

Vitis 高层次综合用户指南

UG1399 (v2025.2) 2026 年 1 月 22 日

本文档为英语文档的翻译版本，若译文与英语原文存在歧义、差异、不一致或冲突，概以英语文档为准。译文可能并未反映最新英语版本的内容，故仅供参考，请参阅最新版本的英语文档获取最新信息。





目录

- 第一部分：简介.....9
 - 按设计进程浏览内容.....9
 - Vitis HLS 支持的操作系统.....9
 - 获取 Vitis HLS 许可证.....10
 - 行为更改.....12
 - 高层次综合的优势.....12
 - Vitis HLS 组件简介.....13
 - 为 HLS 重构 C++ 源代码.....15
 - 教程与示例.....18
- 第二部分：HLS 编程指南.....19
 - 第 1 章：设计原则.....20
 - 可编程逻辑三大范例.....21
 - 组合三种范例.....25
 - 结论 - 性能良方.....27
 - 第 2 章：HLS 抽象并行编程模型.....29
 - 控制驱动的任务和数据驱动的任务.....30
 - 数据驱动的任务级并行度.....31
 - 控制驱动的任务级并行.....34
 - 混用数据驱动的模式与控制驱动的模式.....45
 - 总结.....47
 - 第 3 章：循环入门.....49
 - 循环流水打拍.....49
 - 展开循环.....59
 - 合并循环.....60
 - 处理嵌套循环.....61
 - 处理变量循环边界.....62
 - 第 4 章：阵列入门.....64
 - 将软件阵列映射到硬件存储器.....64
 - 阵列访问和性能.....65
 - 接口上的阵列.....69
 - 阵列初始化和复位.....72
 - 实现 ROM.....73

阵列的 C 语言仿真.....	74
第 5 章：函数入门.....	76
函数内联.....	76
函数流水打拍.....	77
函数例化.....	77
第 6 章：数据类型.....	79
标准类型.....	80
复合数据类型.....	83
任意精度 (AP) 数据类型.....	97
任意精度浮点库 <code>ap_float<int W, int E></code>	102
全局变量.....	106
指针.....	106
矢量数据类型.....	116
位宽传输.....	116
第 7 章：不受支持的 C/C++ 构造.....	118
系统调用.....	118
动态存储器使用.....	119
指针限制.....	120
递归函数.....	121
未定义的行为.....	121
虚拟函数和指针.....	122
第 8 章：HLS 设计接口.....	123
定义接口.....	123
Vitis HLS 存储器布局模型.....	177
HLS 设计的执行模式.....	186
控制初始化与复位行为.....	200
第 9 章：利用 M_AXI 接口进行设计的最佳实践.....	202
第 10 章：顶层性能编译指示.....	204
第 11 章：优化技巧与故障排除提示.....	208
认识高层次综合的调度与绑定.....	209
优化逻辑.....	215
AXI 系统性能优化.....	217
管理面积资源和硬件资源.....	244
展开 C++ 类中的循环.....	249
控制驱动的任务级并行度的限制.....	250
利用静态变量进行流水打拍的局限性.....	255
规范数据流.....	255

第三部分：Vitis HLS 流程步骤.....	260
第 12 章：启动 Vitis Unified IDE.....	261
Vitis Unified IDE 功能特性.....	262
使用 Flow Navigator.....	263
第 13 章：构建和运行 HLS 组件.....	266
创建 HLS 组件.....	267
定义 HLS 配置文件.....	290
运行 C 语言仿真.....	299
运行 C 语言综合.....	315
运行 C/RTL 协同仿真.....	332
封装 RTL 设计.....	345
运行实现.....	349
优化 HLS 工程.....	353
组件比较功能.....	354
L1 Library Wizard 流程.....	355
第 14 章：从命令行创建 HLS 组件.....	358
第四部分：Vitis HLS 命令参考资料.....	361
第 15 章：vitis、v++ 和 vitis-run 命令.....	362
hls_init.tcl.....	364
第 16 章：HLS 配置文件命令.....	365
HLS 常规选项.....	365
阵列分区配置.....	368
C 语言仿真配置.....	368
协同仿真配置.....	369
编译选项.....	372
数据流配置.....	374
调试选项.....	376
接口配置.....	376
封装选项.....	380
运算符配置.....	382
RTL 配置.....	384
调度设置.....	386
存储配置.....	387
实现配置.....	387
展开设置.....	389
HLS 优化指令.....	389
第 17 章：HLS 编译指示.....	431
pragma HLS aggregate.....	432



pragma HLS alias.....	433
pragma HLS allocation.....	434
pragma HLS array_partition.....	436
pragma HLS array_reshape.....	438
pragma HLS array_stencil.....	440
pragma HLS bind_op.....	441
pragma HLS bind_storage.....	444
pragma HLS cache.....	447
pragma HLS dataflow.....	450
pragma HLS dependence.....	452
pragma HLS disaggregate.....	454
pragma HLS expression_balance.....	456
pragma HLS function_instantiate.....	457
pragma HLS inline.....	458
pragma HLS interface.....	460
pragma HLS latency.....	465
pragma HLS loop_flatten.....	466
pragma HLS loop_merge.....	469
pragma HLS loop_tripcount.....	470
pragma HLS occurrence.....	472
pragma HLS performance.....	473
pragma HLS pipeline.....	475
pragma HLS protocol.....	477
pragma HLS reset.....	478
pragma HLS stable.....	479
pragma HLS stream.....	480
pragma HLS top.....	482
pragma HLS unroll.....	483
第 18 章：HLS Tcl 命令.....	485
工程命令.....	485
配置命令.....	508
优化指令.....	525
第五部分：Vitis HLS C 语言驱动程序参考资料.....	568
第 19 章：AXI4-Lite 从接口 C 语言驱动程序参考.....	569
X<DUT>_Initialize.....	569
X<DUT>_CfgInitialize.....	570
X<DUT>_LookupConfig.....	570
X<DUT>_Release.....	571
X<DUT>_Start.....	571
X<DUT>_IsDone.....	571
X<DUT>_IsIdle.....	572
X<DUT>_IsReady.....	572
X<DUT>_Continue.....	572



X<DUT>_EnableAutoRestart.....	573
X<DUT>_DisableAutoRestart.....	573
X<DUT>_Set_ARG.....	573
X<DUT>_Set_ARG_vld.....	574
X<DUT>_Set_ARG_ack.....	574
X<DUT>_Get_ARG.....	574
X<DUT>_Get_ARG_vld.....	575
X<DUT>_Get_ARG_ack.....	575
X<DUT>_Get_ARG_BaseAddress.....	575
X<DUT>_Get_ARG_HighAddress.....	576
X<DUT>_Get_ARG_TotalBytes.....	576
X<DUT>_Get_ARG_BitWidth.....	577
X<DUT>_Get_ARG_Depth.....	577
X<DUT>_Write_ARG_Words.....	577
X<DUT>_Read_ARG_Words.....	578
X<DUT>_Write_ARG_Bytes.....	578
X<DUT>_Read_ARG_Bytes.....	579
X<DUT>_InterruptGlobalEnable.....	579
X<DUT>_InterruptGlobalDisable.....	580
X<DUT>_InterruptEnable.....	580
X<DUT>_InterruptDisable.....	580
X<DUT>_InterruptClear.....	581
X<DUT>_InterruptGetEnabled.....	581
X<DUT>_InterruptGetStatus.....	582
 第六部分：Vitis HLS 库参考资料.....	 583
 第 20 章：C/C++ 语言内置函数.....	 584
 第 21 章：任意精度数据类型库.....	 585
使用任意精度数据类型.....	585
C++ 任意精度整数类型.....	587
C++ 任意精度定点类型.....	606
 第 22 章：HLS Print 函数.....	 630
 第 23 章：HLS 数学库.....	 631
HLS 数学库精度.....	631
HLS 数学库.....	632
定点数学函数.....	634
验证和数学函数.....	636
常见综合错误.....	639
 第 24 章：HLS 串流库.....	 640
C 语言建模与 RTL 实现.....	641
使用 HLS 串流.....	641

第 25 章：HLS 直接 I/O.....	649
使用直接 I/O 串流.....	649
将直接 I/O 串流映射到连线接口.....	650
将直接 I/O 串流映射到 SAXI Lite 接口.....	651
阻塞/非阻塞 API.....	652
第 26 章：HLS 围栏库.....	655
第 27 章：HLS 向量库.....	658
第 28 章：HLS 任务库.....	661
DTLP 中对于标量/存储器接口的支持.....	662
任务与通道.....	663
任务与数据流.....	665
第 29 章：HLS 拆分/合并库.....	667
第 30 章：HLS 块串流库.....	671
块串流建模样式.....	672
检查满的块和空的块.....	674
数据流区域内的建模反馈.....	676
限制.....	677
第 31 章：HLS IP 库.....	678
DSP 本征函数.....	678
快速傅里叶变换 (IP) 库.....	685
FIR 滤波器 IP 库.....	690
DDS IP 库.....	695
SRL IP 库.....	699
第 32 章：处理 OpenCV.....	701
第七部分：Vitis HLS 移植指南.....	702
第 33 章：从 Vitis HLS 移植到 Vitis Unified HLS.....	703
第 34 章：从 Vivado HLS 移植到 Vitis HLS.....	705
关键行为差异.....	705
第 35 章：已弃用功能特性和不受支持的功能特性.....	711
第 36 章：不受支持的功能特性.....	716
编译指示.....	716
HLS 视频库.....	716



C 语言任意精度类型.....717

附录 A：Tcl 到配置文件命令映射.....718

附录 B：指令/运算符说明.....727

附录 C：附加资源与法律声明.....729

 查找其他文档.....729

 支持资源.....729

 参考资料.....730

 修订历史.....730

 请阅读：重要法律声明.....732

第一部分

简介

本部分包含以下章节：

- [按设计进程浏览内容](#)
- [Vitis HLS 支持的操作系统](#)
- [高层次综合的优势](#)
- [Vitis HLS 组件简介](#)
- [教程与示例](#)

按设计进程浏览内容

AMD 自适应计算文档按一组标准设计进程进行组织，以便帮助您查找当前开发任务相关的内容。您可以在[设计中心](#)页面上访问 AMD Versal™ 自适应 SoC 设计进程。您还可以使用[设计流程助手](#)来更深入了解设计流程，并找到特定于预期设计需求的内容。本文档涵盖了以下设计进程：

- 硬件、IP 和平台开发：为硬件平台创建 PL IP 块、创建 PL 内核、功能仿真以及评估 AMD Vivado™ 时序收敛、资源使用情况和功耗收敛。还涉及为系统集成开发硬件平台。本文档中适用于此设计进程的主题包括：
 - [启动 Vitis Unified IDE](#)
 - [运行 C 语言仿真](#)
 - [运行 C 语言综合](#)
 - [运行 C/RTL 协同仿真](#)

Vitis HLS 支持的操作系统

AMD 支持基于 x86 和 x86-64 处理器架构的下列操作系统。

- Red Hat Enterprise Linux Workstation/Server/CentOS：7.9
- Red Hat Enterprise Linux Workstation/Server：8.5、8.6、8.7 和 8.8
- Red Hat Enterprise Linux Workstation/Server：9.0、9.1、9.2 和 9.3
- Ubuntu 20.04.4 LTS、20.04.5 LTS 和 20.04.6 LTS
- Ubuntu 22.04 LTS、22.04.1 LTS、22.04.2 LTS 和 22.04.3 LTS
- Amazon Linux 2 AL2 LTS



提示： Ubuntu 20.04 可能需要 `libtinfo.so.5`，如[答复记录 76616](#) 中所述。

用于嵌入式软件和 HLS 开发的 Windows 64 位：

- Windows 10: 21H2 和 22H2
- Windows 11: 21H2 和 22H2
- Windows Server 2022

获取 Vitis HLS 许可证

在 Vitis Unified IDE 中，HLS 编译器的报告功能（如代码分析器和数据流查看器）需要许可证才能使用。这意味着您需要获得许可证用于安装该工具。您可使用以下步骤获取许可证：

1. 访问 <https://china.xilinx.com/getlicense>
2. 登录您的帐户。



提示： 您必须拥有 AMD 帐户才能下载安装文件和获取许可证。

3. 这样您即可进入“Product Licensing - Name and Address Verification”（产品许可 - 姓名与地址验证）页面。请输入必填信息，然后单击“Next”（下一步）。
4. 这样您即可转至“Product Licensing”（产品许可）页面。如果您有多个帐户，请选择其中一个帐户。
5. 在“Create New Licenses”（创建新许可证）下，勾选“Vitis HLS License”（Vitis HLS 许可证）复选框，然后选择适用的“Generate Floating License”（生成浮动许可证）或“Generate Node-Locked License”（生成单机版许可证）。

Certificate Based Licenses

	Product	Type	License	Available Seats	Status	Subscription End Date
☒	Vitis HLS License	Certificate - No Charge	Node / Float...	1/1	Current	None
☐	SDSoC Environment, 60 Day Evaluation License	Certificate - Evaluation	Node / Float...	1/1	Current	60 days
☐	Vivado ML Enterprise Edition, 30-Day Evaluation License	Certificate - Evaluation	Node	1/1	Current	30 days
☐	2023 AI Engine Tools License	Certificate - No Charge	Node / Float...	1/1	Current	None
☐	Vitis Model Composer (Xilinx toolbox for MATLAB and Simulink), 90 Day Evalua...	Certificate - Evaluation	Node / Float...	1/1	Current	90 days
☐	ISE Embedded Edition License	Certificate - No Charge	Node / Float...	1/1	Current	None
☐	SDAccel OpenCL Development Environment, 30 Day Node Locked Evaluation ...	Certificate - Evaluation	Node	1/1	Expired	31 Oct 2016
☐	SDAccel OpenCL Development Environment, 30 Day Node Locked Evaluation ...	Certificate - Evaluation	Node	1/1	Current	30 days
☐	SDAccel OpenCL Development Environment, 30 Day Floating Evaluation License	Certificate - Evaluation	Floating	1/1	Current	30 days
☐	Vivado Design Suite: ILS WebPACK 2015 and Earlier License	Certificate - No Charge	Node	1/1	Current	None
☐	ISE WebPACK License	Certificate - No Charge	Node	1/1	Current	None

Generate Floating License

Generate Node-Locked License

What is the difference between a floating and a node license? [?](#)

6. 选择浮动许可证与选择单机版许可证打开的对话框稍有不同。要生成浮动许可证，您必须指定“Requested Seats”（请求的席位数）。对于单机版许可证，此数量设置为 1。

1 PRODUCT SELECTION

Product Selections	Product	Type	Available Seats	Subscription End Date	Requested Seats	Borrowed Seats
☒	Vitis HLS License	No Charge	1/1	None	1	

2 SYSTEM INFORMATION

License: Floating

Redundancy: ☒ Non-Redundant ☐ Triple Redundant

Host ID:

3 COMMENTS

Comments: HLS application

7. 在“System Information”（系统信息）下，单击“Select a host”（选择主机），选择现有主机或者添加一个主机。必须选中主机后才能分配许可证。要选中“Add a host”以添加主机，请使用以下步骤：
 - a. “Host Name”（主机名）：指定主机的名称。
 - b. “Select OS type...”（选择操作系统类型）：指定主机的操作系统。
 - c. “Select host type...”（选择主机类型）：根据所选操作系统，可选主机类型包括：
 - Dongle Flex ID
 - “Disk Serial Number”（磁盘序列号）：可在 Windows 命令提示符处输入 `vol c:` 获取
 - “Ethernet MAC”（以太网 MAC）：可从 `ipconfig /all` 命令获取，作为以太网适配器的物理地址
 - Solaris hostid
8. 指定主机后，单击“Next”移至“Review License Request”（复查许可证请求）对话框。
9. 复查所提供的信息，然后单击“Next”以生成许可证文件，并接收“Congratulations”（祝贺）消息。您将收到一封电子邮件，其中包含所需的许可证文件。

如需了解有关使用许可证文件的信息，请参阅《Vivado Design Suite 用户指南：版本说明、安装和许可》(UG973) 中的“管理许可证”部分。如需了解有关许可证的常规信息，请参阅[许可解决方案中心](#)。

行为更改

下表列出了该版本与前版本之间影响移植行为或流程的差异。

表 1：Vitis 软件开发平台的行为更改汇总

范围	详细信息
Vitis Unified IDE	IDE 元数据文件已完成重组。现已引入新的 <code>_ide/.wsdata/version.ini</code> 文件用于标记工作空间版本。打开旧工作空间或者不含 <code>version.ini</code> 的工作空间时，IDE 会提示您更新当前工作空间。重组的目标是通过将重新生成文件保留在 <code>.wsdata</code> 文件夹中，帮助您更轻松地将工作空间添加到源码控制工具中。
Vitis Unified IDE	原先位于 <code>logs</code> 目录中的 Python journal 日志文件 <code>builder.py</code> 现已重命名为 <code>workspace_journal.py</code> 并已移入 <code>_ide</code> 目录。现在，可从“Vitis”→“Workspace Journal”（Vitis > 工作空间日志）下的菜单中访问该文件。
Vitis Unified IDE	IDE 现已支持您在 <code>UserConfig.cmake</code> 中管理应用组件源代码编译设置。原先，IDE 会自动设置构建列表。
Vitis Unified IDE	Vitis IDE 的进程名称已从 <code>Vitisng-ide</code> 改为 <code>Vitis-ide</code> 。
Vitis Unified IDE	用户管理的流程已从“Explorer”（资源管理器）视图扩展至“Vitis Explorer”（Vitis 资源管理器）视图。“Vitis Explorer”可以处理 GUI 管理的流程 and 用户管理的流程，并提供一致的流程窗口支持。

表 2：Vitis 加速软件开发的行为更改汇总

范围	详细信息
Vitis 基础平台	VCK190 基础平台现在使用 Vivado 可定制示例设计 (CED) Vitis 可扩展平台来发起设计，而非从头开始构建。此项更新使 Vitis 平台配置与 CED 和 Vitis 基础平台保持一致。后续版本中，也会将此方法应用于其他基础平台。

高层次综合的优势

高层次综合 (HLS) 是自动设计进程，利用数字系统的抽象行为规范来生成寄存器传输级 (RTL) 结构，以实现给定行为。

使用 HLS 的典型流程包含下列步骤：

1. 围绕目标架构在高抽象层次使用 C/C++ 编写算法
2. 在行为级别验证功能
3. 使用 HLS 工具为给定时钟速度和设计约束生成 RTL
4. 验证生成的 RTL 的功能
5. 使用相同输入源代码探索其他不同的架构

HLS 能够为创建高质量 RTL 铺路，比手动编写没有任何错误的 RTL 更快。

设计师需要以 C/C++ 创建高层次的算法宏架构，这意味着必须审慎全面考量设计意图以及设计与外界世界的交互方式。此 HLS 工具还需要时钟周期、性能约束等设计约束。

在高层次上并不需要制定微架构决策，如创建状态机、数据路径、寄存器流水线等。这些细节可以留给 HLS 工具，通过提供设计约束即可生成经过优化的 RTL。

设计师还可以使用已定义的 C/C++ 算法宏架构通过不同的约束来生成多种 RTL 解决方案，在性能与面积之间做出权衡取舍。因此，单一算法即可得到多种实现，这样即可允许设计师选择最适合满足整体应用需求的实现。

提升效率

设计师利用 HLS 开展高抽象层工作，这表示需编写为 HLS 输入的代码行数更少。由于缩短了用于编写 C++ 代码的时间并且周转更快，因此可以显著减少用户错误数量，从而提升了整体设计效率。设计师们能更专注于在更高的抽象层次上创建高效的设计，而无需担忧机械性的 RTL 实现细节。

HLS 不仅能提升设计效率，也能提升验证效率。借助 HLS 也能在高层次生成或创建测试平台，这意味着原始设计意图可以快速得到验证。由于流程仍在 C/C++ 领域内，因此设计师可以尝试快速调整已验证的算法。在 C/C++ 内完成算法验证后，HLS 工具即可将同样的测试平台用于生成 RTL。然而，生成的 RTL 可与现有 RTL 验证流程加以集成，从而实现更全面的验证覆盖范围。

使用 HLS 给设计和验证带来的好处总结如下：

- 在 C 语言层次开发和确认算法，以便根据硬件实现详细信息在抽象层进行设计。
- 使用 C 语言仿真来确认设计并以比传统 RTL 设计更快的速度进行迭代。
- 根据 C 语言源代码和编译指示创建多个设计解决方案，以探索设计空间并找到最优解。

支持复用

为高层次综合创建的设计属于通用设计，无关实现。不同于给定 RTL，这些源代码并不与任何技术节点或任何给定时钟周期绑定。只需少量更新输入约束，无需更改任何源代码，即可尝试多种不同的架构。对 RTL 采用类似的做法却并不实用。设计师为给定时钟周期创建 RTL，无论对衍生产品进行任何程度的更改，都会导致产生新的复杂工程。设计师利用 HLS 开展高层次工作时，就无需担心微架构，并且可以依靠 HLS 工具来自动重新生成新的 RTL。

Vitis HLS 组件简介

HLS 组件从 C 语言或 C++ 语言函数综合到 RTL 代码中，以供在 AMD Versal™ 自适应 SoC、AMD Zynq™ MPSoC 或 AMD FPGA 器件的可编程逻辑 (PL) 区域中实现。HLS 组件与 Vivado Design Suite 紧密集成用于综合与布局布线，并与 Vitis 核开发套件紧密集成用于异构系统级别设计和应用加速。

HLS 组件可用于开发和导出：

- Vivado IP，以供使用 Vivado Design Suite 集成到硬件设计中，并搭配所提供的软件驱动程序用于在嵌入式系统中进行应用开发
- Vitis 内核，以供在 Vitis 应用加速开发流程中使用、搭配 AI 引擎计算图应用于异构计算系统，或者用于数据中心加速



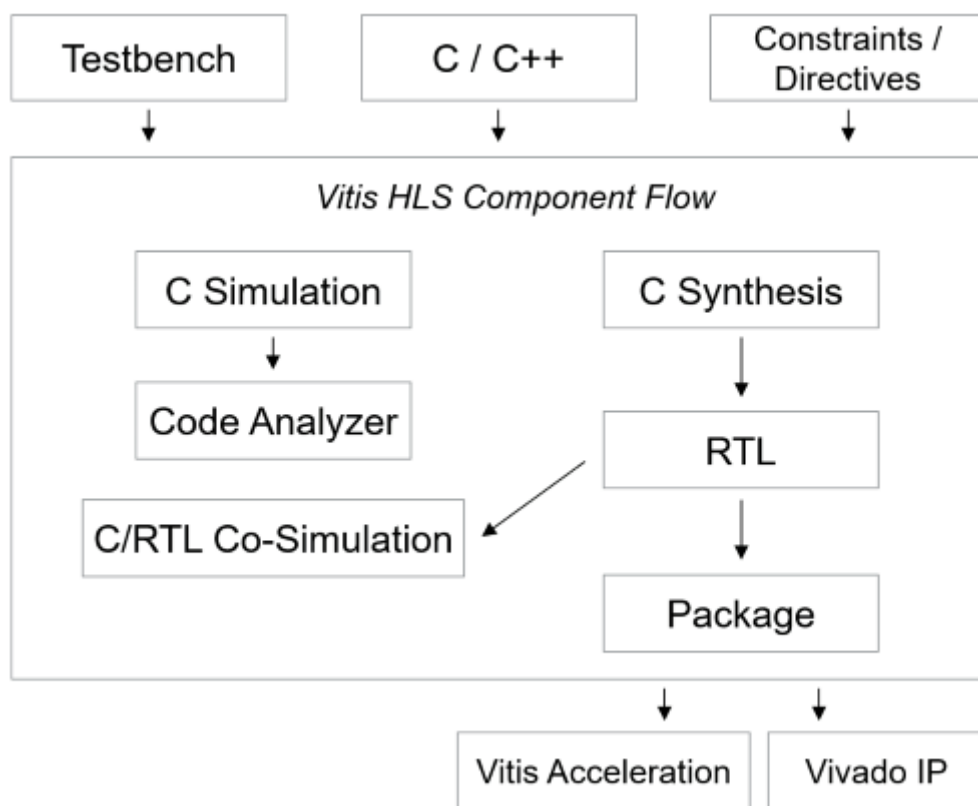
提示：Vitis 内核 (.xo) 具有特定的要求和限制，如 [适用于 Vitis 内核流程的接口](#) 中所述；Vivado IP 几乎没有任何限制，并且可以提供更大的设计灵活性，如 [Vivado IP 流程接口](#) 中所述。

HLS 组件工具流程提供了各种工具，用于在可编程逻辑中仿真、分析、实现和优化 C/C++ 语言代码，以及达成低时延和高吞吐量目标。HLS 组件流程的基础是通过推断所需编译指示来为函数实参生成正确的接口，并对代码中的循环和函数执行流水打拍，无论是通过命令行、Makefile 或 Vitis Unified IDE 来完成此类流程都是如此。

基于 C++ 函数开发 HLS 组件的步骤如下所述：

1. 基于 [设计原则](#) 建立算法架构。
2. （C 语言仿真）利用 C/C++ 语言测试平台验证 C/C++ 代码的功能。
3. （代码分析器）分析 C/C++ 代码的性能、并行度与合规性。
4. （C 语言综合）使用 `v++` 编译器生成 RTL。
5. （C/RTL 协同仿真）使用 C/C++ 测试平台验证生成的 RTL 代码。
6. （封装）复查 HLS 综合报告和实现时序报告。
7. 重新运行前述步骤直至满足性能目标为止。

图 1：HLS 组件开发流程



该工具会基于目标流程、默认工具配置、设计约束和您指定的任意优化编译指示或指令来实现 HLS 组件。您可使用优化指令来修改和控制内部逻辑和 I/O 端口的实现，以覆盖工具的默认行为。

以下提供了有关在 HLS 组件中进行 C++ 函数编码与综合的部分关键概念，在后续章节内将涵盖更多详细信息：

- 硬件接口：HLS 组件的顶层函数的实参均综合到接口和端口内，这些接口和端口通过将多个信号加以组合来定义硬件设计与设计外部的组件之间的通信协议。`v++` 编译器会自动定义接口，并使用业界标准来指定要使用的协议。根据 HLS 组件目标是作为 Vivado IP 还是作为 Vitis 内核来进行封装，默认接口协议不尽相同。接口的默认分配可通过使用 `INTERFACE` 编译指示或指令来覆盖。
- 控制 HLS 组件的执行：HLS 组件的执行模式是由块级控制协议来指定的。HLS 组件可包含控制信号，用于启动/停止执行，也可设置为仅当数据可用时才驱动 HLS 设计。作为设计人员，您需要注意自己的 HLS 设计的执行方式，如 [HLS 设计的执行模式](#) 中所述。

- 任务级并行度：

为了在生成的硬件上实现高性能，综合器工具必须基于顺序代码推断并行度，并利用它来实现更高的性能。[设计原则](#) 章节介绍了三个主要范式，要为 FPGA 平台编写优秀软件，就需要了解这三个范式。HLS 组件通过指定 DATAFLOW 编译指示或者使用 `hls::task` 对象来显式创建并行度，提供了 2 种类型的任务级并行度 (TLP)，如 [HLS 抽象并行编程模型](#) 中所述。

- 存储器架构：

- 存储器架构在 CPU 中已固定，但开发者可以创建自己的架构来优化存储器访问，以便在可编程逻辑上运行应用。
- 在 C++ 程序中，阵列是基础数据结构，用于保存或操作数据。在硬件中完成综合后，这些阵列是作为存储器或寄存器来实现的。存储器可作为本地存储器或全局存储器（通常是 DDR 存储器或 HBM 存储器）来实现。访问全局存储器会产生更高的时延成本，可耗时大量周期，而访问本地存储器通常十分快速，只需一个或多个周期即可。
- 通常在 C++ 程序中，对存储器进行动态分配/解除分配，但这在硬件中无法进行综合。因此，设计师需知晓算法所需存储器的精确数量。
- 存储器访问应加以优化，以减少全局存储器访问的开销。该工具可按编译指示或者在检测到特定访问模式时执行存储器突发访问或合并（例如，通过扩展数据宽度，将多次访问合并为一次访问）。突发访问会隐藏存储器访问时延，并改善存储器带宽。

- 微观级别的优化：

- 在 C++ 程序中，经常需要实现重复算法来处理数据块，例如，信号处理或图像处理。通常，C/C++ 源代码倾向于包含多个循环或多个嵌套循环。v++ 编译器可通过在源代码的相应级别插入编译指示来对循环或嵌套循环进行展开或流水打拍。如需了解更多信息，请参阅 [循环入门](#)。
- 基于设计原则完成算法架构并推断并行度后，您仍需要正确组合微观级别的 HLS 编译指示，如，PIPELINE、UNROLL、ARRAY_PARTITION 等。[PERFORMANCE](#) 编译指示或指令支持您为给定循环体或嵌套循环定义单一顶层性能目标。该工具会自动推断必要的低层次编译指示以满足该目标。借助 PERFORMANCE 编译指示可以减少实现理想 QoR 所需的编译指示，这也是驱动该工具的一种直观方法。

为 HLS 重构 C++ 源代码

以下是以 C++ 编写的简单程序，此程序包含 `compute()` 函数并在 CPU 上执行。此程序在 CPU 上按顺序执行。此示例需重构才能在可编程逻辑上运行时达成最优性能。

```
#include <vector>
#include <iostream>
#include <ap_int.h>
#include "hls_vector.h"

#define totalNumWords 512
unsigned char data_t;

int main(int, char**) {
    // initialize input vector arrays on CPU
    for (int i = 0; i < totalNumWords; i++) {
        in[i] = i;
    }
    compute(data_t in[totalNumWords], data_t Out[totalNumWords]);
    check_results();
}
```

```

void compute (data_t in[totalNumWords ], data_t Out[totalNumWords ]) {
    data_t tmp1[totalNumWords], tmp2[totalNumWords];
    A: for (int i = 0; i < totalNumWords ; ++i) {
        tmp1[i] = in[i] * 3;
        tmp2[i] = in[i] * 3;
    }
    B: for (int i = 0; i < totalNumWords ; ++i) {
        tmp1[i] = tmp1[i] + 25;
    }
    C: for (int i = 0; i < totalNumWords ; ++i) {
        tmp2[i] = tmp2[i] * 2;
    }
    D: for (int i = 0; i < totalNumWords ; ++i) {
        out[i] = tmp1[i] + tmp2[i] * 2;
    }
}

```

此程序 FPGA 上按顺序执行并生成正确结果，但并无任何性能增益。为了提升在 FPGA 上的性能，此程序必须重构以便在硬件内启用并行性操作。并行性的示例包括：

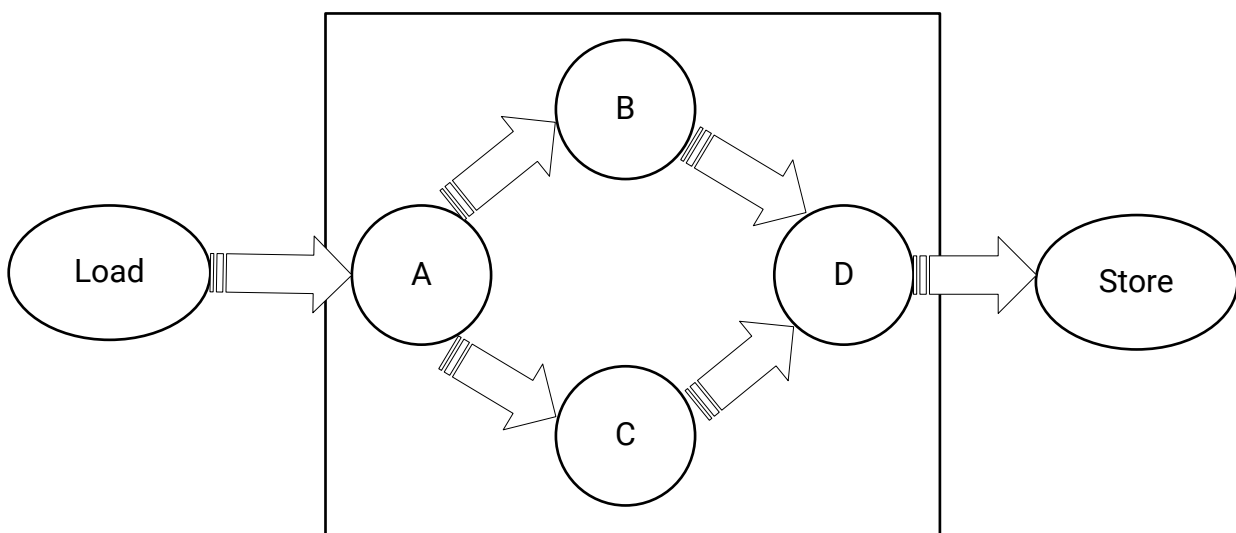
- compute 函数可先启动，随后再将所有数据传递给它
- 多个 compute 函数能以重叠方式运行，例如，“for”循环能够在上一次迭代完成前启动下一次迭代
- “for”循环内的各项操作都能在多个字上并发运行，无需逐字执行

重新架构硬件模块

根据前述示例，compute() 函数需重构，以实现基于 FPGA 的加速。

compute() 函数循环 A 将输入值乘以 3，并创建两条独立路径，分别是 B 和 C。循环 B 和 C 执行操作并将数据馈送给 D。这是一种现实状况的简单表示法，您需在其中逐一执行多项任务，这些任务彼此相连形成如下所示网络。

图 2：模块架构



X27496-120522

以下是硬件模块重构的要点总结：

- 在函数级别实现任务级并行度。为实现任务级并行度，需将循环推送到多个独立的函数中。原始 compute() 函数拆分为多个子函数。根据经验法则，顺序函数可并发执行，顺序循环则可采用流水打拍。

- 指令级并行度是通过从存储器中读取 16 个 32 位字（或 512 位数据）来实现的。在所有这些字上可以并行执行计算。hls::vector 类属于 C++ 模板类，用于对多个样本并发执行矢量运算。
- compute() 函数需重构为多个 load-compute-store（加载 - 计算 - 存储）子函数，如以下示例所示。load 函数和 store 函数用于封装数据访问，并对各 compute 函数执行的计算进行隔离。
- 此外还有以 #pragma 开头的编译器指令，可将顺序代码转换为并行执行。



提示： GitHub 上的《Vitis-HLS 简介示例》中提供了如下所示的 [using_ffos](#) 示例。

```
#include "diamond.h"
#define NUM_WORDS 16
extern "C" {

void diamond(vecOf16Words* vecIn, vecOf16Words* vecOut, int size)
{
    hls::stream<vecOf16Words> c0, c1, c2, c3, c4, c5;
    assert(size % 16 == 0);

    #pragma HLS dataflow
    load(vecIn, c0, size);
    compute_A(c0, c1, c2, size);
    compute_B(c1, c3, size);
    compute_C(c2, c4, size);
    compute_D(c3, c4, c5, size);
    store(c5, vecOut, size);
}
}

void load(vecOf16Words *in, hls::stream<vecOf16Words >& out, int size)
{
    Loop0:
    for (int i = 0; i < size; i++)
    {
        #pragma HLS PERFORMANCE target_ti=32
        #pragma HLS LOOP_TRIPCOUNT max=32
        out.write(in[i]);
    }
}

void compute_A(hls::stream<vecOf16Words >& in, hls::stream<vecOf16Words >&
out1, hls::stream<vecOf16Words >& out2, int size)
{
    Loop0:
    for (int i = 0; i < size; i++)
    {
        #pragma HLS PERFORMANCE target_ti=32
        #pragma HLS LOOP_TRIPCOUNT max=32
        vecOf16Words t = in.read();
        out1.write(t * 3);
        out2.write(t * 3);
    }
}

void compute_B(hls::stream<vecOf16Words >& in, hls::stream<vecOf16Words >&
out, int size)
{
    Loop0:
    for (int i = 0; i < size; i++)
    {
        #pragma HLS PERFORMANCE target_ti=32
        #pragma HLS LOOP_TRIPCOUNT max=32
        out.write(in.read() + 25);
    }
}
```

```

    }
}

void compute_C(hls::stream<vecOf16Words >& in, hls::stream<vecOf16Words >&
out, int size)
{
Loop0:
    for (data_t i = 0; i < size; i++)
    {
        #pragma HLS PERFORMANCE target_ti=32
        #pragma HLS LOOP_TRIPCOUNT max=32
        out.write(in.read() * 2);
    }
}

void compute_D(hls::stream<vecOf16Words >& in1, hls::stream<vecOf16Words >&
in2, hls::stream<vecOf16Words >& out, int size)
{
Loop0:
    for (data_t i = 0; i < size; i++)
    {
        #pragma HLS PERFORMANCE target_ti=32
        #pragma HLS LOOP_TRIPCOUNT max=32
        out.write(in1.read() + in2.read());
    }
}

void store(hls::stream<vecOf16Words >& in, vecOf16Words *out, int size)
{
Loop0:
    for (int i = 0; i < size; i++)
    {
        #pragma HLS PERFORMANCE target_ti=32
        #pragma HLS LOOP_TRIPCOUNT max=32
        out[i] = in.read();
    }
}

```

教程与示例

为您快速上手 Vitis HLS，您可在以下位置找到教程与应用示例：

- Vitis HLS 简介示例 (<https://github.com/Xilinx/Vitis-HLS-Introductory-Examples>)：其中包含小型代码示例，用于演示良好的设计实践、编码准则、常用应用的设计模式以及（最重要的）优化技巧，从而最大程度提升应用性能。所有示例都包含 README 文件、hls_config.cfg 文件和 run_hls.tcl 脚本以帮助使用示例代码。
- Vitis 加速示例仓库 (https://github.com/Xilinx/Vitis_Accel_Examples)：包含示例，用于演示 Vitis 工具和平台的各项功能特性。此仓库提供了小型有效示例，用于演示与 Vitis 应用加速开发流程的主机代码和内核编程相关的具体案例。这些示例中的内核代码均可在 HLS 组件中直接编译。
- Vitis 应用加速开发流程教程 (<https://github.com/Xilinx/Vitis-Tutorials>)：提供多种教程，通过这些教程可以教授有关工具流程和应用开发的具体概念，包括使用 Vitis Unified IDE 来创建 HLS 组件以供在 Vitis 自下而上的设计流程中使用。

第二部分

HLS 编程指南

简介

此编程指南旨在为您提供真实世界的设计技巧以及硬件设计详细信息，帮助您最大限度发挥 AMD Vitis™ HLS 工具的作用。本指南提供了有关各种编程技巧的详细信息，您应运用这些技巧来编写 C/C++ 代码并通过高层次综合将其综合为 RTL 代码，此外它还提供了最佳实践检查表，您应遵循该检查表来创建使用 AXI4 接口的 IP。最后，它还详述了各种优化技巧，供您用于提升代码性能、改善生成的硬件设计的适用性和功能性。

本部分包含以下章节：

- [设计原则](#)
- [HLS 抽象并行编程模型](#)
- [循环入门](#)
- [阵列入门](#)
- [函数入门](#)
- [数据类型](#)
- [不受支持的 C/C++ 构造](#)
- [HLS 设计接口](#)
- [利用 M_AXI 接口进行设计的最佳实践](#)
- [优化技巧与故障排除提示](#)

设计原则

简介

您可能会使用 HLS 工具来充分发挥编写 C/C++ 代码为硬件生成 RTL 的效率增益；或者您可能期望在可编程逻辑中实现的定制硬件上加速各部分 C/C++ 算法。本章旨在帮助您了解根据 C/C++ 编写的软件算法来进行硬件综合的进程。本文档中介绍了用于设计和创建可综合的优秀软件的基本概念，凭借其中所用方法即可使用高层次综合 (HLS) 工具将此类软件成功转换为硬件。本文档中探讨的内容与工具无关，介绍的概念均为大部分 HLS 工具通用的概念。对于有经验的设计师而言，这些材料能够帮助您巩固这些概念的重要性，并帮助您了解如何触达 HLS，尤其是如何构造 HLS 代码来实现高性能设计。

吞吐量和性能

在可编程逻辑中作为定制硬件来实现的 C/C++ 函数的运行速度远高于传统 CPU/GPU 架构上可实现的速度，并且可实现的处理速率和/或性能也更高。首先需要明确在硬件加速上下文中，下列术语的含义。吞吐量定义为在单位时间内执行的特定操作的数量或者在单位时间内生成的结果数量。它是以单位时间内生成的任意对象的单位数量（汽车、摩托车、I/O 示例、存储器字数、迭代数）来度量的。例如，“存储器带宽”一词有时用于指定存储器系统的吞吐量。同样，性能的定义不仅是指提升吞吐量，而是在低功耗的前提下提升吞吐量。在当今世界里，降低功耗的重要性丝毫不亚于提升吞吐量。

架构为要

为了能够更好地理解定制硬件如何对程序中某些部分进行加速，首先您需要了解您的程序在传统计算机上是如何运行的。冯诺依曼架构是现今几乎所有计算的基础，虽然它是在 70 多年前设计完成的。此架构被视为最适合大型应用程序，旨在保障灵活性和可编程性。但随着应用程序需求给与系统的压力开始逐渐增大，CPU 开始支持多个进程的并行执行。多线程处理和/或多进程处理可包含多个系统进程（例如，同时执行两个或两个以上程序），或者可包含单个进程，但此进程中包含多个线程。使用共享存储器系统进行多线程编程允许软件开发者在固定 CPU 架构的前提下，围绕并行性来设计应用程序，因而发展势头如火如荼。但当速度不断提升的多线程 CPU 无法再满足数据处理速度时，多核 CPU 和超线程粉墨登场，用于提升吞吐量，如右图所示。

这种通用的灵活性的代价是功耗和峰值吞吐量。现如今手机、游戏和在线视频会议无处不在，所处理的数据的性质已发生了改变。为了实现更高的吞吐量，您必须将工作负载移至距离存储器更近的位置和/或将工作负载移入专用功能单元内。因此，新的挑战是定义全新可编程架构，使您能够在保留足够的可编程性的同时提升性能并降低功耗成本。

现场可编程门阵列 (FPGA) 不仅能提供这种可编程性，还能提供足够的存储器带宽，从而造就了这一高性能低功耗成本的解决方案。FPGA 与执行程序的 CPU 的不同之处在于，FPGA 可配置为定制硬件电路，其响应输入的方式与专用硬件的行为方式相同。可重配置器件（如 FPGA）包含具备卓越灵活粒度的计算元件，从初级的逻辑门到完整的算术逻辑单元（如数字信号处理 (DSP) 块），都一应俱全。当采用更高粒度时，即可将用户指定的可组合式逻辑单元（称为内核）按策略布局在 FPGA 上，以执行各种角色。可重配置 FPGA 器件的这一特性可支持创建定制宏架构，故而相比传统 CPU/GPU，FPGA 在使用特定于应用程序的并行操作方面优势显著。计算能按空间映射到器件，因而较之以处理器为核心的平台，能够提供更高的操作吞吐量。现今最新 FPGA 还可包含处理器核（基于 Arm）及其他硬化的 IP 块，无需烧录到可编程逻辑中即可直接使用。

可编程逻辑三大范例

虽然 FPGA 可使用 Verilog 或 VHDL 等低层次硬件描述语言 (HDL) 来编程，但现在已有多种高层次综合 (HLS) 工具可以采用以 C/C++ 之类的更高层次的语言编写的算法描述，并将其转换为 Verilog 或 VHDL 等低层次的硬件描述语言。这样下游工具即可对转换后的语言进行处理，以便对 FPGA 器件进行烧录。

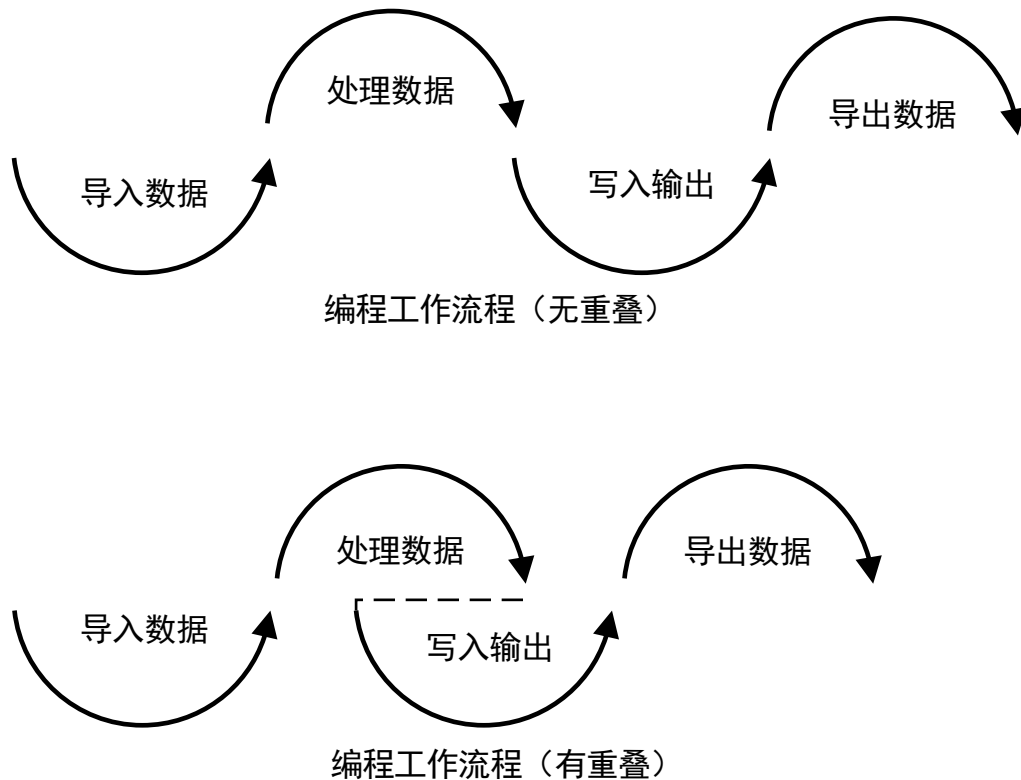
此类流程的主要优势在于，您可使用诸如 C/C++ 等编程语言来编写高效代码，并将代码转换为硬件，但这类编程语言的优势仍能得以完整保留。此外，写好代码乃是软件设计师的专长，比学习新的硬件描述语言更简单。但要实现可接受的结果质量 (QoR)，仍需要额外工作（例如，重写软件）以帮助 HLS 工具实现期望的性能目标。在接下来的几个章节内，将探讨如何首先识别部分宏观级别架构优化以明确程序结构，并聚焦更细化的微观级别架构优化来实现性能目标。

生产者使用者范例

请考虑软件设计师编写多线程程序的方式，通常有一个主线程用于执行某些初始化步骤，并分叉为多个子线程用于执行某些并行计算，当所有并行计算都完成后，主线程会整理结果并写入输出。程序员必须理清哪些部分可以分叉以供并行计算，哪些部分需要按顺序执行。这种分叉/连接类型的并行化操作不仅适用于 CPU，也适用于 FPGA，但 FPGA 上的吞吐量的关键模式之一是生产者使用者范例。您需要将生产者使用者范例应用于顺序程序，并将其转换为可并行执行的抽取功能以便提升性能。

您可借助一条简单的问题语句的帮助来更好地理解这个分解进程。假定您有一份数据手册，您可将其中的项导入列表。随后，您可对列表中的每个项进行处理。处理完每个项需耗时约 2 秒。处理完后，您会把结果写入另一份数据手册，此操作耗时约每项各 1 秒。因此，如果输入 Excel 工作表中有总计 100 个项，那么会耗时总计 300 秒来生成输出。这样做的目的是对此问题进行分解，以便您识别能够并行执行的任务，从而提升系统吞吐量。

图 3：程序工作流程



X25607-030622

第一步是了解程序工作流程，识别独立的任务或函数。整个工作流程分 4 个步骤，类似上图所示的程序工作流程（无重叠）。在此示例中，“写入输出”（步骤 3）任务独立于“处理数据”（步骤 2）处理任务。虽然步骤 3 取决于步骤 2 的输出，但当步骤 2 中的任意项完成处理后，您即可立即将其写入输出文件。您无需等待所有数据都完成处理后再开始将数据写入输出文件。此类型的交织/重叠任务执行方式是非常常见的原则。如上图所示（例如：含重叠的程序工作流程）。如图所示，含重叠的工作流程比不含重叠时更快完成。现在，您可将步骤 2 视作为生产者，将步骤 3 视作为使用者。生产者使用者模式对于 CPU 性能的影响有限。您可交织执行每个线程的步骤，但这需要谨慎分析，以便充分利用底层多线程和 L1 高速缓存架构，因此较为耗时。但在 FPGA 上，由于可采用定制架构，生产者和使用者线程可以同时执行，开销极低甚至没有，因此能够显著提升吞吐量。

首先考量的最简单案例是单一生产者和单一使用者通过大小有限的缓冲器来进行通信。如果缓冲器已满，那么生产者可以选择阻塞/停滞或者丢弃数据。当使用者从缓冲器中移除某个项后，它会通知生产者，生产者随后开始再次填充缓冲器。如果使用者发现缓冲器已空，则可以同样方式停滞。当生产者将数据置入缓冲器后，它会唤醒休眠中的使用者。该解决方案可通过进程间通信（通常使用监控器或信号量）来实现。不充分的解决方案可能导致死锁，即两个进程都停滞并等待唤醒。但在单一生产者和使用者的情况下，通信模式与先入先出 (FIFO) 或乒乓缓冲器 (PIPO) 实现之间存在强映射关系。这种类型的通道无需依赖信号量、互斥体或监控器来进行数据传输，即可提供高效的数据通信。使用此类锁定原语对于性能可能开销较大，并且难以使用和调试。PIPO 和 FIFO 是常用的选择，因为可以避免端到端的原子同步需求。

在此类宏观级别架构优化中，可通过缓冲器封装通信，使程序员可免于担心存储器模型和其他非确定性行为（如争用条件等）。在此类设计中可达成的网络类型为纯粹的“数据流网络”，可在输入侧接受串流数据，对此数据串流执行一些基本处理，然后将其作为数据串流发出。并行程序的复杂性完全被抽离。“导入数据”（步骤 1）和“导出数据”（步骤 4）同样在最大程度提升可用的并行性方面扮演着相应的角色。为了使计算能够与 I/O 成功重叠，重要的是第一步对输入的读取结果进行封装，最后一步则是写入输出。这样即可实现 I/O 与计算的最大程度重叠。在计算步骤中间读取或写入输入/输出端口会限制设计的可用并发性。这同样是在对设计工作流程进行设计时需要牢记的。

最后，此类“数据流网络”的性能依赖于设计师能够持续向网络馈送数据，使数据能够在系统中保持串流。数据流中出现中断可能导致性能下降。视频串流应用就是一个很好的例子，比如在线游戏中，实时高清 (HD) 视频持续流经系统，帧处理率受到持续监控，以确保满足期望的结果质量。游戏玩家可以在屏幕上立即观察到帧处理率下降。假想一下，这能为一大群游戏玩家提供持续性帧率支持，同时功耗相比传统 CPU 或 GPU 架构显著降低 - 这就是硬件加速的魅力。使数据在生产者与使用者之间持续保持流动至关重要。下一步，您可深入了解本节中介绍的这种串流范例。

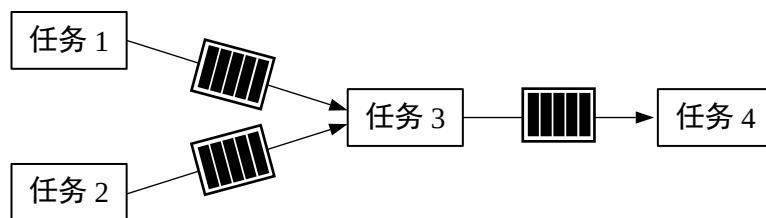
串流数据范例

串流是一种重要的抽象：它表示无限制连续更新数据集，其中“无限制”表示“大小未知或者大小无限”。串流可以是一连串数据（标量或缓冲器）在源（生产者）进程与目标（使用者）进程之间单向流动。串流范例会强制您根据数据访问模式（或序列）来思考。在软件中，随机存储器对数据的访问几乎是免费的（忽略高速缓存成本），但在硬件中，执行顺序访问实际上是很有利的，此类访问可转换为串流。将算法分解为生产者使用者关系并通过网络串流数据来进行通信具有如下所述几大优势。它允许程序员以顺序方式定义算法，并通过其他方式来提取并行度（例如，通过编译器）。诸如任务间同步等复杂性会被抽离。它允许生产者和使用者任务同时处理数据，这是提升吞吐量的关键。另一个优势是代码更清洁且更简单。

如前文所述，对于生产者和使用者范例，数据传输模式与 FIFO 或 PIPO 缓冲器实现之间存在强映射关系。FIFO 缓冲器只是预定义大小/深度的队列，其中插入队列的首个元素也会成为可从队列跳出的首个元素。使用 FIFO 缓冲器的主要优势在于，只要生产者将数据插入缓冲器，使用者进程就可以立即在 FIFO 缓冲器内部开始访问数据。使用 FIFO 缓冲器的唯一问题在于，由于生产者与使用者之间的生产/使用速率不同，可能导致 FIFO 缓冲器大小错误，从而导致死锁。在具有多个生产者和使用者的设计中，这种情况较为常见。乒乓缓冲器属于用于加速进程的双缓冲器，可将 I/O 操作与数据处理操作重叠。其中一个缓冲器用于保存数据块，以便使用者进程能够看到完整（旧）版本的数据，而另一个缓冲器中，生产者进程则正在创建新（部分）版本的数据。当新的数据块完成并有效时，使用者和生产者进程会交换对两个缓冲器的访问。由此导致，使用乒乓缓冲器会增加器件的整体吞吐量，并帮助防止出现最终瓶颈。PIPO 的主要优势在于，工具能够将生产速率与使用速率自动匹配，并创建高性能且无死锁的通信通道。此处值得注意的是，无论使用的是 FIFO 还是 PIPO，关键特性是相同的：生产者将数据块发送或者串流至使用者。数据块可以是单个值，也可以是一组 N 个值。块越大，所需存储器资源越多。

以下是简单的求和应用，用于展示经典的串流网络/数据流网络。在此例中，该应用的目标是成对添加随机数值串流，然后打印这些数值。前两个任务（任务 1 和 2）提供了随机数值串流以供添加。这些数值通过 FIFO 通道发送到求和任务（任务 3），任务 3 会从 FIFO 通道读取值。随后，求和任务将输出发送到打印任务（任务 4），以发布结果。FIFO 通道可在这些独立的执行线程之间提供异步缓冲。

图 4：串流网络/数据流网络



X25608-030622

连接每项“任务”的串流通常是作为 FIFO 队列来实现的。FIFO 能够抽离程序员的并行行为，使其专注于推理任务活动（调度）的“快照”时间。FIFO 能够使并行化更易于实现。这主要得益于它减少了程序员实现并行化框架或容错解决方案时，必须应付的可变空间。独立内核之间的 FIFO 展现出经典的排队行为。对于纯串流系统，可使用排队或网络流模型来对此行为进行建模。这种数据流类型网络和串流优化的另一个主要优势在于它可按不同粒度级别来应用。程序员可以在每项任务内部设计此类网络，也可以为任务或内核系统设计此类网络。实际上，您可以通过串流网络来以分层方式例化并连接多个串流网络或任务。支持更细粒度的并行化的另一项优化措施是流水打拍。

流水线范例

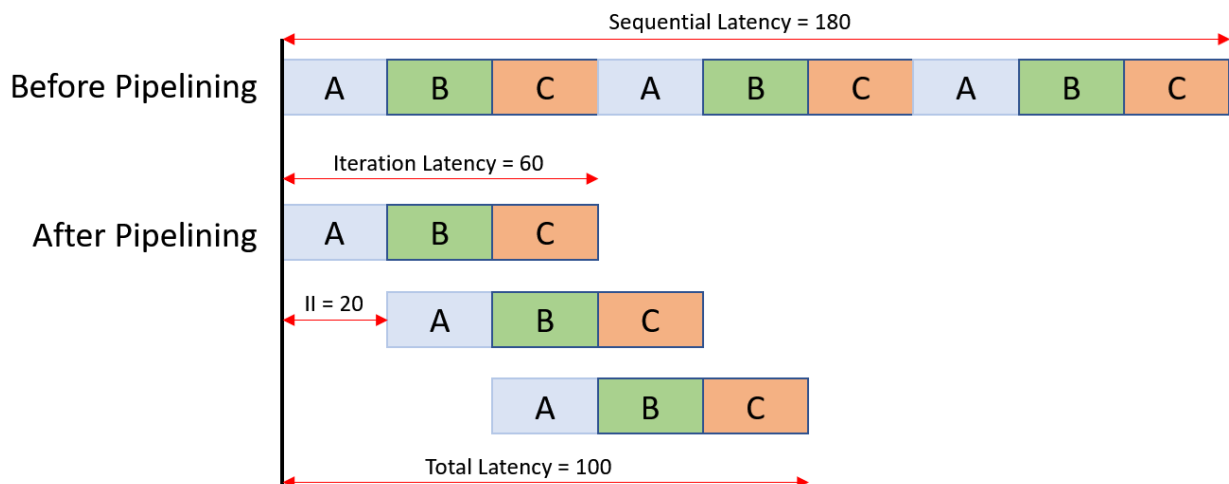
流水线是您日常生活中常用的概念。造车厂生产线就是一个典型的例子，其中每一项具体任务通常都是由一个独立且唯一的工作站来完成的，如安装引擎、安装车门和安装车轮。各工作站各自对一辆不同的车并行执行自己的任务。当某一辆车执行完某一项任务后，它就会移至下一个工作站。完成各项任务的时间差可通过“缓冲”（将一辆或多辆车暂存在各工作站之间的空间内）和/或“停滞”（暂时中止上游工作站的操作）来加以调整，直至下一个工作站变为可用为止。

假设组装一辆车需要执行 3 项任务 A、B 和 C，这 3 项任务分别需要 20、10 和 30 分钟。那么，如果全部 3 项任务均由单个工作站来执行，工厂每 60 分钟才能输出一辆车。通过使用 3 个工作站组成的流水线，该工厂 60 分钟即可输出第一辆车，随后每 30 分钟再输出一辆新车。正如此示例所示，流水线并不会降低时延，即单个项穿越整个系统的总时间。但它会增加系统吞吐量，即，第一个项完成后处理新的项的速率。

由于流水线的吞吐量不可能优于其最慢的元素，因此程序员应尝试在各阶段间拆分工作和资源，以使各阶段耗用相同时间来完成自己的任务。在上述车辆组装线示例中，如果 3 项任务 A、B 和 C 各自耗时 20 分钟，而不是分别耗时 20、10 和 30 分钟，那么时延将仍为 60 分钟，但每隔 20 分钟（而不是 30 分钟）即可完成一辆新车。下图显示了承担制造 3 辆车任务的假想生产线示例。假定任务 A、B 和 C 各耗时 20 分钟，那么顺序生产线将需要 180 分钟才能生产 3 辆车。而流水线式生产线只需 100 分钟即可生产 3 辆车。

生产第一辆车所耗费的时间为 60 分钟，称为流水线的迭代时延。生产完第一辆车后，后两辆车各自只需 20 分钟，这称为流水线的启动时间间隔 (II)。生产三辆车所耗费的总时间为 100 分钟，称为流水线的总时延，例如：总时延 = 迭代时延 + II * (项数 - 1)。因此，改善 II 即可改善总时延，但不影响迭代时延。从程序员视角来看，流水线范例可适用于设计中的函数和循环。确定初始建立时间成本后，可将实现 II 为 1 设为理想的吞吐量目标，例如，初始建立时间延迟过后，在流水线的每个周期都有输出可用。在以上示例中，初始建立时间延迟 60 分钟过后，每隔 20 分钟就有一辆车可用。

图 5：流水打拍



流水打拍是经典的微观级别架构优化，可应用于多个抽象层。前文通过生产者使用者范例解释了任务级别流水打拍。这一概念同样适用于指令级别。这实际上是使生产者使用者流水线（和串流）保持填满并繁忙的关键。仅当每项任务都高速率生产/使用数据，故而需要指令级流水打拍 (ILP) 时，生产者使用者流水线才能保持高效。

由于流水打拍长期使用相同资源来执行相同功能，因此需要有关每项任务时延的完整信息，故而被视为静态优化。有鉴于此，低级指令流水打拍方法无法应用于数据流类型的网络，因为在此类网络中任务的时延是输入数据的函数，故而可能未知。下一章节中将详解如何利用介绍的这三种基本范例来对不同类型的任务并行度进行建模。

组合三种范例

用户程序中，大部分优化的主要焦点是函数和循环。现今的优化工具通常在函数/过程级别工作。每个函数都能转换为特定硬件组件。每个此类硬件组件都与类定义相似，该组件的许多对象（或实例）均可在最终硬件设计中创建和例化。每个硬件组件都可由许多更小的预定义组件组成，这些预定义组件通常用于实现基本函数（例如，加法、减法和乘法）。虽然不支持递归，但是函数可调用其他函数。较小且调用次数较少的函数通常还可以内联到其调用方函数中，这正是软件函数的内联方式。在此情况下，实现函数所需的资源将汇总到调用方函数的组件中，这样能更好地共享公用资源。将设计构造为一组通信函数有助于在执行这些函数时提升推断并行度。

循环是程序中最重要构造之一。由于循环主体多次迭代，因此可轻松利用该属性来提升并行度。可通过多种方式来对循环和循环嵌套进行变换（例如，[流水打拍](#)和[展开](#)），从而提升并行执行的效率。这些变换能实现存储器系统优化，从而映射到多核与 SIMD 执行资源。科学与工程应用中的许多程序都表现为对大型数据结构进行各种运算。这些运算包括对阵列或矩阵进行简单的逐元素运算，或者也可能是具有循环进位依赖关系的更复杂的循环嵌套运算，例如，跨循环迭代的依赖关系。此类数据依赖关系会影响循环内可达成的并行度。在诸多此类情况下，必须对代码进行重构，才能在现代化的并行平台上有效执行并行循环迭代。

下图显示了 4 个连续任务（例如，C/C++ 函数）A、B、C 和 D 的不同重叠执行的简单示例，其中，A 在 2 个不同阵列内分别为 B 和 C 生成数据，D 则使用来自 B 和 C 所生成的 2 个不同阵列的数据。假定这种“菱形”通信模式将运行 2 次，且 2 次运行彼此独立。

```
void diamond(data_t vecIn[N], data_t vecOut[N])
{
    data_t c1[N], c2[N], c3[N], c4[N];
    #pragma HLS dataflow
    A(vecIn, c1, c2);
    B(c1, c3);
    C(c2, c4);
    D(c3, c4, vecOut);
}
```

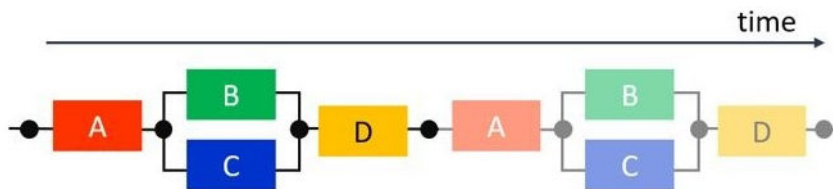
以上代码示例所示 C/C++ 源代码片段显示了这些函数的调用方式。任务 B 和 C 相互之间不存在数据依赖关系。下图对应完全顺序执行方式，其中黑色圆圈表示用于实现串行的某种同步形式。

图 6：顺序执行 - 运行 2 轮



在菱形示例中，B 与 C 完全彼此独立。两者既不相互通信也不访问任何共享存储器资源，因此如果无需共享计算资源，那么两者可并行执行。由此可得结果如下图所示，一轮运行内形成了某种形式的分叉式连接并行运行。当 A 任务结束后，B 和 C 并行执行，而 D 则等待 B 和 C，但下一轮仍按顺序连续执行。

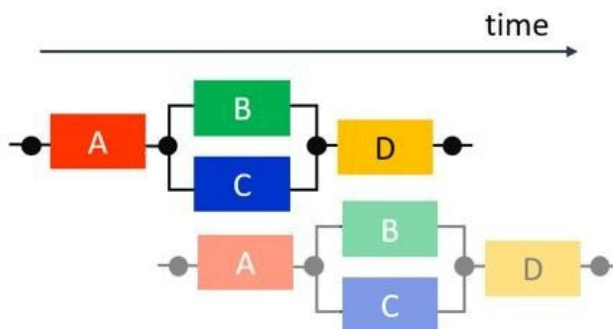
图 7：一轮运行中的任务并行



此类执行方式可总结为 $A; (B \parallel C); D; A; (B \parallel C); D$ ，其中“;”表示串行连续，“ $\parallel$ ”则表示完全并行。这种形式的嵌套分叉连接并行运行对应于一种从属任务子类，称为串并行任务图。一般来说，任何从属任务的有向无环图 (DAG) 均可通过独立的分叉并连接类型同步来实现。此外，同样需要知晓的是，这正是在具有多个线程并使用共享存储器的 CPU 上运行多线程程序的方式。

在 FPGA 上，您可以探索其他可用的并行形式。先前的执行模式利用的是在单次调用内执行任务级别并行操作。那么重叠连续运行又会如何呢？如果每次运行之间真正彼此独立，但每个函数（例如，A、B、C 或 D）复用前一轮的相同计算硬件，那么您仍可能想要将 A 的第二次调用与 B 和 C 的第一次调用并行执行。这是一种跨调用的任务级流水线形式，由此可得结果如下图所示。现在，由于吞吐量受到所有任务间的最大时延的限制，而非所有任务时延总和的限制，因而吞吐量明显改善。虽然每轮运行的时延不变，但多轮运行的总时延得以缩短。

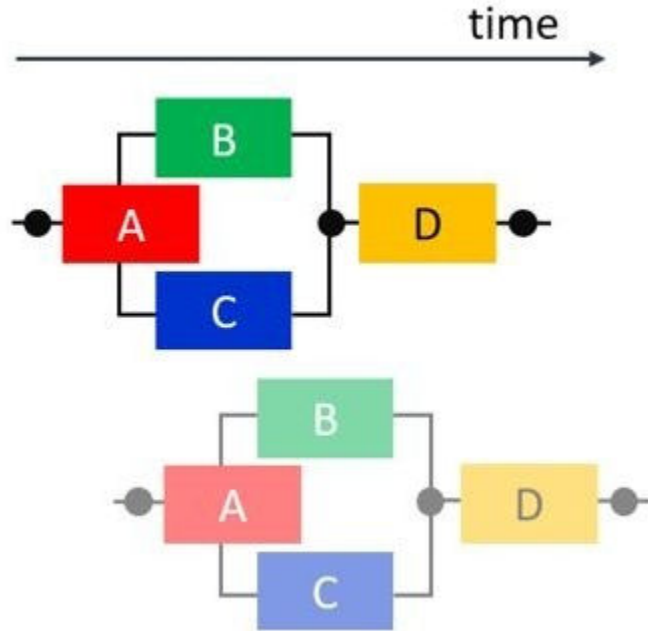
图 8：利用流水打拍实现任务并行



但现在，当 B 首次运行从存储器中执行读取时，A 则已得到其首轮运行的结果，A 的第二轮操作可能已在相同存储器内执行写入。为避免在使用数据之前写入数据，您可以依靠某种形式的存储器扩展（即所谓的双重缓冲或 PIPO）来达成此交织操作。这种交织操作以任务间的黑色圆圈来表示。

有一种有效的方法可用于提升吞吐量和复用计算资源，即对运算符、循环和/或函数进行流水打拍。如果每个任务现在都能与自身重叠，您即可在一轮运行内实现任务并行操作，同时跨多轮运行实现任务流水打拍，这两者都属于宏观级别并行度的例证。多任务内流水打拍则是微观级别并行度的例证。现在，由于每一轮运行依赖于任务间的最小吞吐量而不是任务的最大吞吐量，因此每一轮的总体吞吐量得以进一步提升。最后，根据通信数据的同步方式，仅当生成全部数据 (PIPO) 或者以逐元素方式 (FIFO) 生成全部数据后，才有可能在一轮运行内出现某种程度的额外重叠。例如，在下图中，B 和 C 都比 A 更早启动并以流水打拍方式执行，而 D 则假定仍必须等待 B 和 C 完成。这是最后一种类型的单轮运行内重叠，当 A 通过 FIFO 串流访问（表现为不含圆圈的直线）来与 B 和 C 通信时，才能实现这种方式的重叠。同样，D 也能与 B 和 C 重叠，前提是采用的通道为 FIFO 而不是 PIPO。但不同于前几种执行模式，使用 FIFO 可能导致死锁，因此需要正确设置这些串流 FIFO 的大小。

图 9：单轮运行内的任务并行和流水打拍、多轮运行的流水打拍以及单一任务内的流水打拍



总之，本节中所演示的三种范例为您展示了如何在避免多线程和/或并行编程语言的复杂操作的同时下，仍能在设计中实现并行操作。生产者使用者范例搭配串流通道即可轻松组合小型系统与大型系统在内的各种系统。如上文所述，串流接口支持轻松耦合并行任务，亦或是分层数据流网络也不在话下。这其中部分原因是由于编程语言 (C/C++) 能够灵活支持此类规范，并且还有各种工具可用于在当今 FPGA 器件上可用的异构计算平台上实现这些规范。

结论 - 性能良方

本文档中所演示的设计概念具有一个主要的核心原则，即并行计算模型倾向于通过状态封装和模块化单元或任务内的顺序执行来为并行编程提供更简单的编程模型。随后，任务将与串流相连（以执行同步和通信）。每个串流均可包含不同类型的通道，例如，FIFO 或 PIPO。状态/逻辑区隔化使工具（例如编译器和调度器）能够更加轻松地理清何时何地需要运行应用的哪些部分。基于串流的处理方式日渐流行的另一个原因是，它打破了传统多线程“分叉/连接”模型基于并行执行的观点。通过启用任务级流水打拍和指令级流水打拍，运行时可执行的并发操作数量远远超过了分叉/连接模型现今所能执行的操作数量。这种附加的并行性对于充分利用现今 FPGA 器件上可用的硬件而言至关重要。与启用流水线并行性如出一辙的是，串流同样支持设计师构建并行应用，而无需担心锁定、争用条件等可能导致并行编程无从着手的问题。

最后，建议您依据以下高层次操作检查表为标准，在可重配置 FPGA 平台上实现所期望的性能：

- 专为 CPU 编写的软件与专为 FPGA 编写的软件有着本质上的不同。要想编写可在 CPU 平台与 FPGA 平台之间进行移植的代码，性能牺牲是不可避免的。因此，FPGA 软件与 CPU 软件的编写方式差异巨大，与其抗拒这个事实，不如坦然接受。
- 从工程伊始，就应该建立起能够对源代码更改进行功能性验证的流程。基于参考模型或者使用黄金矢量来测试软件是极为常用的实践。
- 首先专注于设计的宏架构。并考虑使用生产者使用者范例来对解决方案进行建模。

- 明确设计的宏架构后，即可绘制出期望的活动时间线，其中以横轴表示时间，并显示您期望在多次迭代（或调用）过程中执行每一项功能相对于其他功能的时间。这样您就可以对设计中期望的并行性了然于胸，后续即可用于与最终达成的结果进行比较。HLS GUI 通常可用于直观显示所达成的并行度。
- 仅当您已掌握宏架构并确立活动时间线之后，再开始进行程序编码或重构。
- 一般，HLS 编译器仅根据函数调用来推断任务级别并行度。因此，需要在硬件中并发运行的顺序代码块（例如循环）应置于专用函数内。
- 将原始算法分解/分区为较小的组件，这些组件可通过串流来彼此进行通信。这样您就可以在一定程度上掌握数据在设计中流动的方式。
 - 较小的模块化组件的优势在于，可以按需进行赋值，从而提升并行性。
 - 请避免通信通道的位宽过宽。将此类宽通道分解为较窄些的通道，这样有助于在 FPGA 器件上实现。
 - 大型函数（手写或通过内联较小的函数生成）可能包含重要路径，工具可能难以处理此类路径。具有更简单的控制路径的小型函数有助于 FPGA 器件上的实现。
 - 目标是在每个函数内包含单个循环嵌套（可采用 HLS 工具可推断的固定循环边界，或者手动向 HLS 工具提供循环次数信息）。这样可以显著促进吞吐量的测量和优化。此方法虽然可能不适用于所有设计，但对于大部分案例都很有效。
- 吞吐量 - 纵观全局，掌握设计每个阶段期间所需的处理速率，这一点至关重要。这会影响您为 FPGA 编写应用的方式。
 - 思考设计中的关键路径（例如，关键任务级别路径，如，ABD 或 ACD），调查此关键路径中哪个部分可能成为瓶颈。通过设计仿真，观察哪几个任务采用流水打拍，以及任一路径的不同分支是否在吞吐量方面存在不一致。随后，可使用 HLS GUI 工具和/或仿真波形查看器来直观显示此类吞吐量问题。
 - 基于串流的通信允许使用者在生产者开始生产时立即开始处理，由此即可支持重叠执行（从而提升并行性和吞吐量）。
 - 为了使生产者和使用者任务保持持续不间断运行，请使用流水打拍和调整串流的相应大小等方法来对每项任务的执行进行优化，使其能尽快运行。
- 请思考串流通道的同步粒度（和开销）。您可使用 PIPO 通道来重叠任务执行，而无需担心死锁，显式手动串流 FIFO 通道支持您（比 PIPO）更快开始重叠执行，但请谨慎处理 FIFO 大小调整，以避免死锁。
- 了解有关可综合的 C/C++ [编码样式](#) 的更多信息。
- 使用 HLS 编译器生成的报告作为指导来完成优化进程。

请将上述检查表放在附近，以便随时参考。它总结了构建能满足您的性能目标的设计所需的整个设计活动。

设计的另一个需要考量的重要方面在于您的加速函数或内核。内核连接外部的接口是最终系统设计的重要要素。您的内核可能需要插入更大的设计或者与更大的内核系统中的其他内核进行通信，或者与系统外部的存储器或器件进行通信。[利用 M_AXI 接口进行设计的最佳实践](#) 提供了另一份检查表，以供您在设计加速内核的外部接口时考量其中所列出的各项内容。

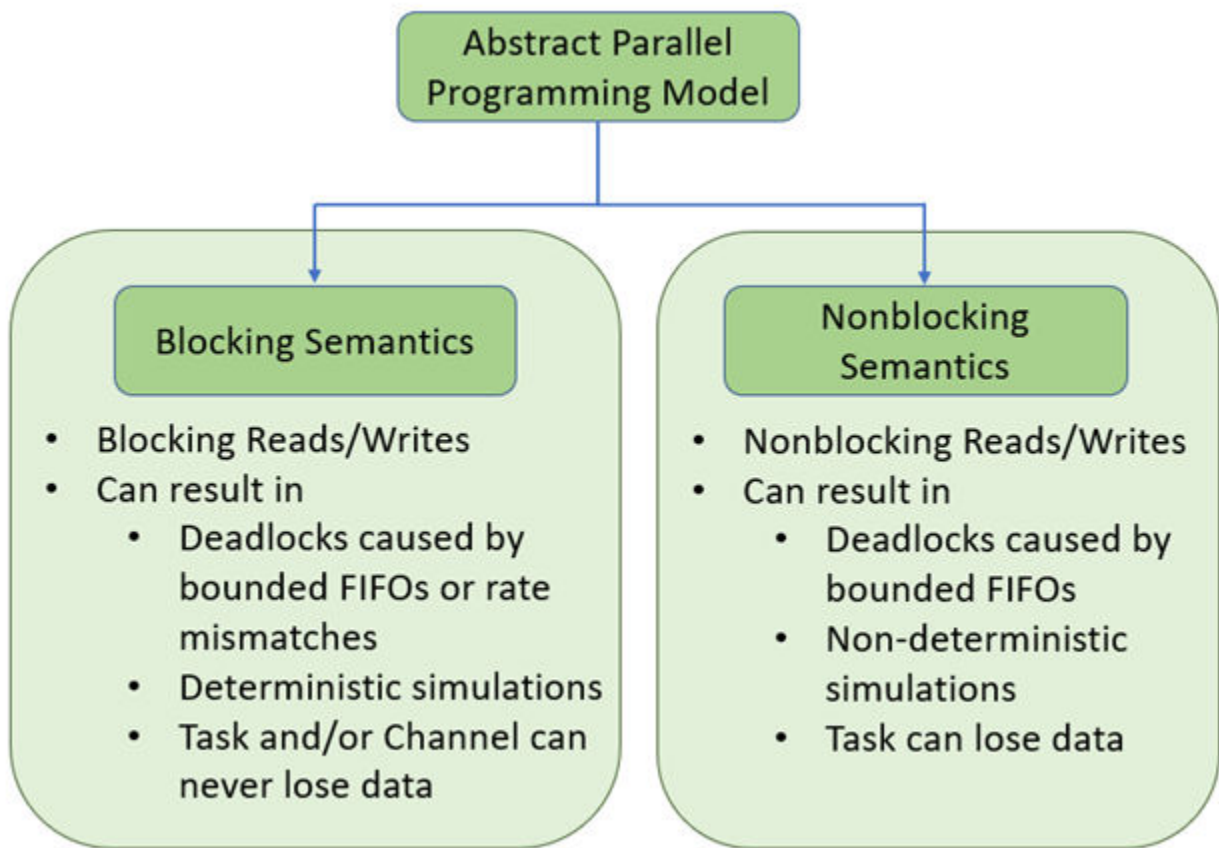
HLS 抽象并行编程模型

为了实现高性能硬件，HLS 工具必须基于顺序代码推断并行度，并利用它来实现更高的性能。要解决这个问题可并不简单。此外，优秀的软件设计通常使用精心定义的规则和实践，例如，运行时类型信息 (RTTI)、递归和动态存储器分配。其中诸多技巧在硬件中都无法找到直接等效的对象，故而给 HLS 工具带来了诸多挑战。这通常意味着现成软件无法高效转换为硬件。最低限度，需检验此类软件中是否存在不可综合的构造，并需要重构代码，使其可综合。即使软件程序可自动转换（或综合）为硬件，但您需要了解编写在 FPGA 器件上执行的优秀软件的最佳实践，从而辅助该工具完成工作。

设计原则 章节介绍了三个主要范例，要为 FPGA 平台编写优秀软件，就需要了解这三个范例：生产者使用者、串流数据和流水打拍。这些范例的底层并行编程模型如下所述：

- 设计/程序需作为任务集合来构造，这些任务通过通信链接（也称为通道）来彼此发送消息。
- 任务可按控制驱动的方式来构造，使用此方式时，任务会等待某些信号后才开始执行；或者任务也可按数据驱动的方式来驱动，在此情况下，通道上只要存在数据，就会驱动任务执行。
- 任务由可执行单元组成，此类单元具有本地存储空间/存储器以及输入/输出 (I/O) 端口集合。
- 本地存储器包含专用数据，例如，任务享有专属访问权的数据。
- 对此专用存储器执行的访问称为本地数据访问，类似块 RAM/URAM 中存储的数据。此类访问十分快速。任务将其本地数据副本发送到其他任务的唯一方式是通过其输出端口，反之，它只能通过其输入端口来接收数据。
- I/O 端口是抽象端口；它对应于任务用于发送或接收数据的通道，并且它由模块的调用程序连接，或者如果它是顶层端口，则在系统集成时连接。
- 通过通道发送或接收的数据称为非本地数据访问。通道是将某一个任务的输出端口连接到另一个任务的输入端口的数据队列。
- 通道应确保可靠，并具有如下行为：
 - 在生产者输出处写入数据的顺序与在使用者输入端口处读取数据的顺序相同以便于 FIFO。对于 PIPO，可按随机顺序写入/读取数据。
 - 不丢失任何数据值。
- 针对通道，支持阻塞和非阻塞读写语义仿真，如 [HLS 串流库](#) 中所述。

图 10：阻塞/非阻塞语义



在该模型中使用阻塞语义时，读取空通道会导致读取进程发生阻塞。同样，写入已满的通道则会导致写入进程发生阻塞。生成的进程/通道网络会展现出**确定性行为**，此类行为不依赖于计算的时序或**通信延迟**。这些样式的模型经验证，非常便于用于对**嵌入式系统**、**高性能计算系统**、**信号处理系统**、**串流处理系统**、**数据流编程**语言及其他计算任务进行建模。

建模的阻塞样式可能导致因下列原因而发生死锁：通道队列大小不足（前提是通道为 FIFO）和/或由于生产者与使用者之间的产率不同。如果在该模型中使用非阻塞语义，那么读取空通道就会导致读取未初始化的数据，或者导致重新读取上一个数据项。同样，写入已满的队列则会导致数据丢失。为避免此类数据丢失，设计必须首先检查队列状态，然后才能执行读写。但这会导致此类模型的仿真呈现不确定性，因为它所依赖的决策是根据通道的运行时状态来作出的。这会导致对此模型的结果进行验证变得极为困难。

Vitis HLS 抽象并行编程模型支持阻塞语义和非阻塞语义。

控制驱动的任务和数据驱动的任务

使用此抽象模型作为基础，有两种类型的任务级并行度 (TLP) 模型可用于构造和设计您的应用。TLP 可采用数据驱动或控制驱动，或者可在单一设计内混用控制驱动任务与数据驱动任务。这两种模型之间的主要区别在于：

- 如果您的应用采用纯数据驱动，无需与外部存储器有任何交互，并且函数可并行执行，无数据依赖关系，那么数据驱动的 TLP 模型则是最适合的选择。您可以设计纯数据驱动的模式，这种模型始终运行，无需控制且仅对数据作出响应。如需了解更多详情，请参阅 [数据驱动的任务级并行度](#)。
- 如果您的应用需要与外部存储器进行某种交互，并且并行执行的任务之间存在数据依赖关系，那么控制驱动的 TLP 模型则是最适合的选择。Vitis HLS 会推断任务之间的并行度，并创建合适的通道（由您定义），以便这些函数可在执行期间重叠。控制驱动的 TLP 模型在 Vitis HLS 中也称为数据流优化，如 [控制驱动的任务级并行](#) 中所述。

在接下来的几个章节内，将对可供使用的这些主要建模选项进行详细描述。您可使用其中任意模型，以 C++ 编写源代码，以便对并行硬件上的程序执行进行优化。

数据驱动的任务级并行度

数据驱动的任务级并行度所使用的任务通道建模样式要求您以静态方式显式例化和连接任务与通道。这些任务不受任何函数调用/返回语义控制，而是始终保持运行，等待其输入串流上的数据。此建模样式中的任务通常仅包含串流类型的输入和输出。但您可以向 TLP 设计中的阵列和标量 I/O 引入指针，如 [DTLP 中对于标量/存储器接口的支持](#) 中所述。

数据驱动的 TLP 模型即存在等待处理的数据时执行的任务。在 Vitis HLS 中，C 语言仿真原先仅限于查看顺序语义和行为。凭借数据驱动的模式，即可在仿真期间通过 FIFO 通道查看并行任务的并发性质及其交互。

在 Vitis HLS 工具中实现数据驱动的 TLP 会使用简单的类来进行任务建模 (`hls::task`) 和通道建模 (`hls::stream/`
`hls::stream_of_blocks`)。



重要提示！ 虽然 Vitis HLS 支持对顶层函数使用 `hls::tasks`，但对于顶层函数中的接口，则无法使用 `hls::stream_of_blocks`。

以如下所示[简单任务通道](#)示例为例：

```
#include "test.h"

void splitter(hls::stream<int> &in, hls::stream<int> &odds_buf,
hls::stream<int> &evens_buf) {
    int data = in.read();
    if (data % 2 == 0)
        evens_buf.write(data);
    else
        odds_buf.write(data);
}

void odds(hls::stream<int> &in, hls::stream<int> &out) {
    out.write(in.read() + 1);
}

void evens(hls::stream<int> &in, hls::stream<int> &out) {
    out.write(in.read() + 2);
}

void odds_and_evens(hls::stream<int> &in, hls::stream<int> &out1,
hls::stream<int> &out2) {
    hls_thread_local hls::stream<int> s1; // channel connecting t1 and
t2
    hls_thread_local hls::stream<int> s2; // channel connecting t1 and t3

    // t1 infinitely runs function splitter, with input in and outputs s1
and s2
    hls_thread_local hls::task t1(splitter, in, s1, s2);
```

```
// t2 infinitely runs function odds, with input s1 and output out1
hls_thread_local hls::task t2(odds, s1, out1);
// t3 infinitely runs function evens, with input s2 and output out2
hls_thread_local hls::task t3(evens, s2, out2);
}
```

特殊的 `hls::task` C++ 类为：

- 源代码中新的对象声明，此声明需要特殊限定符。`hls_thread_local` 限定符的必要性体现在，它能在例化函数（示例中的 `odds_and_evens`）的多次调用之间，使对象（和底层线程）保持活动状态。

`hls_thread_local` 限定符的必要性仅体现在用于确保数据驱动的 TLP 模型的 C 语言仿真与 RTL 仿真展现相同的行为。在 RTL 中，这些函数一旦启动，就会保持处于始终运行模式下。为了确保 C 语言仿真期间的行为不变，`hls_thread_local` 是必需的，它用于确保每项任务仅启动一次，并且即使被多次调用仍能保持相同状态。如无 `hls_thread_local` 限定符，则每次新调用该函数都会产生新的状态。

- 任务对象可隐式管理无限运行函数的线程，为其传递一组实参，这些实参必须为 `hls::stream` 或 `hls::stream_of_blocks`。



提示：不支持任何其他类型的实参。尤其是，`even` 常量值不能作为函数实参来传递。如需向任务主体传递常量，请将函数定义为模板函数，并将常量作为模板实参传递到此模板函数。

- 提供的函数（以上示例中的 `splitter/odds/evens`）被称为任务主体，它周围有隐式无限循环包围，以确保任务保持运行并等待输入。
- 提供的函数包含流水节拍循环，但需将其设为可刷新的流水线 (FLP) 才能阻止死锁。该工具会自动选择正确的流水线样式，用于给定的流水节拍循环或函数。



重要提示！ `hls::task` 不应作为函数调用来处理，需改为将 `hls::task` 视作与通道静态绑定的持久实例来处理。因此，您自行负责确保包含 `hls::tasks` 的任意函数的多次调用保持统一，或者确保这些调用使用相同的 `hls::tasks` 和通道。

通道由特殊模板化 `hls::stream`（或 `hls::stream_of_blocks`）C++ 类来进行建模。此类通道具有以下属性：

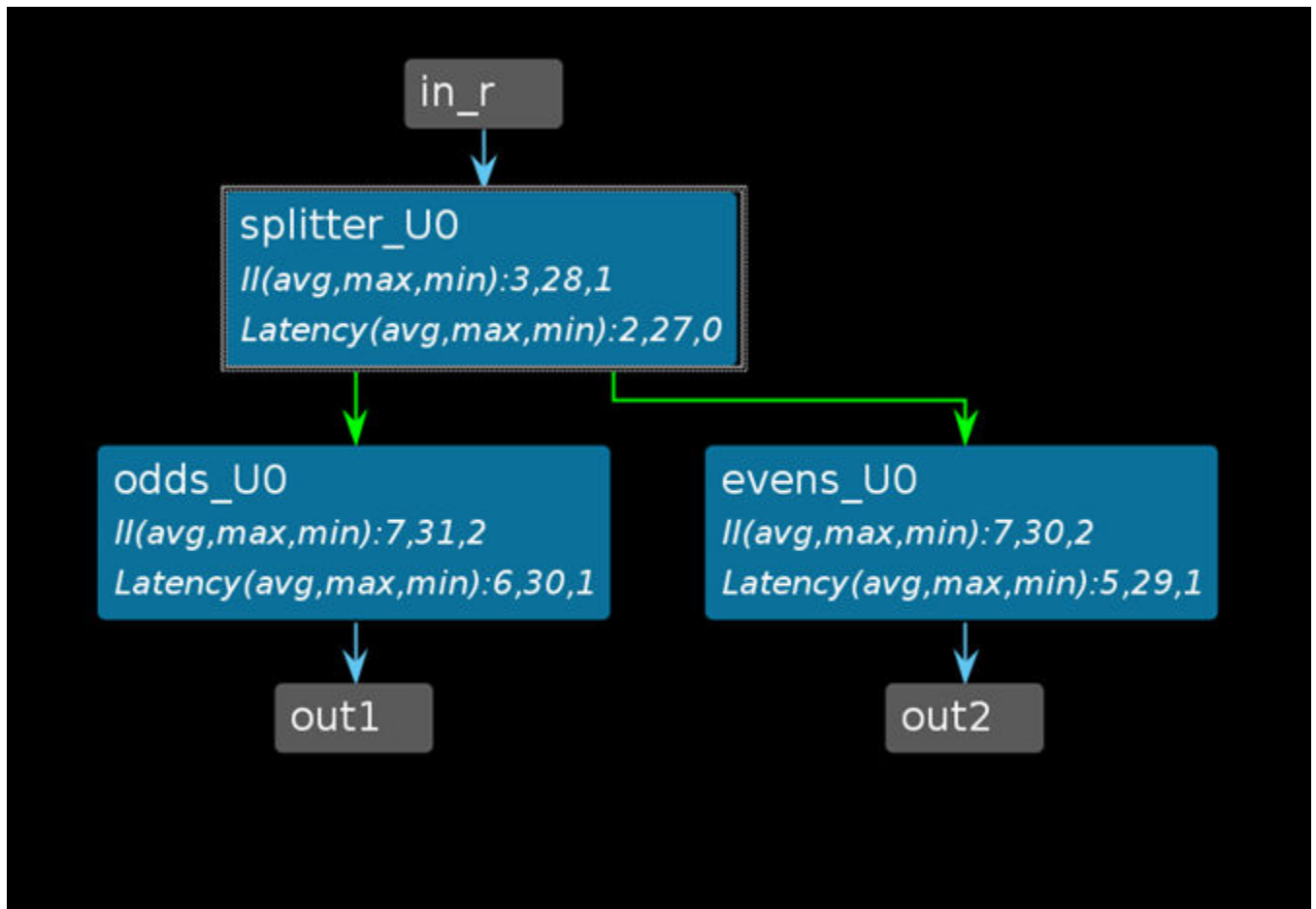
- 在数据驱动的 TLP 模型中，`hls::stream<type,depth>` 对象行为与具有指定深度的 FIFO 类似。此类串流默认深度为 2，用户可覆盖该值。
- 按顺序对这些串流执行读取和写入。这暗含意义是一旦从 `hls::stream<>` 读取数据项，就无法重复读取该数据项。



提示：对不同串流执行访问不经过排序，例如，写入串流和读取另一串流的顺序可通过调度器来更改。

- 串流可采用局部定义或全局定义。全局作用域内定义的串流遵循的规则与任何其他全局变量相同。
- `hls_thread_local` 限定符对于串流（以下示例中的 `s1` 和 `s2`）也是必需的，它能在例化函数（以下代码示例中的 `odds_and_evens`）的多次调用之间，使相同串流保持活动状态。

下图显示了 Vitis HLS 中上述代码示例的图形化表示法。在此图中，绿色箭头为 FIFO 通道，蓝色箭头表示例化函数（`odds_and_evens`）的输入和输出。任务显示为蓝色矩形框。

图 11: `hls::task` 示例的数据流图示

鉴于实际上读取空串流属于阻塞读取，因此可能由于下列原因引发死锁：

- 设计本身内部的进程的生产和耗用率不平衡。
 - 在 C 语言仿真期间，仅限下列原因才会导致死锁：某个进程周期或者从顶层输入启动的进程链尝试读取空的通道。
 - 在 C/RTL 协同仿真期间以及在硬件 (HW) 中运行时，可能由于某个进程周期尝试写入已满的通道或者读取空的通道而发生死锁。
- 如果测试平台提供的数据太少，不足以生成该测试平台在检查计算结果时所需的所有输出，则会发生死锁。

因此，当设计包含 `hls::task` 时，会自动例化死锁检测器。该死锁检测器会检测到死锁并停止 C 语言仿真。此外，还会使用 C 语言调试器（如 `gdb`）进行进一步调试，观察仿真的 `hls::tasks` 尝试读取空的通道时，全部遭到阻塞的具体位置。使用 Vitis HLS GUI 能轻松完成此操作，正如有关调试死锁的 [handling_deadlock](#) 示例中所示。

总之，如果您的设计需要完全数据驱动的纯串流类型的行为，且无需任何种类的控制，那么推荐使用 `hls::task` 模型。此类模型对于反馈和动态多重速率设计也都很有用。只要任务间存在周期性依赖关系，设计中就会发生反馈。动态多重速率模型中如果生产者写入数据或者使用者读取数据的速率与数据相关，那么就只能由数据驱动的 TLP 来处理。如需获取示例，请参阅 GitHub 上的 [simple_data_driven](#) 设计。



提示：静态多重速率设计中如果生产者写入数据或者使用者读取数据的速率与数据无关，则数据驱动的 TLP 和控制驱动的 TLP 均可用于对其进行管理。例如，生产者在某一串流内为每次调用写入 2 个值，使用者为每次调用读取 1 个值。

控制驱动的任务级并行

控制驱动的 TLP 适用于并行度建模，它依靠 C++ 的顺序语义，而非连续运行的线程。示例包括可按并发流水打拍方式（可能在循环内）执行的函数或者可搭配实参（非通道，而是 C++ 标量和阵列变量）执行的函数，这两种方式均适用于片上存储器和片外存储器。对于这类模型，Vitis HLS 在尽可能保留从原始 C++ 顺序执行获取的行为的同时，引入了并行度的概念。控制驱动的 TLP（或数据流）模型可提供：

- 后续函数可在前一个函数完成前启动
- 函数可在完成前重新启动
- 两个或两个以上顺序函数可同时启动

使用数据流模型时，Vitis HLS 会通过任务之间自动插入同步和通信机制来实现 C++ 代码的顺序语义。

数据流模型会利用此顺序函数序列来创建并发进程的任务级流水线架构。本工具通过推断并行任务和通道来完成此操作。设计人员会通过指定 [DATAFLOW](#) 编译指示或指令，以数据流样式（例如，函数体或循环主体）来指定要建模的区域，如下所示。本工具会扫描循环/函数体、抽取并行任务作为并行进程，并在这些进程之间建立通信通道。设计师还可以指引本工具选择通道类型，例如，FIFO (`hls::stream` 或 `#pragma HLS STREAM`) 或 PIPO (`hls::stream_of_blocks`)。数据流模型是可用于改进设计吞吐量和时延的强大方法。

要了解 Vitis HLS 将 C++ 代码变换为数据流模型的方式，请参阅如下所示 [simple_fifos](#) 示例。该示例使用 [DATAFLOW](#) 编译指示将数据流模型应用于顶层 `diamond` 函数，如下所示。

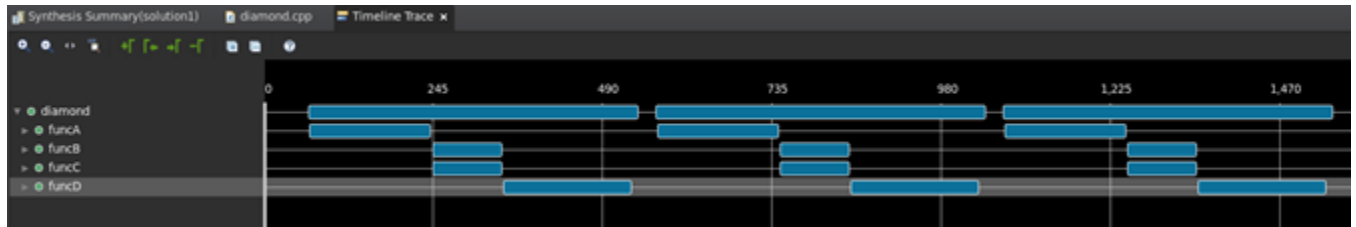
```
#include "diamond.h"

void diamond(data_t vecIn[N], data_t vecOut[N])
{
    data_t c1[N], c2[N], c3[N], c4[N];
    #pragma HLS dataflow
    funcA(vecIn, c1, c2);
    funcB(c1, c3);
    funcC(c2, c4);
    funcD(c3, c4, vecOut);
}
```

在以上示例中有 4 个函数：`funcA`、`funcB`、`funcC` 和 `funcD`。`funcB` 与 `funcC` 之间不存在任何数据依赖关系，因此可以并行执行。`funcA` 会从非本地存储器 (`vecIn`) 读取，需首先执行。同样，`funcD` 写入非本地存储器 (`vecOut`)，因此最后执行。

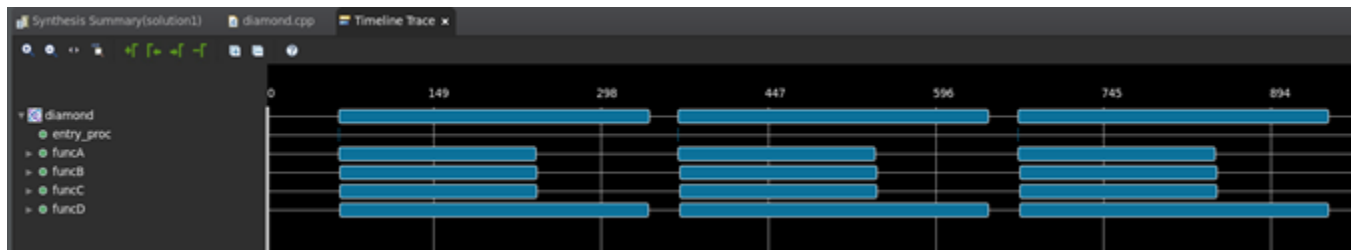
以下波形显示了此设计的执行剖析（无数据流模型）。测试平台对四个函数执行了 3 次调用。`funcA`、`funcB`、`funcC` 和 `funcD` 均按顺序执行。因此，每次调用这四个函数都需耗时总计 475 个周期，如下图所示。

图 12：无数据流的四个函数示例



在下图中，应用数据流模型时并且设计人员选择为通道使用 FIFO 时，控制器会立即启动所有函数，这些函数将停滞并等待输入。一旦输入到达，就会对其进行处理并发出。由于存在此类型的重叠，现在每次调用这四个函数都只需总计 275 个周期，如下所示。请参阅 [组合三种范例](#) 获取更多信息，其中详细探讨了可为此示例实现的并行度的类型。

图 13：含数据流的四个函数示例



要实现这种类型的并行化，势必引发硬件使用率提升。将某个特定区域（例如，函数体或循环主体）识别为要应用数据流模型的区域后，Vitis HLS 就会分析此函数体或循环主体，并基于 C++ 变量（例如，标量、数组或用户定义的通道，如 `hls::streams` 或 `hls::stream_of_blocks`）创建各通道，这些变量用于对数据流区域内的数据流动进行建模。这些通道对于标量变量而言可能只是简单的 FIFO，而对于非标量变量，则可能是乒乓 (PIPO) 缓冲器，此类非标量变量有阵列、块串流（前提是需将 FIFO 和 PIPO 行为与块的显式锁定加以组合）等。

其中每条通道均可包含用于指示通道已满或已空的额外信号。通过采用独立 FIFO 缓冲器和/或 PIPO 缓冲器，Vitis HLS 可使每项任务独立执行，吞吐量仅受输入和输出缓冲器的可用性限制。由此实现的任务执行重叠比正常流水打拍实现更好，但导致增加 FIFO 或块 RAM 寄存器（用于乒乓缓冲器）成本。



提示：仅当您运行设计的协同仿真后，才能看到此重叠执行优化，无法静态观测到此结果（但在 [数据流查看器](#) 中完成 C 语言综合后，能轻松想象到此结果）。

数据流模型并不局限于进程链，使用 `hls::streams` 时，该模型也可用于任何有向无环图 (DAG) 结构或周期结构。它可生成 2 种不同形式的重叠：在迭代内部重叠（前提是使用 FIFO 连接进程）以及在不同迭代之间重叠（前提是使用 PIPO 和 FIFO 连接进程）。这可能达成比静态流水打拍解决方案更好的性能。它将严格集中控制的流水线停滞理念替换为分布式握手架构，后者使用 FIFO 和/或 PIPO。将集中控制结构替换为分布式结构还有益于控制信号的扇出，例如，寄存器使能；此类扇出广泛分布于各进程的控制结构之间。请参阅 GitHub 上提供的[任务级并行度/控制驱动](#)示例，获取有关这些概念的更多示例。

数据流区域编码风格

`#pragma HLS dataflow` 常驻的代码区域被称为“数据流区域”。它包含外层循环或函数体中的所有语句（赋值、条件、循环、函数调用等）。

本节解释了以下两方面的信息：

- 数据流区域的编码风格，这种风格可以让 Vitis HLS 明确推断并行进程和通道的网络，确保“所见即所得”（又称 **WYSIWYG 编码风格**）。

- **更高级的数据流编码风格**，这种风格要求检查 Vitis GUI 中的综合结果，以确保已达成所期望的数据流结构。

上一节的 `diamond` 函数正是一个遵循该编码风格的示例，也称为“规范形式”。以下示例中，由于在数据流区域使用的代码并非函数调用，因此它使用的是“非规范”形式：

```
void noncanonical(int Input0, int Input1[], int &Output0, int Output1[]) {
    #pragma HLS dataflow
    int C0, C1[N], C2;
    C0 = Input0 * 3;
    for (int i = 0; i < N; i++) {
        C1[i] = Input1[0] + 2;
    }
    func2(C0, C1, C2);
    func3(C2, Output0, Output1);
}
```

此规范编码风格所产生的数据流网络会与源代码中给定的函数调用直接匹配，因此对于用户而言，在性能和结构两方面**可预测性更高**。非规范示例要求 Vitis HLS 创建网络，具体方式是分析循环、函数调用、控制、并遵循数据流规则将其分组到函数调用内。因此，对于用户而言，并非直接产生效果。无论采用何种形式，都建议您使用 Vitis IDE HLS 数据流查看器和协同仿真时间线轨迹来检查您的实现和性能。

Vitis HLS 还会针对非规范数据流语法发出一些消息。默认可发出警告消息，但可通过 `syn.dataflow.strict_mode` 配置选项将其改为错误或禁用，如 [数据流配置](#) 所述。

WYSIWYG 编码风格

本节定义了规范数据流编码风格，此编码风格可生成与源代码一一对应的数据流网络。在下一节中会介绍更多高级功能，在此类功能中，以下限制略为宽松。Vitis HLS 会通过对指令和控制进行分组来将非规范数据流区域转换为规范数据流区域，同时保持循环和函数调用之间的并发性。

数据流编译指示可应用在两种不同的上下文中：

- 在**函数**（也称为函数数据流区域）内。
- 在一种特殊类型的 **for 循环**（也称为循环数据流区域）内，目的是：
 - 使该循环成为函数体内部的**唯一语句**。
 - 在循环头中声明循环计数器，其类型为 `int`。
 - 在循环头中，将初始值设为任意非负整数常量。
 - 退出条件为，该循环变量与某个非负数常量或该循环的外围函数的标量实参进行“<”比较。
 - 循环变量按任意正整数常量递增。
 - 除循环计数器外，变量声明位于循环体内。

WYSIWYG 编码风格要求**该函数体或 for 循环只能包含以下序列**：

- 无初始化的**局部变量声明**，包括构造函数自动执行的声明。
 - 这些变量也称为“通道”，必须在生产变量的进程中进行初始化。
 - 对于具有无法重定义默认构造函数的标准数据类型（例如：`std::complex`），可使用 `no_ctor` 属性来避免初始化，例如，`std::complex<float> arr[SIZE] __attribute__((no_ctor));`
- **子函数调用**
 - 涉及类型转换或地址计算的实参不属于规范数据流，可能会导致产生额外进程或不同的网络结构。

· hls::task 例化

换言之，在“规范的”数据流区域内不得存在任何控制（if-then-else 和循环等控制结构会被自动转换为进程，这会导致需要 Vitis HLS 来推断网络，如上所述）。

这些被调用的子函数和 hls::task 可以是：

- 顺序函数或流水打拍函数，或者
- 函数数据流区域，或者
- 循环数据流区域。

建议的数据流函数风格示例如下：

```
void dataflow(int Input0, int Input1[], int &Output0, int Output1[]) {
#pragma HLS dataflow
    int C1[N], C2; // no initialization
    UserDataTypes C0 __attribute__((no_ctor)); // no_ctor must be used if the
    default constructor is not empty
    func1(Input0, Input1, C0, C1); // read Input0, read Input1, write C0,
    write C1
    func2(C0, C1, C2); // read C0, read C1, write C2
    func3(C2, Output0, Output1); // read C2, write Output0, write Output1
}
```

循环中建议的数据流风格示例如下（注意，N 可以是常量值，或者如本例所示，也可取数据流区域的输入实参）：

```
void dataflow(int Input0, int Input1[], int &Output0, int Output1[], int N)
{
    for (int i = 2; i < N; i += 2) {
#pragma HLS dataflow
        int C1[N], C2; // no initialization
        UserDataTypes C0 __attribute__((no_ctor)); // no_ctor must be used if
        the default constructor is not empty
        func1(Input0, Input1, C0, C1, i); // read Input0, read Input1, write
        C0, write C1
        func2(C0, C1, C2, i); // read C0, read C1, write C2
        func3(C2, Output0, Output1, i); // read C2, write Output0, write
        Output1
    }
}
```

注释：发生循环的函数不需要数据流编译指示，但必须只包含该循环。

关于数据流函数体或区域的进一步语义限制如下：

- 局部变量必须是**非静态**标量或阵列：
 - 但在规范数据流中，静态变量可以在（非数据流）进程内声明。但此进程不应有多个实例，否则对静态变量的访问会在这些实例之间创建隐藏的通信通道；
- 规范区域内 hls::task 的实例：
 - 必须声明为 hls_thread_local（例如，hls_thread_local hls::task t1(proc, arg1, arg2, arg3);）。
 - 数据流区域内 hls::stream 和 hls::stream_of_blocks 的实例同样如此（注意，hls_thread_local 类似于静态变量，但在此上下文中更合适，因为它并不会在具有相同函数体的多个 hls::task 之间暗含共享的单变量实例）。

- 进程之间必须使用局部变量来传输数据，这些局部变量也称为“通道”，可以是类似**标量**、**阵列**或**串流**的通道（`hls::streams` 和 `hls::stream_of_blocks`）。若使用下文所述的高级功能，或存在下文所述的例外情况，则不受此限制：
 - 类似阵列和串流的通道**只能有一个写入程序进程和一个读取程序进程**，且写入程序按文本顺序位于读取程序之前：
 - 如果以 `#pragma HLS bind_storage variable=... type=ram_1wnr` 标记阵列，则阵列可以有多个读取程序。在此情况下，编译器会自动复制该变量。
 - 标量通道可以有多个写入程序进程和读取程序进程（顶层标量除外），并自动转换为 FIFO 通道，每个通道有一个生产者和一个使用者。这些通道可以：
 - 在函数体内具有自动生成的显式 FIFO 写入和读取操作（这些被称为“标量传播 FIFO”）；
 - 或者由生产者的 `ap_done` 来自动写入，并由使用者的 `ap_ready` 自动读取（这些被称为“任务级 FIFO”）。
 - 通道**不得有循环携带依赖**：
 - 下文探讨了各种例外情况论，尤其是，类似反馈串流的通道可用于生成循环数据流网络。
- 顶层内核实参也可以传递给进程，但有**不同的限制**，因为：
 - **进程之间不得使用顶层内核实参来发生通信。**
 - 对于映射到 `m_axi` 接口的顶层阵列实参，写入程序进程（如有）必须位于读取程序（如有）之后，因为 Vitis HLS 假定顶层 `m_axi` 阵列不含携带依赖，即，**每次执行内核时都会在 DRAM 中接收到一个新的 `m_axi` 映射缓冲器。**

嵌套数据流区域陷阱

如果在函数/区域层级的最深层之外的任何层次（例如，数据流计算图的叶节点）存在任何非数据流区域，那么嵌套数据流区域的性能可能会受到巨大影响。这是因为顺序 FSM 不允许对其所含的（可能是数据流）区域进行重叠执行。

嵌套数据流陷阱示例 1：中间函数中缺少数据流编译指示。

```
void proc1(...) {
    #pragma HLS dataflow
    ...
}
void proc2(...) {
    ... // missing dataflow pragma
    proc1();
    ...
}
void top(...) {
    #pragma HLS dataflow
    ...
    proc2(...);
    ...
}
```

嵌套数据流陷阱示例 2：代码位于数据流循环外部。

```
void proc1(...) {
    #pragma HLS dataflow
    ...
}
void proc2(...) {
    a = x + y; // this code is NOT implemented as a dataflow process but as a
    sequential FSM
    for (int i = 0; i < N; i++) {
```

```

    #pragma HLS dataflow
    proc1(a, ...);
    ...
}
}
void top(...) {
    #pragma HLS dataflow
    ...
    proc2(...);
    ...
}

```

在上述两个示例中，proc2 不是数据流区域，这意味着其 ap_ready 和 ap_done 是同时生成的，即其 II 和时延完全相同。这会大大限制 proc1 和 proc2 的数据流吞吐量优势，即 II 小于时延。

高级数据流区域示例

除上文讨论的基础数据流编码风格外，还有更多高级功能可供使用。

参数化任务级并行度的展开与分区

第一项额外支持是一种用于**参数化任务级并行度**的机制，通过以下途径实现：

- 在数据流区域内展开数据流循环。需要借助一条数据流编译指示来确保在展开之前检查该区域是否正确，并且在展开之后，代码必须满足规范数据流条件。
- 阵列分区，其中将每个分区均作为一个独立变量（阵列或标量）来传递给一个进程。

例如，以下代码会创建一个流水线，它由 N 个相同进程组成，所有进程都执行相同功能，并通过 hls::streams 链加以级联，其中 N 是一个编译时常数：

```

void dut(int in[M], int out[M]) {
    #pragma HLS dataflow
    hls_thread_local hls::stream<int> chan[N+1]; // arrays of hls::streams
    are fully partitioned automatically
    read_in(in, chan[0]);
    hls_thread_local hls::task t[N]; // array of worker processes
    for (int i=0; i<N; i++) {
        #pragma HLS unroll
        #pragma HLS dataflow
        t[i](worker, chan[i], chan[i+1]);
    }
    write_out(chan[N], out);
}

```

以下是另一个示例，其中改用 PIPO 链（或者如果添加 #pragma HLS stream variable=chan，则改用串流阵列）：

```

void dut(int in[M], int out[M]) {
    #pragma HLS dataflow
    int chan[N+1][M]; // partitioned into N+1 arrays of M elements
    #pragma HLS array_partition complete dim=1 variable=chan
    read_in(in, chan[0]);
    for (int i=0; i<N; i++) {
        #pragma HLS unroll
        #pragma HLS dataflow
        worker(chan[i], chan[i+1]);
    }
    write_out(chan[N], out);
}

```

最后，以下示例执行的操作是部分分区和部分展开，同样能达到创建数据流网络并在分区和展开后满足单生产者单使用者要求的目标。这会生成两个并行进程，各自作用于不同的阵列：

```
void dut(int a[10], int b[10]) {
    #pragma HLS array_partition cyclic factor=2 variable=a
    #pragma HLS array_partition cyclic factor=2 variable=b
    for (int i=0; i<10; i++) {
        #pragma HLS dataflow
        #pragma HLS unroll factor=2
        compute(a[i], b[i]); // read a[i], write b[i]
    }
}
```

对循环携带依赖的特殊支持

“在数据流区域内不得存在循环携带依赖”这一规则，在以下 3 种情况下可以适当放宽：

- 串流和块串流，请参阅下文；
- 单个非数据流进程内的静态标量，先读取（上一次调用）后写入，随后可能还有读取程序：在此情况下，会在该进程本身内部满足任何循环携带依赖；
- 通道阵列，含有单个写入程序，并且该写入程序按文本顺序位于单个读取程序之前（请参阅 #pragma HLS stream type=shared）：在此情况下，流程依赖保持正向，但可以是循环携带依赖。

用于实现反馈与循环携带依赖的串流

如上所述，hls::streams 和 hls::stream_of_blocks 可用于反向传输数据（向按文本顺序更早出现的进程传输数据），并在用户控制下实现循环携带依赖。

若有进程要从这些反馈串流读取数据，那么请务必注意确保这类进程仅在后续进程产生一些数据之后才会尝试读取数据。满足此要求的最常见的方法是在首次执行时，使用变量跳过读取操作。此类变量可以是：

- 在进程函数中计数的静态变量（请注意，如前文所述，不得复制该进程）；
- 或者外围数据流循环的循环计数器，例如，如以下代码所示，计算阵列元素的部分求和（以一种奇怪的方式计算，仅作参考用途）：

```
void read_and_add(int a[N], hls::stream<int> &b, hls::stream<int> &f, int i) {
    #pragma HLS pipeline II=1 style=flp // flushing pipeline is needed to avoid deadlocks
    int t=0;
    if (i>0) t = f.read(); // reading previous sum, except first time
    b.write(a[i]+t); // adding and passing
}

void write_and_feedback(hls::stream<int> &a, int b[N], hls::stream<int> &f, int i) {
    #pragma HLS pipeline II=1 style=flp
    int aa=a.read();
    if (i<N-1) f.write(aa); // passing back current sum
    b[i] = aa; // writing it out
}

void dut(int a[N], int b[N]) {
    for (int i=0; i<N; i++) {
        #pragma HLS DATAFLOW
        static hls::stream<int> f; // feedback stream, here could also be declared without static, above the loop
    }
}
```

```

hls::stream<int> c; // forward stream

read_and_add(a, c, f, i);
write_and_feedback(c, b, f, i);
}
}

```

- 或者 `hls::task` 中的 `hls_thread_local` 变量，用法相同（在 `hls::task` 上下文中，`hls_thread_local` 更好，因为它能确保 `hls::task` 的多个实例之间无隐藏通信）。

配置数据流存储器通道

Vitis HLS 会根据用户选择，将任务之间的通道作为 PIPO 或 FIFO 缓冲器来加以实现：

- 对于标量，Vitis HLS 会自动推断 FIFO 作为通道类型。
- 如果生产者或使用者的参数为阵列，那么用户可以基于以下考量来选择将通道作为 PIPO 还是 FIFO 来实现：
 - 如果始终顺序访问数据，那么用户可以选择将此存储器通道作为 PIPO/FIFO 来实现。选择 PIPO 的优势在于，PIPO 从不发生死锁，但需要耗用更多存储器。选择 FIFO 的优势在于，所需存储器较少，但如果 FIFO 大小不正确，则存在发生死锁的风险。
 - 如果按任意顺序访问数据，那么存储器通道必须作为 PIPO 来实现（默认大小是原始阵列的两倍）。



提示：PIPO 可确保通道始终具有保存在一次迭代内生成的所有样本（不出现丢失）所需的容量。

指定 FIFO 通道大小会覆盖该工具计算所得的默认值，以尝试对吞吐量进行优化。如果设计中的任何函数生成或使用的样本都可大于指定的 FIFO 大小，那么 FIFO 可能变为空（或满）。在此情况下，设计将停止操作，因为它无法读取（或写入）。这可能导致死锁停滞状态。



提示：如果造成死锁状态，那么仅当执行 C/RTL 协同仿真时或者在完整的系统中使用此块时，您才会看到此状态。

设置 FIFO 的深度时，AMD 建议将初始深度设置为传输的数据值的最大数量（例如，任务间传递的阵列的大小）、确认设计能够成功通过 C/RTL 协同仿真，然后减小 FIFO 大小并确认 C/RTL 协同仿真仍可成功完成且不会出现任何问题。如果 RTL 协同仿真失败，那么 FIFO 大小可能太小，而无法阻止出现停滞或死锁状况。Vitis HLS GUI 现在支持自动判定要使用的适当 FIFO 大小。此外，运行 RTL 协同仿真后，Vitis HLS IDE 可显示一长段时间内每个 FIFO/PIPO 缓冲器的大小的直方图。这有助于判定每个缓冲器的最佳深度。

将阵列指定为 PIPO 或 FIFO

默认情况下，所有阵列均作为乒乓缓存来实现以启用随机访问。这些缓冲器也可根据需要调整大小。例如，在某些情况下（如绕过任务时），可能会出现性能劣化。为缓解对性能造成的影响，可通过使用 STREAM 编译指示或指令来调整这些缓冲器的大小，以向生产者 (producer) 和使用者 (consumer) 提供更多裕量，如下所示。

```

void top ( ... ) {
#pragma HLS dataflow
    int A[1024];
#pragma HLS stream type=pipo variable=A depth=3

    producer(A, B, ...); // producer writes A and B
    middle(B, C, ...);   // middle reads B and writes C
    consumer(A, C, ...); // consumer reads A and C
}

```

接口上的阵列默认定义为 `m_axi` 或 `ap_memory`。但如果 `INTERFACE` 编译指示或指令将阵列定义为 `ap_fifo` 或 `axis`，则可将阵列指定为串流。

在设计内，如果实现需要 FIFO，必须使用 `STREAM` 编译指示或指令将阵列指定为串流。



提示：对阵列应用 `STREAM` 指令时，硬件内实现生成的 FIFO 包含的元素数量与阵列同样多。但 `depth` 选项可用于指定 FIFO 的大小，如 [syn.directive.stream](#) 中所述。

`STREAM` 编译指示或指令也可用于覆盖 `syn.dataflow.fifo_depth` 配置命令为 `DATAFLOW` 区域内的任意阵列指定的默认实现，如 [数据流配置](#) 中所述。总体上，建议对给定阵列使用 `STREAM` 编译指示或指令，而不是使用全局选项。

- 如果 `syn.dataflow.default_channel` 设置为乒乓，那么只要对任意阵列应用 `STREAM` 指令，就仍能将该阵列作为 FIFO 来实现。



提示：要使用 FIFO 实现，必须以顺序方式（而非随机访问顺序）来访问阵列。

- 如果 `syn.dataflow.default_channel` 设为 FIFO，那么只要对阵列的 `STREAM` 编译指示或指令应用 `type=pipo`，就仍能将该阵列作为乒乓实现来加以实现。



重要提示！要保留顺序访问以确保正确执行串流，可能需要使用 `volatile` 限定符来阻止编译器优化（尤其是死码消除）。

当 `DATAFLOW` 区域中的阵列指定为作为 FIFO 来进行串流和实现时，通常 FIFO 无需保留与原始阵列相同数量的元素。当数据采样可用后，`DATAFLOW` 区域中的任务会立即使用该数据采样。FIFO 深度可使用 `syn.dataflow.fifo_depth` 选项或者 `STREAM` 编译指示或指令搭配 `depth` 选项来指定。这可用于设置 FIFO 大小，以确保数据流从不发生停滞。

如果选中 `type=pipo` 选项，`depth` 选项会设为 PIPO 的深度（块数）。

将阵列指定为块串流

`hls::stream_of_blocks` 类型可提供用户同步的串流，支持对数据块进行串流以供数据流上下文中的进程级别接口使用，在此类上下文中每个块都是单个阵列或多维阵列。块串流的目标用途是替换数据流区域内的成对进程之间的基于阵列的通信。请参阅 GitHub 上的 [using_stream_of_blocks](#) 示例。

当前，Vitis HLS 可在数据流区域内实现由生产者进程写入并由使用者进程读取的阵列，方法是这些阵列映射到乒乓缓冲器 (PIPO)。在 C++ 中返回生产者函数和调用使用者函数时，PIPO 缓冲器就会发生缓冲器交换。

块串流建模样式

从另一方面来看，对于块串流，生产者与使用者之间的通信以类阵列对象串流方式来建模，这样与通过 PIPO 进行阵列传输相比存在多方面优势。

在代码中使用块串流需要以下 `include` 文件：

```
#include "hls_streamofblocks.h"
```

块串流对象模板为：

```
hls::stream_of_blocks<block_type, depth> v
```

其中：

- `<block_type>` 用于指定块串流所包含的阵列或多位阵列的数据类型
- `<depth>` 是可选实参，可提供与 `hls::stream` 或 `PIPO` 相似的深度控制，并指定块的总数，包括在任意时间生产者获取的块和使用者获取的块。默认值为 2
- `v` 为块串流对象指定变量名称

使用以下步骤访问块串流中的某一个块：

1. 如果生产者或使用者进程要首先访问串流，则需要使用 `hls::write_lock` 或 `hls::read_lock` 对象获取对该串流的访问权。
2. 当生产者获取锁定后，它即可开始写入（或读取）获取的块。此块完全初始化后，当 `write_lock` 对象超出范围时，生产者即可将此块释放。

注释：具有 `write_lock` 的生产者进程也可以读取此块，前提是它从已写入的位置读取，因为新获取的块必须假定包含未初始化的数据。仅限生产者进程才能对块进行写入和读取，不支持使用者执行此操作。

3. 随后，此块将在块串流中以 FIFO 方式排队，当使用者获取 `read_lock` 对象后，此块即可供使用者进程读取。

前述示例中所示的 `hls::stream_of_blocks` 与 `PIPO` 机制之间的主要差异在于一旦 `write_lock` 超出范围，块就会变为对使用者可用，而不是仅在生产者进程返回时才可用。因此，含块串流时的存储量远小于含 `PIPO` 时的存储量：即，此时存储量为 $2N$ 而不是 $2 \times M \times N$ 。

生产者通过构造名为 `b` 的 `hls::write_lock` 对象，并将名为 `s` 的块串流对象引用传递给该对象，即可获取此块。`write_lock` 对象可提供重载阵列访问运算符，这样即可像阵列一样访问此对象，以便按随机顺序访问底层存储器，如以下示例所示。

通过构造 `write_lock/read_lock` 对象即可执行锁定的获取，当对象超出范围时，就会被销毁，从而自动释放锁定。此方法使用常用的“资源获取即初始化” (RAII) 样式的锁定和解锁。

```
#include "hls_streamofblocks.h"
typedef int buf[N];
void producer (hls::stream_of_blocks<buf> &s, ...) {
    for (int i = 0; i < M; i++) {
        // Allocation of hls::write_lock acquires the block for the producer
        hls::write_lock<buf> b(s);
        for (int j = 0; j < N; j++)
            b[f(j)] = ...;
        // Deallocation of hls::write_lock releases the block for the consumer
    }
}

void consumer(hls::stream_of_blocks<buf> &s, ...) {
    for (int i = 0; i < M; i++) {
        // Allocation of hls::read_lock acquires the block for the consumer
        hls::read_lock<buf> b(s);
        for (int j = 0; j < N; j++)
            ... = b[g(j)] ...;
        // Deallocation of hls::write_lock releases the block to be reused by
        the producer
    }
}

void top(...) {
#pragma HLS dataflow
    hls::stream_of_blocks<buf> s;

    producer(b, ...);
    consumer(b, ...);
}
```

此方法的关键特征包括：

- 以上生产者的外层循环的目标性能是达成总体启动时间间隔 (II) 为 1 的目标
- 锁定的块虽可使用，但它仅供生产者或使用者进程专用，直至被释放为止。
- 生产者的阵列对象的初始状态并未定义，但它包含由生产者为用户写入的值。
- 块串流的主要优势在于允许重叠执行使用者和生产者的多次迭代，以提升吞吐量。

资源使用

当深度增加超过默认值 2 时，资源成本与 PIPO 的资源成本类似。换言之，每递增 1，都需要足够一个块使用的存储器，例如，在以上示例中，需要 $N * 32$ 位字。

块串流对象可绑定到特定 RAM 类型，方法是在声明块串流的位置放置 `BIND_STORAGE` 编译指示，例如，在顶层函数内。默认情况下，块串流使用 2 端口块 RAM (`type=RAM_2P`)。

指定编译器创建的 FIFO 深度

起始传输 FIFO

编译器可能自动创建“起始 FIFO”以将 `ap_start/ap_ready` 握手传输到内部进程。此类 FIFO 有时可能成为性能瓶颈，在此情况下，工具可能错误估算默认大小，您可通过 `syn.dataflow.start_fifo_depth` 配置命令增大默认大小。

如果生产者与使用者之间需无限裕量，并且内部进程可永久运行，并完全由其输入或输出（FIFO 或 PIPO）以安全方式驱动，那么针对给定数据流区域，可使用以下编译指示或指令在本地移除这些起始 FIFO（风险由您自行承担）：

```
#pragma HLS DATAFLOW disable_start_propagation
```

标量传输 FIFO

编译器通过进程之间的“标量 FIFO”自动从 C/C++ 代码传输部分标量。此类 FIFO 有时可能成为性能瓶颈或者导致死锁，在此情况下，您可通过 `syn.dataflow.scalar_fifo_depth` 配置命令来设置大小。



提示：默认深度是使用 `syn.dataflow.fifo_depth` 来设置的。

Stable 阵列

`stable` 编译指示可用于标记数据流区域的输入变量或输出变量。其作用是移除其对应的任务级同步，假定用户保证此移除行为确实正确无误。

```
void dataflow_region(int A[...], ...
#pragma HLS stable variable=A
#pragma HLS dataflow
    proc1(...);
    proc2(A, ...);
```

如果不使用 `stable` 编译指示，并假定 A 由 `proc2` 读取，那么 `proc2` 会成为其所在位置的数据流区域内执行的初始同步的一部分。这意味着仅当 `proc2` 同样准备好重新启动后，`proc1` 才能重新启动，这会阻止数据流迭代重叠，从而可能导致性能下降。`stable` 编译指示表明无需此同步亦可保持操作正确性。

通过 `stable` 编译指示，编译器假定：

- 如果 A 由 `proc2` 读取，那么执行 `dataflow_region` 时，读取的存储器位置仍可访问，任何其他进程或调用上下文都无法覆盖这些存储器位置。
- 如果 A 由 `proc2` 写入，那么执行 `dataflow_region` 时，任何其他进程或调用上下文在定义写入的存储器位置前都无法读取这些位置。

典型场景是仅当数据流区域尚未开始执行或者已完成执行时，调用程序才会更新或读取这些变量。

总之，数据流优化是一种强大的优化，可以显著改善设计的吞吐量。由于需依靠 HLS 来对设计内可用并行度执行推断，因此需要设计师来辅助确保代码编写方式可便于 HLS 进行明确推断。最后，在某些情况下，设计师仍会发现需要在同一设计内同时部署“Dataflow”（数据流）模型和“Task-Channel”（任务通道）模型。下一章节描述了这种混用组合模型，这种模型能够产生一些有趣的设计。

混用数据驱动的控制模型与控制驱动的数据模型

下表着重讲解了 HLS 设计的各项要素，这些要素有助于您判定何时应用控制驱动的任务级并行度 (TLP) 或数据驱动的 TLP。

表 3：控制驱动的 TLP 和数据驱动 TLP

控制驱动的 TLP	数据驱动的 TLP
HLS 设计需要通过控制信号来启动/停止进程	HLS 设计使用完全数据驱动的方法，无需控制信号来启动/停止进程
设计需要非本地存储器访问权限	设计使用纯串流数据传输
设计需要与外部软件应用进行交互	设计具有数据相关的多重速率行为： · 生产者写入数据或者使用者读取数据的速率与数据有关 · 为需要在进程间保留反馈的设计简化建模方式
设计具有多个进程，并且这些进程运行的执行数相同	可在 C 语言仿真和 RTL 仿真中观察到任务级并行度。
需要 RTL 仿真来对并行效果进行建模	

正如上表所示，所提供的两张形式的任务级并行度具有不同的用例和优势。但有时，要将整个应用设计成纯数据驱动的 TLP，就不可能同时还将设计的某些部分构造为纯串流设计。在此情况下，适合使用混用控制驱动/数据驱动的建模来创建该应用。请参阅来自 GitHub 的以下 [mixed_control_and_data_driven](#) 示例。

```
void dut(int in[N], int out[N], int n) {
#pragma HLS dataflow
    hls_thread_local hls::split::round_robin<int, NP> split1;
    hls_thread_local hls::merge::round_robin<int, NP> merge1;

    read_in(in, n, split1.in);

    // Task-Channels
    hls_thread_local hls::task t[NP];
    for (int i=0; i<NP; i++) {
#pragma HLS unroll
        t[i](worker, split1.out[i], merge1.in[i]);
    }

    write_out(merge1.out, out, n);
}
```

在以上示例中，有两个不同的区域，一个是数据流区域，其中包含 `read_in/write_out` 函数，这些函数中的顺序语义将予以保留，例如，先执行 `read_in`，后执行 `write_out`，另一个是任务通道区域，其中包含 4 项任务（因为在此示例中 $NP = 4$ ）的动态例化以及一些特殊类型的通道，称为 `split` 通道或 `merge` 通道。`split`（拆分）通道具有单个输入和多个输出，在此例中，拆分通道具有 4 个输出，如 [HLS 拆分/合并库](#) 中所述。同样，`merge`（合并）通道具有多个输入和单个输出。

此外，对于端口，这些通道也支持内部作业调度器。在以上示例中，合并通道与拆分通道均已选中循环调度器来将传入数据逐一分配到这 4 个任务，从 `worker_U0` 开始。如果已选中负载均衡调度器，那么传入数据已分配给首个可用的工作程序任务（这会导致非确定性仿真，因为每次运行仿真时，此顺序都可能不同）。由于这是纯任务通道区域，只要这 4 项任务的传入串流中存在数据，这些任务就会立即并行执行。请参阅 GitHub 上提供的 [merge_split](#) 示例，获取有关这些概念的更多示例。

请务必注意，虽然以上代码可能看似在循环中“调用”了每个任务，并且每次执行循环主体时，任务都可能连接到不同的通道对，但实际上，这种用法暗示静态例化，即：

- 每次执行 `dut()` 时，都只能执行一次 `t[i](...)` 调用。
- 高于 `i` 的循环必须完全展开，以便推断 RTL 中对应的一组 4 个实例。
- 测试平台只能调用一次 `dut()` 函数。
- 每个拆分输出或合并输入必须仅限绑定到一个 `hls::task` 实例。

虽然规范的顺序对于 `hls::task` 对象无关紧要，但对于控制驱动的数据流网络，Vitis HLS 必须能够看到存在的进程链，例如，从 `read_in` 到 `write_out`。Vitis HLS 使用调用顺序来定义此进程链，对于 `hls::tasks` 而言，此调用顺序同时也是声明顺序。这意味着该模型必须在数据流区域内定义从 `read_in` 函数到 `hls::task` 区域以及最后到 `write_out` 函数的显式顺序，如以上示例所示。

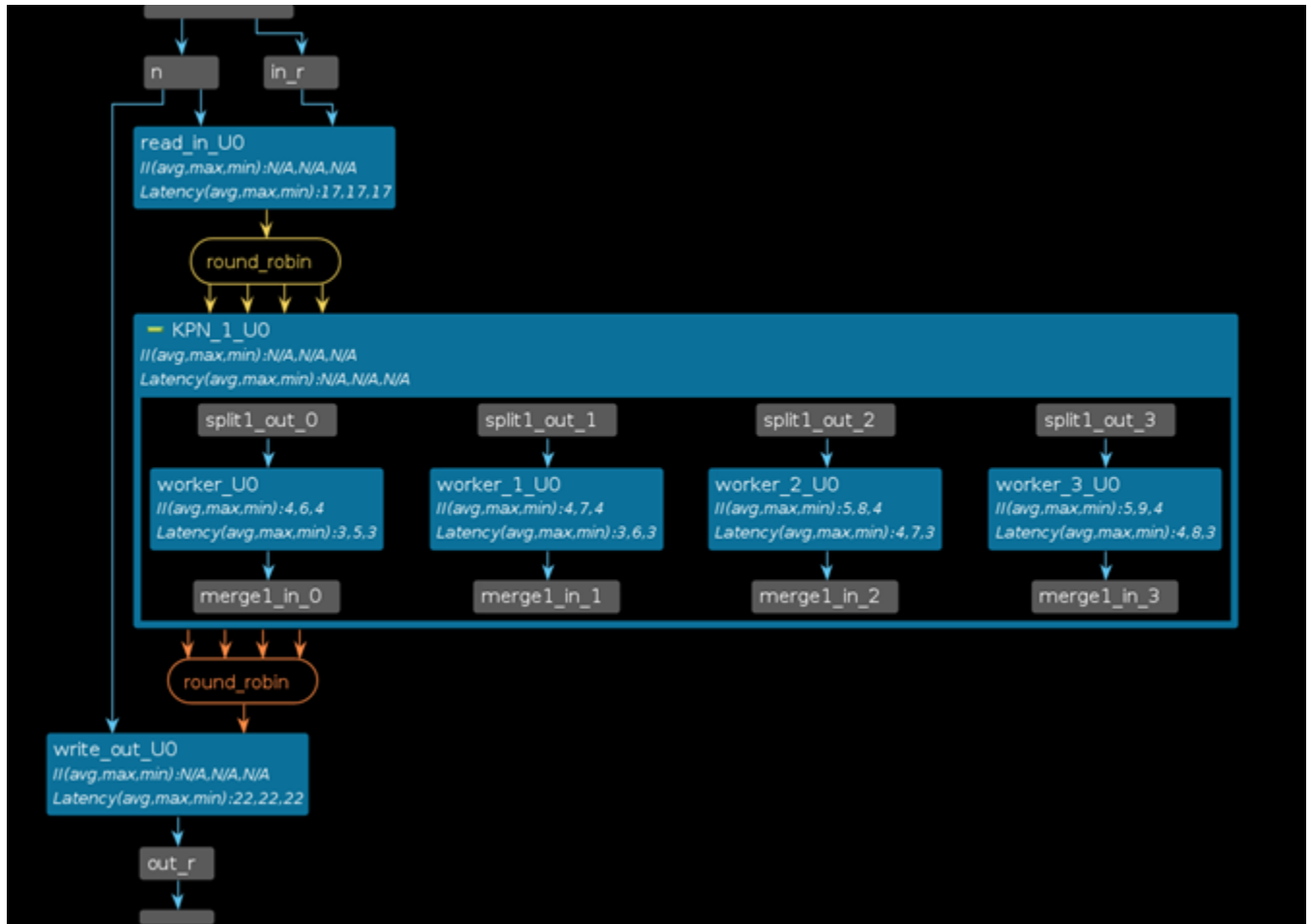
通常：

- 如果基于控制的进程（如，常规数据流）为每个 `hls::task` 生成一条串流，那么必须先调用该进程，然后才能调用代码中的任务声明
- 如果基于控制的进程耗用来自某个 `hls::task` 的一条串流，那么必须先调用代码中的任务声明，然后再调用该进程

违反上述规则可能导致意外结果，因为每个 NP `hls::task` 实例都静态绑定到 `t[i](...)` 的首次调用中使用的通道。

下图显示了此混用任务通道与数据流的图例：

图 14：混用任务通道与数据流



总结

如果 HLS 设计为纯数据驱动的设计，且无需与软件应用进行任何交互，那么可以使用数据驱动的 TLP 模型来建模。此类设计的示例有：

- 基于简单规则的“防火墙”，且“规则”编译到内核中
- 快速傅里叶变换，且配置数据编译到内核中
- FIR 滤波器，且系数编译到内核中

如果设计需要往来外部存储器执行数据传输，那么可使用控制驱动的 TLP 模型。此类设计的示例有：

- 网络路由器，其中路由表必须完全更新后才能执行内核
- 使用散列映射向服务器发送数据的负载均衡器，这类负载均衡器必须同时更新服务器列表、服务器映射以及对应的 IP 地址

但大部分设计均可混用控制驱动与数据驱动的模式，需要对外部存储器进行部分访问，支持 HLS 内的并行任务与流水打拍任务之间的串流。

总之，本章描述了设计以 C/C++ 编写的应用时可供您考量的建模选择。至此，此处探讨了有关构造高层次算法以利用这些特殊模型（例如，任务通道优化或数据流优化）的方法。达成良好的吞吐量的另一个关键要素是，指令级并行度也应一并纳入考量。HLS 中的指令集并行度指的是在循环、函数甚至阵列内部有效执行并行运算的能力。接下来的几个章节会为您逐步详解这些低层次优化与宏级别优化的搭配使用方式。

循环入门

编写用于高层次综合 (HLS) 的代码时，经常需要实现重复算法来处理数据块，例如，信号处理或图像处理。通常，C/C++ 源代码倾向于包含多个循环或多个嵌套循环。

需要进行性能优化时，循环是最适合开始尝试优化的位置。在硬件中，执行每次循环迭代都需要至少 1 个时钟周期。从硬件角度来看，循环主体隐含“等待一个时钟周期”的意义。仅当前一次循环迭代完成之后，才能开始后一次迭代。为了提升性能，循环常采用“流水打拍”或“展开”，以便充分利用 FPGA 架构的高度分布化和并行化，正如后续章节中所述。

循环流水打拍

流水打拍循环允许在前一次循环迭代完成前就启动后一次循环迭代，从而支持部分循环在执行时重叠。默认情况下，循环的每次迭代仅在前一次迭代完成后才会开始。在以下循环示例中，单次循环迭代会添加 2 个变量，并将结果存储在 第 3 个变量中。假定在硬件中，此循环需要 3 个周期才能完成 1 次迭代。同时假定循环变量 `len` 为 20，即，`vadd` 循环在内核内运行 20 次迭代。因此，需要总计 60 个时钟周期（20 次迭代 * 3 个周期），才能完成此循环的所有操作。

```
vadd: for(int i = 0; i < len; i++) {
    c[i] = a[i] + b[i];
}
```



提示：最好始终按以上示例所示方式来标记循环 (`vadd:...`)。此实践有助于在 Vitis HLS 中进行设计调试。有时，未使用的标记在编译期间会生成警告，用户可放心忽略此警告。

循环流水打拍支持该循环的后续迭代发生重叠且以并发方式运行。您可通过在循环主体内添加 PIPELINE 编译指示或指令来启用循环流水打拍，如下所示：

```
vadd: for(int i = 0; i < len; i++) {
    #pragma HLS PIPELINE
    c[i] = a[i] + b[i];
}
```

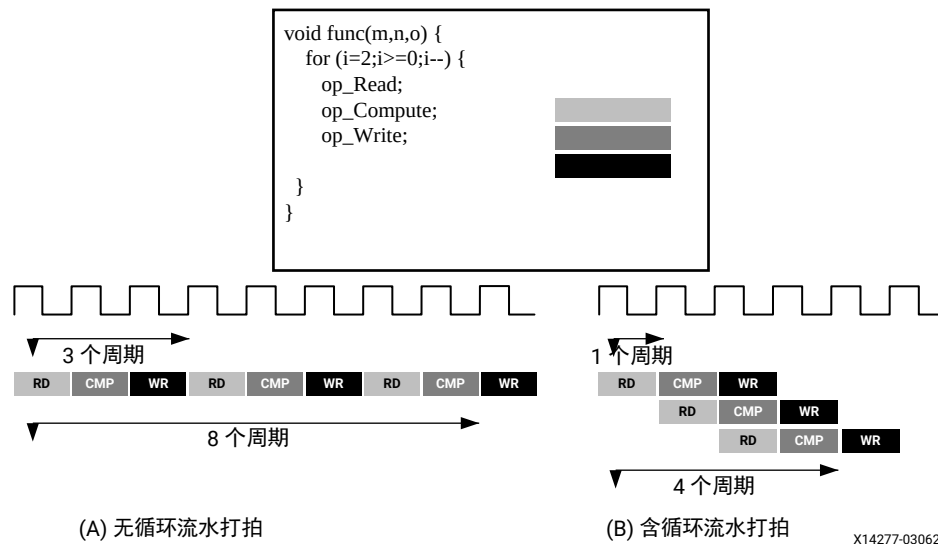


提示：Vitis HLS 会对含超过 64 次迭代的循环自动进行流水打拍。此功能特性可使用 `syn.compile.pipeline_loops` 配置命令加以更改或者禁用，如 [编译选项](#) 中所述。

启动下一次循环迭代所需的周期数称为流水打拍循环的启动时间间隔 (Initiation Interval, II)。因此 $II = 2$ 表示循环的下一次迭代会在当前迭代的 2 个周期后启动。 $II = 1$ 是理想情况，即每个周期内都会启动一次循环迭代。使用 [pragma HLS pipeline](#) 时，您可以指定编译器要实现的 II。如果不指定目标 II，那么默认情况下编译器可尝试实现 $II=1$ 。

下图显示了流水打拍循环与非流水打拍循环的执行差异。在下图中，(A) 显示了默认顺序操作，每次输入读操作间存在 3 个时钟周期 ($II = 3$)，并且需要经过 8 个时钟周期后才会执行最后一次输出写操作。

图 15：循环流水打拍



在 (B) 所示的循环的流水打拍版本中，每个周期都会读取一次新输入样本 ($II = 1$)，仅需 4 个时钟周期后即可写入最终输出，在使用相同硬件资源的前提下显著改善 II 和时延。



重要提示！ 循环流水打拍会导致自动展开流水打拍循环内部嵌套的任意循环。

如果循环内部存在任何数据依赖关系，则可能无法实现 $II = 1$ ，并且可能导致启动时间间隔增大。循环依赖关系属于数据依赖关系，可能对循环优化（通常为流水打拍）施加约束。此类依赖关系可出现在任一循环的单一迭代内或者也可出现在任一循环的不同迭代内。了解循环依赖关系的最简单的方法是检验极端示例。在以下示例中，循环结果用作为循环持续条件或循环退出条件。循环的每次迭代必须完成后才能开始下一次迭代。

```

Minim_Loop: while (a != b) {
  if (a > b) a -= b;
  else b -= a;
}

```

以上示例中的 `Minim_Loop` 循环无法流水打拍，因为前一次迭代结束后，后一次循环迭代才能开始。并非所有循环依赖关系都如此极端，但此示例着重展示了部分操作在某些其他操作完成后才能开始的情况。解决方案是尝试确保尽早执行初始运算。

循环依赖关系可能发生于所有类型的数据。使用阵列时尤其常见。

自动循环流水打拍

`syn.compile.pipeline_loops` 配置命令支持根据迭代计数对循环进行自动流水打拍。循环次数大于指定限值的循环都会自动流水打拍。默认限值为 64。

给定以下示例代码：

```

loop1: for (i = 0; i < 5; i++) {
  // do something 5 times ...
}

```

如果 `syn.compile.pipeline_loops` 命令设为 4，那么 `for` 循环 (`loop1`) 会进行流水打拍。这等同于添加 PIPELINE 编译指示，如下代码片段所示：

```
loop1: for (i = 0; i < 5; i++) {
  #pragma HLS PIPELINE II=1
  // do something 5 times ...
}
```

但对于嵌套循环，该工具会从最内层循环开始，判定它是否能流水打拍，然后沿循环嵌套上移。在以下示例中，`syn.compile.pipeline_loops` 阈值仍为 4，最内层循环 (`loop1`) 标记为等待流水打拍。随后，该工具会对这对循环的父循环 (`loop2`) 进行评估，发现它也能流水打拍。在此情况下，该工具会将 `loop1` 展开到 `loop2` 内，并将 `loop2` 标记为等待流水打拍。随后，它将上移至父循环 (`loop3`) 并重复分析操作。

```
loop3: for (y = 0; y < 480; y++) {
  loop2: for (x = 0; x < 640; x++) {
    loop1: for (i = 0; i < 5; i++) {
      // do something 5 times ...
    }
  }
}
```

如果设计中有部分无需使用自动流水打拍的循环，请对此类循环应用含 `off` 选项的 PIPELINE 指令。`off` 选项可阻止自动循环流水打拍。



重要提示！ 该工具会在执行所有用户指定的指令后应用 `syn.compile.pipeline_loops` 命令。例如，如果向循环应用用户指定的 UNROLL 指令，那么会首先展开该循环，但无法应用自动循环流水打拍，因为此循环已被消除。

如果 `pipeline_loops` 阈值设为 0，则关闭所有自动流水打拍（除非在循环中添加了覆盖该设置的编译指示）。而且，无论如何设置该阈值，所示示例始终会自动执行流水打拍。以下列出了工具是否自动执行流水打拍的具体条件。

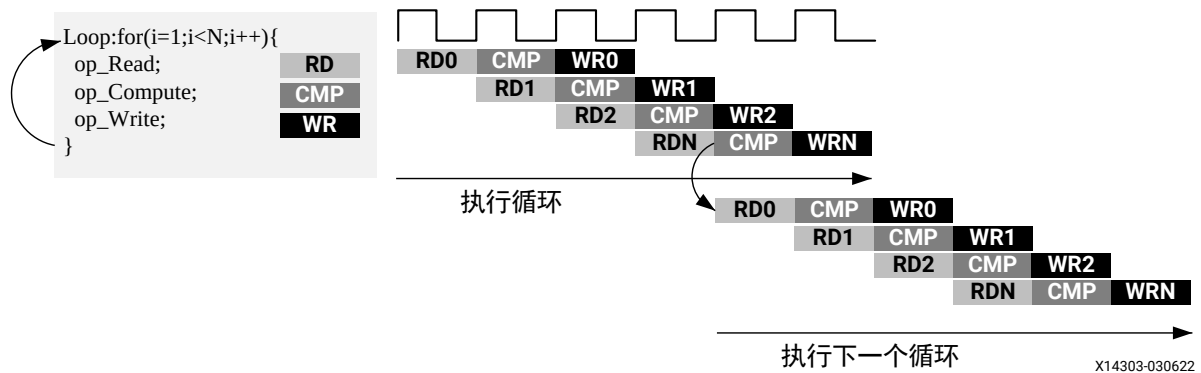
- 当循环位于最内层且循环次数未知，或者循环次数大于等于阈值，或者其上层没有任何循环时，该循环就会执行流水打拍，除非针对该循环显式关闭流水打拍。
- 如果已知循环的循环次数 < 阈值，并且其上层存在循环，那么，请检查是否可从上层执行流水打拍：此类上层循环的条件是只能包含同样也将展开的子循环（例如，其循环次数 < 阈值）。如果可行，则将流水打拍置于该上层循环内。
- 实际上，对于上层循环也适用相同的条件，累计循环次数才是至关重要的。例如，已展开的迭代次数乘以当前循环次数，并且逐层向上计算直至达到阈值。

回绕已流水打拍的循环以保障性能

PIPELINE 编译指示具有名为 `rewind` 的选项。该选项支持以重叠方式执行流水打拍循环的后续调用，前提是此循环为数据流区域或顶层函数的最外层的构造（并且该数据流区域多次执行）。

下图显示了对循环进行流水打拍时使用 `rewind` 选项的操作。循环迭代计数结束时，循环将开始再次执行。虽然一般情况下它立即重新执行，但也可能出现延迟，在 GUI 中可显示并描述此延迟。

图 16: 含回绕 (Rewind) 选项的循环流水打拍



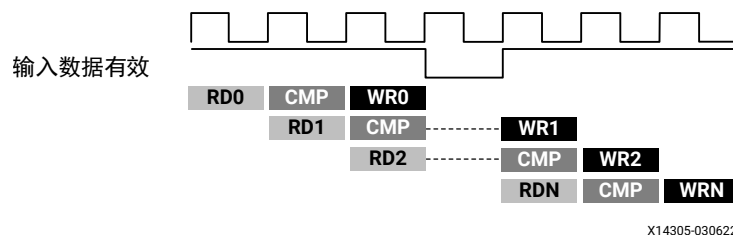
注释：如果在 DATAFLOW 区域周围使用循环，Vitis HLS 会自动将其实现以允许后续执行进行重叠。

刷新流水线和流水线类型

刷新流水线

只要在流水线输入端数据可用，流水线就会持续执行。如果没有可供处理的数据，流水线就会停滞。如下图所示，Input Data Valid 信号转为低电平，表示没有更多有效的输入数据。当此信号变为高电平状态后，即表示有新数据可供处理，那么流水线就会继续操作。

图 17: 含停滞的循环流水打拍



在某些情况下，最好采用可“清空”或可“刷新”的流水线。提供 `flush` 选项即可执行此操作。当“刷新”流水线时，如果没有新输入可用（由流水线起始位置的数据 `valid` 信号来判定），流水线就会停止读取，但仍会继续处理并关闭后续每个流水线阶段，直至最终输入已处理至流水线输出为止。

如下文所述，本工具会自动选择正确的流水线样式，用于给定的流水打拍循环或函数。但您可使用 `syn.compile.pipeline_style` 配置命令指定流水线样式来覆盖此默认行为。

您也可以使用 PIPELINE 编译指示或指令为函数或循环指定流水线样式。该选项适用于编译指示或指令的特定范围，不更改 `syn.compile.pipeline_style` 分配的全局默认设置。

`stp` 和 `flp` 类型的流水线都使用标准流水线逻辑，其中硬件流水线使用各种阻塞信号来使流水线停滞。这些阻塞信号常变为高扇出信号线的驱动程序，在物理阶段数量较多的流水线和处理大量数据的流水线上尤其如此。此类高扇出信号线一旦创建后，就会成为时序收敛问题的主要原因，这些问题在 RTL/逻辑综合或布局布线期间无法修复。为解决此问题，我们创建了名为“自由运行的流水线”（或称为 `frp`）。自由运行的流水线是适用于处理含阻塞信号操作的流水线的最高效的架构。原因在于

- 它完全消除了与寄存器使能的阻塞信号连接

- 它是完全可刷新的流水线，允许为无效传输事务生成气泡
- 不同于先前分发扇出（跨触发器）的架构，此架构可以减少扇出
- 它不依赖综合和/或布局布线优化（如触发器克隆）
- 它创建的结构有助于 PnR，在此结构中，不仅缩短了连线长度，也减少了高扇出

但减少扇出也有代价：

- 阻塞输出端口所需的 FIFO 缓冲器大小导致额外增加资源使用量。
- 这些阻塞输出端口上存在多路复用器延迟
- 由于提前确认正压触发器导致可能影响性能

 **重要提示！** 自由运行的流水线 (frp) 只能从 DATAFLOW 区域内部调用。frp 样式无法应用于在顺序区域或流水打拍区域内调用的循环。

流水线类型

下表汇总了该工具内可用的三种类型的流水线。该工具会自动选择正确的流水线样式，用于给定的流水打拍循环或函数。如果流水线搭配 `hls::tasks` 一起使用，那么会自动选中刷新流水线 (FLP) 样式以避免死锁。如果流水线控制需要高扇出并且满足其他自由运行要求，那么本工具会选择自由运行的流水线 (FRP) 样式以限制高扇出。最后，如果出现上述任一情况，则选中标准流水线 (STP) 样式。

表 4：流水线类型

名称	停滞的流水线	自由运行/可刷新的流水线	可刷新的流水线
用例	· 如果流水线控制上不存在因扇出过高而导致的时序问题，则可使用此项 · 如无需可刷新流水线（如不存在因停滞而导致的性能或死锁问题），则可使用此项	· 如果由于从流水线控制到寄存器使能的扇出，而需要更好的时序，则可使用此项 · 如需可刷新的流水线以换取更好的性能或避免死锁，则可使用此项 · 只能从数据流区域调用。	如需可刷新的流水线以换取更好的性能或避免死锁，则可使用此项。
编译指示/指令	<code>#pragma HLS pipeline style=stp</code>	<code>#pragma HLS pipeline style=frp</code>	<code>#pragma HLS pipeline style=flp</code>
全局设置	<code>syn.compile.pipeline_style=stp</code> （默认）	<code>syn.compile.pipeline_style=frp</code>	<code>syn.compile.pipeline_style=flp</code>
缺点	· 不可刷新，因此可能： ○ 导致数据流中出现更多死锁 ○ 如果缺少下一次迭代的输入，则会阻止交付计算所得的输出 · 由于流水线控制上扇出过高导致时序问题	· 由于输出上添加的 FIFO，资源增加量中等 · 需至少一个阻塞 I/O（串流或 ap_hs）。 · 并非所有流水打拍场景和 I/O 类型都受支持。	· 可采用较大的 II · 由于 II>1 时共享更少，资源使用率更大

表 4：流水线类型 (续)

名称	停滞的流水线	自由运行/可刷新的流水线	可刷新的流水线
优点	- 默认流水线。无使用约束。 - 通常总体资源使用率最低。	- 时序更好，因为 - 扇出更少 - 流水线控制逻辑更简单 - 可刷新	- 可刷新 - 避免死锁



重要提示！ 同时启用 `syn.compile.enable_auto_rewind` 配置命令时，刷新流水线 (flp) 与 PIPELINE 编译指示或指令中指定的 `rewind` 选项兼容。

管理流水线依赖关系

Vitis HLS 会构造对应于 C/C++ 语言源代码的硬件数据路径。如果没有流水线指令，将始终按顺序执行，并且工具不需要考虑任何依赖关系。但如果设计功能特性已流水打拍，那么该工具需要确保在 Vitis HLS 生成的硬件内遵循所有可能依赖关系。

“数据依赖关系”或“存储器依赖关系”的典型用例是在完成上一次读操作或写操作后再次发生读操作或写操作。

- 先写后读 (RAW) 操作也称为真性依赖关系，它表示指令（及其读取/使用的数据）从属于前一次操作的结果。

- l1: $t = a * b;$
- l2: $c = t + 1;$

语句 l2 中的读操作取决于语句 l1 中的 t 的写操作。如果对指令进行重新排序，它会使用 t 的前一个值。

- 先读后写 (WAR) 操作也称为反依赖关系，它表示当前一条指令完成数据读取后，下一条指令才能更新寄存器或存储器（通过写操作）。

- l1: $b = t + a;$
- l2: $t = 3;$

语句 l2 中的写操作无法在语句 l1 之前执行，否则 b 的结果无效。

- 先写后写 (WAW) 依赖关系表示必须按特定顺序写入寄存器或存储器，否则可能破坏其他指令。

- l1: $t = a * b;$
- l2: $c = t + 1;$
- l3: $t = 1;$

语句 l3 中的写操作必须晚于语句 l1 中的写操作。否则，语句 l2 结果将出错。

- 先读后读不含任何依赖关系，因为只要变量未声明为易变，即可随意对指令进行重新排序。如果变量声明为易变，则必须保留指令顺序不变。

例如，生成流水线时，工具需确保先前写操作不会修改后续阶段读取的寄存器或存储器位置。这属于真性依赖关系或先写后读 (RAW) 依赖关系。具体示例如下：

```
int top(int a, int b) {
    int t,c;
    I1: t = a * b;
    I2: c = t + 1;
    return c;
}
```

在语句 I1 完成前，无法对语句 I2 求值，因为与 t 变量之间存在依赖关系。在硬件中，如果乘法需耗时 3 个时钟周期，那么 I2 将发生等同于此时间量的延迟。如果对以上函数进行流水打拍，那么 Vitis HLS 会将其检测为真性依赖关系，并对操作进行相应调度。它使用数据转发优化来移除 RAW 依赖关系，因此函数可按 II=1 来运行。

当此示例应用于阵列而非变量时，就会出现存储器依赖关系。

```
int top(int a) {
    int r=1,rnext,m,i,out;
    static int mem[256];
    L1: for(i=0;i<=254;i++) {
    #pragma HLS PIPELINE II=1
    I1:     m = r * a; mem[i+1] = m;      // line 7
    I2:     rnext = mem[i]; r = rnext; // line 8
    }
    return r;
}
```

在以上示例中，L1 循环的调度导致出现调度警告消息：

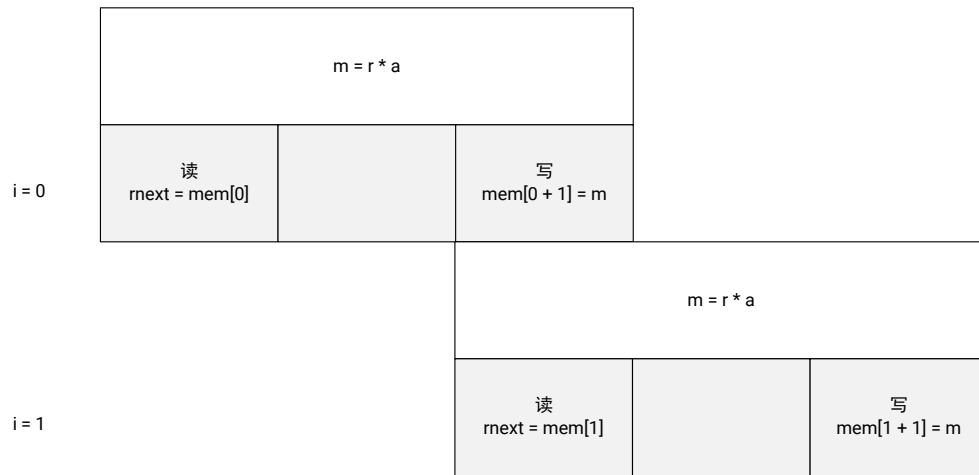
```
WARNING: [SCHED 204-68] Unable to enforce a carried dependency constraint
(II = 1, distance = 1) between 'store' operation (top.cpp:7) of variable
'm', top.cpp:7
on array 'mem' and 'load' operation ('rnext', top.cpp:8) on array 'mem'.
INFO: [SCHED 204-61] Pipelining result: Target II: 1, Final II: 2, Depth: 3.
```

只要写入的索引不同于读取的索引，那么循环的同一次迭代内就不会发生任何问题。而 2 条指令可同时并发执行。但请观测多次迭代中的读写操作：

```
// Iteration for i=0
I1:     m = r * a; mem[1] = m;      // line 7
I2:     rnext = mem[0]; r = rnext; // line 8
// Iteration for i=1
I1:     m = r * a; mem[2] = m;      // line 7
I2:     rnext = mem[1]; r = rnext; // line 8
// Iteration for i=2
I1:     m = r * a; mem[3] = m;      // line 7
I2:     rnext = mem[2]; r = rnext; // line 8
```

当考量 2 次连续迭代时，来自 I1 语句的乘法结果 m（时延 = 2）将写入某一位置，而循环的下一次迭代的 I2 语句将把位于该位置的结果读取到 rnext 中。在此情况下，存在 RAW 依赖关系，因为上一次计算的写操作完成后，下一次循环迭代才能开始读取 mem[i]。

图 18：依赖关系示例



X24685-030622

如果增大时钟频率，那么乘法器将需要更多流水线阶段，从而导致时延增加。这也会迫使 II 增大。

请注意以下代码，其中操作发生交换，导致功能发生更改：

```
int top(int a) {
    int r,m,i;
    static int mem[256];
L1: for(i=0;i<=254;i++) {
#pragma HLS PIPELINE II=1
I1:   r = mem[i];           // line 7
I2:   m = r * a , mem[i+1]=m; // line 8
    }
    return r;
}
```

调度警告为：

```
INFO: [SCHED 204-61] Pipelining loop 'L1'.
WARNING: [SCHED 204-68] Unable to enforce a carried dependency constraint
(II = 1,
distance = 1)
between 'store' operation (top.cpp:8) of variable 'm', top.cpp:8 on array
'mem'
and 'load' operation ('r', top.cpp:7) on array 'mem'.
WARNING: [SCHED 204-68] Unable to enforce a carried dependency constraint
(II = 2,
distance = 1)
between 'store' operation (top.cpp:8) of variable 'm', top.cpp:8 on array
'mem'
and 'load' operation ('r', top.cpp:7) on array 'mem'.
WARNING: [SCHED 204-68] Unable to enforce a carried dependency constraint
(II = 3,
distance = 1)
between 'store' operation (top.cpp:8) of variable 'm', top.cpp:8 on array
'mem'
and 'load' operation ('r', top.cpp:7) on array 'mem'.
INFO: [SCHED 204-61] Pipelining result: Target II: 1, Final II: 4, Depth: 4.
```


请观测多次迭代中的连续读写操作：

```
Iteration with i=0
I1:      r = mem[0];           // line 7
I2:      m = r * a , mem[1]=m; // line 8
Iteration with i=1
I1:      r = mem[1];           // line 7
I2:      m = r * a , mem[2]=m; // line 8
Iteration with i=2
I1:      r = mem[2];           // line 7
I2:      m = r * a , mem[3]=m; // line 8
```

所需 II 延长，因为存在如下 RAW 依赖关系，从 `mem[i]` 读取 `r`、执行乘法并写入 `mem[i+1]`。

移除假性依赖关系以改善循环流水节拍

假性依赖关系即编译器过于保守时出现的依赖关系。这些依赖关系在真实代码中并不存在，但无法由编译器来判定。这些依赖关系可能阻碍循环流水节拍。

假性依赖关系如下示例所示。在此示例中，针对相同循环迭代内的 2 个不同地址执行读写访问。这 2 个地址均依赖于输入数据，可指向 `hist` 阵列中的任一元素。有鉴于此，Vitis HLS 假定这 2 个地址可访问同一个位置。因此，它安排按交替周期对阵列执行读写操作，导致循环 II 为 2。但代码显示 `hist[old]` 和 `hist[val]` 永远无法访问相同地址，因为这两者包含在 `if(old == val)` 条件的 `else` 分支中。

```
void histogram(int in[INPUT SIZE], int hist[VALUE SIZE]) {
    int acc = 0;
    int i, val;
    int old = in[0];
    for(i = 0; i < INPUT SIZE; i++)
    {
        #pragma HLS PIPELINE II=1
        val = in[i];
        if(old == val)
        {
            acc = acc + 1;
        }
        else
        {
            hist[old] = acc;
            acc = hist[val] + 1;
        }

        old = val;
    }

    hist[old] = acc;
}
```

为克服这一缺陷，您可使用 `DEPENDENCE` 指令为 Vitis HLS 提供有关这些依赖关系的附加信息。

```
void histogram(int in[INPUT SIZE], int hist[VALUE SIZE]) {
    int acc = 0;
    int i, val;
    int old = in[0];
    #pragma HLS DEPENDENCE variable=hist type=intra direction=RAW
    dependent=false
    for(i = 0; i < INPUT SIZE; i++)
    {
        #pragma HLS PIPELINE II=1
        val = in[i];
        if(old == val)
```

```

{
    acc = acc + 1;
}
else
{
    hist[old] = acc;
    acc = hist[val] + 1;
}

old = val;
}

hist[old] = acc;

```



重要提示！ 如果实际上依赖关系并非假性 (FALSE) 关系，那么指定假性依赖关系可能导致硬件错误。指定依赖关系前，请确认它是否正确 (TRUE 或 FALSE)。

指定依赖关系时，有 2 种主要类型：

- Inter（循环间）：指定相同循环的不同迭代之间存在不同的依赖关系。如指定为 FALSE，则当循环已流水打拍、已展开或已部分展开时，允许 Vitis HLS 并发执行运算，指定为 TRUE 时则阻止此类并发运算。
- Intra（循环内）：指定循环的相同迭代内的依赖关系，例如，在相同迭代开始和结束时访问的阵列。当 intra 依赖关系指定为 FALSE 时，Vitis HLS 可在循环内自由移动运算、提升运算移动性，从而可能改善性能或面积。当此依赖关系指定为 TRUE 时，必须按指定顺序执行运算。

标量依赖关系

部分标量依赖关系较难以解析，且通常需要更改源代码。标量数据依赖关系如下所示：

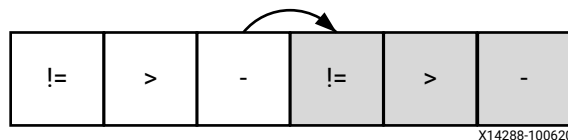
```

while (a != b) {
    if (a > b) a -= b;
    else b -= a;
}

```

此循环的当前迭代完成 a 和 b 的更新值计算后才能启动下一次迭代，如下图所示。

图 19：标量依赖关系



如果必须得到上一次循环迭代结果后才能开始当前迭代，则无法进行循环流水打拍。如果 Vitis HLS 无法以指定的启动时间间隔进行流水打拍，那么它会增加启动时间间隔。如果完全无法流水打拍，则它会停止流水打拍并继续输出非流水打拍设计。

展开循环

循环的迭代执行次数由循环归纳变量来指定。迭代次数也可能受到循环主体内逻辑的影响（例如，`break` 条件或对循环出口变量进行修改）。您可展开循环，在 RTL 设计中创建循环主体的多个副本，从而允许并行执行部分或全部循环迭代。您可使用 `UNROLL` 编译指示展开循环以便增加数据访问和吞吐量。

默认情况下，HLS 循环保持处于未展开状态。这意味着循环的每次迭代都使用相同硬件。展开循环意味着，循环的每次迭代都有其自己的硬件用于执行循环函数。这意味着，展开的循环的性能显著高于未展开的循环。但提升性能的代价是增加面积和资源使用率。

请参阅来自 GitHub 的以下示例：

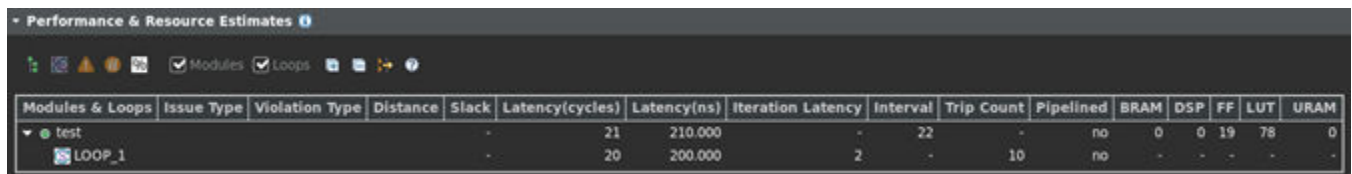
```
#include "test.h"

dout_t test(din_t A[N]) {
    dout_t out_accum=0;
    dsel_t x;

    LOOP_1:for (x=0; x<N; x++) {
        out_accum += A[x];
    }
    return out_accum;
}
```

下图中的“Synthesis Summary”（综合汇总）报告显示，实现以顺序方式执行，且无优化。这可通过观察 `LOOP_1` 的循环次数来确认，报告称迭代次数为 10，“Latency”（时延）为 200。时延是循环可接受新的输入值之前的时间。

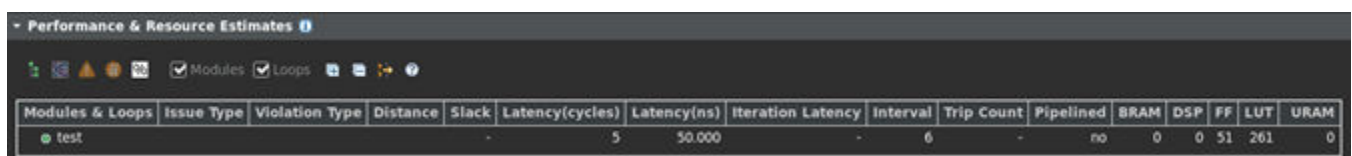
图 20：性能和资源估算



Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
test					21	210.000		22	-	no	0	0	19	78	0
LOOP_1					20	200.000		2	-	no	-	-	-	-	-

为了获得最优吞吐量，时延需尽可能短。为了提升性能，假定循环边界为静态，可使用 `UNROLL` 编译指示来完全展开循环，以创建循环主体的并行实现。在完全展开 `LOOP_1` 后，可见时延显著缩短 (50 ns)，如下图所示。展开循环意味着一种权衡：可以获得更高性能，但代价是使用额外资源（如下所示，FF 和 LUT 数量会增加）。完全展开循环会导致循环本身消失，并被循环主体的并行实现所替代，后者会占用额外资源，如下所示。

图 21：性能和资源估算



Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
test					5	50.000		6	-	no	0	0	51	261	0

在某些情况下，由于资源增加以及平台可用资源所限，会导致无法完全展开循环。在此情况下，首选解决方案是部分展开循环，这样通常可以提升部分性能，同时无需耗用过多资源。定义一个展开因子以便编译指示或指令将循环部分展开。对于此处受约束的情况，将同一循环按因子 2 展开（即复制循环主体，将循环次数减半为 5）是一种可以接受的解决方案，如下所示。

图 22：性能和资源估算

Performance & Resource Estimates

</

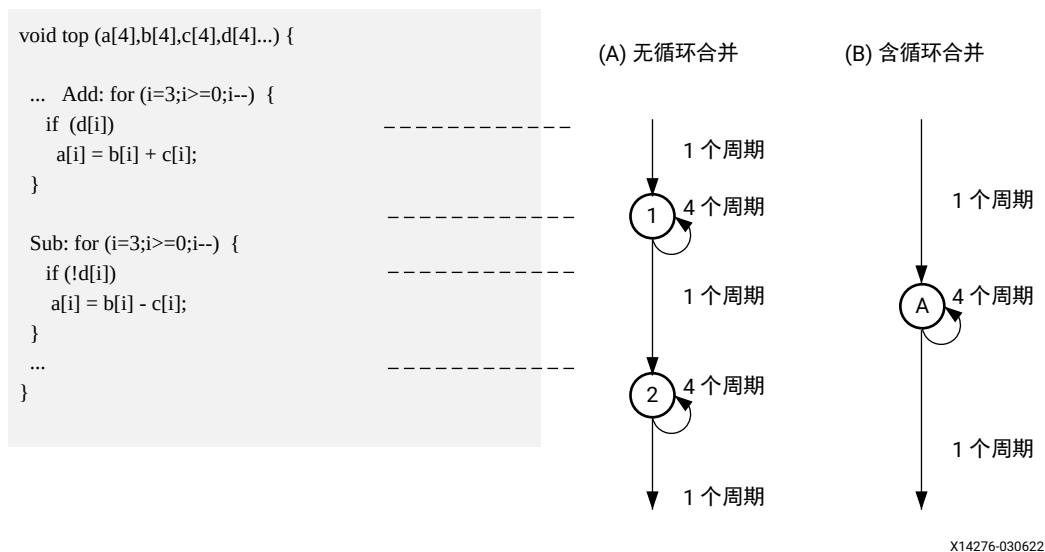
此外，部分展开循环时，HLS 工具会在循环中实现出口检查，以防循环次数无法被展开因子整除。如果循环次数能被展开因子整除，则跳过出口检查。

合并循环

所有收起的循环都在设计 FSM 中指明并创建至少一种状态。当存在多个顺序循环时，它可能会创建其他不必要的时钟周期，并阻止进一步的优化。

下图显示了一个简单的示例，其中看似直观的编码样式对 RTL 设计的性能产生了负面影响。

图 23：循环指令



在这个简单的示例中，很明显，ADD 循环中的 else 分支也可以解决该问题，但是在更复杂的示例中，它可能不那么明显，更直观的编码样式可能具有更大的优势。

LOOP_MERGE 优化指令用于自动合并循环。循环合并优化指令将尝试合并其布局作用域内的所有循环。在上面的示例中，合并循环将创建类似于上图 (B) 所示的控制结构，完成它只需要 6 个时钟周期。

合并循环允许将循环内的逻辑组合在一起进行优化。循环合并变换存在一些限制，并非总能成功。但仍可以同构重构代码来手动合并循环。在上面的示例中，使用双端口块 RAM 可以并行执行加减运算。

处理嵌套循环

仅限不含任何循环（可能在展开部分内层循环后）的循环才能进行流水打拍。因此，包含流水打拍的循环的循环属于数据流循环或顺序循环。对于顺序循环，其迭代会依次执行，而且每次迭代，就会完全执行一次内层循环。在 RTL 实现中，从外层循环移动到内层循环需要一个时钟周期，从内层循环移回外层循环需要一个时钟周期，并且中间要加上完成内层循环的所有迭代的完整时延。处理嵌套循环时为了提升性能（降低时延），创建完美嵌套的循环并将其转换为可流水打拍的单一循环就会变得至关重要。

```
Perfect_nested_loop_1: for (int i = 0; i < N; ++i) {
    Perfect_nested_loop_2: for (int j = 0; j < M; ++j) {
        ...
    }
}

Imperfect_nested_loop_0: for (int i = 0; i < N; ++i) {
    Imperfect_nested_loop_2: for (int j = 0; j < M; ++j) {
        ...
    }
    Imperfect_nested_loop_3: for (int j = 0; j < M; ++j) {
        ...
    }
}
```

完美循环嵌套是其中每个外层循环仅包含一个子循环的嵌套循环，其中不含任何其他内容。所谓近乎完美的循环嵌套，是指嵌套循环中每个外层循环还可包含基本指令，这类循环可以通过将这些指令推入最内层循环来自动变换为完美的循环嵌套。

LOOP_FLATTEN 编译指示或指令用于允许将完美和近乎完美的嵌套循环加以平铺（存在额外的平铺能力要求，这些要求与其循环次数有关，请参阅 loop_flatten 编译指示或指令的描述），这样就无需通过重新编码来达成最优硬件性能，并且还可减少执行循环中的运算所需的周期数。将 LOOP_FLATTEN 优化应用于一组嵌套循环时，应将其应用于包含循环体的最内层循环。

对嵌套循环进行流水打拍时，通常可通过对最内层循环进行流水打拍来实现面积与性能之间的最优平衡。这样同时也可达成最短的运行时间。

处理变量循环边界

当循环包含变量边界时，将禁止执行 Vitis HLS 可应用的某些优化操作。GitHub 上的 [variable_bound_loops](#) 提供了如下所示代码示例，其中，循环边界由顶层输入驱动变量 `width` 来判定。在此情况下，循环被视为包含变量边界，因为 Vitis HLS 无从知晓循环将何时完成。

```
#include "ap_int.h"
#define N 32

typedef ap_int<8> din_t;
typedef ap_int<13> dout_t;
typedef ap_uint<5> dsel_t;

dout_t code028(din_t A[N], dsel_t width) {

    dout_t out_accum=0;
    dsel_t x;

    LOOP_X:for (x=0;x<width; x++) {
        out_accum += A[x];
    }

    return out_accum;
}
```

尝试对上述示例中的设计进行优化时，会报告由变量循环边界造成的问题。变量循环边界的第一个问题是阻止 Vitis HLS 判定循环时延。Vitis HLS 可判定完成一次循环迭代所需的时延，但由于它无法静态判定精确的变量宽度值 `width`，因此无从知晓执行的迭代次数，因而无法报告循环时延（即完全执行循环的所有迭代的周期数）。

存在变量循环边界时，Vitis HLS 会将时延报告为问号 (?) 而不是使用精确值。以下显示了完成前述示例的综合后的结果：

```
+ Summary of overall latency (clock cycles):
* Best-case latency:      ?
* Worst-case latency:    ?
+ Summary of loop latency (clock cycles):
+ LOOP_X:
* Trip count: ?
* Latency:    ?
```

克服此问题的方法是使用 `LOOP_TRIPCOUNT` 编译指示或指令来为循环指定最小和/或最大迭代计数。循环次数 (`tripcount`) 表示循环的迭代次数。如果对第 1 个示例中的 `LOOP_X` 应用 `tripcount` 最大值 32，那么报告将更新为显示如下内容：

```
+ Summary of overall latency (clock cycles):
* Best-case latency:      2
* Worst-case latency:    34
+ Summary of loop latency (clock cycles):
+ LOOP_X:
* Trip count: 0 ~ 32
* Latency:    0 ~ 32
```

用户为 `LOOP_TRIPCOUNT` 指令提供的值仅用于报告，或者用于支持 `PERFORMANCE` 编译指示或指令。指定的循环次数值使 Vitis HLS 能够在报告中判定时延值，以便对来自不同解决方案的值进行比较。要将此循环边界信息用于综合，必须使用断言更新 C/C++ 语言代码，这样会影响综合，由于断言条件假定为 `true`，因此必须谨慎使用。

要对第 1 个示例进行优化以降低启动时间间隔，请执行下列后续步骤：

- 展开循环，允许并行执行累加。
- 通过单一存储器端口对阵列输入进行分区或者限制并行累加。

应用这些代码变换操作后，Vitis HLS 的输出会高亮显示变量边界循环的最显著的问题：

```
WARNING: [HLS 200-936] Cannot unroll loop 'LOOP_X' (loop_var.cpp:22) in
function 'loop_var': cannot completely unroll a loop with a variable trip
count.
```

由于变量边界循环无法完全展开，因此不仅阻止应用展开 (unroll) 指令，而且还会阻止循环上层的层级的流水打拍操作。



重要提示！ 当循环或函数实现流水打拍后，Vitis HLS 即可将函数或循环下层层级中的所有循环展开。如果在此层级中存在具有变量边界的循环，它就会阻止流水打拍。

对于含变量边界的循环，解决方案是通过循环内部的条件执行来将循环迭代次数设置为固定值。可重写变量循环边界示例中的代码，如以下代码示例所示。此处循环边界显式设置为变量宽度值 `width` 的最大值，循环主体则以有条件方式来执行：

```
#include "ap_int.h"
#define N 32

typedef ap_int<8> din_t;
typedef ap_int<13> dout_t;
typedef ap_uint<5> dsel_t;

dout_t loop_max_bounds(din_t A[N], dsel_t width) {

    dout_t out_accum=0;
    dsel_t x;

    LOOP_X:for (x=0; x<N; x++) {
        if (x<width) {
            out_accum += A[x];
        }
    }

    return out_accum;
}
```

以上示例中的 `for` 循环 (LOOP_X) 可完全展开。由于此循环的上限固定，因此 Vitis HLS 知晓需创建的硬件数量。在 RTL 设计中包含 `N(32)` 份循环主体副本。每份循环主体副本都包含与之关联的条件逻辑，并根据变量 `width` 值来执行。请参阅 [Vitis-HLS-Introductory-Examples/Modeling/variable_bound_loops](#) 以获取示例。

阵列入门

将软件阵列映射到硬件存储器

阵列是任意 C++ 软件程序中的基础数据结构。软件程序员将阵列视作为简单的容器，并对阵列进行按需（通常动态）分配/解除分配。如需综合程序以供硬件使用，则不支持对阵列进行此类型的动态存储器分配。要将阵列综合为硬件，需准确知晓存储器量（静态），算法才可用。此外，FPGA 上的存储器架构（也称为“本地存储器”）相比于全局存储器（通常为 DDR 存储器或 HBM 存储体）有着截然不同的侧重点。访问全局存储器会产生很高的时延成本，可耗时大量周期，而访问本地存储器通常十分快速，只需一个或多个周期即可。

当 HLS 设计已适当完成流水打拍和/或展开后，即可确立存储器访问模式。HLS 编译器允许用户将阵列映射到各种类型的资源，其中无论有无握手信号，阵列元素均可并行使用。内部阵列和顶层函数接口内的阵列均可映射到寄存器或存储器。如果阵列位于顶层接口内，那么该工具会自动创建与外部存储器对接所需的地址信号、数据信号和控制信号。如果阵列位于设计内部，那么该工具不仅会创建必要的地址信号、数据信号和控制信号以访问存储器，还会例化存储器模型（随后下游 RTL 综合工具会将该模型推断为存储器）。

综合后，阵列通常作为存储器（RAM、ROM 或移位寄存器）来实现。阵列还可按各寄存器进行完全分区，以创建完全并行的实现，前提是平台具有足够寄存器来支持该步骤。GitHub 上提供的 [initialization_and_reset](#) 示例演示了存储器的不同实现。

顶层函数接口上的阵列作为访问外部存储器的 RTL 端口来进行综合。而在设计内部，较小的阵列将作为移位寄存器来执行综合。根据优化设置，较大的阵列将综合到块 RAM (BRAM)、LUTRAM 或 UltraRAM (URAM) 中（请参阅 `BIND_STORAGE` directive/pragma）。

请参阅以下示例，其中，HLS 编译器在遇到以下代码时推断移位寄存器：

```
int A[N]; // This will be replaced by a shift register

for(...) {
    // The loop below is the shift operation
    for (int i = 0; i < N-1; ++i)
        A[i] = A[i+1];
    A[N-1] = ...;

    // This is an access to the shift register
    ... A[x] ...
}
```

移位寄存器每个周期都能执行一次移位操作，还支持在移位寄存器内任意位置每个周期执行一次随机读取访问，因此，比 FIFO 更灵活。

在下列情况下，阵列在 RTL 中可能引发问题：

- 作为存储器 (BRAM/LUTRAM/URAM) 来实现时，存储器端口的数量可能限制对数据的访问，从而导致流水打拍循环内出现 II 违例
- HLS 编译器可能无法正确推断互斥访问

- 请确保在 RTL 中将只需读访问权的阵列作为 ROM 来实现

HLS 编译器支持指针阵列。每个指针都只能指向 1 个标量或 1 个标量阵列。



提示：阵列大小必须固定。对于如下函数实参，这是必需的：C++ 编译器忽略实参大小，但该工具会使用此大小，例如：Array[10];。但不支持非固定大小的阵列，如：Array[];。

阵列访问和性能

在上一章节中介绍了多个优化概念，例如循环展开和流水打拍，这些均为探索并行化的方法。但这其中均未考量当阵列映射到存储器而不是寄存器时，阵列访问模式可能对此类优化产生的阻碍。映射到存储器的阵列可能会成为设计性能中的瓶颈。Vitis HLS 提供了多种优化措施来消除这些存储器瓶颈，例如，阵列重塑和阵列分区。应尽可能利用这些自动存储器优化措施来最大程度减少代码修改量。但在某些情况下，可能需要对存储器架构进行显式编码来满足性能需求或者允许设计人员实现更好的结果质量。在此类情况下，必须按不会限制性能的方式来对阵列访问进行编码。这意味着在设计中通过分析阵列访问模式并组织存储器来实现所期望的吞吐量和面积。以下代码示例显示了访问阵列导致最终 RTL 设计性能受限的案例。在此示例中，对 mem[N] 阵列执行了 3 次访问以创建求和结果。请参阅 [Vitis-HLS-Introductory-Examples/Interface/Memory/memory_bottleneck](#) 以获取此示例的完整版本。

```
#include "array_mem_bottleneck.h"

dout_t array_mem_bottleneck(din_t mem[N]) {

    dout_t sum=0;
    int i;

    SUM_LOOP:for(i=2;i<N;++i)
        sum += mem[i] + mem[i-1] + mem[i-2];

    return sum;
}
```

在综合期间，该阵列作为 RAM 来实现。如果将此 RAM 指定为单端口 RAM，那么将无法通过对 SUM_LOOP 循环进行流水打拍来实现在每个时钟周期内处理新的循环迭代的目标。

尝试以值为 1 的启动时间间隔对 SUM_LOOP 进行流水打拍将生成如下消息（无法达成吞吐量 1 后，Vitis HLS 会放宽约束）：

```
INFO: [SCHED 61] Pipelining loop 'SUM_LOOP'.
WARNING: [SCHED 69] Unable to schedule 'load' operation ('mem_load_2',
bottleneck.c:62) on array 'mem' due to limited memory ports.
INFO: [SCHED 61] Pipelining result: Target II: 1, Final II: 2, Depth: 3.
```

此处的问题是单端口 RAM 只有 1 个数据端口：每个时钟周期内只能执行 1 次读取（或 1 次写入）。

- SUM_LOOP Cycle1: 读取 mem[i];
- SUM_LOOP Cycle2: 读取 mem[i-1]，对值求和；
- SUM_LOOP Cycle3: 读取 mem[i-2]，对值求和；

可使用双端口 RAM，但这样仅支持每个时钟周期 2 次访问。要计算总和值，需读取 3 次，因此要将循环流水打拍，每个时钟周期需访问 3 次，即每个时钟周期内新增 1 次迭代。

以上示例中的代码可按以下代码示例中所示方式重写，以便按吞吐量为 1 来对代码进行流水打拍。在以下代码示例中，通过执行预读取并手动对数据访问进行流水打拍，即可在循环每次迭代中仅指定 1 次阵列读取。这样可确保只需 1 个单端口 RAM 即可实现性能目标。

```
#include "array_mem_perform.h"

dout_t array_mem_perform(din_t mem[N]) {

    din_t tmp0, tmp1, tmp2;
    dout_t sum=0;
    int i;

    tmp0 = mem[0];
    tmp1 = mem[1];
    SUM_LOOP:for (i = 2; i < N; i++) {
        tmp2 = mem[i];
        sum += tmp2 + tmp1 + tmp0;
        tmp0 = tmp1;
        tmp1 = tmp2;
    }

    return sum;
}
```

需要按如上所述方式更改源代码的场景并不多见。更为常见的场景是使用优化指令/编译指示来实现相同的结果。Vitis HLS 包含用于更改阵列实现和访问方式的优化指令。优化主要分两类：

- “Array Partition”（阵列分区）用于将原始阵列拆分为较小的阵列或者独立的寄存器。
- “Array Reshape”（阵列重塑）用于将阵列重新组织为不同的存储器排列方式，以增加并行度而不拆分原始阵列。

阵列分区

阵列可分区到块中或者分区到其各元素中。在某些情况下，Vitis HLS 会将阵列分区到各独立元素中。这可使用自动分区配置设置来加以控制。将阵列分区到多个块中时，单一阵列将作为多个 RTL RAM 块来实现。分区到元素中时，每个元素都作为 RTL 中的 1 个寄存器来实现。在这 2 种情况下，分区允许并行访问更多元素，并且有助于提升性能；设计将在性能与实现性能所需的 RAM 或寄存器数量之间加以权衡取舍。

以下消息显示了函数流水打拍时常见的问题：

```
INFO: [SCHED 204-61] Pipelining loop 'SUM_LOOP'.
WARNING: [SCHED 204-69] Unable to schedule 'load' operation ('mem_load_2',
bottleneck.c:62) on array 'mem' due to limited memory ports.
WARNING: [SCHED 204-69] The resource limit of core:RAM:mem:p0 is 1, current
assignments:
WARNING: [SCHED 204-69] 'load' operation ('mem_load', bottleneck.c:62)
on array
'mem',
WARNING: [SCHED 204-69] The resource limit of core:RAM:mem:p1 is 1, current
assignments:
WARNING: [SCHED 204-69] 'load' operation ('mem_load_1',
bottleneck.c:62) on array
'mem',
INFO: [SCHED 204-61] Pipelining result: Target II: 1, Final II: 2, Depth: 3.
```

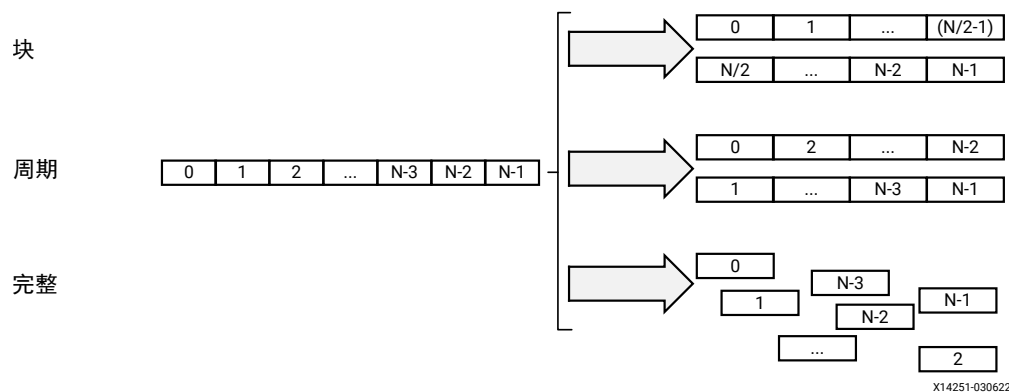
在此示例中，Vitis HLS 声明它无法达成指定的启动时间间隔 (II) 值 1，因为它受存储器端口所限，无法在存储器上调度 load（读取）操作 (mem_load_2)。以上消息指出了第 62 行上的 mem_load 操作所使用的 “core:RAM:mem:p0 is 1” 的资源限制。块 RAM 的第 2 个端口同样仅含 1 项资源，该资源同样供 mem_load_1 操作使用。由于存在此存储器端口争用，Vitis HLS 报告的最终 II 为 2，而不是所期望的值 1。

此问题通常是由阵列所导致的。如果阵列不属于顶层函数的接口，则会作为包含最多 2 个数据端口块 RAM 来实现。这可能限制读写（或加载/存储）密集型算法的吞吐量。通过将该阵列（单一块 RAM 资源）拆分为多个更小的阵列（多个块 RAM）从而有效增加端口数量，即可改善带宽。

阵列可使用 ARRAY_PARTITION 指令来进行分区。Vitis HLS 可提供 3 种类型的阵列分区，如下图所示。这 3 种分区样式分别是：

- **block**：原始阵列分割为原始阵列的连续元素块（大小相同）。
- **cyclic**：原始阵列分割多个大小相同的块，这些块交织成原始阵列的元素。
- **complete**：默认操作是将阵列按其独立元素进行拆分。这对应于将存储器解析为寄存器。

图 24：阵列分区



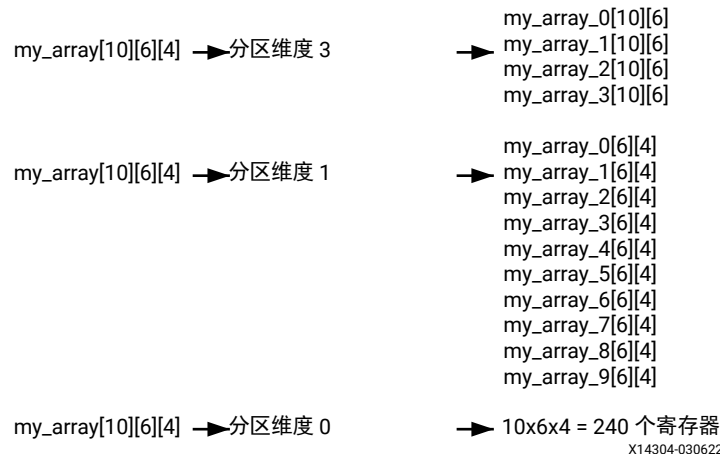
对于 **block** 和 **cyclic** 分区，**factor** 选项可指定要创建的阵列数量。在前图中，使用因子 2，即将阵列分割为 2 个更小的阵列。如果阵列的元素数量并非该因子的整数倍，那么最后一个阵列所含元素数量较少。

对多维阵列进行分区时，**dimension** 选项可用于指定对哪个维度进行分区。下图显示了使用 **dimension** 选项对以下代码示例进行分区的方式：

```
void foo (...) {
    int my_array[10][6][4];
    ...
}
```

此图中的示例演示了如何通过对 **dimension 3** 进行分区来生成 4 个独立阵列，以及如何对 **dimension 1** 进行分区以生成 10 个独立分区。如果针对 **dimension** 指定 0，则将对所有维度进行分区。

图 25：对阵列维度进行分区



自动阵列分区

`syn.array_partition.complete_threshold` 和 `syn.array_partition.throughput_driven` 配置命令可根据元素数量或吞吐量需求来定义阵列的自动分区方式，如 [阵列分区配置](#) 中所述。

阵列重塑

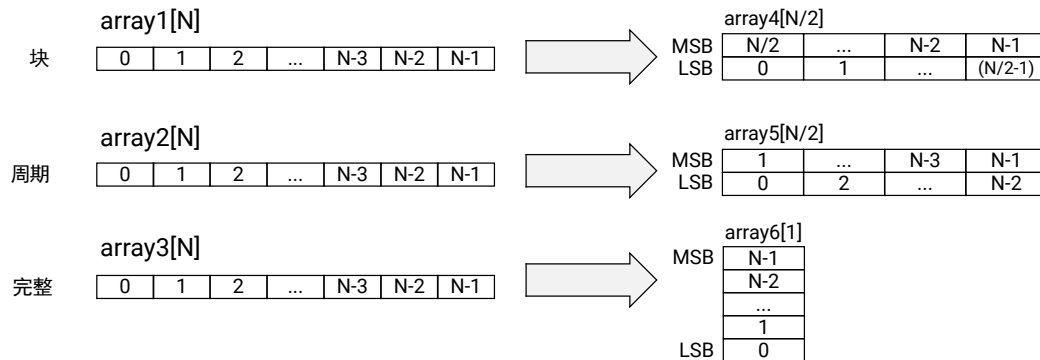
`ARRAY_RESHAPE` 指令通过垂直重映射模式来重构阵列，用于减少所耗用的块 RAM 的数量，同时提供对数据的并行访问。

给定以下示例代码：

```
void foo (...) {
    int array1[N];
    int array2[N];
    int array3[N];
    #pragma HLS ARRAY_RESHAPE variable=array1 type=block factor=2 dim=1
    #pragma HLS ARRAY_RESHAPE variable=array2 type=cyclic factor=2 dim=1
    #pragma HLS ARRAY_RESHAPE variable=array3 type=complete dim=1
    ...
}
```

`ARRAY_RESHAPE` 指令可将阵列转换为下图所示形式。

图 26：阵列重塑



X14307-030622

ARRAY_RESHAPE 指令支持在单一时钟周期内访问更多数据。只要能在单一时钟周期内访问更多数据，Vitis HLS 即可自动展开使用此数据的所有循环，前提是这有助于提升吞吐量。循环可全部或部分展开以创建足够的硬件以便在单一时钟周期内使用更多数据。此功能特性是使用 `syn.unroll.tripcount_threshold` 配置命令来控制的，如 [展开设置](#) 中所述。

在以下示例中，循环次数小于 16 的任何循环都将自动展开（前提是可提高吞吐量）。

```
syn.unroll.tripcount_threshold=16
```

接口上的阵列

使用阵列作为顶层函数的实参时，Vitis HLS 采用下列任一假定：

- 存储器位于偏外，并且 Vitis HLS 会在接口上综合 M_AXI 端口，以便访问该存储器，这是 Vitis 内核流程的默认行为。
- 存储器为标准块 RAM，时延为 1。这是 Vivado IP 流程的默认行为。提供地址后经过 1 个时钟周期后，数据即处于就绪状态。

要配置 Vitis HLS 创建这些端口的方式，请执行以下操作：

- 使用 INTERFACE 编译指示或指令将接口指定为 M_AXI、BRAM 或 FIFO 接口。
- 使用 INTERFACE 编译指示或指令的 `storage_type` 选项，将 RAM 指定为单端口或双端口 RAM。
- 使用 INTERFACE 编译指示或指令的 `latency` 选项，指定 RAM 时延。
- 使用阵列优化指令 ARRAY_PARTITION 或 ARRAY_RESHAPE 重新配置阵列结构，从而重新配置 I/O 端口的数量。



提示：由于通过存储器（RAM 或 FIFO）端口限制对数据的访问，接口上的阵列可能造成性能瓶颈。通常，您可使用优化指令来克服这些瓶颈。

在可综合的代码中使用阵列时，必须调整阵列大小。例如，如果 [阵列接口](#) 中的声明 `d_i[4]` 更改为 `d_i[]`，那么 Vitis HLS 会发出消息称无法综合此设计：

```
@E [SYNCHK-61] array_RAM.c:52: unsupported memory access on variable 'd_i' which is (or contains) an array with unknown size at compile time.
```

阵列接口

INTERFACE 编译指示或指令允许您显式定义要搭配 `storage_type=<value>` 选项使用的 RAM 或 ROM 的类型。这样即可定义创建哪些端口（单端口或双端口）。如不指定 `storage_type`，那么本工具会使用：

- 单端口 RAM（默认情况下）。
- 双端口 RAM，前提是这可以缩短启动时间间隔或减少时延。

ARRAY_PARTITION 和 ARRAY_RESHAPE 编译指示可重配置接口上的阵列。阵列可分区为多个较小的阵列，并在其各自接口上单独实现。其中包含将阵列彻底分区为一组标量的功能。在函数接口上，这导致阵列中每个元素都有专用的端口。由此可提供最大限度并行访问，但会造成端口数量剧增，可能在硬件实现期间导致布线问题。

默认情况下，以下代码示例中所示的函数中的阵列实参将综合到单端口 RAM 接口中。

```
#include "array_RAM.h"

void array_RAM (dout_t d_o[4], din_t d_i[4], didx_t idx[4]) {
    int i;

    For_Loop: for (i=0;i<4;i++) {
        d_o[i] = d_i[idx[i]];
    }
}
```

使用单端口 RAM 接口的原因是因为 `for-loop` 可确保在每个时钟周期内仅读写 1 个元素。使用双端口 RAM 接口并无任何优势。如果展开 `for` 循环，那么 Vitis HLS 会使用双端口 RAM。这样即可支持同时读取多个元素，从而缩短启动时间间隔。通过应用 INTERFACE 编译指示或指令并设置 `storage_type` 即可显式设置 RAM 接口的类型。

接口上阵列相关问题通常与吞吐量有关。例如，如果以上示例中的阵列分区到各独立元素中，并且展开 `for-loop`，那么即可同时访问每个阵列中的全部 4 个元素。

您还可以使用 INTERFACE 编译指示或指令和 `latency=<value>` 选项来指定 RAM 时延。这样本工具即可对接口处时延大于 1 的外部 SRAM 进行建模。

FIFO 接口

Vitis HLS 允许在 RTL 中将阵列实参作为 FIFO 端口来实现。如果要使用 FIFO 端口，请务必确保往来该阵列的访问均为顺序访问。

注释：如果在接口处访问并非顺序访问，则存在 RTL 仿真不匹配。

以下代码示例中显示了工具无法判定访问是否为顺序访问时的情况。在此示例中，`d_i` 和 `d_o` 均已指定为在综合期间作为 FIFO 接口来实现。在此情况下，您必须确保访问为顺序访问，否则将导致系统中出现错误。

```
#include "array_FIFO.h"

void array_FIFO (dout_t d_o[4], din_t d_i[4], didx_t idx[4]) {
    int i;
    #pragma HLS INTERFACE mode=ap_fifo port=d_i
    #pragma HLS INTERFACE mode=ap_fifo port=d_o
    For_Loop: for (i=0;i<4;i++) {
        d_o[i] = d_i[idx[i]];
    }
}
```


在此情况下，`idx` 变量的值将判定是否能作为 `d_i[]` 实参成功创建 FIFO 接口。

- 如果 `idx` 的元素值为顺序值，则可创建 FIFO 接口
- 如果 `idx` 使用随机值，那么在 RTL 中实现时，协同仿真中的 FIFO 接口将失败，运行期间同样可能失败

但由于在编译时无法确认这些条件，Vitis HLS 会在综合期间发出一条消息并创建 FIFO 接口：

```
@W [XFORM-124] Array 'd_i': may have improper streaming access(es).
```

此外，永不读取 `idx` 阵列，原因是假定由于存在 `ap_fifo` INTERFACE 编译指示，阵列元素包含从 0 开始的顺序值。

注释：针对要在同一循环或函数内执行读取和写入的阵列，FIFO 端口无法进行综合。必须创建独立的输入阵列和输出阵列（如上示例所示）。

以下通用规则适用于使用 FIFO 接口实现的阵列：

- 在循环或函数中，阵列必须仅限于读取或者仅限于写入。这可变换为匹配 FIFO 链接特征的点对点连接。
- 阵列读取与阵列写入的顺序必须相同。由于针对 FIFO 通道不支持随机访问，因此必须在遵循先入先出语义的程序内使用该阵列。

如果阵列的数据类型为结构体并按顺序访问阵列（即，指定的阵列包含 `axis` 或 `ap_fifo` 接口，或者以 STREAM 编译指示或指令来标记），则适用以下条件：

- 您无法从使用阵列到串流接口或者用作为串流接口的 I/O 实参直接访问结构体成员。您可生成结构体的本地副本以读/写成员元素。
- 您必须确保顺序访问，如下所示

```
struct A {
    short foo;
    int bar;
};

void dut(A in[N], A out[out], bool flag) {
    #pragma HLS interface ap_fifo port=in,out
    for (unsigned i=0; i<N; i++) {
        A tmp = in[i];
        if (flag)
            tmp.bar += 5;
        out[i] = tmp;
    }
}

Bad example 1:
void dut(A in[N], A out[out], bool flag) {
    #pragma HLS interface ap_fifo port=in,out
    for (unsigned i=0; i<N; i++) {
        out[i] = in[i];
        if (flag)
            out[i].bar += 5;
    }
}

Bad example 2:
void dut(A in[N], A out[out], bool flag) {
    #pragma HLS interface ap_fifo port=in,out
    for (unsigned i=0; i<N; i++) {
        out[i].foo = in[i].foo;
        if (flag)
```

```

    out[i].bar = in[i].bar + 5;
  else
    out[i].bar = in[i].bar;
  }
}

```

存储器映射接口

如前文所述，Vitis HLS 允许用户为接口中的阵列指定 M_AXI 接口。由于此存储器为片外存储器，而非本地存储器，因此访问这些存储器可能需耗费大量周期。为了优化这些访问的方式，Vitis HLS 会执行自动突发优化，以便有效读取/写入这些外部存储器。突发是一种优化方式，它会尝试以智能方式聚集对 DDR 的存储器访问操作，以便尽可能提升吞吐量带宽和/或减小时延。突发是可对内核执行的诸多优化操作之一。突发通常能够实现 4 到 5 倍的提升。DDR 端口上存在争用（源于多个相互竞争的内核）时，适合使用突发。

如需获取有关此突发优化的更多信息以及有关如何编写代码来推断更多突发的详细信息，请参阅 [AXI 系统性能优化](#) 部分。

阵列初始化和复位



建议：AMD 建议（但并不强制）将阵列指定为作为含 `static` 限定符的存储器来实现。这不仅可确保 Vitis HLS 以 RTL 中的存储器来实现阵列，还允许使用“static”（静态）类型的默认初始化行为。

在以下代码中，阵列利用一组值来进行初始化。每次执行函数时，都会为 `coeff` 阵列分配这些值。综合后，每次执行设计时，用于实现 `coeff` 的 RAM 都会随这些值一起加载。对于单端口 RAM，此操作耗时 8 个时钟周期。对于 1024 阵列，当然也就需要 1024 个时钟周期，在此期间无法执行任何依赖于 `coeff` 的运算。

```
int coeff[8] = {-2, 8, -4, 10, 14, 10, -4, 8, -2};
```

以下代码使用 `static` 限定符来定义 `coeff` 阵列。该阵列开始执行时会使用指定的值进行初始化。每次执行该函数时，`coeff` 阵列都会保留上次执行的值。静态阵列在 C/C++ 语言代码中的行为与 RTL 中的存储器行为相同。

```
static int coeff[8] = {-2, 8, -4, 10, 14, 10, -4, 8, -2};
```

此外，如果变量包含 `static` 限定符，那么 Vitis HLS 会对 RTL 设计和 FPGA 比特流中的变量进行初始化。因此，无需经历多个时钟周期来初始化存储器，并且可确保大型存储器初始化不会产生任何运算开销。请参阅 GitHub 上提供的 [initialization_and_reset](#) 示例，以获取更多示例。

RTL 配置命令 `syn.rtl.reset` 可指定在应用复位后，静态变量是否返回其初始状态。这并非默认设置。使用 `syn.rtl.reset=state` 或 `all` 时，会强制作为块 RAM 实现的所有阵列都在复位后返回初始化状态。这可能导致 RTL 设计中出现 2 个非常不利的条件：

- 不同于上电初始化（或上电复位），显式复位要求 RTL 设计对块 RAM 中的每个地址进行迭代以设置该值：如果 N 较大，这可能导致时钟周期数量显著增加，并增加实现复位所需的面积资源。
- 在设计中每个阵列中都添加复位。

为防止在每个此类 BRAM 上添加复位逻辑，并因复位 RAM 内所有元素而产生周期开销，请指定默认 `syn.rtl.reset=control` 复位模式，并使用 RESET 编译指示或指令来识别要复位的个别静态变量或全局变量。

或者，可使用 `syn.rtl.reset=state` 复位模式，并使用 RESET 指令 `off` 选项选择不执行复位的个别静态变量或全局变量。

最后，根据您所选的硬件器件或平台（UltraScale+ 或 Versal 等），块 RAM 和 URAM 的初始化和/或复位方式可能不尽相同。总体上，Vitis HLS 支持两类复位：一种类型是器件上电（也称为上电初始化或上电复位），第二种类型是在器件执行期间，硬件 RESET 信号断言有效。以下显示了不同存储器资源的行为差异：

- 初始化行为：适用于所有平台上的所有块 RAM，仅适用于 Versal URAM。这是上电初始化（或上电复位）期间的行为。
- 如果读取/写入阵列，则保留“initial value array”（初始值阵列）和“runtime array”（运行时阵列）。此行为适用于块 RAM 和 URAM，对应于器件执行期间的硬件 RESET 信号。



提示：将针对相同地址的读取和写入操作映射到相同存储器端口时，URAM 不支持 read-first（先读取）输出 write_mode（不同于块 RAM）。块 RAM 支持以下写入模式：write thru、read first 和 no change。URAM 仅支持 no change。如果 Vitis HLS 在 URAM 端口上无法将存储器操作调度为在同一个周期内执行，那么将发出以下警告消息：

```
Usage of URAM can potentially cause worse II as Vitis HLS does not
exploit
read-first mode for URAMs. Consider using block RAMs instead.
```

实现 ROM

如果数组为只读，则建议对其使用 `const` 限定符，因为 Vitis HLS 并不总是推断设计分析是否应使用 ROM。ROM 的自动推断的一般规则是局部（非全局）推断，`static` 阵列先完全写入而后读取，且不再写入。在代码中遵循下列实践有助于推断 ROM：

- 在使用阵列的函数中尽早初始化该阵列。
- 将写操作组合在一起。
- 勿将 `array(ROM)` 初始化写操作与非初始化代码交织在一起。
- 勿将不同值存储到相同阵列元素中（将所有写操作一起组合到代码中）。
- 元素值计算不得依赖于除初始化循环计数器变量以外的任何非常量（在编译时）设计变量。

如果使用复数赋值来初始化 ROM（例如，来自 `math.h` 库的函数），将阵列初始化置于独立函数中即可允许推断 ROM。在以下示例中，完成 RTL 综合后，将 `sin_table[256]` 阵列推断为存储器，并作为 ROM 来实现。

```
#include "array_ROM_math_init.h"
#include <math.h>

void init_sin_table(din1_t sin_table[256])
{
    int i;
    for (i = 0; i < 256; i++) {
        dint_t real_val = sin(M_PI * (dint_t)(i - 128) / 256.0);
        sin_table[i] = (din1_t)(32768.0 * real_val);
    }
}

dout_t array_ROM_math_init(din1_t inval, din2_t idx)
{
    short sin_table[256];
    init_sin_table(sin_table);
    return (int)inval * (int)sin_table[idx];
}
```



提示：由于 `sin()` 函数生成了常量值，因此 RTL 设计中无需任何核即可实现 `sin()` 函数。

阵列的 C 语言仿真

在 C/C++ 仿真期间，阵列可能引发各种问题，即使在执行综合步骤前也是如此。如果您指定的阵列过大，可能导致 C/C++ 仿真因存储器不足而失败，如下示例所示：

```
#include "ap_int.h"

int i, acc;
// Use an arbitrary precision type
ap_int<32> la0[100000000], la1[100000000];

for (i=0 ; i < 100000000; i++) {
    acc = acc + la0[i] + la1[i];
}
```

仿真可能因存储器不足而失败，因为阵列置于存储器中存在的栈上，而不是置于由操作系统管理并且可使用本地磁盘空间来扩展的堆上。某些问题可能提高出现此问题的概率：

- 在 PC 上，可用存储器通常小于大型 Linux 系统，并且可用存储器可能进一步减少。
- 如以上示例所示，使用任意精度类型可能使此问题更严重，因为这些类型建模时所需存储器比标准 C/C++ 类型所需存储器更多。
- 使用 C/C++ 中的更复杂的定点任意精度类型可能还会提高设计出现存储器不足问题的概率，因为各类型所需存储器更多。

增加 C/C++ 代码开发中的存储器资源的标准方法是使用连接器选项增大栈大小，例如以下选项将栈大小显式设置为 `syn.csimflags -z stack-size=10485760`。

但机器可能没有足够存储器可用，且增大栈大小无效。在此情况下，解决方案是使用动态存储器分配执行仿真，但使用固定大小的阵列执行综合，如下示例所示。这意味着此操作所需存储器将在堆上进行分配，且此堆由操作系统管理并可使用本地磁盘空间来扩展。

```
#include "ap_int.h"

int i, acc;
#ifdef __SYNTHESIS__
    // Use an arbitrary precision type & array for synthesis
    ap_int<32> la0[100000000], la1[100000000];
#else
    // Use an arbitrary precision type & dynamic memory for simulation
    ap_int<32> *la0 = malloc(100000000 * sizeof(ap_int<32>));
    ap_int<32> *la1 = malloc(100000000 * sizeof(ap_int<32>));
#endif
for (i=0 ; i < 100000000; i++) {
    acc = acc + la0[i] + la1[i];
}
```

但这并非理想解决方案，因为仿真的代码与综合的代码不同。但这是完成仿真的唯一途径。如果您采用此方法，请务必确认 C/C++ 测试平台涵盖阵列访问的各方面。由 `cosim_design` 执行的 RTL 仿真将验证综合后的代码中执行的存储器访问是否正确。

注释： `__SYNTHESIS__` 宏仅限在要综合的代码上使用。请勿在测试平台中使用该宏，因为在 C/C++ 语言仿真或 C/C++ RTL 协同仿真中，该宏无关紧要。请参阅 [Vitis-HLS-Introductory-Examples/Pipelining/Functions/hier_func](#) 以获取此示例的完整版本。

函数入门

顶层函数综合后即变为 RTL 设计的顶层模块。未内联的所有子函数都会被综合到 RTL 设计中的独立模块内。顶层函数的实参是作为硬件中的接口端口来实现的，如 [HLS 设计接口](#) 中所述。无法从外部访问内核使用的全局变量。供测试平台（或其他已编译的内核或主机）及内核本身访问的任意变量都应定义为顶层函数的实参。



重要提示！ 顶层函数不得采用静态函数。

综合后，设计中每个函数都有其自己的综合报告和 HDL 文件（Verilog 和 VHDL）。

函数内联

函数内联将函数逻辑分解为调用函数，从而移除函数层级。函数可使用 `INLINE` 指令进行内联。内联函数可支持函数中的组件以更高效的方式与调用函数中的逻辑共享或利用该逻辑进行优化，从而改善面积。Vitis HLS 也会为小型函数自动执行此类型的函数内联。

内联支持对函数共享进行更有效的控制。对于要共享的函数，必须在相同层级内使用。在此代码示例中，`foo_top` 函数会对 `foo` 进行 2 次调用，并调用 `foo_sub` 函数。

```
foo_sub (p, q) {
    int q1 = q + 10;
    foo(p1,q); // foo_3
    ...
}
void foo_top { a, b, c, d} {
    ...
    foo(a,b); //foo_1
    foo(a,c); //foo_2
    foo_sub(a,d);
    ...
}
```

将函数 `foo_sub` 内联并使用 `ALLOCATION` 指令指定仅使用 `foo` 函数的 1 个实例，这样生成的设计仅含 1 个 `foo` 函数的实例：面积为以上示例的三分之一。使用此策略即可对可共享的用户定义资源加以更精细的控制，如需考量面积使用率，那么此策略也是一项很实用的功能特性。

```
foo_sub (p, q) {
#pragma HLS INLINE
    int q1 = q + 10;
    foo(p1,q); // foo_3
    ...
}
void foo_top { a, b, c, d} {
#pragma HLS ALLOCATION instances=foo limit=1 function
    ...
}
```

```
foo(a,b); //foo_1
foo(a,c); //foo_2
foo_sub(a,d);
...
}
```

INLINE 指令可选择使用 `recursive` 选项来允许位于指定函数下层的所有函数以递归方式进行内联。如果对顶层函数使用 `recursive` 选项，那么将移除设计中的所有函数层级。

可选择对函数应用 `INLINE off` 选项以阻止对这些函数进行内联。该选项可用于阻止 Vitis HLS 自动进行函数内联。

INLINE 指令是大幅修改代码结构而不对源代码执行任何实际修改的强大方法，并可提供强大的架构探索方法。

函数流水打拍

函数流水打拍的处理方式与 [循环流水打拍](#) 中所述循环流水打拍相似。Vitis HLS 对于函数体的处理方式是将其视为与多次调用的循环主体相同，唯一区别在于，函数体是多次调用的函数，工具会对这些调用的执行进行流水打拍。因此，与循环类似，函数流水打拍时，函数体以及下层层级中的所有循环都会自动展开。这是继续执行流水打拍的前提。如果循环具有变量边界，且无法展开，那么这将导致无法对函数进行流水打拍。

函数例化

函数例化是一种优化技巧，不仅具有维持函数层级的面积优势，还可提供另一个强大的选项：在函数的特定实例上执行针对性局部优化。这样可以简化围绕函数调用的控制逻辑，也可能改进时延和吞吐量。

鉴于调用函数时，函数的部分输入可能是常量，`FUNCTION_INSTANTIATE` 编译指示或指令可藉此简化周围控制结构，并生成进一步优化的、更小的函数块。如下代码所示示例对此进行了最明确的解释。

```
char func(char inval, char incr) {
#pragma HLS INLINE OFF
#pragma HLS FUNCTION_INSTANTIATE variable=incr
    return inval + incr;
}

void top(char inval1, char inval2, char inval3,
         char *outval1, char *outval2, char *outval3)
{
    *outval1 = func(inval1, 0);
    *outval2 = func(inval2, 1);
    *outval3 = func(inval3, 100);
}
```



提示：Vitis HLS 工具会将小函数自动分解（或内联）到更高层次的调用函数中。将 `INLINE` 编译指示与 `OFF` 选项搭配使用即可阻止此自动内联操作。

很明显，函数 `func` 已写为执行三次互斥运算（根据 `incr` 的值）。函数 `func` 的每个实例都是以相同方式实现的。虽说这对于函数复用和面积优化很实用，但也意味着函数内部的控制逻辑必须更为复杂，才能容纳两次互斥运算。请参阅 [Vitis-HLS-Introductory-Examples/Pipelining/Functions/function_instantiate](#) 以获取此示例的完整版本。

FUNCTION_INSTANTIATE 优化允许对每个实例进行独立优化，从而减少功能和面积。完成 FUNCTION_INSTANTIATE 优化后，以上代码可有效转换为 2 个独立函数，每个函数都针对模式的不同可能值来完成优化，如下所示：

```
void func1() {  
    // code segment 1  
}  
  
void func2() {  
    // code segment 2  
}
```

如果在未经大幅内联或代码修改的情况下，在不同层级使用该函数而导致函数难以共享，那么函数例化可提供改进面积的最佳方法：大量小型局部优化的副本比大量无法共享的大型副本更有效。

数据类型



重要提示！ 鉴于库依赖关系是专为 64 位系统编译的，因此 Vitis HLS 不支持 32 位构建。所以不允许使用 `-m32` 标志，它将导致错误。

如果 C/C++ 语言函数编译为可执行文件，则函数中使用的数据类型会影响结果的准确性以及存储器要求，并且可能影响性能。32 位整数 `int` 数据类型可包含更多数据，因此精度比 8 位 `char` 类型更高，但需更多存储空间。同样，将 C/C++ 语言函数综合到 RTL 实现中时，这些数据类型会影响 RTL 设计的精度、面积和性能。用于变量的数据类型可判定所需的运算符大小，因此可判定 RTL 的面积和性能。

Vitis HLS 支持对所有标准 C/C++ 语言类型进行综合，包括精确宽度整数类型。

- `(unsigned) char`、`(unsigned) short` 和 `(unsigned) int`
- `(unsigned) long` 和 `(unsigned) long long`
- `(unsigned) intN_t`（其中，N 为 8、16、32 或 64，在 `stdint.h` 中定义）
- `float` 和 `double`

精确宽度整数类型可用于确保设计在各种类型的系统之间的可移植性。

C/C++ 语言标准规定 `(unsigned)long` 类型在 64 位操作系统上作为 64 位来实现。综合可根据运行 Vitis HLS 的操作系统类型来匹配此行为并生成不同大小的运算符，从而生成不同 RTL 设计。在 Windows 操作系统上，Microsoft 将 `long` 类型定义为 32 位。

- 针对 32 位，使用 `(unsigned)int` 或 `(unsigned)int32_t` 数据类型代替 `(unsigned)long` 类型。
- 针对 64 位，使用 `(unsigned)long long` 或 `(unsigned)int64_t` 数据类型代替 `(unsigned)long` 类型。

AMD 强烈建议在单一公用头文件中定义所有变量的数据类型，该头文件可包含在所有源文件中。

- 在一般 Vitis HLS 工程中，可对部分数据类型进行调优，例如，用于减小其大小和支持更高效的硬件实现。
- 提高工作的抽象层次的主要作用之一在于能够快速创建新设计实现。通常在后续工程中使用的文件是相同的，但可能使用的数据类型不同（更小、更大或更准确）。

在单一位置更改数据类型更便于完成这 2 项任务：替代方法是对多个文件进行编辑。



重要提示！ 在头文件中使用宏时，请始终使用唯一名称。例如，如果头文件中定义了 1 个名为 `_TYPES_H` 的宏，由于此名称较为常见，在其他系统文件中可能也会定义相同名称，它可能会启用或禁用某些其他代码，从而导致不可预见的意外结果。



提示： `std::complex<long double>` 数据类型在 Vitis HLS 中不受支持，不应使用。

标准类型

以下代码示例显示了所执行的部分基本算术运算。

```
#include "types_standard.h"

void types_standard(din_A inA, din_B inB, din_C inC, din_D inD,
  dout_1 *out1, dout_2 *out2, dout_3 *out3, dout_4 *out4
) {

  // Basic arithmetic operations
  *out1 = inA * inB;
  *out2 = inB + inA;
  *out3 = inC / inA;
  *out4 = inD % inA;

}
```

以上示例中的数据类型是在 `types_standard.h` 头文件中定义的，如下代码示例所示。其中显示了以下类型的使用方式：

- 标准有符号类型
- 无符号类型
- 精确宽度的整数类型（包含 `stdint.h` 头文件）

```
#include <stdio.h>
#include <stdint.h>

#define N 9

typedef char din_A;
typedef short din_B;
typedef int din_C;
typedef long long din_D;

typedef int dout_1;
typedef unsigned char dout_2;
typedef int32_t dout_3;
typedef int64_t dout_4;

void types_standard(din_A inA, din_B inB, din_C inC, din_D inD, dout_1
*out1, dout_2 *out2, dout_3 *out3, dout_4 *out4);
```

这些不同类型经过综合可生成以下运算符和端口大小：

- 用于计算结果 `out1` 的乘法器为 24 位乘法器。8 位 `char` 类型乘以 16 位 `short` 类型需要 24 位乘法器。结果是符号位将扩展为 32 位，以与输出端口宽度匹配。
- 用于 `out2` 的加法器为 8 位。由于输出为 8 位 `unsigned char` 类型，因此仅将底部 8 位 `inB`（16 位 `short`）与 8 位 `char` 类型的 `inA` 相加。
- 对于 `out3` 输出（32 位精确宽度类型），8 位 `char` 类型的 `inA` 的符号位将扩展为 32 位值，并使用 32 位（`int` 类型）`inC` 输入执行 32 位除法运算。

- 使用 64 位 `long long` 类型的 `inD` 来执行 64 位模数运算，8 位 `char` 类型的 `inA` 的符号位将扩展至 64 位，以创建 64 位输出 `out4` 的结果。

根据 `out1` 结果指示，Vitis HLS 使用可用的最小运算符，并扩展结果以与所需输出位宽相匹配。对于 `out2` 结果，即使其中一项输入为 16 位，仍可使用 8 位加法器，因为只需 8 位输出。正如 `out3` 和 `out4` 的结果所示，如需所有位元，则将对完整大小的运算符进行综合。

浮点与双精度

Vitis HLS 支持 `float` 和 `double` 类型的综合。这两种数据类型均依据 IEEE-754 标准部分合规性来进行综合（请参阅《Floating-Point Operator LogiCORE IP 产品指南》(PG060)）。

- 单精度 32 位
 - 24 位小数
 - 8 位指数
- 双精度 64 位
 - 53 位小数
 - 11 位指数



建议：使用浮点数据类型时，AMD 强烈建议您查阅《使用 Vivado HLS 进行浮点设计》(XAPP599)。另请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Modeling/using_float_and_double](#)，以获取浮点和双精度数据类型的使用示例。

除了将浮点和双精度类型用于标准算术运算（例如，+、-、* 等）外，浮点和双精度通常还可配合 `math.h` 一起使用（针对 C++，这 2 个类型还可配合 `cmath.h` 一起使用）。本节讨论了为标准运算符提供的支持。

以下代码示例显示的头文件可配合更新后的 [标准类型](#) 一起使用，以将数据类型定义为 `double` 和 `float` 类型。

```
#include <stdio.h>
#include <stdint.h>
#include <math.h>

#define N 9

typedef double din_A;
typedef double din_B;
typedef double din_C;
typedef float din_D;

typedef double dout_1;
typedef double dout_2;
typedef double dout_3;
typedef float dout_4;

void types_float_double(din_A inA, din_B inB, din_C inC, din_D inD, dout_1
*out1, dout_2 *out2, dout_3 *out3, dout_4 *out4);
```

此更新后的头文件可配合以下代码示例一起使用，此代码中使用了 `sqrtf()` 函数。

```
#include "types_float_double.h"

void types_float_double(
    din_A inA,
    din_B inB,
```

```

din_C   inC,
din_D   inD,
dout_1  *out1,
dout_2  *out2,
dout_3  *out3,
dout_4  *out4
) {

// Basic arithmetic & math.h sqrtf()
*out1 = inA * inB;
*out2 = inB + inA;
*out3 = inC / inA;
*out4 = sqrtf(inD);

}

```

对以上示例进行综合后，可生成 64 位双精度乘法器、加法器和除法器运算符。这些运算符可由相应的浮点 AMD IP 目录核来实现。

使用 `sqrtf()` 的平方根函数是使用 32 位单精度浮点核来实现的。

如果使用双精度平方根函数 `sqrt()`，则将生成额外逻辑以在用于 `inD` 的 32 位单精度浮点类型与其他类型之间进行双向强制转换，`out4: sqrt()` 为双精度 (`double`) 函数，`sqrtf()` 为单精度 (`float`) 函数。

在 C 语言函数中，混用浮点类型和双精度类型时请谨慎处理，因为浮点到双精度和双精度到浮点转换单位是在硬件中推断所得的。

```

float foo_f      = 3.1459;
float var_f = sqrt(foo_f);

```

以上代码可生成如下硬件：

```

wire(foo_t)
-> Float-to-Double Converter unit
-> Double-Precision Square Root unit
-> Double-to-Float Converter unit
-> wire (var_f)

```

使用 `sqrtf()` 函数即可：

- 使硬件无需再使用类型转换器
- 节省面积
- 改进时序

对浮点和双精度类型进行综合时，Vitis HLS 会保留 C 语言代码中执行的运算顺序，以确保结果与 C 语言仿真结果相同。由于执行的饱和与截位操作，在单精度运算和双精度运算中不保证：

```

A=B*C; A=B*F;
D=E*F; D=E*C;
O1=A*D O2=A*D;

```

对于 `float` 和 `double` 类型，不保证 `O1` 和 `O2` 相同。



提示：在某些情况下（视设计而定），展开或部分展开循环等优化可能无法充分利用并行计算，因为 Vitis HLS 对浮点和双精度类型进行综合时会严格保留运算顺序。您可使用 `config_compile -unsafe_math_optimizations` 覆盖此限制。

对于 C++ 设计，Vitis HLS 可提供最常用的数学函数的位近似实现。

浮点累加器和 MAC

浮点累加器 (facc)、乘积累加 (fmacc) 和乘加 (fmadd) 均可使用 `syn.op` 配置命令来启用，如 [运算符配置](#) 中所述。下面给出了 1 个示例：

```
syn.op=op:mul impl:dsp
syn.op=op:add impl:fabric latency:6
syn.op=op:fmacc precision:high
syn.op=op:hdiv latency:5
```

本工具会为这些运算符提供不同精度级别的支持，以期在 Versal 器件和非 Versal 器件上能实现性能、面积和精度的取舍权衡。

- 低精度累加适用于高吞吐量低精度浮点累加和乘积累加，该模式仅在非 Versal 器件上可用。
 - 它使用具有预分频器和后分频器的整数累加器来将输入和输出转换为单精度或双精度浮点。
 - 它分别使用 60 位和 100 位累加器来执行单精度输入和双精度输入。
 - 由于 C++ 仿真相关精度不足，因此它可能导致协同仿真发生不匹配
 - 它始终采用流水打拍，且 II=1，无源代码更改
 - 它使用的资源量约为标准精度浮点累加的 3 倍，因此根据时钟频率和目标器件，II 通常在 3 到 5 之间。

使用低精度的情况下，在所有器件上都支持浮点和双精度累加，启动时间间隔 (II) 均为 1。这意味着以下代码可采用流水打拍，II 为 1 且无需进一步编码：

```
float foo(float A[10], float B[10]) {
    float sum = 0.0;
    for (int i = 0; i < 10; i++) {
        sum += A[i] * B[i];
    }
    return sum;
}
```

- 标准精度累加和乘加适合大部分浮点用例，并且可在 Versal 器件和非 Versal 器件上使用。
 - 它始终使用真正的浮点累加器
 - 在 Versal 器件上，它可采用流水打拍，且 II=1。
 - 在非 Versal 器件上，它可采用流水打拍，且 II 通常介于 3 到 5 之间（取决于时钟频率和目标器件）。标准精度模式在 Versal 器件上比在非 Versal 器件上效率更高。
- 高精度融合乘加适合高精度应用，可在 Versal 器件上使用。
 - 它使用额外的精度位
 - 虽然它使用的资源比非融合乘加更多，但它始终使用单个融合乘加，末尾采用单个舍入
 - 由于 C++ 仿真相关的额外精度，因此它可能导致协同仿真发生不匹配

复合数据类型

HLS 支持对以下复合数据类型执行综合：

- 结构体
- 枚举类型
- 联合体

结构体

代码中的结构体（例如，内部变量和全局变量）默认情况下处于解聚 (disaggregated) 状态。这些结构体解聚为对应每个成员元素的独立对象。创建的元素数量和类型取决于结构体本身的内容。结构体阵列作为多个阵列来实现，每个结构体成员都具有独立的阵列。



重要提示！ 默认情况下，用作为顶层函数的结构体将聚合，如 [接口中的结构体](#) 中所述。

或者，您可使用 [AGGREGATE](#) 编译指示或指令将某一结构体的所有元素都集合到单宽矢量中。这样即可同时读写结构体的所有成员。聚合后的结构体将按需填充，以在 4 字节边界上对齐各元素，如 [结构体填充与对齐](#) 中所述。该结构体的成员元素按 C/C++ 语言代码中所示顺序置于该矢量内：结构体的第一个元素对齐矢量的 LSB，结构体的最后一个元素对齐矢量的 MSB。结构体中的任意阵列都分区到独立阵列元素中，并按从低到高的顺序置于矢量内。



提示： 对含大型阵列的结构体使用 [AGGREGATE](#) 编译指示时，应谨慎处理。如果某个阵列包含 4096 个类型为 `int` 的元素，则将导致矢量（和端口）宽度达 $4096 * 32 = 131072$ 位。虽然 Vitis HLS 可创建此 RTL 设计，但 Vivado 在实现期间很可能无法完成其布线。

使用 [AGGREGATE](#) 指令创建的单宽矢量支持在单一时钟周期内访问更多数据。在单一时钟周期内访问数据时，Vitis HLS 会自动展开使用此数据的所有循环（前提是可提高吞吐量）。循环可全部或部分展开以创建足够的硬件以便在单一时钟周期内使用更多数据。此功能特性是使用 `syn.unroll.tripcount_threshold` 配置命令来控制的，如 [展开设置](#) 中所述。

在以下示例中，循环次数小于 16 的任何循环都将自动展开（前提是可提高吞吐量）：

```
syn.unroll.tripcount_threshold=16
```

如果结构体包含阵列，那么 [AGGREGATE](#) 指令执行的操作与 `ARRAY_RESHAPE` 类似，可将重构的阵列与结构体中的其他元素组合在一起。但是，使用 [AGGREGATE](#) 对结构体进行优化后，就无法对该结构体再进行分区或重构。[AGGREGATE](#)、`ARRAY_PARTITION` 与 `ARRAY_RESHAPE` 指令互斥。

接口中的结构体

默认情况下，Vitis HLS 会使接口上的结构体保持处于聚合状态，将任一结构体的所有元素都组合到一个单宽矢量中。这样即可同时读写结构体的所有成员。您可使用 [DISAGGREGATE](#) 编译指示或指令对接口中的结构体进行解聚。当结构体包含一个或多个 `hls::stream` 对象时，Vitis HLS 将按下文《接口中含 `hls::stream` 元素的结构体》中所述，对结构体进行自动解聚。



重要提示！ 在 Vitis 内核流程中不支持对接口中的结构体进行解聚，因为 Vitis 工具无法将单个 C 语言实参映射到多个 RTL 端口。对接口中的结构体进行解聚时，Vitis HLS 将通过手动或自动方式来构建和导出 Vitis 内核输出（.xo），但搭配 `v++` 命令一起使用时，此输出将出错。要支持 Vitis 内核流程，您必须将此结构体手动拆分为其组成成分元素，并将任意 `hls::stream` 对象定义为使用 [AXIS 接口](#)。

在聚合过程中，针对 Vitis 内核流程，结构体的所有元素都在 4 字节对齐位置对齐，对于 Vivado IP 流程，则在 1 字节对齐位置对齐。此对齐可能需要添加位填充，以对齐各元素或者使各元素保持对齐，如 [结构体填充与对齐](#) 中所述。默认情况下，聚合的结构体将进行填充而不是打包，但在 Vivado IP 流程中，您可使用 [AGGREGATE](#) 编译指示或指令的 `compact=bit` 选项将其打包。但定义为 AXI4 接口的任意端口（`m_axi`、`s_axilite` 或 `axis`）都无法使用 `compact=bit`。

该结构体的成员元素按 C/C++ 语言代码中所示顺序置于该矢量内：结构体的第一个元素对齐矢量的 LSB，结构体的最后一个元素对齐矢量的 MSB。这样即可在单一时钟周期内访问更多数据。结构体中的任意阵列都分区到独立阵列元素中，并按从低到高的顺序置于矢量内。

在以下示例中所头文件内定义 `struct data_t`。此结构体包含 2 个数据成员：

- 无符号的矢量 `varA`，类型为 `short`（16 位）。
- 阵列 `varB`，含 4 种 `unsigned char` 类型（8 位）。

```
typedef struct {
    unsigned short varA;
    unsigned char varB[4];
} data_t;

data_t struct_port(data_t i_val, data_t *i_pt, data_t *o_pt);
```

在接口上聚合结构体会生成单个 48 位端口，其中包含 16 位 `varA` 和 4x8 位 `varB`。



提示：由数据打包创建的任意端口或总线的最大位宽为 8192 位，或者针对 `axis` 串流接口则为 4096 位。

对于 Vitis HLS 可综合的结构体的大小或复杂性并无限制。结构体中可包含所需的任意数量的阵列维度和成员。仅当阵列作为串流（例如，FIFO 接口）来实现时，才会出现结构体实现的唯一限制。在此情况下，请遵循适用于接口（FIFO 接口）上的阵列的常规规则进行操作。

接口上含 `hls::stream` 元素的结构体

包含 `hls::stream` 元素的接口上的用户定义的结构体将由 Vitis HLS 自动进行解聚。在 Vivado IP 流程中，支持解聚的结构体，并且导出的 IP 将能够按期望的方式工作。但 Vitis 内核流程则不支持此解聚的结构体，并且导出的内核（.xo）搭配 `v++ --link` 命令一起使用时，将导致出错。要支持 Vitis 内核流程，您必须将此结构体手动拆分为其组成成分元素，并将 `hls::stream` 对象定义为使用 `AXIS` 接口。

如有结构体已自动解聚，Vitis HLS 会将任意 `INTERFACE` 编译指示应用于已解聚的结构体的各个元素。如果针对此结构体仅指定一条 `INTERFACE` 编译指示，那么它将应用于该结构体的每个元素。如果您为已解聚的结构体的每个元素都提供一个 `INTERFACE` 编译指示，则会按期望方式来应用编译指示。

结构体填充与对齐

Vitis HLS 中的结构体根据所使用的 `__attributes__` 或 `#pragmas`，可能采用不同类型的填充和对齐方式。这些功能如下所述。

- 解聚：默认情况下，代码中作为内部变量的结构体已解聚为个别元素。创建的元素数量和类型取决于结构体本身的内容。Vitis HLS 将根据某些优化条件来判定任一结构体是否将解聚。



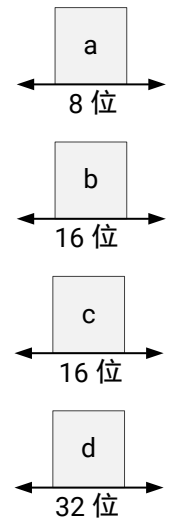
提示：您可使用 `AGGREGATE` 编译指示或指令来阻止代码中的结构体的默认解聚操作。

图 27：解聚结构体

```

struct example {
  ap_int<5> a;
  unsigned short int b;
  unsigned short int c;
  int d;
};
void foo()
{
  example s0;
  #pragma HLS disaggregate variable=s0
}

```



X24681-030622

- 聚合：该工具的默认行为是在接口上聚合结构体，如 [接口中的结构体](#) 中所述。Vitis HLS 会将结构体的元素连接在一起，从而将此结构体聚合为单一数据单元。此默认操作是根据 [AGGREGATE](#) 编译指示或指令完成的，但您无需指定编译指示，因为对于接口上的结构体而言，这是默认行为。聚合过程可能还包含对结构体元素进行位填充，以便按默认 4 字节对齐或指定对齐方式来实现字节结构对齐。



提示：指定 `-Wpadded` 作为编译器标志来添加用于填充结构体的多个位时，该工具会发出警告。

- 已对齐：默认情况下，Vitis HLS 会按 4 字节对齐方式来对齐结构体，通过填充结构体元素来将其对齐到 32 位宽。但您可使用 `__attribute__((aligned(X)))` 在结构体元素间添加填充，以便将其对齐到“X”字节边界。



重要提示！ 请注意，“X”只能定义为 2 的幂。

`__attribute__((aligned))` 不会更改所应用到的变量的大小，而是可以更改结构的存储器布局，方法是在结构体的各元素之间插入填充。由此即可更改结构的大小。

对于含定制数据宽度的结构体中的数据类型（如，`ap_int`），为其分配的大小为 2 的幂。Vitis HLS 会添加填充位，用于将数据类型的大小对齐到 2 的幂。

Vitis HLS 还将填充 `bool` 数据类型，以将其对齐到 8 位。

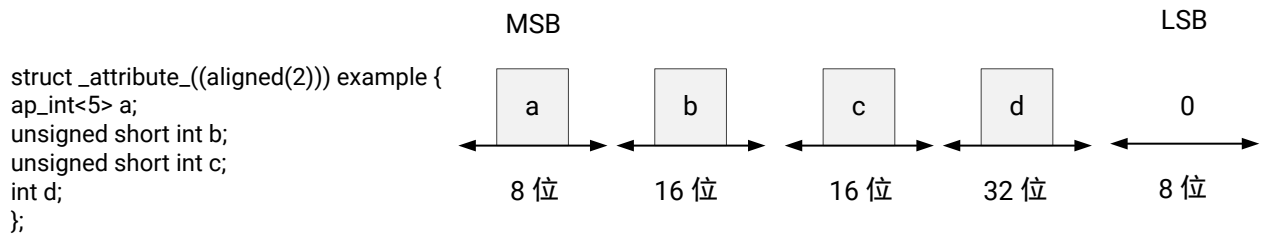
在以下示例中，结构体中 `varA` 的大小将填充到 8 位（而不是 5 位）。

```

struct example {
  ap_int<5> varA;
  unsigned short int varB;
  unsigned short int varC;
  int d;
};

```

图 28：已对齐的结构体实现



X24682-030622

所使用的填充取决于结构体元素的顺序和大小。在以下代码示例中，结构体对齐为 4 字节，Vitis HLS 将在第一个元素 `varA` 之后添加 2 字节用于填充，在第三个元素 `varC` 之后再添加 2 字节用于填充。结构体的总计大小将为 96 位。

```

struct data_t {
  short varA;
  int varB;
  short varC;
};
        
```

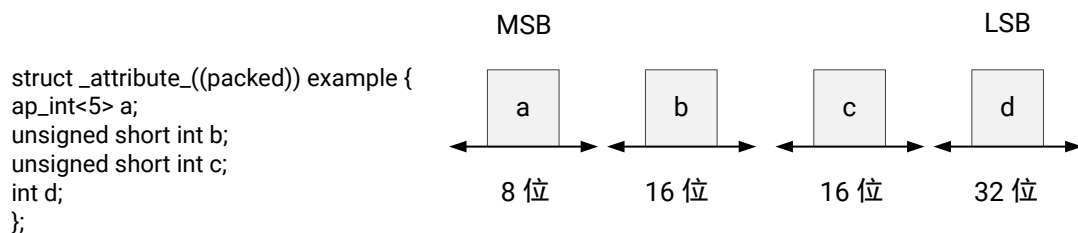
但如果您按如下方式重写该结构体，则无需填充，而结构体的总计大小将为 64 位。

```

struct data_t {
  short varA;
  short varC;
  int varB;
};
        
```

- 已封装：Vitis HLS 通过指定 `__attribute__((packed))` 来封装结构体元素，在此情况下结构体大小取决于结构体每个元素的实际大小。在以下示例中，这意味着结构体大小为 72 位：

图 29：已封装的结构体实现



X24680-030622



提示：这也可通过使用 `AGGREGATE` 编译指示或指令的 `compact=bit` 选项来达成。

C++ 类和模板

Vitis HLS 完全支持对 C++ 类进行综合，但仅支持对顶层函数进行综合，而不支持对顶层的类进行综合。要对类成员函数进行综合，请将类本身例化为函数。请勿直接将顶层类例化为测试平台。以下代码示例显示了如何在 `cpp_FIR` 顶层函数中对后文所述头文件中定义的 `CFir` 类进行例化，以及如何使用该类实现 `FIR` 滤波器。

```
#include "cpp_FIR.h"

// Top-level function with class instantiated
data_t cpp_FIR(data_t x)
{
    static CFir<coef_t, data_t, acc_t> fir1;

    cout << fir1;

    return fir1(x);
}
```



重要提示！ 不支持对位于顶层的类和类成员函数进行综合。请例化顶层函数中的类。

检验以上 C++ FIR 滤波器示例中用于实现设计的类之前，值得注意的是，Vitis HLS 在综合期间会忽略标准输出串流 `cout`。完成综合后，Vitis HLS 会发出以下警告：

```
INFO [SYNCHK-101] Discarding unsynthesizable system call:
'std::ostream::operator<<' (cpp_FIR.h:108)
INFO [SYNCHK-101] Discarding unsynthesizable system call:
'std::ostream::operator<<' (cpp_FIR.h:108)
INFO [SYNCHK-101] Discarding unsynthesizable system call: 'std::operator<<
<std::char_traits<char> >' (cpp_FIR.h:110)
```

以下代码示例显示了 `cpp_FIR.h` 头文件，包括 `CFir` 类的定义及其关联的成员函数。在此示例中，运算符成员函数 `()` 和 `<<` 为重载运算符，分别用于执行主算法以及配合 `cout` 用于数据格式化以便在 C/C++ 语言仿真期间显示。

```
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

#define N 85

typedef int coef_t;
typedef int data_t;
typedef int acc_t;

// Class CFir definition
template<class coef_T, class data_T, class acc_T>
class CFir {
protected:
    static const coef_T c[N];
    data_T shift_reg[N-1];
private:
public:
    data_T operator()(data_T x);
    template<class coef_TT, class data_TT, class acc_TT>
    friend ostream&
    operator<<(ostream& o, const CFir<coef_TT, data_TT, acc_TT> &f);
};
```

```
// Load FIR coefficients
template<class coef_T, class data_T, class acc_T>
const coef_T CFir<coef_T, data_T, acc_T>::c[N] = {
    #include "cpp_FIR.h"
};

// FIR main algorithm
template<class coef_T, class data_T, class acc_T>
data_T CFir<coef_T, data_T, acc_T>::operator()(data_T x) {
    int i;
    acc_t acc = 0;
    data_t m;

    loop: for (i = N-1; i >= 0; i--) {
        if (i == 0) {
            m = x;
            shift_reg[0] = x;
        } else {
            m = shift_reg[i-1];
            if (i != (N-1))
                shift_reg[i] = shift_reg[i - 1];
        }
        acc += m * c[i];
    }
    return acc;
}

// Operator for displaying results
template<class coef_T, class data_T, class acc_T>
ostream& operator<<(ostream& o, const CFir<coef_T, data_T, acc_T> &f) {
    for (int i = 0; i < (sizeof(f.shift_reg)/sizeof(data_T)); i++) {
        o << shift_reg[ << i << ]= << f.shift_reg[i] << endl;
    }
    o << ----- << endl;
    return o;
}

data_t cpp_FIR(data_t x);
```

以下代码示例中显示的 C++ FIR 滤波器示例中的测试平台用于演示对顶层函数 `cpp_FIR` 进行调用和确认的方式。此示例突出显示了适合 Vitis HLS 综合的标准测试平台的部分重要属性：

- 根据已知有效值对输出结果进行检查。
- 如果结果确认正确，测试平台会返回 0。

```
#include "cpp_FIR.h"

int main() {
    ofstream result;
    data_t output;
    int retval=0;

    // Open a file to saves the results
    result.open(result.dat);

    // Apply stimuli, call the top-level function and saves the results
    for (int i = 0; i <= 250; i++)
    {
        output = cpp_FIR(i);
        result << setw(10) << i;
```

```

    result << setw(20) << output;
    result << endl;

}
result.close();

// Compare the results file with the golden results
retval = system(diff --brief -w result.dat result.golden.dat);
if (retval != 0) {
    printf(Test failed  !!!\n);
    retval=1;
} else {
    printf(Test passed !\n);
}

// Return 0 if the test
return retval;
}

```

适用于 `cpp_FIR` 的 C++ 测试平台

要将指令应用于类中定义的对象，请执行以下操作：

1. 打开定义类的文件（通常为头文件）。
2. 使用“Directives”（指令）选项卡应用指令。

就像函数一样，针对类的所有实例将应用相同的优化操作。

全局变量和类

AMD 不建议在类中使用全局变量。此类变量可能阻止执行某些优化。在以下代码示例中，类用于创建筛选工具组件（`polyd_cell` 类用作执行移位、乘法和累加运算的组件）。

```

typedef long long acc_t;
typedef int mult_t;
typedef char data_t;
typedef char coef_t;

#define TAPS 3
#define PHASES 4
#define DATA_SAMPLES 256
#define CELL_SAMPLES 12

// Use k on line 73 static int k;

template <typename T0, typename T1, typename T2, typename T3, int N>
class polyd_cell {
private:
public:
    T0 areg;
    T0 breg;
    T2 mreg;
    T1 preg;
    T0 shift[N];
    int k; //line 73
    T0 shift_output;
    void exec(T1 *pcout, T0 *dataOut, T1 pcin, T3 coeff, T0 data, int col)
    {
        Function_label0;;
    }
};

```

```

        if (col==0) {
SHIFT:for (k = N-1; k >= 0; --k) {
    if (k > 0)
        shift[k] = shift[k-1];
    else
        shift[k] = data;
    }
    *dataOut = shift_output;
    shift_output = shift[N-1];
    }
    *pcout = (shift[4*col]* coeff) + pcin;
    }
};

// Top-level function with class instantiated
void cpp_class_data (
    acc_t *dataOut,
    coef_t coeff1[PHASES][TAPS],
    coef_t  coeff2[PHASES][TAPS],
    data_t dataIn[DATA_SAMPLES],
    int row
) {

    acc_t pcin0 = 0;
    acc_t pcout0, pcout1;
    data_t dout0, dout1;
    int col;
    static acc_t accum=0;
    static int sample_count = 0;
    static polyd_cell<data_t, acc_t, mult_t, coef_t, CELL_SAMPLES>
polyd_cell0;
    static polyd_cell<data_t, acc_t, mult_t, coef_t, CELL_SAMPLES>
polyd_cell1;

    COL:for (col = 0; col <= TAPS-1; ++col) {

        polyd_cell0.exec(&pcout0,&dout0,pcin0,coeff1[row]
[col],dataIn[sample_count],
col);

        polyd_cell1.exec(&pcout1,&dout1,pcout0,coeff2[row][col],dout0,col);

        if ((row==0) && (col==2)) {
            *dataOut = accum;
            accum = pcout1;
        } else {
            accum = pcout1 + accum;
        }
    }
    sample_count++;
}

```


在 `polyd_cell` 类中包含用于数据移位的 `SHIFT` 循环。如果 `SHIFT` 循环中使用的循环索引 `k` 已被移除并替换为对应 `k` 的全局索引（如上示例所示，但带有 `static int k` 注释），则 Vitis HLS 无法对使用 `polyd_cell` 类的任何循环或函数进行流水打拍。Vitis HLS 将发出如下消息：

```
@W [XFORM-503] Cannot unroll loop 'SHIFT' in function 'polyd_cell<char,
long long,
int, char, 12>::exec' completely: variable loop bound.
```

使用局部非全局变量来建立循环索引即可确保 Vitis HLS 能执行所有优化操作。

模板

Vitis HLS 支持在 C++ 中使用模板进行综合。Vitis HLS 不支持对顶层函数使用模板。请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Modeling/using_C++_templates](#)，以获取这些概念的示例。



重要提示！ 不能使用模板作为顶层函数。

使用模板创建专用实例

针对模板实参的每个不同值将复制模板函数中的静态变量。

将不同 C++ 模板值传递给同一函数即可为每个模板值创建唯一的函数实例。Vitis HLS 会在这些副本自己的关联环境内对其单独进行综合。其优势主要体现在工具可以为每个唯一的实例提供特定的优化，从而为该函数生成简单明了的实现。

```
template<int NC, int K>
void startK(int* dout) {
    static int acc=0;
    acc += K;
    *dout = acc;
}

void foo(int* dout) {
    startK<0,1> (dout);
}

void goo(int* dout) {
    startK<1,1> (dout);
}

int main() {
    int dout0,dout1;
    for (int i=0;i<10;i++) {
        foo(&dout0);
        goo(&dout1);
        cout <<"dout0/1 = "<<dout0<<" / "<<dout1<<endl;
    }
    return 0;
}
```

使用模板执行递归

模板还可用于实现标准 C 语言综合中不支持的某种形式的递归（递归函数）。

以下代码示例显示了使用模板化 `struct` 来实现尾递归斐波那契算法。执行综合的关键是用于在递归中实现最终调用的终端类，其中使用的模板大小为 1。

```
//Tail recursive call
template<data_t N> struct fibon_s {
    template<typename T>
        static T fibon_f(T a, T b) {
            return fibon_s<N-1>::fibon_f(b, (a+b));
        }
};

// Termination condition
template<> struct fibon_s<1> {
    template<typename T>
        static T fibon_f(T a, T b) {
            return b;
        }
};

void cpp_template(data_t a, data_t b, data_t &dout){
    dout = fibon_s<FIB_N>::fibon_f(a,b);
}
```

枚举类型

以下代码示例中的头文件定义了一些 `enum` 类型，在 `struct` 中使用。随后在其他 `struct` 中使用了该 `struct`。这样可以直观描述要采集的复杂类型。

以下代码示例演示如何指定和综合复杂定义 (`MAD_NSBSAMPLES`) 语句。

```
#include <stdio.h>

enum mad_layer {
    MAD_LAYER_I    = 1,
    MAD_LAYER_II   = 2,
    MAD_LAYER_III  = 3
};

enum mad_mode {
    MAD_MODE_SINGLE_CHANNEL = 0,
    MAD_MODE_DUAL_CHANNEL  = 1,
    MAD_MODE_JOINT_STEREO  = 2,
    MAD_MODE_STEREO       = 3
};

enum mad_emphasis {
    MAD_EMPHASIS_NONE = 0,
    MAD_EMPHASIS_50_15_US = 1,
    MAD_EMPHASIS_CCITT_J_17 = 3
};

typedef    signed int mad_fixed_t;

typedef struct mad_header {
    enum mad_layer layer;
        enum mad_mode mode;
    int mode_extension;
    enum mad_emphasis emphasis;

    unsigned long long bitrate;
```

```

unsigned int samplerate;

unsigned short crc_check;
unsigned short crc_target;

int flags;
int private_bits;

} header_t;

typedef struct mad_frame {
    header_t header;
    int options;
    mad_fixed_t sbsample[2][36][32];
} frame_t;

# define MAD_NSBSAMPLES(header) \
    ((header)->layer == MAD_LAYER_I ? 12 : \
    (((header)->layer == MAD_LAYER_III && \
    ((header)->flags & 17)) ? 18 : 36))

void types_composite(frame_t *frame);

```

下例中使用了上例中定义的 `struct` 和 `enum` 类型。如果在顶层函数的实参中使用 `enum`，则会将枚举综合为 32 位值以符合标准 C/C++ 语言编译行为。如果枚举类型在设计内部，则 Vitis HLS 会将其优化到必需的最小位数。

以下代码示例显示在综合期间如何忽略 `printf` 语句。

```

#include "types_composite.h"

void types_composite(frame_t *frame)
{
    if (frame->header.mode != MAD_MODE_SINGLE_CHANNEL) {
        unsigned int ns, s, sb;
        mad_fixed_t left, right;

        ns = MAD_NSBSAMPLES(&frame->header);
        printf("Samples from header %d \n", ns);

        for (s = 0; s < ns; ++s) {
            for (sb = 0; sb < 32; ++sb) {
                left = frame->sbsample[0][s][sb];
                right = frame->sbsample[1][s][sb];
                frame->sbsample[0][s][sb] = (left + right) / 2;
            }
        }
        frame->header.mode = MAD_MODE_SINGLE_CHANNEL;
    }
}

```

联合体

在以下代码示例中，已创建含 `double` 和 `struct` 的联合体 (`union`)。不同于 C/C++ 编译，综合并不保证针对 `union` 中的所有字段使用相同存储器（对于综合，则不保证使用相同寄存器）。Vitis HLS 通过执行优化来提供最优硬件。

```
#include "types_union.h"

dout_t types_union(din_t N, dinfp_t F)
{
    union {
        struct {int a; int b; } intval;
        double fpval;
    } intfp;
    unsigned long long one, exp;

    // Set a floating-point value in union intfp
    intfp.fpval = F;

    // Slice out lower bits and add to shifted input
    one = intfp.intval.a;
    exp = (N & 0x7FF);

    return ((exp << 52) + one) & (0x7fffffffffffffffffLL);
}
```

Vitis HLS 不支持：

- 顶层函数接口上的联合体。
- 重新解读指针以供综合。因此，联合体不能保存不同类型的指针，也不能保存指向不同类型的数组的指针。
- 通过其他变量访问联合体。同样使用上述示例中的联合体，不支持：

```
for (int i = 0; i < 6; ++i)
if (i<3)
    A[i] = intfp.intval.a + B[i];
else
    A[i] = intfp.intval.b + B[i];
}
```

- 但可将其显式重编码为：

```
A[0] = intfp.intval.a + B[0];
A[1] = intfp.intval.a + B[1];
A[2] = intfp.intval.a + B[2];
A[3] = intfp.intval.b + B[3];
A[4] = intfp.intval.b + B[4];
A[5] = intfp.intval.b + B[5];
```

对联合体进行综合不支持原生 C/C++ 类型与用户定义的类型之间进行类型强制转换。

在 Vitis HLS 设计中，联合体常用于转换原始位元的数据类型。通常在顶层端口接口处使用浮点值时，需使用此原始位元转换。请参阅以下示例：

```
typedef float T;
unsigned int value; // the "input" of the conversion
T myhalfvalue; // the "output" of the conversion
union
{
```

```

    unsigned int as_uint32;
    T as_floatingpoint;
} my_converter;
my_converter.as_uint32 = value;
myhalfvalue = my_converter.as_floatingpoint;

```

这种类型的代码适用于 C/C++ 语言的浮点 (float) 数据类型，通过修改还可用于双精度 (double) 数据类型。将 `typedef` 和 `int` 更改为 `short` 对于 `half` 数据类型无效，但由于 `half` 属于类，因此在联合体中无法使用。可改用以下代码：

```

typedef half T;
short value;
T myhalfvalue = static_cast<T>(value);

```

同样，相反方式的转换使用的是 `value=static_cast<ap_uint<16>>(myhalfvalue)` 或 `static_cast<unsigned short>(myhalfvalue)`。

```

ap_fixed<16,4> afix = 1.5;
ap_fixed<20,6> bfix = 1.25;
half ahlh = afix.to_half();
half bhlh = bfix.to_half();

```

另一种方法是使用帮助程序类 `fp_struct<half>` 来通过 `data()` 或 `to_int()` 方法进行转换。使用 `hls/utils/x_hls_utils.h` 头文件。

类型限定符

类型 (type) 限定符可直接影响高层次综合创建的硬件。一般情况下，此类限定符会以可预测的方式来影响综合结果，如下所述。Vitis HLS 仅受限符解读的限制，因为它会影响功能行为，并且可执行优化以创建更优化的硬件设计。在每个限定符概述后提供了此行为的示例。

Volatile

在函数接口上多次访问指针时，`volatile` 限定符会影响 RTL 中执行的读取或写入操作次数。虽然 `volatile` 限定符会影响层级内所有函数中的此行为，但 `volatile` 限定符的影响主要常见于顶层接口中，如需了解详细信息，请参阅 [顶层接口](#) 章节。

注释：对往来 `volatile` 变量的访问权限均为保留权限。这意味着：

- 无突发访问
- 无端口拓宽
- 无死码消除

提示：任意精度类型不支持使用易变 (`volatile`) 限定符执行算术运算。对于使用 `volatile` 限定符的所有任意精度数据类型，必须将其指定为非易变数据类型，才可在算术表达式中使用。

静态

函数中的静态 (`static`) 类型在函数调用之间保留其值。硬件设计中的等效行为是寄存变量（触发器或存储器）。如果 C/C++ 语言函数要求变量为静态类型才能正确执行，则在最终的 RTL 设计中它必然属于寄存器。该值必须在函数和设计的调用之间保持不变。

综合后并不是只有 `static` 类型会实现为寄存器。Vitis HLS 会判定在 RTL 设计中需要将哪些变量实现为寄存器。例如，如果必须在多个周期内保持变量赋值不变，则即使 C/C++ 语言函数中的原始变量不是静态类型，Vitis HLS 也会创建一个寄存器来保存该值。

Vitis HLS 会遵循静态函数的初始化行为，并在初始化期间对寄存器赋值 0（或任何显式初始化的值）。这意味着 `static` 变量在 RTL 代码和 FPGA 比特流中初始化。但这并不意味着每次复位信号时都会重新初始化该变量。

请参阅 [RTL 配置](#) 以确定有关系统复位的静态初始化值的实现方式。

Const

`const` 类型用于指定变量值从不更新。该变量仅供读取，而无法写入，因此必须初始化。这通常意味着在 RTL 设计中，大部分 `const` 变量都会减缩为常量。Vitis HLS 会执行常量传输并移除任何不必要的硬件。

对于阵列，`const` 变量在最终 RTL 设计中作为 ROM 来实现（Vitis HLS 不会像对小型阵列一样对此类变量执行自动分区）。使用 `const` 限定符指定的阵列与静态限定符一样在 RTL 和 FPGA 比特流中进行初始化。无需将其复位，因为从不对其执行写入。

ROM 优化

以下显示的代码示例中，即使未以 `static` 或 `const` 限定符指定阵列，Vitis HLS 仍实现为 ROM。其中演示了 Vitis HLS 分析设计和判定最优实现的方式。限定符用于为工具提供指导，但并不支配最终 RTL。

```
#include "array_ROM.h"

dout_t array_ROM(din1_t inval, din2_t idx)
{
    din1_t lookup_table[256];
    dint_t i;

    for (i = 0; i < 256; i++) {
        lookup_table[i] = 256 * (i - 128);
    }

    return (dout_t)inval * (dout_t)lookup_table[idx];
}
```

在此示例中，该工具能够判定采用变量 `lookup_table` 作为最终 RTL 中的存储器元素的情况下，实现结果最佳。

任意精度 (AP) 数据类型

基于 C/C++ 语言的原生数据类型都位于 8 位边界（8 位、16 位、32 位和 64 位）上。但是，RTL 总线（对应于硬件）支持任意数据长度。使用标准 C/C++ 语言数据类型可能导致硬件实现效率低下。例如，AMD 器件中的基本乘法单位是 DSP 库单元。32 位“ints”整数乘法需要多个 DSP 单元，而使用任意精度类型则可以每次乘法仅使用一个单元。

任意精度 (AP) 数据类型允许您的代码使用位宽更小的变量，并且可供 C/C++ 语言仿真用于确认功能保持不变或者可接受。较小的位宽所生成的硬件运算符也更小且运行更快。这样即可允许在 FPGA 中布局更多逻辑，并允许以更高的时钟频率来执行逻辑。

AP 数据类型是专为 C++ 提供的，支持您对 1 位到 1024 位之间的任意位宽的数据类型进行建模。您必须将 AP 库包含在自己的 C++ 源代码中才能指定使用这些 AP 库，如 [任意精度数据类型库](#) 中所述。



提示：任意精度类型通常仅在函数边界处才需要，因为 Vitis HLS 会优化内部逻辑并移除不扇出到输出端口的数据位和逻辑。

AP 示例

例如，含用于通信协议的筛选函数的设计需要 10 位输入数据和 18 位输出数据以满足数据传输需求。使用标准 C/C++ 语言数据类型时，输入数据必须至少为 16 位，输出数据必须至少为 32 位。在最终硬件中，这会在输入和输出之间创建过宽的数据路径、使用过多的资源、延迟过长（例如，32 位乘 32 位的乘法耗时比 18 位乘 18 位的乘法更长），并且需要更多时钟周期才能完成。

在此设计中使用任意精度数据类型时，可以在综合前指定代码中所需的精确位数、对更新后的代码进行仿真，并在综合前验证结果。请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Modeling](#)，以获取有关使用任意精度和定点 AP 数据类型的示例。

AP 数据类型的优势



重要提示！ AP 数据类型的劣势之一是阵列不会以 0 值进行自动初始化。如果需要初始化阵列，必须手动执行。

以下代码用于执行部分基本算术运算：

```
#include "types.h"

void apint_arith(dinA_t inA, dinB_t inB, dinC_t inC, dinD_t inD,
                dout1_t *out1, dout2_t *out2, dout3_t *out3, dout4_t *out4
                ) {

    // Basic arithmetic operations
    *out1 = inA * inB;
    *out2 = inB + inA;
    *out3 = inC / inA;
    *out4 = inD % inA;
}
```

数据类型 `dinA_t`、`dinB_t` 等在 `types.h` 头文件中定义。强烈建议使用适用于全工程的头文件（如 `types.h`），这样可便于从标准 C/C++ 语言类型移植到任意精度类型，并且有助于将任意精度类型优化为最优大小。

如果以上示例中的数据类型定义为：

```
typedef char dinA_t;
typedef short dinB_t;
typedef int dinC_t;
typedef long long dinD_t;
typedef int dout1_t;
typedef unsigned int dout2_t;
typedef int32_t dout3_t;
typedef int64_t dout4_t;
```

综合后，此设计结果如下：

```
+ Timing (ns):
  * Summary:
  +-----+-----+-----+-----+
  | Clock | Target | Estimated | Uncertainty |
  +-----+-----+-----+-----+
  | default | 4.00 | 3.85 | 0.50 |
  +-----+-----+-----+-----+
```

```
+ Latency (clock cycles):
  * Summary:
    +-----+-----+-----+-----+
    | Latency | Interval | Pipeline |
    | min | max | min | max | Type |
    +-----+-----+-----+-----+
    | 66 | 66 | 67 | 67 | none |
    +-----+-----+-----+-----+
  * Summary:
    +-----+-----+-----+-----+
    | Name | BRAM_18K | DSP48E | FF | LUT |
    +-----+-----+-----+-----+
    | Expression | - | - | 0 | 17 |
    | FIFO | - | - | - | - |
    | Instance | - | 1 | 17920 | 17152 |
    | Memory | - | - | - | - |
    | Multiplexer | - | - | - | - |
    | Register | - | - | 7 | - |
    +-----+-----+-----+-----+
    | Total | 0 | 1 | 17927 | 17169 |
    +-----+-----+-----+-----+
    | Available | 650 | 600 | 202800 | 101400 |
    +-----+-----+-----+-----+
    | Utilization (%) | 0 | ~0 | 8 | 16 |
    +-----+-----+-----+-----+
```

如果使用标准 C/C++ 语言类型实现的设计并无数据宽度要求，但如果其宽度比标准 C 类型小、而又比下一个最小的标准 C 语言类型更大，如下所示：

```
typedef int6 dinA_t;
typedef int12 dinB_t;
typedef int22 dinC_t;
typedef int33 dinD_t;
typedef int18 dout1_t;
typedef uint13 dout2_t;
typedef int22 dout3_t;
typedef int6 dout4_t;
```

综合后的结果会显示最大时钟频率有所改善、时延改善，且面积使用率显著降低达 75%。

```
+ Timing (ns):
  * Summary:
    +-----+-----+-----+-----+
    | Clock | Target | Estimated | Uncertainty |
    +-----+-----+-----+-----+
    | default | 4.00 | 3.49 | 0.50 |
    +-----+-----+-----+-----+

+ Latency (clock cycles):
  * Summary:
    +-----+-----+-----+-----+
    | Latency | Interval | Pipeline |
    | min | max | min | max | Type |
    +-----+-----+-----+-----+
    | 35 | 35 | 36 | 36 | none |
    +-----+-----+-----+-----+
  * Summary:
    +-----+-----+-----+-----+
    | Name | BRAM_18K | DSP48E | FF | LUT |
    +-----+-----+-----+-----+
    | Expression | - | - | 0 | 13 |
```


FIFO		-		-		-		-	
Instance		-		1		4764		4560	
Memory		-		-		-		-	
Multiplexer		-		-		-		-	
Register		-		-		6		-	
+-----+-----+-----+-----+-----+									
Total		0		1		4770		4573	
+-----+-----+-----+-----+-----+									
Available		650		600		202800		101400	
+-----+-----+-----+-----+-----+									
Utilization (%)		0		~0		2		4	
+-----+-----+-----+-----+-----+									

这两种设计之间存在的巨大时延差异原因在于除法和余数运算耗费多个周期才完成。使用 AP 数据类型来代替将设计与标准 C/C++ 语言数据类型强制匹配，可以提高硬件实现的质量：准确性不变，但性能更好、耗用资源少。

任意精度整数数据类型概述

Vitis HLS 可以为 C++ 提供整数数据类型和定点任意精度数据类型。

表 5：任意精度数据类型

语言	整数数据类型	所需头文件
C++	ap_[u]int<W> (1024 位) 位宽可扩展至 4K 位，如下所述。	#include "ap_int.h"
C++	ap_[u]fixed<W,I,Q,O,N>	#include "ap_fixed.h"

对于 C++ 语言 ap_[u]int 数据类型，头文件 ap_int.h 用于定义任意精度整数数据类型。要在 C++ 函数中使用任意精度整数数据类型，请执行以下操作：

- 将头文件 ap_int.h 添加到源代码。
- 将位类型更改为 ap_int<N> 或 ap_uint<N>，其中 N 是介于 1 到 1024 之间的位大小。

以下示例显示了如何添加头文件并实现 2 个变量来使用 9 位整数和 10 位无符号的整数类型：

```
#include "ap_int.h"

void foo_top () {

    ap_int<9>    var1;           // 9-bit
    ap_uint<10>  var2;           // 10-bit unsigned
```

针对 ap_[u]int 数据类型允许的默认最大位宽为 1024 位。此默认设置可通过在包含 ap_int.h 头文件之前定义含正整数（小于或等于 4096）的 AP_INT_MAX_W 宏来覆盖。

**重要提示！** AP_INT_MAX_W 的值设置过高可能导致软件编译缓慢、运行时间延长。

以下是覆盖 AP_INT_MAX_W 的示例：

```
#define AP_INT_MAX_W 4096 // Must be defined before next line
#include "ap_int.h"

ap_int<4096> very_wide_var;
```

任意精度定点数据类型概述

定点数据类型以 2 的补码形式将数据作为整数位和小数位进行建模，格式为 `ap_fixed<W,I,[Q,O,N]>`，如下表所述。在以下示例中，Vitis HLS `ap_fixed` 类型用于定义 18 位变量，其中 6 位（包括符号位）指定为表示二进制小数点前的数值，12 位则隐含用于表示小数点后的小数值。此变量指定为有符号变量，量化模式设置为舍入到正无穷。由于未指定上溢模式，因此针对上溢使用默认卷绕模式。

```
#include <ap_fixed.h>
...
ap_fixed<18,6,AP_RND> t1 = 1.5; // internally represented as
0b00'0001.1000'0000'0000 (0x01800)
ap_fixed<18,6,AP_RND> t2 = -1.5; // 0b11'1110.1000'0000'0000 (0x3e800)
...
```



提示： `ap_fixed` 的整数值 (I) 指定了二进制小数点左侧的整数位数（包括符号位）。

执行计算时，如果其中所含变量具有不同位数或不同精度，那么二进制小数点将自动对齐。例如，以不同大小的定点类型变量执行除法时，商的小数部分不大于被除数的小数部分。为了保留商的小数部分，可在赋值前先将结果强制转换为新的变量宽度。

使用定点执行的 C++ 语言仿真行为与生成的硬件相匹配。这使您能够使用 C 语言层次快速仿真来分析位精度、量化和上溢行为。

定点类型适合替代需要多个时钟周期才能完成的浮点类型。除非需要浮点类型的完整范围，否则通常使用定点类型即可实现相同精度，并且硬件更小也更快。

下表中提供了 `ap_fixed` 类型的标识符的汇总。

表 6：定点标识符汇总

标识符	描述	
W	字宽（位）	
I	用于表示整数值的位数，即，二进制小数点左侧的整数位数，包括符号位。 当 I 为负值（如以下示例所示）时，它表示隐式有符号位（对应有符号表示法）的数量，或者小数点右侧的隐式零位（对应无符号表示法）的数量。例如：	
	<pre>ap_fixed<2, 0> a = -0.5; // a can be -0.5, ap_ufixed<1, 0> x = 0.5; // 1-bit representation. x can be 0 or 0.5 ap_ufixed<1, -1> y = 0.25; // 1-bit representation. y can be 0 or 0.25 const ap_fixed<1, -7> z = 1.0/256; // 1-bit representation for z = 2^-8</pre>	
Q	量化模式：用于规定当生成的精度大于用于存储结果的变量中的最小小数位可定义的精度时的行为。	
	ap_fixed 类型	描述
	AP_RND	舍入到正无穷
	AP_RND_ZERO	舍入到 0
	AP_RND_MIN_INF	舍入到负无穷
	AP_RND_INF	舍入到无穷
	AP_RND_CONV	收敛舍入
	AP_TRN	截位到负无穷（默认）
	AP_TRN_ZERO	截位到 0

表 6：定点标识符汇总 (续)

标识符	描述	
O	上溢模式：用于规定当运算结果超出用于存储结果的变量可存储的最大值（或者对于负值为最小值）时的行为。	
	ap_fixed 类型	描述
	AP_SAT ¹	饱和
	AP_SAT_ZERO ¹	饱和到 0
	AP_SAT_SYM ¹	对称饱和
	AP_WRAP	卷绕（默认）
	AP_WRAP_SM	符号量值卷绕
N	定义上溢卷绕模式中的饱和位数。	

注释：

1. 使用 AP_SAT* 模式可以提升资源使用率，因为需要额外逻辑来执行饱和，此额外成本可能额外增加高达 20% 的 LUT 使用率。
2. 来自 hls_math 库的定点数学函数不支持 ap_[u]fixed 模板参数 Q、O 和 N，这三个参数依次分别对应于量化模式、上溢模式和饱和位数。仅当 ap_[u]fixed 变量位于赋值左侧时或者在初始化期间，量化模式和上溢模式才有效，但在变量计算期间无效。

针对 ap_[u]fixed 数据类型允许的默认最大位宽为 1024 位。您可通过在包含 ap_int.h 头文件之前定义 AP_INT_MAX_W 宏为小于或等于 4096 的正整数来覆盖此默认设置。



重要提示！ 使用 ap_[u]fixed 时，ROM 综合耗时较长。将其更改为 int 可加速综合。例如：

```
static ap_fixed<32,0> a[32][depth] =
```

可更改为：

```
static int a[32][depth] =
```

编译器支持将 std::numeric_limit 用于 ap_(u)int 和 ap_(u)fixed。使用示例：

```
using data_u_t = ap_uint<5>;
data_u_t max = std::numeric_limits<data_u_t>::max();

using data_u_t = ap_ufixed<5, 3>;
data_u_t min = std::numeric_limits<data_u_t>::min();
```

任意精度浮点库 ap_float<int W, int E>

新增的任意精度浮点库提供了额外的灵活性，使设计师可以探索哪种精度最适合其设计，并在资源成本和性能与相对于标准浮点格式的精度之间进行权衡。

任意精度浮点库可以匹配其他浮点格式，如 bfloat16、tf32 等。

所需头文件为 ap_float.h，必须使用 #include <ap_float.h> 添加到源代码中。此库是作为模板化的库 ap_float<int W, int E> 来提供的，其中 W 是变量的总位宽，E 是指数的位宽。其余内部位字段与 IEEE 754 浮点标准相同，有 1 位符号，其余 W-E-1 位则用于保存有效位的小数位。有效位包含 W-E 位：隐式位设为“1”，后接 W-E-1 小数位。

使用 `ap_float<>` 库时，标准 32 位 IEEE 754 浮点数类型为 `ap_float<32,8>`；在下表给出了其他常见类型。

表 7: C++ 常用浮点数据类型与 `ap_float<>` 的对应关系

C++ 数据类型	<code>ap_float<W,E></code> 等效类型
<code>float</code>	<code>ap_float<32,8></code>
<code>double</code>	<code>ap_float<64,11></code>
<code>half</code>	<code>ap_float<16,5></code>
<code>bfloat16</code>	<code>ap_float<16,8></code>
<code>tf32 (TensorFloat32)</code>	<code>ap_float<19,8></code>

注释：由于编译器原生支持标准 C++ 浮点和双精度浮点数据类型，因此无需更改设计。

由于 `ap_float<>` 库在生成 RTL 时使用 AMD Floating-Point LogiCORE IP（请参阅《Floating-Point Operator LogiCORE IP 产品指南》(PG060)），因此，该 IP 衍生出的以下限制在此处也同样适用：

1. 总位宽 W 上限为 80 位。
2. 指数位宽 E 范围为从 4 位到 16 位。
3. 尾数位宽从 4 位到 64 位不等。
4. 不支持次正态：次正态结果和实参舍入为 0。
5. 默认且唯一受支持的舍入模式为“Round to Nearest ties to Even”（向最近偶数舍入）。
6. 所有 NaN 均为“Quiet NaN”（安静 NaN），不支持“Signaling NaN”（信号 NaN）。

不支持次正态，因此有效位的隐含位始终设置为 1。

访问原始比特位

`ap_float::bits_ref` API 会返回底层原始比特位作为 `ap_int` 引用。

```
ap_int<W>& ap_float<W, E>::bits_ref();
It can be used as follow:
// Next representable floating point value
ap_float<16, 8> next(ap_float<16, 8> f) {
    ap_float<16, 8> res;
    res.bits_ref() = f.bits_ref() + 1;
    return res;
}
```

此操作是对 `sign_ref/exponent_ref/mantissa_ref` 的补充，后者分别提供了对应于 `sign/exponent/mantissa` 的底层存储子范围。

通过访问整个底层存储，就能更方便地实现序列化和反序列化，以及 ULP 距离计算。

支持的器件

支持所有 Versal 器件和 UltraScale+ FPGA 器件。

受支持的函数

支持所有常用的 C++ 浮点算术运算符：

- 二元 +、-、* 和 /

- 一元 +=、-=、*= 和 /=
- 比较运算符
- 特定 HLS 函数，如 `hls::sqrt`、`hls::fma` 和 `hls::abs`
- 累加是使用运算符 += 或使用特定 `ap_float_acc<>` 帮助程序库来予以支持的，如下所述。

不执行从 `ap_float<>` 到 `float` 的自动转换，因此其他函数将导致编译错误。这是一种设计选择，如下所述。

此处提供了一个简单示例：

```
#include <ap_float.h>

typedef ap_float<18, 6> my_float;

float apf_simple_example( float f_input) {
    #pragma HLS pipeline
    my_float pi    = M_PI;

    // Implicit conversion from float to ap_float<> is allowed
    my_float r     = f_input;

    my_float area = pi*r*r;
    if ( area>1.0f ) { area=1.0f; }

    // Explicit conversion from ap_float<>
    // Otherwise => error: no viable conversion from 'ap_float<18, 6>' to
    'float'
    float ret_value=(float)area;

    return ret_value;
}
```

处理不支持的函数

不存在从 `ap_float<>` 到任何 `float` 数据类型的自动转换：这次该库的设计决策，用于强制设计师在执行类型转换时思考所需的精度和关联的成本。

因此，只要在库中使用含有任何不受支持的函数的 `ap_float<>`，都将触发编译错误，如下示例所示。

```
ap_float<16,8> k;
... = hls::log(k); // log is not expecting ap_float<>
// will give
ERROR: [HLS 207-3339] no matching function for 'log'
INFO: [HLS 207-4378] candidate function not viable: no known conversion
from 'ap_float<16, 8>' for 1st argument
```

面对此示例时，设计可将 `ap_float<>` 转换为以下任意 `float`、`ap_fixed<>` 或 `ap_int<>/int` 数据类型：

```
... = hls::log((float)k);           // using float
... = hls::log((ap_fixed<20,5>)k); // using ap_fixed
... = hls::log((int)k);             // using int or ap_int
```

在 `ap_float<>` 与其他数据类型之间进行双向转换

在某个 `ap_float<>` 与另一个不同精度（即，W 或 E 模板参数不同）的 `ap_float<>` 之间进行转换时，将会为舍入和/或饱和逻辑例化一个 Floating-point IP。仅当参数相同时，或者对于 C 浮点数据类型，仅当使用等效参数时，才能自由转换，例如，在 `float` 与 `ap_float<32,8>` 之间进行双向转换。

在某个 `ap_float<>` 与整数类型或 `ap_fixed<>` 之间进行转换时，将会为量化例化一个 Floating-point IP。

使用 `ap_float_acc<>` 帮助程序类对 `ap_float<>` 进行累加

提供帮助程序类 `ap_float_acc<>` 的目的是使用 `ap_fixed<>` 整数底层数据类型执行累加，以便为累加循环启用 `II=1`。类声明的伪代码如下所示。

```
template <...>
class ap_float_acc< ap_float<W, E>, ap_fixed<WFX, IFX> > {
    ap_float<W, E> accumulate(const ap_float<W, E> &value, bool last);
}
```

第一个模板实参定义要累加的值的输入数据类型 `ap_float<W,E>`。

第二个模板实参定义执行实际累加的累加器的底层数据类型 `ap_fixed<WFX, IFX>`。

成员函数 `accumulate(value,last)` 用于将输入“value”累加到其内部累加器中，输入“last”用于在最后一次累加后将内部累加器清零。返回值为累加器值。

```
using apf_t = ap_float<18, 5>; // using is a C++ syntax
using acc_datatype_t = ap_fixed<32, 16>;

... my_function(...) {
    ap_float_acc<apf_t, acc_datatype_t> acc; // instance of accumulator
    class

    apf_t res;
    for (unsigned int i = 0; i < n; i++) {
        apf_t value = ...; // value to accumulate
        res=acc.accumulate( /*input:ap_float*/ value, /*last:bool*/ i == n-1);
    }
}
```

使用 `ap_float_acc<>` 时的限制

`ap_float_acc<>` 实例阵列必须使用 `array_partition` 编译指示加以完全分区。

`ap_float_acc<>` 帮助程序类不支持跨层级，并且无法复制：

- 在将使用累加成员函数的函数内声明 `ap_float_acc` 实例，这也适用于数据流进程。
- 如果在流水打拍循环中使用累加函数，则只能在该循环中使用该函数（因为编译器会将流水打拍循环抽取到其自己的函数中）。

全局变量

全局变量可在代码中自由使用并且完全可综合。但不允许将全局变量推断为顶层函数的实参，必须改为将其显式指定为 RTL 设计中端口的实参。

以下代码示例显示了全局变量的默认综合行为。其中使用了 3 个全局变量：`idx`、`Ain` 和 `Aout`。虽然此示例使用阵列，但 Vitis HLS 支持所有类型的全局变量。

- 从 `Ain` 阵列读取值。
- `Aint` 阵列用于将值从 `Ain` 变换并传输到 `Aout`。
- 输出写入 `Aout` 阵列。



重要提示！ 在实参列表中，必须显式列出针对全局变量 `Ain` 和 `Aout` 的访问权。

```
#include "top.h"

void top(const int idx, const int Ain[N], int Aout[Nhalf]) {

    int Aint[N];

    // Move elements in the input array
    ILOOP: for (int i = 0; i < N; i++) {

        int iadj = (i + idx) % N;

        Aint[i] = Ain[i] + Ain[iadj];

    } // end ILOOP

    // sum the 1st and 2nd halves
    OLOOP: for (int i = 0; i < Nhalf; i++) {

        Aout[i] = (Aint[i] + Aint[Nhalf + i]);

    } // end OLOOP

} // end top()
```

指针

指针在 C/C++ 语言代码中使用广泛，也支持用于综合，但通常建议避免在代码中使用指针。在以下情况下使用指针时尤其如此：

- 在同一函数内多次访问（读取或写入）指针时。
- 使用指针阵列时，每个指针都必须指向 1 个标量或标量阵列（而不是另一指针）。
- 仅支持标准 C/C++ 语言类型间的指针强制转换，如下所示。

注释：不支持指针到指针。

以下代码示例显示支持针对指向多个对象的指针执行综合。

```
#include "pointer_multi.h"

dout_t pointer_multi (sel_t sel, din_t pos) {
    static const dout_t a[8] = {1, 2, 3, 4, 5, 6, 7, 8};
    static const dout_t b[8] = {8, 7, 6, 5, 4, 3, 2, 1};

    dout_t* ptr;
    if (sel)
        ptr = a;
    else
        ptr = b;

    return ptr[pos];
}
```

Vitis HLS 支持指针到指针类型的综合，但不支持在顶层接口上执行，即不支持将其作为顶层函数的实参来进行综合。如果在多个函数中使用指针到指针，那么 Vitis HLS 会将使用指针到指针的所有函数内联。内联多个函数可能导致运行时间增加。

```
#include "pointer_double.h"

data_t sub(data_t ptr[10], data_t size, data_t**flagPtr)
{
    data_t x, i;

    x = 0;
    // Sum x if AND of local index and pointer to pointer index is true
    for(i=0; i<size; ++i)
        if (**flagPtr & i)
            x += *(ptr+i);
    return x;
}

data_t pointer_double(data_t pos, data_t x, data_t* flag)
{
    data_t array[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    data_t* ptrFlag;
    data_t i;

    ptrFlag = flag;

    // Write x into index position pos
    if (pos >=0 & pos < 10)
        *(array+pos) = x;

    // Pass same index (as pos) as pointer to another function
    return sub(array, 10, &ptrFlag);
}
```

指针阵列同样可进行综合。请参阅以下代码示例，其中使用指针阵列来存储全局阵列的第二维度的起始位置。指针阵列中的指针只能指向标量或标量阵列，不能指向其他指针。

```
#include "pointer_array.h"

data_t A[N][10];

data_t pointer_array(data_t B[N*10]) {
    data_t i,j;
    data_t sum1;
```



```
// Array of pointers
data_t* PtrA[N];

// Store global array locations in temp pointer array
for (i=0; i<N; ++i)
    PtrA[i] = &(A[i][0]);

// Copy input array using pointers
for(i=0; i<N; ++i)
    for(j=0; j<10; ++j)
        *(PtrA[i]+j) = B[i*10 + j];

// Sum input array
sum1 = 0;
for(i=0; i<N; ++i)
    for(j=0; j<10; ++j)
        sum1 += *(PtrA[i] + j);

return sum1;
}
```

如果使用原生 C/C++ 语言类型，则支持对指针强制转换进行综合。在以下代码示例中，`int` 类型将强制转换为 `char` 类型。

```
#define N 1024

typedef int data_t;
typedef char dint_t;

data_t pointer_cast_native (data_t index, data_t A[N]) {
    dint_t* ptr;
    data_t i =0, result = 0;
    ptr = (dint_t*)&A[index];

    // Sum from the indexed value as a different type
    for (i = 0; i < 4*(N/10); ++i) {
        result += *ptr;
        ptr+=1;
    }
    return result;
}
```

Vitis HLS 不支持一般类型间的指针强制转换。例如，如果创建 `struct` 复合类型的有符号值，那么无法通过对该指针进行强制转换来赋值无符号值。

```
struct {
    short first;
    short second;
} pair;

// Not supported for synthesis
*(unsigned*)&pair = -1U;
```

在此类情况下，必须使用原生类型对这些值进行赋值。

```
struct {
    short first;
    short second;
} pair;

// Assigned value
pair.first = -1U;
pair.second = -1U;
```

接口上的指针

指针可作为顶层函数的实参。了解综合期间指针的实现方式至关重要，因为有时指针可能在综合后实现期望的 RTL 接口与设计时引发问题。请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Modeling/Pointers](#)，以获取以下部分概念的示例。

基本指针

如下代码示例所示，顶层接口上具有基本指针的函数不会对 Vitis HLS 产生影响。指针可综合为简单的有线接口或使用握手的接口协议。



提示： 仅限只读或只写指针才能综合为 FIFO 接口。

```
#include "pointer_basic.h"

void pointer_basic (dio_t *d) {
    static dio_t acc = 0;

    acc += *d;
    *d = acc;
}
```

每个函数调用仅对接口上的指针执行一次读取或写入。以下代码示例中显示了测试平台。

```
#include "pointer_basic.h"

int main () {
    dio_t d;
    int i, retval=0;
    FILE *fp;

    // Save the results to a file
    fp=fopen(result.dat,w);
    printf( "Din Dout\n", i, d);

    // Create input data
    // Call the function to operate on the data
    for (i=0;i<4;i++) {
        d = i;
        pointer_basic(&d);
        fprintf(fp, %d \n, d);
        printf( %d %d\n, i, d);
    }
    fclose(fp);

    // Compare the results file with the golden results
```

```

retval = system(diff --brief -w result.dat result.golden.dat);
if (retval != 0) {
    printf(Test failed!!!\n);
    retval=1;
} else {
    printf(Test passed!\n);
}

// Return 0 if the test
return retval;
}

```

C 和 RTL 仿真通过以下简单数据集验证运算是否正确（尽管并未列出所有可能的情况）：

```

Din Dout
0      0
1      1
2      3
3      6
Test passed!

```

指针算术

引入指针算术会限制 RTL 中可综合的接口。以下代码示例显示了相同的代码，但在此实例中使用简单的指针算术来累加数据值（从第 2 个值开始）。

```

#include "pointer_arith.h"

void pointer_arith (dio_t *d) {
    static int acc = 0;
    int i;

    for (i=0;i<4;i++) {
        acc += *(d+i+1);
        *(d+i) = acc;
    }
}

```

以下代码示例显示了支持此示例的测试平台。由于执行累加的循环当前位于 `pointer_arith` 函数内部，测试平台会填充含相应值的 `d[5]` 阵列所指定的地址空间。

```

#include "pointer_arith.h"

int main () {
    dio_t d[5], ref[5];
    int i, retval=0;
    FILE *fp;

    // Create input data
    for (i=0;i<5;i++) {
        d[i] = i;
        ref[i] = i;
    }

    // Call the function to operate on the data
    pointer_arith(d);

    // Save the results to a file
    fp=fopen(result.dat,w);
    printf( Din Dout\n, i, d);
}

```

```

for (i=0;i<4;i++) {
    fprintf(fp, %d \n, d[i]);
    printf(  %d    %d\n, ref[i], d[i]);
}
fclose(fp);

// Compare the results file with the golden results
retval = system(diff --brief -w result.dat result.golden.dat);
if (retval != 0) {
    printf(Test failed!!!\n);
    retval=1;
} else {
    printf(Test passed!\n);
}

// Return 0 if the test
return retval;
}

```

仿真后，会生成如下输出：

```

Din Dout
0    1
1    3
2    6
3   10
Test passed!

```

指针算术能无序访问指针数据。另一方面，连线、握手或 FIFO 接口只能按顺序访问数据：

- 当设计准备好使用数据或写入数据（当数据就绪后）时，连线接口会读取数据。
- 当控制信号允许继续操作时，握手和 FIFO 接口会执行读写操作。

在上述 2 种情况下，数据都必须按顺序到达（并写入），从元素 0 开始。在“含指针算术的接口”示例中，代码从索引 1 开始读取（*i* 从 0 开始，0+1=1）。这是来自测试平台中的 `d[5]` 数组的第 2 个元素。

在硬件中实现此示例后，需要某种形式的数据索引。Vitis HLS 不支持对连线、握手或 FIFO 接口建立数据索引。

或者，必须修改此代码，在接口上使用数组代替指针，如以下示例所示。在综合中可通过 RAM (`ap_memory`) 接口来实现此代码。此接口可使用地址来建立数据索引，并执行乱序或无序访问。

连线、握手或 FIFO 接口只能用于串流数据。除非它从 0 位数据开始建立数据索引，然后按顺序进行操作，否则不能搭配指针算术使用。

```

#include "array_arith.h"

void array_arith (dio_t d[5]) {
    static int acc = 0;
    int i;

    for (i=0;i<4;i++) {
        acc += d[i+1];
        d[i] = acc;
    }
}

```

接口上的多重访问指针



重要提示! 虽然在指针上支持多重访问指针，但强烈建议您使用 `hls::stream` 类替代多重访问指针来实现所需的行为，以避免下述部分困难。如需了解 `hls::stream` 类的详细信息，请参阅 [HLS 串流库](#)。

如果设计使用接口上的顶层函数的实参列表中的指针，那么在使用指针执行多重访问时需考量一些特殊注意事项。当任一指针以相同方式多次执行读取或写入时，就会发生多重访问。

使用多次访问的指针可能会在综合后引发意外行为。在以下错误示例中，对指针 `d_i` 执行了 4 次读取，对指针 `d_o` 执行了 2 次写入：指针执行了多次访问。

```
#include "pointer_stream_bad.h"

void pointer_stream_bad ( dout_t *d_o,  din_t *d_i) {
    din_t acc = 0;

    acc += *d_i;
    acc += *d_i;
    *d_o = acc;
    acc += *d_i;
    acc += *d_i;
    *d_o = acc;
}
```

综合后，此代码产生的 RTL 设计将读取 1 次输入端口，写入 1 次输出端口。与任何标准 C/C++ 语言编译器一样，Vitis HLS 将优化掉多余的指针访问。用于验证此设计的测试平台如下代码示例所示：

```
#include "pointer_stream_bad.h"
int main () {
    din_t d_i;
    dout_t d_o;
    int retval=0;
    FILE *fp;

    // Open a file for the output results
    fp=fopen(result.dat,w);

    // Call the function to operate on the data
    for (d_i=0;d_i<4;d_i++) {
        pointer_stream_bad(&d_o,&d_i);
        fprintf(fp, %d %d\n, d_i, d_o);
    }
    fclose(fp);

    // Compare the results file with the golden results
    retval = system(diff --brief -w result.dat result.golden.dat);
    if (retval != 0) {
        printf(Test failed !!!\n);
        retval=1;
    } else {
        printf(Test passed !\n);
    }

    // Return 0 if the test
    return retval;
}
```

要实现上述代码，即对 `d_i` 读取 4 次，对 `d_o` 写入 2 次，必须将指针指定为 `volatile`，如以下“`pointer_stream_better`”示例所示。

```
#include "pointer_stream_better.h"

void pointer_stream_better ( volatile dout_t *d_o,  volatile din_t *d_i) {
    din_t acc = 0;

    acc += *d_i;
    acc += *d_i;
    *d_o = acc;
    acc += *d_i;
    acc += *d_i;
    *d_o = acc;
}
```

为支持接口上的多重访问指针，应执行以下步骤：

- 执行综合前，确认 C/C++ 语言代码的意图以及 C/C++ 模型是否正确。
- 如果在 Vitis HLS 中使用协同仿真来验证 RTL，则指针实参所含数值必须与端口接口上指定的访问次数相同。

理解易变数据

[接口上的多重访问指针](#) 中的代码编写时的意图是：在 RTL 中将输入指针 `d_i` 和输出指针 `d_o` 作为 FIFO（或握手）接口来实现，以确保：

- 每次在 RTL 端口 `d_i` 上执行读取时，上游生产者模块都可提供新数据。
- 每次写入 RTL 端口 `d_o` 时，下游使用者模块都接受新数据。

通过标准 C/C++ 语言编译器来编译此代码时，针对每个指针的多重访问会减少为单次访问。就编译器而言，并无指示表明 `d_i` 上的数据在函数执行期间发生更改，仅限最终写入 `d_o` 的数据才相关。其他写入在函数完成时被覆盖。

Vitis HLS 与 C++ 编译器的行为相符，将这些读写优化到单一读操作和单一写操作中。检验 RTL 时，每个端口上只有一次读取操作和一次写入操作。

此设计的基本问题在于测试平台和设计未能充分对您期望的 RTL 端口实现方式进行建模：

- 您期望 RTL 端口在每次传输事务期间进行多次读写（并且可通过串流来输入和输出数据）。
- 测试平台仅提供单一输入值，并且仅返回单一输出值。按 [接口上的多重访问指针](#) 所示进行 C/C++ 语言仿真可显示如下结果，证明每次输入累加 4 次。同样的值读取一次，且每次都累加。这并非 4 次不同的读取操作。

Din	Dout
0	0
1	4
2	8
3	12

为了使该设计能够多次读写 RTL 端口，请使用 `volatile` 限定符，如 [接口上的多重访问指针](#) 所示。`volatile` 限定符会告知 C/C++ 语言编译器和 Vitis HLS，不对指针访问作任何假设，且不将这些访问优化掉。即数据为易变数据，可能更改。

`volatile` 限定符：

- 阻止指针访问优化。
- 生成的 RTL 设计会对输入端口 `d_i` 执行期望的 4 次读取，对输出端口 `d_o` 执行 2 次写入。

即使使用 `volatile` 关键字，此编码样式（多次访问指针）仍存在函数和测试平台无法对多次独立的读写操作进行充分建模的问题。在此情况下，将执行 4 次读取，但只对相同数据读取 4 次。存在 2 次独立的写入，每次写入数据正确，但测试平台仅捕获最终写入的数据。



提示：要查看中间访问，请使用 `cosim_design -trace_level` 以在 RTL 仿真期间创建追踪文件，并在相应的查看器中查看此追踪文件。

多重访问易变指针接口可通过连线接口来实现。如果指定 FIFO 接口，Vitis HLS 会创建 RTL 测试平台以在每次读取时串流新数据。由于测试平台未提供新数据，因此 RTL 验证失败。测试平台也就无法对读写操作进行正确建模。

对串流数据接口进行建模

不同于软件，硬件系统的并发性使其能够充分利用串流数据。数据可持续不断提供给设计，设计可持续不断输出数据。RTL 设计可在设计完成处理现有数据之前接受新数据。

如 [理解易变数据](#) 所示，对软件中的串流数据进行建模是很重要的，编写软件以对现有硬件实现进行建模（即并发/串流性质已存在，且需加以建模）时尤其如此。

有多种方法可供使用：

- 添加 `volatile` 限定符，如“多重访问 `volatile` 指针接口”示例中所示。测试平台不会对专用读写进行建模，使用原 C/C++ 语言测试平台的 RTL 仿真可能失败，但通过查看追踪文件波形可以发现执行的读写操作正确。
- 修改代码以对显式专用读写进行建模。请参阅以下示例。
- 修改代码以使用串流数据类型。串流数据类型支持对使用串流数据的硬件进行准确建模。

以下代码示例已更新，以确保它可从测试平台读取 4 个唯一值并写入 2 个唯一值。由于指针访问按顺序进行并从位置 0 开始，因此综合期间可使用串流接口类型。

```
#include "pointer_stream_good.h"

void pointer_stream_good ( volatile dout_t *d_o,  volatile din_t *d_i) {
    din_t acc = 0;

    acc += *d_i;
    acc += *(d_i+1);
    *d_o = acc;
    acc += *(d_i+2);
    acc += *(d_i+3);
    *(d_o+1) = acc;
}
```

测试平台已更新，可对函数在每项传输事务内读取 4 个唯一值的行为进行建模。此新测试平台仅对单一传输事务进行建模。要对多项传输事务进行建模，必须增大输入数据集并多次调用该函数。

```
#include "pointer_stream_good.h"

int main () {
    din_t d_i[4];
    dout_t d_o[4];
    int i, retval=0;
    FILE *fp;

    // Create input data
    for (i=0;i<4;i++) {
        d_i[i] = i;
    }
}
```

```
// Call the function to operate on the data
pointer_stream_good(d_o,d_i);

// Save the results to a file
fp=fopen(result.dat,w);
for (i=0;i<4;i++) {
    if (i<2)
        fprintf(fp, %d %d\n, d_i[i], d_o[i]);
    else
        fprintf(fp, %d \n, d_i[i]);
}
fclose(fp);

// Compare the results file with the golden results
retval = system(diff --brief -w result.dat result.golden.dat);
if (retval != 0) {
    printf(Test failed !!!\n);
    retval=1;
} else {
    printf(Test passed !\n);
}

// Return 0 if the test
return retval;
}
```

测试平台可确认算法是否包含如下结果，结果显示：

- 单一传输事务有 2 项输出。
- 输出分别为前 2 次输入读取的累加，以及后 2 次输入读取的累加，再加上前 2 者的累加。

```
Din Dout
0    1
1    6
2
3
```

- 多次访问位于函数接口处的指针时需要注意的最后一个问题是 RTL 仿真建模。

多重访问指针和 RTL 仿真

当多次访问接口上的指针以进行读写时，Vitis HLS 无法从函数接口判定执行的读写次数。函数接口中的实参也无法告知 Vitis HLS 读写的值的数量。

```
void pointer_stream_good (volatile dout_t *d_o, volatile din_t *d_i)
```

除非代码告知 Vitis HLS 所需的值的数量（例如，阵列的最大大小），否则该工具假定仅含单个值，并仅为单一输入和单一输出进行 C/RTL 协同仿真建模。如果 RTL 端口实际读写多个值，那么 RTL 协同仿真将中止运行。RTL 协同仿真会对通过端口接口连接到 RTL 设计的外部生产者和使用者块进行建模。如果需要多个值，RTL 设计会在尝试读写多个值时停滞，原因是当前没有要读取的值，或者没有可供写入的空间。

在接口上使用多重访问指针时，必须将接口上必需的读写次数告知 Vitis HLS。手动为指针接口指定 INTERFACE 编译指示或指令，并将 `depth` 选项设置为所需的深度。

例如，以上代码样本中的实参 `d_i` 要求 FIFO 深度为 4。这样可确保 RTL 协同仿真提供足够的值以正确验证 RTL。

矢量数据类型

通过提供矢量数据类型即可对单指令流多数据流 (SIMD) 类型的运算轻松进行建模与综合。许多运算符都重载，以便为矢量类型提供 SIMD 行为。AMD Vitis™ HLS 库可提供 `hls::vector<T, N>` 类型的参考实现，用于表示单指令流多数据流 (SIMD) 矢量，其定义如下。

- `T`：表示矢量保存的元素类型，可以是用户定义的类型，该类型必须提供常用的算术运算。
- `N`：矢量包含的元素数量，必须为正整数。
- 当 `T` 的位宽和 `N` 均为 2 的幂整数时，即可实现最佳性能。

Vitis HLS 提供了模板类型 `hls::vector`，可用于定义 SIMD 操作数。在并行执行这些运算的综合期间，使用此类型执行的所有运算都映射到硬件。这些运算可在 `II=1` 的流水打拍循环内执行。以下示例显示了如何定义和使用含 8 个整数元素的矢量：

```
typedef hls::vector<int, 8> t_int8Vec;
t_int8Vec intVectorA, intVectorB;
.
.
.
void processVecStream(hls::stream<t_int8Vec>
&inVecStream1, hls::stream<t_int8Vec> &inVecStream2, hls::stream<int8Vec>
&outVecStream)
{
    for(int i=0; i<32; i++)
    {
        #pragma HLS pipeline II=1
        t_int8Vec aVec = inVecStream1.read();
        t_int8Vec bVec = inVecStream2.read();
        //performs a vector operation on 8 integers in parallel
        t_int8Vec cVec = aVec * bVec;
        outVecStream.write(cVec);
    }
}
```

如需了解更多信息，请参阅 [HLS 向量库](#)。请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Modeling/using_vectors](#) 以获取示例。

位宽传输

编码样式对于函数的主要影响在于影响函数实参和接口。如果准确调整函数实参大小，Vitis HLS 即可在整个设计中传输此信息。无需为每个变量创建任意精度类型。在以下示例中，有 2 个整数相乘，但仅低 24 位用于生成结果。

```
#include "ap_int.h"

ap_int<24> foo(int x, int y) {
    int tmp;

    tmp = (x * y);
    return tmp;
}
```

对此代码进行综合时，可生成 32 位乘法器，输出截位至 24 位。

如果将输入大小正确调整为 12 位类型 (int12) (如下代码示例所示)，则最终 RTL 使用 24 位乘法器。

```
#include "ap_int.h"
typedef ap_int<12> din_t;
typedef ap_int<24> dout_t;

dout_t func_sized(din_t x, din_t y) {
    int tmp;

    tmp = (x * y);
    return tmp;
}
```

针对 2 项函数输入使用任意精度类型即可足以确保 Vitis HLS 使用 24 位乘法器创建设计。这样即可在整个设计中传输 12 位类型。AMD 建议您正确调整层级中所有函数实参的大小，这样即可避免调整局部变量大小。

总而言之，从函数接口（尤其是从顶层函数接口）直接驱动变量时，这些变量即可阻止执行优化。常见用例是使用输入作为循环索引上限。

不受支持的 C/C++ 构造



提示：有些 C/C++ 构造并不受支持。本列表并非详尽无遗，而只是提供与部分不受支持的构造相关的详情。

虽然 HLS 编译器支持各种 C/C++ 语言，但部分构造不可综合，或者可能导致设计流程下游出错。本节探讨了为了对器件中的函数进行综合和实现而必须执行编码更改。

要执行综合，需满足下列要求：

- 该函数及其调用必须包含设计的完整功能。
- 无法通过对操作系统执行系统调用来执行任何功能。
- C/C++ 语言构造必须采用固定大小或受限大小。
- 这些构造必须以无歧义方式来实现。

系统调用

系统调用无法综合，因为系统调用操作与正在运行 C/C++ 程序的操作系统上执行的某些任务有关。

Vitis HLS 会忽略仅显示数据而对算法执行没有影响的常用系统调用，如 `printf()` 和 `fprintf(stdout,)`。一般情况下，系统调用无法综合，应在综合前从函数中移除。其他此类调用示例包括 `getc()`、`time()` 和 `sleep()`，这些操作都会对操作系统执行调用。

Vitis HLS 会在执行综合时定义 `__SYNTHESIS__` 宏。这样 `__SYNTHESIS__` 宏即可从设计中排除不可综合的代码。

注释：`__SYNTHESIS__` 宏仅限在要综合的代码中使用。请勿在测试平台中使用该宏，因为 C/C++ 语言仿真或 C/C++ RTL 协同仿真不会遵循其指示进行操作。



注意！您不得在代码中或者通过编译器选项来定义或取消定义 `__SYNTHESIS__` 宏，否则编译可能失败。

在以下代码示例中，来自子函数的中间结果将保存到硬盘上。`__SYNTHESIS__` 宏用于确保综合期间忽略不可综合的文件写入。

```
#include "hier_func4.h"

int shift_func(dint_t *in1, dint_t *in2, dout_t *outA, dout_t *outB)
{
    *outA = *in1 >> 1;
    *outB = *in2 >> 2;
}

void hier_func4(din_t A, din_t B, dout_t *C, dout_t *D)
{
    dint_t apb, amb;
```

```
sumsub_func(&A,&B,&apb,&amb);
#ifdef __SYNTHESIS__
FILE *fp1; // The following code is ignored for synthesis
char filename[255];
sprintf(filename, Out_apb_%03d.dat, apb);
fp1=fopen(filename,w);
fprintf(fp1, %d \n, apb);
fclose(fp1);
#endif
shift_func(&apb,&amb,C,D);
}
```

使用 `__SYNTHESIS__` 宏即可轻松排除不可综合的代码，而无需从函数中移除代码本身。使用此类宏并不意味着用于仿真的代码与用于综合的代码当前存在差异。



注意！ 如果使用 `__SYNTHESIS__` 宏来更改 C/C++ 语言代码的功能，那么它可能导致 C/C++ 语言仿真与 C/C++ 语言综合之间出现不同结果。此类代码中的错误本身非常难以调试。请勿使用 `__SYNTHESIS__` 宏来更改功能。

动态存储器使用

负责管理系统内的存储器分配的任意系统调用（例如，`malloc()`、`alloc()` 和 `free()`）都使用该操作系统的存储器中现有的资源，并在运行时期间进行创建和释放。为了能对硬件实现进行综合，设计必须完全为自包含的独立设计，并指定所有必要的资源。

必须从设计代码中移除存储器分配系统调用后才能进行综合。由于动态存储器操作用于定义设计功能，因此必须转换为受同等限制的表示法。以下代码示例显示了如何将使用 `malloc()` 的设计变换为可综合版本，并突出显示了 2 项实用的编码样式技巧：

- 此设计不使用 `__SYNTHESIS__` 宏。

用户定义的 `NO_SYNTH` 宏用于选择可综合版本或不可综合版本。这样可确保在 C/C++ 语言中仿真的代码与 Vitis HLS 中综合的代码相同。

- 使用 `malloc()` 的原始设计中的指针无需重写即可配合固定大小的元素一起使用。

可创建固定大小的资源，并且可将现有指针直接设置为指向此固定大小的资源。此技巧可避免对现有设计进行手动重新编码。

```
#include "malloc_removed.h"
#include <stdlib.h>
//#define NO_SYNTH

dout_t malloc_removed(din_t din[N], dsel_t width) {

#ifdef NO_SYNTH
long long *out_accum = malloc (sizeof(long long));
int* array_local = malloc (64 * sizeof(int));
#else
long long _out_accum;
long long *out_accum = &_out_accum;
int _array_local[64];
int* array_local = &_array_local[0];
#endif
}
```

```
int i,j;

LOOP_SHIFT:for (i=0;i<N-1; i++) {
  if (i<width)
    *(array_local+i)=din[i];
  else
    *(array_local[i])=din[i]>>2;
}

*out_accum=0;
LOOP_ACCUM:for (j=0;j<N-1; j++) {
  *out_accum += *(array_local+j);
}

return *out_accum;
}
```

由于在此处执行编码更改会影响设计功能，因此 AMD 不建议使用 `__SYNTHESIS__` 宏。AMD 建议执行以下步骤：

1. 将用户定义的 `NO_SYNTH` 宏添加到代码中，并对代码进行修改。
2. 启用 `NO_SYNTH` 宏、执行 C/C++ 语言仿真，并保存结果。
3. 禁用 `NO_SYNTH` 宏，执行 C/C++ 语言仿真以验证结果是否相同。
4. 在禁用用户定义的宏的情况下执行综合。

此方法可确保通过 C/C++ 语言仿真来确认更新后的代码，然后对相同的代码进行综合。就像 C/C++ 语言中的动态存储器使用限制一样，Vitis HLS 也不支持动态创建或删除 C/C++ 对象（用于综合）。

指针限制

常用指针强制转换

Vitis HLS 不支持常用指针强制转换，但支持原生 C/C++ 语言类型之间的指针强制转换。

指针阵列

Vitis HLS 支持对指针阵列进行综合，前提是每个指针都指向 1 个标量或标量阵列。指针阵列不能指向其他指针。

函数指针

不支持函数指针。

注释：不支持指针到指针。

递归函数

递归函数无法综合。这适用于可形成多次递归的函数：

```
unsigned foo (unsigned n)
{
    if (n == 0 || n == 1) return 1;
    return (foo(n-2) + foo(n-1));
}
```

Vitis HLS 也不支持其中包含有限数量的函数调用的尾递归。

```
unsigned foo (unsigned m, unsigned n)
{
    if (m == 0) return n;
    if (n == 0) return m;
    return foo(n, m%n);
}
```

在 C++ 中，模板可实现尾递归，并且后续可用于可综合的尾递归设计。



注意！ 不支持虚拟函数。

标准模板库

许多 C++ 标准模板库 (STL) 都包含函数递归并使用动态存储器分配。因此，Vitis HLS 无法对 STL 进行综合。对于 STL 的解决方案是创建局部函数，该函数具有相同功能但不呈现递归、动态存储器分配或动态创建和解构对象特征。

注释：支持对标准数据类型（例如 `std::complex`）进行综合。但是，`std::complex<long double>` 数据类型在 Vitis HLS 中不受支持，不应使用。

未定义的行为

C/C++ 未定义的行为可能导致仿真与综合中出现不同的行为。此类行为的示例如下所示：

```
for (int i=0; i<N; i++) {
    int val; // uninitialized value
    if (i==0) val=0;
    else if (cond) val=1;
    // val may have indeterminate value here
    A[i] = val; // undefined behavior
    val++; // dead code
}
```

在以上示例中，只要 `i==0` 和 `(cond)` 值均非 `true`，`A[i]` 就不应从上一次循环迭代得到 `val` 值。您甚至不应期望会发生递增 (`val++`) 操作。对于完全分区后获取的标量值也同样如此。

对于此类 C/C++ 未定义的行为，编译代码时，CPU 编译器与 Vitis HLS 之间的行为可能有别，这可能导致 RTL/协同仿真期间发生不匹配。这是由于在为 CPU 执行编译时，通常不同循环迭代之间的 `val` 保留在相同寄存器或栈位置，因此不同循环迭代之间的行为是保留 `val` 的值不变。

解决方案是在每次迭代时初始化 `val`（前提是，此行为与期望相符）或者将 `val` 的声明移至循环上层（尽可能高），使其生存期范围与所需复用相符。您不应期望编译器会根据未定义的 C/C++ 行为来推断已定义的特定 RTL 行为。

虚拟函数和指针

不支持。

HLS 设计接口

完成 C++ 代码并且综合将其转换为 RTL 设计后，您还可以控制硬件实现中的多项要素。具体来说，您需要考量进出 HLS 设计的输入和输出、存储器的布局、数据对齐的管理以及 HLS 设计的执行模型。本节将讨论下列主题：

- [定义接口](#)
- [Vitis HLS 存储器布局模型](#)
- [HLS 设计的执行模式](#)
- [控制初始化与复位行为](#)

定义接口

接口综合简介

HLS 组件的顶层函数的实参均综合到接口和端口内，这些接口和端口通过将多个信号加以组合来定义 HLS 组件与设计外部的元件之间的通信协议。本工具会自动定义接口，并使用业界标准来指定要使用的协议。本工具选择的接口类型取决于顶层函数的参数的数据类型和方向、HLS 组件的目标流程、指定的默认接口配置设置（如 [接口配置](#) 中所述）以及任何指定的 `INTERFACE` 编译指示或指令。



提示： 您可使用 `INTERFACE` 编译指示或指令手动分配接口。如需了解更多信息，请参阅 [添加编译指示和指令](#)。

如 [目标流程概述](#) 中所述，HLS 组件支持的目标流程包括：

- 作为工具的默认流程的 AMD Vivado™ IP 流程
- Vitis 内核流程，它是自下而上的设计流程，适用于 Vitis 应用加速开发流程

您可在创建 HLS 组件时指定目标流程，如 [第三部分：Vitis HLS 流程步骤](#) 中所述。

此接口可定义内核的 3 个元素：

1. 此接口可为流入或流出 HLS 设计的数据定义通道。数据可从多种外部来源流入内核或 IP，外部来源包括：主机应用、外部相机或传感器或者 AMD 器件上实现的其他内核或 IP 等。Vitis 内核的默认通道是 AXI 适配器，如 [适用于 Vitis 内核流程的接口](#) 中所述。
2. 此接口负责定义端口协议，此协议用于控制流经数据通道的数据流，定义何时数据有效并且可供读取或可供写入，如 [Vivado IP 流程的端口级协议](#) 中所述。



提示： 这些端口协议可在 Vivado IP 流程中自定义，但在 Vitis 内核流程中设置且在大多数情况下不可更改。

3. 此接口还负责定义 HLS 设计的执行控制方案，指定内核或 IP 操作方式为流水打拍还是顺序，如 [块级控制协议](#) 中所述。

如 [利用 M_AXI 接口进行设计的最佳实践](#) 中所述，接口的选择与配置是设计成功与否的关键。但本工具会尝试通过为目标流程选择默认接口来简化此进程，如上所述。

完成综合后，您可在 [综合汇总](#) 报告的“软件 I/O 信息”部分中复查将 C/C++ 代码的软件实参映射到硬件端口或接口的方式是否正确。

适用于 Vitis 内核流程的接口

Vitis 内核流程为已编译的内核对象 (.xo) 提供支持，以便从主机应用和 Xilinx Runtime (XRT) 来执行软件控制。如《使用 Vitis 进行数据中心加速》(UG1700) 中的 [PL 内核属性](#) 中所述，此流程具有非常具体的接口要求，Vitis HLS 必须满足这些要求。

Vitis HLS 支持多种存储器、串流和寄存器接口范例，其中每个范例都遵循某个接口协议并使用适配器来与外部世界进行通信。

- 存储器范例 (m_axi)：内核通过存储器（如 DDR、HBM、PLRAM/BRAM/URAM）来访问数据
- 串流范例 (axis)：数据从其他串流源（例如，视频处理器或其他内核）串流至内核中，也可从该内核流出。
- 寄存器范例 (s_axilite)：内核通过寄存器接口来访问数据，软件则通过寄存器读/写来访问数据。



提示：AXI 协议要求采用低电平有效复位。如果 `syn.rtl.reset_level` 设为高电平有效状态（默认设置），那么本工具会定义此复位并显示警告。

Vitis 内核流程默认会实现下列接口：

C 语言实参类型	范例	接口协议 (I/O/Inout)
标量（通过值来传递）	寄存器	AXI4-Lite (s_axilite)
阵列	存储器	AXI4 存储器映射 (m_axi)
指向阵列的指针	存储器	m_axi
指向标量的指针	寄存器	s_axilite
引用	寄存器	s_axilite
hls::stream	串流	AXI4-Stream (axis)

如上表所示，指向阵列的指针是作为 m_axi 接口（用于数据传输）来实现的，指向标量的指针则是使用 s_axilite 接口来实现的。作为常量来传递的标量值不需要读取权限，而指向标量值的指针则需要读取和写入权限。s_axilite 接口会根据 C 语言实参类型来实现一项额外的内部协议。此内部实现可使用 [Vivado IP 流程的端口级协议](#) 来控制。但如非必要，不应在 Vitis 内核流程中修改默认端口协议。

注释：当元素需要不同接口类型时，Vitis HLS 将不会自动推断结构体/类的成员元素的默认接口。例如，当某个结构体的某个元素需要串流接口，而另一个成员元素需要 s_axilite 接口时，就是如此。您不能依靠默认接口分配，而必须为结构体的每个元素显式定义 INTERFACE 编译指示。如未定义任何 INTERFACE 编译指示或指令，则 Vitis HLS 会发出以下错误消息：

```
ERROR: [HLS 214-312] Vitis mode requires explicit INTERFACE pragmas for structs in the interface. Please add one INTERFACE pragma for each struct member field for argument 'd' of function 'dut(A&)' (example.cpp:19:0)
```

Vitis 内核流程的默认执行模式为流水节拍执行，即启用内核的重叠执行以改善吞吐量。这是通过 s_axilite 接口上的 ap_ctrl_chain 块控制协议来指定的。



提示： Vitis 环境支持含所有受支持的块控制协议的内核，如 [块级控制协议](#) 中所述。

以下代码中的 `vadd` 函数提供了接口综合的示例。

```
#define VDATA_SIZE 16

typedef struct v_datatype { unsigned int data[VDATA_SIZE]; } v_dt;

extern "C" {
void vadd(const v_dt* in1, // Read-Only Vector 1
          const v_dt* in2, // Read-Only Vector 2
          v_dt* out_r, // Output Result for Addition
          const unsigned int size // Size in integer
) {

    unsigned int vSize = ((size - 1) / VDATA_SIZE) + 1;

    // Auto-pipeline is going to apply pipeline to this loop
    vadd1:
    for (int i = 0; i < vSize; i++) {
        vadd2:
        for (int k = 0; k < VDATA_SIZE; k++) {
            out_r[i].data[k] = in1[i].data[k] + in2[i].data[k];
        }
    }
}
}
```

`vadd` 函数包括：

- 两个指针输入：`in1` 和 `in2`
- 指针输出：`out_r`，结果写入其中
- 标量值 `size`

如果 Vitis HLS 为 Vitis 内核流程使用默认接口综合设置，那么设计将综合到含下图所示端口和接口的 RTL 块中。

图 30：默认接口综合后的 RTL 端口



该工具会在 RTL 设计上创建三种类型的接口端口，以处理数据流和控制流。

- 时钟、复位和中断端口：在内核中添加 `ap_clk`、`ap_rst_n` 和 `interrupto`。
- AXI4-Lite 接口：`s_axi_control` 接口中包含标量实参（如 `size`）、管理 `m_axi` 接口的地址偏移并定义块控制协议。
- AXI4 存储器映射接口：`m_axi_gmem` 接口，其中包含指针实参：`in1`、`in2` 和 `out_r`。

Vitis 的 M_AXI 接口的详细信息

AXI4 存储器映射 (`m_axi`) 接口允许内核在全局存储器（DDR、HBM 和 PLRAM）中读取和写入数据，存储器映射接口便于跨加速应用的不同元素进行数据共享，例如，在主机与内核之间或者在加速器卡上的内核之间。以下列出了 `m_axi` 接口的主要优势：

- 该接口均有独立的读取通道和写入通道
- 它支持基于突发的访问
- 它可提供未完成传输事务队列
- 认识突发访问：AXI4 存储器映射接口通过单一地址阶段即可支持高达 4K 字节的高吞吐量突发。对于突发模式传输，Vitis HLS 会使用单一基地址读取或写入数据，后接多次顺序数据采样，因此，该模式支持更高的数据吞吐量。使用流水打拍的 `for` 循环来复制存储器时，可使用操作的突发模式。如需了解更多信息，请参阅 [控制 AXI4 突发行](#) 或 [AXI 突发传输](#)。
- 自动端口拓宽和端口宽度对齐：

如 [端口宽度自动调整](#) 中所述，Vitis HLS 能够自动拓宽端口宽度以便执行数据传输并改善突发访问，前提是该工具可发现突发访问。因此，突发所需的所有前置条件（如 [AXI 突发传输](#) 中所述）都必须得到满足后才能调整端口大小。

在 Vitis 内核流程中，默认启用端口宽度自动调整，可采用以下配置命令来执行此操作（请注意，其中一条命令指定为位，另一条命令则指定为字节）：

```
syn.interface.m_axi_max_widen_bitwidth=512
syn.interface.m_axi_alignment_byte_size=64
```

- 偏移规则：



重要提示！ 在 Vitis 内核流程中，操作的默认模式为 `syn.interface.m_axi_offset=direct` 和 `syn.interface.default_slave_interface=s_axilite`，并且不应更改。

正确指定偏移即可将 HLS 内核正确集成到 Vitis 系统中。如需了解更多信息，请参阅 [偏移和操作模式](#)。

- 捆绑接口 - 性能对比资源使用情况：

默认情况下，Vitis HLS 将具有兼容选项的函数实参组合到单个 `m_axi` 接口适配器中，如 [M_AXI 捆绑](#) 中所述。将端口捆绑到单个接口中可消除 AXI4 逻辑，因而有助于节省器件资源，在拥挤的设计内工作时可能需要这些资源。

但是，单一接口捆绑可能会限制内核性能，因为所有存储器传输都必须穿越单个接口。`m_axi` 接口具有独立的读取 (READ) 和写入 (WRITE) 通道，因此单个接口即可同时执行读取和写入，但仅限单一位置。使用多个捆绑即可创建多个接口以连接到存储体，从而能够增大内核带宽和吞吐量。

Vitis 的 S_AXILITE 接口的详细信息

在 C++ 语言中，从父函数调用函数时，该函数就会开始处理数据。函数调用将在调用时推送到栈上，处理完成时则会从栈中移除以便将控制权返还给调用函数。此进程可确保父函数明确知晓子函数的状态。

由于主机和内核在 Vitis 内核流程中占用两个独立的计算空间，因此“栈”由 Xilinx Runtime (XRT) 来管理，通信则通过 `s_axilite` 接口来管理。此内核由软件通过 XRT 来控制，方法是读取和写入 `s_axilite` 接口的控制寄存器，如 [S_AXILITE 控制寄存器映射](#) 中所述。此接口具有下列特性：

- 控制协议：块控制协议可定义 `s_axilite` 接口中的控制寄存器，以便您设置控制信号来管理内核的执行和操作。
- 标量实参：内核上的标量数据为典型输入，可视作为编程常量或参数。主机应用通过 `s_axilite` 接口来传输这些值。
- 指向标量实参的指针：如果指向标量值的指针分配给 `s_axilite` 接口，那么 Vitis HLS 允许您在该指针上执行读取或写入。指针默认分配给 `m_axi` 接口，因此您必须使用 `INTERFACE` 编译指示或指令将该指针手动分配给 `s_axilite`：

```
int top(int *a, int *b) {
#pragma HLS interface s_axilite port=a
```

- 偏移规则：

注释： Vitis 内核流程会判定所需的偏移。请勿在该流程中指定 `offset` 选项。

- 捆绑规则：

Vitis 内核流程仅支持单个 `s_axilite` 接口，这意味着所有 `s_axilite` 接口都必须捆绑在一起。

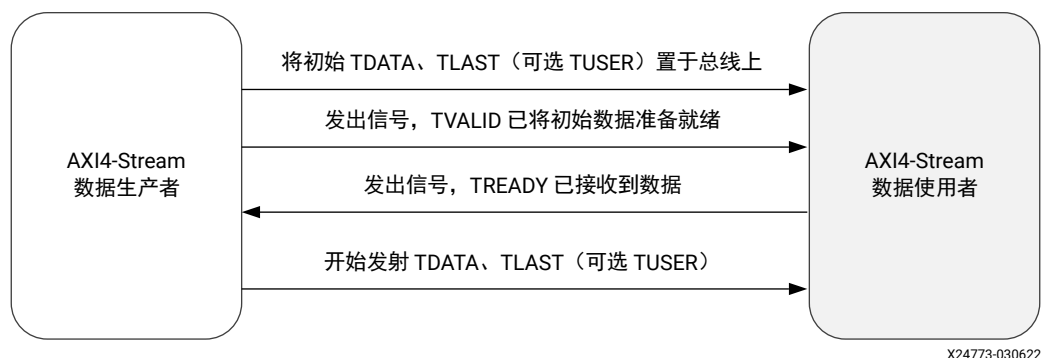
- 如不指定捆绑，那么该工具会自动创建名为 `Control` 的默认捆绑。
- 如果您出于某些原因想要手动指定捆绑名称，则必须将此捆绑应用于所有 `s_axilite` 接口以创建单一捆绑。

Vitis 的 AXIS 接口的详细信息

AXI4-Stream 协议 (AXIS) 可定义单向通道，用于按顺序串流数据。AXI4-Stream 接口可突发的数据量是无限的，故而能显著提升性能。不同于需要地址才能读/写存储器的 AXI4 存储器映射接口，AXIS 接口可直接将数据传递至另一个 AXIS 接口，无需地址，因此使用的器件资源更少。通过将这些功能特性相结合，即可使串流接口成为轻量级高性能接口。

AXI4-Stream 可用于生产者与使用者之间的业界标准 `ready/valid` 握手，如下图所示。当生产者发送 `TVALID` 信号，并且使用者通过发送 `TREADY` 信号来响应后，即可开始数据传输。此数据和控制握手应持续进行，直至 `TREADY` 或 `TVALID` 设为低位或者生产者断言 `TLAST` 信号有效（表示这是传输的最后一个数据包）为止。

图 31：AXI4-Stream 握手



X24773-030622



重要提示！ AXIS 接口只能分配给内核或 IP 的顶层实参（端口），不能分配给设计内部函数实参。HLS 设计内部使用的串流通道应使用 `hls::stream`，而不能使用 AXIS 接口。

您应使用 `hls::stream<T_data_type>` 来定义串流的数据类型，使用 `ap_axis` 结构体类型来实现 AXIS 接口。如 [AXI4-Stream 接口](#) 中所述，`ap_axis` 结构体支持您选择接口实现（含旁路通道或不含旁路通道）：

- 不含旁路的 AXI4-Stream 接口 可通过使用更少的资源来将 AXIS 接口实现为超轻量级接口
- 含旁路的 AXI4-Stream 接口 可实现全功能接口以提供更强大的控制能力



提示：您不应自行定义结构体来对 AXIS 信号（旁路通道、TLAST、TVALID）进行建模。您可改为重载 TDATA 信号来实现自己的数据类型。

Vivado IP 流程接口

由于需要为各种应用提供 FPGA 设计支持，因此 Vivado IP 流程支持各种 I/O 协议和握手。此流程支持将多个 IP 集成到单个系统内的传统系统流程。IP 可通过 Vitis HLS 来生成。在此 IP 流程中，有两种控制模式可用于系统执行：

- 软件控制：通过在嵌入式 Arm 处理器或外部 x86 处理器上运行的软件应用来控制系统，使用驱动程序来访问硬件设计的各种元件，并通过读取和写入硬件中的寄存器来控制系统中 IP 的执行。
- 自同步：在此模式下，IP 将公开各种信号，用于启动和停止内核。这些信号受系统设计的其他 IP 或其他元件驱动，此系统设计负责处理 IP 的执行。

Vivado IP 流程支持存储器、串流和寄存器接口范例，其中每个范例都支持不同接口协议与外部世界进行通信，如下表所示。请注意，虽然 Vitis 内核流程仅支持 AXI4 接口适配器，但此流程支持多种不同的接口类型。

表 8：接口类型

范例	描述	接口类型
存储器	内核通过存储器（如 DDR、HBM、PLRAM/BRAM/URAM）来访问数据	<code>ap_memory</code> 和 AXI4 存储器映射 (<code>m_axi</code>)
串流	数据将从其他串流源（如视频处理器或其他内核）串流至内核，并且可从内核向外串流。	<code>ap_fifo</code> , AXI4-Stream (<code>axis</code>)
寄存器	内核通过寄存器读取和写入所执行的寄存器接口来访问数据。	<code>ap_none</code> 、 <code>ap_hs</code> 、 <code>ap_ack</code> 、 <code>ap_ovld</code> 、 <code>ap_vld</code> 和 AXI4-Lite 适配器 (<code>s_axilite</code>)。

默认接口由顶层函数中的 C 语言实参类型和默认范例来定义，如下表所示。

表 9：默认接口

C 语言实参类型	受支持的范例	默认范例	默认接口协议		
			输入	输出	输入输出
标量变量（通过值来传递）	寄存器	寄存器	<code>ap_none</code>	不适用	不适用
阵列	存储器、串流	存储器	<code>ap_memory</code>	<code>ap_memory</code>	<code>ap_memory</code>
指针	存储器、串流、寄存器	寄存器	<code>ap_none</code>	<code>ap_vld</code>	<code>ap_ovld</code>
参考	寄存器	寄存器	<code>ap_none</code>	<code>ap_vld</code>	<code>ap_vld</code>
<code>hls::stream</code>	串流	串流	<code>ap_fifo</code>	<code>ap_fifo</code>	不适用

Vivado IP 流程的默认执行模式为顺序执行，要求 HLS IP 完成前一次迭代后才能开始下一次迭代。这是通过 `ap_ctrl_hs` 块控制协议指定的。控制协议可按 [块级控制协议](#) 中指定的方式来更改。

以下代码中的 `vadd` 函数提供了在 Vivado IP 流程中进行接口综合的示例。

```
#define VDATA_SIZE 16

typedef struct v_datatype { unsigned int data[VDATA_SIZE]; } v_dt;

extern "C" {
void vadd(const v_dt* in1, // Read-Only Vector 1
          const v_dt* in2, // Read-Only Vector 2
          v_dt* out_r, // Output Result for Addition
          const unsigned int size // Size in integer
) {

    unsigned int vSize = ((size - 1) / VDATA_SIZE) + 1;

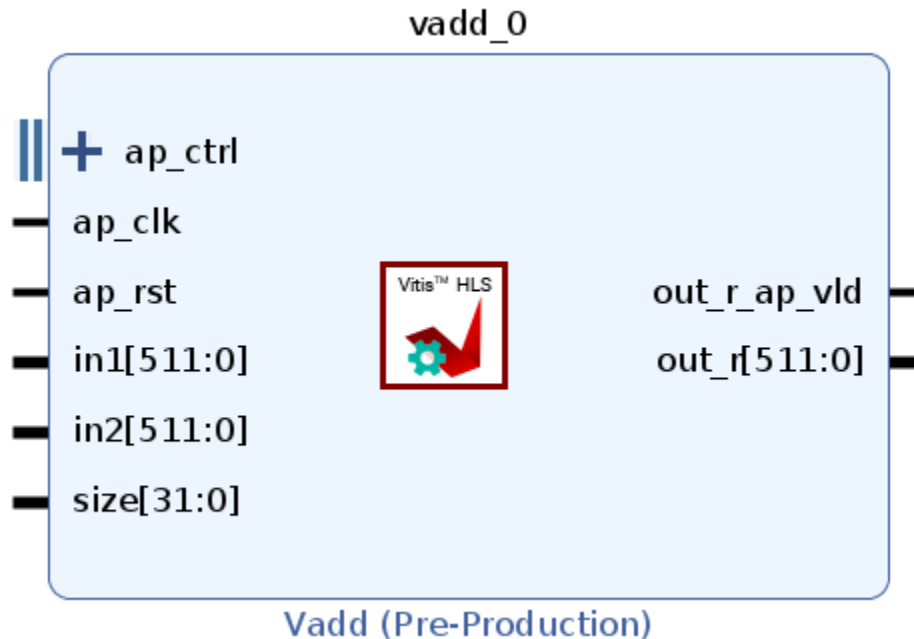
    // Auto-pipeline is going to apply pipeline to this loop
    vadd1:
    for (int i = 0; i < vSize; i++) {
        vadd2:
        for (int k = 0; k < VDATA_SIZE; k++) {
            out_r[i].data[k] = in1[i].data[k] + in2[i].data[k];
        }
    }
}
}
```

`vadd` 函数包括：

- 两个指针输入： `in1` 和 `in2`
- 指针： `out_r`，结果写入该指针
- 标量值 `size`

通过 Vivado IP 流程所使用的默认接口综合设置，设计将综合到含端口的 RTL 块中，如下图所示。

图 32：默认接口综合后的 RTL 端口



在默认 Vivado IP 流程中，该工具会在 RTL 上创建三种类型的接口端口，以处理数据流和控制流。

- 时钟和复位端口：ap_clk 和 ap_rst 均添加到内核中。
- 块级控制协议：ap_ctrl 接口是作为 s_axilite 接口来实现的。
- 端口级协议：这些协议是针对顶层函数和函数返回（如果函数返回值）中的每个实参创建的。如上表所述，大部分实参都使用端口协议 ap_none，并且不含控制信号。在以上 vadd 示例中，这些端口包括：in1、in2 和 size。但是，out_r_o 输出端口使用 ap_vld 协议，因此与 out_r_o_ap_vld 信号相关联。

Vivado IP 流程中的 AP_Memory

ap_memory 是上表中所述存储器范例的默认接口。在 Vivado IP 流程中，它用于与存储器资源（如 BRAM 和 URAM）进行通信。ap_memory 协议同样遵循地址和数据阶段方式来进行操作。该协议首先请求读/写资源，并等待至它收到资源可用确认为止。随后，它会发起读/写的数据传输阶段。

ap_memory 的重要注意事项之一是它仅执行单节拍数据传输并传输至单一地址，这有别于 m_axi，后者可执行突发访问。因此，相比其他协议，ap_memory 属于轻量级协议。

- 存储器资源：默认情况下，Vitis HLS 会实现协议以便与单端口 RAM 资源进行通信。您可在 INTERFACE 编译指示或指令中指定 storage_type 来控制该协议的实现。storage_type 指令允许您显式定义使用的 RAM 类型，以及所创建的 RAM 端口（单端口或双端口）。如果不指定 storage_type，那么 Vitis HLS 会使用：
 - 单端口 RAM（默认情况下）。
 - 双端口 RAM，前提是这样可缩短启动时间间隔或减少时延。

Vivado IP 流程中的 M_AXI 接口

AXI4 存储器映射 (m_axi) 接口允许 IP 读取和写入全局存储器（DDR、HBM 或 PLRAM）中的数据，存储器映射接口是跨多个 IP 共享数据的便利方法。以下列出了 m_axi 接口的主要优势：

- 每个接口均有独立的读取通道和写入通道
- 它支持基于突发的访问
- 它可提供未完成传输事务队列
- 认识突发访问：AXI4 存储器映射接口支持高达 4K 字节的高吞吐量突发，且只需单一地址阶段即可。对于突发模式传输，Vitis HLS 会使用单一基址读取或写入数据，后接多次顺序数据采样，因此，该模式支持更高的数据吞吐量。使用 `C memcpy` 函数或经流水打拍的 `for` 循环时，即可执行突发模式操作。如需了解更多信息，请参阅 [控制 AXI4 突发行为](#) 或 [AXI 突发传输](#)。
- 自动端口拓宽和端口宽度对齐：

如 [端口宽度自动调整](#) 中所述，Vitis HLS 能够自动拓宽端口宽度以便执行数据传输，并改善突发访问，前提是突发的所有前置条件均得到满足。在 Vivado IP 流程中，默认情况下，以下配置设置会禁用端口宽度自动调整。要启用此功能，必须更改这些配置选项（请注意，其中一条命令指定为位，另一条命令则指定为字节）：

```
syn.interface.m_axi_max_widen_bitwidth=0
syn.interface.m_axi_alignment_byte_size=0
```

- 为 Vivado IP 模式指定对齐：

`m_axi` 端口的对齐允许该端口根据指定对齐方式来读取和写入存储器。选择正确的对齐方式之所以如此重要，是因为最好情况下，它会影响性能，最差情况下，则会影响功能。

对齐的存储器访问意味着指针（或数据起始地址）是特定类型的值（称为对齐）的倍数。对齐是自然地址倍数，其类型必须或者应该存储在存储器上（例如，出于性能原因）。例如，标准 32 位架构在存储器中存储的数据字为 32 位，每个字均为 4 字节。此数据对齐到单个字或 4 字节边界。

对齐方式在整个系统中应保持一致。当 IP 在 AXI4 主模式下工作时，即可确定并且应该指定对齐方式，如采用 4 字节对齐的 32 位架构。当此 IP 在从模式下工作时，对齐应与主模式对齐方式相匹配。

- 偏移规则：

`m_axi` 偏移默认设置为 `offset=direct` 和 `syn.interface.default_slave_interface=s_axilite`。但在 Vivado IP 流程中，您可按 [偏移和操作模式](#) 中所述方式对其进行更改。

- 捆绑接口 - 性能对比资源使用情况：

默认情况下，Vitis HLS 将具有兼容选项的函数实参组合到单个 `m_axi` 接口适配器中，如 [M_AXI 捆绑](#) 中所述。将端口捆绑到单个接口中可消除 AXI4 逻辑，因而有助于节省器件资源，在拥挤的设计内工作时可能需要这些资源。

但是，单一接口捆绑可能会限制 IP 性能，因为所有存储器传输都必须穿越单个接口。`m_axi` 接口具有独立的读取 (READ) 和写入 (WRITE) 通道，因此单个接口即可同时执行读取和写入，但仅限单一位置。使用多个捆绑即可创建多个接口以连接到存储体，从而能够改善性能。

Vivado IP 流程中的 S_AXILITE

在 Vivado IP 流程中，默认执行流程由寄存器读取和写入通过 `s_axilite` 接口使用默认 `ap_ctrl_hs` 控制协议来进行管理。此 IP 由软件控制，方法是读取和写入 `s_axilite` 接口的控制寄存器，如 [S_AXILITE 控制寄存器映射](#) 中所述。

`s_axilite` 接口可提供下列功能特性：

- 控制协议：采用 [块级控制协议](#) 中指定的块控制协议。
- 标量实参：来自顶层函数的标量实参可映射到 `s_axilite` 接口，此接口会为值创建寄存器，如 [S_AXILITE 控制寄存器映射](#) 中所述。软件可对此寄存器空间执行读/写操作。

- 偏移规则：Vivado IP 流程可以基于顶层函数中关联的 C 语言实参的数据类型来定义端口的大小或分配给端口的地址范围。但该工具还允许您手动定义偏移大小，如 [S_AXILITE 偏移选项](#) 中所述。
- 捆绑规则：在 Vivado IP 流程中，您可使用 `s_axilite` 接口来指定多个捆绑，这样将为您定义的每个捆绑创建一个独立的接口适配器。但您应熟悉一些有关使用多个捆绑的规则，如 [S_AXILITE 捆绑规则](#) 中所述。

Vivado IP 流程中的 AP_FIFO

在 Vivado IP 流程中，`ap_fifo` 接口协议是默认接口，用于供该接口上的串流范例与存储器资源 FIFO 进行通信，并用作为 IP 内不同函数之间的通信通道。仅当数据采用顺序访问时，才应使用此协议，并且 AMD 强烈建议使用 `hls::stream<data type>`，它可实现 FIFO。



提示： `<data type>` 不应与 `T_data_type` 相同，后者应仅限在接口上使用。

Vivado IP 流程中的 AXIS 接口

AXI4-Stream 协议 (`axis`) 可作为备选用于串流接口，该协议定义了一条单向通道用于按顺序串流数据。不同于 `m_axi` 协议，AXI4-Stream 接口可突发的数据量是无限的，故而能显著提升性能。不同于需要地址才能读/写存储器的 AXI4 存储器映射接口，`axis` 接口可直接将数据传递至另一个 `axis` 接口，无需地址，因此使用的器件资源更少。通过这些功能特性相结合，即可使串流接口成为轻量级高性能接口，如 [AXI4-Stream 接口](#) 中所述。

AXI 适配器接口协议



重要提示！ 如 [适用于 Vitis 内核流程的接口](#) 中所述，AXI4 适配器接口即 Vitis HLS 用于 Vitis 应用加速开发流程的默认接口，但在 Vivado IP 流程中也同样支持这些接口。AXI4-Stream 加速器适配器是软核 AMD LogiCORE™ 知识产权 (IP)，作为基础架构块用于将硬件加速器连接到嵌入式 CPU。

Vitis HLS 支持的 AXI4 接口包括 AXI4-Stream 接口 (`axis`)、AXI4-Lite (`s_axilite`) 和 AXI4 主接口 (`m_axi`)。如需了解有关 AXI4 接口的完整说明（包括时序和端口），请参阅《Vivado Design Suite：AXI 参考指南》([UG1037](#))。如前述章节所述，AXI4 接口通过实现适配器来根据协议管理通信。没有任何其他 Vitis HLS 接口可用于实现此类适配器。

- `m_axi`：仅在阵列和指针（以及 C++ 中的引用）上指定。`m_axi` 模式用于指定 [AXI4 存储器映射接口](#)。



提示： 您可将捆绑实参组合到单个 `m_axi` 接口内。

- `s_axilite`：仅限在除串流外的任意类型的实参上指定此协议。`s_axilite` 模式用于指定 [AXI4-Lite 从接口](#)。



提示： 您可将多个实参捆绑到单个 `s_axilite` 接口内。

- `axis`：仅限在输入实参或输出实参上指定此协议，而不得在输入/输出实参上指定。`axis` 模式用于指定 [AXI4-Stream 接口](#)。



提示： AXI 协议要求采用低电平有效复位。如果您的设计使用 AXI 接口，那么该工具将定义此复位级别并显示警告，前提是 `syn.rtl.reset_level` 设为高电平有效，这是默认设置。

AXI4 主接口

AXI4 存储器映射 (`m_axi`) 接口允许内核在全局存储器（DDR、HBM、和 PLRAM）内读写数据。存储器映射接口便于跨加速应用的不同元素共享数据，例如，在主机与内核之间或者在加速器卡上的内核之间共享数据。请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Interface/Memory](#)，以获取这些概念的部分示例。

以下列出了 `m_axi` 接口的主要优势：

- 此接口有独立的读取通道和写入通道
- 它支持基于突发的访问，潜在性能可达 ~17 GB/s
- 它可为未完成传输事务提供支持
- 它可包含高速缓存用于改善性能，此高速缓存由 `CACHE` 编译指示或指令来表示。

在 Vitis 内核流程中，默认情况下，会将 `m_axi` 接口分配到指针和实参。在此流程中，它支持以下默认功能：

- 将指针和阵列实参自动映射到 `m_axi` 接口
- 在 Vitis 中，默认操作模式是 `offset=slave`，并且不应更改此模式
- 所有指针和阵列实参都映射到单个接口捆绑，以节省器件资源，并且各端口在活动期间共享读取和写入访问
- Vitis 流程中的默认对齐设置为 64 字节
- 默认情况下，最大读/写突发长度设置为 16

虽然 Vivado IP 流程默认不使用 `m_axi` 接口，但指定该接口时，它具有下列默认功能特性：

- 默认操作模式为 `offset=off`，但可对此进行更改，如 [偏移和操作模式](#) 中所述
- 已分配的指针和阵列实参都映射到单个接口捆绑，以节省器件资源，并且在活动期间共享该接口
- Vivado IP 流程中的默认对齐设置为 1 字节
- 默认情况下，最大读/写突发长度设置为 16

在 Vivado IP 流程与 Vitis 内核流程中，`m_axi` 接口的全局默认值均由 `syn.interface.xxx` 命令来定义，如 [接口配置](#) 中所述。`INTERFACE` 编译指示或指令可用于按需修改特定接口的默认值。有些自定义设置有助于提升设计性能，如 [AXI 系统性能优化](#) 中所述。您还可按 `pragma HLS cache` 中的描述指定 `CACHE` 编译指示或指令，以改善 `m_axi` 接口的性能。

您可在阵列或指针/参考实参上使用 AXI4 主接口，Vitis HLS 通过以下任一模式来实现该接口：

- 单独数据传输
- 突发模式数据传输

对于单独数据传输，Vitis HLS 会为每个地址读取或写入单个数据元素。以下示例显示了单一读取操作和单一写入操作。在此示例中，Vitis HLS 可在 AXI 接口上生成 2 个地址，1 个用于读取单个数据值，另 1 个用于写入单个数据值。该接口会为每个地址传输 1 个数据值。

```
void bus (int *d) {
    static int acc = 0;

    acc += *d;
    *d = acc;
}
```

对于突发模式传输，Vitis HLS 会使用单一基址读取或写入数据，后接多次顺序数据采样，因此，该模式支持更高的数据吞吐量。使用流水打拍的 `for` 循环时，可使用操作的突发模式。如需了解更多信息，请参阅 [AXI 突发传输](#)。

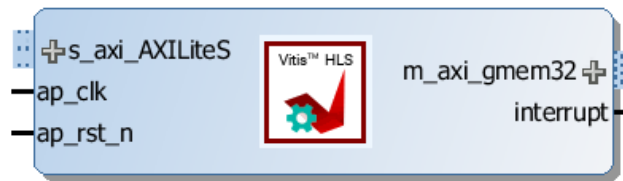


重要提示！ 由于 `memcpy` 无法内联或流水打拍，因此不鼓励使用。由于使用它时，会将实参的类型更改为 `char`，使用 `array_partition`、`array_reshape` 或结构体 `disaggregate` 时此更改可能导致出错，这就可能会导致各种问题。建议您改为自行编写 `memcpy`，利用显式阵列和循环来提供更好的控制。

对先前的示例进行综合时，会生成如下图所示接口。

注释：在此图中，AXI4 接口处于折叠状态。

图 33：AXI4 接口



使用 `for` 循环实现突发读取或写入时，请遵循如下要求：

- 对循环进行流水打拍
- 按升序访问地址
- 请勿将访问置于条件语句内
- 对于嵌套循环，请勿手动将循环平铺（使用 `LOOP_FLATTEN` 编译指示或指令），因为这样有碍突发操作

注释：在任一 `for` 循环内只允许 1 次读取和 1 次写入，除非端口捆绑在不同 AXI 端口内。

偏移和操作模式



重要提示！ 在 Vitis 内核流程中，默认操作模式为 `offset=slave` 且不应更改。在 Vivado IP 流程中，默认模式同样是 `offset=slave`，但可更改。

AXI4 主接口具有读/写地址通道，可用于读/写特定地址。默认情况下，`m_axi` 接口会从地址 `0x00000000` 启动所有读写操作。例如，给定以下代码时，设计会读取从地址 `0x00000000` 到 `0x000000C7`（50 个 32 位字，提供 200 字节）的数据，它表示 50 个地址值。随后，设计会将数据重新写入相同地址。

```
#include <stdio.h>
#include <string.h>

void example(int *a){

#pragma HLS INTERFACE mode=m_axi port=a depth=50

    int i;
    int buff[50];

    //The use of memcpy is discouraged because multiple calls of memcpy
    cannot be
    //pipelined and will be scheduled sequentially. Use a dedicated for loop
    instead
    //memcpy(buff,(const int*)a,50*sizeof(int));
    //Burst read input
    for(i=0; i<50; i++){
        buff[i] = a[i];
    }

    // Compute value
    for(i=0; i < 50; i++){
        buff[i] = buff[i] + 100;
    }
}
```

```
//Burst write output
for(i=0; i<50; i++){
    a[i] = buff[i];
}
}
```

该 HLS 编译器提供了如下功能：允许在 Vivado IP 中静态配置这些基址，或者由软件应用或其他 IP 在运行时进行动态配置。

m_axi 接口可作为发起传输事务的主接口，也可以作为接收数据并发送确认的从接口。根据随 INTERFACE 编译指示的 offset 选项指定的模式，HLS IP 可使用多种方法来设置基址。



提示：syn.interface.m_axi_offset 配置命令可以为偏移提供全局设置，对于特定 m_axi 接口，可使用 INTERFACE 编译指示 offset 选项来覆盖此全局设置。

- 主模式：使用不同 offset 选项充当主接口时，m_axi 接口起始地址可采用硬编码，或者也可在运行时设置。
 - offset=off：将 m_axi 接口的基址设为 0x00000000，在 Vivado IP integrator 中此设置不可更改。这种方法的缺点之一是您在运行时无无法更改基址。请参阅 [在 IP integrator 中自定义 AXI4 主接口](#) 以获取有关设置基址的信息。

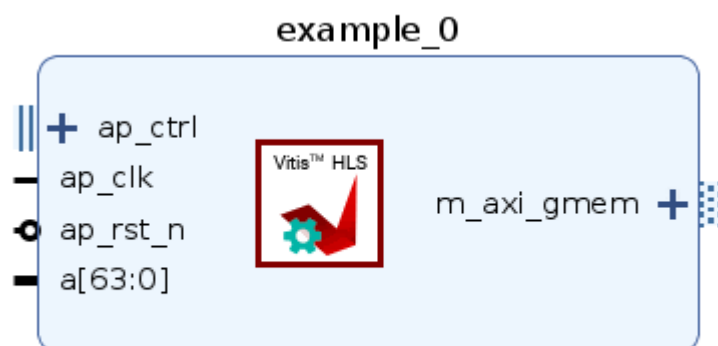
以下显示了为函数设置 offset=off 的 INTERFACE 编译指示。

```
void example(int *a){
#pragma HLS INTERFACE m_axi depth=50 port=a offset=off
...
}
```

- offset=direct：Vitis HLS 会在 IP 上生成端口用于设置地址。请注意在下图中添加的 a 端口。它支持您在运行时更新地址，这样即可通过同一个 m_axi 接口来读取和写入不同位置。例如，一个 HLS 模块用于将数据从 ADC 读取到 RAM 内，另一个 HLS 模块则用于处理此数据。由于您可更改模块上的地址，因此当其中一个 HLS 模块在处理初始数据集时，另一个模块则可将更多数据读取到其他地址内。

```
void example(int *a){
#pragma HLS INTERFACE m_axi depth=50 port=a offset=direct
...
}
```

图 34：offset=direct

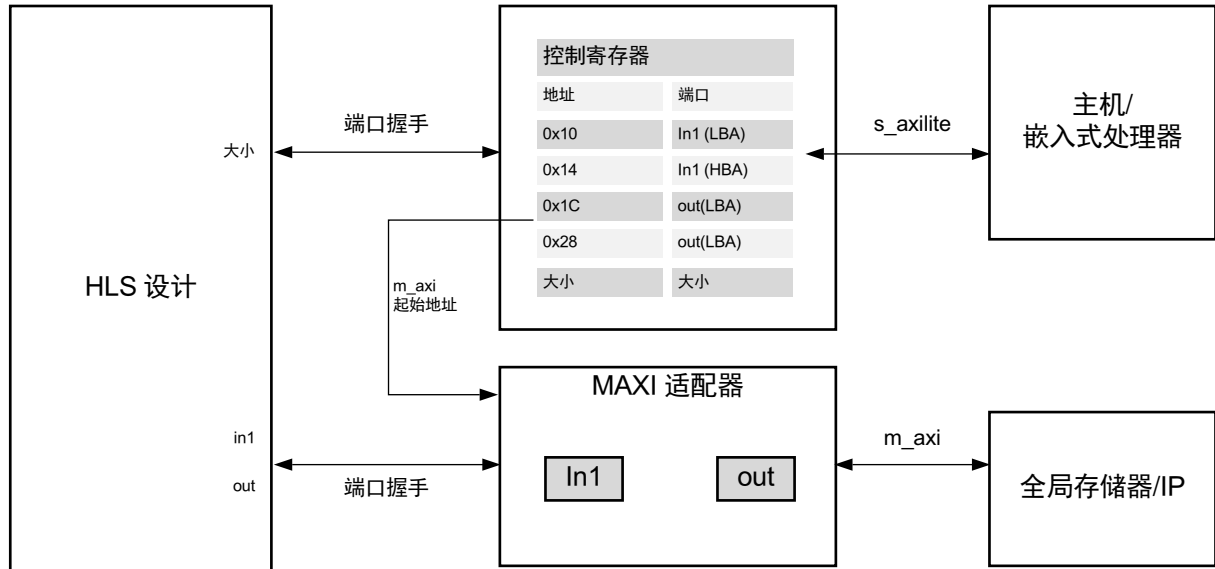


- 从模式：接口的从模式是使用 offset=slave 来设置的。在此模式下，IP 将由主机应用控制或者由微控制器通过 s_axilite 接口来控制。这是 Vitis 内核流程与 Vivado IP 流程的默认行为。以下是操作流程：

1. 初始情况下，主机/CPU 将使用块级控制协议启动 IP 或内核，此协议需映射到 `s_axilite` 适配器。
2. 主机将通过 `s_axilite` 适配器为 `m_axi` 接口发送地址偏移。
3. `m_axi` 适配器将从 `s_axilite` 适配器读取起始地址，并将其存储在队列中。
4. HLS 设计会开始从全局存储器读取数据。

如下图所示，HLS 设计将包含 `s_axilite` 适配器用于基址，也包含 `m_axi` 用于对全局存储器执行读写传输。

图 35：从模式下的 AXI 适配器



X26768-060822

偏移规则

以下是与 `offset` 选项关联的规则：

- 完全指定偏移：当用户显式设置偏移值时，该工具会使用指定的设置。用户还可在设计中为不同 `m_axi` 接口设置不同偏移值，工具将使用指定的偏移。

```
#pragma HLS INTERFACE s_axilite port=return
#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=out offset=direct
#pragma HLS INTERFACE mode=m_axi bundle=BUS_B port=in1 offset=slave
#pragma HLS INTERFACE mode=m_axi bundle=BUS_C port=in2 offset=off
```

- 不指定偏移：如果在 `INTERFACE` 编译指示中未指定任何偏移，那么该工具将遵从 `syn.interface.m_axi_offset` 指定的设置。

注释：如果指定了全局 `syn.interface.m_axi_offset` 设置，并且设计具有 `s_axilite` 接口，则会忽略全局设置，并假定 `offset=slave`。

```
void top(int *a) {
#pragma HLS interface mode=m_axi port=a
#pragma HLS interface mode=s_axilite port=a
}
```

M_AXI 捆绑

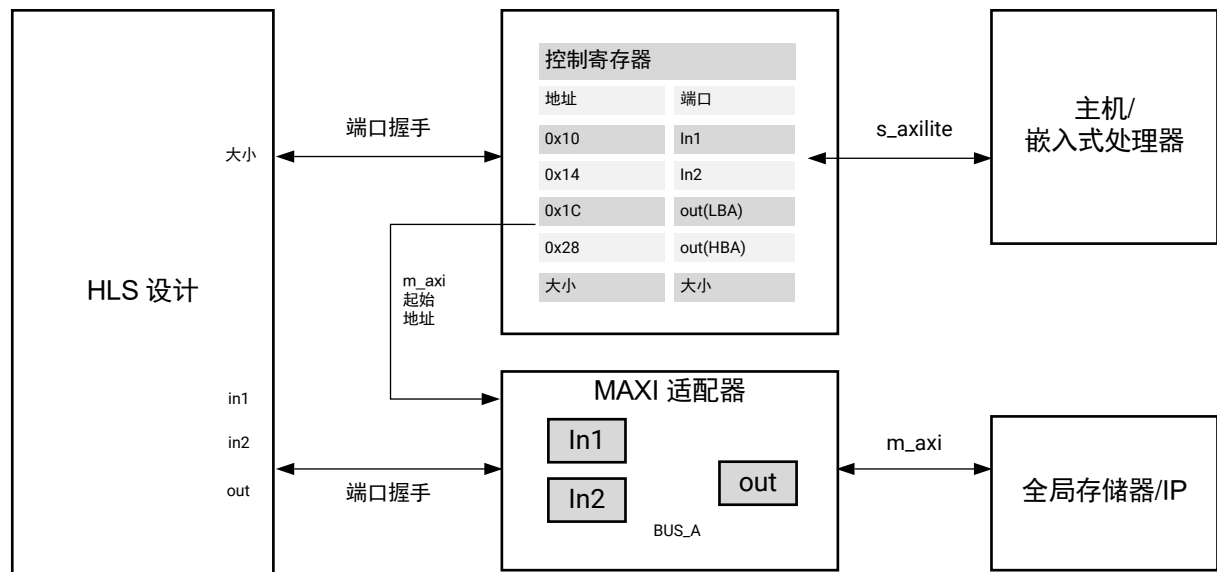
Vitis Unified IDE 和 `v++` 命令会将兼容选项的函数实参组合到单个 `m_axi` 接口适配器中。将端口捆绑到单个接口中有助于通过消除 AXI 逻辑来节省 FPGA 资源，但它可能限制内核性能，因为所有存储器传输都必须经过同一个端口。`m_axi` 接口具有独立的读取 (READ) 和写入 (WRITE) 通道，因此单个接口即可同时执行读取和写入，但仅限单一位置。使用多个捆绑会创建多个接口来连接到多个存储体，故而会导致内核带宽和吞吐量增加。

在以下示例中，所有指针实参都使用接口选项 `bundle=BUS_A` 组合到单个 `m_axi` 适配器中，并为 `m_axi` 偏移、标量实参 `size` 和函数返回添加单个 `s_axilite` 适配器。

```
extern "C" {
void vadd(const unsigned int *in1, // Read-Only Vector 1
          const unsigned int *in2, // Read-Only Vector 2
          unsigned int *out,        // Output Result
          int size                  // Size in integer
        ) {

#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=out
#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=in1
#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=in2
#pragma HLS INTERFACE mode=s_axilite port=in1
#pragma HLS INTERFACE mode=s_axilite port=in2
#pragma HLS INTERFACE mode=s_axilite port=out
#pragma HLS INTERFACE mode=s_axilite port=size
#pragma HLS INTERFACE mode=s_axilite port=return
}
```

图 36: MAXI 和 S_AXILITE



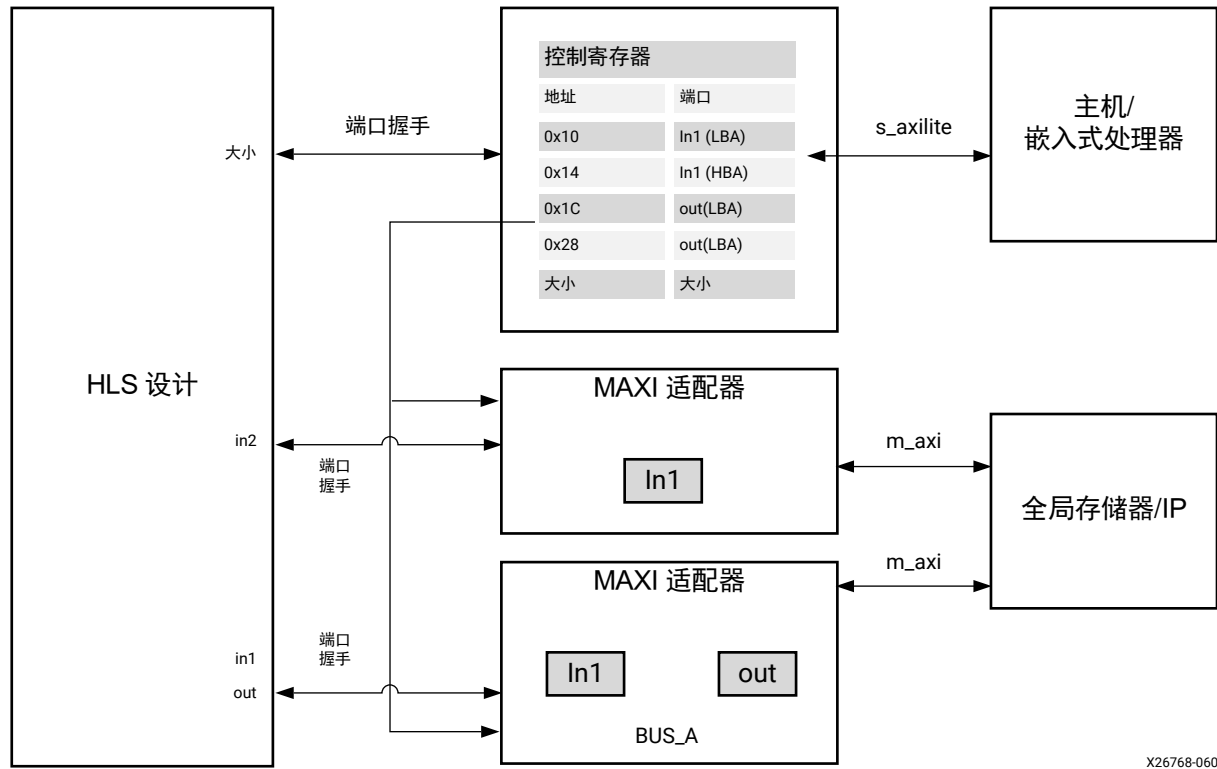
X26771-060722

您还可选择将函数实参捆绑到独立接口适配器内，如以下代码所示。此处实参 `in2` 通过 `bundle=BUS_B` 组合到独立的接口适配器内。这样会为端口 `in2` 创建新的 `m_axi` 接口适配器。

```
extern "C" {
void vadd(const unsigned int *in1, // Read-Only Vector 1
          const unsigned int *in2, // Read-Only Vector 2
          unsigned int *out,        // Output Result
          int size                  // Size in integer
        ) {
}
```

```
#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=out
#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=in1
#pragma HLS INTERFACE mode=m_axi bundle=BUS_B port=in2
#pragma HLS INTERFACE mode=s_axilite port=in1
#pragma HLS INTERFACE mode=s_axilite port=in2
#pragma HLS INTERFACE mode=s_axilite port=out
#pragma HLS INTERFACE mode=s_axilite port=size
#pragma HLS INTERFACE mode=s_axilite port=return
```

图 37：2 个 MAXI 捆绑



X26768-060722

捆绑规则

全局配置命令 `syn.interface.m_axi_auto_max_ports=false` 将把接口捆绑数量限制到所需的最小值。它允许工具将兼容的端口组合到单个 `m_axi` 接口内。此命令默认设置为禁用 (`false`)，但可通过为每个端口创建独立的 `m_axi` 适配器来将其启用，从而最大化带宽。

禁用 `m_axi_auto_max_ports` 时，对于该工具在不同环境下处理捆绑的方式存在相应的规则，包括但不限于：

1. **默认捆绑名称：**该工具将不含捆绑名称的所有接口端口都组合到单个 `m_axi` 接口端口内，并使用工具默认名称 `bundle=<default>`，将 RTL 端口命名为 `m_axi-<default>`。以下编译指示：

```
#pragma HLS INTERFACE mode=m_axi port=a depth=50
#pragma HLS INTERFACE mode=m_axi port=a depth=50
#pragma HLS INTERFACE mode=m_axi port=a depth=50
```

会生成以下消息：

```
INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to 'm_axi'.
INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to 'm_axi'.
INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to 'm_axi'.
```

2. 用户指定的捆绑名称：该工具会将具有相同用户指定的 `bundle=<string>` 的所有接口端口都组合到同一个 `m_axi` 接口端口中，并以 `m_axi_<string>` 指定的值来命名 RTL 端口。未分配 `bundle` 的端口则组合到默认捆绑中，如上所述。以下编译指示：

```
#pragma HLS INTERFACE mode=m_axi port=a depth=50 bundle=BUS_A
#pragma HLS INTERFACE mode=m_axi port=b depth=50
#pragma HLS INTERFACE mode=m_axi port=c depth=50
```

会生成以下消息：

```
INFO: [RTGEN 206-500] Setting interface mode on port 'example/BUS_A' to 'm_axi'.
INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to 'm_axi'.
INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to 'm_axi'.
```

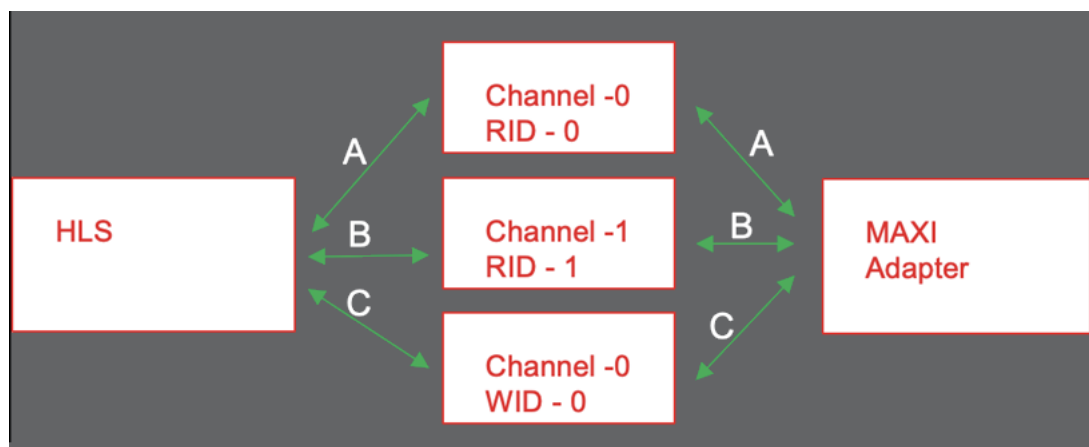


重要提示！ 如果对不兼容的接口进行捆绑，那么该工具会发出消息并忽略捆绑分配。

M_AXI 通道

M_AXI 通道会为映射到单个 AXI 接口的每个指针实参实现一条单独的通道，而无需单独的适配器。

图 38：Maxi 适配器



这具有如下优势：

- 内核使用的 M_AXI 适配器更少，耗用的硬件资源也更少。
- 在数据流区域内部可以使用映射到单个 AXI 接口的多个指针实参。
- 为每个指针实参使用唯一的 AXI ID 能够支持突发交织，从而提升 AXI 总线带宽的使用率。

在您的设计中可通过 2 种方法来启用该功能特性：

- 在 `m_axi` 接口上使用 `syn.interface.m_axi_auto_id_channel=true` 配置命令全局启用该功能特性，如 [接口配置](#) 中所述。启用该功能特性时，HLS 工具会将通道自动添加到 `m_axi` 适配器。
- 在特定 `m_axi` 接口上使用 `INTERFACE` 编译指示或指令的 `channel` 选项来启用该功能特性，如 [syn.directive.interface](#) 中所述。

从架构角度来看，每个 MAXI 捆绑都分为两部分：

- 每个通道每个方向一个负载单元和/或一个存储单元 (LSU)，例如，如果输入指针与输出指针具有相同的通道 ID，那么您即可得到一个负载单元和一个存储单元。
 - 如果您不使用通道选项，那么映射到该捆绑的所有 MAXI 顶层指针只有一个负载单元和一个存储单元。
 - 该 LSU 包含高速缓存（如果已启用）。此图引用的是含高速缓存的 LSU。若无高速缓存，就没有标记存储器，当然也就没有有效的存储器。
 - 如果多个顶层指针映射到同一个通道（或映射到默认的单一通道），那么所有这些指针都必须有相同的高速缓存配置，或者所有指针都没有高速缓存。
 - LSU 包含数据缓冲器和地址缓冲器，支持执行内核侧突发，执行次数取决于用户配置或者 FE 自动推断所得的突发长度和暂挂请求数量。它是适配器中 BRAM 最密集的部分。
 - 每个 LSU 都有自己的 AXI ID，因此不同 LSU 可生成交叉式突发，如果内核并发访问映射到同一个捆绑的多个顶层指针，那么通道越多，暗示整体吞吐量越好。
- 一个读适配器，一个写适配器。
 - 这些适配器不会缓冲数据，而是缓冲暂挂请求，直至返回响应为止。
 - 它们会将内核生成的潜在超长突发细分为满足 AXI 强制要求的突发长度限制的若干区块。

M_AXI 资源

AXI 主适配器用于将自定义的 AXI 命令从 HLS 调度器转换为标准 AXI AMBA® 协议，并将其发送至外部存储器。MAXI 适配器使用 FIFO 等资源来存储请求/数据和确认 (ack)。以下提供了它所耗用的模块和资源综述：

- 写入模块：总线写入模块负责执行写入操作。
 - `FIFO_wreq`：此 FIFO 模块负责存储未来写入请求。当 AW 通道可用时，将从该 FIFO 跳出针对全局存储器的新写入请求。
 - `buff_wdata`：此 FIFO 负责存储未来写入数据，这些数据需发送至全局存储器。当 W 通道可用并且满足 AXI 协议条件时，将从该 FIFO 跳出大小与 `burst_length` 相等的写入数据，并发送至全局存储器。
 - `FIFO_resp`：该模块负责控制向模块发送的流水线写入响应的数量。
- 读取模块：这些模块负责执行读取操作。它使用以下资源
 - `FIFO_rreq`：此 FIFO 模块负责存储未来写入请求。当 AR 通道空闲时，将从该 FIFO 跳出针对全局存储器的读取请求。
 - `buff_rdata`：此 FIFO 负责存储从全局存储器接收到的读取数据。

M_AXI 适配器的器件资源耗用量是所有写入模块（`FIFO_wreq module` 模块、`buff_wdata`、和 `FIFO_resp` 的大小）总和与所有读取模块总和相加所得。总之，FIFO 大小的计算方式为“宽度 * 深度”。当您引用 1 KB FIFO 存储器时，它可采用 32*32、8*64 等任一配置，具体配置根据设计规格来选择。同样，可使用以下配置命令来为设计全局配置该适配器的 FIFO 存储器：

- `syn.interface.m_axi_latency`
- `syn.interface.m_axi_max_read_burst_length/`
`syn.interface.m_axi_max_write_burst_length`

- `syn.interface.m_axi_num_read_outstanding/syn.interface.m_axi_num_write_outstanding`
- `syn.interface.m_axi_addr64`



提示：您还可使用 [INTERFACE](#) 编译指示或指令上的类似选项来配置特定 `m_axi` 接口。

这些配置选项用于控制 FIFO 的宽度和深度，如下所示。

- `FIFO_wreq/rreq module 大小 = (width( syn.interface.m_axi_addr64)) * Depth( syn.interface.m_axi_latency )`。Vivado 工具会将此 FIFO 作为移位寄存器来实现。
- `buff_wdata/buff_rdata 模块大小 = (HLS 综合后的端口宽度 * 深度 (syn.interface.m_axi_num_write_outstanding * syn.interface.m_axi_max_write_burst_length))`。



提示：默认情况下，此 FIFO 将作为块 RAM 来实现，但可在 LUTRAM 或 URAM（由 `syn.interface.m_axi_buffer_impl` 判定）中实现。

- `FIFO_resp 模块大小 = 深度 (syn.interface.m_axi_num_write_outstanding)`。

控制 AXI4 突发行为

当设计在等待访问总线时，从不会在最优 AXI4 接口上停滞，授予总线访问后，总线等待设计执行读/写时从不停滞。为创建优化 AXI4 接口，在 [INTERFACE](#) 编译指示或指令中提供了以下选项以指定突发访问行为，并优化 AXI4 接口的效率。如需了解有关突发传输的更多信息，请参阅 [AXI 突发传输](#)。

在某些情况下，如果自动突发访问失败，那么有效的解决方案是重写代码或者使用手动突发，如 [使用手动突发](#) 中所述。如果这些方法无效，那么另一个解决方案可能是在 AXI4 接口中通过 [CACHE](#) 编译指示或指令来使用高速缓存存储器。



重要提示！变量上的 `volatile` 限定符会阻止突发访问对其进行读取或写入。

其中部分选项使用内部存储空间来缓冲数据，并且可能影响面积和资源：

- `latency`：指定 AXI4 接口的期望时延，允许设计发起总线请求的时间比执行期望的读取或写入操作早数个周期（时延）。如果该值太低，设计将过早达成就绪状态，可能停滞并等待总线；如果该值太高，则可能授予总线访问权时，总线仍处于停滞状态并等待设计发起访问。
- `max_read_burst_length`：指定突发传输期间读取的数据值的最大数量。
- `num_read_outstanding`：指定在设计停滞前可对 AXI4 总线发出的读取请求的数量（无响应）。此操作暗示设计中的内部存储空间，即 FIFO 大小为：`num_read_outstanding*max_read_burst_length*word_size`。
- `max_write_burst_length`：指定突发传输期间写入的数据值的最大数量。
- `num_write_outstanding`：指定在设计停滞前可对 AXI4 总线发出的写入请求的数量（无响应）。此操作暗示设计中的内部存储空间，即 FIFO 大小为：`num_write_outstanding*max_write_burst_length*word_size`。

以下示例可用于帮助解释这些选项：

```
#pragma HLS interface mode=m_axi port=input offset=slave bundle=gmem0
depth=1024*1024*16/(512/8)
latency=100
num_read_outstanding=32
num_write_outstanding=32
max_read_burst_length=16
max_write_burst_length=16
```

接口时延指定为 100。Vitis HLS 希望将突发访问请求发生时间调度为比设计准备好访问 AXI4 总线的时间早 100 个时钟周期。为了进一步提高总线效率，`num_write_outstanding` 和 `num_read_outstanding` 选项可用于确保设计所含缓冲足以存储最多 32 次读写突发。这样即可允许设计持续处理，直至处理总线请求为止。最后，`max_read_burst_length` 和 `max_write_burst_length` 选项可确保最大突发量为 16，并且 AXI4 接口暂存总线的时间不超过该设置。

这些选项允许对 AXI4 接口行为进行优化以满足运行该接口的系统的要求。操作效率取决于是否准确设置这些值。

端口宽度自动调整

在 Vitis 工具流程中，Vitis HLS 能够自动将 `m_axi` 接口端口的大小调整为 512 位，以改善突发访问能力。但是，端口宽度自动调整仅支持标准 C 语言数据类型和 2 的幂值 `struct` 大小，其中对指针分配拓宽后期望的字节大小。如果该工具无法自动拓宽此端口，那么您可通过使用矢量或任意精度 (AP) 作为该端口的数据类型来手动更改端口宽度。



重要提示！ 接口上的结构体会阻止端口的自动拓宽操作。您必须将结构体拆分为多个独立元素才能启用该功能。

Vitis HLS 使用以下 2 条命令来自动调整端口宽度：

- `syn.interface.m_axi_max_widen_bitwidth=<N>`：指令该工具将 M-AXI 接口上的突发自动拓宽至指定的位宽。<N> 的值必须为 2 的幂（介于 0 到 1024 之间）。
- `syn.interface.m_axi_alignment_byte_size=<N>`：请注意，突发拓宽还需要强大的对齐属性。假定映射到 `m_axi` 接口的指针至少对齐到所提供的宽度（以字节为单位，值为 2 的幂）。这样有助于自动进行突发拓宽。

在 Vitis 内核流程中，默认通过以下命令启用端口宽度自动调整功能：

```
syn.interface.m_axi_max_widen_bitwidth=512
syn.interface.m_axi_alignment_byte_size=64
```

在 Vivado IP 流程中，默认禁用此功能：

```
syn.interface.m_axi_max_widen_bitwidth=0
syn.interface.m_axi_alignment_byte_size=1
```

仅当工具可看到突发访问时，端口宽度自动调整功能才能调整端口大小。因此，突发所需的所有前置条件（如 [AXI 突发传输](#) 中所述）都必须得到满足后才能调整端口大小。这些前置条件包括：

- 必须按单调递增访问顺序（访问的存储器位置和访问时间两方面都是如此）。不能访问位于先前访问的两处存储器位置之间的存储器位置，即没有重叠。
- 从全局存储器进行访问时使用的模式应采用顺序访问模式，并具有下列附加要求：
 - 顺序访问需在原语类型或非矢量的 2 的幂大小的聚合类型上进行
 - 顺序访问的起始位置需对齐到拓宽后的字大小
 - 顺序访问的长度需被拓宽因子整除

以下代码示例在后续计算过程中使用：

```
vadd_pipeline:
  for (int i = 0; i < iterations; i++) {
#pragma HLS LOOP_TRIPCOUNT min = c_len/c_n max = c_len/c_n

    // Pipelining loops that access only one variable is the ideal way to
    // increase the global memory bandwidth.
    read_a:
    for (int x = 0; x < N; ++x) {
```

```
#pragma HLS LOOP_TRIPCOUNT min = c_n max = c_n
#pragma HLS PIPELINE II = 1
    result[x] = a[i * N + x];
}

read_b:
    for (int x = 0; x < N; ++x) {
#pragma HLS LOOP_TRIPCOUNT min = c_n max = c_n
#pragma HLS PIPELINE II = 1
        result[x] += b[i * N + x];
    }

write_c:
    for (int x = 0; x < N; ++x) {
#pragma HLS LOOP_TRIPCOUNT min = c_n max = c_n
#pragma HLS PIPELINE II = 1
        c[i * N + x] = result[x];
    }
}
}
```

上述代码的自动优化宽度需通过 3 个步骤来执行：

1. 该工具检查 read_a 循环中的访问模式数量。每个循环迭代期间有一次访问，因此优化判定接口位宽为 32= 32 *1（整数变量位宽 * 访问次数）。
2. 该工具会尝试使用以下表达式来达到 config_interface -m_axi-max-widen-bitwidth 512 指定的默认最大值：

```
length = (ceil((loop-bound of index inner loops) *
(loop-bound of index - outer loops)) * #(of access-patterns))
```

- 在上述代码中，该工具支持非完美循环嵌套。如果 a 和 b 捆绑到相同端口，那么由于冲突，该工具将不会在 a 和 b 上扩展突发。如果 a 和 b 捆绑到不同端口，那么该工具将在 a 和 b 上将突发扩展至外层循环。因此，该公式将缩短为：

```
length = (ceil((loop-bound of index inner loops)) * #(of access-
patterns))
```

或者：length = ceil(128) * 32 = 4096

3. 计算所得是否为 2 的幂？如果是，那么长度将限为 syn.interface.m_axi-max-widen-bitwidth 所指定的宽度。

使用端口宽度自动调整功能有利有弊，您需在使用时审慎考量。此功能特性改善的是访问吞吐量，而不是数据类型大小。它还会添加更多必要资源，以缓冲巨型矢量，并且可能需要将数据相应转换为数据路径大小。

创建含 32 位地址的 AXI4 接口

默认情况下，Vitis HLS 会实现含 64 位地址总线的 AXI4 端口。但在某些器件（如 Zynq 7000）上，存在 32 位地址总线。在此情况下，您可通过设置 syn.interface.m_axi_addr64=0 配置命令禁用 64 位地址总线，以实现含 32 位地址总线的 AXI4 接口。



重要提示！ 如果禁用 syn.interface.m_axi_addr64 选项，那么 HLS 工具会在含 32 位地址总线的设计中实现所有 AXI4 接口。

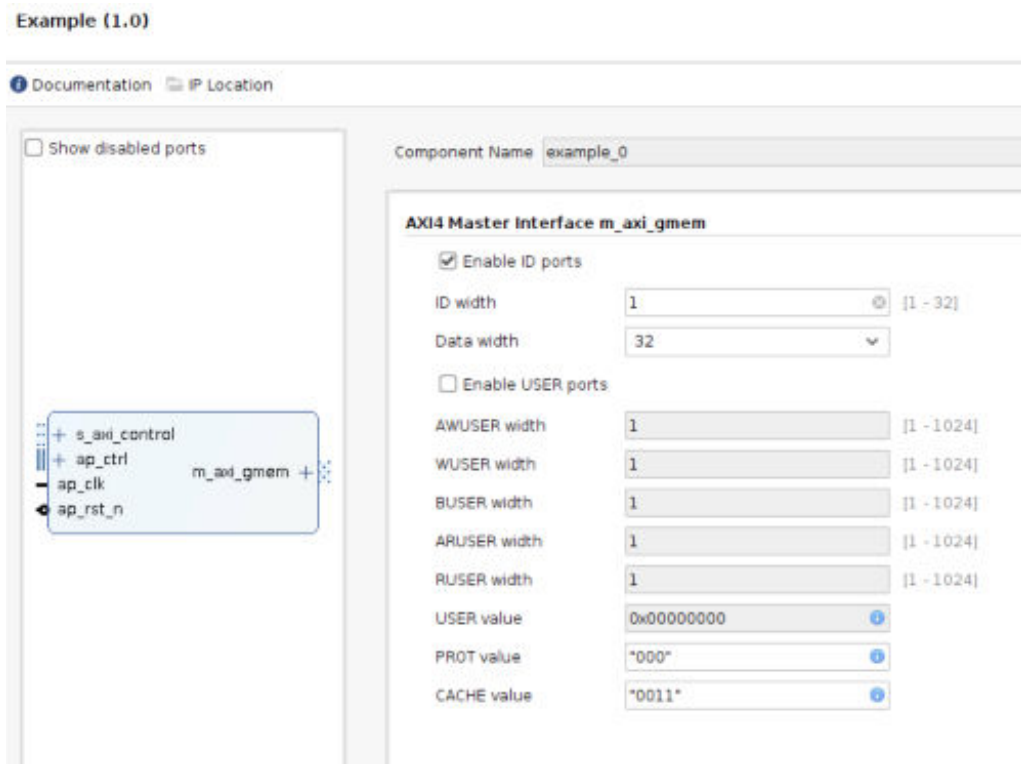
Vitis HLS

在 IP integrator 中自定义 AXI4 主接口

将使用 AXI4 主接口的 HLS RTL 设计整合到 Vivado IP integrator 中的设计中时，可以自定义块。在 IP integrator 模块框图中，选择 HLS 块、右键单击，然后选择“Customize Block”（自定义块）以自定义所提供的任何设置。如需获取 AXI4 参数的完整描述，请参阅《Vivado Design Suite：AXI 参考指南》(UG1037)。

下图显示了如下所示设计的“Re-Customize IP”（重新自定义 IP）对话框。此设计包含 1 个 AXI4-Lite 端口。

图 39：在 IP integrator 中自定义 AXI4 主接口



AXI4-Lite 接口

概述

主机应用或嵌入式处理器可使用 AXI4-Lite 从接口 (s_axilite) 对 HLS IP 或内核进行控制，该接口充当系统总线，用于处理器与内核之间通信。主机或嵌入式处理器可使用 s_axilite 接口启动和停滞内核，以及对内核进行数据读取或写入。当 Vitis HLS 对设计进行综合时，s_axilite 接口是作为适配器来实现的，用于捕获来自适配器上的寄存器中的主机的通信信息。请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Interface/Register](#)，以获取其中部分概念的示例。

AXI4-Lite 接口在 Vivado IP 或 Vitis 内核中执行多项功能：

- 它可映射用于启动和停滞内核的块级控制机制。
- 它可提供通道，用于将 m_axi 接口的标量实参、指向标量值的指针、函数返回值和地址偏移从主机传递到 IP 或内核
- 对于 Vitis 内核流程：

- 该工具将自动推断 `s_axilite` 接口编译指示，以向分配给 `m_axi` 接口的指针实参、标量值和函数返回类型提供偏移。
- 如果指向标量值的指针分配给 `s_axilite` 接口，那么 Vitis HLS 允许您在该指针上执行读取或写入。指针默认分配给 `m_axi` 接口，因此您必须使用 `INTERFACE` 编译指示或指令将该指针手动分配给 `s_axilite`：

```
int top(int *a, int *b) {
#pragma HLS interface s_axilite port=a
```

- Bundle:** 在 Vitis 内核流程中，针对 `s_axilite` 适配器，请勿指定 `bundle` 选项。该工具将创建单个 `s_axilite` 接口，用于服务整个设计。



重要提示！ 如果针对 Vitis 内核流程指定了多个捆绑，HLS 将返回错误。

- Offset:** 该工具将自动选择接口的偏移。请勿在此流程中指定任何偏移。
- 对于 Vivado IP 流程：
 - 默认情况下，该流程将不使用 `s_axilite` 接口。
 - 要使用 `s_axilite` 作为标量实参、指向标量值的指针、`m_axi` 指针地址的偏移和函数返回类型的通信通道，必须手动指定 `INTERFACE` 编译指示或指令。
 - Bundle:** 此流程支持按捆绑指定多个 `s_axilite` 接口。如需了解更多信息，请参阅 [S_AXILITE 捆绑规则](#)。
 - Offset:** 默认情况下，该工具将在控制寄存器映射内按顺序从 0x10 开始布局实参。如需了解更多详情，请参阅 [S_AXILITE 偏移选项](#)。

S_AXILITE 示例

以下示例演示了 Vitis HLS 将多个实参（包括函数返回）作为 `s_axilite` 接口来实现的方式。由于针对 `bundle` 选项每条编译指示都使用相同名称，因此每个端口都分组到单个接口中。

```
void example(char *a, char *b, char *c)
{
#pragma HLS INTERFACE mode=s_axilite port=return bundle=BUS_A
#pragma HLS INTERFACE mode=s_axilite port=a bundle=BUS_A
#pragma HLS INTERFACE mode=s_axilite port=b bundle=BUS_A
#pragma HLS INTERFACE mode=s_axilite port=c bundle=BUS_A
#pragma HLS INTERFACE mode=ap_vld port=b

  *c += *a + *b;
}
```

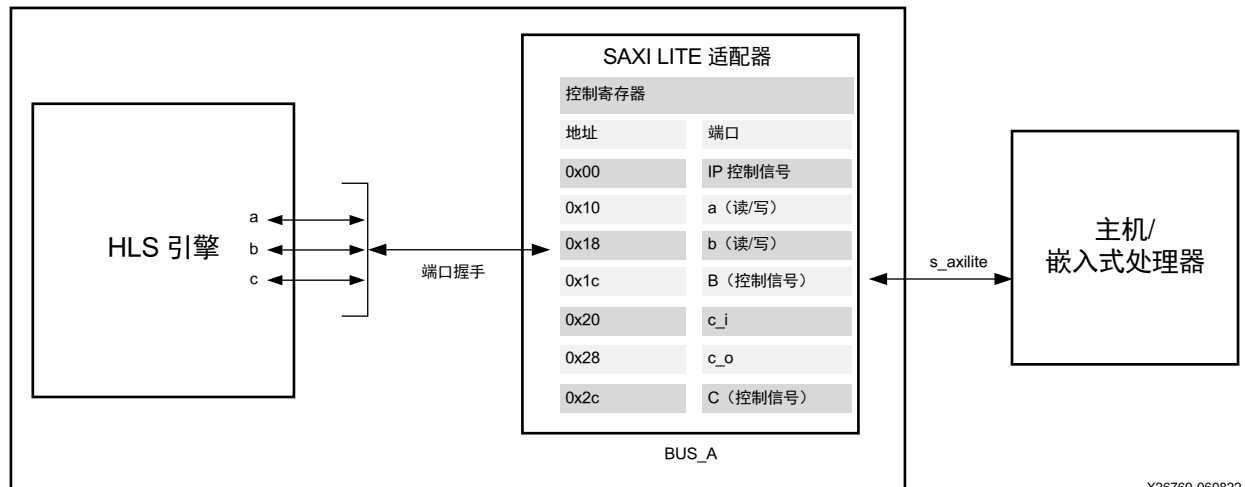


提示： 如果不指定 `bundle` 选项，Vitis HLS 会将所有实参组合到同一个 `s_axilite` 捆绑中，并对该端口进行自动命名。

综合后的示例将作为系统的一部分，该系统具有 3 个重要元素，如下图所示：

- 与 IP 或内核交互的 x86 或嵌入式处理器上运行的主机应用
- SAXI Lite Adapter：INTERFACE 编译指示用于实现 `s_axilite` 适配器。该适配器具有 2 个主要函数：用于实现接口协议以便与主机进行通信，并向 IP 或内核提供控制寄存器映射。
- 实现设计逻辑的 HLS 引擎或函数

图 40: S_AXILITE 适配器



X26769-060822

默认情况下，Vitis HLS 会为分组到单个 `s_axilite` 接口的每个端口分配地址。分配给端口的大小或地址范围取决于使用的实参数据类型和端口协议，如下所述。您也可以使用 `offset` 选项来显式定义地址，如 [S_AXILITE 偏移选项](#) 中所述。

- 端口 a：默认情况下作为 `ap_none` 来实现。为数据信号分配 1 个字，并且仅使用 3 个位来进行寻址，因为实参数据类型为 `char`。剩余位保留不用。
- 端口 b：作为 `ap_vld` 来实现，由 `INTERFACE` 编译指示定义，如示例中所示。对应的控制寄存器大小为 2 字节（16 位），并分为 2 节，如下所示：
 - (0x1c) 控制信号：为控制信号分配 1 个字。
 - (0x18) 数据信号：为数据信号地址分配 1 个字，并且仅使用 3 个位，因为实参数据类型为 `char`。剩余位保留不用。
- 端口 c：默认情况下，作为 `ap_ovld` 来实现（作为输出）。对应的控制寄存器大小为 4 字节（32 位），并分为 3 节：
 - (0x20) `c_i` 的数据信号：为输入数据信号分配 1 个字，并且因为实参数据类型为 `char`，仅使用 3 个位进行寻址，剩余位保留不用。
 - (0x24) 保留空间
 - (0x28) `c_o` 的数据信号：为输出数据信号分配 1 个字。
 - (0x2c) `c_o` 的控制信号：为控制信号 `ap_ovld` 分配 1 个字，并且因为实参数据类型为 `char`，所以仅使用 3 个位进行寻址。剩余位保留不用。

在操作中，主机应用将通过写入控制地址空间 (0x00) 来初始启动内核。主机/CPU 通过写入其他地址空间来完成初始设置，这些地址空间与示例中定义的各函数实参相关联。

端口 b 的控制信号将断言有效，随后内核才能读取端口 a 和 b（端口 a 为 `ap_none` 且不含控制信号）。在此之前，设计处于停滞状态并等待端口 b 的 `valid` 寄存器置位。每次 HLS 引擎准备好端口 b 后，输入 `valid` 寄存器就会清零，并且寄存器会复位为逻辑 0。

当 HLS 引擎完成计算后，端口 c 上的输出值将存储在控制寄存器内，对应的 `valid` 位将置位，以供主机读取。主机读取数据后，HLS 引擎会在控制寄存器 (0x00) 内写入 `ap_done` 位，以标记 IP 计算结束。

Vitis HLS 会在 [S_AXILITE 控制寄存器映射](#) 中报告分配的地址，并在 [C 语言驱动程序文件](#) 中提供这些地址以辅助您进行软件开发。您可使用 `s_axilite` 接口以利用 C 语言驱动程序文件，通过使用提供的应用编程接口 (API) 函数，可将这些文件搭配嵌入式或 x86 处理器上运行的代码来一起使用，以便您通过自己的软件来控制硬件。

S_AXILITE 控制寄存器映射

Vitis Unified IDE 和 `v++` 命令会自动生成控制寄存器映射 (Control Register Map) 用于控制 Vivado IP 或 Vitis 内核以及组合到 `s_axilite` 接口中的各端口。此寄存器映射会被添加到生成的 RTL 文件中，并且可拆分为两部分：

- 1. 块级控制信号
- 2. 映射到 `s_axilite` 接口内的函数实参

在 Vitis 内核流程中，默认块协议是 `ap_ctrl_chain` 并分配至 `s_axilite` 接口。默认设置不应更改。

但在 Vivado IP 流程中，默认块控制协议是 `ap_ctrl_hs` 并分配至其自己的接口，如 [Vivado IP 流程接口](#) 中所示。如果您在自己的 IP 内使用的是 `s_axilite` 接口，那么也可以使用以下 `INTERFACE` 编译指示来向该接口分配块控制协议，如以下示例所示：

```
#pragma HLS INTERFACE mode=s_axilite port=return
```

要更改块控制协议，也可以使用 `INTERFACE` 编译指示或指令，如下所示：

```
#pragma HLS INTERFACE mode=ap_ctrl_chain port=return
```

在 `s_axilite` 接口的控制寄存器映射中，Vitis HLS 会保留地址 `0x00` 到 `0x18` 用于顶层协议控制、中断控制、邮箱控制和自动重启控制。仅当启用带计数的自动重启和邮箱时，后一种情况才存在，如下所示：

表 10：地址

地址	描述
0x00	控制信号
0x04	全局中断使能寄存器
0x08	IP 中断使能寄存器（读/写）
0x0c	IP 中断状态寄存器（读取/TOW）
0x10	自动重启计数器（写入；仅当有带计数的自动重启时才存在）
0x14	输入邮箱写入（读/写；仅当启用输入邮箱时才存在）
0x18	输出邮箱读取（读/写；仅当启用输出邮箱时才存在）

控制信号 (0X00) 包含 `ap_start`、`ap_done`、`ap_ready` 和 `ap_idle`；对于 `ap_ctrl_chain`，块协议还包含 `ap_continue`。这些均为块级接口信号，通过 `s_axilite` 适配器来访问。

要开始块操作，控制寄存器中的 `ap_start` 位必须设置为 1。随后，HLS 引擎将继续从 AXI4-Lite 从接口中的寄存器读取已分组到该接口中的任意输入。

当此块完成操作后，`ap_done`、`ap_idle` 和 `ap_ready` 寄存器将由硬件输出端口进行设置，并由相应的寄存器读取已分组到 `s_axilite` 接口中的任何输出端口的结果。

对于函数实参，本工具会自动为分配给 `s_axilite` 接口的每个实参或端口分配地址。该工具将为每个端口分配一个偏移，从 `0x10` 开始，低位地址则保留用于控制信号。分配给端口的大小或地址范围取决于使用的实参数据类型和端口协议。

由于已组合到 AXI4-Lite 接口中的变量均为函数实参，在 C 语言代码中不含默认值，因此无法为 `s_axilite` 接口中的任何实参寄存器赋予默认值。这些寄存器可通过使用 `config_rtl` 命令进行复位来实现，但不能为其赋予任何其他默认值。

以下提供了 Vitis HLS 为 `ap_ctrl_chain` 块协议生成的控制寄存器映射：

```
...
//-----Address Info-----
// 0x00 : Control signals
//      bit 0 - ap_start (Read/Write/COH)
//      bit 1 - ap_done (Read)
//      bit 2 - ap_idle (Read)
//      bit 3 - ap_ready (Read/COR)
//      bit 4 - ap_continue (Read/Write/SC)
//      bit 7 - auto_restart (Read/Write)
//      bit 9 - interrupt (Read)
//      others - reserved
// 0x04 : Global Interrupt Enable Register
//      bit 0 - Global Interrupt Enable (Read/Write)
//      others - reserved
// 0x08 : IP Interrupt Enable Register (Read/Write)
//      bit 0 - enable ap_done interrupt (Read/Write)
//      bit 1 - enable ap_ready interrupt (Read/Write)
//      others - reserved
// 0x0c : IP Interrupt Status Register (Read/TOW)
//      bit 0 - ap_done (Read/TOW)
//      bit 1 - ap_ready (Read/TOW)
//      others - reserved
// 0x10 : Data signal of a
//      bit 7~0 - a[7:0] (Read/Write)
//      others - reserved
// 0x14 : reserved
// 0x18 : Data signal of b
//      bit 7~0 - b[7:0] (Read/Write)
//      others - reserved
// 0x1c : reserved
// 0x20 : Data signal of c_i
//      bit 7~0 - c_i[7:0] (Read/Write)
//      others - reserved
// 0x24 : reserved
// 0x28 : Data signal of c_o
//      bit 7~0 - c_o[7:0] (Read)
//      others - reserved
// 0x2c : reserved
// (SC = Self Clear, COR = Clear on Read, TOW = Toggle on Write, COH =
// Clear on Handshake)
...
```

S_AXILITE 和端口级协议

端口级 I/O 协议将来自 `s_axilite` 适配器的数据按顺序输入输出 HLS 引擎，如 [S_AXILITE 示例](#) 中所示。在 Vivado IP 流程中，您可向捆绑到 `s_axilite` 接口的个别端口和信号分配端口级 I/O 协议。在 Vitis 内核流程中，如非必要，否则不建议更改默认端口级 I/O 协议。该工具会根据与端口关联的实参的类型和方向来向端口分配默认端口协议。端口可包含以下一个或多个对象：

- 实参的数据信号
- 有效信号 (`ap_vld/ap_ovld`)，用于指示何时数据可供读取
- 确认信号 (`ap_ack`)，用于指示何时已读取数据

各实参类型的默认端口协议分配如下：

表 11：受支持的实参类型

实参类型	默认	受支持
标量	ap_none	也可使用 ap_ack 和 ap_vld
指针/参考		
输入	ap_none	ap_ack 和 ap_vld
输出	ap_vld	也可使用 ap_none、ap_ack 和 ap_ovld
输入输出	ap_ovld	也支持 ap_none、ap_ack 和 ap_vld

 **重要提示！** 阵列默认采用 ap_memory。

S_AXILITE 示例 将端口 b 组合到 s_axilite 接口内并将端口 b 指定为搭配 INTERFACE 编译指示使用 ap_vld 协议。由此导致，s_axilite 适配器包含用于端口 b 的数据的寄存器和用于端口 b 的输入有效信号的寄存器。

如果输入有效寄存器未设置为逻辑 1，那么 b 数据寄存器中的数据将不会被视为有效，设计将停滞并等待有效寄存器完成置位。每次读取端口 b 时，Vitis HLS 都会自动清除输入有效寄存器，并将该寄存器复位为逻辑 0。

 **建议：**为简化设计操作，AMD 建议您使用与 s_axilite 接口关联的默认端口协议。

S_AXILITE 捆绑规则

在 **S_AXILITE 示例** 中，所有函数实参均组合到单一 s_axilite 接口适配器内，该接口适配器由 INTERFACE 编译指示中的 bundle=BUS_A 选项来指定。bundle 选项支持您直接将端口组合到同一个接口内。

在 Vitis 内核流程中，应仅有单个接口捆绑，通常工具将其命名为 s_axi_control。因此，您在该流程中不应指定 bundle 选项，否则综合期间可能会出错。但是，在 Vivado IP 流程中，您可使用 s_axilite 接口来指定多个捆绑，这样将为您定义的每个捆绑创建一个独立的接口适配器。如以下示例所示：

```
void example(char *a, char *b, char *c)
{
  #pragma HLS INTERFACE mode=s_axilite port=a bundle=BUS_A
  #pragma HLS INTERFACE mode=s_axilite port=b bundle=BUS_A
  #pragma HLS INTERFACE mode=s_axilite port=c bundle=OUT
  #pragma HLS INTERFACE mode=s_axilite port=return bundle=BUS_A
  #pragma HLS INTERFACE mode=ap_vld port=b
    *c += *a + *b;
}
```

综合完成后，“Synthesis Summary”（综合汇总）报告会提供有关生成的 s_axilite 适配器数量的反馈。此报告的“SW-to-HW Mapping”（软件到硬件映射）部分包含硬件信息，可显示每个端口的控制寄存器偏移和地址范围。

但捆绑与 s_axilite 接口的搭配使用存在一些规则。

- 1. 默认捆绑名称：此规则将不含捆绑名称的所有接口端口都显式组合到同一个 AXI4-Lite 接口端口中，并使用工具默认捆绑名称，通常将 RTL 端口命名为 s_axi-<default>（一般采用 s_axi_control）。

在此示例中，所有端口均映射到默认捆绑：

```
void top(char *a, char *b, char *c)
{
    #pragma HLS INTERFACE mode=s_axilite port=a
    #pragma HLS INTERFACE mode=s_axilite port=b
    #pragma HLS INTERFACE mode=s_axilite port=c
    *c += *a + *b;
}
```

2. 用户指定的捆绑名称：此规则将具有相同 `bundle` 名称的所有接口端口都显式组合到同一个 AXI4-Lite 接口端口中，并以 `s_axi-<string>` 指定的值来命名 RTL 端口。

以下示例生成的接口分别名为 `s_axi_BUS_A`、`s_axi_BUS_B` 和 `s_axi_OUT`：

```
void example(char *a, char *b, char *c) { #pragma HLS INTERFACE
mode=s_axilite port=a bundle=BUS_A #pragma HLS INTERFACE mode=s_axilite
port=b bundle=BUS_B #pragma HLS INTERFACE mode=s_axilite port=c
bundle=OUT #pragma HLS INTERFACE mode=s_axilite port=return bundle=OUT
#pragma HLS INTERFACE mode=ap_vld port=b *c += *a + *b; }
```

3. 部分指定的捆绑名称：如果您为部分实参指定 `bundle` 名称，但其他实参则保留不分配，那么该工具会按如下方式来捆绑这些实参：

- 按 `INTERFACE` 编译指示的要求将所有端口组合到指定捆绑内。
- 将不含捆绑分配的所有端口都组合到一个采用默认命名的捆绑内。默认名称可以是标准工具默认名称，如果用户已指定工具默认名称，那么也可采用备用默认名称。

在以下示例中，用户已指定 `bundle=control`，即工具默认名称。在此情况下，端口 `c` 将根据用户指定分配到 `s_axi_control`，剩余端口将捆绑到 `s_axi_control_r` 下，即工具使用的备用默认名称。

```
void top(char *a, char *b, char *c) {
#pragma HLS INTERFACE mode=s_axilite port=a
#pragma HLS INTERFACE mode=s_axilite port=b
#pragma HLS INTERFACE mode=s_axilite port=c bundle=control
}
```

S_AXILITE 偏移选项

注释： Vitis 内核流程会判定所需的偏移。请勿在该流程中指定 `offset` 选项。

在 Vivado IP 流程中，Vitis HLS 根据使用的实参数据类型和端口协议来定义分配给 [S_AXILITE 控制寄存器映射](#) 中的端口的地址大小和范围。但是，`INTERFACE` 编译指示还包含 `offset` 选项，可供您用于指定 AXI4-Lite 接口中的地址偏移。

为实参指定偏移后，必须考量数据大小并保留部分额外地址用于端口控制协议。保留的地址范围应基于 32 位字。您应保留足够的 32 位字，以满足您的实参数据类型需求，并额外保留一个字用于控制协议，即使对于 `ap_none` 也是如此。



提示： 对于阵列的 `ap_memory` 协议，您无需为控制协议保留额外的字。在此情况下，只需保留足够的 32 位字以满足实参数据类型即可。

例如，要为双精度保留足够的空间，则需为 64 位数据类型保留两个 32 位字，然后保留另一个 32 位字用于控制协议。因此总计需要保留三个 32 位字或 96 位。如果您的实参偏移从 0x020 开始，那么下一个可用偏移将从 0x02c 开始，以便为您的实参保留所需的地址范围。

如果您错误设置了实参偏移，未保留足够的地址范围以供数据类型和控制协议使用，那么 Vitis HLS 将会识别该错误、向您发出问题警告，并进行恢复，方法是将其错误布局的实参寄存器移动到控制寄存器映射末尾。这样您的构建即可继续，但如果您的主机应用或驱动程序已写入指定偏移，那么主机应用或驱动程序可能无效。

C 语言驱动程序文件

当实现 AXI4-Lite 从接口时，会自动创建一组 C 语言驱动程序文件。这些 C 语言驱动程序文件可提供一组 API 以集成到 CPU 上运行的任意软件中，并且这些 API 可用于通过 AXI4-Lite 从接口与器件进行通信。

这些 C 语言驱动程序文件是在 IP 目录中将设计封装为 IP 时创建的。

针对独立模式和 Linux 模式会创建驱动程序文件。在独立模式下，驱动程序的使用方式与任何其他 AMD 独立驱动程序的使用方式并无不同。在 Linux 模式下，会将所有 C 语言文件 (.c) 和头文件 (.h) 都复制到软件工程中。

驱动程序文件和 API 函数的名称衍生自综合的顶层函数。在上述示例中，顶层函数称为“example”。如果顶层函数名为“DUT”，那么在以下描述中名称“example”将被替换为“DUT”。驱动程序文件是在封装的 IP 内创建的（位于 solution 内的 impl 目录中）。

表 12：名为“Example”的设计的 C 语言驱动程序文件

文件路径	使用模式	描述
data/example.mdd	独立模式	驱动程序定义文件。
data/example.tcl	独立模式	供 SDK 用于将软件集成到 SDK 工程中。
src/xexample_hw.h	双模式	定义所有内部寄存器的地址偏移。
src/xexample.h	双模式	API 定义
src/xexample.c	双模式	独立 API 实现
src/xexample_sinit.c	独立模式	初始化 API 实现
src/xexample_linux.c	Linux	初始化 API 实现
src/Makefile	独立模式	Makefile

在 xexample.h 文件中定义了 2 个结构体。

- XExample_Config：用于保存 IP 实例的配置信息（每个 AXI4-Lite 从接口的基址）。
- XExample：用于保存 IP 实例指针。大部分 API 使用此实例指针作为首个实参。

标准 API 实现在文件 xexample.c、xexample_sinit.c 和 xexample_linux.c 中提供，可提供用于以下操作的函数。

- 初始化器件
- 控制器件并查询其状态
- 读写寄存器
- 设置、监测和控制中断

请参阅 [第五部分：Vitis HLS C 语言驱动程序参考资料](#) 以获取 C 语言驱动程序文件中提供的 API 函数的描述。



重要提示！ C 语言驱动程序 API 始终使用无符号的 32 位类型 (U32)。您可能需要将 C 语言代码中的数据强制转换为期望的类型。

C 语言驱动程序文件和浮点类型

C 语言驱动程序文件始终使用 32 位无符号整数 (U32) 数据进行数据传输。在以下示例中，该函数使用浮点类型实参 `a` 和 `r1`。它用于设置 `a` 的值，并返回 `r1` 的值：

```
float calculate(float a, float *r1)
{
#pragma HLS INTERFACE mode=ap_vld register port=r1
#pragma HLS INTERFACE mode=s_axilite port=a
#pragma HLS INTERFACE mode=s_axilite port=r1
#pragma HLS INTERFACE mode=s_axilite port=return

    *r1 = 0.5f*a;
    return (a>0);
}
```

综合后，Vitis HLS 会将所有端口组合到默认 AXI4-Lite 接口，并创建 C 语言驱动程序文件。但驱动程序文件使用的类型为 U32，如以下示例所示：

```
// API to set the value of A
void XCalculate_SetA(XCalculate *InstancePtr, u32 Data) {
    Xil_AssertVoid(InstancePtr != NULL);
    Xil_AssertVoid(InstancePtr->IsReady == XIL_COMPONENT_IS_READY);
    XCalculate_WriteReg(InstancePtr->Hls_periph_bus_BaseAddress,
XCALCULATE_HLS_PERIPH_BUS_ADDR_A_DATA, Data);
}

// API to get the value of R1
u32 XCalculate_GetR1(XCalculate *InstancePtr) {
    u32 Data;

    Xil_AssertNonvoid(InstancePtr != NULL);
    Xil_AssertNonvoid(InstancePtr->IsReady == XIL_COMPONENT_IS_READY);

    Data = XCalculate_ReadReg(InstancePtr->Hls_periph_bus_BaseAddress,
XCALCULATE_HLS_PERIPH_BUS_ADDR_R1_DATA);
    return Data;
}
```

如果这些函数直接用于处理浮点类型，那么写入和读取值与期望的浮点类型不一致。在软件中使用这些函数时，可在代码中使用以下类型强制转换：

```
float a=3.0f,r1;
u32 ua,url;

// cast float "a" to type U32
XCalculate_SetA(&calculate,*((u32*)&a));
url=XCalculate_GetR1(&calculate);

// cast return type U32 to float type for "r1"
r1=*((float*)&url);
```

控制硬件



提示：以下示例演示的是 `ap_ctrl_hs` 块控制协议，它是 Vivado IP 流程的默认协议。请参阅 [块级控制协议](#)，以获取有关 `ap_ctrl_chain` 协议的更多信息和描述，该协议是 Vitis 内核流程的默认协议。

在此示例中，硬件头文件 `xexample_hw.h` 提供了存储器映射位置完整列表，对应分组到 AXI4-Lite 从接口内的所有端口，如 [S_AXILITE 控制寄存器映射](#) 中所述。

```
// 0x00 : Control signals
//      bit 0 - ap_start (Read/Write/SC)
//      bit 1 - ap_done (Read/COR)
//      bit 2 - ap_idle (Read)
//      bit 3 - ap_ready (Read)
//      bit 7 - auto_restart (Read/Write)
//      others - reserved
// 0x04 : Global Interrupt Enable Register
//      bit 0 - Global Interrupt Enable (Read/Write)
//      others - reserved
// 0x08 : IP Interrupt Enable Register (Read/Write)
//      bit 0 - Channel 0 (ap_done)
//      bit 1 - Channel 1 (ap_ready)
// 0x0c : IP Interrupt Status Register (Read/TOW)
//      bit 0 - Channel 0 (ap_done)
//      others - reserved
// 0x10 : Data signal of a
//      bit 7~0 - a[7:0] (Read/Write)
//      others - reserved
// 0x14 : reserved
// 0x18 : Data signal of b
//      bit 7~0 - b[7:0] (Read/Write)
//      others - reserved
// 0x1c : reserved
// 0x20 : Data signal of c_i
//      bit 7~0 - c_i[7:0] (Read/Write)
//      others - reserved
// 0x24 : reserved
// 0x28 : Data signal of c_o
//      bit 7~0 - c_o[7:0] (Read)
//      others - reserved
// 0x2c : Control signal of c_o
//      bit 0 - c_o_ap_vld (Read/COR)
//      others - reserved
// (SC = Self Clear, COR = Clear on Read, TOW = Toggle on Write, COH =
// Clear on
// Handshake)
```

要对 `s_axilite` 接口中的寄存器进行正确编程，您必须了解硬件端口使用默认端口协议或定制协议来工作的方式，如 [S_AXILITE 和端口级协议](#) 中所述。

例如，要启动块操作，`ap_start` 寄存器必须设置为 1。随后，器件将继续从 AXI4-Lite 从接口中的寄存器读取已分组到该接口中的任意输入。当此块完成操作后，`ap_done`、`ap_idle` 和 `ap_ready` 寄存器将由硬件输出端口进行设置，并由相应的寄存器读取已分组到 AXI4-Lite 从接口中的任何输出端口的结果。

此示例中的函数实参 `c` 的实现突显了了解硬件端口工作的重要性。函数实参 `c` 可供读取和写入，因此作为独立的输入端口和输出端口 `c_i` 和 `c_o` 来实现，如 [S_AXILITE 示例](#) 中所述。

对 `s_axilite` 接口进行编程的第 1 个建议流程对应于函数的一次性执行流程：

- 使用 [C 语言驱动程序文件](#) 中提供的中断函数标准 API 实现来判定期望的中断工作方式。
- 为块输入端口加载寄存器值。在以上示例中，此操作是使用 API 函数 `XExample_Set_a`、`XExample_Set_b` 和 `XExample_Set_c_i` 来执行的。
- 使用 `XExample_Start` 将 `ap_start` 位设置为 1，以开始执行函数。此寄存器为自我清除型寄存器，如以上头文件中所述。完成一次传输事务后，此块将暂停操作。
- 允许执行此函数。解决生成的所有中断。

- 读取输出寄存器。在以上示例中，这是使用 API 函数 `XExample_Get_c_o_vld`（用于确认数据有效）和 `XExample_Get_c_o` 来执行的。

注释： `s_axilite` 接口中的寄存器遵循的 I/O 协议与端口相同。在此情况下，输出有效设置为逻辑 1，以指示数据是否有效。

- 针对下一项传输事务重复此过程。

第 2 个建议流程对应于块的连续执行流程。在此模式下，AXI4-Lite 接口中包含的输入端口应仅限于执行配置的端口，如需获取有关此模式的详细描述，请参阅下一章节。通常，此块运行速度必须比 CPU 更快。如果块必须等待输入，那么块的大部分时间将用于等待：

- 使用中断函数来判定期望的中断工作方式。
- 为块输入端口加载寄存器值。在以上示例中，此操作是使用 API 函数 `XExample_Set_a`、`XExample_Set_a` 和 `XExample_Set_c_i` 来执行的。
- 使用 API `XExample_EnableAutoRestart` 设置自动启动函数。
- 允许执行此函数。个别端口 I/O 协议将同步通过时钟处理的数据。
- 解决生成的所有中断。执行此操作期间可访问输出寄存器，但数据可能多次发生变化。
- 使用 API 函数 `XExample_DisableAutoRestart` 来停止执行。
- 读取输出寄存器。在以上示例中，此操作是使用 API 函数 `XExample_Get_c_o` 和 `XExample_Set_c_o_vld` 来执行的。

控制软件

在 CPU 上运行的软件中可使用 API 函数来控制硬件块。此进程概述为：

- 创建硬件实例
- 查找器件配置
- 初始化器件
- 设置 HLS 块的输入参数
- 启动器件并读取结果

应用示例如下所示。

```
#include "xexample.h"      // Device driver for HLS HW block
#include "xparameters.h"

// HLS HW instance
XExample HlsExample;
XExample_Config *ExamplePtr

int main() {
    int res_hw;

    // Look Up the device configuration
    ExamplePtr = XExample_LookupConfig(XPAR_XEXAMPLE_0_DEVICE_ID);
    if (!ExamplePtr) {
        print("ERROR: Lookup of accelerator configuration failed.\n\r");
        return XST_FAILURE;
    }

    // Initialize the Device
    status = XExample_CfgInitialize(&HlsExample, ExamplePtr);
    if (status != XST_SUCCESS) {
```



```

print("ERROR: Could not initialize accelerator.\n\r");
exit(-1);
}

//Set the input parameters of the HLS block
XExample_Set_a(&HlsExample, 42);
XExample_Set_b(&HlsExample, 12);
XExample_Set_c_i(&HlsExample, 1);

// Start the device and read the results
XExample_Start(&HlsExample);
do {
res_hw = XExample_Get_c_o(&HlsExample);
} while (XExample_Get_c_o(&HlsExample) == 0); // wait for valid data output
print("Detected HLS peripheral complete. Result received.\n\r");
}

```

控制 AXI4-Lite 接口中的时钟和复位

注释：如果在使用不同时钟频率的总线互连结构中例化 AXI4-Lite 从寄存器，那么 Vivado IP integrator 将自动生成时钟域交汇 (CDC) slice，其执行的功能与上述控制时钟相同，故而无需用选项。

默认情况下，Vitis HLS 针对 AXI4-Lite 接口和已综合的设计使用相同时钟。Vitis HLS 将 AXI4-Lite 接口中的所有寄存器都连接到用于综合逻辑的时钟 (ap_clk)。

(可选) 您可使用 INTERFACE 指令 clock 选项来为每个 AXI4-Lite 端口指定独立的时钟。将时钟连接到 AXI4-Lite 接口时，必须使用以下协议：

- AXI4-Lite 接口时钟必须同步到用于综合逻辑的时钟 (ap_clk)。即，这 2 个时钟必须衍生自同一个主生成器时钟。
- AXI4-Lite 接口时钟频率必须等于或小于用于综合逻辑的时钟 (ap_clk) 的频率。

如果将 clock 选项与 INTERFACE 指令搭配使用，那么只需针对每个捆绑中的 1 个函数实参指定 clock 选项即可。Vitis HLS 通过相同的时钟和复位来实现捆绑中的所有其他函数实参。Vitis HLS 以 ap_rst_ 作为前缀并后接时钟名称的方式来对生成的复位信号命名。生成的复位信号处于低电平有效状态，与 config_rtl 命令无关。

以下示例显示了 Vitis HLS 如何将函数实参 a 和 b 组合到同一个 AXI4-Lite 端口内，且该端口含名为 AXI_clk1 的时钟和关联的复位端口。

```

// Default AXI-Lite interface implemented with independent clock called
AXI_clk1
#pragma HLS interface mode=s_axilite port=a clock=AXI_clk1
#pragma HLS interface mode=s_axilite port=b

```

在以下示例中，Vitis HLS 将函数实参 c 和 d 组合到 AXI4-Lite 端口 CTRL1 内，该端口含名为 AXI_clk2 的独立时钟和关联的复位端口。

```

// CTRL1 AXI-Lite bundle implemented with a separate clock (called AXI_clk2)
#pragma HLS interface mode=s_axilite port=c bundle=CTRL1 clock=AXI_clk2
#pragma HLS interface mode=s_axilite port=d bundle=CTRL1

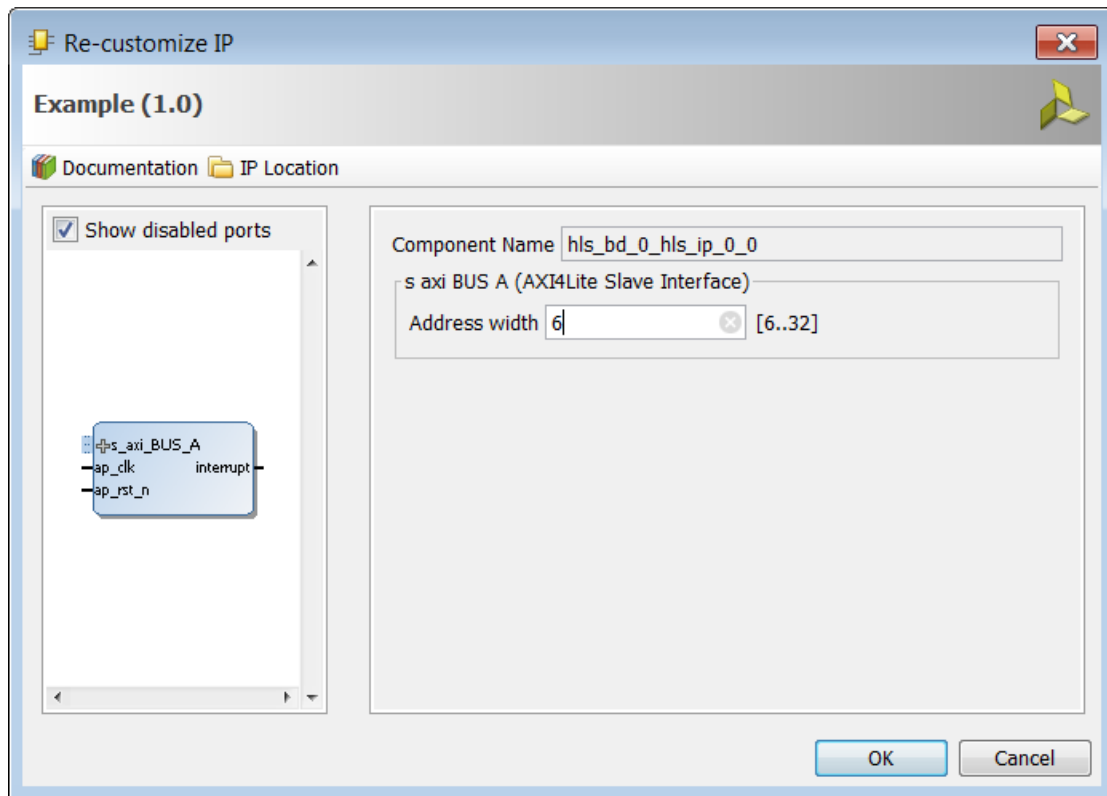
```

在 IP integrator 中自定义 AXI4-Lite 从接口

在将使用 AXI4-Lite 从接口的 HLS RTL 设计整合到 Vivado IP integrator 中的设计中时，可对块进行自定义。在 IP integrator 的模块框图中，选中 HLS 块、右键单击并选择“Customize Block”。

默认情况下，地址宽度配置为所需的最小大小。对此设置进行修改即可连接到地址大小小于 32 位的块。

图 41：在 IP integrator 中自定义 AXI4-Lite 从接口



AXI4-Stream 接口



重要提示！ `hls::axis`（以及 `ap_axiu/ap_axis`）无法在内部函数或变量上使用，因为 AXI4-Stream 协议仅在顶层函数接口上受支持。对于内部函数和变量，必须使用 `hls::stream` 对象，如 [HLS 串流库](#) 中所述。

AXI4-Stream 接口可应用于任意输入实参和任意数组或指针输出实参。由于 AXI4-Stream 接口按顺序串流方式来传输数据，因此不可用于同时支持读写的实参。就数据布局而言，AXI4-Stream 的数据类型会对齐到下一个字节。例如，如果数据类型大小为 12 位，则将扩展至 16 位。根据选择的是有符号接口还是无符号接口，扩展的位将采用符号位扩展或零位扩展。

如果串流数据类型为用户定义的结构体，那么默认过程是使该结构体保持处于聚合状态，并将其对齐到最大数据元素大小（最接近的字节）。此规则的唯一例外是包含 `hls::stream` 对象的结构体。在此特殊情况下，该结构体将解聚，并为该结构体的每个成员元素创建一个 AXI 串流。



提示： 受支持的最大端口位宽为 4096 位，对于聚合的结构体或重构的阵列也都同样如此。

以下代码示例显示了如何根据结构体类型来封装对齐。如果结构体仅包含一个字符 (`char`) 类型，如以下示例所示，那么它将以 1 字节对齐方式进行封装。该结构体的总大小将为 2 字节：

```
struct A {
    char foo;
    char bar;
};
```

但如果结构体所含元素具有不同数据类型，如下所示，那么它将封装并对齐到最大的数据元素的大小，在此示例中即 4 字节。元素 `bar` 将填充 3 字节，从而使该结构体的总大小达到 8 字节：

```
struct A {
    int foo;
    char bar;
};
```

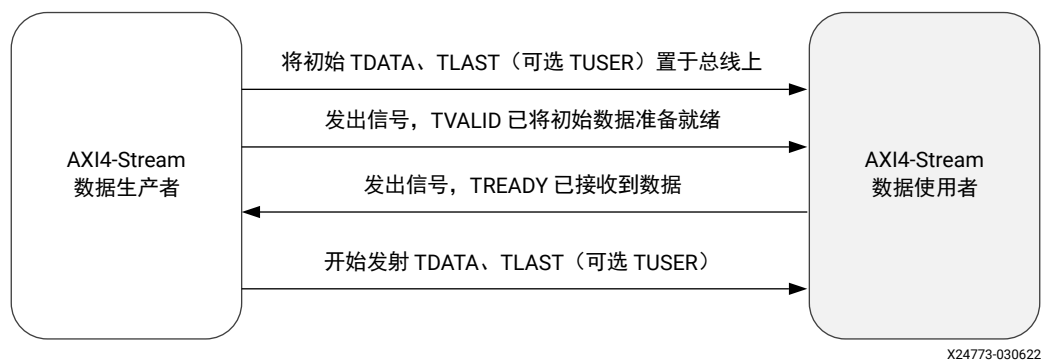


重要提示！ AXI4-Stream 接口 (axis) 中包含的结构体默认情况下处于聚合状态，串流本身无法解聚。如果结构体的成员元素需要独立串流，则将其手动编码为独立元素，从而为每个元素生成独立的 AXIS 接口。请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Interface/Aggregation_Disaggregation/disaggregation_of_axis_port](#) 以获取示例。

AXI4-Stream 工作原理

AXI4-Stream 是专为传输任意单向数据而设计的协议。在 AXI4-Stream 中，TDATA 位宽是按时钟周期来传输的。当生产者发出 TVALID 信号，并且使用者通过发送 TREADY 信号来响应（使用者已使用初始 TDATA）后，就会立即开始传输。此时，生产者会开始发送 TDATA 和 TLAST（如果需要，它还会发送 TUSER 以承载其他用户定义的旁路数据）。TLAST 信号表示串流的最后一个字节。因此，使用者会持续使用传入的 TDATA，直至 TLAST 断言有效为止。

图 42：AXI4-Stream 握手



X24773-030622

AXI4-Stream 具有其他可选功能特性，如随 TKEEP 端口和 TSTRB 端口发送定位数据，这样即可在 TDATA 信号上对数据位置和数据本身进行多路复用。通过使用 TID 和 TDIST 信号，即可对这些串流进行布线，因为这些字段与串流标识符和串流目标标识符之间存在大致对应关系。如需了解更多信息，请参阅《Vivado Design Suite：AXI 参考指南》(UG1037) 或《AMBA AXI4-Stream 协议规范》(ARM IHI 0051A)。

AXI4-Stream 实现方式

如果设计需要串流接口，可首先定义和使用串流数据结构，例如，Vitis HLS 中的 `hls::stream`。此简单对象可封装串流的要求，其串流接口则默认在 RTL 中作为 FIFO 来实现 (ap_fifo)，但（可选）可作为握手接口 (ap_hs) 或 AXI4-Stream 接口 (axis) 来实现。请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Interface/Streaming](#) 以获取串流接口的不同示例。

如果通过接口编译指示模式选项指定 AXI4-Stream 接口 (axis)，那么接口实现将通过定义 TDATA、TVALID 和 TREADY 信号来模仿 AXIS 接口的样式。

如需更正式的 AXIS 实现，那么 Vitis HLS 需使用特殊数据类型 (`ap_axi_sdata.h` 中定义的 `hls::axis`) 来封装 AXI4-Stream 协议的要求，并实现此接口所需的特殊 RTL 信号。

AXI4-Stream 接口是作为 Vitis HLS 中的结构体类型来实现的，并且具有如下特征（在 `ap_axi_sdata.h` 中定义）：

```
template <typename T, size_t WUser, size_t WId, size_t WDest> struct axis
{ .. };
```

其中：

- T：要串流的数据类型。



提示：可支持任意数据类型，包括 `ap_fixed`。

- WUser：TUSER 信号的宽度
- WId：TID 信号的宽度
- WDest：TDEST 信号的宽度

当串流数据类型 (T) 为简单的整数类型时，有 2 种预定义的 AXI4-Stream 实现类型可用：

- AXI4-Stream 类的有符号实现（或者使用 `ap_axis<Wdata, WUser, WId, WDest>` 则更简单）

```
hls::axis<ap_int<WData>, WUser, WId, WDest>
```

- AXI4-Stream 类的无符号实现（或者使用 `ap_axiu<WData, WUser, WId, WDest>` 则更简单）

```
hls::axis<ap_uint<WData>, WUser, WId, WDest>
```

为 WUser、WId 和 WDest 模板参数指定的值将控制 AXI4-Stream 接口中旁路信号的使用。

使用 `hls::axis` 类时，生成的 RTL 通常将包含实际数据信号 `TDATA`，和以下额外信号：`TVALID`、`TREADY`、`TKEEP`、`TSTRB`、`TLAST`、`TUSER`、`TID` 和 `TDEST`。

`TVALID`、`TREADY` 和 `TLAST` 都是 AXI4-Stream 协议的必要控制信号。`TKEEP`、`TSTRB`、`TUSER`、`TID` 和 `TDEST` 信号均为可选特殊信号，可用于围绕额外的簿记数据来传递。



提示：如果 WUser、WId 和 WDest 设置为 0，那么在接口中生成的 RTL 将不包含可选 `TUSER`、`TID` 和 `TDEST` 信号。

寄存 AXI4-Stream 接口

默认情况下，AXI4-Stream 接口始终作为已寄存的接口来实现，以便在将含有 AXI4-Stream 接口的多个 HLS IP 块集成到更大的设计中时，确保不会创建组合反馈路径。对于 AXI4-Stream 接口，提供了 4 种类型的寄存器模式，以控制接口寄存器的实现方式：

- Forward：即“正向”；仅寄存 `TDATA` 和 `TVALID` 信号。
- Reverse：即“反向”；仅寄存 `TREADY` 信号。
- Both：即“双向”；寄存所有信号（`TDATA`、`TREADY` 和 `TVALID`）。这是默认方式。
- Off：即“关闭”；不寄存任何端口信号。

AXI4-Stream 旁路信号被视为数据信号，随 `TDATA` 一起寄存。



建议：将 HLS 生成的 IP 块与 AXI4-Stream 接口相连时，应将至少 1 个接口实现为寄存接口，或者这些块应通过 AXI4-Stream Register Slice 连接。

在设计中可通过 2 种基本方法来使用 AXI4-Stream：

- 使用不含旁路的 AXI4-Stream。
- 使用含旁路的 AXI4-Stream。

第 2 种使用模型可提供额外功能，允许将属于 AXI4-Stream 标准的可选旁路直接用于 C/C++ 代码中。

不含旁路的 AXI4-Stream 接口

如果数据类型为 `ap_axis` 或 `ap_axiu` 的函数实参不包含任何 AXI4 旁路元素（即 `WUser`、`Wid` 和 `WDest` 参数设置为 0），则使用不含旁路的 AXI4-Stream。在以下示例中，2 种接口均使用 AXI4-Stream 来实现：

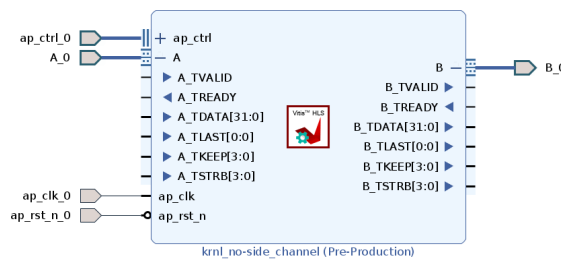
```
#include "ap_axi_sdata.h"
#include "hls_stream.h"

typedef ap_axiu<32, 0, 0, 0> trans_pkt;

void example(hls::stream< trans_pkt > &A, hls::stream< trans_pkt > &B)
{
    #pragma HLS INTERFACE mode=axis port=A
    #pragma HLS INTERFACE mode=axis port=B
    trans_pkt tmp;
    A.read(tmp);
    tmp.data += 5;
    B.write(tmp);
}
```

综合后，2 个实参均使用数据端口 (TDATA) 和标准 AXI4-Stream 协议端口 (TVALID、TREADY、TKEEP、TLAST 和 TSTRB) 来实现，如下图所示。

图 43：不含旁路的 AXI4-Stream 接口



提示：如果指定的 `hls::stream` 对象的数据类型并非 `ap_axis` 或 `ap_axiu`，那么该工具会推断不含 TLAST 信号或任何旁路信号的 AXI4-Stream 接口。此 AXI4-Stream 接口实现耗用的器件资源较少，但无法查看串流何时结束。

通过使用结构体可将多个变量组合到同一个 AXI4-Stream 接口中，此结构体默认是由 Vitis HLS 聚合而成的。将单个结构体的多个元素聚合到单宽矢量中即可在同一个 AXI4-Stream 接口中实现该结构体的所有元素。

含旁路的 AXI4-Stream 接口

以下两个示例显示了如何在 C/C++ 语言代码中直接使用旁路，并在接口上实现这些旁路。代码使用 `#include "ap_axi_sdata.h"` 提供 API 来处理 AXI4-Stream 接口的旁路。

第一个示例使用推荐的 API，下文列出了其中使用的 get 方法或 set 方法，而第二个示例则使用了传统的编码风格。

Table for getters/setters APIs:

Getters	Setters
get_data()	set_data()
get_keep()	set_keep()
get_strb()	set_strb()
get_user()	set_user()
get_last()	set_last()
get_id()	set_id()
get_dest()	set_dest()

First example:

```
#include "ap_axi_sdata.h"
#include "ap_int.h"
#include "hls_stream.h"

#define DWIDTH 32
typedef ap_axiu<DWIDTH, 1, 1, 1> trans_pkt;

extern "C"
{
    void krnl_stream_vmult(hls::stream<trans_pkt> &A,
                           hls::stream<trans_pkt> &B)
    {
        #pragma HLS INTERFACE mode = axis port = A
        #pragma HLS INTERFACE mode = axis port = B
        #pragma HLS INTERFACE mode = s_axilite port = return bundle = control
        bool eos = false;
        vmult: do
        {
            #pragma HLS PIPELINE II = 1
            trans_pkt t2 = A.read();

            // Packet for Output
            trans_pkt t_out;

            // Reading data from input packet
            ap_uint<DWIDTH> in2 = t2.get_data();
            ap_uint<DWIDTH> tmpOut = in2 * 5;

            // Setting data and configuration to output packet
            t_out.set_data(tmpOut);
            t_out.set_last(t2.get_last());
            t_out.set_keep(-1); // Enabling all bytes
            // Writing packet to output stream
            B.write(t_out);
            if (t2.get_last())
            {
                eos = true;
            }
        } while (eos == false);
    }
}
```

采用传统编码风格的第二个示例：

```
#include "ap_axi_sdata.h"
#include "ap_int.h"
#include "hls_stream.h"

#define DWIDTH 32

typedef ap_axiu<DWIDTH, 1, 1, 1> trans_pkt;

extern "C" {
    void krnl_stream_vmult(hls::stream<trans_pkt> &A,
                          hls::stream<trans_pkt> &B) {
#pragma HLS INTERFACE mode=axis port=A
#pragma HLS INTERFACE mode=axis port=B
#pragma HLS INTERFACE mode=s_axilite port=return bundle=control
        bool eos = false;

        vmult: do {
#pragma HLS PIPELINE II=1
            trans_pkt t2 = A.read();

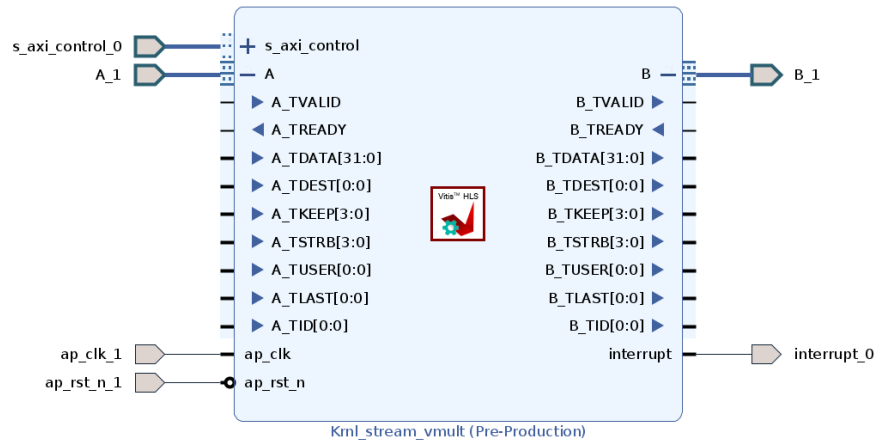
            // Packet for Output
            trans_pkt t_out;

            // Reading data from input packet
            ap_uint<DWIDTH> in2 = t2.data;
            ap_uint<DWIDTH> tmpOut = in2 * 5;

            // Setting data and configuration to output packet
            t_out.data = tmpOut;
            t_out.last = t2.last;
            t_out.keep = -1; //Enabling all bytes
            // Writing packet to output stream
            B.write(t_out);
            if (t2.last) {
                eos = true;
            }
        } while (eos == false);
    }
}
```

完成综合后，A 和 B 实参均随数据端口、标准 AXI4-Stream 协议端口、TVALID 和 TREADY 以及结构体中所描述的所有可选端口一起实现。

图 44: 含旁路的 AXI4-Stream 接口



阵列到串流的编码样式

虽然阵列可转换为串流，但这样通常会导致出现编码及综合问题，因为阵列可按随机顺序访问，而串流则需采用按顺序访问模式，即每个元素都按顺序读取。为避免出现此类问题，需要串流接口时，强烈建议使用 `hls::stream` 对象，如 [使用 HLS 串流](#) 中所述。使用此构造将强制执行源代码中的串流语义。

但要将阵列转换为串流，您应对临时变量执行所有运算。读取输入串流、处理临时变量并写入输出串流，如以下示例所示。此方法允许您保留数据串流的读取和写入顺序，而不是尝试多次或随机读取或写入。

```
struct A {
    short varA;
    int varB;
};

void dut(A in[N], A out[N], bool flag) {
#pragma HLS interface mode=axis port=in,out
    for (unsigned i=0; i<N; i++) {
        A tmp = in[i];
        if (flag)
            tmp.varB = tmp.varA + 5;
        out[i] = tmp;
    }
}
```

如不遵循此编码样式，它将导致串流处理功能失败。

建议的方法是将实参定义为 `hls::stream` 对象，如下所示：

```
void dut(hls::stream<A> &in, hls::stream<A> &out, bool flag) {
#pragma HLS interface mode=axis port=in,out

    for (unsigned i=0; i<N; i++) {
        A tmp = in.read();
        if (flag)
            tmp.varB = tmp.varA + 5;
        out.write(tmp);
    }
}
```

自定义 AXI4-Stream 接口

不含旁路的 AXI4-Stream 实例（例如，`hls::stream<ap_axiu<32, 0, 0, 0>>`）会导致在接口上出现以下 RTL 信号：

- TVALID
- TREADY
- TDATA[]
- TLAST[]
- TKEEP[]
- TSTRB[]

在您的设计中可能并不需要或者并不希望出现其中部分信号（TLAST、TKEEP 和 TSTRB）。此外，在某些用例中，不需要 TDATA，因为建模者会使用 TUSER 来移动数据。为支持这些额外用例，`hls::axis<T>` 定义支持以下选项：

```
template <typename T,
          std::size_t WUser = 0,
          std::size_t WId = 0,
          std::size_t WDest = 0,
          uint8_t EnableSignals = (AXIS_ENABLE_KEEP |
                                   AXIS_ENABLE_LAST |
                                   AXIS_ENABLE_STRB),
          bool StrictEnablement = false>
struct axis {
    // members
    T data;
    ap_uint<bytesof<T>> strb;
    ap_uint<bytesof<T>> keep;
    ap_uint<WUser> user;
    ap_uint<WId> id;
    ap_uint<WDest> dest;
    ap_uint<1> last;

    // get_* methods
    ...
    // set_* methods
    ...
};
```

EnableSignals 字段和 StrictEnablement 字段允许您使用特殊宏规范来指定以下三个信号中的哪个信号应出现在您的设计中：TKEEP、TLAST 和 TSTRB。完整的宏列表如下所示：

```
// Enablement for axis signals
#define AXIS_ENABLE_DATA 0b00000001
#define AXIS_ENABLE_DEST 0b00000010
#define AXIS_ENABLE_ID 0b00000100
#define AXIS_ENABLE_KEEP 0b00001000
#define AXIS_ENABLE_LAST 0b00010000
#define AXIS_ENABLE_STRB 0b00100000
#define AXIS_ENABLE_USER 0b01000000

// Disablement mask for DATA axis signals
#define AXIS_DISABLE_DATA (0b11111111 ^ AXIS_ENABLE_DATA) & \
                          (0b11111111 ^ AXIS_ENABLE_KEEP) & \
```



```

                                (0b11111111 ^ AXIS_ENABLE_STRB)

// Enablement/disablement of all axis signals
#define AXIS_ENABLE_ALL 0b01111111
#define AXIS_DISABLE_ALL 0b00000000

```



提示：这些宏还支持某些错误检查，以识别规范不完整的情况。

除了允许您启用或禁用特定旁路信号外，这组宏已扩展至允许指定信号的 `all` 或 `none` 规范。这样即可允许定义特殊的 `hls::axis_data<>` 和 `hls::axis_user<>` 帮助程序类，以便于准确选取信号规范。这些特殊帮助程序类的定义如下所示：

```

// Struct: axis_data (alternative to axis)
//   DATA signal always enabled
//   All other signals are optional, disabled by default
template <typename TData,
          uint8_t EnableSignals = AXIS_ENABLE_DATA,
          std::size_t WUser = 0,
          std::size_t WId = 0,
          std::size_t WDest = 0,
          bool StrictEnablement = true>
using axis_data = axis<TData, WUser, WId, WDest,
                      (EnableSignals | AXIS_ENABLE_DATA),
                      StrictEnablement>;

// Struct: axis_user (alternative to axis)
//   USER signal always enabled
//   DATA signal always disabled
//   All other signals are optional, disabled by default
template <std::size_t WUser,
          uint8_t EnableSignals = AXIS_ENABLE_USER,
          std::size_t WId = 0,
          std::size_t WDest = 0,
          bool StrictEnablement = true>
using axis_user = axis<void, WUser, WId, WDest,
                      (EnableSignals & AXIS_DISABLE_DATA),
                      StrictEnablement>;

```

以下用例展示了各种选项，以支持您自定义与 AXI4-Stream 接口关联的信号：

```

// Allow user to control existence of TKEEP, TSTRB and TLAST by template
parameter `EnableSideChannels`

// Enable only TKEEP
using data_t = hls::axis<int, 0, 0, 0, AXIS_ENABLE_KEEP>; Or using data_t =
hls::axis_data<int, AXIS_ENABLE_KEEP>

// Enable two channels TKEEP and TLAST
using data_t = hls::axis_data<int, AXIS_ENABLE_KEEP|AXIS_ENABLE_LAST>;

// Disable all side-channels
using data_t = hls::axis_data<int, AXIS_DISABLE_ALL>;

// Enable all side-channels
using data_t = hls::axis_data<int, AXIS_ENABLE_ALL, 4, 2, 3>;

// Error case: All signals enabled but zero size specified for WUser/WId/
WDest
using data_t = hls::axis_data<int, AXIS_ENABLE_ALL>;

```

```
// Allow user to control existence of TDATA by specifying template
parameter `T` with `void` type
// and only use TUSER channel
using data_t = hls::axis<void, 5, 0, 0, AXIS_DISABLE_ALL>; Or using data_t
= hls::axis_user<5>

// Use default enabling of TDATA, TKEEP, TSTRB and TLAST
using data_t = hls::axis<int>;
```

Vivado IP 流程的端口级协议



重要提示！ 此处所述端口级协议可供在 Vivado IP 流程中使用，如 [Vivado IP 流程接口](#) 中所述。在 Vitis 内核流程中不支持这些协议。

默认情况下，输入指针和按值传递 (pass-by-value) 实参作为简单的线型端口来实现，无关联的握手信号。例如，在 [Vivado IP 流程接口](#) 所探讨的 `vadd` 函数中，输入端口实现时不含 I/O 协议，仅含数据端口。如果此端口不含默认或按设计指定的 I/O 协议，那么输入数据必须保持稳定直至读取为止。

默认输出指针实现时含关联的输出有效信号，用于指示何时输出数据有效。在 `vadd` 函数示例中，输出端口实现时含关联的输出有效端口 (`out_r_o_ap_vld`) 以指示何时端口上的数据有效并且可供读取。如果不存在与输出端口关联的 I/O 协议，那么将难以确定何时读取数据。



提示： 最好始终在输出上使用 I/O 协议。

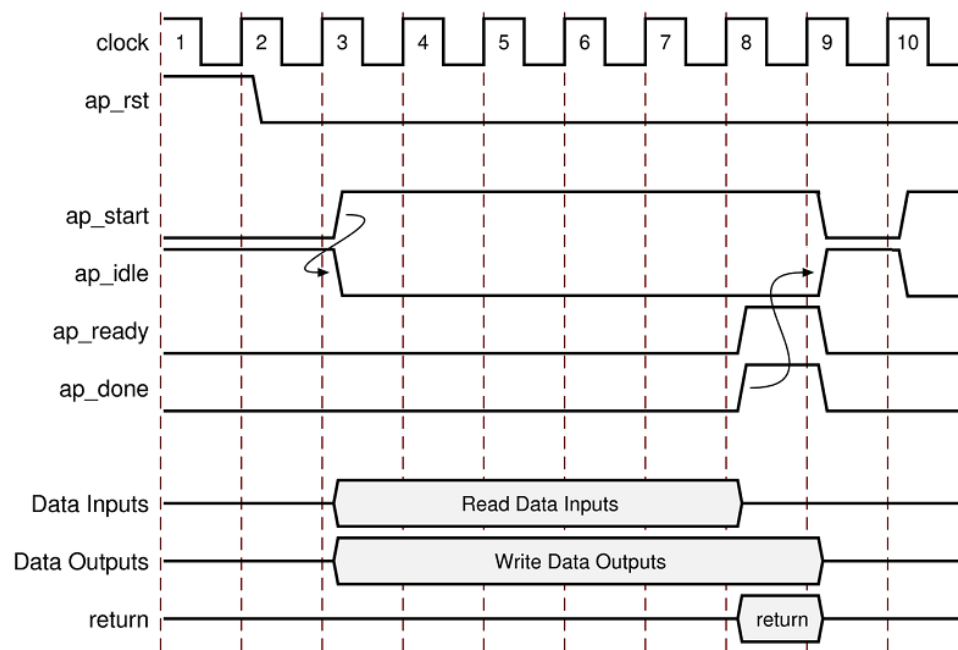
同时支持读取和写入的函数实参将拆分为独立的输入端口和输出端口。在 `vadd` 函数示例中，`out_r` 实参是作为输入端口 `out_r_i` 和输出端口 `out_r_o` 来实现的，并具有关联的 I/O 协议端口 `out_r_o_ap_vld`。

如果函数具有返回值，则实现输出端口 `ap_return` 以提供返回值。当 RTL 设计完成 1 项传输事务时（等同于执行 1 次 C/C++ 语言函数），块级协议会以 `ap_done` 信号来表明函数已完成。这也表示 `ap_return` 端口上的数据有效且可读。

注释： 返回顶层函数的值不能为指针。

对于所示的示例代码，时序行为如下图所示（假定目标技术和时钟频率允许每个时钟周期执行一次加法）。

图 45：含默认综合的 RTL 端口时序



- 当 `ap_start` 断言为高位有效时，即开始设计。
- `ap_idle` 信号断言为低位有效，表示设计正在运行中。
- 输入数据可在首个周期后的任意时钟内读取。Vitis HLS 会调度读取发生的时间。读取所有输入后，`ap_ready` 信号即断言为高位有效 (High)。
- 计算输出 `sum` 时，关联的输出握手 (`sum_o_ap_vld`) 表示数据有效。
- 当函数完成时，即断言 `ap_done` 有效。这也表示 `ap_return` 上的数据有效。
- 端口 `ap_idle` 断言高位有效时，即表示设计正在等待再次启动。

端口级 I/O：无协议

`ap_none` 指定不向端口添加任何 I/O 协议。指定该选项时，实参作为不含任何其他关联信号的数据端口来实现。
`ap_none` 模式是标量输入的默认模式。

`ap_none`

`ap_none` 端口级 I/O 协议是最简单的接口类型，没有与之关联的其他信号。输入和输出数据信号都没有关联的控制端口以指示何时读取或写入数据。RTL 设计中仅有的端口是源代码中指定的端口。

`ap_none` 接口无需额外硬件开销。但是，`ap_none` 接口需满足以下条件：

- 生产者块执行以下操作之一：
 - 在正确的时间向输入端口提供数据，通常是在设计开始之前提供。
 - 在执行传输事务期间保留数据，直到设计发出 `ap_ready` 信号为止。
- 设计完成时，使用者块会在设计重新启动之前读取输出端口。

注释： `ap_none` 接口不能与阵列实参一起使用。

端口级 I/O：有线握手

接口模式 `ap_hs` 包含与数据端口的双向握手信号。此握手属于业界标准的有效和确认握手。`ap_vld` 模式同样如此，但仅含有效端口，`ap_ack` 仅含确认端口。

`ap_ovld` 模式用于输入输出实参。将输入输出拆分为独立输入端口和输出端口时，`ap_none` 模式适用于输入端口，`ap_vld` 适用于输出端口。这是支持读写的指针实参的默认行为。

`ap_hs` 模式可应用于按顺序读写的阵列。如果 Vitis HLS 可判定读访问或写访问为无序访问，它将停止综合并报错。如果无法判定访问顺序，Vitis HLS 将发出警告。

`ap_hs` (`ap_ack`、`ap_vld` 和 `ap_ovld`)

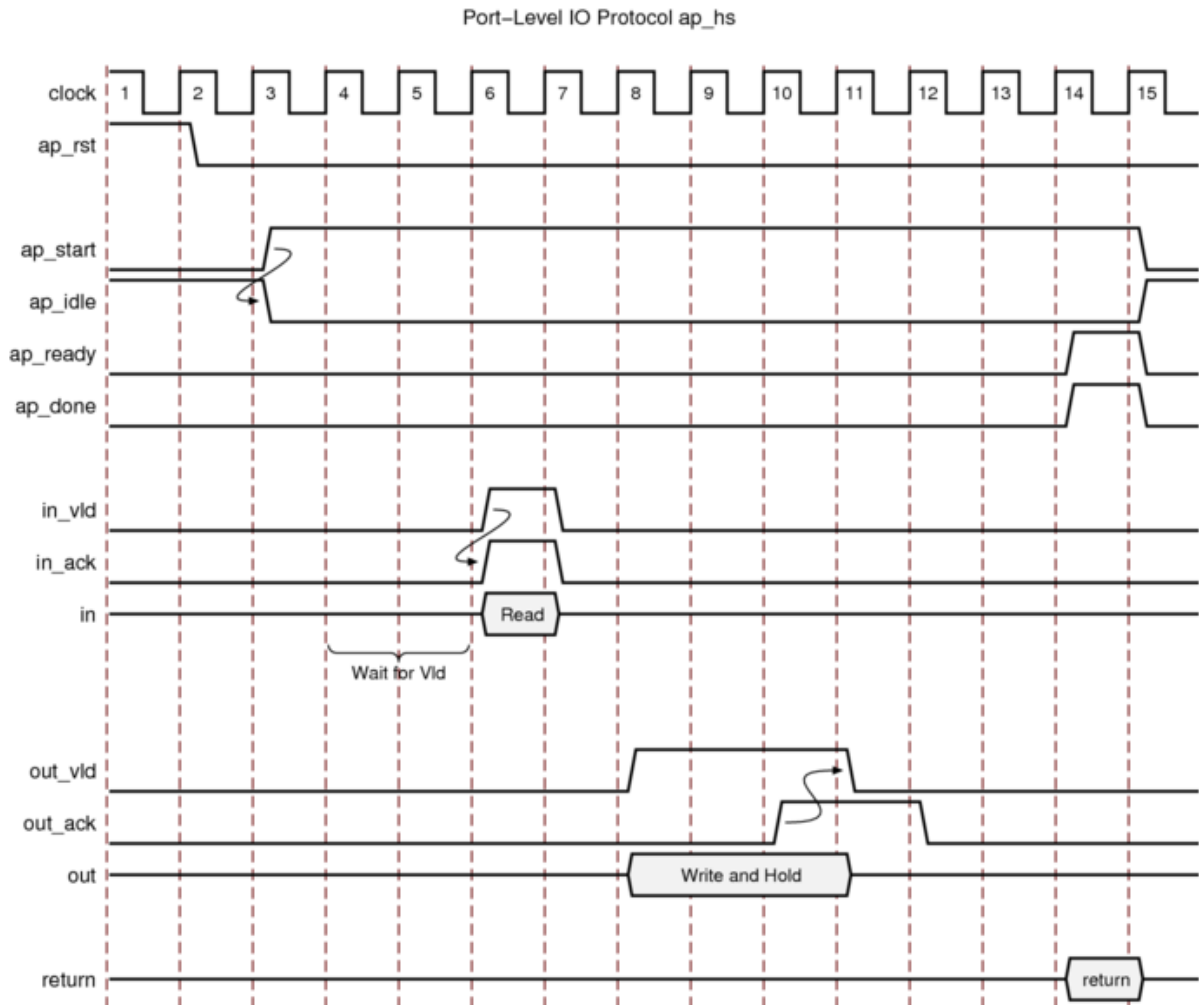
`ap_hs` 端口级 I/O 协议在开发过程中提供了最大的灵活性，允许采用自下而上和自上而下的设计流程。双向握手可安全执行所有块内通信，无需人为干预或假设即可正确执行。`ap_hs` 端口级 I/O 协议提供以下信号：

- 数据端口
- 用于指示数据信号有效并且可读的有效信号
- 用于指示何时已读取数据的确认信号

下图显示了 `ap_hs` 接口对应输入和输出端口的行为。在此示例中，输入端口名为 `in`，输出端口名为 `out`。

注释： 控制信号名称基于原始端口名称。例如，用于数据输入 `in` 的 `valid` 端口名为 `in_vld`。

图 46: ap_hs 接口的行为



对于输入，将执行以下操作：

- 应用 start 后，该块开始正常操作。
- 如果设计已准备好输入数据，但输入 valid 处于低电平状态，则设计将停滞并等待断言输入 valid 有效以指示存在新的输入值。

注释：上图显示了此行为。在此示例中，设计已准备好在时钟周期 4 上读取数据输入 in、停滞并等待输入 valid，然后再读取数据。

- 当输入 valid 断言为高电平有效 (High) 时，输出确认将断言为高电平有效 (High)，表明已读取数据。

对于输出，将执行以下操作：

- 应用 start 后，该块开始正常操作。
- 写入输出端口时，将同时断言其关联的输出 valid 信号有效，以指示端口上存在有效数据。
- 如果关联的输入确认为低电平有效 (Low)，则设计将停滞并等待断言输入确认有效。
- 当断言输入确认有效以表示已读取数据后，将在下一个时钟沿断言输出 valid 无效。

ap_ack

ap_ack 端口级 I/O 协议是 ap_hs 接口类型的子集。ap_ack 端口级 I/O 协议提供以下信号：

- 数据端口
- 用于指示何时使用数据的确认信号
 - 对于输入实参，设计会在读取输入的周期中生成高电平有效输出确认。
 - 对于输出实参，Vitis HLS 会实现输入确认端口以确认已读取输出。

注释：写入操作之后，设计将停滞并等待，直到输入确认断言为高电平有效为止，这表明使用者块已读取输出。但是，不存在可用于指示何时可以使用数据的关联输出端口。



注意！ 您不能使用 C/RTL 协同仿真来验证在输出端口上使用 ap_ack 的设计。

ap_vld

ap_vld 是 ap_hs 接口类型的子集。ap_vld 端口级 I/O 协议提供以下信号：

- 数据端口
- 用于指示数据信号有效并且可读的有效信号
 - 对于输入实参，设计在 valid 端口有效后立即读取数据端口。即使设计尚未准备好读取新数据，该设计也会对数据端口进行采样并在内部保留数据，直到需要时为止。
 - 对于输出实参，Vitis HLS 会实现输出 valid 端口以指示输出端口上的数据何时有效。

ap_ovld

ap_ovld 是 ap_hs 接口类型的子集。ap_ovld 端口级 I/O 协议提供以下信号：

- 数据端口
- 用于指示数据信号有效并且可读的有效信号
 - 对于输入实参和输入输出参数中的输入部分，设计默认为 ap_none 类型。
 - 对于输出实参和输入输出参数中的输出部分，设计实现 ap_vld 类型。

端口级 I/O：存储器接口协议

默认情况下，使用字寻址将阵列实参作为 ap_memory 接口来实现。这是含数据、地址、芯片使能和写使能端口的标准块 RAM 接口。

ap_memory 接口可作为单端口接口或双端口接口来实现。如果 Vitis HLS 可判定使用双端口接口将缩短启动时间间隔，那么它将自动实现双端口接口。BIND_STORAGE 编译指示或指令可用于指定存储器资源，如果在含单端口块 RAM 的阵列上指定该指令，那么将实现单端口接口。相反，如果使用 BIND_STORAGE 编译指示指定双端口接口，并且 Vitis HLS 判定此接口并无益处，那么它将自动实现单端口接口。

如果按顺序访问阵列，可使用 ap_fifo 接口。就像 ap_hs 接口一样，如果 Vitis HLS 判定未按顺序进行数据访问，那么它将停止；如果无法判定是否采用顺序访问，则将发出警告；如果判定已采用顺序方式访问，则不发出任何消息。ap_fifo 接口只能用于读取或写入，不能用于同时读写。

ap_memory

ap_memory 接口端口级 I/O 协议用于实现实参阵列。当实现要求随机访问存储器地址位置时，这种类型的端口级 I/O 协议会连接到存储器元件（例如，RAM 和 ROM）。

注释：如果只需顺序访问存储器元件，请改用 ap_fifo 接口。ap_fifo 接口用于减少硬件开销。

ap_memory 接口使用基于字的寻址方式，并生成一个芯片使能控制信号。

从 v2024.1 版起，ap_memory 接口显示为单一端口（已组合）。在 IP integrator 中，可使用单一连接来创建到所有端口的连接。

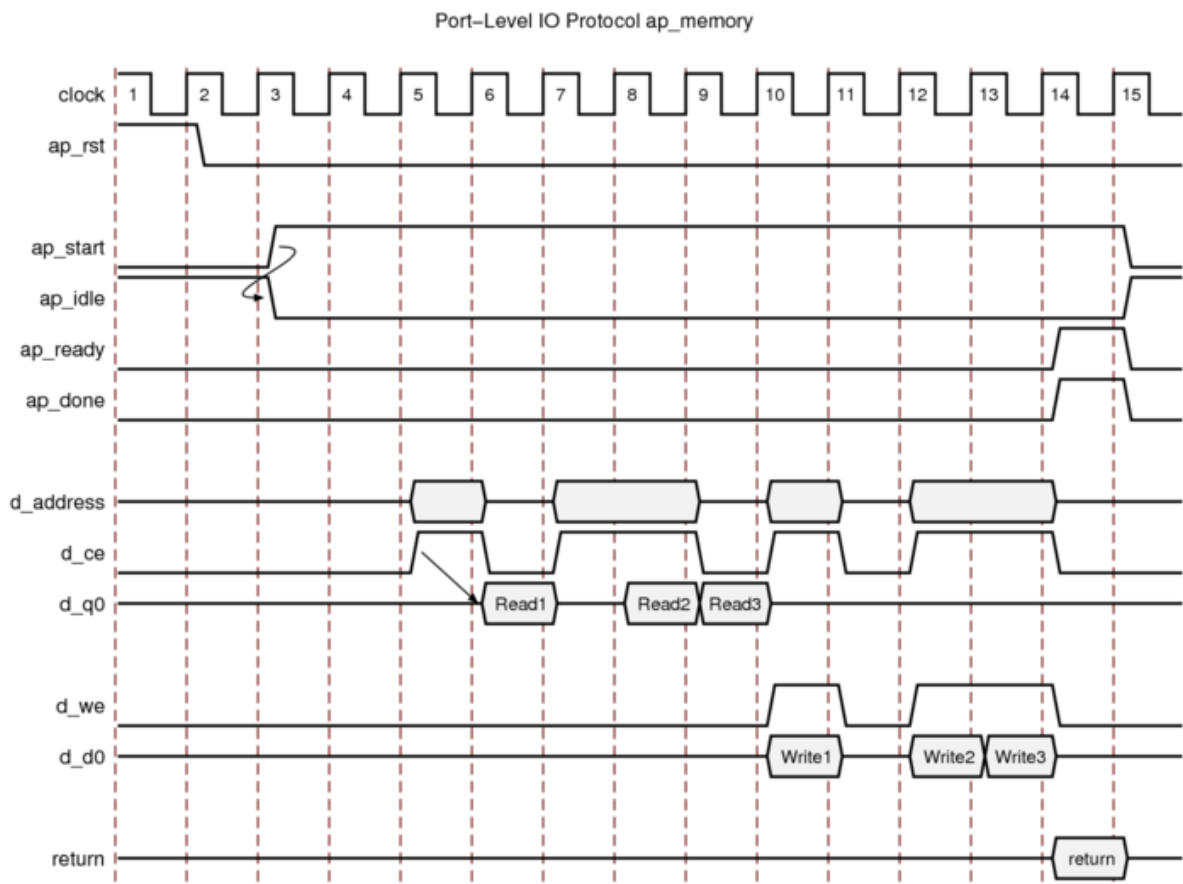
使用存储器接口时，使用 BIND_STORAGE 编译指示来指定实现。如果没有为阵列指定目标，则 Vitis HLS 会判定是使用单端口还是双端口 RAM 接口。



提示：运行综合前，请使用 BIND_STORAGE 编译指示确保将阵列实参定向到正确的存储器类型。使用其他存储器进行重新综合可能会导致调度和 RTL 不同。

下图显示了一个名为 d 的阵列，该阵列指定为单端口块 RAM。端口名称基于 C/C++ 函数实参。例如，如果 C/C++ 实参为 d，则根据块 RAM 的 output/q 端口，芯片使能为 d_ce，输入数据为 d_q0。

图 47：ap_memory 接口的行为



复位后，将执行以下操作：

- 应用 start 后，该块开始正常操作。
- 通过在断言输出信号 d_ce 有效的同时在输出地址端口上应用地址来执行读取。

注释：对于默认的块 RAM，设计期望在下一个时钟周期中，输入数据 d_q0 可用。您可以使用 BIND_STORAGE 编译指示来指示 RAM 具有更长的读取时延。

注释：当 d_ce 为低电平时，外部存储器必须停滞并保持输出数据处于有效状态。

- 通过断言输出端口 d_ce 和 d_we 有效并同时应用地址和输出数据 d_d0 来执行写操作。

ap_fifo

写入输出端口时，当设计需要访问存储器元件并且访问始终以顺序方式执行时，即不需要随机访问，则与其关联的输出 valid 信号接口是最节省硬件的方法。ap_fifo 端口级 I/O 协议支持以下操作：

- 允许端口连接到 FIFO
- 启用完整的双向 empty-full 通信
- 适用于数组、指针和按引用传递实参类型

注释：可以使用 ap_fifo 接口的函数通常使用指针，并且可能多次访问同一变量。要了解使用这种编码样式时 volatile 限定符的重要性，请参阅 [接口上的多重访问指针](#)。

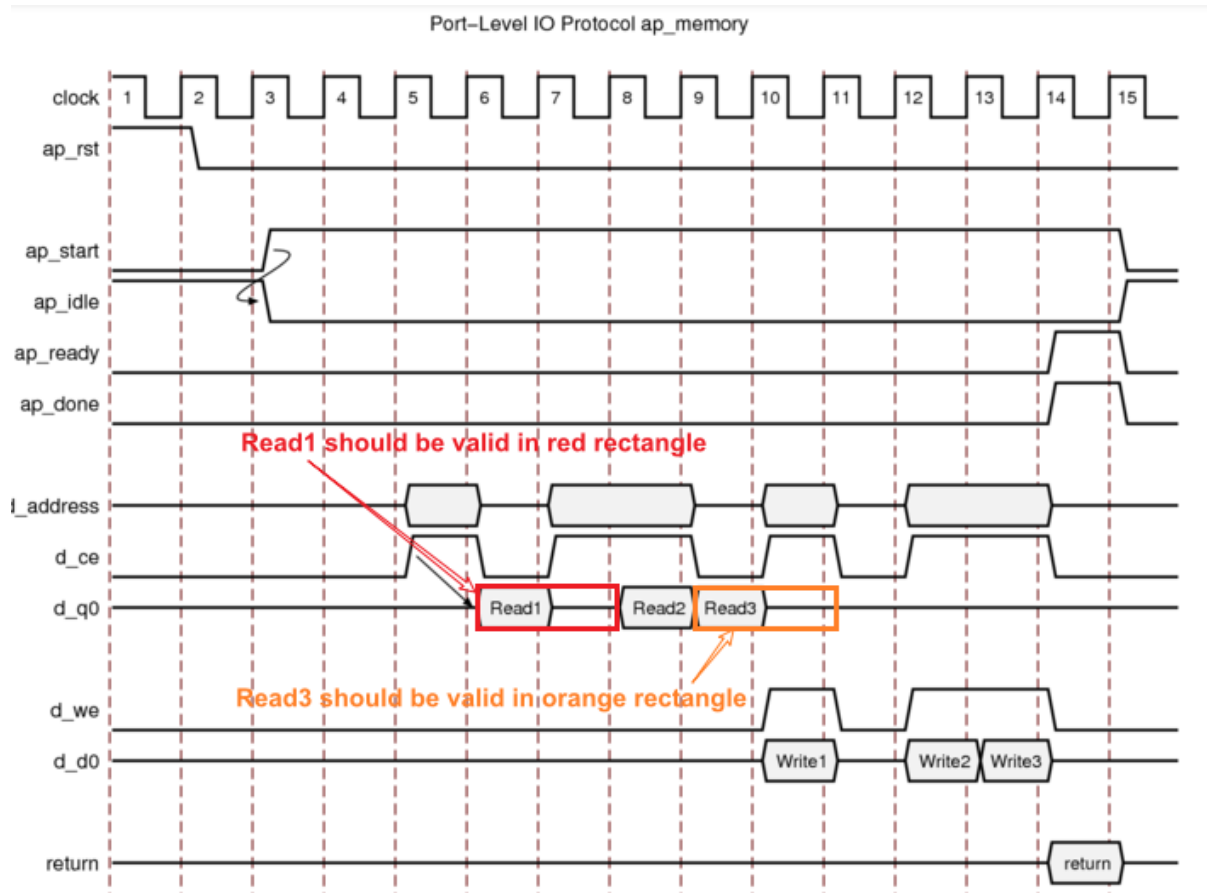
在以下示例中，in1 是一个指针，该指针访问当前地址，然后访问当前地址上面的两个地址，最后访问下面的一个地址。

```
void foo(int* in1, ...) {
    int data1, data2, data3;
    ...
    data1= *in1;
    data2= *(in1+2);
    data3= *(in1-1);
    ...
}
```

如果将 in1 指定为 ap_fifo 接口，则 Vitis HLS 会检查访问、判定访问并非按顺序进行，随即发出错误消息并中止。要读取非顺序地址位置，请使用 ap_memory 接口。

您不能在同时支持读取和写入的实参上指定 ap_fifo 接口。您只能在输入或输出实参上指定 ap_fifo 接口。含输入实参 in 和输出实参 out（指定为 ap_fifo 接口）的设计的行为如下图所示。

图 48：时延为 1 的 ap_memory 接口的行为



对于输入，将执行以下操作：

- 应用 `ap_start` 后，该块开始正常操作。
- 如果输入端口已准备好读取，但输入端口 `in_empty_n` 处于低电平状态，指示 FIFO 为空，则设计将停滞并等待数据可用。
- 当输入端口 `in_empty_n` 处于高电平状态，指示 FIFO 包含数据时，将断言输出确认 `in_read` 为高电平有效以指示当前周期中已读取数据。

对于输出，将执行以下操作：

- 应用 `start` 后，该块开始正常操作。
- 如果输出端口已准备好写入，但 `out_full_n` 处于低电平状态，指示 FIFO 已满，则数据将置于输出端口上，但设计将停滞并等待 FIFO 中的空间可用。
- 当 `out_full_n` 处于高电平状态，指示 FIFO 中有可用空间时，将断言输出确认信号 `out_write` 有效以指示输出数据为 `valid`。
- 如果顶层函数或顶层循环已使用 `-rewind` 选项进行流水打拍，则 Vitis HLS 将创建另一个后缀为 `_lwr` 的输出端口。当最后一次写入 FIFO 接口完成时，`_lwr` 端口将变为高电平有效。

HBM 中多端口访问的编程模型

高带宽存储器（或称 HBM）可通过在设计中将阵列拆分为不同 bank 或伪通道来提供高带宽。这是在高性能计算中将阵列分区为不同存储器区域的常用做法。与硬件交互的软件应用程序会分配单个缓冲器，该缓冲器将分布在各伪通道上。

支持在不同实参之间进行依赖关系分析

Vitis HLS 会把不同指针视作为各独立通道，并移除任何依赖关系分析。但该软件应用会为两个指针分配单个缓冲器，这样该工具即可通过 `pragma HLS ALIAS` 来保留依赖关系分析。ALIAS 编译指示可以提供有关指针距离的数据依赖关系分析。如需了解更多信息，请参阅 [ALIAS](#) 编译指示或指令。

在以下代码示例中，在 bank0 内分配 arg0 内核，在 bank1 内分配 arg1 内核。指针距离应在 ALIAS 编译指示的 `distance` 选项内指定，如下所示：

```
//Assume that the host code looks like this:
int *buf = clCreateBuffer(ctxt, CL_MEM_READ_ONLY, 2*bank_size, ...);
clSetKernelArg(kernel, 0, 0x20000000, buf);           // bank0
clSetKernelArg(kernel, 1, 0x20000000, buf+bank_size); // bank1

//The ALIAS pragma informs data dependence analysis about the pointer
distance
void kernel(int *bank0, int *bank1, ...)
{
#pragma HLS alias ports=bank0,bank1 distance=bank_size
```

ALIAS 编译指示可使用以下任一形式来指定：

- 恒定距离：

```
#pragma HLS alias ports=arr0,arr1,arr2,arr3 distance=1024
```

- 可变距离：

```
#pragma HLS alias ports=arr0,arr1,arr2,arr3 offset=0,512,1024,2048
```

约束：

- INTERFACE 编译指示中所有端口的深度都必须相同
- 所有端口都必须分配到不同捆绑，并绑定到不同 HBM 控制器
- 第二种形式下指定的端口数量与指定的偏移数量必须相同，每个偏移对应一个端口。不支持 `#pragma HLS interface offset=off`
- 每个端口仅限在一条 ALIAS 编译指示内使用

利用多个 M_AXI 接口改善 HBM 带宽

Vitis HLS 支持利用顶层阵列或指针实参上的各种功能特性，通过更改少量代码即可提高 C++ 代码的并行度和吞吐量。

改善 ARRAY_PARTITION 编译指示

[ARRAY_PARTITION](#) 编译指示或指令可应用于顶层实参上位于顶层函数接口处的阵列。该编译指示或指令与 INTERFACE 编译指示搭配一起使用，支持 Vitis HLS 创建为单个软件实参多个 M_AXI 接口，以便用于 HBM 伪通道。ARRAY_PARTITION `factor=N` 用于在 M_AXI 接口与 HBM 伪通道（或 HBM_PC）之间创建 N 个点点对点连接，以便将吞吐量提升至单个接口的 N 倍。

对于微架构，需要将阵列访问与关联数据路径显式并行化，以与 M_AXI 接口增加相匹配。这从编码角度亦可解读为相似的并行化操作。

阵列分区类型可设为 complete（完整）、cyclic（循环）或 block（块），但对于此应用，仅使用 cyclic 或 block。每个类型都能产生不同的微架构。

- 循环分区可在循环内使用：针对阵列的连续访问将连接到不同的 M_AXI 及其匹配的 HBM_PC，您可以使用相同因子来部分展开此循环。

```
constexpr int partitions = 4;
constexpr int NWORDS=32<<20;
void example(int a[NWORDS] , int b[1]) {
    #pragma HLS INTERFACE m_axi port=a depth=NWORDS bundle=gmem
    max_widen_bitwidth=512
    #pragma HLS ARRAY_PARTITION cyclic variable=a factor=partitions
    #pragma HLS INTERFACE m_axi port=b depth=1 bundle=outmem
    int tot=0;
accessloop:
    for (int i=0; i < NWORDS; ++i) {
        #pragma HLS PIPELINE II=1
        #pragma HLS UNROLL factor=partitions
        tot+= a[i] * i; // computation on a[i]
    }
    b[0]=tot;
}
```

前述示例显示了使用明确大小的阵列而不是指针，并通过 INTERFACE 编译指示提供深度选项。这两种方法效果相同，C-RTL 协同仿真需要采取其中至少一种方法来正确调整 RTL 缓冲器大小。搭配 Vitis 目标流程使用时，并不需要有关实参和接口的大小信息，因为指针大小自动设为 64 位。此外，循环分区与大小无关。

- 如果设计包含相同计算函数的多个实例，并且每个计算函数都将通过不同 M_AXI 及其匹配的 HBM_PC 来访问该阵列的不同块，那么可使用块分区。

```
constexpr int partitions = 4;
constexpr int NWORDS=32<<20;
void my_pe(int a[NWORDS/partitions], int &b, int offset) { // PE
    processes a fraction of the data
    // offset is needed to adjust the coefficient to multiply the array
    value
    int tot=0;
accessloop:
    for (int i=0; i < NWORDS/partitions; ++i) {
        #pragma HLS PIPELINE II=1
        tot += a[i] * (i+offset); // computation on a[i]
    }
    b=tot;
}

void reduce(int *partial_totals, int &out) {
    for (int i=tot=0; i < partitions; ++i) {
        #pragma HLS PIPELINE off
        tot+=partial_totals[i];
    }
    out = tot;
}

void example(int a[NWORDS ], int &b) {
    #pragma HLS INTERFACE m_axi port=a depth=NWORDS bundle=gmem
    max_widen_bitwidth=MAXWBW
    #pragma HLS ARRAY_PARTITION block variable=a factor=partitions
    #pragma HLS INTERFACE m_axi port=b depth=1 bundle=outmem
    int partial_totals[partitions], tot;
```

```
#pragma HLS ARRAY_PARTITION complete variable=partial_totals
#pragma HLS DATAFLOW
instanceloop_unrolled:
for (int i=0; i < partitions; ++i) {
#pragma HLS UNROLL
    // each instance accesses a different block
    my_pe( &a[NWORDS*i/partitions], partial_totals[i], NWORDS*i/
partitions);
}
    reduce(partial_totals, b);
}
```

在以上示例中，由于块分区与原始阵列的大小相关，因此阵列大小信息是必需的。

所生成的 M_AXI 接口将带有其捆绑名称，此名称衍生自原始捆绑名称并附加了十进制后缀。例如，pragma HLS INTERFACE bundle=gmem 采用分区因子 4，由此生成 4 个 M_AXI，分别名为 m_axi_gmem_0、m_axi_gmem_1、m_axi_gmem_2 和 m_axi_gmem_3。您将需要更新连接设置，以便系统集成所有 M_AXI 接口。如需了解更多信息，请参阅《使用 Vitis 进行嵌入式设计开发》(UG1701) 中的《将内核端口映射到存储器》。

在以上示例中，软件应用用于调用 example() 模块的函数签名将因使用 ARRAY_PARTITION factor=4 而发生改变。此函数签名会从 example(int a[NWORDS], int b[1]) 改为：

```
void example(
    int a_0[NWORDS/4],
    int a_1[NWORDS/4],
    int a_2[NWORDS/4],
    int a_3[NWORDS/4],
    int b[1]);
```

为主机侧阵列分区生成的帮助程序函数

该软件应用将不得不把包含该阵列的缓冲器分区为相同循环分区或块分区以供 HLS 设计使用，如上所述。为帮助完成此任务，Vitis HLS 会生成帮助程序函数，以便将主机阵列缓冲器拆分为多个分区，并生成逆向帮助程序函数以重新将多个分区组合为单一阵列。这些帮助程序函数包括：

- <top_name>_Set_<array_name> 用于将阵列划分为多个分区。例如，搭配 factor=4 和 int 数据类型使用：

```
void <DUT>_Set_<ARG>(int *dst[4], int *src, unsigned long long
num_elements);
```

- <top_name>_Get_<array_name> 用于将多个分区重新组合为单个阵列。例如，搭配 factor=4 和 int 数据类型使用：

```
void <DUT>_Get_<ARG>(int *dst, int *src[4], unsigned long long
num_elements)
```



提示：实参顺序遵循如下约定：destination(s)、source(s) 和 number of elements。

- 生成的帮助程序函数可作为汇编文件以供使用：./project/solution/impl/ip/drivers/example_v1_0/src/hbm_helper_x86_64.s & hbm_helper_aarch64.s
- 生成的帮助程序函数也可作为编译文件以供使用：./project/solution/impl/ip/drivers/example_v1_0/src/hbm_helper_x86_64.o & hbm_helper_aarch64.o

注释：头文件并不会自动生成，因此您需要在软件应用中包含函数声明。以下演示了 factor=4 和 int 数据类型的示例：

```
extern "C" void example_Set_a(int **, int *, long);
extern "C" void example_Get_a(int *, int **, long);
```

用法示例：

```
int *host_in=new int[NWORDS];
for (size_t i = 0; i < NWORDS; ++i) {
    host_in[i] = ... // initialize the host input memory
}

xrt::bo bo_Inputs[partitions];
int* bufIn_map[partitions];
for (int i = 0; i < partitions; i++) {
    // buffer object matching kernel arguments/memory banks
    bo_Inputs[i] = xrt::bo(device,NWORDS*sizeof(int)/partitions,
krnl.group_id(i));
    // Map buffer object data to user pointer for manipulation
    bufIn_map[i] = bo_Inputs[i].template map<int*>();
}

// partition the input data into the mapped partitions
example_Set_a(bufIn_map, host_in, NWORDS);

xrt::run run(krnl); // run object for that kernel

for (int i = 0; i < partitions; i++) {
    // sync updated bo contents to board
    bo_Inputs[i].sync(XCL_BO_SYNC_BO_TO_DEVICE);
}
for (int i = 0; i < partitions; i++) {
    // set arguments according to function signature
    run.set_arg(i,bo_Inputs[i]);
}
...
run.start();
run.wait();
```

利用 SSI 技术器件管理接口

某些 AMD 器件使用堆叠硅片互联 (SSI) 技术。在此类器件中，全部可用资源被划分为多个超级逻辑区域 (SLR)。SLR 之间的连接使用超长线路 (SLL) 布线。SLL 布线产生的延迟成本通常大于标准 FPGA 布线。为确保设计以最大性能运行，请遵循如下准则：

- 将跨 SLR 的所有信号都寄存在 SLR 输出和 SLR 输入中。
- 通过 I/O 缓冲器进出 SLR 的信号无需寄存。
- 请确保 Vitis HLS 所创建的逻辑适用于单一 SLR 内。

注释：选择 SSI 技术器件作为目标技术时，使用率报告将包含有关 SLR 使用率和器件总使用率的详细信息。

如果逻辑包含在单一 SLR 器件内，HLS 工具提供了 `syn.rtl.register_all_io` 配置命令。如果启用该选项，则会寄存所有输入和输出。如果禁用该选项，则不寄存任何输入或输出。

Vitis HLS 存储器布局模型

Vitis 应用加速开发流程提供了相应的框架，可通过使用标准编程语言来为软件和硬件组件开发和交付 FPGA 加速应用。软件组件或主机程序是使用 C/C++ 语言开发的，可在 x86 或嵌入式处理器上运行，借助 OpenCL 或 XRT 本机 API 调用来管理与加速器的运行时间交互。硬件组件或内核（在实际 FPGA 卡/平台上运行）则可使用 C/C++、OpenCL C 或 RTL 来开发。Vitis 软件平台有助于促进对异构应用的硬件和软件元素进行并发开发和测试。因此，在主机上运行的软件程序需要使用精确定义的接口和协议来与在 FPGA 硬件模型上运行的加速内核进行通信。因此，精确定义所使用的存储器模型以便正确处理读写的数据就显得尤为重要。存储器模型可以定义计算机存储器中排列和访问数据的方式。其中涉及两个独立但相关的问题：数据对齐和数据结构填充。此外，Vitis HLS 编译器支持指定特殊属性（和编译指示）来更改默认的数据对齐和数据结构填充规则。

数据对齐

软件程序员习惯于将存储器视为简单的字节阵列，而将基本数据类型视为是由一个或多个存储器块组成的。但计算机处理器并不会在单个字节大小的区块内对存储器执行读取和写入。现今的现代化 CPU 实际上每次访问 2、4、8、16 甚至是 32 字节的区块，虽说最常用的指令集架构 (ISA) 是 32 位和 64 位的。鉴于系统内存存储器的组织方式，这些区块的地址应为其大小的倍数。如果地址满足此要求，则将其称为已对齐。高级程序员对于存储器的看法与现代化处理器实际处理存储器的方式之间的差别在应用程序的正确性和性能方面实际上是非常重要的。例如，如果您不了解软件中的地址对齐问题，那么就可能发生下列情况：

- 软件运行缓慢
- 应用将锁定/挂起
- 操作系统可能崩溃
- 软件将静默失败，产生错误结果

C++ 语言可以提供一系列不同大小的基本类型。为了能快速操作这些类型的变量，生成的对象代码将尝试使用能立即读取/写入整个数据类型的 CPU 指令。这也就意味着这些类型的变量在存储器内的布局方式应确保其地址能以合适方式保持对齐。由此导致每个基本类型除大小之外还有另一个属性：即对齐要求。基本类型的对齐可能看似与其大小相同。但实际情况往往并非如此，因为最适合特定类型的 CPU 指令可能每次只能访问其数据中的一部分。例如，32 位 x86 GNU/Linux 机器可能每次只能读取最多 4 字节，因此，64 位 `long long` 类型的大小为 8，对齐为 4。下表显示了 C/C++ 中对应 32 位和 64 位 x86-64 GNU/Linux 机器的基本原生数据类型的大小和对齐（以字节数为单位）。

表 13：数据类型

类型	32 位 x86 GNU/Linux		64 位 x86 GNU/Linux	
	大小	对齐	大小	对齐
bool	1	1	1	1
char	1	1	1	1
short int	2	2	2	2
int	4	4	4	4
long int	4	4	8	8
long long int	8	4	8	8
float	4	4	4	4
double	8	4	8	8
long double	12	4	16	16
void*	4	4	8	8

根据以上排列，程序员为什么需要更改对齐？原因有多个，但主要原因是在存储器要求与性能之间的取舍。在主机与加速器之间往返发送数据时，发射的每个字节都有成本。幸好，C++ 编译器可提供语言扩展 `__attribute__((aligned(X)))`，用于为变量、结构/类或结构字段更改默认对齐（以字节数为单位来测量）。例如，以下声明会导致编译器在 16 字节边界上分配全局变量 `x`。

```
int x __attribute__((aligned (16))) = 0;
```

`__attribute__((aligned (X)))` 不会更改所应用到的变量的大小，而是可以更改结构的存储器布局，方法是在结构体的各元素之间插入填充。由此即可更改结构的大小。如果您在已对齐的属性中不指定对齐因子，那么编译器会将声明的变量或字段的对齐设置为要编译的目标机器上的任意数据类型所使用的最大对齐。这样通常能够提升复制操作的效率，因为编译器在对已通过这种方式对齐的变量或字段执行往来复制时，可以使用任何能复制最大存储器区块的指令。`aligned` 属性只能增大对齐，而不能减小对齐。C++ 函数 [offsetof](#) 可用于判定结构中每个成员元素的对齐。

数据结构填充

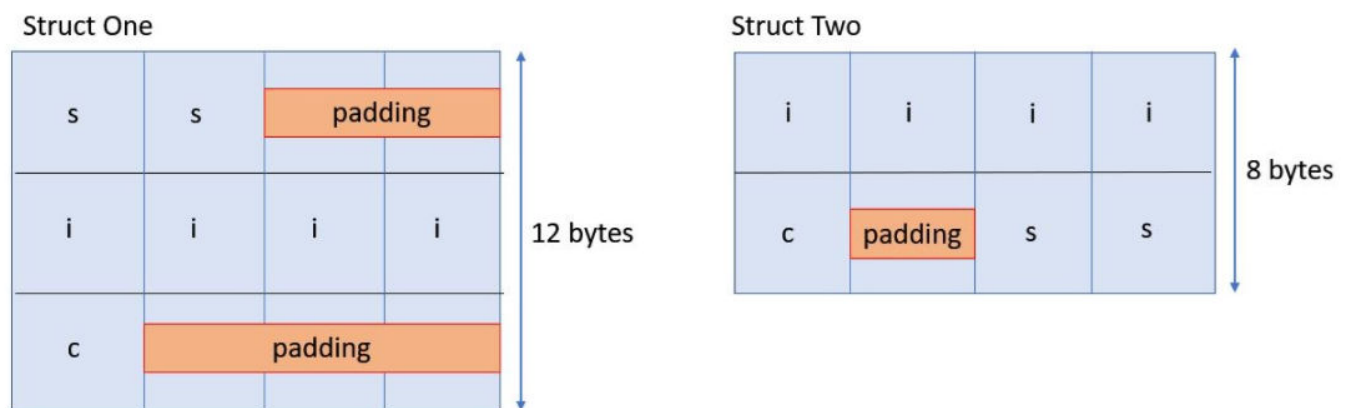
如 [数据对齐](#) 中的表格所示，原生数据类型具有明确定义的对齐结构，那么用户定义的数据类型又如何呢？C++ 编译器也需要确保结构体或类中的所有成员变量都正确对齐。为此，编译器会在成员变量之间插入填充字节。此外，为了确保用户定义类型的阵列中的每个元素都对齐，编译器可在最后一个数据成员之后添加额外的填充。请考虑以下示例：

<pre>struct One { short int s; int i; char c; }</pre>	<pre>struct Two { int i; char c; short int s; }</pre>
---	---

编译器始终假定 `struct One` 的实例的起始地址对齐到所有结构体成员的要求最严格的对齐地址处，在此例中，即 `int`。实际上，用户定义的对齐要求正是以此方式计算所得的。假定存储器为 x86-64 对齐，其中 `short int` 的对齐为 2，`int` 的对齐为 4，为了使 `struct One` 的 `i` 数据成员得以适当对齐，编译器需要在 `s` 与 `i` 之间插入 2 个额外的填充字节以创建对齐，如下图所示。同样，为了对齐数据成员 `c`，编译器需要在 `c` 之后插入 3 字节。

对于 `struct One`，编译器会基于结构体元素的排列方式推断总大小为 12 字节。但如果结构体的元素已重新排序（如 `struct Two` 中所示），那么编译器现在即可推断得到较小的大小，即 8 字节。

图 49：结构体填充



默认情况下，C/C++ 编译器会按结构体成员的声明顺序来完成这些成员的布局，在成员间或者最后一个成员之后可能按需插入填充字节，以确保每个成员都能正确对齐。但是，C++ 编译器会提供语言扩展 `__attribute__((packed))`，以告知编译器不插入填充，而是改为允许结构体成员间不对齐。例如，如果系统正常情况下要求所有 `int` 对象都采用 4 字节对齐，那么使用 `__attribute__((packed))` 可能导致将 `int` 结构体成员分配至非对齐偏移处。

您必须审慎考量 `__attribute__((packed))` 的使用，因为访问未对齐的存储器可能导致编译器插入代码以逐个字节读取存储器，而不是一次性读取多个存储器区块。

Vitis HLS 对齐规则和语义

鉴于前述 C++ 编译器的行为，本节将详述 Vitis HLS 如何使用 `aligned` 和 `packed` 属性来创建高效硬件。首先，您需要了解 Vitis HLS 中的 **聚合** 和 **解聚** 功能特性。代码中的结构或类对象（例如，内部变量和全局变量）默认情况下处于解聚 (disaggregated) 状态。解聚暗示此结构/类被分解为多个独立对象，每个结构体/类成员对应一个对象。创建的元素数量和类型取决于结构体本身的内容。结构体阵列作为多个阵列来实现，每个结构体成员都具有独立的阵列。

但默认情况下，用作为顶层函数的实参的结构体则保持聚合状态。聚合暗示某个结构体的所有元素都被收集到单个宽向量内。这样即可同时读写结构体的所有成员。结构体的成员元素按 C/C++ 代码中所示顺序置于该向量内：结构体的第一个元素对齐向量的 LSB，结构体的最后一个元素对齐向量的 MSB。结构体中的任意阵列都分区到独立阵列元素中，并按从低到高的顺序置于向量内。

表 15：接口实参和内部变量

	无 AGGREGATE 编译指示时的行为		含 AGGREGATE 编译指示时的行为 (<code>compact=auto</code> 或者未指定 <code>compact</code>)	
	接口实参	内部变量	接口实参	内部变量
AXI 协议接口 (<code>m_axi/s_axilite/axis</code>)	aggregate <code>compact=none</code>	不适用	<code>compact=none</code>	不适用
包含 <code>hls::stream</code> 对象的结构体/类	自动解聚结构体/类	自动解聚结构体/类	不适用	不适用
其他接口协议	aggregate <code>compact=bit</code>	自动解聚	<code>compact=bit</code>	<code>compact=bit</code>

Vitis HLS 中的默认聚合目的是在硬件接口顶层使用 `x86_64-gnu-linux` 存储器布局，同时优化内部硬件以提升结果质量 (QoR)。上表显示了 Vitis HLS 的默认行为。该表中显示了 2 种模式：用户不指定 AGGREGATE 编译指示（默认模式），和用户指定 AGGREGATE 编译指示的模式。

对于 AXI4 接口 (`m_axi/s_axilite/axis`)，默认按照结构体元素的顺序对该结构体进行填充，如 **数据结构填充** 中所述。这样即可将结构大小聚合为最接近的 2 的幂值，并且在此情况下可应用部分填充。这样即可有效推断 AGGREGATE 编译指示上的 `compact=none` 选项。

对于其他接口协议，按位级来对结构体进行打包，因此聚合的向量的大小仅等于该结构体的各元素大小之和。这样即可有效推断 AGGREGATE 编译指示上的 `compact=bit` 选项。

以上规则的唯一例外是在接口中以间接方式使用 `hls::stream` 时（即，在结构体/类内部使用 `hls::stream` 对象，随后将此结构体/类用作为接口端口的类型）。包含 `hls::stream` 对象的结构体将始终解聚为其各独立成员元素。

聚合示例

聚合存储器映射接口

这是 `m_axi` 接口的 AGGREGATE 编译指示或指令的示例。GitHub 上的 [aggregation_of_m_axi_ports](#) 提供了如下所示示例。

```

struct A {
    char foo;        // 1 byte
    short bar;       // 2 bytes
};

int dut(A* arr) {
    #pragma HLS interface m_axi port=arr depth=10
    #pragma HLS aggregate variable=arr compact=auto
    int sum = 0;
    for (unsigned i=0; i<10; i++) {
        auto tmp = arr[i];
        sum += tmp.foo + tmp.bar;
    }
    return sum;
}

```

对于以上示例，`m_axi` 接口端口 `arr` 的大小为 3 字节（或 24 位），但由于 AGGREGATE `compact=auto` 编译指示的作用，该端口的大小将对齐到 4 字节（或 32 位），因为这是最接近的 2 的幂值。Vitis HLS 将在 log 日志文件中发出以下消息：

```

INFO: [HLS 214-241] Aggregating maxi variable 'arr' with compact=none mode
in 32-bits (example.cpp:19:0)

```



提示：仅当使用 AGGREGATE 编译指示时，才会发出以上消息。但即使不使用编译指示，该工具仍将自动聚合接口端口 `arr` 并将其填充至 4 字节，这是 AXI 接口端口的默认行为。

在接口上聚合结构体

这是 `ap_fifo` 接口的 AGGREGATE 编译指示或指令的示例。GitHub 上的 [aggregation_of_struct](#) 提供了如下所示示例。

```

struct A {
    int myArr[3];      // 4 bytes per element (12 bytes total)
    ap_int<23> length; // 23 bits
};

int dut(A arr[N]) {
    #pragma HLS interface ap_fifo port=arr
    #pragma HLS aggregate variable=arr compact=auto
    int sum = 0;
    for (unsigned i=0; i<10; i++) {
        auto tmp = arr[i];
        sum += tmp.myArr[0] + tmp.myArr[1] + tmp.myArr[2] + tmp.length;
    }
    return sum;
}

```

对于 `ap_fifo` 接口，无论是否使用聚合编译指示，结构体都将在位级进行封装。

在以上示例中，AGGREGATE 编译指示将为端口 `arr` 创建大小为 119 位的端口。阵列 `myArr` 将取 12 字节（或 96 位），元素 `length` 将取 23 位，总计 119 位。Vitis HLS 将在 log 日志文件中发出以下消息：

```
INFO: [HLS 214-241] Aggregating fifo (array-to-stream) variable 'arr' with
compact=bit mode
      in 119-bits (example.cpp:19:0)
```

聚合嵌套结构体端口

这是 Vivado IP 流程中 AGGREGATE 编译指示或指令的示例。GitHub 上的 [aggregation_of_nested_structs](#) 提供了如下所示示例。

```
#define N 8

struct T {
    int m;    // 4 bytes
    int n;    // 4 bytes
    bool o;   // 1 byte
};

struct S {
    int p;    // 4 bytes
    T q;      // 9 bytes
};

void top(S a[N], S b[N], S c[N]) {
#pragma HLS interface bram port=c
#pragma HLS interface ap_memory port=a
#pragma HLS aggregate variable=a compact=byte
#pragma HLS aggregate variable=b compact=bit
#pragma HLS aggregate variable=c compact=byte
    for (int i=0; i<N; i++) {
        c[i].q.m = a[i].q.m + b[i].q.m;
        c[i].q.n = a[i].q.n - b[i].q.n;
        c[i].q.o = a[i].q.o || b[i].q.o;
        c[i].p = a[i].q.n;
    }
}
```

在以上示例中，聚合算法将为端口 `a` 和端口 `c` 创建大小为 104 位的端口，因为在聚合编译指示中已指定 `compact=byte` 选项，但针对端口 `b` 使用默认选项 `compact=bit`，其打包大小将为 97 位。嵌套的结构 `S` 和 `T` 将聚合以包含 3 个 32 位成员变量（`p`、`m` 和 `n`）以及 1 个位/字节成员变量（`o`）。



提示：此示例使用 Vivado IP 流程来演示聚合行为。在 Vitis 内核流程中，端口 `b` 将自动推断为 `m_axi` 端口，故而不允许使用 `compact=bit` 设置。

Vitis HLS 将在 log 日志文件中发出以下消息：

```
INFO: [HLS 214-241] Aggregating bram variable 'b' with compact=bit mode in
97-bits (example.cpp:19:0)
INFO: [HLS 214-241] Aggregating bram variable 'a' with compact=byte mode in
104-bits (example.cpp:19:0)
INFO: [HLS 214-241] Aggregating bram variable 'c' with compact=byte mode in
104-bits (example.cpp:19:0)
```

解聚示例

解聚 AXIS 接口

这是 `axis` 接口的 `DISAGGREGATE` 编译指示或指令的示例。GitHub 上的 `disaggregation_of_axis_port` 提供了如下所示示例。

表 16：解聚 AXIS 接口上的结构体

HLS 源代码	已综合的 IP 模块
<pre>#define N 10 struct A { char c; int i; }; void dut(A in[N], A out[N]) { #pragma HLS interface axis port=in #pragma HLS interface axis port=out #pragma HLS disaggregate variable=in #pragma HLS disaggregate variable=out int sum = 0; for (unsigned i=0; i<N; i++) { out[i].c = in[i].c; out[i].i = in[i].i; } }</pre>	<pre>module dut (ap_local_block, ap_local_deadlock, ap_clk, ap_rst_n, ap_start, ap_done, ap_idle, ap_ready, in_c_TVALID, in_i_TVALID, out_c_TREADY, out_i_TREADY, in_c_TDATA, in_c_TREADY, in_i_TDATA, in_i_TREADY, out_c_TDATA, out_c_TVALID, out_i_TDATA, out_i_TVALID);</pre>

在以上解聚示例中，结构体实参 `in` 和 `out` 均映射到 `AXIS` 接口，然后进行解聚。这导致 Vitis HLS 为每个实参创建 2 条 AXI 串流：`in_c`、`in_i`、`out_c` 和 `out_i`。结构体 `A` 的每个成员都各成一条独立串流。

生成的模块的 RTL 接口显示在上表右侧，其中成员元素 `c` 和 `i` 均为独立 AXI 串流端口，每个端口都有自己的 `TVALID`、`TREADY` 和 `TDATA` 信号。

Vitis HLS 将在 log 日志文件中发出以下消息：

```
INFO: [HLS 214-210] Disaggregating variable 'in' (example.cpp:19:0)
INFO: [HLS 214-210] Disaggregating variable 'out' (example.cpp:19:0)
```

解聚 HLS::STREAM

这是搭配 `hls::stream` 类型一起使用的 `DISAGGREGATE` 编译指示或指令的示例。

表 17: HLS::STREAM 的结构体（已解聚）

HLS 源代码	已综合的 IP 模块
<pre>#define N 1024 struct A {, hls::stream<int> s_in; long arr[N]; }; long dut(struct A &d) { long sum = 0; while(!d.s_in.empty()) sum += d.s_in.read(); for (unsigned i=0; i<N; i++) sum += d.arr[i]; return sum; }</pre>	<pre>module dut (ap_local_block, ap_local_deadlock, ap_clk, ap_rst, ap_start, ap_done, ap_idle, ap_ready, d_s_in_dout, d_s_in_empty_n, d_s_in_read, d_arr_ce0, d_arr_q0, ap_return);</pre>

如果在用于接口中的结构内部使用 `hls::stream` 对象，则会导致 Vitis HLS 编译器将该结构体端口自动解聚。如以上示例所示，生成的 RTL 接口将为 `hls::stream` 对象 `s_in` 包含独立的 RTL 端口（名为 `d_s_in_*`），并为阵列 `arr` 包含独立的 RTL 端口（名为 `d_arr_*`）。

Vitis HLS 将在 log 日志文件中发出以下消息：

```
INFO: [HLS 214-210] Disaggregating variable 'd'
INFO: [HLS 214-241] Aggregating fifo (hls::stream) variable 'd_s_in' with compact=bit mode in 32-bits
```

结构体大小对于流水打拍的影响

函数接口中使用的结构体的大小对于循环中的流水打拍可能存在不利影响，主要影响的是有权访问函数体中的接口的函数。以下列包含 2 个 M_AXI 接口的代码为例：

```
struct A { /* Total size = 192 bits (32 x 6) or 24 bytes */
    int s_1;
    int s_2;
    int s_3;
    int s_4;
    int s_5;
    int s_6;
};

void read(A *a_in, A buf_out[NUM]) {
    READ:
    for (int i = 0; i < NUM; i++)
    {
        buf_out[i] = a_in[i];
    }
}

void compute(A buf_in[NUM], A buf_out[NUM], int size) {
    COMPUTE:
    for (int j = 0; j < NUM; j++)
    {
        buf_out[j].s_1 = buf_in[j].s_1 + size;
        buf_out[j].s_2 = buf_in[j].s_2;
        buf_out[j].s_3 = buf_in[j].s_3;
    }
}
```

```

        buf_out[j].s_4 = buf_in[j].s_4;
        buf_out[j].s_5 = buf_in[j].s_5;
        buf_out[j].s_6 = buf_in[j].s_6 % 2;
    }
}

void write(A buf_in[NUM], A *a_out) {
    WRITE:
    for (int k = 0; k < NUM; k++)
    {
        a_out[k] = buf_in[k];
    }
}

void dut(A *a_in, A *a_out, int size)
{
    #pragma HLS INTERFACE m_axi port=a_in bundle=gmem0
    #pragma HLS INTERFACE m_axi port=a_out bundle=gmem1
    A buffer_in[NUM];
    A buffer_out[NUM];

    #pragma HLS dataflow
    read(a_in, buffer_in);
    compute(buffer_in, buffer_out, size);
    write(buffer_out, a_out);
}

```

在以上示例中，结构体 A 的大小为 192 位，这并非 2 的幂。如前文所述，默认情况下，所有 AXI4 接口大小均设为 2 的幂。Vitis HLS 将自动把这 2 个 M_AXI 接口（a_in 和 a_out）的大小调整为 256 - 即与 192 位大小最接近的 2 的幂值（并在 log 日志文件中报告此行为，如下所示）。

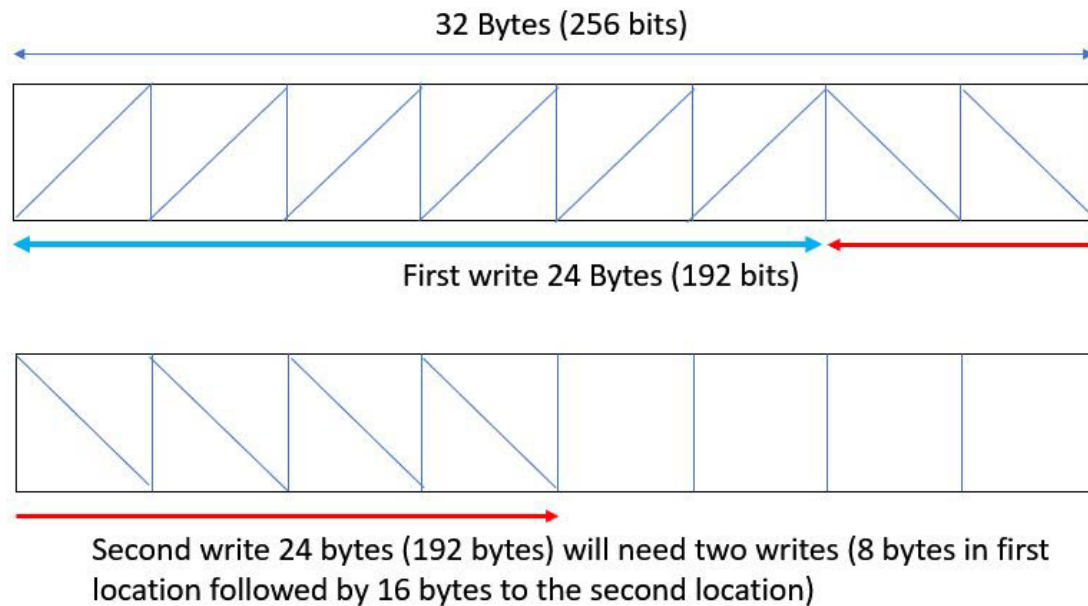
```

INFO: [HLS 214-241] Aggregating maxi variable 'a_out' with compact=none
mode in
256-bits (example.cpp:49:0)
INFO: [HLS 214-241] Aggregating maxi variable 'a_in' with compact=none mode
in 256-bits
(example.cpp:49:0)

```

这其中隐含的意义是在写入结构数据并输出时，首次写入将在一个周期内把 24 字节写入第一个缓冲器，但第二次写入将必须把 8 字节写入第一个缓冲器内的剩余 8 字节，然后将 16 字节写入第二个缓冲器，这样就会生成 2 次写入，如下图所示。

图 50: 对齐错误的写入周期



这将导致函数 `write()` 中的 `WRITE` 循环的 II 发生 II 违例, 因为它需要 `II=2` 而不是 `II=1`。读取时也会发生类似的行为, 因此 `read()` 函数同样会发生 II 违例, 因为它需要 `II=2`。Vitis HLS 将为函数 `read()` 和 `write()` 中的 II 违例发出以下警告:

```
WARNING: [HLS 200-880] The II Violation in module 'read_r' (loop 'READ'):
Unable
to enforce a carried dependence constraint (II = 1, distance = 1, offset =
1) between
bus read operation ('gmem0_addr_read_1', example.cpp:23) on port 'gmem0'
(example.cpp:23)
and bus read operation ('gmem0_addr_read', example.cpp:23) on port 'gmem0'
(example.cpp:23).
```

```
WARNING: [HLS 200-880] The II Violation in module 'write_Pipeline_WRITE'
(loop 'WRITE'):
Unable to enforce a carried dependence constraint (II = 1, distance = 1,
offset = 1)
between bus write operation ('gmem1_addr_write_ln44', example.cpp:44) on
port 'gmem1'
(example.cpp:44) and bus write operation ('gmem1_addr_write_ln44',
example.cpp:44) on
port 'gmem1' (example.cpp:44).
```

修复此类 II 问题的方法是以 8 个额外字节填充结构体 A, 这样即可始终同时写入 256 位 (32 字节), 或者使用下表所示的其他替代方法。这将允许调度器在 `READ/WRITE` 循环内按 `II=1` 来调度读/写。

表 18：结构体对齐

码块	描述
<pre> struct A { int s_1; int s_2; int s_3; int s_4; int s_5; int s_6; int pad_1; int pad_2; }; </pre>	通过添加所需的填充元素，将结构体的总大小定义为 256 位 (32 x 8) 或 32 字节。
<pre> struct A { int s_1; int s_2; int s_3; int s_4; int s_5; int s_6; } __attribute__((aligned(32))); </pre>	使用标准 <code>__aligned__</code> 属性。
<pre> struct alignas(32) A { int s_1; int s_2; int s_3; int s_4; int s_5; int s_6; } </pre>	使用 C++ 标准 <code>alignas</code> 类型说明符来为变量和用户定义的类型指定定制对齐方式。

HLS 设计的执行模式

HLS 设计的执行模式表示设计作为块（或模块）的工作方式（涉及块本身以及块内的函数）、设计与其他块或模块之间的关系，或者设计在对此块进行寻址的软件外部进行工作的方式。这些模式是由分配给 HLS 设计的块控制协议（如[块级控制协议](#)中所述）以及 HLS 设计的内部结构（如[HLS 抽象并行编程模型](#)中所述）来确定的。

内核的执行模式包括：

- 重叠模式：允许在当前传输事务完成之前就开始执行下一项新传输事务。函数流水打拍、循环回绕或数据流执行允许在设计就绪后，重叠块运行立即开始处理其他数据。

控制协议：使用具有反压支持的 `ap_ctrl_chain` 或 `ap_ctrl_hs`。反压是一种允许下游模块控制数据流动的机制。如果下游模块尚未准备好接收新数据，它可以断言一个信号，使上游模块暂时停滞。这样可以防止数据丢失，并确保流水线不同阶段的同步。

- 顺序模式：要求当前传输事务完成后，才能开始新的传输事务。

控制协议：可使用不含反压支持的 `ap_ctrl_hs` 或 `ap_ctrl_chain` 来实现。由于消除反压，设计假定数据耗用的速率保持不变，因此各阶段之间无需流量控制。

- 连续运行模式：连续运行的内核会无限期运行，直到显式复位或运行达到一定次数为止，这使该模式与传统软件加速内核有着本质区别。与受调用、执行并在完成时返回的传统软件内核相反，连续运行的内核会模仿 `while(1)` 循环连续执行。在硬件环境中这些内核尤为有用，因为在此类环境中，数据串流输入没有预定义的边界或已知的完成点。

连续运行的内核常用于满足以下条件的应用：在此类应用中，将数据连续不断馈送到系统内，并且该内核必须不间断处理这些数据。示例包括：

- **联网：**在主机系统未提前知晓数据包总数的情况下，处理从以太网端口传入的数据包。例如，内核可能以连续执行的方式处理从 `TVALID`（有效数据）到 `TLAST`（最后一个数据包）的数据包。
- **防火墙：**基于规则的简单防火墙，其中规则由主机在复位时写入一次，内核会连续处理数据包。主机可以随时读取丢包计数器，但这些计数器是内核单次执行的结果。
- **网络路由：**该网络路由器会在执行期间更新路由表，但连续处理传入数据包的操作不会停止。
- **负载均衡：**该负载均衡器会根据散列映射对数据进行路由，在持续处理流量的同时更新服务器列表和 IP 地址。
- **金融科技：**在金融应用中会连续处理来自以太网端口的数据包，主机则会松散监控执行状态，而不中断进行中的操作。

控制协议：在下述两种场景下，均可对连续运行模式建模。

- 处于自动重启模式下使用 `ap_ctrl_chain` 块级协议的控制驱动的任务级并行 (CDTLP)。
- 使用 `ap_ctrl_none` 块级协议的数据驱动的任务级并行 (DTLP)。
- 自动重启：自动重启模式允许 HLS 设计在准备好开始处理其他数据后自动重新启动该模块。此模式支持重叠执行和顺序执行。自动重启是适用于数据驱动的 TLP 设计的方法，但也可按 [自动重启模式](#) 中所述在控制驱动的 TLP 设计内实现。

块级控制协议

Vitis 内核或 Vivado IP 的执行模式是由块级控制协议和 HLS 设计内子函数的结构来定义的。对于控制驱动的 TLP，`ap_ctrl_chain` 协议和 `ap_ctrl_hs` 协议支持顺序执行和流水打拍执行。对于数据驱动的 TLP，`ap_ctrl_none` 是必需的控制协议。

`ap_ctrl_chain` 控制协议是 Vitis 内核流程的默认协议，如 [适用于 Vitis 内核流程的接口](#) 中所述。`ap_ctrl_hs` 块级控制协议是 Vivado IP 流程的默认协议，如 [Vivado IP 流程接口](#) 中所述。但要将 HLS 设计链接起来以便为流水打拍执行提供更好的支持，应使用 `ap_ctrl_chain`。

您可在函数返回时使用 [INTERFACE](#) 编译指示或指令指定块级控制协议。如果 C/C++ 语言代码不返回值，您仍可在函数返回时指定控制协议。如果 C/C++ 语言代码使用函数返回，那么 Vitis HLS 会为返回值创建 `ap_return` 输出端口。

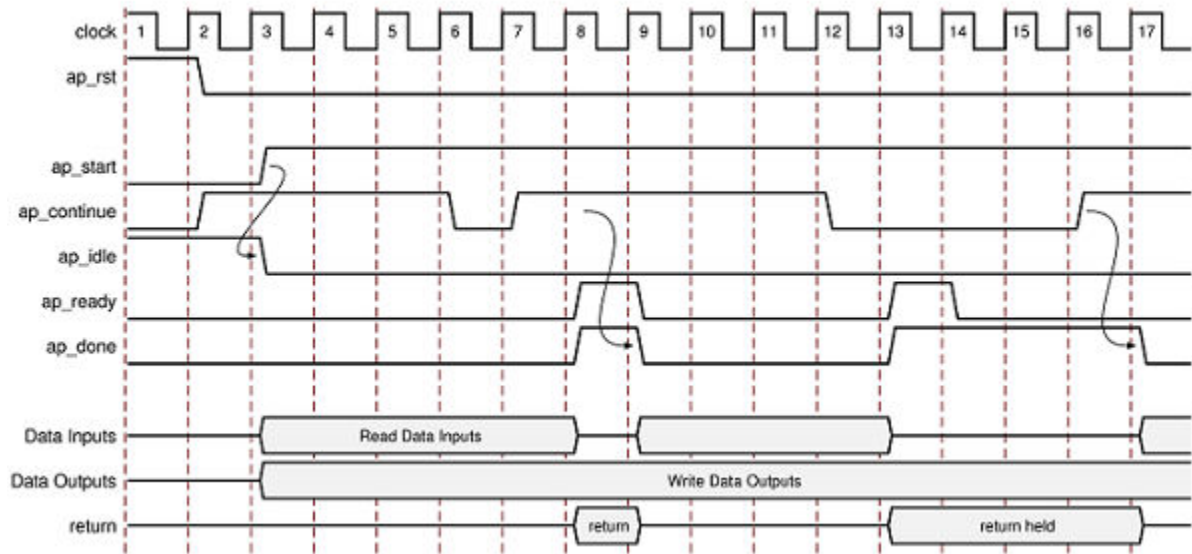


提示：如果函数返回指定为 AXI4-Lite 接口 (`s_axilite`)，那么控制协议中的所有端口都会捆绑到 `s_axilite` 接口。使用应用或软件驱动程序来配置和控制块的启动和停止操作时，这是软件可控内核或 IP 的常用实践。这是 XRT 和 Vitis 内核流程的要求。

ap_ctrl_chain

下图显示了由 ap_ctrl_chain 控制协议为顺序执行所创建的块级握手信号的行为。在下图中，HLS 设计的第 1 项传输事务完成后，第 2 项传输事务立即启动，因为 ap_continue 为高电平状态，且 ap_done 为高电平状态。但设计在第 2 项传输事务结束后将停止，直至 ap_continue 断言为高电平有效为止。

图 51: ap_ctrl_chain 接口的行为



时序图会在复位后显示以下行为：

1. 此块会等待 ap_start 达到高电平，然后再开始操作。
2. ap_idle 输出会立即变为低电平，以指示设计不再处于空闲状态。
3. ap_start 信号必须保持处于高电平状态，直至 ap_ready 达到高电平状态。当 ap_ready 达到高电平状态后：
 - 如果 ap_start 保持高电平，设计将启动下一项传输事务。
 - 如果 ap_start 变为低电平，设计将完成当前传输事务，然后停止操作。
4. 可读取输入端口上的数据。
5. 可将数据写入输出端口。

注释：输入和输出端口还可指定独立于控制协议的端口级 I/O 协议。欲知详情，请参阅 [Vivado IP 流程的端口级协议](#)。

6. 当块完成操作后，ap_done 输出会变为高电平状态。

注释：如果存在 ap_return 端口，那么当 ap_done 处于高电平状态时，此端口上的数据将变为有效。因此，ap_done 信号还用于指示 ap_return 输出上的数据有效。

7. ap_ctrl_chain 控制协议会提供处于高电平有效状态的 ap_continue 信号，表示使用输出数据的下游块已准备好处理新的数据输入。这样即可允许下游块提供反压，以阻止数据流动。
 - 如果 ap_continue 信号为高电平状态，且 ap_done 为高电平状态，那么设计将继续操作。
 - 如果下游块无法使用新数据输入，那么 ap_continue 信号处于低电平状态。如果 ap_continue 信号为低电平状态，而 ap_done 为高电平状态，那么设计会停止操作，ap_done 信号会保持高电平状态，并等待 ap_continue 变为高电平状态。

8. 当设计准备好接受新输入后，`ap_ready` 信号会脉冲至高电平状态并保持 1 个时钟周期。下游块的 `ap_ready` 端口可直接驱动 `ap_continue` 端口。以下是有关 `ap_ready` 信号的其他信息：
- `ap_ready` 信号处于不活动状态，直至设计开始操作为止。
 - 在非流水打拍设计中，`ap_ready` 信号与 `ap_done` 同时断言有效。
 - 在流水打拍设计中，当 `ap_start` 采样结果为高电平后，`ap_ready` 信号可能在任意周期变为高电平状态。这取决于设计的流水打拍方式。
 - 如果在 `ap_ready` 转至高电平后 `ap_start` 保持处于高电平状态，那么下一项传输事务将立即启动。
 - 如果 `ap_ready` 转至高电平后 `ap_start` 立即转至低电平，那么设计会持续执行操作，直至 `ap_done` 变为高电平状态后停止操作，除非在此期间 `ap_start` 再次转至高电平，而这样则会启动新的传输事务。
9. `ap_idle` 信号可用于指示设计何时处于空闲且不执行操作状态。以下是有关 `ap_idle` 信号的其他信息：
- 如果 `ap_start` 信号为低电平状态，而 `ap_ready` 为高电平状态，那么设计将停止操作，而 `ap_idle` 信号将在达成 `ap_done` 后，再经过 1 个周期后变为高电平状态。
 - 如果 `ap_start` 信号为高电平状态，且 `ap_ready` 为高电平状态，那么设计将继续操作，且 `ap_idle` 信号保持处于低电平状态。

ap_ctrl_hs

`ap_ctrl_hs` 控制协议与 `ap_ctrl_chain` 具有相同信号，但它将 `ap_continue` 信号设为 1 以保持高电平。此控制协议支持顺序执行模式和流水打拍执行模式，但不提供来自下游设计模块的反压以控制数据流动。

ap_ctrl_none

`ap_ctrl_none` 同样与 `ap_ctrl_chain` 具有相同信号，但握手信号端口（`ap_start`、`ap_idle`、`ap_ready` 和 `ap_done`）设为高电平并且会被优化掉。

`ap_ctrl_none` 区域或者一个或多个 `hls::tasks` 可通过以下任一方法来加以例化：

- 上至顶层都仅含 `ap_ctrl_none` 区域，即，上层不含任何顺序 FSM 或非 `ap_ctrl_none` 数据流，哪怕有编码错误也是如此）；
- 或者在数据流区域内满足下列条件：
 - 其输入串流是由先前调用的一个或多个非 `ap_ctrl_none` 进程或区域生成的
 - 其输出串流供后续调用的一个或多个非 `ap_ctrl_none` 进程或区域使用

前者支持消除 `ap_start/ap_ready/ap_done/ap_continue` 握手，上至顶层都是如此。后者支持为高于先前进程或区域的数据流区域生成 `ap_start/ap_ready` 握手，并支持后续进程或区域生成 `ap_done/ap_continue` 握手。



重要提示！ 如果您在设计中使用 `ap_ctrl_none` 控制协议，则必须满足 [接口综合的联合仿真要求](#) 中所述的至少一个 C/RTL 协同仿真条件，以便验证 RTL 设计。如果这些条件都未能满足，C/RTL 协同仿真将中止并提示下列消息：

```
@E [SIM-345] Cosim only supports the following 'ap_ctrl_none' designs:
(1)
combinational designs; (2) pipelined design with task interval of 1;
(3) designs with
array streaming or hls_stream ports.
@E [SIM-4] *** C/RTL co-simulation finished: FAIL ***
```

连续运行模式

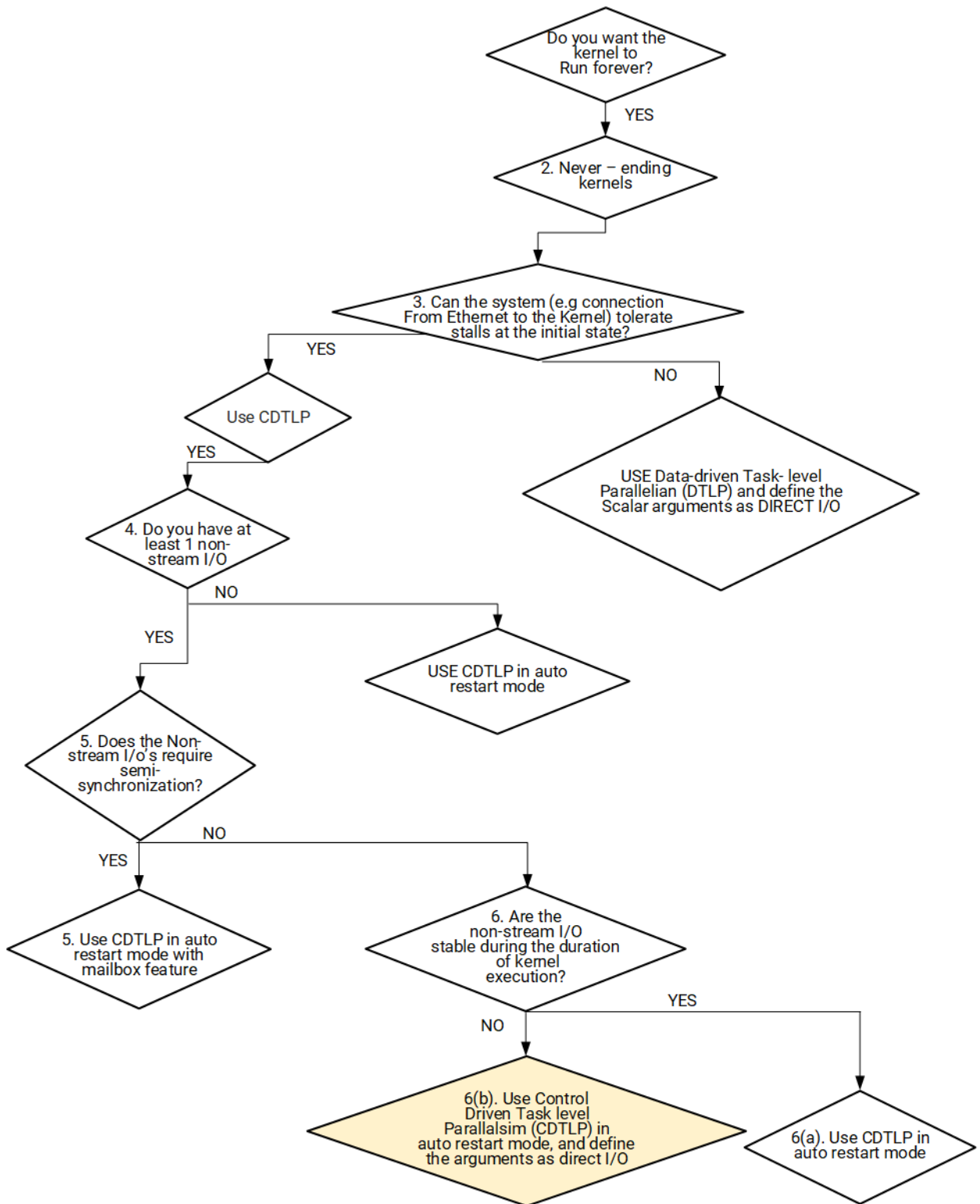
连续运行的内核是指永久运行直至复位为止的内核。永不停止的内核概念是典型的硬件设计关联环境。这些内核与 VHDL 或 Verilog 中作为进程来建模的硬件 FSM 密切相关。它们与传统软件加速内核截然不同，后者旨在“受调用”，完成后以类似软件中的函数调用的方式“返回”。永不停止的内核行为与软件加速内核的行为不同。请看以下示例：传统软件加速内核旨在“受调用”并在完成时“返回”。但永不停止的内核将重复执行直至复位，并非由主机显式启动，与围绕函数的 while (1) 循环类似。

其用例可见于视频、网络 and 金融科技领域，在此类领域中，内核会连续接收数据，不会收到来自以太网端口或 DAC 等的中断。此用例之所以需要连续运行的内核，是因为主机无法提前知晓内核应处理的包的数量（其中包的数量计算方式为从第一个 TVALID 包开始到最后一个 TLAST 包为止）。以连续运行模式对内核建模时，此问题即可解决，此模式允许内核永续运行，而无需提前知晓包的数量。

其他示例包括但不限于：

- 基于简单规则的“防火墙”，其中“规则”由主机代码在复位时编写，并且其中主机代码可以随时读取一组被丢弃的包的计数器，但所有计数器值都必须来自单次内核执行。
- 网络路由器，其中路由表必须完全更新后才能执行内核。
- 使用散列映射向服务器发送数据的负载均衡器必须同时更新服务器列表、服务器映射以及对应的 IP 地址。
- 金融科技示例，其中主机对加速器卡上运行的内核的执行状态进行松散监控，这些加速器卡会处理通过以太网端口传入的请求。

图 52：内核流程图



X50004-022725

1. 系统是否能够承受在初始状态发生停滞

连续运行的内核有两种建模方式，但选择选项 1 还是选项 2 取决于系统设计师。

- **选项 1：**数据驱动的任务级并行 (DTLP)。
- **选项 2：**自动重启模式下的控制和数据驱动的任务级并行 (CDTLP)。

选项 1：在数据驱动的任务级并行下，一旦将 `xclbin` 加载到器件上，就会立即启动内核，内核启动将不存在任何开销。该选项最适合**无法承受或者倾向于不承受以太网/DAC 输出侧发生任何停滞**的设计。

选项 2：在控制和数据驱动的任务级并行 (CDTLP) 下，主机必须将内核置于自动重启模式下。相较于选项 1，该选项存在一些启动开销，但内核设计师只需为整轮应用运行进行一次此操作即可。同样，内核设计师负责在应用完成时停止内核。

总之，使用选项 1 还是选项 2 的决策取决于**内核启动方式及其优缺点**。一旦决定建模方式，内核设计师就必须决定是否包含至少一个非串流 I/O。

2. 什么是非串流 I/O

Vitis HLS 支持存储器范例、串流范例和寄存器范例，其中每个范例都遵循某个接口协议来与外部世界进行交互。如需了解各种不同范例的描述，请参阅 [适用于 Vitis 内核流程的接口](#)。

- **非串流 I/O：**
 - 非串流 I/O 属于以下接口范例：
 - 存储器范例
 - 寄存器范例
- **串流 I/O：**
 - 串流 I/O 属于以下接口范例：
 - 串流范例

3. 非串流 I/O 需要半同步

半同步以异步非阻塞方式与自动重启内核进行 I/O 数据往来交换。在软件加速内核中，I/O 完全同步。这意味着内核会等待所有输入都可用后再启动内核。在半同步 I/O 中，从第二个函数启动开始，内核将使用旧值，并且对于更新后的新值，不会阻塞内核执行。半同步 I/O 可在连续串流的视频应用中使用。连续串流的设计可能需要以异步非阻塞方式来更改要求。例如，在应用运行过程中，视频应用可能希望将输出图像的分辨率从 HD 高清提高到 4K。通过使用半同步功能，应用可以更改输入参数，而不会阻塞内核执行。在 HLS 中，该功能是作为“邮箱”功能来实现的。请参阅 [邮箱语义](#)。

一旦内核设计确定了半同步，内核设计师就需要注意 I/O 端口的行为。

4. I/O 端口的行为：稳定/直接 I/O

根据输入的行为，在两种情况下，输入（非串流）实参会发生与内核执行相关的更改：

- 情况 (a)：在整个内核执行过程中，I/O 从不更改。
- 情况 (b)：可以随时更改，与内核是处于执行中还是空闲状态无关。

情况 (a)：在整个内核执行过程中，I/O 从不更改。

- 默认情况下，编译器会将仅当内核空闲时才会更改的输入作为稳定输入来处理。这意味着编译器假定这些输入在内核活跃执行期间将保持不变。在以下示例中，在内核空闲状态期间，会对实参“a”进行采样，在内核执行的整个过程中，该实参都会保持相同的值不变。

```
void func(hls::stream<int> in, int a, hls::stream<int> out)
{
    #pragma HLS INTERFACE mode=ap_ctrl_chain
    do {
        out.write(in.read()+a);
    }
}
```

情况 (b)：可以随时更改，与内核是处于执行中还是空闲状态无关。

- 如果内核实参是使用 [hls::direct I/O](#) 类来建模的，那么内核执行中任意时间点，这些内核都能更改该值。内核设计师对更改的时机并无特定要求，只要内核能在未来某个时间点看到更新后的值即可。在以下示例中，只要内核执行期间存在有效的输入，就会对实参“reset_myCounter”进行采样。

```
void krnl_stream_vdatamover(hls::stream<pkt> &in,
                           hls::stream<pkt> &out,
                           int mem[DATA],
                           hls::ap_vld<int> &reset_value,
                           hls::ap_vld<int> &reset_myCounter
                           )

{
    for(int i=0; i<DATA; i++)
    {
        ...
        if(reset_myCounter.valid())

        {
            int reset = reset_myCounter.read();
        }
        ...
    }
}
```

自动重启模式

Vitis HLS 设计可以是控制驱动或数据驱动，如 [HLS 抽象并行编程模型](#) 中所述。这意味着，HLS 设计的执行是通过控制信号来管理的，并由外部软件应用或软件驱动程序来管理，或者由输入是否存在数据来驱动。

数据驱动的设计仅根据有更多数据需要处理这一事实来自动重启设计。HLS 设计完成一项传输事务的执行，然后有数据可用时即开始执行下一项传输事务。如不存在数据，则执行停滞，存在数据时则重新启动。本节并不涉及这些数据驱动的设计。

但对于控制驱动的设计，负责管理的软件应用或者已连接的 HLS 设计必须触发模块的 `ap_start` 信号才能开始处理，如 [块级控制协议](#) 中所述。虽然默认情况下，这些设计不会自动重启，但可按此处所述方式使用自动重启机制来实现。当 HLS 设计使用串流 I/O 进行数据输入或输出时，此自动重启模式对于控制驱动的 TLP 很有用，但其中也有控制信号，使其可供软件应用进行管理。在此情况下，软件应用会启动一次内核，而后内核就会按指定的次数运行传输事务，或者持续运行直至复位或由软件应用显式停止为止。

这种控制驱动的自动重启设计对于如下设计很有用：设计内具有串流数据并且这些数据进出 AMD 器件的 I/O 管脚（以太网、SerDes），或者设计内具有串流数据并且这些数据往来于已连接的 HLS 设计模块之间。换言之，HLS 设计以数据驱动的设计为主，但并非全部如此。

处理自动重启设计

在软件应用启动一次后，自动重启设计即可持续运行。此类设计可以持续运行直至复位和重启，也可以编程为按预定义的次数运行迭代，而无需软件应用以称为“带计数的自动重启”的模式来多次显式调用这些设计。此功能类似于软件代码中的 `while(running)` 循环，其中 `running` 变量由软件应用控制。

设计的控制在硬件中进行管理，因此在软件应用启动此块后，此块将自动重启，直至超过迭代计数或者直至主机代码将其显式停止为止。此外，应用可查询设计的状态，以检查特定寄存器状态或者提供新参数以供后续使用。

自动重启设计使用了以下几个独特功能：

1. 控制寄存器中的自动重启位用于持续重新启动设计或者按指定迭代次数重新启动设计，而无需在每次执行时进行显式软件调用。
2. Xilinx Runtime (XRT) 的邮箱功能可支持软件应用与设计偶尔同步，以设置新的操作参数或者检查当前运行的状态，如 [使用邮箱](#) 中所述。
3. `software_reset` 功能允许软件应用复位设计以停止执行。

支持的接口

自动重启 HLS 设计需要 `ap_ctrl_chain` 协议或 `ap_ctrl_hs` 协议。

- 设计应在 INTERFACE 编译指示或指令上指定 `mode=ap_ctrl_hs`。
- 自动重启设计支持串流接口 (`axis`) 以及标量实参 (`s_axilite`) 和存储器映射 (`m_axi`) 实参（可读取和写入）。

可在多个不同的场景中使用自动重启设计：

1. 仅限包含串流接口 (`axis`) 的自动重启内核，此内核完全不需要与主机应用交互。例如，要将配置数据编译到内核的快速傅里叶变换 (FFT)，或者要将系数编译到内核的 FIR 滤波器。
2. 使用标量和存储器映射 (`m_axi`) 实参的自动重启内核，包含或不包含 `axis` 接口。标量和存储器映射实参需要邮箱才能在需要时更新内核参数。例如，基于规则的简单防火墙（其规则由主机应用在复位时编写），以及丢包计数器（可由主机代码读取，所有值都必须来自单次内核执行）。又例如，使用散列映射向服务器发送数据的负载均衡器（必须同时更新服务器列表、服务器映射以及对应的 IP 地址）。

启用自动重启

要启用 HLS 设计的连续重启，请在顶层接口编译指示中指定：

```
#pragma HLS interface s_axilite port=return autorestart
```

您可以使用以下选项启用软件复位功能以便自动重启：

```
syn.interface.s_axilite_sw_reset=1
```



提示：复位内核会停止端口上的所有传入流量，并刷新掉所有未完成的 MAXI 传输事务。如需了解有关使用复位的更多信息，请参阅 [将自动重启与邮箱搭配使用](#)。

使用邮箱

自动重启 HLS 设计的主要优势在于可以像数据驱动的设计一样半自动运行，而无需与软件应用频繁交互，也无需软件控制。但是，自动重启内核还通过称为邮箱的功能来提供半同步，邮箱能够以异步、非阻塞的安全方式与软件应用交换数据。

HLS 设计经由软件应用启动后，会自动重启直至被显式停止为止。软件应用也可以查询设计的状态，以确定在得到执行指示后实际完成执行的时间。应用和自动重启设计都使用以下通信协议：

- 为了将实参值从软件应用传递到设计，邮箱会实现一组双重缓冲的 `s_axilite` 映射寄存器，以确保软件应用的输入与设计输出之间能够以非阻塞方式进行通信，并且能够以一致方式来传递软件应用的输入和设计输出。
- 每当软件应用写入输入实参时，都会更改软件侧副本。硬件中运行的 HLS 设计看不到此更改。软件应用请求邮箱写入后，下次自动重启设计时，将以最新输入来更新设计所看到的寄存器副本。因此，软件应用可按任意顺序写入任意数量的实参，在软件应用请求邮箱写入后，HLS 设计才会看到这些更新，并且设计会重新启动。



提示：如果某些实参是映射到 `s_axilite` 寄存器文件的阵列，则必须在连续邮箱写入之间写入整个阵列，因为它是作为乒乓缓冲器来实现的。

- 当 HLS 设计写入到设计侧的寄存器内并请求邮箱读取时，在输出侧会发生相同的过程。下次执行设计时，将更新 `s_axilite` 映射输出实参的值，并且软件应用可按需读取这些值。

因此，软件应用具有以下条件：

- 不负责在每次执行 HLS 设计时提供输入数据且不负责在完成时收集输出数据，就像控制驱动的设计一样。
- 不定时参与设置和更新某些设计输入参数（例如，布线表等）并检查设计执行的状态。通常在软件应用与 HLS 设计之间无固定通信速率的情况下完成此半同步操作。
- 当软件应用要将一组新参数发送到 HLS 设计时，它会忽略设计的执行状态。当软件应用需要检查 HLS 设计的状态时，它会忽略设计的执行状态。它只要求为硬件持续执行参数更新和状态检查即可。

邮箱语义

邮箱功能可用于输入和输出，并且需要为 HLS 组件的所有 `s_axilite` I/O 指定此功能。除接口编译指示的 `autorestart` 选项外，您还可使用以下配置命令搭配全局选项来为各接口启用该功能：

```
syn.interface.s_axilite_mailbox=both
```

设置配置选项后，邮箱会实现一对名为 `HW copy` 和 `SW copy` 的寄存器。输入邮箱和输出邮箱具有一对独立的寄存器。软件应用与 HLS 组件的通信包括：

- 输入邮箱：
 - 应用将部分或全部元素写入邮箱的 `SW copy` 寄存器。
 - 应用通知邮箱，`SW copy` 已更新。
 - 当 HLS 设计重新启动时，`SW copy` 寄存器将被复制到 `HW copy` 寄存器。
 - 应用会收到关于 `HW copy` 已更新的通知，并且可根据需要再次更改 `SW copy` 寄存器。



提示：HLS 设计可在无软件应用更新的情况下进行多次读取。

- 输出邮箱：
 - 应用通知邮箱，它想要读取更新后的邮箱副本。在当前执行结束时，HLS 设计将部分或全部元素写入邮箱的 `HW copy` 寄存器。
 - 在设计完成后，会将 `HW copy` 复制到 `SW copy` 寄存器。
 - 应用会收到关于 `SW copy` 已更新的通知，并且可随时读取该寄存器。



提示：硬件可在无任何软件更新请求的情况下进行多次写入。

在协同仿真（C/RTL 协同仿真）中使用自动重启

“Auto-Restart”（自动重启）支持内核或 IP 在每次完成后立即重新启动自己，迭代次数有限，无需软件干预，也没有间隔。在 AXI4-Lite 的传统协同仿真流程中，如果不使用自动重启功能，协同仿真框架会在连续的 DUT 启动之间创建 4 个周期的间隔，以使用 AXI4-Lite 传输事务执行 `ap_ctrl_hs` 握手，这会导致顶层启动时间间隔较低 ($II \leq 5$) 的 DUT 无法实现无缝无间隔执行。

Auto-Restart 帮助程序类消除了协同仿真 (cosim) 期间的启动间隔，确保内核能连续无间隔执行指定次数的迭代。

编程模型

1. 包含头文件：添加 `#include "hls_task.h"` 以访问 Auto-Restart 帮助程序类
2. 设置测试平台：利用 DUT 数据初始化变量和缓冲器
3. 利用 Auto-Restart 进行调用：使用 `hls::autorestart name_of_class_instance(num_iters, dut_function, arg1, arg2 等)` 调用 DUT；其中 `num_iters` 是迭代次数

调用 AXI4-Lite (SAXI4-Lite) 接口时，它会将迭代计数 (`num_iters`) 写入内核的“Auto-Restart”（自动重启）计数器寄存器，该寄存器地址为 `0x10`。随后，该内核会无间隔连续执行指定次数的迭代。

示例

```
#include "hls_task.h"

void top(...) {
    #pragma HLS INTERFACE s_axilite autorestart port=return
    ...
}

void main()
{
    for(i.. )
    {
        input[i] = i;
    }
    // Run 3 iterations back-to-back with test
    hls::autorestart run_top(top, input, output);
}

for(i.. )
{
    input[i] = i*i;
}
// Run 10 iterations back-to-back
run_top(10, top, input, output);
```

注释：顶层函数必须在 `s_axilite adapter` 内具有 `autorestart` 计数器，该计数器是通过 `#pragma HLS interface s_axilite port=return autorestart` 启用的

自动重启协同仿真限制

自动重启存在限制。

1. 无法无限自动重启：必须指定有限次数的重启。
2. 不支持直接 I/O 变量。
3. 在测试平台中只能创建一个 `hls::autorestart` 实例。连续调用应使用该实例的 `operator()`，如上所示。

4. 完成指定次数的迭代之前，不得更改任何输入。
5. 完成 `num_iter` 次迭代之前，测试平台无法观察到任何输出。
6. 控制协议：仅支持 `ap_ctrl_hs`；不支持 `ap_ctrl_chain` 和 `ap_ctrl_none`。
7. 接口：不支持阵列到串流 FIFO。
8. 不支持邮箱。

在 XRT 中使用自动重启

以下章节描述了自动重启 HLS 设计示例。

将自动重启与邮箱搭配使用

邮箱功能支持与软件应用进行半同步。邮箱是一种用于更新 HLS 设计参数的非阻塞机制。下次启动设计时，将提取通过邮箱提供的任何更新。

该示例设计使用的标量值将从软件应用进行编程，并且设计将在下一次软件调用时提取这些标量值。标量 `adder1` 和 `adder2` 将从软件应用进行异步更新。

除接口编译指示的 `autorestart` 选项外，您还可使用以下配置命令将 HLS 编译器配置为自动重新启动模式并启用邮箱功能：

```
syn.interface.s_axilite_mailbox=both
```

这会告知本工具使用 `autorestart` 机制创建 IP 或内核。HLS 设计代码示例如下所示：

```
#define DWIDTH 32
11
12 typedef ap_axiu<DWIDTH, 0, 0, 0> pkt;
13
14 extern "C" {
15 void krnl_stream_vdatamover(hls::stream<pkt> &in,
16                             hls::stream<pkt> &out,
17                             int adder1,
18                             int adder2
19                             ) {
20
21 #pragma HLS interface ap_ctrl_hs autorestart port=return
22 #pragma HLS INTERFACE s_axilite port=adder2
25 #pragma HLS port=adder1 stable
    #pragma HLS port=adder2 stable
27 bool eos = false;
28 vdatamover:
29 do {
30     // Reading a and b streaming into packets
31     pkt t1 = in.read();
32
33     // Packet for output
34     pkt t_out;
35
36     // Reading data from input packet
37     ap_uint<DWIDTH> in1 = t1.data;
38
39     // Vadd operation
40     ap_uint<DWIDTH> tmpOut = in1+adder1+adder2;
41
42     // Setting data and configuration to output packet
43     t_out.data = tmpOut;
```

```

44     t_out.last = t1.last;
45     t_out.keep = -1; // Enabling all bytes
46
47     // Writing packet to output stream
48     out.write(t_out);
49
50     if (t1.last) {
51         eos = true;
52     }
53 } while (eos == false);
54

```

创建邮箱以更新标量值 `adder1` 和 `adder2`。

从软件应用使用 `set_arg` 和 `write` 方法更新设计参数，如下所示。自动重启 HLS 设计将不会自行停止，因为串流接口没有启动和停止机制。此模块必须显式停止或复位。应用代码可以使用 `abort()` 方法显式停止设计运行。

```

// add(in1, in2, nullptr, data_size)
xrt::kernel add(device, uuid, "krnl_stream_vadd");
xrt::bo in1(device, data_size_bytes, add.group_id(0));
auto in1_data = in1.map<int*>();
xrt::bo in2(device, data_size_bytes, add.group_id(1));
auto in2_data = in2.map<int*>();

// mult(in3, nullptr, out, data_size)
xrt::kernel mult(device, uuid, "krnl_stream_vmult");
xrt::bo in3(device, data_size_bytes, mult.group_id(0));
auto in3_data = in3.map<int*>();
xrt::bo out(device, data_size_bytes, mult.group_id(2));
auto out_data = out.map<int*>();

xrt::kernel incr(device, uuid, "krnl_stream_vdatamover");
int adder1 = 20; // arbitrarily chosen to be different from 0
int adder2 = 10; // arbitrarily chosen to be different from 0

// create run objects for re-use in loop
xrt::run add_run(add);
xrt::run mult_run(mult);
std::cout <<"performing never-ending mode with infinite auto
restart"<<std::endl;
auto incr_run = incr(xrt::autostart{0}, nullptr, nullptr, adder1, adder2);

// create mailbox to programatically update the incr scalar adder
xrt::mailbox incr_mbox(incr_run);

// computed expected result
std::vector<int> sw_out_data(data_size);

std::cout << " for loop started" <<std::endl;
bool error = false; // indicates error in any of the iterations
for (unsigned int cnt = 0; cnt < iter; ++cnt) {

    // Create the test data and software result
    for(size_t i = 0; i < data_size; ++i) {
        in1_data[i] = static_cast<int>(i);
        in2_data[i] = 2 * static_cast<int>(i);
        in3_data[i] = static_cast<int>(i);
        out_data[i] = 0;
        sw_out_data[i] = (in1_data[i] + in2_data[i] + adder1 + adder2) *
in3_data[i];
    }
}

```

```

    }

    // sync test data to kernel
    in1.sync(XCL_BO_SYNC_BO_TO_DEVICE);
    in2.sync(XCL_BO_SYNC_BO_TO_DEVICE);
    in3.sync(XCL_BO_SYNC_BO_TO_DEVICE);

    // start the pipeline
    add_run(in1, in2, nullptr, data_size);
    mult_run(in3, nullptr, out, data_size);

    // wait for the pipeline to finish
    add_run.wait();
    mult_run.wait();

    // prepare for next iteration, update the mailbox with the next
    // value of 'adder'.
    incr_mbox.set_arg(2, ++adder1); // update the mailbox
    incr_mbox.set_arg(3, --adder2); // update the mailbox

    // write the mailbox content to hw, the write will not be picked
    // up until the next iteration of the pipeline (incr).
    incr_mbox.write(); // requests sync of mailbox to hw

    // sync result from device to host
    out.sync(XCL_BO_SYNC_BO_FROM_DEVICE);

    // compare with expected scalar adders
    for (size_t i = 0 ; i < data_size; i++) {
        if (out_data[i] != sw_out_data[i]) {
            std::cout << "error in iteration = " << cnt
                << " expected output = " << sw_out_data[i]
                << " observed output = " << out_data[i]
                << " adder1 = " << adder1 - 1
                << " adder2 = " << adder2 + 1 << '\n';
            throw std::runtime_error("result mismatch");
        }
    }
    incr_run.abort();
}

```

使用带计数的自动重启

此示例使用的代码与前述示例相同，但在此例中，HLS 设计配置为运行并重启 3 次迭代，然后停止。唯一要更改的是从软件应用指定迭代次数，如以下代码示例中所示。



提示：在“带计数的自动重启”用例中，不需要使用 `abort()`，因为软件应用知道何时停止。

```

143 xrt::kernel incr(device, uuid, "increment");
144 int adder1 = 20; // arbitrarily chosen to be different from 0
145 int adder2 = 10; // arbitrarily chosen to be different from 0
151 // start the incr kernel in auto restart mode with default adders
152 // since it is a streaming kernel it will be stalled waiting for
153 // input
154 auto incr_run = incr(xrt::autostart{3}, nullptr, nullptr, adder1);

```

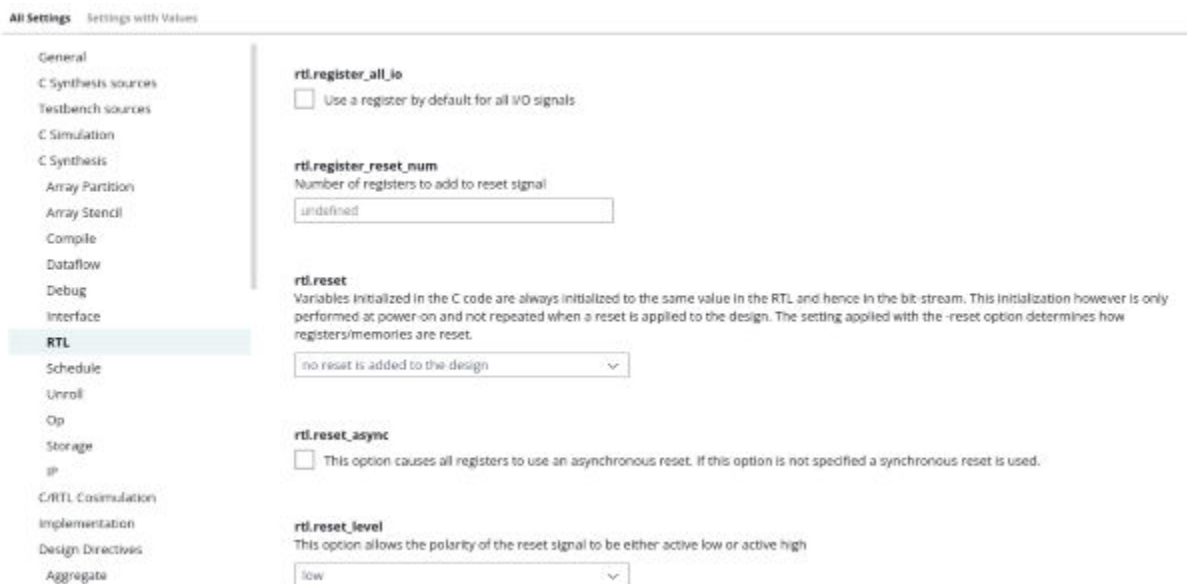
控制初始化与复位行为

复位端口在器件中用于在应用复位信号时，立即将连接到复位端口的寄存器和块 RAM 还原为初始值。通常 RTL 配置中最重要的操作即选择复位行为。

注释：对于复位行为，重要的是理解初始化与复位之间的差异。如需了解更多信息，请参阅 [初始化行为](#)。

配置命令可用于控制 RTL 复位端口是否存在及其行为，如 [RTL 配置](#) 中所述，如下图所示。

图 53：RTL 配置



复位设置包含设置复位极性以及使用同步复位还是异步复位的功能，但更重要的是，它可通过“reset”选项来控制应用复位信号时要复位的寄存器。



重要提示！当在设计上使用 AXI4 接口时，复位极性会自动更改为低电平有效，而与 `config_rtl` 配置中的设置无关。这符合 AXI4 标准的要求。

“reset”选项包含 4 项设置：

- “none”：在设计中不添加复位。
- “control”：这是默认设置，用于确保将所有控制寄存器复位。控制寄存器即状态机中使用的寄存器，用于生成 I/O 协议信号。此设置可确保设计可立即启动其操作状态。
- “state”：该选项可为控制寄存器添加复位（与“control”设置相同），并且还可为衍生自 C/C++ 语言代码中的静态变量和全局变量的任意寄存器或存储器添加复位。此设置可确保应用复位后，C/C++ 语言代码中初始化的静态变量和全局变量均复位为其初始值。
- “all”：此设置用于为设计中的所有寄存器和存储器添加复位。

通过 RESET 编译指示或指令可提供更精细的复位控制，如 [syn.directive.reset](#) 中所述。静态变量和全局变量均可通过 RESET 指令来添加复位。还可使用 RESET 指令的 `off` 选项从复位的变量中移除变量。



重要提示！ 使用 `state` 或 `all` 复位选项时，考量其对于阵列复位的影响是至关重要的。

初始化行为

在 C/C++ 中，默认情况下，以静态限定符定义的变量和全局作用域中定义的变量都初始化为 0。（可选）对于这些变量，可赋予特定初始值。对于这些已初始化的变量，C/C++ 语言代码中的值在编译时（时序为 0 时）进行赋值，并且不再进行赋值。上述 2 种情况下，在 RTL 中实现初始值。

- 在 RTL 仿真期间，为这些变量设置的初始值与 C/C++ 语言代码中相同。
- 在用于对 FPGA 进行烧录的比特流中，也会对这些变量进行初始化。当器件上电时，变量将以其初始状态启动。

在 RTL 中，虽然变量启动时使用的初始值与 C/C++ 语言代码相同，但无法强制该变量返回至此初始状态。要复原初始状态，必须通过复位信号来实现这些信号变量。



重要提示！ 顶层函数实参可在 AXI4-Lite 接口内实现。由于在 C/C++ 中无法为函数实参提供初始值，因此在 RTL 中无法对这些变量进行初始化，否则由此创建的 RTL 设计将与 C/C++ 代码产生不同的功能行为，从而导致在 C/RTL 协同仿真期间验证失败。

利用 M_AXI 接口进行设计的最佳实践

对于负责实现 AMD Vitis™ 内核的设计师而言，处理 FPGA 器件上可用的器件存储器（PLRAM、HBM 和 DDR）时，有多方面因素需要折衷考量。以下是为应用设计 AXI4 存储器映射接口时使用的最佳实践检查表。

如果首要的优化目标是吞吐量，那么显而易见，第一步是使用宏架构和微架构优化来加速应用的计算部分，但往来内核进行数据传输所耗用的时间也会影响与吞吐量目标相关的应用架构。由于数据传输的高开销，考虑如何将计算与应用中存在的通信（数据移动）加以重叠就显得尤为重要。

对于给定应用而言：

- 通过如下方式来分解内核算法：构建生产者 - 使用者任务并以加载、计算和存储 (LCS) 编码模式来对任务进行建模
 - 所有外部 I/O 访问都必须包含在加载和存储任务中。
 - 如果内核需要并行读取或写入不同端口，则应存在多个加载或存储任务。
 - 计算任务应仅包含标量、阵列、串流或块串流实参。
 - 请确保这全部三项任务（指定为函数）都能以重叠方式来执行（由编译器启用任务级并行操作）。
 - 计算任务能够进一步拆分为更小的计算任务，这样即可进一步执行优化操作（例如，流水打拍）。适用于 LCS 的这些规则同样适用于这些较小的计算函数。
 - 请始终使用本地存储器来通过计算任务进行数据的往来传递。
- 加载和存储块负责在全局存储器与计算块之间以尽可能高效的方式移动数据。
 - 一方面，这些块必须根据内核内部的计算任务强制要求的（时间）顺序通过串流接口来读取或写入数据
 - 另一方面，这些块必须根据软件应用所设置的（空间）排列顺序通过存储器映射接口来读取或写入数据
- 利用 HLS 构建正确的硬件设计的关键是改变您对于数据访问的思维模式
 - 在软件中，用户经常会思考数据“访问”方式（算法提取所需的数据）。
 - 在硬件中，思考数据“流经”算法的方式则更为有效（数据推送到算法）
 - 在软件中，用户需要推理阵列索引以及“在何处”访问数据
 - 在硬件中，用户需要推理串流及“何时”访问数据
- 全局存储器的访问时间更长（DRAM 和 HBM），并且带宽有限 (DRAM)。为了减少访问全局存储器的开销，接口函数需要
 - 高效访问大型连续数据块（以便通过 [AXI 突发传输](#) 来获益）
 - 相比随机访问和/或无需数据（突发分析在此情况下将失败），顺序访问数据的突发更高（数据吞吐量效率更高）
 - 避免冗余访问（节省带宽）

- 数据顺序出入计算任务大多数情况下与全局存储器中的数据排列顺序不同。
 - 在此情况下，接口函数优化需要创建内部高速缓存结构以便收集充足的数据，并对其加以相应组织，从而尽可能减少全局存储器访问开销，同时仍能满足串流接口所期望的顺序。
 - 为了简化数据移动逻辑，开发者还可思考如何通过不同方法将数据存储到存储器中。例如，通过以列为主的方式来访问 DRAM 中的数据效率非常低。相比在内核中实现专用数据移动器，在软件中对数据进行转置并改用以行为主的方式来存储将显著简化硬件访问模式，因此这种方式更有效。
- 请尽可能增大接口的端口宽度，将每个 AXI 端口的位宽设置为 512 位（64 字节）。
 - 使用 `hls::vector` 或 `ap_(u)int<512>` 作为端口的数据类型，以便推断最大突发长度。在接口中使用结构体可能导致突发性能欠佳。
 - 访问全局存储器开销极大，因此增大访问的数据字大小更为有效。
 - 请将接口端口想象成用于向内核馈送数据的管道。管道越宽，那么可供访问和处理的数据就越多，发回的数据也就越多。
 - 从全局器件存储器传输大型数据块。一次大规模传输要比多次小规模传输效率更高。带宽受 PCIe 性能限制。请运行 [DMA 测试](#) 以测量 PCIe® 传输的最大有效吞吐量。通常，读取和写入的最大吞吐量各自均在 10-17 GB/s 范围内。
 - 存储器资源包括 PLRAM（小尺寸，快速访问，时延最低）、HBM（中等尺寸，中等访问速度，存在一定时延）和 DRAM（大尺寸，访问速度较慢，高时延）。
 - 鉴于读取本质上是异步操作，因此快速缓冲的理想选择是分布式 RAM。您可立即使用读取的值，而无需等待至下一个时钟周期。您也可以使用分布式 RAM 来创建小型 ROM。但分布式 RAM 不适合大型存储器，使用块 RAM 和 UltraRAM 能够为大小超过 128 位的存储器提升性能（并降低功耗）。
- 确定并发端口的最优数量，例如，并发 AXI 存储器映射端口的数量
 - 如果加载任务需要获取多个输入数据集以供馈送给计算任务，那么它可选择使用多个接口端口来并行访问此数据。
 - 但此数据需要存储在[不同的存储体](#)中，或者需要顺序访问。如有 [32 条 HBM 通道](#)，那么 FPGA 上有最多 4 个 DDR 存储体。
 - 当有多个进程访问同一个存储器端口或存储体时，仲裁器将对同一个存储器端口或存储体执行的并发访问转变为顺序访问。
- 为每个 AXI 端口设置正确的突发长度，即最大突发访问长度（以元素数量来表示）。
 - 设置与最大 4k 字节传输相等的突发长度。例如，使用的 AXI 数据宽度为 512 位（64 字节）时，突发长度应设置为 64。
 - 以突发方式传输数据可隐藏存储器访问时延，并提高存储器控制器的带宽使用率和效率
 - 按此方式编写应用代码能够为往来全局存储器的读取和写入推断最大突发长度
- 设置 AXI 端口在停滞前可维持的未完成存储器请求的数量
 - 设置合理的[未完成请求](#)数使系统能在停滞前提交多个存储器请求，这种请求流水打拍方式允许系统隐藏部分存储器时延，但代价是 BRAM/URAM 资源数增加。

顶层性能编译指示

顶层性能编译指示

`top-level performance pragma`（顶层性能编译指示）提供了一套机制，用于定义分配给设计顶层的性能目标。它包含吞吐量约束以充当高层次指令，用于指导综合工具根据目标来优化设计。

利用顶层性能编译指示，编译器会对整个设计开展吞吐量分析。它会对若干编译指示进行评估，以便通过推断来判定达成吞吐量目标的可行性。根据分析结果，该工具会自动为整个设计层级中的单个循环和循环嵌套推断适当的循环层性能编译指示。

循环层性能编译指示

循环层性能编译指示适用于单个循环，能为该工具提供显式指导。这些编译指示可以由顶层性能编译指示算法进行自动推断与校准，也可以由用户进行手动应用。它们通过自动推断一个或多个传统低层编译指示（如，流水线、展开、平铺、重塑和分区），即可实现细粒度的性能变换。

“优势”

- “精确控制循环行为”：循环层编译指示使设计师能够优化特定循环的吞吐量，从而更精准地控制关键路径的性能。
- “支持自上而下的目标”：与顶层性能编译指示相结合使用时，循环层编译指示能够通过精细的调控，来帮助弥合当前性能与指定的系统级目标之间的差距。

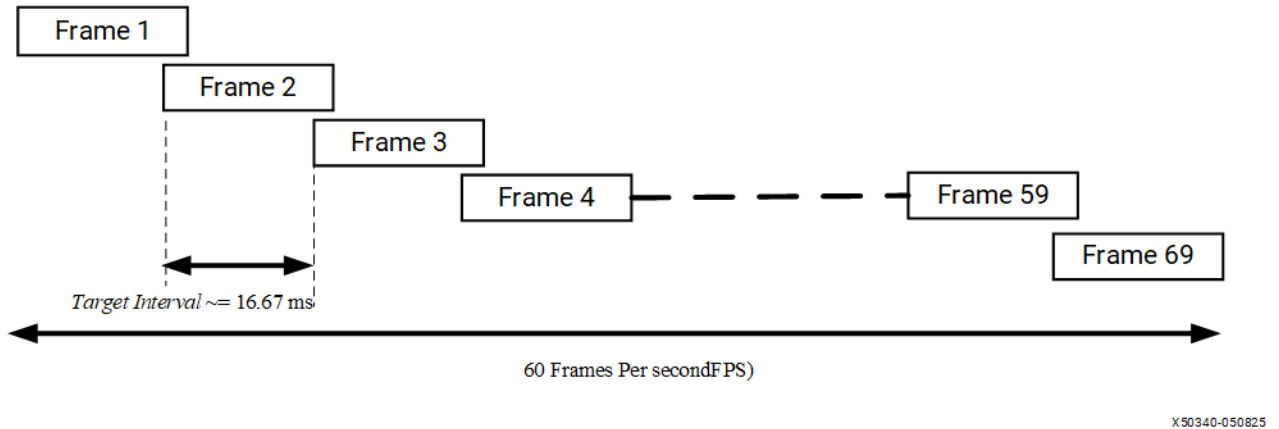
性能编译指示方法论

顶层性能编译指示方法论是一套经简化的方法，用于指导优化进程，达成苛刻的带宽目标。

第 1 步：根据吞吐量目标计算性能目标

计算视频应用的性能目标

- 假设有一个图像处理设计，其目标帧速率为每秒 60 帧 (60 fps)。
- **第 1 步：判定总帧数**
 - 每秒总帧数 = 60。
- **第 2 步：判定顶层性能目标 (target_ti)：**
 - 要达到 60 FPS 的帧速，函数 top 必须在 1/60 秒内准备好重新启动并读取新的帧。
 - $\text{target_ti} = 1/\text{FPS} = 1/60 \approx 16.67$ 毫秒
 - 这意味着函数 top 必须在 16.7 毫秒内准备好重新启动并读取新的帧，以保持 60 FPS 的目标。



第 3 步：为数据流重构代码

在应用性能编译指示和深入开展任何 Vitis HLS 优化操作之前，必须确保代码结构能适应高效的硬件实现。这包括遵循规范的数据流形式来创建任务（循环与函数）。这将为 Vitis HLS 有效优化代码并实现并行度打下坚实的基础。

第 4 步：运行 CSIM 并判定循环次数

- 该方法论的下一步是运行 C 语言仿真。在 C 语言仿真期间，启用循环次数剖析选项可自动捕获实际的循环迭代次数。此数据至关重要，因为性能编译指示算法必须为设计中的所有循环准确确定预算；精确的循环次数可以避免依赖默认的假设，并提高性能估算的准确性。在 C 语言仿真期间，启用循环次数剖析选项可自动捕获实际的循环迭代次数。该指标非常重要，因为性能编译指示算法需要精确计算设计中所有循环的预算。默认情况下，性能编译指示会将变量循环的循环边界估算为“1024”，对于边界大于 1024 的循环，这可能导致性能估算不准确。
- 请参阅“CSIM 循环次数选项”。

第 5 步：添加顶层性能目标

- 该方法论的下一步是应用性能目标。使用 `target_ti` 参数（目标时间间隔），在代码中直接指定所需性能目标，如下所示。为了满足 60 fps 的帧率，`target_ti` 设为 16.7 毫秒。

```
#pragma HLS performance target_ti = <> ms/cycles
```

第 6 步：识别循环瓶颈（如有）

- 运行 C 语言综合：**应用顶层性能编译指示后，在 Vitis HLS 中运行 C 语言综合。
- 分析报告：**仔细查阅 C 语言综合报告。识别不符合 `target_ti` 要求的循环/函数

第 7 步：添加/更新局部性能目标

- 对于被识别为瓶颈的关键循环，使用相同的 `target_ti` 参数指定循环层性能目标。这样会引导 Vitis HLS 将优化重点放在这些循环上。
- 重新运行 C 语言综合并分析更新后的报告。继续完善循环层目标，直到设计满足总体性能目标为止。
- 通过遵循这一分析与优化迭代进程来使用性能编译指示，您即可引导 Vitis HLS 在自己的硬件设计中实现最佳性能。

理解顶层性能编译指示的优化策略

性能编译指示：确定时序约束的优先顺序

性能编译指示的主要优化目标是满足指定的时序要求。它优先考虑达成时序约束，不会牺牲时序约束以尝试满足性能目标。因此，即使不能完全达成性能目标，编译指示也将确保设计满足其时序要求。如果无法满足所需性能目标，则建议使用更细化的传统编译指示（如，`unroll` 和 `pipeline`），在不违反已确立的时序约束的前提下进一步增强性能。

为顶层性能编译指示优化重构代码

- 为了有效使用顶层性能编译指示，设计需重构为“Load-compute-store”（加载 - 计算 - 存储）(LCS) 范式，并采用数据流编译指示。

动态循环次数信息

- 在 C 语言仿真期间启用循环次数剖析选项以自动捕获实际循环迭代次数。若没有循环次数信息，性能编译指示会将动态循环的循环边界估算为“1024”，对于边界大于 1024 的循环，这可能导致性能估算不准确。
- 对于变量循环，您应使用 HLS 循环次数编译指示 (`pragma HLS loop_tripcount max=N`) 提供动态循环次数信息

锁定推断得到的编译指示

将性能编译指示所推断的低层次编译指示连同任何手动指定的编译指示一起导出到用户配置文件。将这些编译指示重新注入您的源文件或指令文件，以锁定所选择的优化。这样即可防止性能预算器在后续运行轮次中重新探索编译指示组合，从而减少额外的周期并缩短迭代时间。

限制

“编译指示优先顺序”

以下准则概述了将各种编译指示应用于循环和阵列时的优先顺序：

- **PIPELINE OFF 编译指示：**
 - 将 PIPELINE OFF 编译指示应用于循环时，“循环层性能编译指示”不会为该循环自动推断 PIPELINE 编译指示
- **UNROLL OFF 编译指示：**
 - 如果将 UNROLL OFF 编译指示应用于任何循环，则会阻止“循环层性能编译指示”为该循环推断任何 UNROLL 编译指示
- **FLATTEN 编译指示：**
 - 如果在某个循环上使用 FLATTEN 编译指示，它将不允许“循环层性能编译指示”推断任何其他 FLATTEN 编译指示
- **ARRAY_PARTITION OFF 编译指示：**
 - 将 ARRAY_PARTITION OFF 应用于局部阵列时，将阻止“循环层性能编译指示”在该阵列上推断任何 ARRAY_PARTITION 编译指示

接口端口限制

- 默认情况下，ARRAY_PARTITION 编译指示不会自动推断顶层函数的接口上的阵列。这可能导致性能瓶颈。您可通过以下命令启用该功能特性来解决此问题：

```
config_array_partition -throughput_driven=aggressive
```

不支持的库

顶层性能编译指示使用代码分析器技术来对整个设计开展性能分析，从而达成此目标。与代码分析器关联的限制也适用于顶层性能编译指示。行为/限制如下

功能特性	行为/限制
ap_cint	工具退出时会发出显式警告消息
ap_(u)int / ap_(u)fixed	性能模型不准确
ap_int / ap_fixed 的大型常量阵列	将这些阵列与 NON_C99STRING 宏一起使用将导致编译错误
std::complex<ap_fixed>	导致在 Windows 上出现编译器错误
HLS IP 块 (FFT、FIR 等)、hls_math.h、ap_wait, hls::vector、ap_axis/ap_axiu	性能模型不准确
hls::stream_of_blocks、hls::task、hls::split/hls::merge、hls::print、hls::half、ap_utils.h、hls_fpo.h、ap_float、RTL 黑盒、hls::burst_maxi、hls::fence 和 hls::directio	工具退出时会发出显式警告消息
已弃用的 HLS 编译指示	生成警告（忽略已弃用的编译指示）
带子循环的函数流水线	断言失败

已知问题

编译时间长

- 顶层性能编译指示会对整个设计执行全面的性能分析。通过类比可以看出，这一过程就像在众多可能路径中，寻找从 A 点到 B 点的最佳路线。通过在整个设计中探索各种优化策略，该编译指示旨在识别最有效的优化，以满足性能目标。然而，这段广泛的探索阶段虽然对实现最优结果至关重要，但有时也会导致编译时间过长。如果您遇到问题，请联系 AMD 技术支持。

激进推断得到的性能目标

- 顶层性能编译指示旨在通过对整个设计开展分析，使用最少量的资源来达成最优性能。请注意，对于某些特定的循环，该工具的自动优化可能会提出激进的目标，从而可能导致资源消耗过多。如果特定循环的资源使用量很高，那么使用传统编译指示即可对资源进行更精细的控制。

优化技巧与故障排除提示

本章节概括的各项优化技巧可供您用于指导 AMD Vitis™ HLS 生成可满足所期望的性能和面积目标的宏架构。使用 Vitis HLS 时可以将不同的优化指令应用于设计，包括：

- 任务流水打拍，允许在当前任务完成前即开始执行下一项任务。
- 指定完成函数、循环和区域的目标时延。
- 指定使用的资源数量限制。
- 覆盖代码中的固有或隐含依赖关系，以允许执行特定操作。例如，如果可以放弃或忽略初始数据值（例如在视频流中），则允许对存储器先读取后写入，前提是这样做可提高性能。
- 指定 I/O 协议以确保函数实参可以连接到具有相同 I/O 协议的其他硬件块。

注释： Vitis HLS 自动确定任何子函数使用的 I/O 协议。您不能控制这些端口，只能指定端口是否寄存。

这样有助于了解用于对来自 C/C++ 源代码的 RTL 硬件描述进行综合的进程。[认识高层次综合的调度与绑定](#) 描述了有关此进程的部分重要详细信息，这些信息有助于您了解如何为其进行优化。

您可使用各种 HLS 编译指示，将优化指令作为编译器的编译指示直接添加到源代码中，或者也可以使用这些指令作为配置命令，如 [添加编译指示和指令](#) 中所述。下表列出了 HLS 编译器提供的优化指令（作为编译指示或配置命令）。

表 19：Vitis HLS 优化指令

指令	描述
AGGREGATE	AGGREGATE 编译指示用于将任一结构体的所有元素都组合到一个单宽矢量中，以允许同时读取和写入该结构体的所有成员。
ALIAS	ALIAS 编译指示支持在 Vitis HLS 中通过定义访问相同 DRAM 缓冲器的多个指针之间的距离来执行数据依赖关系分析。
ALLOCATION	用于指定所使用的运算、实现或函数的数量限制。这样会强制共享硬件资源并可能增大时延。
ARRAY PARTITION	把大型阵列分区为多个较小阵列或分区为单独的寄存器，以改善数据访问并消除块 RAM 瓶颈。
ARRAY_RESHAPE	把阵列从多元素阵列重塑为字宽更宽的阵列。用于在不增加使用的块 RAM 数量的前提下提升块 RAM 访问。
BIND_OP	用于为运算定义特定的 RTL 实现。
BIND_STORAGE	用于为 RTL 中的存储元素或存储器定义特定的实现。
DATAFLOW	支持任务层次的流水打拍，允许函数和循环并发执行。用于优化吞吐量和/或时延。
DEPENDENCE	用于提供可克服循环进位依赖关系并允许循环流水打拍（或以更低的时间间隔进行流水打拍）的其他信息。
DISAGGREGATE	将结构体细分为各独立元素。
EXPRESSION_BALANCE	允许关闭自动表达式平衡。
INLINE	内联函数，移除该层次的函数层级。用于跨越函数边界实现逻辑优化，通过减少函数调用开销来改善时延/时间间隔。
INTERFACE	指定如何根据函数描述创建 RTL 端口。
LATENCY	允许指定最小和最大时延约束。

表 19: Vitis HLS 优化指令 (续)

指令	描述
LOOP_FLATTEN	允许把嵌套循环折叠为已改善时延的单一循环。
LOOP_MERGE	合并连续循环，以缩短总体时延、增加共享和提升逻辑优化。
LOOP_TRIPCOUNT	用于含变量边界的循环。提供估算的循环迭代计数。这对综合没有影响，只对报告功能有影响。
OCCURRENCE	在对函数或循环进行流水打拍时使用，用于指定某个位置的代码执行速度低于外围函数或循环中的代码执行速度。
PERFORMANCE	指定期望的循环传输事务时间间隔，允许工具判定达成此结果的最佳方式。
PIPELINE	允许在单一循环或函数内重叠执行运算，从而缩短启动时间间隔。
PROTOCOL	此命令用于指定代码区域（协议区域），除非代码中明确指定，否则 Vitis HLS 不会在其中插入时钟操作。
RESET	本指令用于在特定状态变量（全局或静态）上添加或移除复位。
STABLE	用于指示在数据流区域的入口和出口处生成同步时，可忽略某个该数据流区域的变量输入或输出。
STREAM	指定在数据流优化期间将特定阵列作为 FIFO 或 RAM 存储器通道来实现。使用 hls::stream 时，STREAM 优化指令用于覆盖 hls::stream 的配置。
TOP	综合的顶层函数在工程设置中指定。该指令可用于将任意函数指定为综合的顶层函数。这样即可允许将相同工程内的不同解决方案指定为综合的顶层函数，而无需创建新工程。
UNROLL	展开 for 循环以创建循环主体的多个实例，随后即可独立调度其指令。

除了优化指令外，Vitis HLS 还提供了多条配置命令，这些命令可能影响综合结果的性能。如需了解有关使用配置命令的详细信息，请参阅 [HLS 配置文件命令](#)。下表反映了其中部分命令。

表 20: Vitis HLS 配置

GUI 指令	描述
阵列分区配置	用于判断包括全局阵列在内的阵列分区方式，以及分区是否会影响阵列端口。
编译选项	控制综合专用优化功能，例如自动循环流水打拍和浮点运算优化。
数据流配置	用于指定数据流优化中的默认存储器通道和 FIFO 深度。
接口配置	用于控制与顶层函数实参无关的 I/O 端口，允许从最终 RTL 中去除未使用的端口。
运算符配置	配置指定运算的默认时延和实现。
RTL 配置	提供对输出 RTL 的控制权，包括文件和模块命名以及复位控制。
调度设置	判断综合调度阶段的工作量以及输出消息的详细程度
存储配置	配置指定存储类型的默认时延和实现。
展开设置	配置展开循环的默认循环次数阈值。

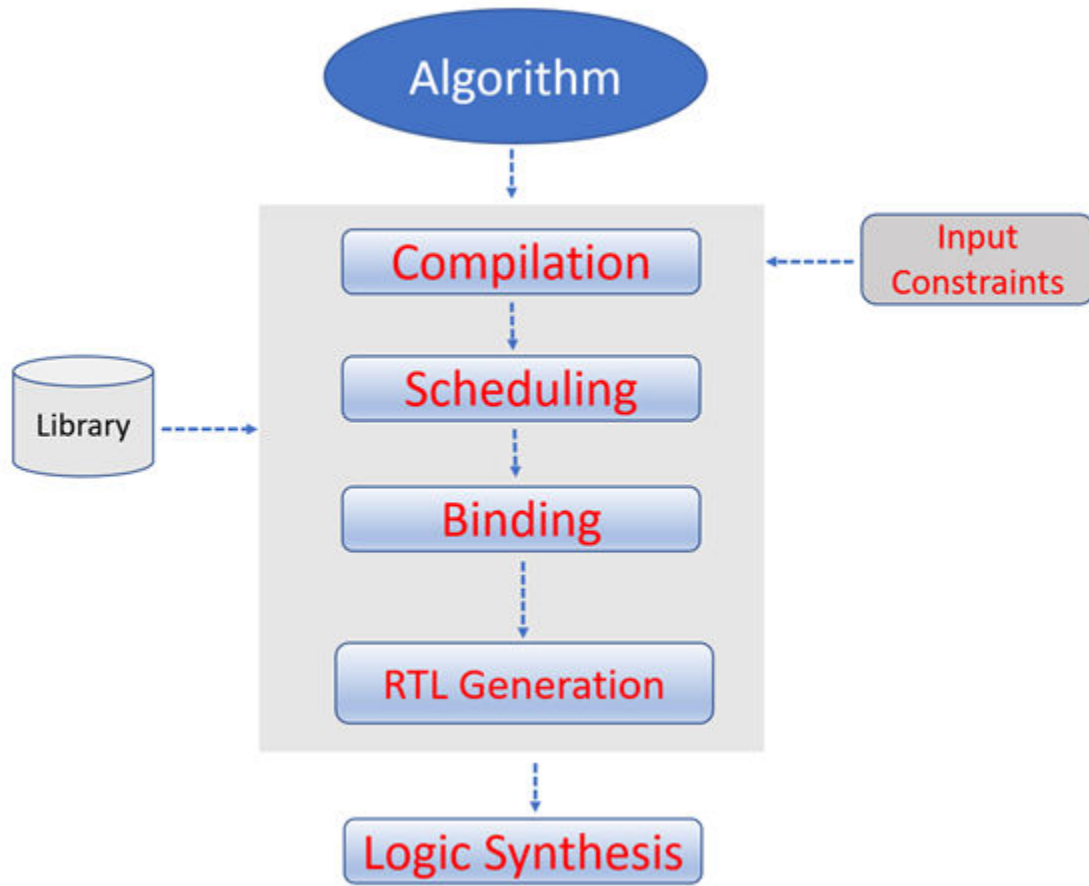
认识高层次综合的调度与绑定

高层次综合工具用于将未定时的高层次规范转变为完全定时的实现。在此变换期间，会通过实现定制架构来满足规范要求。生成的架构包含数据路径、控制逻辑、存储器接口以及 RTL 与外部世界的通信方式。数据路径由一组存储元素（例如，寄存器、寄存器文件或存储器）、一组功能单元（例如，ALU、乘法器、移位器和其他定制功能）以及互连元素（例如，三态驱动器、多路复用器和总线）组成。每个组件的执行都需耗时一个或多个时钟周期、可采用流水打拍并且可包含输入或输出寄存器。此外，完整数据路径和控制器在多个阶段内均可流水打拍。

设计师应将工程初期部分用于重新定义算法架构，以满足性能，同时将算法保持在较高层次。任何特定 HLS 工具都有设计原则和最佳实践，必须遵循这些原则和最佳实践才能生成满足目标性能优化的 RTL。

HLS 工具会执行下列任务，如下图所示。

图 54：HLS 步骤



1. 对编写的算法进行编译以满足规范：该步骤包含多项代码优化，例如，死码消除、常量折叠、报告不受支持的构造等。
2. 调度给定时钟周期的运算：
 - a. “Schedule”（调度）阶段用于基于下列条件来判定每个时钟周期内发生的运算：
 - 运算的依赖关系何时已得到满足或者变为可用。
 - 时钟周期长度或时钟频率。
 - 运算完成所需时间（由目标器件来定义）。时钟周期越长，单个时钟周期内可完成的运算越多。部分运算可能需要作为多周期资源来实现。HLS 会将运算自动调度到多个时钟周期内
 - 可用资源。
 - 整合任意用户指定的优化指令。
 - b. 在“Schedule”阶段内，本工具会判定将在给定周期内执行的运算符以及所需组件的数量。下一步是判定运算与对应资源之间的绑定。

- 3. 将运算绑定到功能组件，将变量绑定到存储元素
 - a. 绑定任务会分配硬件资源用于实现调度的每项运算，并将运算符（例如，加法、乘法和移位）映射到特定 RTL 实现。例如，mult 运算可在 RTL 内作为组合乘法器或流水打拍乘法器来实现。
 - b. 绑定任务会将存储器和/或寄存器分配到函数内的阵列变量以满足目标性能。
 - c. 如果多项运算使用相同资源，那么此步骤可执行资源共享，前提是不在同一个周期内使用。
- 4. 控制逻辑抽取可创建有限状态机 (FSM)，根据定义的调度按顺序执行 RTL 设计中的运算。
- 5. 创建逻辑以与外部世界进行通信：生成的 RTL 将与外部世界进行通信，例如，对来自外部端口或起始/停止逻辑的数据进行串流，或者访问外部存储器。
- 6. 最后，生成 RTL 架构

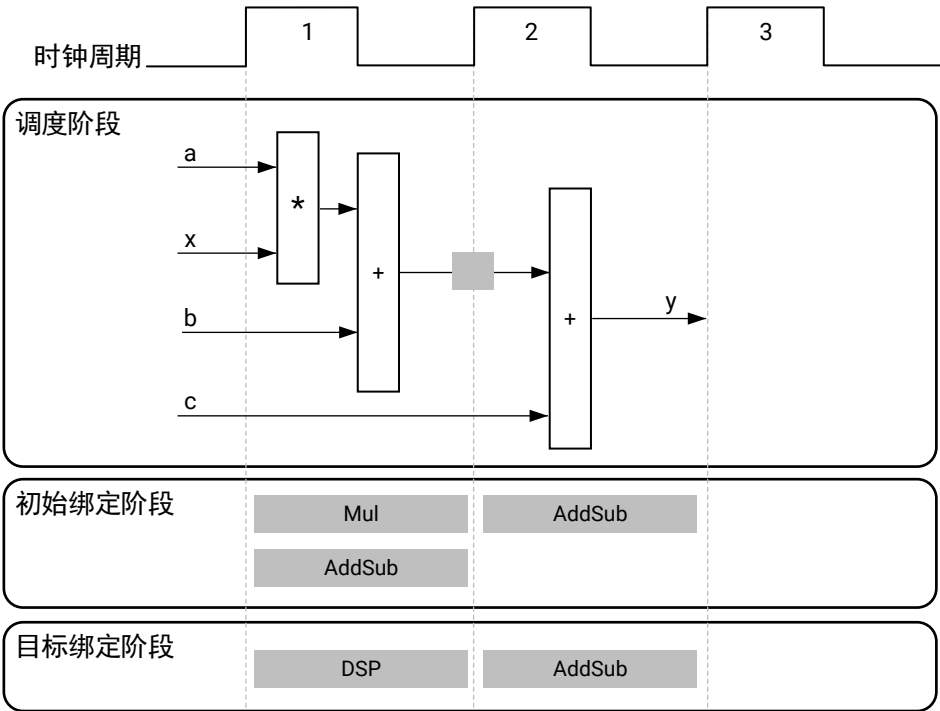
下一章按概念逐步讲解了 HLS 工具基于输入约束（例如，时钟周期）调度运算符以及将运算符绑定到可用硬件资源的总体方式。

调度和绑定示例

下图显示了此代码示例的调度和绑定阶段的示例：

```
int foo(char x, char a, char b, char c) {  
    char y;  
    y = x*a+b+c;  
    return y;  
}
```

图 55：调度和绑定示例



X14220-030622

在此示例的调度阶段，根据高层次综合的调度，在每个时钟周期内将执行以下操作：

- 第一个时钟周期：乘法和第一次加法
- 第二个时钟周期：如果在第二个时钟周期内第一次加法的结果可用，则执行第二次加法并输出生成

注释：在上图中，第一个和第二个时钟周期之间的方块指示内部寄存器存储变量的时间。在此示例中，高层次综合仅要求在一个时钟周期内寄存加法的输出。第一个周期读取 `x`、`a` 和 `b` 数据端口。第二个周期读取数据端口 `c` 并生成输出 `y`。

在最终的硬件实现中，高层次综合将顶层函数的实参实现为输入和输出 (I/O) 端口。在此示例中，实参是简单数据端口。因为每个输入变量都是 `char` 类型，所以输入数据端口位宽均为 8 位。`return` 函数为 32 位 `int` 数据类型，而输出数据端口位宽为 32 位。



重要提示！在硬件中实现 C 语言代码的优点是所有运算都在更少的时钟周期内完成。在此示例中，完成运算仅需 2 个时钟周期。在中央处理器 (CPU) 中，即使这个简单的代码示例也需要更多时钟周期才能完成。

在此示例的初始绑定阶段，高层次综合使用组合乘法器 (Mul) 实现乘法运算，并使用组合加法器/减法器 (AddSub) 实现两次加法运算。

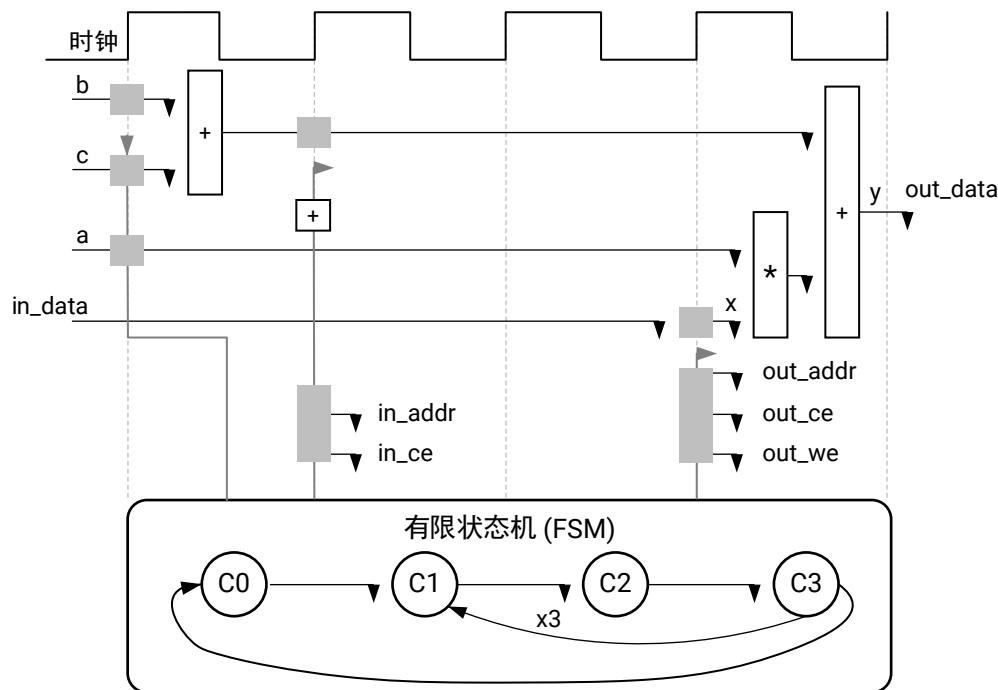
在目标绑定阶段，高层次综合使用 DSP 模块资源实现乘法器和其中一项加法运算。部分应用使用大量二进制乘法器和累加器，这些乘法器和累加器最好在专用 DSP 资源内实现。DSP 模块是 FPGA 架构中可用的计算块，可在高性能和高效实现之间达成理想的平衡。

控制逻辑提取和 I/O 端口实现示例

下图显示了对应此代码示例的控制逻辑的提取和 I/O 端口的实现过程：

```
void foo(int in[3], char a, char b, char c, int out[3]) {
    int x,y;
    for(int i = 0; i < 3; i++) {
        x = in[i];
        y = a*x + b + c;
        out[i] = y;
    }
}
```

图 56：控制逻辑提取和 I/O 端口实现示例



X14218-030622

此代码示例执行的运算与前述示例相同。但它在 for 循环内执行运算，且 2 个函数实参为阵列。调度此代码后，生成的设计在 for 循环内执行 3 次该逻辑。高层次综合会从 C 语言代码自动提取控制逻辑，并在 RTL 设计中创建 FSM 以对这些运算进行排序。顶层函数实参在最终 RTL 设计中变为端口。类型为 `char` 的标量变量将映射到标准 8 位数据总线端口。阵列实参（如 `in` 和 `out`）包含完整数据集。

在高层次综合中，阵列默认综合到块 RAM 中，但还有其他选项可供选择，例如，FIFO、分布式 RAM 和独立寄存器。使用阵列作为顶层函数中的实参时，高层次综合假定块 RAM 位于顶层函数外，并自动创建端口以访问位于设计外的块 RAM，例如，数据端口、地址端口和任何必需的芯片使能信号或写使能信号。

FSM 用于控制寄存器何时存储数据并控制任意 I/O 控制信号的状态。FSM 启动状态为 C0。在下一个时钟上，它会按顺序依次进入 C1 状态、C2 状态和 C3 状态。它会返回 C1、C2 和 C3 状态总计 3 次，最后返回 C0 状态。

注释：这与 C 语言代码中 for 循环的控制结构非常相似。状态的完整顺序为：C0、{C1, C2, C3}、{C1, C2, C3}、{C1, C2, C3}，然后返回 C0。

设计要求仅添加 1 次 b 和 c。高层次综合会将运算移出 for 循环，并移入 C0 状态。每次设计进入 C3 状态时，都会复用加法的结果。

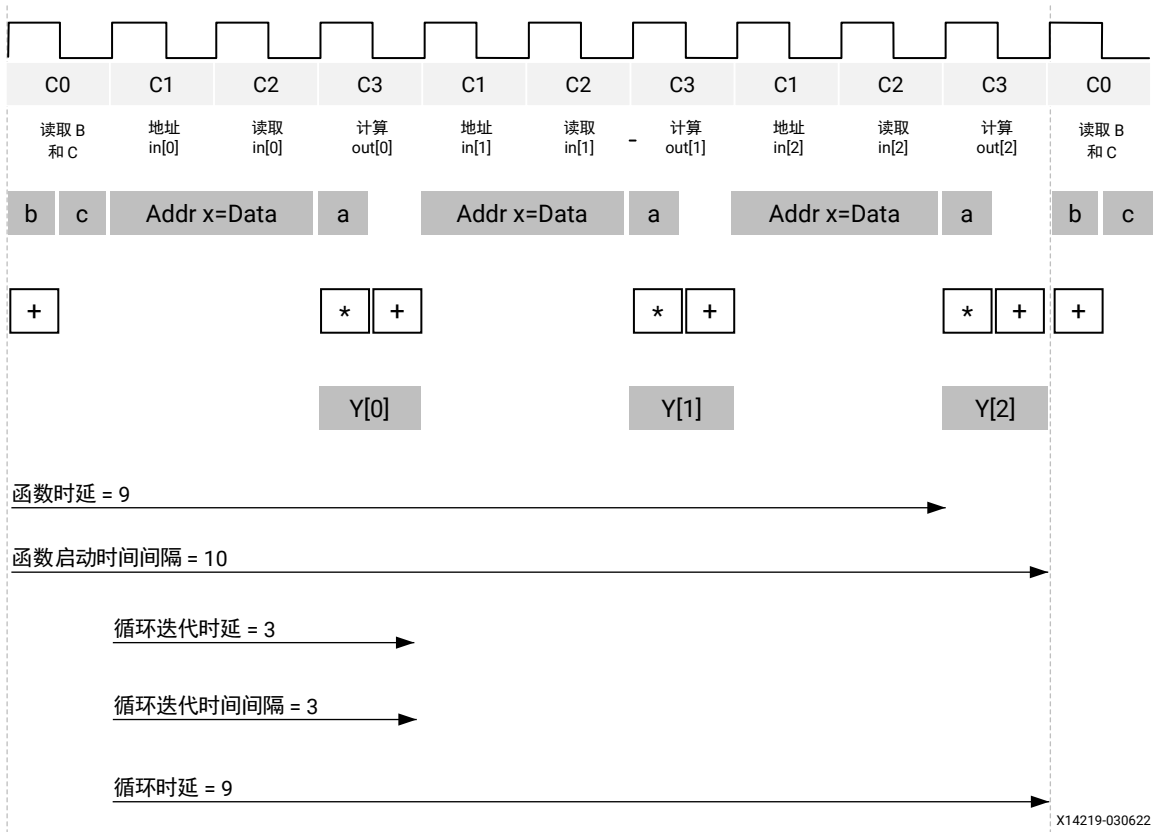
设计会从 `in` 读取数据，并将数据存储到 `x` 中。FSM 会为处于 C1 状态的首个元素生成地址。此外，在 C1 状态中，加法器会递增以记录设计必须围绕 C1、C2 和 C3 状态进行迭代的次数。在 C2 状态下，块 RAM 会返回 `in` 的数据，并将其存储在 `x` 变量中。

高层次综合会从含其他值的 `a` 端口读取数据，以执行计算，并生成首个 `y` 输出。FSM 可确保生成正确的地址和控制信号，以将该值存储在块范围外。随后，设计会返回 C1 状态以从阵列/块 RAM `in` 读取下一个值。此进程将持续直至写完所有输出为止。随后，设计会返回 C0 状态，以读取 b 和 c 的下一个值，以重新开始此进程。

性能指标示例

下图显示了上一个示例中每个周期的代码完整执行过程，包括每个时钟周期的状态、读取操作、计算操作和写入操作。

图 57：时延和启动时间间隔示例



以下是此示例的性能指标：

- 时延：该函数需要 9 个时钟周期才能输出所有值。
注释：当输出为阵列时，时延测量到最后一个阵列值输出为止。
- 启动时间间隔 (II)：II 为 10，这意味着该函数需要 10 个时钟周期后才能启动一组新的输入读取并开始处理下一组输入数据。
注释：函数的一次完整执行的时间称为一项传输事务。在此示例中，该函数需要 11 个时钟周期后才能接受下一项传输事务的数据。
- 循环迭代时延：每次循环迭代的时延为 3 个时钟周期。
- 循环 II：时间间隔为 3。
- 循环时延：时延为 9 个时钟周期。

优化逻辑

推断移位寄存器

Vitis HLS 现在将在遇到以下代码时推断移位寄存器：

```
int A[N]; // This will be replaced by a shift register

for(...) {
    // The loop below is the shift operation
    for (int i = 0; i < N-1; ++i)
        A[i] = A[i+1];
    A[N] = ...;

    // This is an access to the shift register
    ... A[x] ...
}
```

移位寄存器每个周期都能执行一次移位操作，还支持在移位寄存器内任意位置每个周期执行一次随机读取访问，因此，比 FIFO 更灵活。

优化逻辑表达式

在综合期间会执行多次优化（例如，强度折减和位宽最小化）。在自动优化列表中也包括表达式平衡。

表达式平衡会重新排列运算符以构造平衡的树结构并降低时延。

- 对于整数运算，默认情况下表达式平衡处于开启状态，但可使用 [EXPRESSION_BALANCE](#) 编译指示或指令将其禁用。
- 对于浮点运算，默认情况下表达式平衡处于关闭状态，但可使用 `config_compile -unsafe_math_optimizations` 来启用，如下所述。

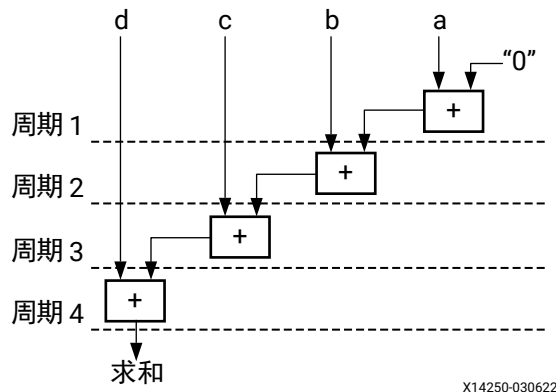
对于使用如下示例所示或者由循环展开而产生的 `+=` 和 `*=` 之类的赋值运算符的高度循序代码：

```
data_t foo_top (data_t a, data_t b, data_t c, data_t d)
{
    data_t sum;

    sum = 0;
    sum += a;
    sum += b;
    sum += c;
    sum += d;
    return sum;
}
```

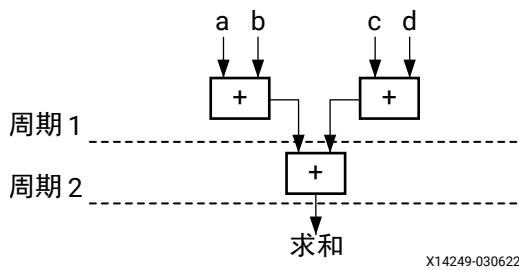
如果不使用表达式平衡并且假定每个加法都需要 1 个时钟周期，那么完整计算 `sum` 需要 4 个时钟周期，如下图所示。

图 58：加法树



但加法 $a+b$ 和 $c+d$ 可并行执行以缩减时延。经过平衡后，计算可在 2 个时钟周期内完成，如下图所示。表达式平衡禁止共享，导致面积增加。

图 59：平衡后的加法树



对于整数，您可使用 `EXPRESSION_BALANCE` 优化指令配合 `off` 选项来禁用表达式平衡。默认情况下，HLS 工具对于 `float` 类型或 `double` 类型的运算不执行 `EXPRESSION_BALANCE` 优化。对 `float` 和 `double` 类型进行综合时，该工具会保留 C/C++ 语言代码中执行的运算顺序，以确保结果与 C/C++ 语言仿真相同。例如，在以下代码示例中，所有变量类型均为 `float` 或 `double`。O1 和 O2 即使看似执行相同的基本操作，其值也并不同。

```
A=B*C; A=B*F;
D=E*F; D=E*C;
O1=A*D O2=A*D;
```

此行为是 C/C++ 语言标准中执行 `float` 或 `double` 类型的运算时饱和和舍入所导致的结果。因此，存在类型为 `float` 或 `double` 的函数时，HLS 编译器始终保留运算顺序不变，并且默认不执行表达式平衡。

您可为特定运算启用表达式平衡，或者也可配置该工具，以使用 `syn.compile.unsafe_math_optimizations=1` 配置命令来对 `float` 类型和 `double` 类型启用表达式平衡。

启用此设置后，该工具可能会更改运算顺序，以生成更优化的设计。但 C/RTL 协同仿真的结果可能与 C/C++ 语言仿真不同。

`unsafe_math_optimizations` 功能还支持 `syn.compile.no_signed_zeros` 优化。`no_signed_zeros` 优化可确保以下表达式配合 `float` 类型和 `double` 类型使用时结果完全相同：

```
x - 0.0 = x;
x + 0.0 = x;
0.0 - x = -x;
x - x = 0.0;
x*0.0 = 0.0;
```

如果不使用 `no_signed_zeros` 优化，由于舍入，以上表达式结果将不同。如果仅选中 `syn.compile.no_signed_zeros` 配置命令，那么就能选择在不使用表达式平衡的情况下执行此优化。



提示：使用 `unsafe_math_optimizations` 和 `no_signed_zero` 优化时，RTL 实现结果将不同于 C/C++ 语言仿真。测试平台应可忽略结果中的轻微差异：检查范围，不执行精确比对。

AXI 系统性能优化

简介

Vitis 加速系统包含一个全局存储器子系统，用于在内核与主机应用之间共享数据。AMD 器件外部的宿主系统上可用的全局存储器可以提供超大量的存储空间，但代价是所需访问时间远超过 AMD 器件上的本地存储器。系统/应用性能的衡量指标之一就是吞吐量，其定义为给定时间范围内传输的字节数。因此，往来全局存储器的数据传输不足将导致冗长的存储器访问时间，从而给系统性能及内核执行时间造成不利影响。

在 Vitis HLS 中开发加速应用包含 2 个阶段：内核开发和提升系统性能。[设计原则](#) 建议采用如下内核开发方法：实现类似高速缓存的加载 - 计算 - 存储结构，其中加载 - 计算函数用于对全局存储器执行读取/访问。提升系统性能则主要用于实现高效的加载存储设计，从而改善内核执行时间。本章主要描述可能影响和改善加载 - 存储 (LS) 函数的吞吐量的功能特性和指标。请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Interface/Memory](#)，获取以下部分概念的示例。

AXI 突发传输

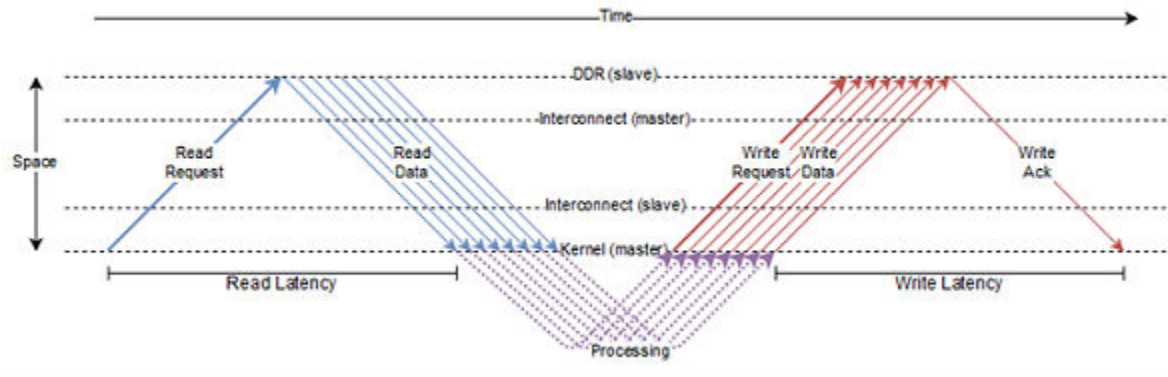
突发传输概述

突发是一种优化方式，它会尝试以智能方式聚集对 DDR 的存储器访问操作，以便尽可能提升吞吐量带宽和/或减小延迟。突发是可对内核执行的诸多优化操作之一。突发通常可以实现 4 到 5 倍的提升，而其他优化（例如，访问拓宽或者确保不存在通过 DDR 存储器的依赖关系）甚至可提供更大的性能提升。通常 DDR 端口上存在争用（源于多个相互竞争的内核）时，适合使用突发。

Vitis HLS 工具支持自动突发推断，并且支持多种功能特性，例如，自动拓宽端口以适应自动突发访问。在某些情况下，如果自动突发访问失败，那么有效的解决方案是重写代码或者使用手动突发，如 [使用手动突发](#) 中所述。如果这些方法无效，那么另一个解决方案可能是在 AXI4 接口中通过 [CACHE](#) 编译指示或指令来使用高速缓存存储器。

AXI4 协议的突发功能特性能够通过单个请求内在全局存储器上读取/写入多个数据块来提升加载存储功能的吞吐量。数据越大，吞吐量越高。这种指标的计算方式如下： $((\text{传输的字节数}) * (\text{内核频率})) / (\text{时间})$ 。最大内核接口位宽为 512 位，如果内核编译为按 300 MHz 频率运行，那么理论上每个 DDR 存储器可达成 $(512 * 300 \text{ MHz}) / 1 \text{ 秒} = \sim 17 \text{ GB/s}$ 。

图 60: AXI 协议



上图显示 AXI 协议的工作方式。HLS 内核会发出读取请求（突发长度为 8），然后发出写入请求（突发长度为 8）。读取时延定义为发送读取请求突发直到内核接收到来自突发中的首个读取请求的数据之间所耗费的时间。同样，写入时延定义为发送写入突发中的最后一次写入操作的数据的时间与内核接收到写入确认的时间之间所耗用的时间。读取请求通常在写入请求入队并首次变为可用即开始发送，直至突发中的每次写入的数据变为可用为止。



提示：此行为是由 `syn.interface.m_axi_conservative_mode` 选项来判定的（如 [接口配置](#) 中所述），该选项默认启用。

要了解突发传输的底层语义，请考量如下代码片段：

```
for(size_t i = 0; i < size; i++) {
    out[f(i)] = in[f(i)];
}
```

Vitis HLS 会自动执行突发优化，它以智能方式聚集来自用户代码的循环/函数内部的存储器访问，并对特定大小的全局存储器执行读/写。这些读/写会被转换为针对全局存储器的读取请求、写入请求和写入响应。根据存储器访问模式，Vitis HLS 会将这些读取和写入请求自动插入循环边界外部或者循环主体内部。根据这些请求的布局，Vitis HLS 可定义两种突发请求类型：顺序突发和流水打拍突发。

突发语义

对于给定内核，HLS 编译器会以多通道优化（而非基于每个函数）的方式来实现突发分析优化。仅对单个函数执行突发，不支持跨函数执行突发。在 [综合汇总](#) 报告中会提供突发优化报告，并且还会报告错失的突发机会，以便帮助您改善突发优化。

首先，HLS 编译器会在函数基本块中查找存储器访问，例如，函数内部一组连续的语句中的存储器访问。假定已满足突发的前置条件，这些基本块中推断的每次突发都被称为顺序突发。编译器将自动扫描基本块以将最长的连续访问构建为单一顺序突发。

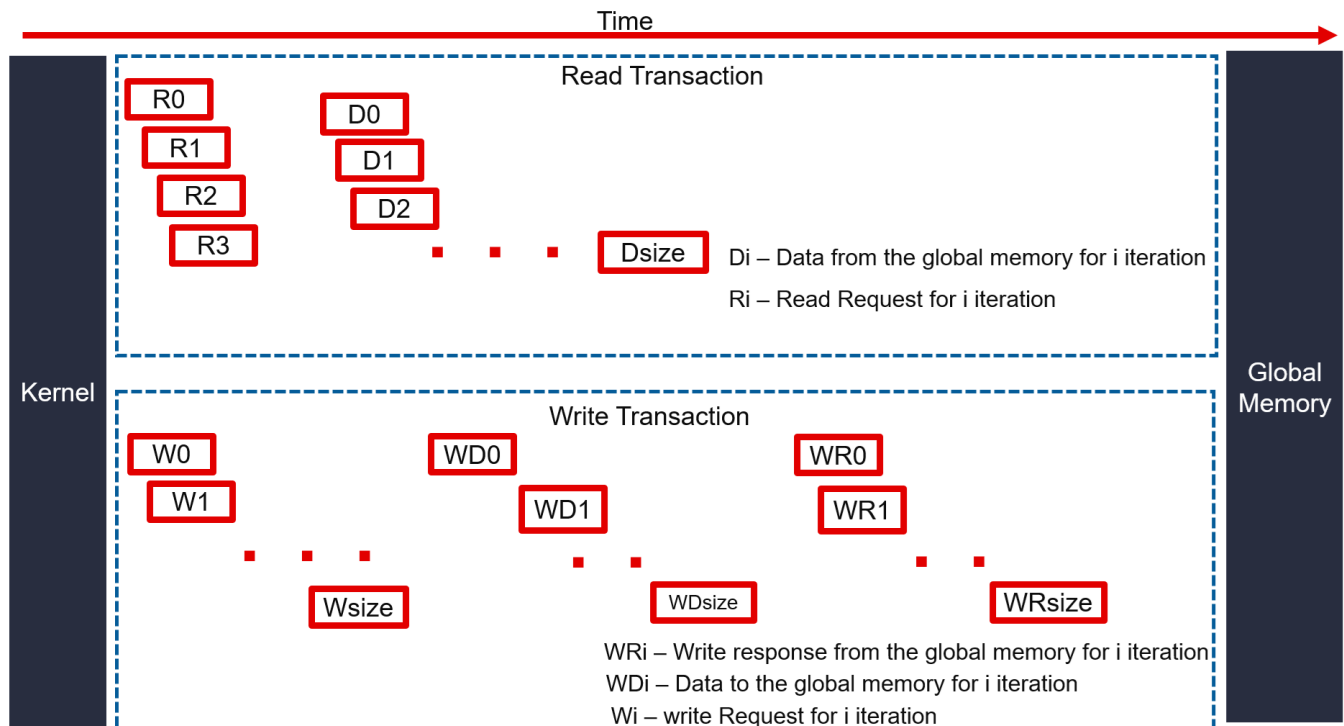
随后，编译器会查找循环，并尝试推断哪些突发属于流水线突发。所谓流水线突发，是指跨循环迭代的连续读/写操作。编译器会尝试通过分析循环归纳变量和循环次数来推断突发长度。如果分析成功，编译器即可将每个循环迭代中的一连串读/写链接到单一长条流水线突发中。现如今，编译器会自动推断流水线突发或顺序突发，但无法请求特定类型的突发。需编写代码以使该工具推断流水线突发或顺序突发。

流水线突发

流水线突发通过在单个请求内读取或写入大量数据或大量的数据来改善函数吞吐量。流水线突发的优势在于后续请求 (i+1) 无需等待当前请求 (i) 完成，因为读取请求、写入请求和写入响应均位于循环主体外部，并且流水线突发可以尽快执行请求，如以下代码示例所示。这样可以显著提升函数吞吐量，因为读/写整个循环边界所需的时间更短。

```
rb = ReadReq(i, size);
wb = WriteReq(i, size);
for(size_t i = 0; i < size; i++) {
    Write(wb, i) = f(Read(rb, i));
}
WriteResp(wb);
```

图 61：流水线突发



如果编译器可成功减少来自归纳变量 (*i*) 和循环次数 (*size*) 的突发长度，那么它将推断一个大型流水线突发，并且将 `ReadReq`、`WriteReq` 和 `WriteResp` 调用移至循环外部，如“流水线突发”代码示例所示。因此，所有循环迭代的读取请求都会组合为单个读取请求，所有写入请求也都会组合为单个写入请求。所有读取请求通常会立即发出，而写入请求仅在数据变为可用后才会发出。

在此情况下，每个循环迭代的读取请求和写入请求均可组合为一个读取或写入请求。但如有任何突发前置条件未能得到满足（如 [突发传输的前置条件和限制](#) 中所述），那么编译器将无法推断流水线突发，但是将改为尝试推断顺序突发，其中，`ReadReq`、`WriteReq` 和 `WriteResp` 与要进行突发优化的读/写访问位于同一处，如“顺序突发”代码示例中所示。

顺序突发

顺序突发由大小较小的数据组成，其中读取请求、写入请求和写入响应均位于循环主体内部，如以下代码示例中所示。

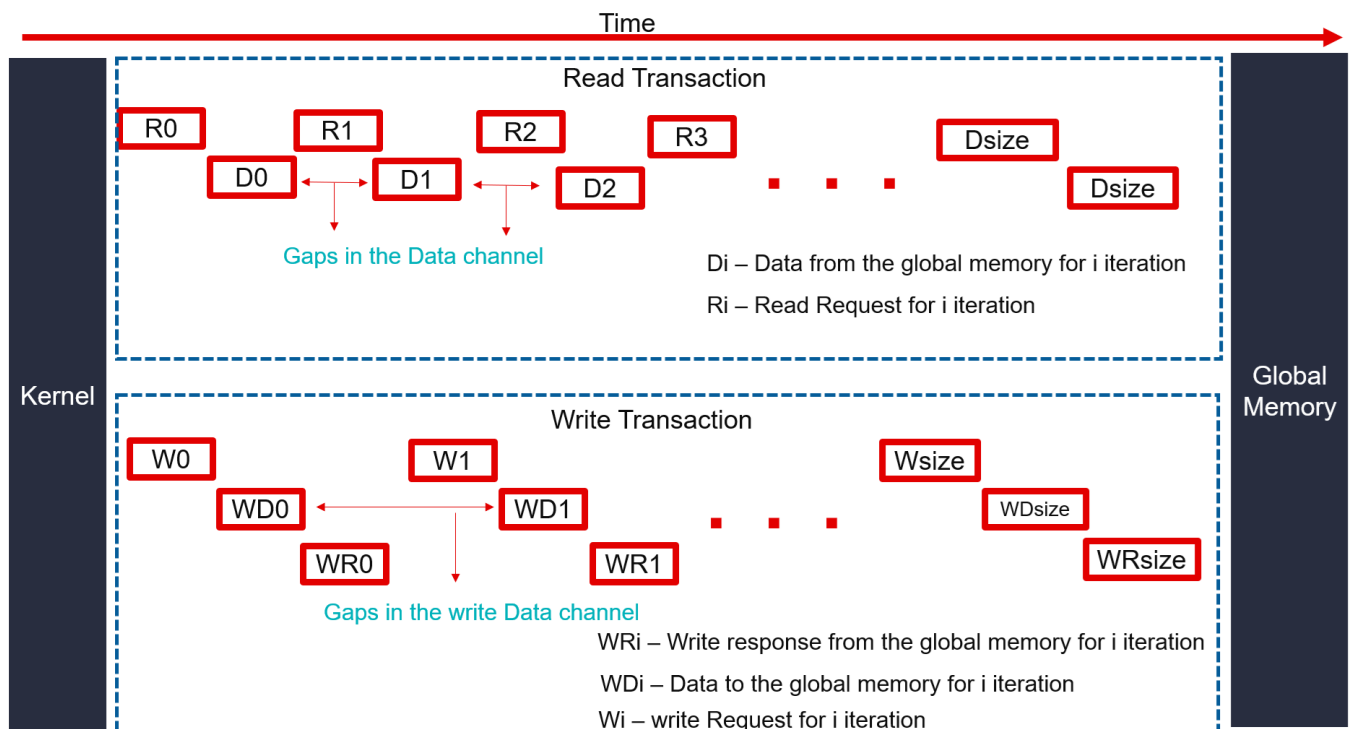
```

for(size_t i = 0; i < size; i++) {
    rb = ReadReq(i, 1);
    wb = WriteReq(i, 1);
    Write(wb, i) = f(Read(rb, i));
    WriteResp(wb);
}

```

顺序突发的缺点在于后续请求 (i+1) 依赖于先前请求 (i) 完成后才能发出，因为它会等待读取请求、写入请求和写入响应完成。这将导致请求之间出现间隔，如下图所示。

图 62：顺序突发



顺序突发效率不及流水线突发，因为它多次读取或写入少量数据，以补偿循环边界。虽然这将显著影响吞吐量，但是顺序突发仍好于无突发。如果您的代码未遵循 [突发传输的前置条件和限制](#)，那么 Vitis HLS 使用此突发方法。



提示：突发请求的大小可以进一步分区为多个请求，大小由用户使用 INTERFACE 编译指示或指令的 `max_read_burst_length` 和 `max_write_burst_length` 来指定，如 [控制 AXI4 突发行为的选项](#) 中所述。

突发传输的前置条件和限制

突发前置条件

突发即汇总连续的存储器访问请求。这些连续突发必须满足下列前置条件才能成功启动突发优化：

- 请求必须全部为读取请求或者全部为写入请求 - 不支持突发读取和写入。

- 必须按单调递增访问顺序（访问的存储器位置和访问时间两方面都是如此）。不能访问位于先前访问的两处存储器位置之间的存储器位置。
- 必须为连续存储器 - 两个存储器之间按正序彼此相邻且无间隔或重叠。
- 读取/写入访问的次数（或突发长度）必须可在发出请求之前确定。这意味着即使在运行时计算得到突发长度，仍必须在发出读取/写入请求之前完成计算。
- 如果将两个阵列绑定到同一个 M-AXI 端口，那么任意给定时间内，任一方向上最多仅针对一个阵列执行突发。
- 如果在同一区域的同一捆绑的同一通道上进行同向访问，则所有这些访问都不会出现突发。
- 突发请求的发起时间与完成时间之间不得存在任何依赖关系问题。



提示： `volatile` 限定符会阻止往来此变量的突发访问。

由于存储器访问重叠导致的外层循环突发失败

在以下示例中外层循环突发推断将失败，因为循环 L1 的迭代 0 和迭代 1 访问的是阵列中的相同元素 a 和 b。突发推断属于全有或全无类型的优化，即，该工具不会推断部分突发。它是一种贪婪的算法，会尝试最大程度增加突发长度。自动突发推断将尝试以自下而上的方式来推断突发，即从内层向外层，当任一前置条件无法得到满足时即停止。在以下示例中，突发推断发现重复读取元素 8 时就会停止，因此，在此例中，将推断得到内层突发长度为 9。

```
L1: for (int i = 0; i < 8; ++i)
    L2: for (int j = 0; j < 9; ++j)
        b[i*8 + j] = a[i*8 + j];

itr 0: | 0 1 2 3 4 5 6 7 8 |
itr 1: |                   8 9 10 11 12 13 14 15 16 |
```

使用 `ap_int`/`ap_uint` 类型作为循环归纳变量

由于突发推断取决于循环归纳变量和循环次数，因此使用非原生类型可能阻碍优化的触发。建议循环归纳变量始终使用无符号的整数类型。

必须至少进入循环一次

在某些情况下，编译器可能无法推断出循环归纳变量最大值从不为零 (0)，即，它无法证明始终能够进入循环。在此类情况下，断言语句将有助于编译器完成此推断。

```
assert (N > 0);
L1: for(int a = 0; a < N; ++a) { ... }
```

阵列上的循环间或循环内依赖关系

如果写入阵列位置，然后在同一迭代或下一次迭代内从该位置读取，可能导致优化难以对此类阵列依赖关系进行解密。基本上，对于此类案例，优化将失败，因为它无法保证写入将先于读取发生。

针对存储器的有条件访问

如果执行的存储器访问是有条件访问，则可能导致突发推断算法失败，因为它无法通过条件语句来完成推理。在某些情况下，编译器将简化条件语句，甚至将其移除，但一般建议在存储器访问周围避免使用条件语句。

在从循环调用的函数内部执行 M-AXI 访问

跨函数阵列访问分析并不适合编译器转换（如突发推断）。在此类情况下，用户可以使用 `INLINE` 编译指示或指令来内联函数，以避免突发失败。

```

void my_function(hls::stream<T> &out_pkt, int *din, int input_idx) {
    T v;
    v.data = din[input_idx];
    out_pkt.write(v);
}

void my_kernel(hls::stream<T> &out_pkt,
               int *din,
               int num_512_bytes,
               int num_times) {
    #pragma HLS INTERFACE mode=m_axi port = din offset=slave bundle=gmem0
    #pragma HLS INTERFACE mode=axis port=out_pkt
    #pragma HLS INTERFACE mode=s_axilite port=din bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=num_512_bytes bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=num_times bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=return bundle=control

    unsigned int idx = 0;
    L0: for (int i = 0; i < num_times; ++i) {
        L1: for (int j = 0; j < num_512_bytes; ++j) {
            #pragma HLS PIPELINE
            my_function(out_pkt, din, idx++);
        }
    }
}

```

由于从调用的函数执行存储器访问，因此导致突发推断将会失败。为了正常完成突发推断，建议用户对访问 M-AXI 存储器的任意此类函数执行内联。

在此示例中，突发推断失败的另一个原因是通过 `my_function` 中的 `din` 来访问存储器，这是通过变量 `(idx)` 来定义的，此变量不属于循环归纳变量 `i` 和 `j` 的函数，因此可能并非顺序或单调变量。请勿传递 `idx`，可改用 `(i*num_512_bytes+j)`。

数据流循环上的流水打拍突发推断

在具有 `DATAFLOW` 编译指示或指令的循环上不支持突发推断。但数据流循环内的每个进程/任务都具有突发。并且，由于任务可以并行执行，因此不支持在数据流区域内共享 M-AXI 端口。

控制 AXI4 突发行为的选项

当设计在等待访问总线时，从不会在最优 AXI4 接口上停滞，授予总线访问后，总线等待设计执行读/写时从不停滞。影响系统性能和突发传输的设计元素有很多，例如：

- 时延
- 端口宽度
- 多个端口
- 指定突发长度
- 未完成的读/写操作数

时延

读取时延定义为从发送突发读取请求直到内核接收到来自突发中的首个读取请求之间所耗用的时间。同样，写入时延定义为发送突发中的最后一次写入操作的数据的时间与内核接收到写入响应的的时间之间所耗用的时间。这些时延取决于诸如 DDR 存储器访问拥塞等系统特性，因此可采用非确定性值。有鉴于此，HLS 编译器无法准确判定综合期间的存储器读/写时延，故而使用 64 个内核周期的默认时延来调度请求和操作，如下所示。

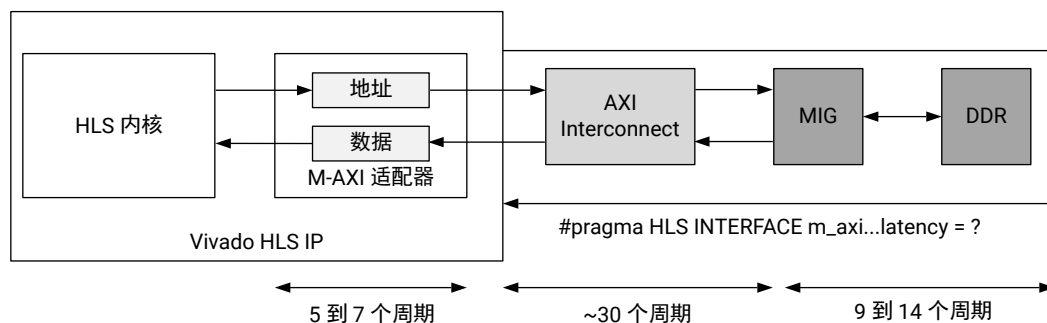
- 它会在调度读/写请求并等待数据的同时，并行执行与存储器无关的各项操作，例如，处理串流或计算
- 等待调度新的读/写请求



提示：默认工具时延可使用 **LATENCY** 编译指示或指令来进行更改。特定接口的默认时延可使用 **INTERFACE** 编译指示或指令的时延选项来更改。

为帮助您理解系统中可能存在的各种时延，下图显示了当 HLS 内核向 DDR 存储器发出突发时所发生的操作。

图 63：突发传输事务图示



X24687-030622

当设计发出读/写请求时，请求将通过多个专用帮助程序模块发送至 DDR 存储器。首先，`m_axi` 适配器会充当缓冲器，以供 HLS 内核所创建的请求使用。该适配器包含用于将大型突发剪切为小型突发的逻辑，目的是为了避免占用通道或者避免请求超过 4 KB 边界，请参阅《Vivado Design Suite：AXI 参考指南》(UG1037)。

默认情况下，适配会暂停发送突发请求（根据未完成请求参数的最大值），直至所有数据变为可用为止，以便安全缓冲每个内核的完整数据。这样做的目的是减少因存储器子系统上的并发请求（读取或写入请求）而导致的死锁，但这可能导致写入时延有所增加。您可通过按 [接口配置](#) 中所述设置

`syn.interface.m_axi-conservative-mode=false` 来禁用此写入请求的保持时间，但会增大发生死锁的风险。

穿越此适配器需耗费数个时延周期，通常需 5 到 7 个周期。随后，请求会穿越 AXI Interconnect，后者将把内核请求路由至 MIG 并最终发送到 DDR 存储器。穿越 AXI Interconnect 需耗费大量时延，可能需约 30 个周期。最终到达 DDR，返回则需耗费 9 到 14 个周期。这些提供的数值并非精确测得的时延，仅为估算，用于展示这些专用模块的相对时延成本。如需获取更精确的测量值，您可使用特定于您的系统的“Application Timeline”（应用时间线）报告来测试并观察这些时延，如 [AXI 性能案例研究](#) 中所述。



提示：如需了解有关“Application Timeline”报告的更多信息，请参阅《使用 Vitis 进行数据中心加速》(UG1700) 中的[时间线轨迹](#)。

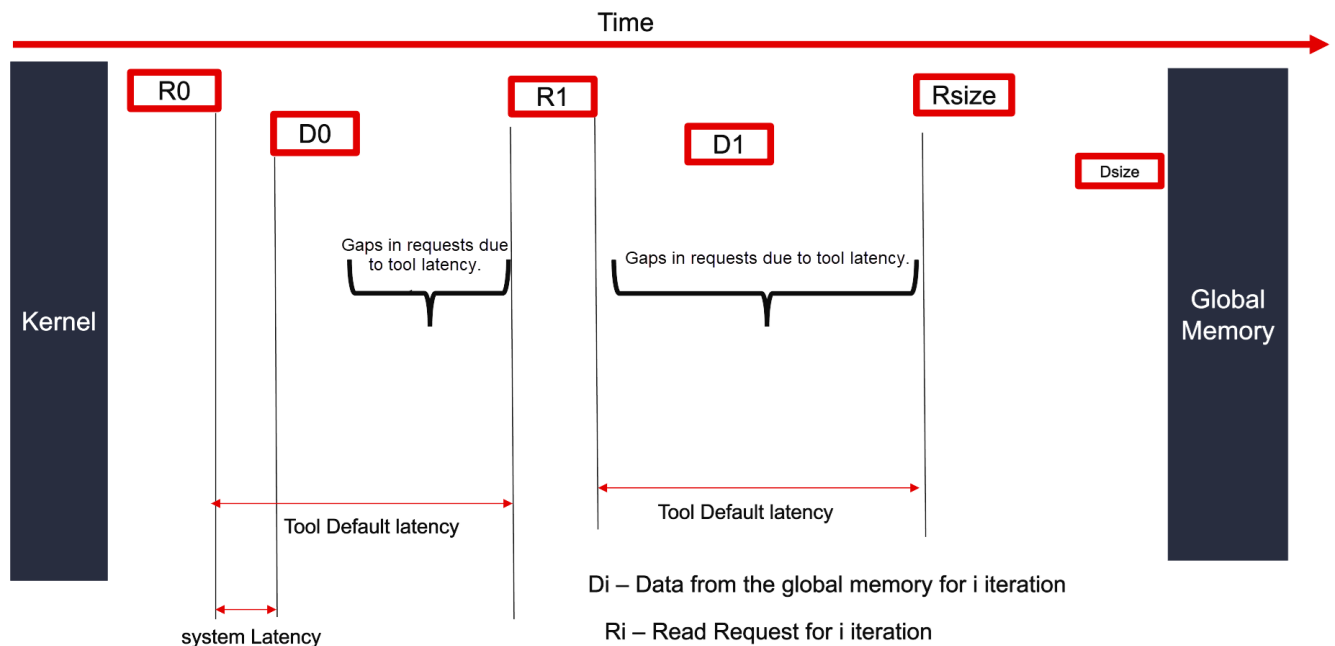
以下提供了查看系统中的时延的另一种方法：互连接口的平均启动时间间隔 (II) 为 2，而 DDR 存储器控制器用于请求的平均 II 则为 4-5 周期（两者在数据上耗费的 II 均为 1）。互连接口的仲裁测量是根据读/写请求大小来进行仲裁的，因此所请求的数据的突发长度越长，则优先级越高（从而导致发生争用时，分配给较长的突发的通道带宽更大）。当然，大型突发请求的副作用是阻止其他任何请求访问 DDR 存储器，因此，在突发长度与减少 DDR 端口争用之间必须有所取舍。幸好，大延迟有助于防止出现某些此类端口争用问题，而对请求进行有效的流水打拍则可以显著提升系统中可用的带宽吞吐量。

时延不影响具有流水打拍的突发的循环/函数，因为突发在单次请求内即请求最大大小。

时延可能会在以下两方面影响具有顺序突发的循环/函数：

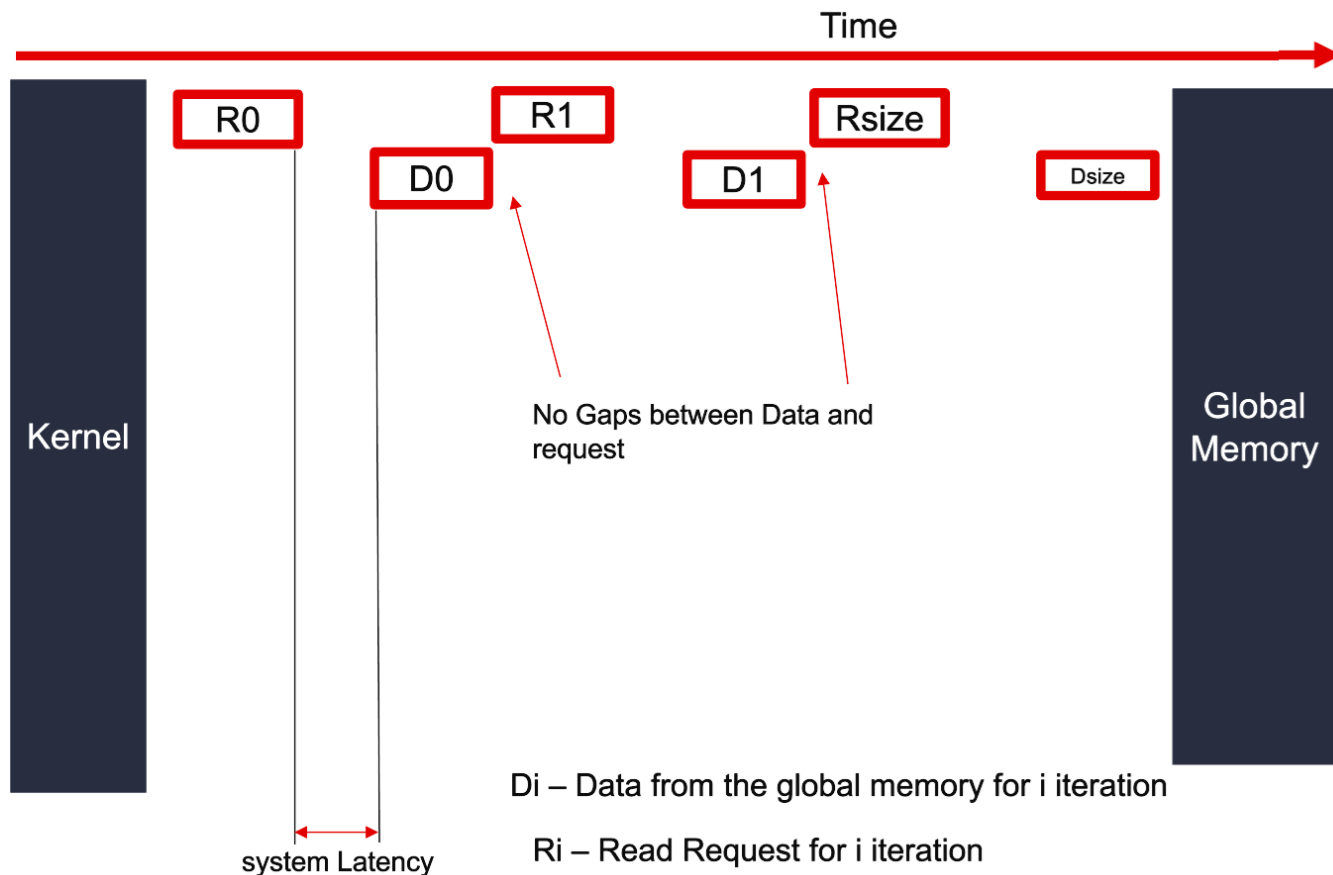
- 如果系统读/写时延大于默认工具时延，则 Vitis HLS 必须等待数据。更改时延将无法改善系统性能。
- 如果读/写时延小于工具默认时延，那么 Vitis HLS 会保持空闲 (Idle) 状态，并浪费剩余内核周期。由于在此空闲状态期间不执行任务，因此可能影响设计性能。如下图所示，系统时延与默认时延参数之间的差异会导致顺序请求被进一步延迟。这会导致吞吐量显著损失。

图 64：默认工具时延



但当您使用 LATENCY 编译指示或指令减少工具时延时，本工具将紧缩请求以供单一顺序突发使用，如下图所示。

图 65：调整后的工具时延



建议：时延对顺序突发存在显著影响。减少默认工具时延可以提升系统性能。

端口宽度

通过最大限度提升传输的字节数，可以进一步改善加载 - 存储函数的吞吐量。Vitis Unified IDE 和 `v++` 支持最大位宽为 1024 位的内核，这意味着内核每个端口每个时钟周期可以读取或写入多达 128 字节。



建议：您应尽可能增大接口的端口宽度，将每个 AXI 端口的位宽设置为 512 位（64 字节）。

Vitis HLS 还支持通过分析源代码的存储器访问模式来进行自动端口宽度优化。如果代码满足突发方法的前置条件和限制，那么它将在 Vitis 内核流程中将端口大小自动调整为 512 位。



重要提示！如果在编译时，迭代大小和次数可变，那么该工具将不会自动拓宽端口宽度。

如果该工具无法自动拓宽此端口，那么您可通过使用 [矢量数据类型](#) 或 [任意精度 \(AP\) 数据类型](#) 作为该端口的数据类型来手动更改端口宽度。

多个端口和通道

通过最大限度提升并发读/写，可以进一步改善加载 - 存储函数的吞吐量。在 Vitis HLS 中，默认情况下函数实参捆绑/映射/组合到单个端口。将多个端口捆绑到单个端口有助于节省资源。但是，单一端口可能会限制内核性能，因为所有存储器传输都必须穿越单个端口。`m_axi` 接口具有独立的 READ 和 WRITE 通道，因此单个端口可同时执行读取和写入。`m_axi` 接口还在单一端口中提供了多个通道（如 [M_AXI 通道](#) 中所述）以增加该端口内的读取和写入通道数。

使用多个端口可以创建多个接口以连接到不同存储体（如 [多 DDR 教程](#) 中所示）或者可执行顺序访问，因而可以增加内核带宽和吞吐量。当有多个实参访问同一个存储器端口或存储体时，仲裁器将把对同一个存储器端口或存储体执行的并发访问转变为顺序访问。将多个端口连接到不同的存储体可能增加加载和存储函数的吞吐量，从而导致计算块也会一并伸缩调整以满足加载和存储函数的吞吐量需求，否则将在加载存储函数上放置反压或停滞。



建议：分析并发存储器读取/写入，并采用单一专用/独立端口来执行并发访问。

未完成的读/写操作数

通过允许系统隐藏部分存储器时延，可以进一步改善加载 - 存储函数的吞吐量。`config_interface` 命令或 `INTERFACE` 编译指示或指令的 `m_axi_num_read_outstanding` 和 `m_axi_num_write_outstanding` 选项允许内核控制流水打拍的存储器请求数量，这些请求无需等待上一个请求完成即可发送到全局存储器。

增加流水打拍请求的数量会增大读/写请求的流水线深度，这将耗用额外的 BRAM/URAM 资源。

注释：大部分情况下，当突发长度 ≥ 16 时，未完成的读/写操作数应足矣。对于大小小于 16 的突发，AMD 建议将未完成的操作数量从默认值 16 加倍。

利用 INTERFACE 编译指示定义突发属性

为创建优化 AXI4 接口，在 `INTERFACE` 指令中提供了以下命令选项以指定突发的行为并优化 AXI4 接口的效率。

请注意，其中部分选项可使用内部存储空间来缓冲数据，这可能影响面积和资源：

- `latency`：指定期望的 AXI4 接口时延，允许设计发起总线请求的时间比执行期望的读取或写入操作早数个周期（时延）。如果该值太低，设计将过早达成就绪状态，可能停滞并等待总线；如果该值太高，则可能授予总线访问权时，总线仍处于停滞状态并等待设计发起访问。Vitis HLS 中的默认时延为 64。
- `max_read_burst_length`：指定突发传输期间读取的数据值的最大数量。默认值：16。
- `num_read_outstanding`：指定在设计停滞前可对 AXI4 总线发出的读取请求的数量（无响应）。此操作暗示设计中的内部存储空间：即 FIFO 大小为 `num_read_outstanding * max_read_burst_length * word_size`。默认值：16。
- `max_write_burst_length`：指定突发传输期间写入的数据值的最大数量。默认值：16。
- `num_write_outstanding`：指定在设计停滞前可对 AXI4 总线发出的写入请求的数量（无响应）。此操作暗示设计中的内部存储空间：即 FIFO 大小为 `num_write_outstanding * max_write_burst_length * word_size`。默认值：16。

以下 `INTERFACE` 编译指示示例可用于帮助解释这些选项：

```
#pragma HLS interface mode=m_axi port=input offset=slave
bundle=gmem0
depth=1024*1024*16/(512/8) latency=100 num_read_outstanding=32
num_write_outstanding=32
max_read_burst_length=16 max_write_burst_length=16
```

- 接口时延指定为 100。HLS 编译器会尝试将突发访问请求调度为比设计准备好访问 AXI4 总线早 100 个时钟周期。

- 为进一步提升总线效率，`num_write_outstanding` 选项和 `num_read_outstanding` 选项可确保设计包含的缓冲足以存储多达 32 次读取和/或写入突发。每个请求都将需要自己的缓冲器。这样即可允许设计持续处理，直至处理总线请求为止。
- 最后，`max_read_burst_length` 和 `max_write_burst_length` 选项可确保最大突发量为 16，并且 AXI4 接口暂存总线的时间不超过该设置。HLS 工具将根据指定的总线长度来对较长的突发进行分区，并通过如下消息来报告此状况：

```
Multiple burst reads of length 192 and bit width 128 in loop
'VITIS_LOOP_2' (./src/filter.cpp:247:21) has been inferred on port
'mm_read'.
These burst requests might be further partitioned into multiple requests
during RTL generation based on the max_read_burst_length settings.
```

用于配置突发的命令

这些命令可为工具配置全局设置，对 AXI4 接口进行优化，以便运行该工具的系统使用此接口。操作效率取决于能否正确设置这些值。提供的默认值均为保守值，根据您的设计的存储器访问剖析，可能需要进行更改。

表 21：Vitis HLS 控制

Vitis HLS 命令	值	描述
<code>syn.interface.m_axi_conservative_mode</code>	bool 默认值 = true	延迟 M-AXI 的每个写入请求，直至关联写入数据变为完全可用（通常，缓冲至适配器内或者已发射）。这样可能导致写入时延有所增加，但是可以解决由于存储器子系统上的并发请求（读取或写入请求）而导致的死锁。
<code>syn.interface.m_axi_latency</code>	uint 0 表示自动 对于 Vivado IP 流程，默认值为 0 对于 Vitis 内核流程，默认值为 64	为调度器提供期望的 M-AXI 访问时延值。时延即读取请求与首次读取数据之间的延迟或者最后一次写入数据与写入响应之间的延迟。请注意，该数值并不需要精确，估算过低会导致调度的时延更低，但存在更长的动态停滞。调度器将考虑其他适配器的时延并增加几个周期。
<code>syn.interface.m_axi_min_bitwidth</code>	uint 默认值为 8	M-AXI 接口数据通道的最大位宽。必须为 2 的幂（介于 8 到 1024 之间）。请注意，如果实际访问小于所需接口，那么这样并不一定会增加吞吐量。
<code>syn.interface.m_axi_max_bitwidth</code>	uint 默认值为 1024	M-AXI 接口数据通道的最大位宽。必须为 2 的幂（介于 8 到 1024 之间）。请注意，如果实际访问大于所需接口，那么这样会降低吞吐量，因为访问将被拆分为多周期访问突发。
<code>syn.interface.m_axi_max_widen_bitwidth</code>	uint 对于 Vivado IP 流程，默认值为 0 对于 Vitis 内核流程，默认值为 512	允许该工具将 M-AXI 接口上的突发自动拓宽至所选位宽。必须为 2 的幂（介于 8 到 1024 之间）。请注意，突发拓宽需要强大的对齐属性（不只限于突发）。
<code>syn.interface.m_axi_auto_max_ports</code>	bool 默认值为 false	如果该选项为 false，那么未显式捆绑的所有 M-AXI 接口都将捆绑到单一通用接口上，从而最大限度降低资源使用率（单一适配器）。如果该选项为 true，那么未显式捆绑的所有 M-AXI 接口都将映射到独立接口，从而增大资源使用率（多个适配器）。
<code>syn.interface.m_axi_alignment_byte_size</code>	uint 对于 Vivado IP 流程，默认值为 1 对于 Vitis 内核流程，默认值为 64	假定映射到 M-AXI 接口的主要函数指针至少对齐到所提供的宽度（以字节为单位，值为 2 的幂）。这样有助于自动进行突发拓宽。警告：如果指针在运行时未能准确对齐，则会发生错误行为。

表 21: Vitis HLS 控制 (续)

Vitis HLS 命令	值	描述
<code>syn.interface.m_axi_num_read_outstanding</code>	uint 默认值为 16	M-AXI num_read_outstanding 接口参数的默认值。
<code>syn.interface.m_axi_num_write_outstanding</code>	uint 默认值为 16	M-AXI num_write_outstanding 接口参数的默认值。
<code>syn.interface.m_axi_max_read_burst_length</code>	uint 默认值为 16	M-AXI max_read_burst_length 接口参数的默认值。
<code>syn.interface.m_axi_max_write_burst_length</code>	uint 默认值为 16	M-AXI max_write_burst_length 接口参数的默认值。

模板优化

模板模式算法是一种在科学计算、图像处理和数值仿真领域广泛使用的计算模式。

- 这种算法通过应用一种预定的模式或“模板”来表明相邻元素的值的合并方式，从而更新阵列中每个元素的值。
- 例如，在图像处理上下文中，模板计算可以计算像素值与其毗邻的近邻的加权平均值，以达到模糊效果。模板计算对于 FPGA 等硬件的存储器带宽具有严苛的要求，因为模板像素位置的非顺序性本质决定了它需要多次执行 DDR 存储器读取操作，这显著延长了计算时间。
- 为缓解这一问题，将利用窗口缓冲器和行缓冲器技巧来实现模板模式算法。这些技巧能优化数据访问模式，通过在片上存储器中缓存相关数据来减少多次片外存储器读取的需求。

这些技巧可以提高存储器带宽使用率，并提高并行效率和流水节拍效率。

调整算法以利用行缓冲器和窗口缓冲器可能非常耗时，需对代码进行大量重构。Vitis HLS 引入了模板优化/编译指示，可自动实现行缓冲器和窗口缓冲器并实现相同的性能。

```
for(int y=0; y<30; ++y)
{
    for(int x=0; x<1000; ++x)
    {
#pragma HLS pipeline II=1
#pragma HLS array_stencil variable=src

        // Apply 2D filter to the pixel window
        int sum = 0;
        for(int row=0; row<FILTER_V_SIZE; row++)
        {
            for(int col=0; col<FILTER_H_SIZE; col++)
            {
                unsigned char pixel;
                int xoffset = (x+col-(FILTER_H_SIZE/2));
                int yoffset = (y+row-(FILTER_V_SIZE/2));
                // Deal with boundary conditions : clamp pixels to 0
                when outside of image
                if ( (xoffset<0) || (xoffset>=1000) || (yoffset<0) ||
                (yoffset>=30) ) {
                    pixel = 0;
                } else {
                    pixel = src[yoffset][xoffset];
                }
                sum += pixel*coeffs[row][col];
            }
        }
    }
}
```

建议的编码样式示例

如 [综合汇总](#) 中所述，Vitis HLS 会发出报告，汇总突发活动并识别突发故障。如果完成可变长度突发，那么报告将提及已推断可变长度突发。编译器还会提供突发消息，在编译器 log 日志文件 `vitis_hls.log` 中可找到这些消息。这些消息是在调度步骤之前发出的。

简单读写突发推断

以下示例是读取和写入 DDR 并推断读取和写入突发的标准方法。Vitis HLS 编译器将为以下示例报告以下突发推断：

```
INFO: [HLS 214-115] Burst read of variable length and bit width 32 has been
inferred on port 'gmem'
INFO: [HLS 214-115] Burst write of variable length and bit width 32 has
been inferred on port 'gmem' (./src/vadd.cpp:75:9).
```

此示例的代码如下所示：

```
/* ***** BEGIN EXAMPLE ***** */

#define DATA_SIZE 2048
// Define internal buffer max size
#define BURSTBUFFERSIZE 256

//TRIPCOUNT identifiers
const unsigned int c_min = 1;
const unsigned int c_max = BURSTBUFFERSIZE;
const unsigned int c_chunk_sz = DATA_SIZE;

extern "C" {
void vadd(int *a, int size, int inc_value) {
    // Map pointer a to AXI4-master interface for global memory access
    #pragma HLS INTERFACE mode=m_axi port=a offset=slave bundle=gmem
    max_read_burst_length=256 max_write_burst_length=256
    // We also need to map a and return to a bundled axilite slave interface
    #pragma HLS INTERFACE mode=s_axilite port=a bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=size bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=inc_value bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=return bundle=control

    int burstbuffer[BURSTBUFFERSIZE];

    // Per iteration of this loop perform BURSTBUFFERSIZE vector addition
    for (int i = 0; i < size; i += BURSTBUFFERSIZE) {
    #pragma HLS LOOP_TRIPCOUNT min=c_min*c_min max=c_chunk_sz*c_chunk_sz/
    (c_max*c_max)
        int chunk_size = BURSTBUFFERSIZE;
        //boundary checks
        if ((i + BURSTBUFFERSIZE) > size)
            chunk_size = size - i;

        // Use a for loop to create a burst access to memory
        // memcpy is not recommended
        // memcpy(burstbuffer, &a[i], chunk_size * sizeof(int));
        for (i=0; i < chunk_size; i++) {
            burstbuffer[i] = a[i];
        }

        // Calculate and write results to global memory, the sequential write
        in a for loop can be
        // inferred as a memory burst access
```

```

    calc_write:
        for (int j = 0; j < chunk_size; j++) {
            #pragma HLS LOOP_TRIPCOUNT min=c_size_max max=c_chunk_sz
            #pragma HLS PIPELINE II=1
            burstbuffer[j] = burstbuffer[j] + inc_value;
            a[i + j] = burstbuffer[j];
        }
    }
}

```

突发之间的流水打拍

以下示例将推断长度为 N 的突发：

```

for(int x=0; x < k; ++x) {
    int off = f(x);
    for(int i = 0; i < N; ++i) {
        #pragma HLS PIPELINE II=1
        ... = gmem[off + i];
    }
}

```

但请注意，外层循环并不采用流水打拍。这意味着虽然突发内部存在流水打拍，但是突发之间不存在任何流水打拍。

为了做出补救，您可以将内层循环展开，并将外层循环流水打拍，使突发之间也采用流水打拍。以下示例仍将推断长度为 N 的突发，但现在突发之间同样采用流水打拍，因而吞吐量更高：

```

for(int x=0; x < k; ++x) {
    #pragma HLS PIPELINE II=N
    int off = f(x);
    for(int i = 0; i < N; ++i) {
        #pragma HLS UNROLL
        ... = gmem[off + i];
    }
}

```

从二维阵列访问行数据

以下是读写二维阵列的示例。Vitis HLS 会推断读取突发和写入突发，并发出以下消息：

```

INFO: [HLS 214-115] Burst read of length 256 and bit width 512 has been
inferred on port 'gmem' (./src/row_array_2d.cpp:43:5)
INFO: [HLS 214-115] Burst write of length 256 and bit width 512 has been
inferred on port 'gmem' (./src/row_array_2d.cpp:56:5)

```

请注意，在此示例内达成的位宽为 512。这比上述简单示例中达成的 32 位宽更高效。增大突发位宽是优化突发的另一种方法，如 [端口宽度自动调整](#) 中所述。

此示例的代码如下所示：

```

/***** BEGIN EXAMPLE *****/
// Parameters Description:
//     NUM_ROWS:           matrix height
//     WORD_PER_ROW:       number of words in a row
//     BLOCK_SIZE:         number of words in an array
#define NUM_ROWS    64
#define WORD_PER_ROW 64
#define BLOCK_SIZE (WORD_PER_ROW*NUM_ROWS)

```

```

// Default datatype is integer
typedef int DTYPE;
typedef hls::stream<DTYPE> my_data_fifo;

// Read data function: reads data from global memory
void read_data(DTYPE *inx, my_data_fifo &inFifo) {
read_loop_i:
    for (int i = 0; i < NUM_ROWS; ++i) {
        read_loop_jj:
            for (int jj = 0; jj < WORD_PER_ROW; ++jj) {
                #pragma HLS PIPELINE II=1
                inFifo << inx[WORD_PER_ROW * i + jj];
            }
        }
    }

// Write data function - writes results to global memory
void write_data(DTYPE *outx, my_data_fifo &outFifo) {
write_loop_i:
    for (int i = 0; i < NUM_ROWS; ++i) {
        write_loop_jj:
            for (int jj = 0; jj < WORD_PER_ROW; ++jj) {
                #pragma HLS PIPELINE II=1
                outFifo >> outx[WORD_PER_ROW * i + jj];
            }
        }
    }

// Compute function is pretty simple because this example is focused on
// efficient memory access pattern.
void compute(my_data_fifo &inFifo, my_data_fifo &outFifo, int alpha) {
compute_loop_i:
    for (int i = 0; i < NUM_ROWS; ++i) {
        compute_loop_jj:
            for (int jj = 0; jj < WORD_PER_ROW; ++jj) {
                #pragma HLS PIPELINE II=1
                DTYPE inTmp;
                inFifo >> inTmp;
                DTYPE outTmp = inTmp * alpha;
                outFifo << outTmp;
            }
        }
    }

extern "C" {
    void row_array_2d(DTYPE *inx, DTYPE *outx, int alpha) {
        // AXI master interface
        #pragma HLS INTERFACE mode=m_axi port = inx offset = slave bundle = gmem
        #pragma HLS INTERFACE mode=m_axi port = outx offset = slave bundle =
gmem
        // AXI slave interface
        #pragma HLS INTERFACE mode=s_axilite port = inx bundle = control
        #pragma HLS INTERFACE mode=s_axilite port = outx bundle = control
        #pragma HLS INTERFACE mode=s_axilite port = alpha bundle = control
        #pragma HLS INTERFACE mode=s_axilite port = return bundle = control

        my_data_fifo inFifo;
        // By default the FIFO depth is 2, user can change the depth by
using
        // #pragma HLS stream variable=inFifo depth=256
    }
}

```

```

        my_data_fifo outFifo;

        // Dataflow enables task level pipelining, allowing functions and
        loops to execute
        // concurrently. For more details please refer to UG902.
        #pragma HLS DATAFLOW
        // Read data from each row of 2D array
        read_data(inx, inFifo);
        // Do computation with the acquired data
        compute(inFifo, outFifo, alpha);
        // Write data to each row of 2D array
        write_data(outx, outFifo);
        return;
    }
}

```

总结

以可推断突发的方式来编写代码。确保满足所有前置条件。突发并不意味着一次性得到所有数据，意义在于将多个请求合并为单个请求，但数据将按顺序逐一到达。

理想突发长度为 16，但突发长度即使为 8 也足以应对。突发越大时延越长，较短的突发可采用流水打拍。请勿将突发与流水打拍相混淆，但请注意，多个突发相互间可采用流水打拍方式来执行。

如果突发为固定长度，那么您可展开用于推断突发的内层循环，而对外层循环执行流水打拍。这样将在保持突发长度不变的同时，在突发之间达成流水打拍，从而提升吞吐量。

要提升吞吐量，请集中精力将接口拓宽至 1024 位，而不是简单地加长突发。

相比于 AXI Interconnect，突发越大，优先级越高。在内核内部不执行动态仲裁。

您可将 2 个 `m_axi` 端口连接到同一个 DDR 存储器，以便对内核内部互斥的访问进行建模，但内核外部的 AXI Interconnect 将对完成请求进行仲裁。

绕过无序访问限制的变通方法之一是在块 RAM 中创建您自己的缓冲器、将突发存储在该缓冲器内，然后使用该缓冲器来执行无序访问。这是视频处理中常用的优化技巧，通常称为行缓冲器 (line buffer)。

或者，您还可考虑添加 CACHE 编译指示或指令，以改善 `m_axi` 接口的性能，如 [pragma HLS cache](#) 中所述。

请复查 [综合汇总](#) 报告的“突发优化”部分，了解有关设计中的突发优化的更多信息以及有关错失突发机会的信息。如果在设计中并未发生自动突发，则可使用 `hls::burst_maxi` 数据类型执行手动突发，如 [使用手动突发](#) 中所述。

使用手动突发

突发传输通过在全局存储器上读取或写入大量数据区块来改善内核 I/O 的吞吐量。突发越大，吞吐量越高，该指标计算方式如下： $((\text{传输的字节数}) * (\text{内核频率}) / (\text{时间}))$ 。最大内核接口带宽为 512 位，如果内核以 300 MHz 频率进行编译，那么理论上每个 DDR 可以实现 $= (80\text{-}95\% \text{ 的 DDR 效率}) * (512 * 300 \text{ MHz}) / 1 \text{ 秒} = \sim 17\text{-}19 \text{ GB/s}$ 。如前文所述，Vitis HLS 可以执行自动突发优化，以智能化方式从用户代码汇总循环/函数的存储器访问，并在单个突发请求内执行特定大小的读取/写入操作。但正如 [突发传输的前置条件和限制](#) 中所述，突发传输也有要求，这些要求有时可能负担过重或者难以满足。

在某些情况下，如果自动突发访问失败，那么有效的解决方案是重写代码或者使用手动突发。在此类情况下，如果您熟悉 AXI4 `m_axi` 协议，并且理解硬件传输事务建模，那么您可以使用 `hls::burst_maxi` 类来实现手动突发传输，如下所述。请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Interface/Memory/manual_burst](#)，以获取这些概念的示例。另一个解决方案可能是在 AXI4 接口中通过 [CACHE](#) 编译指示或指令来使用高速缓存存储器。

hls::burst_maxi 类

`hls::burst_maxi` 类可以提供一种机制，用于对 DDR 存储器执行读/写访问。这些方法将把类方法的使用行为转换为相应的 AXI4 协议，并在 AXI4 总线信号上发送和接收请求：AW、AR、WDATA、BVALID 和 RDATA。这些方法可以控制 HLS 调度器的突发行为。适配器可从调度器接收命令，并负责向 DDR 存储器发送数据。这些请求将遵循用户指定的 INTERFACE 编译指示选项，例如，`max_read_burst_length` 和 `max_write_burst_length`。类方法应仅在内核代码中使用，不得在测试平台中使用（类构造函数除外，如下所述）。

· 构造函数：

- `burst_maxi(const burst_maxi &B) : Ptr(B.Ptr) {}`
- `burst_maxi(T *p) : Ptr(p) {}`



重要提示！ HLS 设计和测试平台必须位于不同文件内，因为构造函数 `burst_maxi(T *p)` 仅在 C 语言仿真模型内可用。

· 读取方法：

- `void read_request(size_t offset, size_t len);`

此方法用于对 `m_axi` 适配器执行读取请求。如果 `m_axi` 适配器内部的读取请求队列未满，那么该函数就会立即返回，否则它会等待至空间变为可用为止。

- `offset`：指定存储器偏移，将从该指定偏移位置读取数据
- `len`：指定调度器突发长度。此突发长度将发送至适配器，随后，适配器即可将其转换为标准 AXI AMBA 协议

- `T read();`

此方法用于将数据从 `m_axi` 适配器传输至调度器 FIFO。如果数据不可用，`read()` 将执行阻塞。`read()` 方法应调用 `len` 次，具体次数在 `read_request()` 中指定。

· 写入方法：

- `void write_request(size_t offset, size_t len);`

此方法用于对 `m_axi` 适配器执行写入请求。如果 `m_axi` 适配器内部的写入请求队列未满，那么该函数就会立即返回。

- `offset`：指定存储器偏移，数据应写入该指定偏移位置
- `len`：指定调度器突发长度。此突发长度将发送至适配器，随后，适配器即可将其转换为标准 AXI AMBA 协议

- `void write(const T &val, ap_int<sizeof(T)> byteenable_mask = -1);`

此方法用于将数据从调度器的内部缓冲器传输到 `m_axi` 适配器。如果内部写入缓冲器已满，那么它会执行阻塞。`byteenable_mask` 用于在 WDATA 中启用字节。默认情况下，它将启用传输的所有字节。`write()` 方法应调用 `len` 次，具体次数在 `write_request()` 中指定。

- `void write_response();`

此方法会执行阻塞，直至从全局存储器返回所有写入响应为止。此方法的调用次数应与 `write_request()` 相同。

在 HLS 设计中使用手动突发

在 HLS 设计中，如果您发现自动突发传输未按期望方式工作，并且您无法按需对设计进行优化，那么您可以使用 `hls::burst_maxi` 对象来实现读取和写入传输事务。在此情况下，您将需要修改自己的代码，将原始指针实参替换为 `burst_maxi`（作为函数实参）。这些实参必须供 `burst_maxi` 类的 `read` 和 `write` 显式方法访问，如以下示例所示。

以下显示的是原始代码样本，它使用指针实参从全局存储器读取数据。

```
void dut(int *A) {
    for (int i = 0; i < 64; i++) {
        #pragma pipeline II=1
        ... = A[i]
    }
}
```

在以下修改后的代码中，该指针被替换为 `hls::burst_maxi<int>` 类对象和方法。在此示例中，HLS 调度器将 4 个请求（`len` 均为 16）从端口 A 放置到 `m_axi` 适配器上。该适配器将其存储在 FIFO 内部，只要 AW/AR 总线可用，它就会将请求发送到全局存储器。在 64 次循环迭代中，`read()` 命令会发出阻塞调用，调用将等待全局存储器返回的数据。数据可用后，HLS 调度器将从 `m_axi` 适配器 FIFO 读取此数据。

```
#include "hls_burst_maxi.h"
void dut(hls::burst_maxi<int> A) {
    // Issue 4 burst requests
    A.read_request(0, 16); // request 16 elements, starting from A[0]
    A.read_request(128, 16); // request 16 elements, starting from A[128]
    A.read_request(256, 16); // request 16 elements, starting from A[256]
    A.read_request(384, 16); // request 16 elements, starting from A[384]
    for (int i = 0; i < 64; i++) {
        #pragma pipeline II=1
        ... = A.read(); // Read the requested data
    }
}
```

在以下示例 2 中，HLS 调度器/内核会将 2 个请求从端口 A 放置到适配器上，第一个请求的 `len` 为 2，第二个请求的 `len` 为 1，总计 2 个写入请求。随后，它会发出对应的请求，因为总计突发长度为 3 条写入命令。该适配器将这些请求存储在 FIFO 内部，只要 AW 和 W 总线可用，它就会将请求和数据发送到全局存储器。最后，使用 2 条 `write_response` 命令等待 2 个 `write_requests` 的响应。

```
void trf(hls::burst_maxi<int> A) {
    A.write_request(0, 2);
    A.write(x); // write A[0]
    A.write_request(10, 1);
    A.write(x, 2); // write A[1] with byte enable 0010
    A.write(x); // write A[10]
    A.write_response(); // response of write_request(0, 2)
    A.write_response(); // response of write_request(10, 1)
}
```

在 C 语言仿真中使用手动突发

手动突发支持在不同数据流进程中执行 `read_request()` 和 `read()`，以启用 `read_request()` 和 `read()` 的并行处理（请参阅以下示例）。`write_request()`、`write()` 和 `write_response()` 也同样受支持。

```

void foo(hls::burst_maxi<int> a) {
    for (int i = 0; i < 10; i++) {
        #pragma HLS pipeline
        a.read_request(i * 16, 10);
    }
}

void bar(hls::burst_maxi<int> a, int *b) {
    for (int i = 0; i < 100; i++) {
        #pragma HLS pipeline
        b[i] = a.read();
    }
}

void test(hls::burst_maxi<int> a, int b[1000]) {
    #pragma HLS interface m_axi port=a depth=1000
    #pragma HLS dataflow
    foo(a);
    bar(a, b);
}

```

您可将常规阵列传递到顶层函数，构造函数将把此阵列自动转换为 `hls::burst_maxi`。



重要提示！ HLS 设计和测试平台必须位于不同文件内，因为 `burst_maxi(T *p)` 构造函数只能在 C 语言仿真模型内使用。

```

#include "hls_burst_maxi.h"
void dut(hls::burst_maxi<int> A);

int main() {
    int Array[1000];
    dut(Array);
    .....
}

```

使用手动突发进行性能优化

Vitis HLS 将突发行行为划分为两种类型：流水线突发和顺序突发。

- 流水线突发：流水线突发通过在单个请求内读取或写入最大量的数来改善吞吐量。如果 `read_request`、`write_request` 和 `write_response` 调用位于循环外部，那么编译器就会推断流水线突发，如以下代码示例所示。在以下示例中，大小 (size) 是从测试平台发送的变量。

```

9  int buf[8192];
10 in.read_request(0, size);
11 for (int i = 0; i < size; i++) {
12     #pragma HLS PIPELINE II=1
13     buf[i] = in.read();
14     out.write_request(0, size*NT);
17     for (int i = 0; i < NT; i++) {
19         for (int j = 0; j < size; j++) {
20             #pragma HLS PIPELINE II=1

```



```
21         int a = buf[j];  
22         out.write(a);  
23     }  
24 }  
25 out.write_response();
```

图 66：综合结果

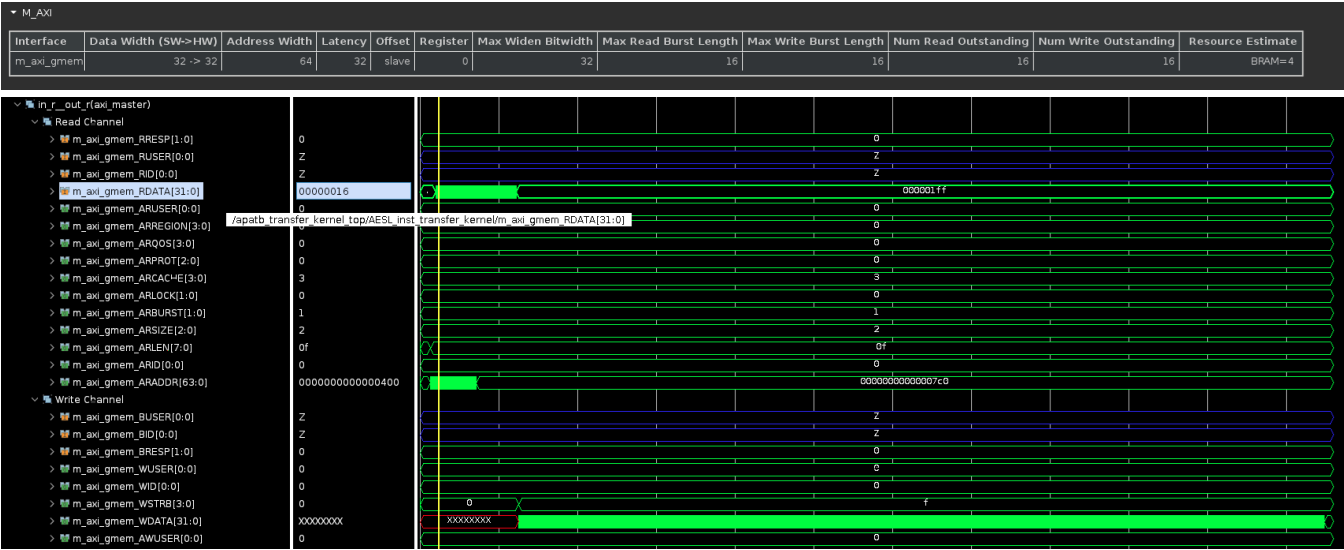
M_AXI Burst Information

Manual Burst (burst_maxi)

HW Interface	Direction	Length	Width	Location
m_axi_gmem	read	variable	32	example.cpp:10:12
m_axi_gmem	write	variable	32	example.cpp:16:7

如上图所示，该工具从用户代码推断突发，并在编译时提及长度（作为变量）。

图 67：性能优势



在运行时期间，HLS 编译器会发送 `length = size` 突发请求，适配器按用户指定的 `burst_length` 编译指示选项对该请求进行分区。在此情况下，默认突发长度设置为 16，并在 `ARlen` 通道和 `AWlen` 通道内使用。由于传输期间没有气泡，因此读/写通道已实现最大吞吐量。

图 68：协同仿真结果

Modules & Loops	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency
transfer_kernel				5735	5735	5735
transfer_kernel_Pipeline_VITIS_LOOP_11_1				517	517	517
transfer_kernel_Pipeline_VITIS_LOOP_18_2_VITIS_LOOP_19_3				5122	5122	5122

· 顺序突发：

此突发为数据大小较小的顺序突发，其中读取请求、写入请求和写入响应均位于循环主体内部，如以下代码片段中所示。顺序突发的缺点在于后续请求 (i+1) 依赖于先前请求 (i) 才能完成，因为它会等待读取请求、写入请求和写入响应完成，这将导致请求之间出现间隔。顺序突发效率不及流水线突发，因为它多次读取或写入少量数据，以补偿循环边界。虽然这将限制吞吐量改善，但是顺序突发仍好于无突发。

```
void transfer_kernel(hls::burst_maxi<int> in, hls::burst_maxi<int> out,
const int size )
{
    #pragma HLS INTERFACE m_axi port=in depth=512 latency=32 offset=slave
    #pragma HLS INTERFACE m_axi port=out depth=5120 offset=slave latency=32

    int buf[8192];

    for (int i = 0; i < size; i++) {
        in.read_request(i, 1);
        #pragma HLS PIPELINE II=1
        buf[i] = in.read();
    }

    for (int i = 0; i < NT; i++) {
        for (int j = 0; j < size; j++) {
            out.write_request(j, 1);
#pragma HLS PIPELINE II=1
            int a = buf[j];
            out.write(a);
            out.write_response();
        }
    }
}
```

图 69：综合结果

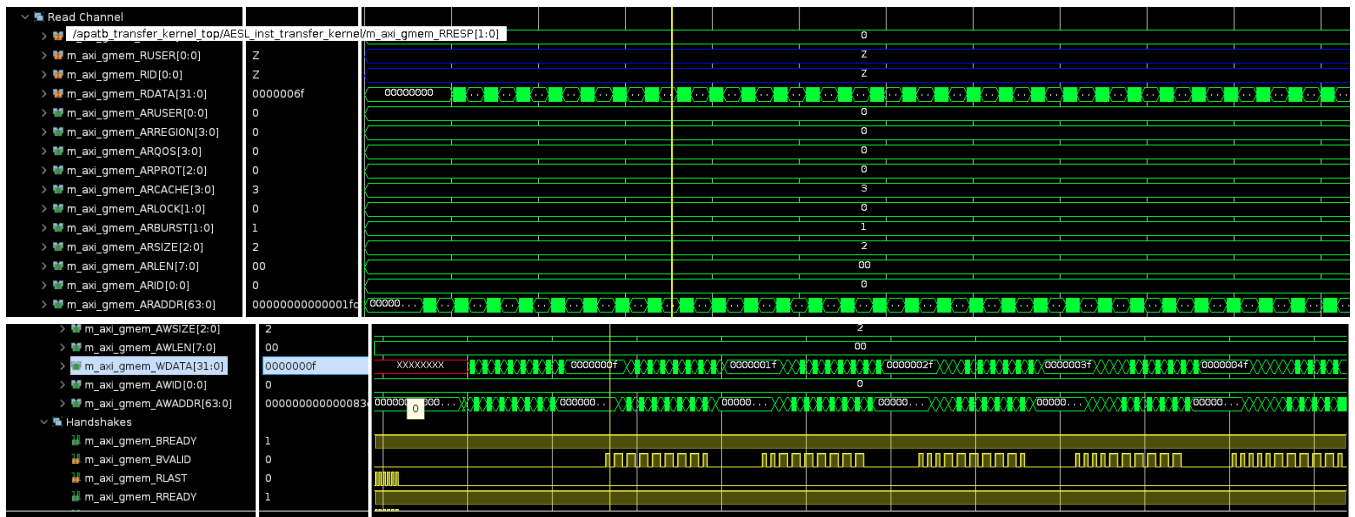
▼ M_AXI Burst Information

▼ Manual Burst (burst_maxi)

HW Interface	Direction	Length	Width	Location
m_axi_gmem	read	1	32	example.cpp:13:8
m_axi_gmem	write	1	32	example.cpp:23:15

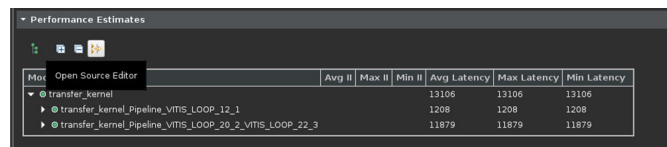
如上报告样本所示，该工具实现了长度为 1 的突发。

图 70：性能影响



读/写循环 R/WDATA 通道的间隔与读/写时延相等，如 **AXI4 主接口** 中所述。对于读取通道，循环会等待全局存储器返回所有读取数据。对于写入通道，最内层循环会等待全局存储器返回响应 (BVALID)。这将导致性能降级。协同仿真结果同样显示，对于此突发语义，性能减半。

图 71：性能估算



功能特性和局限性

- 如果 `m_axi` 元素为结构体：
 - 此结构体将打包到宽整型 (wide int) 内。不允许对此结构体执行分解。
 - 结构体大小必须为 2 的幂，不应超过 1024 位或 `config_interface -m_axi_max_bitwidth` 命令指定的最大宽度。
- 不允许 `burst_maxi` 端口的 `ARRAY_PARTITION` 和 `ARRAY_RESHAPE`。
- 您可将 `INTERFACE` 编译指示或指令应用于 `hls::burst_maxi`，以定义 `m_axi` 接口。如果 `burst_maxi` 端口与其他端口捆绑，则此捆绑中的所有端口都必须设置为 `hls::burst_maxi` 并且必须具有相同的元素类型。

```
void dut(hls::burst_maxi<int> A, hls::burst_maxi<int> B, int *C,
hls::burst_maxi<short> D) {
    #pragma HLS interface m_axi port=A offset=slave bundle=gmem // OK
    #pragma HLS interface m_axi port=B offset=slave bundle=gmem // OK
    #pragma HLS interface m_axi port=C offset=slave bundle=gmem // Bad. C
    must also be hls::burst_maxi type, because it shares the same bundle
    'gmem' with A and B
    #pragma HLS interface m_axi port=D offset=slave bundle=gmem // Bad. D
    should have 'int' element type, because it shares the same bundle
    'gmem' with A and B
}
```

4. 您可以使用 INTERFACE 编译指示或指令来指定 `num_read_outstanding` 和 `num_write_outstanding`，并使用 `max_read_burst_length` 和 `max_write_burst_length` 来定义 `m_axi` 适配器的内部缓冲器大小。

```
void dut(hls::burst_maxi<int> A) {
    #pragma HLS interface m_axi port=A num_read_outstanding=32
    num_write_outstanding=32 max_read_burst_length=16
    max_write_burst_length=16
}
```

5. 不支持 INTERFACE 编译指示或指令 `max_widen_bitwidth`，因为 HLS 将不会更改 `hls::burst_maxi` 端口的位宽。
6. 您必须先发出 `read_request`，再发出 `read`，或者先发出 `write_request`，再发出 `write`：

```
void dut(hls::burst_maxi<int> A) {
    ... = A.read(); // Bad because read() before read_request(). You can
    catch this error in C-sim.
    A.read_request(0, 1);
}
```

7. 如果读取组的地址和生存期 (`read_request() > read()`) 与写入组 (`write_request() > write() > write_response()`) 重叠，那么该工具将无法保证访问顺序。C 语言仿真将报告错误。

```
void dut(hls::burst_maxi<int> A) {
    A.write_request(0, 1);
    A.write(x);
    A.read_request(0, 1);
    ... = A.read(); // What value is read? It is undefined. It could be
    original A[0] or updated A[0].
    A.write_response();
}

void dut(hls::burst_maxi<int> A) {
    A.write_request(0, 1);
    A.write(x);
    A.write_response();
    A.read_request(0, 1);
    ... = A.read(); // this will read the updated A[0].
}
```

8. 如有多个 `hls::burst_maxi` 端口捆绑到同一个 `m_axi` 适配器，并且其传输事务生存期重叠，那么行为将不可预测。

```
void dut(hls::burst_maxi<int> A, hls::burst_maxi<int> B) {
    #pragma HLS INTERFACE m_axi port=A bundle=gmem depth = 10
    #pragma HLS INTERFACE m_axi port=B bundle=gmem depth = 10
    A.read_request(0, 10);
    B.read_request(0, 10);

    for (int i = 0; i < 10; i++) {
        #pragma HLS pipeline II=1
        ..... = A.read(); // get value of A[0], A[2], A[4] ...
        ..... = B.read(); // get value of A[1], A[3], A[5] ...
    }
}
```

9. 不支持将读取/写入请求与读取/写入操作包含在不同数据流进程内。数据流检查程序将报告错误：`multiple writes in different dataflow processes are not allowed`。

例如：

```
void transfer(hls::burst_maxi<int> A) {
#pragma HLS dataflow
    IssueRequests(A); // issue multiple write_request() of A
    Write(A); // multiple writes to A
    GetResponse(A); // write_response() of A
}
```

潜在陷阱

以下是您在实现手动突发方法时必须留意的部分问题：

- 死锁：错误使用手动突发可能导致死锁。

在 `read()` 命令前发出过多 `read_requests` 将导致死锁，因为 `read_request` 循环将把请求推入读取请求 FIFO，仅在全局存储器读取操作完成后才会清空此 FIFO。`read()` 命令的工作是从适配器 FIFO 读取数据，并将请求标记为已完成，然后，将从 FIFO 迅速发出 `read_request`，随后即可将新请求推送到此 FIFO。

```
//reads/writes. will deadlock if N is larger
for (i = 0; i < N; i++)
{
    A.read_request(i * 128, 16);}
for (i = 0; i < 16 * N; i++) { ... = A.read();}

for (int i = 0; i < N; i++) {
    p.write_request(i * 128, 16);
}

for (int i = 0; i < N * 16; i++) {
    p.write(i);
}

for (int i = 0; i < N; i++) {
    p.write_response();
}
```

在以上示例中，如果 `N` 大于 `read_request`，并且读取 FIFO 将满，因为它趋向于 `N/2`。读取请求循环将无法完成，读取命令循环将无法启动，从而导致死锁。

注释：对于 `write_request()` 和 `write()` 命令也同样如此。

- AXI 协议违例：写入请求与写入响应数量应相等。请求与响应数量不相等将导致发生 AXI 协议违例

AXI 性能案例研究

简介

案例研究的目标是通过展示逐步优化，使用 HLS 指标来提升读取或写入循环或函数的吞吐量。这些优化将通过执行从全局存储器到内核的高效数据传输来改善系统的内核时间和吞吐量。以下 `transfer_kernel` 示例会对变量大小和 `NUM_ITERATIONS` 执行简单的 DDR 读/写。



提示：主机代码（此处未显示）仅传输数据，并按有序队列方式对内核进行排队。

```

1 #include "config.h"
2 #include "assert.h"
3 extern "C" {
4     void transfer_kernel(wd* in, wd* out, const int size, const int iter )
5 {
6     ...
7     wd buf[256];
8     int off = (size/16);
9     read_loop: for (int i = 0; i < off; i++)
10    {
11        buf[i] = in[i];
12    }
13
14    write_loop: L1: for (int i = 0; i < iter; i++) {
15        L2: for (int j = 0; j < off; j++) {
16            #pragma HLS PIPELINE II=1
17            out[j+off*i] = buf[j];
18        }
19    }
20    ...
21 }
22 }

```

此案例研究分为 4 个步骤：

1. 设置内核运行时间基线，其中端口宽度设为 512 位
2. 通过更改时延参数来提升性能
3. 改善写入循环的自动突发推断。
4. 使用多个端口和未完成写入操作数无法再继续改善

步骤 1：设置内核基线，端口位宽为 512 位

使用默认设置来为内核时间设定基线。运行期间，自动突发会针对读取和写入循环执行以下推断：

- 由于该工具可以预测连续存储器访问模式，故而读取循环会达成流水线突发。因此流水打拍请求会读取 DDR 变量大小。
- 写入外层循环 L1 将达成顺序突发，因为编译器会基于所有组合进行迭代，并发现由于在编译时大小未知，因此在 L2 循环开始前，它会在 L1 循环内插入一项 if 条件。与此同时，最内层循环 L2 会达成流水线突发。L2 循环会发出写入变量大小请求，同时，L1 会等待 L2 循环的所有数据从 DDR 返回后再开始下一次 L1 迭代。

构建并运行应用后，可使用 Vitis 分析器工具来查看构建进程或者运行汇总生成的报告，以便评估性能。请复查 Vitis HLS 的“Synthesis Report”（综合报告）中提供的“Burst Summary”（突发汇总）。它会确认读取循环和写入循环的突发是否成功。

图 72：综合报告 - 突发汇总

BURST SUMMARY		
HW Interface	Loop	Message
m_axi_gmem	VITIS_LOOP_11_1	Multiple burst reads of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based on
m_axi_gmem	VITIS_LOOP_17_3	Multiple burst writes of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based on

BURSTS AND WIDENING MISSED					
HW Interface	Variable	Loop	Problem	Resolution	Location
m_axi_gmem	out	VITIS_LOOP_16_2	Access call is in the conditional branch	214-232	krnl_tf.cpp:16:22
m_axi_gmem	out		Could not widen since the size of type 'S12' is greater than or equal to the max_widen_bitwidth threshold of '64'.		krnl_tf.cpp:17:27
m_axi_gmem	in		Could not widen since the size of type 'S12' is greater than or equal to the max_widen_bitwidth threshold of '64'.		krnl_tf.cpp:11:26

在 Vitis 分析器中，“Profile Summary”（剖析汇总）报告和“Timeline Trace”（时间线轨迹）报告同样是很实用的工具，可用于分析经 FPGA 加速的应用的性能。在“Profile Summary”（剖析汇总）中，“Kernels & Compute Unit: Kernel Execution”（内核与计算单元：内核执行）会报告基线构建中 `transfer_kernel` 所需的总时间。

图 73：剖析汇总 - 内核执行

Kernels & Compute Units						
Kernel Execution						
Kernel	Enqueues	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)	
 transfer_kernel	300	1683.150	3.766	5.610	7.947	

步骤 2：改善性能时延

Vitis HLS 使用的默认时延为 64 个内核周期，在某些情况下可能过高。时延取决于系统特性。对于本示例，时延从默认值减少到 21 个内核周期。使用 `INTERFACE` 编译指示或指令可更改此代码以指定时延，如下示例所示：

```
1 #include "config.h"
2 #include "assert.h"
3 extern "C" {
4     void transfer_kernel(wd* in,wd* out, const int size, const int iter )
5 {
6     #pragma HLS INTERFACE m_axi port=in0_index offset=slave latency=21
7     #pragma HLS INTERFACE m_axi port=out offset=slave latency=21
8
9     ...
10 }
```

构建和运行应用，并使用 Vitis 分析器来复查构建进程或运行汇总所生成的报告。在 Vitis HLS 中复查“Synthesis Report”（综合报告），并检查“HW Interface”（硬件接口）表，查看是否已应用指定的时延。

图 74：综合报告 - 硬件接口

M_AXI									
Interface	Data Width (SW->HW)	Address Width	Latency	Offset	Register	Max Widen Bitwidth	Max Read Burst Length	Max Write Burst Length	Num Read Outstanding
m_axi_gmem	512 -> 512	64	21	slave	0	512	16	16	

复查“Burst Summary”（突发汇总）以检查该进程是否成功。

图 75：综合报告 - 突发汇总 2

BURST SUMMARY			
HW Interface	Loop	Message	
m_axi_gmem	VITIS_LOOP_11_1	Multiple burst reads of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based on	
m_axi_gmem	VITIS_LOOP_17_3	Multiple burst writes of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based on	

BURSTS AND WIDENING MISSED					
HW Interface	Variable	Loop	Problem	Resolution	Location
m_axi_gmem	out	VITIS_LOOP_16_2	Access call is in the conditional branch	214-232	krnl_tf.cpp:16:22
m_axi_gmem	out		Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.		krnl_tf.cpp:17:27
m_axi_gmem	in		Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.		krnl_tf.cpp:11:26

检查“Profile Summary”报告中的“Kernel Execution”（内核执行），留意因设置接口时延而达成的性能提升。

图 76：剖析汇总 - 内核执行 2

Kernels & Compute Units					
Kernel Execution					
Kernel	Enqueues	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)
 transfer_kernel	300	1656.180	3.445	5.521	8.002

步骤 3：改善写入循环的自动突发推断

编译器对自动突发推断持悲观态度，因为在编译时大小和循环次数均未知。您可修改代码以帮助编译器推断流水节拍突发，如下所示。

```

1 #include "config.h"
2 #include "assert.h"
3 extern "C" {
4     void transfer_kernel(wd* in, wd* out, const int size, const int
iter ) {
5         #pragma HLS INTERFACE m_axi port=in offset=slave latency=21
6         #pragma HLS INTERFACE m_axi port=out offset=slave latency=21
7
8         int k=0;
9         wd buf[256];
10        int off = (size/16);
11
12        read_loop: for (int i = 0; i < off; i++)
13        {
14            buf[i] = in[i];
15        }
16
17        write_loop: for (int j = 0; j < off*iter; j++) {
18            #pragma HLS PIPELINE II=1
19            out[k++] = buf[j%off];
20        }
21    }
22 }
```

构建和运行应用，并使用 Vitis 分析器来复查构建进程或运行汇总所生成的报告。“Synthesis Report”会确认提供给编译器的突发提示已修复了写入循环的顺序突发。“Burst and Widening Missed”（错失突发与拓宽）消息与将端口拓宽至 512 位有关。由于此示例的端口宽度已设为 512 位，因此可忽略此消息。如果在您的代码中宽度并非 512 位，那么您可能需要集中精力解决这些消息中所示问题。

图 77：综合报告 - 突发汇总 3

BURST SUMMARY		
HW Interface	Loop	Message
m_axi_gmem	VITIS_LOOP_12_1	Multiple burst reads of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation basec
m_axi_gmem	VITIS_LOOP_18_2	Multiple burst writes of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation basec

BURSTS AND WIDENING MISSED			
HW Interface	Variable	Problem	Location
m_axi_gmem	out	Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.	krnl_tf.cpp:18:27
m_axi_gmem	in	Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.	krnl_tf.cpp:12:26

检验“Profile Summary”报告中的“Kernel Execution”，并留意因步骤 2 的时延更改而达成的性能提升以及当前步骤中写入循环的流水线突发。

图 78：剖析汇总 - 内核执行 3

Kernels & Compute Units						
Kernel Execution						
Kernel	Enqueues	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)	
 transfer_kernel	300	1206.800	1.748	4.023	6.720	

总结

Vitis HLS 接口指标无法进一步改善性能。此案例研究示例并不含并发读取或写入，因此以多个端口为目标对此案例并无帮助。在此示例中，该工具已为最大吞吐量实现了流水线突发，因此未完成读取和写入数量同样可以忽略。内核时间方面无法确认进一步的性能改善。

正如此案例研究所示，实现有效的加载存储函数取决于 HLS 接口指标，包括端口宽度、突发访问、时延、多个端口以及未完成的读取和写入数量。AMD 建议遵循如下准则来改善系统性能：

- 端口宽度：最大程度为接口增加端口宽度，例如，对于每个 AXI 端口的位宽，使用 `hls::vector` 或 `ap_(u)int<512>` 作为端口的数据类型。
- 分析并发存储器读取/写入，并采用单一专用/独立端口来执行并发访问。
- 流水线突发：AXI 时延参数不影响流水打拍突发，建议用户编写代码以达成流水打拍突发，这样可以显著提升性能。
- 顺序突发：AXI 时延参数对顺序突发具有显著影响，降低工具的默认时延的时延数将能改善性能。
- 未完成操作数：在大多数情况下，当突发长度 ≥ 16 时，默认未完成数应已足够。对于大小小于 16 的突发，AMD 建议将未完成操作数的默认值 16 加倍。
- 数据重排序：始终建议达成流水打拍突发，但有时候，由于存储器访问模式所限，编译器只能达成顺序突发。为了提升性能，开发者还可思考如何通过不同方法将数据存储到存储器中。例如，通过以列为主的方式来访问 DRAM 中的数据效率非常低。相比在内核中实现专用数据移动器，在软件中对数据进行转置并改用以行为主的方式来存储将显著简化硬件访问模式，因此这种方式更有效。

管理面积资源和硬件资源

综合期间，Vitis HLS 会执行以下基本任务：

- 将 C 和 C++ 源代码细化到内部数据库中，该数据库包含 C 语言代码运算符，如加法、乘法、阵列读取和写入。
- 将运算符映射到硬件中的实现上。

所谓实现，即用于创建设计的特定硬件组件（例如，加法器、乘法器、经流水打拍的乘法器和块 RAM）。

您可使用命令、编译指示和指令来控制其中每个步骤，从而对硬件实现进行极为详细的控制。

限制运算符数量

显式限制运算符的数量以减小某些情况下所需的面积：Vitis HLS 的默认操作是首先最大限度提升性能。限制设计中的运算符数量是一项减小设计面积的实用技巧：它通过强制共享运算来减小面积。但是，这可能导致性能下降。

ALLOCATION 指令允许您限制设计中使用的运算符数量。例如，名为 `foo` 的设计包含 317 次乘法，但 FPGA 仅有 256 项乘法器资源（DSP 宏单元）。以下所示 ALLOCATION 编译指示可指令 Vitis HLS 创建含最多 256 个乘法 (`mul`) 运算符的设计：

```
dout_t array_arith (dio_t d[317]) {
    static int acc;
    int i;
    #pragma HLS ALLOCATION instances=fmul limit=256 operation

    for (i=0;i<317;i++) {
    #pragma HLS UNROLL
        acc += acc * d[i];
    }
    rerun acc;
}
```

注释：如果指定的 ALLOCATION 限制超出所需数量，Vitis HLS 会尝试使用此项限制所指定的资源数量，从而导致减少共享量。

您可使用 `type` 选项来指定 ALLOCATION 指令是否限制运算、实现和函数数量。下表列出了可使用 ALLOCATION 指令控制的所有运算。

注释：下列运算均受 ALLOCATION 编译指示或指令支持。BIND_OP 编译指示或指令可支持该命令语法中描述的小部分运算符。

表 22：Vitis HLS 运算符

运算符	描述
add	整数加法
ashr	算术右移
dadd	双精度浮点加法
dcmp	双精度浮点比较
ddiv	双精度浮点除法
dmul	双精度浮点乘法
drecip	双精度浮点倒数
drem	双精度浮点余数
drsqrt	双精度浮点平方根倒数
dsub	双精度浮点减法
dsqrt	双精度浮点平方根
fadd	单精度浮点加法
fcmp	单精度浮点比较
fdiv	单精度浮点除法

表 22: Vitis HLS 运算符 (续)

运算符	描述
fmul	单精度浮点乘法
frecip	单精度浮点倒数
frem	单精度浮点余数
frsqr	单精度浮点平方根倒数
fsub	单精度浮点减法
fsqrt	单精度浮点平方根
icmp	整数比较
lshr	逻辑右移
mul	乘法
sdiv	有符号的除法器
shl	左移
srem	有符号取余
sub	减法
udiv	无符号除法
urem	无符号取余

控制硬件实现

执行综合时，Vitis HLS 使用由时钟指定的时序约束、由目标器件指定的时延搭配您指定的任意指令来判定哪些硬件实现将用于代码中的各种运算符。例如，要实现乘法器运算，Vitis HLS 可以使用组合乘法器或者使用流水线乘法器。

综合期间映射到运算符的实现可通过指定 ALLOCATION 编译指示或指令来加以限制，与运算符的方式相同。您无需限制乘法运算总数，而可改为选择限制组合乘法器的数量以强制使用流水打拍乘法器来执行所有剩余乘法（或反之亦然）。

BIND_OP 或 BIND_STORAGE 编译指示或指令用于显式指定哪些实现要用于特定运算或存储类型。以下命令会告知 Vitis HLS 使用二阶流水打拍乘法器，并为变量 c 使用互连结构逻辑。由 Vitis HLS 判定用于变量 d 的实现。

```
int foo (int a, int b) {
    int c, d;
    #pragma HLS BIND_OP variable=c op=mul impl=fabric latency=2
    c = a*b;
    d = a*c;

    return d;
}
```

在以下示例中，BIND_OP 编译指示指定使用 dsp 实现来实现变量 temp 的加法运算。这样可确保在最终设计中使用 DSP 模块原语来实现运算。默认情况下，加法运算是使用 LUT 来实现的。

```
void apint_arith(dinA_t inA, dinB_t inB,
                dout1_t *out1
                ) {

    dout2_t temp;
    #pragma HLS BIND_OP variable=temp op=add impl=dsp
```

```
temp = inB + inA;
*out1 = temp;
}
```

请参阅 BIND_OP 或 BIND_STORAGE 编译指示或指令来获取有关可供分配到运算或存储类型的实现的详细信息。

在以下示例中，BIND_OP 编译指示指定使用 3 阶流水打拍乘法器来实现 out1 的乘法。

```
void foo(...) {
#pragma HLS BIND_OP variable=out1 op=mul latency=3

// Basic arithmetic operations
*out1 = inA * inB;
*out2 = inB + inA;
*out3 = inC / inA;
*out4 = inD % inA;
}
```

如果赋值指定多个相同的运算符，则必须修改此代码以确保针对要控制的每个运算符都存在单一变量。例如，在以下代码中，如果首个乘法 (inA * inB) 要以流水打拍乘法器来实现：

```
*out1 = inA * inB * inC;
```

那么应使用 Result_tmp 变量上指定的编译指示来将代码更改为：

```
#pragma HLS BIND_OP variable=Result_tmp op=mul latency=3
Result_tmp = inA * inB;
*out1 = Result_tmp * inC;
```

DSP 多重运算匹配

HLS 编译器会尝试识别加法-乘法、乘法-加法和 AMA 运算（以及减法-乘法、乘法-减法运算），使其分配与物理器件上 DSP 的相匹配。HLS 编译器会使用以下进程将运算与 DSP 匹配：

- 内联函数（按需）
- 提取常用子表达式
- 匹配无扇出 add->mul、mul->add 和 add->mul->add（其中，add 也可以是 sub）代码片段，这些代码片段满足 DSP 的位宽限制
- 调度
- 按 BIND_OP 编译指示的指引，选择 DSP 或 LUT 中的实现

HLS 编译器将运算或表达式与 DSP 实现相匹配的能力取决于：

1. 数据类型和位宽
2. 编码样式和表达式
3. BIND_OP 和其他编译指示的使用

数据类型和位宽

DSP 匹配受限于目标器件上 DSP48 和 DSP58 的设计限制。请参阅《UltraScale 架构 DSP Slice》(UG579)，了解针对 DSP48 的具体限制，并请参阅《Versal 自适应 SoC DSP 引擎架构手册》(AM004)，了解针对 DSP58 的具体限制。

例如，在 Versal 器件中，DSP58 的端口宽度如下：

- B 24 位
- A 34 位
- D 27 位
- C 58 位
- P 58 位

编码样式

代码的编写样式至关重要：

- 位宽必须足够小以适应 DSP，包括中间结果
- 不允许扇出，即使因常见的子表达式消除操作而导致的扇出也不允许。例如，由于抽取子表达式 $a*b$ ，导致以下代码与 DSP 分配不匹配：

```
f1 = a*b+c;
f2= a*b+d;
```



提示：为避免这一问题，请使用非内联函数，这些函数应精确包含要匹配的表达式。

- 含指定时延的非内联函数（例，时延为 2 的 MULADD）无法与 II=1 的累加器相匹配。原因是乘法器输入和加法器输入的就绪时间必须比产生累加输出早 2 个周期。在此情况下，MULADD 或 MAC 函数（如有）必须内联。

BIND_OP 和其他编译指示

对于 DSP 匹配，部分编译指示或指令可阻止在 DSP 中实现代码匹配，而其他编译指示或指令则可能有助于此。在以下代码示例中，**BIND_OP** 编译指示实际上阻止将 MULADD 表达式分配给 DSP。通过分配 `mul` 运算，它会阻止 HLS 编译器识别和分配 MULADD 表达式。在此情况下移除该编译指示将允许编译器完成分配。

```
static const int NFRAC = 14;
typedef ap_fixed<3+NFRAC, 3, AP_TRN, AP_WRAP> DATA_T;
...
ACC_T MAC( DATA_T din0, COEF_T coef, ACC_T acc ) {
    acc = din0 * coef + acc;
    #pragma HLS BIND_OP variable=acc op=mul impl=dsp latency=2
    return acc;
}
void process ( DATA_T din, DATA_T* dout0) {
    #pragma HLS pipeline II=1
    ...
    LOOP_MAC: for (int i=0; i<l_COEFF; i++) {
        acc = MAC (sr[tdl_index], coeff[i], acc);
        tdl_index = tdl_index-1;
    }
}
```

PIPELINE 编译指示会导致循环展开，并执行表达式平衡，这同样会阻止 DSP 匹配。在此情况下，添加 EXPRESSION_BALANCE OFF 以禁用表达式平衡将有助于 HLS 编译器识别用于 DSP 匹配的多重运算表达式。以下重做后的代码示例将改善 DSP 匹配。

```
static const int NFRAC = 14;
typedef ap_fixed<3+NFRAC, 3, AP_TRN, AP_WRAP> DATA_T;
...
ACC_T MAC( DATA_T din0, COEF_T coef, ACC_T acc ) {
    acc = din0 * coef + acc;
    return acc;
}
void process ( DATA_T din, DATA_T* dout0 ) {
    #pragma HLS pipeline II=1
    #pragma HLS expression_balance off
    ...
    LOOP_MAC: for (int i=0; i<l_COEFF; i++) {
        acc = MAC (sr[tdl_index], coeff[i], acc);
        tdl_index = tdl_index-1;
    }
}
```

控制运算符流水打拍

HLS 编译器会自动判定用于内部运算的流水打拍级别。您可将 BIND_OP 或 BIND_STORAGE 编译指示与 -latency 选项搭配使用，以显式指定流水线阶段的数量，并覆盖该工具判定的数量。

RTL 综合可使用多个额外流水线寄存器来帮助改善布局布线后可能导致的时序问题。在运算输出中添加寄存器通常有助于改善输出数据路径中的时序。在运算输入中添加寄存器通常有助于改善输入数据路径和来自 FSM 的控制逻辑中的时序。

您可使用 [运算符配置](#) 命令对设计中特定运算的具有相同流水线深度的所有实例进行流水打拍。

展开 C++ 类中的循环



重要提示！ 在 C++ 类中使用循环时，请确保循环感应变量的不属于该类的成员，因为这会导致展开此循环。

在此示例中，循环感应变量的 `k` 是类 `loop_class` 的成员。

```
template <typename T0, typename T1, typename T2, typename T3, int N>
class loop_class {
private:
    pe_mac<T0, T1, T2> mac;
public:
    T0 areg;
    T0 breg;
    T2 mreg;
    T1 preg;
    T0 shift[N];
    int k; // Class Member
    T0 shift_output;
    void exec(T1 *pcout, T0 *dataOut, T1 pcin, T3 coeff, T0 data, int col)
    {
        Function_label0;;
        #pragma HLS inline off
        SRL:for (k = N-1; k >= 0; --k) {
```

```
#pragma HLS unroll // Loop will fail UNROLL
if (k > 0)
  shift[k] = shift[k-1];
else
  shift[k] = data;
}

*dataOut = shift_output;
shift_output = shift[N-1];
}

*pcout = mac.exec1(shift[4*col], coeff, pcin);
};
```

为了使 Vitis HLS 能够按 UNROLL 编译指示指令指定的方式展开此循环，应重写此代码，从类成员中移除 `k`，使其成为 `exec` 函数的本地类成员。

控制驱动的任务级并行度的限制



提示：控制驱动的 TLP 要求在代码相应位置指定 [DATAFLOW](#) 编译指示或指令。

控制驱动的 TLP 模型可对任务（函数和循环）之间的数据流以及按理想方式流水打拍的函数和循环进行优化，从而最大限度提升性能。这些任务无需逐一链接，但数据传输方式存在某些限制。以下行为可能阻止或限制本工具可在数据流模型内执行的重叠：

- 在数据流区域中间读取函数输入或写入函数输出
- 单一生产者使用者违例
- 任务的有条件执行
- 含多个退出条件的循环



重要提示！如果存在上述任一编码样式，HLS 编译器就会发出一条描述状况的消息。

读取输入/写入输出

读取函数输入应在数据流区域起始位置执行，写入输出则应在数据流区域结束位置执行。读取/写入函数端口可能导致各进程按顺序执行，而非按重叠方式执行，从而对性能产生不利影响。

单一生产者使用者违例

为便于本工具使用数据流模型，任务间传递的所有元素都必须遵循单一生产者使用者模型。每个变量都必须从单一任务驱动，并且仅限供单一任务使用。在以下代码示例中，`temp1` 将扇出并供 `Loop2` 和 `Loop3` 使用。这违背了单一生产者使用者模型的要求。

```
void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {
  int temp1[N];

  Loop1: for(int i = 0; i < N; i++) {
    temp1[i] = data_in[i] * scale;
  }
  Loop2: for(int j = 0; j < N; j++) {
```

```

    data_out1[j] = temp1[j] * 123;
}
Loop3: for(int k = 0; k < N; k++) {
    data_out2[k] = temp1[k] * 456;
}
}

```

此代码的修改后版本使用 `Split` 函数来创建单一生产者使用者设计。以下代码块示例显示了通过 `Split` 对带有函数的数据流执行拆分的方式。现在，数据在全部 4 个任务之间流动，并且本工具可使用数据流模型。

```

void Split (in[N], out1[N], out2[N]) {
// Duplicated data
L1:for(int i=1;i<N;i++) {
    out1[i] = in[i];
    out2[i] = in[i];
}
}
void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {

    int temp1[N], temp2[N], temp3[N];
    Loop1: for(int i = 0; i < N; i++) {
        temp1[i] = data_in[i] * scale;
    }
    Split(temp1, temp2, temp3);
    Loop2: for(int j = 0; j < N; j++) {
        data_out1[j] = temp2[j] * 123;
    }
    Loop3: for(int k = 0; k < N; k++) {
        data_out2[k] = temp3[k] * 456;
    }
}

```

绕过任务和通道大小调整

此外，通常数据应在不同任务间流动。如果您绕过这些任务，则可能降低数据流模型的性能。在以下示例中，`Loop1` 会为 `temp1` 和 `temp2` 生成值。但下一项任务 `Loop2` 仅使用 `temp1` 的值。直至 `Loop2` 后才会使用 `temp2` 的值。因此，`temp2` 会绕过序列中的下一项任务，这可能限制数据流模型的性能。

```

void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {
    int temp1[N], temp2[N], temp3[N];
    Loop1: for(int i = 0; i < N; i++) {
        temp1[i] = data_in[i] * scale;
        temp2[i] = data_in[i] >> scale;
    }
    Loop2: for(int j = 0; j < N; j++) {
        temp3[j] = temp1[j] + 123;
    }
    Loop3: for(int k = 0; k < N; k++) {
        data_out[k] = temp2[k] + temp3[k];
    }
}

```

在此情况下，应将用于存储 `temp2` 的 PIPO 缓冲器的深度增大为 3，而不是采用默认深度 2。这样即可允许该缓冲器在执行 `Loop2` 的同时，存储用于 `Loop3` 的值。同样，绕过 2 个进程的 PIPO 深度应设为 4。使用 `STREAM` 编译指示或指令设置缓冲器深度：

```
#pragma HLS STREAM type=pipo variable=temp2 depth=3
```




重要提示！ 通道大小调整也会对性能产生类似的影响。FIFO/PIPO 深度不匹配可能因 FIFO/PIPO 的反压而导致数据流区域内无意间出现同步点。

任务间的反馈

当任务输出供数据流区域中前一项任务使用时，就会发生反馈。在数据流区域中不支持非串流反馈通道（例如，标量或阵列）。当 HLS 编译器检测到反馈时，它会根据情况发出警告，并且可能无法使用数据流模型。

但是，数据流搭配 `hls::streams` 使用即可支持反馈。以下示例演示了此例外。

```
#include "ap_axi_sdata.h"
#include "hls_stream.h"

void firstProc(hls::stream<int> &forwardOUT, hls::stream<int> &backwardIN) {
    static bool first = true;
    int fromSecond;

    //Initialize stream
    if (first)
        fromSecond = 10; // Initial stream value
    else
        //Read from stream
        fromSecond = backwardIN.read(); //Feedback value
    first = false;

    //Write to stream
    forwardOUT.write(fromSecond*2);
}

void secondProc(hls::stream<int> &forwardIN, hls::stream<int> &backwardOUT)
{
    backwardOUT.write(forwardIN.read() + 1);
}

void top(...) {
    #pragma HLS dataflow
    hls::stream<int> forward, backward;
    firstProc(forward, backward);
    secondProc(forward, backward);
}
```

在此简单设计中，执行 `firstProc` 时，它使用 10 作为输入的初始值。由于 `hls::streams` 不支持初始值，此方法可用于在不违反单一生产者使用者规则的前提下提供初始值。在后续迭代中，`firstProc` 会通过 `backwardIN` 接口读取 `hls::stream`。

`firstProc` 会处理该值，并通过串流将其发送给 `secondProc`，此串流将按原始 C++ 函数执行顺序正向传输。`secondProc` 会读取 `forwardIN` 上的值，对其加 1，并通过反馈串流将其发回 `firstProc`，此反馈串流按执行顺序反向传输。

从第二次执行开始，`firstProc` 会使用从串流中读取的值来执行其计算，两个进程可通过正向和反向通信来保持永久运行，针对首次执行则使用初始值即可。

任务的有条件执行

数据流模型不会对有条件执行的任务进行优化。以下示例着重演示了此限制。在此示例中，Loop1 和 Loop2 的有条件执行会阻止本工具对这些循环之间的数据流进行优化，因为在循环之间不发生数据流动。

```

void foo(int data_in1[N], int data_out[N], int sel) {

    int temp1[N], temp2[N];

    if (sel) {
        Loop1: for(int i = 0; i < N; i++) {
            temp1[i] = data_in[i] * 123;
            temp2[i] = data_in[i];
        }
    } else {
        Loop2: for(int j = 0; j < N; j++) {
            temp1[j] = data_in[j] * 321;
            temp2[j] = data_in[j];
        }
    }
    Loop3: for(int k = 0; k < N; k++) {
        data_out[k] = temp1[k] * temp2[k];
    }
}

```

要确保在所有情况下都能执行每个循环，必须按以下示例中所示方式对代码进行转换。在此示例中，条件语句已移至首个循环内。这两个循环都始终执行，数据始终在循环间进行串流。

```

void foo(int data_in[N], int data_out[N], int sel) {

    int temp1[N], temp2[N];

    Loop1: for(int i = 0; i < N; i++) {
        if (sel) {
            temp1[i] = data_in[i] * 123;
        } else {
            temp1[i] = data_in[i] * 321;
        }
    }
    Loop2: for(int j = 0; j < N; j++) {
        temp2[j] = data_in[j];
    }
    Loop3: for(int k = 0; k < N; k++) {
        data_out[k] = temp1[k] * temp2[k];
    }
}

```

含多个退出条件的循环

在数据流区域中无法使用含多个出口点的循环。在以下示例中，Loop2 具有 3 项退出条件：

- 由 N 的值定义的出口：当 $k \geq N$ 时循环将退出。
- 由 break 语句定义的出口。

- 由 `continue` 语句定义的出口。

```
#include "ap_int.h"
#define N 16

typedef ap_int<8> din_t;
typedef ap_int<15> dout_t;
typedef ap_uint<8> dsc_t;
typedef ap_uint<1> dsel_t;

void multi_exit(din_t data_in[N], dsc_t scale, dsel_t select, dout_t
data_out[N]) {
    dout_t temp1[N], temp2[N];
    int i,k;

    Loop1: for(i = 0; i < N; i++) {
        temp1[i] = data_in[i] * scale;
        temp2[i] = data_in[i] >> scale;
    }

    Loop2: for(k = 0; k < N; k++) {
        switch(select) {
            case 0: data_out[k] = temp1[k] + temp2[k];
            case 1: continue;
            default: break;
        }
    }
}
```

由于循环的退出条件始终由循环边界来定义，因此使用 `break` 或 `continue` 语句将禁止在 DATAFLOW 区域内使用循环。

最后，数据流模型化不含任何分层实现。如果子函数或循环包含可能受益于数据流的其他任务，那么必须对该循环、子函数或子函数的内联应用数据流模型。

您也可以在数据流区域内使用 `std::complex`。但应搭配 `__attribute__((no_ctor))` 一起使用，如下示例所示：

```
void proc_1(std::complex<float> (&buffer)[50], const std::complex<float>
*in);
void proc_2(hls::Stream<std::complex<float>> &fifo, const
std::complex<float> (&buffer)[50], std::complex<float> &acc);
void proc_3(std::complex<float> *out, hls::Stream<std::complex<float>>
&fifo, const std::complex<float> acc);

void top(std::complex<float> *out, const std::complex<float> *in) {
#pragma HLS DATAFLOW

    std::complex<float> acc __attribute__((no_ctor)); // Here
    std::complex<float> buffer[50] __attribute__((no_ctor)); // Here
    hls::Stream<std::complex<float>, 5> fifo; // Not here

    proc_1(buffer, in);
    proc_2(fifo, buffer, acc);
    proc_3(out, fifo, acc);
}
```

利用静态变量进行流水打拍的局限性

静态变量用于在循环迭代间保留数据，这通常在导致最终实现时生成寄存器。如果在以流水节拍方式执行的函数中遇到此变量，Vitis HLS 可能无法对设计进行充分优化，这可能导致启动时间间隔超过所需的时间。

以下是此状况的典型示例：

```
function_foo()  
{  
    static bool change = 0  
    if (condition_xyz){  
        change = x; // store  
    }  
    y = change; // load  
}
```

如果 Vitis HLS 无法优化此代码，则存储操作需要 1 个周期，加载操作也需要 1 个周期。如果此函数包含在流水线中，那么流水线必须以最小启动时间间隔 2 来实现，因为静态变更变量会创建循环附带的依赖关系。

用户可通过重写代码来避免此问题，如下例所示。它可确保在循环的每次迭代中仅存在读操作或写操作，这样即可以 II=1 来调度设计。

```
function_readstream()  
{  
    static bool change = 0  
    bool change_temp = 0;  
    if (condition_xyz)  
    {  
        change = x; // store  
        change_temp = x;  
    }  
    else  
    {  
        change_temp = change; // load  
    }  
    y = change_temp;  
}
```

规范数据流

当 Vitis HLS 发现 #pragma HLS dataflow 时，它会将出现该编译指示的函数或循环体（以下称为“数据流区域”）转换为进程集合。

这组进程集合可以通过以下方式识别：

- 由设计师决定，设计师可以为每个进程调用一个函数或例化一个 hls::task，例如：

```
void dataflow(int Input0, int Input1[], int &Output0, int Output1[]) {
#pragma HLS dataflow
    int C0, C1[N], C2;
    func1(Input0, Input1, C0, C1); // first process
    func2(C0, C1, C2); // second process
    func3(C2, Output0, Output1); // third process
}
```

- 或者由工具决定（如果数据流区域包含函数调用以外的代码），例如：

```
void dataflow(int Input0, int Input1[], int &Output0, int Output1[]) {
#pragma HLS dataflow
    int C0, C1[N], C2;      C0 = Input0 * 3; // first process
    for (int i = 0; i < N; i++) { // first process or second process?
        C1[i] = Input1[0] + 2;
    }
    func2(C0, C1, C2); // second process or third process?
    func3(C2, Output0, Output1); // third process or fourth process?
}
```

第一种形式生成的数据流网络**可预测性更高**，无论是性能还是结构方面都是如此，而第二种形式的可预测性则低得多，因为可能存在多种选择。

本节讨论如何通过遵循一套**编码风格准则**（也称为“规范数据流”）来实现良好的可预测性。

这里按照从基础到高级的三个级别来探讨这些准则。

每当违反编码风格规则时，Vitis HLS 前端都会生成警告。

在此情况下，Vitis HLS 会尽力实现所生成的数据流（如果违例是致命的，例如阵列有多个写入程序进程或多个读取程序进程，则会报错退出）。

但您应始终检查 GUI 数据流查看器和协同仿真时间线轨迹，以确保数据流和达成的性能都与期望相符。

具体来说，数据流层级内存在的任何非数据流层级（即顺序 FSM）**通常表示性能损失**，因为这些层级**不允许对其所含的数据流区域进行重叠执行**。

基础级别

在最基础的级别，可在两种不同的上下文中应用数据流编译指示：

- 在**函数**（也称为函数数据流区域）内
- 在一种特殊类型的**for 循环**（也称为循环数据流区域）内，目的是：
 - 使该循环成为函数体内部的**唯一语句**
 - 禁止使用位于该循环外部的变量声明
 - 循环变量的类型为“integer”（整数）
 - 在循环头中声明初始值，并将其设置为任意非负整数常量。
 - 退出条件为，该循环变量与某个非负数常量或该循环的外围函数的标量实参进行“<”比较。
 - 循环变量按任意正整数常量递增。

为了保证可预测性，建议采用如下编码风格：**该函数体或 for 循环只能包含以下序列**

- 子函数调用（使用 `ap_ctrl_chain`）和/或
- `hls::task` 声明（使用 `ap_ctrl_none`）。

数据流区域内不得存在任何控制（if-then-else 和循环会被自动转换为进程，导致上述数据流结构不可预测）。

这些被调用的子函数和 `hls::task`：

- 返回类型必须为 `void`
- 可以是：
 - 顺序函数或流水节拍函数（称为**进程**）或者
 - 函数数据流区域，或者
 - 循环数据流区域。

其他语法限制：

- 不允许变量初始化（包括构造函数自动执行的初始化）：
 - 对于具有无法重定义的默认构造函数的标准数据类型（例如：`std::complex`），可使用 `no_ctor` 属性来避免初始化
 - 例如：

```
std::complex<float> arr[SIZE] __attribute__((no_ctor));
```

- 不允许按值传递表达式给进程。
 - 在上述示例中，第一个进程必须按如下方式声明（注意 `Input1` 是按引用传递的）：

```
void func1(int Input0, int Input1[], int &Output0, int Output1[])
```

建议的数据流函数风格示例如下：

```
void dataflow(int Input0, int Input1[], int &C0, int C1[]) {
#pragma HLS dataflow
    int C1[N], C2; // no initialization
    UserDataTypes C0 __attribute__((no_ctor)); // no_ctor must be used if the
    default constructor is not empty
    func1(Input0, Input1, C0, C1); // read Input0, read Input1, write C0,
    write C1
    func2(C0, C1, C2); // read C0, read C1, write C2
    func3(C2, Output0, Output1); // read C2, write Output0, write Output1
}
```

循环中建议的数据流风格示例如下：

```
void dataflow(int Input0, int Input1[], int &Output0, int Output1[]) {
    for (int i = 0; i < N; i++) {
#pragma HLS dataflow
        int C1[N], C2; // no initialization
        UserDataTypes C0 __attribute__((no_ctor)); // no_ctor must be used if
        the default constructor is not empty
        func1(Input0, Input1, C0, C1, i); // read Input0, read Input1, write
        C0, write C1
    }
```

```

    func2(C0, C1, C2, i);           // read C0, read C1, write C2
    func3(C2, Output0, Output1, i); // read C2, write Output0, write
Output1
}
}

```

注释：发生循环的函数不需要数据流编译指示。

关于数据流函数体或区域的进一步语义限制如下：

- 局部变量必须是**非静态**标量或阵列（仅在被调用的进程中允许包含静态变量）。
 - 规范区域内 `hls::task` 和 `hls::thread` 的实例必须声明为 `hls_thread_local`（例如，`hls_thread_local hls::task t1(proc, arg1, arg2, arg3);`）。
 - 请注意，`hls_thread_local` 类似于静态变量，但更安全，因为它并不会在具有相同函数体的多个 `hls::task` 之间暗含共享的单变量实例。
- 函数调用和/或 `hls::task` 的序列必须在以下条件下使用局部变量来传输数据：
 - 阵列：
 - 只能有一个写入程序进程和一个读取程序进程。
 - 该写入程序必须按文本顺序位于读取程序之前（即，无循环携带依赖）
 - `hls::streams` 和 `hls::stream_of_blocks`：
 - 只能有一个读取程序进程和一个写入程序进程。
 - 也可用于反向传输数据（向按文本顺序更早出现的进程传输数据）
 - 若有进程要从这些反馈串流读取数据，那么请务必注意确保这类进程仅在后续进程产生一些数据之后才会尝试读取数据。
 - 满足此要求的最常见的方法是在这些进程内使用 `hls_thread_local` 变量，用于在首次执行时跳过读取操作
 - **强烈不建议**出于同一目的使用非阻塞读取，因为它本质上会导致**非确定性实现和协同仿真不匹配**。
 - 标量：
 - 可包含多个写入程序进程和读取程序进程
 - 自动转换为 FIFO 通道
 - 在循环内数据流情况下，不得有循环携带依赖

高级级别

除了上文所讨论的基础级别外，建议的可预测数据流编码风格还支持：

- 完全展开的循环（在数据流区域内）。循环体只能包含函数调用和 `hls::task` 例化。
- 完全分区的阵列，其中将每个分区均作为一个独立变量（阵列或标量）来传递给一个进程。

例如，以下代码会创建一个流水线，它由 N 个相同进程组成，所有进程都执行相同功能，并通过 `hls::streams` 链加以级联：

```

void dut(int in[M], int out[M]) {
    #pragma HLS dataflow
    hls_thread_local hls::stream<int> chan[N+1]; // arrays of hls::streams
are fully partitioned
    read_in(in, chan[0]);
    hls_thread_local hls::task t[N]; // array of worker processes

```

```
for (int i=0; i<N; i++) {  
    #pragma HLS unroll  
    t[i](worker, chan[i], chan[i+1]);  
}  
write_out(chan[N], out);  
}
```

以下是另一个示例，其中改用 PIPO 链（或者如果添加 #pragma HLS stream variable=chan，则改用串流阵列）：

```
void dut(int in[M], int out[M]) {  
    #pragma HLS dataflow  
    int chan[N+1][M]; // partitioned into N+1 arrays of M elements  
    #pragma HLS array_partition complete dim=1 variable=chan  
    read_in(in, chan[0]);  
    for (int i=0; i<N; i++) {  
        #pragma HLS unroll  
        worker(chan[i], chan[i+1]);  
    }  
    write_out(chan[N], out);  
}
```


第三部分

Vitis HLS 流程步骤

本部分包含以下章节：

- [启动 Vitis Unified IDE](#)
- [创建 HLS 组件](#)
- [运行 C 语言仿真](#)
- [运行 C 语言综合](#)
- [运行 C/RTL 协同仿真](#)
- [封装 RTL 设计](#)
- [运行实现](#)
- [优化 HLS 工程](#)
- [从命令行创建 HLS 组件](#)

启动 Vitis Unified IDE



提示：AMD Vitis™ Unified IDE 应用是随 Vitis 软件平台一起安装的。它无需额外单独安装。如需了解有关安装和启动 Vitis Unified IDE 的更多信息，请参阅《使用 Vitis 进行嵌入式设计开发》(UG1701)。

1. 使用以下命令设置 Vitis 软件平台环境。

```
source <Vitis_Installation_Directory>/settings64.sh
```

2. 使用以下命令启动下一代 Vitis Unified IDE。

```
vitis -w <workspace>
```

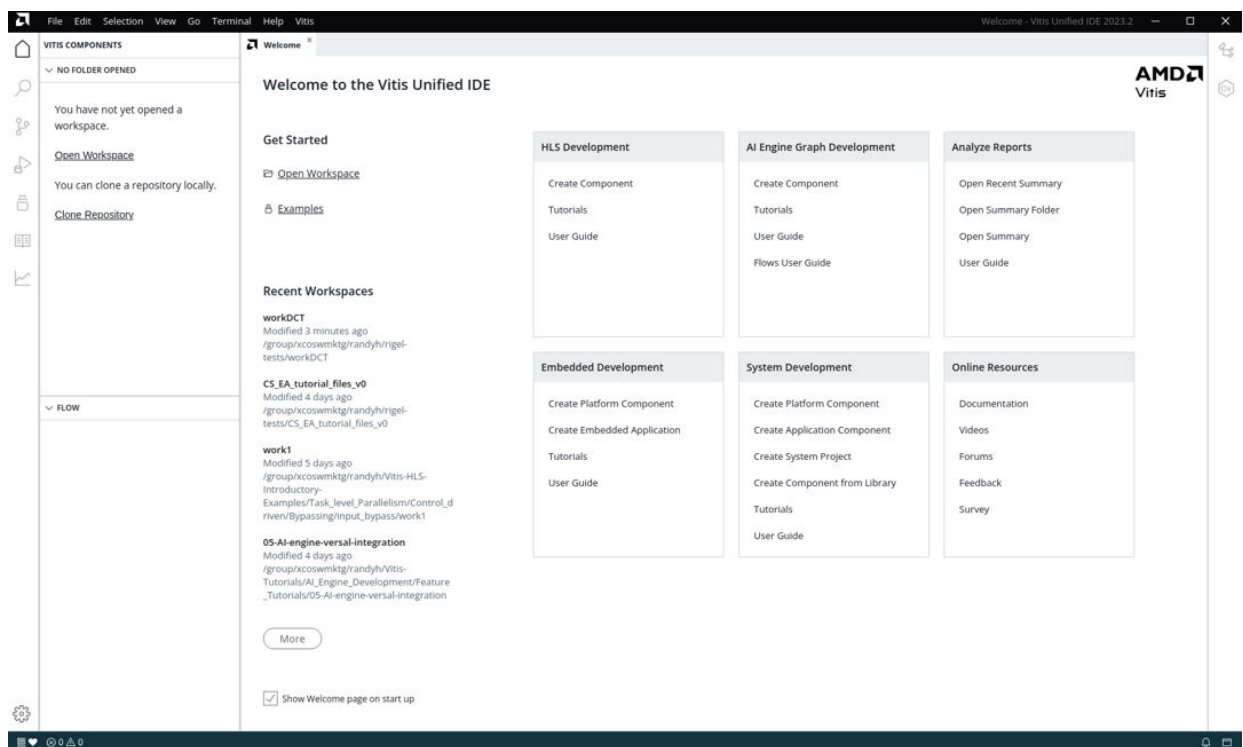
其中，<workspace> 表示用于保存 HLS 组件元素或其他设计工程的文件夹。该工作空间用于将构成某一设计或多个设计的源文件和数据文件组合在一起，并存储工具配置以供该工作空间使用。



提示：您还可从 Windows 桌面双击应用图标来为 HLS 组件启动 Vitis Unified IDE。

这将打开 Vitis Unified IDE，并显示“Welcome”（欢迎）屏幕，如下图所示。

图 79: “Welcome” 屏幕



在“HLS Development”（HLS 开发）下，包含以下选项。

- “Create Component”（创建组件）：启动“Create HLS Component” Wizard（创建 HLS 组件向导）以创建新组件。如需了解更多信息，请参阅 [构建和运行 HLS 组件](#)。
- “Create Component from Library”（从库创建组件）：从库函数创建新的 HLS 组件。
- “Tutorials”（教程）：打开 Web 浏览器并指向 GitHub 上的 [Vitis-HLS-Introductory-Examples](#)。请参阅 [教程与示例](#)。
- “User Guide”（用户指南）：在 Web 浏览器中打开此文档。

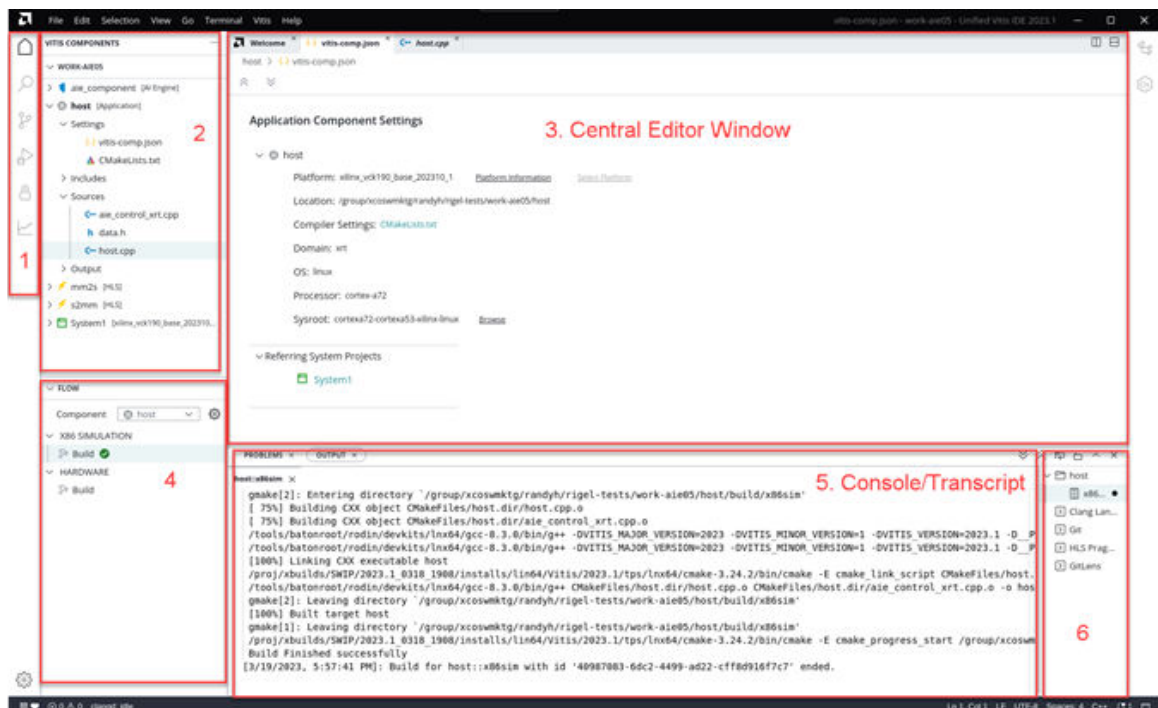
您可看到，该工具中还有其他命令块和功能特性可供使用。在“Online Resources”（在线资源）下，可找到有助于您使用该工具的文档和教程。

如果您先前已启动 Vitis Unified IDE，那么您可加载“Recent Workspaces”（最近的工作空间）下列出的先前加载的工作空间之一。您还可在“File”（文件）菜单下查看其他可用选项。

Vitis Unified IDE 功能特性

全新的 AMD Vitis™ Unified IDE 提供了诸多全新的功能特性，以下简述了这些功能特性：

图 80：Vitis Unified IDE



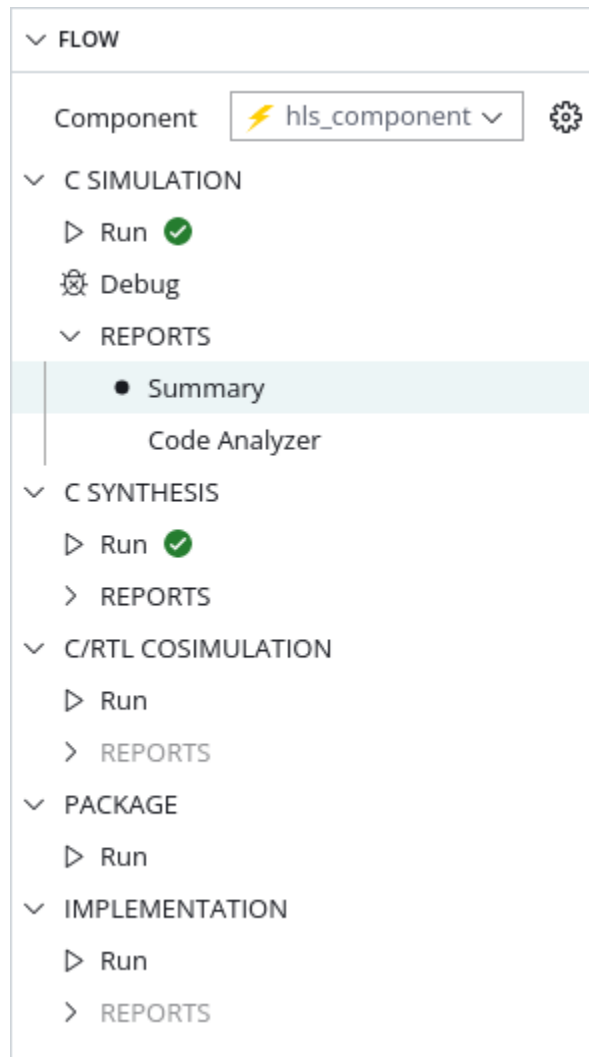
1. 位于屏幕左侧的工具栏支持轻松访问该工具的主要功能特性：Vitis Component Explorer（组件资源管理器）、搜索功能、“Source Control”（源码控制）、“Debug”（调试）视图、“Examples”（示例）和“Analysis”（分析）视图。

2. Vitis Component Explorer 可用于查看工作空间的虚拟层级。此视图显示了一个工作空间，其结构有助于您了解组件或工程的不同元素，例如，磁盘上不存在的“Sources and Outputs”（源码和输出）文件夹。
3. “Central Editor”（中央编辑器）窗口用于编辑组件、配置和源文件。
4. Flow Navigator 显示了活动组件的设计流程。不同组件具有不同的工作流程，在 Flow Navigator 中显示的是活动组件的工作流程。您可在 Flow Navigator 或“Component Explorer”中选中组件以将其指定为活动组件。
5. “Console/Terminal”（控制台/终端）区域显示的是该工具的输出转录文本，“Terminal”窗口和“Pipeline”（流水线）视图等其他窗口也位于此处。“Terminal”窗口显示的是工作空间的文件夹，可用于对内容运行脚本。
6. “Index”（索引）显示的是会话期间运行的各种构建步骤的转录文本列表，支持您重新打开先前关闭的进程转录文本。

使用 Flow Navigator

对于 Vitis Unified IDE 中的 HLS 组件，“Flow Navigator”是一种进程流表示法，进程中每个步骤均以操作来表示，您可启动这些操作通过流程化方式完成工作。每个步骤完成后，即可通过 Flow Navigator 浏览所有报告和报告查看器。

图 81: Flow Navigator



HLS 组件的进程步骤包括：

- “C SIMULATION”（C 语言仿真）：使用指定的源代码和测试平台以及按 [运行 C 语言仿真](#) 中所述提供的任何输入数据来运行 C 语言仿真和代码分析器。
- “C SYNTHESIS”（C 语言综合）：打开“C Synthesis”对话框，并列出运行综合后可用的报告，如 [运行 C 语言综合](#) 中所述。
- “C/RTL CO-SIMULATION”（C/RTL 协同仿真）：打开“C/RTL Cosimulation”对话框，并列出运行仿真后可用的报告，如 [运行 C/RTL 协同仿真](#) 中所述。
- “PACKAGE”（封装）：生成 Vivado IP (.zip) 或 Vitis 内核 (.xo) 并将其从综合后的 RTL 设计中导出，如 [封装 RTL 设计](#) 中所述。
- “IMPLEMENTATION”（实现）：运行 Vivado 综合与实现以生成更详细的使用情况和时序报告，如 [运行实现](#) 中所述。

“Flow Navigator” 还包含一些功能特性以帮助您查看关联的文件和设计状态。如上图所示，在 “Component”（组件）菜单旁以及设计中的每个进程步骤旁都提供了 “Open Settings”（打开设置）命令 ()。选中 “Component” 旁的 “Open Settings”（打开设置）即可打开该组件的 `vitis-comp.json` 文件。在进程步骤旁选中此项即可打开 “Config File Editor”（配置文件编辑器）。

您可在 “Flow Navigator” 中使用 “Cancel Run”（取消运行）命令来取消运行中的进程。此命令默认隐藏，只需将鼠标悬停在运行中的步骤上即可显示。

当某些步骤未完成或者由于设计源文件或关联文件中发生更改而需要重新运行某些步骤时，“Flow Navigator” 会显示警告符号 ()。将鼠标悬停在该警告上即可显示消息以指明可能存在的问题。

构建和运行 HLS 组件

AMD Vitis™ Unified IDE 和 `v++` 通用命令行提供了使用自下而上设计技术创建 HLS 组件的功能，以确保组件满足设计要求后再将其与其他要素集成到系统设计中。在 HLS 组件中，该工具会将 C 语言或 C++ 语言函数综合到 RTL 代码中，以供在 AMD Versal™ 自适应 SoC、AMD Zynq™ MPSoC 或 AMD FPGA 器件的可编程逻辑 (PL) 区域中实现。

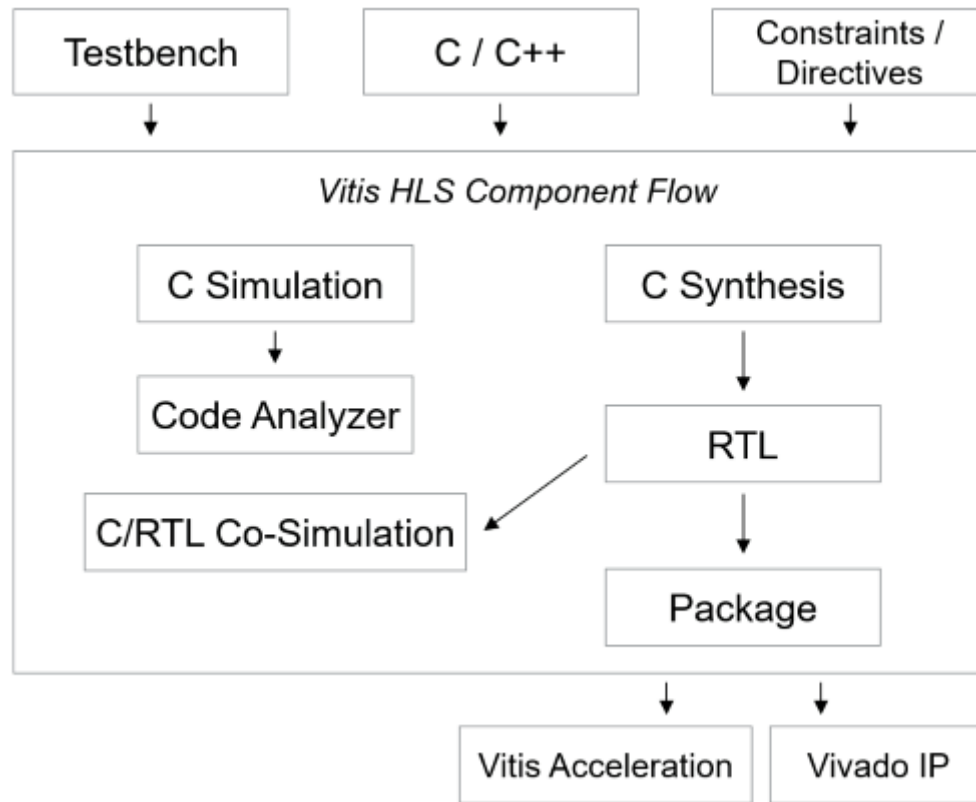
HLS 组件可用于开发和导出：

- Vivado IP，以便使用 Vivado Design Suite 和嵌入式软件开发流程将其集成到硬件设计中，如《Vitis 统一软件平台文档：嵌入式软件开发》(UG1400) 中所述。
- Vitis 内核，以便在异构计算系统和数据中心应用加速中使用，如《使用 Vitis 进行嵌入式设计开发》(UG1701) 中所述。

C++ 函数的开发步骤如下所述。

1. 基于 [设计原则](#) 来架构 C/C++ 算法
2. 使用 C 语言测试平台（C 语言仿真）验证源代码逻辑
3. 分析设计的并行度和性能（代码分析器）
4. 根据源代码生成 RTL（C 语言综合）
5. 使用 C 语言测试平台和 RTL 逻辑仿真来验证生成的 RTL 模块（C/RTL 协同仿真）
6. 复查并分析 HLS 综合报告和协同仿真报告（分析）
7. 重复前述步骤直至满足性能目标为止

图 82: Vitis HLS 开发流程



HLS 组件基于目标流程、默认工具配置、配置命令以及您为设计指定的任何优化编译指示或指令来实现设计。以下章节描述了如何创建和构建 HLS 组件。

创建 HLS 组件

有多种方法可用于在 Vitis Unified IDE 中创建 HLS 组件。以下步骤描述了其中一种建议的方法。



提示：本节呈现了 Vitis IDE 中的 HLS 组件。如需了解命令行方法，请参阅 [从命令行创建 HLS 组件](#)。

1. 打开 Vitis IDE 后，请从主菜单中依次选中“File”→“New Component”→“HLS”（文件 > 新建组件 > HLS）。



提示：您还可在“Welcome”（欢迎）页面上选择“New Component”（新建组件），或者从“Explorer”（资源管理器）视图的右键单击菜单中选择该命令。

2. 这样即可打开“Create HLS Component” Wizard（创建 HLS 组件向导）的“Choose Name and Location”（选择名称和位置）页面，如下所示。

图 83：创建 HLS 组件 - 名称和位置

The screenshot shows a 'Create Integration Project' dialog box with a dark header bar. Below the header, there is a breadcrumb trail: 'Name and Location > Build Output > Input Selection > Summary'. The 'Name and Location' tab is active. The instructions state: 'Choose a name for your integration project and specify a directory where integration project data files will be stored'. There are two input fields: 'Integration project name' with the text 'integration_project' and 'Integration project location' with a dropdown menu showing '/TEST_WORKSPACE/integration_...' and a 'Browse' button. Below these fields, it says 'Integration project will be created at: /TEST_WORKSPACE/integration_project'. At the bottom, there are 'Cancel' and 'Next' buttons.

输入“Component name”（组件名称）和“Component location”（组件位置），然后单击“Next”（下一步）。

3. 这样即可打开该向导的“Configuration File”（配置文件）页面，如下所示。HLS 组件配置文件包含用于组件综合、仿真和导出的命令和指令。

图 84：创建 HLS 组件 - 配置文件

The screenshot shows a web-based wizard titled "Create HLS Component - Empty HLS Component". The breadcrumb navigation is "Name and Location > Configuration File > Source Files > Hardware > Settings > Summary". The "Configuration File" step is active. It asks, "A configuration file stores component settings like source files, compiler settings, etc. How would you like to set up your configuration file?". There are three radio button options: "Empty File" (selected), "Existing File", and "Generate from Vitis HLS Project". Below the options, it says "Create a new, empty configuration file". There is a text input field for "New configuration file name" with the value "hls_config". Below that, it says "Component configuration file will be created at:" followed by a text area. At the bottom, there are "Cancel", "Back", and "Next" buttons.

“Create HLS Component” Wizard 的“Configuration File”（配置文件）页面提供了 3 种方法用于向您的组件添加配置文件：

- “Empty File”（空文件）：指定创建新的空配置文件，并在创建组件后填充该文件，如 [定义 HLS 配置文件](#) 中所述。选择该选项即可允许您在“New configuration file name”（新配置文件名称）字段中指定名称。
- “Existing File”（现有文件）：指定使用现有配置文件。选择该选项即可允许您在“File”（文件）字段中输入路径和名称，或者选择“Browse”（浏览）以查找文件。
- “Generate from Vitis HLS Project”（从 Vitis HLS 工程生成）：该选项会基于现有 Vitis HLS 工程 `hls.app` 文件生成新的配置文件。该工具允许您识别工程内的 `hls.app` 文件并为基于 `hls.app` 文件生成的配置文件输入“New configuration file name”（新配置文件名称）。

请选择用于向您的 HLS 组件添加配置文件的方法，并按需指定名称或文件。单击“Next”以继续至“Source Files”（源文件）。

4. 这样即可打开该向导的“Source Files”（源文件）页面，如下所示。

图 85：创建 HLS 组件 - 源文件

Create HLS Component - Empty HLS Component

Name and Location > Configuration File > Source Files > Hardware > Settings > Summary

Add Source Files

Specify design files, test bench files and flags for your component. You can also skip this step now and add sources later.

DESIGN FILES

FILE(S)

Flags common to all files

CFLAGS

CSIMFLAGS

Top Function: Select Top Function [Browse](#)

TEST BENCH FILES

FILE/FOLDER(S)

Flags common to all files

CFLAGS

[Cancel](#) [Back](#) [Next](#)

您可在“Design Files”（设计文件）下单击“Add files”（添加文件）为设计选择主要的源文件，或者按需添加多个文件。添加一个或多个文件后，您可识别“Top Function”（顶层函数），它将作为 HLS 组件的顶层函数进行综合。顶层函数调用的任何函数都将综合到 RTL 中。

您可在“Test Bench Files”（测试激励文件）下单击“Add files”（添加文件）或者选择“Add folders”（添加文件夹）来为 HLS 组件添加文件（包含测试激励文件和关联文件）。

对于设计文件和测试激励文件，您可添加 CFLAGS 和 CSIMFLAGS 以指定要在编译和仿真期间使用的指令。您可指定要应用于所有文件或应用于来自每个类别的特定文件的指令。添加文件、文件夹和指令后，单击“Next”继续。

5. 这样即可打开该向导的“Select Part”（选择部件）页面，如下所示。

图 86：创建 HLS 组件 - 部件

Create HLS Component - Empty HLS Component

Name and Location > Configuration File > Source Files > **Part** > Settings > Summary

Select Part

Specify a board, device, or platform you are compiling your component for

☒ Part

☐ Platform

Product Category

All

Family

All

Sub Family

All

Packages

All

Speed Grade

All

Temp Grade

All

Reset Filter

PART	FAMILY	PACKAGE	SPEED	LUT	FF	DSP	BRAM
xcvu7p_civ-flvc2104-1-i	Virtex UltraScale+	flvc2104	-1	788160	1576320	4560	1440
xcvu9p-flga2104-3-e	Virtex UltraScale+	flga2104	-3	1182240	2364480	6840	2160
xcvu9p-flga2104-2-e	Virtex UltraScale+	flga2104	-2	1182240	2364480	6840	2160
xcvu9p-flga2104-2-i	Virtex UltraScale+	flga2104	-2	1182240	2364480	6840	2160
xcvu9p-flga2104-2L-e	Virtex UltraScale+	flga2104	-2L	1182240	2364480	6840	2160

Cancel

Back

Next

为 HLS 组件选择“Part”（部件）或者选择“Platform”（平台），然后单击“Next”继续。

提示：如果创建 HLS 组件时选择了“Platform”，那么 Vitis Unified IDE 会从指定平台提取部件，并将 `part=` 添加至您的配置文件中。

6. 这样即可显示该向导的“Settings”（设置）页面，如下所示。

UG1399 (v2025.2) 2026 年 1 月 22 日
Vitis HLS 用户指南

Send Feedback

271

图 87：HLS 空组件

“Settings” 页面允许您为设计指定时钟周期或频率，以及时钟不确定性。本工具的默认时钟设为 10 ns（或 100 MHz）的时钟周期，且时钟不确定性为 27%，如 [指定时钟频率](#) 中所述。

“Settings”（设置）页面还允许您将 HLS 组件构建进程的“flow_target”（流程目标）指定为生成 Vivado IP 或 Vitis 内核，如 [目标流程概述](#) 中所述。Vivado IP 或 Vitis 内核的接口要求不同，如 [HLS 设计接口](#) 中所述。默认“flow_target”设为 Vivado IP 流程。

在“flow_target”下，您还可指定封装设计时工具要生成的“package.output.format”（封装输出格式）。默认输出格式为 Vivado IP，即，生成 .zip 文件以添加到您的 IP 目录中，支持您在其他设计中使用 HLS 生成的 RTL 设计。

注释：选择 Vitis 作为“flow_target”并不会将输出格式自动更改为 Vitis 内核。默认输出格式仍为 Vivado IP，但您可按需手动更改输出格式。

另请谨记，HLS 组件使用的是自下而上的设计流程，在您封装输出之前，仅生成 RTL 输出，如 [封装 RTL 设计](#) 中所述。自下而上的流程支持您在逐个解决设计难题的过程中开发多次设计迭代，并且仅在设计准备就绪后才封装输出。这种方法可在您每次运行综合或仿真时节省大量时间。

单击“Next”以继续至“Summary”（汇总）页面。

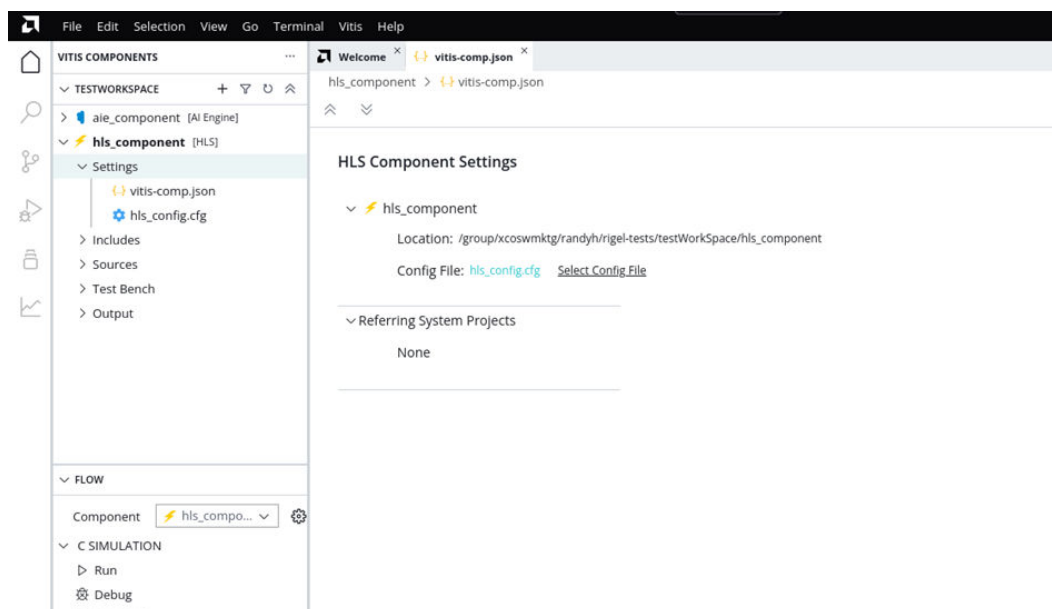
7. “Summary” 页面反映了您在前几个页面上的选择。请复查汇总信息并单击“Finish”（完成）以创建 HLS 组件，或者单击“Back”（返回）以返回至先前页面并更改您的选择。

创建 HLS 组件时，会在中央编辑器窗口中打开此组件的 `vitis-comp.json` 文件，并且在 Flow Navigator 中该 HLS 组件会变为活动状态，如下所示。



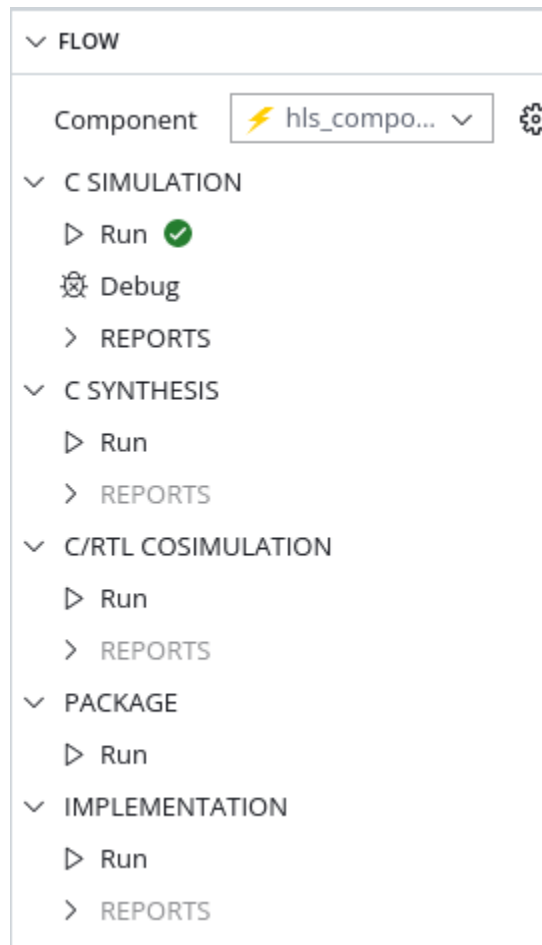
提示：打开多个 HLS 组件时，在 `vitis-comp.json` 文件名旁的选项卡中会一并显示组件名称以帮助您识别不同组件。

图 88: HLS 组件 - vitis-comp.json



创建 HLS 组件后，它会在 Flow Navigator 中变为活动组件，或者您可以选中“Component”（组件）在该工具中将其设为活动组件。当 HLS 组件为活动组件时，Flow Navigator 支持运行 C 语言仿真、C 语言综合、C/RTL 协同仿真、封装与实现，如下图所示。

图 89: HLS 组件 - Flow Navigator



但如果您要管理这些步骤的操作，就需要编辑 HLS 配置文件以达成设计目标，如下所述。

目标流程概述

HLS 组件旨在处理 Vivado IP 流程或 Vitis 内核流程。HLS 组件是基于目标流程、默认工具配置、设计约束和您指定的任意优化编译指示或指令来打造的。您可使用优化指令来修改和控制内部逻辑和 I/O 端口的实现，以覆盖默认行为。

Vivado IP 流程

当您在“Create HLS Component”Wizard（创建 HLS 组件向导）的“Settings”（设置）页面上选中“Vivado IP Flow”（Vivado IP 流程）时，或者在 HLS 配置文件中指定 `flow_target=vivado` 时，即表示您正在将该工具配置为生成 RTL 文件以供在 Vivado Design Suite 中使用、搭配其他 RTL 文件和 Vivado IP 使用，以及搭配软件应用来使用，这些软件应用使用软件来与硬件进行交互。

HLS IP 可实现到 AMD Versal™ 自适应 SoC 器件、AMD Zynq™ MPSoC 器件或 AMD FPGA 器件的可编程逻辑 (PL) 区域内。此 IP 是使用器件驱动程序生成的，因此，PL 可与嵌入式软件应用相集成以管理和运行硬件功能，如《Vitis 嵌入式软件开发流程文档》(UG1400) 中所述。

相较于 Vitis 内核流程，Vivado IP 流程给设计选择赋予了更大的灵活性，但也将 IP 集成和管理责任交给了您。Vivado IP 流程可支持各种接口规范和数据传输协议，但给函数实参分配了默认接口，如 [Vivado IP 流程接口](#) 中所述。您也可以通过使用 `INTERFACE` 编译指示或 `set_directive_interface` 命令来为函数实参手动分配接口规范以覆盖默认设置，满足 Vivado 设计所需。

Vitis 内核流程

当您在“Create HLS Component”Wizard 的“Settings”页面上选中“Vitis Kernel Flow”（Vitis 内核流程）时，或者在 HLS 配置文件中指定 `flow_target=vitis` 时，即表示您正在配置该工具，使其生成适合用于 Vitis 开发环境和 Xilinx Runtime (XRT) 的 RTL 设计。Vitis 开发流程可在嵌入式系统设计中用于 AMD Versal™ 自适应 SoC、AMD Zynq™ MPSoC 器件或 Alveo 加速器卡上运行的数据中心应用加速的异构计算系统，如《使用 Vitis 进行数据中心加速》(UG1700) 中所述。Vitis 开发流程局限性比 Vivado IP 流程更大，因为 HLS 工具所生成的 RTL 的设计要求必须满足平台和 Xilinx Runtime (XRT) 的特定要求。

在 IDE 中指定“Vitis Kernel Flow”（Vitis 内核流程）时，该工具会使用 AXI 标准来实现接口端口，如 [适用于 Vitis 内核流程的接口](#) 中所述。如果代码中不含任何现有 `INTERFACE` 编译指示或指令，则默认将应用以下接口协议。

- AXI4-Lite 接口 (`s_axilite`) 将分配至标量实参、阵列的控制信号以及软件函数的返回值。
- AXI4 主接口 (`m_axi`) 分配到 C/C++ 函数的指针与阵列实参。
- Vitis HLS 会尽可能自动尝试推断 BURST 传输事务，聚合存储器访问以便最大程度提升吞吐量带宽和/或减小时延。
- 使用 `hls::stream` 数据类型定义软件函数暗示使用的是 AXI4-Stream (`axis`) 端口。

Vivado/Vitis 流程的默认设置

对于 Vivado IP 流程与 Vitis 内核流程，该工具的默认设置不尽相同。下表显示了这两个流程的默认设置。



提示：除了默认配置外，该工具还有其他功能特性可用于支持其中某一流程，而不支持另一流程，或者两个流程之间采用不同配置。在本文档中，会高亮显示此类差异。

表 23：默认配置

配置	Vivado	Vitis
clock_uncertainty	27%	27%
syn.compile.pipeline_loops	64	64
syn.compile.name_max_length	255	255
vivado.optimization_level	0	0
vivado.phys_opt	无	无
syn.rtl.module_auto_prefix	true	true
syn.rtl.register_reset_num	0	3
syn.schedule.enable_dsp_full_reg	true	true
syn.directive.interface	IP 模式	内核模式
syn.interface.m_axi_addr64	true	true
syn.interface.m_axi_latency	0	64
syn.interface.m_axi_alignment_byte_size	1	64
syn.interface.m_axi_max_widen_bitwidth	0	512
syn.interface.default_slave_interface	s_axilite	s_axilite
syn.interface.m_axi_offset	从	从

流程封装设置

除这些默认设置外，您还必须对生成的 RTL 文件进行封装，以供下游进程使用，例如，供 Vivado 设计用于将多个 RTL 文件或 Vivado IP 集成为单个设计。或者供系统工程内的 Vitis 内核用于与其他 PL 内核及 AI 引擎计算图应用集成。生成的输出由 `package.output.format` 和 `package.output.syn` 设置来判定，如 [封装 RTL 设计](#) 中所述。输出格式允许您从设计的 RTL 文件创建封装格式。默认输出格式为 Vivado IP .zip 文件格式，可添加到您的 IP 目录中，也可在 Vivado IP integrator 功能特性中使用。您还可将输出格式指定为 XO 格式，以便将 RTL 文件封装为内核，以供在 Vitis 开发流程中使用。选择 `flow_target` 并不会自动指定输出格式。您必须自行完成此操作。

处理源文件

Vitis HLS 输入包括：

- 以 C 语言和 C++11/C++14 语言编写的 C 语言函数。这是 Vitis HLS 的主要输入。函数可包含子函数层级。
- 用于指定时钟周期、时钟不确定性和器件目标的配置文件。
- 指令为可选，用于指示综合流程实现特定行为或优化操作。
- C 语言测试平台及在综合前对 C 语言函数进行仿真所需的所有关联文件，以及使用 C/RTL 协同仿真来验证 RTL 输出所需的文件。

您可在 Vitis Unified IDE 中向 HLS 组件添加 C/C++ 源文件、指令和约束，如 [创建 HLS 组件](#) 中所述。您还可创建 HLS 配置文件，并使用 `v++ -c --mode hls` 命令，如 [从命令行创建 HLS 组件](#) 中所述。

C/C++ 函数编码

编码样式

在任意 C 程序中，顶层函数都称为 `main()`，通常可在 C 语言仿真的测试平台中找到该函数。在 HLS 组件中，您可将 `main()` 调用的任意子函数指定为高层次综合的顶层函数。您无法综合顶层函数 `main()`。部分其他规则包括：

- 仅允许指定 1 个函数作为综合的顶层函数。
- 层级中综合的顶层函数下的任意子函数也同样会进行综合。
- 如果要综合的函数不包含在综合的顶层函数下的层级内，则必须将此类函数合并到综合的单一顶层函数中。

C/C++ 语言支持

HLS 组件支持许多 C 和 C++11/C++14 语言构造以及每一种语言的所有原生数据类型，包括浮点和双精度类型，如 [数据类型](#) 中所述。但是，针对部分构造不支持综合，包括：

- 动态存储器分配：FPGA 具有一组固定资源，不支持动态创建和清空存储器资源。
- 操作系统 (OS) 操作：往来 FPGA 的所有数据都必须从输入端口读取或写入输出端口。不支持操作系统操作（例如，文件读写或时间和日期之类的操作系统查询）。改由主机应用或测试平台执行这些操作，并将数据作为函数实参传递到函数中。

在 HLS 组件中使用库

Vitis Unified IDE 和 `v++` 通用命令行支持基础 C/C++ 库，支持在 C/C++ 中对常用硬件设计构造和函数轻松建模，并将其综合到 RTL 中。以下库随这些工具一并提供，如 [第六部分：Vitis HLS 库参考资料](#) 中所述：

- [任意精度数据类型库](#)：任意精度数据类型允许您的 C 语言代码使用位宽小于标准 C 语言或 C++ 语言数据类型的变量，从而提升性能并减小硬件中的面积。

- **HLS 数学库**：用于指定标准数学运算，以供综合到 RTL 内和在 AMD 器件上执行实现。
- **HLS 串流库**：用于对串流数据结构进行建模和编译。

您可在代码中包含 C 语言库头文件，这样即可在设计中使用每一种 C 语言库。这些头文件位于 Vitis HLS 安装区域内的 `include` 目录中。



重要提示！ 如果设计是在 AMD Vitis™ Unified IDE 中构建的，那么无需在 `include` 路径中包含库的头文件，因为将自动添加指向库头文件的路径。

Vitis 库

此外，Vitis 加速库可搭配 Vitis Unified IDE 和 `++` 通用命令行一起使用。这些库包含数学、统计数据、线性代数和 DSP 的常用函数，并且还支持特定领域的应用，例如，视觉和图像处理、计量金融、数据库、数据分析以及数据压缩等。这些库可从 https://github.com/Xilinx/Vitis_Libraries 下载，并且可通过设置以供在 Vitis Unified IDE 的“Libraries”（库）视图中使用，如《Vitis 参考指南》(UG1702) 中的 [处理 Vitis Unified IDE](#) 中所述。

这些库包含专为在 AMD 器件上实现而优化的函数和构造。使用这些库有助于确保高质量结果 (QoR)；即，综合结果可以获得资源使用率优化的高性能设计。由于这些库是以 C 语言和 C++ 语言提供的，您可将这些库整合到顶层函数中，并在综合前对其进行仿真以验证功能正确性。



提示： Vitis 应用加速库在 Windows 操作系统上不可用。

处理 OpenCV

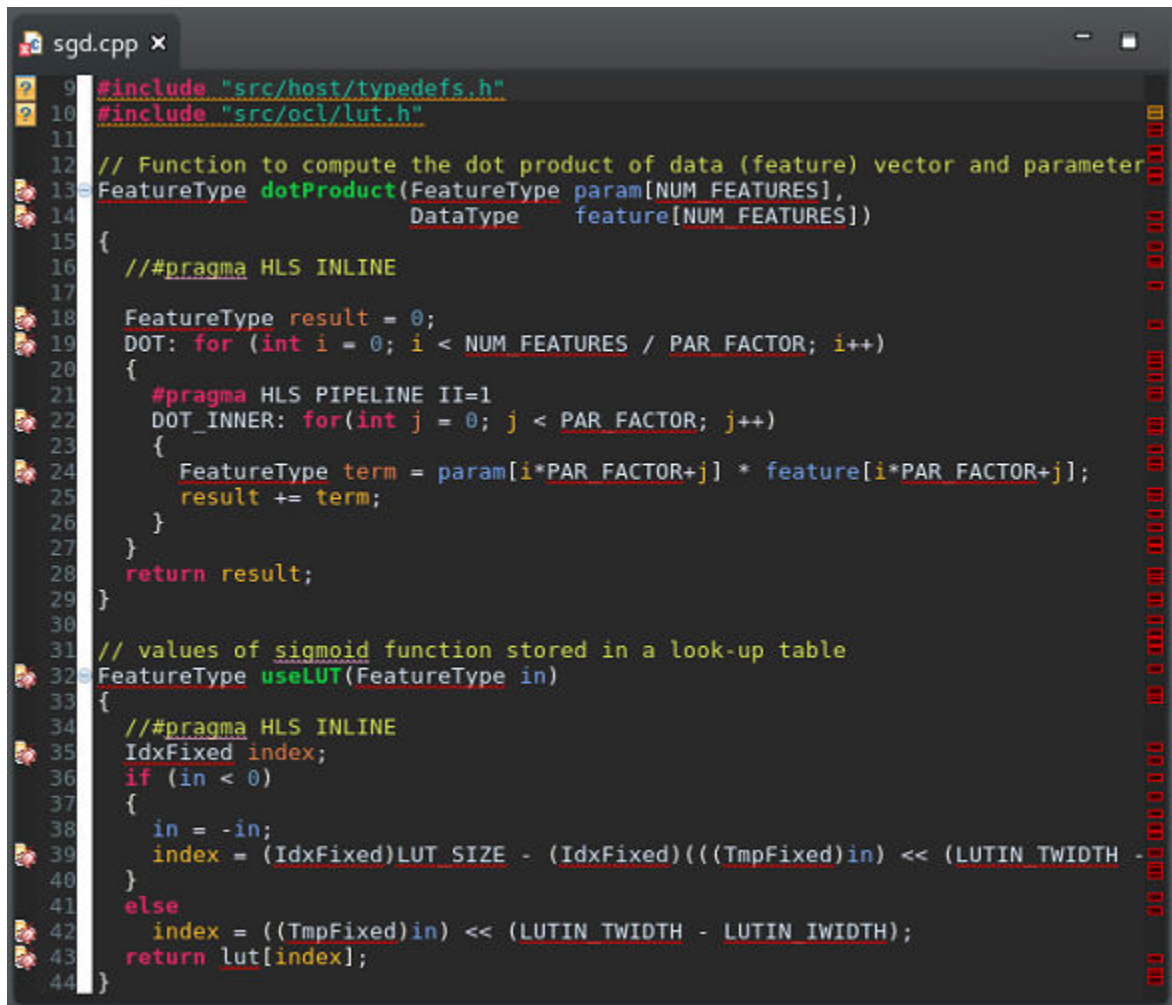
如 https://github.com/Xilinx/Vitis_Libraries/tree/main/vision 中所述，您还必须已安装 OpenCV 计算机视觉库，才能使用 Vitis 视觉库 (Vitis Vision Library)。Vitis HLS 并不含预安装的 OpenCV，如需了解如何查找和安装 OpenCV，请访问以下链接：[Vitis 库 - 编译和安装 OpenCV 库以搭配视觉库一起使用](#)。

解析引用并查看 `#include` 文件

Vitis Unified IDE 会持续解析所有头文件，以解析编码引用。有效的引用当然不仅支持代码正确完成编译，也允许您右键单击 `#include` 语句以使用“Go to Declaration”（转至声明）命令来打开包含的文件。您还可选择函数名称、变量或数据类型，并使用“Go to Definition”（转至定义）命令查看其定义。

GUI 会高亮未解析的引用，如下图所示：

图 90：未解析的引用



- 左侧边栏：在源代码行号处高亮未解析的引用。
- 右侧边栏：显示相对于整个文件的未解析的引用。

如果您的所有工程设置（如 include 路径）都已正确定义，那么默认本工具将为 C 语言源文件自动建立索引。无法解析头文件（扩展名为 .h 或 .hpp）中定义的代码时，就会出现未解析的引用。出现未解析的引用的主要原因包括：

- 代码是最近添加到此文件中的。

如果此代码为新代码，请确保头文件已保存。保存头文件后，本工具会自动为头文件建立索引，并更新代码引用。

- 头文件不在搜索路径中。

请使用 #include 语句确保头文件包含在 C 语言代码中，并且在相同目录中可找到此头文件（作为 C 语言源文件），或者在搜索路径中包含头文件的位置。



提示：要为源文件显式添加搜索路径，请编辑 `syn.cflags` 或 `syn.file_cflags` 以指定要在编译和仿真期间用于源文件的指令，如 [HLS 常规选项](#) 中所述。

解决源代码中的注释

在某些本地化文件中，源文件中的非英语注释显示为怪异的字符。这可使用以下步骤来纠正：

1. 选择“File” → “Preferences” → “Open Settings (UI)”（文件 > 首选项 > 打开设置 (UI)）菜单命令。
2. 在“Preferences”（首选项）视图中，选中“Text Editor”（文本编辑器）标题，然后选中“Files”（文件）。
3. 选中“Encoding”（编码）下拉菜单，然后选择相应的编码。
4. 保存更改。

添加 RTL 黑盒函数

RTL 黑盒支持在 HLS 工程中使用现有 Verilog RTL IP。这样您即可将 RTL 代码添加到 C/C++ 代码中，供 Vitis HLS 用于对工程进行综合。此 RTL IP 可在顺序区域、流水线区域或数据流区域内使用。请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Misc/rtl_as_blackbox](#)，以获取此方法的示例。



提示：在设计中添加 RTL 黑盒将导致工具无法输出 VHDL 代码，因为 RTL 黑盒必须为 Verilog，输出将仅限 Verilog。

将 RTL IP 集成到 Vitis HLS 工程中需要以下文件：

- RTL 代码的 C 语言函数签名。此签名可置于头文件 (.h) 内。
- 黑盒 JSON 描述文件，如 [适用于 RTL 黑盒的 JSON 文件](#) 中所述。
- RTL IP 文件。

要在 HLS 工程中使用 RTL 黑盒，请使用以下步骤。

1. 从顶层函数内或者从 Vitis HLS 工程的子函数内调用 C 语言函数签名。
2. 在 Vitis HLS IDE 中使用“Add Files”（添加文件）命令（如 [创建 HLS 组件](#) 中所述）或者使用 `add_files` 命令将黑盒 JSON 描述文件添加到 HLS 工程中：

```
add_files -blackbox my_file.json
```



提示：如下一节所述，全新的“RTL Blackbox” Wizard 可帮助您生成 JSON 文件并将 RTL IP 添加到工程中。

3. 运行 Vitis HLS 设计文件照常进行仿真、综合和协同仿真。

要求与限制

RTL 黑盒功能特性中使用的 RTL IP 具有下列要求：

- 应为 Verilog (.v) 代码。
- 必须包含唯一的时钟信号和唯一的高电平有效复位信号。
- 必须具有用于启用或停用 RTL IP 的 CE 信号。
- 可使用 `ap_ctrl_chain` 或 `ap_ctrl_none` 块级控制协议，如 [块级控制协议](#) 中所述。

在 Vitis HLS 内，RTL 黑盒功能特性：

- 仅支持 C++。
- 无法连接到顶层接口 I/O 信号。

- 无法直接充当被测设计 (DUT)。
- 不支持 `struct` 或 `class` 类型的接口。
- 对于 `ap_ctrl_none`，支持以下接口协议，如 [适用于 RTL 黑盒的 JSON 文件](#) 中所述：
 - 连线输入和输出：适用于 II=1 的流水线（请参阅 [适用于 RTL 黑盒的 JSON 文件](#) 中的“JSON 文件格式”表中的 `rtl_ports` 属性）
 - FIFO 输入和输出：适用于含串流的数据流
- 对于 `ap_ctrl_chain`，支持以下接口协议，如 [适用于 RTL 黑盒的 JSON 文件](#) 中所述：
 - `hls::stream`：RTL 黑盒 IP 支持 `hls::stream` 接口。在 C 语言函数中使用此数据类型时，请在 RTL 黑盒 IP 中针对该实参使用 FIFO RTL 端口协议。
 - 阵列：RTL 黑盒 IP 针对阵列支持 RAM 接口。对于 C 语言函数中的阵列实参，请将以下 RTL 端口协议之一用于 RTL 黑盒 IP 中的对应实参：
 - 单端口 RAM - `RAM_1P`
 - 双端口 RAM - `RAM_T2P`
 - 标量和输入指针：RTL 黑盒 IP 仅支持在顺序区域和流水线区域中使用 C 语言标量和输入指针。在数据流区域内则不受支持。在 C 语言函数中使用这些构造时，请在 RTL IP 内使用 `wire` 端口协议。
 - 输入输出指针和输出指针：RTL 黑盒 IP 仅支持在顺序区域和流水线区域中使用输入输出指针和输出指针。在数据流区域内则不受支持。在 C 语言函数中使用这些构造时，RTL IP 应针对输出指针使用 `ap_vld`，针对输入输出指针使用 `ap_ovld`。

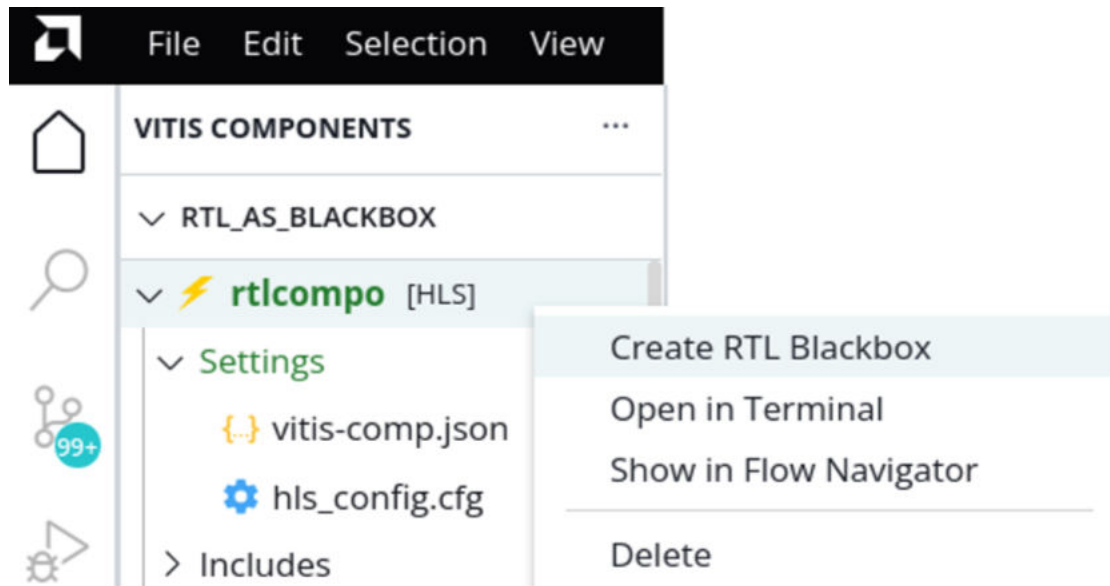


提示：在工程内使用 RTL 黑盒时，所有其他 Vitis HLS 设计限制也都适用。

使用 RTL Blackbox Wizard

导航到工程，右键单击并打开“RTL Blackbox Wizard”（RTL 黑盒向导），如下图所示：

图 91：打开 RTL Blackbox Wizard

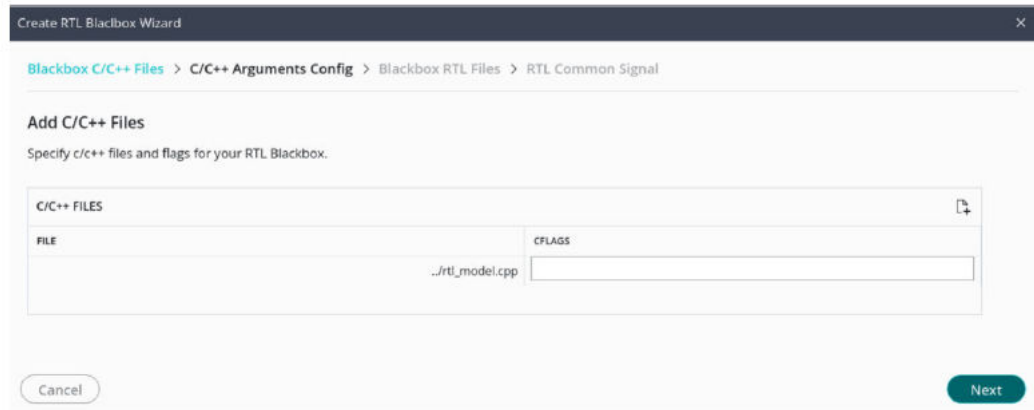


该向导按细分 JSON 文件创建进程的方式来组织其中页面。要在页面之间导航，请单击“Next”（下一步），然后选择“Back”（返回）。最终确定各选项后，即可单击“OK”（确定）生成 JSON。以下每个部分都描述了每个页面及其输入选项。

C++ 语言模型和头文件

在“Blackbox C/C++ files”（黑盒 C/C++ 文件）页面中，提供了 C++ 文件用于构成 RTL IP 的功能模型。此 C++ 模型仅在 C++ 仿真和 C++/RTL 协同仿真期间使用。此 RTL IP 与 Vitis HLS 相结合即可构成综合输出。

图 92：“Blackbox C/C++ Files” 页面



在此页面中，可执行以下操作：

- 单击“Add Files”（添加文件）以添加文件。
- 单击“Edit CFLAGS”（编辑 CFLAGS）以向 C 语言功能模型提供连接器标志。
- 单击“Next”（下一步）以继续。

“C File Wizard”（C 语言文件向导）页面允许您指定用于 RTL IP 的 C 语言功能模型的值。其中字段包括：

- “C Function”（C 语言函数）：用于指定 RTL IP 的 C 语言函数名称。
- “C Argument Name”（C 语言实参名称）：用于指定函数实参的名称。这些名称应与 IP 上的端口相关。
- “C Argument Type”（C 语言实参类型）：用于指定用于每个实参的数据类型。
- “C Port Direction”（C 语言端口方向）：用于指定实参的端口方向，对应于 IP 中的端口。
- “RAM Type”（RAM 类型）：用于指定在接口处使用的 RAM 类型。
- “RTL Group Configuration”（RTL 组配置）：用于指定对应的 RTL 信号名称。

图 93: “C File Wizard” 页面

Create RTL Blackbox Wizard

Blackbox C/C++ Files > C/C++ Arguments Config > Blackbox RTL Files > RTL Common Signal

Config C/C++ Files

Specify c/c++ arguments and rtl ports for your RTL Blackbox.

C Function

rtl_model

Browse

FUNCTION ARGUMENTS DEFINITION

C ARGUMENT NAME	C ARGUMENT TYPE	C PORT DIRECTION	RAM TYPE	RTL GROUPS CONFIGURATION
a4	data_t	in		data_read_in=a44
b1	data_t	in		data_read_in=b1
b2	data_t	in		data_read_in=b2
b3	data_t	in		data_read_in=b3
b4	data_t	in		data_read_in=b4
z1	data_t &	out		data_write_out=z1,data_writ...
z2	data_t &	out		data_write_out=z2,data_writ...
z3	data_t &	out		data_write_out=z3,data_writ...
z4	data_t &	out		data_write_out=z4,data_writ...

Cancel

Back

Next

单击 “Next”（下一步）以继续。

RTL IP 定义

图 94: RTL 黑盒向导

Create RTL Blackbox Wizard

Blackbox C/C++ Files > C/C++ Arguments Config > Blackbox RTL Files > RTL Common Signal

Config RTL Files

Specify RTL files and settings for your RTL Blackbox.

RTL FILES

FILE

../rtl_model.v

RTL Module Name

rtl_model

Performance

Latency

2

II

1

Resource

FF

64

LUT

32

BRAM

0

URAM

0

DSP

1

RTL Wizard 页面允许您为 IP 定义 RTL 源文件。要定义的字段包括：

- “RTL Files”（RTL 文件）：该选项用于添加 RTL IP 文件或移除原先存在的 RTL IP 文件。
- “RTL Module Name”（RTL 模块名称）：在该字段内可指定顶层 RTL IP 模块的名称。
- “Performance”（性能）：该字段用于为 IP 指定性能目标。
- “Latency”（时延）：时延表示设计完成所需的时间。在此字段内可指定时延信息。
- II：该字段用于定义目标启动时间间隔 (II)。II 表示可应用新输入之前的时钟周期数。
- “Resource”（资源）：该字段用于为 RTL IP 指定器件资源使用率。此处提供的资源信息将与来自综合的使用率信息相结合，生成总体设计资源使用率报告。您应从 Vivado Design Suite 提取此信息

单击“Next”（下一步）继续转至“RTL Common Signal”（RTL 常用信号）页面，如下所示。

图 95：RTL 常用信号

RTL Signal	Value
module_clock	ap_clk
module_reset	ap_rst
module_clock_enable	ap_ce
ap_ctrl_chain_protocol_idle	ap_idle
ap_ctrl_chain_protocol_start	ap_start
ap_ctrl_chain_protocol_ready	ap_ready
ap_ctrl_chain_protocol_done	ap_done
ap_ctrl_chain_protocol_continue	ap_continue

- module_clock：指定 RTL IP 内使用的时钟名称。
- module_reset：指定 IP 内使用的复位信号名称。
- module_clock_enable：指定 IP 内时钟使能信号的名称。
- ap_ctrl_chain_protocol_start：指定 IP 内使用的块控制起始信号名称。
- ap_ctrl_chain_protocol_ready：指定 IP 内使用的块控制就绪信号名称。
- ap_ctrl_chain_protocol_done：指定 IP 内使用的块控制完成信号名称。
- ap_ctrl_chain_protocol_continue：指定 RTL IP 内使用的块控制继续信号名称。

单击“Finish”（完成）即可为指定的 IP 自动生成 JSON 文件。可通过如下所示日志消息来确认此文件。

日志消息：

```
"[2019-08-29 16:51:10] RTL Blackbox Wizard Information: the "foo.json" file
has been created in the rtl_blackbox/Source folder."
```

JSON 文件可通过 Source 文件夹来访问，并且将按下一节所述方式生成。

适用于 RTL 黑盒的 JSON 文件

JSON 文件格式

下表描述了 JSON 文件格式：

表 24：JSON 文件格式

项目	JSON 属性	描述
c_function_name		黑盒的 C++ 函数名称。c_function_name 必须与 C 函数仿真模型保持一致。
rtl_top_module_name		黑盒的 RTL 函数名称。rtl_top_module_name 必须与 c_function_name 一致。
c_files	c_file	指定要用于黑盒模块的 C 语言文件。
	cflag	提供对应 C 语言文件所需的任何编译选项。
rtl_files		指定要用于黑盒模块的 RTL 文件。
c_parameters	c_name	指定用于黑盒 C++ 函数的实参名称。 未使用的 c_parameters 字段应从模板中删除。
	c_port_direction	对应 C 语言实参的访问方向。 - in：仅供黑盒 C++ 函数读取。 - out：仅供黑盒 C++ 函数写入。 - inout：将可供黑盒 C++ 函数读写。
	RAM_type	指定对应 C 语言实参使用 RTL 的 RAM 协议时要使用的 RAM 类型。可使用 2 种类型的 RAM： - RAM_1P：用于 1 端口 RAM 模块 - RAM_T2P：用于 2 端口 RAM 模块 当对应 C 语言实参不使用 RTL 的 RAM 协议时，省略该属性。
	rtl_ports	指定对应 C 语言实参 (c_name) 的 RTL 端口协议信号。每个 c_parameter 都应 与 rtl_port 关联。 使用 5 种 RTL 端口协议，ap_ctrl_none 控制协议只支持 wire 和 FIFO。请参阅《RTL 端口协议》表以获取详细信息。 - wire：实参如果是包含“in”或“out”方向的标量，或者是包含“in”方向的指针，那么就能映射到 wire。 - ap_vld：实参如果使用“out”方向的指针，则可映射到 ap_vld。 - ap_ovld：实参如果使用含“inout”方向的指针，则可映射到 ap_ovld。 - FIFO：实参如果使用 hls::stream 数据类型，则可映射到 FIFO。 - RAM：实参如果使用阵列 (array) 类型，则可映射到 RAM。阵列类型支持输入输出方向。 指定的 RTL 端口协议具有关联的控制信号，这些关联信号同样需要在 JSON 文件内指定。

表 24: JSON 文件格式 (续)

项目	JSON 属性	描述
c_return	c_port_direction	必须设置为 out。
	rtl_ports	指定 RTL 黑盒 IP 中使用的对应 RTL 端口名称。
rtl_common_signal	module_clock	RTL 黑盒模块的专用时钟信号。
	module_reset	指定 RTL 黑盒模块的复位信号。复位信号必须处于高电平有效或正电平有效状态。
	module_clock_enable	指定 RTL 黑盒模块的时钟使能信号。使能信号必须处于高电平有效或正电平有效状态。
	ap_ctrl_chain_protocol_idle	RTL 黑盒模块的 ap_ctrl_chain 协议中的 ap_idle 信号。对于 ap_ctrl_none 控制协议，该信号必须为空字符串。
	ap_ctrl_chain_protocol_start	RTL 黑盒模块的 ap_ctrl_chain 协议中的 ap_start 信号。对于 ap_ctrl_none 控制协议，该信号必须为空字符串。
	ap_ctrl_chain_protocol_ready	RTL 黑盒 IP 的 ap_ctrl_chain 协议中的 ap_ready 信号。对于 ap_ctrl_none 控制协议，该信号必须为空字符串。
	ap_ctrl_chain_protocol_done	黑盒 RTL 模块的 ap_ctrl_chain 协议中的 ap_done 信号。对于 ap_ctrl_none 控制协议，该信号必须为空字符串。
	ap_ctrl_chain_protocol_continue	RTL 黑盒模块的 ap_ctrl_chain 协议中的 ap_continue 信号。对于 ap_ctrl_none 控制协议，该信号必须为空字符串。
rtl_performance	latency	指定 RTL 黑盒模块的时延。该值必须为非负整数值。对于组合型 RTL IP，请指定 0，否则请指定 RTL 模块的精确时延。
	II	即“启动时间间隔”；表示函数接受新输入数据之前的时钟周期数。该值必须为非负整数值。0 表示无法对此黑盒进行流水打拍。否则，它表示黑盒模块可流水打拍。
rtl_resource_usage	FF	指定 RTL 黑盒模块的寄存器使用率。
	LUT	指定 RTL 黑盒模块的 LUT 使用率。
	block RAM	指定 RTL 黑盒模块的块 RAM 使用率。
	URAM	指定 RTL 黑盒模块的 URAM 使用率。
	DSP	指定 RTL 黑盒模块的 DSP 使用率。

表 25：RTL 端口协议

RTL 端口协议	C 端口方向	JSON 属性	RTL 端口方向	用户定义的名称	注释
wire	输入	data_read_in	输入	指定 RTL 黑盒 IP 中使用的用户定义的名称。作为对应 wire 的示例，如果 RTL 端口名称为“flag”，则 JSON 文件格式为 "data_read-in" : "flag"。	
ap_vld	输出	data_write_out	输出		
		data_write_valid	输出		
ap_ovld	输入输出	data_read_in	输入		必须处于负电平有效状态。
		data_write_out	输出		
		data_write_valid	输出		
FIFO	输入	FIFO_empty_flag	输入		必须处于负电平有效状态。
		FIFO_read_enable	输出		
		FIFO_data_read_in	输入		
	输出	FIFO_full_flag	输入		必须处于负电平有效状态。
		FIFO_write_enable	输出		
		FIFO_data_write_out	输出		
RAM (RAM_1P)	输入	RAM_address	输出		
		RAM_clock_enable	输出		
		RAM_data_read_in	输入		
	输出	RAM_address	输出		
		RAM_clock_enable	输出		
		RAM_write_enable	输出		
		RAM_data_write_out	输出		
	输入输出	RAM_address	输入		
		RAM_clock_enable	输入		
		RAM_write_enable	输出		
		RAM_data_write_out	输出		
		RAM_data_read_in	输入		

表 25: RTL 端口协议 (续)

RTL 端口协议	C 端口方向	JSON 属性	RTL 端口方向	用户定义的名称	注释
RAM (RAM_T2P)	输入	RAM_address	输出	指定 RTL 黑盒 IP 中使用的用户定义的名称。作为对应 wire 的示例，如果 RTL 端口名称为“flag”，则 JSON 文件格式为 "data_read-in" : "flag"。	含 _snd 的信号属于 RAM 的第 2 端口。不含 _snd 的信号属于第 1 端口。
		RAM_clock_enable	输出		
		RAM_data_read_in	输入		
		RAM_address_snd	输出		
		RAM_clock_enable_snd	输出		
		RAM_data_read_in_snd	输入		
	输出	RAM_address	输出		
		RAM_clock_enable	输出		
		RAM_write_enable	输出		
		RAM_data_write_out	输出		
		RAM_address_snd	输出		
		RAM_clock_enable_snd	输出		
		RAM_write_enable_snd	输出		
		RAM_data_write_out_snd	输出		
	输入输出	RAM_address	输出		
		RAM_clock_enable	输出		
		RAM_write_enable	输出		
		RAM_data_write_out	输出		
		RAM_data_read_in	输入		
		RAM_address_snd	输出		
		RAM_clock_enable_snd	输出		
		RAM_write_enable_snd	输出		
		RAM_data_write_out_snd	输出		
		RAM_data_read_in_snd	输入		

注释：RTL 黑盒的 C 语言函数行为模型必须遵循建议的 HLS 编码样式。

JSON 文件示例

本节提供了有关手动编写 RTL 黑盒所需的 JSON 文件的详细信息。以下是 JSON 文件示例：

```
{
  "c_function_name" : "foo",
  "rtl_top_module_name" : "foo",
  "c_files" :
  [
    {
      "c_file" : "../a/top.cpp",
      "cflag" : ""
    },
    {
      "c_file" : "xx.cpp",
      "cflag" : "-D KF"
    }
  ]
}
```

```

    ],
    "rtl_files" : [
        "../././foo.v",
        "xx.v"
    ],
    "c_parameters" : [{
        "c_name" : "a",
        "c_port_direction" : "in",
        "rtl_ports" : {
            "data_read_in" : "a"
        }
    },
    {
        "c_name" : "b",
        "c_port_direction" : "in",
        "rtl_ports" : {
            "data_read_in" : "b"
        }
    },
    {
        "c_name" : "c",
        "c_port_direction" : "out",
        "rtl_ports" : {
            "data_write_out" : "c",
            "data_write_valid" : "c_ap_vld"
        }
    },
    {
        "c_name" : "d",
        "c_port_direction" : "inout",
        "rtl_ports" : {
            "data_read_in" : "d_i",
            "data_write_out" : "d_o",
            "data_write_valid" : "d_o_ap_vld"
        }
    },
    {
        "c_name" : "e",
        "c_port_direction" : "in",
        "rtl_ports" : {
            "FIFO_empty_flag" : "e_empty_n",
            "FIFO_read_enable" : "e_read",
            "FIFO_data_read_in" : "e"
        }
    },
    {
        "c_name" : "f",
        "c_port_direction" : "out",
        "rtl_ports" : {
            "FIFO_full_flag" : "f_full_n",
            "FIFO_write_enable" : "f_write",
            "FIFO_data_write_out" : "f"
        }
    },
    {
        "c_name" : "g",
        "c_port_direction" : "in",
        "RAM_type" : "RAM_1P",
        "rtl_ports" : {
            "RAM_address" : "g_address0",
            "RAM_clock_enable" : "g_ce0",
            "RAM_data_read_in" : "g_q0"
        }
    }
  ]
}

```

```

},
{
  "c_name" : "h",
  "c_port_direction" : "out",
  "RAM_type" : "RAM_1P",
  "rtl_ports" : {
    "RAM_address" : "h_address0",
    "RAM_clock_enable" : "h_ce0",
    "RAM_write_enable" : "h_we0",
    "RAM_data_write_out" : "h_d0"
  }
},
{
  "c_name" : "i",
  "c_port_direction" : "inout",
  "RAM_type" : "RAM_1P",
  "rtl_ports" : {
    "RAM_address" : "i_address0",
    "RAM_clock_enable" : "i_ce0",
    "RAM_write_enable" : "i_we0",
    "RAM_data_write_out" : "i_d0",
    "RAM_data_read_in" : "i_q0"
  }
},
{
  "c_name" : "j",
  "c_port_direction" : "in",
  "RAM_type" : "RAM_T2P",
  "rtl_ports" : {
    "RAM_address" : "j_address0",
    "RAM_clock_enable" : "j_ce0",
    "RAM_data_read_in" : "j_q0",
    "RAM_address_snd" : "j_address1",
    "RAM_clock_enable_snd" : "j_ce1",
    "RAM_data_read_in_snd" : "j_q1"
  }
},
{
  "c_name" : "k",
  "c_port_direction" : "out",
  "RAM_type" : "RAM_T2P",
  "rtl_ports" : {
    "RAM_address" : "k_address0",
    "RAM_clock_enable" : "k_ce0",
    "RAM_write_enable" : "k_we0",
    "RAM_data_write_out" : "k_d0",
    "RAM_address_snd" : "k_address1",
    "RAM_clock_enable_snd" : "k_ce1",
    "RAM_write_enable_snd" : "k_we1",
    "RAM_data_write_out_snd" : "k_d1"
  }
},
{
  "c_name" : "l",
  "c_port_direction" : "inout",
  "RAM_type" : "RAM_T2P",
  "rtl_ports" : {
    "RAM_address" : "l_address0",
    "RAM_clock_enable" : "l_ce0",
    "RAM_write_enable" : "l_we0",
    "RAM_data_write_out" : "l_d0",
    "RAM_data_read_in" : "l_q0",
    "RAM_address_snd" : "l_address1",

```

```

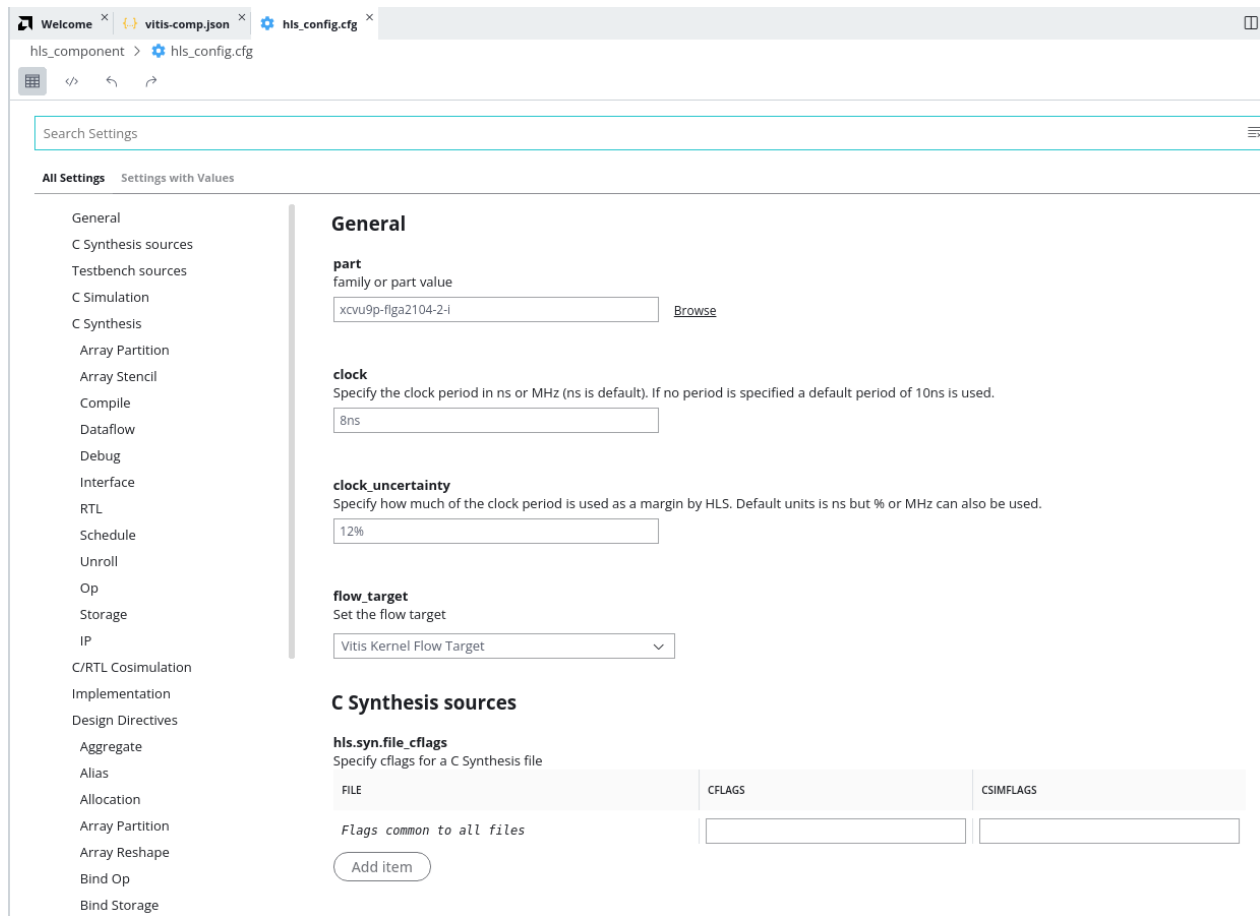
        "RAM_clock_enable_snd" : "l_ce1",
        "RAM_write_enable_snd" : "l_we1",
        "RAM_data_write_out_snd" : "l_d1",
        "RAM_data_read_in_snd" : "l_q1"
    },
    },
    "c_return" : {
        "c_port_direction" : "out",
        "rtl_ports" : {
            "data_write_out" : "ap_return"
        }
    },
    "rtl_common_signal" : {
        "module_clock" : "ap_clk",
        "module_reset" : "ap_rst",
        "module_clock_enable" : "ap_ce",
        "ap_ctrl_chain_protocol_idle" : "ap_idle",
        "ap_ctrl_chain_protocol_start" : "ap_start",
        "ap_ctrl_chain_protocol_ready" : "ap_ready",
        "ap_ctrl_chain_protocol_done" : "ap_done",
        "ap_ctrl_chain_protocol_continue" : "ap_continue"
    },
    "rtl_performance" : {
        "latency" : "6",
        "II" : "2"
    },
    "rtl_resource_usage" : {
        "FF" : "0",
        "LUT" : "0",
        "BRAM" : "0",
        "URAM" : "0",
        "DSP" : "0"
    }
}

```

定义 HLS 配置文件

HLS 组件配置文件包含用于组件综合、仿真和导出的命令和设置。要访问此配置文件，请选中 HLS 组件的 `vitis-comp.json` 选项卡上的“Config File”（配置文件）超链接，或者选中位于 HLS 组件的“Settings”（设置）下的“Vitis Component Explorer”（Vitis 组件资源管理器）中的 `hls-config.cfg`。这将在中央编辑器窗口中打开“Config File Editor”（配置文件编辑器），如下所示。

图 96：HLS 组件配置文件

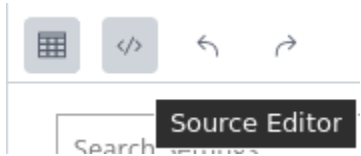


对于 HLS 组件，配置文件包含以下几个部分：

- “General”（常规）：为设计提供常规选项，例如，目标部件、时钟频率和流程目标。如需了解这些选项的描述，请参阅 [HLS 常规选项](#)。
- “C Synthesis sources”（C 语言综合源文件）：指定 HLS 设计的 C/C++ 源文件。如需了解这些文件的描述，同样请参阅 [运行 C 语言综合](#)。
- “Testbench sources”（测试平台源文件）：指定在 C 语言仿真和 C/RTL 协同仿真期间使用的 C/C++ 测试激励文件和关联文件。
- “C Simulation”（C 语言仿真）：指定用于运行 C 语言仿真的编译选项，如 [运行 C 语言仿真](#) 中所述。
- “C Synthesis Settings”（C 语言综合设置）：指定以下类别的全局综合控制：“Array Partition”（阵列分区）、“Array Stencil”（阵列模具）、“Compile”（编译）、“Dataflow”（数据流）、“Debug”（调试）、“Interface”（接口）、“RTL”、“Schedule”（调度）、“Unroll”（展开）、“Op”（运算符）、“Storage”（存储）和“IP”。这些配置设置与 [HLS 配置文件命令](#) 中所述的配置文件命令相关。
- “C/RTL Cosimulation”（C/RTL 协同仿真）：提供编译用于运行 C/RTL 协同仿真的选项，如 [运行 C/RTL 协同仿真](#) 中所述。
- “Package”（封装）：指定选项用于控制 C 语言综合期间生成的 RTL 的导出格式，如 [封装 RTL 设计](#) 中所述。
- “Implementation”（实现）：指定用于控制 Vivado 综合与实现的选项，如 [运行实现](#) 中所述。

- “Design Directives”（设计指令）：这些指令可作为 [HLS 编译指示](#) 直接添加至源代码，或者作为 [HLS 优化指令](#) 添加至配置文件，以供在 HLS 组件中使用。在 “Config File Editor”（配置文件编辑器）中，会列出各种指令，您可选中 “Add Item”（添加项）打开 “Directive”（指令）编辑器，如 [添加编译指示和指令](#) 中所述。
- “Config File Editor”（配置文件编辑器）提供了基于格式的配置文件选项视图和基于文本的编辑器，您可启用后者以便直接编辑配置文件。您可使用编辑器窗口顶部的图标在这两者之间进行切换。

图 97：基于格式的编辑和基于文本的编辑



在 “Config File Editor” 顶部有一个搜索栏，可供您使用关键字或短语快速搜索各选项。您也可以选择仅显示 “Settings with Values”（含有值的设置）以限制显示选项。

**提示：**编辑配置文件后，请记得启用 “File” → “AutoSave”（文件 > 自动保存）功能或者选中 “File” → “Save”（文件 > 保存）命令或 `Ctrl-S` 来保存您的更改。

指定时钟频率

对于 C 和 C++ 设计，仅支持单一时钟。对设计中的所有函数将应用同一个时钟。

创建 HLS 组件时，会以纳秒 (ns) 或 MHz 为单位来指定时钟周期，或者可在 HLS 配置文件中的 “General”（常规）类别下更改此时钟周期，如下所示。

图 98：部件和时钟设置

General

part
family or part value

[Browse](#)

clock
Specify the clock period in ns or MHz (ns is default). If no period is specified a default period of 10ns is used.

clock_uncertainty
Specify how much of the clock period is used as a margin by HLS. Default units is ns but % or MHz can also be used.

默认时钟周期为 10 ns。HLS 组件还使用时钟不确定性概念来提供用户定义的时序裕度，用于在设计内调度进程。您可以在指定时钟的同时，采用同样方法来定义设计的时钟不确定性。默认时钟不确定性如不指定，则为时钟周期的 27%。

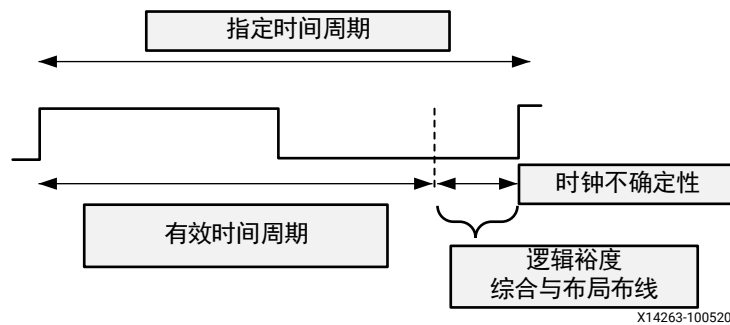


重要提示！ 如果 HLS 配置文件指定 `platform=` 而不是 `part=`，那么您还必须使用 `freqhz=` 代替 `clock=`。这是因为，平台以 `freqhz` 形式指定时钟，您只能使用相同形式来将其覆盖。

该工具通过为 HLS 组件使用时钟频率和部件信息，即可消除设计中的操作时序。但它无从知晓最终组件布局和信号线布线，因为这些操作是由输出 RTL 的逻辑综合来执行的。因此，该工具只能估算延迟。

为计算用于综合的时钟周期，该工具会从时钟周期减去时钟不确定性，如下图所示。

图 99：时钟周期和裕度



这样可为用户提供指定裕度以确保下游进程（例如，逻辑综合与布局布线）具有足够的时序裕度来完成其操作。如果器件资源主要供 HLS 组件使用，那么单元的布局和用于连接各单元的信号线布线可能过于紧密，导致设计延迟超出预期的时序延迟。面对此类情况，通过增加时钟不确定性可避免 HLS 所创建的设计在每个时钟周期内封装过多逻辑，并在布局布线选项不太理想的情况下使 RTL 综合仍能满足时序。

Vitis Unified IDE 和 `v++` 命令会尝试满足所有约束：时序、吞吐量和时延。但如果无法满足约束，那么该工具仍会生成 RTL 设计。如果无法满足时钟周期推断的时序约束，该工具会发出 `SCHED-644` 消息（如下所示）并以可实现的最佳性能来创建设计。

```
@W [SCHED-644] Max operation delay (<operation_name> 2.39ns) exceeds the
effective
cycle time
```



重要提示！ 请务必在综合后复查约束报告以判定是否满足所有约束，这一行为至关重要：实际上虽然综合可以生成输出设计，但并不能保证设计满足所有性能约束。请复查设计报告的“Performance Estimates”（性能估算）部分。

综合完成后，针对层级内每个函数都会生成一份设计报告，并可在解决方案报告文件夹中查看。整个设计的最差情况时序将在各函数报告中报告为最差情况。无需复查层级中的每份报告。

如果时序违例过于严格而无法进一步优化，且下游进程无法对其加以更正，请先复查指定精确的时延和指定精确的实现核的技巧，然后再考虑是否采用更快的目标技术。

时钟和复位端口

如果设计耗时超过 1 个周期才能完成操作，大部分设计都是如此，那么可以选择使用 `syn.interface.clock_enable` 命令将时钟使能端口 (`ap_ce`) 添加到设计的配置文件中，如 [接口配置](#) 中所述。

如需了解有关复位操作的描述，请参阅 [控制初始化与复位行为](#)，复位操作可使用 [RTL 配置](#) 设置来修改。

添加编译指示和指令

HLS 编译指示和指令允许您配置代码的综合结果。

- 在源代码中添加了 [HLS 编译指示](#) 以启用在原始源代码内执行优化或更改。每次综合代码时，都会根据指定编译指示来实现代码。
- [HLS 优化指令](#) 可指定为与 HLS 组件关联的 HLS 配置文件命令。这些配置文件可跨多个 HLS 组件复用，以便您跨不同组件自定义同一代码库的综合结果。

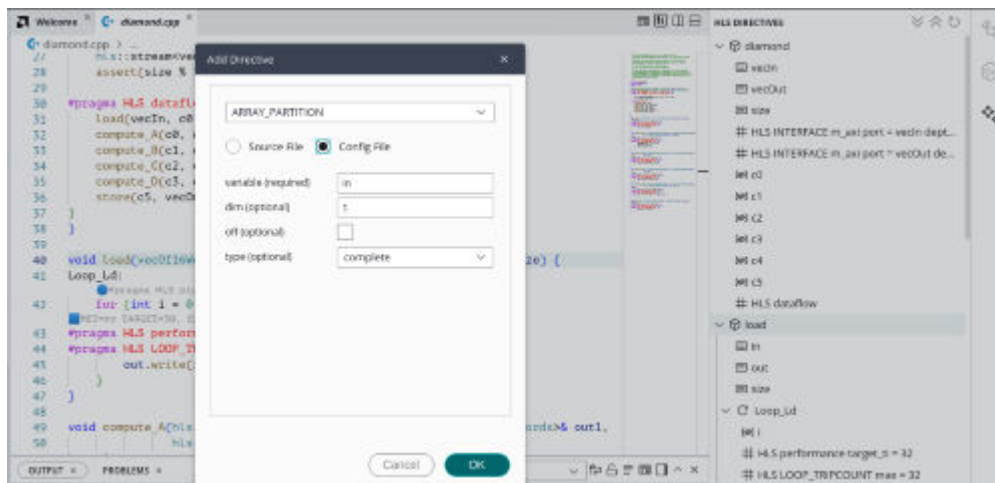


重要提示！ 在某些情况下，如果编译指示或指令与其他编译指示或指令存在冲突，那么综合进程会返回错误，直至冲突得以解决为止。但在某些情况下，遇到的第一条编译指示或指令优先于第二条编译指示或指令，并忽略第二条编译指示或指令。此信息应在 log 日志文件或控制台窗口内予以报告。

要将编译指示或指令添加到工程中，请执行以下操作：

- 在“Vitis Component Explorer”（Vitis 组件资源管理器）中的“Sources”（源文件）文件夹下选中 HLS 组件的源文件以将其打开。
- 打开源文件后，从 Vitis Unified IDE 靠右侧工具栏菜单中选中“HLS Directive”（HLS 指令）命令 ()，以打开“HLS Directive”编辑器，如下图所示。

图 100: “HLS Directive” 编辑器

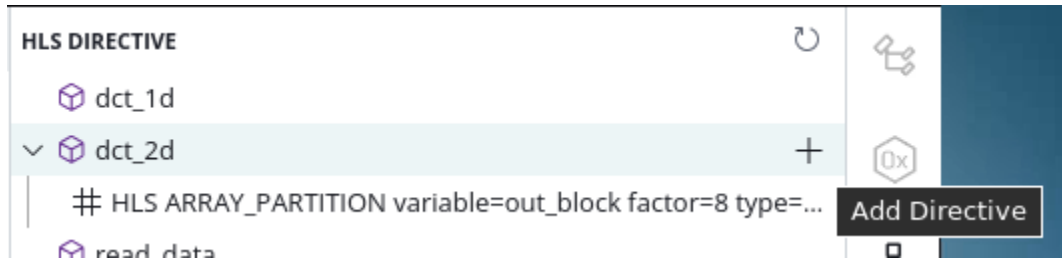


- “HLS Directive”编辑器会显示当前源代码的层级，以便您将指令或编译指示添加到设计中，指令会进入配置文件，编译指示则进入源文件。在上图中，添加到配置文件中的条目是：

```
syn.directive.array_partition=dct_2d in_block factor=8 type=cyclic
```

单击“OK”后，您即可看到它被添加到“Config File”（配置文件）编辑器中。

- 在“HLS Directive”编辑器中选择“Add Directive”（添加指令）命令，方法是选中要将指令添加到其中的函数、循环或变量，然后单击右侧“+”，如下图所示。



选中指令旁的“Edit Directive”（编辑指令）命令即可修改现有指令。

编译指示的条件控制

HLS 编译指示的条件控制支持您在编译时应用各种条件来分析不同编译指示的性能和资源使用情况。您可以利用这一功能特性，根据常量表达式有选择地启用或禁用特定的编译指示，从而更轻松地尝试各种优化策略。

工作原理

- 您可以给编译指示添加 if（条件）子句，将其置于指令名称之前。
- 语法如下：

```
#pragma HLS if(condition) <Pragma> {pragma options}
```

- 条件必须是常量表达式，可以在编译时进行求值。
- 针对 C/C++ 源代码编译指示和 Tcl `set_directive` 命令支持此方法。
- 模板函数条件可以基于类型模板参数或非类型模板参数。

使用预处理器定义

在此示例中，应用的编译指示取决于 `OPT` 宏的值。如果 `OPT` 为 1，则对循环进行流水打拍；如果 `OPT` 为 2，则循环以 16 为因子进行展开。

```
#define OPT 2

int dot_product(int A[SIZE], int B[SIZE]) {
    #pragma HLS INTERFACE mode=ap_memory port=A
    #pragma HLS INTERFACE mode=ap_memory port=B
    #pragma HLS INTERFACE mode=s_axilite port=return

    int result = 0;

    DP_LOOP: for (int i = 0; i < SIZE; i++) {
        #pragma HLS if (OPT==1) PIPELINE II=1
        #pragma HLS if (OPT==2) UNROLL factor=16

        result += A[i] * B[i];
    }
    return result;
}
```

使用模板函数参数

在此示例中，循环中应用的编译指示取决于 TripCount 非类型模板参数。如果 TripCount 大于 20，则使用流水打拍，否则展开循环。

```
template<int TripCount>
void dot_product(const int* a, const int* b, int& output) {
    output = 0;
    DP: for (int i = 0; i < TripCount; ++i) {
        #pragma HLS if (TripCount > 20) pipeline II = 1
        #pragma HLS if (TripCount <=20) unroll
        output += a[i] * b[i];
    }
}

void top(const int* a, const int* b, int& output) {
    #pragma HLS INTERFACE m_axi port=a depth=10 bundle=gmem
    #pragma HLS INTERFACE m_axi port=b depth=10 bundle=gmem
    #pragma HLS INTERFACE m_axi port=output depth=10 bundle=gmem

    dot_product<10>(a, b, output); // Instantiate with TripCount = 30
}
```

使用脚本中的指令对比使用代码中的编译指示

在“HLS Directive”（HLS 指令）编辑器中，您可指定以下任一“Destination”（目标）设置：

- “Source File”（源文件）：Vitis Unified IDE 会将指令作为编译指示直接插入 C 语言源文件。
- “Config File”（配置文件）：IDE 会将指令插入 HLS 配置文件。

下表描述了这 2 种方法的优缺点。

表 26：配置文件指令对比 HLS 编译指示

指令格式	优点	缺点
HLS 配置文件	源代码保持不变，而设计探索则通过配置文件条目来处理。此方法适合设计探索。	如果将 C 语言源文件传输给第三方或存档，那么必须包含 directives.tcl 文件。 如果要重现结果，directives.tcl 文件是必需的。
源代码编译指示	优化指令嵌入 C 语言源代码。 适合将 C 语言源文件作为 C IP 交付给第三方。无需其他文件来重现相同结果。 此方法适用于不太可能改变的指令，如 TRIPCOUNT 和 INTERFACE 等。	如在代码中嵌入优化指令，那么会将这些指令自动应用于引用该源代码的每个 HLS 组件。



提示：您可以使用指令来发掘不同的设计可能性，但在最终确定指令时，最好将其转换为 HLS 编译指示。这样即可包含优化编译指示作为设计源代码的一部分。

指定编译指示实参的值时，可使用字面值（如 1、55 或 3.14），或者使用 #define 来传递宏。以下示例显示了含字面值的编译指示：

```
#pragma HLS ARRAY_PARTITION variable=k_matrix_val type=cyclic factor=5
```

该示例使用已定义的宏：

```
#define E 5  
#pragma HLS ARRAY_PARTITION variable=k_matrix_val type=cyclic factor=E
```

将指令应用于正确的作用域

虽然 Vitis Unified IDE 允许您将指令应用于特定的代码对象，但这些指令会被添加到包含该对象的作用域内。例如，您可将 `INTERFACE` 编译指示应用于 HLS 组件中的接口对象，但该指令将应用于顶层函数（作用域）。接口端口（对象）将在该指令中识别。

您可向以下对象和作用域应用优化指令：

- 函数：向函数应用指令时，HLS 编译器会将该指令应用于函数作用域内的所有对象。任意指令的影响都止于函数层级的下一层，并且不适用于子函数。



提示：包含递归选项的指令（如 `PIPELINE` 指令）则可以递归方式应用于整个层级内。

- 接口：HLS 编译器将指令应用于顶层函数，即包含接口的作用域。
- 循环：指令应用于循环作用域内的所有对象。

例如，如果对循环应用 `LOOP_MERGE` 指令，那么 HLS 编译器会将该指令应用于该循环内的所有子循环，但不会将其应用于该循环本身。指令应用到的循环不会与相同层级内的同辈循环合并。

- 阵列：指令应用于包含阵列的作用域。

将优化指令应用于全局变量

指令只能应用于多个作用域，或某个作用域内的多个对象。因此，无法将其直接应用于在任意函数作用域外声明的全局变量。所以，要将指令应用于全局变量，必须使用如下进程来手动分配该指令：

1. 打开“HLS Directive”（HLS 指令）编辑器并显示源代码
2. 在“Directive”编辑器内选中要使用全局变量的作用域（函数、循环或区域）。
3. 选中“Add Directive”（添加指令），并编辑 `variable` 名称，以将其分配给全局变量。
4. 单击“OK”（确定）以将该指令添加至源代码或配置文件。

对类对象应用优化指令

优化指令同样可应用于类中定义的对象或作用域。差别通常在于类是在头文件中定义的。使用以下任一操作即可打开头文件：

1. 从“Vitis Component Explorer”（组件资源管理器）视图中打开 HLS 组件的 `Includes` 文件夹，然后双击头文件以将其打开。或者，在开源代码文件中，将光标置于头文件的 `#include` 语句中，按住“Ctrl”键并单击头文件以在“Code Editor”中将其打开。
2. 打开“HLS Directive”（HLS 指令）编辑器并显示源代码
3. 在“Directive”编辑器内选中要使用全局变量的作用域（函数、循环或区域）。
4. 选中“Add Directive”（添加指令），并编辑 `variable` 名称，以将其分配给类或变量。
5. 单击“OK”（确定）以将该指令添加至源代码或配置文件。



注意！ 将指令作为编译指示应用于头文件时应谨慎处理。其他人或其他工程可能使用此文件。每次在设计中包含头文件时，都会应用作为编译指示添加的指令。

对模板应用优化指令

使用配置文件命令时，要在模板上手动应用优化指令，请参阅类方法来指定模板实参和类。例如，给定以下 C++ 代码：

```
template <uint32 SIZE, uint32 RATE>
void DES10<SIZE,RATE>::calcRUN() {}
```

使用以下配置文件命令即可在函数上指定 `INLINE` 指令：

```
syn.directive.inline DES10<SIZE,RATE>::calcRUN
```

将常量与编译指示搭配使用

您可将 `const int` 或 `constexpr` 等常量与编译指示或指令搭配使用。例如：

```
const int MY_DEPTH=1024;
#pragma HLS stream variable=my_var depth=MY_DEPTH
```

在 C 语言代码中也可使用宏来实现此功能。使用宏的关键在于使用宏中的层级。这样即可正确执行扩展。代码将按如下方式进行编译：

```
#include <hls_stream.h>
using namespace hls;

#define PRAGMA_SUB(x) _Pragma (#x)
#define PRAGMA_HLS(x) PRAGMA_SUB(x)
#define STREAM_IN_DEPTH 8

void foo (stream<int> &InStream, stream<int> &OutStream) {

// Legal pragmas
PRAGMA_HLS(HLS stream depth=STREAM_IN_DEPTH variable=InStream)
#pragma HLS stream depth=8 variable=OutStream

}
```

无法满足优化指令

应用优化指令时，HLS 编译器会将详述进程的信息输出至控制台（和 log 日志文件）。在以下示例中，PIPELINE 指令已应用于 C 语言函数且 `II=1`（启动时间间隔为 1），但综合未能满足此目标。

```
INFO: [SCHED 11] Starting scheduling ...
INFO: [SCHED 61] Pipelining function 'array_RAM'.
WARNING: [SCHED 63] Unable to schedule the whole 2 cycles 'load' operation
('d_i_load', array_RAM.c:98) on array 'd_i' within the first cycle (II = 1).
WARNING: [SCHED 63] Please consider increasing the target initiation
interval of the
pipeline.
```



```
WARNING: [SCHED 69] Unable to schedule 'load' operation ('idx_load_2',  
array_RAM.c:98) on array 'idx' due to limited memory ports.  
INFO: [SCHED 61] Pipelining result: Target II: 1, Final II: 4, Depth: 6.  
INFO: [SCHED 11] Finished scheduling.
```



重要提示！ 如果 HLS 工具未能满足优化指令，它会自动放宽优化目标，并尝试以更低性能目标来创建设计。如果无法放宽目标，它将停止并报错。

通过尝试创建满足较低优化目标的设计，本工具可提供 3 种重要信息：

- 使用当前 C 语言代码和优化指令可达成的目标性能。
- 无法满足更高性能目标的原因列表。
- 可供分析的设计，以提供更多见解并帮助了解失败原因。

在以上 SCHED-69 消息中，所提供的无法达成目标 II 的原因是由于端口受限。设计必须访问块 RAM，块 RAM 最多只有 2 个端口。

此类失败发生后，下一步是分析问题。在此示例中，即分析代码第 52 行和/或使用“Analysis”（分析）透视图来判定瓶颈以及是否可降低需要 2 个以上端口的要求，或者判定如何增加端口的数量。

设计完成优化并达成期望的性能后，可验证 RTL，并将综合结果封装为 IP。

运行 C 语言仿真

HLS 组件的验证可分为不同的进程，分别发生在高层次综合之前和之后。这些进程包括：C 仿真和代码分析，用于在高层次综合之前确认 C/C++ 源代码；C/RTL 协同仿真，用于确认生成的 RTL 设计。

对 HLS 组件运行 C 语言仿真需要测试平台，如 [撰写测试平台](#) 中所述。测试平台可以自查，并确认要综合的函数的结果是否正确。如果结果正确，测试平台将向 `main()` 返回 0 值。否则，测试平台应返回任何非零值。但在运行仿真前，必须配置 HLS 组件以支持仿真。C 语言仿真可自行独立运行，或者搭配启用的代码分析器来运行。自行运行时，C 语言仿真会使用测试平台和提供的数据来确认源代码逻辑。搭配代码分析器时，仍会完成仿真以确认算法，此外还会对设计的源代码和预期性能进行更深入的分析。代码分析器可让您直观查看任务级并行度的可能性、估算设计的潜在性能，并了解改善设计性能所需的架构更改。

加载测试激励文件

正如 C 语言综合一样，可在创建 HLS 组件时加载仿真所需的源文件。但您可在创建时跳过此步骤，并在运行仿真前添加测试激励文件。也可以更改或添加用于仿真的文件。此操作是在“Config Editor”（配置编辑器）的“Testbench sources”（测试平台源文件）部分中完成的。

图 101: HLS 组件测试平台源文件

“Testbench sources”显示了当前已添加的测试激励文件，并允许您编辑或删除当前文件、添加新文件以及添加或修改测试平台的 CFLAGS。

- “Add item”（添加项）：选择此命令即可向 HLS 组件添加新的测试激励文件。这样会打开“File Browser”（文件浏览器）并允许您导航到并选中要添加的文件。
- “Add CFLAGS”（添加 CFLAGS）：您可添加编译标志 (CFLAGS)，以供应用于所有源文件或者添加到特定源文件。如上图所示，只需将这些标志添加到“Flags common to all files”（所有文件的通用标志）的相应文本输入框中即可。要为特定源文件添加标志，请选中该源文件，然后选中“Edit item”（编辑项）以添加标志或者修改文件名和路径。

这些选项会生成如下配置文件条目：

```
[HLS]
tb.file=../././src/in.dat
tb.file=../././src/out.golden.dat
tb.file=../././src/dct_test.cpp
tb.cflags=<cflag>
tb.file_cflags=../././src/dct_test.cpp,<cflag>
```

配置仿真器

您可在运行仿真前使用“Config Editor”的“C Simulation”（C 语言仿真）部分来配置 C 语言仿真器，如下所示。

图 102: HLS 组件仿真设置

这些配置命令允许您指定仿真的运行方式。配置选项包括：

- `O`：默认情况下，使用调试信息编译设计，以便对编译进行分析和调试。编译设计时，“Optimizing Compile”（优化编译）选项使用更高级别的优化，但不会添加调试器所需的任何信息。这会增加编译时间，但会减少仿真运行时间。
- `argv`：指定测试平台 `main()` 函数所需的任意输入。
- `clean`：该选项用于在编译代码之前，从工程中移除所有现有的可执行文件和对象文件。
- `code_analyzer`：启用 Code Analyzer 工具，如 [使用代码分析器](#) 中所述。
- `ldflags`：指定传递给连接器以进行语言仿真的选项。该选项通常用于为 C/C++ 语言测试平台传递 include 路径信息或库信息。
- `mflags`：提供要传递给编译器以进行 C 语言仿真的选项。通常用于加快编译速度。
- `setup`：指定该选项时，将在当前 HLS 组件的 `csim` 目录中创建仿真二进制文件，但不执行仿真。稍后，可从已编译的可执行文件启动仿真。默认为 `false`，完成设置后运行仿真。

这些选项会生成如下示例所示配置文件条目：

```
[HLS]
csim.clean=true
csim.O=true
csim.code_analyzer=0
csim.argv=arg1 arg2
```

运行仿真

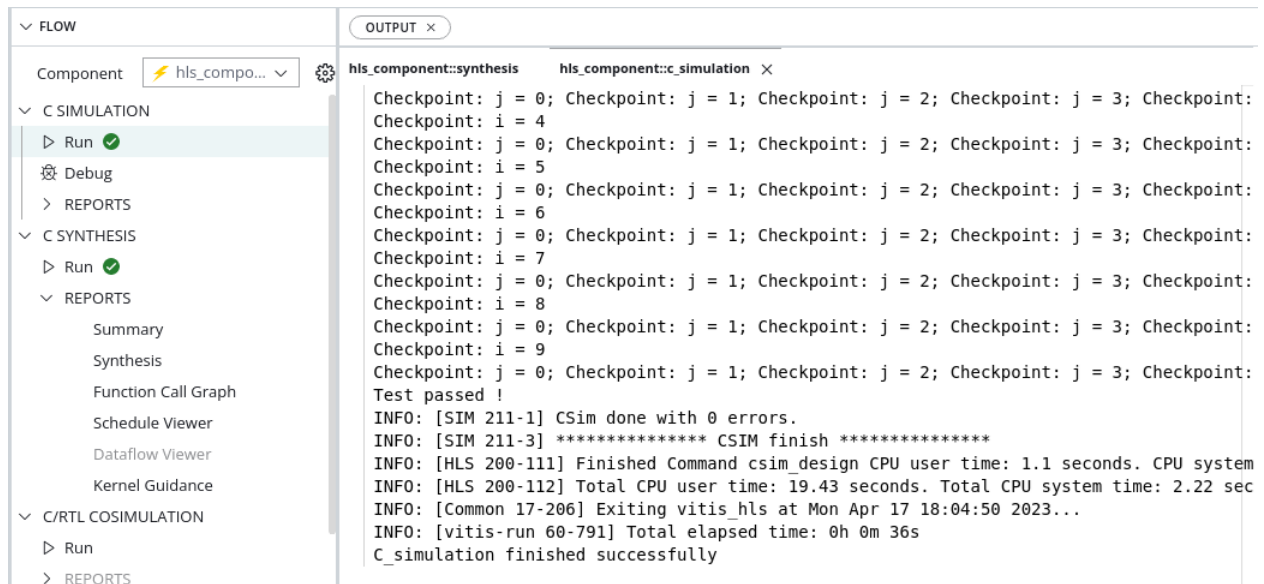
要运行仿真，请在“Flow Navigator”中，确保 HLS 组件处于活动状态，或者在“Component”（组件）菜单中将其选中，在该工具中将其设为活动组件。当 HLS 组件为活动组件时，“Flow Navigator”支持运行 C 语言综合、C 语言仿真、C/RTL 协同仿真、封装与实现来构建和分析该 HLS 组件。要仿真 HLS 组件，请选中“Flow Navigator”的“C SIMULATION”（C 语言仿真）标题下的“Run”（运行）。



提示：您还可从“Flow Navigator”选中“Debug”（调试）命令以启动调试进程，如 [调试 HLS 组件](#) 中所述。

您可在“Output”（输出）窗口中跟踪仿真进度。综合运行的转录文本所含顶层函数名为 `<component-name>::c_simulation`，如下所示。

图 103: 运行仿真的 HLS 组件



提示： Vitis IDE 中的仿真运行使用的是 `vitis-run --mode hls --csim` 命令，如 [vitis](#)、[v++](#) 和 [vitis-run](#) 命令中所述。

C 语言仿真结果

在此对话框中单击“OK”后，就会编译 C/C++ 语言代码并运行 C 语言仿真。运行仿真时，该控制台会显示来自测试平台的任何 `printf` 语句或者来自内核或 IP 的 `hls::print` 语句。仿真成功完成后，还会向控制台返回以下消息：

```
INFO: [SIM 211-1] CSim done with 0 errors.
INFO: [SIM 211-3] ***** CSIM finish *****
Finished C simulation.
```

仿真失败时，则返回错误：

```
@E Simulation failed: Function 'main' returns nonzero value '1'.
ERROR: [SIM 211-100] 'csim_design' failed: nonzero return value.
INFO: [SIM 211-3] ***** CSIM finish *****
```

仿真完成后，会生成“Summary”（汇总）报告，以显示仿真命令和运行的时间戳。您可从“Report”（报告）标题中选中“Summary”（汇总）报告和仿真 log 日志文件来查看此报告和日志，也可在“Analysis”（分析）视图中查看。如果先前已运行 Code Analyzer，那么也将显示其报告。

撰写测试平台

使用 HLS 组件流程时，对错误编码的 C/C++ 语言函数进行综合，然后分析实现详细信息，以判定此函数功能与期望不符的原因，这个过程较为耗时。因此，高层次综合中的第一步应该是验证 C 语言函数是否正确，再生成 RTL 代码，然后使用精心编写的测试平台执行仿真。准确编写测试平台有助于显著提升工作效率，因为 C 语言函数的执行速度比 RTL 仿真快数个量级。与开发和调试 RTL 代码相比，在综合之前使用 C/C++ 语言来开发和确认算法要快得多。

测试平台包括 `main()` 函数以及任何其他必要的子函数，这些子函数不包含在综合所指定的顶层函数的层级内。`main` 函数会验证综合的顶层函数是否正确，方法是从测试平台提供激励并调用该函数，然后使用并确认其输出。



重要提示！ 测试平台可接受输入实参，这些实参可在启动 C 语言仿真时提供，如 [运行 C 语言仿真](#) 中所述。但测试平台执行期间不得要求交互式用户输入。HLS 工具无法与仿真器进行交互，故而在执行测试平台时不能接受用户输入。

例如，以下代码显示了自检测测试平台的重要功能：

```
int main () {
    //Establish an initial return value. 0 = success
    int ret=0;

    // Call any preliminary functions required to prepare input for the test.

    // Call the top-level function multiple times, passing input stimuli as
    // needed.
    for(i=0; i<NUM_TRANS; i++){
        top_func(input, output);
    }

    // Capture the output results of the function, write to a file

    // Compare the results of the function against expected results
    ret = system("diff --brief -w output.dat output.golden.dat");

    if (ret != 0) {
        printf("Test failed  !!!\n");
        ret=1;
    } else {
        printf("Test passed !\n");
    }

    return ret;
}
```

测试平台应针对多项传输事务执行顶层函数，以便尽可能应用和验证更多不同的数据值。测试平台的效率受到所执行的测试类型的限制。此外，如果您想要在 RTL 仿真期间计算 II，那么您的测试平台必须向综合后的函数提供重复的调用，如 [运行 C/RTL 协同仿真](#) 中所述。

以上自检测测试平台会将函数 `output.dat` 的结果与 `output.golden.dat` 中已知良好的结果进行比对。这只是自检测测试平台的示例之一。还有多种方法可用于确认顶层函数，您必须根据自己的代码来对测试平台进行适当的编码。

在 HLS 组件流程中，函数 `main()` 的返回值的指示如下：

- 0 值：结果正确。
- 非 0 值：结果不正确。

测试平台可返回任意非 0 值。复杂的测试平台可根据失败类型返回不同的值。如果测试平台在 C 语言仿真或 C/RTL 协同仿真后返回非 0 值，那么该工具会报告错误和仿真失败。



提示： 由于系统环境（例如，Linux、Windows 或 Tcl）可解读 `main()` 函数的返回值，因此建议将返回值的范围约束为 8 位以保障可移植性和安全性。

当然，仿真结果的准确性受到您提供的测试平台所限。您负责确保测试平台返回的结果正确无误。如果测试平台返回 0，那么无论仿真期间发生了什么，该工具都会指示仿真已通过。

示例测试平台

AMD 建议您将顶层综合函数与测试平台分开，并使用头文件。在以下代码示例显示的设计中，HLS 工程的顶层函数 `hier_func` 会调用 2 个子函数：

- `sumsub_func` 执行加法和减法。
- `shift_func` 执行移位。

数据类型在头文件 (`hier_func.h`) 中定义。此函数的代码如下所示：

```
#include "hier_func.h"

int sumsub_func(dint_t *in1, dint_t *in2, dint_t *outSum, dint_t *outSub)
{
    *outSum = *in1 + *in2;
    *outSub = *in1 - *in2;
}

int shift_func(dint_t *in1, dint_t *in2, dout_t *outA, dout_t *outB)
{
    *outA = *in1 >> 1;
    *outB = *in2 >> 2;
}

void hier_func(dint_t A, dint_t B, dout_t *C, dout_t *D)
{
    dint_t apb, amb;

    sumsub_func(&A, &B, &apb, &amb);
    shift_func(&apb, &amb, C, D);
}
```

如上所示，该顶层函数可包含多个子函数。综合仅限于单个顶层函数。要对多个函数进行综合，请将这些函数组合为单个顶层函数的子函数。

如下所示，头文件 (`hier_func.h`) 演示了如何使用宏以及如何通过 `typedef` 语句提升代码可移植性和可读性。



提示：任意精度 (AP) 数据类型 探讨了任意精度数据类型相关内容，以及如何使用 `typedef` 语句来优化变量类型从而优化变量位宽，并在最终的 FPGA 实现中改善面积和性能。

```
#ifndef _HIER_FUNC_H_
#define _HIER_FUNC_H_

#include <stdio.h>

#define NUM_TRANS 40

typedef int din_t;
typedef int dint_t;
typedef int dout_t;

void hier_func(dint_t A, dint_t B, dout_t *C, dout_t *D);

#endif
```

以上头文件包含 `hier_func` 函数不需要的部分 `#define` 语句（如 `NUM_TRANS`），这些语句是为包含相同头文件的测试平台提供的。

以下代码用于为 `hier_func` 设计定义测试平台：

```
#include "hier_func.h"

int main() {
    // Data storage
    int a[NUM_TRANS], b[NUM_TRANS];
    int c_expected[NUM_TRANS], d_expected[NUM_TRANS];
    int c[NUM_TRANS], d[NUM_TRANS];

    //Function data (to/from function)
    int a_actual, b_actual;
    int c_actual, d_actual;

    // Misc
    int retval=0, i, i_trans, tmp;
    FILE *fp;

    // Load input data from files
    fp=fopen(tb_data/inA.dat,r);
    for (i=0; i<NUM_TRANS; i++){
        fscanf(fp, %d, &tmp);
        a[i] = tmp;
    }
    fclose(fp);

    fp=fopen(tb_data/inB.dat,r);
    for (i=0; i<NUM_TRANS; i++){
        fscanf(fp, %d, &tmp);
        b[i] = tmp;
    }
    fclose(fp);

    // Execute the function multiple times (multiple transactions)
    for(i_trans=0; i_trans<NUM_TRANS-1; i_trans++){

        //Apply next data values
        a_actual = a[i_trans];
        b_actual = b[i_trans];

        hier_func(a_actual, b_actual, &c_actual, &d_actual);

        //Store outputs
        c[i_trans] = c_actual;
        d[i_trans] = d_actual;
    }

    // Load expected output data from files
    fp=fopen(tb_data/outC.golden.dat,r);
    for (i=0; i<NUM_TRANS; i++){
        fscanf(fp, %d, &tmp);
        c_expected[i] = tmp;
    }
    fclose(fp);

    fp=fopen(tb_data/outD.golden.dat,r);
    for (i=0; i<NUM_TRANS; i++){
        fscanf(fp, %d, &tmp);
        d_expected[i] = tmp;
    }
    fclose(fp);

    // Check outputs against expected
```

```

for (i = 0; i < NUM_TRANS-1; ++i) {
    if(c[i] != c_expected[i]){
        retval = 1;
    }
    if(d[i] != d_expected[i]){
        retval = 1;
    }
}

// Print Results
if(retval == 0){
    printf(    *** *** *** *** \n);
    printf(    Results are good \n);
    printf(    *** *** *** *** \n);
} else {
    printf(    *** *** *** *** \n);
    printf(    Mismatch: retval=%d \n, retval);
    printf(    *** *** *** *** \n);
}

// Return 0 if outputs are corre
return retval;
}

```

设计文件和测试激励文件

由于 HLS 工具复用 C 语言测试平台进行 RTL 验证，因此要求在将测试平台和任何关联文件添加到 HLS 组件时，将这些文件全部标记为测试激励文件。符合下列条件的任意文件均为测试平台的关联文件：

- 供测试平台访问的文件。
- 测试平台正常运行所需的文件。

测试平台示例中，此类文件示例包括数据文件 `inA.dat` 和 `inB.dat`。您必须将这些文件作为测试激励文件添加到 HLS 配置文件中。

在 HLS 组件中识别测试激励文件的要求并不强制要求设计文件与测试平台分为不同文件（虽然推荐采用不同文件）。作为演示，此处使用 [示例测试平台](#) 中所用的相同代码定义了一个新示例，唯一区别是定义了新的顶层函数。在此示例中，函数 `sumsub_func` 被定义为 HLS 组件中的顶层函数。



提示：您可在 HLS 配置文件中更改顶层函数，方法是选中“C Synthesis”（C 语言综合）设置或者使用 `syn.top` 配置文件条目。

将 `sumsub_func` 函数定义为顶层函数后，更高层次的函数 `hier_func` 将包含到测试平台内，因为它是对应 `sumsub_func` 的调用函数。同层的 `shift_func` 函数现在同样已包含在测试平台内，因为它是测试必要的组成部分。即使这些函数与顶层 `sumsub_func` 函数包含在同一个代码文件内，但同样包含在测试平台内。

单文件测试平台和设计

您还可将设计与测试平台包含到同一个设计文件内。以下示例所含 `hier_func` 函数与 [示例测试平台](#) 中相同，但所有一切都通过单一文件来编码，包括：测试平台的顶层函数、子函数和 `main` 函数。



重要提示! 要在单个文件中同时包含测试平台和设计, 您必须将该文件同时作为综合源文件和测试激励文件添加到 HLS 配置中。

```
#include <stdio.h>

#define NUM_TRANS 40

typedef int din_t;
typedef int dint_t;
typedef int dout_t;

int sumsub_func(din_t *in1, din_t *in2, dint_t *outSum, dint_t *outSub)
{
    *outSum = *in1 + *in2;
    *outSub = *in1 - *in2;
}

int shift_func(dint_t *in1, dint_t *in2, dout_t *outA, dout_t *outB)
{
    *outA = *in1 >> 1;
    *outB = *in2 >> 2;
}

void hier_func(din_t A, din_t B, dout_t *C, dout_t *D)
{
    dint_t apb, amb;

    sumsub_func(&A,&B,&apb,&amb);
    shift_func(&apb,&amb,C,D);
}

int main() {
    // Data storage
    int a[NUM_TRANS], b[NUM_TRANS];
    int c_expected[NUM_TRANS], d_expected[NUM_TRANS];
    int c[NUM_TRANS], d[NUM_TRANS];

    //Function data (to/from function)
    int a_actual, b_actual;
    int c_actual, d_actual;

    // Misc
    int retval=0, i, i_trans, tmp;
    FILE *fp;
    // Load input data from files
    fp=fopen(tb_data/inA.dat,r);
    for (i=0; i<NUM_TRANS; i++){
        fscanf(fp, %d, &tmp);
        a[i] = tmp;
    }
    fclose(fp);

    fp=fopen(tb_data/inB.dat,r);
    for (i=0; i<NUM_TRANS; i++){
        fscanf(fp, %d, &tmp);
        b[i] = tmp;
    }
    fclose(fp);

    // Execute the function multiple times (multiple transactions)
    for(i_trans=0; i_trans<NUM_TRANS-1; i_trans++){
```



```

//Apply next data values
a_actual = a[i_trans];
b_actual = b[i_trans];

hier_func(a_actual, b_actual, &c_actual, &d_actual);

//Store outputs
c[i_trans] = c_actual;
d[i_trans] = d_actual;
}

// Load expected output data from files
fp=fopen(tb_data/outC.golden.dat,r);
for (i=0; i<NUM_TRANS; i++){
fscanf(fp, %d, &tmp);
c_expected[i] = tmp;
}
fclose(fp);

fp=fopen(tb_data/outD.golden.dat,r);
for (i=0; i<NUM_TRANS; i++){
fscanf(fp, %d, &tmp);
d_expected[i] = tmp;
}
fclose(fp);

// Check outputs against expected
for (i = 0; i < NUM_TRANS-1; ++i) {
if(c[i] != c_expected[i]){
retval = 1;
}
if(d[i] != d_expected[i]){
retval = 1;
}
}

// Print Results
if(retval == 0){
printf(    *** *** *** *** \n);
printf(    Results are good \n);
printf(    *** *** *** *** \n);
} else {
printf(    *** *** *** *** \n);
printf(    Mismatch: retval=%d \n, retval);
printf(    *** *** *** *** \n);
}

// Return 0 if outputs are correct
return retval;
}

```

C 语言仿真期间可用的清理器

现在，C 语言仿真期间有一项新功能可用：清理器。清理器可帮助识别代码（测试平台和设计）中未定义的行为。如果代码正确，但在 C/C++ 语言规范中指定代码包含不可预测的效果，那么就会出现未定义行为。如果设计代码可能触发未定义的行为，那么综合后的硬件执行计算时就很可能出错。C/C++ 中的未定义的行为示例包括整数上溢或阵列访问出界。

Vitis HLS 提供了 2 种清理器用于识别 HLS 组件中的未定义行为。

- “Address Sanitizer”（地址清理器）能在运行 C 语言仿真时识别未定义的存储器访问。

- “Undefined-behavior Sanitizer”（未定义行为清理器）可识别 C 语言仿真期间出现的与地址无关的其他未定义行为。

运行 C 语言仿真之前，可在 HLS 工程选项中使用 GUI（需要截屏）或者使用以下 Tcl 命令启用这两种清理器：
`config\_csim -sanitize\_address=true` 和 `config\_csim -sanitize\_undefined=true`。

在某些情况中，这些清理器会在控制台中生成栈追踪以指出问题所在。如下示例所示：

```
AddressSanitizer:DEADLYSIGNAL
=====
==44041==ERROR: AddressSanitizer: SEGV on unknown address 0x0000000000c0
(pc 0x7ff6d5c4c284 bp 0x7ffeccaald80 sp 0x7ffeccaalaa0 T0)
==44041==The signal is caused by a READ memory access.
==44041==Hint: address points to the zero page.
#0 0x7ff6d5c4c284 in _IO_vfscanf (/lib64/libc.so.6+0x59284) (BuildId:
398944d32cf16a67af51067a326e6c0cc14f90ed)
#1 0x274110 in __interceptor_vfscanf /path/compiler-rt/lib/asan/./
sanitizer_common/sanitizer_common_interceptors.inc:1523:1
#2 0x274110 in fscanf /path/compiler-rt/lib/asan/./sanitizer_common/
sanitizer_common_interceptors.inc:1541:1
#3 0x313c74 in main /project/hls/csim/build/../../../../testbench.cpp:36:3
#4 0x7ff6d5c15504 in __libc_start_main (/lib64/libc.so.6+0x22504)
(BuildId: 398944d32cf16a67af51067a326e6c0cc14f90ed)
#5 0x253198 in _start (/project/hls/csim/build/csim.exe+0x253198)

AddressSanitizer can not provide additional info.
SUMMARY: AddressSanitizer: SEGV (/lib64/libc.so.6+0x59284) (BuildId:
398944d32cf16a67af51067a326e6c0cc14f90ed) in _IO_vfscanf
==44041==ABORTING
```

在此示例中，开头带有 # 和数字的行表示从 0（最深层次的函数调用）开始的栈条目到列表中的最后一个条目（您的应用中的初始函数调用）。通常，问题都与发生错误时 HLS 工程中最深的函数调用有关。要找到有问题的源代码行，您可以从 #0 开始向下追踪，识别属于您的设计或测试平台的第一行代码。例如，在执行测试激励文件第 36 行（栈追踪中的第 3 个条目）时发生了分段错误，应对其进行检查。

C 语言仿真输出

在 Vitis Unified IDE 中，HLS 组件层级的结构如下：<workspace>/<component>/<component>/hls/。您可在“Vitis Component Explorer”（Vitis 组件资源管理器）中选中 HLS 组件，然后从右键单击菜单中选中“Open In Terminal”（在终端中打开），以浏览组件文件夹的内容。

C 语言仿真完成后，就会在 <component>/hls 文件夹内创建 csim 文件夹。该文件夹包含以下元素：

- csim/build：C 语言仿真相关的所有文件的主要位置
 - 测试平台所读取的所有文件都会复制到此文件夹中。
 - 在此文件夹中创建并运行 C 语言可执行文件 csim.exe。
 - 测试平台写入的所有文件都是在此文件夹中创建的。
 - csim/obj：包含用于已编译的源代码的对象文件 (.o) 和用于源代码构建的依赖关系文件 (.d)。
- csim/report：包含用于 C 语言仿真构建和运行的 log 日志文件。

使用代码分析器

要在高层次综合 (HLS) 中达成最佳结果，通常需要更改代码以改善设计的宏架构。为助您实现此目标，Vitis Unified IDE Code Analyzer（代码分析器）提供了各种功能特性，可支持您直观查看任务级并行度的可行性，了解性能优化所需的架构更改。

您可在 C 语言仿真步骤中运行代码分析器，具体方法是在 HLS 配置文件中启用 `csim.code_analyzer` 命令，如 [C 语言仿真配置](#) 所述。运行代码分析器后，会在“Flow Navigator”的 C 语言仿真步骤中的“Reports”（报告）下或者在 Vitis Unified IDE 的“Analysis”（分析）视图中显示“Code Analyzer”报告。代码分析器的功能特性包括：

- 数据流计算图抽取：代码分析器报告会抽取数据流计算图 (DFG)，其中顶层语句会变为数据流进程（DFG 节点），这些进程的数据依赖关系会变为数据流通道（DFG 边沿）。计算图可从函数或循环体（即使这两者并非数据流）生成，以帮助更准确判断如何以数据流形式重写代码，如 [HLS 抽象并行编程模型](#) 所述。
- 性能指标：数据量、传输事务时间间隔 (TI) 和吞吐量等性能指标由代码分析器来确定。数据量和访问模式可通过 C 语言测试平台根据剖析信息来确定。例如，该工具可确定变量 A 推断端口位宽为 32 位，进程 1 先写入 8 Kb 的数据，然后进程 2 读取这些数据。代码分析器会在综合之前对设计进行分析，估算传输事务时间间隔或数据传输完成所需的时间，以及通道的吞吐量。不过，由于估算发生在综合之前，因此其准确性不及根据已综合的 RTL 的性能计算得出的结果。
- 性能指南：识别 HLS 组件源代码可能存在的阻碍性能的主要因素。这些阻碍因素包括循环依赖关系和存储器端口争用。性能指南可帮助您了解可能限制设计性能的代码结构，或者识别哪些指标能助您了解设计可达成的性能级别。
- 计算图变换：根据数据流计算图分解以及测量和估算指标，您可能会判断计算图并非所见般理想。您可修改计算图，包括合并进程以执行假设类型的设计探索。计算图修改后，可根据新的架构判断新的性能指标。这种设计进程的迭代可以产生一个理想架构的解决方案蓝图，供您作为重构源代码的基础。

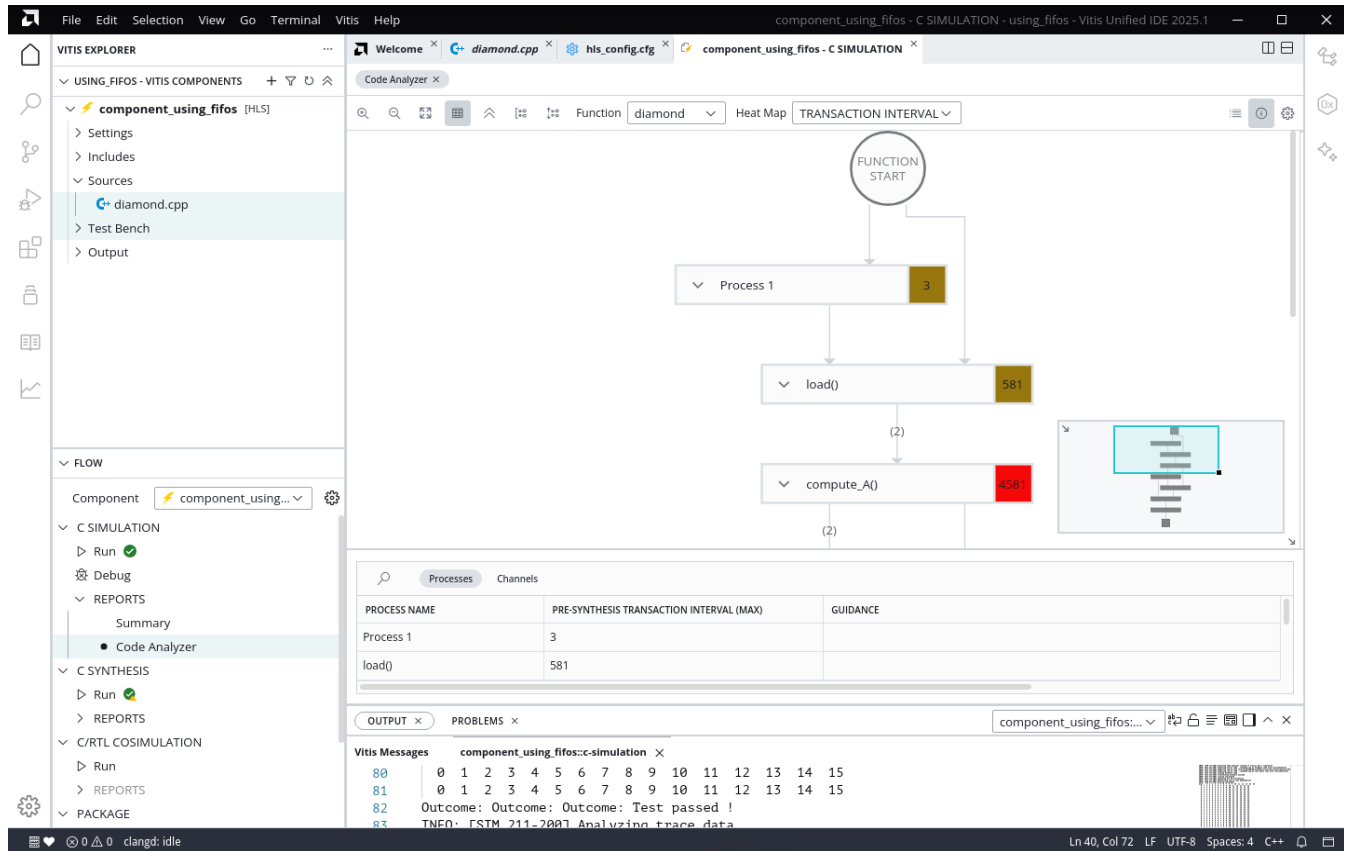


重要提示！ 在代码分析器报告中可以合并和拆分代码，但您要沿用的任何更改都需要在原始设计源码中手动重新实现。

使用代码分析器报告

启用代码分析器并运行 C 语言仿真命令后，就会生成代码分析器报告，此报告在 Vitis Unified IDE 的“Analysis”视图或“Flow Navigator”的“Reports”报头下查看。代码分析器报告最初会显示组件顶层函数定义的进程和通道计算图，如下例所示。您可使用工具栏菜单中的“Function”（函数）选择器或者单击计算图中已展开的进程中的向右箭头来更改报告范围。

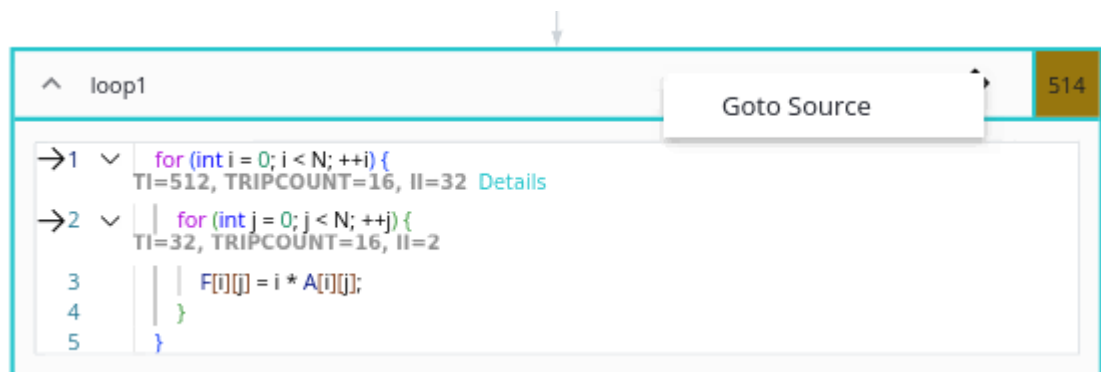
图 104：代码分析器报告



代码分析器报告的功能特性如下：

- 计算图：报告中的“计算图”视图以数据流计算图的形式显示设计的进程和通道。这样即可推断是否存在 DATAFLOW 编译指示或指令，即使源代码中不存在该编译指示或指令也是如此。每个进程都会显示该元素的传输事务时间间隔和性能指南，并包含该元素的源代码，可通过展开代码视图查看。以下示例显示了综合前分析期间估算的性能。数据流进程（计算图节点）的 TI 显示在右上角的黄色/红色框中。形成数据流进程的代码中的函数调用和循环也会在调用或循环报头后紧接着显示一些性能指标，其中可包含详情链接，并提供可识别阻碍该循环或函数调用性能的因素的额外指导信息。

图 105：计算图视图



此外，您可右键单击进程报头，并选中“Goto Source”（转至源码）命令以打开源文件并高亮其中的源码。

- 表：在计算图下方，“Code Analyzer”报告会显示一个表格，其中有两个选项卡：“Processes”（进程）和“Channels”（通道）。它提供了不同要素的快速汇总，以便您在一张表中查看分析结果。
 - “Processes”会显示计算图的进程，还包括估算的综合前传输事务时间间隔 (TI)，并提供分析可能生成的任何设计指导信息。
 - “Channels”会在单独一行中显示进出每个进程的数据流通道。通道以定义通道的变量命名，其中包含变量声明的详细信息，例如，位宽、预计通过该通道交付的数据量、预计的吞吐量、访问模式以及生产者和使用者任务或进程。
- 工具栏：“Code Analyzer”报告的工具栏菜单提供了多项命令用于帮助配置和查看报告。



上图从左到右显示了以下命令：

- “Zoom In/Zoom Out/Zoom Fit”（放大/缩小/缩放适应）：按需放大计算图。
- “Toggle Table”（切换表格）：显示或隐藏进程和通道表。这样即可按需为计算图腾出空间。
- “Collapse All”（全部折叠）：关闭计算图中所有已展开的进程。
- “Group All/Ungroup All”（全部组合/全部取消组合）：将含有相同源和目标的通道加以组合或取消组合。
- “Function”（函数）：为当前计算图提供上下文。可通过以下方法更改上下文：从列表中选择一个新函数，或者单击进程代码中循环和函数调用旁边的箭头。
- “Heat Map”（热图）：将计算图的报告数据指定为循环或进程的传输事务时间间隔 (TI) 或性能指导消息。就 TI 而言，最大的结果以红色高亮显示，表明这些进程最慢，会限制整个数据流区域的性能。
- “Properties”（属性）：显示或隐藏显示性能瓶颈的面板。面板内容通过单击进程代码中的“Details”链接来设置。
- “Info”（信息）：提供与工具使用、返回的指标或设计常见附加说明相关的信息。这些信息值得不时查阅。
- “Settings”（设置）：指定计算图中的吞吐量单位和边沿标签。可用单位是每个（周期或秒）的（位数或字节数）。您还可通过设置“Channel Volume Filter”（通道量筛选器）将低数据量信号（例如，控制信号）灰显来指定数据量下限，以便您专注于高数据量通道。
- 概述：“Overview”（概述）是整个计算图的缩影，为放大计算图时显示的其中相应部分提供参考。您可以使用“Overview”通过操作边界来管理计算图视图。您也可以按需关闭“Overview”以便腾出计算图上的空间。

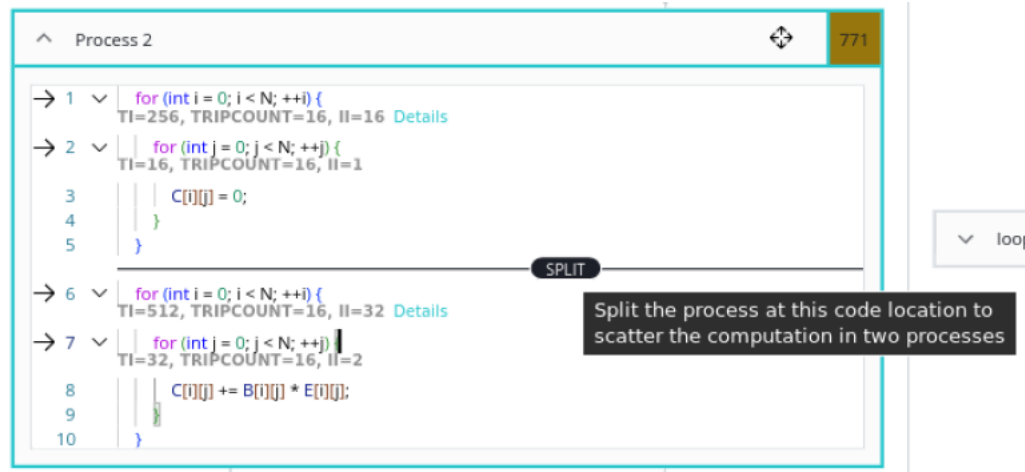
处理计算图

开始设计时，您需要深入了解 HLS 组件源代码，识别主要进程以及这些进程之间的依赖关系。“Code Analyzer”对此予以支持，具体方式是以数据流计算图的形式显示代码，作为 C 语言仿真的输出。

“Code Analyzer”报告会显示每个进程的传输事务时间间隔 (TI)，并在热图中使用红色背景显示值最高的 TI。红色表示设计的问题区域。然而，计算图中使用的综合前估算并不能提供与综合后或实现后指标相同的保真度。“Code Analyzer”支持您快速判定性能潜力、识别问题并加以解决。如需更精确的信息，应使用综合与实现。

您可以合并源代码中的连续进程，探索设计的不同数据流结构，然后按需将代码重新拆分成独立的进程。只需将一个进程拖放到另一个进程上，即可将其合并。请记住，在原始源代码中，进程必须按序执行。下图显示了两个进程先合并，然后使用代码中的“SPLIT”行重新拆分的过程。

图 106：处理计算图



如果您识别出当前设计中存在较大的瓶颈，并希望将其转化为数据流区域，您可以将计算图重新聚焦到函数体或循环体，然后继续对设计进行分析。努力有条不紊解决问题，并充分利用代码分析器。

最终必须以数据流的形式重写代码，以反映代码分析器的结果。通常，您需要将进程提取到各自的函数中，并在函数、循环或区域内添加数据流编译指示。要加速此进程，请单击计算图中的每个进程上的“Goto Source”、右键单击所选源代码，然后选中“Refactor”（重构）。

用例

· 合规性：

代码分析器支持先识别数据流设计中的合规性问题，然后再尝试综合。可以识别的关键问题有：

- 读取和写入接口可通过对始于“开始节点”或止于“结束节点”的通道进行“R+W”（读取和写入）访问来找到。
- 可从该表中识别多个生产者/使用者违例，按通道名称排序并识别具有相同变量的多个通道。对开始节点和结束节点的访问一般都可以取消。
- 可以在表中找到反馈回路，其通道访问模式可能是“R → W”（先读取后写入）模式或可能是“R+W → W”（先读写后写入）模式。此分析可通过通道类型来加以补充，以区分合规与违规的反馈通道。
- 未提取的进程可从进程代码中识别：您应确保在顶层数据流区域中，每个进程只有单次函数调用，并且尽可能在此次调用中使用变量或常量作为实参。

这些问题可以直接在代码中修复，然后新一轮 C 语言仿真就能用更新的指标和结构来刷新计算图。

· 改善性能：

数据流区域性能的关键组成部分之一就是构成该区域的进程的 TI。代码分析器可用于有效提高数据流进程的性能，而无需 HLS 综合。

选中工具栏菜单的“Heat Map”选择器中的“Performance Guidance”（性能指南）即可使用计算图节点上存在的问题标记来识别存在性能问题的进程。展开进程代码可呈现在进程中识别出的特定问题的详情。您可以调查这些问题，并根据可行性、这些问题的位置和最终性能目标来决定是否解决这些问题。例如，如果您希望在某些特定进程的内层循环中 II=1，那么您就需要重写代码，以解决该特定循环嵌套上呈现的所有问题。

在相关用例中，您可能出于教育培训目的或验证目的，希望了解特定进程的 TI 是如何计算的。函数调用和循环旁的 TI 和 II 注解可与进程源代码内联，以便达成您的目的。

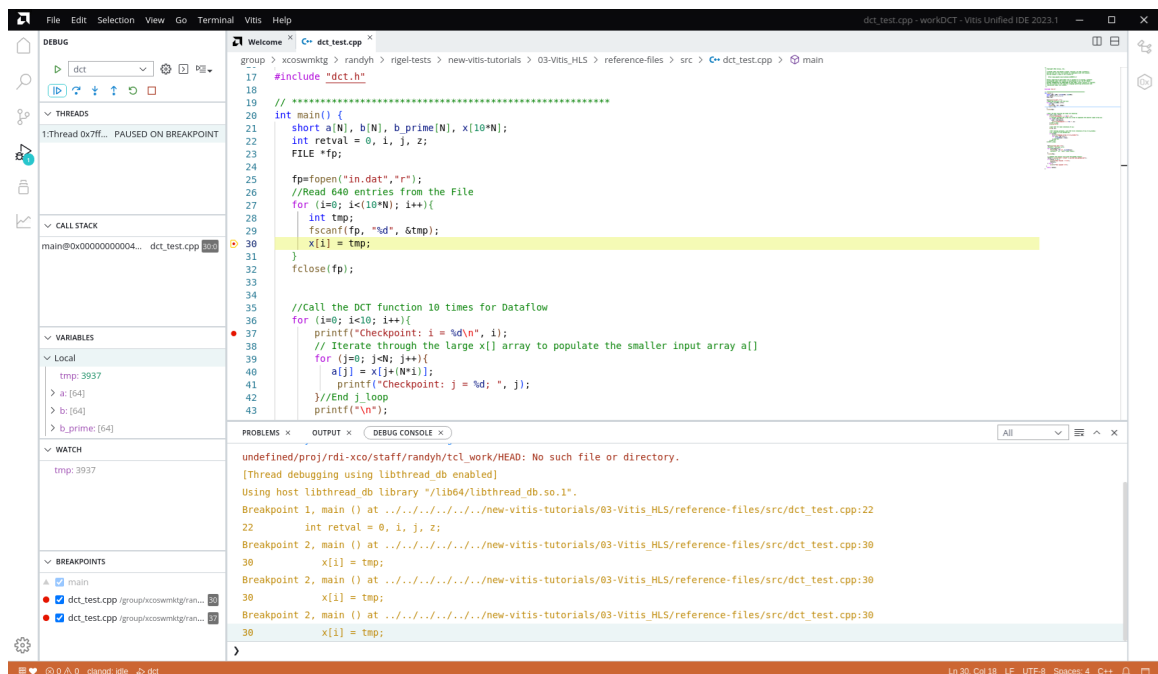
- 吞吐量分析：

代码分析器会呈现通道的吞吐量估算结果。为了辅助分析和更好地了解设计性能，您还可以访问通道宽度及其容量（区域每次执行的访问总数）。不过，您应尽可能通过综合设计来确认吞吐量估算结果。代码分析器依赖于综合前的估算，其保真度低于其他综合后和实现后的指标。

调试 HLS 组件

仅在 C 语言仿真之后或 C 语言仿真过程中才能调试 HLS 组件。下图显示了 HLS 组件的“Debug”（调试）视图。您可在“Debug”视图中直接查看变量值和表达式。例如，在“Variables”（变量）视图中，您可编辑变量值以强制变量变为特定状态。在“Watch”（观察）视图中，您可以监控表达式的值。在“Watch”视图使用“+”命令即可添加要观察的表达式。如需了解有关可用功能特性和视图的更多详细信息，请参阅《使用 Vitis 进行嵌入式设计开发》(UG1701) 中描述的“调试视图”。

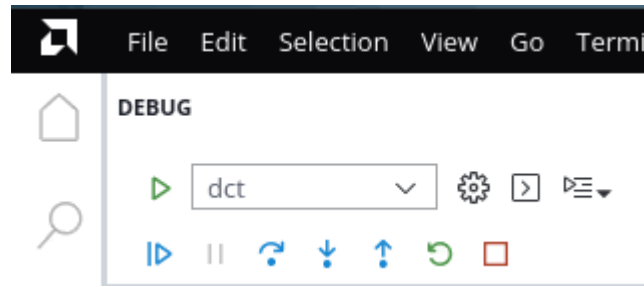
图 107：调试视图 - HLS 组件



调试视图的功用与大部分传统 C/C++ 软件调试工具类似。它能够在代码中插入断点，也能单步跳过或单步跳入特定代码节。有许多不同的窗口或小组件可用于管理调试进程。以下提供了其中大部分窗口或小组件的列表，并描述了您在调试 HLS 组件时如何使用这些窗口或小组件。

- “Control Panel”（控制面板）：“Debug”视图“Control Panel”显示在屏幕左上角，如下图所示。调试期间，您可以使用“Continue”（继续）、“Step Over”（单步跳过）、“Step Into”（单步进入）、“Step Out”（单步跳出）、“Restart”（重新启动）和“Stop”（停止）等控制按钮来控制调试进程。

图 108：调试控制面板



- “Threads”（线程）：“Threads”（线程）用于显示相关调试线程。线程是在调试进程中创建和销毁的。您可在多个线程之间进行切换。
- “Call Stack”（调用栈）：“Call Stack”（调用栈）显示了运行应用时更新的函数调用栈。
- “Variables”（变量）：您可在“Debug”视图中使用“Variables”窗口直接查看变量值和表达式，该窗口中显示了全局变量和局部变量的当前值。切换线程时，会更新变量信息。您可选中特定变量，然后从右键单击菜单中选中“Set Value”（设置值）命令来强制将该变量设为特定值。
- “Watch”（观察）：显示了您指定要关注的变量和表达式。选中“Add Expression”（添加表达式）(+) 添加观察点。
- “Breakpoints”（断点）：Vitis Unified IDE 会在测试平台的 main 函数处设置断点。要添加断点，可打开源文件并单击行号左侧显示红点处。您可单击先前添加的断点以将其移除。

显示红点时，您可右键单击并选中“Add Conditional Breakpoint”（添加条件断点）来添加条件断点。您还可右键单击并选中“Add Logpoint”（添加记录点）插入到达断点时要记录的消息。

- “Source Code Editor”（源代码编辑器）：从“Flow Navigator”或“Launch Configuration”（启动配置）视图启动“Debug”时，会打开“Source Code”（源代码）视图。
- “Debug Console”（调试控制台）：此调试控制台会显示调试进程的转录文本，以及从被测应用接收到的任何消息。

运行 C 语言综合

在“Flow Navigator”中，确保 HLS 组件处于活动状态，或者在“Component”（组件）菜单中将其选中，在该工具中将其设为活动组件。当 HLS 组件为活动组件时，“Flow Navigator”支持运行 C 语言仿真、C 语言综合、C/RTL 协同仿真、封装与实现来构建和分析该 HLS 组件。要综合 HLS 组件，请选中“Flow Navigator”的“C SYNTHESIS”（C 语言综合）标题下的“Run”（运行）。对该 HLS 组件运行 C 语言综合会从 C/C++ 源代码生成 RTL。



提示：在 Vitis Unified IDE 中使用 `v++ -c --mode hls` 命令来运行综合。

指定流程目标和通用设置

HLS 组件可用于为 Vivado IP 流程或 Vitis 内核流程创建综合结果。流程目标用于判定应用于 IP 或内核的默认接口端口以及综合后设计的其他详细信息。

图 109: HLS 组件通用设置

General

part

family or part value

[Browse](#)

clock

Specify the clock period in ns or MHz (ns is default). If no period is specified a default period of 10ns is used.

clock_uncertainty

Specify how much of the clock period is used as a margin by HLS. Default units is ns but % or MHz can also be used.

flow_target

Set the flow target

在上图中，当前为 Vitis 流程设置了“flow_target”。默认流程目标为 Vivado IP 流程。通常在创建 HLS 组件时设置此目标，但也可按需修改。

在“General”（通用）设置下还可指定“part”（部件）或“platform”（平台）。这些选项用于确定物理器件，以便为这些器件执行 C/C++ 代码综合。所选器件可采用相应的硬件资源，这些资源会扩展或限制可供 RTL 综合选择的实现选项。

最后，可以指定“clock”（时钟）周期和“clock_uncertainty”（时钟不确定性）来定义 RTL 设计的时序详细信息。这些都可能会影响设计中的操作的时序和调度。

这些选项会生成如下配置文件条目：

```
part=xcvc1902-vsva2197-2MP-e-S

[hls]
flow_target=vitis
clock=8ns
clock_uncertainty=15%
```

加载源文件和识别顶层函数

通常，HLS 组件的源代码是在创建时定义或加载的。但您可在创建组件时绕过此步骤，然后您需要在综合设计前添加源文件。要向现有 HLS 组件添加源文件，请打开组件配置文件，导航到“C Synthesis sources”（C 语言综合源文件）部分，如下所示。“Config Editor”（配置编辑器）的这部分支持您添加一个或多个源文件，并指定顶层函数。

图 110: HLS 组件综合源文件

C Synthesis sources

hls.syn.file_cflags
Specify cflags for a C Synthesis file

FILE	CFLAGS	CSIMFLAGS
Flags common to all files		
/group/xcoswmktg/andyh/rigel-tests/03-Vit...		

Add item

top
Name of the function to be synthesized

Select Top Function [Browse](#)

“C Synthesis sources” 会显示当前添加的源文件，并允许您编辑或删除当前源文件、添加新的源文件以及添加或修改 CFLAGS 和 CSIMFLAGS。

- “Add item”（添加项）：选择此命令即可向 HLS 组件添加新的源文件。这样会打开“File Browser”（文件浏览器）并允许您导航到并选中要添加的源文件。
- “Add CFLAGS or CSIMFLAGS”（添加 CFLAGS 或 CSIMFLAGS）：您可添加综合编译标志 (CFLAGS) 和仿真编译标志 (CSIMFLAGS)，以供应用于所有源文件或者添加到特定源文件。如上图所示，只需将这些标志添加到“Flags common to all files”（所有文件的通用标志）的相应文本输入框中即可。要为特定源文件添加标志，请选中该源文件，然后选中“Edit item”（编辑项）以添加标志或者修改文件名和路径。
- “top”（顶层函数）：允许您指定用作为综合的顶层 RTL 模块的函数。顶层模块用于判定将添加的 RTL 端口以及要包含在 HLS 组件中的子函数。这样会打开“Select Top Function”（选择顶层函数）对话框，其中会显示源文件中已定义的函数列表以供您从中选择顶层函数。

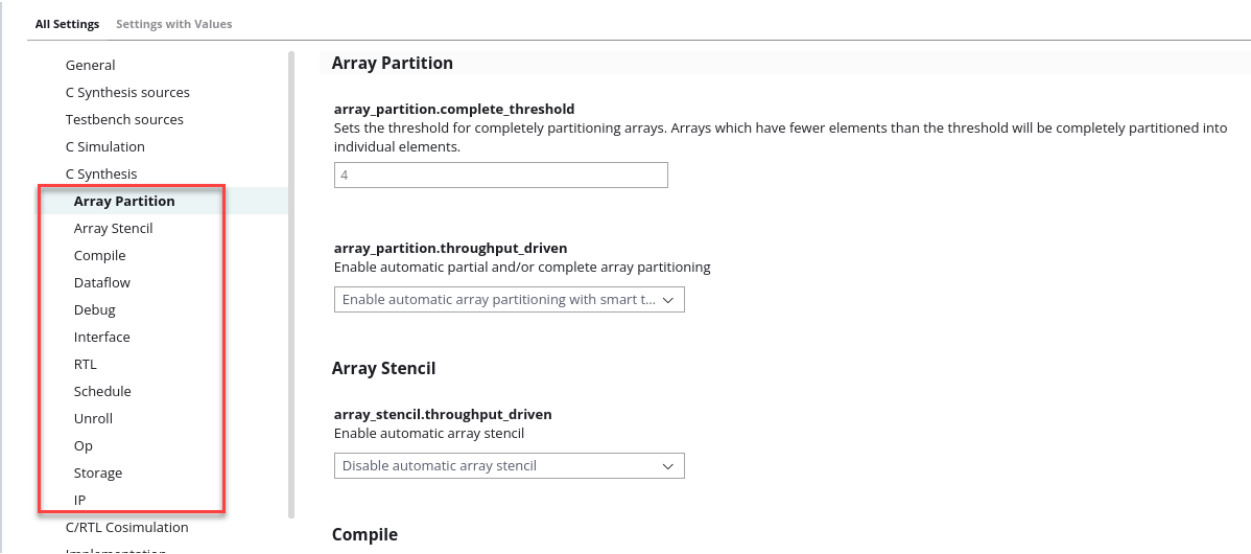
这些选项会生成如下配置文件条目：

```
[HLS]
syn.file=<path/to/file.cpp>
syn.file_cflags=<path/to/file.cpp>,<cflag>
syn.file_csimflags=<path/to/file.cpp>,<csimflag>
syn.cflags=<cflag for all files>
syn.csimflags=<csimflag for all files>
syn.top=<top function name>
```

配置默认设置

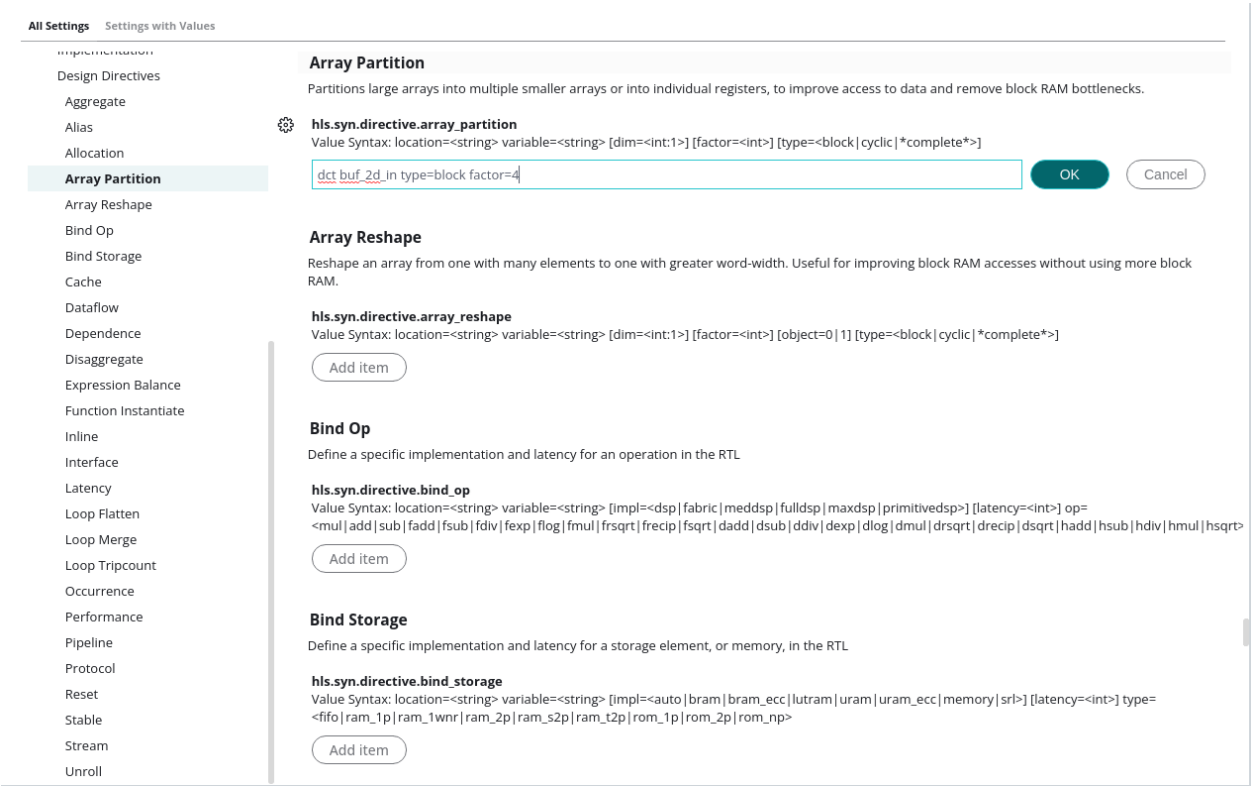
对于 HLS 组件，Vitis Unified IDE 会提供各种命令，这些命令可用于配置该工具的默认综合与仿真设置。这些配置命令显示在“Config Editor”的“C Synthesis”（C 语言综合）标题下，如下图所示。通常，这些选项按图中所示方式记述在文档中，如 [编译选项](#)、[接口配置](#) 和 [RTL 配置](#)。

图 111：HLS 组件默认设置



分配设计指令

图 112：HLS 组件指令



设计指令允许您为源代码自定义综合结果。更改多轮综合运行中使用的指令即可更改结果或优化设计。这些指令可作为 HLS 编译指示直接添加至源代码，或者作为 [HLS 优化指令](#) 添加至配置文件，以供在 HLS 组件中使用。在“Config File Editor”（配置文件编辑器）中，会列出各种指令，您可选中“Add Item”（添加项）打开“Directive”（指令）编辑器。

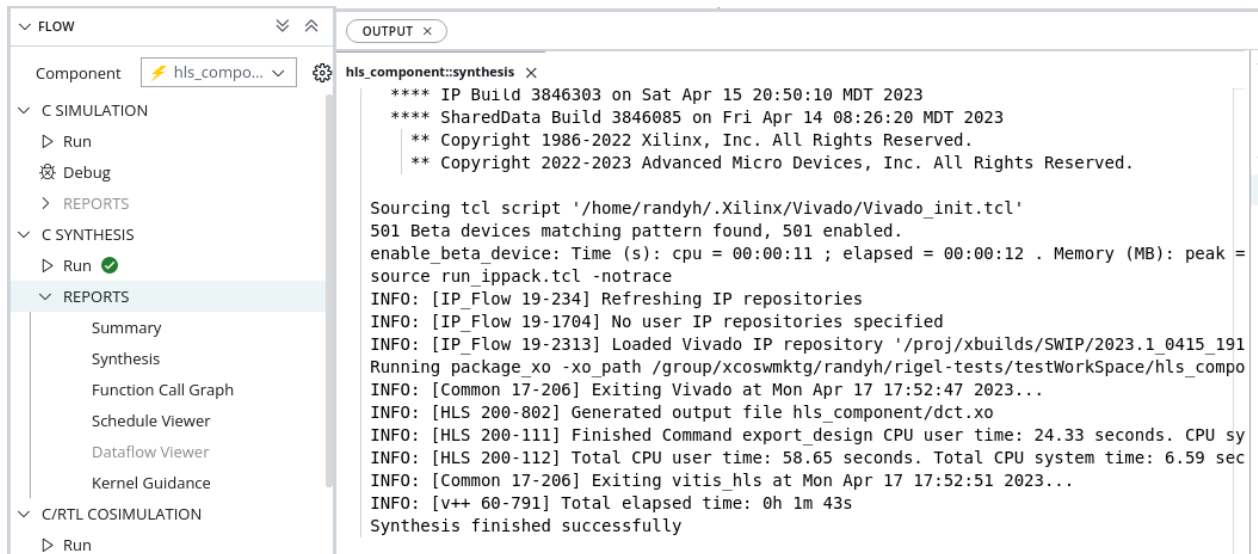
配置文件中的设计指令输入的部分示例：

```
[HLS]
syn.directive.dataflow=dct
syn.directive.array_partition=dct buf_2d_in type=block factor=4
syn.directive.pipeline=dct2d II=4
```

运行综合

在配置文件中定义 HLS 组件的关键元素后，即可运行综合。从“Flow Navigator”中选中“Run”即可开始综合。您可在“Output”（输出）窗口中跟踪综合运行的进度。综合运行的转录文本所含顶层函数名为 `<component-name>::synthesis`。综合进程期间，消息将被转录到控制台窗口内，并记录到 `vitis_hls.log` 文件中。

图 113：运行综合的 HLS 组件



完成综合后，应在转录文本底部显示“Synthesis finished successfully”（综合成功完成）消息。在“Run”命令下还会显示填充的“Reports”文件夹，如上图所示。综合后可用报告如下所示：

- “Summary”（汇总）：报告使用的命令行和结果的时间戳。
- “Synthesis”（综合）：报告综合结果及结果质量、硬件接口、突发传输事务等信息。如需了解更多信息，请参阅[综合汇总](#)。
- “Function Call Graph”（函数调用计算图）：在 C 语言综合或 C/RTL 协同仿真完成后，显示 HLS 组件，以显示设计的时延和 II 方面的吞吐量，如[函数调用计算图](#)所述。
- “Schedule Viewer”（调度查看器）：显示函数的每项操作和每个控制步骤，以及执行操作和步骤的时钟周期，如[调度查看器](#)所述。
- “Dataflow Viewer”（数据流查看器）：仅当设计中使用 [DATAFLOW](#) 编译指示或指令时才可用。此报告会显示本工具推断所得的数据流结构，如[数据流查看器](#)所述。
- “Kernel Guidance”（内核指南）：报告指南消息，为源代码与综合结果提供设计建议。

改善综合运行时间和容量

HLS 编译器会调度对象。无论对象是浮点乘法运算还是单寄存器，它都仍属于要调度的对象。浮点乘法可能需多个周期才能完成，并且使用大量资源来实现，但在调度层次上，它仍是对象。

HLS 编译器会按层级来调度运算。调度顺序如下：循环中的运算、其次循环，最后是函数中的子函数和运算。编译器的运行时间会在下列情况下增加：

- 需调度的对象数量增加。
- 有更大自由度和更多可能性可供探索。

展开循环和对阵列进行分区会创建更多需调度的对象并且可能增加运行时间。函数内联会在此层级创建更多需调度的对象，同样会增加运行时间。仅执行下列操作时，请务必谨慎处理：所有阵列的分区、所有循环的展开以及所有函数的内联操作。这些优化可能是满足性能目标所必需的，但可以预见，综合运行时间会因此增加。请使用 [第二部分：HLS 编程指南](#) 中探讨的优化策略，并谨慎采用这些优化。

如果必须展开循环，或者如果在上层层级中使用 PIPELINE 指令已自动展开循环，请考虑将循环主体作为独立函数来捕获。这将把所有逻辑捕获到单一函数内，而不是在展开逻辑时创建多个逻辑副本：在已定义的层级内调度一组对象速度更快。谨记，如果在流水打拍区域内使用展开的循环，请对此函数进行流水打拍。

代码自由程度同样会影响运行时间。请将 HLS 编译器视为设计专家，默认负责的任务是查找含最高吞吐量、最长时延和最小面积的设计。该工具所受约束程度越高，可供使用的选项越少，运行速度越快。请考虑对代码中的作用域使用时延约束：循环、函数或区域。设置所含最小值和最大值相同的 LATENCY 指令可以减少该作用域内可能的优化搜索次数。

C 语言综合的输出

在 Vitis Unified IDE 中，HLS 组件层级的结构如下：<workspace>/<component>/<component>/hls/。您可在“Vitis Component Explorer”（Vitis 组件资源管理器）中选中 HLS 组件，然后从右键单击菜单中选中“Open In Terminal”（在终端中打开），以浏览组件文件夹的内容。

完成综合后，就会在 <component>/hls 文件夹内创建 syn 文件夹。该文件夹包含以下元素：

- verilog 和 vhdl 文件夹包含输出 RTL 文件。
 - 顶层文件与综合的顶层函数同名。
 - 针对尚未内联到更高层次的函数中的每个子函数，都会创建一个 RTL 文件。
 - 此外还可能包含其他 RTL 文件，用于实现 RTL 层级的子块，例如，块 RAM 和流水打拍乘法器。
- report 文件夹包含顶层函数的报告文件，对于 HLS 编译器尚未内联到更高层次函数中的每个子函数，还各包含一个报告文件。顶层函数的报告提供了有关整个设计的详细信息。



重要提示！ 请勿将 syn/verilog 或 syn/vhdl 文件夹内生成的 RTL 文件直接用于在 Vivado 工具内进行综合。必须改用由封装进程生成并导出的输出文件，如 [封装 RTL 设计](#) 中所述。HLS 编译器有时会使用来自已综合的 RTL 代码中的 Vivado IP 目录的 IP（例如，浮点设计），并且 verilog 和 vhdl 文件夹将不包含完整的设计。

综合汇总

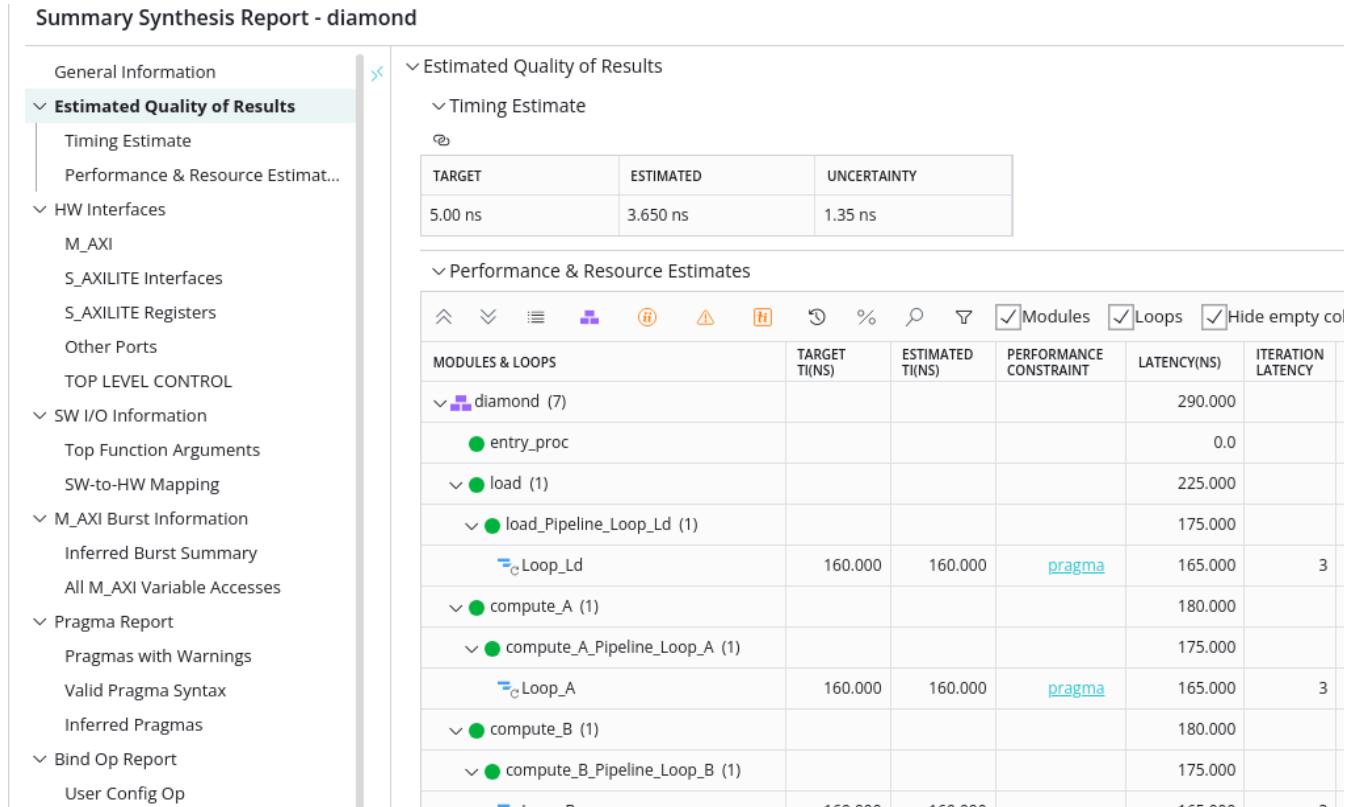
当综合完成后，Vitis HLS 会在信息窗格中自动打开顶层函数的“Synthesis Summary”（综合汇总）报告。

以下详细介绍了“Synthesis Summary”中的各部分。

“General Information”（常规信息）

此“常规信息”部分可提供有关报告生成时间、所用软件版本、工程名称、解决方案名称和目标流程以及技术详情的信息。

图 114：综合汇总报告



时序估算

此“Timing Estimate”（时序估算）部分可显示解决方案所指定的时序的快速估算结果，如 [指定时钟频率](#) 所述。其中包括指定的“Target”（目标）时钟周期和“Uncertainty”（不确定性）的周期。时钟周期减去不确定性即可生成“Estimated”（估算）时钟周期。



提示：这些值只是 HLS 工具的估算值。Vivado 综合与实现可提供更精确的时序估算，如 [运行实现](#) 所述。

性能和资源估算

“Performance & Resource Estimates”（性能和资源估算）部分中的“Performance Estimate”（性能估算）列可报告顶层函数和顶层中例化的任何子块的时序裕量、时延和启动时间间隔。C/C++ 源代码中在此级别调用的每个子函数都是生成的 RTL 块中的一个实例，除非使用 `INLINE` 编译指示或指令将该子函数内联（或者自动内联）到顶层函数内。

“Slack”（裕量）列可显示实现中的任何时序问题。

这两个“Latency”（时延）列可显示生成输出所需的周期数，并显示时延 (ns)。

“Iteration Latency”（迭代时延）表示单个循环的单次迭代的时延。

“Initiation Interval”（启动时间间隔）是可应用新输入之前的时钟周期数。如果没有任何 PIPELINE 指令，则时延比启动时间间隔少 1 个周期（写入最终输出后，读取下一个输入）。



提示：如果时延显示为“？”，则表示本工具无法确定循环迭代次数。如果设计的时延或吞吐量取决于含可变索引的循环，那么 HLS 编译器会将此循环的时延报告为未知。在此情况下，请使用 LOOP_TRIPCOUNT 编译指示或指令来手动指定循环迭代次数。LOOP_TRIPCOUNT 值仅用于确保生成的报告能够显示有意义的时延和时间间隔范围，而不影响综合结果。

“Trip Count”（循环次数）列显示的是特定循环在已实现的硬件内的迭代次数。它用于反映硬件中的任意循环展开行为。

此报告的资源估算列用于指示实现 RTL 代码中的软件函数所需的资源估算结果。其中提供了块 RAM、DSP、FF 和 LUT 的估算值。

HW 接口

综合报告的“HW Interfaces”（硬件接口）部分提供了对应于综合期间生成的不同硬件接口的表格。该工具生成的硬件接口类型取决于解决方案所指定的流程目标以及应用于代码的任何 INTERFACE 编译指示或指令。在下图中，该解决方案目标为 Vitis 内核流程，因此会按需生成 AXI 接口。

图 115：HW 接口

Summary Synthesis Report - diamond

General Information

Estimated Quality of Results

Timing Estimate

Performance & Resource Estimati...

HW Interfaces

M_AXI

S_AXILITE Interfaces

S_AXILITE Registers

Other Ports

TOP LEVEL CONTROL

SW I/O Information

Top Function Arguments

SW-to-HW Mapping

M_AXI Burst Information

Inferred Burst Summary

All M_AXI Variable Accesses

Pragma Report

Pragmas with Warnings

Valid Pragma Syntax

Inferred Pragmas

Bind Op Report

HW Interfaces

M_AXI

INTERFACE	READ/WRITE	DATA WIDTH (SW->HW)	ADDRESS WIDTH	LATENCY	OFFSET	REGISTER
m_axi_gmem	READ_WRITE	512 -> 512	64	0	slave	0

S_AXILITE Interfaces

INTERFACE	DATA WIDTH	ADDRESS WIDTH	OFFSET	REGISTER
s_axi_control	32	6	16	0

S_AXILITE Registers

INTERFACE	REGISTER	OFFSET	WIDTH	ACCESS	DESCRIPTION
s_axi_control	vecIn_1	0x10	32	W	Data signal of vecIn
s_axi_control	vecIn_2	0x14	32	W	Data signal of vecIn
s_axi_control	vecOut_1	0x1c	32	W	Data signal of vecOut
s_axi_control	vecOut_2	0x20	32	W	Data signal of vecOut

Other Ports

PORT	MODE	DIRECTION	BITWIDTH
size	ap_none	in	32

查看这些表格时，应留意下列信息：

- 针对不同接口提供了不同的表格。
- 所提供的列用于显示接口的不同属性。对于 M_AXI 接口，包括“Data Width”（数据宽度）和“Max Widen Bitwidth”（最大拓宽位宽）列，用于指示是否发生了“Automatic Port Width Resizing”（端口宽度自动调整）以及发生的程度。在以上示例中，可以看到，此端口已从软件中指定的 16 位拓宽至 512 位。如需了解更多信息，请参阅 [端口宽度自动调整](#)。

- “Latency”（时延）列可显示接口的时延：
 - 在 `ap_memory` 接口中，该列可显示驱动此接口的 RAM 资源的读取时延。
 - 对于 `m_axi` 接口，该列会显示 AXI4 接口的期望时延，允许设计发起总线请求的时间比执行期望的读取或写入操作早数个周期（时延）。
- “Bundle”（捆绑）列可显示来自 **INTERFACE** 编译指示或指令的任意指定的捆绑名称。
- 此外，这些列还会显示 M_AXI 接口的突发和读取写入属性，如“INTERFACE 编译指示或指令”所述。
- “Bit Fields”（位字段）列可显示 `s_axilite` 接口中寄存器所使用的位。

软件 I/O 信息

此“SW I/O Information”（软件 I/O 信息）部分会高亮来自 C/C++ 源代码的函数实参与生成的 RTL 代码中的端口名称之间的关联方式。其中提供的软件和硬件端口的其他详细信息如下所示。软件实参可扩展至多个硬件接口中。例如，`input` 实参与 3 个硬件接口相关，`m_axi` 对应于数据，`s_axi_lite` 则对应于所需的控制信号。

图 116：软件 I/O 信息

Summary Synthesis Report - diamond				
- General Information - Estimated Quality of Results - Timing Estimate - Performance & Resource Estimati... - HW Interfaces - M_AXI - S_AXILITE Interfaces - S_AXILITE Registers - Other Ports - TOP LEVEL CONTROL SW I/O Information - Top Function Arguments - SW-to-HW Mapping - M_AXI Burst Information - Inferred Burst Summary - All M_AXI Variable Accesses - Pragma Report - Pragmas with Warnings - Valid Pragma Svntax	SW I/O Information			
	Top Function Arguments			
	ARGUMENT	DIRECTION	DATATYPE	
	vecIn	inout	vector<unsigned int 16>*	
	vecOut	inout	vector<unsigned int 16>*	
	size	in	int	
	SW-to-HW Mapping			
	ARGUMENT	HW INTERFACE	HW TYPE	HW USAGE
	vecIn	m_axi_gmem	interface	channel=0
	vecIn	s_axi_control	register	offset
	vecIn	s_axi_control	register	offset
	vecOut	m_axi_gmem	interface	channel=0
	vecOut	s_axi_control	register	offset
	vecOut	s_axi_control	register	offset
	size	size	port	

M_AXI 突发信息

在“M_AXI Burst Information”（M_AXI 突发信息）部分中，“Burst Summary”（突发汇总）表中报告了成功的突发传输，并包含指向关联源代码的链接。报告的突发长度表示 `max_read_burst_length` 或 `max_write_burst_length`，并表示突发传输期间读取/写入的数据值的数量。例如，如果输入类型为整数（32 位），且 HLS 自动将此接口拓宽至 512 位，那么每次突发会传输 1024 个整数。由于拓宽后的接口每次可承载 16 个整数，因此结果是 64 拍突发。“Burst Missed”（突发未命中）表格用于报告突发未能命中的原因，并包含指向突发失败相关的指导性消息的链接，以帮助解决问题。

图 117: M_AXI 突发信息

Summary Synthesis Report - diamond

General Information

> Estimated Quality of Results

> HW Interfaces

> SW I/O Information

▼ M_AXI Burst Information

Inferred Burst Summary

All M_AXI Variable Accesses

> Pragma Report

> Bind Op Report

> Bind Storage Report

✕

▼ M_AXI Burst Information

▼ Inferred Burst Summary

HW INTERFACE	DIRECTION	LENGTH	WIDTH	LOOP	LOOP LOCATION
m_axi_gmem	read	variable	512	Loop_Ld	diamond.cpp:42
m_axi_gmem	write	variable	512	Loop_St	diamond.cpp:92

▼ All M_AXI Variable Accesses

HW INTERFACE	VARIABLE	ACCESS LOCATION	DIRECTION	BURST STATUS	LENGTH	LOOP
m_axi_gmem	in	diamond.cpp:45	read	Widen Fail		Loop_Ld
m_axi_gmem	in	diamond.cpp:45	read	Inferred	variable	Loop_Ld
m_axi_gmem	out	diamond.cpp:95	write	Widen Fail		Loop_St
m_axi_gmem	out	diamond.cpp:95	write	Inferred	variable	Loop_St

编译指示报告

此“Pragma Report”（编译指示报告）部分显示设计中的编译指示：valid（有效）、ignored（已忽略）和 inferred（已推断）。此报告旨在汇总 log 日志文件中显示的问题。它使您能够快速识别设计中使用的编译指示所存在的问题，以便您查看哪些编译指示未能按期望的方式来使用。其中还会单独报告有效的编译指示，以便您查看设计中使用的所有编译指示。



提示：其中提供了链接，指向源代码中的编译指示所在位置。单击链接即可查看源代码。

图 118: 编译指示报告

Summary Synthesis Report - diamond

General Information	✕	▼ Pragma Report																																						
- Estimated Quality of Results - HW Interfaces - SW I/O Information - M_AXI Burst Information ▼ Pragma Report - Pragmas with Warnings - Valid Pragma Syntax - Inferred Pragmas - Bind Op Report - Bind Storage Report		▼ Pragmas with Warnings <table> <tr> <th>TYPE</th><th>OPTIONS</th><th>LOCATION</th><th>FUNCTION</th><th>MESSAGES</th></tr> <tr> <td>dataflow</td><td></td><td>diamond.cpp:30</td><td>diamond</td><td>Since the only kind of statements allowed in a c</td></tr> </table> > Valid Pragma Syntax ▼ Inferred Pragmas <table> <tr> <th>INFERRED PRAGMA</th><th>OPTIONS</th><th>LOCATION</th><th>INFERRED FROM</th></tr> <tr> <td>pipeline</td><td>ii=1</td><td>diamond.cpp:42 Loop_Ld in load</td><td>performance diamond.cpp:43</td></tr> <tr> <td>pipeline</td><td>ii=1</td><td>diamond.cpp:52 Loop_A in compute_a</td><td>performance diamond.cpp:53</td></tr> <tr> <td>pipeline</td><td>ii=1</td><td>diamond.cpp:64 Loop_B in compute_b</td><td>performance diamond.cpp:65</td></tr> <tr> <td>pipeline</td><td>ii=1</td><td>diamond.cpp:74 Loop_C in compute_c</td><td>performance diamond.cpp:75</td></tr> <tr> <td>pipeline</td><td>ii=1</td><td>diamond.cpp:83 Loop_D in compute_d</td><td>performance diamond.cpp:84</td></tr> <tr> <td>pipeline</td><td>ii=1</td><td>diamond.cpp:92 Loop_St in store</td><td>performance diamond.cpp:93</td></tr> </table>	TYPE	OPTIONS	LOCATION	FUNCTION	MESSAGES	dataflow		diamond.cpp:30	diamond	Since the only kind of statements allowed in a c	INFERRED PRAGMA	OPTIONS	LOCATION	INFERRED FROM	pipeline	ii=1	diamond.cpp:42 Loop_Ld in load	performance diamond.cpp:43	pipeline	ii=1	diamond.cpp:52 Loop_A in compute_a	performance diamond.cpp:53	pipeline	ii=1	diamond.cpp:64 Loop_B in compute_b	performance diamond.cpp:65	pipeline	ii=1	diamond.cpp:74 Loop_C in compute_c	performance diamond.cpp:75	pipeline	ii=1	diamond.cpp:83 Loop_D in compute_d	performance diamond.cpp:84	pipeline	ii=1	diamond.cpp:92 Loop_St in store	performance diamond.cpp:93
TYPE	OPTIONS	LOCATION	FUNCTION	MESSAGES																																				
dataflow		diamond.cpp:30	diamond	Since the only kind of statements allowed in a c																																				
INFERRED PRAGMA	OPTIONS	LOCATION	INFERRED FROM																																					
pipeline	ii=1	diamond.cpp:42 Loop_Ld in load	performance diamond.cpp:43																																					
pipeline	ii=1	diamond.cpp:52 Loop_A in compute_a	performance diamond.cpp:53																																					
pipeline	ii=1	diamond.cpp:64 Loop_B in compute_b	performance diamond.cpp:65																																					
pipeline	ii=1	diamond.cpp:74 Loop_C in compute_c	performance diamond.cpp:75																																					
pipeline	ii=1	diamond.cpp:83 Loop_D in compute_d	performance diamond.cpp:84																																					
pipeline	ii=1	diamond.cpp:92 Loop_St in store	performance diamond.cpp:93																																					

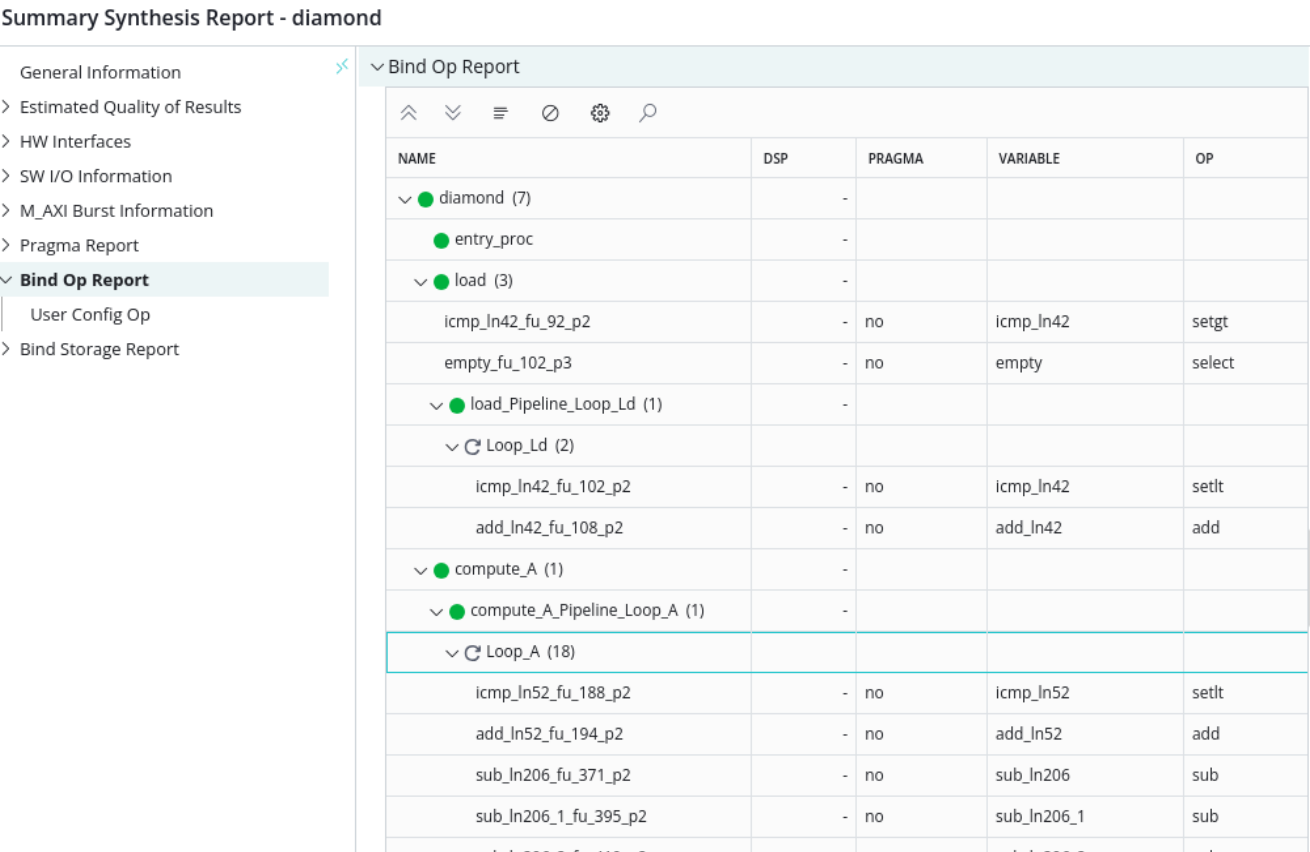
绑定操作报告和绑定存储报告

在“Synthesis Summary”报告中新增了“Bind Op”（绑定操作）报告和“Bind Storage”（绑定存储）报告。这两份报告都有助于您了解 HLS 编译器将操作映射到资源时所做的选择。该工具将把操作映射到具有适当时延的适当资源。您可通过使用 BIND_OP 编译指示或指令并请求特定资源映射和时延来影响此进程。“Bind Op”（绑定操作）报告将显示已自动完成的映射和通过使用编译指示来强制完成的映射。同样，“Bind Storage”（绑定存储）报告会显示块 RAM/LUTRAM/URAM 等平台上的阵列到存储器资源的映射。

“Bind Op”报告可显示内核或 IP 的实现详细信息。其中会显示顶层函数的层次结构，并列出变量和已应用的任何 HLS 编译指示或指令、已定义的操作、HLS 工具使用的实现以及任何已应用的时延。

此报告适用于检验 RTL 设计所指定的可编程逻辑实现详细信息。

图 119：综合汇总



如上所示，“Bind Op”报告高亮显示了您的设计中的部分重要特性。当前，它调出设计中使用的 DSP 数量并按层次结构显示这些 DSP 在设计中的使用位置。该表还高亮显示是否已根据用户指定的编译指示完成特定资源分配，如果已完成分配，则会在“Pragma”列中显示“yes”。如果“Pragma”列中不存在任何条目，这表示工具已自动推断资源分配。该表还显示为用户设计中的每个模块分配的资源的 RTL 名称，您可以按层次结构降序逐级向下查看各种资源。

它不会显示所有推断的资源，而是改为显示相关的资源（例如，算术、浮点和 DSP）。此外还会显示互连结构（使用 LUT 实现）或 DSP 的具体实现选择。最后，还会显示资源的时延。这些信息有助于了解并按需增加资源的时延，以便向设计添加流水线阶段。在尝试解决实现期间的时序问题时，如需尝试对冗长的组合路径进行细分，那么这些信息是非常实用的。

每个资源分配都与源代码行相关，对应操作正是根据此源代码行来推断得出的，您可右键单击资源，并选择“Goto Source”（转至源代码）选项以查看此关联。最后，“Bind Op”报告下的第二个表格还显示了各项全局配置设置，这些设置全都同样可用于更改工具使用的资源分配算法。在以上示例中，dadd（双精度浮点加法）操作的实现选择已固定到 fulldsp 实现。同样，ddiv 操作的时延已固定为 2。

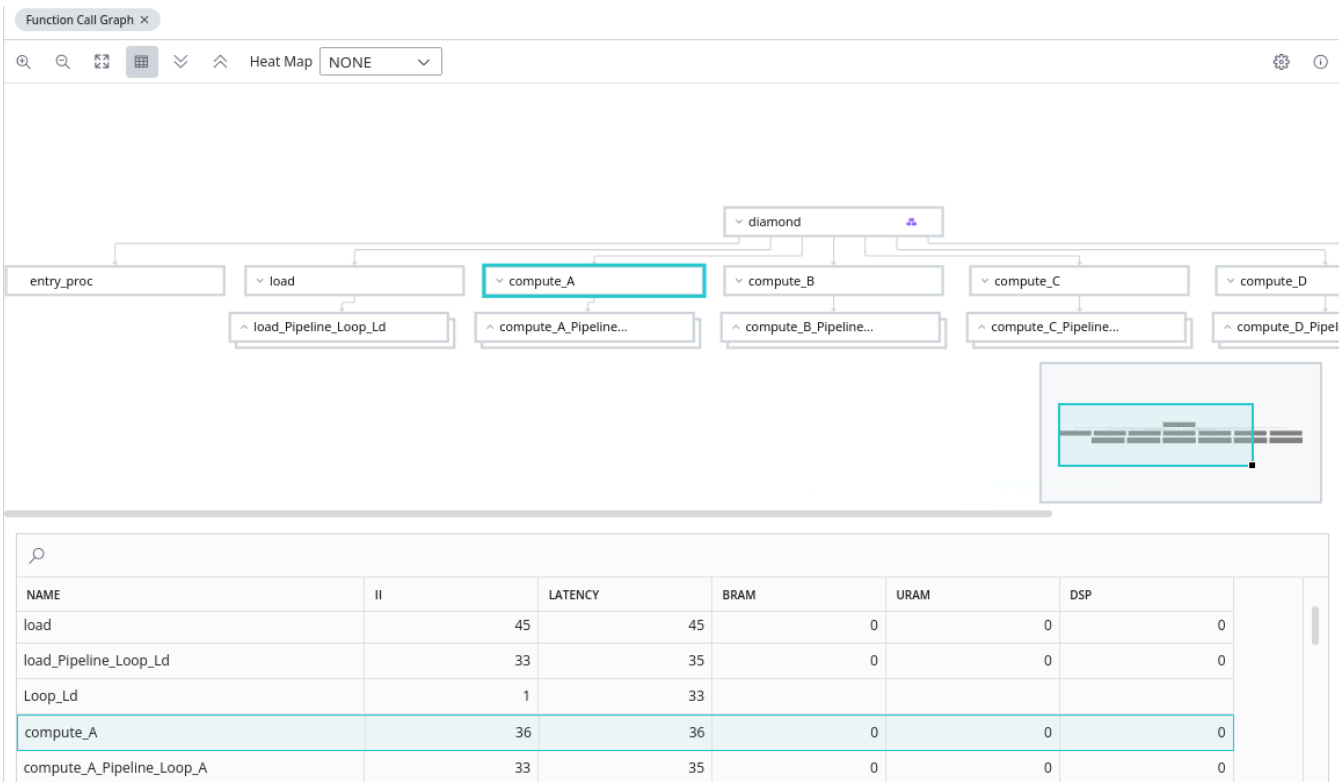
与 BIND_OP 编译指示类似，BIND_STORAGE 编译指示可用于选择特定存储器类型（例如，单端口或双端口）和/或特定存储器实现（例如，BRAM/LUTRAM/URAM/SRL 等）以及时延值。“Bind Storage”报告会高亮显示设计中使用的存储映射。当前，它会调出设计中使用的块 RAM 和 URAM 的数量。该表还高亮显示是否已根据用户指定的编译指示完成特定存储资源分配，如果已完成分配，则会在“Pragma”列中显示“yes”。如果“Pragma”列中不存在任何条目，则表示工具已自动推断存储资源分配。特定的存储类型和实现选择也会随变量名和时延一并显示。

您可使用此信息来复查设计中的存储资源分配情况，并根据可用资源来更改最终的存储实现，从而在设计中选择相应的选项。最后，如有任何全局设置（如 [存储配置](#) 所述）可用于更改本工具所使用的存储资源分配算法，那么“Bind Storage”报告中还会显示另一个表格。

函数调用计算图

“Function Call Graph”（函数调用计算图）为您演示了 C 语言综合或 C/RTL 协同仿真后的完整设计。此报告从时延和启动时间间隔 (II) 两方面显示了设计的吞吐量。它有助于识别您的设计中的关键路径，并帮助您识别设计中的瓶颈，以提升吞吐量。它还可显示穿过设计的各条路径中可能存在不平衡而导致 FIFO 停滞和/或死锁的路径。

图 120：性能指标综合



在某些情况下，显示的设计层级可能与源代码不同，因为 HLS 优化将循环转换为函数流水线等原因。内联的函数不再显示在调用计算图中，因为在综合后的代码中，这些函数不再是独立的函数。如果为某个函数创建了多个实例，那么该函数的每个唯一实例都会显示在此调用计算图中。这样您即可查看哪些函数影响了调用函数的时延和 II。

以上显示的计算图中，将函数显示为矩形框，将循环显示为椭圆框。该计算图会显示层级和嵌套的函数或循环。该计算图下的表格中显示了每个函数或循环，根据具体视图，还会显示 II、时延和资源或时序数据。

“Function Call Graph” 报告位于 Flow Navigator 的“C SYNTHESIS”（C 语言综合）和“C/RTL COSIMULATION”（C/RTL 协同仿真）标题下。计算图中显示的来自“C SYNTHESIS”阶段的性能和资源指标均为来自 HLS 工具的估算值。完成“C/RTL COSIMULATION”后所得“Function Call Graph”报告则更详细，其中会提供报告的实际 II 和时延数值以及停滞百分比。

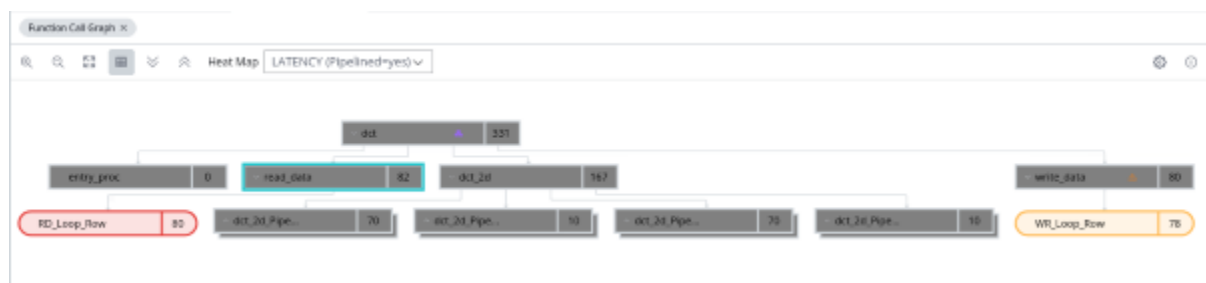


提示：如需获取更准确的资源和时序估算结果，可在 [运行实现](#) 中执行逻辑综合或实现。

您也可以使用“Heat Map”（热图）功能特性来高亮多个感兴趣的指标：

- II（最小值、最大值、平均值）
- 时延（最小值、最大值、平均值）
- 资源使用情况
- 停滞时间百分比

图 121：性能指标



热图使用颜色编码来高亮存在问题的模块。它使用红色到绿色的色度，其中红色表示指标值高（最高的 II 或最高的时延），而绿色则表示指标值低（存在问题）。非红非绿的颜色表示介于最高值与最低值之间的值的范围。如上所示，这样有助于快速识别需要注意的模块。在以上示例中，显示了 LATENCY 热图，红色模块表示观测到高时延的位置。

如下所述，Function Call Graph 显示了高层次设计的吞吐量值。用户可以查看 Function Call Graph 并将其作为总控制台，从中执行进一步深入调查。右键单击任意显示的模块即可显示选项菜单，供您用于显示更多信息。这样您即可查看整体设计，随后跳转到设计中需要额外注意的具体部分。其他报告还包括“Schedule Viewer”（调度查看器）、“Synthesis Summary”（综合汇总）报告、“Dataflow Viewer”（数据流查看器）和源文件。“Function Call Graph”允许您纵观整个设计，并获取每个模块的时延和 II 信息，这些信息可用于分析。

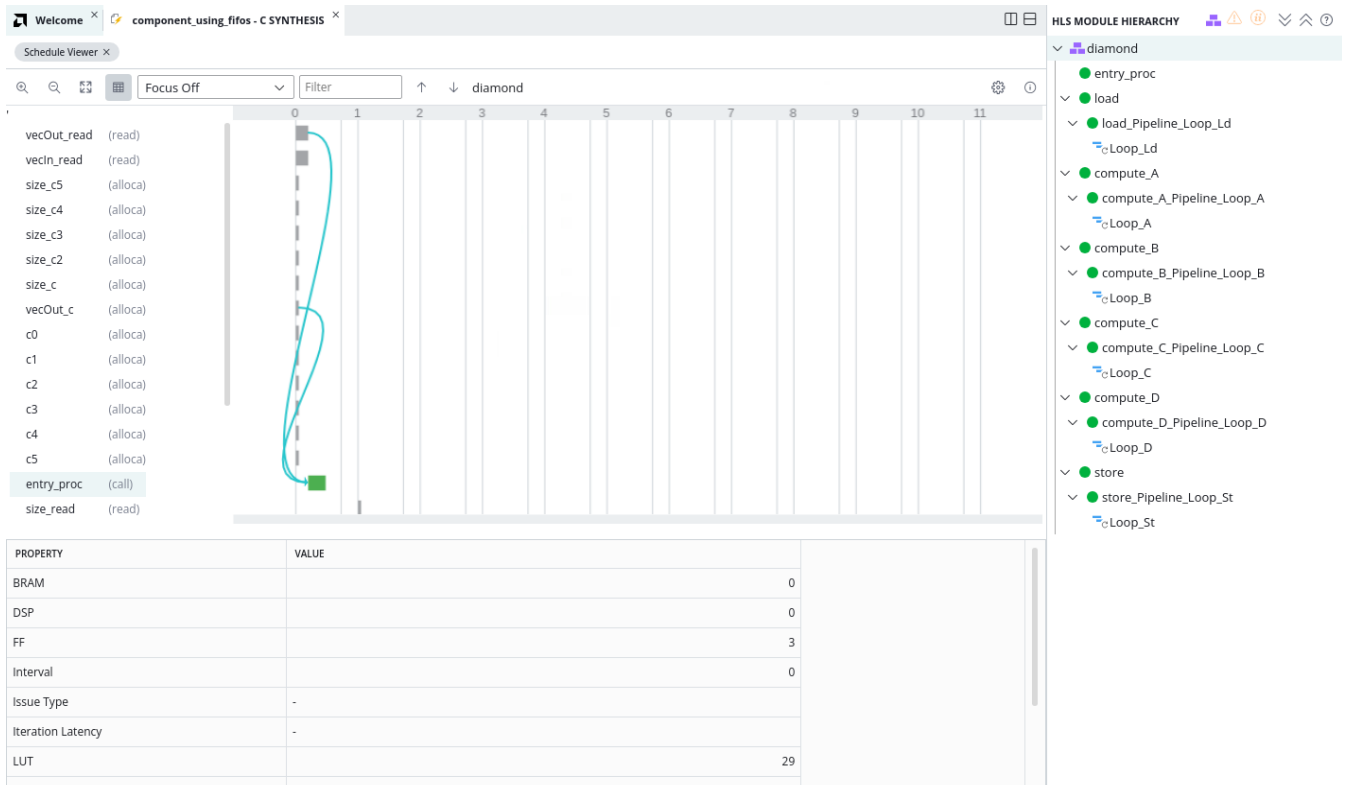


提示：在报告右侧有“Settings”（设置）命令和“Legend”（图例）命令，以便您显示报告的附加详细信息。

调度查看器

“Schedule Viewer”（调度查看器）可提供已综合 RTL 的详细视图，用于展示功能的每个操作和控制步骤，以及执行步骤的时钟周期。它有助于您识别阻碍并行化的任意循环依赖关系、时序违例以及数据依赖关系。

图 122：调度查看器



- 左侧纵轴显示的是将在 RTL 层级中作为逻辑来实现的操作和循环的名称。操作按拓扑顺序执行，这暗示任一行上的操作只能由前一行的操作驱动，并且只能驱动后一行中的操作。根据发现的违例类型，“Schedule Viewer”会显示每项操作的额外信息



提示：每项操作都与源代码行关联。右键单击操作即可使用“Goto Source”（转至源代码）命令打开与操作关联的输入源代码。

- 顶部横轴显示连续的时钟周期。
- 每个时钟周期内的竖轴虚线显示由于时钟不确定性而保留的时钟周期部分。此时间即工具保留用于 Vivado 后端进程（如布局布线）的时间。
- 每项操作在表中均显示为灰框。根据操作延迟占总时钟周期百分比来设置此框的横向大小。对于函数调用，提供的周期信息与操作时延相等。多周期操作显示为灰框，框中间有一条水平线贯穿。
- “Schedule Viewer”还会将常规运算符数据依赖关系显示为蓝色实线。如上图所示，选择操作时，您可看到实体蓝色箭头高亮特定的运算符依赖关系。这使您能够对数据依赖关系执行详细分析。绿色虚线表示相互迭代数据依赖关系。存储器依赖关系则以金色线条来显示。

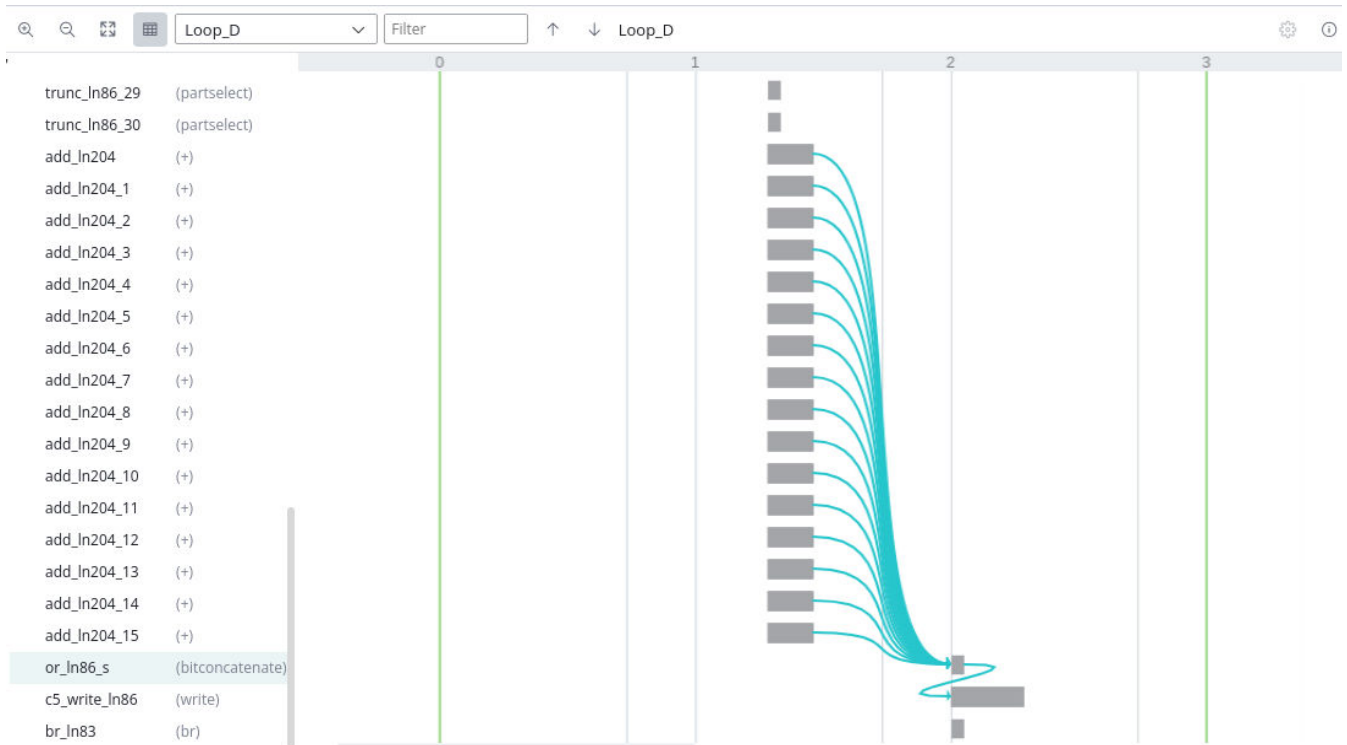


提示：默认情况下，其中可显示关键时序路径中的每项操作之间的所有依赖关系（蓝色线条）。

- 在右侧远端的“Schedule Viewer”（调度查看器）旁会显示“HLS Module Hierachy”（HLS 模块层级），以便您快速浏览设计层级。

在下图中，已选中名为 COL_DCT_LOOP_DCT_OUTER_LOOP 的循环。这是经流水打拍的循环，在循环栏中已显式声明启动时间间隔 (II)。所有流水打拍循环均处于以可视化方式展开状态，这表示在调度查看器中显示 1 次完整迭代。II 定义的重叠以循环标记上加粗的时钟边界来标记。单次迭代的总时延等同于循环标记所覆盖的周期数。在此情况下，总时延为 6 个时钟周期。

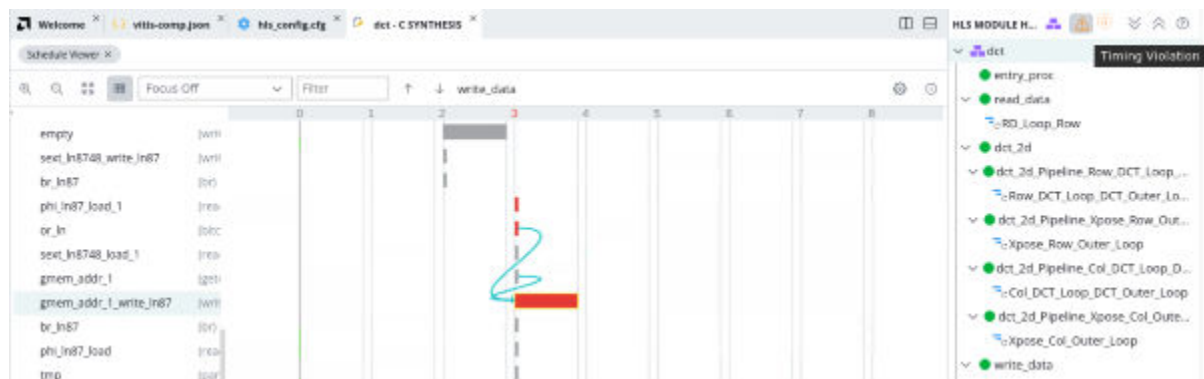
图 123：循环流水打拍



“Schedule Viewer” 会在报告顶部显示一个菜单栏，其中包含以下功能特性：

- “Zoom In”（放大）、“Zoom Out”（缩小）/ “Zoom Fit”（缩放适应）命令
- 用于为特定操作或步骤输入搜索词的文本字段；以及用于在与搜索词匹配的对象列表中上下滚动的方向键命令：“Previous Match”（前一项匹配）或“Next Match”（后一项匹配）
- “Legend”（图例）命令用于显示图例。

图 124：时序违例



您可使用“Module Hierarchy”视图中的工具栏来快速定位时序违例和 II 违例，如上图所示。要定位导致源代码中出现违例的操作，请右键单击操作并使用“Goto Source”（转至源码）命令。时序违例是一条操作路径，其所需时间超过可用时钟周期。为便于直观显示，在“Schedule Viewer”中会以红色框显示存在问题的操作。

属性视图

如上图所示，在“Schedule Viewer”底部有“Properties”（属性）视图，该视图可显示“Schedule Viewer”中当前所选对象的属性。它可便于您查看“Schedule Viewer”中选中的特定函数、循环或操作的详细信息。可选中的元素类型以及显示的属性包括：

- 函数或循环
 - “Initiation Interval (II)”（启动时间间隔）：表示函数或循环可接受新输入数据之前的时钟周期数。
 - “Loop Iteration Latency”（循环迭代时延）：表示完成循环的单次迭代所需的时钟周期数。
 - “Latency”（时延）：表示函数计算所有输出值或者循环完成所有迭代所需的时钟周期数（和时间）。
 - “Name”（名称）：函数或循环的名称。
 - “Pipelined”（流水打拍）：表示在 RTL 设计中对函数或循环进行流水打拍。
 - “Slack”（裕量）：表示函数或循环的时序裕量。
 - “Tripcount”（循环次数）：表示循环完成的迭代数。
 - “Resource Utilization”（资源使用率）：显示用于实现函数或循环的 BRAM、DSP、LUT 或 FF 的数量。
- 运算和存储映射
 - “Bitwidth”（位宽）：对应运算的位宽。
 - “Impl”（实现）：定义用于指定运算或存储的实现。
 - “Name”（名称）：操作名称。
 - “Op Latency”（运算时延）：显示运算或存储的绑定的默认时延或指定时延。
 - “Opcode”（操作代码）：表示调度的运算，例如，`add`、`sub` 和 `mult`。如需了解更多信息，请参阅 `BIND_OP` 或 `BIND_STORAGE` 编译指示或指令。
 - “Schedule Delay”（调度延迟）：指定与操作关联的延迟。

数据流查看器

DATAFLOW 优化属于动态优化，只有在完成 RTL 协同仿真后才能完全理解。有鉴于此，完成 C 语言综合后，“Dataflow”（数据流）查看器允许您查看工具推断所得的数据流结构、检查通道 (FIFO/PIPO) 并检验通道深度对于性能的影响。完成协同仿真后，将在“Dataflow”查看器中给性能数据添加反标注解。



重要提示！ 您无需运行 RTL 协同仿真亦可打开“Dataflow”查看器，但这样视图中将不包含重要的性能信息，例如读/写阻塞时间、协同仿真深度和停滞时间。

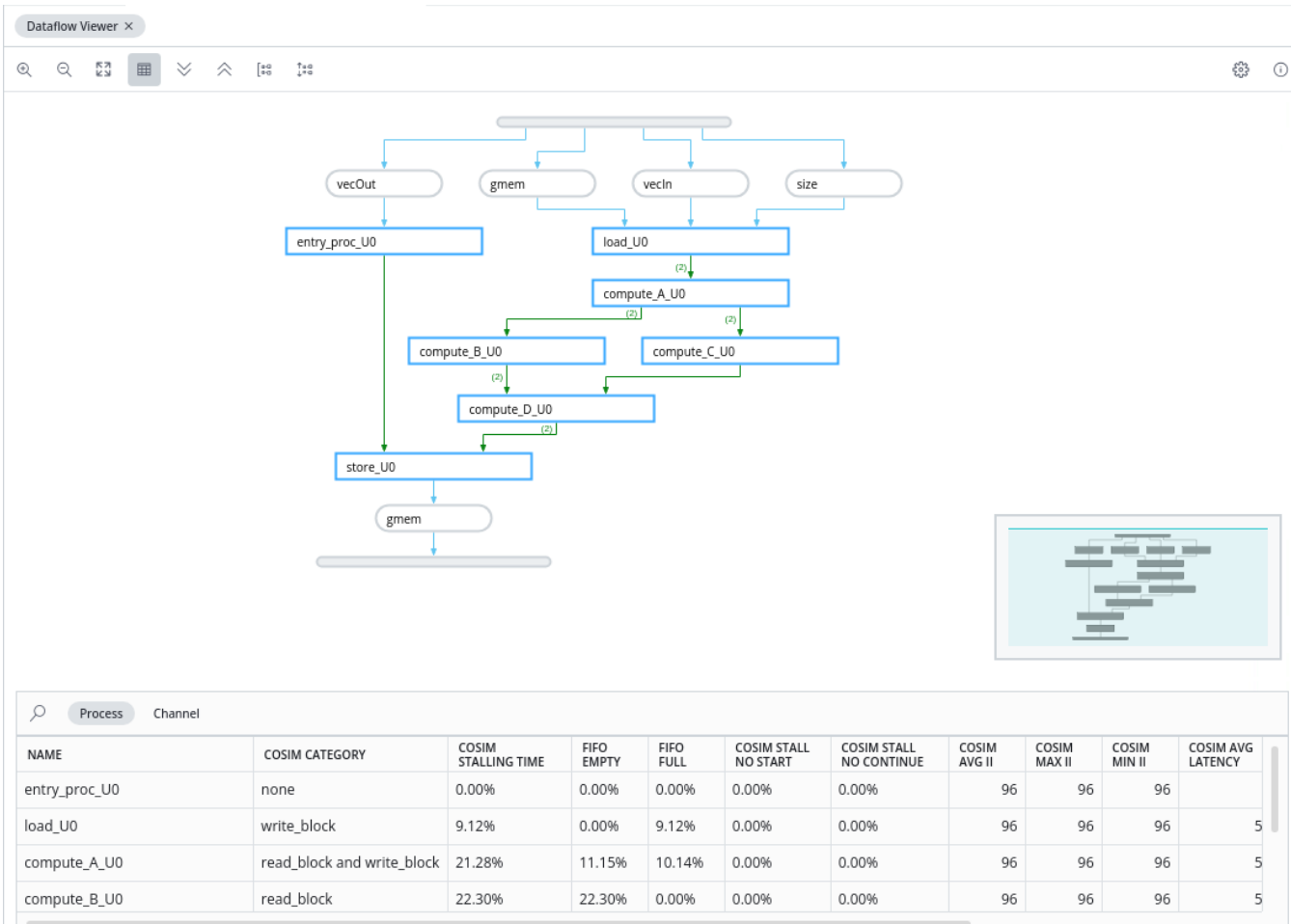
您必须向设计应用 DATAFLOW 编译指示或指令方可在“Dataflow Viewer”报告中生成这些信息。您可将数据流应用于顶层函数，或者应用于函数或循环的特定区域。“Dataflow”查看器会显示数据流图结构表示法，以显示不同进程和底层生产者/使用者连接。“Synthesis Summary”（综合汇总）、“Module Hierarchy”（模块层级）和“Function

Call Graph”（函数调用计算图）将在顶层函数旁显示  图标，以表明存在 DATAFLOW 编译指示。



提示： 下图是从 Vitis 内核流程的默认 `m_axi` 接口生成的。使用 Vivado IP 流程可能导致生成不同的数据流图。

图 125：数据流查看器



“Dataflow” 查看器的功能特性包括：

- 源代码浏览器
- 从进程/通道到源代码的自动交叉探测。
- 筛选端口和通道类型。
- “Process and Channel”（进程和通道）表详列了设计的特性：
 - “Channel Profiling”，即“通道剖析”（FIFO 大小等），在“Config File Editor”（配置文件编辑器）的“C/RTL Cosimulation”（C/RTL 协同仿真）设置中启用。

★ **重要提示！** 您必须在 HLS 配置文件中使用 `cosim.enable_dataflow_profiling=true` 为“Dataflow”查看器捕获数据，并且测试平台必须运行顶层函数的至少 2 次迭代。

- RTL 协同仿真后报告进程读取阻塞/写入阻塞/停滞时间。
- 显示进程时延和 II。
- 通道类型和宽度均显示在“Channel”表中。
- 从“Process”和“Channel”表到“Graph”和“Source”浏览器之间的自动交叉探测。

- 悬停于通道或进程上即可显示工具提示（含设计信息）。

“Dataflow”查看器可帮助设计进行性能调试。RTL 协同仿真期间，如果设计发生死锁，那么 GUI 将打开“Dataflow”查看器并高亮死锁所设计的通道和进程，以便您判定原因，如 FIFO 深度不足等。

如果设计表现不及预期，那么“Process”和“Channel”表还能帮助您了解原因。进程可能发生停滞并等待读取输入，或者可能无法写入输出而导致停滞。“Channel”表可以提供停滞百分比并识别进程是处于“读取阻塞”还是“写入阻塞”状态。

“Dataflow Viewer”（数据流查看器）会在报告顶部显示一个菜单栏，其中包含以下功能特性：

- “Zoom In”（放大）、“Zoom Out”（缩小）/“Zoom Fit”（缩放适应）命令
- “Toggle Table”（切换表格）命令，用于显示或隐藏计算图下方的表格
- “Group All” / “Ungroup All”（全部组合/全部取消组合）用于将关联的通道组合在一起以简化此图例
- “Legend”（图例）命令用于显示图例。

运行 C/RTL 协同仿真

在“Flow Navigator”中，确保 HLS 组件处于活动状态，或者在“Component”（组件）菜单中将其选中，在该工具中将其设为活动组件。当 HLS 组件为活动组件时，“Flow Navigator”支持运行 C 语言仿真、C 语言综合、C/RTL 协同仿真、封装与实现来构建和分析该 HLS 组件。要对 HLS 组件进行 C/RTL 协同仿真，请选中 Flow Navigator 的“C/RTL COSIMULATION”（C/RTL 协同仿真）标题下的“Run”（运行）。

在 HLS 组件上运行协同仿真需要一个测试平台或多个测试激励文件，您必须加载这些文件。运行协同仿真前，还必须配置 HLS 组件以支持 C/RTL 协同仿真。使用以下步骤来配置设计。



重要提示！ 不支持对 C/C++ 测试平台（例如：`std::random_device`）中的随机输入测试矢量执行协同仿真。建议一次性生成随机值并保存到一个文件中，随后，C/C++ 测试平台可在协同仿真期间使用此文件。

配置仿真器

您可在运行仿真前使用“Config Editor”（配置编辑器）的“C/RTL Co-Simulation”（C/RTL 协同仿真）部分来配置仿真器，如下所示。

图 126：HLS 组件 RTL 协同仿真设置

C/RTL Cosimulation**o**☐ Enables optimize compilation of the C testbench and RTL wrapper. Without optimization, cosim_design will compile the testbench as quickly as possible.**argv**

Option to specify the argument list for the behavioral testbench. The <string> will be passed on to the main C function.

compiled_library_dir

Option to specify the compiled library directory during simulation with 3rd party simulators.

 [Browse](#)**coverage**☐ This option enables the coverage feature during simulation with the VCS simulator.**disable_binary_tv**☐ Use plain text binary test vector files instead of binary compressed format**disable_deadlock_detection**☐ Disables deadlock detection

这些配置命令允许您指定仿真的运行方式。[协同仿真配置](#) 中记述了所有这些配置命令。请参阅其中内容获取更详细的信息。“C/RTL Co-Simulation”的部分设置包括：

- “trace_level”（转储追踪）：该设置可指定在执行仿真时，写入当前解决方案的 `sim/Verilog` 或 `sim/VHDL` 目录的追踪文件输出级别。选项包括：
 - “all”（全部）：表示输出当前正保存到追踪文件的所有端口和信号波形数据。
 - “port”（端口）：表示仅为顶层端口输出波形追踪数据。
 - “port_hier”（端口层级）：表示完整端口层级的输出波形追踪数据。
 - “none”（无）：表示不输出追踪数据。
- “random_stall”（随机停滞）：表示对每次数据发射都应用随机停滞。
- “wave_debug”（波形调试）：表示在 RTL 仿真中启用所有进程的波形可视化。仅当使用 Vivado 逻辑仿真器时，才支持该选项。启用该选项将启动仿真器 GUI 以便您检查波形中仿真生成的数据流活动。如需了解有关该工具的更多信息，请参阅《Vivado Design Suite 用户指南：逻辑仿真》([UG900](#))。
- “disable_deadlock_detection”（禁用死锁检测）：禁用死锁检测，并在协同仿真中打开协同仿真死锁检查。
- “enable_dataflow_profiling”（启用数据流剖析）：用于启用捕获剖析数据，以供显示在 [数据流查看器](#) 中。
- “Dynamic Deadlock Prevention”（防止动态死锁）：表示在协同仿真期间，为数据流剖析启用自动 FIFO 通道大小调整，从而防止出现死锁。

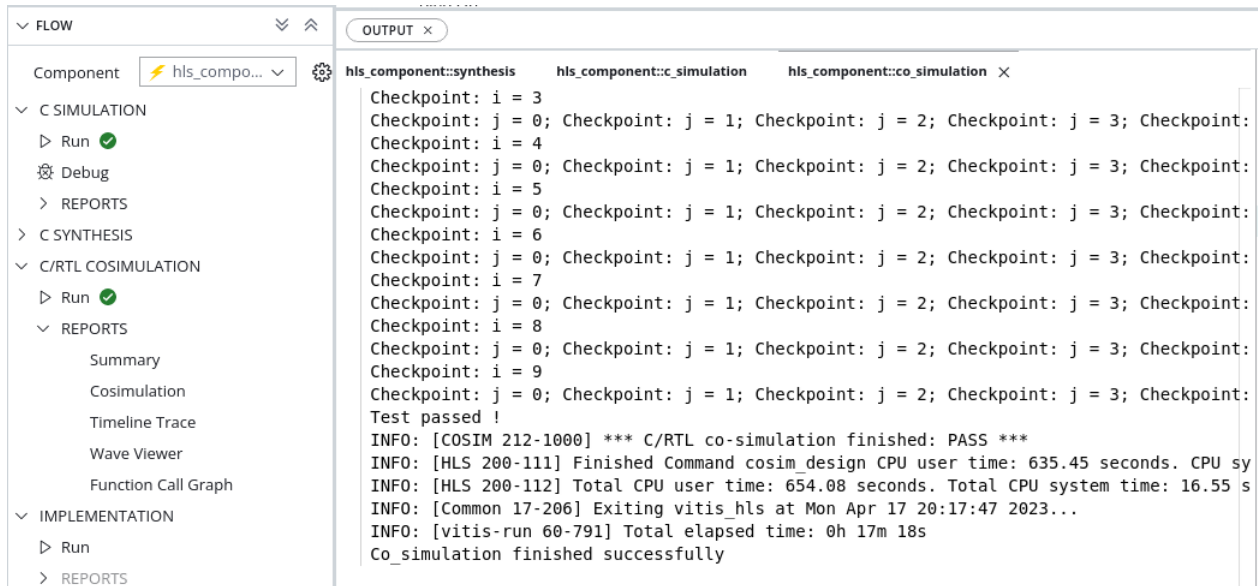
运行 C/RTL 协同仿真

在配置文件中定义 C/RTL 协同仿真设置后，即可从“Flow Navigator”中选中“Run”开始仿真。您可在“Output”（输出）窗口中跟踪仿真进度。综合运行的转录文本所含顶层函数名为 `<component-name>::co_simulation`，如下所示。



提示： Vitis Unified IDE 中的实现运行使用的是 `vitis-run --mode hls --cosim` 命令。

图 127：运行 C/RTL 协同仿真的 HLS 组件



完成综合后，应在转录文本底部显示“Co_simulation finished successfully”（协同仿真成功完成）消息。在“Run”命令下还会显示填充的“Reports”文件夹，如上图所示。协同仿真后可用的报告包括：

- “Summary”（汇总）：报告使用的命令行和结果的时间戳。
- “Cosimulation”（协同仿真）：显示设计的常规信息，显示协同仿真过程中使用的特定选项，并显示设计层级的性能和资源估算。如果启用 `enable_dataflow_profiling`，那么性能数据将反标注解回“Dataflow Viewer”（数据查看器）报告，如 [数据流查看器](#) 所述。



重要提示！ 如果要计算 II，则必须确保在 RTL 仿真中至少有 2 项传输事务。

- “Timeline Trace”（时间线轨迹）：如 [时间线轨迹查看器](#) 所述。
- “Wave Viewer”（波形查看器）：如 [查看仿真波形](#) 所述。
- “Function Call Graph”（函数调用计算图）：显示协同仿真后的调用计算图，如 [函数调用计算图](#) 所述。

C/RTL 协同仿真输出

在 Vitis Unified IDE 中，HLS 组件层级的结构如下：`<workspace>/<component>/<component>/hls/`。您可在“Vitis Component Explorer”（Vitis 组件资源管理器）中选中 HLS 组件，然后从右键单击菜单中选中“Open In Terminal”（在终端中打开），以浏览组件文件夹的内容。

C/RTL 协同仿真完成后，就会在 solution 文件夹内创建 `sim` 文件夹。该文件夹包含以下元素：

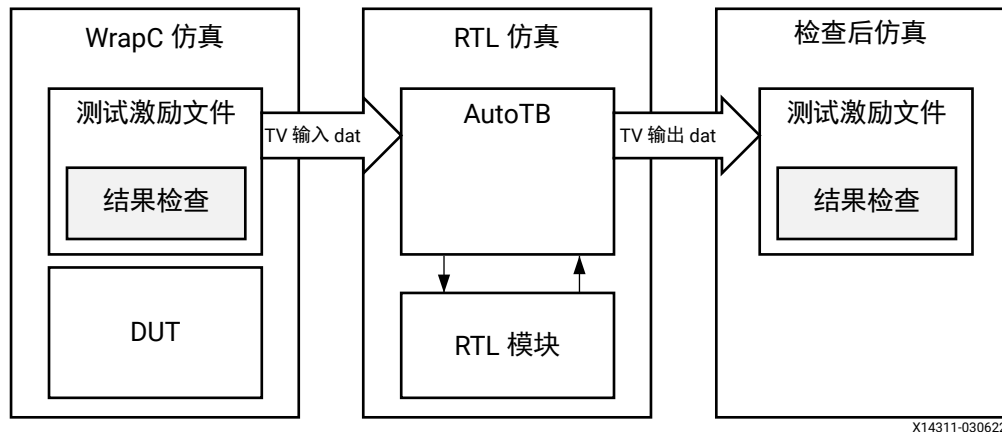
- 对于验证的每一种 RTL 语言，都会创建一个验证文件夹，名为 `sim/verilog` 或 `vhdl`。
 - 用于仿真的 RTL 文件存储在 `verilog` 或 `vhdl` 文件夹内。
 - RTL 仿真在验证文件夹内执行。
 - 所有输出（如追踪文件和波形文件）都写入 `verilog` 或 `vhdl` 文件夹。
- `sim/report` 文件夹中包含对应仿真的每一类 RTL 的报告和 log 日志文件。
- 其他文件夹（如 `sim/autowrap`、`tv`、`wrap` 和 `wrap_pc`）均为 HLS 编译器使用的工作文件夹。这些文件夹内不含任何用户文件。



提示：如果在配置文件中选中了 `cosim.setup` 选项，那么会在 `sim/<hdl>` 文件夹内创建一个可执行文件，但不会运行仿真。您可通过在终端内执行仿真 `.sh` 或 `.exe` 来手动运行仿真。

自动验证 RTL

图 128：C/RTL 验证流程



X14311-030622

C/RTL 协同仿真流程使用运行 `main()` 函数的 C 语言测试平台来自动验证行为仿真中运行的 RTL 设计。C/RTL 验证进程由 3 个阶段组成：

- 执行 C 语言仿真，顶层函数输入或被测设计 (DUT) 将保存为“输入矢量”。
- “输入矢量”在 RTL 仿真中使用，此类 RTL 仿真使用的是在 Vivado 仿真器或受支持的第三方 HDL 仿真器中由 C 语言综合创建的 RTL。RTL 输出或仿真结果将保存为“输出矢量”。
- RTL 仿真返回的“输出矢量”将返回至 C 语言测试平台的 `main()` 函数，以验证结果是否正确。C 语言测试平台在某些情况下通过与已知良好的结果相比较来执行结果验证。

在验证进程中，HLS 编译器会输出以下消息：

运行 C 语言仿真时：

```
INFO: [COSIM 212-14] Instrumenting C test bench ...  
Build using ".../bin/g++"  
Compiling dct_test.cpp_pre.cpp.tb.cpp  
Compiling dct_inline.cpp_pre.cpp.tb.cpp  
Compiling apatb_dct.cpp  
Generating cosim.tv.exe  
INFO: [COSIM 212-302] Starting C TB testing ...  
Test passed !
```

在此阶段中，由于已执行 C 语言仿真，C 语言测试平台编写的消息都将输出到“Console”（控制台）窗口和 log 日志文件中。

运行 RTL 仿真时：

```
INFO: [COSIM 212-333] Generating C post check test bench ...  
INFO: [COSIM 212-12] Generating RTL test bench ...  
INFO: [COSIM 212-1] *** C/RTL co-simulation file generation completed. ***  
INFO: [COSIM 212-323] Starting verilog/vhdl simulation.  
INFO: [COSIM 212-15] Starting XSIM ...
```

在此阶段中，RTL 仿真生成的所有消息都将输出到控制台窗口或 log 日志文件中。

检查 C 语言测试平台返回的结果时：

```
INFO: [COSIM 212-316] Starting C post checking ...  
Test passed !  
INFO: [COSIM 212-1000] *** C/RTL co-simulation finished: PASS ***
```

C/RTL 协同仿真的要求如下所述：

- 测试平台必须进行自我检查（如 [撰写测试平台](#) 中所述），如果测试成功，则返回 0 值，如果测试失败，则返回非零值。
- [接口综合的联合仿真要求](#) 必须得到满足。
- 设计接口上的任意 arrays 或 structs 都不得使用 [不受支持的协同仿真优化](#) 中列示的优化指令。
- 搜索路径中必须包含所有第三方仿真器，以供 Vitis Unified IDE 或 vitis-run 命令启动。
- 如果使用第三方仿真器，那么 IP 仿真库必须经过编译后才能搭配第三方仿真器一起使用，如 [仿真 IP 核](#) 中所述。

接口综合的联合仿真要求

要使用 C/RTL 协同仿真功能来验证 RTL 设计，必须满足以下至少 1 项条件：

- 顶层函数必须使用 ap_ctrl_chain 或 ap_ctrl_hs 块级协议来进行综合
- 必须采用纯组合型设计
- 顶层函数的启动时间间隔必须为 1
- 接口包含的阵列必须全部使用 axis 或 ap_hs 接口模式来进行串流和实现

注释：hls::stream 变量作为 ap_fifo 接口自动实现。

如果这些条件都未能满足，C/RTL 协同仿真将中止并提示下列消息：

```
@E [SIM-345] Cosim only supports the following 'ap_ctrl_none' designs: (1)
combinational designs; (2) pipelined design with task interval of 1; (3)
designs with
array streaming or hls_stream ports.
@E [SIM-4] *** C/RTL co-simulation finished: FAIL ***
```



重要提示！ 如果设计指定为使用块级 I/O 协议 `ap_ctrl_none` 且设计包含采用非阻塞行为的任意 `hls::stream` 变量，则无法保证能否完成 C/RTL 协同仿真。

如果任意顶层函数实参指定为 AXI4-Lite 接口，则函数返回也必须指定为 AXI4-Lite 接口。

DATAFLOW 和 DEPENDENCE 的验证

C/RTL 协同仿真会自动验证 DATAFLOW 和 DEPENDENCE 指令的各方面。

如果 DATAFLOW 指令用于对任务进行流水打拍，那么它会在任务间插入通道以改善任务间的数据流。通常，通道是以使用 STREAM 指令或 `syn.dataflow.fifo_depth` 配置命令指定的 FIFO 和 FIFO 深度来实现的，如 [数据流配置](#) 中所述的。

如果 FIFO 深度太小，RTL 仿真可能停滞。例如，如果指定的 FIFO 深度为 2，但在使用者任务读取任意数据值之前，生产者任务写入 3 个值，那么 FIFO 会阻止生产者操作。在某些情况下，这可能导致整个设计停滞，如 [协同仿真死锁检测](#) 中所述。

在此情况下，C/RTL 协同仿真会发出如下消息，以表明当前 DATAFLOW 区域中的通道导致 RTL 仿真停滞。

```
////////////////////////////////////
//
// ERROR!!! DEADLOCK DETECTED at 1292000 ns! SIMULATION WILL BE STOPPED! //
////////////////////////////////////
//
// Dependence cycle 1:
// (1): Process: hls_fft_1kxburst.fft_rank_rad2_nr_man_9_U0
//      Channel: hls_fft_1kxburst.stage_chan_in1_0_V_s_U, FULL
//      Channel: hls_fft_1kxburst.stage_chan_in1_1_V_s_U, FULL
//      Channel: hls_fft_1kxburst.stage_chan_in1_0_V_1_U, FULL
//      Channel: hls_fft_1kxburst.stage_chan_in1_1_V_1_U, FULL
// (2): Process: hls_fft_1kxburst.fft_rank_rad2_nr_man_6_U0
//      Channel: hls_fft_1kxburst.stage_chan_in1_2_V_s_U, EMPTY
//      Channel: hls_fft_1kxburst.stage_chan_in1_2_V_1_U, EMPTY
////////////////////////////////////
// Total 1 cycles detected!
////////////////////////////////////
```

“Dataflow Viewer”（数据流查看器）将显示以红色显示的死锁中所涉及的任何进程。它还将显示哪些通道已满（红色）、哪些通道已空（白色）。在此情况下，请复查任务间通道的实现，确保所有 FIFO 大小足够保存所生成的数据。

RTL 测试平台也可采用类似方式来配置，以自动检查使用 DEPENDENCE 指令指定的假性依赖关系的有效性。协同仿真期间会显示警告消息，表明依赖关系并非假性依赖关系，并且必须移除对应方向以实现功能有效的的设计。



提示： `cosim.disable_deadlock_detection=true` 配置命令会禁用这些检查。

不受支持的协同仿真优化

对于 Vivado IP 模式，RTL 自动验证不支持对接口上的阵列或结构体内的阵列执行多次变换。



重要提示！ Vitis 内核流程不支持该功能特性。

为了执行自动验证，函数接口上的阵列或者函数接口上的结构体内的阵列可使用以下任一优化，但不得同时使用 2 种或 2 种以上优化操作：

- 对相同大小的阵列执行垂直映射
- 重构
- 分区（适用于阵列的维度 1）

RTL 自动验证不支持对顶层函数接口使用下列任一优化：

- 水平映射。
- 对不同大小的阵列执行垂直映射。
- 对已启用寄存器切片 (slice) 的 AXI4-Stream 执行条件访问。
- 将阵列映射到串流。

仿真 IP 核

使用浮点核来实现设计时，浮点核的位精确模型必须可供 RTL 仿真器使用。如果使用 Vivado 逻辑仿真器执行 RTL 仿真，则会自动完成此操作。但是，对于受支持的 HDL 第三方仿真器，AMD 浮点库必须预编译并添加到仿真器库中。

例如，要在 Verilog 中编译 AMD 浮点库以搭配 VCS 仿真器一起使用，请打开 Vivado IDE，在 Tcl 控制台窗口中输入以下命令：

```
compile_simlib -simulator vcs_mx -family all -language verilog
```

这样即可在 VCS 的当前目录中创建浮点库。请参阅 Vivado 的“Tcl Console”窗口以获取目录名称。在此示例中，名称为 `./rev3_1`。

您必须在 Vitis Unified IDE 中引用此库，方法是指定 `cosim.compiled_library_dir` 配置文件（如 [协同仿真配置](#) 中所述），或者使用以下命令运行 C/RTL 协同仿真：

```
cosim_design -tool vcs -compiled_library_dir <path_to_library>/rev3_1
```

分析 RTL 仿真

当 C/RTL 协同仿真完成后，会打开仿真报告并显示测量所得时延和 II。这些结果将不同于 HLS 综合后报告的值，后者为基于穿过设计的绝对最短路径和绝对最长路径计算所得。C/RTL 协同仿真后提供的结果会显示给定仿真数据集的时延和 II 的实际值（如果改变使用的输入激励，则结果可能发生改变）。

在非流水打拍设计中，C/RTL 协同仿真会测量 `ap_start` 与 `ap_done` 信号之间的时延。II 比时延多 1，因为设计会在所有操作完成 1 个周期后读取新输入。设计仅在当前传输事务完成后才启动下一项传输事务。

在流水打拍设计中，设计可能在第一项传输事务完成之前读取新输入，并且在传输事务完成前可能存在多个 `ap_start` 和 `ap_ready` 信号。在此情况下，C/RTL 协同仿真会测量数据输入值与数据输出值之间的周期数作为时延。II 是 `ap_ready` 信号之间的周期数，设计会使用该值来请求新输入。

注释：对于流水打拍设计，仅当设计针对多个传输事务进行仿真时，才能判定 C/RTL 协同仿真的 II 值。

查看仿真波形

要在 RTL 协同仿真期间查看波形数据，必须在“Config File Editor”（配置文件编辑器）或配置文件中启用以下项：

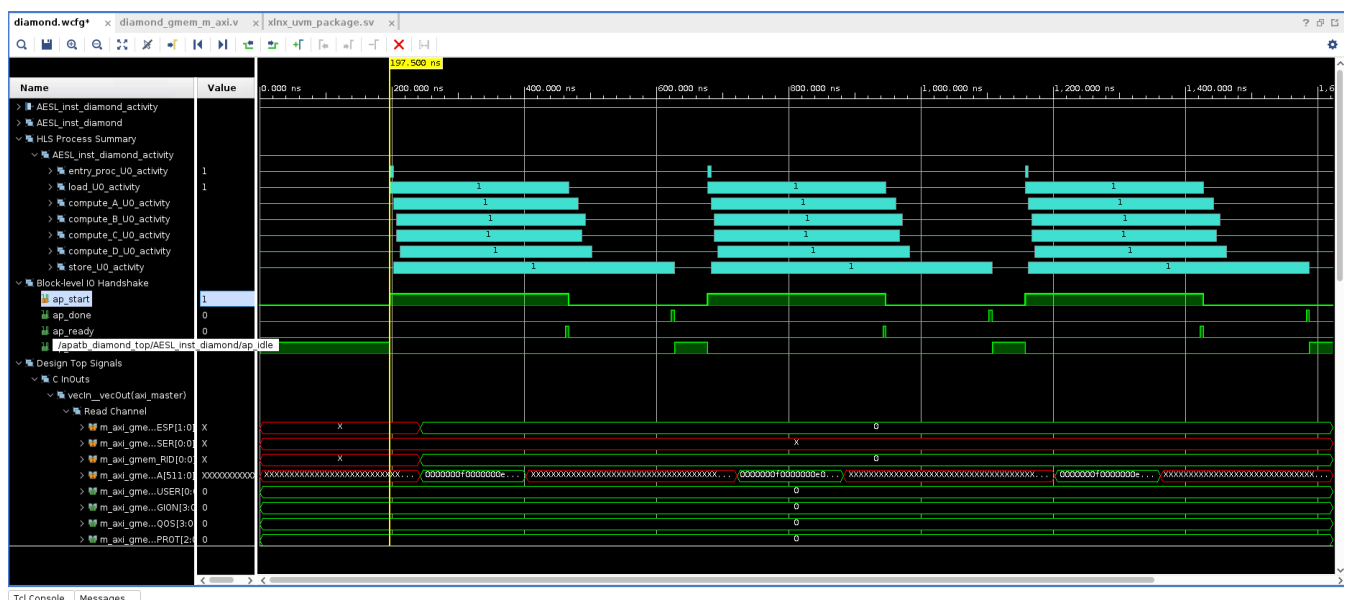
- 在“tool”（工具）下，选中“xsim”作为 RTL 仿真器，或者在配置文件中设置 `cosim.tool=xsim`
- 在“trace_level”（追踪级别）下，选中“port”（端口）或“all”（全部），或者在配置文件中设置 `cosim.trace_level=port`

这样会打开 Vivado 仿真器 GUI，并显示 RTL 设计中的所有进程。将 HLS 设计内的活动进程可视化，以便对顶部模块每次激活期间的进程活动和持续时间进行详细剖析。这些可视化有助于对各进程的性能以及独立进程的总体并发执行情况进行分析。只要能够缩短进程执行时间，主导总体执行的进程就极有可能提升性能。

此可视化分为 2 个部分：

- HLS 进程汇总信息包含所有进程的活动报告的分层表示法。
 - “DUT name”：<名称>
 - “Function”：<函数名称>
- 数据流分析可提供有关数据流区域内的任务的详细活动信息。
 - “DUT name”：<名称>
 - “Function”：<函数名称>
 - “Dataflow/Pipeline Activity”：用于显示作为数据流进程实现时，函数的并行执行数。
 - “Active Iterations”：用于显示数据流的当前活动迭代数。行数将动态递增以适应任意并发执行的可视化。
 - “StallNoContinue”：这是一个停滞信号，表示数据流进程是否遇到任何输出停滞（函数已执行，但是未能从相邻的数据流进程接收到继续执行信号）。
 - “RTL Signals”：这是底层 RTL 控制信号，用于解读数据流进程的传输事务视图。

图 129：波形查看器



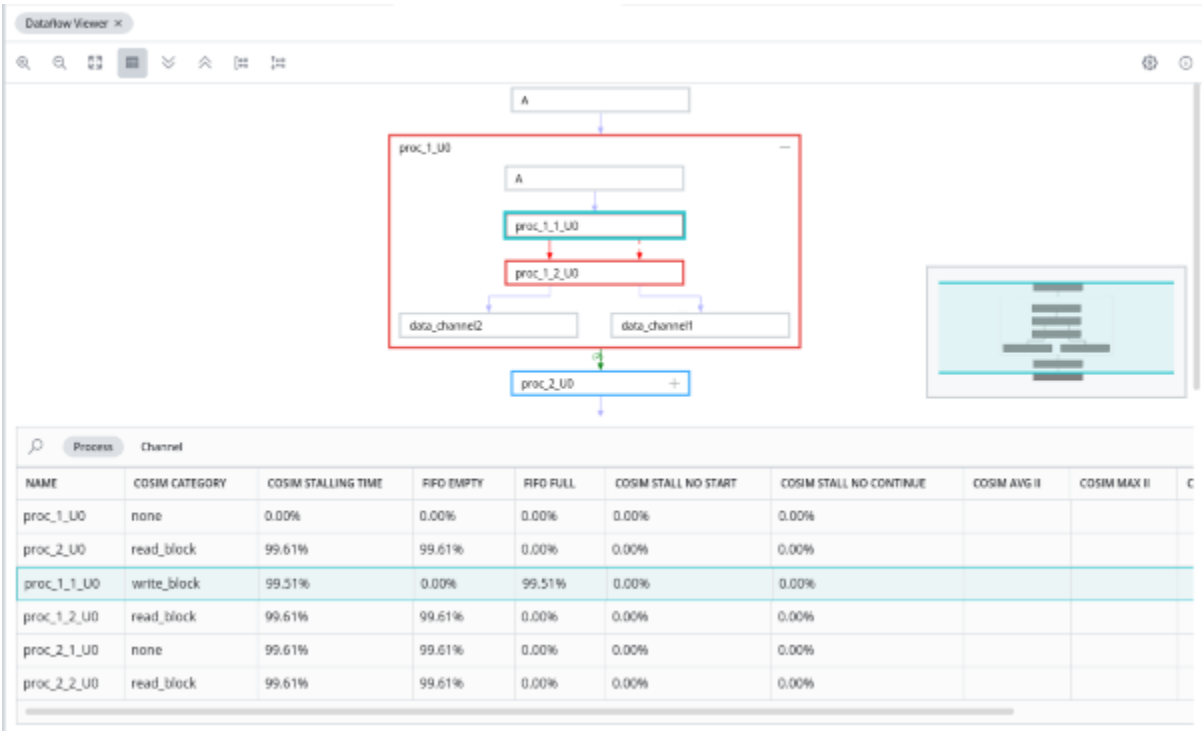
完成 C/RTL 协同仿真后，可在 Vivado IDE 中选中“C/RTL COSIMULATION”（C/RTL 协同仿真）的“REPORTS”（报告）报头下的“Wave Viewer”（波形查看器）来重新打开 RTL 波形。

 **重要提示！** 使用此方法打开 Vivado IDE 后，由于先前已生成波形，因此您只能使用波形分析功能，例如，缩放、平移和波形基数。

协同仿真死锁检测

所谓死锁指的是在某个 DATAFLOW 区域内，由于进程共享相同的通道，阻碍彼此读取或写入该区域，从而导致这两个进程都卡死。如果 DATAFLOW 内部存在 FIFO 通道或者混用 PIPO 和 FIFO 作为通道，那么这种状况很常见。如下图所示，死锁检测可在“Dataflow Viewer”（数据流查看器）报告中直观显示这种情况。

图 130：死锁查看器



死锁检测功能默认启用，它可在 C/RTL 协同仿真过程中遇到有问题的进程和通道时，在数据流查看器中将其高亮显示。当协同仿真检测到死锁状况后，“Dataflow”报告会自动打开。此查看器还提供交叉探测功能，在有问题的数据通道与关联的源代码之间建立链接。您可使用这些信息来快速轻松解决问题。您可使用 `cosim.disable_deadlock_detection` 配置文件命令禁用此功能。

在以上示例中，数据流区域包含 2 个通过 PIPO 和 FIFO 来进行通信的进程。proc_1 中的首个循环会在 data_array 中写入任何内容之前，先在 data_channel1 中写入 10 个数据项。由于 FIFO 深度不足，data_channel1 无法完成，导致其余进程无法完成。随后，proc_2 也会停滞，因为它无法从 data_channel2 读取数据（因为该通道为空），并且无法从 data_channel1 移除任何数据。这样就会造成死锁，需将 data_channel1 大小至少增大至 10。在 GitHub 上 [03-dataflow_debug_and_optimization](#) 教程的死锁检测部分中演示了该示例。

C/RTL 协同仿真进程输出如下所示。由于死锁检测，仿真会结束并报错，并且“Dataflow Viewer”报告会自动打开。

```

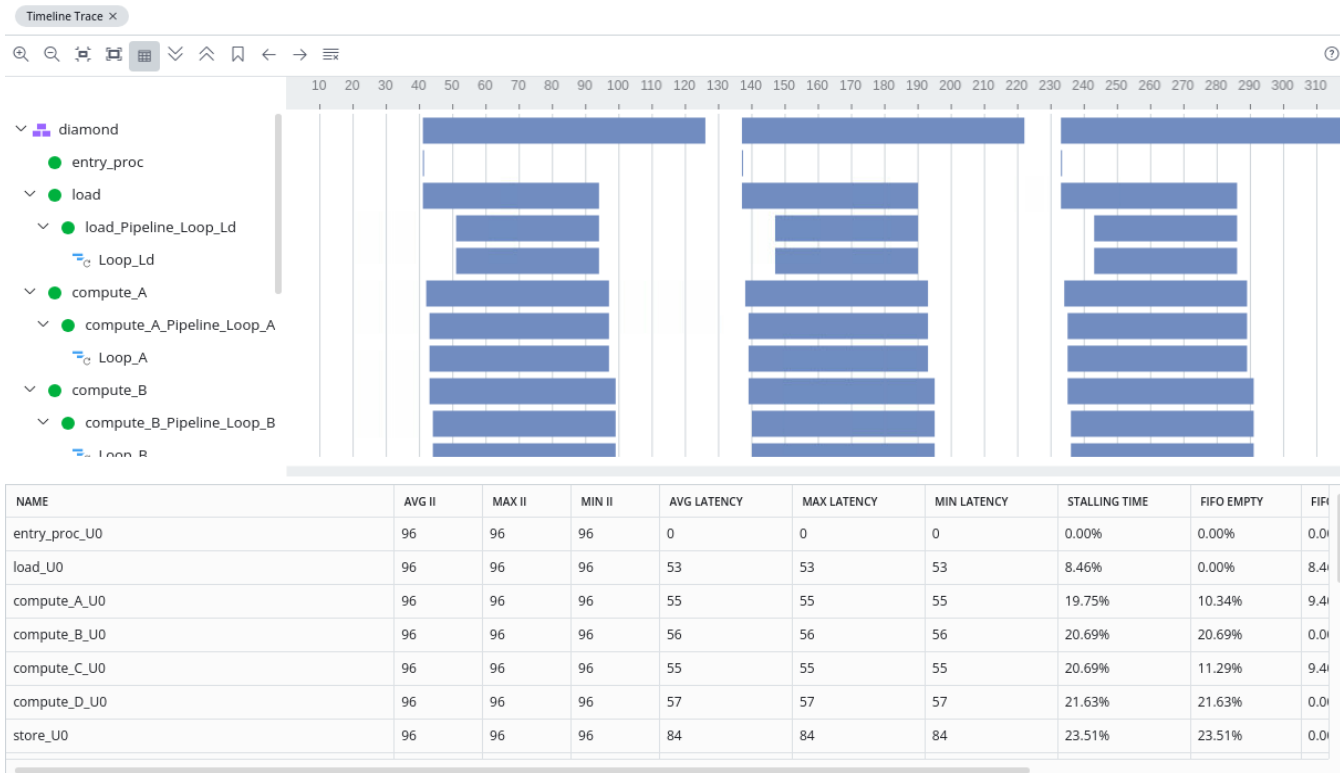
/////////////////////////////////////////////////////////////////
// Inter-Transaction Progress: Completed Transaction / Total Transaction
// Intra-Transaction Progress: Measured Latency / Latency Estimation * 100%
//
// RTL Simulation : "Inter-Transaction Progress" ["Intra-Transaction
Progress"] @ "Simulation Time"
/////////////////////////////////////////////////////////////////
//
// RTL Simulation : 0 / 1 [0.00%] @ "105000"
/////////////////////////////////////////////////////////////////
//
// ERROR!!! DEADLOCK DETECTED at 132000 ns! SIMULATION WILL BE STOPPED! //
/////////////////////////////////////////////////////////////////
//
// Dependence cycle 1:
// (1): Process: example_example.proc_1_U0
//       Channel: example_example.data_channel1_U, FULL
// (2): Process: example_example.proc_2_U0
//       Channel: example_example.data_array_U, EMPTY
/////////////////////////////////////////////////////////////////
// Totally 1 cycles detected!
/////////////////////////////////////////////////////////////////

```

时间线轨迹查看器

完成 C/RTL 协同仿真后，“Timeline Trace”（时间线轨迹）报告即可供使用。“Timeline Trace”查看器可显示设计的函数的运行时剖析。它最适合用于查看协同仿真后数据流区域的行为，因为无需启动 Vivado 逻辑仿真器即可查看时间线。

图 131：时间线轨迹查看器



时间线轨迹查看器会显示通过数据流区域的各子函数的多次迭代，如上图所示。它会在时间线下的表格中显示函数的起始和结束位置并显示协同仿真数据。要在 C/RTL 协同仿真期间生成时间线轨迹，应在配置文件中或者从“Config File Editor”（配置文件编辑器）启用 `cosim.trace_level=all` 选项和 `cosim.enable_dataflow_profiling=true` 选项。

“Timeline Trace”视图还会以“Full”（满）标记和“Empty”（空）标记显示 FIFO 和 PIPO 通道的停滞/匮乏状态，如上所示。在上图中可见，`read_data` PIPO 为空，导致有 68% 的时间出于停滞状态，如计算图下方的表格中报告所示。

“Dataflow Viewer”（数据流查看器）会在报告顶部显示一个菜单栏，其中包含以下功能特性：

- “Zoom In” / “Zoom Out” / “Zoom Fit” / “Zoom Full”（放大/缩小/缩放适应/缩放至全屏）命令
- “Toggle Table”（切换表格）命令，用于显示或隐藏计算图下方的表格
- “Expand All” / “Collapse All”（全部展开/全部折叠）用于展开或折叠设计层级
- “Previous Marker” / “Next Marker”（上一个标记/下一个标记）允许您在时间线上从一个标记移至另一个标记
- “Delete All Markers”（删除所有标记）会移除时间线上的标记
- “Legend”（图例）命令用于显示图例

对 C/RTL 协同仿真进行故障排除

C/RTL 协同仿真完成后，Vitis HLS 通常会指示仿真成功且 RTL 设计的功能与初始 C 语言代码相匹配。C/RTL 协同仿真失败时，Vitis HLS 会发出以下消息：

```
@E [SIM-4] *** C/RTL co-simulation finished: FAIL ***
```

以下是 C/RTL 协同仿真失败的主要原因：

- 环境设置错误
- 应用的优化指令不受支持或不正确
- C 语言测试平台或 C 语言源代码问题

要对 C/RTL 协同仿真失败进行调试，请运行如下章节中所述的检查。如果无法解决 C/RTL 协同仿真失败问题，请访问[自适应计算技术支持](#)，以获取支持资源，如答复记录、文档、下载和论坛。

设置环境

检查环境设置是否如下表中所示。

表 27：调试环境设置

问题	需采取的行动
当前是否使用第三方仿真器？	确认在系统搜索路径中已指定指向仿真器可执行文件的路径。 使用 Vivado 仿真器时，无需指定搜索路径。 请确保已完成仿真库的编译，如 仿真 IP 核 中所述。
当前是否正在运行 Linux？	确保设置文件（例如， <code>.cshrc</code> 或 <code>.bashrc</code> ）不含更改目录命令。当 C/RTL 协同仿真开始后，会生成新的 shell 进程。如果在设置文件中存在 <code>cd</code> 命令，它会导致更改 shell 运行位置，并最终导致 C/RTL 协同仿真失败。

优化指令

检查优化指令是否如下图所示。

表 28：调试优化指令

问题	需采取的行动
您当前使用的是 DEPENDENCE 指令吗？	从设计中移除 DEPENDENCE 指令，查看 C/RTL 协同仿真是否成功。 如果协同仿真成功，则可能表明 DEPENDENCE 指令的 TRUE 或 FALSE 设置错误，如 DATAFLOW 和 DEPENDENCE 的验证 中所述。
设计在顶层接口上是否使用 volatile 指针？	确保在 INTERFACE 指令上指定 DEPTH 选项。 在接口上使用 volatile 指针时，必须指定在每个传输事务中或者每次执行 C 语言函数中，在端口上执行的读写次数。
是否正在对 FIFO 使用 DATAFLOW 优化？	检查确认通过标准乒乓缓冲器执行的 C/RTL 协同仿真是否成功。 检查确认不指定 FIFO 通道大小的情况下，C/RTL 协同仿真是否成功。这样可确保通道默认设置为 C 语言代码中的阵列大小。 减小 FIFO 通道大小，直至 C/RTL 协同仿真停滞为止。停滞表明通道大小太小。复查设计，判定 FIFO 的最优大小。您可使用 STREAM 指令来指定各 FIFO 的大小。

表 28：调试优化指令 (续)

问题	需采取的行动
您当前使用的接口是否受支持？	确保您使用受支持的接口模式。欲知详情，请参阅 接口综合的联合仿真要求 。
是否正在向接口上的阵列应用多项优化指令？	确保使用设计为可协同运行的优化。欲知详情，请参阅 不受支持的协同仿真优化 。
当前是否正在映射到串流的接口上使用阵列？	要使用接口级串流（DUT 的顶层函数），请使用 <code>hls::stream</code> 。

C 语言测试平台和 C 语言源代码

请查看下表中所示的 C 语言测试平台和 C 语言源代码。

表 29：对 C 语言测试平台和 C 语言源代码进行调试

问题	需采取的行动
C 语言测试平台是否检查结果并在结果正确时返回 0（零）值？	确保 C 语言测试平台返回 0 值以便进行 C/RTL 协同仿真。即使结果正确，如果 C 语言测试平台不返回 0 值，那么 C/RTL 协同仿真功能仍会报告失败。
C 语言测试平台当前是否基于随机数字创建输入数据？	将测试平台更改为使用固定种子进行任意随机数字生成。如果用于随机数字生成的种子基于变量（例如，基于时间的种子），那么每次执行测试平台时用于仿真的数据都将不尽相同，故而导致结果不同。
当前是否正在多次访问的顶层接口上使用指针？	对于单一传输事务（执行 1 次 C 语言函数）内多次访问的任意指针，请使用 <code>volatile</code> 指针。如果不使用 <code>volatile</code> 指针，那么会根据 C 语言标准将除首次读取和最后一次写入外的所有一切都优化掉。
C 语言代码是否包含未定义的值或者执行界外阵列访问？	确认所有阵列大小都设置正确，以匹配所有访问。超出阵列大小的循环边界常引发各种问题（例如，针对大小为 N-1 的阵列执行 N 次访问）。 确认 C 语言仿真结果符合预期，并且未针对输出值指定随机数据值。 请考虑在 HLS 设计环境外使用业界标准 Valgrind 应用来确认 C 语言代码不含未定义的问题或出界问题。 即使某些变量未定义或者出界，C 语言函数仍可执行并完成。在 C 语言仿真中，将为未定义的值赋予随机数字。在 RTL 仿真中，将为未定义的值赋予未知值或 X 值。
当前在设计中是否使用浮点数学运算？	请检查 C 语言测试平台结果是否在可接受的误差范围内，而不是执行精确比较。对于某些浮点数学运算，RTL 实现不同于 C。欲知详情，请参阅 验证和数学函数 。 请确保向第三方仿真器提供适用于浮点核的 RTL 仿真模型。欲知详情，请参阅 仿真 IP 核 。
当前是否使用 AMD IP 块和第三方仿真器？	确保已向第三方仿真器提供指向 AMD IP 仿真模型的路径。
当前设计中使用的 <code>hls::stream</code> 构造是否会更改数据速率（例如，抽取或内插）？	分析设计，并使用 <code>STREAM</code> 指令增大用于实现 <code>hls::stream</code> 的 FIFO 的大小。 默认情况下， <code>hls::stream</code> 作为 FIFO 来实现，深度为 2。如果设计导致数据速率增加（例如，内插操作），那么默认 FIFO 大小 2 可能太小，并导致 C/RTL 协同仿真停滞。

表 29：对 C 语言测试平台和 C 语言源代码进行调试 (续)

问题	需采取的行动
当前仿真中使用的数据集是否过大？	执行 C/RTL 协同仿真时使用 <code>reduce_diskspace</code> 选项。在此模式下，HLS 每次仅执行 1 项传输事务。仿真可能运行稍慢，但这会导致存储空间受限，从而导致系统容量问题。 C/RTL 协同仿真功能可同时验证所有传输事务。如果多次调用顶层函数（例如，用于对多个视频帧进行仿真），那么所有仿真输入和输出数据都存储在磁盘上。根据机器设置和操作系统，这可能导致性能问题或执行问题。

封装 RTL 设计

HLS 组件流程的最后一步是将 RTL 设计封装成可供设计流程中其他工具使用的形式，例如，在 Vivado Design Suite 中作为更大的系统设计的一部分。单击 Flow Navigator 中的“Package”（封装）命令即可将 RTL 导出为 Vivado IP、Vitis 内核或已综合的检查点。

配置封装

您可在运行封装之前根据自己的设置配置封装进程以导出 IP 或内核。“Config File Editor”（配置文件编辑器）会将“Packaging”封装拆分为“Output”（输出）部分和“IP”部分，如下所示。

图 132：配置文件封装设置

All Settings Settings with Values C Synthesis Basic Settings

General
C Synthesis sources
Testbench sources
Blackbox sources
> Simulation
> C Synthesis
▼ **IP/XO Packaging**
 Output
 IP
 Implementation
> Design Directives

IP/XO Packaging

Output

output.file
Specify the output file path for the file generated via the format option

[Browse](#)

output.format
Specify the packaged format.

package.output.syn
☒ Enable packaging of IP/XO during v++ --mode=hls C-Synthesis

IP

ip.description
IP description

指定输出格式和输出文件

您必须指定 RTL 综合结果的“output.format”（输出格式）。默认格式为 RTL，此格式允许该工具从 C/C++ 源文件综合 Verilog 和 VHDL 代码，但不会生成 Vivado IP 或 Vitis 内核作为输出文件。此方法支持您快速运行综合，而无需在每次迭代时都生成硬件文件。但要在嵌入式软件设计或应用工程之类的下游进程中使用 RTL 设计，您必须生成硬件文件。您的设计中当前支持的流程能判定输出格式的最佳选择。

以下列出了“Package”（封装）选项及其默认值：

- package.output.file:
“output.file”名称和位置是由该工具判定的，默认采用为 HLS 组件指定的顶层函数的名称。但您可以指定名称和位置以覆盖默认值。
- package.output.format:
output.format 命令支持多种不同格式，如下表所示。

表 30：封装输出格式

封装格式	默认位置	注释
<code>package.out.format=ip_catalog</code>	<code><comp-name>.zip</code>	此 IP 作为 ZIP 文件导出，可添加到 Vivado IP 目录内。 <code>impl/ip</code> 文件夹还包含解压后的 IP 的内容。
<code>package.out.format=xo</code>	<code><comp-name>.xo</code>	XO 文件输出可在应用加速开发流程内供 Vitis 编译器用于执行链接。 您可将 Vitis 内核与其他内核及目标加速器卡相链接，来为加速应用构建 <code>xclbin</code> 文件。
<code>package.out.format=sysgen</code>	<code><comp-name>.zip</code>	将此 IP 导出为 ZIP 文件，搭配 System Generator for DSP 的 Vivado 版本使用。 <code>impl/ip</code> 文件夹还包含解压后的 IP 的内容。
<code>package.out.format=rtl</code>	在 HLS 组件工作目录中创建 Verilog 和 VHDL 文件夹。	该选项支持您可以跳过封装的 IP 或 XO 模块的生成，这样即可在初始设计上进行迭代时节省时间。

- `package.out.syn`：启用或禁用综合期间创建 IP 或 XO 的操作。指定 `false` 以禁用生成封装的 IP 或 XO 的操作，或者指定 `true` 以启用此操作。该选项支持您可以跳过封装的 IP 或 XO 模块的生成，这样即可在初始设计上进行迭代时节省时间。

这些选项会生成如下配置文件条目：

```
[HLS]
package.output.format=xo
package.output.file=../../<filename>
package.output.syn=false
```

IP 配置

在 HLS 配置文件中指定 `package.output.format=ip_catalog` 时，您还可以指定将应用于生成的 IP 的其他字段，例如，IP 的供应商 (Vendor)、库 (Library)、名称 (Name) 和版本 (Version)，即 IP 的 VLVN。

“Configuration”（配置）信息用于在将某一 IP 加载到 Vivado IP 目录中时，区分此 IP 的多个实例。例如，如果针对 IP 目录封装某个实现，然后创建新解决方案，并将其封装为 IP，那么默认情况下新解决方案包含相同的名称和配置信息。如果将新解决方案也添加到 IP 目录中，那么 IP 目录会将其识别为相同 IP 的更新版本，并且将使用添加到 IP 目录的最新版本。

以下列出了“IP”选项及其默认值：

- `package.ip.vendor`：xilinx.com
- `package.ip.library`：hls
- `package.ip.name`：默认采用 HLS 组件名称
- `package.ip.version`：1.0
- `package.ip.description`：由 HLS 组件生成的 IP
- `package.ip.display_name`：该字段留空
- `package.ip.taxonomy`：该字段留空
- `package.ip.xdc_file`：该字段留空
- `package.ip.xdc_ooc_file`：该字段留空



提示： Vitis Unified IDE 中的封装进程会使用 `vitis-run --mode hls --package` 命令，如 `vitis`、`v++` 和 `vitis-run` 命令所述。

完成封装进程后，会将 ZIP 文件存档或 XO 内核写入指定的 `package.output.file` 位置，或者写入 HLS 组件文件夹。导出的 IP 文件可导入 Vivado IP 目录，并在任意设计中使用。导出的 Vitis 内核可搭配 `v++ --link` 命令用作更大的系统设计的一部分。

软件驱动程序文件

对于包含 AXI4-Lite 从接口的设计，导出流程中将创建一组软件驱动程序文件。这些 C 语言驱动程序文件可包含在 Vitis 嵌入式软件开发工程中，并用于访问 AXI4-Lite 从端口。

软件驱动程序文件则写入 `solution/impl/ip/drivers` 目录，并包含在封装的 IP `export.zip` 中。请参阅“AXI4-Lite 接口”章节，获取有关 C 语言驱动程序文件的详细信息。

RTL 导出的输出

运行“Package”（封装）命令时，HLS 编译器会写入活动解决方案的 `impl` 文件夹。

输出文件和文件夹包括：

- `component.xml`：IP 组件文件，用于定义接口和架构。
- `<component_name>.zip`：IP 及其内容的 zip 存档。此 zip 文件可直接添加到 Vivado IP 目录中。
- `<component_name>.xo`：已编译的内核对象，可供在 Vitis 应用加速开发流程中使用。
- `impl/ip`：解压后的 IP 内容。
- `impl/ip/example`：此文件夹内包含用于生成封装后的 IP 的 Tcl 脚本和用于导出此 IP 的 shell 脚本。
- `impl/report`：综合后或布局布线后的 IP 报告将写入此文件夹。
- `impl/verilog`：包含 Verilog 格式的 RTL 输出文件。
- `impl/vhdl`：包含 VHDL 格式的 RTL 输出文件。



提示： 如果执行“Flow Navigator”中的“Implementation”（实现）步骤，那么输出文件夹还会包含 `project.xpr` 文件，此文件可在 Vivado Design Suite 中打开。



重要提示！ 您不应将 `verilog` 或 `vhdl` 输出文件中的文件直接用于自己的用途。而应改为使用已封装的 IP 输出文件 (`impl/ip`)，原因如下。

如果 HLS 编译器在已编译的设计中使用 Vivado IP（例如，配合浮点设计使用），那么 `impl/verilog` 或 `impl/vhdl` 目录包含用于在 RTL 综合期间创建 IP 的脚本。如果将 `verilog` 或 `vhdl` 文件夹中的文件复制出并用于 RTL 综合，那么您自行承担正确使用这些文件夹中存在的任意脚本文件的责任。如果使用封装 IP 输出文件 (`impl/ip`)，则会自动执行该进程。如果在 HLS 组件中已执行 C/RTL 协同仿真，那么 Vivado 工程还将包含 RTL 测试平台，并且可对设计进行仿真。

运行实现

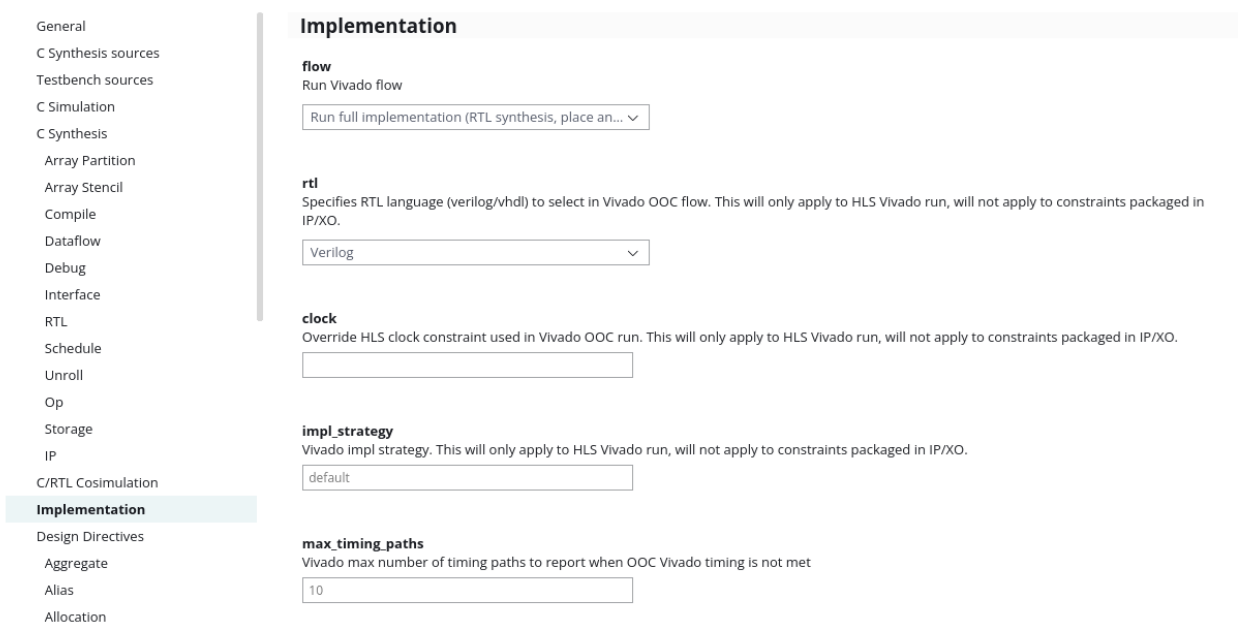
在“Flow Navigator”中，确保 HLS 组件处于活动状态，或者在“Component”（组件）菜单中将其选中，在该工具中将其设为活动组件。当 HLS 组件为活动组件时，“Flow Navigator”支持运行 C 语言仿真、C 语言综合、C/RTL 协同仿真、封装与实现来构建和分析该 HLS 组件。选中“Flow Navigator”的“IMPLEMENTATION”（实现）标题下的“Run”（运行）。

当 HLS 编译器报告高层次综合的结果时，它会提供估算结果，包括预测的时钟频率和资源使用率（LUT、DSP、BRAM 等）。之所以仅预测这些结果，原因是该工具无从知晓最终综合或实现后的设计中的优化或布线延迟。为了能更准确分析 RTL 设计，您可以在生成的 RTL 设计上实际运行 Vivado 综合和布局布线，并复查时序和资源使用率的结果。但这也只是改进后的估算结果，因为如果使用当前 IP 或内核的更大的设计中还包含其他 IP 或内核，则会产生不同的结果。

配置实现

您可以先配置 Vivado 综合与实现，然后再使用“Config File Editor”（配置文件编辑器）的“Implementation”（实现）部分（如下所示）或者直接编辑 HLS 配置文件来运行综合与实现。

图 133：HLS 组件实现设置



“Implementation”的配置命令包括：

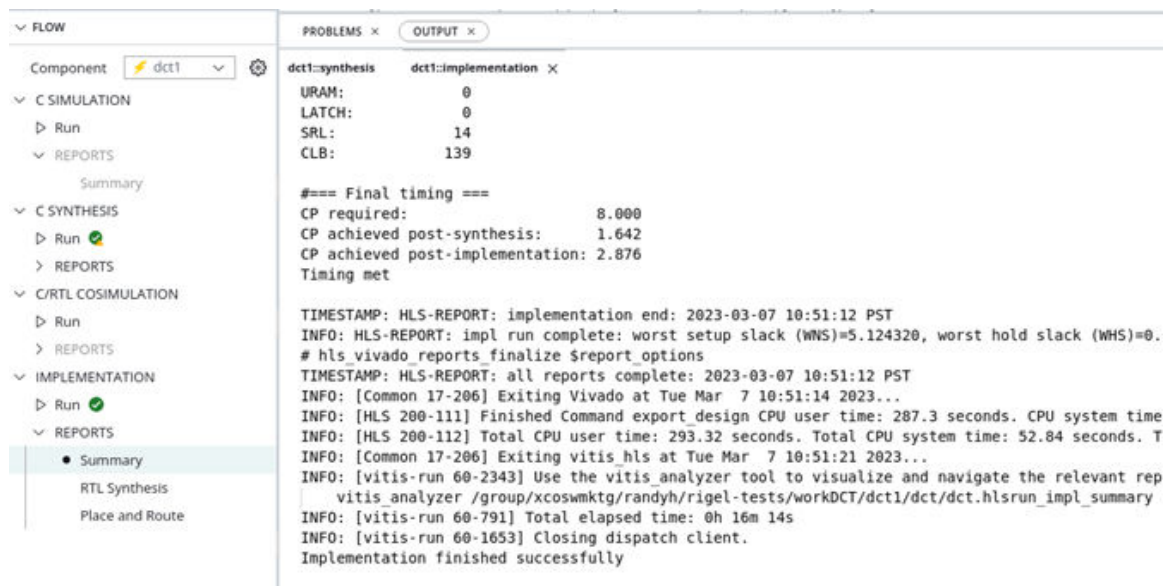
- **vivado.flow**：指定仅运行综合或者同时运行综合与实现。仅运行综合比同时运行综合与实现更快，但缺少实现运行的部分详细信息。默认值为 `syno`。
- **vivado.rtl**：指定运行 Vivado 非关联流程时使用的语言。默认为 Verilog。
- **vivado.clock**：用于指定综合或实现期间使用的时钟周期。如不指定此项，则使用创建 HLS 组件时指定的默认时钟。
- **vivado.impl_strategy**：用于指定要在实现运行期间采用的策略。这仅在运行实现用于估算资源使用率和时序期间才适用，并且不影响生成的 Vivado IP 或 Vitis 内核。

- `vivado.max_timing_paths`：用于指定要从“Timing Summary”（时序汇总）报告抽取的时序路径数量。它会返回指定数量的最差情况路径。
- `vivado.optimization_level`：这是用于管理 Vivado 工具所执行的优化的通用功能特性。设置越高，采用的优化越多，运行时间也越长。
- `vivado.pblock`：指定 Pblock 范围或值，用于在布局布线期间限制设计可用面积。
- `vivado.phys_opt`：用于指定要运行的物理优化。选项包括：`none`、`place`、`route` 和 `all`。
- `vivado.report_level`：用于定义综合或实现期间生成的报告级别。此报告可包含使用情况和时序汇总信息、时序路径详细信息或 Failfast（快速失败）报告（默认）。
- `vivado.synth_design_args`：用于为 `synth_design` 命令指定选项。
- `vivado.synth_strategy`：用于指定要在 Vivado 综合运行期间采用的策略。

Run Implementation

在配置文件中定义实现设置后，即可从“Flow Navigator”中选择“Run”（运行）。您可在“Output”（输出）窗口中跟踪实现运行的进度。运行的转录文本所含顶层函数名为 `<component-name>::implementation`，如下所示。

图 134：运行实现的 HLS 组件



提示： Vitis Unified IDE 中的实现运行使用的是 `vitis-run --mode hls --impl` 命令。

完成实现后，应在转录文本底部显示“Implementation finished successfully”（实现成功完成）消息。在“IMPLEMENTATION”报头下的“Reports”（报告）文件夹中，还会填充以下报告：

- “Summary”（汇总）：报告使用的命令行和结果的时间戳
- “RTL Synthesis”（RTL 综合）：报告综合结果，包括资源使用率和时序
- “Place and Route”（布局布线）：如 [实现报告](#) 所述



提示：您可在“Flow Navigator”中使用“Cancel Run”（取消运行）命令来取消运行实现。

实现报告

“Implementation Report”（实现报告）包含已运行的综合和布局布线的结果。此报告包含以下几个部分：

- “General Information”（常规信息）：提供与设计和实现相关的常规信息。
- “Run Constraints and Options”（运行约束和选项）：用于报告为运行的 RTL 综合和/或布局布线所设置的约束和选项。它可为您显示针对运行所设置和/或修改的约束。
- “Resource Usage”（资源使用）/ “Final Timing”（最终时序）：“Resource Usage”部分和“Final Timing”部分可显示 RTL 综合运行或布局布线运行后实现的资源和时序的摘要信息。这两部分提供了有关是否满足时序目标的高层次资源使用情况和状态概述。后续章节中的信息将提供有关调试时序问题的实用详细信息。
- “Resources”（资源）：该表中显示了按模块对资源进行拆分的详细信息。此外，该表还显示了来自源代码的原始变量和源码位置信息。如果用户指定的编译指示生成了特定资源，那么该表中也可显示该资源。这样您即可将自己的 C 语言代码关联到综合后的 RTL 实现内。检查此报告可谓一举多得，因为它是在 Vivado 完成设计综合后生成的，故而 DSP 之类的功能块和其他逻辑单元在电路内全都已完成例化。
- “Fail Fast”（快速失败）：Vivado 提供的“Fail Fast”报告可指导您调查该工具遇到的特定问题。在“Fail Fast”报告中，您应关注状态为“REVIEW”（复查）的所有问题，以改善实现和时序收敛。“Fail Fast”报告的不同部分包括：
 - “Design Characteristics”（设计特性）：默认使用率准则基于 SSI 技术器件，对非 SSI 技术器件可以放宽。有一项或多项 REVIEW 检查标记的设计虽可行但难以实现。
 - “Clocking Checks”（时钟设置检查）：这些检查至关重要，必须解决。
 - “LUT and Net Budgeting”（LUT 和信号线预算）：使用保守方法能更准确预测在器件使用率高的情况下布局后，哪些逻辑路径无法满足时序要求的可能性最高。

图 135：设计特性

Criteria	Guideline	Actual	Status
LUT	70%	7.45%	OK
FD	50%	3.99%	OK
LUTRAM+SRL	25%	1.49%	OK
LOOKAHEAD8	25%	0.07%	OK
DSP	80%	0.20%	OK
RAMB	80%	6.51%	OK
URAM	80%	0.00%	OK
DSP+RAMB+URAM (Avg)	70%	3.35%	OK
BUFGCE* + BUFGCTRL	24	0	OK
DONT_TOUCH (cells/nets)	0	0	OK
MARK_DEBUG (nets)	0	0	OK
Control Sets	16872	407	OK
Average Fanout for modules > 100k cells	4	3.56	OK
Non-FD high fanout nets > 10k loads	0	2	REVIEW
TIMING-6 (No common primary clock between related clocks)	0	0	OK
TIMING-7 (No common node between related clocks)	0	0	OK
TIMING-8 (No common period between related clocks)	0	0	OK
TIMING-14 (LUT on the clock tree)	0	0	OK
TIMING-35 (No common node in paths with the same clock)	0	0	OK
Number of paths above max LUT budgeting (0.449ns)	0	100	REVIEW
Number of paths above max Net budgeting (0.302ns)	0	100	REVIEW

- “Timing Paths”（时序路径）：“Timing Paths”报告可显示导致设计的最差裕量的时序关键路径。默认情况下，该工具会显示排名前 10 的最差负时序裕量路径。表中每条路径都包含详细信息，可显示各触发器之间的组合路径。要解决时序问题，就需要中断这些冗长的组合路径。因此，您需要分析这些路径，推理其来源，并将这些路径映射回您的 C 语言代码。将这些路径和先前的资源表搭配使用有助于判定路径与源代码之间的关系，并将路径反向关联到源代码。

在下图中，可以看到“Place & Route”（布局布线）报告中排名前 10 的负时序裕量路径的实际逻辑级数 (9) 比完成“RTL Synthesis”（RTL 综合）后的逻辑级数 (5) 更高，最大扇出也更糟 (64 → 9366)。这明显表明设计中的拥堵导致逻辑级数和扇出增高，从而导致出现问题，无法满足时序。根据这些线索，您可修改自己的设计，通过重写 C 语言代码或者改变有关 BRAM/LUTRAM/URAM 资源选择的设计决策，从而消除其中部分拥塞。

图 136：时序路径报告

Timing Paths	
Worst Negative Slack:	0.313ns(met)
Total Negative Slack:	0.000ns(met)
Max levels:	4
Max fanout:	1
Full Timing Report:	verilog/report/example_timing_routed.rpt
NAME	VALUE
✓ Path 1 (4)	slack=0.313ns(met) levels=4 fanout=1
Modules	"
Startpoint Pin	i_synth/MULT.OP/i_non_prim.MULT/MULT_GEN_VARIANT.FIX_MULT/MULT/gDSP.gDSP_only.iDSP/use_prim.appDSP[0].bppDSP[0].use_dsp.use_dsp48e2
Endpoint Pin	i_synth/MULT.OP/i_non_prim.MULT/MULT_GEN_VARIANT.FIX_MULT/MULT/gDSP.gDSP_only.iDSP/use_prim.appDSP[0].bppDSP[0].use_dsp.use_dsp48e2
> Cells in Path (6)	6
> Path 2 (4)	slack=0.313ns(met) levels=4 fanout=1
> Path 3 (4)	slack=0.313ns(met) levels=4 fanout=1
> Path 4 (4)	slack=0.313ns(met) levels=4 fanout=1

优化 HLS 工程

分析后，您将很可能需要或者想要对函数性能进行优化。即使函数当前性能表现良好，也可能仍有改善空间。本节主要探讨为工程应用优化的机制。如需了解有关您可执行的各种类型的优化的相关信息，请参阅 [优化技巧与故障排除提示](#)。

您可各种 [HLS 编译指示](#) 将优化指令作为编译器的编译指示直接添加到源代码中使用，或者也可按 [HLS 优化指令](#) 中所述使用配置命令来配置优化指令，并将其应用于 HLS 组件。

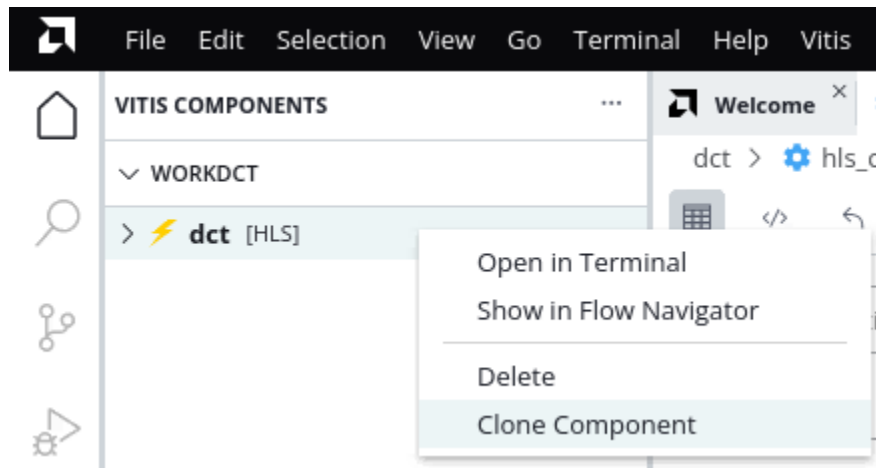
除优化编译指示和指令外，HLS 编译器还提供了多项配置命令，供您用于管理仿真与综合的默认结果，如 [HLS 配置文件命令](#) 中所述。

克隆 HLS 组件

HLS 编译器的最常用用例是用于创建初始设计、分析结果，然后执行优化以满足期望的面积和性能目标。这通常是一个迭代性进程，需要执行多个步骤和多次优化才能达成期望的目标。开始探索应对设计挑战的可能解决方案时，您可以克隆一个 HLS 组件，以便保存一个组件，同时使用另一个组件进行探索。

要克隆 HLS 组件，请在 Vitis 的“Component Explorer”（组件资源管理器）中右键单击该组件，然后选中“Clone Component”（克隆组件）命令，如下图所示。这样即可打开“Clone Component”（克隆组件）对话框，它会提示您给该组件输入一个新名称。

图 137: 克隆组件



创建克隆的组件后，设计将包含相同的源文件、测试激励文件、顶层函数和 HLS 配置文件设置。您可编辑此新 HLS 组件的配置文件设置或其他设置，然后开始逐步执行构建进程。您可将原始 HLS 组件保留其当前状态，或者开发两种不同的设计分支方法。

组件比较功能

“Component Comparison”（组件比较）工具允许用户从性能和使用率两方面来比较一组 HLS 组件。可按表格或图形格式生成报告。此外，报告数据还可以 CSV 格式导出。

仅限为 HLS 组件生成比较报告，并且在已打开的 Vitis 统一软件平台工作空间中，这些组件必须存在。要使用此功能，选中进行比较的 HLS 组件应至少已成功完成以下任一阶段：

- C 语言综合
- C/RTL 协同仿真
- 实现



重要提示！ 在生成的 HLS 比较报告中，仅提供已完成的阶段的数据。

要生成报告，请转至 Vitis Unified IDE 中的“VITIS COMPONENTS”（Vitis 组件）面板，将鼠标悬停在任意 HLS 组件上，选中其“HLS Compare Reports”（HLS 比较报告）图标。将鼠标悬停在该图标上时，它将变得可见。或者，转至“View”（视图），然后选择“HLS Compare Reports”。

这样就会显示“HLS Compare Reports”面板。从可用 HLS 组件列表中勾选所需组件的复选框。最后，选择“Compare”（比较）。

这样就会生成表格格式的 HLS 比较报告文件，并显示在编辑器面板中。针对所选 HLS 组件的完成阶段，会以表格形式填充组件信息和指标值。

- 要将表格数据导出为 CSV 格式文件，请选中报告右上角的导出（带向下箭头的云）图标。这样就会出现“Compare Reports Export CSV File”（比较报告导出 CSV 文件）面板。输入文件名，然后选择“Save”（保存）。
- 要生成图形格式的 HLS 比较报告，请从表格“HLS Compare Report”中选中报告右上角的“Graph Compare”（图形化比较）图标，这样会得到一个条形图。

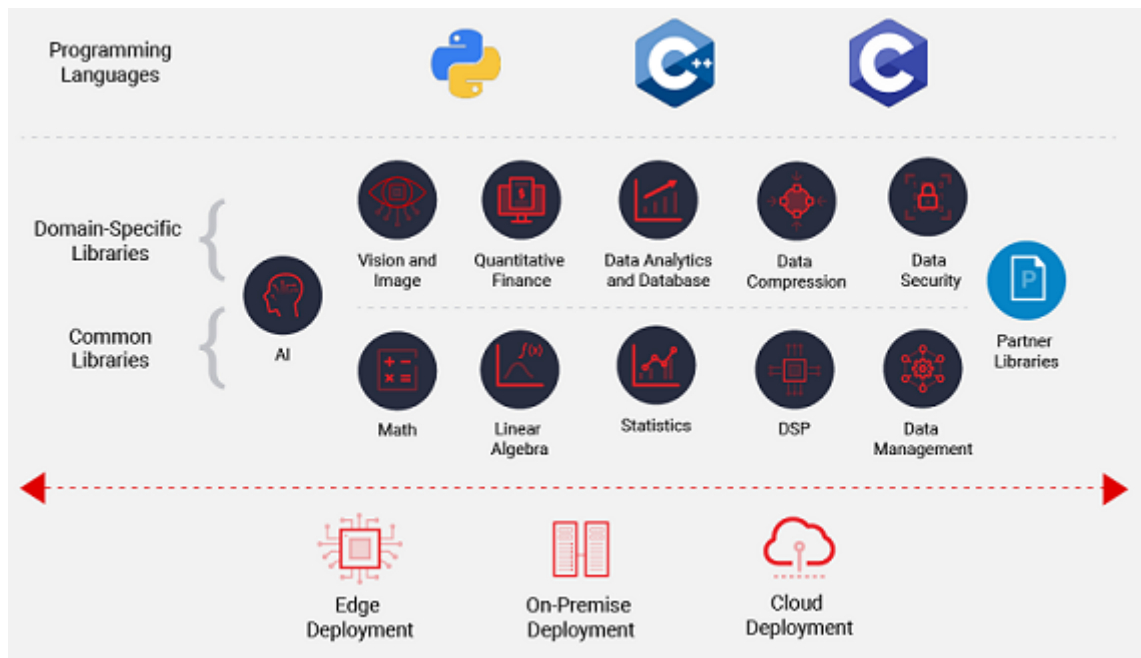
- 要切换回表格报告，请选择报告右上角的“Table Compare”（表格比较）图标，这样会得到一个电子数据表。

L1 Library Wizard 流程

“L1 Library Wizard”（L1 库向导）流程功能提供了一种工具辅助的方法，用于访问一小部分 Vitis 加速库 HLS 功能。您可选择库函数，并生成示例以及参数化的模板函数，这些函数可以整合到您的设计中。这样您即可从基于 HLS 预构建的 IP 快速为自己的设计添加基于 IP 的功能。

Vitis 库包含丰富的开源库，这些库专为性能优化，只需对您的现有应用进行些许更改、甚至无需更改，即可提供开箱即用的应用加速能力。适用于数学、统计、线性代数和数字信号处理 (DSP) 的各种通用 Vitis 加速库能够为广泛而多样的应用提供一整套核心功能。针对特定领域的 Vitis 加速库则可为视觉和图像处理、计量金融、数据库分析、数据分析以及数据压缩等工作负载提供开箱即用的加速。

图 138：Vitis 库



Vitis 库中提供了三种实现类型，分别是 L1 原语、L2 内核与 L3 软件 API。这些实现被分门别类放置在各自相应的目录中：L1、L2 和 L3。FPGA 硬件开发者可以利用 L1 原语实现。L2 内核实现为 Vitis 主机代码开发者提供了使用示例。L3 软件 API 实现提供 C、C++ 和 Python 函数接口，允许纯软件开发者将运算分载到预构建的 FPGA 镜像（也称为覆层）中。Vitis 视觉库为 FPGA 和 AI 引擎器件上加速的计算机视觉函数提供了一个软件接口。

Vitis L1 Library Flow 功能提供了一种工具辅助的方法，可将 L1 库 HLS 函数快速纳入设计中，仅适用于 Vitis Solver（求解器）和 Vision Library（视觉库）子文件夹。“Solver Library”（求解器库）提供一系列矩阵分解运算、线性求解器和特征值求解器集合。Vitis 视觉库是专为在 Vitis 开发环境中工作而设计的，它为使用 Vitis HLS 在 FPGA 上加速的计算机视觉功能提供了一个软件接口。Vitis 视觉库函数的功能大多与 OpenCV 的对应函数相似。

注释：要使用 Vitis 视觉库，您的系统上必须已安装并且可访问 OpenCV 库 v4.4.0。如需获取有关如何在 Windows 上安装 OpenCV 库的指示信息，请参阅在 [Windows 10 上安装/设置 OpenCV](#)，如需获取 Linux 相关信息，请参阅[编译和安装 OpenCV 库以搭配视觉库一起使用](#)。

有 3 个工具辅助流程可用于访问 L1 HLS 函数：

1. “Examples” 视图
2. 在 “Welcome” 页面中，从库创建组件
3. 在 Vitis Unified IDE 编辑器中，从 `xf:: namespace` 自动补全



提示：为确保正常访问 L1 库 HLS 函数，请确保已下载 Vitis 库的 Solver 和 Vision 子文件夹。

流程 1 - 示例 - 创建 L1 HLS 函数示例

“Examples VIEW” 流程允许您选择并生成 L1 Library 示例。由此即可得到指定的 L1 Library HLS 组件工程，其中包含测试平台和源代码文件。

要生成 L1 库示例，请执行以下步骤：

1. 在 Vitis Unified IDE 中选中 “Examples”（示例）图标。
2. 在 “Examples” 面板中，转至 “HLS Examples” → “Vitis Accelerated Libraries Repository” → “Vitis Vision Library” → “L1” → “Examples”（HLS 示例 > Vitis 加速库存储库 > Vitis 视觉库 > L1 > 示例）。
3. 选择所需的示例，例如 AMD 3D LUT L1 Test。
4. 单击 “Create HLS Component Template”（创建 HLS 组件模板）按钮。
5. 设置组件名称和位置。选择 “Next”（下一步）。
6. 选择 “Finish”（完成）生成 HLS 组件示例。

流程 2 - 从库创建组件

“Create component from Library”（从库创建组件）流程允许您选择 HLS 模板函数并将其参数化。在可综合文件的 C++ 封装文件中，会创建一个带有已例化的模板函数的 HLS 组件工程。其中不提供测试激励文件。要执行此流程，请遵循以下步骤进行操作：

1. 在 “Welcome” 页面中，选中 “Create Component from Library”。
2. 在弹出的 “Create Component from Library” 面板中输入组件名称和位置，然后选择 “Next”。
3. 选择所需的 L1 HLS 函数，然后选择 “Next”。
4. 在参数化面板中，配置参数。
 - a. 灰色文本字段为必填项。在某些情况中，会指定合法的整数范围。例如，“`int(16~2160)`”表示该字段值可以是 16 至 2160（含）范围内的整数。
5. 定义参数后，选择 “Next”。
6. 选择 “Part”（部件），然后选择 “Next”。您可以立即选择 “Next” 以使用默认部件。
7. 编辑 “Settings”（设置），然后选择 “Next”。您可以立即选择 “Next” 以使用默认设置。
8. 复查新 HLS 组件的设置，然后选择 “Finish”。

流程 3 - 在 IDE 编辑器中从 `xf:: namespace` 自动补全

“auto-completion from the `xf:: namespace`”（从 `xf:: namespace` 自动补全）流程提供了将参数化 HLS 模板函数插入当前正在编辑的源代码的方法。

1. 在 Vitis Unified IDE 编辑器中，打开一个 C++ 源代码文件。
2. 在文件编辑器面板的空行中键入 “`xf::`”。
3. 这样将显示可用 HLS 函数列表，用于自动补全。选择一个函数，例如 `xf::cv::absdiff`。

4. 选中函数后，就会打开 HLS 函数的参数化窗口，请填写该面板中的“Parameters”（参数）和“Ports”（端口）部分。
 - a. 在“Parameters”部分中，灰色文本字段为必填字段。在某些情况中，会指定合法的整数范围。例如，“int(16~2160)”表示该字段值可以是 16 至 2160（含）范围内的整数。
 - b. 在“Ports”部分中，您可重命名实参，使其与源代码上下文相匹配。

从命令行创建 HLS 组件

前述内容描述了如何使用 Vitis Unified IDE 构建和运行 HLS 组件，本节主要聚焦如何使用可用的命令行工具来完成同样的操作。

`v++ -c --mode hls` 编译器命令用于构建 HLS 组件。该命令使用配置文件命令语言。C 语言仿真、代码分析器、C/RTL 协同仿真、实现和导出全都依赖于 `vitis-run` 命令。以下描述了命令行流程的不同步骤。



提示：以下示例是基于《[Vitis HLS 入门](#)》中的 DCT 设计来提供的。此示例包含 HLS 组件、源文件和测试平台。

运行 C 语言综合

用于构建 `dct` HLS 组件的 `v++` 命令行如下所示：

```
v++ -c --mode hls --config ./dct/hls_config.cfg --work_dir dct
```

其中：

- `--config` 指定配置文件，其中包含用于构建的编译器指令以及用于配置仿真器（用于运行仿真）的编译器指令
- `--work_dir` 提供了工作目录用于构建组件



提示：从命令行创建 HLS 组件时，`--work_dir` 会指定 HLS 组件文件夹，`--work_dir` 的父文件夹会变为工作空间，用于启动 Vitis IDE。

配置文件的内容各异，但针对综合，该 `dct` HLS 组件要求 `hls_config.cfg` 文件中包含下列命令：

```
part=xczu9eg-ffvb1156-2-e

[hls]
syn.file=./src/dct.cpp
syn.top=dct
flow_target=vitis
clock=8ns
clock_uncertainty=12%
syn.output.format=rtl
syn.directive.pipeline=dct_2d II=4
```



重要提示！如果 HLS 配置文件使用 `platform=` 代替 `part=`，那么您还必须指定 `freqhz=` 代替 `clock=`（[此处所示](#)）以更改平台的默认时钟频率。

综合命令的成功很大程度上取决于配置文件的内容。其中有几个元素是必需的，还有一些选项可由您指定。在以上提供的配置文件中，综合所需的元素包括 `part`、`syn.file` 和 `syn.top`。`flow_target`、`clock` 和 `clock_uncertainty` 并非必需，仅当用于覆盖默认值时才需要使用。`syn.directive.xxx` 命令用于为函数综合提供特定的优化。

运行 C 语言仿真和代码分析器

要在 C 语言仿真或代码分析器中运行 HLS 组件，请使用 `vitis-run` 命令。C 语言仿真并不强制要求先运行 C 语言综合，因为它不需要综合所生成的 RTL 代码。

```
vitis-run --mode hls --csim --config ./dct/hls_config.cfg --work_dir dct
```

其中：

- `--csim` 指定运行目标。
- `--config` 会按照指示为综合指定配置文件，但会包含 C 语言仿真的特定要求，如下所示。
- `--work_dir` 会按照指示为综合提供工作目录用于构建组件。

配置文件的内容各异，但对于 C 语言仿真，配置文件不仅需要要求为综合指定的源文件和顶层文件，还需要测试平台和输入文件，以及 `csim` 配置设置，如 [C 语言仿真配置](#) 所述：

```
part=xczu9eg-ffvb1156-2-e

[hls]
clock=8ns
clock_uncertainty=12%
flow_target=vitis
syn.output.format=rtl
syn.file=./src/dct.cpp
syn.file=./src/dct.h
syn.top=dct
tb.file=./src/dct_coeff_table.txt
tb.file=./src/dct_test.cpp
tb.file=./src/in.dat
tb.file=./src/out.golden.dat
csim.clean=true
csim.code_analyzer=false
syn.directive.pipeline=dct_2d II=4
```



提示：将 `csim.code_analyzer` 更改为 `true` 以启用代码分析器和仿真。

运行 C/RTL 协同仿真

要在 HLS 组件上运行 C/RTL 协同仿真，请使用 `vitis-run` 命令，如下所示：

```
vitis-run --mode hls --cosim --config ./dct/hls_config.cfg --work_dir dct
```

C/RTL 协同仿真所需的配置文件内容包括：

```
part=xczu9eg-ffvb1156-2-e

[hls]
clock=8ns
clock_uncertainty=12%
flow_target=vitis
syn.output.format=rtl
syn.file=./src/dct.cpp
syn.file=./src/dct.h
syn.top=dct
tb.file=./src/dct_coeff_table.txt
```

```
tb.file=./src/dct_test.cpp
tb.file=./src/in.dat
tb.file=./src/out.golden.dat
syn.directive.pipeline=dct_2d II=4
cosim.enable_dataflow_profiling=true
cosim.enable_fifo_sizing=true
cosim.trace_level=port
cosim.wave_debug=true
```

注释：以上 `hls_config.cfg` 文件显示了 C/RTL 协同仿真工具的部分设置。如需了解更多信息，请参阅 [协同仿真配置](#)。

运行实现

要在 HLS 组件上运行 Vivado 综合或实现，请使用 `vitis-run` 命令，如下所示：

```
vitis-run --mode hls --impl --config ./dct/hls_config.cfg --work_dir dct
```

导出 IP/XO

要从已综合的 HLS 组件导出 Vivado IP 或 Vitis 内核，可使用 `Vitis-run --package` 命令，如下所示：

```
vitis-run --mode hls --package --config ./dct/hls_config.cfg --work_dir dct
```

导出封装 IP 或 XO 所需的配置文件内容包括：

```
part=xcvu9p-flga2104-2-i

[hls]
syn.file=/group/xcoswmktg/randyh/rigel-tests/03-Vitis_HLS/reference-
files/src/dct.cpp
syn.top=dct
syn.output.format=xo
```

注释：`--package` 命令，用于从先前综合的 HLS 组件导出 IP 或 XO。您可将 Vitis 内核作为 IP 来导出。但您无法将 Vivado 流程 IP 作为 XO 来导出，除非它满足 Vitis 内核的特定要求。

第四部分

Vitis HLS 命令参考资料

本部分包含以下章节：

- [vitis、v++ 和 vitis-run 命令](#)
- [HLS 配置文件命令](#)
 - [HLS 配置选项](#)
 - [HLS 优化指令](#)
- [HLS 编译指示](#)
- [HLS Tcl 命令](#)
 - [配置命令](#)
 - [工程命令](#)
 - [优化指令](#)

vitis、v++ 和 vitis-run 命令

vitis 命令

使用 `vitis` 命令按以下语法启动 Vitis Unified IDE：

```
vitis -w <workspace>
```

使用 Vitis 工具时，您也可以使用各种其他模式。您可使用 `-h` 或 `--help` 来显示 `vitis` 命令所支持的模式。

```
vitis -h
```

```
Syntax: vitis [-g (default) | -a | -w | -i | -s | -h | -v]
```

Options:

```
  Launches New Vitis IDE (default option).
  -a/--analyze [<summary file | folder | waveform file: *.*[wdb|wcfg]>]
      Open the summary file in the Analysis view.
      Opening a folder opens the summary files found in the folder.
      Open the waveform file in a waveform view tab.
      If no file or folder is specified, opens the Analysis view.
  -w/--workspace <workspace_location>
      Launches Vitis IDE with the given workspace location.
  -i/--interactive
      Launches Vitis python interactive shell.
  -s/--source <python_script>
      Runs the given python script.
  -j/--jupyter
      Launches Vitis Jupyter Web UI.
  -h/--help
      Display help message.
  -v/--version
      Display Vitis version.
```

Vitis Unified IDE 会启动 `v++ -c --mode hls` 命令以综合 HLS 组件，或者启动 `vitis-run` 命令以运行 C 语言仿真、C/RTL 协同仿真、封装和实现，如 [从命令行创建 HLS 组件](#) 中所述。这些命令解释如下。

v++ 命令

`v++ -c --mode hls` 命令用于为 Vivado IP 或 Vitis 内核流程目标执行 HLS 组件综合（如 [目标流程概述](#) 中所述），或者编译 HLS 组件，用于在 Vitis Unified IDE 中的系统工程上进行软件仿真（如 [构建和运行 HLS 组件](#) 中所述）。

在 HLS 组件模式下使用 `v++` 命令时，配置文件会为工具指定多个选项，并为源代码指定优化指令，如 [HLS 配置文件命令](#) 中所述。您可运行以下命令来查看其中部分选项：

```
v++ -c --mode hls -h
usage: v++ [options] <input file...>

Options allowed only on a command line:
  -c [ --compile ]      Run a compile flow
```

```
-h [ --help ]           Display usage message
-m [ --mode ] arg       [aie|hls] Specify a mode. Must be used with
                        --compile/-c option
-v [ --version ]        Display version information
--config arg            Config file
--work_dir arg          Specify a working directory for output files and
                        directories

Options allowed in a config file or a command line:

-f [ --platform ] arg   Specify path to a platform specification(XPFM) or
                        hardware specification(XSA)
--freqhz arg            Specify a PL clock frequency in Hz or MHz
--part arg              Specify a part

[hls] section:
--hls.* arg             Specify hls section options. For more
information,            see: <https://docs.amd.com/r/en-US/ug1702-vitis-
accel
                        erated-reference/v-Mode-HLS>
```

vitis-run 命令

vitis-run 命令用于在 HLS 组件上搭配 --mode hls 选项来运行处理步骤。要查看 vitis-run 命令支持的命令选项，请使用以下命令：

```
vitis-run --mode hls -h
usage: vitis-run [options] <input file...>

Options allowed only on a command line:
-h [ --help ]           Display usage message
-v [ --version ]        Display version information
--config arg            Config file
--cosim                 Specify hls run option cosim
--csim                  Specify hls run option csim
--impl                  Specify hls run option impl for vivado ooc
--input_file arg        Specify input Tcl file; input file can also be
                        specified positionally without using the --
input_file              option
--itcl                  Interactive hls Tcl shell
--mode arg (=hls)       [hls] Specify a mode
--package               Specify hls run package option
--tcl                   Evaluate hls Tcl file
--work_dir arg          Specify a working directory for output files and
                        directories

Options allowed in a config file or a command line:

-f [ --platform ] arg   Specify path to a platform specification(XPFM) or
                        hardware specification(XSA)
--freqhz arg            Specify a PL clock frequency in Hz or MHz
--part arg              Specify a part

[hls] section:
--hls.* arg             Specify hls section options. For more
information,
```



```
accel
```

```
see: <https://docs.amd.com/r/en-US/ug1702-vitis-generated-reference/vitis-run-Command>
```

hls_init.tcl

启动 AMD Vitis™ HLS 时，该工具会在下列位置查找 Tcl 初始化脚本：

1. 特定用户脚本：在本地用户目录中，适用于工具的所有版本：

- · 在 Windows 上：%APPDATA%/Xilinx/HLS_init.tcl
- · 在 Linux 上：\$HOME/.Xilinx/HLS_init.tcl

HLS_init.tcl 允许您使用 AMD Vitis™ HLS 命令先执行工具初始化，然后再打开工程。



提示：在软件安装位置中没有 HLS_init.tcl 脚本。如需该脚本，则您必须手动创建。

HLS 配置文件命令

HLS 编译模式支持访问各种功能特性，用于开发、优化、分析和导出 Vitis 内核 (.xo) 文件或 Vivado IP (.zip) 文件。您可使用以下命令或通过启动 Vitis Unified IDE 进入 HLS 模式：

```
v++ -c --mode hls -h [options] <input_files...>
```

HLS 编译选项应输入配置文件以搭配 v++ 命令和 --config 选项一起使用。HLS 选项应置于配置文件中 [hls] 节头下。例如，以下配置文件指定了部件、源文件、测试激励文件和流程目标。请注意，在 [HLS] 头文件下并未指定 part，因为这是 v++ 编译器的常规选项。

```
part=xcvu11p-flga2577-1-e

[hls]
clock=8
flow_target=vitis
syn.file=../../src/dct.cpp
syn.top=dct
tb.file=../../src/out.golden.dat
tb.file=../../src/in.dat
tb.file=../../src/dct_test.cpp
tb.file=../../src/dct_coeff_table.txt
syn.output.format=xo
clock_uncertainty=15%
```

配置文件中或命令上允许使用的选项

使用 Vitis Unified IDE 时，可从配置文件内使用以下 v++ 命令行选项。这些选项不属于配置文件中的任何报头。

- --platform <arg>：指定平台规格 (.xpfm) 的路径，以此平台规格即为当前组件或工程的目标。
- --freqhz <arg>：指定以 Hz 为单位的时钟频率。



重要提示！ 如果 HLS 配置文件指定 platform= 而不是 part=，那么您还必须使用 freqhz= 代替 clock=。这是因为，平台以 freqhz 形式指定时钟，您只能使用相同形式来将其覆盖。

- --part <arg>：指定 HLS 组件的目标器件。

HLS 常规选项



重要提示！ 在 HLS 配置文件中的 [hls] 头文件下必须出现下列选项。

- clock:

指定时钟周期，以 ns 或 MHz 为单位，默认单位为 ns。如不提供周期，系统会使用默认的 10 ns 周期。

```
clock=8ns
```



重要提示！ 如果 HLS 配置文件使用 `platform=` 代替 `part=`，那么您还必须指定 `freqhz=` 代替 `clock=`（如此处所示）以更改平台的默认时钟频率。

· `clock_uncertainty`:

由 HLS 指定时钟周期作为时钟不确定性裕度。系统会从时钟周期中扣除此不确定性裕度，以创建有效的时钟周期。时钟不确定性以 ns 为单位，或者以时钟周期的百分比来定义。时钟不确定性默认设为时钟周期的 27%。指定该值时，默认单位为 ns，但也可以使用 % 或 MHz 作为单位。

```
clock_uncertainty=15%
```

· `flow_target`:

将流程目标设为综合 Vitis 内核 (`vitis`) 或 Vivado IP (`vivado`)。应用加速流程使用 AMD Vitis™ 内核，而嵌入式设计流程则使用 Vivado IP。



重要提示！ Vivado IP 与 Vitis 内核的接口定义不同。

· `relative_roots`:

指定一个或多个绝对路径的有序列表。通过搜索该列表即可解析源文件的相对文件路径，还可解析与 PATH 环境变量具有相似行为的 `cflags include` 路径。该列表的第一个条目还可用作任意相对输出文件路径的根。

`relative_roots` 列表中的所有条目都必须是绝对路径，或者是以任一下列项开头的路径，下列项均有特殊意义：

- `file`: 包含配置文件的目录，`relative_root` 是在该配置文件内指定的
- `cwd`: `v++/vitis-run` 进程的当前工作目录

通过任一配置文件中的多个条目即可指定 `relative_roots` 列表。每个连续的值都会被追加到条目列表中。在每个条目中，可以用分号分隔符来指定多条路径。分号前后的空白会忽略或移除。

```
[hls]
relative_roots=/tmp/aaa ; /tmp/bbb ; file/../ccc ; cwd
syn.file=top.cpp
```

注释：如果在 `v++/vitis-run` 命令上指定 `relative_roots`，则会将其忽略并导致出现一条警告消息。

C 语言综合源文件

· `syn.cflags`:

定义要应用于所有 `syn.file` 定义的源文件的编译标志，以供在综合期间使用。

```
syn.cflags=-I../src/
```

· `syn.csimflags`:

定义要应用于所有 `syn.file` 源文件的编译标志，以供在 C 语言仿真或 RTL/协同仿真期间使用。

· `syn.file`:

指定要在 HLS 组件综合期间使用的源文件的文件路径和名称。要指定多个文件，需添加多条 `syn.file` 语句。

文件路径可指定为绝对路径或相对路径。相对路径相对于配置文件的位置，无论是在 HLS 组件内部还是外部都是如此。

```
syn.file=../../src/dct.cpp
```

- `syn.file_cflags`:

将综合的编译标志应用于指定源文件。首先指定文件路径和名称，后接逗号，然后接 `cflags`：

```
syn.file_cflags=../../src/dct.cpp,-I../../src/
```

- `syn.file_csimflags`:

将仿真的编译标志应用于指定源文件。首先指定文件路径和名称，后接逗号，然后接 `csimflags`。

```
syn.file_csimflags=../../src/dct.cpp,-Wno-unknown-pragmas
```

- `syn.blackbox.file`:

指定用于 RTL 黑盒的 JSON 文件。HLS 编译器会在综合期间和运行 RTL/协同仿真时使用该文件。

```
syn.blackbox.file=../../RTL/fft.json
```

- `syn.top`:

指定函数名称，该函数会作为 HLS 组件的顶层函数来进行综合。帮助识别源代码中定义多个函数的顶层函数。

```
syn.top=dct
```



重要提示！ 顶层函数调用的函数也会成为 HLS 组件的一部分。

测试平台源文件

- `tb.cflags` arg:

为 `tb.file` 中定义的所有源文件定义编译标志，用于仿真或协同仿真。

```
tb.cflags=-Wno-unknown-pragmas
```

- `tb.file` arg:

指定测试平台源文件的文件路径和名称，用于 HLS 组件仿真或协同仿真。要指定多个文件，需添加多条 `tb.file` 语句。

文件路径可以是绝对路径，也可以是相对路径。相对路径相对于配置文件的位置，无论是在 HLS 组件内部还是外部都是如此。

```
tb.file=../../src/dct_test.cpp
```

- `tb.file_cflags` arg:

将用于仿真或协同仿真的编译标志应用于指定的测试平台源文件。首先指定文件路径和名称，后接逗号，然后接 `cflags`：

```
syn.file_cflags=../../src/dct.cpp,-Wno-unknown-pragmas
```

阵列分区配置

`syn.array_partition` 命令用于指定整个设计的阵列分区的默认行为。您可使用 `syn.directive.array_partition` 为特定阵列覆盖这些设置。

- `syn.array_partition.complete_threshold <value>`：

设置完整阵列分区的阈值。所含元素数量少于指定值的阵列会被分区为若干独立元素。

```
syn.array_partition.complete_threshold=12
```

- `syn.array_partition.throughput_driven [ auto | off ]`：

启用自动部分和/或完整阵列分区。

- `auto`：启用自动阵列分区，支持智能平衡面积与吞吐量。这是默认值。
- `off`：禁用自动阵列分区。

```
syn.array_partition.throughput_driven=off
```

C 语言仿真配置

`csim` 选项适用于 C 语言仿真进程，此进程用于确认设计的 C/C++ 语言。如需了解更多信息，请参阅《Vitis 高层次综合用户指南》(UG1399) 的“运行 C 语言仿真”部分。

- `csim.O`：

启用优化编译，以消除调试构造。默认值为 `false`。以调试模式执行编译以启用调试。

```
csim.O=true
```

- `csim.argv`：

指定行为测试平台的实参列表。在 C 语言测试平台中将 `<arg>` 传递到 `main()` 函数。

```
csim.argv=arg1 arg2
```

- `csim.clean`：

启用纯净构建。默认为 `false`。如不使用该选项，设计会以递增方式进行编译。

```
csim.clean=true
```

- `csim.profile_tripcount`: 启用循环次数剖析。

```
csim.profile_tripcount=true
```

- `csim.ldflags`:

指定传递给连接器以进行仿真的选项。该选项通常用于为 C/C++ 语言测试平台传递 `include` 路径信息或库信息。

```
csim.ldflags=ldExample
```

- `csim.mflags`:

提供传递给 C 语言仿真的编译器的选项，以加快编译速度。

```
csim.mflags=mExample
```

- `csim.profile_tripcount`:

启用循环次数剖析，默认为 `false`。

```
csim.profile_tripcount=true
```

- `csim.setup`:

在当前 HLS 组件的 `csim` 目录下创建仿真二进制文件。稍后，您可从已编译的可执行文件启动仿真。默认值为 `false`。完成设置后运行仿真。

```
csim.setup=true
```

协同仿真配置

`cosim` 选项适用于 C/RTL 协同仿真进程，此进程用于确认 HLS 综合所生成的 RTL。选项包括使用先前在 C 语言仿真中使用的 C/C++ 测试平台。选项还包括在行为仿真中使用 RTL 设计，如《Vitis 高层次综合用户指南》(UG1399) 的“Vitis HLS 中的 C/RTL 协同仿真”所述。

- `cosim.O`:

启用优化编译，以消除调试构造。默认值为 `false`。以调试模式执行编译以启用调试。启用 C/C++ 测试平台和 RTL 封装文件的优化编译会增加编译时间，但是能换取更好的运行时性能。

```
cosim.O=true
```

- `cosim.argv`:

指定行为测试平台的实参列表。在 C 语言测试平台中，会将指定的 `<arg>` 传递给 `main()` 函数。

```
cosim.argv=arg1 arg2
```

- `cosim.compiled_library_dir`:

指定使用第三方仿真器进行仿真期间已编译的库目录。<arg> 是已编译的库目录的路径名称。使用 `compile_simlib` 命令提前编译该库，如《Vivado Design Suite Tcl 命令参考指南》(UG835) 所述。

```
cosim.compiled_library_dir=../../simLib
```

- `cosim.coverage`:

启用使用 VCS 仿真器进行仿真期间的覆盖率功能。

```
cosim.coverage=true
```

- `cosim.disable_binary_tv`:

在协同仿真中禁用二进制测试向量。

```
cosim.disable_binary_tv=true
```

- `cosim.disable_deadlock_detection`:

禁用死锁检测，并在协同仿真中打开协同仿真死锁查看器。

```
cosim.disable_deadlock_detection=true
```

- `cosim.disable_dependency_check`:

运行协同仿真时禁用依赖关系检查。

```
cosim.disable_dependency_check=true
```

- `cosim.enable_dataflow_profiling`:

该选项会启用数据流通道剖析以跟踪协同仿真期间的通道大小。您必须启用该功能特性才能捕获数据流数据，如《Vitis 高层次综合用户指南》(UG1399) 的“数据流查看器”章节所述。

```
cosim.enable_dataflow_profiling=true
```

- `cosim.enable_fifo_sizing`:

在协同仿真期间，为数据流剖析启用自动 FIFO 通道大小调整。

```
cosim.enable_fifo_sizing=true
```

- `cosim.enable_tasks_with_m_axi`:

启用稳定的 `m_axi` 接口，以搭配 `hls::task` 一起使用。

```
cosim.enable_tasks_with_m_axi=true
```

- `cosim.hwemu_trace_dir`:

指定测试向量的位置，这些向量是在硬件仿真期间生成的，用作协同仿真期间的测试平台。

`syn.rtl.cosim_trace_generation` 命令会生成测试向量，如 [RTL 配置](#) 所述。该实参用于在硬件仿真的仿真结果中指定 Vitis 内核的内核和实例名称。其目的是帮助您为 HLS 组件定位测试向量。

```
cosim.hwemu_trace_dir=../../dct/dct_2
```

- `cosim.ldflags <arg>`:

指定传递给连接器以进行仿真的选项。为 C/C++ 语言测试平台传递 include 路径信息或库信息。

```
cosim.ldflags=ldExample
```

- `cosim.mflags <arg>`:

提供若干选项，这些选项会被传递给编译器用于 C 语言仿真。这样可以加速编译。

```
cosim.mflags=mExample
```

- `cosim.random_stall`:

在协同仿真期间启用顶层接口的随机停滞。

```
cosim.random_stall=true
```

- `cosim.rtl`:

指定 Verilog 或 VHDL 作为用于 C/RTL 协同仿真的语言。默认为 Verilog。

```
cosim.rtl=vhdl
```

- `cosim.setup`:

系统会在当前 HLS 组件的 `cosim` 目录中创建仿真二进制文件。但这样并不会执行仿真。稍后，您可从已编译的可执行文件启动仿真。默认值为 `false`。完成设置后会运行协同仿真。

```
cosim.setup=true
```

- `cosim.stable_axilite_update`:

启用 `s_axilite` 以配置寄存器，这些寄存器与传输事务前相比均保持稳定状态。

```
cosim.stable_axilite_update=true
```

- `cosim.tool`:

指定 HDL 仿真器，用于将 RTL 与 C 语言测试平台一起执行协同仿真。除非另行指定，否则 Vivado 仿真器 (`xsim`) 为默认仿真器。

- `auto`
- `vcs`
- `modelsim`
- `riviera`
- `isim`
- `xsim`
- `ncsim`
- `xcelium`

```
cosim.tool=modelsim
```


- `cosim.trace_level`:

判定 C/RTL 协同仿真期间，要保存的波形追踪数据级别。

- `none` 表示不保存追踪数据。这是默认方式。
- `all` 表示生成要保存到追踪文件的所有端口和信号波形。
- `port` 表示仅为顶层端口保存波形追踪数据。
- `port_hier` 表示保存设计层级内的所有端口的追踪信息。

```
cosim.trace_level=port
```

执行仿真时，会将追踪文件保存在组件的 `sim/Verilog` 文件夹或 `sim/VHDL` 文件夹中。具体位置取决于 `cosim.rtl` 选项中所做的选择。

- `cosim.user_stall`:

指定协同仿真期间使用的 JSON 停滞文件。此停滞文件可使用 `cosim_stall` 命令来生成。

```
cosim.user_stall=../../stall.json
```

- `cosim.wave_debug`:

打开 Vivado 仿真器 GUI 以查看波形和仿真结果。这样会在生成的 RTL 中启用查看所有进程的波形的功能。这些进程包含数据流和顺序进程。仅当使用 Vitis 仿真器执行协同仿真时才可使用该选项。您必须设置 `cosim.tool=xsim`。如需了解更多信息，请参阅 [查看仿真波形](#)。

```
cosim.wave_debug=true
```

编译选项

`syn.compile` 命令用于指定 HLS 组件编译的默认行为。

- `syn.compile.design_size_max_warning`: 指定会触发与编译缓慢或 QoR 欠佳相关警告的设计大小。

```
syn.compile.design_size_maximum_warning=300000
```

- `syn.compile.enable_auto_rewind`:

设为 TRUE 时，为使用自动循环回绕的流水打拍循环启用备用 HLS 实现。如需了解更多信息，请参阅《Vitis 高层次综合用户指南》(UG1399) 的[回绕已流水打拍的循环以保障性能](#)章节。

```
syn.compile.enable_auto_rewind=1
```

- `syn.compile.ignore_long_run_time`:

禁用运行时过长警告。

```
syn.compile.ignore_long_run_time=1
```

- `syn.compile.name_max_length <arg>`:

指定函数名称的最大长度。如果长度超过阈值，系统会将名称的最后一部分截断。

```
syn.compile.name_max_length=13
```

- `syn.compile.no_signed_zeros`:

`no_signed_zeros` 选项忽略浮点 0 值的符号类型：通过忽略符号类型，编译器即可对浮点运算执行激进的优化措施。

```
syn.compile.no_signed_zeros=1
```

- `syn.compile.performance_budgeter`:

启用该选项后，顶层性能编译指示会对整个设计进行全面的性能分析。就像在从 A 点到 B 点的众多布线可能性中选择最有效的布线一样，此分析会在整个设计中考虑多种优化策略。该分析有助于识别满足性能目标的最佳路径。

“enable”（启用）：激活性能预算，对整个设计进行综合性能分析。

“disable”（禁用）：停用性能预算，阻止执行任何综合分析。

“auto”（自动）：如果系统检测到性能编译指示，就会自动启用覆盖整个设计的性能分析。

- `syn.compile.pipeline_flush_in_task` arg:

指定默认情况下，在 `hls::tasks` 中会刷新流水线 (flp) 以降低 C/RTL 协同仿真中出现死锁的概率。该选项适用于以默认选项 `ii1` 达成 `II=1` 的流水线。但您也可以通过应用 `always` 启用在 `hls::tasks` 或数据流中刷新流水线来指定该选项，或者也可以使用 `never` 来完全禁用该选项。



重要提示！ 同时指定 `syn.compile.enable_auto_rewind` 选项时，刷新流水线 (flp) 与 PIPELINE 编译指示或指令中指定的 `rewind` 选项兼容。

- `always`：在 `hls::tasks` 或数据流中始终使流水线可刷新，忽略 `II`。

- `never`：从不使流水线变为可刷新，除非其他指令或编译指示明确覆盖此设置。

- `ii1`：在 `hls::tasks` 中将可实现 `II=1` 的流水线设为可刷新。这是默认设置。

```
syn.compile.pipeline_flush_in_task=always
```

- `syn.compile.pipeline_loops` arg:

当循环次数大于或等于您在此设置中指定的值时，循环会自动流水打拍。

```
syn.compile.pipeline_loops=20
```

- `syn.compile.pipeline_style` arg:

设置默认流水线样式，这是偏好设置，而非硬性约束。如需了解更多信息，请参阅《Vitis 高层次综合用户指南》(UG1399) 中的“刷新流水线和流水线类型”。有 3 种样式：可停滞 (stp)、可刷新 (flp) 和自由运行 (frp)。默认是自由运行。

```
syn.compile.pipeline_style=flp
```

- `syn.compile.pragma_strict_mode`:

对未识别和不正确的编译指示语法启用错误，而不是警告。

```
syn.compile.pragma_strict_mode=1
```

- `syn.compile.unsafe_math_optimizations`:

`unsafe_math_optimizations` 选项会忽略浮点 0 值的符号类型并启用关联的浮点运算：允许编译器对浮点运算执行激进的优化。

```
syn.compile.unsafe_math_optimizations=1
```



重要提示！ 使用该选项可能改变任意浮点运算的结果，导致 C/RTL 协同仿真不匹配。请确保测试平台容许差异，并检查差异的裕度，而不是精确值。

- `syn.compile_use_csim_directives`:

使用由 C 语言仿真在 C 语言综合期间生成的指令。默认设为 `false`。

```
syn.compile_use_csim_directives=true
```

数据流配置

综合：数据流选项

`syn.dataflow` 命令会为整个设计配置数据流分析。这些设置用于指定 `syn.directive.dataflow` 所使用的默认存储器通道和 FIFO 深度。

- `syn.dataflow.default_channel`:

默认情况下以乒乓方式并使用数据流流水打拍来配置 RAM 存储器时，Vitis 工具会使用函数或循环之间的数据。使用串流时（其中数据始终采用连续读取和连续写入），FIFO 存储器更高效。您可选择 FIFO 存储器作为默认存储器类型。

可用通道包括 `fifo` 和 `pingpong`。默认值为 `pingpong`。

```
syn.dataflow.default_channel=fifo
```



提示： 您必须使用 `set_directive_stream` 命令将阵列设置为串流以便执行 FIFO 访问。

- `syn.dataflow.disable_fifo_sizing_opt`:

禁用导致增大资源使用率的 FIFO 大小调整优化，这样可改善性能并减少死锁。

```
syn.dataflow.disable_fifo_sizing_opt=1
```

- `syn.dataflow.fifo_depth`:

用于指定 FIFO 默认深度的整数值。使用 `pingpong` 存储器时，该选项无效。默认情况下，通道内 FIFO 深度会设置为生产者或使用者中的较大者的值。在某些情况下，此方法可过于保守，导致 FIFO 过大，超出所需。如果您已知 FIFO 大于所需值，那么可使用该选项指定深度。

```
syn.dataflow.fifo_depth=6
```



重要提示！ 错误使用该选项可能导致设计无法正常工作。

- `syn.dataflow.override_user_fifo_depth`:

指定每个 `hls::stream` 的深度，覆盖任何用户设置。

```
syn.dataflow.override_user_fifo_depth=12
```

此设置可用于检查是否因设计中 FIFO 深度不足而导致出现死锁。如果将 `override` 设置为一个超大值，且没有发生死锁，您就可以：

- 使用协同仿真的 FIFO 深度剖析选项。
- 使用 GUI 找到既能确保性能又能避免死锁的最小深度。

仿真结束时协同仿真打印的最大深度就是一个较大的值的例子。

- `syn.dataflow.scalar_fifo_depth`:

用于指定标量值传输 FIFO 的最小深度的整数值。请参阅《Vitis 高层次综合用户指南》(UG1399) 的“指定编译器创建的 FIFO 深度”章节。这些 FIFO 用于将数据流区域的标量实参值转发到满足下列条件的进程：在区域本身内包含前代进程。它们不会影响功能正确性。如果自动计算所得大小不足，则可能导致性能损失，甚至死锁。

```
syn.dataflow.scalar_fifo_depth=4
```

如果不指定该选项，那么最小深度即 `syn.dataflow.fifo_depth` 选项的值，或者设为 2。根据经验，转发标量值的进程能够比读取该值的上一个进程先启动的平均次数即为该选项的有效值。

- `syn.dataflow.start_fifo_depth`:

用于指定开始传输 FIFO 的最小深度的整数值。请参阅《Vitis 高层次综合用户指南》(UG1399) 的“指定编译器创建的 FIFO 深度”。这些 FIFO 会将 `ap_start` 握手信号转发到满足下列条件的进程：在区域本身内包含前代进程。它们不影响功能正确，但如果自动计算所得大小不足，则可能导致性能损失。

```
syn.dataflow.start_fifo_depth=5
```

如果不指定该选项，那么最小深度即 `syn.dataflow.fifo_depth` 选项的值，或者值为 2。根据经验，有效值是允许进程先于其后代进程启动的预期平均次数。

- `syn.dataflow.strict_mode`:

为数据流规范形式消息设置严重性。可用模式包括：`error`、`warning` 和 `off`。默认值为 `warning`。

```
syn.dataflow.strict_mode=error
```

- `syn.dataflow.strict_stable_sync`:

强制对 `ap_done` 的稳定端口进行同步。如需了解更多信息，请参阅《Vitis 高层次综合用户指南》(UG1399) 的“Stable 阵列”章节。

```
syn.dataflow.strict_stable_sync=1
```

- `syn.dataflow.task_level_fifo_depth`:

默认任务级 FIFO 深度。用于为了在进程间传输标量而自动创建的 FIFO。通过 `ap_ctrl_chain` 来同步 FIFO。写操作是生产者的 `ap_done`，读操作则是使用者的 `ap_ready`。同步方面与 PIPO 相似，访问方面则与 FIFO 相似。

```
syn.dataflow.task_level_fifo_depth=7
```

调试选项

`syn.debug` 命令用于在 HLS 组件中启用调试并指定调试输出的写入位置。

- `syn.debug.enable`:

启用调试文件生成。如果您不启用该选项，则可在编译期间优化 HLS 组件。但该组件不支持调试。

```
syn.debug.enable=1
```



提示：该选项与 `v++ -c -g` 选项相关。在应用程序级别调试时，您必须在 HLS 组件中手动启用该选项。

- `syn.debug.directory`:

指定 HLS 调试输出的写入位置。如果不指定该选项，那么此位置设为 `hls/.debug`。

```
syn.debug.directory=../../debug
```

接口配置

`syn.interface` 命令用于配置应用于 HLS 组件的默认接口设置。对于特定顶层接口端口，可使用 `syn.directive.interface` 覆盖这些设置。

- `syn.interface.clock_enable`:

向设计添加时钟使能端口 (`ap_ce`)。时钟使能处于低电平有效状态时会阻止所有时钟操作，并禁用所有后续操作。默认为 `false`。

```
syn.interface.clock_enable=1
```

- `syn.interface.default_slave_interface`:

指定默认从接口。支持 2 种模式：`s_axilite` 和 `none`。默认值为 `s_axilite`。

```
syn.interface.default_slave_interface=none
```

- `syn.interface.m_axi_addr64`:

为所有 `m_axi` 接口启用 64 位寻址。默认启用该选项。禁用此设置时，`m_axi` 接口会使用 32 位寻址。

```
syn.interface.m_axi_addr64=1
```

- `syn.interface.m_axi_alignment_byte_size`:

为所有 `m_axi` 接口指定默认分配字节大小。对于 Vitis 内核流程，不指定该选项时，默认值为 64 字节。对于 Vivado IP 流程，默认值为 1 字节。请参阅《Vitis 高层次综合用户指南》(UG1399) 的“Vivado/Vitis 流程”章节。

```
syn.interface.m_axi_alignment_byte_size=16
```



提示： 值为 0 无效。

- `syn.interface.m_axi_auto_id_channel`:

为 `m_axi` 接口启用自动通道 ID 分配。默认禁用该选项。如需了解更多信息，请参阅《Vitis 高层次综合用户指南》(UG1399) 的“AXI4 主接口”章节。

```
syn.interface.m_axi_auto_id_channel=1
```

- `syn.interface.m_axi_auto_max_ports`:

为接口上的每个实参或端口启用自动创建独立 `m_axi` 接口适配器的功能。默认禁用该选项，从而将 `m_axi` 接口数量减少到所需的最小数量。如需了解更多信息，请参阅《Vitis 高层次综合用户指南》(UG1399) 的“M_AXI 捆绑”章节。

```
syn.interface.m_axi_auto_max_ports=1
```

- `syn.interface.m_axi_buffer_impl`:

为 `m_axi` 适配器内部的所有缓冲器指定实现资源。选项包括：`auto`、`lutram`、`bram` 和 `uram`。默认值为 `bram`。

```
syn.interface.m_axi_buffer_impl=lutram
```

- `syn.interface.m_axi_cache_impl`:

指定实现资源以供添加到 `m_axi` 适配器的高速缓存使用。选项包括：`auto`、`lutram`、`bram` 和 `uram`。默认值为 `auto`。

```
syn.interface.m_axi_cache_impl=lutram
```

- `syn.interface.m_axi_conservative_mode`:

配置所有 `m_axi` 适配器，使其在保守模式下工作。Vitis 工具会等待发出写入请求，直到关联的写入数据完全可用为止。通常该工具会等待数据缓冲到适配器中或已经发出为止。该工具会使用 M_AXI 适配器内的缓冲器来存储突发的所有数据，读取和写入时都是如此。默认启用此功能特性，这样可能导致写入时延有所增加，但是可以解决由于存储器子系统上的并发请求（读取或写入请求）而导致的死锁。将其设为 `false` 即可禁用保守模式。

```
syn.interface.m_axi_conservative_mode=0
```

- `syn.interface.m_axi_flush_mode`:

将所有 `m_axi` 适配器配置为可刷新，写入或读取垃圾数据。仅当因流水线阻塞而导致突发中断时，该选项才适用。例如，在非保守模式下缺少数据输入时或者缺少输出空间时。默认禁用该选项。

```
syn.interface.m_axi_flush_mode=1
```

• `syn.interface.m_axi_latency`:

全局指定 `m_axi` 接口期望的时延。允许设计先发起总线请求，经过数个周期（时延）后再执行期望的读取或写入操作。对于 Vitis 内核流程，默认值为 64。此默认值适用于 Vivado IP 流程。请参阅《Vitis 高层次综合用户指南》(UG1399) 中的“Vivado 和 Vitis 流程默认值”章节。

```
syn.interface.m_axi_latency=5
```

• `syn.interface.m_axi_max_bitwidth`:

指定 `m_axi` 接口数据通道的最大位宽。默认为 1024 位。指定的值必须为 2 的幂（介于 8 到 1024 之间）。如果实际访问大于所需接口，那么这样会减小吞吐率。这是因为吞吐量被拆分为多个周期的突发访问。

```
syn.interface.m_axi_max_bitwidth=128
```

• `syn.interface.m_axi_max_read_burst_length`:

指定突发传输期间所有 `m_axi` 接口读取的数据值的全局最大数量。默认值为 16。

```
syn.interface.m_axi_max_read_burst_length=12
```

• `syn.interface.m_axi_max_widen_bitwidth`:

启用端口宽度自动调整，以便为 `m_axi` 接口拓宽突发，最高可拓宽至所选位宽。指定的值必须为 2 的幂（介于 8 到 1024 之间），并且必须与 `-m_axi_alignment_size` 对齐。对于 Vitis 内核流程，默认值为 512，对于 Vivado IP 流程则为 0。

```
syn.interface.m_axi_max_widen_bitwidth=64
```

• `syn.interface.m_axi_max_write_burst_length`:

指定突发传输期间所有 `m_axi` 接口写入的数据值的全局最大数量。默认值为 16。

```
syn.interface.m_axi_max_write_burst_length=12
```

• `syn.interface.m_axi_min_bitwidth`:

指定 `m_axi` 接口数据通道的最小位宽。默认为 8 位。值必须为 2 的幂（介于 8 到 1024 之间）。如果实际访问小于所需接口，那么这样并不一定会增加吞吐量。

```
syn.interface.m_axi_min_bitwidth=64
```

• `syn.interface.m_axi_num_read_outstanding`:

指定在设计停滞前可对 `m_axi` 接口发出读取请求的数量（无响应）。此操作暗示设计中的内部存储空间，且 FIFO 大小为：

```
num_read_outstanding*max_read_burst_length*word_size
```

默认值为 16。

```
syn.interface.m_axi_num_read_outstanding=8
```

- `syn.interface.m_axi_num_write_outstanding`:

指定在设计停滞前可对 `m_axi` 接口发出写入请求的数量（无响应）。此操作暗示设计中的内部存储空间，且 FIFO 大小为：

```
num_write_outstanding*max_write_burst_length*word_size
```

默认值为 16。

```
syn.interface.m_axi_num_write_outstanding=8
```

- `syn.interface.m_axi_offset`:

指定所有 `m_axi` 接口的默认偏移机制。选项为：

- `off`: 不生成偏移端口。
- `slave`: 生成偏移端口并自动将其映射到 `s_axilite` 接口。该选项是默认值。
- `direct`: 生成标量输入偏移端口，以便通过偏移端口将地址偏移直接传递到 IP 内。

```
syn.interface.m_axi_offset=slave
```

- `syn.interface.register_io`:

为顶层函数上的所有标量输入、所有输出或所有标量端口全局启用寄存器。始终寄存阵列。选项包括：`off`、`scalar_in`、`scalar_out` 和 `scalar_all`。默认值为 `off`。

```
syn.interface.register_io=scalar_out
```

- `syn.interface.s_axilite_data64`:

为 `s_axilite` 接口启用 64 位数据宽度。使用 1 即可启用 64 位数据宽度，使用 0 则将其禁用。默认为 32 位数据宽度。

```
syn.interface.s_axilite_data64=1
```

- `syn.interface.s_axilite_interrupt_mode`:

为将在读取时清零 (`cor`) 或者在写入时切换 (`tow`) 的 `s_axilite` 接口指定中断模式。Vitis 工具可以在单一传输事务内完成 UI 读取中断的清除操作，而 `tow` 则需要两个传输事务才能完成此操作。`Tow` 是默认中断模式。

```
syn.interface.s_axilite_interrupt_mode=cor
```

- `syn.interface.s_axilite_mailbox`: 启用为非串流非稳定 `s_axilite` 实参创建邮箱的功能。邮箱功能特性等会设置和管理自动重启内核。请参阅《Vitis 高层次综合用户指南》(UG1399) 中的“自动重启模式”章节。可用选项包括：

- `in`: 仅为输入实参启用邮箱
- `out`: 仅为输出实参启用邮箱
- `both`: 为输入和输出实参启用邮箱

- `none`：不创建邮箱。该选项是默认值。

```
syn.interface.s_axilite_mailbox=in
```

- `syn.interface.s_axilite_status_regs`:

启用在 `s_axilite` 寄存器文件中暴露 ECC 错误位的功能。通过为启用 ECC 的每个块 RAM 或 URAM 设置两个读取时清零 (COR) 计数器，启用 ECC 错误位的暴露。选项包括：`ecc` 启用该功能，或者 `off` 禁用该功能。默认设置为禁用。

```
syn.interface.s_axilite_status_regs=ecc
```

- `syn.interface.s_axilite_sw_reset`:

在 `s_axilite` 适配器中启用软件复位。

```
syn.interface.s_axilite_sw_reset=1
```

封装选项

输出选项

`package.output` 选项用于指定已综合的 RTL 设计所生成的导出文件的性质。这些选项包括：

- `package.output.file`：为导出的文件指定输出文件路径和名称。如不指定该选项，那么 Vitis Unified IDE 会根据 HLS 组件来命名输出文件。请参阅以下示例命令：

```
package.output.file=../kernel.xo
```

- `package.output.format`：指定综合后生成的输出文件的导出格式。受支持的值包括：
 - `package.output.format=ip_catalog`：适合添加到 Vivado IP 目录的格式。
 - `package.output.format=xo`：v++ 编译器接受的格式，用于在 Vitis 应用加速流程中进行链接。
 - `package.output.format=syn_dcp`：Vivado Design Suite 的已综合的检查点。使用该选项来执行 RTL 综合。可选使用 `vivado.flow=impl` 来添加 Vivado 实现
 - `package.output.format=sysgen`：生成 IP 和 ZIP 存档以供在 System Generator 中使用。
 - `package.output.format=rtl`：从 C 语言综合输出 RTL 文件。这样并不会封装 IP 或 XO 用于下游进程。



提示：RTL 格式适用于 HLS 组件的开发、分析和调试。但要导出文件，您必须指定任一其他输出格式。

- `package.output.syn`：启用在 C 语言综合步骤中运行“Package”步骤，以便在综合期间生成所需的导出文件。

IP 选项

`package.ip` 命令用于指定 HLS 组件生成的 IP 的详细信息。这些设置用于定义 IP 供应商、库、名称和版本 (VLNV)，例如，用于在 IP 目录内识别此 IP。

- `package.ip.description`:

提供生成的 IP 的目录条目描述，在封装 IP 时使用。

```
package.ip.description=details of IP
```

- `package.ip.display_name`:

提供生成的 IP 的目录条目的显示名称，在封装 IP 时使用。

```
package.ip.display_name=randy1
```

- `package.ip.library`:

为生成的 IP 的 `<Vendor>:<Library>:<Name>:<Version>` (VLNV) 标识符提供库作为该标识符的组成部分。

```
package.ip.library=testLib
```

- `package.ip.name`:

为生成的 IP 的 `<Vendor>:<Library>:<Name>:<Version>` (VLNV) 标识符提供名称作为该标识符的组成部分。

```
package.ip.name=randy1
```

- `package.ip.taxonomy`:

指定生成的 IP 的目录条目的分类法，在封装 IP 时使用。Taxonomy（分类法）是有助于识别 IP 用途或应用的分类方法。

```
package.ip.taxonomy=video
```

- `package.ip.vendor`:

为生成的 IP 的 `<Vendor>:<Library>:<Name>:<Version>` (VLNV) 标识符提供供应商作为该标识符的组成部分。

```
package.ip.vendor=randyCom
```

- `package.ip.version`:

为生成的 IP 的 `<Vendor>:<Library>:<Name>:<Version>` (VLNV) 标识符提供版本作为该标识符的组成部分。

```
package.ip.version=2.3
```

- `package.ip.xdc_file`:

指定 XDC 文件。Vivado 工具中的实现会使用封装的 IP 的内容。

```
package.ip.xdc_file=../../ip.xdc
```

- `package.ip.xdc_ooc_file`:

指定非关联 (OOC) XDC 文件。此文件的内容包含在已封装的 IP 内，以供在针对导出的 IP 执行非关联 Vivado 综合期间使用。

```
package.ip.xdc_ooc_file=../..ooc,xdc
```

运算符配置

`syn.op` 命令允许您为用于 HLS 组件的不同运算符配置默认实现风格、时延和精度。您可向配置文件添加多条 `syn.op` 命令，以指定不同运算符的详细信息。如果未指定运算符，那么 Vitis 工具会判定该组件的默认值。

您可使用 `syn.directive.bind_op` 命令为特定变量覆盖 `syn.op` 命令所指定的默认设置。

- `syn.op`:

`syn.op` 命令语法如下所示。

```
syn.op=op:mul impl:dsp
syn.op=op:add impl:fabric latency:6
syn.op=op:fmacc precision:high
syn.op=op:hdiv latency:5
```

- `syn.op=`: 启动命令
- `op:<operator>`: 指定 `op` 关键字，后接当前要定义的运算符。
 - `mul`: 整数乘法运算
 - `add`: 整数加法运算
 - `sub`: 整数减法运算
 - `fadd`: 单精度浮点加法运算
 - `fsub`: 单精度浮点减法运算
 - `fdiv`: 单精度浮点除法运算
 - `fexp`: 单精度浮点指数运算
 - `flog`: 单精度浮点对数运算
 - `fmul`: 单精度浮点乘法运算
 - `frsqrt`: 单精度浮点平方根倒数运算
 - `frecip`: 单精度浮点倒数运算
 - `fsqrt`: 单精度浮点平方根运算
 - `dadd`: 双精度浮点加法运算
 - `dsub`: 双精度浮点减法运算
 - `ddiv`: 双精度浮点除法运算
 - `dexp`: 双精度浮点指数运算

- `dlog`: 双精度浮点对数运算
- `dmul`: 双精度浮点乘法运算
- `drsqrt`: 双精度浮点平方根倒数运算
- `drecip`: 双精度浮点倒数运算
- `dsqrt`: 双精度浮点平方根运算
- `hadd`: 半精度浮点加法运算
- `hsub`: 半精度浮点减法运算
- `hdiv`: 半精度浮点除法运算
- `hmul`: 半精度浮点乘法运算
- `hsqrt`: 半精度浮点平方根运算
- `facc`: 单精度浮点累加运算
- `fmac`: 单精度浮点乘积累加运算
- `fmadd`: 单精度浮点乘加运算



提示：比较运算符（如，`dcmp`）会在 LUT 内实现。这些运算符无法在互连结构外实现，也无法映射到 DSP。您无法使用 `syn.op` 命令或 `syn.directive.bind_op` 命令来配置运算符。

- `impl:<value>`: 指定实现 (`impl`) 关键字，后接指定运算符的值。不指定 `impl` 时，默认工具会为给定运算符判定最佳实现。支持值包括：
 - `all`: 所有实现。这是默认设置。
 - `dsp`: 使用 DSP 资源
 - `fabric`: 使用非 DSP 资源
 - `meddsp`: 浮点 IP 中度使用 DSP 资源
 - `fulldsp`: 浮点 IP 完全使用 DSP 资源
 - `maxdsp`: 浮点 IP 最大程度使用 DSP 资源
 - `primitivedsp`: 浮点 IP 原语使用 DSP 资源
 - `auto`: 启用组合 `facc` | `fmac` | `fmadd` 运算符的推断
 - `none`: 禁用组合 `facc` | `fmac` | `fmadd` 运算符的推断
- `latency:<value>`: 指定 `latency` 关键字，后接值。定义默认时延，用于将运算符绑定到实现资源。有效值范围因运算的每个实现而异 (`impl`)。默认值为 -1，即允许该工具为实现资源应用标准时延。



提示：您无需指定实现，即可为特定运算指定时延。这样会交由该工具在管理时延的同时选择实现。

- `precision:<value>`: 用于浮点运算符 (`facc`、`fmac` 和 `fmadd`)，它会指定 `precision` 关键字，后接以下值之一：
 - `low`: 使用低精度（60 位和 100 位整数）累加实现（如果可用）。该选项仅在某些非 AMD Versal™ 器件上可用，由于 C++ 仿真相关精度不足，因此它可能导致 RTL/协同仿真发生不匹配。但器件无需源代码更改即可进行流水打拍，且 `II=1`。该选项使用的资源约为 `standard` 精度浮点累加资源所用资源的三倍。

- **high**：使用高精度（1 个额外的位）融合乘加实现（如果可用）。该选项适用于高精度应用。该选项在 Versal 器件上非常高效，但由于 C++ 仿真相关的额外精度，因此可能导致 RTL/协同仿真发生不匹配。该选项使用的资源比 **standard** 精度浮点累加更多。
- **standard**：标准精度浮点累加和乘加适合大部分浮点用例，它是默认设置。它始终使用真正的浮点累加器，可在 Versal 器件上采用 $II=1$ 来进行流水打拍，在非 Versal 器件上， II 通常介于 3 到 5 之间（取决于时钟频率和目标器件）。

RTL 配置

`syn.rtl` 命令用于配置已编译的 RTL 的各属性、所使用的复位类型以及状态机的编码。此命令还允许您在 RTL 中使用特定标识。默认情况下，这些选项适用于顶层设计以及设计中的所有 RTL 块。

· `syn.rtl.cosim_trace_generation`:

在 Vitis 工具流中执行硬件仿真期间生成测试向量。当内核综合为 Vitis 内核时，适用该选项。在未来设计迭代中，此进程可作为实用的 C/RTL 协同仿真测试平台。

```
syn.rtl.cosim_trace_generation=1
```

· `syn.rtl.deadlock_detection`:

在已导出的 IP/XO 文件的顶层 RTL 中启用仿真或综合死锁检测。其中选项如下：

- “none”（无）：禁用死锁检测。
- “sim”（仿真）：仅对仿真启用死锁检测。这是默认设置。
- “hw”（硬件）：在已综合的 RTL IP 中启用死锁检测。将 `ap_local_deadlock` 和 `ap_local_block` 信号添加到 IP 以启用局部和全局死锁检测。

```
syn.rtl.deadlock_detection=hw
```

· `syn.rtl.deadlock_diagnosis`:

在应用的硬件仿真期间，为 Vitis 内核 (.xo) 启用死锁检测诊断。

```
syn.rtl.deadlock_diagnosis=1
```

· `syn.rtl.fsm_encoding`:

指定 RTL 属性值为 “fsm_encoding” 来引导 RTL 综合。其中选项如下：

- “auto”（自动）：允许 RTL 综合工具判定最佳状态机编码
- “gray”（格雷码）：使用格雷码作为状态机编码。
- “johnson”（约翰逊码）：使用约翰逊码作为状态机编码。
- “one_hot”（独热码）：使用 one_hot 独热码作为状态机编码。
- “sequential”（顺序）：使用顺序状态机编码。
- “none”（无）：禁用状态机编码。状态机使用 RTL 中指定的状态码 (one_hot) 进行精确综合。

```
syn.rtl.fsm_encoding=gray
```

- `syn.rtl.fsm_safe_state`:

指定 RTL 属性值为 “`fsm_safe_state`” 来引导 RTL 综合。该属性会影响综合结果的质量，通常会导致面积越大性能越差。其中选项如下：

- `auto_safe_state`: 暗示采用 Hamming-3 编码。
- `default_state`: 将状态机恢复到默认状态。
- `power_on_state`: 将状态机恢复到 `POWER_ON` 状态。
- `reset_state`: 将状态机恢复到 `RESET` 状态。
- `none`: 不在 RTL 中添加属性，状态机将不包含安全状态逻辑。

```
syn.rtl.fsm_safe_state=auto_safe_state
```

- `syn.rtl.header`:

指定一个文件，其中内容将插入生成的所有 RTL 文件的开头处。这样您即可确保生成的 RTL 文件包含用户指定的内容。

```
syn.rtl.header=../../myHeader.txt
```

- `syn.rtl.kernel_profile`:

添加内核剖析所需的顶层事件和停滞端口。

```
syn.rtl.kernel_profile=1
```



重要提示！ 该选项与 `v++ -c --profile.stall` 命令相关。您必须手动将此选项添加到 HLS 组件中，以确保停滞剖析可用于已链接的应用。

- `syn.rtl.module_auto_prefix`:

指定顶层函数名称，作为生成的 RTL 模块的前缀值。如果同时指定 `syn.rtl.module_prefix`，则忽略该选项。默认启用该选项。

```
syn.rtl.module_auto_prefix=1
```

- `syn.rtl.module_prefix`:

指定要用于所有生成的 RTL 模块名称的前缀。此设置可用于覆盖顶层函数的默认模块前缀。

```
syn.rtl.module_prefix=newTop
```

- `syn.rtl.mult_keep_attribute`:

启用 `keep` 属性。

```
syn.rtl.mult_keep_attribute=1
```

- `syn.rtl.register_all_io`:

默认为所有 I/O 信号使用单个寄存器。

```
syn.rtl.register_all_io=1
```

- `syn.rtl.register_reset_num`:

要添加到复位信号的寄存器数量。

```
syn.rtl.register_reset_num=2
```

- `syn.rtl.reset`: C/C++ 语言代码中变量的初始化值与 RTL 中初始化的值始终相同，因此与比特流中初始化值也始终相同。此初始化仅在上电时执行，对设计应用复位时不会重复此操作。

通过 `-reset` 选项应用该设置，以判定寄存器和存储器的复位方式。

- `none`: 不向设计添加复位。
- `control`: 复位控制寄存器，例如，状态机中使用的寄存器和用于生成 I/O 协议信号的寄存器。这是默认设置。
- `state`: 复位控制寄存器和从 C/C++ 语言代码中的静态变量或全局变量衍生的寄存器或存储器。C/C++ 语言代码中已初始化的任意静态变量或全局变量都复位为其初始值。
- `all`: 复位设计中的所有寄存器和存储器。C/C++ 语言代码中已初始化的任意静态变量或全局变量都复位为其初始值。

```
syn.rtl.reset=state
```

- `syn.rtl.reset_async`:

导致所有寄存器都使用异步复位。如果您不指定该选项，则使用同步复位。

```
syn.rtl.reset_async=1
```

- `syn.rtl.reset_level`:

将复位信号的极性定义为“active-Low”（低电平有效）或“active-High”（高电平有效）。默认设置为 active-High。

```
syn.rtl.reset_level=low
```



提示：AXI 协议要求采用低电平有效复位。如果您的设计使用 AXI 接口，那么 Vitis 工具会定义此复位级别并显示警告，前提是 `syn.rtl.reset_level` 设为高电平有效。

调度设置

`syn.schedule` 命令用于配置所执行的调度的默认类型。

- `syn.schedule.enable_dsp_full_reg`:

指定需要完全寄存的 DSP 信号。默认启用该选项。

```
syn.schedule.enable_dsp_full_reg=0
```

存储配置

指定 HLS 微架构将 FIFO 存储元件绑定到存储器资源时使用的全局默认选项。

您可针对特定设计元素运行 `syn.directive.bind_storage` 指令来覆盖 `syn.storage` 为 FIFO 存储所定义的默认配置。您也可通过为接口上的对象运行 `syn.directive.interface` 命令指定 `storage_type` 选项来覆盖默认配置。

- `syn.storage`:

请参阅以下示例，了解 `syn.storage` 命令的语法。

```
syn.storage=fifo impl=auto auto_srl_max_bits=512 auto_srl_max_depth=3
```

- `syn.storage=fifo`: 启动命令以配置 FIFO。

注释: 目前 FIFO 是唯一受支持的类型。

- `impl=<value>`: 指定实现 (`impl`) 关键字后接值。如果不指定 `impl`, 则默认值为 `auto`。使用默认值表示允许 Vitis 工具为给定运算符判定最佳实现。该选项支持以下值: `auto`、`bram`、`lutram`、`uram`、`memory` 和 `srl`。
- `auto_srl_max_bits=<value>`: 仅对 `impl:auto` (默认设置) 有效。指定允许的 SRL 总位数 (深度 * 宽度) 最大值即可自动实现。默认值为 1024。
- `auto_srl_max_depth=<value>`: 为 `auto-srl` 实现指定允许的最大 SRL 深度。默认值为 2。

实现配置

`syn.vivado` 命令用于配置 Vivado 综合与实现运行，以便衍生出资源使用率和时序估算信息。这些设置通常不影响为 HLS 组件创建的 RTL，但可能影响用于估算的硬件设计示例。

- `vivado.clock`:

覆盖 Vivado 非关联运行中使用的 HLS 时钟约束，用于时序分析的判定。这不影响 IP 或 XO 文件的输出。

```
vivado.clock=37
```

- `vivado.flow`:

运行 Vivado 流程，可选仅运行综合流程 (`syn`) 或仅运行实现流程 (`impl`)。运行综合比运行实现更快，但缺少实现运行的部分详细信息。默认设置为 `impl`。

```
vivado.flow=syn
```



提示: 这有别于 `flow_target` 选项，后者将目标定义为 Vitis 内核流程或 Vivado IP 流程。

- `vivado.impl_strategy`:

指定 Vivado 综合策略或实现策略。策略名称可在《Vivado Design Suite 用户指南：实现》(UG904) 的“定义实现策略”中找到。此设置用于资源使用情况和时序分析，不影响输出文件。

```
vivado.impl_strategy=Performance_Explore
```

- **vivado.max_timing_paths:**

指定在 Vivado 综合或实现中不满足时序的情况下，要报告的时序路径的最大数量。

```
vivado.max_timing_paths=12
```

- **vivado.optimization_level:**

Vivado 优化级别，用于设置其他 `vivado_*` 选项。这不适用于 IP/XO 输出。选项包括 0、1、2 和 3。它仅用于报告生成，将不适用于导出的 IP。默认设置为 0。

```
vivado.optimization_level=2
```

- **vivado.pblock:**

指定要在实现期间用于报告目的的 Pblock 范围。

```
vivado.pblock={SLICE_X8Y105:SLICE_X23Y149}
```

- **vivado.phys_opt:**

在指定的实现阶段中运行 Vivado 物理优化。选项包括：无优化 (`none`)、布局优化 (`place`)、布线优化 (`route`) 或布局布线 (`all`) 优化。该选项仅适用于实现运行，此运行用于生成资源和时序估算。

```
vivado.phys_opt=route
```

- **vivado.report_level:**

指定 Vivado 综合与实现运行的报告级别。有效值和关联的报告包括：

- 0：综合后使用率。实现后使用率和时序。
- 1：综合后使用率、时序和分析。实现后使用率、时序和分析。
- 2：综合后使用率、时序、分析和快速失败 (`failfast`)。实现后使用率、时序和快速失败。这是默认设置。

```
vivado.report_level=1
```

- **vivado.rtl:**

指定要在 Vivado 流程中选择的 RTL 语言 (`verilog/vhdl`)。

```
vivado.rtl=vhdl
```

- **vivado.synth_design_args:**

指定 Vivado `synth_design` 命令的实参。可用的 `synth_design` 实参可在《Vivado Design Suite Tcl 命令参考指南》(UG835) 中找到。

```
vivado.synth_design_args=arg1 arg2
```

- `vivado.synth_strategy`:

指定生成设计示例时要使用的 Vivado 综合策略，用于资源和时序估算。具体综合策略可在《Vivado Design Suite 用户指南：综合》(UG901) 中找到。

```
vivado.synth_strategy=Synthesis_Defaults
```

展开设置

`syn.unroll` 设置用于提供全局循环次数阈值，低于该阈值的循环会自动展开。所含循环次数更大的循环可使用 `syn.directive.unroll` 来展开。

- `syn.unroll.tripcount_threshold`:

所含迭代数少于指定值的所有循环都将自动展开。默认值为 0，这表示不会自动展开循环。

```
syn.unroll.tripcount_threshold=6
```

HLS 优化指令



重要提示！ 在 HLS 配置文件中的 `[hls]` 头文件下必须出现下列选项。

指令或 `syn.directive.xxx` 命令允许您为跨多个实现的同一源代码自定义综合结果。更改指令即可更改结果。
`syn.directive.xxx` 命令供 Vitis Unified IDE 在配置文件内使用。请参阅《Vitis 高层次综合用户指南》(UG1399) 中的“构建和运行 HLS 组件”。

注释： 您也可以在源代码中的使用编译指示，代替配置文件中使用的指令。这样做的好处是可以将编译指示直接存储在源代码中。如需了解更多信息，请参阅《Vitis 高层次综合用户指南》(UG1399) 的“HLS 编译指示”章节。

通过配置文件应用的指令必须包含 `<location>` 作为指令的实参。`<location>` 用于定义该指令应用于哪些源代码的元素，例如，函数、循环、区域或变量。

`syn.directive.xxx` 命令的语法示例包括指令的位置和实参。例如：

```
syn.directive.pipeline=dct2d II=4
```

其中，`dct2d` 表示 PIPELINE 指令应用到的位置（函数名称），`II=4` 是指令的可用实参之一。

`syn.directive.aggregate`

描述

该指令用于将结构体的数据字段收集到单个宽型标量中。Vitis HLS 针对结构体内部阵列执行的操作与 `syn.directive.array_reshape` 类似。Vitis HLS 会将阵列彻底分区并重构为一个宽型标量，并将其与结构体的其他元素一起打包。



提示： Vitis HLS 会将结构体的阵列重构为聚合元素组成的阵列。此优化不支持任何结构体包含其他结构体。

您可根据结构体元素的声明顺序推断出生成的新字宽标量的位对齐方式。第一个元素占用此字的最低有效位，依此类推，直到映射完成所有字段为止。

语法

```
syn.directive.aggregate=[OPTIONS] <location> <variable>
```

- <location> 是包含打包的变量的位置（格式为 `function[/label]`）。
- <variable> 是要打包的结构体变量。

选项

- `compact=[bit | byte | none | auto]`：指定已聚合的结构体的对齐方式。可选设置包括：按位级对齐（打包）、按字节级对齐（填充）、无对齐或者由工具自动与打包联合判定对齐方式（默认行为）。

示例

以下示例聚合了变量 AB。该变量是一个结构体指针，其中在 `func` 函数内包含 3 个 8 位字段 (`typedef struct {unsigned char R, G, B;}pixel`)。这些字段被聚合为一个新的 24 位指针，并在位级别对齐数据。

```
syn.directive.aggregate=func AB compact=bit
```

另请参阅

- [syn.directive.array_reshape](#)
- [syn.directive.disaggregate](#)

syn.directive.alias

描述

指定两个或两个以上的 `M_AXI` 指针实参指向存储器（DDR 或 HBM）内的同一个底层缓冲器。通过设置指针间的距离或偏移来设置指针间的混叠。



重要提示！ `syn.directive.alias` 适用于已映射到 `M_AXI` 接口的顶层函数实参。

Vitis HLS 将不同指针视为独立通道，通常不提供任何依赖关系分析。但如果主机为多个指针分配单个缓冲器，则可通过 `syn.directive.alias` 来告知此关系。这种通信方式可保留依赖关系分析。通过 `syn.directive.alias` 进行通信还可定义缓冲器内指针之间的距离，从而在 Vitis HLS 中启用数据依赖关系分析。

`syn.directive.alias` 的要求：

- 您必须将所有端口都分配给 `M_AXI` 接口，并将其分配给不同的捆绑。请参阅以下示例以获取详细信息。
- 在 `syn.directive.alias` 命令中，每个端口只能使用一次。
- 分配给 `syn.directive.alias` 的所有端口都必须设为相同深度。
- 如果您指定偏移，那么每个端口均可有一个偏移量。

- 您必须为 M_AXI 端口指定偏移，支持将偏移指定为 `slave` 或 `direct`。不支持 `offset=off`。

语法

```
syn.directive.alias=[OPTIONS] <location> <ports>
```

- `<location>` 是位置字符串，格式为 `function[/label]`，ALIAS 编译指示即应用于此指定位置。
- `<ports>` 用于指定要混叠的端口。

选项

- `distance=<integer>`：指定传递到列表中的端口的指针值之间的距离。
- `offset=<string>`：指定传递到 `ports` 列表中每个端口的指针相对于阵列原始指针的偏移。

示例

对于以下 `top` 函数：

```
void top(int *arr0, int *arr1, int *arr2, int *arr3, ...) {
    #pragma HLS interface M_AXI port=arr0 bundle=hbm0 depth=0x40000000
    #pragma HLS interface M_AXI port=arr1 bundle=hbm1 depth=0x40000000
    #pragma HLS interface M_AXI port=arr2 bundle=hbm2 depth=0x40000000
    #pragma HLS interface M_AXI port=arr3 bundle=hbm3 depth=0x40000000
}
```

以下命令用于为指定的阵列指针定义混叠，并定义指针之间的 `distance`：

```
syn.directive.alias=top arr0,arr1,arr2,arr3 distance=10000000
```

或者，以下命令用于指定指针之间的 `offset`，达成效果与以上命令相同：

```
syn.directive.alias=top arr0,arr1,arr2,arr3
offset=00000000,10000000,20000000,30000000
```

另请参阅

- [syn.directive.interface](#)

syn.directive.allocation

描述

指定资源分配的实例限制。

`syn.directive.allocation` 可限制用于实现特定函数、循环或运算的 RTL 实例和硬件资源的数量。如果 C/C++ 语言源代码的 `foo_sub` 函数有 4 个实例，则 `syn.directive.allocation` 会确保在最终 RTL 中只有一个 `foo_sub` 实例。全部 4 个实例均使用相同的 RTL 块来实现。这样即可减少函数所使用的资源量，但由于共享这些资源，故而对性能会产生负面影响。

`syn.directive.allocation` 命令还会限制 C/C++ 代码中的运算（加法、乘法、阵列读取和写入）。

语法

```
syn.directive.allocation=[OPTIONS] <location> <instances>
```

- `<location>` 是位置字符串，格式为 `function[/label]`。
- `<instances>` 是函数或运算符。该函数可以是原始 C/C++ 语言代码中未由 `syn.directive.allocation` 命令内联或未由 Vitis HLS 自动内联的任何函数。

选项

- `limit=<integer>`:

设置要在 RTL 设计中使用的实例（类型由 `type` 选项定义）数量上限。

- `type=[function|operation]`: 实例类型可以是 `function`（默认）或 `operation`。要获取受支持的运算的完整列表，请参阅 [运算符配置](#)。

示例

给定 `foo_top` 设计中有多个 `foo` 函数的实例，在 RTL 中将 `foo` 的实例数限制为 2。

```
syn.directive.allocation=limit=2 type=function foo_top foo
```

将 `My_func` 实现中使用的乘法器数限制为 1。此限制不适用于可能驻留在 `My_func` 的子函数中的任何乘法器。要限制任何子函数实现过程中使用的乘法器数量，请在子函数上指定分配指令，或将子函数内联到 `My_func` 函数中。

```
syn.directive.allocation=limit=1 type=operation My_func mul
```

另请参阅

- [syn.directive.inline](#)

syn.directive.array_partition

描述



重要提示！ 针对顶层函数上的 M_AXI 接口，不支持 `syn.directive.array_partition` 和 `syn.directive.array_reshape`。您可改用 `hls::vector` 数据类型，如《Vitis 高层次综合用户指南》(UG1399) 的“向量数据类型”章节所述。

`syn.directive.array_partition` 将阵列分区为更小的阵列或者独立元素。

此分区可执行以下操作。

- 生成包含多个小型存储器或多个寄存器（而不是一个大型存储器）的 RTL
- 为存储器增加读写端口数量
- 能改善设计吞吐量
- 需要更多存储器实例或寄存器

语法

```
syn.directive.array_partition=[OPTIONS] <location> <array>
```

- <location> 是包含阵列变量的位置（格式为 `function[/label]`）。
- <array> 是要分区的阵列变量。

选项

- `dim=<integer>`：指定要执行分区的阵列的维度。默认值为 1。
 - 该维度仅与多维阵列相关。
 - 如果使用的值为 0，那么会使用指定选项对所有维度进行分区。
 - 设置任何其他值都仅对该值对应的维度进行分区。例如，如果值为 1，则仅对第 1 个维度进行分区。
- `type=(block|cyclic|complete)`：
 - `block` 型分区会从原始阵列的连续块创建更小阵列。这样可将阵列有效分区为 N 个相等的块，其中 N 为 `-factor` 选项定义的整数。
 - `cyclic` 型分区会通过交织来自原始阵列的元素来创建更小的阵列。例如，如果使用 `-factor 3`，结果如下。
 - 该指令会向第 1 个新阵列分配元素 0。
 - 向第 2 个新阵列分配元素 1。
 - 向第 3 个新阵列分配元素 2。
 - 再次向第 1 个新阵列分配元素 3。
 - `complete` 分区可将阵列分解为多个独立元素。对于一维阵列，这对应于将存储器解析为独立寄存器。对于多维阵列，请指定每个维度的分区方式，或者使用 `-dim 0` 选项来对所有维度进行分区。

默认值为 `complete`。

- `factor=<integer>`：该选项仅用于 `block` 或 `cyclic` 分区，用于指定要创建的更小的阵列数量。
- `off=true`：为指定变量和指定的 `dim`（如有）禁用 `ARRAY_PARTITION` 功能特性。

示例 1

将 `func` 函数中的 `AB[13]` 阵列分区为 4 个阵列。由于 4 不是 13 的整数因子，因此：

- 有 3 个阵列各包含 3 个元素。
- 另 1 个阵列包含 4 个元素 (`AB[9:12]`)。

```
syn.directive.array_partition=func AB type=block factor=4
```

将 `func` 函数中的 `AB[6][4]` 阵列分区为 2 个阵列，每个阵列维度为 `[6][2]`。

```
syn.directive.array_partition=func AB type=block factor=2 dim=2
```

将 `func` 函数中的 `AB[4][10][6]` 的所有维度分区为独立元素。

```
syn.directive.array_partition=func AB type=complete dim=0
```

示例 2

您可通过已分区阵列的新结构来对自己的代码中已分区的阵列进行寻址。使用以下指令时，请参阅以下示例代码。

```
syn.directive.array_partition=top b type=complete dim=1
```

代码结构如下：

```
struct SS
{
    int x[N];
    int y[N];
};

int top(SS *a, int b[4][6], SS &c) {...}

syn.directive.interface mode=ap_memory top b[0]
syn.directive.interface mode=ap_memory top b[1]
syn.directive.interface mode=ap_memory top b[2]
syn.directive.interface mode=ap_memory top b[3]
```

另请参阅

- [syn.directive.array_reshape](#)

syn.directive.array_reshape

描述



重要提示！ 针对顶层函数上的 M_AXI 接口，不支持 `syn.directive.array_partition` 和 `syn.directive.array_reshape`。您可改用 `hls::vector` 数据类型，如《Vitis 高层次综合用户指南》(UG1399) 的“向量数据类型”所述。

`syn.directive.array_reshape` 将阵列分区与垂直阵列映射相结合以创建所含元素更少但字宽更宽的新阵列。

`syn.directive.array_reshape` 命令提供下列功能特性：

- 将阵列拆分为多个阵列（类似 `syn.directive.array_partition`）。
- 自动按垂直方式重新组合阵列，创建字宽更宽的新阵列。

语法

```
syn.directive.array_reshape=[OPTIONS] <location> <array>
```

- `<location>` 是包含阵列变量的位置（格式为 `function[/label]`）。
- `<array>` 是要重塑的阵列变量。

选项

- `dim=<integer>`：指定要执行分区的阵列的维度。
 - 该维度仅与多维阵列相关。
 - 如果使用的值为 0，那么会使用指定选项对所有维度进行分区。

- 设置任何其他值都仅对该值对应的维度进行分区。例如，如果使用的值为 1，则仅对第 1 个维度进行分区。
- `type=(block|cyclic|complete):`
 - `block` 型重塑会从原始阵列的连续块创建更小阵列。这样即可将阵列有效等分为 N 个块，其中 N 为 `-factor` 选项定义的整数，然后使用 `word-width*N` 将 N 个块整合到单一阵列中。默认值为 `complete`。
 - `cyclic` 型重塑会通过交织来自原始阵列的元素来创建更小的阵列。例如，如果使用 `-factor 3`，则将向第 1 个新阵列分配元素 0，向第 2 个新阵列分配元素 1，向第 3 个新阵列分配元素 2，然后再次向第 1 个新阵列分配元素 3。最后一个阵列是将新阵列垂直串联（字串联以创建更长的字）成单个阵列。
 - `complete` 型重塑可将阵列分解为临时独立元素，然后将其重新组合为含字宽更宽的单个阵列。对于一维阵列，这相当于创建超宽寄存器。例如，如果原始阵列有 N 个元素，每个元素为 M 位，那么即可得到一个 N*M 位的寄存器。该选项为默认选项。
- `factor=<integer>`: 该选项仅用于 `block` 或 `cyclic` 分区，用于指定要创建的更小的阵列数量。
- `object`:

注释： 仅与容器阵列相关。

对容器内的对象应用重塑。如果指定该选项，则会对各对象的所有维度进行重塑。但容器的所有维度均保持不变。

- `off=true`: 为指定变量禁用 `ARRAY_RESHAPE` 功能特性。

示例 1

将函数 `func` 中的 8 位阵列 `AB[17]` 重塑为含 5 个元素的新的 32 位阵列。

由于 4 不是 17 的整数因子，因此：

- 阵列 `AB[17]` 的索引 17 位于重塑后第五个元素的低八位中。
- 第五个元素的高八位则不使用。

```
syn.directive.array_reshape=type=block factor=4 func AB
```

将函数 `func` 中的阵列 `AB[6][4]` 分区到维度 `[6][2]` 的新阵列中，其中维度 2 为 2 倍宽度。

```
syn.directive.array_reshape=type=block factor=2 dim=2 func AB
```

将函数 `func` 中的 8 位阵列 `AB[4][2][2]` 重塑到新的单元素阵列（寄存器）中，其位宽为 $4*2*2*8 (= 128)$ 。

```
syn.directive.array_reshape=type=complete dim=0 func AB
```

示例 2

您可通过新的阵列结构来对自己的代码中重塑后的阵列进行寻址。使用以下指令时，请参阅以下代码。

```
syn.directive.array_reshape=top b type=complete dim=0
```


代码结构如下：

```
struct SS
{
    int x[N];
    int y[N];
};

int top(SS *a, int b[4][6], SS &c) {...}
```

阵列接口定义如下：

```
syn.directive.interface=mode=ap_memory top b[0]
```

另请参阅

- [syn.directive.array_partition](#)
- [syn.directive.interface](#)

syn.directive.bind_op

描述



提示：此编译指示或指令在 DSP 多重运算匹配 (MULADD、AMA、ADDMUL 和相应的减法运算) 中不起作用。事实上，使用 BIND_OP 为 DSP 分配运算符会妨碍 HLS 编译器匹配多重运算表达式。

Vitis HLS 使用特定硬件资源或实现来实现代码中的不同运算。该工具会自动判定要使用的资源。您还可定义 `syn.op` 来指定实现（请参阅 [运算符配置](#)）。

`syn.directive.bind_op` 命令用于指定变量，并指定某项运算将该变量映射到 RTL 中的特定实现。例如，`mul`、`add` 或 `sub` 可映射到 RTL 中的 `impl`。例如，您可使用 `syn.directive.bind_op` 命令来指明在器件互连结构内（而不是在 DSP 内）实现一项特定乘法器运算 (`mul`)。

您也可以通过 `syn.directive.bind_op` 使用 `latency` 选项来指定时延，如下所述。



重要提示！要使用 `latency` 选项，运算必须具有 1 个可用的多阶段实现。HLS 工具可为所有基本算术运算（加减乘除）和所有浮点运算提供 1 个多阶段实现。

语法

```
syn.directive.bind_op=[OPTIONS] <location> <variable>
```

- `<location>` 是包含变量的位置（格式为 `function[/label]`）。
- `<variable>` 是要分配的变量。在此例中，定义的变量就是该指令的目标运算结果所分配到的变量。

选项

- `op=<value>`：用于定义要绑定到特定实现资源的运算。受支持的函数运算包括：`mul`、`add` 和 `sub`

受支持的浮点运算包括：`fadd`、`fsub`、`fdiv`、`fexp`、`flog`、`fmul`、`frsqrt`、`frecip`、`fsqrt`、`dadd`、`dsub`、`ddiv`、`dexp`、`dlog`、`dmul`、`drsqrt`、`drecip`、`dsqrt`、`hadd`、`hsub`、`hdiv`、`hmul` 和 `hsqrt`

提示：浮点运算包括单精度 (f)、双精度 (d) 和半精度 (h)。

- `impl=<value>`：定义用于指定运算的实现。受支持的函数运算实现包括 `fabric` 和 `dsp`。受支持的浮点运算实现包括：`fabric`、`meddsp`、`fulldsp`、`maxdsp` 和 `primitivedsp`。

注释：`primitivedsp` 仅在 Versal 器件上可用。

- `latency=<int>`：定义运算的实现的默认时延。有效的时延值因指定的 `op` 和 `impl` 而异。默认值为 -1，即交由 Vitis HLS 选择时延。下表反映了受支持的运算、实现和时延组合。

表 31：受支持的函数运算、实现和时延组合

运算	实现	最小时延	最大时延
add	fabric	0	4
add	dsp	0	4
mul	fabric	0	4
mul	dsp	0	4
sub	fabric	0	4
sub	dsp	0	0

提示：比较运算符（如 `dcmp`）是在 LUT 中实现的，不得在互连结构外部实现，也不能映射到 DSP，因此无法通过 `config_op` 命令或 `bind_op` 命令来配置。

表 32：受支持的浮点运算、实现和时延组合

运算	实现	最小时延	最大时延
fadd	fabric	0	13
fadd	fulldsp	0	12
fadd	primitivedsp	0	3
fsub	fabric	0	13
fsub	fulldsp	0	12
fsub	primitivedsp	0	3
fdiv	fabric	0	29
fexp	fabric	0	24
fexp	meddsp	0	21
fexp	fulldsp	0	30
flog	fabric	0	24
flog	meddsp	0	23
flog	fulldsp	0	29
fmul	fabric	0	9
fmul	meddsp	0	9
fmul	fulldsp	0	9
fmul	maxdsp	0	7
fmul	primitivedsp	0	4
fsqrt	fabric	0	29

表 32：受支持的浮点运算、实现和时延组合 (续)

运算	实现	最小时延	最大时延
frsqr	fabric	0	38
frsqr	fulldsp	0	33
frecip	fabric	0	37
frecip	fulldsp	0	30
dadd	fabric	0	13
dadd	fulldsp	0	15
dsub	fabric	0	13
dsub	fulldsp	0	15
ddiv	fabric	0	58
dexp	fabric	0	40
dexp	meddsp	0	45
dexp	fulldsp	0	57
dlog	fabric	0	38
dlog	meddsp	0	49
dlog	fulldsp	0	65
dmul	fabric	0	10
dmul	meddsp	0	13
dmul	fulldsp	0	13
dmul	maxdsp	0	14
dsqr	fabric	0	58
drsqr	fulldsp	0	111
drecip	fulldsp	0	36
hadd	fabric	0	9
hadd	meddsp	0	12
hadd	fulldsp	0	12
hsub	fabric	0	9
hsub	meddsp	0	12
hsub	fulldsp	0	12
hdiv	fabric	0	16
hmul	fabric	0	7
hmul	fulldsp	0	7
hmul	maxdsp	0	9
hsqr	fabric	0	16

示例

在以下示例中，指定了使用互连结构逻辑的二阶流水节拍乘法器来为函数 `foo` 的变量 `c` 实现乘法运算。

```
int foo (int a, int b) {
  int c, d;
  c = a*b;
  d = a*c;
  return d;
}
```

`syn.directive.bind_op` 命令如下所示：

```
syn.directive.bind_op=op=mul impl=fabric latency=2 foo c
```



提示：由于未给 `d` 变量指定任何 `syn.directive.bind_op`，因此 HLS 工具会选择用于该变量的实现。

另请参阅

- [syn.directive.bind_storage](#)

syn.directive.bind_storage

描述

Vitis HLS 在代码中使用特定存储器资源（或类型）来分配阵列。该工具会自动判定要使用的资源。或者，您也可以定义 `syn.storage` 来笼统指定存储器类型，如 [存储配置](#) 所述。

`syn.directive.bind_storage` 命令用于将代码中的特定变量（阵列或函数实参）分配到 RTL 中的特定存储器类型 (`type`)。例如，您可使用 `syn.directive.bind_storage` 命令指定用于阵列变量的存储器的类型以及要使用的实现。您可控制此阵列是作为单端口 RAM 还是双端口 RAM 来实现。



重要提示！该功能特性对于顶层函数接口上的阵列非常重要。与阵列关联的存储器类型用于判定 RTL 中所需端口的数量和类型。请参阅《Vitis 高层次综合用户指南》(UG1399) 的“接口上的阵列”章节。对于分配给顶层函数实参的变量，您必须使用 `syn.directive.interface` 的 `storage_type` 选项和 `storage_impl` 选项来分配存储器类型和实现。

您也可使用 `syn.directive.bind_storage` 的 `latency` 选项来指定实现的时延。对于接口上的块 RAM，`latency` 选项允许您对接口上的片外非标准 SRAM 进行建模。例如，针对时延为 2 或 3 的 SRAM 的建模支持。对于内部运算，`latency` 选项允许使用多个流水节拍阶段来实现运算。这些附加的流水线阶段可帮助解决 RTL 综合期间的时序问题。



重要提示！要使用 `latency` 选项，存储器必须具有 1 个可用的多阶段实现。HLS 工具可以为所有块 RAM 提供多阶段实现。

语法

```
syn.directive.bind_storage=[OPTIONS] <location> <variable>
```

- `<location>` 是包含变量的位置（格式为 `function[/label]`）。
- `<variable>` 是要分配的变量。

 **提示：**如果变量是顶层函数实参，则使用 `syn.directive.interface` 的 `storage_type` 选项和 `storage_impl` 选项。

选项

- `type=<value>`：定义要绑定到指定变量的存储器的类型。受支持的类型包括：`fifo`、`ram_1p`、`ram_1wnr`、`ram_2p`、`ram_s2p`、`ram_t2p`、`rom_1p`、`rom_2p` 和 `rom_np`。

表 33：存储类型

类型	描述
FIFO	FIFO。Vitis HLS 可判定如何在 RTL 中将其实现，除非指定 <code>-impl</code> 选项。
RAM_1P	单端口 RAM。Vitis HLS 可判定如何在 RTL 中将其实现，除非指定 <code>-impl</code> 选项。
RAM_1WNR	含 1 个写入端口和 N 个读取端口的 RAM，内部使用 N 个 bank。
RAM_2P	双端口 RAM，允许在某一端口上执行读操作并在另一个端口上执行读写操作。
RAM_S2P	双端口 RAM，允许在某一端口上执行读取操作，并在另一个端口上执行写入操作。
RAM_T2P	真正的双端口 RAM，支持在 2 个端口上执行读写操作。
ROM_1P	单端口 ROM。Vitis HLS 可判定如何在 RTL 中将其实现，除非指定 <code>-impl</code> 选项。
ROM_2P	双端口 ROM。
ROM_NP	多端口 ROM。

 **提示：**如果您为 PIPO 指定了单端口 RAM，那么绑定程序通常会分配一个合并的 PIPO，只有一个 bank，读取和写入分别访问 bank 的不同部分。HLS 编译器报告的参考消息会显示 “Implementing PIPO using a single memory for all blocks.”，即，“为所有块使用同一个存储器来实现 PIPO”。对于小型 PIPO 来说，这种方法通常比较经济，因为所有 bank 可以共享单个 RAM 块。但是，如果您为 PIPO 指定了双端口 RAM 以获取更高带宽，而调度器在生产者或使用者中使用了这两个端口，那么绑定程序通常会分配一个拆分 PIPO，即生产者使用 1 个端口，使用者使用不同 bank 的 2 个端口。在此情况下，HLS 编译器报告的参考消息为 “Implementing PIPO using a separate memory for each block.”，即，“为每个块使用一个独立存储器来实现 PIPO”。

- `impl=<value>`：定义指定存储器类型的实现。受支持的实现包括：`bram`、`bram-ecc`、`lutram`、`uram`、`uram-ecc`、`srl`、`memory` 和 `auto`，如下所述。

表 34：受支持的实现

名称	描述
MEMORY	通用 FIFO 存储器，允许 Vivado 工具选择实现。
URAM	UltraRAM 资源
URAM_ECC	含 ECC 的 UltraRAM
SRL	移位寄存器逻辑资源
LUTRAM	分布式 RAM 资源
BRAM	块 RAM 资源
BRAM_ECC	含 ECC 的块 RAM
AUTO	Vitis HLS 会自动判定变量的实现。

表 35: FIFO/RAM/ROM 支持的实现

类型	命令/编译指示	范围	受支持的实现
FIFO	<code>bind_storage</code> ¹	局部	AUTO 、BRAM、BRAM_ECC、LUTRAM、URAM、URAM_ECC、MEMORY、SRL
FIFO	<code>config_storage</code>	全局	AUTO 、BRAM、BRAM_ECC、LUTRAM、URAM、URAM_ECC、MEMORY、SRL
RAM* ROM*	<code>bind_storage</code>	局部	AUTO BRAM、BRAM_ECC、LUTRAM、URAM 和 URAM_ECC
RAM* ROM*	<code>config_storage</code> ²	全局	不适用
RAM_1P	<code>set_directive_interface_s_axilite_storage_impl</code>	局部	AUTO 、BRAM 和 URAM
	<code>config_interface-m_axi_buffer_impl</code>	全局	AUTO 、 BRAM 、LUTRAM 和 URAM

注释：

- 1. 如无任何实现，该指令会使用 AUTOSRL 行为作为默认实现。但您无法指定该值。
- 2. `config_storage` 仅支持 FIFO 类型。

· `latency=<int>`：定义用于将存储类型绑定到实现的默认时延。有效的时延值因指定的 `type` 和 `impl` 而异。默认值为 -1，即交由 Vitis HLS 选择时延。

表 36: 受支持的存储器类型、实现和时延组合

类型	实现	最小时延	最大时延
FIFO	BRAM	1	5
FIFO	LUTRAM	1	5
FIFO	MEMORY	1	5
FIFO	SRL	1	5
FIFO	URAM	1	5
RAM_1P	AUTO	1	5
RAM_1P	BRAM	1	5
RAM_1P	LUTRAM	1	5
RAM_1P	URAM	1	5
RAM_1WNR	AUTO	1	5
RAM_1WNR	BRAM	1	5
RAM_1WNR	LUTRAM	1	5
RAM_1WNR	URAM	1	5
RAM_2P	AUTO	1	5
RAM_2P	BRAM	1	5
RAM_2P	LUTRAM	1	5
RAM_2P	URAM	1	5
RAM_S2P	BRAM	1	5

表 36：受支持的存储器类型、实现和时延组合 (续)

类型	实现	最小时延	最大时延
RAM_S2P	BRAM_ECC	1	5
RAM_S2P	LUTRAM	1	5
RAM_S2P	URAM	1	5
RAM_S2P	URAM_ECC	1	5
RAM_T2P	BRAM	1	5
RAM_T2P	URAM	1	5
ROM_1P	AUTO	1	5
ROM_1P	BRAM	1	5
ROM_1P	LUTRAM	1	5
ROM_2P	AUTO	1	5
ROM_2P	BRAM	1	5
ROM_2P	LUTRAM	1	5
ROM_NP	BRAM	1	5
ROM_NP	LUTRAM	1	5

注释：除 ROM 外的所有其他类型都支持 BRAM_ECC 和 URAM_ECC。

注释：FIFO 的最小和最大时延为 0。应从 bind_storage 编译指示部分复制有关 config_storage -read_after_write_latency 的提示。



重要提示！ `syn.directive.bind_storage` 仅支持上表中的存储器类型与实现的组合。

示例

在以下示例中，`coeffs[128]` 变量是 `func1` 函数的实参。该指令指定 `coeffs` 使用来自库的 BRAM 核上实现的单端口 RAM。

```
syn.directive.bind_storage=func1 coeffs type=RAM_1P impl=bram
```



提示：在 RAM_1P 核中定义 RTL 中创建的用于访问 `coeffs` 值的端口。

另请参阅

· [syn.directive.bind_op](#)

syn.directive.cache

描述

在 FPGA 设计中，对片外 DDR 存储器进行非突发访问的成本很高，而且会使性能劣化。为了缓解该问题，Vitis HLS 在 M_AXI 适配器中支持一种高速缓存机制，通过利用引用的局部性（时间性和空间性）来减少平均访问时延。访问某个存储器区域时，很可能很快会再次访问该区域，并且还可能访问附近的位置。

这种高速缓存机制既支持单端口高速缓存，也支持多端口层级：

- 单端口高速缓存：直接映射、只读高速缓存，连接到单个 m_axi 端口。这样在内核重复访问同一行时可以减少对 DRAM 的重复读取。
- 多端口高速缓存：一种分层高速缓存，每个端口都有 L1 高速缓存，并共享 L2 高速缓存。它支持在单个时钟周期内对同一个顶层 m_axi 指针进行最多 N 次读取访问，而无需更改源代码，适用于类似 stencil 的访问模式以及编译器无法推断窗口缓冲器的其他动态访问模式。相比于编译器推断的窗口缓冲器，多端口高速缓存更灵活（处理动态模式），但通常会耗用更多的资源。

多端口高速缓存架构

- L2 高速缓存（共享）：
 - 所有端口共享一个直接映射的高速缓存。
 - 通过突发传输从片外 DRAM 加载。
 - 将各行广播到发出请求的 L1 高速缓存，通过避免冗余 DRAM 加载来优化存储器带宽。
- L1 高速缓存（各端口）：
 - 每个高速缓存端口一个直接映射的 L1 高速缓存。
 - 每个 L1 均可在每个周期内提供一个数据项，允许每个周期对同一个 m_axi 指针进行最多 N 次读取访问。
 - 从共享的 L2 高速缓存加载 L1 行。



提示：您可使用 `syn.interface.m_axi_cache_impl` 来控制高速缓存实现资源。

语法

```
syn.directive.cache=<location> port=<name> lines=<value> depth=<value>
[ports=<N>] [l2_lines=<M>]
```

其中：

- `<location>`：指定函数，可在该函数所在位置找到指定端口。这是顶层函数。
- `port=<name>`：指定高速缓存要添加到的只读端口（顶层 m_axi 指针）。
- `lines=<value>`：每个 L1 高速缓存中的高速缓存行数（针对单端口的情况：唯一的一个高速缓存）。针对单行指定 1，针对多行指定大于 1 的 2 的幂值。可选；如不指定，则默认值为 1。
- `depth=<value>`：每个高速缓存行的大小，以字数为单位，必须为 2 的幂值。适用于指针的元素类型（例如，深度以 int 和 float 等为单位）。可选；默认值为最大突发长度。最大突发长度默认为 16，但可通过 `syn.interface.m_axi_max_read_burst_length` 来全局指定，或通过 `syn.directive.interface` 来按接口指定。

注释：在多端口高速缓存中，L1 和 L2 缓存共享相同的行深度；深度同时适用于两个级别。

- `ports=<N>`：启用多端口高速缓存功能特性。定义在一个时钟周期内可以针对对应顶层 m_axi 指针调度执行的读取访问次数。仅适用于映射到其 m_axi 捆绑的只读指针；当端口数量大于 1 时，只能将一个只读阵列映射到该捆绑（请参阅“限制”）。
- `l2_lines=<M>`：可选；默认情况下，L1 和 L2 高速缓存的行数是通过行选项来选择的。该选项可以独立设置共享 L2 高速缓存内的行数，其值必须大于 L1 高速缓存行数。每个 L2 行的大小与 L1 行大小相等，并受深度控制。

行为与性能注释

- 多端口高速缓存是专为类似 stencil 的访问模式以及无法推断窗口缓冲器的动态访问模式而设计的。它可以在每个周期内允许多个读取请求，同时减少通过共享 L2 高速缓存的冗余 DRAM 突发，从而提高吞吐量。
- L1 与 L2 高速缓存均为直接映射。命中/未命中行为由局部性决定；增大行数并选择合适的深度可以提高命中率，但会增加资源消耗。
- 共享 L2 高速缓存每次未命中时会加载一个 DRAM 行，并将该行广播到需要它的任意 L1 高速缓存，从而优化与窗口缓冲器类似的带宽，但支持动态模式。

限制

- 仅限只读端口才支持高速缓存。写端口可以与缓存的只读指针共享一个捆绑，因为它们使用的是独立通道，但不支持 inout 输入输出端口。
- 在多端口模式（端口数大于 1）下：
 - 缓存的顶层 m_axi 指针必须是映射到其捆绑的唯一读端口。
 - 写端口也可以映射到同一个捆绑，但 inout 端口对此不予支持。
- 高速缓存在 L1 和 L2 级处均为直接映射（单路）。
- 指定的 l2_lines 值必须大于行数。
- 只能将一个只读阵列映射到多端口缓存指针的 m_axi 捆绑。

其他控制

- syn.interface.m_axi_cache_impl 可用于控制高速缓存实现资源。
- syn.interface.m_axi_max_read_burst_length 和 syn.directive.interface 可通过最大突发长度来控制默认深度。

示例

以下示例显示的设计中，重叠访问会导致突发失败。使用 CACHE 编译指示或指令会改善设计性能。

```
extern "C" {
    void dut(
        const double *in,    // Read-Only Vector 1
        double *out,         // Output Result
        int size             // Size in integer
    )
    #pragma HLS INTERFACE m_axi port=in bundle=aximm depth = 1026
    #pragma HLS INTERFACE m_axi port=out bundle=aximm depth = 1024
    #pragma HLS cache port=in lines=8 depth=128
    for(int i = 0; i < size; i++)
    {
        out[i] = in[i] + in[i + 1];
    }
}
```

另请参阅

- [pragma HLS interface](#)
- [接口配置](#)
- [pragma HLS cache](#)

syn.directive.dataflow

描述

该指令会按 C/C++ 描述中的顺序来执行所有运算。如无限制资源的任何指令（如 `set_directive_allocation`），则 Vitis HLS 会尝试最大限度减小时延并提高并发。数据依赖关系可对此加以限制。例如，访问阵列的函数或循环必须先完成对阵列的所有读写访问后才能完成操作。这样会阻止下一个使用该数据的函数或循环开始操作。

但函数或循环中的操作可在前一个函数或循环尚未完成其所有操作时就开始操作。`syn.directive.dataflow` 用于指定要对函数或循环执行的数据流优化，用于提高 RTL 实现的并发。当指定 `syn.directive.dataflow` 时，HLS 工具会分析顺序函数或循环之间的数据流并创建通道。这些通道是根据乒乓 RAM 或 FIFO 创建的，这些乒乓 RAM 或 FIFO 允许使用者函数或循环在生产者函数或循环完成前即开始操作。函数或循环可以并行操作，从而降低时延，并提升 RTL 吞吐量。



提示： `syn.dataflow.xxx` 命令会指定 `syn.directive.dataflow` 所使用的默认存储器通道和 FIFO 深度，如 [数据流配置](#) 所述。

要使 DATAFLOW 优化正常工作，数据必须逐一流经设计中的每个任务。请务必留意下列情况。如需了解更多详细信息，请参阅 [控制驱动的任务级并行度的限制](#) 和 [数据流区域编码风格](#)。

- 单一生产者使用者违例
- 任务间的反馈
- 任务的有条件执行
- 含多个退出条件的循环



重要提示！ 如果存在上述任一编码样式，则 HLS 工具会发出一条消息，且不会执行 DATAFLOW 优化。

最后，DATAFLOW 优化默认不加以分层。如果子函数或循环包含可能受益于 DATAFLOW 优化的其他任务，那么请对该循环或子函数应用此优化。您还可以内联子函数。

语法

```
syn.directive.dataflow=<location> disable_start_propagation
```

- `<location>` 是数据流优化的执行位置（格式为 `function[/label]`）。
- `disable_start_propagation`：禁用起始 FIFO 的创建，起始 FIFO 用于向内部进程传播起始令牌。此类 FIFO 有时可能会成为性能瓶颈。

示例

指定 DATAFLOW 优化在函数 `foo` 内执行。

```
syn.directive.dataflow=foo
```

另请参阅

- [syn.directive.allocation](#)

syn.directive.dependence

描述

Vitis HLS 会检测循环内的依赖关系。任一循环的同一迭代内的依赖关系均为独立于循环的依赖。任一循环的不同迭代之间的依赖关系均为循环携带依赖。

调度运算时（尤其是在函数和循环流水打拍期间），会对这些依赖关系产生影响。

- 独立于循环的依赖关系：访问单一循环迭代内的相同元素。

```
for (i=1;i<N;i++) {
  A[i]=x;
  y=A[i];
}
```

- 循环携带依赖：访问不同循环迭代的相同元素。

```
for (i=1;i<N;i++) {
  A[i]=A[i-1]*2;
}
```

在某些情况下，依赖关系分析可能过于保守，无法筛选掉假性依赖关系。例如，在进行变量依赖的阵列索引期间。又例如，在需要强制执行外部要求时（如，两个输入不共享相同的索引）。`syn.directive.dependence` 命令允许您显式定义依赖关系并消除假性依赖关系。



重要提示！ 如果依赖关系并非假性 (false) 关系，那么指定假性依赖关系可能导致硬件错误。指定依赖关系前，请确认它是否正确 (true 或 false)。

语法

```
syn.directive.dependence=[OPTIONS] <location>
```

- `<location>`：表示代码中定义依赖关系的位置，指定方式为 `function[/label]`。

选项

- `class=(array | pointer)`：指定需要澄清其中所含依赖关系的变量的类。该选项与 `variable` 选项互斥。
- `variable=<variable>`：定义特定变量，以供应用依赖关系指令。该选项与 `class` 选项互斥。



重要提示！ 对于与 `m_axi` 接口内其他实参捆绑的函数实参，不能指定 `dependence`。这是函数上的 `m_axi` 接口的默认配置。并且除非结构体已解聚，否则也不能为结构体的元素指定依赖关系。

- `dependent=(true | false)`：指定该实参可用于指示依赖关系如为 `true` 则需要强制实施，如为 `false` 则应移除。但如果不指定该实参，则工具将返回警告，称未指定该值，并且假定值为 `false`。
- `direction=(RAW | WAR | WAW)`：

注释： 仅与循环携带依赖有关。

指定依赖关系的方向：

- `RAW`（先写后读 - 真性依赖关系）：写指令使用的值供读指令使用。
- `WAR`（先读后写 - 反依赖关系）：读指令获取的值被写指令覆盖。

- WAW（先写后写 - 输出依赖关系）：2 条写指令按顺序写入相同位置。
- `distance=<integer>`:

注释：仅与循环携带依赖有关，其中 `-dependent` 设置为 `true`。

指定阵列访问的迭代间距离。

- `type=(intra | inter)`：指定依赖关系：
 - 位于相同循环迭代内 (`intra`)，或者
 - 位于不同循环迭代之间 (`inter`)（默认值）。

示例

移除 `func` 函数的 `loop_1` 的相同迭代中的 `Var1` 之间的依赖关系。

```
syn.directive.dependence=variable=Var1 type=intra \
dependent=false func/loop_1
```

`func` 函数的 `loop_2` 中的所有阵列的依赖关系将告知 Vitis HLS，在相同循环迭代内，所有读操作都必须发生在写操作之后。

```
syn.directive.dependence=class=array type=intra \
dependent=true direction=RAW func/loop_2
```

另请参阅

- [syn.directive.disaggregate](#)
- [syn.directive.pipeline](#)

syn.directive.disaggregate

描述

`syn.directive.disaggregate` 命令允许您按 `struct` 变量所含各独立元素来对其进行解构。`struct` 本身的内容用于判定元素的数量和类型。



重要提示！默认情况下，作为顶层函数实参的结构体将聚合在一起，但可通过该指令或编译指示来对其进行解聚。

语法

```
syn.directive.disaggregate=<location> <variable>
```

- `<location>` 对应可在其中找到要解聚的变量的位置（格式为 `function[/label]`）。
- `<variable>` 则用于指定结构体变量名称。

选项

此命令不含任何选项。

示例 1

以下示例显示的是如何对 `top` 函数中的结构体变量 `a` 进行解聚：

```
syn.directive.disaggregate=top a
```

示例 2

您可以使用标准的 C/C++ 语言编码风格对解聚的结构体进行寻址。使用这些指令时的代码如下所示。

```
syn.directive.disaggregate=top a
syn.directive.disaggregate=top c
```

代码结构如下。

```
struct SS
{
    int x[N];
    int y[N];
};

int top(SS *a, int b[4][6], SS &c) {

    syn.directive.interface=mode=s_axilite top a->x
    syn.directive.interface=mode=s_axilite top a->y

    syn.directive.interface=mode=ap_memory top c.x
    syn.directive.interface=mode=ap_memory top c.y
```



重要提示！ 请注意，访问指针元素 (a) 与访问参考元素 (c) 的方法存在差异，如上所示

示例 3

以下示例显示的 `Dot` 结构体内包含 `RGB` 结构体作为元素。如果您将 `syn.directive.disaggregate` 应用于 `Arr` 变量，则仅对顶层 `Dot` 结构体进行解聚。

```
struct Pixel {
    char R;
    char G;
    char B;
};

struct Dot {
    Pixel RGB;
    unsigned Size;
};

#define N 1086
void DUT(Dot Arr[N]) {
    ...
}

syn.directive.disaggregate=DUT Arr
```

如果您要将整个结构体（包括 Dot 和 RGB）进行解聚，则可按如下所示对 `syn.directive.disaggregate` 进行赋值。

```
void DUT(Dot Arr[N]) {
#pragma HLS disaggregate variable=Arr->RGB
...
}

syn.directive.disaggregate=DUT Arr->RGB
```

以下代码展示了操作结果。

```
void DUT(char Arr_RGB_R[N], char Arr_RGB_G[N], char Arr_RGB_B[N], unsigned
Arr_Size[N]) {
...
}
```

另请参阅

- [syn.directive.aggregate](#)

syn.directive.expression_balance

描述

有时，C/C++ 代码含一连串运算，故而会在 RTL 中生成一长串运算链。如果时钟周期较短，这就可能增加设计中的时延。默认情况下，Vitis HLS 工具会使用结合律和交换律来重新组织运算。通过重新组织可以创建平衡的树结构以缩短运算链，从而以增加硬件为代价来缩短设计时延。

表达式平衡会重新排列运算符以构造平衡的树结构并降低时延。

- 对于整数运算，默认情况下表达式平衡处于开启状态，但可将其禁用。
- 对于浮点运算，默认情况下表达式平衡处于关闭状态，但可将其启用。

`syn.directive.expression_balance` 命令支持在指定作用域内开关这种表达式平衡方法。

语法

```
syn.directive.expression_balance=[OPTIONS] <location>
```

- `<location>` 表示位置，格式为 `function[/label]`。它用于指定禁用或启用表达式平衡的具体位置。

选项

- `off`：在指定位置关闭表达式平衡。指定 `syn.directive.expression_balance` 命令即可在指定作用域内启用表达式平衡。添加 `off` 选项即可将其禁用。

示例

在 `My_Func` 函数内禁用表达式平衡。

```
syn.directive.expression_balance=off My_Func
```

在 `My_Func2` 函数内显式启用表达式平衡。

```
set_directive_expression_balance=My_Func2
```

syn.directive.function_instantiate

描述

默认情况下：

- 函数在 RTL 中作为独立层级块保留，或者分解（或内联）到更高层次的函数中。
- 同一层级上的任一函数的所有实例都使用相同 RTL 实现（块）。

`syn.directive.function_instantiate` 命令用于为函数的每个实例创建唯一的 RTL 实现。每个实例均可围绕特定实参或变量来进行优化。

默认情况下，以下代码可为全部 3 个实例的 `func_sub` 函数生成单一 RTL 实现，或者如果 `func_sub` 是小函数，则将其内联到 `func` 函数中。



提示：默认情况下，Vitis HLS 工具会自动内联小函数。即使对于函数例化也同样如此。请使用 `set_directive_inline off` 选项来阻止此自动内联操作。

```
char func_sub(char inval, char incr)
{
    return inval + incr;
}
void func(char inval1, char inval2, char inval3,
          char *outval1, char *outval2, char * outval3)
{
    *outval1 = func_sub(inval1, 1);
    *outval2 = func_sub(inval2, 2);
    *outval3 = func_sub(inval3, 3);
}
```

按如下示例部分所示使用指令可生成 3 个版本的 `func_sub` 函数，每个版本都独立针对 `incr` 变量进行优化。

语法

```
syn.directive.function_instantiate=<location> <variable>
```

- `<location>` 即生成函数的唯一实例的位置（格式为 `function[/region label]`）。
- `<variable>` 用于指定函数实参作为各种函数例化中的常量。

选项

此命令不含任何选项。

示例

对于以上所示示例代码，以下 Tcl（或置于 `func_sub` 函数中的编译指示）支持对 `func_sub` 函数的每个实例进行与 `incr` 输入相关的独立优化。

```
syn.directive.inline off=func_sub
syn.directive.function_instantiate=func_sub incr
```

另请参阅

- [syn.directive.allocation](#)
- [syn.directive.inline](#)

syn.directive.inline

描述

`syn.directive.inline` 移除 RTL 层级中作为独立实体的函数。完成内联后，函数会消隐到调用函数内，不再显示为层级的独立层次。



重要提示！ 内联子函数也会将应用于该函数的所有编译指示或指令消隐掉。在 Vitis HLS 中，忽略子关联环境中应用的任意指令。

在某些情况下，将函数内联即可使函数内的运算以更有效的方式与调用函数进行共享和优化。但内联函数无法共享或复用，因此如果父函数多次调用内联函数，则可能导致增加面积和资源使用率。

默认情况下，仅在函数层级的下一个层次上执行内联。

语法

```
syn.directive.inline=[OPTIONS] <location>
```

- `<location>` 是要执行内联的位置（格式为 `function[/label]`）。

选项

- `off`：默认情况下，Vitis HLS 会在代码中为较小的函数执行内联。通过使用 `off` 选项则可为指定函数禁用内联。
- `recursive`：默认情况下，仅执行一级函数内联。所指定的函数内部包含的函数则不进行内联。`recursive` 选项会在指定函数层级内按递归方式内联所有函数。

示例

以下示例可内联 `func_sub1` 函数，但 `func_sub1` 不调用任何子函数。

```
syn.directive.inline=func_sub1
```

此示例会按递归方式向下内联 `func_sub1` 函数（`func_sub2` 函数除外）：

```
syn.directive.inline=recursive func_sub1
syn.directive.inline=off func_sub2
```


另请参阅

- [syn.directive.allocation](#)

syn.directive.interface

描述

仅限顶层函数才支持 `syn.directive.interface` 指令。对于 HLS 组件的子函数，您无法使用该指令。INTERFACE 编译指示或指令会指定在接口综合期间函数实参创建 RTL 端口的方式。请参阅《Vitis 高层次综合用户指南》(UG1399) 的“定义接口”章节。Vitis HLS 工具会自动确定任何子函数使用的 I/O 协议。

RTL 实现中的端口来自以下要素。

- 顶层函数与函数返回的实参的数据类型和方向。
- HLS 组件的 `flow_target`。
- 由 `syn.interface.xxx` 指定的默认接口配置设置（请参阅 [接口配置](#)）。
- `syn.directive.interface`。

每个函数实参均可包含其自己的 I/O 协议（例如，有效握手或确认握手）。



提示：接口上必需的全局变量必须显式定义为顶层函数的实参，如《Vitis 高层次综合用户指南》(UG1399) 的“全局变量”章节所述。然而，如果要访问全局变量，但所有读写操作均为设计的本地操作，则会在该设计中创建此资源。RTL 中无需 I/O 端口。

该接口还定义了 HLS 组件的执行控制协议。欲知详情，请参阅《Vitis 高层次综合用户指南》(UG1399) 的“块级控制协议”章节。控制协议用于控制 HLS 组件（或块）何时开始执行，以及该块在完成操作后何时处于空闲状态并准备就绪以接收新输入。

语法

```
syn.directive.interface=[OPTIONS] <location> <port>
```

- `<location>` 对应位置，在此位置中，可指定函数接口或寄存的输出（格式为 `function[/label]`）。
- `<port>` 是用于对接口进行综合的参数（函数实参）。下列块控制模式无需端口名称：`ap_ctrl_chain`、`ap_ctrl_hs` 或 `ap_ctrl_none`。

选项



提示：以下许多选项都采用 `syn.interface.xxx` 命令定义的默认值。您可为此处定义的接口定义局部值以覆盖默认值。

- `mode=<mode>:`

以下三个类别演示了这些模式以及工具如何在 RTL 中实现这些模式。

1. 端口级协议：

- `ap_none`：无端口协议。此接口是简单的数据端口。
- `ap_stable`：无协议。此接口是简单的数据端口。Vitis HLS 工具假定复位后数据端口始终保持稳定。允许通过优化来移除不必要的寄存器。
- `ap_vld`：用于实现含关联 `valid` 信号的数据端口，以指示何时数据有效且可供读取或写入。

- `ap_ack`：用于实现含关联 `acknowledge` 信号的数据端口，以确认数据写入或读取。
- `ap_hs`：用于实现同时包含 `valid` 信号和 `acknowledge` 信号的数据端口，提供两路握手以指示数据何时有效且可供读取和写入。确认数据已被读取或写入
- `ap_ovld`：用于实现含关联 `valid` 信号的输出数据端口，以指示何时数据有效且可供读取或写入。



提示：对于 `ap_ovld`，Vitis HLS 通过 `ap_none` 模式来实现输入实参或任意读取/写入实参的输入部分。

- `ap_memory`：用于实现阵列实参（作为标准 RAM 接口）。如果在 Vivado IP integrator 中使用 RTL 设计，请将组成该接口的各端口分离。
- `ap_fifo`：使用含关联低电平有效 FIFO `empty` 端口和 `full` 端口的数据输入和输出端口来实现含标准 FIFO 接口的端口。

注释：您只能对读取实参或写入实参使用 `ap_fifo` 接口。`ap_fifo` 模式不支持双向读写实参。

2. AXI 接口协议：

- `s_axilite`：用于将该端口作为 AXI4-Lite 接口来实现。为 HLS 组件导出生成的 RT 时，该工具会生成一组关联的 C 语言驱动程序文件。
- `m_axi`：用于将该端口作为 AXI4 接口来实现。您可使用 `syn.interface.m_axi_addr64` 命令来指定 32 位（默认）地址端口或 64 位地址端口，并控制任何地址偏移。
- `axis`：用于将该端口作为 AXI4-Stream 接口来实现。

3. 块级控制协议：

- `ap_ctrl_chain`：实现一组块级控制端口，以执行下列操作。
 - 使用 `start` 表示开始设计操作
 - 指示何时设计处于 `idle` 状态
 - 指示何时设计处于 `done` 状态
 - 使用 `ready` 表示接收新输入数据
- `ap_ctrl_hs`：实现一组块级控制端口以通过 `start` 开始设计操作。同时指示何时设计处于 `idle`、`done` 和 `ready` 状态（接收新输入数据）。
- `ap_ctrl_none`：无块级 I/O 协议。

注释：使用 `ap_ctrl_none` 模式可阻止使用 C/RTL 协同仿真来验证设计。

- `bundle=<string>`：

默认情况下，HLS 工具会将兼容的函数实参组合（或捆绑）到 RTL 代码中的单个接口端口内。含兼容选项（例如，`mode`、`offset` 和 `bundle`）的所有接口端口都组合到单一接口端口内。



提示：此默认行为可使用 `syn.interface.m_axi_auto_max_ports` 命令来进行更改。

`bundle=<string>` 选项允许您定义捆绑，用于将端口组合在一起，这会覆盖默认行为。`<string>` 用于指定捆绑名称。`mode` 与 `bundle` 的组合可在生成的 RTL 中提供端口名称。`name` 也可指定端口名称。



重要提示！您必须使用小写字母来指定 `bundle` 名称。

- `clock=<string>`：默认情况下，AXI4-Lite 接口时钟与系统时钟为相同时钟。该选项会为 AXI4-Lite 接口指定独立时钟。如果使用 `bundle` 选项将多个顶层函数实参组合到单一 AXI4-Lite 接口中，那么只需在其中一个捆绑成员上指定时钟选项即可。

- `channel=<string>`: 要在 `m_axi` 接口上启用多个通道, 请指定通道 ID。您可使用不同的通道 ID 将多个 `m_axi` 接口组合为单个 `m_axi` 适配器。
- `depth=<int>`: 指定供测试平台处理的最大采样数。此设置用于指示 HLS 工具为 RTL 协同仿真所创建的验证适配器中所需 FIFO 的最大大小。



提示: 虽然 `depth` 选项通常为可选, 但需要通过该选项来为 RTL 协同仿真指定指针实参大小。

- `interrupt=<int>`: 仅限 `ap_vld/ap_hs` 才会使用该选项。该选项会启用中断中的 I/O 管理。该选项会在 `s_axilite` 寄存器文件中的 `ISR` 和 `IER` 中创建对应的位。整数值 `N=16..31` 用于指定这两个寄存器中的位元位置。默认情况下, 位元的位置是从 16 开始连续分配的。
- `latency=<value>`: 该选项可用于 `ap_memory` 和 `m_axi` 接口。
 - 在 `ap_memory` 接口中, 该选项用于指定驱动接口的 RAM 资源的读取时延。默认执行 1 个时钟周期的读取操作。该选项允许对读取时延超过 1 个时钟周期的外部 RAM 进行建模。
 - 在 `m_axi` 接口中, 该选项用于指定 AXI4 接口的目标时延, 允许设计先发起总线请求, 然后在经过指定的周期数 (时延) 后再执行所期望的读取或写入操作。如果该值太低, 设计可能过早达成就绪状态, 并且可能停滞并等待总线; 如果该值太高, 总线访问可能处于空闲状态并等待设计发起访问。
- `max_read_burst_length=<int>`: 该选项适用于 `m_axi` 接口, 可指定突发传输期间读取的数据值的最大数量。如需了解更多信息, 请参阅《Vitis 高层次综合用户指南》(UG1399) 的“AXI 突发传输”章节。
- `max_write_burst_length=<int>`: 该选项适用于 `m_axi` 接口, 可指定突发传输期间写入的数据值的最大数量。
- `max_widen_bitwidth=<int>`: 自动拓宽接口时, 请指定接口可用的最大位宽。此设置将覆盖由 `syn.interface.m_axi_max_bitwidth` 命令指定的默认值。如需了解更多信息, 请参阅《Vitis 高层次综合用户指南》(UG1399) 的“端口宽度自动调整”章节。
- `name=<string>`: 指定端口名称以供生成的 RTL 使用。`mode` 与 `bundle` 组合即可指定端口名称。但以 `name` 指定名称时则是例外。
- `num_read_outstanding=<int>`: 该选项适用于 `m_axi` 接口, 用于指定在设计停滞前可对 AXI4 总线发起的读取请求数量 (无响应)。此操作暗示设计中的内部存储空间, 且 FIFO 大小为:

```
num_read_outstanding*max_read_burst_length*word_size
```

- `num_write_outstanding=<int>`: 搭配 `m_axi` 接口一起使用。该选项指定 AXI4 总线在设计停滞之前可以接受的写请求 (无响应) 数量。此操作暗示设计中的内部存储空间, 且 FIFO 大小为:

```
num_write_outstanding*max_write_burst_length*word_size
```

- `offset=<string>`: 为指定端口控制 AXI4-Lite (`s_axilite`) 和 AXI4 存储器映射 (`m_axi`) 接口中的地址偏移。
 - 在 `s_axilite` 接口中, `<string>` 用于指定寄存器映射中的地址。
 - 在 `m_axi` 接口中, 该选项会覆盖 `config-interface -m_axi_offset` 选项和 `<string>` 所指定的全局选项。请参阅下文以获取详细信息。
 - `off`: 不生成偏移端口。
 - `direct`: 生成标量输入偏移端口。
 - `slave`: 生成偏移端口并自动将其映射到 AXI4-Lite 从接口。这是默认偏移。

- `register`：寄存信号和所有关联的协议信号。指示各信号保持不变直至至少完成函数执行的最后一个周期为止。`syn.interface.register_io` 命令用于控制顶层函数上的所有接口的默认寄存，该选项则允许您覆盖当前接口的默认设置。该选项适用于以下接口模式：

- `s_axilite`
- `ap_fifo`
- `ap_none`
- `ap_stable`
- `ap_hs`
- `ap_ack`
- `ap_vld`
- `ap_ovld`



提示： `register` 选项不适用于函数的返回端口 (`port=return`)。请改用 `syn.directive.latency`。

- `register_mode=(both|forward|reverse|off)`：该选项适用于 AXI4-Stream 接口，它会指定是否在以下位置布局寄存器。
 - 在正向路径上布局寄存器 (`TDATA` 和 `TVALID`)，或者
 - 在反向路径 (`TREADY`) 上布局寄存器，在两条路径上都布局寄存器 (`both`)，或者
 - 不在任何端口信号上布局寄存器 (`off`)。

默认值为 `register_mode=both`。



提示： AXI4-Stream 旁路信号为数据信号。寄存 `TDATA` 时也会寄存器这些信号。

- `storage_impl=<impl>`：该选项仅适用于 `mode=s_axilite`，用于定义分配给该接口的存储实现。受支持的 `<impl>` 值包括：`auto`、`bram` 和 `uram`。默认值为 `auto`。



提示： `uram` 是同步存储器，能为两个端口提供单一时钟（仅限某些器件上可用）。因此不能将 `uram` 指定用于含 2 个时钟的 `s_axilite` 适配器，当指定器件不支持 `uram` 时也不可用。

- `storage_type=<type>`：

该选项仅适用于 `mode=ap_memory` 或 `mode=bram`，用于定义要分配给变量的存储类型（例如，`RAM_T2P`）。

受支持的类型包括：`ram_1p`、`ram_1wnr`、`ram_2p`、`ram_s2p`、`ram_t2p`、`rom_1p`、`rom_2p` 和 `rom_np`。



提示： 对于顶层函数上的对象，可使用 `syn.directive.bind_storage` 定义该选项。

示例 1

此示例用于演示为 `func` 函数禁用函数级握手。

```
syn.directive.interface=mode=ap_ctrl_none func return
```

示例 2

函数 `func` 中的实参 `InData` 具有 `ap_vld` 接口。需寄存输入。

```
syn.directive.interface=func InData mode=ap_vld register
```

另请参阅

- [syn.directive.bind_storage](#)
- [syn.directive.latency](#)

syn.directive.latency

描述

在函数、循环或区域上指定最大和/或最小时延值。

Vitis HLS 始终以最小时延为目标。下面显示了指定最小与最大时延值后工具的行为。

- 当时延小于最小值时：如果 Vitis HLS 可以实现小于最小指定值的时延，那么它可将时延扩展至指定值。此操作可支持更多共享。
- 当时延大于最小值时：满足约束。该工具不会执行进一步的优化。
- 当时延小于最大值时：满足约束。该工具不会执行进一步的优化。
- 当时延大于最大值时：如果 Vitis HLS 无法调度到最大值范围内，那么它会尽力实现指定约束。如果仍无法满足最大时延，则会发出警告。随后，Vitis HLS 会以可实现的最小时延来生成设计。



提示：您还可以使用 `syn.directive.latency` 来限制该工具寻找最优解决方案的范围。指定代码内作用域的时延约束。为循环、函数或区域等作用域指定时延约束可减少该作用域内可能的解决方案数量，从而改善工具编译时间。如需了解更多信息，请参阅《Vitis 高层次综合用户指南》(UG1399) 的“改善综合运行时间和容量”章节。

要限制所有循环迭代的总时延，请将时延指令应用于包含整个循环的区域。例如：`syn.directive.latency Region_All_Loop_A max=10`

```
Region_All_Loop_A: {
  Loop_A: for (i=0; i<N; i++)
  {
    ..Loop Body...
  }
}
```

在此情况下，即使展开循环，时延指令仍会对所有循环操作设置最大限制。

语法

```
syn.directive.latency=[OPTIONS] <location>
```

- `<location>` 是要约束的函数、循环或区域的位置（格式为 `function[/label]`）。

选项

- `max=<integer>`：限制最大时延。

- `min=<integer>`：限制最小时延。

示例

`foo` 函数采用的最小时延值为 4，最大时延值为 8。

```
syn.directive.latency=min=4 max=8 foo
```

在 `foo` 函数中，指定 `loop_row` 循环采用最大时延 12。

```
syn.directive.latency=max=12 foo/loop_row
```

syn.directive.loop_flatten

描述

允许将内层嵌套循环折叠（平铺）为单个循环，以便在循环的所有迭代上应用流水打拍，从而实现更低的时延。

只能对最内层的循环（完成可能的内部展开后）进行流水打拍。外层循环只能是数据流循环或顺序循环。当流水打拍循环上层的循环为顺序循环时，其迭代会依次执行，而且每次迭代，就会完全执行一次内层循环。在 RTL 实现中，从外层循环移动到内层循环需要一个时钟周期，从内层循环移回外层循环需要一个时钟周期，并且中间要加上完成内层循环的所有迭代的完整时延。与此相反，平铺嵌套循环可以将其作为单个循环来进行优化和流水打拍，这样外层循环的迭代与内层循环不同调用的迭代之间就可能发生重叠（流水打拍）。总之，平铺可以提高性能，但是：

- 平铺并非总是可行，需要遵循一定的编码风格。
- 在某些情况下，根据外层循环中的运算和/或依赖关系，时序甚至 II 都可能会劣化。

将 `syn.directive.loop_flatten` 应用于循环层级的最内层循环的循环体。只有完美循环和近乎完美的嵌套循环（在完成可能的初步函数内联或循环展开后）才能以这种方式加以平铺：

- 完美循环嵌套：
 - 每个非最内层循环的主体都包含一个且仅包含一个子循环，不包含任何其他指令。
- 近乎完美的循环嵌套：
 - 每个非最内层循环的主体都包含一个且仅包含一个子循环，无任何其他控制流。
 - 每个非最内层循环的主体都不得包含任何含循环的函数调用。

对于近乎完美的循环，编译器会将两个循环之间存在的任何指令自动“推入”最内层的循环，从而使这些循环完美嵌套。

此外具备平铺能力的循环还需要满足一些要求：

- 每个循环都应该是 `for` 循环，而不是 `while` 循环；并且不含 `break` 语句。
- 在平铺循环之前，编译器应能计算出每个循环的循环次数（但它不必是一个常数）。典型的编码风格如下：
 - 所含循环计数器按常数递增的循环。
 - 循环计数器的下限和上限不依赖于要平铺的循环（应满足“循环不可变”条件）。

举例来说，如果循环的内层循环次数取决于外层循环计数器，那么这类循环就不具备平铺能力。

编译器无法平铺不完美的循环嵌套（即，当循环包含多个子循环或控制流时）。在此情况下，需要手动执行平铺，方法是重构代码，将指令推入最内层的循环，或者展开内层循环以在其上层创建完美循环嵌套。

语法

```
syn.directive.loop_flatten=[OPTIONS] <location>
```

- <location> 表示最内层循环的位置，格式为 `function[/label]`。

选项

- `off`：可选关键字。阻止将包含 `loop_flatten off` 的循环与其子循环（如有）一起进行平铺。



重要提示！ 存在 `LOOP_FLATTEN` 编译指示和指令即表示启用优化。添加 `off` 则表示禁用优化。

示例

将 `foo` 函数中的 `loop_1` 及循环层级中位于其上层的所有（完美或近乎完美）循环平铺为单一循环。

```
syn.directive.loop_flatten=foo/loop_1
```

防止在 `foo` 函数的 `loop_2` 中进行循环平铺。

```
syn.directive.loop_flatten=off foo/loop_2
```

如需获取更完整的示例，请参阅对应的 [pragma HLS loop_flatten](#) 章节。

另请参阅

- [set_directive_loop_flatten](#)
- [pragma_HLS_loop_merge](#)
- [pragma_HLS_unroll](#)

syn.directive.loop_merge

描述

将所有循环合并到单一循环中。合并循环：

- 减少 RTL 中循环主体实现间转换所需时钟周期数。
- 允许并行实现循环（若可能）。

循环合并规则为：

- 如果循环边界为变量，则值（迭代数）必须相同。
- 如果循环边界是常量，那么该工具会使用最大常量值作为合并的循环的边界。
- 具有变量边界的循环与具有常量边界的循环无法合并。
- 合并循环之间的代码不得产生副作用。多次执行此代码需生成相同结果。
 - 允许 `a=b`

- 不允许 `a=a+1`。
- 包含 FIFO 读取的循环无法合并。合并会更改读取顺序。从 FIFO 或 FIFO 接口执行读取必须始终按顺序进行。

语法

```
syn.directive.loop_merge=[options] <location>
```

- `<location>` 是循环所在位置（格式为 `function[/label]`）。

选项

- `force`：即使 Vitis HLS 发出警告仍强制合并循环。确保合并的循环正常运行。

示例

将函数 `foo` 中的所有连续循环都合并到单一循环中。

```
syn.directive.loop_merge=foo
```

函数 `foo` 的 `loop_2` 内的所有循环（不包含 `loop_2` 本身）均通过使用 `force` 选项来合并。

```
syn.directive.loop_merge=force foo/loop_2
```

另请参阅

- [syn.directive.loop_flatten](#)
- [syn.directive.unroll](#)

syn.directive.loop_tripcount

描述

“循环次数”是指循环执行的迭代总数。Vitis HLS 可报告每个循环的总时延（执行循环的所有迭代所需周期数）。因此，此循环时延为循环次数（循环迭代次数）的函数。



重要提示！ `syn.directive.loop_tripcount` 仅用于分析，不影响综合结果。

循环次数可为常量值。它可能取决于循环表达式（例如，`x<y`）中使用的变量值或循环内使用的控制语句。

在某些情况下，Vitis HLS 无法判定循环次数。例如，如果用于判定循环次数的变量属于：

- 输入实参，或者
- 采用动态运算来计算的变量

在以下示例中，输入 `num_samples` 的值用于判定 `for` 循环的最大迭代数。在 C 语言函数中不定义 `num_samples` 的值，该值从外部进入函数。

```
void foo (num_samples, ...) {
    int i;
    ...
    loop_1: for(i=0;i< num_samples;i++) {
        ...
        result = a + b;
    }
}
```

如果循环时延未知，那么 `syn.directive.loop_tripcount` 允许您指定循环的最小、最大和平均迭代数。该工具可分析循环时延给报告中的设计总时延造成的影响，以帮助您为设计决定合适的优化措施。



提示：如果您使用 C 语言断言宏来限制循环变量的大小，那么 Vitis HLS 会执行以下操作。Vitis HLS 会使用该宏来为报告定义循环限制，创建硬件并根据这些限制来精确调整硬件大小。

语法

```
syn.directive.loop_tripcount=[OPTIONS] <location>
```

- `<location>` 表示循环中指定循环次数的位置（格式为 `function[/label]`）。

选项

- `avg=<integer>`：指定平均迭代次数。
- `max=<integer>`：限制最大迭代次数。
- `min=<integer>`：限制最小迭代次数。

示例

函数 `foo` 中的 `loop_1` 的最小循环次数指定为 12，最大循环次数则为 16：

```
syn.directive.loop_tripcount=min=12 max=16 avg=14 foo/loop_1
```

syn.directive.occurrence

描述

执行函数和循环流水打拍时，OCCURRENCE 编译指示或指令会指定以下内容。在流水打拍函数或循环内，流水打拍函数调用中的代码的执行速率低于其外层函数或循环的执行速率。以较低的速率执行代码即可支持流水打拍的调用以较低速率进行流水打拍，并且可能在顶层流水线中共享。例如：

- 循环迭代 N 次。
- 循环的一部分受条件语句保护，并且仅执行 M 次，其中 N 是 M 的整数倍。
- 假设受条件保护的代码将执行 N/M 次。

如果以 OCCURRENCE 速率标识某一区域，则在此区域中将执行流水打拍的函数和循环的启动时间间隔可比外层函数或循环更长。

语法

```
syn.directive.occurrence=[OPTIONS] <location>
```

- <location> 用于指定代码块，此代码块中包含执行速率较低的流水打拍函数调用。

选项

- `cycle=<int>`：指定执行次数 N/M，其中：
 - N 是执行外层函数或循环的次数。
 - M 是执行条件区域的次数。



重要提示！ N 必须是 M 的整数倍。

- `off=true`：为指定函数禁用 `occurrence`。

示例

`foo` 函数中的 `Cond_Region` 区域执行 4 次。它的执行速度比其外层的代码更慢，仅为外层代码的四分之一。

```
syn.directive.occurrence=cycle=4 foo/Cond_Region
```

另请参阅

- [syn.directive.pipeline](#)

syn.directive.performance

描述



提示： `syn.directive.performance` 适用于循环和循环嵌套，需要已知循环次数才能判定性能。如果您的循环包含可变循环计数，则必须同时指定 `syn.directive.tripcount`。

`syn.directive.performance` 允许您指定高层次约束：`target_ti` 或 `target_tl`。该指令允许您定义循环的连续启动之间的时钟周期数。该指令允许工具推断更低层次的 `UNROLL`、`PIPELINE`、`ARRAY_PARTITION` 和 `INLINE` 指令，以达成所期望的结果。`syn.directive.performance` 不保证将达成指定的值，仅作为目标。

`target_ti` 是循环的连续启动的间隔时间。它也是开始循环的首次迭代与下一次开始该循环的首次迭代之间的时间间隔。在以下代码示例中，`target_ti=T` 意味着在 L1 的 2 次连续迭代之间，开始 L2 循环的目标时间间隔是 100 个周期。

```
const int T = 100;
L1: for (int i=0; i<N; i++){
  L2: for (int j=0; j<M; j++){
    #pragma HLS PERFORMANCE target_ti=T
    ...
  }
}
```

`target_tl` 是循环启动与结束之间的时间间隔，或者循环的首次迭代开始与循环的最后一次迭代完成之间的时间间隔。在前述代码示例中，`target_tl=T` 意味着对于 L1 的单个迭代，L2 循环的目标完成时间应为 100 个周期。

注释： `syn.directive.inline` 会自动应用于 `II=1` 的任意流水打拍循环内部的函数，以改善吞吐量。如果您应用 `PERFORMANCE` 编译指示，那么该工具还会触发自动内联优化。如果您应用某条指令，并且该指令推断的流水线 `II=1`，那么该工具也会触发自动内联优化。您可使用 `syn.directive.inline off` 来为特定函数禁用此操作。

传输事务时间间隔是循环的启动时间间隔 (`II`) 乘以迭代次数或循环次数： $\text{target_ti} = \text{II} * \text{loop tripcount}$ 。反之， $\text{target_ti} = \text{频率 (Hz)} / \text{每秒操作数}$ 。

例如，假设某个图像处理函数每次调用都处理一帧，且吞吐量目标为 60 fps。该函数的目标吞吐量即为每秒 60 次调用。如果时钟频率为 180 MHz，则 `target_ti` 为 $180\text{M}/60$ ，或每次函数调用 3 百万个时钟周期。

语法

指定已标记的循环的指令。

```
syn.directive.performance=<location> [Options]
```

其中：

`<location>` 以 `function/loop_label` 格式来指定循环。

选项

- `target_ti=<value>`：指定目标传输事务时间间隔是循环完成一次迭代的时钟周期数。传输事务时间间隔是指从循环或嵌套循环的第一项传输事务开始到该循环的下一项传输事务开始之间的时钟周期数。`<value>` 可取整数、浮点或由该工具解析为整数的常量表达式。

注释： 如果发生截位，该工具会返回警告。

- `target_tl=<value>`：

指定目标时延是循环完成所有迭代的时钟周期数。传输事务时延即为循环的首次迭代开始与该循环的最后一次迭代完成之间的时间间隔。`<value>` 可取整数、浮点或由该工具解析为整数的常量表达式。

- `unit=[sec | cycle]`：指定与 `target_ti` 值或 `target_tl` 值关联的单位。此单位可取秒数或时钟周期数。当指定的单位为秒数时，您可用表示纳秒 (ns)、皮秒 (ps) 或微秒 (us) 的相应的值来指定该单位。

示例 1

标记为 `loop_1` 的循环的目标传输事务时间间隔为 4 个时钟周期：

```
syn.directive.performance=loop_1 target_ti=4
```

另请参阅

- [syn.directive.inline](#)

syn.directive.pipeline

描述

允许并发执行操作，以缩短函数或循环的启动时间间隔 (`II`)。欲知详情，请参阅《Vitis 高层次综合用户指南》(UG1399) 的“流水打拍范例”小节。流水线函数或循环每隔 `N` 个时钟周期即可处理新输入，其中 `N` 为启动时间间隔 (`II`)。如果 `II` 为 1，则表示每个时钟周期都处理新输入。

该工具默认根据指定的时钟周期约束，为设计生成最小 `II`。除非指定 `-II`，否则重点强调满足时序，而不是实现 `II`。

`syn.compile.pipeline_style` 用于定义流水线命令的默认类型，如 [编译选项](#) 所述。您可使用 PIPELINE 编译指示或指令将其覆盖。

如果 Vitis HLS 无法使用指定 II 创建设计，那么它会发出警告，并以可达成的最低 II 来创建设计。您可利用警告消息来分析此设计，以判定要采取哪些步骤才能创建满足所需启动时间间隔的设计。

语法

```
syn.directive.pipeline=[OPTIONS] <location>
```

其中：

- `<location>` 是要流水打拍的位置（格式为 `function[/label]`）。

选项

- `II=<integer>`：指定期望的流水线启动时间间隔。Vitis HLS 会尝试满足此请求。根据数据依赖关系，实际结果的 II 可能更大。
- `off`：关闭特定循环或函数的流水线。使用 `config_compile -pipeline_loops` 全局进行循环流水打拍时，请使用该选项。
- `rewind`：

注释：仅适用于循环。

可选关键字。启用回绕，如《Vitis 高层次综合用户指南》(UG1399) 的“回绕已流水打拍的循环以保障性能”所述。这样即可支持持续性循环流水打拍，但在循环执行结束与下一次执行开始之间没有暂停。仅当顶层函数内只有一个循环（或完美循环嵌套）时，回绕才有效。循环前的代码段：

- 被视为初始化。
- 在流水线中仅执行一次。
- 不能包含任何条件运算 (`if-else`)。
- `style=<stp | frp | flp>`：指定要用于指定的函数或循环的流水线类型。如需了解有关流水线样式的更多信息，请参阅《Vitis 高层次综合用户指南》(UG1399) 的“刷新流水线和流水线类型”章节。流水线类型包括：
 - `stp`：停止流水线。仅当输入数据可用时才运行，否则停滞。这是默认设置，也是 Vitis HLS 用于循环和函数流水打拍的流水线类型。无需可刷新的流水线时，请使用该选项。例如，没有导致停滞的性能或死锁问题时即可用。
 - `flp`：将流水线定义为可刷新流水线。此类型的流水线通常耗用更多资源并且 II 可能较大，因为您无法在流水线迭代间共享资源。
 - `frp`：自由运行的可刷新流水线。即使输入数据不可用也可运行。如果由于流水线控制信号扇出减少而导致需要更好的时序，或者如果需要提升性能以免死锁，则可使用此类型。但此流水线样式可能功耗更大，因为即使没有数据也会对流水线寄存器进行计时。



重要提示！这是约束，但并非硬性约束。工具会检查设计启用流水打拍的条件。部分循环可能不遵循特定样式，如果需要，该工具会还原为默认样式 (`stp`)。

示例

函数 `func` 按指定启动时间间隔进行流水打拍。

```
syn.directive.pipeline=func II=1
```

另请参阅

- [syn.directive.dependence](#)

syn.directive.protocol

描述

指定代码的某个协议区域，除非此代码另有明确指示，否则 Vitis HLS 在此区域中不会插入时钟操作。该工具不会在该区域内的运算之间插入任何时钟，包括读取或写入函数实参的时钟。已综合的 RTL 严格遵守读取和写入的顺序。

在 C/C++ 代码中可以通过使用 “{}” 将区域括起或者对该区域进行命名来创建代码区域。以下示例定义了名为 `io_section` 的区域：

```
io_section:{
...
lines of code
...
}
```

您可以使用 `ap_wait()` 语句来指定 C 语言代码中的时钟操作。您可以使用 `wait()` 语句来指定 C++ 代码中的时钟操作。



提示： `ap_wait` 和 `wait` 语句对于设计的仿真没有影响。

语法

```
syn.directive.protocol=[OPTIONS] <location>
```

<location> 用于指定您定义协议区域的具体位置（格式为 `function[/label]`）。

选项

- `mode=[floating | fixed]:`
 - `floating`：表示浮动模式，允许协议区域外的代码语句与最终 RTL 内协议区域中的语句重叠且并行执行。协议区域可保持时钟精确，但仍可同时发生外部运算。这是默认模式。
 - `fixed`：表示固定模式，`fixed` 模式可确保协议区域外部的语句不会与协议区域并行执行。

示例

此代码示例在函数 `foo` 内定义了协议区域 `io_section`。以下指令用于将该区域定义为固定模式协议区域：

```
syn.directive.protocol=mode=fixed foo/io_section
```

syn.directive.reset

描述

RESET 编译指示或指令用于为特定状态变量（全局变量或静态变量）添加或禁用复位端口。

使用复位端口将与端口连接的寄存器和块 RAM 复原为初始值。您可在应用复位信号时，随时执行此操作。

`syn.rtl.reset` 配置设置会对 RTL 复位端口的存在和行为进行全局配置。复位配置设置不仅能定义复位极性，还能指定采用同步复位还是异步复位。更重要的是，复位配置设置能控制在应用复位信号时对哪些寄存器进行复位。如需了解更多信息，请参阅《Vitis 高层次综合用户指南》(UG1399) 中的“控制初始化与复位行为”。

RESET 编译指示可对复位进行更具体的控制。对于全局变量或静态变量，如果不存在任何复位，那么 RESET 编译指示会显式启用一个复位。您也可以通过 `off` 来关闭编译指示，以便从复位移除变量。当设计中存在静态阵列或全局阵列时，该选项非常实用。

注释：对于类的公开变量，您必须使用 RESET 编译指示，因为复位配置设置仅适用于在函数级别或文件级别声明的变量。此外，您还必须将 RESET 编译指示应用于顶层函数或子函数中的类的对象。您无法将其应用于类的私有变量。

语法

将 C 语言源代码中的编译指示置于变量生命周期边界内。

```
syn.directive.reset=<location> [ variable=<a> | off=true ]
```

其中：

- `location>`：指定您定义变量的位置（格式为 `function[/label]`）。
- `variable=<a>`：指定 RESET 编译指示应用到的变量。
- `off` 或 `off=true`：指示针对指定变量不生成复位。

示例

为 `foo` 函数中的 `a` 变量添加复位，即使全局复位 (`syn.rtl.reset`) 设置为 `none` 或 `control` 也是如此。

```
syn.directive.reset=foo a
```

从 `foo` 函数中的 `static int a` 变量移除复位，即使全局复位设置为 `state` 或 `all` 也是如此。

```
syn.directive.reset=off foo a
```

以下示例显示了如何将 RESET 编译指示或指令与类变量和方法搭配使用。在类中声明变量时，该变量必须是该类中的公共静态变量。您可以在类的方法中为变量创建 RESET 编译指示。在 HLS 设计的顶层函数或子函数中创建对象后，您也可以创建 RESET 编译指示。

```
class bar {
public:
    static int a; // class level static; must be public
    int a1;      // class-level non-static; must be public
    bar(...) {
        // #pragma reset does not work here for a or a1
    }
    // this is the function that is called in the top
    void run(...) {
        // #pragma reset does not work here for non-static variable a1
        // but does work for static variable a
        #pragma reset variable=a
    }
};
static int c; // file level
int d; // file level
void top(...) {
```

```
#pragma HLS top
static int b; // function level
bar t;
static bar s;

// #pragma reset works here for b, c and d, as well as t and s members
// except for t.a1 because it is not static
#pragma reset variable=b
#pragma reset variable=c
#pragma reset variable=d
#pragma reset variable=t.a
#pragma reset variable=s.a
#pragma reset variable=s.a1
t.run(...);
s.run(...);
}
```

以下显示了为 top 函数指定的 `syn.directive.reset` 命令。

```
syn.directive.reset=top variable=b
syn.directive.reset=top variable=c
syn.directive.reset=top variable=d
syn.directive.reset=top variable=t.a
syn.directive.reset=top variable=s.a
syn.directive.reset=top variable=s.a1
```

另请参阅

- [RTL 配置](#)

syn.directive.stable

描述

将 `syn.directive.stable` 应用于 DATAFLOW（数据流）区域或 PIPELINE（流水线）区域的实参。该编译指示表明在 DATAFLOW 区域的入口和出口处生成同步时，可忽略该区域的输入或输出。在输入的任务级（相应的细粒度）流水线的“第一阶段”中无需包含对该实参的读取访问。在输出的任务级（相应的细粒度）流水线的最后一个阶段中无需包含写入访问。

可在层级中的任意点（针对标量或阵列）指定该编译指示。该编译指示会自动应用到低于层级中该点的所有 DATAFLOW 区域或 PIPELINE 区域。DATAFLOW 区域或 PIPELINE 区域可以启动另一次迭代，即使尚未读取上一次迭代的值也是如此。对于输出，即使上一次迭代尚未完成，亦可发生下一次迭代写入操作。

语法

```
syn.directive.stable=<location> <variable>
```

- `<location>` 是用于约束指令的函数名或循环名。
- `<variable>` 是要约束的阵列的名称。

示例

在以下示例中，不使用 STABLE 指令的情况下，proc1 会与 proc2 同步，以确认其输入的读取操作（包括 A）。使用该指令后，则不再将 A 视为需要同步的输入。

```
void dataflow_region(int A[...], int B[...] ...
    proc1(...);
    proc2(A, ...);
```

以下对该示例的指令进行了描述。

```
syn.directive.stable=dataflow_region variable=A
syn.directive.dataflow dataflow_region
```

另请参阅

- [syn.directive.dataflow](#)
- [syn.directive.pipeline](#)

syn.directive.stream

描述

默认情况下，阵列变量作为 RAM 来实现。

- 顶层函数阵列参数作为 RAM 接口端口来实现。
- 一般阵列作为 RAM 来实现以便进行读写访问。
- 包含在子函数内的阵列或者基于循环的 DATAFLOW 优化都作为 RAM 乒乓缓冲器通道来实现。

如果按顺序来使用或生成阵列中的数据，那么使用串流数据更有效，其中使用 FIFO 代替 RAM。将顶层函数实参指定为 `INTERFACE mode=ap_fifo` 时，该阵列会作为串流来实现。如需了解更多信息，请参阅《Vitis 高层次综合用户指南》(UG1399) 的“Vivado IP 流程接口”章节。



重要提示！ 要保留访问，可能需要使用 `volatile` 限定符来阻止编译器优化（尤其是死码消除）。请参阅《Vitis 高层次综合用户指南》(UG1399) 的“类型限定符”。

语法

```
syn.directive.stream=[OPTIONS] <location> <variable>
```

- `<location>` 是包含阵列变量的位置（格式为 `function[/label]`）。
- `<variable>` 是要作为 FIFO 来实现的阵列变量。

选项

- `depth=<integer>:`

注释： 仅与数据流通道中的阵列串流相关。

默认情况下，RTL 中 FIFO 的深度与 C 语言代码中的阵列大小相同。该选项允许您修改 FIFO 的大小。

在 DATAFLOW 区域内实现阵列时，可使用 `-depth` 选项来减小 FIFO 的大小。例如，如果 DATAFLOW 区域中的所有循环和函数均以 `II = 1` 的速率来处理数据，则无需大型 FIFO。在每个时钟周期内都会生产和使用数据。您可使用 `-depth` 将 FIFO 大小减小为 2 以便大幅减小 RTL 设计的面积。

`config_dataflow` 命令搭配 `-depth` 选项即可对 DATAFLOW 区域内的所有阵列提供相同的功能。`-depth` 选项搭配 `set_directive_stream` 一起使用即可覆盖 `config_dataflow` 中的默认设置。

- `type=<arg>`：指定某种机制，可选机制包括 FIFO、PIPO、已同步的共享 (`shared`) 和未同步的共享 (`unsync`)。受支持的类型包括：
 - `fifo`：含指定 `depth` 的 FIFO 缓冲器。
 - `pipo`：常规乒乓缓冲器，所含“bank”数量与指定深度相同（默认值为 2）。
 - `shared`：含深度的共享通道，其同步方式与常规乒乓缓冲器类似，但不复制阵列数据。通过将深度设置为足够小的值以充当生产者与使用者之间的同步距离，即可确保一致性。



提示： `shared` 默认深度值为 1。

- `unsync`：除了个别存储器读取和写入外，无任何同步。设计必须自行确保一致性（读取 - 写入和写入 - 写入顺序）。

示例

指定 `func` 函数中要进行串流的 `A[10]` 阵列，并作为 FIFO 实现。

```
syn.directive.stream=func A type=fifo
```

`func` 函数中名为 `loop_1` 的循环中的阵列 B 设置为作为 FIFO（深度为 12）来进行串流。在此情况下，请将编译指示置于 `loop_1` 内。

```
syn.directive.stream=depth=12 type=fifo func/loop_1 B
```

阵列 C 的串流以 PIPO 方式来实现。

```
syn.directive.stream=type=pipo func C
```

另请参阅

- [syn.directive.dataflow](#)
- [syn.directive.interface](#)
- [数据流配置](#)

syn.directive.top

描述

给函数附加名称。随后此名称可供 `set_top` 命令用于将指定函数设置为顶层函数。此命令通常用于对 C++ 中的某个类的成员函数进行综合。

语法

```
syn.directive.top=[OPTIONS] <location>
```

- `<location>` 表示要重命名的函数。

选项

- `name=<string>`: 指定要供 `syn.top` 命令使用的函数名称，如 [HLS 常规选项](#) 所述。

示例

将 `foo_long_name` 函数重命名为 `DESIGN_TOP`。随后，将顶层函数指定为 `DESIGN_TOP`。如果在代码中放置编译指示，则仍必须在 GUI 工程设置中指定的顶层发出 `set_top` 命令。

```
syn.directive.top=name=DESIGN_TOP foo_long_name
```

随后发出 `syn.top=DESIGN_TOP` 命令。

另请参阅

- [HLS 常规选项](#) 中的 `syn.top`

syn.directive.unroll

描述

通过创建循环主体的多个副本来变换循环。

循环感应变量用于指定循环执行的迭代次数。循环主体内的逻辑（例如，对任意循环出口变量进行中断或修改）也会影响迭代次数。代表循环主体的逻辑块会在 RTL 中实现此循环。该逻辑会执行相同次数的迭代。

`syn.directive.unroll` 命令允许将此循环部分或全部展开。完全展开循环会在 RTL 中创建循环主体副本，副本数量与其中的循环迭代次数相同。按因子 `N` 部分展开循环会创建 `N` 份循环主体副本并对循环迭代进行相应调整。

如果用于部分展开的因数 `N` 并非原始循环迭代计数的整数倍，那么在循环主体的每个展开片段后请检查原始退出条件。

要完全展开循环，在编译时循环边界必须已知。对于部分展开则无需满足此条件。

语法

```
syn.directive.unroll=[OPTIONS] <location>
```

- `<location>` 表示要展开的循环位置（格式为 `format function[/label]`）。

选项

- `factor=<integer>`: 指定非零整数，表示已请求部分展开。

循环主体会重复整数次。迭代信息会进行相应的调整。

- `skip_exit_check`: 仅当指定因数时才有效（部分展开）。

- 固定边界

如果迭代计数为因数的倍数，则不执行出口条件检查。

如果迭代计数并非因数的整数倍，则该工具将执行以下操作：

- 阻止展开。
- 发出警告，称必须执行出口检查后才能继续。
- 变量边界
 - 移除出口条件检查。您必须确保：
 - 变量边界为因数的整数倍。
 - 无需出口检查。
- `off=true`：为指定循环禁用展开。

示例

展开 `foo` 函数内的 L1 循环。将编译指示置于 L1 循环的主体内。

```
syn.directive.unroll=foo/L1
```

在 `foo` 函数的 L2 循环上指定展开因数 4。移除出口检查。将编译指示置于 L2 循环的主体内。

```
syn.directive.unroll=skip_exit_check factor=4 foo/L2
```

另请参阅

- [syn.directive.loop_flatten](#)
- [syn.directive.loop_merge](#)

HLS 编译指示

Vitis HLS 中的优化

在 AMD Vitis™ 软件平台中，以 C/C++ 语言或 OpenCL™ C 语言定义的内核必须编译到寄存器传输级 (RTL)，并且此 RTL 必须可实现到 AMD 器件的可编程逻辑内。v++ 编译器通过调用 Vitis 高层次综合 (HLS) 工具来实现来自内核源代码的 RTL 代码。

HLS 工具旨在以无交互方式处理 Vitis IDE 工程。但是，HLS 工具还提供各种编译指示，用于优化设计、降低时延、提升吞吐量性能，以及减少生成的 RTL 代码的面积和器件资源使用率。这些编译指示可直接添加到内核源代码中。

HLS 编译指示可包含下表中指定的优化类型。

表 37: Vitis HLS 编译指示 (按类型)

类型	属性
内核优化	• pragma HLS aggregate • pragma HLS alias • pragma HLS disaggregate • pragma HLS expression_balance • pragma HLS latency • pragma HLS performance • pragma HLS protocol • pragma HLS reset • pragma HLS top • pragma HLS stable
函数内联	• pragma HLS inline
接口综合	• pragma HLS interface • pragma HLS stream
任务级流水线	• pragma HLS dataflow • pragma HLS stream
流水线	• pragma HLS pipeline • pragma HLS occurrence
循环展开	• pragma HLS unroll • pragma HLS dependence
循环优化	• pragma HLS loop_flatten • pragma HLS loop_merge • pragma HLS loop_tripcount
阵列优化	• pragma HLS array_partition • pragma HLS array_reshape
结构封装	• pragma HLS aggregate • pragma HLS dataflow

表 37: Vitis HLS 编译指示（按类型）（续）

类型	属性
资源使用情况	• pragma HLS allocation • pragma HLS bind_op • pragma HLS bind_storage • pragma HLS function_instantiate

pragma HLS aggregate

描述

收集结构体的数据字段并将其组合到字段更宽的单一标量中。

AGGREGATE 编译指示用于将任一结构体的所有元素都组合到一个单宽矢量中，以允许同时读取和写入该结构体的所有成员。可根据结构体元素的声明顺序推断出生成的新字宽标量的位对齐方式。结构体的第一个元素取矢量的 LSB，最后一个元素则与矢量的 MSB 对齐。

如果结构体包含阵列，那么 AGGREGATE 编译指示执行的操作与 ARRAY_RESHAPE 类似，可将重构的阵列与结构体中的其他元素组合在一起。在该结构体内部声明的所有阵列都将全部分区并重构为单一大宽度标量，与其他标量元素封装在一起。



重要提示！在含大型阵列的结构体对象上使用 AGGREGATE 优化时，应谨慎处理。如果某个阵列包含 4096 个类型为 int 的元素，则将导致矢量（和端口）宽度达 $4096 \times 32 = 131072$ 位。Vitis HLS 工具可以创建此 RTL 设计，但是逻辑综合在 FPGA 实现期间很可能无法完成其布线。

语法

将编译指示布局在要聚合的结构体变量定义附近：

```
#pragma HLS aggregate variable=<variable> compact=<arg>
```

其中：

- `variable=<variable>`：指定要组合的变量。
- `compact=[bit | byte | none | auto]`：指定已聚合的结构体的对齐方式。可选设置包括：按位级对齐、按字节级对齐、无对齐或者由工具自动判定对齐方式（默认行为）。

示例 1

将函数 `func` 内含 3 个 8 位字段 (`typedef struct {unsigned char R, G, B;} pixel`) 的结构体指针 `AB` 聚合到一个新的 24 位指针内并按位级对齐。

```
typedef struct{
    unsigned char R, G, B;
} pixel;

pixel AB;
#pragma HLS aggregate variable=AB compact=bit
```

示例 2

将含 3 个 8 位字段（R、G 和 B）的结构体阵列 AB[17] 聚合到一个含 17 个元素的 24 位新阵列内。

```
typedef struct{
    unsigned char R, G, B;
} pixel;

pixel AB[17];
#pragma HLS aggregate variable=AB
```

另请参阅

- [set_directive_aggregate](#)
- [pragma HLS array_reshape](#)
- [pragma HLS disaggregate](#)

pragma HLS alias

描述

指定两个或两个以上的 M_AXI 指针实参指向存储器（DDR 或 HBM）中的同一个底层缓冲器，并通过设置相互间的距离或偏移来表明指针之间的任何混叠。



重要提示！ ALIAS 编译指示适用于已映射到 M_AXI 接口的顶层函数实参。

Vitis HLS 将不同指针视为独立通道，通常不提供任何依赖关系分析。但如果主机为多个指针分配单个缓冲器，则可通过 ALIAS 编译指示或指令来告知此关系，并且可维护依赖关系分析。ALIAS 编译指示支持在 Vitis HLS 中通过定义缓冲器内指针之间的距离来执行数据依赖关系分析。

ALIAS 的要求：

- 分配给 ALIAS 编译指示的所有端口都必须分配给 M_AXI 接口，并分配给不同捆绑或通道。请参阅以下捆绑示例。
- 每个端口仅限在一条 ALIAS 编译指示或指令内使用
- 分配给 ALIAS 编译指示的所有端口的 depth 必须相同
- 指定偏移时，指定的端口数与偏移数必须相同，一个端口对应一个偏移
- INTERFACE 的偏移必须指定为从偏移或直接偏移，不支持 offset=off

语法

```
pragma HLS alias ports=<list> [distance=<int> | offset=<list...>]
```

其中：

- ports=<list>：用于指定要混叠的端口。
- distance=<integer>：指定传递到列表中的端口的指针值之间的距离。

- `offset=<list>`: 指定传递到 `ports` 列表中每个端口的指针相对于阵列原始指针的偏移。

注释： `offset` 与 `distance` 互斥。

示例

对于以下 `top` 函数：

```
void top(int *arr0, int *arr1, int *arr2, int *arr3, ...) {
    #pragma HLS interface mode=m_axi port=arr0 bundle=hbm0 depth=0x40000000
    #pragma HLS interface mode=m_axi port=arr1 bundle=hbm1 depth=0x40000000
    #pragma HLS interface mode=m_axi port=arr2 bundle=hbm2 depth=0x40000000
    #pragma HLS interface mode=m_axi port=arr3 bundle=hbm3 depth=0x40000000
```

以下编译指示用于为指定的阵列指针定义混叠，并定义指针之间的 `distance`：

```
#pragma HLS ALIAS ports=arr0,arr1,arr2,arr3 distance=10000000
```

或者，以下编译指示用于指定指针之间的 `offset`，达成效果与以上命令相同：

```
#pragma HLS ALIAS ports=arr0,arr1,arr2,arr3
offset=00000000,10000000,20000000,30000000
```

另请参阅

- [set_directive_alias](#)
- [pragma HLS interface](#)

pragma HLS allocation

描述

指定相应限制，以便对已实现的内核中的资源分配加以限制。ALLOCATION 编译指示或指令可限制用于实现特定函数、循环或运算的 RTL 实例和硬件资源的数量。ALLOCATION 编译指示是在代码的函数、循环或区域主体内部指定的。

例如，如果 C 语言代码包含函数 `foo_sub` 的 4 个实例，那么 ALLOCATION 编译指示可确保最终 RTL 中仅有 `foo_sub` 的 1 个实例。C 语言函数的全部 4 个实例都是使用相同 RTL 块来实现的。这样即可减少函数所使用的资源量，但由于共享这些资源，故而对性能会产生负面影响。

对于 ALLOCATION 还可指定模板函数，只需指定函数指针代替函数名称即可，如下示例所示。

C 语言代码中的运算（例如，加法、乘法、阵列读取和写入）均可通过 ALLOCATION 编译指示来加以限制。

语法

将编译指示置于函数、循环或区域的主体内适用的位置。



重要提示！ 以下实参顺序至关重要。`<type>` 作为运算或函数必须位于 `allocation` 关键字之后。

```
#pragma HLS allocation <type> instances=<list>
limit=<value>
```

其中：

- `<type>`：可指定以下任一类型：
 - `function`：指定分配适用于 `instances=` 列表中的函数。该函数可以是原始 C 语言或 C++ 语言代码中的任意函数，但满足下列任一类型的函数除外：
 - 由 `pragma HLS inline` 或 `set_directive_inline` 命令内联的函数，或者
 - 由 Vitis HLS 工具自动内联的函数。
 - `operation`：指定分配适用于 `instances=` 列表中的运算。
- `instances=<list>`：指定来自 C 代码的函数名称或运算符。
要获取可使用 `ALLOCATION` 编译指示加以限制的运算符的完整列表，请参阅 [config_op](#) 命令。
- `limit=<value>`：（可选）指定要在内核中使用的实例的限制。

示例 1

在设计中具有函数 `foo` 的多个实例的情况下，此示例用于将硬件内核的 RTL 中的 `foo` 实例数量限制为 2。

```
#pragma HLS allocation function instances=foo limit=2
```

示例 2

将函数 `my_func` 的实现中使用的乘法器运算数量限制为 1。此限制不适用于超出 `my_func` 范围的任意乘法器或者可能位于 `my_func` 的子函数内的乘法器。



提示： 要限制任何子函数实现过程中使用的乘法器数量，请在子函数上指定分配指令，或将子函数内联到 `my_func` 函数中。

```
void my_func(data_t angle) {
#pragma HLS allocation operation instances=mul limit=1
...
}
```

示例 3

`ALLOCATION` 编译指示还可在模板函数上使用，如下所示。标识通常基于函数名称，但对于模板函数，标识则基于函数指针：

```
template <typename DT>
void foo(DT a, DT b){
}
// The following is valid
#pragma HLS ALLOCATION function instances=foo<DT>
...
// The following is not valid
#pragma HLS ALLOCATION function instances=foo
```


另请参阅

- [set_directive_allocation](#)
- [pragma HLS inline](#)

pragma HLS array_partition

描述

将阵列分区为更小的阵列或者独立元素，并提供下列特性：

- 生成包含多个小型存储器或多个寄存器（而不是一个大型存储器）的 RTL。
- 有效增加存储器读写端口数量。
- 可能改善设计吞吐量。
- 需要更多存储器实例或寄存器。

语法

将 C 语言源代码中的编译指示置于定义阵列变量的函数边界内。

```
#pragma HLS array_partition variable=<name> \  
type=<type> factor=<int> dim=<int> off=true
```

其中：

- `variable=<name>`：必要实参，用于指定要分区的阵列变量。
- `type=<type>`：（可选）指定分区类型。默认类型为 `complete`。支持以下类型：
 - `cyclic`：循环分区会通过交织来自原始阵列的元素来创建更小的阵列。该阵列按循环进行分区，具体方式是在每个新阵列中放入一个元素，然后回到第一个阵列以重复该循环直至阵列完全完成分区为止。例如，如果使用 `factor=3`：
 - 向第 1 个新阵列分配元素 0。
 - 向第 2 个新阵列分配元素 1。
 - 向第 3 个新阵列分配元素 2。
 - 向第 4 个新阵列分配元素 3。
 - `block`：块分区会从原始阵列的连续块创建更小阵列。这样可将阵列有效分区为 N 个相等的块，其中 N 为 `factor=` 实参定义的整数。
 - `complete`：完全分区可将阵列分解为多个独立元素。对于一维阵列，这对应于将存储器解析为独立寄存器。这是默认 `<type>`。
- `factor=<int>`：指定要创建的更小的阵列数量。



重要提示！ 对于 `complete` 型分区，不指定该因子。对于 `block` 和 `cyclic` 型分区，需指定 `factor=`。

- `dim=<int>`：指定要分区的多维阵列的维度。默认值为 1。针对含 `<N>` 维的阵列，指定范围介于 0 到 `<N>` 之间的整数：
 - 如果使用 0 值，则使用指定的类型和因子选项对多维阵列的所有维度进行分区。
 - 任意非零值均表示只对指定维度进行分区。例如，如果使用的值为 1，则仅对第 1 个维度进行分区。
- `off=true`：为指定变量和指定的 `dim`（如有）禁用 `ARRAY_PARTITION` 功能特性。

示例 1

此示例使用 `block` 分区将 13 个元素的阵列 `AB[13]` 分区为 4 个阵列：

```
#pragma HLS array_partition variable=AB type=block factor=4
```



提示：由于 4 不是 13 的整数因子，因此：

- 其中 3 个新阵列各含 3 个元素
- 另 1 个阵列则包含 4 个元素 (`AB[9:12]`)

示例 2

此示例分区将二维阵列 `AB[6][4]` 的维度 2 分区为 2 个维度为 `[6][2]` 的新阵列：

```
#pragma HLS array_partition variable=AB type=block factor=2 dim=2
```

示例 3

此示例将二维 `in_local` 阵列的第二个维度分区为各独立元素。

```
int in_local[MAX_SIZE][MAX_DIM];
#pragma HLS ARRAY_PARTITION variable=in_local type=complete dim=2
```

示例 4

在您的代码中，可以按阵列的新结构来对已分区的阵列进行寻址，如以下代码示例所示；

```
struct SS
{
    int x[N];
    int y[N];
};

int top(SS *a, int b[4][6], SS &c) {
#pragma HLS array_partition type=complete dim=1 variable=b
#pragma HLS interface mode=ap_memory port = b[0]
#pragma HLS interface mode=ap_memory port = b[1]
#pragma HLS interface mode=ap_memory port = b[2]
#pragma HLS interface mode=ap_memory port = b[3]
```

另请参阅

- [set_directive_array_partition](#)
- [pragma_HLS_array_reshape](#)

pragma HLS array_reshape

描述

 **重要提示！** 针对顶层函数上的 M_AXI 接口，不支持 Array_Partition 和 Array_Reshape 编译指示和指令。您可改用 hls::vector 数据类型，如 [矢量数据类型](#) 中所述。

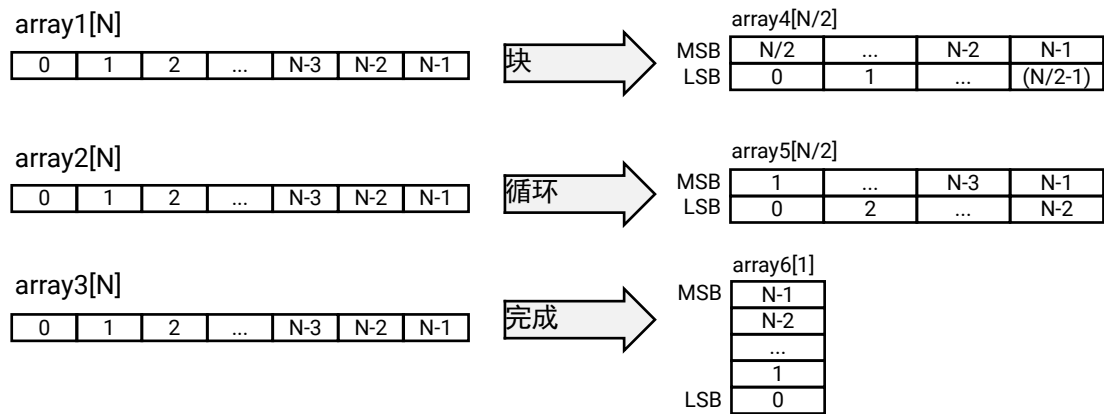
ARRAY_RESHAPE 编译指示用于通过增加位宽来对阵列元素进行垂直重映射和串联，以重构此阵列。这样即可减少耗用的块 RAM 数量，同时提供对数据的并行访问。该编译指示能够创建新阵列，其中包含元素数量更少但位宽更大，这样即可在单一时钟周期内访问更多数据。

给定如下代码：

```
void foo (...) {  
    int array1[N];  
    int array2[N];  
    int array3[N];  
    #pragma HLS ARRAY_RESHAPE variable=array1 type=block factor=2 dim=1  
    #pragma HLS ARRAY_RESHAPE variable=array2 type=cyclic factor=2 dim=1  
    #pragma HLS ARRAY_RESHAPE variable=array3 type=complete dim=1  
    ...  
}
```

ARRAY_RESHAPE 编译指示将阵列转换为下图所示形式。

图 139：ARRAY_RESHAPE 编译指示



X14307-030622

语法

将 C 语言源代码中的编译指示置于定义阵列变量的函数区域内。

```
#pragma HLS array_reshape variable=<name> type=<type> factor=<int>  
dim=<int> off=true
```

其中：

- `variable=<name>`：必要实参，用于指定要重构的阵列变量。

- `type=<type>`: (可选) 指定分区类型。默认类型为 `complete`。支持以下类型:
 - `cyclic`: 循环重塑会通过交织来自原始阵列的元素来创建更小的阵列。例如, 如果使用 `factor=3`, 则将向第 1 个新阵列分配元素 0, 向第 2 个新阵列分配元素 1, 向第 3 个新阵列分配元素 2, 然后再次向第 1 个新阵列分配元素 3。最后一个阵列是将新阵列垂直串联 (字串联以创建更长的字) 成单个阵列。
 - `block`: 块重塑会从原始阵列的连续块创建更小阵列。这样即可将阵列有效等分为 $<N>$ 个相等的块, 其中 $<N>$ 是 `factor=` 定义的整数, 然后使用 `word-width*N` 将 $<N>$ 个块整合到单一阵列中。
 - `complete`: 完整重塑可将阵列分解为临时独立元素, 然后将其重新组合为含更宽的字的单个阵列。对于一维阵列, 这等同于创建一个超宽寄存器 (如果原始阵列为 N 个 M 位的元素, 结果可生成含 $N*M$ 位的寄存器)。这是默认阵列重塑类型。
- `factor=<int>`: 指定当前阵列的拆分量 (或者要创建的临时阵列数)。因子为 2 表示将阵列对半拆分, 并将位宽加倍。因子为 3 表示将阵列拆分为 3 份, 并将位宽变为三倍。



重要提示! 对于 `complete` 型分区, 不指定该因子。对于 `block` 和 `cyclic` 型重塑, 需指定 `factor=`。

- `dim=<int>`: 指定要分区的多维阵列的维度。针对含 $<N>$ 维的阵列, 指定范围介于 0 到 $<N>$ 之间的整数:
 - 如果使用 0 值, 则使用指定的类型和因子选项对多维阵列的所有维度进行分区。
 - 任意非零值均表示只对指定维度进行分区。例如, 如果使用的值为 1, 则仅对第 1 个维度进行分区。
- `object`: 仅与容器阵列相关的关键字。指定此关键字时, `ARRAY_RESHAPE` 编译指示将应用于该容器内的对象, 并对容器内的对象的所有维度进行重塑, 但容器本身的所有维度都保留不变。不指定关键字时, 该编译指示将应用于容器阵列, 而不应用于对象。
- `off=true`: 为指定变量禁用 `ARRAY_RESHAPE` 功能特性。

示例 1

使用块映射, 将含 17 个元素的 8 位阵列 `AB[17]` 重塑为含 5 个元素的全新 32 位阵列。

```
#pragma HLS array_reshape variable=AB type=block factor=4
```



提示: `factor=4` 表示阵列应拆分为 4 份, 这表示将 17 个元素重塑为含 5 个元素的阵列, 位宽增加至原来的 4 倍。在此案例中, 最后一个元素 `AB[17]` 映射到第五个元素的下 8 位, 第五个元素的其余部分则为空。

示例 2

将二维阵列 `AB[6][4]` 重塑为新阵列 (维度为 `[6][2]`), 其中维度 2 位宽增至原来的 2 倍。

```
#pragma HLS array_reshape variable=AB type=block factor=2 dim=2
```

示例 3

将函数 `func` 中的三维 8 位阵列 `AB[4][2][2]` 重塑为全新的单元素阵列 (寄存器), 位宽为 128 位 ($4 \times 2 \times 2 \times 8$)。

```
#pragma HLS array_reshape variable=AB type=complete dim=0
```



提示: `dim=0` 表示重塑该阵列的所有维度。

示例 4

在您的代码中，可以按阵列的新结构来对已分区的阵列进行寻址，如以下代码示例所示；

```
struct SS
{
    int x[N];
    int y[N];
};

int top(SS *a, int b[4][6], SS &c) {
#pragma HLS array_reshape type=complete dim=0 variable=b
#pragma HLS interface mode=ap_memory port=b[0]
```

另请参阅

- [syn.directive.array_reshape](#)
- [pragma HLS array_partition](#)

pragma HLS array_stencil

描述

`#pragma HLS array_stencil variable=<name> <off>`

“`#pragma HLS array_stencil variable=xxx off`” 用于在循环中的某个变量上禁用自动模板。

全局自动控制

`config_array_stencil -throughput_driven <off|on>`

代码的规范形式和限制：

- 模板优化只能应用于顶层。
- 对于多维模板模式，循环边界需保持常数。
- 编译指示必须位于循环内。
- 应用模板优化的循环需要 `II=1` 的流水线编译指示。
- 模板优化要求嵌套循环为完美嵌套循环。
- 访问模式应采用固定模式。
- 访问模式顺序必须遵循循环顺序，例如，它必须有一个循环增量 ± 1 。

pragma HLS bind_op

描述



提示：此编译指示或指令在 DSP 多重运算匹配 (MULADD、AMA、ADDMUL 和相应的减法运算) 中不起作用。事实上，使用 BIND_OP 为 DSP 分配运算符会妨碍 HLS 编译器匹配多重运算表达式。如需了解更多信息，请参阅 [DSP 多重运算匹配](#)。

Vitis HLS 使用特定实现来实现代码中的运算。BIND_OP 编译指示用于指定针对每个特定变量，都应将一项运算 (mul、add、div) 映射到特定器件资源，以便在 RTL 内实现 (impl)。如果不指定 BIND_OP 编译指示，Vitis HLS 会自动判定用于运算的资源。

例如，您可使用 BIND_OP 编译指示来指明在器件互连结构内（而不是在 DSP 内）实现一项特定乘法器运算 (mul)。

您也可以使用 latency 选项来指定运算的时延。



重要提示！要使用 latency 选项，运算必须具有 1 个可用的多阶段实现。HLS 工具可为所有基本算术运算（加减乘除）和所有浮点运算提供 1 个多阶段实现。

语法

将 C 语言源代码中的编译指示置于定义变量的函数体内。

```
#pragma HLS bind_op variable=<variable> op=<type>\
impl=<value> latency=<int>
```

其中：

- **variable=<variable>**：用于定义要将 BIND_OP 编译指示分配到的变量。在此例中，定义的变量就是该编译指示的目标运算结果所分配到的变量。
- **op=<type>**：用于定义要绑定到特定实现资源的运算。受支持的函数运算包括：mul、add 和 sub 受支持的浮点运算包括：fadd、fsub、fddiv、fexp、flog、fmul、frsqrt、frecip、fsqrt、dadd、dsub、ddiv、dexp、dlog、dmul、drsqrt、drecip、dsqrt、hadd、hsqrt、hdiv、hmul 和 hsqrt



提示：浮点运算包括单精度 (f)、双精度 (d) 和半精度 (h)。

- **impl=<value>**：定义用于指定运算的实现。受支持的函数运算实现包括 fabric 和 dsp。受支持的浮点运算实现包括：fabric、meddsp、fulldsp、maxdsp 和 primitivedsp。

注释：primitivedsp 仅在 AMD Versal™ 器件上可用。

- **latency=<int>**：定义运算的实现的默认时延。有效的时延值因指定的 op 和 impl 而异。默认值为 -1，即交由 Vitis HLS 选择时延。下表反映了受支持的运算、实现和时延组合。

表 38：受支持的函数运算、实现和时延组合

运算	实现	最小时延	最大时延
add	fabric	0	4
add	dsp	0	4
mul	fabric	0	4

表 38：受支持的函数运算、实现和时延组合 (续)

运算	实现	最小时延	最大时延
mul	dsp	0	4
sub	fabric	0	4
sub	dsp	0	0



提示：比较运算符（如 `dcmp`）是在 LUT 中实现的，不得在互连结构外部实现，也不能映射到 DSP，因此无法通过 `config_op` 命令或 `bind_op` 命令来配置。

表 39：受支持的浮点运算、实现和时延组合

运算	实现	最小时延	最大时延
fadd	fabric	0	13
fadd	fulldsp	0	12
fadd	primitivedsp	0	3
fsub	fabric	0	13
fsub	fulldsp	0	12
fsub	primitivedsp	0	3
fdiv	fabric	0	29
fexp	fabric	0	24
fexp	meddsp	0	21
fexp	fulldsp	0	30
flog	fabric	0	24
flog	meddsp	0	23
flog	fulldsp	0	29
fmul	fabric	0	9
fmul	meddsp	0	9
fmul	fulldsp	0	9
fmul	maxdsp	0	7
fmul	primitivedsp	0	4
fsqrt	fabric	0	29
frsqrt	fabric	0	38
frsqrt	fulldsp	0	33
frecip	fabric	0	37
frecip	fulldsp	0	30
dadd	fabric	0	13
dadd	fulldsp	0	15
dsub	fabric	0	13
dsub	fulldsp	0	15
ddiv	fabric	0	58
dexp	fabric	0	40
dexp	meddsp	0	45

表 39：受支持的浮点运算、实现和时延组合 (续)

运算	实现	最小时延	最大时延
dexp	fulldsp	0	57
dlog	fabric	0	38
dlog	meddsp	0	49
dlog	fulldsp	0	65
dmul	fabric	0	10
dmul	meddsp	0	13
dmul	fulldsp	0	13
dmul	maxdsp	0	14
dsqrt	fabric	0	58
drsqrt	fulldsp	0	111
drecip	fulldsp	0	36
hadd	fabric	0	9
hadd	meddsp	0	12
hadd	fulldsp	0	12
hsub	fabric	0	9
hsub	meddsp	0	12
hsub	fulldsp	0	12
hdiv	fabric	0	16
hmul	fabric	0	7
hmul	fulldsp	0	7
hmul	maxdsp	0	9
hsqrt	fabric	0	16

示例

在以下示例中，指定了使用互连结构逻辑的二阶流水节拍乘法器来为函数 `foo` 的变量 `c` 实现乘法运算。

```
int foo (int a, int b) {  
  int c, d;  
  #pragma HLS BIND_OP variable=c op=mul impl=fabric latency=2  
  c = a*b;  
  d = a*c;  
  return d;  
}
```

 **提示：** HLS 工具会选择用于变量 `d` 的实现。

另请参阅

- [set_directive_bind_op](#)
- [pragma_HLS_bind_storage](#)

pragma HLS bind_storage

描述

BIND_STORAGE 编译指示用于将代码中的变量（阵列或函数实参）分配给 RTL 中的特定存储器类型 (type)。如果不指定此编译指示，那么 Vitis HLS 工具会判定要分配的存储器类型。HLS 工具在硬件中使用指定的实现 (impl) 来实现该存储器。

例如，您可使用此编译指示来指定要用于实现阵列的存储器类型。例如，这样您即可控制阵列是作为单端口 RAM 还是作为双端口 RAM 来实现。



重要提示！ 此功能特性对于顶层函数接口上的阵列尤为重要，因为与阵列关联的存储器类型可用于判定 RTL 中所需的端口数量和类型，如 [接口上的阵列](#) 中所述。但对于分配给顶层函数实参的变量，您必须使用 INTERFACE 编译指示或指令的 -storage_type 选项和 -storage_impl 选项来分配存储器类型和实现。

您还可以指定实现的时延。对于接口上的块 RAM，latency 选项允许您对接口上的片外非标准 SRAM 进行建模，例如，用于支持时延为 2 或 3 的 SRAM。对于内部运算，latency 选项允许使用多个流水打拍阶段来实现存储器。这些附加的流水线阶段可帮助解决 RTL 综合期间的时序问题。



重要提示！ 要使用 latency 选项，运算必须具有 1 个可用的多阶段实现。HLS 工具可以为所有块 RAM 提供多阶段实现。

语法

将 C/C++ 源代码中的编译指示置于定义变量的函数体内。

```
#pragma HLS bind_storage variable=<variable> type=<type>\
[ impl=<value> latency=<int> ]
```

其中：

- variable=<variable>：定义要将 BIND_STORAGE 编译指示分配到的变量。指定编译指示时，此变量是必需的。



提示： 如果变量是顶层函数实参，则使用 INTERFACE 编译指示或指令的 -storage_type 选项和 -storage_impl 选项。

- type=<type>：定义要绑定到指定变量的存储器的类型。受支持的类型包括：fifo、ram_1p、ram_1wnr、ram_2p、ram_s2p、ram_t2p、rom_1p、rom_2p、rom_npo。

表 40：存储类型

类型	描述
FIFO	FIFO。Vitis HLS 可判定如何在 RTL 中将其实现，除非指定 -impl 选项。
RAM_1P	单端口 RAM。Vitis HLS 可判定如何在 RTL 中将其实现，除非指定 -impl 选项。
RAM_1WNR	含 1 个写入端口和 N 个读取端口的 RAM，内部使用 N 个 bank。
RAM_2P	双端口 RAM，允许在某一端口上执行读操作并在另一个端口上执行读写操作。
RAM_S2P	双端口 RAM，允许在某一端口上执行读取操作，并在另一个端口上执行写入操作。
RAM_T2P	真正的双端口 RAM，支持在 2 个端口上执行读写操作。
ROM_1P	单端口 ROM。Vitis HLS 可判定如何在 RTL 中将其实现，除非指定 -impl 选项。

表 40：存储类型 (续)

类型	描述
ROM_2P	双端口 ROM。
ROM_NP	多端口 ROM。

 **提示：**如果您为 PIPO 指定了单端口 RAM，那么绑定程序通常会分配一个合并的 PIPO，只有一个 bank，读取和写入分别访问 bank 的不同部分。HLS 编译器报告的参考消息会显示 “Implementing PIPO using a single memory for all blocks.”，即，“为所有块使用同一个存储器来实现 PIPO”。对于小型 PIPO 来说，这种方法通常比较经济，因为所有 bank 可以共享单个 RAM 块。但是，如果您为 PIPO 指定了双端口 RAM 以获取更高带宽，而调度器在生产者或使用者中使用了这两个端口，那么绑定程序通常会分配一个拆分 PIPO，即生产者使用 1 个端口，使用者使用不同 bank 的 2 个端口。在此情况下，HLS 编译器报告的参考消息为 “Implementing PIPO using a separate memory for each block.”，即，“为每个块使用一个独立存储器来实现 PIPO”。

· `impl=<value>`：定义指定存储器类型的实现。受支持的实现包括：`bram`、`bram_ecc`、`lutram`、`uram`、`uram_ecc`、`srl`、`memory` 和 `auto`，如下所述。

表 41：受支持的实现

名称	描述
MEMORY	通用存储器，允许 Vivado 工具选择实现。
URAM	UltraRAM 资源
URAM_ECC	含 ECC 的 UltraRAM
SRL	移位寄存器逻辑资源
LUTRAM	分布式 RAM 资源
BRAM	块 RAM 资源
BRAM_ECC	含 ECC 的块 RAM
AUTO	Vitis HLS 会自动判定变量的实现。

表 42：FIFO/RAM/ROM 支持的实现

类型	命令/编译指示	范围	受支持的实现
FIFO	<code>bind_storage</code> ¹	局部	AUTO 、BRAM、BRAM_ECC、LUTRAM、URAM、URAM_ECC、MEMORY、SRL
FIFO	<code>config_storage</code>	全局	AUTO 、BRAM、BRAM_ECC、LUTRAM、URAM、URAM_ECC、MEMORY、SRL
RAM* ROM*	<code>bind_storage</code>	局部	AUTO BRAM、BRAM_ECC、LUTRAM、URAM 和 URAM_ECC
RAM* ROM*	<code>config_storage</code> ²	全局	不适用
RAM_1P	<code>set_directive_interface_s_axilite_storage_impl</code>	局部	AUTO 、BRAM 和 URAM

表 42: FIFO/RAM/ROM 支持的实现 (续)

类型	命令/编译指示	范围	受支持的实现
	<code>config_interface</code> <code>-m_axi_buffer_impl</code>	全局	AUTO、 BRAM 、LUTRAM 和 URAM

注释：

- 如不指定任何实现，该指令会使用 AUTOSRL 行为作为默认实现。但您无法指定该值。
- `config_storage` 仅支持 FIFO 类型。

- `latency=<int>`：定义用于绑定类型的默认时延。如下表所示，有效的时延值因指定的 `type` 和 `impl` 而异。默认值为 -1，即交由 Vitis HLS 选择时延。

表 43: 受支持的存储器类型、实现和时延组合

类型	实现	最小时延	最大时延
FIFO	BRAM	0	0
FIFO	LUTRAM	0	0
FIFO	MEMORY	0	0
FIFO	SRL	0	0
FIFO	URAM	0	0
RAM_1P	AUTO	1	5
RAM_1P	BRAM	1	5
RAM_1P	LUTRAM	1	5
RAM_1P	URAM	1	5
RAM_1WNR	AUTO	1	5
RAM_1WNR	BRAM	1	5
RAM_1WNR	LUTRAM	1	5
RAM_1WNR	URAM	1	5
RAM_2P	AUTO	1	5
RAM_2P	BRAM	1	5
RAM_2P	LUTRAM	1	5
RAM_2P	URAM	1	5
RAM_S2P	BRAM	1	5
RAM_S2P	BRAM_ECC	1	5
RAM_S2P	LUTRAM	1	5
RAM_S2P	URAM	1	5
RAM_S2P	URAM_ECC	1	5
RAM_T2P	BRAM	1	5
RAM_T2P	URAM	1	5
ROM_1P	AUTO	1	5
ROM_1P	BRAM	1	5
ROM_1P	LUTRAM	1	5
ROM_2P	AUTO	1	5
ROM_2P	BRAM	1	5

表 43: 受支持的存储器类型、实现和时延组合 (续)

类型	实现	最小时延	最大时延
ROM_2P	LUTRAM	1	5
ROM_NP	BRAM	1	5
ROM_NP	LUTRAM	1	5

注释：除 ROM 外的所有其他类型都支持 BRAM_ECC 和 URAM_ECC。



提示：对于时延为 0 的 FIFO，含义如下：

- 在 FIFO 输出上的读取数据可用的同一周期内，读取输入处于活动状态。
- 在 FIFO 输入接受写入数据的同一个周期内，写入输入处于活动状态。

以先写后读时延为例，写入数据完成后经过若干周期之后方可从 FIFO 读取该数据，此周期数只能通过以下方式来自全局配置：`config_storage fifo -read_after_write_latency N`



重要提示！上表中未列出的存储器类型与实现的任意组合均不受 `set_directive_bind_storage` 支持。

示例

该编译指示指定 `coeffs` 变量使用来自库的 BRAM 核上实现的单端口 RAM。

```
#pragma HLS bind_storage variable=coeffs type=RAM_1P impl=bram
```



提示：在 RAM_1P 中定义 RTL 中创建的用于访问 `coeffs` 值的端口。

另请参阅

- [set_directive_bind_storage](#)
- [pragma HLS bind_op](#)
- [config_storage](#)

pragma HLS cache

描述

在 FPGA 设计中，对片外 DDR 存储器进行非突发访问的成本很高，而且会使性能劣化。为了缓解该问题，Vitis HLS 在 M_AXI 适配器中支持一种高速缓存机制，通过利用引用的局部性（时间性和空间性）来减少平均访问时延。访问某个存储区域时，很可能很快会再次访问该区域，并且还可能访问附近的位置。

这种高速缓存机制既支持单端口高速缓存，也支持多端口层级：

- 单端口高速缓存：直接映射、只读高速缓存，连接到单个 `m_axi` 端口。这样在内核重复访问同一行时可以减少对 DRAM 的重复读取。

- 多端口高速缓存：一种分层高速缓存，每个端口都有 L1 高速缓存，并共享 L2 高速缓存。它支持在单个时钟周期内对同一个顶层 `m_axi` 指针进行最多 `N` 次读取访问，而无需更改源代码，适用于类似 `stencil` 的访问模式以及编译器无法推断窗口缓冲器的其他动态访问模式。相比于编译器推断的窗口缓冲器，多端口高速缓存更灵活（处理动态模式），但通常会耗用更多的资源。

多端口高速缓存架构

- L2 高速缓存（共享）：
 - 所有端口共享一个直接映射的高速缓存。
 - 通过突发传输从片外 DRAM 加载。
 - 将各行广播到发出请求的 L1 高速缓存，通过避免冗余 DRAM 加载来优化存储器带宽。
- L1 高速缓存（各端口）：
 - 每个高速缓存端口一个直接映射的 L1 高速缓存。
 - 每个 L1 均可在每个周期内提供一个数据项，允许每个周期对同一个 `m_axi` 指针进行最多 `N` 次读取访问。
 - 从共享的 L2 高速缓存加载 L1 行。



提示： 您可使用 `syn.interface.m_axi.cache_impl` 命令控制高速缓存实现资源。

语法

```
pragma HLS cache port=<name> lines=<value> depth=<value>
```

其中：

- `port=<name>`：指定高速缓存要添加到的只读端口（顶层 `m_axi` 指针）。
- `lines=<value>`：每个 L1 高速缓存中的高速缓存行数（针对单端口的情况：唯一的一个高速缓存）。针对单行指定 1，针对多行指定大于 1 的 2 的幂值。可选；如不指定，则默认值为 1。
- `depth=<value>`：每个高速缓存行的大小，以字数为单位，必须为 2 的幂值。适用于指针的元素类型（例如，深度以 `int` 或 `float` 为单位）。可选；默认值为最大突发长度。最大突发长度默认为 16，但可通过 `syn.interface.m_axi_max_read_burst_length` 来全局指定，或通过 `syn.directive.interface` 来按接口指定。

注释： 在多端口高速缓存中，L1 和 L2 缓存共享相同的行深度；深度同时适用于两个级别。

- `ports=<N>`：启用多端口高速缓存功能特性。定义在一个时钟周期内可以针对对应顶层 `m_axi` 指针调度执行的读取访问次数。仅适用于映射到其 `m_axi` 捆绑的只读指针；当端口数量大于 1 时，只能将一个只读阵列映射到该捆绑（请参阅“限制”）。
- `l2_lines=<M>`：可选；默认情况下，L1 和 L2 高速缓存的行数由深度选项来选择。在共享 L2 高速缓存中显式设置行数。每个 L2 行的大小与 L1 行大小相等，并受深度控制。必须大于行数；L2 高速缓存行数必须大于 L1 高速缓存行数。

不含 L2 高速缓存的单端口高速缓存示例：

```
#pragma HLS cache port=A lines=4 depth=32
```

启用每个周期最多 3 次读取的多端口高速缓存，显式设置共享 L2 的大小：

```
#pragma HLS cache port=A lines=4 depth=32 ports=3 l2_lines=16
```

行为与性能注释

- 多端口高速缓存是专为类似 stencil 的访问模式以及无法推断窗口缓冲器的动态访问模式而设计的。它可以在每个周期内允许多个读取请求，同时减少通过共享 L2 的冗余 DRAM 突发，从而提高吞吐量。
- L1 与 L2 高速缓存均为直接映射。命中/未命中行为由局部性决定；增大行数并选择合适的深度可以提高命中率，但会增加资源消耗。
- 共享 L2 高速缓存每次未命中时会加载一个 DRAM 行，并将该行广播到需要它的任意 L1 高速缓存，从而优化与窗口缓冲器类似的带宽，但支持动态模式。

限制

- 仅限只读端口才支持高速缓存。写端口可以与缓存的只读指针共享一个捆绑，因为它们使用的是独立通道，但不支持 inout 输入输出端口。
- 在多端口模式（端口数大于 1）下：
 - 缓存的顶层 m_axi 指针必须是映射到其捆绑的唯一读端口。
 - 写端口也可以映射到同一个捆绑，但 inout 端口对此不予支持。
- 高速缓存在 L1 和 L2 级处均为直接映射（单路）。
- 指定的 l2_lines 值必须大于行数。
- 只能将一个只读阵列映射到多端口缓存指针的 m_axi 捆绑。

其他控制

- syn.interface.m_axi_cache_impl 可用于控制高速缓存实现资源。
- syn.interface.m_axi_max_read_burst_length 和 syn.directive.interface 可通过最大突发长度来控制默认深度。

示例

以下示例显示的设计中，重叠访问会导致突发失败。使用 CACHE 编译指示或指令会改善设计性能。

```
extern "C" {
    void dut(
        const double *in,    // Read-Only Vector 1
        double *out,         // Output Result
        int size             // Size in integer
    )
    #pragma HLS INTERFACE m_axi port=in bundle=aximm depth = 1026
    #pragma HLS INTERFACE m_axi port=out bundle=aximm depth = 1024
    #pragma HLS cache port=in lines=8 depth=128
    for(int i = 0; i < size; i++)
    {
        out[i] = in[i] + in[i + 1];
    }
}
```

另请参阅

- [pragma HLS interface](#)
- [接口配置](#)
- [syn.directive.cache](#)

· [set_directive_cache](#)

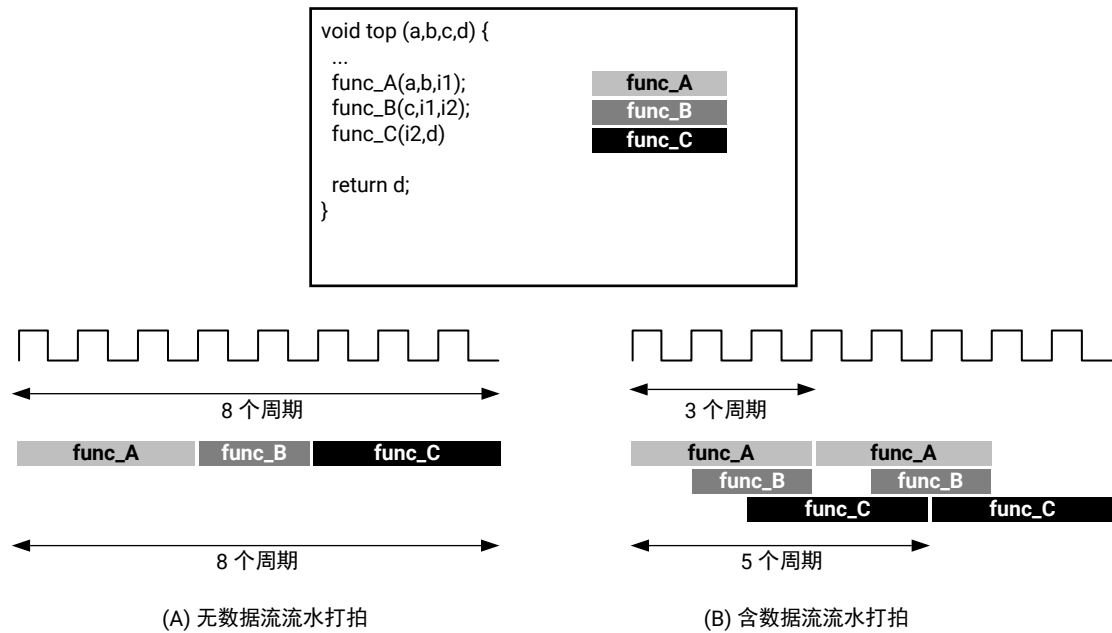
pragma HLS dataflow

描述

DATAFLOW 编译指示可启用任务级流水打拍，允许函数和循环在其操作过程中重叠，增加 RTL 实现的并发度，并增加设计的整体吞吐量。

在 C 语言描述中，所有操作均按顺序执行。如 unlimited 资源的任何指令（如 `pragma HLS allocation`），则 Vitis HLS 工具会尝试最大限度减小时延并提高并发。但是，数据依赖关系可能对此施加限制。例如，访问阵列的函数或循环必须先完成对阵列的所有读写访问后才能完成操作。这样会阻止下一个使用该数据的函数或循环开始操作。DATAFLOW 优化支持函数或循环中的操作在前一个函数或循环尚未完成其所有操作时就开始操作。

图 140：DATAFLOW 编译指示



X14266-030622

当指定 DATAFLOW 编译指示化后，HLS 工具会分析顺序函数或循环之间的数据流，并基于乒乓 RAM 或 FIFO 创建通道，以允许使用者 (consumer) 函数或循环在生产者 (producer) 函数或循环完成前即开始操作。这样即可允许函数或循环并行操作，从而降低时延，并提升 RTL 吞吐量。



提示： `config_dataflow` 命令可用于指定数据流优化中使用的默认存储器通道和 FIFO 深度。

要使 DATAFLOW 优化正常工作，数据必须逐一流经设计中的每个任务。请务必留意下列情况。如需了解更多详细信息，请参阅 [控制驱动的任务级并行度的限制](#) 和 [数据流区域编码风格](#)。

- 单一生产者使用者违例
- 任务间的反馈

- 任务的有条件执行
- 含多个退出条件的循环



重要提示！ 如果存在上述任一编码样式，那么 HLS 工具会发出一条消息。

最后，DATAFLOW 优化默认不加以分层。如果子函数或循环包含可能受益于 DATAFLOW 优化的其他任务，那么必须对该循环、子函数或子函数的内联应用 DATAFLOW 优化。

语法

将 C 语言源代码中的编译指示置于区域、函数或循环的边界内。

```
#pragma HLS dataflow [disable_start_propagation]
```

- `disable_start_propagation`：（可选）禁用起始 FIFO 的创建，起始 FIFO 用于向内部进程传输起始令牌。此类 FIFO 有时可能会成为性能瓶颈。

示例

在 `wr_loop_j` 循环内指定 DATAFLOW 优化。

```
void read_fifo(int tile[TILE_HEIGHT][TILE_WIDTH], hls::stream<int> &inFifo)
{
    for (int m = 0; m < TILE_HEIGHT; ++m) {
        for (int n = 0; n < TILE_WIDTH; ++n) {
            #pragma HLS PIPELINE
            tile[m][n] = inFifo.read();
        }
    }
}

void write_out(int tile[TILE_HEIGHT][TILE_WIDTH], int *outx, int I, int j) {
    for (int m = 0; m < TILE_HEIGHT; ++m) {
        for (int n = 0; n < TILE_WIDTH; ++n) {
            #pragma HLS PIPELINE
            outx[TILE_HEIGHT*TILE_PER_ROW*TILE_WIDTH*i
+TILE_PER_ROW*TILE_WIDTH*m+TILE_WIDTH*j+n] = tile[m][n];
        }
    }
}

...
wr_loop_i:
for (int i = 0; i < TILE_PER_COL; ++i) {
    wr_loop_j:
    for (int j = 0; j < TILE_PER_ROW; ++j) {
        #pragma HLS DATAFLOW
        int tile[TILE_HEIGHT][TILE_WIDTH];
        read_fifo(tile, inFifo);
        write_out(tile, outx, i, j);
    }
}
```

另请参阅

- [set_directive_dataflow](#)
- [config_dataflow](#)

- [pragma_HLS_allocation](#)
- [pragma_HLS_pipeline](#)

pragma HLS dependence

描述

Vitis HLS 会检测循环内的依赖关系：循环的相同迭代内的依赖关系为独立于循环的依赖关系，循环的不同迭代之间的依赖关系则为循环进位依赖关系。DEPENDENCE 编译指示允许您提供额外信息用于定义、抵消循环依赖关系，并允许循环以更低的时间间隔来进行流水打拍。

- 独立于循环的依赖关系：在单个循环迭代内访问相同元素。

```
for (i=1;i<N;i++) {  
    A[i]=x;  
    y=A[i];  
}
```

- 循环进位依赖关系：从不同循环迭代内访问相同元素。

```
for (i=1;i<N;i++) {  
    A[i]=A[i-1]*2;  
}
```

这些依赖关系会影响运算的调度时间，尤其是在函数和循环流水打拍期间。

在诸如建立变量相关阵列索引或需强制实施外部要求（例如，2 项输入从不采用相同索引）之类的情况下，依赖关系分析可能过于保守，无法滤除假性依赖关系。DEPENDENCE 编译指示允许您显式定义依赖关系并消除假性依赖关系。

依赖关系编译指示也适用于函数调用实参：

- Inter：连续调用之间的依赖关系
- Intra：单次调用内部的依赖关系

```
void foo(int a[3], int x) {  
    #pragma HLS pipeline  
    #pragma HLS dependence variable=a inter true distance=3  
    a[x] += ...;  
}  
  
void dut(int a[3], int x, ...) {  
    #pragma HLS dataflow  
    foo(a, x);  
    bar(...);  
}
```



重要提示！ 如果实际上依赖关系并非假性 (false) 关系，那么指定假性依赖关系可能导致硬件错误。指定依赖关系前，请确认它是否正确 (true 或 false)。

语法

将编译指示置于函数边界内定义依赖关系的位置。

```
#pragma HLS dependence variable=<variable> <class> \
<type> <direction> distance=<int> <dependent>
```

其中：

- `variable=<variable>`：（可选）指定为了确认依赖关系而需要考量的变量。



重要提示！ 对于与 `m_axi` 接口内其他实参捆绑的函数实参，不能指定 `dependence`。这是函数上的 `m_axi` 接口的默认配置。并且除非结构体已解聚，否则也不能为结构体的元素指定依赖关系。

- `class=[array | pointer]`：（可选）指定需要澄清其中所含依赖关系的变量的类。有效值包括 `array` 或 `pointer`。



提示： `<class>` 和 `variable=` 不应一起指定，因为您可以在函数内为某一个变量或者某一类变量指定依赖关系。

- `type=[inter | intra]`：有效值包括 `intra` 或 `inter`。指定依赖关系：
 - `intra`：相同循环迭代内的依赖关系。当依赖关系 `<type>` 指定为 `intra` 并且 `<dependent>` 为 `false` 时，HLS 工具可在循环内自由移动操作，从而增大其可移动性并潜在改善性能或面积。当 `<dependent>` 指定为 `true` 时，必须按指定顺序执行操作。
 - `inter`：不同循环迭代间的依赖关系。这是默认 `<type>`。如果依赖关系 `<type>` 指定为 `inter`，并且 `<dependent>` 为 `false`，那么它允许 HLS 工具并行执行多项操作，前提是函数或循环采用流水打拍或者循环已展开或部分展开，当 `<dependent>` 指定为 `true` 时，则阻止此类并发操作。
- `direction=[RAW | WAR | WAW]`：仅与循环进位依赖关系有关，用于指定依赖关系的方向：
 - `RAW`（先写后读 - 真性依赖关系）：写指令使用的值供读指令使用。
 - `WAR`（先读后写 - 反依赖关系）：读指令获取的值被写指令覆盖。
 - `WAW`（先写后写 - 输出依赖关系）：2 条写指令按顺序写入相同位置。
- `distance=<int>`：指定阵列访问的迭代间距离。仅与循环进位依赖关系有关，其中依赖关系设为 `true`。
- `<dependent>`：指定 `<dependent>` 实参可用于指示依赖关系如为 `true` 则需要强制实施，如为 `false` 则应移除。但如果不指定该实参，则工具将返回警告，称未指定该值，并且假定值为 `false`。接受的值为 `true` 或 `false`。

示例 1

在以下示例中，HLS 工具不知晓 `cols` 的值，并且保守假定在写入 `buff_A[1][col]` 与读取 `buff_A[1][col]` 之间始终存在依赖关系。在诸如此类的算法中，`cols` 几乎不可能为 0，但 HLS 工具无法就数据依赖关系做出任何判断。为了克服这一缺陷，您可使用 `DEPENDENCE` 编译指示来声明在循环迭代之间（在此示例中，针对 `buff_A` 和 `buff_B`）没有依赖关系。

```
void foo(int rows, int cols, ...){
    for (row = 0; row < rows + 1; row++) {
        for (col = 0; col < cols + 1; col++) {
            #pragma HLS PIPELINE II=1
            #pragma HLS dependence variable=buff_A type=inter false
            #pragma HLS dependence variable=buff_B type=inter false
            if (col < cols) {
```

```

        buff_A[2][col] = buff_A[1][col]; // read from buff_A[1][col]
        buff_A[1][col] = buff_A[0][col]; // write to buff_A[1][col]
        buff_B[1][col] = buff_B[0][col];
    }
}
}
}

```

示例 2

移除 func 函数的 loop_1 的相同迭代中的 Var1 之间的依赖关系。

```
#pragma HLS dependence variable=Var1 type=intra false
```

示例 3

在 func 函数的 loop_2 中的所有阵列上定义依赖关系，以便告知 HLS 工具，在相同循环迭代内，所有读操作都必须发生在写操作之后 (RAW)。

```
#pragma HLS dependence class=array type=intra direction=RAW true
```

另请参阅

- [set_directive_dependence](#)
- [pragma_HLS_disaggregate](#)
- [pragma_HLS_pipeline](#)

pragma HLS disaggregate

描述

DISAGGREGATE 编译指示允许您按 struct 变量所含各独立元素来对其进行解构。创建的元素数量和类型取决于结构体本身的内容。



重要提示！ 默认情况下，作为顶层函数实参的结构体将聚合在一起，但可通过该指令或编译指示来对其进行解聚。如需了解有关如何对与串流关联的结构体进行解聚的更多信息，请参阅 [AXI4-Stream 接口](#)。

语法

将 C 语言源代码中的编译指示置于区域、函数或循环的边界内。

```
#pragma HLS disaggregate variable=<variable>
```

选项

其中：

- `variable=<variable>`：指定要解聚的结构体变量。

示例 1

以下示例显示函数 `top` 中的结构体变量 `a` 将进行解聚：

```
#pragma HLS disaggregate variable=a
```

示例 2

解聚后的结构体可在您的代码中使用标准 C/C++ 编码样式来进行寻址，如下所示。请注意访问指针元素 (a) 与访问参考元素 (c) 的方法差异；

```
struct SS
{
    int x[N];
    int y[N];
};

int top(SS *a, int b[4][6], SS &c) {
#pragma HLS disaggregate variable = a
#pragma HLS interface s_axilite port = a->x
#pragma HLS interface s_axilite port = a->y

// Following is now supported
#pragma HLS disaggregate variable = c
#pragma HLS interface ap_memory port = c.x
#pragma HLS interface ap_memory port = c.y
```

示例 3

以下示例显示的 `Dot` 结构体内包含 `RGB` 结构体作为元素。如果您将 `DISAGGREGATE` 编译指示应用于 `Arr` 变量，则仅对顶层 `Dot` 结构体进行解聚。

```
struct Pixel {
    char R;
    char G;
    char B;
};

struct Dot {
    Pixel RGB;
    unsigned Size;
};

#define N 1086
void DUT(Dot Arr[N]) {
#pragma HLS disaggregate variable=Arr
...
}
```

如果您要将整个结构体（包括 `Dot` 和 `RGB`）进行解聚，则可按如下所示对 `DISAGGREGATE` 编译指示进行赋值。

```
void DUT(Dot Arr[N]) {
#pragma HLS disaggregate variable=Arr->RGB
...
}
```

此例中的结果为：

```
void DUT(char Arr_RGB_R[N], char Arr_RGB_G[N], char Arr_RGB_B[N], unsigned
Arr_Size[N]) {
#pragma HLS disaggregate variable=Arr->RGB
...
}
```

另请参阅

- [set_directive_disaggregate](#)
- [pragma_HLS_aggregate](#)

pragma HLS expression_balance

描述

有时，C/C++ 代码是以一连串运算来编写的，故而会在 RTL 中生成一长串运算链。如果时钟周期较短，这就可能增加设计中的时延。默认情况下，Vitis HLS 工具会使用结合律和交换律来重新组织运算。通过重新组织可以创建平衡的树结构以缩短运算链，从而以增加硬件为代价来缩短设计时延。

表达式平衡会重新排列运算符以构造平衡的树结构并降低时延。

- 对于整数运算，默认情况下表达式平衡处于开启状态，但可将其禁用。
- 对于浮点运算，默认情况下表达式平衡处于关闭状态，但可将其启用。

EXPRESSION_BALANCE 编译指示允许在指定作用域内禁用或明确启用此表达式平衡。

语法

将 C 语言源代码中的编译指示置于所需位置的边界内。

```
#pragma HLS expression_balance off
```

其中：

- **off**：在此位置关闭表达式平衡。指定 `#pragma HLS expression_balance` 即可在指定作用域内启用表达式平衡。添加 **off** 则会将其禁用。

示例 1

在 `my_Func` 函数内禁用表达式平衡。

```
void my_func(char inval, char incr) {
#pragma HLS expression_balance off
```

示例 2

此示例会在 `my_func` 函数内显式启用表达式平衡。

```
void my_func(char inval, char incr) {  
    #pragma HLS expression_balance
```

另请参阅

- [set_directive_expression_balance](#)

pragma HLS function_instantiate

描述

FUNCTION_INSTANTIATE 编译指示化是一种优化技巧，不仅具有维持函数层级的面积优势，还可提供另一个强大的选项：在函数的特定实例上执行针对性的局部优化。这样可以简化围绕函数调用的控制逻辑，也可能改进时延和吞吐量。

默认情况下：

- 函数在 RTL 中作为独立层级块保留，或者分解（或内联）到更高层次的函数中。
- 同一层级上的任一函数的所有实例都使用单一 RTL 实现（块）。

FUNCTION_INSTANTIATE 编译指示用于为函数的每个实例创建唯一的 RTL 实现，允许根据函数调用对每个实例进行局部优化。鉴于调用函数时部分函数输入可能是常量，此编译指示可藉此简化周围控制结构，并生成进一步优化的、更小的函数块。

如不使用 FUNCTION_INSTANTIATE 编译指示，则以下代码会为 `func` 中的函数的全部 3 个实例生成函数 `func_sub` 的单一 RTL 实现。函数 `func_sub` 的每个实例都是以相同方式实现的。这对于函数复用并无影响，并且可以减少函数的每次实例调用所需的面积，但是函数内部的控制逻辑必须更复杂，以便应对每次调用 `func_sub` 时产生的变化。

```
char func_sub(char inval, char incr) {  
    #pragma HLS INLINE OFF  
    #pragma HLS FUNCTION_INSTANTIATE variable=incr  
    return inval + incr;  
}  
void func(char inval1, char inval2, char inval3,  
          char *outval1, char *outval2, char * outval3)  
{  
    *outval1 = func_sub(inval1, 1);  
    *outval2 = func_sub(inval2, 2);  
    *outval3 = func_sub(inval3, 3);  
}
```



提示：Vitis HLS 工具会将小函数自动分解（或内联）到更高层次的调用函数中。即使对于函数例化也同样如此。将 `INLINE` 编译指示与 `OFF` 选项搭配使用即可阻止此自动内联操作。

在以上代码样本中，FUNCTION_INSTANTIATE 编译指示会生成函数 `func_sub` 的 3 个不同实现，每个实现都会按 `incr` 指定值进行优化，从而减少面积并改善函数实现的性能。

语法

将 C 语言源代码中的编译指示置于所需位置的边界内。

```
#pragma HLS FUNCTION_INSTANTIATE variable=<variable>
```

其中：

- `variable=<variable>`：这是必需的实参，用于定义要用作为常量的函数实参。

示例

在以下示例中，布局在函数 `swInt` 内的 `FUNCTION_INSTANTIATE` 编译指示允许按 `maxv` 函数实参来对函数 `swInt` 的每个实例进行独立优化。

```
void swInt(unsigned int *readRefPacked, short *maxr, short *maxc, short *maxv){
#pragma HLS FUNCTION_INSTANTIATE variable=maxv
    uint2_t d2bit[MAXCOL];
    uint2_t q2bit[MAXROW];
#pragma HLS array partition variable=d2bit,q2bit cyclic factor=FACTOR

    intTo2bit<MAXCOL/16>((readRefPacked + MAXROW/16), d2bit);
    intTo2bit<MAXROW/16>(readRefPacked, q2bit);
    sw(d2bit, q2bit, maxr, maxc, maxv);
}
```

另请参阅

- [set_directive_function_instantiate](#)
- [pragma HLS allocation](#)
- [pragma HLS inline](#)

pragma HLS inline

描述

移除层级中作为独立实体的函数。完成内联后，函数将消隐到调用函数内，不再显示为 RTL 中的层级的独立层次。



重要提示！ 内联函数时也会将应用于该函数的所有编译指示或指令消隐掉。在 Vitis HLS 中，忽略应用于内联函数的任意编译指示或指令。

在某些情况下，将函数内联即可使函数内的运算以更有效的方式与调用函数进行共享和优化。但内联函数无法共享或复用，因此如果父函数多次调用内联函数，则可能导致增加 RTL 实现所需的面积。

根据指定 `INLINE` 编译指示的方式不同，在定义该编译指示的作用域中，它会以不同方式应用于该作用域：

- `INLINE`：如不指定实参，那么该编译指示表示，指定该编译指示的函数应向上内联到任意调用函数内。
- `INLINE off`：该编译指示表示，指定该编译指示的函数不应向上内联到任意调用函数内。这表示禁用特定函数的内联操作，此类特定函数原本可能自动内联或者在递归过程中内联。

- `INLINE recursive`：表示将该编译指示应用于它分配到的函数体。它向下应用，以递归方式内联该函数的内容。

默认情况下，只能对函数层级的下一级执行内联，而不能对子函数执行内联。但 `recursive` 选项允许您指定穿过层级的多个层次进行内联。

语法

将 C 语言源代码中的编译指示置于函数或代码区域的主体内。

```
#pragma HLS inline <off> <recursive>
```

其中：

- `recursive`：默认情况下，仅执行一级函数内联，并且不内联指定函数内部的函数。`recursive` 选项会在指定函数或区域内按递归方式内联所有函数。



重要提示！ 在数据流区域中执行函数递归内联可能导致该区域内的单个函数不满足数据流的条件。在此情况下，会忽略 `INLINE recursive` 编译指示或指令。

- `off`：禁用函数内联以防止指定函数发生内联。例如，如果在函数中指定 `recursive`，该选项会阻止调用的特定函数进行内联，同时所有其他函数均可进行内联。



提示： Vitis HLS 工具会自动内联小型函数，并将 `INLINE` 编译指示与 `off` 选项搭配使用以阻止此自动内联操作。

示例 1

以下示例用于对 `func_top` 主体内部的所有函数执行内联，以递归方式向下内联穿越整个函数层级，但 `func_sub` 函数不执行内联。递归编译指示置于 `func_top` 函数内。用于禁用内联的编译指示则置于 `func_sub` 函数内：

```
func_sub (p, q) {
#pragma HLS inline off
int q1 = q + 10;
func(p1,q);// foo_3
...
}
void func_top { a, b, c, d} {
#pragma HLS inline recursive
...
func(a,b);//func_1
func(a,c);//func_2
func_sub(a,d);
...
}
```



提示： 请注意，在此示例中，`INLINE RECURSIVE` 向下应用于 `func_top` 函数内容，但 `INLINE OFF` 则直接应用于 `func_sub`。



提示： 使用带递归的 `off` 可以保留添加该编译指示的层级，但会平铺该层级之下的任何内容。

示例 2

此示例用于将 `copy_output` 函数内联到调用 `copy_output` 的任意函数或区域内。

```

void copy_output(int *out, int out_lcl[OSize * OSize], int output) {
#pragma HLS INLINE
    // Calculate each work_item's result update location
    int stride = output * OSize * OSize;

    // Work_item updates output filter/image in DDR
    writeOut: for(int itr = 0; itr < OSize * OSize; itr++) {
#pragma HLS PIPELINE
        out[stride + itr] = out_lcl[itr];
    }
}

```

另请参阅

- [set_directive_inline](#)
- [pragma_HLS_allocation](#)

pragma HLS interface

描述

仅支持对顶层函数使用 `INTERFACE` 编译指示或指令，无法将其用于 HLS 组件的子函数。它可指定在接口综合期间如何根据函数实参创建 RTL 端口，如 [定义接口](#) 所述。Vitis HLS 工具会自动确定任何子函数使用的 I/O 协议。

RTL 实现中的端口衍生自顶层函数和函数返回的实参的数据类型和方向、HLS 组件的 `flow_target`、[config_interface](#) 及 `INTERFACE` 编译指示或指令所指定的默认接口配置设置。每个函数实参均可指定为包含其自己的 I/O 协议（例如，有效握手或确认握手）。



提示：接口上必需的全局变量必须显式定义为顶层函数的实参，如 [全局变量](#) 中所述。如果访问全局变量，但所有读写操作均为设计的本地操作，则会在该设计中创建此资源。RTL 中无需 I/O 端口。

该接口还定义了 HLS 组件的执行控制协议，如 [块级控制协议](#) 中所述。控制协议用于控制 HLS 组件（或块）何时开始执行，以及该块在完成操作后何时处于空闲状态并准备就绪以接收新输入。

语法

将编译指示布局在函数边界内。

```
#pragma HLS interface mode=<mode> port=<name> direct_io=<value>[OPTIONS]
```

其中：

- `mode=<mode>`:

受支持的模式以及该工具在 RTL 中实现这些模式的方式均可细分为如下 3 个类别：

1. 端口级协议：

- `ap_none`：无端口协议。此接口是简单的数据端口。

- `ap_vld`：用于实现含关联 `valid` 信号的数据端口，以指示何时数据有效且可供读取或写入。
- `ap_ack`：用于实现含关联 `acknowledge` 信号的数据端口，以确认数据已读取或写入。
- `ap_hs`：用于实现同时包含 `valid` 信号和 `acknowledge` 信号的数据端口，提供两路握手以指示数据有效且可供读取和写入，并确认数据已读取或写入。
- `ap_ovld`：用于实现含关联 `valid` 信号的输出数据端口，以指示何时数据有效且可供读取或写入。



提示：对于 `ap_ovld`，Vitis HLS 通过 `ap_none` 模式来实现输入实参或任意读取/写入实参的输入部分。

- `direct_io=<true/false>`：针对 INTERFACE 应用到的函数实参，指定值为 `true`。请参阅 [将直接 I/O 串流映射到 SAXI Lite 接口](#) 章节以获取该选项的用例。
- `ap_memory`：用于实现阵列实参（作为标准 RAM 接口）。如果在 Vivado IP integrator 中使用 RTL 设计，那么该接口将由多个不同端口组成。
- `bram`：用于实现阵列实参（作为标准 RAM 接口）。如果在 Vivado IP integrator 中使用 RTL 设计，那么该存储器接口含单个端口。
- `ap_fifo`：使用含关联低电平有效 FIFO `empty` 端口和 `full` 端口的数据输入和输出端口来实现含标准 FIFO 接口的端口。

注释：您只能对读取实参或写入实参使用 `ap_fifo` 接口。`ap_fifo` 模式不支持双向读写实参。

2. AXI 接口协议：

- `s_axilite`：用于将该端口作为 AXI4-Lite 接口来实现。为 HLS 组件导出生成的 RT 时，该工具会生成一组关联的 C 语言驱动程序文件。
- `m_axi`：用于将该端口作为 AXI4 接口来实现。您可使用 `config_interface -m_axi_addr64` 命令来指定 32 位（默认）地址端口或 64 位地址端口，并控制任何地址偏移。
- `axis`：用于将该端口作为 AXI4-Stream 接口来实现。

3. 块级控制协议：

- `ap_ctrl_chain`：实现块级控制端口以启动 (`start`) 设计操作、`continue` 操作，以及指示设计何时处于 `idle`、`done` 和 `ready` 状态，以便处理新输入数据。
- `ap_ctrl_hs`：实现块级控制端口以启动 (`start`) 设计操作，并指示设计何时处于 `idle`、`done` 和 `ready` 状态，以便处理新输入数据。
- `ap_ctrl_none`：无块级 I/O 协议。

注释：使用 `ap_ctrl_none` 模式可阻止使用 C/RTL 协同仿真来验证设计。

- `port=<name>`：用于指定 INTERFACE 编译指示所应用到的函数实参或函数返回的名称。



提示：块级 I/O 协议 (`ap_ctrl_none`、`ap_ctrl_hs` 或 `ap_ctrl_chain`) 可分配到端口，以供函数 `return` 值使用。

选项



提示：以下指定的许多选项都具有 `config_interface` 命令中定义的全局值。为此处定义的接口设置局部值即可覆盖全局值。

- `bundle=<string>`：默认情况下，HLS 工具会将函数实参与兼容选项组合或捆绑到 RTL 中的接口端口内。所有 AXI4-Lite (`s_axilite`) 接口都会尽可能捆绑到 AXI4-Lite 端口内。同样，默认指定为 AXI4 (`m_axi`) 接口的所有函数实参也都会捆绑到单一 AXI4 端口内。含兼容选项（例如，`mode`、`offset` 和 `bundle`）的所有接口端口都组合到单一接口端口内。端口名称自动衍生自模式与捆绑的组合，或者可通过 `-name` 来指定名称。



重要提示！ 指定 `bundle` 名称时，应使用全小写字母。

- `channel=<string>`: 要在 `m_axi` 接口上启用多个通道，请指定通道 ID。您可使用不同的通道 ID 将多个 `m_axi` 接口组合为单个 `m_axi` 适配器。
- `clock=<name>`: 默认情况下，AXI4-Lite 接口时钟与系统时钟为相同时钟。该选项用于为 AXI4-Lite 接口指定独立时钟。如果使用 `-bundle` 选项将多个顶层函数实参组合到单一 AXI4-Lite 接口中，那么只需在其中一个捆绑成员上指定时钟选项即可。
- `depth=<int>`: 指定供测试平台处理的最大采样数。此设置用于指示 HLS 工具为 RTL 协同仿真所创建的验证适配器中所需 FIFO 的最大大小。基于顶层端口实参，该选项接受简单的表达式，例如：

```
int dut(int *arr, int height, int width)
{
#pragma HLS interface m_axi port=arr depth=height*width
...
}
```



提示： 虽然 `depth` 选项通常为可选，但需要通过该选项来为 RTL 协同仿真指定指针实参大小。

- `interrupt=<int>`: 仅供 `ap_vld/ap_hs` 使用。该选项用于启用要在中断中管理的 I/O，方法是在 `s_axilite` 寄存器文件的 `ISR` 和 `IER` 中创建对应的位元。整数 `N=16..31` 用于指定这两个寄存器中的位元位置（默认从 16 开始分配连续的值）。
- `latency=<value>`: 该选项可用于 `ap_memory` 和 `M_AXI` 接口。
 - 在 `ap_memory` 接口中，接口选项用于指定驱动接口的 RAM 资源的读取时延。默认情况下，使用 1 个时钟周期的读取操作。该选项允许对读取时延超过 1 个时钟周期的外部 RAM 进行建模。
 - 在 `M_AXI` 接口中，该选项用于指定 AXI4 接口的期望时延，允许设计发起总线请求的时间比执行期望的读取或写入操作早 `<value>` 个周期（时延）。如果该值太低，设计将过早达成就绪状态，可能停滞并等待总线；如果该值太高，总线访问可能处于空闲状态并等待设计发起访问。
- `max_read_burst_length=<int>`: 该选项适用于 `M_AXI` 接口，可指定突发传输期间读取的数据值的最大数量。如需了解更多信息，请参阅 [AXI 突发传输](#)。
- `max_write_burst_length=<int>`:
 - 对于 AXI4 (`m_axi`) 接口，该选项用于指定突发传输期间写入的数据值的最大数量。



提示： 如果此端口为只读端口，则请设置 `num_write_outstanding=1` 和 `max_write_burst_length=2` 以节省存储器资源。对于仅限写入的端口，请设置 `num_read_outstanding=1` 和 `max_read_burst_length=2`。

- `max_widen_bitwidth=<int>`: 自动拓宽接口时，请指定接口可用的最大位宽。此设置将覆盖由 `config_interface -m_axi_max_bitwidth` 命令指定的全局值。
- `name=<string>`: 指定将在生成的 RTL 中使用的端口的名称。
- `num_read_outstanding=<int>`: 对于 AXI4 (`m_axi`) 接口，该选项用于指定在设计停滞前可对 AXI4 总线发起的读取请求数量（无响应）。此操作暗示设计中的内部存储空间，且 FIFO 大小为：

```
num_read_outstanding*max_read_burst_length*word_size
```

- `num_write_outstanding=<int>`: 对于 AXI4 (`m_axi`) 接口, 该选项用于指定在设计停滞前可对 AXI4 总线发起的写入请求数量 (无响应)。此操作暗示设计中的内部存储空间, 且 FIFO 大小为:

```
num_write_outstanding*max_write_burst_length*word_size
```

- `offset=<string>`: 为指定端口控制 AXI4-Lite (`s_axilite`) 和 AXI4 存储器映射 (`m_axi`) 接口中的地址偏移。
 - 在 `s_axilite` 接口中, `<string>` 用于指定寄存器映射中的地址。
 - 在 `m_axi` 接口中, 该选项会覆盖 `config_interface -m_axi_offset` 选项所指定的全局选项, 并且 `<string>` 指定为:
 - `off`: 不生成偏移端口。
 - `direct`: 生成标量输入偏移端口。
 - `slave`: 生成偏移端口并自动将其映射到 AXI4-Lite 从接口。这是默认偏移。
- `autorestart`: 启用在 AXI4-Lite 适配器中生成 `auto-restart` 计数器的操作。

当该选项设为“true”时, 编译后的内核必须通过测试平台中的 `hls::autorestart` 类来驱动。该选项适用于以下使用约束:

- `mode` 必须设为 `s_axilite`
- `port` 必须设为 `return`
- 内核的顶层协议必须设为 `ap_ctrl_hs`
- `register`: 此可选关键字可用于寄存信号和任何相关协议信号, 并导致保持直至至少完成函数执行的最后一个周期为止。 `config_interface` 命令的 `-register_io` 选项用于全局控制在顶层函数上寄存所有输入/输出的操作。该选项适用于以下接口模式:
 - `ap_fifo`
 - `ap_none`
 - `ap_hs`
 - `ap_ack`
 - `ap_vld`
 - `ap_ovld`



提示: 不支持在函数的返回端口 (`port=return`) 上使用寄存器选项。请改为使用 LATENCY 编译指示或指令:

```
#pragma HLS LATENCY min=1 max=1
```

- `register_mode=<forward|reverse|both|off>`: 该选项适用于 AXI4-Stream 接口, 并用于指定寄存器布局到正向路径 (TDATA 和 TVALID)、反向路径 (TREADY) 或同时布局到这两条路径 (TDATA、TVALID 和 TREADY) 上, 或者不寄存任何端口信号 (`off`)。默认值为 `both`。AXI4-Stream 旁路信号被视为数据信号, 随 TDATA 一起寄存。
- `storage_impl=<impl>`: 仅限搭配 `s_axilite` 一起使用。该选项可定义要分配给接口的存储实现。受支持的实现值包括 `auto`、`bram` 和 `uram`。默认值为 `auto`。



提示: `uram` 是同步存储器, 对于两个端口仅含单个时钟。因此, 针对含第二个时钟的 `s_axilite` 适配器不能指定 `uram`。

- `storage_type=<value>`: 仅限搭配 `ap_memory` 接口一起使用。该选项用于指定要分配给变量的存储类型（即，RAM_T2P）。受支持的类型包括：`ram_1p`、`ram_1wnr`、`ram_2p`、`ram_s2p`、`ram_t2p`、`rom_1p`、`rom_2p` 和 `rom_np`。



提示：对于不在接口上的对象，也可以使用 `BIND_STORAGE` 编译指示或指令来指定该选项。

示例 1

在此示例中，两个函数实参都是使用 AXI4-Stream 接口来实现的：

```
void example(int A[50], int B[50]) {
    //Set the HLS native interface types
    #pragma HLS INTERFACE mode=axis port=A
    #pragma HLS INTERFACE mode=axis port=B
    int i;
    for(i = 0; i < 50; i++){
        B[i] = A[i] + 5;
    }
}
```

示例 2

以下设置会关闭块级 I/O 协议，并分配至函数返回值：

```
#pragma HLS interface mode=ap_ctrl_none port=return
```

指定函数实参 `InData` 以使用 `ap_vld` 接口，并指示应寄存输入：

```
#pragma HLS interface mode=ap_vld register port=InData
```

示例 3

此示例定义了顶层 `transpose` 函数的端口的 `INTERFACE` 标准。请注意，其中使用了 `bundle=` 选项来对信号进行分组。

```
// TOP LEVEL - TRANSPOSE
void transpose(int* input, int* output) {
    #pragma HLS INTERFACE mode=m_axi port=input offset=slave bundle=gmem0
    #pragma HLS INTERFACE mode=m_axi port=output offset=slave bundle=gmem1

    #pragma HLS INTERFACE mode=s_axilite port=input bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=output bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=return bundle=control

    #pragma HLS dataflow
}
```

另请参阅

- [set_directive_interface](#)
- [pragma_HLS_bind_storage](#)

pragma HLS latency

描述

指定最小时延值或最大时延值，用于完成函数、循环和区域。

- 时延：生成输出所需的时钟周期数。
- 函数时延：函数计算并返回所有输出值所需的时钟周期数。
- 循环时延：执行所有循环迭代的周期数。

Vitis HLS 始终尝试将设计中的时延最小化。指定 LATENCY 编译指示时，工具行为如下所示：

- 当时延大于最小值时，或者小于最大值时：满足约束。不再执行进一步优化。
- 当时延小于最小值时：如果 HLS 工具可以实现小于最小指定值的时延，那么它可将时延扩展至指定值，这样可能增加共享。
- 当时延大于最大值时：如果 HLS 工具无法调度到最大限值范围内，那么它会尽力实现指定约束。如果仍无法满足最大时延，则会发出警告，并以超出最大限值前提下可实现的最小时延来生成设计。



提示：您还可以使用 LATENCY 编译指示来限制该工具寻找最优解决方案的范围。为代码中的作用域（包括循环、函数或区域）指定时延约束即可减少该作用域内的可能的解决方案数量，从而改善工具运行时间。如需了解更多信息，请参阅 [改善综合运行时间和容量](#)。

如果要限制所有循环迭代的总时延，应将时延指令应用于包含整个循环的区域，如下示例所示：

```
Region_All_Loop_A: {  
#pragma HLS latency max=10  
Loop_A: for (i=0; i<N; i++)  
{  
    ..Loop Body...  
}  
}
```

在此情况下，即使展开循环，时延指令仍会对所有循环操作设置最大限制。

如果 Vitis HLS 无法满足最大时延约束，它会放宽时延约束，并尝试尽可能实现最佳结果。

如果设置最小时延约束，并且 Vitis HLS 生成的设计时延低于所需的最小值，它会插入虚拟时钟周期以满足最小时延。

语法

请在代码的函数、循环或区域的边界内必须对时延加以管理的位置处布局该编译指示。

```
#pragma HLS latency min=<int> max=<int>
```

其中：

- min=<int>：（可选）指定代码的函数、循环或区域的最小时延。
- max=<int>：（可选）指定代码的函数、循环或区域的最大时延。

注释：虽然最小值和最大值均描述为可选，但必须至少指定二者之一。

示例 1

指定 `foo` 函数采用最小时延值 4 和最大时延值 8。

```
int foo(char x, char a, char b, char c) {
    #pragma HLS latency min=4 max=8
    char y;
    y = x*a+b+c;
    return y
}
```

示例 2

如下示例所示，`loop_1` 指定为采用最大时延 12。将编译指示置于循环主体内，如图所示。

```
void foo (num_samples, ...) {
    int i;
    ...
    loop_1: for(i=0;i< num_samples;i++) {
        #pragma HLS latency max=12
        ...
        result = a + b;
    }
}
```

示例 3

以下示例会创建代码区域和分组信号，这些区域和信号需要在同一时钟周期内通过指定 0 时延来进行更改。

```
// create a region { } with a latency = 0
{
    #pragma HLS LATENCY max=0 min=0
    *data = 0xFF;
    *data_vld = 1;
}
```

另请参阅

- [set_directive_latency](#)

pragma HLS loop_flatten

描述

允许将内层嵌套循环折叠（平铺）为单个循环，以便在循环的所有迭代上应用流水打拍，从而实现更低的时延。

只能对最内层的循环（完成可能的内部展开后）进行流水打拍。外层循环只能是数据流循环或顺序循环。当流水打拍循环上层的循环为顺序循环时，其迭代会依次执行，而且每次迭代，就会完全执行一次内层循环。在 RTL 实现中，从外层循环移动到内层循环需要一个时钟周期，从内层循环移回外层循环需要一个时钟周期，并且中间要加上完成内层循环的所有迭代的完整时延。与此相反，平铺嵌套循环可以将其作为单个循环来进行优化和流水打拍，这样外层循环的迭代与内层循环不同调用的迭代之间就可能发生重叠（流水打拍）。总之，平铺可以提高性能，但是：

- 平铺并非总是可行，需要遵循一定的编码风格。

- 在某些情况下，根据外层循环中的运算和/或依赖关系，时序甚至 II 都可能会劣化。

将 `LOOP_FLATTEN` 编译指示应用于循环层级的最内层循环的循环主体。只有完美循环和近乎完美的嵌套循环（在完成可能的初步函数内联或循环展开后）才能以这种方式加以平铺：

- 完美循环嵌套：
 - 每个非最内层循环的主体都包含一个且仅包含一个子循环，不包含任何其他指令。
- 近乎完美的循环嵌套：
 - 每个非最内层循环的主体都包含一个且仅包含一个子循环，无任何其他控制流。
 - 每个非最内层循环的主体都不得包含任何含循环的函数调用。

对于近乎完美的循环，编译器会将两个循环之间存在的任何指令自动“推入”最内层的循环，从而使这些循环完美嵌套。

此外具备平铺能力的循环还需要满足一些要求：

- 每个循环都应该是 `for` 循环，而不是 `while` 循环；并且不含 `break` 语句。
- 在平铺循环之前，编译器应能计算出每个循环的循环次数（但它不必是一个常数）。典型的编码风格如下：
 - 所含循环计数器按常数递增的循环。
 - 循环计数器的下限和上限不依赖于要平铺的循环（应满足“循环不可变”条件）。

举例来说，如果循环的内层循环次数取决于外层循环计数器，那么这类循环就不具备平铺能力。

编译器无法平铺不完美的循环嵌套（例如，当循环包含多个子循环或控制流时）。在此情况下，需要手动执行平铺，方法是重构代码，将指令推入最内层的循环，或者展开内层循环以在其上层创建完美循环嵌套。

语法

只要满足平铺能力要求，即可将 C 语言源代码的编译指示置于如下循环内：该循环通常是最内层的循环，并且要与其上层（完美循环或近乎完美的）循环一起平铺。

```
#pragma HLS loop_flatten
```

选项：

- `off`：可选关键字。阻止将 `loop_flatten` 设为 `off` 的循环与其子循环（如有）一起进行平铺。如果在最内层的循环中指定 `loop_flatten off`，则不会发生任何平铺行为（即使有自动平铺也不予执行）。

示例 1

在循环层级内找到要与上层的完美循环或近乎完美的循环（此处的 `loop_0`）一起平铺的循环，将编译指示置于该循环的（此处的 `loop_1`）主体内，前提是满足平铺能力要求。

```
void foo (N, M, ...) {
    int i, j;
    loop_0: for (i=0; i<N; i++) {
        loop_1: for (j=0; j<M; j++) {
            #pragma HLS loop_flatten
            ...
        }
    }
}
```


示例 2

阻止将 `loop_1` 循环与 `loop_0` 一起进行平铺。仅将 `loop_1` 和 `loop_2` 一起平铺。

```
void foo (N, M, P, ...) {
    int i, j;
    loop_0: for (i=0; i<N; i++) {
        #pragma HLS loop_flatten off
        loop_1: for (j=0; j<M; j++) {
            loop_2: for (k=0; k<P; k++) {
                #pragma HLS loop_flatten
                ...
            }
        }
    }
}
```

示例 3

如果 `loop_flatten` 编译指示上层有两个以上嵌套循环，那么以下情况下编译器可以决定不平铺所有循环：如果预计可能有 II 劣化，或者如果它需要明确知晓（但不能证明）内层循环迭代至少一次。在此情况下，可以在要与周围循环一起平铺的循环内执行一次额外的 `loop_flatten` 编译指示来强制平铺，例如，此处 `loop_1` 中，强制与 `loop_0` 一起平铺。

```
void foo (N, M, P, ...) {
    int i, j;
    loop_0: for (i=0; i<N; i++) {
        ... // some instruction with side effect
        loop_1: for (j=0; j<M; j++) {
            #pragma HLS loop_flatten
            loop_2: for (k=0; k<P; k++) {
                #pragma HLS loop_flatten
                ...
            }
        }
    }
}
```

备注：

- 对于 `loop_flatten` 编译指示，如果要某些有副作用的指令推入循环（针对近乎完美的循环），那么编译器假定包含该编译指示的循环至少迭代一次。
- 平铺时，内层循环中的任何依赖编译指示（`false` 或有距离）都会应用于平铺后获得的单一循环。

示例 4

以下示例显示了可平铺的近乎完美的循环。

```
void foo (...) {
    int i, j;
    loop_0: for (i=0; i<N; i++) {
        int s = 0;
        loop_1: for (j=0; j<M; j++) {
            #pragma HLS loop_flatten
            ...
        }
    }
}
```

```
s += a[i][j];  
b[i][j] = s;  
}  
}
```

示例 5

以下示例显示了不可平铺的循环。

```
void foo (...) {  
    int i, j;  
    loop_0: for (i=0; i<N; i++) {  
        loop_1: for (j=i; j<M; j++) {  
            #pragma HLS loop_flatten  
            ...  
        }  
    }  
}
```

另请参阅

- [set_directive_loop_flatten](#)
- [pragma_HLS_loop_merge](#)
- [pragma_HLS_unroll](#)

pragma HLS loop_merge

描述

将连续循环合并到单个循环内，以缩短总体时延、增加共享，并提升逻辑优化。合并循环：

- 减少 RTL 中循环主体实现间转换所需时钟周期数。
- 允许并行实现循环（如果可能）。

LOOP_MERGE 编译指示将尝试合并其布局范围内的所有循环。例如，如果对循环应用 LOOP_MERGE 编译指示，那么 Vitis HLS 工具会将编译指示应用于该循环内的所有子循环，但不会将其应用于该循环本身。

合并循环的规则是：

- 如果循环边界为变量，则值（迭代数）必须相同。
- 如果循环边界为常量，那么最大常量值用作为合并循环的边界。
- 具有变量边界的循环与具有常量边界的循环无法合并。
- 要合并的循环之间的代码不得产生不同结果。多次执行此代码应生成相同结果（允许使用 `a=b`，不允许使用 `a=a+1`）。
- 包含 FIFO 读取的循环无法合并。合并会更改读取顺序。从 FIFO 或 FIFO 接口执行读取必须始终按顺序进行。

语法

将 C 语言源代码中的编译指示置于所需的代码范围或区域内。

```
#pragma HLS loop_merge force
```

其中：

- **force**：（可选）关键字，使用此关键字表示即使 HLS 工具发出警告，仍强制合并循环。



重要提示！ 在此情况下，您必须手动确保合并后的循环功能正常。

示例

将函数 `foo` 中的所有连续循环都合并到单一循环中。

```
void foo (num_samples, ...) {  
    #pragma HLS loop_merge  
    int i;  
    ...  
    loop_1: for(i=0;i< num_samples;i++) {  
        ...  
    }
```

`loop_2` 内的所有循环（不包含 `loop_2` 本身）均通过使用 `force` 选项来合并。将编译指示置于 `loop_2` 主体内。

```
loop_2: for(i=0;i< num_samples;i++) {  
    #pragma HLS loop_merge force  
    ...  
}
```

另请参阅

- [set_directive_loop_merge](#)
- [pragma_HLS_loop_flatten](#)
- [pragma_HLS_unroll](#)

pragma HLS loop_tripcount

描述

将该编译指示手动应用于循环，用于指定循环执行的迭代总数。



重要提示！ `LOOP_TRIPCOUNT` 编译指示或指令仅用于分析，不影响综合结果。

Vitis HLS 工具会报告每个循环的总时延，即执行循环的所有迭代的时钟周期数。因此，循环时延即为循环迭代次数（或循环次数）的函数。

循环次数可为常量值。它取决于循环表达式（例如，`x < y`）中使用的变量值或循环内使用的控制语句。在某些情况下，HLS 工具无法判定循环次数，因此时延未知。在此类情况下，用于判定循环次数的变量可能是：

- 输入实参，或者
- 采用动态运算计算所得的变量。

在以下示例中，for 循环的最大迭代次数由输入 `num_samples` 的值来确定。在 C 语言函数中不定义 `num_samples` 的值，该值从外部进入函数。

```
void foo (num_samples, ...) {
    int i;
    ...
    loop_1: for(i=0;i< num_samples;i++) {
        ...
        result = a + b;
    }
}
```

如果循环时延未知或者无法计算，那么 `LOOP_TRIPCOUNT` 编译指示会要求您指定循环迭代次数的最小、最大和平均值。这样该工具即可分析循环时延给报告中的设计总时延造成的影响，并帮助您为设计决定合适的优化措施。



提示：如果使用 C 语言断言宏来限制循环变量的大小，那么 Vitis HLS 可将其用于定义循环限制以供报告之用，并根据这些限制创建大小精确的硬件。

语法

将 C 语言源代码中的编译指示置于循环的主体内。

```
#pragma HLS loop_tripcount min=<int> max=<int> avg=<int>
```

其中：

- `max= <int>`：指定循环迭代次数的最大值。
- `min=<int>`：指定循环迭代次数的最小值。
- `avg=<int>`：指定循环迭代次数的平均值。

示例

在以下示例中，函数 `foo` 中的 `loop_1` 的最小循环次数指定为 12，最大循环次数则为 16：

```
void foo (num_samples, ...) {
    int i;
    ...
    loop_1: for(i=0;i< num_samples;i++) {
        #pragma HLS loop_tripcount min=12 max=16
        ...
        result = a + b;
    }
}
```

另请参阅

- [set_directive_loop_tripcount](#)

pragma HLS occurrence

描述

OCCURRENCE 编译指示或指令可应用于区域内的函数调用。该功能特性适用于条件块内的流水打拍函数调用，例如，`if` 语句或 `switch` 语句。该编译指示用于指定当前流水线内调用流水打拍函数的频率。如果以 OCCURRENCE 编译指示来标识条件块内的某个函数，即可允许该区域中的函数和循环采用流水打拍、启动时间间隔更高，且比外层函数或循环更慢。

执行函数或循环流水打拍时，OCCURRENCE 编译指示可用于指定流水打拍函数或循环内某个流水打拍函数调用的执行频率低于其外层函数或循环的执行频率。这样流水打拍的函数调用在以较低频率执行时即可以较慢的速率进行流水打拍，这可能有助于改善顶层流水线内的资源共享。

以下进程可用于判定 OCCURRENCE 周期值：

- 循环迭代 $\langle N \rangle$ 次。
- 但由于循环主体中某部分是通过条件语句来启用的，因此仅执行 $\langle M \rangle$ 次，其中 $\langle N \rangle$ 为 $\langle M \rangle$ 的整数倍。
- 条件代码的出现次数比循环主体的其他部分慢 N/M 次。

例如，在执行 10 次的循环中，其中的条件语句仅执行 2 次，则出现次数为 5（即 $10/2$ ）。

语法

将 C 语言源代码中的编译指示布局在包含流水打拍的子函数调用的条件代码块中。

```
#pragma HLS occurrence cycle=<int> off=true
```

其中：

- `cycle=<int>`：指定出现次数值为 N/M ，其中 $\langle N \rangle$ 是外层函数或循环的执行次数， $\langle M \rangle$ 是条件区域的执行次数。



重要提示！ $\langle N \rangle$ 必须为 $\langle M \rangle$ 的整数倍。

- `off=true`：为指定函数禁用 occurrence。

示例

在此示例中，在 `if` 语句中调用子函数（但不包括存储器读写，因为它们不在流水打拍函数内）会发生 4 次（其执行频率是包含此代码的外部代码的四分之一以下）。因此，虽然没有 OCCURRENCE 编译指示，但仍会以与调用程序相同的 II 来对其进行流水打拍，包含 OCCURRENCE 编译指示时，将以 $II=4$ 来执行流水打拍。这将在其自身内部以及与其他函数之间增加资源共享的机会。

```
void subfunction(...) {
#pragma HLS pipeline II=...
// Without the occurrence pragma,
// this will be automatically pipelined with an II=1,
// regardless of its own pipeline pragma,
// since it is called in a pipeline with II=1
// With the pragma, it has an II=4.
...
}
for (int i = 0; i < 16; i++) {
```

```
#pragma HLS pipeline II=1
    if (i % 4 == 0) {
#pragma HLS occurrence cycle=4
        subfunction(...);
        a[i+1] = b[i];
    }
}
```

另请参阅

- [set_directive_occurrence](#)
- [pragma_HLS_pipeline](#)

pragma HLS performance

描述

提示：顶层 PERFORMANCE 编译指示全局应用于整个被测设计 (DUT) 的顶层函数，而循环层 PERFORMANCE 编译指示则以设计中的特定循环或循环嵌套为目标。

Vitis HLS 中的 PERFORMANCE 编译指示为您的设计提供了用于定义高层次性能目标的机制，以便引导综合工具的优化工作。它可应用于两个不同的作用域：设计的顶层（内核/IP）和设计中的特定循环层。

顶层 PERFORMANCE 编译指示

在内核或 IP 顶层函数上应用 PERFORMANCE 编译指示允许您指定整体性能目标，主要侧重于达成所需的吞吐量。在内核顶层函数上应用该编译指示时，会触发 HLS 工具对整个设计执行性能分析。该分析对整个设计结构进行评估，以判定满足指定性能目标的可行性。根据整体目标及此项分析，该工具会自动为设计层级内的单个循环和循环嵌套推断适当的循环层性能编译指示。

循环层 PERFORMANCE 编译指示

直接将 PERFORMANCE 编译指示应用于特定循环或循环嵌套使您能定义循环或循环嵌套的该特定部分的性能目标。循环层 PERFORMANCE 编译指示或指令支持您指定高层次约束 `target_ti` 来定义循环的连续启动之间的时钟周期数，并允许该工具推断达成期望结果所需的低层次 UNROLL、PIPELINE、ARRAY_PARTITION 和 INLINE 指令。PERFORMANCE 编译指示或指令不保证能达成指定的值，仅作为目标。

适用于顶层（整个函数/DUT）和循环层的 PERFORMANCE 编译指示使用 `target_ti` 参数来引导优化以达成特定性能目标。

`target_ti`：（目标启动时间间隔）定义 DUT 或循环的连续启动之间所需的时钟周期数。

顶层性能编译指示：

- 在顶层应用 `target_ti` 时，它会指定单次函数执行的开始时间与下一次函数执行的开始时间之间的目标时间间隔。来看一个示例，假设要为图像处理设计计算顶层性能编译指示，目标帧数为每秒 60 帧 (FPS)。要达到 60 FPS 的帧速，函数/DUT 必须在 1/60 秒内准备好重新启动并读取新的帧。
- $\text{Target TI} = 1/\text{FPS} = 1/60 \approx 16.67 \text{ 毫秒}$ 。
- 这意味着函数/DUT 必须在 16.7 毫秒内准备好重新启动并读取新的帧，以保持每秒 60 帧 (FPS) 的目标。

循环层性能编译指示：

- 在循环层应用 `target_ti` 时，它会指定在连续的外层循环迭代中，循环首次迭代的启动之间的目标时间间隔。在以下示例中，循环 L2 上的 `target_ti=T` 将 L1 的一次迭代中开始执行 L2 循环与 L1 的下一迭代中开始 L2 循环之间的目标时间间隔设为 `T` 个时钟周期。

```
1 const int T = 100;
2 L1: for (int i=0; i<N; i++)
3   L2: for (int j=0; j<M; j++)
4   { pragma HLS performance target_ti=T    ... }
```

语法

```
#pragma HLS performance target_ti=<value>
```

其中：

- `target_ti=<value>`：指定目标传输事务时间间隔定义为代码的函数、循环或区域完成一次迭代的时钟周期数。`<value>` 可指定为整数、浮点或由该工具解析为整数的常量表达式。

注释：如果发生截位，则会返回警告。

- `unit=[sec | cycle]`：指定与 `target_ti` 值或 `target_tl` 值关联的单位。此单位可指定为秒数或时钟周期数。当指定的单位为秒数时，可用表示纳秒 (ns)、皮秒 (ps) 或微秒 (us) 的相应的值来指定单位。

示例 1

如果 `target_ti` 所需目标值为 100 us，那么应如何指定编译指示？例如，如果值为 100 us，则 `target_ti=1e-4 unit=sec`。

- `#pragma HLS performance target_ti=100 unit=us`
- `#pragma HLS performance target_ti=100us unit=sec`
- `#pragma HLS performance target_ti=100e-6 unit=sec`

示例 2

指定外层循环的目标传输事务时间间隔为 1000 个时钟周期。

```
for (int i =0; i < 1000; ++i) {
#pragma HLS performance target_ti=1000
  for (int j = 0; j < 8; ++j) {
    int tmp = b_buf[j].read();
    b[i * 8 + j] = tmp + 2;
  }
}
```

另请参阅

- [set_directive_performance](#)
- [pragma HLS inline](#)

pragma HLS pipeline

描述

允许并发执行操作，以缩短函数或循环的启动时间间隔 (II)。流水线的默认样式是由 `config_compile -pipeline_style` 命令定义的，但可在 PIPELINE 编译指示或指令内进行覆盖。

流水打拍函数或循环每 $\langle N \rangle$ 个时钟周期即可处理新输入，其中 $\langle N \rangle$ 是该循环或函数的 II。如果 II 为 1，则表示每个时钟周期都处理新输入。您可通过为编译指示使用 II 选项来指定启动时间间隔。

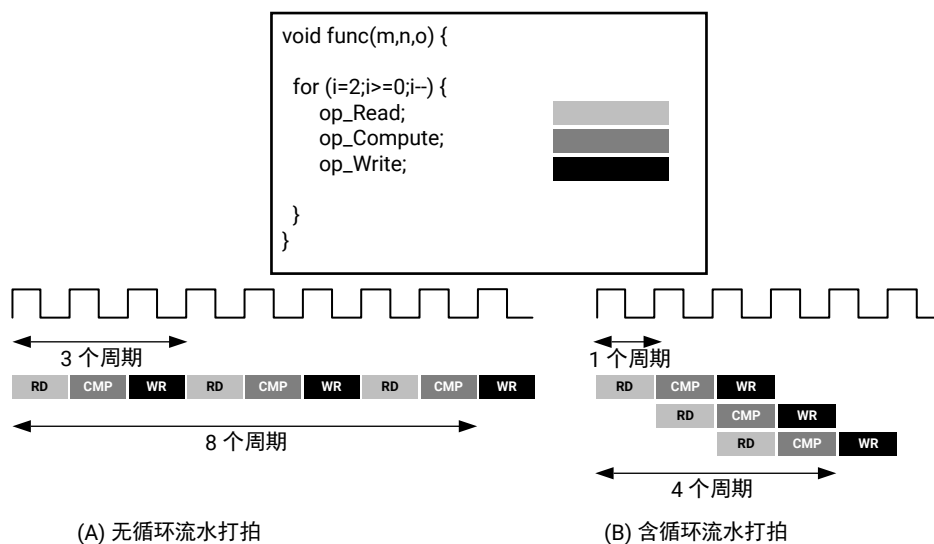
Vitis HLS 使用 PIPELINE 编译指示或指令时的默认行为是根据指定的时钟周期约束，为设计生成最小 II。除非指定 II 选项，否则重点强调满足时序，而不是实现 II。

如果 Vitis HLS 工具无法使用指定 II 创建设计，那么它会发出警告，并以可能的最低 II 创建设计。

随后，您可利用警告消息来分析此设计，以判定必须采取哪些步骤才能创建满足所需启动时间间隔的设计。

循环流水打拍允许以并发方式来实现循环的操作，如下图所示。在此图中，(A) 显示了默认顺序操作，每次输入读操作间存在 3 个时钟周期 (II = 3)，并且需要经过 8 个时钟周期后才会执行最后一次输出写操作。(B) 显示的流水打拍操作表明每次读操作之间有 1 个时钟周期 (II=1)，并且距离最后的写操作有 4 个周期。

图 141：循环流水打拍



X14277-030622



重要提示！ 循环携带依赖会阻碍流水打拍。使用 DEPENDENCE 编译指示或指令可提供附加信息，这些信息可用于克服循环携带依赖，并支持循环流水打拍（或以较低时间间隔进行流水打拍）。

语法

将 C 语言源代码中的编译指示置于函数或循环的主体内。

```
#pragma HLS pipeline II=<int> off rewind style=<value>
```

其中：

- `II=<int>`: 指定期望的流水线启动时间间隔。HLS 工具会尝试满足此请求。根据数据依赖关系, 实际结果的启动时间间隔可能更大。
- `off`: 可选关键字。关闭特定循环或函数的流水线。当使用 `config_compile -pipeline_loops` 对循环进行全局流水打拍时, 可使用该选项来为特定循环禁用流水打拍。
- `rewind=<true|false>`: 可选关键字。默认情况下, 如果不使用此关键字, 编译器可能会按照 [回绕已流水打拍的循环以保障性能](#) 中的描述回绕循环。请记住, 使用 `rewind` 本身等同于 `rewind=true`。这样即可支持持续性循环流水打拍, 但在循环执行结束与下一次执行开始之间没有暂停。仅当顶层函数内只有一个循环 (或完美循环嵌套) 时, 回绕才有效。循环前的代码段:
 - 被视为初始化。
 - 在流水线中仅执行一次。
 - 无法包含任何条件操作符 (if-else)。



提示: 仅针对流水打拍循环才支持此功能: 针对流水打拍函数不支持。

- `style=<stp | frp | flp>`: 指定要用于指定的函数或循环的流水线类型。如需了解有关流水线样式的更多信息, 请参阅 [刷新流水线和流水线类型](#)。流水线类型包括:
 - `stp`: 停滞型流水线。仅当输入数据可用时才运行, 否则停滞。这是默认设置, 也是 Vitis HLS 用于循环和函数流水打拍的流水线类型。无需可刷新流水线时, 请使用此类型。例如, 没有导致停滞的性能或死锁问题时即可用。
 - `flp`: 该选项用于将流水线定义为可刷新流水线。此类型的流水线通常耗用更多资源和/或可能 `II` 较大, 因为无法在流水线迭代间共享资源。
 - `frp`: 自由运行的可刷新流水线。即使输入数据不可用也可运行。如果由于流水线控制信号扇出减少而导致需要更好的时序, 或者如果需要提升性能以免死锁, 则可使用此类型。但此流水线样式可能功耗更大, 因为即使没有数据也会对流水线寄存器进行时钟驱动。



重要提示! 这是提示, 而非硬性约束。工具会检查设计启用流水打拍的条件。部分循环可能不遵循特定样式, 如果需要, 工具会还原为默认样式 (`stp`)。

示例

在此示例中, `func` 函数按启动时间间隔 1 来进行流水打拍。

```
void func { a, b, c, d } {
  #pragma HLS pipeline II=1
  ...
}
```

另请参阅

- [set_directive_pipeline](#)
- [pragma_HLS_dependence](#)
- [config_compile](#)

pragma HLS protocol

描述

此命令用于指定代码区域（协议区域），除非代码中明确指定，否则 Vitis HLS 不会在其中插入时钟操作。Vitis HLS 不会在该区域内的运算之间插入任何时钟，包括读取或写入函数实参的运算。因此，读取和写入顺序严格遵循已综合的 RTL 中的顺序。

在 C/C++ 代码中可以通过使用 “{}” 将区域括起或者对该区域进行命名来创建代码区域。以下示例定义了名为 `io_section` 的区域：

```
io_section:{  
  ...  
  lines of code  
  ...  
}
```

在 C/C++ 代码中，可使用 `ap_wait()` 语句来显式指定时钟运算，在 C++ 代码中则可使用 `wait()` 语句来指定该运算。`ap_wait` 和 `wait` 语句对于设计的仿真没有影响。

语法

将 C 语言源代码中的编译指示置于函数或协议区域的主体内。

```
#pragma HLS protocol [floating | fixed]
```

选项

- `floating`：表示浮动模式，允许协议区域外的代码语句与最终 RTL 内协议区域中的语句重叠且并行执行。协议区域可保持时钟精确，但仍可同时发生外部运算。这是默认模式。
- `fixed`：表示固定模式，`fixed` 模式可确保协议区域外部的语句不会与协议区域并行执行。

示例

此示例在函数 `foo` 内定义了 `io_section` 协议区域，其中通过编译指示将该区域定义为浮动协议区域（默认模式）：

```
io_section: {  
  #pragma HLS protocol  
  ...  
}
```

另请参阅

- [set_directive_protocol](#)

pragma HLS reset

描述

RESET 编译指示或指令用于为特定状态变量添加或禁用复位端口。

复位端口用于在应用复位信号时，立即将连接到复位端口的寄存器和块 RAM 还原为初始值。在全局范围内，`syn.rtl.reset` 配置设置（或 `config_rtl -reset`）可用于控制 RTL 复位端口是否存在及其行为。复位配置设置包含定义复位极性以及指定使用同步复位还是异步复位的功能，但更重要的是，它可通过复位选项来控制应用复位信号时所复位的寄存器。如需了解更多信息，请参阅 [控制初始化与复位行为](#)。

通过 RESET 编译指示可提供更具体的复位控制。对于全局变量或静态变量，RESET 编译指示用于显式启用复位（前提是不存在任何变量），或者可通过设为 `off` 关闭此编译指示来从复位中移除变量。当设计中存在静态阵列或全局阵列时，该选项非常实用。

注释：对于任一类的公开变量，必须使用 RESET 编译指示，因为复位配置设置仅适用于在函数级别或文件级别声明的变量。此外，RESET 编译指示必须应用于顶层函数或子函数中的类的对象，不能应用于类的私有变量。

语法

将 C 语言源代码中的编译指示置于变量生命周期边界内。

```
#pragma HLS reset variable=<a> off
```

其中：

- `variable=<a>`：指定 RESET 编译指示应用到的变量。
- `off`：指示针对指定变量不生成复位。

示例 1

此示例会为 `foo` 函数中的 `a` 变量添加复位，即使全局复位设置为 `none` 或 `control` 也是如此。

```
void foo(int in[3], char a, char b, char c, int out[3]) {  
#pragma HLS reset variable=a
```

示例 2

从 `foo` 函数中的 `a` 变量移除复位，即使全局复位设置为 `state` 或 `all` 也是如此。

```
void foo(int in[3], char a, char b, char c, int out[3]) {  
#pragma HLS reset variable=a off
```

示例 3

以下示例显示了如何将 RESET 编译指示或指令与类变量和方法搭配使用。在类中声明的变量必须是该类中的公开静态变量，并且可在类的方法中为该变量指定 RESET 编译指示，或者在 HLS 设计的顶层函数或子函数中创建类对象之后为该变量指定 RESET 编译指示。

```
class bar {
public:
    static int a; // class level static; must be public
    int a1;      // class-level non-static; must be public
    bar(...) {
        // #pragma reset does not work here for a or a1
    }
    // this is the function that is called in the top
    void run(...) {
        // #pragma reset does not work here for non-static variable a1
        // but does work for static variable a
        #pragma reset variable=a
    }
};

static int c; // file level
int d; // file level
void top(...) {
    #pragma HLS top
    static int b; // function level
    bar t;
    static bar s;

    // #pragma reset works here for b, c and d, as well as t and s members
    // except for t.a1 because it is not static
    #pragma reset variable=b
    #pragma reset variable=c
    #pragma reset variable=d
    #pragma reset variable=t.a
    #pragma reset variable=s.a
    #pragma reset variable=s.a1
    t.run(...);
    s.run(...);
}
```

另请参阅

- [set_directive_reset](#)
- [config_rtl](#)

pragma HLS stable

描述

标记为 stable 的实参在内核执行期间不得更改。仅当内核未运行时才能更改的数据（例如，提供配置数据的端口）应以 STABLE 编译指示或指令来加以标记。STABLE 编译指示只能用于控制驱动的任务建模，因为，在数据驱动的任务级并行化中，无法检测内核何时处于空闲状态。

STABLE 编译指示会将以下信息告知 Vitis HLS：

- 应用于端口的数据在正常操作期间保持稳定，但不是可以优化的常量值。
- 此端口的扇出无需寄存。

DATAFLOW 网络中的编译指示 STABLE 会告知编译器，网络中第一个数据流进程无需读取输入，网络中最后一个数据流进程无需写入输出。这样即可显著改善 II。

语法

```
#pragma HLS stable variable=<a>
```

其中：

- `variable=<a>`：指定 STABLE 编译指示应用到的标量或阵列变量的名称。
- （新增选项）：- off：关闭 stable 推断。

示例

在以下示例中，不使用 STABLE 编译指示的情况下，`proc1` 与 `proc2` 将同步，以确认其输入的读取操作（包括 A）。使用该编译指示后，A 将不再被视为需同步的输入。

```
void dataflow_region(int A[...], int B[...] ...
#pragma HLS stable variable=A
#pragma HLS dataflow
    proc1(...);
    proc2(A, ...);
```

另请参阅

- [set_directive_stable](#)
- [pragma HLS dataflow](#)

pragma HLS stream

描述

默认情况下，阵列变量作为 RAM 来实现：

- 顶层函数阵列参数作为 RAM 接口端口来实现。
- 一般阵列作为 RAM 来实现以便进行读写访问。
- 包含在子函数内的阵列或者基于循环的 DATAFLOW 优化都作为 RAM 乒乓缓冲器通道来实现。

如果按顺序来使用或生产阵列中存储的数据，那么更有效的通信机制是使用串流数据（根据 STREAM 编译指示所指定的方式），其中使用 FIFO 代替 RAM。



重要提示！ 当顶层函数实参的 INTERFACE 类型被指定为 `ap_fifo` 时，该阵列会以串流方式自动实现。如需了解更多信息，请参阅 [定义接口](#)。

语法

将 C 语言源代码中的编译指示置于所需位置的边界内。

```
#pragma HLS stream variable=<variable> type=<type> depth=<int>
```

其中：

- `variable=<variable>`：指定要作为串流接口实现的阵列的名称。
- `depth=<int>`：仅与 DATAFLOW 通道中的阵列串流相关。默认情况下，RTL 中实现的 FIFO 的深度与 C 语言代码中指定的阵列大小相同。该选项允许您修改 FIFO 大小以指定不同深度。

在 DATAFLOW 区域内实现阵列时，经常使用 `depth` 选项来减小 FIFO 的大小。例如，如果 DATAFLOW 区域中的所有循环和函数均以 `II = 1` 的速率来处理数据，则无需大型 FIFO，因为在每个时钟周期内都会生产和使用数据。在此情况下，`depth` 选项可用于将 FIFO 大小减小为 1 以便大幅减小 RTL 设计的面积。



提示： `config_dataflow -depth` 命令能够对 DATAFLOW 区域内的所有阵列进行串流。STREAM 编译指示内指定的 `depth` 选项会覆盖指定的 `<variable>` 的 `config_dataflow -depth` 设置。

- `type=<arg>`：指定机制，可选机制包括 FIFO、PIPO、已同步的共享 (`shared`) 和未同步的共享 (`unsync`)。受支持的类型包括：
 - `fifo`：含指定 `depth` 的 FIFO 缓冲器。
 - `pipo`：乒乓缓冲器，所含“bank”数量与指定深度相同（默认值为 2）
 - `shared`：含深度的共享通道，其同步方式与常规乒乓缓冲器类似，但不复制阵列数据。通过将深度设置为足够小的值以充当生产者与使用者之间的同步距离，即可确保一致性。



提示： `shared` 默认深度值为 1。

- `unsync`：除了个别存储器读取和写入外，无任何同步。设计必须自行确保一致性（读取 - 写入和写入 - 写入顺序）。

示例 1

以下示例指定要进行串流的阵列 `A[10]`，并作为 FIFO 来实现。

```
#pragma HLS STREAM variable=A
```

示例 2

在此示例中，阵列 `B` 设置为以 FIFO 深度 12 来进行串流。

```
#pragma HLS STREAM variable=B depth=12 type=fifo
```

示例 3

阵列 `C` 的串流以 PIPO 方式来实现。

```
#pragma HLS STREAM variable=C type=pipo
```

另请参阅

- [set_directive_stream](#)
- [pragma HLS dataflow](#)
- [pragma HLS interface](#)
- [config_dataflow](#)

pragma HLS top

描述

给函数附加名称，随后此名称可搭配 `set_top` 命令一起用于对该函数以及从指定顶层调用的任意函数进行综合。此命令通常用于对 C/C++ 中的某个类的成员函数进行综合。

在活动解决方案中指定 TOP 编译指示，然后搭配新名称来使用 `set_top` 命令。

语法

将 C 语言源代码中的编译指示置于所需位置的边界内。

```
#pragma HLS top name=<string>
```

其中：

- `name=<string>`：指定将供 `set_top` 命令使用的名称。

示例

函数 `foo_long_name` 被指定为顶层函数，并重命名为 `DESIGN_TOP`。在代码中放置编译指示后，仍必须从 Tcl 命令行或者从 IDE 工程设置中指定顶层发出 `set_top` 命令。

```
void foo_long_name () {  
    #pragma HLS top name=DESIGN_TOP  
    ...  
}
```

随后发出 `set_top DESIGN_TOP` 命令。

另请参阅

- [set_directive_top](#)
- [set_top](#)

pragma HLS unroll

描述

您可展开循环以创建多个独立操作而非单个操作集。UNROLL 编译指示会通过 RTL 设计中创建循环主体的多个副本来变换循环，从而允许部分或全部循环迭代并行发生。

C/C++ 语言函数中的循环默认情况下全部保持收起状态。收起循环时，综合会为循环的单次迭代创建逻辑，RTL 设计会为序列中循环的每次迭代都执行此逻辑。循环的迭代执行次数由循环归纳变量来指定。迭代次数也可能受到循环主体内的逻辑影响（例如，对循环出口变量条件应用 `break` 条件或修改）。您可使用 UNROLL 编译指示展开循环以便增加数据访问和吞吐量。

UNROLL 编译指示支持将循环完全展开或部分展开。完全展开循环会在 RTL 内为每个循环迭代创建一份循环主体副本，因此整个循环可并发运行。部分展开循环允许您指定因子 `N` 以创建 `N` 份循环主体副本，并相应减少循环迭代。



提示：要完全展开循环，在编译时循环边界必须已知。对于部分展开则无需满足此条件。

部分循环展开不要求 `N` 为最大循环迭代计数的整数因子。Vitis HLS 工具会添加出口检查以确保部分展开的循环的功能与原始循环相同。例如，给定以下代码：

```
for(int i = 0; i < X; i++) {
    pragma HLS unroll factor=2
    a[i] = b[i] + c[i];
}
```

按因子 2 展开的循环可将代码有效变换为如下所示代码，其中 `break` 构造函数用于确保功能保持不变，并且循环会在相应的点退出。

```
for(int i = 0; i < X; i += 2) {
    a[i] = b[i] + c[i];
    if (i+1 >= X) break;
    a[i+1] = b[i+1] + c[i+1];
}
```

在以上示例中，由于最大迭代计数 `x` 可用，因此 HLS 工具可能无法判定其值，故而它给部分展开的循环添加了出口检查和控制逻辑。但如果您已知指定的展开因子（在此示例中为 2）是最大迭代计数 `x` 的整数因子，那么 `skip_exit_check` 选项允许您移除出口检查和关联的逻辑。这有助于最大限度降低面积并简化控制逻辑。



提示：当使用 `ARRAY_PARTITION` 或 `ARRAY_RESHAPE` 之类的编译指示支持在单个时钟周期内访问更多数据时，HLS 工具会自动展开耗用该数据的任意循环，前提是这样可提高吞吐量。循环可全部或部分展开以创建足够的硬件以便在单一时钟周期内使用更多数据。这种自动展开可使用 `config_unroll` 命令来控制。

语法

将 C 语言源代码中的编译指示置于要展开的循环主体内。

```
#pragma HLS unroll factor=<N> skip_exit_check off=true
```

其中：

- `factor=<N>`：指定非零整数，表示已请求部分展开。循环主体将重复指定次数，迭代信息将进行相应的调整。如不指定 `factor=`，则循环将完全展开。

- `skip_exit_check`：可选关键字，仅当使用 `factor=` 指定部分展开时才适用。根据循环迭代计数为已知还是未知来判断是否消除出口检查：
 - 固定边界

如果迭代计数为因数的倍数，则不执行出口条件检查。

如果迭代计数并非因数的整数倍，则该工具将执行以下操作：

 - 阻止展开。
 - 发出警告，称必须执行出口检查后才能继续。
 - 变量边界

移除出口条件检查。您必须确保：

 - 变量边界为因数的整数倍。
 - 实际上无需出口检查。
- `off=true`：为指定循环禁用展开。

示例 1

以下示例在 `foo` 函数内完全展开 `loop_1`。将编译指示置于 `loop_1` 主体内，如图所示。

```
loop_1: for(int i = 0; i < N; i++) {
    #pragma HLS unroll
    a[i] = b[i] + c[i];
}
```

示例 2

此示例指定展开因子为 4，以部分展开 `foo` 函数的 `loop_2`，并移除出口检查。

```
void foo (...) {
    int8 array1[M];
    int12 array2[N];
    ...
    loop_2: for(i=0;i<M;i++) {
        #pragma HLS unroll skip_exit_check factor=4
        array1[i] = ...;
        array2[i] = ...;
        ...
    }
    ...
}
```

另请参阅

- [set_directive_unroll](#)
- [pragma HLS loop_flatten](#)
- [pragma HLS loop_merge](#)

HLS Tcl 命令

工程命令

工程命令是允许您创建和管理工程与解决方案的 AMD Vitis™ HLS 命令。这些命令必须以交互模式 `vitis_hls -i` 来运行，或者也可以使用 `-f` 选项作为脚本来运行，如 [vitis](#)、[v++](#) 和 [vitis-run](#) 命令 中所述。

通过 Vitis HLS GUI 执行特定函数时也可以使用这些命令的功能特性，如 [创建 HLS 组件](#) 和 [运行 C/RTL 协同仿真](#) 中所述。



提示：通过 GUI 运行命令时，会将 Tcl 命令添加至您的工程的脚本中，并写入 `solution/constraints/script.tcl`。

add_files

描述

将设计源文件添加到当前工程中。

该工具会搜索当前目录中是否存在设计源文件中包含的任何头文件。要使用存储在其他目录中的头文件，请使用 `-cflags` 选项将这些目录包含在搜索路径中。

语法

```
add_files [OPTIONS] <src_files>
```

- `<src_files>` 用于列出一个或多个受支持的源文件。

选项

- `-blackbox <file_name.json>`：指定用于 RTL 黑盒的 JSON 文件。此文件中的信息供 HLS 编译器在综合期间以及运行 C/C++ 语言仿真和协同仿真时使用。
- `-cflags <string>`：含任意编译选项的字符串。
- `-csimflags <string>`：含任意所需仿真编译选项的字符串。使用该选项指定的标志仅适用于仿真编译（包括 C/C++ 语言仿真和 RTL 协同仿真），不适用于综合编译。该选项不影响 `-cflags` 选项。
- `-tb`：指定设计测试平台使用的任意文件。这些文件不进行综合。运行仿真时，这些文件将供 `csim_design` 或 `cosim_design` 命令来使用。

为设计添加源文件时，不使用 `-tb` 选项。请单独使用 `add_files` 命令来添加设计文件和仿真文件。

示例

将 3 个设计文件添加到工程中。

```
add_files a.cpp
add_files b.cpp
add_files c.cpp
```

使用单一命令行添加多个文件。

```
add_files "a.cpp b.cpp c.cpp"
```

使用 `-tb` 选项将测试激励文件添加到工程中。此示例使用一条命令添加多个文件，包括：

- 测试平台 `a_test.cpp`
- 测试平台读取的所有数据文件：
 - `input_stimuli.dat`
 - `out.gold.dat`

```
add_files -tb "a_test.cpp input_stimuli.dat out.gold.dat"
```

如果先前示例中的测试平台数据文件存储在单独的目录（例如，`test_data`）中，可将此目录添加到工程中，以代替添加个别数据文件。

```
add_files -tb a_test.cpp
add_files -tb test_data
```

apply_ini

描述

将指定 INI 文件应用于当前工程。INI 文件可替代 Tcl 脚本，用于在 Vitis HLS 工具中创建和配置工程。INI 文件是由 Vitis HLS 工具使用 `write_ini` 命令写入的。

注释：INI 文件表示当前工程中的解决方案。在工程中使用 `apply_ini` 命令之前，必须先使用 `open_project` 和 `open_solution` 命令。

语法

```
apply_ini <ini_file> [OPTIONS]
```

- `<ini_file>` 用于指定要读入 AMD Vitis™ HLS 工具的 INI 文件。

选项

- `-exclude <string>`：指定读取 INI 文件时要从当前工程中排除的 INI 节 glob 模式。
- `-include <string>`：指定读取 INI 文件时要包含在当前工程中的 INI 节 glob 模式。
- `-quiet <true | false>`：读取 INI 文件时禁止所有警告消息和参考消息。请注意，仍将返回错误。
- `-show <true | false>`：读取 INI 文件时显示详细消息。

示例

读取以下 INI 文件，并将其应用于当前工程和解决方案：

```
apply_ini project1.ini
```

cat_ini

描述

将一个或多个 INI 文件串联为单个文件，并将结果输出到 stdout 或指定文件。apply_ini 命令可用于将生成的 INI 读入 Vitis HLS 工具。

语法

```
cat_ini { <ini_file1> <ini_file2> ... } [OPTIONS]
```

- { <ini_file1> <ini_file2> ... } 用于指定要读入 AMD Vitis™ HLS 工具并串联到单个文件的一个或多个 INI 文件。“{}” 括号是对多项输入进行分组所必需的。

选项

- -exclude <string>: 指定写入 INI 文件时要排除的 INI 节 glob 模式。
- -include <string>:

指定 INI 文件中的 glob 模式，将其包含在输出文件内。不包含的所有其他内容都将被排除。



提示： -exclude 与 -include 互斥，不能一起使用。

- -out <file name>: 指定要作为输出写入的 INI 文件名。如不指定，则输出写入 stdout。
- -quiet <true | false>: 写入 INI 文件时禁止所有警告消息和参考消息。请注意，仍将返回错误。默认值为 false。
- -show <true | false>: 写入 INI 文件时显示详细消息。默认值为 false。

示例

此示例将指定的 INI 文件连接在一起并将其返回至 stdout：

```
cat_ini {./run.ini ./test.ini}
```

close_project

描述

关闭当前工程。在 Vitis HLS 会话中，此工程不再处于活动状态。

close_project 命令：

- 可阻止您输入任何特定于工程的命令或特定于解决方案的命令。

- 并非必需命令。打开或创建新工程即可关闭当前工程。

语法

```
close_project
```

选项

此命令不含任何选项。

示例

```
close_project
```

- 关闭当前工程。
- 保存所有结果。

close_solution

描述

关闭当前解决方案。在 Vitis HLS 会话中，当前解决方案不再处于活动状态。

`close_solution` 命令用于：

- 可阻止您输入任何特定于解决方案的命令。
- 并非必需命令。打开或创建新解决方案即可关闭当前解决方案。

语法

```
close_solution
```

选项

此命令不含任何选项。

示例

```
close_solution
```

- 关闭当前解决方案。
- 保存所有结果。

cosim_design

描述

利用原先基于 C/C++ 的测试平台来为已综合的 RTL 执行综合后协同仿真。



提示：要指定测试平台的文件，请运行以下命令：

```
add_files -tb
```

根据 `-rtl` 选项，仿真结果将写入处于活动状态的解决方案的 `sim/Verilog` 或 `sim/VHDL` 文件夹。

语法

```
cosim_design [OPTIONS]
```

选项

- `-O`：启用 C/C++ 测试平台和 RTL 封装文件的优化编译。这样会增加编译时间，但是能提供更好的运行时性能。
- `-argv <string>: <string>` 传递到 C/C++ 主函数。

指定行为测试平台的实参列表。

- `-compiled_library_dir <string>`：指定使用第三方仿真器进行仿真期间已编译的库目录。`<string>` 是已编译的库目录的路径名称。此库必须事先使用 `compile_simlib` 命令完成编译，如《Vivado Design Suite Tcl 命令参考指南》(UG835) 所述。
- `-coverage`：启用使用 VCS 仿真器进行仿真期间的覆盖率功能。
- `-disable_binary_tv`：使用纯文本测试矢量文件代替压缩二进制格式。
- `-disable_deadlock_detection`：禁用协同仿真中的死锁检测功能。
- `-disable_dependency_check`：运行协同仿真时禁用依赖关系检查。
- `-enable_changing_stable_inputs`：如果将此项置位，那么 `cosim` 测试平台可以在协同仿真中更改顶层调用之间的 `ap_ctrl_none` 顶层设计的稳定输入。这通常会导致 `cosim` 错误，仅当设计处于非空闲状态时，其中测试平台能够承受稳定输入发生更改的情况下，才能在此类设计中使用此项。
- `-enable_dataflow_profiling`：该选项会开启数据流通道剖析以跟踪协同仿真期间的通道大小。
- `-enable_fifo_sizing`：在协同仿真期间，该选项可开启自动 FIFO 通道大小调整，以便进行数据流剖析。
- `-enable_tasks_with_m_axi`：该选项用于启用 `hls::task` 和 `ap_ctrl_none` 的仿真，并具有稳定的顶层指针。
- `-hwemu_trace_dir <kernel_name>/<instance_name>`：指定测试矢量的位置，这些矢量是在硬件仿真期间生成的，将在协同仿真期间使用。这些测试矢量是由 `config_export -cosim_trace_generation` 命令生成的。该实参允许您在硬件仿真的仿真结果中指定 Vitis 内核的内核和实例名称，以便定位这些测试矢量。
- `-ldflags <string>`：指定传递到连接器用于协同仿真的选项。
该选项通常用于为 C/C++ 语言测试平台传递 `include` 路径信息或库信息。
- `-random_stall`：在协同仿真期间启用顶层接口的随机停滞。
- `-rtl [verilog | vhdl]`：指定用于 C/RTL 协同仿真的 RTL 语言。默认为 Verilog。
- `-setup`：创建仿真文件，包括在处于活动状态的解决方案的 `sim/<HDL>` 目录中创建的所有仿真文件。不执行仿真，但稍后可从命令外壳运行仿真。
- `-stable_axilite_update`：启用 `s_axilite` 以配置寄存器，这些寄存器与传输事务前相比均保持稳定状态。

- `-trace_level [*none* | all | port | port_hier]`: 判定 C/RTL 协同仿真期间，要保存的波形追踪数据级别。
 - `none` 表示不保存追踪数据。这是默认方式。
 - `all` 表示生成要保存到追踪文件的所有端口和信号波形。
 - `port` 表示仅为顶层端口保存波形追踪数据。
 - `port_hier` 表示保存设计层级内的所有端口的追踪信息。

根据 `-rtl` 选项中使用的选择，执行仿真时，追踪文件将保存在当前解决方案的 `sim/Verilog` 或 `sim/VHDL` 文件夹内。

- `-user_stall <string>`: 指定协同仿真期间要使用的 JSON 停滞文件。此停滞文件可使用 `cosim_stall` 命令来生成。
- `-wave_debug`: 打开 Vivado 仿真器 GUI 以查看波形和仿真结果。在生成的 RTL 中启用查看所有进程的波形的功能，就像在数据流进程和顺序进程中一样。仅当通过设置 `-tool xsim` 使用 Vivado 仿真器来执行系统仿真时，才支持该选项。如需了解更多信息，请参阅 [查看仿真波形](#)。

示例

使用 Vivado 仿真器执行验证：

```
cosim_design
```

使用 VCS 仿真器验证 Verilog RTL，并启用波形追踪文件保存功能：

```
cosim_design -tool VCS -rtl verilog -coverage -trace_level all
```

使用 ModelSim 验证 VHDL RTL。值 5 和 1 会传递到测试平台函数，以供在 RTL 验证中使用：

```
cosim_design -tool modelsim -rtl vhdl -argv "5 1"
```

cosim_stall

描述

用于开发系统仿真停滞文件（JSON 格式）的命令。

语法

```
cosim_stall [OPTIONS]
```

选项

- `-check <string>`: 指定运行协同仿真时使用的 JSON 格式的停滞文件。
- `-generate <string>`: 生成要在协同仿真期间使用的 JSON 停滞文件。
- `-list`: 列出在协同仿真期间可应用停滞的所有端口。该选项会基于当前设计返回端口列表。
- `-generate_burst_default`: 在使用 `cosim_design` 时，该选项能为流至外部存储器（如 DDR）的突发流量启用更准确的建模。

示例

以下示例用于生成指定的停滞文件，然后指定要在协同仿真期间使用的文件：

```
cosim_stall -generate my_cosim_stall.json  
cosim_stall -check my_cosim_stall.json
```

create_clock

描述

针对当前解决方案创建 1 个虚拟时钟。

此命令只能在处于活动状态的解决方案关联环境中执行。时钟周期是用于驱动优化的约束（在给定时钟周期内将尽可能多的可行运算链接起来）。

C 和 C++ 设计仅支持单个时钟。

语法

```
create_clock -period <number> [OPTIONS]
```

选项

- `-name <string>`：指定时钟名称。如果不提供名称，将使用默认名称。
- `-period <number>`：指定时钟周期（以 ns 或 MHz 为单位）。
 - 如果不指定单位，将使用 ns。
 - 如果不指定周期，将使用默认周期 10 ns。

示例

指定时钟周期为 50 ns：

```
create_clock -period 50
```

使用默认名称和周期 10 ns 来指定时钟：

```
create_clock
```

指定以 MHz 为单位的时钟频率：

```
create_clock -period 100MHz
```

create_derived_clock

描述

为 `#pragma HLS 接口 s_axilite` 时钟选项中使用的辅助时钟创建相对于基准时钟的约束。

语法

```
create_derived_clock [OPTIONS]
```

选项

- `-name`: 辅助时钟管脚的名称。
- `-multiply_by`: 倍频器（默认值：1）。
- `-divide_by`: 分频器（默认值：1）。

csim_design

描述

使用提供的 C/C++ 语言测试平台编译并运行综合前的 C/C++ 语言仿真。



提示：要指定测试平台的文件，请运行以下命令：

```
add_files -tb
```

仿真结果会写入活动解决方案内部的 `csim` 文件夹。

语法

```
csim_design [OPTIONS]
```

选项

- `-O`: 启用 C/C++ 测试平台的优化编译。这样会增加编译时间，但是能提供更好的运行时性能。
- `-argv <string>`: 指定行为测试平台的实参列表。`<string>` 将传递到测试平台的 `main()` C/C++ 函数。
- `-clean`: 启用纯净构建。如果不选择此选项，`csim_design` 将进行增量编译。
- `-ldflags <string>`: 指定传递给连接器以进行语言仿真的选项。该选项通常用于为 C/C++ 语言测试平台传递 `include` 路径信息或库信息。
- `-sanitize_address`: 该选项用于控制在 C 语言仿真期间是否启用 Address Sanitizer（地址清理器）。遇到寻址问题时，Address Sanitizer 会报错，例如，出界阵列访问。
- `-sanitize_undefined`: 该选项用于控制在 C 语言仿真期间是否启用 Undefined Behavior Sanitizer（未定义的行为清理器）。在 C 语言仿真过程中执行具有未定义的行为的 C/C++ 构造时，该清理器会报错。
- `-setup`: 指定该选项时，会在活动解决方案的 `csim` 目录中创建仿真二进制文件，但不执行仿真。稍后，可从已编译的可执行文件启动仿真。

示例

编译并运行 C/C++ 仿真：

```
csim_design
```

编译源代码设计和测试平台以生成仿真二进制文件。不执行仿真二进制文件：

```
csim_design -O -setup
```



提示：要运行仿真，请在命令终端内从活动解决方案的 `csim/build` 目录执行 `run.sh`。

csynth_design

描述

为处于活动状态的解决方案综合 Vitis HLS 工程。

此命令只能在处于活动状态的解决方案关联环境中执行。数据库中细化后的设计将根据已设置的任意约束进行调度并映射到 RTL 上。

语法

```
csynth_design [OPTIONS]
```

选项

- `-dump_cfg`：编写综合前控制流图 (CFG)。
- `-dump_post_cfg`：编写综合后控制流图 (CFG)。
- `-synthesis_check`：运行综合前设计规则检查，但不生成 RTL。

示例

在顶层设计上运行 Vitis HLS 综合。

```
csynth_design
```

delete_project

描述

删除与工程关联的目录。

`delete_project` 命令用于检查对应的工程目录 `<project>`，以确保它是有效的 Vitis HLS 工程，然后再将其删除。如果指定的工程目录在当前工作目录中不存在，则此命令无效。

语法

```
delete_project <project>
```

- `<project>` 是工程名称。

选项

此命令不含任何选项。

示例

通过移除 `Project_1` 目录及其所有内容来删除该目录。

```
delete_project Project_1
```

delete_solution

描述

从处于活动状态的工程中移除解决方案，并从工程目录中删除 `<solution>` 子目录。

如果解决方案在工程目录中不存在，则此命令无效。

语法

```
delete_solution <solution>
```

- `<solution>` 表示要删除的解决方案。

选项

此命令不含任何选项。

示例

删除子目录以便从活动工程删除 `solution1` 解决方案。

```
delete_solution solution1
```

enable_beta_device

描述

在 Vitis HLS 工具集中启用指定的 Beta 测试访问器件。

语法

```
enable_beta_device <pattern>
```

- `<pattern>` 指定将 Beta 测试器件匹配到使能的模式。

选项

此命令不含任何选项。

示例

以下示例会启用该版本中的所有 Beta 测试器件：

```
enable_beta_device *
```

export_design

描述

导出并封装生成的 RTL 代码，作为 Vivado Design Suite 内使用的封装 IP，或者作为已编译的 Vitis 内核对象 (.xo) 用于 Vitis 应用加速开发流程。

受支持的格式包括：

- Vivado IP，包含在 IP 目录内。
- Vitis 应用加速内核 (.xo)。
- 已综合或已实现的设计检查点 (DCP) 格式。
- Vivado IP 和 ZIP 归档，在 System Generator for DSP 工具内使用。

封装的工程将写入活动解决方案的 `solution/impl` 文件夹内。

语法

```
export_design [OPTIONS]
```

选项

- `-description <string>`：提供生成的 IP 的目录条目描述，在封装 IP 时使用。
- `-display_name <string>`：提供生成的 IP 的目录条目的显示名称，在封装 IP 时使用。
- `-flow (syn | impl)`：使用 Vivado 综合与实现为生成的 RTL 获取更准确的时序和资源使用率数据。`syn` 选项用于执行 RTL 综合。`impl` 选项用于执行 RTL 综合与实现，包括 RTL 网表的详细布局布线。在 Vitis HLS IDE 中，这些选项显示为带有“Vivado Synthesis”（Vivado 综合）和“Vivado Synthesis, place and route stage”（Vivado 综合与布局布线阶段）标记的复选框。
- `-format (ip_catalog | xo | syn_dcp | sysgen)`：指定 IP 封装格式。受支持的格式为：
 - `ip_catalog`：适合添加到 AMD IP 目录的格式。
 - `xo`：v++ 编译器接受的格式，用于在 Vitis 应用加速流程中进行链接。
 - `syn_dcp`：Vivado Design Suite 的已综合的检查点。如果使用该选项，则将自动执行 RTL 综合。无法以可选方式添加 Vivado 实现。
 - `sysgen`：生成 Vivado IP 和 .zip 存档，以供在 System Generator 内使用。
- `-ipname <string>`：为生成的 IP 的 `Vendor:Library:Name:Version (VLNV)` 标识符提供名称作为该标识符的组成部分。
- `-library <string>`：为生成的 IP 的 `Vendor:Library:Name:Version (VLNV)` 标识符提供库作为该标识符的组成部分。
- `-output <string>`：指定输出 IP、XO 或 DCP 文件的路径和文件名。默认情况下，如果不指定 `-output`，那么此文件将写入当前工程的 `solution/impl` 文件夹。
- `-rtl (verilog | VHDL)`：指定在执行 `-flow` 选项时使用的 HDL。如果不指定此项，那么对于 Vivado 已综合的网表，默认语言为 Verilog。
- `-taxonomy <string>`：指定生成的 IP 的目录条目的分类法，在封装 IP 时使用。

- `-vendor <string>`: 为生成的 IP 的 `Vendor:Library:Name:Version (VLNV)` 标识符提供供应商作为该标识符的组成部分。
- `-version <string>`: 为生成的 IP 的 `Vendor:Library:Name:Version (VLNV)` 标识符提供版本作为该标识符的组成部分。

示例

为 Vitis 应用加速流程导出 RTL:

```
export_design -format xo
```

以 Vivado IP 目录格式导出 RTL 作为 VHDL 代码。VHDL 将在 Vivado 综合工具内进行综合，以获取更好的时序和使用率数据:

```
export_design -rtl vhdl -format ip_catalog -flow syn
```

get_clock_period

描述

此命令用于返回指定时钟对象的时钟周期，或者返回活动解决方案的默认时钟周期。

语法

```
get_clock_period [OPTIONS]
```

选项

- `-default`: 如未指定时钟周期，则返回默认周期。
- `-name <string>`: 获取指定时钟的时钟周期。
- `-units [ns | ps | hz | MHz | GHz]`: 按指定单位返回时钟周期。默认情况下，Vitis HLS 返回的时钟周期所使用的单位与先前指定单位相同。

示例

以下示例用于创建时钟 `ap_clk`，并以 MHz 为单位来指定时钟周期。然后，它将以 ns 为单位获取该时钟的时钟周期:

```
create_clock -name ap_clk -period 200MHz  
get_clock_period -name ap_clk -ns
```

get_clock_uncertainty

描述

此命令用于返回指定时钟的时钟不确定性，或者返回活动解决方案的默认时钟不确定性。

语法

```
get_clock_uncertainty [clock_name]
```

- `<clock_name>` 表示要获取其不确定性的时钟。

选项

- `-default (true | false)`:

如果用户尚未设置不确定性值，则返回默认不确定性值。如为 `true`，则返回默认不确定性（除非用户已设置不确定性值）。如为 `false`，则不返回默认值。

示例

以下示例用于为指定时钟获取时钟不确定性：

```
get_clock_uncertainty clk1
```

get_component

描述

该命令用于获取活动的组件。

语法

```
get_component [OPTIONS]
```

选项

- `-data_json [=true|false]`：返回组件元数据 JSON 文件的绝对路径。
- `-directory [=true|false]`：返回组件目录的绝对路径。
- `-ide_json [=true|false]`：返回组件 Vitis Unified IDE JSON 文件的绝对路径。
- `-ini [=true|false]`：返回组件配置 ini 文件的绝对路径。
- `-name [=true|false]`：返回组件名称。
- `-work_dir [=true|false]`：返回组件 `work_dir` 目录的绝对路径。

get_derived_clock

描述

描述：此命令会为指定时钟的 AXI4-Lite 辅助时钟返回衍生时钟。

语法

```
get_derived_clock [clock_name] <clock_name> indicates the derived clock.
```

get_files

描述

此命令用于获取已添加到活动解决方案中的文件。

语法

```
get_files [OPTIONS]
```

选项

- `-cflags`：返回随文件指定的任意编译器标志。
- `-csimflags`：返回随文件指定的任意 C 语言仿真标志。
- `-fullpath`：返回文件的完整路径。
- `-tb`：仅返回作为测试平台的一部分（使用 `-tb` 选项）添加的文件。

示例

以下示例用于获取从当前解决方案添加的测试激励文件，并返回这些文件的完整路径：

```
get_files -tb -fullpath
```

get_part

描述

此命令用于返回活动解决方案中使用的 AMD 器件。

语法

```
get_part
```

选项

本命令无选项。

示例

以下示例返回活动解决方案中使用的部件：

```
get_part
```

get_project

描述

此命令可获取当前打开的工程的信息。

语法

```
get_project [OPTIONS]
```

选项

- `-directory`：返回指向工程路径的完整路径。
- `-ide (classic|vitis)`：（默认为 `classic`）返回工程 IDE 设置。
- `-name`：返回工程名称。
- `-solutions`：返回工程中所有解决方案名称的列表。

示例

以下示例用于获取当前工程的完整路径：

```
get_project -directory
```

get_solution

描述

此命令用于返回活动解决方案的相关信息。

语法

```
get_solution [OPTIONS]
```

选项

- `-directory`：返回指向处于活动状态的解决方案的完整路径。
- `-flow_target`：返回处于活动状态的解决方案的给定流程目标。
- `-json`：返回指向解决方案元数据 JSON 文件的绝对路径。
- `-name`：返回解决方案名称。

示例

以下示例用于返回活动解决方案的流程目标：

```
get_solution -flow_target
```

get_top

描述

此命令用于返回已打开的 Vitis HLS 工程的顶层函数名称。

语法

```
get_top
```

选项

本命令无选项。

示例

以下示例会返回已打开的工程顶层函数：

```
get_top
```

help

描述

- 指定的 `help` 命令如果不含命令名称，则会列出所有 Vitis HLS Tcl 命令。
- 搭配 Vitis HLS Tcl 命令作为实参来使用时，`help` 命令则会返回指定命令的详细信息。



提示：对于已识别 Vitis HLS 命令，输入命令名称时支持使用 Tab 键自动补全。

语法

```
help <cmd>
```

- `<cmd>` 指定要为其返回帮助信息的命令名称。如不指定任何命令，则返回所有 Vitis HLS 命令的列表。

选项

- `cmd <string>`：命令名称
- `-ini [=true|false]`：显示 ini 命令的帮助信息
- `-list [=true|false]`：列出匹配的帮助条目，而不是显示完整帮助
- `-width <int:120>`：设置输出宽度

示例

显示所有命令和指令的帮助：

```
help
```

显示 `add_files` 命令的帮助：

```
help add_files
```

list_part

描述

此命令用于返回受支持的器件家族、部件或开发板的名称。

如不提供实参，则此命令将返回所有受支持的部件家族。要返回某一家族的特定部件，请提供该家族的名称作为实参。

语法

```
list_part [name] [OPTIONS]
```

- `<family>` 指定要返回的特定器件所属的器件家族。如不指定 `<family>`，则 `list_part` 命令会返回可用器件家族的列表。

选项

- `-name <string>`：家族、部件、开发板名称或 glob 模式。
- `-board[=false|true]`：返回开发板名称（而不是部件名称）的列表。
- `-clock_regions`：返回指定部件的时钟区域列表。必须搭配 `-name` 命令一起指定。
- `-slr_pblocks`：返回指定部件的 SLR Pblock 词典。必须搭配 `-name` 命令一起指定。

示例

返回所有受支持的器件家族。

```
list_part
```

返回活动解决方案中当前部件的时钟区域。

```
list_part -name [get_part] -clock_regions
```

open_component

描述

`open_component` 命令可打开一个现有组件或创建一个新组件。

语法

```
open_component [name] [OPTIONS]
```

选项

- `-config <string>`：覆盖已生成的 `config.ini` 文件的默认位置。
- `-flow_target <vivado|vitis>`：为接口、流程和工具默认值指定目标。
 - Vivado：Vivado IP 流程目标

- Vitis: Vitis 内核流程目标
- `-reset [=true|false]`: 移除已存在的所有数据，以将组件复位。
- `-write_ini [=true|false]`: 自动将所有解决方案数据保存到 `config.ini` 文件。

open_project

描述

打开现有工程，或者如果指定工程不存在，则创建新工程。



重要提示! 在 Vitis HLS 内，使用 `open_project` 命令打开 Vivado HLS 工程时，除非使用 `-upgrade` 或 `-reset` 选项，否则此命令将返回错误。

在每个 Vitis HLS 会话内，仅限存在一个处于活动状态的工程。要关闭工程，请运行以下操作：

- 使用 `close_project` 命令，或者
- 使用 `open_project` 或 `open_tcl_project` 命令打开或创建另一个工程。

使用 `delete_project` 命令可完整删除工程目录（将其从硬盘中移除）以及与之关联的所有解决方案。

语法

```
open_project [OPTIONS] <name>
```

- `<name>` 用于指定工程名称。

选项

- `-ide (classic|vitis)`: 返回工程 IDE 设置。
- `-reset`:
 - 移除工程内已存在的所有数据即可将工程复位。
 - 移除设计源文件上的所有原有工程信息、头文件搜索路径和顶层函数。关联的解决方案目录和文件都予以保留，但可能包含无效的结果。



建议: 使用 Tcl 脚本执行 Vitis HLS 时，请使用 `-reset` 选项。否则，每条新的 `add_files` 命令都会向现有数据添加更多文件。

- `-upgrade`: 将 Vivado HLS 工程升级到 Vitis HLS。

示例

打开名为 `Project_1` 的现有工程，或者如果该工程不存在，则创建新工程：

```
open_project Project_1
```

打开工程，并移除所有现有数据：

```
open_project -reset Project_2
```

open_solution

描述

在当前活动的工程内打开现有解决方案或创建新解决方案。在 Vitis HLS 会话中，任意给定时间仅限存在 1 个处于活动状态的解决方案。如 [目标流程概述](#) 中所述，该解决方案目标为 Vivado IP 流程或 Vitis 内核流程。如不指定流程目标，则默认流程为 Vivado IP 流程。



重要提示！ 没有任何工程处于打开状态的前提下尝试打开或创建解决方案都将导致出错。

在当前工程中，每个解决方案有 1 个对应子目录用于对其进行管理。如果在打开的工程内不存在指定的解决方案，则会创建新的解决方案。要关闭解决方案，请运行以下操作：

- 运行 `close_solution` 命令，或者
- 使用 `open_solution` 命令打开另一个解决方案。

使用 `delete_solution` 命令从工程中移除解决方案并删除对应的子目录。

语法

```
open_solution [OPTIONS] <name>
```

- `<name>` 用于指定解决方案名称。



提示： 您可指定工程名称和解决方案名称，这样使用 `open_solution` 通过一条命令即可打开工程和解决方案：`open_solution dctProj/solution1`

选项

- `-flow_target [vitis | vivado]:`
 - `vivado`：将解决方案配置为在支持 Vivado IP 生成流程的前提下运行，其中包括要求严格按标准使用编译指示和指令，并将结果导出为 Vivado IP。不指定 `-flow_target` 时，这是默认流程。
 - `vitis`：配置解决方案，以供在 Vitis 应用加速开发流程中使用。这样即可将 Vitis HLS 工具配置为无需指定 `INTERFACE` 编译指示或指令即可正确推断函数实参的接口，并将综合后的 RTL 代码作为 Vitis 内核对象文件 (`.xo`) 来输出。
- `-reset:`
 - 如果解决方案已存在，则复位解决方案数据。这样将删除有关库、约束和指令的所有先前的解决方案信息。
 - 同时还会移除综合、验证和实现结果。

示例

在打开的工程内，打开名为 `Solution_1` 的现有解决方案，或者如果不存在含有指定名称的解决方案，则创建新的解决方案。该解决方案经配置后，即可用于创建内核对象 (`.xo`)，用于 Vitis 工具流程。

```
open_solution -flow_target vitis Solution_1
```

在打开的工程内，打开并复位指定的解决方案。从该解决方案中移除所有现有数据。

```
open_solution -reset Solution_2
```

open_tcl_project

描述

使用 `source` 命令运行 Tcl 文件来创建工程，但跳过 Tcl 脚本中的所有设计命令：`cosim_design`、`csynth_design` 和 `csim_design`。此命令仅从 Tcl 脚本创建并配置工程。这样您即可使用来自现有工程的 Tcl 脚本创建工程，而无需运行仿真或综合。

在每个 Vitis HLS 会话内，仅限存在一个处于活动状态的工程。要关闭工程，请运行以下操作：

- 使用 `close_project` 命令，或者
- 使用 `open_tcl_project` 或 `open_project` 命令打开或创建另一个工程。

使用 `delete_project` 命令可完整删除工程目录（将其从硬盘中移除）以及与之关联的所有解决方案。

语法

```
open_tcl_project <tclfile>
```

- `<tclfile>` 用于指定在创建工程时要使用的 Tcl 脚本的路径和名称。

选项

此命令不含任何选项。

示例

从指定的 Tcl 脚本创建并打开工程：

```
open_tcl_project run_hls.tcl
```

project2ini

描述

从现有工程创建配置文件（INI 格式）。这样您就可以指定工程名称，并从工程中生成配置文件。将 HLS 工程的解决方案转换为 INI 文件。

语法

```
project2ini <project> [OPTIONS]
```

- `<project>` 指定要用于生成 INI 文件的工程名称。

选项

- `-all [=true|false]`：写入所有选项，与是否已设置当前值无关。可能的值为 `true` 和 `false`，默认值为 `false`。
- `-filepaths <absolute|none|relative>`：控制输出中的相对文件路径的处理方式
- `-fileroot <string>`：指定文件根目录路径，后续生成的文件路径均为该路径的相对路径（默认为 `inifile` 目录）

- `-outdir <string>`: 输出目录名称，默认为工程目录的名称
- `-package_defaults [=true|false]`: 包含封装默认值
- `-quiet [=true|false]`: 禁止所有消息（仍将抛出错误）
- `-show [=true|false]`: 显示详细消息
- `-solution <string:*>`: 要转换的解决方案名称（glob 模式）

示例

:

```
project2ini Project_1
```

set_clock_uncertainty

描述

在 `create_clock` 定义的时钟周期上设置裕度。

将从时钟周期中扣除不确定性的裕度，以创建有效的时钟周期。时钟不确定性是以 `ns` 或者时钟周期的百分比来定义的。时钟不确定性默认设为时钟周期的 27%。

Vitis HLS 会根据有效时钟周期对设计进行优化，为下游工具提供裕度以便进行逻辑综合和布线。此命令只能在处于活动状态的解决方案关联环境中执行。Vitis HLS 仍使用所有输出文件中指定的时钟周期进行验证和实现。

语法

```
set_clock_uncertainty <uncertainty> <clock_list>
```

- `<uncertainty>`: 是表示时钟周期中用作为裕度的量的值（以 `ns` 为单位）。不确定性还可按时钟周期百分比来指定。默认不确定性为时钟周期的 27%。
- `<clock_list>`: 是对其应用不确定性的时钟的列表。如果未提供任何时钟，则表示对所有时钟应用不确定性。

选项

此命令不含任何选项。

示例

在时钟上指定不确定性或裕度为 0.5 ns。这样可将 Vitis HLS 可使用的时钟周期有效缩短 0.5 ns。

```
set_clock_uncertainty 0.5
```

set_part

描述

为当前解决方案设置目标器件、器件家族或开发板。此命令只能在处于活动状态的解决方案关联环境中执行。



提示：工程中的每个解决方案均可将一个独立器件或器件家族定为目标。

语法

```
set_part <device_specification>
```

- <device_specification> 表示器件规格，用于为 Vitis HLS 综合与实现设置目标器件。
- 器件规格包括 <device>、<package> 和 <speed_grade> 信息。
- 指定 <device_family> 表示使用器件家族的默认器件。

选项

- -board：按开发板上的定义来指定部件。

示例

您可以通过提供器件家族名称，将随 Vitis HLS 提供的 FPGA 库添加到当前解决方案中，如下所示。在此情况下，将使用 Vitis HLS FPGA 库中为 Virtex 7 器件家族指定的默认器件、封装和速度等级。

```
set_part virtex7
```

借助随 Vitis HLS 提供的 FPGA 库，您也可以选择指定含有封装和等级信息的特定器件。

```
set_part xc6v1x240tff1156-1
```

按开发板的定义来指定部件。

```
set_part -board u200
```

set_top

描述

定义要综合的顶层函数。



重要提示！从顶层函数调用的任意函数都将成为 HLS 设计的一部分。

语法

```
set_top <name>
```

- <name> 是 HLS 要综合的函数。

选项

此命令不含任何选项。

示例

将顶层函数设置为 `foo`。

```
set_top foo
```

write_component

描述

生成 Vitis Unified IDE `vitis-comp.json` 文件。

语法

```
vitis-comp.json [OPTIONS]
```

选项

- `-config <string>`：配置 ini 文件。
- `-directory <string.>`：输出目录。
- `-name <string>`：组件名称。
- `-reset [=true|false]`：若为 `true`，则覆盖所有现有信息。
- `-show [=true|false]`：显示详细消息。
- `-work_dir <string>`：组件目录。

write_ini

描述

将当前工程设置写入指定 INI 文件。这可用于将工程导出到 INI 文件，而不是为该工程创建 Tcl 脚本。您可使用 `apply_ini` 将 INI 读入 Vitis HLS 工具，或者将其与 `v++ -c --mode hls` 命令搭配使用。

语法

```
write_ini <ini_file> [OPTIONS]
```

`<ini_file>` 用于指定要创建的 INI 文件。

选项

- `-all <true | false>`：写入所有选项，与是否已设置当前值无关。这样可确保写入 INI 文件时，连默认值也会一并导出到此 INI 文件。
- `-appfile <string>`：将使用此处指定的工程 `app` 文件代替当前打开的工程。可用于从当前未打开的工程生成 INI 文件。
- `-apsfile <string>`：将使用此处指定或来自外部工程的解决方案 `aps` 文件代替当前打开的解决方案。

- `-filepaths <absolute|relative|none>`: 控制输出中的相对文件路径的处理方式。默认值为 `none`。
- `-fileroot <string>`: 该选项搭配 `-filepaths relative` 选项一起使用，用于指定文件根目录路径，后续生成的文件路径均为该目录路径的相对路径。默认设为从中启动工具的目录的相对路径。
- `-include <string>`: 指定 glob 模式要包含的字符串。
- `-package_defaults [=true|false]`: 将各包的配置设置默认值写入 INI 文件。
- `-quiet <true | false>`: 写入 INI 文件时禁止所有警告消息和参考消息。但仍将返回错误。
- `-show <true | false>`: 写入 INI 文件时显示详细消息。
- `-solution_only [=true|false]`: 在生成的 INI 文件中不包含工程数据。
- `-workdir <string>`: 在生成的 INI 文件中，指定工作目录。

示例

这将把当前工程导出到指定 INI 文件，并导出所有选项，包括用户指定的选项和默认值：

```
write_ini project1.ini -all=true
```

配置命令

配置命令允许您配置 AMD Vitis™ HLS 工具以控制综合与仿真的结果、指定编译指示和指令的默认设置并指定默认生成的输出。这些命令必须以交互模式 `vitis_hls -i` 来运行，或者也可以使用 `-f` 选项作为脚本来运行，如 [vitis](#)、[v++](#) 和 [vitis-run](#) 命令中所述。

您还可在 Vitis HLS IDE 中使用“Solution Settings”（解决方案设置）对话框设置这些配置命令，如 [定义 HLS 配置文件](#) 中所述。

config_array_partition

描述

指定阵列分区的默认行为。

语法

```
config_array_partition [OPTIONS]
```

选项

- `-throughput_driven <off | auto>`:

启用自动部分和/或完整阵列分区（也适用于含 HLS STREAM 编译指示的阵列）。

- `auto`: 启用自动阵列分区，支持智能平衡面积与吞吐量。这是默认值。
- `off`: 禁用自动阵列分区。

- `-complete_threshold <uint:4>`:

设置完整阵列分区的阈值。所含元素数量少于指定阈值的阵列将完整分区为各元素。

示例

将设计中含不足 12 个元素的所有阵列（全局阵列除外）都分区为各元素。

```
config_array_partition -complete_threshold 12
```

config_compile

描述

配置前端编译的默认行为。

语法

```
config_compile [OPTIONS]
```

选项

- `-double_acc_bits=N`: 浮点类型 `double` 的累加器精度。默认值为 269。
- `-enable_auto_rewind[=true|false]`: 设为 `TRUE` 时，使用流水打拍循环的备用 HLS 实现，此实现支持自动循环回绕。接受的值为 `TRUE` 或 `FALSE`。默认值为 `TRUE`。
- `-float_acc_bits=M`: 浮点类型 `float` 的累加器精度。默认值为 45。
- `-ignore_long_run_time[=true|false]`: 不报告“long runtime”（运行时过长）警告。接受的值为 `TRUE` 或 `FALSE`。默认值为 `FALSE`。
- `-name_max_length <value>`: 指定函数名称的最大长度。如果名称长度超过阈值，则名称的最后一部分将被截位，并按需添加数字以使名称唯一。默认值为 256。
- `-no_signed_zeros[=true|false]`: 忽略浮点 0 值的符号类型，以便编译器对浮点运算执行主动优化。接受的值为 `TRUE` 或 `FALSE`。默认值为 `FALSE`。



重要提示！ 使用该选项可能改变任意浮点运算的结果，导致 C/RTL 协同仿真不匹配。请确保测试平台容许差异，并检查差异的裕度，而不是精确值。

- `-pipeline_flush_in_task <always | never | ii1>`: 指定默认情况下，在 `hls::tasks` 中将刷新流水线 (`flp`) 以降低 C/RTL 协同仿真中出现死锁的概率。该选项适用于以默认选项 `ii1` 达成 `II=1` 的流水线。但您也可以通过应用 `always` 启用在 `hls::tasks` 或数据流中刷新流水线来指定该选项，或者也可以使用 `never` 来完全禁用该选项。如需了解更多信息，请参阅[刷新流水线和流水线类型](#)。



重要提示！ 同时启用 `config_compile -enable_auto_rewind` 选项时，刷新流水线 (`flp`) 与 PIPELINE 编译指示或指令中指定的 `rewind` 选项兼容。

- `always`: 在 `hls::tasks` 或数据流中始终使流水线可刷新，忽略 `II`。
- `never`: 从不使流水线变为可刷新，除非其他指令或编译指示明确覆盖此设置。
- `ii1`: 在 `hls::tasks` 中将可实现 `II=1` 的流水线设为可刷新。这是默认设置。

- `-pipeline_loops <threshold>`:

指定对循环和嵌套循环进行自动流水打拍的下限。对于单个未嵌套循环，如果该循环的循环次数大于指定阈值，那么该循环将采用流水打拍。默认值为 64，这会导致 Vitis HLS 对循环次数高于 64 的循环进行自动流水打拍。例如，如果循环次数为 65，那么该循环将按默认阈值进行流水打拍，但循环次数为 64 的循环则不执行流水打拍。

对于嵌套循环，将从最内层循环开始，对这些循环成对进行求值。如果内层循环次数超过指定阈值，那么会将其标记为流水打拍，并继续对成对循环中的父循环执行分析。如果父循环的循环次数超出指定阈值，那么也会将其标记为流水打拍，而内层循环（先前已标记为流水打拍）会展开到父循环内。如果还有其他循环，则会继续向上执行分析。

但其他优化操作会影响此分析，并且可能阻碍您要执行的流水打拍行为。例如，您可能需要使用 `loop_flatten off` 防止平铺并促进循环流水打拍。

- `-pipeline_style <stp | flp | frp>`: 指定 Vitis HLS 基于以上指定的 `-pipeline_loops` 阈值，用于 PIPELINE 编译指示或指令的流水线的默认类型，或者用于循环流水打拍的默认类型。如需了解有关流水线样式的更多信息，请参阅 [刷新流水线和流水线类型](#)。



重要提示！ 这是提示，而非硬性约束。工具会检查设计启用流水打拍的条件。部分循环可能不遵循特定样式，如果需要，工具会还原为默认样式 (stp)。

- `flp`: 可刷新流水线架构：当输入数据不可用时刷新，随后停滞并等待新数据。
- `frp`: 自由运行的可刷新流水线。即使输入数据不可用也可运行。如果由于流水线控制信号扇出减少而导致需要更好的时序，或者如果需要提升性能以免死锁，则可使用此类型。但此流水线样式可能功耗更大，因为即使没有数据，流水线寄存器仍受时钟驱动。
- `stp`: 停滞型流水线。仅当输入数据可用时才运行，否则停滞。无需可刷新的流水线（例如，不存在因停滞引发的性能或死锁问题）时，这是默认风格。
- `-pragma_strict_mode [=true|false]`: 为位置错误或使用错误的编译指示启用错误消息。
- `-pre_tcl <arg>`: 指定在启动 `csynth_design` 命令前运行的 Tcl 脚本。
- `-performance_budgeter <autodisable|enable>`:

启用该选项后，顶层性能编译指示会对整个设计进行全面的性能分析。就像在从 A 点到 B 点的众多路线可能性中选择最有效的路线一样，它在整个设计中考考虑多种优化策略，以识别达到性能目标的最佳路径。

“enable”（启用）：激活性能预算，对整个设计进行综合性能分析。

“disable”（禁用）：停用性能预算，阻止执行任何综合分析。

“auto”（自动）：如果检测到性能编译指示，则自动为整个设计启用性能分析；否则保持禁用性能预算。

- `-unsafe_math_optimizations [=true|false]`: 忽略浮点 0 值的符号类型并启用可结合的浮点运算，以便编译器对浮点运算执行积极优化。接受的值为 TRUE 或 FALSE。默认值为 FALSE。

注释： 使用该选项可能改变任意浮点运算的结果，导致 C/RTL 协同仿真不匹配。请确保测试平台容许差异，并检查差异的裕度，而不是精确值。

- `-use_csim_directives [=true|false]`: 在 C 语言综合期间使用由 C 语言仿真生成的指令。

示例

对所含循环次数高于 30 的最内层循环进行流水打拍，或者如果最内层循环的循环次数小于或等于 30，则对其父循环进行流水打拍：

```
config_compile -pipeline_loops 30
```

忽略浮点 0 值的符号类型：

```
config_compile -no_signed_zeros
```

忽略浮点 0 值的符号类型并启用可结合的浮点运算：

```
config_compile -unsafe_math_optimizations
```

config_cosim

描述

支持您配置 C/RTL 协同仿真命令 (`cosim_design`) 的设置。

语法

```
config_cosim [OPTIONS]
```

选项

- `-argv <string>: <string>` 传递到 C/C++ 主函数。
指定行为测试平台的实参列表。
- `-compiled_library_dir <string>`：指定使用第三方仿真器进行仿真期间已编译的库目录。`<string>` 是已编译的库目录的路径名称。此库必须事先使用 `compile_simlib` 命令完成编译，如《Vivado Design Suite Tcl 命令参考指南》(UG835) 所述。
- `-coverage`：启用使用 VCS 仿真器进行仿真期间的覆盖率功能。
- `-disable_deadlock_detection`：禁用协同仿真中的死锁检测功能。
- `-disable_dependency_check`：运行协同仿真时禁用依赖关系检查。
- `-enable_dataflow_profiling`：该选项会开启数据流通道剖析以跟踪协同仿真期间的通道大小。
- `-enable_fifo_sizing`：在协同仿真期间，该选项可开启自动 FIFO 通道大小调整，以便进行数据流剖析。
- `-enable_tasks_with_m_axi`：启用稳定的 `m_axi` 接口，以搭配 `hls::task` 一起使用。
- `-hwemu_trace_dir <kernel_name>/<instance_name>`：指定测试矢量的位置，这些矢量是在硬件仿真期间生成的，将在协同仿真期间使用。这些测试矢量是由 `config_export -cosim_trace_generation` 命令生成的。该实参允许您在硬件仿真的仿真结果中指定 Vitis 内核的内核和实例名称，以便定位这些测试矢量。
- `-random_stall`：在协同仿真期间启用顶层接口的随机停滞。
- `-rtl [verilog | vhdl]`：指定用于 C/RTL 协同仿真的 RTL 语言。默认为 Verilog。

- `-setup`: 创建仿真文件，包括在处于活动状态的解决方案的 `sim/<HDL>` 目录中创建的所有仿真文件。不执行仿真，但稍后可从命令外壳运行仿真。
- `-stable_axilite_update`: 启用 `s_axilite` 以配置寄存器，这些寄存器与传输事务前相比均保持稳定状态。
- `-trace_level [*none* | all | port | port_hier]`: 判定 C/RTL 协同仿真期间，要保存的波形追踪数据级别。
 - `none` 表示不保存追踪数据。这是默认方式。
 - `all` 表示生成要保存到追踪文件的所有端口和信号波形。
 - `port` 表示仅为顶层端口保存波形追踪数据。
 - `port_hier` 表示保存设计层级内的所有端口的追踪信息。

根据 `-rtl` 选项中使用的选择，执行仿真时，追踪文件将保存在当前解决方案的 `sim/Verilog` 或 `sim/VHDL` 文件夹内。

- `-user_stall <string>`: 指定协同仿真期间要使用的 JSON 停滞文件。此停滞文件可使用 `cosim_stall` 命令来生成。
- `-wave_debug`: 打开 AMD Vivado™ 仿真器 GUI 以查看波形和仿真结果。在生成的 RTL 中启用查看所有进程的波形的功能，就像在数据流进程和顺序进程中一样。仅当通过设置 `-tool xsim` 使用 Vivado 仿真器来执行系统仿真时，才支持该选项。如需了解更多信息，请参阅 [查看仿真波形](#)。

config_csim

描述

允许您配置 C 语言仿真命令 (`csim_design`) 的设置。

语法

```
config_csim [OPTIONS]
```

选项

- `-argv <string>`: 指定行为测试平台的实参列表。`<string>` 会被传递到测试平台的 `main()` C/C++ 函数。
- `-clean`: 启用纯净构建。如果不选择此选项，`csim_design` 将进行增量编译。
- `-sanitize_address`: 为地址启用 Clang Sanitizer。
- `-sanitize_undefined`: 为未定义行为启用 Clang Sanitizer。
- `-setup`: 指定该选项时，将在活动解决方案的 `csim` 目录中创建仿真二进制文件，但不执行仿真。稍后，可从已编译的可执行文件启动仿真。
- `-profile_tripcount`: 启用循环次数剖析。

config_dataflow

描述

- 指定数据流流水打拍（通过 `set_directive_dataflow` 命令实现）的默认行为。

- 允许您指定默认通道存储器类型和深度。

语法

```
config_dataflow [OPTIONS]
```

选项

- `-default_channel [fifo | pingpong]`: 默认情况下, 在使用数据流流水打拍时, 以 `pingpong` 方式配置的 RAM 存储器用于缓冲函数或循环之间的数据。使用串流数据时 (即数据始终连续读写), FIFO 存储器更高效且可选作为默认存储器类型。



提示: 使用 `set_directive_stream` 命令将阵列设置为串流以执行 FIFO 访问。

- `-disable_fifo_sizing_opt`: 禁用导致可能增大资源使用量的 FIFO 大小调整优化, 这样可能可改善性能并减少死锁。
- `-enable_canonicalization [=true|false]`: 启用变换为数据流规范形式 (若可行)。默认该选项设为 `true`。
- `-fifo_depth <integer>`: 指定 FIFO 的默认深度。默认深度为 2。

使用乒乓 c 存储器时, 该选项无效。如果不指定该选项, 默认深度为 2, 或者如果这是转换为 FIFO 的阵列, 则默认大小为原始阵列的大小。在某些情况下, 这可能过于保守, 导致 FIFO 过大。当您确认 FIFO 过大时, 可使用该选项。



注意! 请谨慎使用该选项。FIFO 深度不足可能导致死锁。

- `-override_user_fifo_depth <value>`:

针对每个 `hls::stream` 使用指定深度, 覆盖任何用户设置。

注释: 这可用于检查是否因设计中 FIFO 深度不足而导致出现死锁。只需将其设置为超大值 (例如, 仿真结束时协同仿真打印的最大深度), 如果存在死锁, 那么您就可以使用协同仿真的 FIFO 深度剖析选项和 GUI 来找到最小的深度, 以保障性能并避免死锁。

- `-scalar_fifo_depth <integer>`: 针对标量传播 FIFO 请指定最小值。

通过标量传播, 编译器会将标量从 C/C++ 语言代码转换为 FIFO。可通过 `-scalar_fifo_depth` 设置这些 FIFO 的最小大小。如果不提供该选项, 则使用 `-fifo_depth` 的值。

- `-start_fifo_depth <integer>`: 指定起始传播 FIFO 的最小深度。

仅当生产者与使用者间通道为 FIFO 时, 该选项才有效。该选项使用的默认值与 `-fifo_depth` 选项同为 2。此 FIFO 有时可能导致死锁, 在此情况下您可使用该选项来增加 FIFO 的深度。

- `-strict_mode [off | warning | error]`: 为数据流规范形式相关的消息设置严重性。
- `-strict_stable_sync[=true|false]`: 强制对 `ap_done` 的稳定端口进行同步。
- `-task_level_fifo_depth <integer>`: 指定任务级别 FIFO 的深度。

通过 `ap_ctrl_chain` 来同步 FIFO。写操作是生产者的 `ap_done`, 读操作则是使用者的 `ap_ready`。同步方面与 PIPO 相似, 访问方面则与 FIFO 相似。

示例

将默认通道从乒乓存储器更改为 FIFO：

```
config_dataflow -default_channel fifo
```

将默认通道从乒乓存储器更改为深度为 6 的 FIFO：

```
config_dataflow -default_channel fifo -fifo_depth 6
```



注意！ 如果设计实现需要包含超过 6 个元素的 FIFO，此设置会导致设计的 RTL 验证失败。请谨慎使用该选项，因为它属于用户覆盖选项。

要找到死锁原因，请尝试显著增大所有 FIFO 深度，尤其是 C/RTL 协同仿真报告的 FIFO。如果当“N”值较大时死锁消失，则问题是由于 FIFO 深度不足导致的。您可按如下方式进行测试：

```
config_dataflow -fifo_depth N -start_fifo_depth N -scalar_fifo_depth N  
-task_level_fifo_depth N
```

config_debug

生成 Vitis 应用加速开发流程中使用的 HLS 调试文件。

描述

配置前端编译的默认行为。

语法

```
config_debug [OPTIONS]
```

选项

- `-directory <path>`：指定 HLS 调试输出的位置。如不指定，则此位置设置为 `solution/.debug`。
- `-enable[=true|false]`：启用生成 HLS 调试文件，以供在 Vitis 调试视图内使用。

示例

以下示例会启用调试文件：

```
config_debug -enable true
```

config_export

描述

为 [export_design](#) 配置选项，用于运行下游工具，或者用于封装 Vivado IP 或 Vitis 编译的内核对象（.xo）。

语法

```
config_export [OPTIONS]
```

选项

- `-cosim_trace_generation=<true | false>`: 硬件仿真期间, 当内核作为 Vitis 内核进行综合时, 在 Vitis 工具流中生成测试矢量, 以供后续迭代中的 C/RTL 协同仿真期间使用。
- `-description <string>`: 提供生成的 IP 的目录条目描述, 在封装 IP 时使用。
- `-display_name <string>`: 提供生成的 IP 的目录条目的显示名称, 在封装 IP 时使用。
- `-enable_verbose_report`: 启用详细报告。
- `-flow (none | syn | impl)`: 支持您使用 Vivado 综合与实现为生成的 RTL 获取更准确的时序和资源使用率数据。`syn` 选项用于执行 RTL 综合。`impl` 选项用于执行 RTL 综合与实现, 包括 RTL 网表的详细布局布线。默认选项是 `none`, 表示不运行综合或实现。



提示: 在 Vitis HLS IDE 中, 这些选项显示为带有“Vivado Synthesis” (Vivado 综合) 和“Vivado Synthesis, place and route stage” (Vivado 综合与布局布线阶段) 标记的复选框。

- `-format (ip_catalog | xo | syn_dcp)`: 指定 IP 封装格式。受支持的格式为:
 - `ip_catalog`: 适合添加到 AMD IP 目录的格式。
 - `xo`: v++ 编译器接受的格式, 用于在 Vitis 应用加速流程中进行链接。
 - `syn_dcp`: Vivado Design Suite 的已综合的检查点。如果使用该选项, 则将自动执行 RTL 综合。无法以可选方式添加 Vivado 实现。
 - `sysgen`: 生成 Vivado IP 和 .zip 存档, 以供在 System Generator 内使用。
- `-ip_xdc_file <arg>`: 指定 XDC 文件, 其中内容包含在已封装的 IP 内, 以供在 Vivado 工具实现期间使用。
- `-ip_xdc_ooc_file <arg>`: 指定非关联 (OOC) XDC 文件, 其中内容包含在已封装的 IP 内, 以供在针对导出的 IP 执行非关联 Vivado 综合期间使用。
- `-ipname <string>`: 为生成的 IP 的 `<Vendor>:<Library>:<Name>:<Version>` (VLNV) 标识符提供名称作为该标识符的组成部分。
- `-library <string>`: 为生成的 IP 的 `<Vendor>:<Library>:<Name>:<Version>` (VLNV) 标识符提供库作为该标识符的组成部分。
- `-output <string>`: 指定生成的 IP、.xo 文件或 DCP 文件的输出位置。如果不指定输出路径, 那么此文件将写入当前工程的 `solution/impl` 文件夹。
- `-rtl (verilog | VHDL)`: 指定在执行 `-flow` 选项时使用的 HDL。如果不指定此项, 那么对于 Vivado 已综合的网表, 默认语言为 Verilog。
- `-taxonomy <string>`: 指定生成的 IP 的目录条目的分类法, 在封装 IP 时使用。
- `-vendor <string>`: 为生成的 IP 的 `<Vendor>:<Library>:<Name>:<Version>` (VLNV) 标识符提供供应商作为该标识符的组成部分。
- `-version <string>`: 为生成的 IP 的 `<Vendor>:<Library>:<Name>:<Version>` (VLNV) 标识符提供版本作为该标识符的组成部分。

- `-vivado_clock <arg>`: 覆盖 Vivado OOC 运行期间使用的指定 HLS 时钟约束。此选项仅用于报告，将不适用于导出的 IP。
- `-vivado_impl_strategy <string>`: 指定 Vivado 实现策略名称。默认名称为 “default”。
- `-vivado_max_timing_paths <uint:10>`: 指定在 Vivado 综合或实现中不满足时序的情况下，要报告的时序路径的最大数量。
- `-vivado_optimization_level (0 | 1 | 2 | 3)`: Vivado 优化级别。该选项用于设置其他 `vivado_*` 选项。它仅用于报告生成，将不适用于导出的 IP。默认设置为 0。
- `-vivado_pblock <arg>`: 指定要在实现期间用于报告目的的 Pblock 范围。这将不适用于导出的 IP。
- `-vivado_phys_opt (none | place | route | all)`: 指定在 Vivado 实现期间是否应运行 Vivado 物理优化。有效值包括：
 - `none`: 不运行（默认选项）。
 - `place`: 布局后运行。
 - `route`: 布线后运行。
 - `all`: 布局后和布线后运行。
- `-vivado_report_level (0 | 1 | 2)`: 指定生成的 Vivado 报告的数量，不适用于导出的 IP。有效值和关联的报告包括：
 - 0: 综合后使用率。实现后使用率和时序。
 - 1: 综合后使用率、时序和分析。实现后使用率、时序和分析。
 - 2: 综合后使用率、时序、分析和快速失败 (failfast)。实现后使用率、时序和快速失败。这是默认设置。
- `-vivado_synth_design_args <string>`: 指定要传递给 Vivado `synth_design` 命令的其他实参。默认值为 `-directive sdx_optimization_effort_high`。
- `-vivado_synth_strategy <string>`: 指定 Vivado 综合策略名称。默认策略为 “default”。

示例

以下示例用于将 Vitis .xo 导出至指定文件：

```
export_design -description "Kernel Export" -display_name kernel_export \
-flow impl -format xo -output "tmp/hls_tests/kernel.xo"
```

config_interface

描述

指定默认接口选项，这些选项用于在接口综合期间实现每个函数的 RTL 端口。

语法

```
config_interface [OPTIONS]
```

选项

- `-ap_memory_io`: [=bus | port]: 默认设置为 port。指定将在 RTL 和 IP 中对 ap_memory 接口进行建模的方式。
- `-clock_enable` [=true|false]: 向设计添加时钟使能端口 (ap_ce)。默认为 false。

时钟使能处于低电平有效状态时会阻止所有时钟操作。它会禁用所有后续操作

- `-default_slave_interface` [none | s_axilite]: 启用从接口的默认值，将其设为 none (Vivado IP 流程的默认值) 或者设为 s_axilite (Vitis 内核流程的默认值)，如 [目标流程概述](#) 中所述。
- `-m_axi_addr64` [=true|false]: 在设计中全局启用所有 m_axi 端口的 64 位寻址。默认情况下，针对 Vitis 流程启用该选项，否则禁用。
- `-m_axi_alignment_byte_size` <size>: 为 m_axi 接口指定存储器对齐边界，这些接口作为位宽提供。<size> 值必须为有效的 2 次幂值。值为 0 无效。当 `open_solution -flow_target vitis` 时，默认值为 64，当 `-flow_target=vivado` 时，值为 1，并对齐到单个字节。



重要提示！ 如果指针在运行时未能对齐，则会发生错误的突发行为。

- `-m_axi_auto_id_channel` [=true|false]: 为 m_axi 接口启用自动通道 ID 分配。如需了解更多信息，请参阅 [M_AXI 通道](#)。
- `-m_axi_auto_max_ports` [=true|false]: 如果该选项为 true，那么未通过 INTERFACE 编译指示或指令显式捆绑的所有 m_axi 接口都会映射到各独立接口，从而增加资源使用（多个适配器）。默认值为 false 且 m_axi 端口捆绑到单个接口。
- `-m_axi_buffer_impl` [auto | lutram | uram | bram]: 选择所有 m_axi 内部缓冲器的实现。选项包括：auto、lutram、bram 和 uram。默认值为 bram。
- `-m_axi_cache_impl` [auto | lutram | uram | bram]:

指定实现资源以供添加到 m_axi 适配器的高速缓存使用。选项包括：auto、lutram、bram 和 uram。默认值为 auto。

- `-m_axi_conservative_mode` [=true|false]: 此模式会告知 m_axi 不发出写入请求，直至关联的写入数据完全可用为止（通常，缓冲至适配器内或者已发射）。它使用 MAXI 适配器内的缓冲器来存储突发的所有数据，读取和写入时都是如此。默认启用此模式 (true)，这样可能导致写入时延有所增加，但是可以解决由于存储器子系统中的并发请求（读取或写入请求）而导致的死锁。将此功能设为 false 即可将其禁用。



提示： 如果设计实现的缓冲大于使用替代方法时的最大写入突发长度，即可安全地将其模式设为 false 以节省此内部缓冲。

- `-m_axi_flush_mode`: 将 m_axi 配置器配置为可刷新，如果由于下列原因导致突发中断，则写入或读取垃圾数据：流水线阻塞、缺失数据输入（非保守模式下），或者缺失输出空间。默认值为 false。指定该选项时启用此设置。
- `-m_axi_latency` <latency>: 全局指定 m_axi 接口的期望时延，允许设计发起总线请求的时间比执行期望的读取或写入操作早数个周期（时延）。当 `open_solution -flow_target vitis` 时，默认值为 64，当 `-flow_target vivado` 时，值为 0。
- `-m_axi_max_bitwidth` <size>: 指定 m_axi 接口数据通道的最大位宽。默认为 1024 位。指定的值必须为 2 的幂（介于 8 到 1024 之间）。请注意，如果实际访问大于所需接口，那么这样会降低吞吐量，因为访问将被拆分为多周期访问突发。
- `-m_axi_max_read_buffer_size` <size>: 如果 MAXI 互连结构已提供部分缓冲，则覆盖 MAXI 适配器中用于读取数据的缓冲器的大小，目前计算方法是读取突发量乘以待处理的读取突发数。

- `-m_axi_max_read_burst_length <size>`: 指定突发传输期间所有 `m_axi` 接口读取的数据值的全局最大数量。默认值为 16。
- `-m_axi_max_widen_bitwidth <size>`: 端口宽度自动调整, 以便为 `m_axi` 接口拓宽突发, 最高可拓宽至所选位宽。指定的值必须为 2 的幂 (介于 8 到 1024 之间), 并且必须与 `-m_axi_alignment_size` 对齐。当 `open_solution -flow_target vitis` 时, 默认值为 512, 当 `-flow_target vivado` 时, 值为 0。
- `-m_axi_max_write_buffer_size <size>`: 该选项允许设计师在 MAXI 结构已提供部分缓冲的情况下, 覆盖 MAXI 适配器中用于写入数据的缓冲器大小, 目前计算方法是写入突发量乘以待处理的写入突发数。
- `-m_axi_max_write_burst_length <size>`: 指定突发传输期间所有 `m_axi` 接口写入的数据值的全局最大数量。默认值为 16。
- `-m_axi_min_bitwidth <size>`: 指定 `m_axi` 接口数据通道的最小位宽。默认为 8 位。值必须为 2 的幂 (介于 8 到 1024 之间)。如果实际访问小于所需接口, 那么这样并不一定会增加吞吐量。
- `-m_axi_min_pending_requests <num>`: 设置 `m_axi` 接口中待处理请求的最小数量, 以避免因返回首次响应之前调度的请求数量过多而导致 `m_axi` 适配器发生死锁。默认值: 0。
- `-m_axi_num_read_outstanding <size>`: 指定在设计停滞前可对 `m_axi` 接口发出读取请求的数量 (无响应)。默认值为 16。此操作暗示设计中的内部存储空间, 且 FIFO 大小为:

```
num_read_outstanding*max_read_burst_length*word_size
```

- `-m_axi_num_write_outstanding <size>`: 指定在设计停滞前可对 `m_axi` 接口发出写入请求的数量 (无响应)。默认值为 16。此操作暗示设计中的内部存储空间, 且 FIFO 大小为:

```
num_write_outstanding*max_write_burst_length*word_size
```

- `-m_axi_offset [off | direct | slave]`: 全局控制设计中的所有 `m_axi` 接口的偏移端口。
 - `off`: 不生成偏移端口。这是 Vivado IP 流程中的默认值。
 - `direct`: 生成标量输入偏移端口, 以便通过偏移端口将地址偏移直接传递到 IP 内。
 - `slave`: 生成偏移端口并自动将其映射到 AXI4-Lite 从接口。这是默认值。
- `-s_axilite_data64 [=true|false]`: 将 `s_axilite` 接口的数据宽度设为 64 位。
- `-s_axilite_interrupt_mode [=cor|tow]`: 为将在读取时清零 (`cor`) 或者在写入时切换 (`tow`) 的 `s_axilite` 接口指定中断模式。“Clear on Read” (读取时清零) 中断可在单一传输事务内完成, 而 `tow` 则需要两项传输事务。`Tow` 是默认中断模式。
- `-s_axilite_mailbox [both | in | out]`: 启用为非串流非稳定 `s_axilite` 实参创建邮箱的功能。邮箱功能用于设置和管理自动重启内核。实参值可指定:
 - “both”: 为输入和输出实参启用邮箱
 - “in”: 仅为输入实参启用邮箱
 - “out”: 仅为输出实参启用邮箱
 - “none”: 不创建邮箱 (默认)
- `-s_axilite_status_regs [ecc | off]`:

在 `s_axilite` 寄存器映射中通过为启用 ECC 的每个 BRAM 或 URAM 设置两个读取时清零 (COR) 计数器, 来启用 ECC 错误位的公开。

- `off`: 不生成状态寄存器。这是默认设置。

- `ecc`：为 BRAM 和 URAM 启用 ECC 错误计数器
- `-s_axilite_sw_reset[=false|true]`：在 `s_axilite` 适配器中启用内核的软件复位。

示例

以下示例为 IP 添加了时钟使能端口：

```
config_interface -clock_enable
```

config_op

描述

设置默认选项，用于将运算符（`add`、`mul`、`sub`...）的微架构绑定到 FPGA 实现资源，并指定其时延。

绑定是将运算符（如、加法、乘法和位移）映射到特定 RTL 实现的过程。例如，作为组合或流水打拍 RTL 乘法器来实现的 `mult` 运算。

此命令可多次使用以配置不同运算类型到不同实现资源的默认绑定，或者为该运算指定默认时延。可通过为特定设计元素指定 `BIND_OP` 编译指示或指令来覆盖 `config_op` 所定义的默认配置。

语法

```
config_op [OPTIONS] <op>
```

- `<op>`：为指定的变量指定运算类型。支持值包括：

- `mul`：整数乘法运算
- `add`：整数加法运算
- `sub`：整数减法运算
- `fadd`：单精度浮点加法运算
- `fsub`：单精度浮点减法运算
- `fdiv`：单精度浮点除法运算
- `fexp`：单精度浮点指数运算
- `flog`：单精度浮点对数运算
- `fmul`：单精度浮点乘法运算
- `frsqrt`：单精度浮点平方根倒数运算
- `frecip`：单精度浮点倒数运算
- `fsqrt`：单精度浮点平方根运算
- `dadd`：双精度浮点加法运算
- `dsub`：双精度浮点减法运算
- `ddiv`：双精度浮点除法运算
- `dexp`：双精度浮点指数运算

- `dlog`: 双精度浮点对数运算
- `dmul`: 双精度浮点乘法运算
- `drsqrt`: 双精度浮点平方根倒数运算
- `drecip`: 双精度浮点倒数运算
- `dsqrt`: 双精度浮点平方根运算
- `hadd`: 半精度浮点加法运算
- `hsub`: 半精度浮点减法运算
- `hdiv`: 半精度浮点除法运算
- `hmul`: 半精度浮点乘法运算
- `hsqrt`: 半精度浮点平方根运算
- `facc`: 单精度浮点累加运算
- `fmac`: 单精度浮点乘积累加运算
- `fmadd`: 单精度浮点乘加运算



提示：比较运算符（如 `dcmp`）是在 LUT 中实现的，不得在互连结构外部实现，也不能映射到 DSP，因此无法通过 `config_op` 命令或 `bind_op` 命令来配置。

选项

- `-impl [dsp | fabric | meddsp | fulldsp | maxdsp | primitivedsp | auto | none | all]`: 定义指定运算的默认实现样式。默认为允许工具选择要使用的实现。选项包括：
 - `all`: 所有实现。这是默认设置。
 - `dsp`: 使用 DSP 资源
 - `fabric`: 使用非 DSP 资源
 - `meddsp`: 浮点 IP 中度使用 DSP 资源
 - `fulldsp`: 浮点 IP 完全使用 DSP 资源
 - `maxdsp`: 浮点 IP 最大程度使用 DSP 资源
 - `primitivedsp`: 浮点 IP 原语使用 DSP 资源
 - `auto`: 启用组合 `facc` | `fmac` | `fmadd` 运算符的推断
 - `none`: 禁用组合 `facc` | `fmac` | `fmadd` 运算符的推断
- `-latency <value>`: 定义默认时延，用于将类型绑定到实现资源。有效值范围因运算的每个实现而异（`-impl`）。默认值为 -1，即为实现资源应用标准时延。



提示：无需指定实现详细信息，即可为特定运算指定此时延。这将交由 Vitis HLS 在管理时延的同时选择实现。

- `-precision [low | high | standard]`: 应用于 `facc`、`fmac` 和 `fmadd` 运算符。指定给定运算符的精度。

- **low**: 使用低精度（60 位和 100 位整数）累加实现（如果可用）。该选项仅在某些非 Versal 器件上可用，由于 C++ 仿真相关精度不足，因此它可能导致 RTL/协同仿真发生不匹配。它始终采用流水打拍，且 II=1，无源代码更改，但所使用的资源量约为 **standard** 精度浮点累加所用资源量的 3 倍。
- **high**: 使用高精度（1 个额外的位）融合乘加实现（如果可用）。该选项可用于高精度应用，在 Versal 器件上非常高效，但由于 C++ 仿真相关的额外精度，因此它可能导致 RTL/协同仿真发生不匹配。它使用的资源比 **standard** 精度浮点累加更多。
- **standard**: 标准精度浮点累加和乘加适合大部分浮点用例，它是默认设置。它始终使用真正的浮点累加器，可在 Versal 器件上采用 II=1 来进行流水打拍，在非 Versal 器件上，II 通常介于 3 到 5 之间（取决于时钟频率和目标器件）。

示例 1

以下示例用于将加法运算绑定到互连结构，并具有指定的时延：

```
config_op add -impl fabric -latency 2
```

示例 2

以下示例启用低精度的浮点累加器，在非 Versal 器件上实现 II=1：

```
config_op fadd -impl auto -precision low
```

config_rtl

描述

配置输出 RTL 的各种属性以及所使用的复位类型。它还允许您在 RTL 中使用特定标识。默认情况下，这些选项适用于顶层设计以及设计中的所有 RTL 块。

语法

```
config_rtl [OPTIONS]
```

选项

- **-deadlock_detection <none | sim | hw | hw_diagnosis>**: 在已导出的 IP/XO 文件的顶层 RTL 中启用仿真或综合死锁检测。其中选项如下：
 - **none**: 禁用死锁检测
 - **sim**: 仅对仿真启用死锁检测（默认选项）
 - **hw**: 在已综合及可仿真的 RTL IP 中已启用死锁检测。将 **ap_local_deadlock** 和 **ap_local_block** 信号添加到 IP 以启用局部和全局死锁检测。
 - **hw_diagnosis**: 在生成的 RTL 代码中已启用死锁检测并包含额外的诊断逻辑。
- **-fsm_encoding [one_hot|sequential|johnson|gray|auto|none]**: 默认值为 **None**。指定 RTL 属性值为“fsm_encoding”来引导 RTL 综合。
 - **auto**: 允许 RTL 综合工具判定最佳状态机编码。
 - **gray**: 使用格雷码作为状态机编码。
 - **johnson**: 使用约翰逊码作为状态机编码。

- `one_hot`：使用 `one_hot` 独热码作为状态机编码。
- `sequential`：使用顺序状态机编码。
- `none`：禁用状态机编码，状态机会作为逻辑进行综合。
- `-fsm_safe_state [auto_safe_state|reset_state|power_on_state|default_state|none]`：默认值为 `None`。指定 RTL 属性值为“`fsm_safe_state`”来引导 RTL 综合。该属性会影响综合结果的质量，通常会导致面积越大性能越差。
 - `auto_safe_state`：暗示采用 Hamming-3 编码。
 - `default_state`：将状态机恢复到默认状态。
 - `power_on_state`：将状态机恢复到 `POWER_ON` 状态。
 - `reset_state`：将状态机恢复到 `RESET` 状态。
 - `none`：不在 RTL 中添加属性，状态机将不包含安全状态逻辑。
- `-header <string>`：将文件 `<string>` 的内容置于所有输出 RTL 和仿真文件的顶部（作为注释）。



提示：该选项可用于确保输出 RTL 文件包含用户指定的标识。

- `-kernel_profile`：添加内核剖析所需的顶层事件和停滞端口。
- `-module_auto_prefix`：指定顶层函数名称作为前缀值。如果同时指定 `config_rtl -module_prefix`，则忽略该选项。默认情况下，已启用该选项。
- `-module_prefix <string>`：指定要添加到所有 RTL 实体/模块名称的用户定义的前缀。
- `-mult_keep_attribute`：启用 `keep` 属性。
- `-register_all_io`：默认寄存所有 I/O 信号。默认值为 `false`。指定该选项时启用此设置。
- `-register_reset_num <int>`：指定要添加到复位信号的寄存器数量。在 Vivado IP 流程中，默认值为 0。对于 Vitis 内核流程，默认值为 3。
- `-reset [none | control | state | all]`：C/C++ 语言代码中变量的初始化值与 RTL 中初始化的值始终相同，因此与比特流中初始化值也始终相同。此初始化在上电时执行，对设计应用复位时不会重复此操作。

通过 `-reset` 选项应用的置位操作用于判定寄存器和存储器的复位方式。

- `none`：在设计中不添加复位。
- `control`：复位控制寄存器，例如，状态机中使用的寄存器和用于生成 I/O 协议信号的寄存器。这是默认设置。
- `state`：复位控制寄存器和从 C/C++ 语言代码中的静态变量或全局变量衍生的寄存器或存储器。C/C++ 语言代码中已初始化的任意静态变量或全局变量都复位为其初始值。
- `all`：复位设计中的所有寄存器和存储器。C/C++ 语言代码中已初始化的任意静态变量或全局变量都复位为其初始值。
- `-reset_async`：这导致所有寄存器都使用异步复位。如果不指定该选项，则使用同步复位。
- `-reset_level (low | high)`：允许复位信号的极性设置为低电平有效或高电平有效。默认值为 `High`。



提示：AXI 协议要求采用低电平有效复位。如果您的设计使用 AXI 接口，那么该工具将定义此复位级别并显示警告，前提是 `config_rtl -reset_level` 设为高电平有效。

示例

将输出 RTL 配置为以异步低电平有效复位对所有寄存器进行复位。

```
config_rtl -reset all -reset_async -reset_level low
```

将 my_message.txt 的内容作为注释添加到所有 RTL 输出文件中。

```
config_rtl -header my_message.txt
```

config_schedule

描述

配置 Vitis HLS 执行的调度的默认类型。

语法

```
config_schedule [OPTIONS]
```

选项

- `-enable_dsp_full_reg[=true|false]`: 指定应完全寄存 DSP 信号。默认值为 `true`。

示例

以下示例会禁用寄存 DSP 信号的功能：

```
config_schedule -enable_dsp_full_reg=false
```

config_sim

描述

配置 C 语言仿真。

语法

```
config_sim [OPTIONS]
```

选项

- `O:`: 为 `csim` 和 `cosim` 启用优化编译，禁用 `csim` 代码分析器。如为 `false`，则使用调试模式编译。
- `argv <string>:`: 指定行为测试平台的实参列表。`<string>` 会传递给测试平台的 `main()` C/C++ 函数。
- `ldflags <string>:`: 指定要传递给连接器的标志，用于 `csim` 和 `cosim`。此项通常用于为 C 语言测试平台和设计传递库信息。
- `- clang_version <value>:`: 选择仿真中使用的 clang 版本 (14|16)。默认值：16。

config_storage

描述

将 FIFO 存储元件的 Vitis HLS 微架构绑定的全局默认选项设置为存储器资源。

您可通过为特定设计元件指定 BIND_STORAGE 编译指示或指令或者通过为接口上的对象指定 INTERFACE 编译指示或指令的 storage_type 选项来覆盖 config_storage 为 FIFO 存储空间定义的默认配置。

语法

```
config_storage [OPTIONS] <type>
```

- <type>: 配置 fifo 类型。

选项

- -auto_srl_max_bits <value>: 为 auto-srl 实现 (-impl autosrl) 指定允许的最大 SRL 总位数（深度 * 宽度）。默认值为 1024。
- -auto_srl_max_depth <value>: 为 auto-srl 实现 (-impl autosrl) 指定允许的最大 SRL 深度。默认值为 2。
- -impl [auto | bram | lutram | uram | memory | srl]: 定义要在绑定指定类型时使用的器件资源。

示例

以下示例用于配置 fifo 的默认绑定：

```
config_storage fifo -impl uram
```

config_unroll

描述

基于循环索引限制（或循环次数）自动展开循环。

语法

```
config_unroll [OPTIONS] <value>
```

选项

- -tripcount_threshold <value>: 所含迭代数少于指定值的所有循环都将自动展开。默认值为 0。

示例

以下命令可确保所含迭代数少于 18 的所有循环在调度期间都自动展开。

```
config_unroll -tripcount_threshold 18
```

优化指令

指令或 `set_directive_*` 命令可指定为 Tcl 命令，这些命令可与特定的解决方案或一组解决方案相关联。允许您为跨不同解决方案的同一源代码自定义综合结果。这样您即可在保留源码的同时，参与设计的假设分析。

指令必须以交互模式 `vitis_hls -i` 来运行，或者也可以使用 `-f` 选项作为脚本来运行，如 [vitis](#)、[v++](#) 和 [vitis-run](#) 命令中所述。

编译指示即为您可在源代码中应用的指令，而非 Tcl 脚本，因此会更改代码的所有实现的综合结果。每条 `set_directive` 命令都有 HLS 编译指示，因此您可选择用于处理自己的 AMD Vitis™ HLS 工程的编译指示。如需了解有关不同编译指示的信息，请参阅 [HLS 编译指示](#)。

Vitis HLS IDE 也提供了指令和编译指示，可供分配至您的源代码的特定元素，如 [添加编译指示和指令](#) 中所述。



提示：通过 IDE 运行命令时，会将 Tcl 命令添加至您的工程的脚本中，并写入 `solution/constraints/script.tcl`。

set_directive_aggregate

描述

该指令用于将结构体的数据字段收集到单宽标量中。结构体和 Vitis HLS 中声明的任意阵列均可执行诸如 `set_directive_array_reshape` 等操作，将阵列彻底分区并重构为宽型标量，并将其与结构体的其他元素一起打包。



提示：结构体的阵列经重构后，变为聚合元素阵列。

可根据结构体元素的声明顺序推断出生成的新字宽标量的位对齐方式。第一个元素占用此字的最低有效位，依此类推，直到映射完成所有字段为止。

注释：AGGREGATE 优化无法对结构体进行打包，并且无法在包含其他结构体的结构体上使用。

语法

```
set_directive_aggregate [OPTIONS] <location> <variable>
```

- `<location>` 是包含要打包的变量的位置（格式为 `function[/label]`）。
- `<variable>` 是要打包的结构体变量。

选项

- `-compact [bit | byte | none | auto]`：指定已聚合的结构体的对齐方式。可选设置包括：按位级对齐（打包）、按字节级对齐（填充）、无对齐或者由工具自动与打包联合判定对齐方式（默认行为）。

示例

将函数 `func` 中含有 3 个 8 位字段（`typedef struct {unsigned char R, G, B;} 像素`）的结构体指针 `AB` 聚合到一个新的 24 位指针内，并按位级对齐数据。

```
set_directive_aggregate func AB -compact bit
```

另请参阅

- [pragma HLS aggregate](#)
- [set_directive_array_reshape](#)
- [set_directive_disaggregate](#)

set_directive_alias

描述

指定两个或两个以上的 `M_AXI` 指针实参指向存储器（DDR 或 HBM）中的同一个底层缓冲器，并通过设置相互间的距离或偏移来表明指针之间的任何混叠。



重要提示！ ALIAS 编译指示适用于已映射到 `M_AXI` 接口的顶层函数实参。

Vitis HLS 将不同指针视为独立通道，通常不提供任何依赖关系分析。但如果主机为多个指针分配单个缓冲器，则可通过 ALIAS 编译指示或指令来告知此关系，并且可维护依赖关系分析。ALIAS 编译指示支持在 Vitis HLS 中通过定义缓冲器内指针之间的距离来执行数据依赖关系分析。

ALIAS 的要求：

- 分配给 ALIAS 编译指示的所有端口都必须分配给 `M_AXI` 接口，并分配给不同捆绑，如以下示例所示
- 每个端口仅限在一条 ALIAS 编译指示或指令内使用
- 分配给 ALIAS 编译指示的所有端口的 `depth` 必须相同
- 指定偏移时，指定的端口数与偏移数必须相同，一个端口对应一个偏移
- INTERFACE 的偏移必须指定为从偏移或直接偏移，不支持 `offset=off`

语法

```
set_directive_alias [OPTIONS] <location> <ports>
```

- `<location>` 是位置字符串，格式为 `function[/label]`，ALIAS 编译指示即应用于此指定位置。
- `<ports>` 用于指定要混叠的端口。

选项

- `-distance <integer>`：指定传递到列表中的端口的指针值之间的距离。
- `-offset <string>`：指定传递到 `ports` 列表中每个端口的指针相对于阵列原始指针的偏移。

示例

对于以下 `top` 函数：

```
void top(int *arr0, int *arr1, int *arr2, int *arr3, ...) {
    #pragma HLS interface M_AXI port=arr0 bundle=hbm0 depth=0x40000000
    #pragma HLS interface M_AXI port=arr1 bundle=hbm1 depth=0x40000000
    #pragma HLS interface M_AXI port=arr2 bundle=hbm2 depth=0x40000000
    #pragma HLS interface M_AXI port=arr3 bundle=hbm3 depth=0x40000000
```

以下命令用于为指定的阵列指针定义混叠，并定义指针之间的 `distance`：

```
set_directive_alias "top" arr0,arr1,arr2,arr3 -distance 10000000
```

或者，以下命令用于指定指针之间的 `offset`，达成效果与以上命令相同：

```
set_directive_alias top arr0,arr1,arr2,arr3 -offset  
00000000,10000000,20000000,30000000
```

另请参阅

- [pragma HLS alias](#)
- [set_directive_interface](#)

set_directive_allocation

描述

指定资源分配的实例限制。

ALLOCATION 编译指示或指令可限制用于实现特定函数、循环或运算的 RTL 实例和硬件资源的数量。例如，如果 C/C++ 语言源代码的 `foo_sub` 函数有 4 个实例，则 `set_directive_allocation` 命令可以确保在最终 RTL 中 `foo_sub` 只有一个实例。全部 4 个实例均使用相同的 RTL 块来实现。这样即可减少函数所使用的资源量，但由于共享这些资源，故而对性能会产生负面影响。

C/C++ 语言代码中的运算（例如，加法、乘法、阵列读取和写入）均可通过 `set_directive_allocation` 命令来加以限制。

语法

```
set_directive_allocation [OPTIONS] <location> <instances>
```

- `<location>` 是位置字符串，格式为 `function[/label]`。
- `<instances>` 是函数或运算符。

该函数可以是原始 C/C++ 语言代码中未由 `set_directive_inline` 命令内联或未由 Vitis HLS 自动内联的任何函数。

要获取可使用 ALLOCATION 编译指示加以限制的运算符的完整列表，请参阅 [config_op](#) 命令。

选项

- `-limit <integer>`：

设置要在 RTL 设计中使用的实例（类型由 `-type` 选项定义）数量上限。

- `-type [function|operation]`：实例类型可以是 `function`（默认）或 `operation`。

示例

给定 `foo_top` 设计中有多个 `foo` 函数的实例，在 RTL 中将 `foo` 的实例数限制为 2。

```
set_directive_allocation -limit 2 -type function foo_top foo
```

将 `My_func` 实现中使用的乘法器数限制为 1。此限制不适用于可能驻留在 `My_func` 的子函数中的任何乘法器。要限制任何子函数实现过程中使用的乘法器数量，请在子函数上指定分配指令，或将子函数内联到 `My_func` 函数中。

```
set_directive_allocation -limit 1 -type operation My_func mul
```

另请参阅

- [pragma HLS allocation](#)
- [set_directive_inline](#)

set_directive_array_partition

描述



重要提示！ 针对顶层函数上的 `M_AXI` 接口，不支持 `Array_Partition` 和 `Array_Reshape` 编译指示和指令。您可改用 `hls::vector` 数据类型，如 [矢量数据类型](#) 中所述。

将阵列分区为更小的阵列或者独立元素。

这种分区方式可以：

- 生成包含多个小型存储器或多个寄存器（而不是一个大型存储器）的 RTL。
- 有效增加存储器读写端口数量。
- 可能改善设计吞吐量。
- 需要更多存储器实例或寄存器。

语法

```
set_directive_array_partition [OPTIONS] <location> <array>
```

- `<location>` 是包含阵列变量的位置（格式为 `function[/label]`）。
- `<array>` 是要分区的阵列变量。

选项

- `-dim <integer>`:

注释： 仅与多维阵列相关。

指定要分区的阵列的维度。默认值为 1。

- 如果使用 0 值，则使用指定选项对所有维度进行分区。
- 设置任何其他值都仅对该值对应的维度进行分区。例如，如果使用的值为 1，则仅对第 1 个维度进行分区。

- `-factor <integer>`:

注释： 仅与 `block` 或 `cyclic` 分区类型有关。

指定要创建的更小的阵列数量。

- `-type (block|cyclic|complete)`:

- `block` 型分区会从原始阵列的连续块创建更小阵列。这样可将阵列有效分区为 N 个相等的块，其中 N 为 `-factor` 选项定义的整数。
- `cyclic` 型分区会通过交织来自原始阵列的元素来创建更小的阵列。例如，如果使用 `-factor 3`：
 - 向第 1 个新阵列分配元素 0。
 - 向第 2 个新阵列分配元素 1。
 - 向第 3 个新阵列分配元素 2。
 - 向第 4 个新阵列分配元素 3。
- `complete` 分区可将阵列分解为多个独立元素。对于一维阵列，这对应于将存储器解析为独立寄存器。对于多维阵列，请指定每个维度的分区方式，或者使用 `-dim 0` 选项来对所有维度进行分区。

默认值为 `complete`。

- `-off` 或 `off=true`：为指定变量和指定的 `dim`（如有）禁用 `ARRAY_PARTITION` 功能特性。

示例 1

将 `func` 函数中的 `AB[13]` 阵列分区为 4 个阵列。由于 4 不是 13 的整数因子，因此：

- 有 3 个阵列各包含 3 个元素。
- 另 1 个阵列包含 4 个元素 (`AB[9:12]`)。

```
set_directive_array_partition -type block -factor 4 func AB
```

将 `func` 函数中的 `AB[6][4]` 阵列分区为 2 个阵列，每个阵列维度为 `[6][2]`。

```
set_directive_array_partition -type block -factor 2 -dim 2 func AB
```

将 `func` 函数中的 `AB[4][10][6]` 的所有维度分区为独立元素。

```
set_directive_array_partition -type complete -dim 0 func AB
```

示例 2

在您的代码中，可以按阵列的新结构来对已分区的阵列进行寻址，如以下代码示例所示；

```
struct SS
{
    int x[N];
    int y[N];
};

int top(SS *a, int b[4][6], SS &c) {...}

set_directive_array_partition top b -type complete -dim 1
set_directive_interface -mode ap_memory top b[0]
set_directive_interface -mode ap_memory top b[1]
set_directive_interface -mode ap_memory top b[2]
set_directive_interface -mode ap_memory top b[3]
```

另请参阅

- [pragma HLS array_partition](#)

- [set_directive_array_reshape](#)

set_directive_array_reshape

描述



重要提示！ 针对顶层函数上的 M_AXI 接口，不支持 `Array_Partition` 和 `Array_Reshape` 编译指示和指令。您可改用 `hls::vector` 数据类型，如 [矢量数据类型](#) 中所述。

将阵列分区与垂直阵列映射相结合以创建所含元素更少但字宽更宽的新阵列。

`set_directive_array_reshape` 命令提供下列功能特性：

- 将阵列拆分为多个阵列（类似 `set_directive_array_partition`）。
- 自动按垂直方式重新组合阵列，创建字宽更宽的新阵列。

语法

```
set_directive_array_reshape [OPTIONS] <location> <array>
```

- `<location>` 是包含阵列变量的位置（格式为 `function[/label]`）。
- `<array>` 是要重塑的阵列变量。

选项

- `-dim <integer>`:

注释： 仅与多维阵列相关。

指定要重塑的阵列的维度。

- 如果该值设置为 0，则使用指定选项对所有维度进行分区。
- 设置任何其他值都仅对该值对应的维度进行分区。默认值为 1。

- `-factor <integer>`:

注释： 仅与 `block` 或 `cyclic` 类型的重塑有关。

指定要临时创建的更小的阵列的数量。

- `-object`:

注释： 仅与容器阵列相关。

对容器内的对象应用重塑。如果指定该选项，则将重塑对象的所有维度，但将保留容器的所有维度。

- `-type (block|cyclic|complete)`:

- `block` 型重塑会从原始阵列的连续块创建更小阵列。这样即可将阵列有效等分为 N 个块，其中 N 为 `-factor` 选项定义的整数，然后使用 `word-width*N` 将 N 个块整合到单一阵列中。默认值为 `complete`。
- `cyclic` 型重塑会通过交织来自原始阵列的元素来创建更小的阵列。例如，如果使用 `-factor 3`，则将向第 1 个新阵列分配元素 0，向第 2 个新阵列分配元素 1，向第 3 个新阵列分配元素 2，然后再次向第 1 个新阵列分配元素 3。最后一个阵列是将新阵列垂直串联（字串联以创建更长的字）成单个阵列。

- `complete` 型重塑可将阵列分解为临时独立元素，然后将其重新组合为含字宽更宽的单个阵列。对于一维阵列，这等同于创建一个超宽寄存器（如果原始阵列为 N 个 M 位的元素，结果可生成含 $N*M$ 位的寄存器）。这是默认方式。
- `-off` 或 `off=true`：为指定变量禁用 `ARRAY_RESHAPE` 功能特性。

示例 1

将函数 `func` 中的 8 位阵列 `AB[17]` 重塑为含 5 个元素的新的 32 位阵列。

由于 4 不是 17 的整数因子，因此：

- 阵列 `AB[17]` 的索引 17 位于重塑后第五个元素的低八位中。
- 第五个元素的高八位则不使用。

```
set_directive_array_reshape -type block -factor 4 func AB
```

将函数 `func` 中的阵列 `AB[6][4]` 分区到维度 `[6][2]` 的新阵列中，其中维度 2 为 2 倍宽度。

```
set_directive_array_reshape -type block -factor 2 -dim 2 func AB
```

将函数 `func` 中的 8 位阵列 `AB[4][2][2]` 重塑到新的单元素阵列（寄存器）中，其位宽为 $4*2*2*8 (= 128)$ 。

```
set_directive_array_reshape -type complete -dim 0 func AB
```

示例 2

在您的代码中，可以按阵列的新结构来对已分区的阵列进行寻址，如以下代码示例所示；

```
struct SS
{
    int x[N];
    int y[N];
};

int top(SS *a, int b[4][6], SS &c) {...}

set_directive_array_reshape top b -type complete -dim 0
set_directive_interface -mode ap_memory top b[0]
```

另请参阅

- [pragma HLS array_reshape](#)
- [set_directive_array_partition](#)

set_directive_bind_op

描述



提示：此编译指示或指令在 DSP 多重运算匹配（`MULADD`、`AMA`、`ADDMUL` 和相应的减法运算）中不起作用。事实上，使用 `BIND_OP` 为 DSP 分配运算符会妨碍 HLS 编译器匹配多重运算表达式。如需了解更多信息，请参阅 [DSP 多重运算匹配](#)。

Vitis HLS 使用特定实现来实现代码中的运算。`set_directive_bind_op` 命令用于指定针对每个指定的变量，都将一项运算 (`mul`、`add`、`sub`) 映射到特定器件资源，以便在 RTL 内实现 (`impl`)。如果不指定此命令，Vitis HLS 就会自动判定要使用的资源。

例如，您可使用 `set_directive_bind_op` 命令来指明在器件互连结构内（而不是在 DSP 内）实现一项特定乘法器运算 (`mul`)。

您也可以使用 `-latency` 选项来指定运算的时延。



重要提示！ 要使用 `-latency` 选项，运算必须具有 1 个可用的多阶段实现。HLS 工具可为所有基本算术运算（加减乘除）和所有浮点运算提供 1 个多阶段实现。

语法

```
set_directive_bind_op [OPTIONS] <location> <variable>
```

- `<location>` 是包含变量的位置（格式为 `function[/label]`）。
- `<variable>` 是要分配的变量。在此例中，定义的变量就是该指令的目标运算结果所分配到的变量。

选项

- `-op <value>`：用于定义要绑定到特定实现资源的运算。受支持的函数运算包括：`mul`、`add` 和 `sub`

受支持的浮点运算包括：`fadd`、`fsub`、`fdiv`、`fexp`、`flog`、`fmul`、`frsqrt`、`frecip`、`fsqrt`、`dadd`、`dsb`、`ddiv`、`dexp`、`dlog`、`dmul`、`drsqrt`、`drecip`、`dsqrt`、`hadd`、`hsub`、`hdiv`、`hmul` 和 `hsqrt`



提示：浮点运算包括单精度 (f)、双精度 (d) 和半精度 (h)。

- `-impl <value>`：定义用于指定运算的实现。受支持的函数运算实现包括 `fabric` 和 `dsp`。受支持的浮点运算实现包括：`fabric`、`meddsp`、`fulldsp`、`maxdsp` 和 `primitivedsp`。

注释：`primitivedsp` 仅在 Versal 器件上可用。

- `-latency <int>`：定义运算的实现的默认时延。有效的时延值因指定的 `op` 和 `impl` 而异。默认值为 -1，即交由 Vitis HLS 选择时延。下表反映了受支持的运算、实现和时延组合。

表 44：受支持的函数运算、实现和时延组合

运算	实现	最小时延	最大时延
add	fabric	0	4
add	dsp	0	4
mul	fabric	0	4
mul	dsp	0	4
sub	fabric	0	4
sub	dsp	0	0



提示：比较运算符（如 `dcmp`）是在 LUT 中实现的，不得在互连结构外部实现，也不能映射到 DSP，因此无法通过 `config_op` 命令或 `bind_op` 命令来配置。

表 45：受支持的浮点运算、实现和时延组合

运算	实现	最小时延	最大时延
fadd	fabric	0	13
fadd	fulldsp	0	12
fadd	primitivedsp	0	3
fsub	fabric	0	13
fsub	fulldsp	0	12
fsub	primitivedsp	0	3
fdiv	fabric	0	29
fexp	fabric	0	24
fexp	meddsp	0	21
fexp	fulldsp	0	30
flog	fabric	0	24
flog	meddsp	0	23
flog	fulldsp	0	29
fmul	fabric	0	9
fmul	meddsp	0	9
fmul	fulldsp	0	9
fmul	maxdsp	0	7
fmul	primitivedsp	0	4
fsqrt	fabric	0	29
frsqrt	fabric	0	38
frsqrt	fulldsp	0	33
frecip	fabric	0	37
frecip	fulldsp	0	30
dadd	fabric	0	13
dadd	fulldsp	0	15
dsub	fabric	0	13
dsub	fulldsp	0	15
ddiv	fabric	0	58
dexp	fabric	0	40
dexp	meddsp	0	45
dexp	fulldsp	0	57
dlog	fabric	0	38
dlog	meddsp	0	49
dlog	fulldsp	0	65
dmul	fabric	0	10
dmul	meddsp	0	13
dmul	fulldsp	0	13
dmul	maxdsp	0	14
dsqrt	fabric	0	58

表 45: 受支持的浮点运算、实现和时延组合 (续)

运算	实现	最小时延	最大时延
drsqrt	fulldsp	0	111
drecip	fulldsp	0	36
hadd	fabric	0	9
hadd	meddsp	0	12
hadd	fulldsp	0	12
hsub	fabric	0	9
hsub	meddsp	0	12
hsub	fulldsp	0	12
hdiv	fabric	0	16
hmul	fabric	0	7
hmul	fulldsp	0	7
hmul	maxdsp	0	9
hsqrt	fabric	0	16

示例

在以下示例中，指定了使用互连结构逻辑的二阶流水拍乘法器来为函数 `foo` 的变量 `<c>` 实现乘法运算。

```
int foo (int a, int b) {
  int c, d;
  c = a*b;
  d = a*c;
  return d;
}
```

`set_directive` 命令如下所示：

```
set_directive_bind_op -op mul -impl fabric -latency 2 "foo" c
```

 **提示：**HLS 工具会选择用于变量 `<d>` 的核。

另请参阅

- [pragma HLS bind_op](#)
- [set_directive_bind_storage](#)

set_directive_bind_storage

描述

`set_directive_bind_storage` 命令可用于将代码中的变量（阵列或函数实参）分配给 RTL 中的特定存储器类型（type）。如果不指定此命令，那么 Vitis HLS 工具会判定要分配的存储器类型。HLS 工具在硬件中使用指定的实现（impl）来实现该存储器。例如，您可使用 `set_directive_bind_storage` 命令指定用于阵列变量的存储器的类型以及要使用的实现。并且，此命令还允许您控制此阵列是作为单端口 RAM 还是双端口 RAM 来实现。



重要提示！ 此功能特性对于顶层函数接口上的阵列尤为重要，因为与阵列关联的存储器类型可用于判定 RTL 中所需的端口数量和类型，如 [接口上的阵列](#) 中所述。但对于分配给顶层函数实参的变量，您必须使用 INTERFACE 编译指示或指令的 `-storage_type` 选项和 `-storage_impl` 选项来分配存储器类型和实现。

您可使用 `-latency` 选项来指定实现的时延。对于接口上的块 RAM，`-latency` 选项允许您对接口上的片外非标准 SRAM 进行建模，例如，用于支持时延为 2 或 3 的 SRAM。对于内部运算，`-latency` 选项允许使用多个流水节拍阶段来实现运算。这些附加的流水线阶段可帮助解决 RTL 综合期间的时序问题。



重要提示！ 要使用 `-latency` 选项，运算必须具有 1 个可用的多阶段实现。HLS 工具可以为所有块 RAM 提供多阶段实现。

为实现最佳结果，AMD 建议使用 `-std=c99`（针对 C）和 `-fno-builtin`（针对 C 和 C++）。要指定 `-std=c99` 等 C 语言编译选项，请使用 Tcl 命令 `add_files` 搭配 `-cflags` 选项。或者，选中“Project Settings”（工程设置）对话框中的“Edit CFLAGS”（编辑 CFLAG）按钮，如 [创建 HLS 组件](#) 中所述。

语法

```
set_directive_bind_storage [OPTIONS] <location> <variable>
```

- `<location>` 是包含变量的位置（格式为 `function[/label]`）。
- `<variable>` 是要分配的变量。



提示： 如果变量是顶层函数实参，则使用 INTERFACE 编译指示或指令的 `-storage_type` 选项和 `-storage_impl` 选项。

选项

- `-type`：定义要绑定到指定变量的存储器的类型。受支持的类型包括：`fifo`、`ram_1p`、`ram_1wnr`、`ram_2p`、`ram_s2p`、`ram_t2p`、`rom_1p`、`rom_2p` 和 `rom_np`。

表 46：存储类型

类型	描述
FIFO	FIFO。Vitis HLS 可判定如何在 RTL 中将其实现，除非指定 <code>-impl</code> 选项。
RAM_1P	单端口 RAM。Vitis HLS 可判定如何在 RTL 中将其实现，除非指定 <code>-impl</code> 选项。
RAM_1WNR	含 1 个写入端口和 N 个读取端口的 RAM，内部使用 N 个 bank。
RAM_2P	双端口 RAM，允许在某一端口上执行读操作并在另一个端口上执行读写操作。
RAM_S2P	双端口 RAM，允许在某一端口上执行读取操作，并在另一个端口上执行写入操作。
RAM_T2P	真正的双端口 RAM，支持在 2 个端口上执行读写操作。
ROM_1P	单端口 ROM。Vitis HLS 可判定如何在 RTL 中将其实现，除非指定 <code>-impl</code> 选项。
ROM_2P	双端口 ROM。
ROM_NP	多端口 ROM。

 **提示：**如果您为 PIPO 指定了单端口 RAM，那么绑定程序通常会分配一个合并的 PIPO，只有一个 bank，读取和写入分别访问 bank 的不同部分。HLS 编译器报告的参考消息会显示 “Implementing PIPO using a single memory for all blocks.”，即，“为所有块使用同一个存储器来实现 PIPO”。对于小型 PIPO 来说，这种方法通常比较经济，因为所有 bank 可以共享单个 RAM 块。但是，如果您为 PIPO 指定了双端口 RAM 以获取更高带宽，而调度器在生产者或使用者中使用了这两个端口，那么绑定程序通常会分配一个拆分 PIPO，即生产者使用 1 个端口，使用者使用不同 bank 的 2 个端口。在此情况下，HLS 编译器报告的参考消息为 “Implementing PIPO using a separate memory for each block.”，即，“为每个块使用一个独立存储器来实现 PIPO”。

- `-impl <value>`：定义指定存储器类型的实现。受支持的实现包括：`bram`、`bram_ecc`、`lutram`、`uram`、`uram_ecc`、`srl`、`memory` 和 `auto`，如下所述。

表 47：受支持的实现

名称	描述
MEMORY	通用 FIFO 存储器，允许 Vivado 工具选择实现。
URAM	UltraRAM 资源
URAM_ECC	含 ECC 的 UltraRAM
SRL	移位寄存器逻辑资源
LUTRAM	分布式 RAM 资源
BRAM	块 RAM 资源
BRAM_ECC	含 ECC 的块 RAM
AUTO	Vitis HLS 会自动判定变量的实现。

 **提示：**将针对相同地址的读取和写入操作映射到相同存储器端口时，URAM 不支持 `read-first`（先读取）输出 `write_mode`（不同于 BRAM）。BRAM 支持以下写入模式：`write thru`、`read first` 和 `no change`。URAM 仅支持 `no change`。如果 Vitis HLS 在 URAM 端口上无法将存储器操作调度为在同一个周期内执行，那么将发出以下警告消息：

```
Usage of URAM can potentially cause worse II as Vitis HLS does not
exploit
read-first mode for URAMs. Consider using BRAMs instead.
```

表 48：FIFO/RAM/ROM 支持的实现

类型	命令/编译指示	范围	受支持的实现
FIFO	<code>bind_storage</code> ¹	局部	AUTO 、BRAM、LUTRAM、URAM、MEMORY 和 SRL
FIFO	<code>config_storage</code>	全局	AUTO 、BRAM、LUTRAM、URAM、MEMORY 和 SRL
RAM* ROM*	<code>bind_storage</code>	局部	AUTO BRAM、BRAM_ECC、LUTRAM、URAM 和、URAM_ECC
RAM* ROM*	<code>config_storage</code> ²	全局	不适用
RAM_1P	<code>set_directive_interface s_axilite -storage_impl</code>	局部	AUTO 、BRAM 和 URAM

表 48: FIFO/RAM/ROM 支持的实现 (续)

类型	命令/编译指示	范围	受支持的实现
	<code>config_interface</code> <code>-m_axi_buffer_impl</code>	全局	AUTO、 BRAM 、LUTRAM 和 URAM

注释：

- 如不指定任何实现，该指令会使用 AUTOSRL 行为作为默认实现。但您无法指定该值。
- `config_storage` 仅支持 FIFO 类型。

- `-latency <int>`：定义用于将存储类型绑定到实现的默认时延。有效的时延值因指定的 `type` 和 `impl` 而异。默认值为 -1，即交由 Vitis HLS 选择时延。

表 49: 受支持的存储器类型、实现和时延组合

类型	实现	最小时延	最大时延
FIFO	BRAM	1	4
FIFO	LUTRAM	1	4
FIFO	MEMORY	1	4
FIFO	SRL	1	4
FIFO	URAM	1	4
RAM_1P	AUTO	1	3
RAM_1P	BRAM	1	3
RAM_1P	LUTRAM	1	3
RAM_1P	URAM	1	3
RAM_1WNR	AUTO	1	3
RAM_1WNR	BRAM	1	3
RAM_1WNR	LUTRAM	1	3
RAM_1WNR	URAM	1	3
RAM_2P	AUTO	1	3
RAM_2P	BRAM	1	3
RAM_2P	LUTRAM	1	3
RAM_2P	URAM	1	3
RAM_S2P	BRAM	1	3
RAM_S2P	BRAM_ECC	1	3
RAM_S2P	LUTRAM	1	3
RAM_S2P	URAM	1	3
RAM_S2P	URAM_ECC	1	3
RAM_T2P	BRAM	1	3
RAM_T2P	URAM	1	3
ROM_1P	AUTO	1	3
ROM_1P	BRAM	1	3
ROM_1P	LUTRAM	1	3
ROM_2P	AUTO	1	3
ROM_2P	BRAM	1	3

表 49：受支持的存储器类型、实现和时延组合 (续)

类型	实现	最小时延	最大时延
ROM_2P	LUTRAM	1	3
ROM_NP	BRAM	1	3
ROM_NP	LUTRAM	1	3



重要提示！ 上表中未列出的存储器类型与实现的任意组合均不受 `set_directive_bind_storage` 支持。

示例

在以下示例中，`coeffs[128]` 变量是 `func1` 函数的实参。该指令指定 `coeffs` 使用来自库的 BRAM 核上实现的单端口 RAM。

```
set_directive_bind_storage -impl bram "func1" coeffs RAM_1P
```



提示： 在 RAM_1P 核中定义 RTL 中创建的用于访问 `coeffs` 值的端口。

另请参阅

- [pragma HLS bind_storage](#)

set_directive_cache

描述

在 FPGA 设计中，对片外 DDR 存储器进行非突发访问的成本很高，而且会使性能劣化。为了缓解该问题，Vitis HLS 在 M_AXI 适配器中支持一种高速缓存机制，通过利用引用的局部性（时间性和空间性）来减少平均访问时延。访问某个存储器区域时，很可能很快会再次访问该区域，并且还可能访问附近的位置。

这种高速缓存机制既支持单端口高速缓存，也支持多端口层级：

- 单端口高速缓存：直接映射、只读高速缓存，连接到单个 m_axi 端口。这样在内核重复访问同一行时可以减少对 DRAM 的重复读取。
- 多端口高速缓存：一种分层高速缓存，每个端口都有 L1 高速缓存，并共享 L2 高速缓存。它支持在单个时钟周期内对同一个顶层 m_axi 指针进行最多 N 次读取访问，而无需更改源代码，适用于类似 stencil 的访问模式以及编译器无法推断窗口缓冲器的其他动态访问模式。相比于编译器推断的窗口缓冲器，多端口高速缓存更灵活（处理动态模式），但通常会耗用更多的资源。

多端口高速缓存架构

- L2 高速缓存（共享）：
 - 所有端口共享一个直接映射的高速缓存。
 - 通过突发传输从片外 DRAM 加载。
 - 将各行广播到发出请求的 L1 高速缓存，通过避免冗余 DRAM 加载来优化存储器带宽。

- L1 高速缓存（各端口）：
 - 每个高速缓存端口一个直接映射的 L1 高速缓存。
 - 每个 L1 均可在每个周期内提供一个数据项，允许每个周期对同一个 m_axi 指针进行最多 N 次读取访问。
 - 从共享的 L2 高速缓存加载 L1 行。



提示：您可使用 `syn.interface.m_axi.cache.impl` 来控制高速缓存实现资源。

语法

```
set_directive_cache <location> port=<name> lines=<value> depth=<value>
[ports=<N>] [l2_lines=<M>]
```

其中：

- `<location>`：指定函数，可在该函数所在位置找到指定端口。这是顶层函数。
- `port=<name>`：指定高速缓存要添加到的只读端口（顶层 m_axi 指针）。
- `lines=<value>`：每个 L1 高速缓存中的高速缓存行数（针对单端口的情况：唯一的一个高速缓存）。针对单行指定 1，针对多行指定大于 1 的 2 的幂值。可选；如不指定，则默认值为 1。
- `depth=<value>`：每个高速缓存行的大小，以字数为单位，必须为 2 的幂值。适用于指针的元素类型（例如，深度以 int 和 float 为单位）。可选；默认值为最大突发长度。最大突发长度默认为 16，但可通过 `syn.interface.m_axi_max_read_burst_length` 来全局指定，或通过 `syn.directive.interface` 来按接口指定。注释：在多端口高速缓存中，L1 和 L2 缓存共享相同的行深度；深度同时适用于两个级别。
- `ports=<N>`：启用多端口高速缓存功能特性。定义在一个时钟周期内可以针对对应顶层 m_axi 指针调度执行的读取访问次数。仅适用于映射到其 m_axi 捆绑的只读指针；当端口数量大于 1 时，只能将一个只读阵列映射到该捆绑（请参阅“限制”）。
- `l2_lines=<M>`：可选；默认情况下，L1 和 L2 高速缓存的行数由深度选项来选择。在共享 L2 高速缓存中显式设置行数。每个 L2 行的大小与 L1 行大小相等，并受深度控制。必须大于行数；L2 高速缓存行数必须大于 L1 高速缓存行数。

行为与性能注释

- 多端口高速缓存是专为类似 stencil 的访问模式以及无法推断窗口缓冲器的动态访问模式而设计的。它可以在每个周期内允许多个读取请求，同时减少通过共享 L2 的冗余 DRAM 突发，从而提高吞吐量。
- L1 与 L2 高速缓存均为直接映射。命中/未命中行为由局部性决定；增大行数并选择合适的深度可以提高命中率，但会增加资源消耗。
- 共享 L2 高速缓存每次未命中时会加载一个 DRAM 行，并将该行广播到需要它的任意 L1 高速缓存，从而优化与窗口缓冲器类似的带宽，但支持动态模式。

限制

- 仅限只读端口才支持高速缓存。写端口可以与缓存的只读指针共享一个捆绑，因为它们使用的是独立通道，但不支持 inout 输入输出端口。
- 在多端口模式（端口数大于 1）下：
 - 缓存的顶层 m_axi 指针必须是映射到其捆绑的唯一读端口。
 - 写端口也可以映射到同一个捆绑，但 inout 端口对此不予支持。

- 高速缓存在 L1 和 L2 级处均为直接映射（单路）。
- 指定的 `l2_lines` 值必须大于行数。
- 只能将一个只读阵列映射到多端口缓存指针的 `m_axi` 捆绑。

其他控制

- `syn.interface.m_axi_cache_impl` 可用于控制高速缓存实现资源。
- `syn.interface.m_axi_max_read_burst_length` 和 `syn.directive.interface` 可通过最大突发长度来控制默认深度。

示例

以下示例显示的设计中，重叠访问会导致突发失败。使用 `CACHE` 编译指示或指令会改善设计性能。

```
extern "C" {
    void dut(
        const double *in,    // Read-Only Vector 1
        double *out,         // Output Result
        int size             // Size in integer
    )
    #pragma HLS INTERFACE m_axi port=in bundle=aximm depth = 1026
    #pragma HLS INTERFACE m_axi port=out bundle=aximm depth = 1024
    #pragma HLS cache port=in lines=8 depth=128
    for(int i = 0; i < size; i++)
    {
        out[i] = in[i] + in[i + 1];
    }
}
```

另请参阅

- [pragma HLS interface](#)
- [config_interface](#)
- [pragma HLS cache](#)

set_directive_dataflow

描述

指定要对函数或循环执行的数据流优化，用于提高 RTL 实现的并发。

在 C/C++ 语言描述中，所有操作均按顺序执行。如无限制资源的任何指令（如 `set_directive_allocation`），则 Vitis HLS 会尝试最大限度减小延迟并提高并发。数据依赖关系可对此加以限制。例如，访问阵列的函数或循环必须先完成对阵列的所有读写访问后才能完成操作。这会阻止下一个使用该数据的函数或循环开始操作。

函数或循环中的操作可在前一个函数或循环尚未完成其所有操作时就开始操作。当指定 `DATAFLOW` 优化后，HLS 工具会分析顺序函数或循环之间的数据流，并基于乒乓 RAM 或 FIFO 创建通道，以允许使用者 (consumer) 函数或循环在生产者 (producer) 函数或循环完成前即开始操作。这样即可允许函数或循环并行操作，从而降低延迟，并提升 RTL 吞吐量。



提示： `config_dataflow` 命令可用于指定 `DATAFLOW` 优化中使用的默认存储器通道和 FIFO 深度。

要使 DATAFLOW 优化正常工作，数据必须逐一流经设计中的每个任务。请务必留意下列情况。如需了解更多详细信息，请参阅 [控制驱动的任务级并行度的限制](#) 和 [数据流区域编码风格](#)。

- 单一生产者使用者违例
- 任务间的反馈
- 任务的有条件执行
- 含多个退出条件的循环



重要提示！ 如果存在上述任一编码样式，则 HLS 工具会发出一条消息，且不会执行 DATAFLOW 优化。

最后，DATAFLOW 优化默认不加以分层。如果子函数或循环包含可能受益于 DATAFLOW 优化的其他任务，那么必须对该循环、子函数或子函数的内联应用 DATAFLOW 优化。

语法

```
set_directive_dataflow <location> -disable_start_propagation
```

- `<location>` 是数据流优化的执行位置（格式为 `function[/label]`）。
- `-disable_start_propagation`：禁用起始 FIFO 的创建，起始 FIFO 用于向内部进程传播起始令牌。此类 FIFO 有时可能会成为性能瓶颈。

示例

指定数据流优化在函数 `foo` 内执行。

```
set_directive_dataflow foo
```

另请参阅

- [pragma HLS dataflow](#)
- [set_directive_allocation](#)
- [config_dataflow](#)

set_directive_dependence

描述

Vitis HLS 会检测循环内的依赖关系：循环的相同迭代内的依赖关系为独立于循环的依赖关系，循环的不同迭代之间的依赖关系则为循环进位依赖关系。

执行运算调度时，这些依赖关系受到影响，尤其是在函数和循环流水打拍期间。

- 独立于循环的依赖关系：在单个循环迭代内访问相同元素。

```
for (i=1; i<N; i++) {
    A[i]=x;
    y=A[i];
}
```

- 循环进位依赖关系：从不同循环迭代内访问相同元素。

```
for (i=1;i<N;i++) {
    A[i]=A[i-1]*2;
}
```

在诸如建立变量相关阵列索引或需强制实施外部要求（例如，2 项输入从不采用相同索引）之类的情况下，依赖关系分析可能过于保守，无法滤除假性依赖关系。`set_directive_dependence` 命令允许您显式定义依赖关系并消除假性依赖关系。



重要提示！ 如果依赖关系并非假性 (false) 关系，那么指定假性依赖关系可能导致硬件错误。指定依赖关系前，请确认它是否正确 (true 或 false)。

语法

```
set_directive_dependence -dependent <arg> [OPTIONS] <location>
```

- `-dependent (true | false)`：指定该实参可用于指示依赖关系如为 `true` 则需要强制实施，如为 `false` 则应移除。但如果指定该实参，则工具将返回警告，称未指定该值，并且假定值为 `false`。
- `<location>`：表示代码中定义依赖关系的位置，指定方式为 `function[/label]`。

选项

- `-class (array | pointer)`：指定需要澄清其中所含依赖关系的变量的类。该选项与 `-variable` 选项互斥。
- `-dependent (true | false)`：指定需强制实施依赖关系 (`true`) 还是移除依赖关系 (`false`)。
- `-direction (RAW | WAR | WAW)`：

注释： 仅与循环进位依赖关系有关。

指定依赖关系的方向：

- `RAW`（先写后读 - 真性依赖关系）：写指令使用的值供读指令使用。
- `WAR`（先读后写 - 反依赖关系）：读指令获取的值被写指令覆盖。
- `WAW`（先写后写 - 输出依赖关系）：2 条写指令按顺序写入相同位置。
- `-distance <integer>`：

注释： 仅与循环进位依赖关系有关，其中 `-dependent` 设置为 `true`。

指定阵列访问的迭代间距离。

- `-type (intra | inter)`：指定依赖关系：
 - 位于相同循环迭代内 (`intra`)，或者
 - 位于不同循环迭代之间 (`inter`)（默认值）。
- `-variable <variable>`：定义特定变量，以供应用依赖关系指令。该选项与 `-class` 选项互斥。



重要提示！ 对于与 `m_axi` 接口内其他实参捆绑的函数实参，不能指定 `dependence`。这是函数上的 `m_axi` 接口的默认配置。并且除非结构体已解聚，否则也不能为结构体的元素指定依赖关系。

示例

移除 func 函数的 loop_1 的相同迭代中的 Var1 之间的依赖关系。

```
set_directive_dependence -variable Var1 -type intra \
-dependent false func/loop_1
```

func 函数的 loop_2 中的所有阵列的依赖关系将告知 Vitis HLS，在相同循环迭代内，所有读操作都必须发生在写操作之后。

```
set_directive_dependence -class array -type intra \
-dependent true -direction RAW func/loop_2
```

另请参阅

- [pragma HLS dependence](#)
- [set_directive_disaggregate](#)
- [set_directive_pipeline](#)

set_directive_disaggregate

描述

set_directive_disaggregate 命令允许您按 struct 变量所含各独立元素来对其进行解构。创建的元素数量和类型取决于结构体本身的内容。



重要提示！默认情况下，作为顶层函数实参的结构体将聚合在一起，但可通过该指令或编译指示来对其进行解聚。如需了解有关如何对与串流关联的结构体进行解聚的更多信息，请参阅 [AXI4-Stream 接口](#)。

语法

```
set_directive_disaggregate <location> <variable>
```

- <location> 对应可在其中找到要解聚的变量的位置（格式为 function[/label]）。
- <variable> 则用于指定结构体变量名称。

选项

此命令不含任何选项。

示例 1

以下示例显示函数 top 中的结构体变量 a 将进行解聚：

```
set_directive_disaggregate top a
```

示例 2

解聚后的结构体可在您的代码中使用标准 C/C++ 编码样式来进行寻址，如下所示。请注意访问指针元素 (a) 与访问参考元素 (c) 的方法差异；

```
struct SS
{
    int x[N];
    int y[N];
};

int top(SS *a, int b[4][6], SS &c) {

    set_directive_disaggregate top a
    set_directive_interface -mode s_axilite top a->x
    set_directive_interface -mode s_axilite top a->y

    set_directive_disaggregate top c
    set_directive_interface -mode ap_memory top c.x
    set_directive_interface -mode ap_memory top c.y
```

示例 3

以下示例显示的 Dot 结构体内包含 RGB 结构体作为元素。如果您将 set_directive_disaggregate 应用于 Arr 变量，则仅对顶层 Dot 结构体进行解聚。

```
struct Pixel {
    char R;
    char G;
    char B;
};

struct Dot {
    Pixel RGB;
    unsigned Size;
};

#define N 1086
void DUT(Dot Arr[N]) {
    ...
}

set_directive_disaggregate DUT Arr
```

如果您要将整个结构体（包括 Dot 和 RGB）进行解聚，则可按如下所示对 set_directive_disaggregate 进行赋值。

```
void DUT(Dot Arr[N]) {
    #pragma HLS disaggregate variable=Arr->RGB
    ...
}

set_directive_disaggregate DUT Arr->RGB
```

此例中的结果为：

```
void DUT(char Arr_RGB_R[N], char Arr_RGB_G[N], char Arr_RGB_B[N], unsigned
Arr_Size[N]) {
    ...
}
```

另请参阅

- [pragma HLS disaggregate](#)
- [set_directive_aggregate](#)

set_directive_expression_balance

描述

有时，C/C++ 代码是以一连串运算来编写的，故而在 RTL 中生成一长串运算链。如果时钟周期较短，这就可能增加设计中的时延。默认情况下，Vitis HLS 工具会使用结合律和交换律来重新组织运算。如 [优化逻辑表达式](#) 中所述，通过这种重新组织方式，可以创建平衡的树结构以缩短运算链，从而以增加硬件为代价来缩短设计时延。

表达式平衡会重新排列运算符以构造平衡的树结构并降低时延。

- 对于整数运算，默认情况下表达式平衡处于开启状态，但可将其禁用。
- 对于浮点运算，默认情况下表达式平衡处于关闭状态，但可将其启用。

`set_directive_expression_balance` 命令支持在指定作用域内开关这种表达式平衡方法。

语法

```
set_directive_expression_balance [OPTIONS] <location>
```

- `<location>` 是应禁用或启用该表达式平衡的位置（格式为 `function[/label]`）。

选项

- `-off`：在指定位置关闭表达式平衡。指定 `set_directive_expression_balance` 命令即可在指定作用域内启用表达式平衡。添加 `-off` 选项即可将其禁用。

示例

在 `My_Func` 函数内禁用表达式平衡。

```
set_directive_expression_balance -off My_Func
```

在 `My_Func2` 函数内显式启用表达式平衡。

```
set_directive_expression_balance My_Func2
```

另请参阅

- [pragma HLS expression_balance](#)

set_directive_function_instantiate

描述

默认情况下：

- 函数在 RTL 中作为独立层级块保留，或者分解（或内联）到更高层次的函数中。
- 同一层级上的任一函数的所有实例都使用相同 RTL 实现（块）。

`set_directive_function_instantiate` 命令用于为函数的每个实例创建唯一的 RTL 实现，允许围绕特定实参或变量对每个实例进行优化。

默认情况下，以下代码可为全部 3 个实例的 `func_sub` 函数生成单一 RTL 实现，或者如果 `func_sub` 是小函数，则将其内联到 `func` 函数中。



提示：默认情况下，Vitis HLS 工具会自动内联小函数。即使对于函数例化也同样如此。

`set_directive_inline off` 选项可用于阻止此自动内联操作。

```
char func_sub(char inval, char incr)
{
    return inval + incr;
}
void func(char inval1, char inval2, char inval3,
          char *outval1, char *outval2, char * outval3)
{
    *outval1 = func_sub(inval1, 1);
    *outval2 = func_sub(inval2, 2);
    *outval3 = func_sub(inval3, 3);
}
```

按如下示例部分所示使用指令可生成 3 个版本的 `func_sub` 函数，每个版本都独立针对 `incr` 变量进行优化。

语法

```
set_directive_function_instantiate <location> <variable>
```

- `<location>` 即生成函数的唯一实例的位置（格式为 `function[/region label]`）。
- `<variable>` 可指定函数实参，所指定的函数实参将在各种函数例化中被指定为常量。

选项

此命令不含任何选项。

示例

对于以上所示示例代码，以下 Tcl（或置于 `func_sub` 函数中的编译指示）支持对 `func_sub` 函数的每个实例进行与 `incr` 输入相关的独立优化。

```
set_directive_inline -off func_sub
set_directive_function_instantiate func_sub incr
```

另请参阅

- [pragma HLS function_instantiate](#)

- [set_directive_allocation](#)
- [set_directive_inline](#)

set_directive_inline

描述

移除 RTL 层级中作为独立实体的函数。内联后，函数会消隐到调用函数内，不再作为单独的层级出现。



重要提示！ 内联函数时也会将应用于该函数的所有编译指示或指令消隐掉。在 Vitis HLS 中，忽略应用于内联函数的任意编译指示或指令。

在某些情况下，将函数内联即可使函数内的运算以更有效的方式与调用函数进行共享和优化。但内联函数无法共享或复用，因此如果父函数多次调用内联函数，则可能导致增加面积和资源使用率。

默认情况下，仅在函数层级的下一个层次上执行内联。

语法

```
set_directive_inline [OPTIONS] <location>
```

- <location> 是要执行内联的位置（格式为 `function[/label]`）。

选项

- `-off`：默认情况下，Vitis HLS 会在代码中为较小的函数执行内联。通过使用 `-off` 选项则可为指定函数禁用内联。
- `-recursive`：默认情况下，仅执行一级函数内联。所指定的函数内部包含的函数则不进行内联。`-recursive` 选项会在指定函数层级内按递归方式内联所有函数。



重要提示！ 在数据流区域中执行函数递归内联可能平铺该区域内的层级，使其不满足数据流的条件。在此情况下，会忽略 `INLINE -recursive` 编译指示或指令。

示例

以下示例会内联 `func_sub1` 函数，但不会内联 `func_sub1` 调用的任何子函数。

```
set_directive_inline func_sub1
```

此示例会按递归方式向下内联 `func_sub1` 函数（`func_sub2` 函数除外）：

```
set_directive_inline -recursive func_sub1
set_directive_inline -off func_sub2
```

另请参阅

- [pragma HLS inline](#)
- [set_directive_allocation](#)

set_directive_interface

描述

仅支持对顶层函数使用 `set_directive_interface`，无法将其用于 HLS 组件子函数。INTERFACE 编译指示或指令可指定在接口综合期间如何根据函数实参创建 RTL 端口，如 [定义接口](#) 所述。Vitis HLS 工具会自动确定任何子函数使用的 I/O 协议。

RTL 实现中的端口衍生自顶层函数和函数返回的实参的数据类型和方向、HLS 组件的 `flow_target`、[config_interface](#) 及该指令所指定的默认接口配置设置。每个函数实参均可指定为包含其自己的 I/O 协议（例如，有效握手或确认握手）。



提示：接口上必需的全局变量必须显式定义为顶层函数的实参，如 [全局变量](#) 中所述。如果访问全局变量，但所有读写操作均为设计的本地操作，则会在该设计中创建此资源。RTL 中无需 I/O 端口。

该接口还定义了 HLS 组件的执行控制协议，如 [块级控制协议](#) 中所述。控制协议用于控制 HLS 组件（或块）何时开始执行，当此块完成操作时，控制协议处于空闲状态并准备就绪等待新输入。

语法

```
set_directive_interface [OPTIONS] <location> <port>
```

- `<location>` 对应位置，用户必须指定该位置中的函数接口或寄存的输出（格式为 `function[/label]`）。
- `<port>` 对应参数（函数实参），用户必须对该参数的接口进行综合。如果指定如下块控制模式，则无需指定端口名称：`ap_ctrl_chain`、`ap_ctrl_hs` 或 `ap_ctrl_none`。

选项



提示：以下指定的许多选项都具有 [config_interface](#) 命令中定义的全局值。为此处定义的接口设置局部值即可覆盖全局值。

- `-address_mode <byte|word>` 用于指定寻址是使用字节增量还是字增量。`mode=<mode>`：

受支持的模式以及该工具在 RTL 中实现这些模式的方式均可细分为如下 3 个类别：

1. 端口级协议：

- `ap_none`：无端口协议。此接口是简单的数据端口。
- `ap_vld`：用于实现含关联 `valid` 信号的数据端口，以指示何时数据有效且可供读取或写入。
- `ap_ack`：用于实现含关联 `acknowledge` 信号的数据端口，以确认数据已读取或写入。
- `ap_hs`：用于实现同时包含 `valid` 信号和 `acknowledge` 信号的数据端口，提供两路握手以指示数据有效且可供读取和写入，并确认数据已读取或写入。
- `ap_ovld`：用于实现含关联 `valid` 信号的输出数据端口，以指示何时数据有效且可供读取或写入。



提示：对于 `ap_ovld`，Vitis HLS 通过 `ap_none` 模式来实现输入实参或任意读取/写入实参的输入部分。

- `ap_memory`：用于实现阵列实参（作为标准 RAM 接口）。如果在 Vivado IP integrator 中使用 RTL 设计，那么该接口将由多个不同端口组成。
- `ap_fifo`：使用含关联低电平有效 FIFO `empty` 端口和 `full` 端口的数据输入和输出口来实现含标准 FIFO 接口的端口。

注释：您只能对读取实参或写入实参使用 `ap_fifo` 接口。`ap_fifo` 模式不支持双向读写实参。

2. AXI 接口协议：

- `s_axilite`：用于将该端口作为 AXI4-Lite 接口来实现。为 HLS 组件导出生成的 RT 时，该工具会生成一组关联的 C 语言驱动程序文件。
- `m_axi`：用于将该端口作为 AXI4 接口来实现。您可使用 `config_interface -m_axi_addr64` 命令来指定 32 位（默认）地址端口或 64 位地址端口，并控制任何地址偏移。
- `axis`：用于将该端口作为 AXI4-Stream 接口来实现。

3. 块级控制协议：

- `address_mode (byte|word)`：指定寻址是使用字节增量还是字增量。
- `ap_ctrl_chain`：实现块级控制端口以启动 (`start`) 设计操作、`continue` 操作，以及指示设计何时处于 `idle`、`done` 和 `ready` 状态，以便处理新输入数据。
- `ap_ctrl_hs`：实现块级控制端口以启动 (`start`) 设计操作，并指示设计何时处于 `idle`、`done` 和 `ready` 状态，以便处理新输入数据。
- `ap_ctrl_none`：无块级 I/O 协议。

注释：使用 `ap_ctrl_none` 模式可阻止使用 C/RTL 协同仿真来验证设计。

- `-bundle <string>`：

默认情况下，HLS 工具会将兼容的函数实参组合（或捆绑）到 RTL 代码中的单个接口端口内。含兼容选项（例如，`mode`、`offset` 和 `bundle`）的所有接口端口都组合到单一接口端口内。



提示：此默认行为可使用 `config_interface -m_axi_auto_max_ports` 命令来进行更改。

该 `-bundle <string>` 选项允许您定义捆绑，用于将端口组合在一起，这会覆盖默认行为。`<string>` 用于指定捆绑名称。生成的 RTL 代码中使用的端口名称是根据 `mode` 与 `bundle` 的组合自动衍生而成的，或者是按 `name` 指定的方式命名的。



重要提示！ `bundle` 名称必须使用小写字符来指定。

- `-channel <string>`：要在 `m_axi` 接口上启用多个通道，请指定通道 ID。您可使用不同的通道 ID 将多个 `m_axi` 接口组合为单个 `m_axi` 适配器。
- `-clock <string>`：默认情况下，AXI4-Lite 接口时钟与系统时钟为相同时钟。该选项用于为 AXI4-Lite 接口指定独立时钟。如果使用 `-bundle` 选项将多个顶层函数实参组合到单一 AXI4-Lite 接口中，那么只需在其中一个捆绑成员上指定时钟选项即可。
- `-depth <int>`：指定供测试平台处理的最大采样数。此设置用于指示 HLS 工具为 RTL 协同仿真所创建的验证适配器中所需 FIFO 的最大大小。



提示：虽然 `depth` 选项通常为可选，但需要通过该选项来为 RTL 协同仿真指定指针实参大小。

- `-interrupt <int>`：仅供 `ap_vld/ap_hs` 使用。该选项用于启用要在中断中管理的 I/O，方法是在 `s_axilite` 寄存器文件的 `ISR` 和 `IER` 中创建对应的位元。整数 N=16..31 用于指定这两个寄存器中的位元位置（默认从 16 开始分配连续的值）。
- `-latency <value>`：该选项可用于 `ap_memory` 和 `M_AXI` 接口。
- 在 `ap_memory` 接口中，接口选项用于指定驱动接口的 RAM 资源的读取时延。默认情况下，使用 1 个时钟周期的读取操作。该选项允许对读取时延超过 1 个时钟周期的外部 RAM 进行建模。

- 在 M_AXI 接口中，该选项用于指定 AXI4 接口的期望时延，允许设计发起总线请求的时间比执行期望的读取或写入操作早 <value> 个周期（时延）。如果该值太低，设计将过早达成就绪状态，可能停滞并等待总线；如果该值太高，总线访问可能处于空闲状态并等待设计发起访问。
- `-max_read_burst_length <int>`：该选项适用于 M_AXI 接口，可指定突发传输期间读取的数据值的最大数量。如需了解更多信息，请参阅 [AXI 突发传输](#)。
- `-max_write_burst_length <int>`：该选项适用于 M_AXI 接口，可指定突发传输期间写入的数据值的最大数量。
- `-max_widen_bitwidth <int>`：自动拓宽接口时，请指定接口可用的最大位宽。此设置将覆盖由 `config_interface -m_axi_max_bitwidth` 命令指定的全局值。
- `-name <string>`：指定将在生成的 RTL 中使用的端口的名称。除非 `name` 指定了端口名称，否则默认端口名称是根据 `mode` 与 `bundle` 的组合自动衍生的。
- `-num_read_outstanding <int>`：该选项适用于 M_AXI 接口，用于指定在设计停滞前可对 AXI4 总线发起的读取请求数量（无响应）。此操作暗示设计中的内部存储空间，且 FIFO 大小为：

```
num_read_outstanding*max_read_burst_length*word_size
```

- `-num_write_outstanding <int>`：该选项适用于 M_AXI 接口，用于指定在设计停滞前可对 AXI4 总线发起的写入请求数量（无响应）。此操作暗示设计中的内部存储空间，且 FIFO 大小为：

```
num_write_outstanding*max_write_burst_length*word_size
```

- `-offset <string>`：为指定端口控制 AXI4-Lite (`s_axilite`) 和 AXI4 存储器映射 (`m_axi`) 接口中的地址偏移。
 - 在 `s_axilite` 接口中，`<string>` 用于指定寄存器映射中的地址。
 - 在 `m_axi` 接口中，该选项会覆盖 `config_interface -m_axi_offset` 选项所指定的全局选项，并且 `<string>` 指定为：
 - `off`：不生成偏移端口。
 - `direct`：生成标量输入偏移端口。
 - `slave`：生成偏移端口并自动将其映射到 AXI4-Lite 从接口。这是默认偏移。
- `-register`：寄存信号和任何关联协议信号，并指示信号保持直至至少完成函数执行的最后一个周期为止。该选项适用于顶层函数的以下标量接口：

```
· s_axilite
· ap_fifo
· ap_none
· ap_hs
· ap_ack
· ap_vld
· ap_ovld
```



提示：不支持在函数的返回端口 (`port=return`) 上使用寄存器选项。请改为使用 LATENCY 编译指示或指令：

```
#pragma HLS LATENCY min=1 max=1
```

- `-register_mode (both|forward|reverse|off)`：该选项适用于 AXI4-Stream 接口，并用于指定寄存器布局到正向路径 (TDATA 和 TVALID)、反向路径 (TREADY) 或同时布局到这两条路径 (TDATA、TVALID 和 TREADY) 上，或者不寄存任何端口信号 (off)。默认值为 `both`。AXI4-Stream 旁路信号被视为数据信号，随 TDATA 一起寄存。
- `-storage_impl=<impl>`：仅限搭配 `s_axilite` 一起使用。该选项可定义要分配给接口的存储实现。受支持的实现值包括 `auto`、`bram` 和 `uram`。默认值为 `auto`。



提示： `uram` 是同步存储器，对于两个端口仅含单个时钟。因此，针对含第二个时钟的 `s_axilite` 适配器不能指定 `uram`。

- `-storage_type=<type>`：仅限搭配 `ap_memory` 接口一起使用。该选项用于定义要分配给变量的存储类型（例如，RAM_T2P）。受支持的类型包括：`ram_1p`、`ram_1wnr`、`ram_2p`、`ram_s2p`、`ram_t2p`、`rom_1p`、`rom_2p` 和 `rom_np`。



提示： 对于接口上未定义的对象，也可以使用 `BIND_STORAGE` 编译指示或指令来指定变量的类型。

示例

为 `func` 函数关闭函数级握手。

```
set_directive_interface -mode ap_ctrl_none func
```

`func` 函数中的 `InData` 实参指定为包含 `ap_vld` 接口，并且输入应进行寄存。

```
set_directive_interface -mode ap_vld -register func InData
```

另请参阅

- [pragma HLS interface](#)
- [set_directive_bind_storage](#)

set_directive_latency

描述

在函数、循环或区域上指定最大和/或最小时延值。

Vitis HLS 始终以最小时延为目标。指定最小和最大时延值时，该工具的行为如下所示：

- 当时延小于最小值时：如果 Vitis HLS 可以实现小于最小指定值的时延，那么它可将时延扩展至指定值，这样可能增加共享。
- 当时延大于最小值时：满足约束。不再执行进一步优化。
- 当时延小于最大值时：满足约束。不再执行进一步优化。
- 当时延大于最大值时：如果 Vitis HLS 无法调度到最大值范围内，那么它会尽力实现指定约束。如果仍无法满足最大时延，则会发出警告。随后，Vitis HLS 会以可实现的最小时延来生成设计。



提示： 您还可以使用 `LATENCY` 编译指示或指令来限制该工具寻找最优解决方案的范围。为代码中的作用域（包括循环、函数或区域）指定时延约束即可减少该作用域内的可能的解决方案数量，从而改善工具运行时间。如需了解更多信息，请参阅 [改善综合运行时间和容量](#)。

如果要限制所有循环迭代的总时延，应将时延指令应用于包含整个循环的区域，如下示例所示： `set_directive Region_All_Loop_A`

```
Region_All_Loop_A: {
  Loop_A: for (i=0; i<N; i++)
  {
    ..Loop Body...
  }
}
```

在此情况下，即使展开循环，时延指令仍会对所有循环操作设置最大限制。

如果 Vitis HLS 无法满足最大时延约束，它会放宽时延约束，并尝试尽可能实现最佳结果。

如果设置最小时延约束，并且 Vitis HLS 生成的设计时延低于所需的最小值，它会插入虚拟时钟周期以满足最小时延。

语法

```
set_directive_latency [OPTIONS] <location>
```

- `<location>` 是要约束的函数、循环或区域的位置（格式为 `function[/label]`）。

选项

- `-max <integer>`：指定最大时延。
- `-min <integer>`：指定最小时延。

示例

指定 `foo` 函数采用最小时延值 4 和最大时延值 8。

```
set_directive_latency -min=4 -max=8 foo
```

在 `foo` 函数中，指定 `loop_row` 循环采用最大时延 12。将编译指示置于循环主体内。

```
set_directive_latency -max=12 foo/loop_row
```

另请参阅

- [pragma HLS latency](#)

set_directive_loop_flatten

描述

允许将内层嵌套循环折叠（平铺）为单个循环，以便在循环的所有迭代上应用流水打拍，从而实现更低的时延。

只能对最内层的循环（完成可能的内部展开后）进行流水打拍。外层循环只能是数据流循环或顺序循环。当流水打拍循环上层的循环为顺序循环时，其迭代会依次执行，而且每次迭代，就会完全执行一次内层循环。在 RTL 实现中，从外层循环移动到内层循环需要一个时钟周期，从内层循环移回外层循环需要一个时钟周期，并且中间要加上完成内层循环的所有迭代的完整时延。与此相反，平铺嵌套循环可以将其作为单个循环来进行优化和流水打拍，这样外层循环的迭代与内层循环不同调用的迭代之间就可能发生重叠（流水打拍）。总之，平铺可以提高性能，但是：

- 平铺并非总是可行，需要遵循一定的编码风格。
- 在某些情况下，根据外层循环中的运算和/或依赖关系，时序甚至 II 都可能会劣化。

将 `set_directive_loop_flatten` 编译指示应用于循环层级的最内层循环的循环主体。只有完美循环和近乎完美的嵌套循环（在完成可能的初步函数内联或循环展开后）才能以这种方式加以平铺：

- 完美循环嵌套：
 - 每个非最内层循环的主体都包含一个且仅包含一个子循环，不包含任何其他指令。
- 近乎完美的循环嵌套：
 - 每个非最内层循环的主体都包含一个且仅包含一个子循环，无任何其他控制流。
 - 每个非最内层循环的主体都不得包含任何含循环的函数调用。

对于近乎完美的循环，编译器会将两个循环之间存在的任何指令自动“推入”最内层的循环，从而使这些循环完美嵌套。

此外具备平铺能力的循环还需要满足一些要求：

- 每个循环都应该是 for 循环，而不是 while 循环；并且不含 break 语句。
- 在平铺循环之前，编译器应能计算出每个循环的循环次数（但它不必是一个常数）。典型的编码风格如下：
 - 所含循环计数器按常数递增的循环。
 - 循环计数器的下限和上限不依赖于要平铺的循环（应满足“循环不可变”条件）。

举例来说，如果循环的内层循环次数取决于外层循环计数器，那么这类循环就不具备平铺能力。

编译器无法平铺不完美的循环嵌套（即，当循环包含多个子循环或控制流时）。在此情况下，需要手动执行平铺，方法是重构代码，将指令推入最内层的循环，或者展开内层循环以在其上层创建完美循环嵌套。

语法

```
set_directive_loop_flatten [OPTIONS] <location>
```

- `<location>` 表示最内层循环的位置，格式为 `function[/label]`。

选项

- `-off` 或 `off=true`：

可选关键字。阻止将 `loop_flatten` 设为 `off` 的循环与其子循环（如有）一起进行平铺。



重要提示！ 存在 `LOOP_FLATTEN` 编译指示和指令即表示启用优化。添加 `-off` 则表示禁用优化。

示例

将 `foo` 函数中的 `loop_1` 及循环层级中位于其上层的所有（完美或近乎完美）循环平铺为单一循环。将编译指示置于 `loop_1` 主体内。

```
set_directive_loop_flatten foo/loop_1
```

防止在 `foo` 函数的 `loop_2` 中进行循环平铺。将编译指示置于 `loop_2` 主体内。

```
set_directive_loop_flatten -off foo/loop_2
```

如需获取更完整的示例，请参阅对应的 [pragma HLS loop_flatten](#) 章节。

另请参阅

- [pragma_HLS_loop_merge](#)
- [pragma_HLS_unroll](#)

set_directive_loop_merge

描述

将所有循环合并到单一循环中。合并循环：

- 减少 RTL 中循环主体实现间转换所需时钟周期数。
- 允许并行实现循环（如果可能）。

循环合并规则为：

- 如果循环边界为变量，则值（迭代数）必须相同。
- 如果循环边界为常量，那么最大常量值用作为合并循环的边界。
- 具有变量边界和常量边界的循环无法合并。
- 要合并的循环之间的代码不得产生不同结果。多次执行此代码应生成相同结果。
 - 允许 `a=b`
 - 不允许 `a=a+1`。
- 包含 FIFO 读取的循环无法合并。合并会更改读取顺序。从 FIFO 或 FIFO 接口执行读取必须始终按顺序进行。

语法

```
set_directive_loop_merge <location>
```

- `<location>` 是循环所在位置（格式为 `function[/label]`）。

选项

- `-force`：即使 Vitis HLS 发出警告仍强制合并循环。您必须确保合并后的循环功能正常。

示例

将函数 `foo` 中的所有连续循环都合并到单一循环中。

```
set_directive_loop_merge foo
```

函数 `foo` 的 `loop_2` 内的所有循环（不包含 `loop_2` 本身）均通过使用 `-force` 选项来合并。

```
set_directive_loop_merge -force foo/loop_2
```

另请参阅

- [pragma HLS loop_merge](#)
- [set_directive_loop_flatten](#)
- [set_directive_unroll](#)

set_directive_loop_tripcount

描述

“循环次数”是指循环执行的迭代总数。Vitis HLS 可报告每个循环的总时延（执行循环的所有迭代所需周期数）。因此，此循环时延为循环次数（循环迭代次数）的函数。



重要提示！ LOOP_TRIPCOUNT 编译指示或指令仅用于分析，不影响综合结果。

循环次数可为常量值。它取决于循环表达式（例如， $x < y$ ）中使用的变量值或循环内使用的控制语句。

在某些情况下，Vitis HLS 无法判定循环次数。例如，如果用于判定循环次数的变量属于：

- 输入实参，或者
- 采用动态运算来计算的变量

在以下示例中，for 循环的最大迭代次数由输入 `num_samples` 的值来确定。在 C 语言函数中不定义 `num_samples` 的值，该值从外部进入函数。

```
void foo (num_samples, ...) {
    int i;
    ...
    loop_1: for(i=0;i< num_samples;i++) {
        ...
        result = a + b;
    }
}
```

如果循环时延未知或者无法计算，那么 `set_directive_loop_tripcount` 允许您指定循环迭代次数的最小、最大和平均值。这样该工具即可分析循环时延给报告中的设计总时延造成的影响，并帮助您为设计决定合适的优化措施。



提示： 如果使用 C 语言断言宏来限制循环变量的大小，那么 Vitis HLS 可将其用于定义循环限制以供报告之用，并根据这些限制创建大小精确的硬件。

语法

```
set_directive_loop_tripcount [OPTIONS] <location>
```

- `<location>` 表示循环中指定循环次数的位置（格式为 `function[/label]`）。

选项

- `-avg <integer>`：指定平均迭代次数。
- `-max <integer>`：指定最大迭代次数。
- `-min <integer>`：指定最小迭代次数。

示例

函数 `foo` 中的 `loop_1` 的最小循环次数指定为 12，最大循环次数则为 16:

```
set_directive_loop_tripcount -min 12 -max 16 -avg 14 foo/loop_1
```

另请参阅

- [pragma HLS loop_tripcount](#)

set_directive_occurrence

描述

`set_directive_occurrence` 可应用于区域内的函数调用。该功能特性适用于条件块内的流水打拍函数调用，例如，`if` 语句或 `switch` 语句。该指令用于指定当前流水线内调用流水打拍函数的频率。如果以 `set_directive_occurrence` 来标识条件块内的某个函数，即可允许该区域中的函数和循环采用流水打拍、启动时间间隔更高，且比外层函数或循环更慢。

执行函数或循环流水打拍时，`set_directive_occurrence` 可用于指定流水打拍函数或循环内某个流水打拍函数调用的执行频率低于其外层函数或循环的执行频率。这样流水打拍的函数调用在以较低频率执行时即可以较慢的速率进行流水打拍，这可能有助于改善顶层流水线内的资源共享。

以下进程可用于判定 `set_directive_occurrence -cycle` 值：

- 循环迭代 $\langle N \rangle$ 次。
- 但由于循环主体中某部分是通过条件语句来启用的，因此仅执行 $\langle M \rangle$ 次，其中 $\langle N \rangle$ 为 $\langle M \rangle$ 的整数倍。
- 条件代码的出现次数比循环主体的其他部分慢 N/M 次。

例如，在执行 10 次的循环中，其中的条件语句仅执行 2 次，则出现次数为 5（即 $10/2$ ）。

语法

```
set_directive_occurrence [OPTIONS] <location>
```

- `<location>` 用于指定代码块，此代码块中包含以较低速率执行的流水打拍函数调用。

选项

- `-cycle <int>`：指定出现次数为 N/M ，其中 $\langle N \rangle$ 是外层函数或循环的执行次数， $\langle M \rangle$ 是条件区域的执行次数。



重要提示！ N 必须是 M 的整数倍。

- `-off` 或 `off=true`：为指定函数禁用 `occurrence`。

示例

在此示例中，在 if 语句中调用子函数（但不包括存储器读写，因为它们不在流水打拍函数内）会发生 4 次（其执行频率是包含此代码的外部代码的四分之一以下）。因此，虽然没有 `set_directive_occurrence`，但仍会以与调用程序相同的 II 来对其进行流水打拍，包含该指令时，将以 II=4 来执行流水打拍。这将在其自身内部以及与其他函数之间增加资源共享的机会。

```
void subfunction(...) {
#pragma HLS pipeline II=...
// Without the occurrence pragma,
// this will be automatically pipelined with an II=1,
// regardless of its own pipeline pragma,
// since it is called in a pipeline with II=1
// With the pragma, it has an II=4.
...
}
for (int i = 0; i < 16; i++) {
#pragma HLS pipeline II=1
Cond_Region:
    if (i % 4 == 0) {
        subfunction(...);
        a[i+1] = b[i];
    }
}
```

subfunction 函数中的 Cond_Region 区域执行 4 次。它的执行速度比其外层的代码更慢，仅为外层代码的四分之一。

```
set_directive_occurrence -cycle 4 foo/Cond_Region
```

另请参阅

- [pragma HLS occurrence](#)
- [set_directive_pipeline](#)

set_directive_performance

描述



提示： `set_directive_performance` 全局应用于整个被测设计 (DUT) 的顶层函数。它也可作为循环层 PERFORMANCE 指令应用于循环。

Vitis HLS 中的 `set_directive_performance` 指令为您的设计提供了用于定义高层次性能目标的机制，以便引导综合工具的优化工作。它可应用于两个不同的作用域：设计的顶层（内核/IP）和设计中的特定循环层。

顶层 PERFORMANCE 指令

在内核或 IP 顶层函数上应用 `set_directive_performance` 指令允许您指定整体性能目标，主要侧重于达成所需的吞吐量。在内核顶层函数上应用该指令时，会触发 HLS 工具对整个设计执行性能分析。该分析对整个设计结构进行评估，以判定满足指定性能目标的可行性。根据整体目标及此项分析，该工具会自动为设计层内的单个循环和循环嵌套推断适当的循环层性能指令。

循环层 PERFORMANCE 指令

直接将 `set_directive_performance` 指令应用于特定循环或循环嵌套使您能定义循环或循环嵌套的该特定部分的性能目标。循环层的 PERFORMANCE 指令支持您指定高层次约束 `target_ti` 来定义循环的连续启动之间的时钟周期数，并允许该工具推断达成期望结果所需的低层次 UNROLL、PIPELINE、ARRAY_PARTITION 和 INLINE 指令。PERFORMANCE 指令不保证将达成指定的值，仅作为目标。

适用于顶层（整个函数/DUT）和循环层的 `set_directive_performance` 指令使用 `target_ti` 参数来引导优化以达成特定性能目标。

- **target_ti（目标启动时间间隔）：** 定义 DUT 或循环的连续启动之间的所需时钟周期数。
 - **顶层性能指令：**
 - 在顶层应用 `target_ti` 时，它会指定单次函数执行的开始时间与下一次函数执行的开始时间之间的目标时间间隔。来看一个示例，假设要为图像处理设计计算顶层性能指令，目标帧数为每秒 60 帧 (FPS)。要达到 60 FPS 的帧速，函数/DUT 必须在 1/60 秒内准备好重新启动并读取新的帧。
 - Target TI = 1/FPS = 1/60 ≈ 16.67 毫秒。
 - 这意味着函数/DUT 必须在 16.7 毫秒内准备好重新启动并读取新的帧，以保持每秒 60 帧 (FPS) 的目标。
 - **循环层性能指令：**
 - 在循环层应用 `target_ti` 时，它会指定连续多次外层循环迭代中的循环的首次迭代开始时间与下一次开始迭代之间的目标时间间隔。在以下示例中，循环 L2 上的 `target_ti=T` 将循环 L2 开始执行一次 L1 迭代与循环 L2 开始执行 L1 的下一次迭代之间的目标时间间隔设为 T 个周期。

```
const int T = 100;
L1: for (int i=0; i<N; i++)
L2: for (int j=0; j<M; j++)
{ directive HLS performance target_ti=T ... }
```

传输事务时间间隔是循环的启动时间间隔 (II) 乘以迭代次数或循环次数：`target_ti = II * loop tripcount`。反之，`target_ti = 频率 (Hz) / 每秒操作数`。

例如，假定图像处理函数每次调用处理一帧，吞吐量目标为 60 fps，则此函数的目标吞吐量为每秒 60 次调用。如果时钟频率为 180 MHz，则 `target_ti` 为 180M/60，或每次函数调用 3 百万个时钟周期。

语法

指定已标记的循环的指令。

```
set_directive_performance <location> [OPTIONS]
```

其中：

<location> 用于指定顶层函数或循环。

选项

- `-target_ti=<value>`：指定目标传输事务时间间隔定义为循环完成一次迭代的时钟周期数。传输事务时间间隔指的是循环开始处理传输事务的时间与它开始处理下一项传输事务之间的时间间隔。<value> 可指定为整数、浮点或由该工具解析为整数的常量表达式。

注释：如果发生截位，则返回警告。

- `unit=[sec | cycle]`：指定与 `target_ti` 值或 `target_tl` 值关联的单位。此单位可指定为秒数或时钟周期数。当指定的单位为秒数时，可用表示纳秒 (ns)、皮秒 (ps) 或微秒 (us) 的相应的值来指定单位。

示例 1

指定函数 `func` 中标记为 `loop_1` 的循环的目标传输事务时间间隔为 4 个时钟周期：

```
set_directive_performance func/loop_1 -target_ti=4
```

另请参阅

- [pragma HLS performance](#)
- [set_directive_inline](#)

set_directive_pipeline

描述

通过允许并发执行操作来缩短函数或循环的启动时间间隔 (II)，如 [循环流水打拍](#) 中所述。流水线函数或循环每隔 N 个时钟周期即可处理新输入，其中 N 为启动时间间隔 (II)。如果 II 为 1，则表示每个时钟周期都处理新输入。

Vitis HLS 使用 PIPELINE 编译指示或指令时的默认行为是根据指定的时钟周期约束，为设计生成最小 II。除非指定 `-II` 选项，否则重点强调满足时序，而不是实现 II。

流水线的默认样式是由 `config_compile -pipeline_style` 命令定义的，但可在 PIPELINE 编译指示或指令内进行覆盖。

如果 Vitis HLS 无法按指定的 II 创建设计，它将：

- 发出警告。
- 以可能的最低 II 创建设计。

随后，您可利用警告消息来分析此设计，以判定必须采取哪些步骤才能创建满足所需启动时间间隔的设计。

语法

```
set_directive_pipeline [OPTIONS] <location>
```

其中：

- `<location>` 是要流水打拍的位置（格式为 `function[/label]`）。

选项

- `-II <integer>`：指定期望的流水线启动时间间隔。Vitis HLS 会尝试满足此请求。根据数据依赖关系，实际结果的 II 可能更大。
- `-off`：关闭特定循环或函数的流水线。当使用 `config_compile -pipeline_loops` 对循环进行全局流水打拍时，可使用该选项。
- `-rewind`：

注释：仅适用于循环。

可选关键字。支持回绕，如 [回绕已流水打拍的循环以保障性能](#) 中所述。这样即可支持持续性循环流水打拍，但在循环执行结束与下一次执行开始之间没有暂停。仅当顶层函数内只有一个循环（或完美循环嵌套）时，回绕才有效。循环前的代码段：

- 被视为初始化。
- 在流水线中仅执行一次。
- 不能包含任何条件运算 (`if-else`)。
- `-style <stp | frp | flp>`: 指定要用于指定的函数或循环的流水线类型。如需了解有关流水线样式的更多信息, 请参阅 [刷新流水线和流水线类型](#)。流水线类型包括:
 - `stp`: 停止流水线。仅当输入数据可用时才运行, 否则停滞。这是默认设置, 也是 Vitis HLS 用于循环和函数流水打拍的流水线类型。无需可刷新流水线时, 请使用此类型。例如, 没有导致停滞的性能或死锁问题时即可用。
 - `flp`: 该选项用于将流水线定义为可刷新流水线。此类型的流水线通常耗用更多资源和/或可能 II 较大, 因为无法在流水线迭代间共享资源。
 - `frp`: 自由运行的可刷新流水线。即使输入数据不可用也可运行。如果由于流水线控制信号扇出减少而导致需要更好的时序, 或者如果需要提升性能以免死锁, 则可使用此类型。但此流水线样式可能功耗更大, 因为即使没有数据也会对流水线寄存器进行计时。



重要提示! 这是提示, 而非硬性约束。工具会检查设计启用流水打拍的条件。部分循环可能不遵循特定样式, 如果需要, 工具会还原为默认样式 (`stp`)。

示例

函数 `func` 按指定启动时间间隔进行流水打拍。

```
set_directive_pipeline func II=1
```

另请参阅

- [pragma HLS pipeline](#)
- [set_directive_dependence](#)
- [config_compile](#)

set_directive_protocol

描述

此命令用于指定代码区域（协议区域），除非代码中明确指定，否则 Vitis HLS 不会在其中插入时钟操作。Vitis HLS 将不会在该区域内的运算之间插入任何时钟，包括读取或写入函数实参的运算。因此，读取和写入顺序严格遵循已综合的 RTL 中的顺序。

在 C/C++ 代码中可以通过使用 “{ }” 将区域括起或者对该区域进行命名来创建代码区域。以下示例定义了名为 `io_section` 的区域：

```
io_section:{
...
lines of code
...
}
```

在 C 语言代码中，可使用 `ap_wait()` 语句来显式指定时钟运算，在 C++ 代码中则可使用 `wait()` 语句来指定该运算。`ap_wait` 和 `wait` 语句对于设计的仿真没有影响。

语法

```
set_directive_protocol [OPTIONS] <location>
```

<location> 用于指定定义协议区域的位置（格式为 `function[/label]`）。

选项

- `-mode [floating | fixed]`:
 - `floating`: 表示浮动模式，允许协议区域外的代码语句与最终 RTL 内协议区域中的语句重叠且并行执行。协议区域可保持时钟精确，但仍可同时发生外部运算。这是默认模式。
 - `fixed`: 表示固定模式，`fixed` 模式可确保协议区域外部的语句不会与协议区域并行执行。

示例

此代码示例在函数 `foo` 内定义了协议区域 `io_section`。以下指令用于将该区域定义为固定模式协议区域：

```
set_directive_protocol -mode fixed foo/io_section
```

另请参阅

- [pragma HLS protocol](#)

set_directive_reset

描述

RESET 编译指示或指令用于为特定状态变量添加或禁用复位端口。

复位端口用于在应用复位信号时，立即将连接到复位端口的寄存器和块 RAM 还原为初始值。在全局范围内，`config_rtl -reset` 命令可用于控制 RTL 复位端口是否存在及其行为。复位配置设置包含定义复位极性以及指定使用同步复位还是异步复位的功能，但更重要的是，它可通过复位选项来控制应用复位信号时所复位的寄存器。如需了解更多信息，请参阅 [控制初始化与复位行为](#)。

通过 RESET 编译指示可提供更具体的复位控制。对于全局变量或静态变量，RESET 编译指示用于显式启用复位（前提是不存在任何变量），或者可通过设为 `off` 关闭此编译指示来从复位中移除变量。当设计中存在静态阵列或全局阵列时，该选项非常实用。

注释：对于任一类的公开变量，必须使用 RESET 编译指示，因为复位配置设置仅适用于在函数级别或文件级别声明的变量。此外，RESET 编译指示必须应用于顶层函数或子函数中的类的对象，不能应用于类的私有变量。

语法

将 C 语言源代码中的编译指示置于变量生命周期边界内。

```
set_directive_reset <location> [ variable=<a> | off ]
```

其中：

- `location`: 指定定义变量的位置（格式为 `function[/label]`）。
- `variable=<a>`: 指定 RESET 编译指示应用到的变量。

- `off` 或 `off=true`：指示针对指定变量不生成复位。

示例

为 `foo` 函数中的 `a` 变量添加复位，即使全局复位 (`config_rtl -reset`) 设置为 `none` 或 `control` 也是如此。

```
set_directive_reset foo a
```

从 `foo` 函数中的 `static int a` 变量移除复位，即使全局复位设置为 `state` 或 `all` 也是如此。

```
set_directive_reset -off foo a
```

以下示例显示了如何将 RESET 编译指示或指令与类变量和方法搭配使用。在类中声明的变量必须是该类中的公开静态变量，并且可在类的方法中为该变量指定 RESET 编译指示，或者在 HLS 设计的顶层函数或子函数中创建类对象之后为该变量指定 RESET 编译指示。

```
class bar {
public:
    static int a; // class level static; must be public
    int a1;      // class-level non-static; must be public
    bar(...) {
        // #pragma reset does not work here for a or a1
    }
    // this is the function that is called in the top
    void run(...) {
        // #pragma reset does not work here for non-static variable a1
        // but does work for static variable a
        #pragma reset variable=a
    }
};

static int c; // file level
int d; // file level
void top(...) {
    #pragma HLS top
    static int b; // function level
    bar t;
    static bar s;

    // #pragma reset works here for b, c and d, as well as t and s members
    // except for t.a1 because it is not static
    #pragma reset variable=b
    #pragma reset variable=c
    #pragma reset variable=d
    #pragma reset variable=t.a
    #pragma reset variable=s.a
    #pragma reset variable=s.a1
    t.run(...);
    s.run(...);
}
```

为以上所示 `top` 函数指定的 `set_directive_reset` 命令将如下所示：

```
set_directive_reset top variable=b
set_directive_reset top variable=c
set_directive_reset top variable=d
set_directive_reset top variable=t.a
set_directive_reset top variable=s.a
set_directive_reset top variable=s.a1
```

示例 3

另请参阅

- [pragma HLS reset](#)
- [config_rtl](#)

set_directive_stable

描述

标记为 `stable` 的实参在内核执行期间不得更改。仅当内核未运行时才能更改的数据（例如，提供配置数据的端口）应以 `STABLE` 编译指示或指令来加以标记。`STABLE` 编译指示只能用于控制驱动的任务建模，因为，在数据驱动的任务级并行化中，无法检测内核何时处于空闲状态。

`STABLE` 编译指示会将以下信息告知 Vitis HLS：

- 应用于端口的数据在正常操作期间保持稳定，但不是可以优化的常量值。
- 此端口的扇出无需寄存。

`DATAFLOW` 网络中的编译指示 `STABLE` 会告知编译器，网络中第一个数据流进程无需读取输入，网络中最后一个数据流进程无需写入输出。这样即可显著改善 II。

```
set_directive_stable <location> <variable>
```

- `<location>` 是要在其中对指令进行约束的函数名称或循环名称。
- `<variable>` 是要约束的标量或阵列变量的名称。

示例

在以下示例中，不使用 `STABLE` 指令的情况下，`proc1` 与 `proc2` 将同步，以确认其输入的读取操作（包括 `A`）。使用该指令后，`A` 将不再被视为需同步的输入。

```
void dataflow_region(int A[...], int B[...] ...  
    proc1(...);  
    proc2(A, ...);
```

此示例对应的指令脚本如下：

```
set_directive_stable dataflow_region A  
set_directive_dataflow dataflow_region
```

另请参阅

- [pragma HLS stable](#)
- [set_directive_dataflow](#)

set_directive_stream

描述

默认情况下，阵列变量作为 RAM 来实现：

- 顶层函数阵列参数作为 RAM 接口端口来实现。
- 一般阵列作为 RAM 来实现以便进行读写访问。
- 包含在子函数内的阵列或者基于循环的 DATAFLOW 优化都作为 RAM 乒乓缓冲器通道来实现。

如果按顺序来使用或生成阵列中的数据，那么更有效的通信机制是使用串流数据，其中使用 FIFO 代替 RAM。当顶层函数实参的 INTERFACE 类型被指定为 `ap_fifo` 时，该阵列会以串流方式自动实现。如需了解更多信息，请参阅 [定义接口](#)。



重要提示！ 要保留访问，可能需要使用 `volatile` 限定符来阻止编译器优化（尤其是死码消除），如 [类型限定符](#) 中所述。

语法

```
set_directive_stream [OPTIONS] <location> <variable>
```

- `<location>` 是包含阵列变量的位置（格式为 `function[/label]`）。
- `<variable>` 是要作为 FIFO 来实现的阵列变量。

选项

- `-depth <integer>`:

注释： 仅与数据流通道中的阵列串流相关。

默认情况下，RTL 中实现的 FIFO 的深度与 C 语言代码中指定的阵列大小相同。该选项允许您修改 FIFO 的大小。

当在 DATAFLOW 区域内实现阵列时，经常使用 `-depth` 选项来减小 FIFO 的大小。例如，如果 DATAFLOW 区域中的所有循环和函数均以 `II = 1` 的速率来处理数据，则无需大型 FIFO，因为在每个时钟周期内都会生产和使用数据。在此情况下，`-depth` 选项可用于将 FIFO 大小减小为 2 以便大幅减小 RTL 设计的面积。

通过使用 `config_dataflow` 命令搭配 `-depth` 选项即可为 DATAFLOW 区域内所有阵列提供该功能。`-depth` 选项搭配 `set_directive_stream` 一起使用即可覆盖使用 `config_dataflow` 指定的默认设置。

- `-type <arg>`: 指定机制，可选机制包括 FIFO、PIPO、已同步的共享 (`shared`) 和未同步的共享 (`unsync`)。受支持的类型包括：
 - `fifo`: 含指定 `depth` 的 FIFO 缓冲器
 - `pipo`: 乒乓缓冲器，所含“bank”数量与指定深度相同（默认值为 2）
 - `shared`: 含深度的共享通道，其同步方式与常规乒乓缓冲器类似，但不复制阵列数据。通过将深度设置为足够小的值以充当生产者与使用者之间的同步距离，即可确保一致性



提示： `shared` 默认深度值为 1。

- `unsync`: 除了个别存储器读取和写入外，无任何同步。设计必须自行确保一致性（读取 - 写入和写入 - 写入顺序）

示例

指定 `func` 函数中要进行串流的 `A[10]` 阵列，并作为 FIFO 实现。

```
set_directive_stream func A -type fifo
```

`func` 函数中名为 `loop_1` 的循环中的阵列 `B` 设置为作为 FIFO（深度为 12）来进行串流。在此情况下，请将编译指示置于 `loop_1` 内。

```
set_directive_stream -depth 12 -type fifo func/loop_1 B
```

阵列 `C` 的串流以 PIPO 方式来实现。

```
set_directive_stream -type pipo func C
```

另请参阅

- [pragma HLS stream](#)
- [set_directive_dataflow](#)
- [set_directive_interface](#)
- [config_dataflow](#)

set_directive_top

描述

给函数附加名称，随后此名称可供 `set_top` 命令用于将指定函数设置为顶层函数。此命令通常用于对 C++ 中的某个类的成员函数进行综合。



建议：指定当前处于活动状态的解决方案中的指令。将 `set_top` 命令与新名称搭配使用。

语法

```
set_directive_top [OPTIONS]
```

选项

- `-name <string>`：指定要供 `set_top` 命令使用的函数名称。

示例

`foo_long_name` 函数重命名为 `DESIGN_TOP`，随后将其指定为顶层函数。如果在代码中放置编译指示，则仍必须在 GUI 工程设置中指定的顶层发出 `set_top` 命令。

```
set_directive_top -name DESIGN_TOP foo_long_name
```

随后发出 `set_top DESIGN_TOP` 命令。

另请参阅

- [pragma HLS top](#)
- [set_top](#)

set_directive_unroll

描述

通过创建循环主体的多个副本来变换循环。

循环的迭代执行次数由循环归纳变量来指定。迭代次数也可能受到循环主体内的逻辑影响（例如，对任意循环出口变量进行中断或修改）。在 RTL 中，循环按表示循环主体的逻辑块来实现，此类逻辑块会执行相同次数的迭代。

`set_directive_unroll` 命令支持将循环完全展开。展开循环会在 RTL 中创建循环主体副本，副本数量与循环迭代次数相同，或者也可按因数 N 部分展开，从而创建循环主体的 N 份副本并对循环迭代次数进行相应的调整。

如果用于部分展开的因数 N 并非原始循环迭代计数的整数倍，那么必须在循环主体的每个展开片段后检查原始退出条件。

要完全展开循环，在编译时循环边界必须已知。对于部分展开则无需满足此条件。

语法

```
set_directive_unroll [OPTIONS] <location>
```

- `<location>` 表示要展开的循环位置（格式为 `format function[/label]`）。

选项

- `-factor <integer>`：指定非零整数，表示已请求部分展开。

循环主体将按此次数重复。迭代信息将进行相应的调整。

- `-skip_exit_check`：仅当指定因数时才有效（部分展开）。

- 固定边界

如果迭代计数为因数的倍数，则不执行出口条件检查。

如果迭代计数并非因数的整数倍，则该工具将执行以下操作：

- 阻止展开。
- 发出警告，称必须执行出口检查后才能继续。

- 变量边界

移除出口条件检查。您必须确保：

- 变量边界为因数的整数倍。
- 实际上无需出口检查。

- `-off` 或 `off=true`：为指定循环禁用展开。

示例

展开 `foo` 函数内的 L1 循环。将编译指示置于 L1 循环的主体内。

```
set_directive_unroll foo/L1
```

在 `foo` 函数的 L2 循环上指定展开因数 4。移除出口检查。将编译指示置于 L2 循环的主体内。

```
set_directive_unroll -skip_exit_check -factor 4 foo/L2
```

另请参阅

- [pragma HLS unroll](#)
- [set_directive_loop_flatten](#)
- [set_directive_loop_merge](#)

第五部分

Vitis HLS C 语言驱动程序参考资料

本部分包含以下章节：

- [AXI4-Lite 从接口 C 语言驱动程序参考](#)

AXI4-Lite 从接口 C 语言驱动程序参考

X<DUT>_Initialize

语法

```
int X<DUT>_Initialize(X<DUT> *InstancePtr, uintptr BaseAddress);
int X<DUT>_Initialize(X<DUT> *InstancePtr, u16 DeviceId);
int X<DUT>_Initialize(X<DUT> *InstancePtr, const char* InstanceName);
```

描述

`int X<DUT>_Initialize(X<DUT> *InstancePtr, uintptr BaseAddress)`: 对器件指定的 `BaseAddress` 进行初始化以供在独立系统上使用。此 API 将把正确的值写入 `InstancePtr`，随后即可在其他 API 内使用。

注释：`DeviceId` 不再填充至驱动程序配置结构体内。而是改用 `BaseAddress` 来识别驱动程序实例。`DeviceId` 将不填充进 `xparameters.h` 文件内。对于在传统 Vitis 或 Vitis HLS 工具内使用这些驱动程序开发的软件应用，需要通过使用 `#ifdef SDT` 进行更新才能反映此更改，如下示例所示：

```
#ifndef SDT
#define GPIO_EXAMPLE_DEVICE_ID  XPAR_GPIO_0_DEVICE_ID
#else
#define XGPIO_AXI_BASEADDRESS    XPAR_XGPIO_0_BASEADDR
#endif
```

`int X<DUT>_Initialize(X<DUT> *InstancePtr, u16 DeviceId)`: 对器件进行初始化以供在独立系统上使用。此 API 将把正确的值写入 `InstancePtr`，随后即可在其他 API 内使用。AMD 建议调用此 API 来初始化器件，但在系统中使用 MMU 时除外，在此情况下请参阅 `X<DUT>_CfgInitialize` 函数。

`int X<DUT>_Initialize(X<DUT> *InstancePtr, const char* InstanceName)`: 初始化为 `uio` 的器件，以供在 Linux 系统上使用。使用 `sysfs` 中的 `uio` 器件信息创建最多 5 个存储器映射并按 `mmap` 分配从基址。

其中，各实参定义如下：

- `InstancePtr`: 指向器件实例的指针。
- `DeviceId`: `xparameters.h` 中定义的器件 ID。
- `BaseAddress`: 此 `BaseAddress` 是在 `xparameters.h` 中定义的。
- `InstanceName`: `uio` 器件的名称。

返回

`XST_SUCCESS` 表示成功，否则即为失败。

X<DUT>_CfgInitialize

语法

```
X<DUT>_CfgInitializeint X<DUT>_CfgInitialize(X<DUT> *InstancePtr,
X<DUT>_Config *ConfigPtr);
```

描述

在系统中使用 MMU 时初始化器件。在此类情况下，AXI4-Lite 从接口的有效地址不同于 `xparameters.h` 中指定的地址，并且需要使用 API 来初始化器件。

- `InstancePtr`：指向器件实例的指针。
- `ConfigPtr`：指向 `X<DUT>_Config` 的指针。

返回

`XST_SUCCESS` 表示成功，否则即为失败。

X<DUT>_LookupConfig

语法

```
X<DUT>_Config* X<DUT>_LookupConfig(uintptr BaseAddress);
X<DUT>_Config* X<DUT>_LookupConfig(u16 DeviceId);
```

描述

此函数用于获取指定 `BaseAddress` 或 `DeviceId` 的配置信息。其中：

- `BaseAddress`：此 `BaseAddress` 是在 `xparameters.h` 中定义的。
- `DeviceId`：`xparameters.h` 中定义的器件 ID。

注释： `DeviceId` 不再填充至驱动程序配置结构体内。而是改用 `BaseAddress` 来识别驱动程序实例。`DeviceId` 将不填充进 `xparameters.h` 文件内。对于在传统 Vitis 或 Vitis HLS 工具内使用这些驱动程序开发的软件应用，需要通过使用 `#ifdef SDT` 进行更新才能反映此更改，如下示例所示：

```
#ifndef SDT
#define GPIO_EXAMPLE_DEVICE_ID XPAR_GPIO_0_DEVICE_ID
#else
#define XGPIO_AXI_BASEADDRESS XPAR_XGPIO_0_BASEADDR
#endif
```

返回

指向 `X<DUT>_LookupConfig` 变量的指针，此变量包含 ID 为 `DeviceId` 的器件或指定 `BaseAddress` 的配置信息。如未找到匹配项，则返回 `NULL`。

X<DUT>_Release

语法

```
int X<DUT>_Release(X<DUT> *InstancePtr);
```

描述

释放 uio 器件。使用 `munmap` 删除映射。如果此进程终止，将自动删除映射。

- `InstanceName`: uio 器件的名称。

返回

`XST_SUCCESS` 表示成功，否则即为失败。

X<DUT>_Start

语法

```
void X<DUT>_Start(X<DUT> *InstancePtr);
```

描述

启动器件。此函数将在器件上断言 `ap_start` 端口有效。仅当器件上存在 `ap_start` 端口时才可用。

- `InstancePtr`: 指向器件实例的指针。

X<DUT>_IsDone

语法

```
void X<DUT>_IsDone(X<DUT> *InstancePtr);
```

描述

检查器件是否已完成先前执行的操作：此函数将返回器件上 `ap_done` 端口的值。仅当器件上存在 `ap_done` 端口时才可用。

- `InstancePtr`: 指向器件实例的指针。

X<DUT>_IsIdle

语法

```
void X<DUT>_IsIdle(X<DUT> *InstancePtr);
```

描述

检查器件是否处于空闲状态：此函数将返回 `ap_idle` 端口的值。仅当器件上存在 `ap_idle` 端口时才可用。

- `InstancePtr`：指向器件实例的指针。

X<DUT>_IsReady

语法

```
void X<DUT>_IsReady(X<DUT> *InstancePtr);
```

描述

检查器件是否已准备好处理下一项输入：此函数将返回 `ap_ready` 端口的值。仅当器件上存在 `ap_ready` 端口时才可用。

- `InstancePtr`：指向器件实例的指针。

X<DUT>_Continue

语法

```
void XExample_Continue(XExample *InstancePtr);
```

描述

断言 `ap_continue` 端口有效。仅当器件上存在 `ap_continue` 端口时才可用。

- `InstancePtr`：指向器件实例的指针。

X<DUT>_EnableAutoRestart

语法

```
void X<DUT>_EnableAutoRestart(X<DUT> *InstancePtr);
```

描述

在器件上启用“auto restart”（自动重启）。启用后，

- 当器件断言 `ap_done` 有效后将立即断言 `ap_start` 端口有效，器件将自动启动下一项传输事务。
- 或者，如果在器件上实现块级 I/O 协议 `ap_ctrl_chain`，那么当器件断言 `ap_ready` 有效后，下一项传输事务将自动重启（将断言 `ap_start`），并且当器件断言 `ap_done` 有效时，将断言 `ap_continue` 有效。

仅当存在 `ap_start` 端口时才可用。

- `InstancePtr`：指向器件实例的指针。

X<DUT>_DisableAutoRestart

语法

```
void X<DUT>_DisableAutoRestart(X<DUT> *InstancePtr);
```

描述

禁用“自动重启”功能。仅当存在 `ap_start` 端口时才可用。

- `InstancePtr`：指向器件实例的指针。

X<DUT>_Set_ARG

语法

```
void X<DUT>_Set_ARG(X<DUT> *InstancePtr, u32 Data);
```

描述

将值写入 `ARG` 端口（顶层函数的标量实参）。仅当 `ARG` 为输入端口时才可用。

- `InstancePtr`：指向器件实例的指针。
- `Data`：要写入的值。

X<DUT>_Set_ARG_vld

语法

```
void X<DUT>_Set_ARG_vld(X<DUT> *InstancePtr);
```

描述

断言 ARG_vld 端口有效。仅当 ARG 为输入端口并使用 ap_hs 或 ap_vld 接口协议实现时才可用。

- InstancePtr：指向器件实例的指针。

X<DUT>_Set_ARG_ack

语法

```
void X<DUT>_Set_ARG_ack(X<DUT> *InstancePtr);
```

描述

断言 ARG_ack 端口有效。仅当 ARG 为输出端口并使用 ap_hs 或 ap_ack 接口协议实现时才可用。

- InstancePtr：指向器件实例的指针。

X<DUT>_Get_ARG

语法

```
u32 X<DUT>_Get_ARG(X<DUT> *InstancePtr);
```

描述

读取 ARG 中的值。仅当 ARG 端口为器件上的输出端口时才可用。

- InstancePtr：指向器件实例的指针。

返回

ARG 的值。

X<DUT>_Get_ARG_vld

语法

```
u32 X<DUT>_Get_ARG_vld(X<DUT> *InstancePtr);
```

描述

读取 ARG_vld 中的值。仅当 ARG 端口为器件上的输出端口并使用 ap_hs 或 ap_vld 接口协议实现时才可用。

- InstancePtr：指向器件实例的指针。

返回

ARG_vld 的值。

X<DUT>_Get_ARG_ack

语法

```
u32 X<DUT>_Get_ARG_ack(X<DUT> *InstancePtr);
```

描述

读取 ARG_ack 中的值。仅当 ARG 端口为器件上的输入端口并使用 ap_hs 或 ap_ack 接口协议实现时才可用。

- InstancePtr：指向器件实例的指针。

返回

ARG_ack 的值。

X<DUT>_Get_ARG_BaseAddress

语法

```
u32 X<DUT>_Get_ARG_BaseAddress(X<DUT> *InstancePtr);
```

描述

返回接口内的阵列基址。仅当 ARG 为组合到 AXI4-Lite 接口中的阵列时才可用。

- InstancePtr：指向器件实例的指针。

返回

阵列的基址。

X<DUT>_Get_ARG_HighAddress

语法

```
u32 X<DUT>_Get_ARG_HighAddress(X<DUT> *InstancePtr);
```

描述

返回阵列最上方元素的地址。仅当 ARG 为组合到 AXI4-Lite 接口中的阵列时才可用。

- InstancePtr：指向器件实例的指针。

返回

阵列最上方元素的地址。

X<DUT>_Get_ARG_TotalBytes

语法

```
u32 X<DUT>_Get_ARG_TotalBytes(X<DUT> *InstancePtr);
```

描述

返回用于存储阵列的字节总数。仅当 ARG 为组合到 AXI4-Lite 接口中的阵列时才可用。

如果阵列中的元素少于 16 位，AMD Vitis™ HLS 会将多个元素分组到 AXI4-Lite 接口的位宽为 32 位的数据中。如果元素的位宽超过 32 位，Vitis HLS 会将每个元素存储在多个连续地址上。

- InstancePtr：指向器件实例的指针。

返回

用于存储阵列的字节总数。

X<DUT>_Get_ARG_BitWidth

语法

```
u32 X<DUT>_Get_ARG_BitWidth(X<DUT> *InstancePtr);
```

描述

返回阵列中每个元素的位宽。仅当 ARG 为组合到 AXI4-Lite 接口中的阵列时才可用。

如果阵列中的元素少于 16 位，Vitis HLS 会将多个元素分组到 AXI4-Lite 接口的位宽为 32 位的数据中。如果元素的位宽超过 32 位，Vitis HLS 会将每个元素存储在多个连续地址上。

- InstancePtr：指向器件实例的指针。

返回

阵列中每个元素的位宽。

X<DUT>_Get_ARG_Depth

语法

```
u32 X<DUT>_Get_ARG_Depth(X<DUT> *InstancePtr);
```

描述

返回阵列中元素总数。仅当 ARG 为组合到 AXI4-Lite 接口中的阵列时才可用。

如果阵列中的元素少于 16 位，Vitis HLS 会将多个元素分组到 AXI4-Lite 接口的位宽为 32 位的数据中。如果元素的位宽超过 32 位，Vitis HLS 会将每个元素存储在多个连续地址上。

- InstancePtr：指向器件实例的指针。

返回

阵列中的元素总数。

X<DUT>_Write_ARG_Words

语法

```
u32 X<DUT>_Write_ARG_Words(X<DUT> *InstancePtr, int offset, int *data, int length);
```

描述

将长度为 32 位的字写入 AXI4-Lite 接口的指定地址。此 API 需要来自 `BaseAddress` 的偏移地址以及要存储的数据长度。仅当 `ARG` 为组合到 AXI4-Lite 接口中的阵列时才可用。

- `InstancePtr`：指向器件实例的指针。
- `offset`：AXI4-Lite 接口中的地址。
- `data`：指向要存储的数据值的指针。
- `length`：要存储的数据长度。

返回

从指定地址写入的数据长度。

X<DUT>_Read_ARG_Words

语法

```
u32 X<DUT>_Read_ARG_Words(X<DUT> *InstancePtr, int offset, int *data, int length);
```

描述

从阵列中读取长度为 32 位的字。此 API 需要数据目标、来自 `BaseAddress` 的偏移地址以及要存储的数据长度。仅当 `ARG` 为组合到 AXI4-Lite 接口中的阵列时才可用。

- `InstancePtr`：指向器件实例的指针。
- `offset`：ARG 中的地址。
- `data`：指向数据缓冲器的指针。
- `length`：要存储的数据长度。

返回

从指定地址读取的数据长度。

X<DUT>_Write_ARG_Bytes

语法

```
u32 X<DUT>_Write_ARG_Bytes(X<DUT> *InstancePtr, int offset, char *data, int length);
```

描述

将字节长度写入 AXI4-Lite 接口的指定地址。此 API 需要来自 BaseAddress 的偏移地址以及要存储的数据长度。仅当 ARG 为组合到 AXI4-Lite 接口中的阵列时才可用。

- InstancePtr：指向器件实例的指针。
- offset：ARG 中的地址。
- data：指向要存储的数据值的指针。
- length：要存储的数据长度。

返回

从指定地址写入的数据长度。

X<DUT>_Read_ARG_Bytes

语法

```
u32 X<DUT>_Read_ARG_Bytes(X<DUT> *InstancePtr, int offset, char *data, int length);
```

描述

从阵列读取字节长度。此 API 需要数据目标、距离基址的偏移地址以及要加载的数据长度。仅当 ARG 为组合到 AXI4-Lite 接口中的阵列时才可用。

- InstancePtr：指向器件实例的指针。
- offset：ARG 中的地址。
- data：指向数据缓冲器的指针。
- length：要加载的数据长度。

返回

从指定地址读取的数据长度。

X<DUT>_InterruptGlobalEnable

语法

```
void X<DUT>_InterruptGlobalEnable(X<DUT> *InstancePtr);
```


描述

启用中断输出。仅当有 `ap_start` 时，中断功能才可用。

- `InstancePtr`：指向器件实例的指针。

X<DUT>_InterruptGlobalDisable

语法

```
void X<DUT>_InterruptGlobalDisable(X<DUT> *InstancePtr);
```

描述

禁用中断输出。

- `InstancePtr`：指向器件实例的指针。

X<DUT>_InterruptEnable

语法

```
void X<DUT>_InterruptEnable(X<DUT> *InstancePtr, u32 Mask);
```

描述

启用中断源。可存在最多 2 个中断源（`source 0` 对应 `ap_done`，`source 1` 对应 `ap_ready`）。

- `InstancePtr`：指向器件实例的指针。
- `Mask`：位掩码。
 - `Bit n = 1`：启用中断源 `n`。
 - `Bit n = 0`：无更改。

X<DUT>_InterruptDisable

语法

```
void X<DUT>_InterruptDisable(X<DUT> *InstancePtr, u32 Mask);
```

描述

禁用中断源。

- `InstancePtr`：指向器件实例的指针。
- `Mask`：位掩码。
 - Bit $n = 1$ ：禁用中断源 n 。
 - Bit $n = 0$ ：无更改。

X<DUT>_InterruptClear

语法

```
void X<DUT>_InterruptClear(X<DUT> *InstancePtr, u32 Mask);
```

描述

清除中断状态。

- `InstancePtr`：指向器件实例的指针。
- `Mask`：位掩码。
 - Bit $n = 1$ ：切换中断源 n 。
 - Bit $n = 0$ ：无更改。

X<DUT>_InterruptGetEnabled

语法

```
u32 X<DUT>_InterruptGetEnabled(X<DUT> *InstancePtr);
```

描述

检查启用的中断源。

- `InstancePtr`：指向器件实例的指针。

返回

位掩码。

- 位 $n = 1$ ：已启用。
- 位 $n = 0$ ：已禁用。

X<DUT>_InterruptGetStatus

语法

```
u32 X<DUT>_InterruptGetStatus(X<DUT> *InstancePtr);
```

描述

检查触发的中断源。

- InstancePtr：指向器件实例的指针。

返回

位掩码。

- Bit n = 1：已触发。
- Bit n = 0：未触发。

第六部分

Vitis HLS 库参考资料

本部分包含以下章节：

- [C/C++ 语言内置函数](#)
- [任意精度数据类型库](#)
- [HLS Print 函数](#)
- [HLS 数学库](#)
- [HLS 串流库](#)
- [HLS 向量库](#)
- [HLS 任务库](#)
- [HLS 拆分/合并库](#)
- [HLS 块串流库](#)
- [HLS IP 库](#)

C/C++ 语言内置函数

Vitis HLS 支持以下 C/C++ 语言内置函数：

- `__builtin_clz(unsigned int x)`：返回前置 0 位数量（以 `x` 表示），从最高有效位的位置开始。如果 `x` 为 0，则结果为未定义。
- `__builtin_ctz(unsigned int x)`：返回后置 0 位数量（以 `x` 表示），从最低有效位的位置开始。如果 `x` 为 0，则结果为未定义。

以下示例显示这些函数的使用方式。此示例返回 `in0` 中前置 0 的数量和 `in1` 中后置 0 的数量之和：

```
int foo (int in0, int in1) {  
    int ldz0 = __builtin_clz(in0);  
    int ldz1 = __builtin_ctz(in1);  
    return (ldz0 + ldz1);  
}
```

任意精度数据类型库

基于 C 语言的原生数据类型位于 8 位边界（8 位、16 位、32 位和 64 位）上。RTL 总线（对应于硬件）支持任意长度。HLS 需要相应的机制以支持任意精度位宽规格，并且不依赖于原生 C 语言数据类型的人工边界：如需 17 位乘法器，则不应强制使用 32 位乘法器来实现。

AMD Vitis™ HLS 可以为 C++ 提供整数数据类型和定点任意精度数据类型。任意精度数据类型的优势在于允许更新 C 语言代码以使用位宽较小的变量，然后允许重新执行 C 语言仿真以确认功能保持不变或可接受。

使用任意精度数据类型

Vitis HLS 提供了任意精度整数数据类型，用于管理指定宽度边界内整数的值，如下表所示。

表 50: 任意精度数据类型

语言	整数数据类型	所需头文件
C++	ap_[u]int<W>（1024 位） 位宽可扩展至 4K 位，如 C++ 任意精度整数数据类型 中所述。	#include "ap_int.h"
C++	ap_[u]fixed<W,I,Q,O,N>	#include "ap_fixed.h"

Vitis HLS 也随附定义任意精度类型的头文件作为独立封装，并且您有权在自己的源代码中使用这些任意精度类型。在 Vitis HLS 安装区域的 include 目录中提供了 xilinx_hls_lib_<release_number>.tgz 封装。

用于 C++ 的任意整数精度类型

头文件 ap_int.h 用于为 C++ ap_[u]int 数据类型定义任意精度整数数据类型。要在 C++ 函数中使用任意精度整数数据类型，请执行以下操作：

- 将头文件 ap_int.h 添加到源代码。
- 针对有符号的类型将位类型更改为 ap_int<N>，或者针对无符号的类型使用 ap_uint<N>，其中 N 表示范围介于 1 到 1024 之间的位大小。

以下示例显示了如何添加头文件并实现 2 个变量来使用 9 位整数和 10 位无符号的整数类型：

```
#include "ap_int.h"

void foo_top () {

    ap_int<9>   var1;           // 9-bit
    ap_uint<10> var2;           // 10-bit unsigned
```



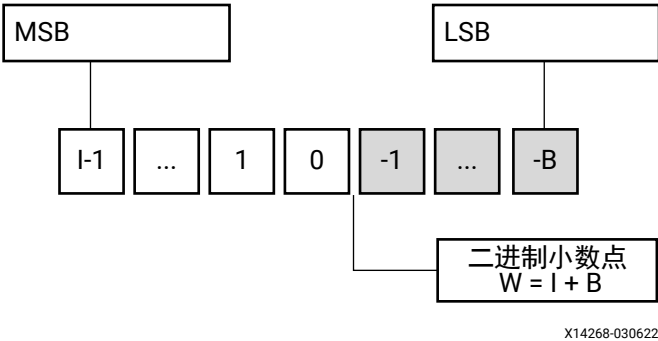
重要提示！ AP 数据类型的劣势之一是阵列不会以 0 值进行自动初始化。如果需要初始化阵列，必须手动执行。

任意精度定点数据类型

在 Vitis HLS 中，使用定点数据类型至关重要，因为使用定点数据类型执行的 C++ 语言仿真的行为与综合创建的硬件的行为相匹配。这使您能够使用 C 语言层次快速仿真来分析位精度、量化和上溢的影响。

这些数据类型借助表达式 $(W = I + B)$ 来管理指定总宽度 (W) 和整数宽度 (I) 边界内的实数（非整数）的值，如下图所示。

图 142：定点数据类型



提示：在上图中，整数值 (I) 指定了二进制小数点左侧的整数位数（包括符号位）。

定点标识符汇总

下表提供了定点类型支持的运算的简要概述。

表 51：定点标识符汇总

标识符	描述
W	字宽（位）
I	用于表示整数值的位数，即，二进制小数点左侧的整数位数，包括符号位。 当 I 为负值（如以下示例所示）时，它表示隐式有符号位（对应符号表示法）的数量，或者小数点右侧的隐式零位（对应无符号表示法）的数量。例如： <pre>ap_fixed<2, 0> a = -0.5; // a can be -0.5, ap_ufixed<1, 0> x = 0.5; // 1-bit representation. x can be 0 or 0.5 ap_ufixed<1, -1> y = 0.25; // 1-bit representation. y can be 0 or 0.25 const ap_fixed<1, -7> z = 1.0/256; // 1-bit representation for z = 2^-8</pre>

表 51：定点标识符汇总 (续)

标识符	描述	
Q	量化模式：用于规定当生成的精度大于用于存储结果的变量中的最小小数位可定义的精度时的行为。	
	ap_fixed 类型	描述
	AP_RND	舍入到正无穷
	AP_RND_ZERO	舍入到 0
	AP_RND_MIN_INF	舍入到负无穷
	AP_RND_INF	舍入到无穷
	AP_RND_CONV	收敛舍入
	AP_TRN	截位到负无穷（默认）
	AP_TRN_ZERO	截位到 0
O	上溢模式：用于规定当运算结果超出用于存储结果的变量可存储的最大值（或者对于负值为最小值）时的行为。	
	ap_fixed 类型	描述
	AP_SAT ¹	饱和
	AP_SAT_ZERO ¹	饱和到 0
	AP_SAT_SYM ¹	对称饱和
	AP_WRAP	卷绕（默认）
	AP_WRAP_SM	符号量值卷绕
N	定义上溢卷绕模式中的饱和位数。	

注释：

1. 使用 AP_SAT* 模式可以提升资源使用率，因为需要额外逻辑来执行饱和，此额外成本可能额外增加高达 20% 的 LUT 使用率。
2. 来自 hls_math 库的定点数学函数不支持 ap_[u]fixed 模板参数 Q、O 和 N，这三个参数依次分别对应于量化模式、上溢模式和饱和位数。仅当 ap_[u]fixed 变量位于赋值左侧时或者在初始化期间，量化模式和上溢模式才有效，但在变量计算期间无效。

使用 ap_fixed 的示例

在此示例中，Vitis HLS ap_fixed 类型用于定义 18 位变量，其中 6 位用于表示小数点前的数值（包括符号位），12 位用于表示小数点后的值。此变量指定为有符号变量，量化模式设置为舍入到正无穷，针对上溢使用默认卷绕模式。

```
#include <ap_fixed.h>
...
ap_fixed<18,6,AP_RND > my_type;
...
```

C++ 任意精度整数类型

C++ 语言中的原生数据类型以 8 位为边界（8 位、16 位、32 位，和 64 位）。RTL 信号和运算支持任意位长。

Vitis HLS 为 C++ 提供任意精度数据类型，允许以任意位宽指定 C++ 代码中的变量和运算：6 位、17 位、234 位以及（最高）1024 位。



提示：允许的默认最大宽度为 1024 位。可通过在包含 `ap_int.h` 头文件之前定义含正整数（小于或等于 4096）的 `AP_INT_MAX_W` 宏来覆盖此默认值。

任意精度数据类型相比于原生 C++ 类型，存在 2 项主要优势：

- 硬件质量更高：例如，如需 17 位乘法器，任意精度类型可指定在计算中精确使用 17 位。
如无任意精度数据类型，则必须使用 32 位整数数据类型实现 17 位乘法，从而导致以多个 DSP 模块来实现乘法。
- 准确的 C++ 仿真/分析：C++ 代码中的任意精度数据类型允许使用精确位宽执行 C++ 仿真，并允许 C++ 仿真在执行综合前确认算法的功能（和准确性）。

C++ 中的任意精度类型不具有 C 的任何劣势：

- C++ 任意类型可使用标准 C++ 编译器来进行编译（C++ 不含等效于 `apcc` 的命令）。
- C++ 任意精度类型不存在“整数提升问题”。

用户经常将文件扩展名从 `.c` 更改为 `.cpp`，因此此类文件可作为 C++ 来编译，并且不存在上述问题。

对于 C++ 语言，头文件 `ap_int.h` 用于定义任意精度整数数据类型 `ap_(u)int<W>`。例如，`ap_int<8>` 表示 8 位有符号的整数数据类型，而 `ap_uint<234>` 表示 234 位无符号的整数类型。

`ap_int.h` 文件位于 `$HLS_ROOT/include` 目录中，其中 `$HLS_ROOT` 是 Vitis HLS 安装目录。

以下示例中所示的代码与 [标准类型](#) 中的“基本算术”示例中所示的代码相同。在此示例中，要综合的顶层函数中的数据类型指定为 `dinA_t`、`dinB_t`，以此类推。

```
#include "cpp_ap_int_arith.h"

void cpp_ap_int_arith(din_A inA, din_B inB, din_C inC, din_D inD,
    dout_1 *out1, dout_2 *out2, dout_3 *out3, dout_4 *out4
) {

    // Basic arithmetic operations
    *out1 = inA * inB;
    *out2 = inB + inA;
    *out3 = inC / inA;
    *out4 = inD % inA;

}
```

在此示例的最新更新中，使用的是 C++ 任意精度类型：

- 将头文件 `ap_int.h` 添加到源代码。
- 将原生 C++ 类型更改为任意精度类型 `ap_int<N>` 或 `ap_uint<N>`，其中 `N` 表示介于 1 到 1024 之间的位大小（如上所述，可根据需要扩展至 4K 位）。

数据类型在 `cpp_ap_int_arith.h` 头文件中定义。

通过与 [标准类型](#) 中的“基本算术”示例比较可以发现，输入数据类型已减缩，以表示实际输入数据的最大大小（例如，8 位输入 `inA` 减缩至 6 位输入）。输出类型已优化，准确性更高，例如，`out2`（`inA` 与 `inB` 之和）只需 13 位，而不是 32 位。

以下示例显示了 C++ 任意精度类型的基本算术。

```
#ifndef _CPP_AP_INT_ARITH_H_
#define _CPP_AP_INT_ARITH_H_

#include <stdio.h>
#include "ap_int.h"

#define N 9

// Old data types
//typedef char dinA_t;
//typedef short dinB_t;
//typedef int dinC_t;
//typedef long long dinD_t;
//typedef int dout1_t;
//typedef unsigned int dout2_t;
//typedef int32_t dout3_t;
//typedef int64_t dout4_t;

typedef ap_int<6> dinA_t;
typedef ap_int<12> dinB_t;
typedef ap_int<22> dinC_t;
typedef ap_int<33> dinD_t;

typedef ap_int<18> dout1_t;
typedef ap_uint<13> dout2_t;
typedef ap_int<22> dout3_t;
typedef ap_int<6> dout4_t;

void cpp_ap_int_arith(dinA_t inA, dinB_t inB, dinC_t inC, dinD_t inD, dout1_t
*out1, dout2_t *out2, dout3_t *out3, dout4_t *out4);

#endif
```

任意精度整数完成综合后，所生成的设计的功能与 [标准类型](#) 相同。它使用内置 `ap_int` 方法 `.to_int()` 将 `ap_int` 结果转换为整数类型，以配合标准 `fprintf` 函数一起使用，而不是使用 C++ `cout` 运算符将结果输出到文件。

```
fprintf(fp, "%d*%d=%d; %d+%d=%d; %d/%d=%d; %d mod %d=%d;\n",
inA.to_int(), inB.to_int(), out1.to_int(),
inB.to_int(), inA.to_int(), out2.to_int(),
inC.to_int(), inA.to_int(), out3.to_int(),
inD.to_int(), inA.to_int(), out4.to_int());
```

C++ 任意精度整数类型：参考信息

Vitis HLS 可提供 C++ 模板类 `ap_[u]int<N>` 以实现任意精度（或位精确）整数数据类型，软件与硬件建模之间可实现一致的位精确行为。

该类可提供原生 C 整数类型允许的所有算术、按位、逻辑和关系运算符。此外，该类还可提供多种方法用于处理某些实用的硬件操作，例如允许对宽度大于 64 位的变量进行初始化和转换。

本章节包含有关使用 `ap_(u)int<N>` 任意精度整数类型的方法、综合行为和各方面的全面信息。

- 用于将常量和初始化值赋值给任意精度整数（包括大于 1024 位的值）的方法。
- 描述 Vitis HLS 帮助程序方法，包括打印、串联、位切片 (bit-slicing) 和范围选择函数。
- 描述运算符行为，包括移位操作（即导致反向移位的负移位值）的描述。

编译 ap_[u]int<> 类型

要使用 ap_[u]int<> 类，必须将 ap_int.h 头文件包含在引用 ap_[u]int<> 变量的所有源文件中。

编译使用这些类的软件模型时，可能需要指定 Vitis HLS 头文件的位置，例如，为 g++ 编译添加 -I/<HLS_HOME>/include 选项。

声明/定义 ap_[u] 变量

有符号的类与无符号的类不同：

- ap_int<int_W> (有符号)
- ap_uint<int_W> (无符号)

int_W 模板参数可指定要声明的变量的总宽度。

用户定义的类型可使用 C/C++ typedef 声明来创建，如以下示例所示：

```
include "ap_int.h" // use ap_[u]fixed<> types

typedef ap_uint<128> uint128_t; // 128-bit user defined type
ap_int<96> my_wide_var; // a global variable declaration
```

允许的默认最大宽度为 1024 位。此默认设置可通过在包含 ap_int.h 头文件之前定义含正整数（小于或等于 4096）的 AP_INT_MAX_W 宏来覆盖。



注意！ AP_INT_MAX_W 的值设置过高可能导致软件编译缓慢、运行时间延长。

以下是覆盖 AP_INT_MAX_W 的示例：

```
#define AP_INT_MAX_W 4096 // Must be defined before next line
#include "ap_int.h"

ap_int<4096> very_wide_var;
```

从常量初始化和赋值（字面值）

类构造函数和赋值运算符重载后，允许使用标准 C/C++ 整数字面值对 ap_[u]int<> 变量进行初始化和赋值。

这种向 ap_[u]int<> 变量赋值的方法受到 C++ 限制和运行软件的系统的影响。这通常导致整数字面值（例如，LL 或 ULL 后缀的整数字面值）限制为 64 位。

为支持使用宽度超过 64 位的值进行赋值，ap_[u]int<> 类提供了允许从任意长度（小于或等于变量宽度）的字符串进行初始化的构造函数。

默认情况下，提供的字符串解释为十六进制值，前提是它仅包含有效的十六进制数字（即，0-9 和 a-f）。要从此类字符串进行赋值，需对字符串进行显式 C++ 语言样式强制转换，将其转换为适当的类型。

以下是初始化和赋值示例（包括大于 64 位的值）：

```
ap_int<42> a_42b_var(-1424692392255LL); // long long decimal format
a_42b_var = 0x14BB648B13FLL; // hexadecimal format

a_42b_var = -1; // negative int literal sign-extended to full width

ap_uint<96> wide_var( "76543210fedcba9876543210" , 16); // Greater than 64-bit
wide_var = ap_int<96>( "0123456789abcdef01234567" , 16);
```

从 2023.2 版起，`ap_[u]int/ap_[u]fixed` 数据类型支持的初始化行为发生了更改，提升了与 C 语言内置类型（例如，`int` 和 `float`）的兼容性。

```
ap_int<N> A[N]; // Such a declaration would result in uninitialized array
elements in all releases.

ap_int<N> B[N] = {0}; // Starting with the 2023.2 release, the whole array
will be initialized to zero. With previous releases, only the first element
is initialized to zero, other elements remained uninitialized.

ap_int<N> B[N] = { }; // With 23.2, the whole array is initialized to zero.
With previous releases, the array elements remained uninitialized.
```

注释：此更改可能导致现在设计无法通过数据流检查，因为阵列元素的初始化将被推断为针对阵列的附加写入访问。

可显式指示 `ap_[u]<>` 构造函数将该字符串解释为以基数 2、8、10 或 16 进制格式来显示数字。具体方法为在构造函数调用中添加相应的基数值作为第 2 个参数。

如果字符串面值包含的任何字符对于指定的基数属于无效数字，那么会发生编译错误。

以下示例使用不同基数格式：

```
ap_int<6> a_6bit_var( "101010" , 2); // 42d in binary format
a_6bit_var = ap_int<6>( "40" , 8); // 32d in octal format
a_6bit_var = ap_int<6>( "55" , 10); // decimal format
a_6bit_var = ap_int<6>( "2A" , 16); // 42d in hexadecimal format

a_6bit_var = ap_int<6>( "42" , 2); // COMPILE-TIME ERROR! "42" is not
binary
```

字符串中已编码的数字的基数还可通过构造函数来推断，前提是以 1 个零 (0) 作为其前缀，后接以下任一字符：`b`、`o` 或 `x`。前缀 `0b`、`0o` 和 `0x` 分别对应于二进制、八进制和十六进制格式。

以下示例使用其他初始化程序字符串格式：

```
ap_int<6> a_6bit_var( "0b101010" , 2); // 42d in binary format
a_6bit_var = ap_int<6>( "0o40" , 8); // 32d in octal format
a_6bit_var = ap_int<6>( "0x2A" , 16); // 42d in hexadecimal format

a_6bit_var = ap_int<6>( "0b42" , 2); // COMPILE-TIME ERROR! "42" is not
binary
```

如果位宽大于 53 位，必须使用如下字符串对 `ap_[u]int<>` 值进行初始化：

```
ap_uint<72> Val( "2460508560057040035.375" );
```

控制台 I/O 支持（打印）

正如 `ap_[u]fixed<>` 变量初始化和赋值一样，Vitis HLS 支持打印显示超 64 位的值。

使用 C++ 标准输出串流

输出存储在 `ap_[u]int` 变量中的任意值的最简单方法是使用 C++ 标准输出串流：

```
std::cout (#include <iostream> 或 <iostream.h>)
```

串流插入运算符 (`<<`) 已重载，可正确输出任意给定 `ap_[u]fixed` 变量的全部可能值。以下串流操控程序同样受支持：

- `dec`（十进制）
- `hex`（十六进制）
- `oct`（八进制）

这些操控程序允许按指定方式对值进行格式化。

以下示例使用 `cout` 来打印值：

```
#include <iostream.h>
// Alternative: #include <iostream>

ap_ufixed<72> Val( "10fedcba9876543210" );

cout << Val << endl; // Yields: "313512663723845890576"
cout << hex << val << endl; // Yields: "10fedcba9876543210"
cout << oct << val << endl; // Yields: "41773345651416625031020"
```

使用标准 C 语言库

您还可使用标准 C 语言库 (`#include <stdio.h>`) 来打印大于 64 位的值：

1. 使用 `ap_[u]fixed` 类方法 `to_string()` 将值转换为 C++ `std::string`。
2. 使用 `std::string` 类方法 `c_str()` 将结果转换为以 `null` 结尾的 C 语言字符串。

可选实参 1（指定基数）

您可为 `ap[u]int::to_string()` 方法传递 1 个可选实参以指定期望的数字格式的基数。有效的基数实参值包括：

- 2（二进制，默认值）
- 8（八进制）
- 10（十进制）
- 16（十六进制）

可选实参 2（打印为有符号的值）

`ap_[u]int::to_string()` 的第 2 个可选实参可指定是否将非十进制格式打印为有符号的值。该实参为布尔值。默认值为 `false`，导致非十进制格式打印为无符号的值。

以下示例使用 `printf` 来打印值：

```
ap_int<72> Val( "80fedcba9876543210" );

printf( "%s\n", Val.to_string().c_str()); // => "80FEDCBA9876543210"
printf( "%s\n", Val.to_string(10).c_str()); // => "-2342818482890329542128"
printf( "%s\n", Val.to_string(8).c_str()); // =>
"401773345651416625031020"
printf( "%s\n", Val.to_string(16, true).c_str()); // =>
"-7F0123456789ABCDF0"
```

涉及 `ap_[u]<>` 类型的表达式

类型为 `ap_[u]<>` 的变量通常可在涉及 C/C++ 运算符的表达式中随意使用。这可能导致某些意外行为。此类行为将在下文中详细讲解。

从较窄的变量赋值到较宽的变量时的零位扩展和符号位扩展

将位宽较窄的有符号 (`ap_int<>`) 变量赋值到较宽的变量时，该值将采用符号位扩展以扩展至目标变量的宽度，这与其是否有符号无关。

同样，无符号的源变量在赋值前将采用零位扩展。

可能需要进行显式源变量类型强制转换，以确保赋值行为符合期望。请参阅以下示例：

```
ap_uint<10> Result;

ap_int<7> Val1 = 0x7f;
ap_uint<6> Val2 = 0x3f;

Result = Val1; // Yields: 0x3ff (sign-extended)
Result = Val2; // Yields: 0x03f (zero-padded)

Result = ap_uint<7>(Val1); // Yields: 0x07f (zero-padded)
Result = ap_int<6>(Val2); // Yields: 0x3ff (sign-extended)
```

从较宽的变量赋值到较窄的变量时的截位

将较宽的源变量赋值到较窄的变量会导致值被截位。超出目标变量的最高有效位 (MSB) 位置的所有位都将丢失。

截位期间对于符号信息不进行任何特殊处理。这可能导致出现意外行为。显式类型强制转换可有助于避免此类意外行为。

方法与运算符分类

`ap_[u]int` 类型不支持从 `wide ap_[u]int` (>64 位) 隐式转换为内置 C/C++ 整数类型。例如，由于在 `if` 语句中从 `ap_int[65]` 到 `bool` 的隐式强制转换返回 0，因此以下代码示例返回 `s1`。

```
bool nonzero(ap_uint<65> data) {
    return data; // This leads to implicit truncation to 64b int
}

int main() {
    if (nonzero((ap_uint<65>)1 << 64)) {
```

```

        return 0;
    }
    printf(FAIL\n);
    return 1;
}

```

要将较宽的 `ap_[u]int` 类型转换为内置整数，请使用 `ap_[u]int` 类型包含的显式转换函数：

- `to_int()`
- `to_long()`
- `to_bool()`

一般情况下，支持针对 `ap_[u]int` 类型使用运算符重载来执行在原生 C/C++ 整数数据类型上执行的任意有效操作。

除这些重载运算符外，还包含部分特定于类的运算符和方法以简化位级运算。

二进制算术运算符

标准二进制整数算术运算符通过重载可提供任意精度算术。这些运算符采用：

- 2 个 `ap_[u]int` 操作数，或者
- 1 个 `ap_[u]int` 类型和 1 个 C/C++ 基本整数数据类型

例如：

- `char`
- `short`
- `int`

在基于目标变量（或表达式）宽度应用符号位扩展、0 值填充或截位前，生成的值的宽度和符号有无取决于操作数的宽度和符号类型。对应每个运算符提供返回值的详细描述。

当表达式混用 `ap_[u]int` 与 C/C++ 基本整数类型时，C++ 类型假定采用如下宽度：

- `char` (8 位)
- `short` (16 位)
- `int` (32 位)
- `long` (32 位)
- `long long` (64 位)

加法

```
ap_(u)int::RType ap_(u)int::operator + (ap_(u)int op)
```

返回以下值之和：

- 2 个 `ap_[u]int`，或
- 1 个 `ap_[u]int` 和 1 个 C/C++ 整数类型

和值的宽度为：

- 比 2 个操作数的宽度多 1 位，或者
- 2 位（仅当较宽的操作数无符号而较窄的操作数有符号时）

如果任一操作数或两个操作数均为有符号类型，则所得的和作为有符号来处理。

减法

```
ap_(u)int::RType ap_(u)int::operator - (ap_(u)int op)
```

返回 2 个整数之差。

差值的宽度为：

- 比 2 个操作数的宽度多 1 位，或者
- 2 位（仅当较宽的操作数无符号而较窄的操作数有符号时）

在赋值前已基于目标变量的宽度执行了符号位扩展、0 值填充或截位的情况下，可满足此条件。

差值作为有符号来处理，与操作数符号类型无关。

乘法

```
ap_(u)int::RType ap_(u)int::operator * (ap_(u)int op)
```

返回 2 个整数值积。

积的宽度是操作数的宽度之和。

如果任一操作数类型为有符号，则积作为有符号类型来处理。

除法

```
ap_(u)int::RType ap_(u)int::operator / (ap_(u)int op)
```

返回 2 个整数值的商。

商的宽度为被除数的宽度（前提是除数为无符号类型）。否则，其宽度为被除数宽度加 1。

如果任一操作数类型为有符号，则商作为有符号类型来处理。

模数

```
ap_(u)int::RType ap_(u)int::operator % (ap_(u)int op)
```

返回 2 个整数值的整数除法的模数或余数。

模数的宽度为操作数宽度的最小值（前提是 2 个操作数符号类型相同）。

如果除数类型为无符号，而被除数类型为有符号，则宽度为除数宽度加 1。

商的符号类型与被除数相同。



重要提示！ Vitis HLS 对模数 (%) 运算符进行综合会导致对生成的 RTL 中已适当参数化的 AMD LogiCORE 被除数核进行例化。

以下是算术运算符的示例：

```
ap_uint<71> Rslt;

ap_uint<42> Val1 = 5;
ap_int<23> Val2 = -8;

Rslt = Val1 + Val2; // Yields: -3 (43 bits) sign-extended to 71 bits
Rslt = Val1 - Val2; // Yields: +3 sign extended to 71 bits
Rslt = Val1 * Val2; // Yields: -40 (65 bits) sign extended to 71 bits
Rslt = 50 / Val2; // Yields: -6 (33 bits) sign extended to 71 bits
Rslt = 50 % Val2; // Yields: +2 (23 bits) sign extended to 71 bits
```

按位逻辑运算符

按位逻辑运算符返回的值的宽度全部为 2 个操作数的最大宽度。仅当 2 个操作数均为无符号时，才将其作为无符号来处理。否则，将其作为有符号类型来处理。

根据表达式（而不是目标变量）的符号类型，可能发生符号位扩展（或 0 值填充）。

按位 OR

```
ap_(u)int::RType ap_(u)int::operator | (ap_(u)int op)
```

返回 2 个操作数的按位 OR。

按位 AND

```
ap_(u)int::RType ap_(u)int::operator & (ap_(u)int op)
```

返回 2 个操作数的按位 AND。

按位 XOR

```
ap_(u)int::RType ap_(u)int::operator ^ (ap_(u)int op)
```

返回 2 个操作数的按位 XOR。

一元运算符

加法

```
ap_(u)int ap_(u)int::operator + ()
```

返回 `ap_[u]int` 操作数的自我复制。

减法

```
ap_(u)int::RType ap_(u)int::operator - ()
```

运行以下操作：

- 操作数的负值，位宽相同（前提是其类型为有符号）；或者

- 如果操作数无符号，则位宽为其位宽加 1。

返回值类型始终为有符号。

按位反转

```
ap_(u)int::RType ap_(u)int::operator ~ ()
```

返回操作数的按位 NOT，位宽相同，符号类型相同。

逻辑反相

```
bool ap_(u)int::operator ! ()
```

仅当操作数不等于零 (0) 时，返回布尔值 `false`。

如果操作数等于零 (0)，则返回布尔值 `true`。

三元运算符

对标准 C 语言 `int` 类型使用三元运算符时，必须从一种类型显式强制转换为另一种类型，以确保 2 个结果的类型相同。例如：

```
// Integer type is cast to ap_int type
ap_int<32> testc3(int a, ap_int<32> b, ap_int<32> c, bool d) {
    return d?ap_int<32>(a):b;
}
// ap_int type is cast to an integer type
ap_int<32> testc4(int a, ap_int<32> b, ap_int<32> c, bool d) {
    return d?a+1:(int)b;
}
// Integer type is cast to ap_int type
ap_int<32> testc5(int a, ap_int<32> b, ap_int<32> c, bool d) {
    return d?ap_int<33>(a):b+1;
}
```

移位运算符

每个移位运算符都带有 2 个版本：

- 1 个版本对应于无符号的右侧 (RHS) 操作数
- 另 1 个版本对应于有符号的右侧 (RHS) 操作数

对有符号的 RHS 版本提供负值将导致移位操作方向反转。即按 RHS 操作数的绝对值发生反方向移位。

移位运算符返回的值的宽度与左侧 (LHS) 操作数相同。就像 C/C++ 一样，如果右移的 LHS 操作数为有符号类型，那么符号位将复制到最高有效位的位置，并保留 LHS 操作数的符号。

无符号的整数右移

```
ap_(u)int ap_(u)int::operator >> (ap_uint<int_W2> op)
```

整数右移

```
ap_(u)int ap_(u)int::operator >> (ap_int<int_W2> op)
```

无符号的整数左移

```
ap_(u)int ap_(u)int::operator << (ap_uint<int_W2> op)
```

整数左移

```
ap_(u)int ap_(u)int::operator << (ap_int<int_W2> op)
```



注意！ 将左移运算符结果赋值到更宽的目标变量时，可能丢失部分或全部信息。AMD 建议您将移位表达式显式强制转换为目标类型以避免发生意外行为。

以下是移位运算示例：

```
ap_uint<13> Rslt;
ap_uint<7> Val1 = 0x41;

Rslt = Val1 << 6; // Yields: 0x0040, i.e. msb of Val1 is lost
Rslt = ap_uint<13>(Val1) << 6; // Yields: 0x1040, no info lost

ap_int<7> Val2 = -63;
Rslt = Val2 >> 4; //Yields: 0x1ffc, sign is maintained and extended
```

复合赋值运算符

Vitis HLS 支持复合赋值运算符：

```
*=
/=
%=
+=
-=
<<=
>>=
&=
^=
|=
```

RHS 表达式首先进行求值，然后作为 RHS 操作数提供给基本运算符，其结果重新赋值给 LHS 变量。表达式大小、符号类型和可能的符号位扩展或截位规则适用方式与上述相关运算所述规则相同。

```
ap_uint<10> Val1 = 630;
ap_int<3> Val2 = -3;
ap_uint<5> Val3 = 27;

Val1 += Val2 - Val3; // Yields: 600 and is equivalent to:

// Val1 = ap_uint<10>(ap_int<11>(Val1) +
// ap_int<11>((ap_int<6>(Val2) -
// ap_int<6>(Val3))));
```

增量运算符和减量运算符

递增运算符和递减运算符均已提供。返回值的宽度全都与操作数相同，仅当 2 个操作数类型均为无符号时，返回值类型为无符号，否则为有符号。

递增前

```
ap_(u)int& ap_(u)int::operator ++ ()
```

返回操作数的递增值。

将递增值赋值给操作数。

递增后

```
const ap_(u)int ap_(u)int::operator ++ (int)
```

返回的值即为将递增值赋值给操作数变量之前的操作数。

递减前

```
ap_(u)int& ap_(u)int::operator -- ()
```

返回操作数的递减值并将递减值赋值给操作数。

递减后

```
const ap_(u)int ap_(u)int::operator -- (int)
```

返回的值即为将递减值赋值给操作数变量之前的操作数。

关系运算符

Vitis HLS 支持所有关系运算符。根据比较结果，这些运算符会返回布尔值。您可将 `ap_[u]int` 类型的变量与含有以下运算符的 C/C++ 基本整数类型进行比较。

等于

```
bool ap_(u)int::operator == (ap_(u)int op)
```

不等于

```
bool ap_(u)int::operator != (ap_(u)int op)
```

小于

```
bool ap_(u)int::operator < (ap_(u)int op)
```

大于

```
bool ap_(u)int::operator > (ap_(u)int op)
```

小于或等于

```
bool ap_(u)int::operator <= (ap_(u)int op)
```

大于或等于

```
bool ap_(u)int::operator >= (ap_(u)int op)
```

其他类方法、运算符和数据成员

以下章节探讨了其他类方法、运算符和数据成员。

位级运算

以下方法有助于对 `ap_[u]int` 类型变量中存储的值执行常见位级运算。

长度

```
int ap_(u)int::length ()
```

返回整数值，以提供 `ap_[u]int` 变量中的总位数。

串联

```
ap_concat_ref ap_(u)int::concat (ap_(u)int low)
ap_concat_ref ap_(u)int::operator , (ap_(u)int high, ap_(u)int low)
```

串联 2 个 `ap_[u]int` 变量，返回的值的宽度为操作数宽度之和。

High 和 Low 实参分别置于结果的高阶位和低阶位；`concat()` 方法用于将实参置于低阶位。

使用已重载的逗号运算符时，括号是必需的。逗号运算符版本还可显示在赋值的 LHS 上。



建议：为避免意外结果，请在串联前将 C/C++ 原生类型（包括整数字面值）强制转换为相应的 `ap_[u]int` 类型。

```
ap_uint<10> Rslt;

ap_int<3> Val1 = -3;
ap_int<7> Val2 = 54;

Rslt = (Val2, Val1); // Yields: 0x1B5
Rslt = Val1.concat(Val2); // Yields: 0x2B6
(Val1, Val2) = 0xAB; // Yields: Val1 == 1, Val2 == 43
```

位选择

```
ap_bit_ref ap_(u)int::operator [] (int bit)
```

从任意精度整数值中选择 1 个位，并将其返回。

返回的值为可用于设置或清除此 `ap_[u]int` 中的对应位的参考值。

位实参必须为 `int` 值。它用于指定要选择的位的索引。最低有效位为索引 0。允许的最高索引是 `ap_[u]int` 的位宽减 1。

生成的 `ap_bit_ref` 类型表示引用该 `ap_[u]int` 实例的 1 个位（按位指定）。

范围选择

```
ap_range_ref ap_(u)int::range (unsigned Hi, unsigned Lo)
ap_range_ref ap_(u)int::operator () (unsigned Hi, unsigned Lo)
```

返回以实参指定的位的范围来表示的值。

`Hi` 实参用于指定此范围的最高有效位 (MSB) 的位置，`Lo` 用于指定最低有效位 (LSB)。

源变量的 LSB 为位置 0。如果 `Hi` 实参的值小于 `Lo` 的值，则会逆序返回这些位。

```
ap_uint<4> Rslt;

ap_uint<8> Val1 = 0x5f;
ap_uint<8> Val2 = 0xaa;

Rslt = Val1.range(3, 0); // Yields: 0xF
Val1(3,0) = Val2(3, 0); // Yields: 0x5A
Val1(3,0) = Val2(4, 1); // Yields: 0x55
Rslt = Val1.range(4, 7); // Yields: 0xA; bit-reversed!
```

注释：范围选择所返回的对象并非 `ap_(u)int` 对象，其中缺少运算符，但可用于赋值。要在链式表达式中将范围选择结果与 `ap_(u)int` 方法搭配使用，请添加显式构造函数，如下所示。

```
ap_uint<32> v = 0x8fff0000;
bool r = ap_uint<16>(v.range(23, 8)).xor_reduce();
```

AND 减缩

```
bool ap_(u)int::and_reduce ()
```

- 对此 `ap_(u)int` 中的所有位应用 AND 运算。
- 返回生成的单个位。
- 等效于将该值与 -1 比较（全部为 1），如果匹配，则返回 `true`，否则返回 `false`。

OR 减缩

```
bool ap_(u)int::or_reduce ()
```

- 对此 `ap_(u)int` 中的所有位应用 OR 运算。
- 返回生成的单个位。
- 等效于将该值与 0 比较（全部为 0），如果匹配，则返回 `false`，否则返回 `true`。

XOR 减缩

```
bool ap_(u)int::xor_reduce ()
```

- 对此 `ap_int` 中的所有位应用 XOR 运算。

- 返回生成的单个位。
- 等效于计算该值中 1 位的数量，如果计数结果为偶数，则返回 `false`，如果为奇数，则返回 `true`。

NAND 减缩

```
bool ap_(u)int::nand_reduce ()
```

- 对此 `ap_int` 中的所有位应用 NAND 运算。
- 返回生成的单个位。
- 等效于将该值与 -1 比较（全部为 1），如果匹配，则返回 `false`，否则返回 `true`。

NOR 减缩

```
bool ap_int::nor_reduce ()
```

- 对此 `ap_int` 中的所有位应用 NOR 运算。
- 返回生成的单个位。
- 等效于将该值与 0 比较（全部为 0），如果匹配，则返回 `true`，否则返回 `false`。

XNOR 减缩

```
bool ap_(u)int::xnor_reduce ()
```

- 对此 `ap_(u)int` 中的所有位应用 XNOR 运算。
- 返回生成的单个位。
- 等效于计算该值中 1 位的数量，如果计数结果为偶数，则返回 `true`，如果为奇数，则返回 `false`。

位减缩方法示例

```
ap_uint<8> Val = 0xaa;

bool t = Val.and_reduce(); // Yields: false
t = Val.or_reduce();      // Yields: true
t = Val.xor_reduce();     // Yields: false
t = Val.nand_reduce();    // Yields: true
t = Val.nor_reduce();     // Yields: false
t = Val.xnor_reduce();    // Yields: true
```

位反转

```
void ap_(u)int::reverse ()
```

将 `ap_[u]int` 实例的内容反转：

- LSB 变为 MSB。
- MSB 变为 LSB。

反转方法示例

```
ap_uint<8> Val = 0x12;
Val.reverse(); // Yields: 0x48
```

测试位值

```
bool ap_(u)int::test (unsigned i)
```

检查 `ap_(u)int` 实例的指定位是否是 1。

如果是，则返回 `true`，如果否，则返回 `false`。

测试方法示例

```
ap_uint<8> Val = 0x12;
bool t = Val.test(5); // Yields: true
```

设置位值

```
void ap_(u)int::set (unsigned i, bool v)
void ap_(u)int::set_bit (unsigned i, bool v)
```

将 `ap_(u)int` 实例的指定位设置为整数 `v` 的值。

设置位（设为 1）

```
void ap_(u)int::set (unsigned i)
```

将 `ap_(u)int` 实例的指定位设置为值 1（一）。

清除位（设为 0）

```
void ap_(u)int::clear(unsigned i)
```

将 `ap_(u)int` 实例的指定位设置为值 0（零）。

对位取反

```
void ap_(u)int::invert(unsigned i)
```

对 `ap_(u)int` 实例的函数实参中指定的位取反。指定的位原始值如果为 1，则变为 0，反之亦然。

位设置、清除和取反方法示例：

```
ap_uint<8> Val = 0x12;
Val.set(0, 1); // Yields: 0x13
Val.set_bit(4, false); // Yields: 0x03
Val.set(7); // Yields: 0x83
Val.clear(1); // Yields: 0x81
Val.invert(4); // Yields: 0x91
```


右转

```
void ap_(u)int:: rrotate(unsigned n)
```

将 `ap_(u)int` 实例向右旋转 `n` 位。

左转

```
void ap_(u)int:: lrotate(unsigned n)
```

将 `ap_(u)int` 实例向左旋转 `n` 位。

```
ap_uint<8> Val = 0x12;

Val.rrotate(3); // Yields: 0x42
Val.lrotate(6); // Yields: 0x90
```

按位 NOT

```
void ap_(u)int:: b_not()
```

- 补足 `ap_(u)int` 实例的每个位。

```
ap_uint<8> Val = 0x12;

Val.b_not(); // Yields: 0xED
```

按位 NOT 示例

测试符号

```
bool ap_int:: sign()
```

- 检查 `ap_(u)int` 实例是否为负。
- 如果为负，则返回 `true`。
- 如果为正，则返回 `false`。

显式转换方法**转换为 C/C++ (u)int 型**

```
int ap_(u)int::to_int ()
unsigned ap_(u)int::to_uint ()
```

- 返回 `ap_[u]int` 中包含的值对应的原生 C/C++（大部分系统上为 32 位）整数。
- 如果值大于 `[unsigned] int` 可显示的值，则发生截位。

转换为 C/C++ 64 位 (u)int 型

```
long long ap_(u)int::to_int64 ()
unsigned long long ap_(u)int::to_uint64 ()
```

- 返回 `ap_[u]int` 中包含的值对应的原生 C/C++ 64 位整数。
- 如果值大于 `[unsigned] int` 可显示的值，则发生截位。

转换为 C/C++ double 型

```
double ap_(u)int::to_double ()
```

- 返回 `ap_[u]int` 中包含的值对应的原生 C/C++ double 64 位浮点表示法。
- 如果 `ap_[u]int` 宽度超过 53 位（double 的尾数中的位数），生成的 double 所包含的值可能与期望的值不同。



建议：AMD 建议显式调用成员函数，而不是使用 C 语言样式强制转换来将 `ap_[u]int` 转换为其他数据类型。

Sizeof

标准 C++ `sizeof()` 函数不应搭配 `ap_[u]int` 或者对象的其他类或实例一起使用。`ap_int<>` 数据类型为类，`sizeof` 会返回类或实例对象使用的存储空间。`sizeof(ap_int<N>)` 始终返回所使用的字节数。例如：

```
sizeof(ap_int<127>)=16
sizeof(ap_int<128>)=16
sizeof(ap_int<129>)=24
sizeof(ap_int<130>)=24
```

编译时间访问数据类型属性

`ap_[u]int<>` 类型随静态成员一并提供，此静态成员允许在编译时判定变量大小。数据类型随静态常量成员 `width` 一并提供，数据类型的宽度将自动赋值给该成员：

```
static const int width = _AP_W;
```

您可使用 `width` 数据成员来提取现有 `ap_[u]int<>` 数据类型的数据宽度，以在编译时创建另一个 `ap_[u]int<>` 数据类型。以下示例显示 `Res` 变量的大小定义为比 `Val1` 变量和 `Val2` 变量位宽大 1 位：

```
// Definition of basic data type
#define INPUT_DATA_WIDTH 8
typedef ap_int<INPUT_DATA_WIDTH> data_t;
// Definition of variables
data_t Val1, Val2;
// Res is automatically sized at compile-time to be 1-bit greater than data
type
data_t
ap_int<data_t::width+1> Res = Val1 + Val2;
```

这样可确保 Vitis HLS 对加法导致的位增长进行正确建模，即使为 `data_t` 更新 `INPUT_DATA_WIDTH` 的值也是如此。

C++ 任意精度定点类型

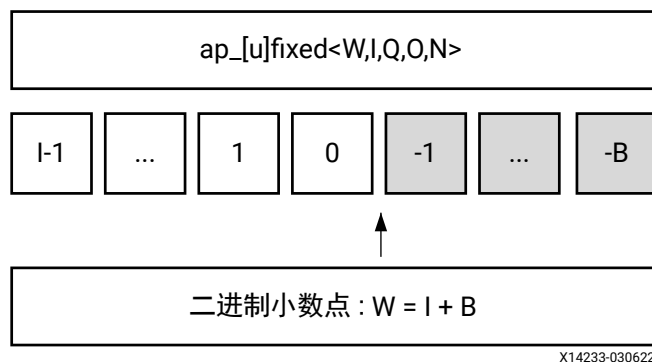


重要提示！ 任意精度定点类型要求代码中包含 `ap_fixed.h` 头文件。

C++ 函数可利用 Vitis HLS 所含的任意精度定点类型。下图汇总了这些定点类型的基本功能：

- 字型可采用有符号 (`ap_fixed`) 或无符号 (`ap_ufixed`) 类型。
- 可定义任意大小 W 的字。
- 小数点 I 前的位数还可定义字的小数位数 $W-I$ （在下图中以 B 来表示）。
- 可选择舍入或量化 (Q) 类型。
- 可选择上溢行为 (O 和 N)。

图 143：任意精度定点类型



提示： 在上图中，整数值 (I) 指定了二进制小数点左侧的整数位数（包括符号位）。

在使用 `ap_[u]fixed` 类型的超大阵列时以及在 C 语言仿真期间，任意精度定点类型会占用更多存储器。

使用定点类型的优势包括：

- 支持轻松显示小数。
- 当变量包含的整数位数和小数位数不同时，可处理小数点的对齐。
- 当小数位数太少无法准确表示结果精度时，有多个选项可用于处理舍入方式。
- 当结果大于可显示的整数位数时，有多个选项可用于处理变量上溢方式。

请查看如下示例中的代码，其中汇总了各项属性。首先，包含 `ap_fixed.h` 头文件。随后使用 `typedef` 语句定义 `ap_fixed` 类型：

- 10 位输入：8 位整数值和 2 个小数位。
- 6 位输入：3 位整数值和 3 个小数位。
- 用于累加的 22 位变量：17 位整数值和 5 个小数位。
- 用于表示结果的 36 位变量：30 位整数值和 6 个小数位。

函数不含用于管理执行运算后的小数点对齐的代码。对齐自动执行。

以下代码样本显示的类型为 `ap_fixed`。

```
#include "ap_fixed.h"

typedef ap_ufixed<10,8, AP_RND, AP_SAT> din1_t;
typedef ap_fixed<6,3, AP_RND, AP_WRAP> din2_t;
typedef ap_fixed<22,17, AP_TRN, AP_SAT> dint_t;
typedef ap_fixed<36,30> dout_t;

dout_t cpp_ap_fixed(din1_t d_in1, din2_t d_in2) {

    static dint_t sum;
    sum += d_in1;
    return sum * d_in2;
}
```

通过使用 `ap_(u)fixed` 类型，C++ 仿真可保证位精度。快速仿真可确认算法及其准确性。综合后，RTL 会显示完全相同的位精确行为。

在此代码中可为任意精度定点类型自由赋予字面值。如以上示例中使用的测试平台中所示（请参阅以下示例），其中已声明 `in1` 和 `in2` 的值，并对其赋予常量值。

赋予含运算符的字面值时，字面值必须首先强制转换为 `ap_(u)fixed` 类型。否则，C 语言编译器和 Vitis HLS 会将字面值解释为整数或 `float/double` 类型，并且可能无法找到合适的运算符。如以下示例所示，在 `in1 = in1 + din1_t(0.25)` 的赋值中，字面值 `0.25` 强制转换为 `ap_fixed` 类型。

```
#include <cmath>
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;
#include "ap_fixed.h"

typedef ap_ufixed<10,8, AP_RND, AP_SAT> din1_t;
typedef ap_fixed<6,3, AP_RND, AP_WRAP> din2_t;
typedef ap_fixed<22,17, AP_TRN, AP_SAT> dint_t;
typedef ap_fixed<36,30> dout_t;

dout_t cpp_ap_fixed(din1_t d_in1, din2_t d_in2);
int main()
{
    ofstream result;
    din1_t in1 = 0.25;
    din2_t in2 = 2.125;
    dout_t output;
    int retval=0;

    result.open(result.dat);
    // Persistent manipulators
    result << right << fixed << setbase(10) << setprecision(15);

    for (int i = 0; i <= 250; i++)
    {
        output = cpp_ap_fixed(in1,in2);

        result << setw(10) << i;
        result << setw(20) << in1;
        result << setw(20) << in2;
        result << setw(20) << output;
    }
```

```
result << endl;

in1 = in1 + din1_t(0.25);
in2 = in2 - din2_t(0.125);
}
result.close();

// Compare the results file with the golden results
retval = system(diff --brief -w result.dat result.golden.dat);
if (retval != 0) {
printf(Test failed !!!\n);
retval=1;
} else {
printf(Test passed !\n);
}

// Return 0 if the test passes
return retval;
}
```

定点标识符汇总

下表显示了量化和上溢模式。



提示：不超过标准硬件算术（卷绕和截位）的默认行为的量化和上溢模式可导致运算符含更多关联硬件。要实现更高级的模式需耗用逻辑 (LUT)，例如，舍入到负无穷或对称饱和。

表 52：定点标识符汇总

标识符	描述	
W	字宽（位）	
I	用于表示整数值的位数，即，二进制小数点左侧的整数位数，包括符号位。 当 I 为负值（如以下示例所示）时，它表示隐式有符号位（对应符号表示法）的数量，或者小数点右侧的隐式零位（对应无符号表示法）的数量。例如： ap_fixed<2, 0> a = -0.5; // a can be -0.5, ap_ufixed<1, 0> x = 0.5; // 1-bit representation. x can be 0 or 0.5 ap_ufixed<1, -1> y = 0.25; // 1-bit representation. y can be 0 or 0.25 const ap_fixed<1, -7> z = 1.0/256; // 1-bit representation for z = 2^-8	
Q	量化模式：用于规定当生成的精度大于用于存储结果的变量中的最小小数位可定义的精度时的行为。	
	ap_fixed 类型	描述
	AP_RND	舍入到正无穷
	AP_RND_ZERO	舍入到 0
	AP_RND_MIN_INF	舍入到负无穷
	AP_RND_INF	舍入到无穷
	AP_RND_CONV	收敛舍入
	AP_TRN	截位到负无穷（默认）
	AP_TRN_ZERO	截位到 0

表 52：定点标识符汇总 (续)

标识符	描述	
O	上溢模式：用于规定当运算结果超出用于存储结果的变量可存储的最大值（或者对于负值为最小值）时的行为。	
	ap_fixed 类型	描述
	AP_SAT ¹	饱和
	AP_SAT_ZERO ¹	饱和到 0
	AP_SAT_SYM ¹	对称饱和
	AP_WRAP	卷绕（默认）
	AP_WRAP_SM	符号量值卷绕
N	定义上溢卷绕模式中的饱和位数。	

注释：

1. 使用 AP_SAT* 模式可以提升资源使用率，因为需要额外逻辑来执行饱和，此额外成本可能额外增加高达 20% 的 LUT 使用率。
2. 来自 hls_math 库的定点数学函数不支持 ap_[u]fixed 模板参数 Q、O 和 N，这三个参数依次分别对应于量化模式、上溢模式和饱和位数。仅当 ap_[u]fixed 变量位于赋值左侧时或者在初始化期间，量化模式和上溢模式才有效，但在变量计算期间无效。

C++ 任意精度定点类型：参考信息



重要提示！ 任意精度定点类型要求代码中包含 ap_fixed.h 头文件。

本章节包含有关使用 ap_(u)fixed<N> 任意精度定点数据类型的方法、综合行为和各方面的全面信息。

- 用于将常量和初始化值赋值给任意精度整数（包括大于 1024 位的值）的方法。
- 上溢和饱和模式的详细描述。
- 描述 Vitis HLS 帮助程序方法，包括打印、串联、位切片 (bit-slicing) 和范围选择函数。
- 描述运算符行为，包括移位操作（即导致反向移位的负移位值）的描述。

Vitis HLS 支持定点类型，定点类型支持轻松处理小数运算。定点运算的优势如下示例所示。

```
ap_fixed<11, 6> Var1 = 22.96875; // 11-bit signed word, 5 fractional bits
ap_ufixed<12,11> Var2 = 512.5; // 12-bit word, 1 fractional bit
ap_fixed<16,11> Res1; // 16-bit signed word, 5 fractional bits

Res1 = Var1 + Var2; // Result is 535.46875
```

即使 Var1 与 Var2 精度不同，但定点类型在执行运算前（此处为加法）仍可确保小数点正确对齐。您无需在 C 语言代码中执行任何操作来使小数点对齐。

用于存储任意定点运算结果的类型大小（包括整数位和小数位）必须足以存储完整结果。

否则，ap_fixed 类型会执行以下操作：

- 使处理上溢（前提是结果所含 MSB 多于指定类型支持的数量）
- 量化（或舍入，前提是结果所含 LSB 少于指定类型支持的数量）

ap_[u]fixed 类型可提供有关上溢和量化执行方式的多种不同选项。这些选项如下所述。

ap_[u]fixed 表示法

在 `ap_[u]fixed` 类型中，定点值以位序列的形式来表示，具有对应二进制小数点的指定位置。

- 二进制小数点左侧的位表示值的整数部分，包括符号位。
- 二进制小数点右侧的位表示值的小数部分。

`ap_[u]fixed` type 定义如下：

```
ap_[u]fixed<int W,
  int I,
  ap_q_mode Q,
  ap_o_mode O,
  ap_sat_bits N>;
```

量化模式

舍入到正无穷	AP_RND
舍入到 0	AP_RND_ZERO
舍入到负无穷	AP_RND_MIN_INF
舍入到无穷	AP_RND_INF
收敛舍入	AP_RND_CONV
截位	AP_TRN
截位到 0	AP_TRN_ZERO

AP_RND

- 将值舍入到特定 `ap_[u]fixed` 类型最接近的可表示值。

```
ap_fixed<3, 2, AP_RND, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.5
ap_fixed<3, 2, AP_RND, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.0
```

AP_RND_ZERO

- 将值舍入到最接近的可表示值。
- 向 0 舍入。
 - 对于正值，删除冗余位。
 - 对于负值，添加最低有效位以获取最接近的可表示值。

```
ap_fixed<3, 2, AP_RND_ZERO, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.0
ap_fixed<3, 2, AP_RND_ZERO, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.0
```

AP_RND_MIN_INF

- 将值舍入到最接近的可表示值。
- 向负无穷舍入。
 - 对于正值，删除冗余位。

- 对于负值，添加最低有效位。

```
ap_fixed<3, 2, AP_RND_MIN_INF, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.0
ap_fixed<3, 2, AP_RND_MIN_INF, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.5
```

AP_RND_INF

- 将值舍入到最接近的可表示值。
- 舍入方式取决于最低有效位。
 - 对于正值，如果已设置最低有效位，则向正无穷舍入。否则，向负无穷舍入。
 - 对于负值，如果已设置最低有效位，则向负无穷舍入。否则，向正无穷舍入。

```
ap_fixed<3, 2, AP_RND_INF, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.5
ap_fixed<3, 2, AP_RND_INF, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.5
```

AP_RND_CONV

- 舍入到最近的可表示值，并将舍入“绑定”到偶数，即舍入后的最低有效位强制为零(0)。
- “绑定”是两个可表示值的中点，当紧随舍入后的最低有效位之后的位为 1 并且其下方所有位全部为 0 时，即发生“绑定”。

```
// For the following examples, bit3 of the 8-bit value becomes the
// LSB of the final 5-bit value (after rounding).
// Notes:
// * bit7 of the 8-bit value is the MSB (sign bit)
// * the 3 LSBs of the 8-bit value (bit2, bit1, bit0) are treated as
//   guard, round and sticky bits.
// * See http://pages.cs.wisc.edu/~david/courses/cs552/S12/handouts/
//   guardbits.pdf

ap_fixed<8,3> p1 = 1.59375; // p1 = 001.10011
ap_fixed<5,3,AP_RND_CONV> rconv1 = p1; // rconv1 = 1.5 (001.10)

ap_fixed<8,3> p2 = 1.625; // p2 = 001.10100 => tie with bit3 (LSB-to-be)
= 0
ap_fixed<5,3,AP_RND_CONV> rconv2 = p2; // rconv2 = 1.5 (001.10) => lsb is
already zero, just truncate

ap_fixed<8,3> p3 = 1.375; // p3 = 001.01100 => tie with bit3 (LSB-to-be)
= 1
ap_fixed<5,3,AP_RND_CONV> rconv3 = p3; // rconv3 = 1.5 (001.10) => lsb is
made zero by rounding up

ap_fixed<8,3> p3 = 1.65625; // p3 = 001.10101
ap_fixed<5,3,AP_RND_CONV> rconv3 = p3; // rconv3 = 1.75 (001.11) => round
up
```

AP_TRN

- 始终将值向负无穷舍入。

```
ap_fixed<3, 2, AP_TRN, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.0
ap_fixed<3, 2, AP_TRN, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.5
```


AP_TRN_ZERO

将值舍入为：

- 对于正值，舍入方式与 AP_TRN 模式相同。
- 对于负值，向 0 舍入。

```
ap_fixed<3, 2, AP_TRN_ZERO, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.0
ap_fixed<3, 2, AP_TRN_ZERO, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.0
```

上溢模式

饱和

AP_SAT

饱和到 0

AP_SAT_ZERO

对称饱和

AP_SAT_SYM

卷绕

AP_WRAP

符号量值卷绕

AP_WRAP_SM

AP_SAT

使值达到饱和。

- 最大值，以防上溢。
- 设为负最大值，以防负上溢。

```
ap_fixed<4, 4, AP_RND, AP_SAT> UAPFixed4 = 19.0; // Yields: 7.0
ap_fixed<4, 4, AP_RND, AP_SAT> UAPFixed4 = -19.0; // Yields: -8.0
ap_ufixed<4, 4, AP_RND, AP_SAT> UAPFixed4 = 19.0; // Yields: 15.0
ap_ufixed<4, 4, AP_RND, AP_SAT> UAPFixed4 = -19.0; // Yields: 0.0
```

AP_SAT_ZERO

强制值为 0，以防上溢或负上溢。

```
ap_fixed<4, 4, AP_RND, AP_SAT_ZERO> UAPFixed4 = 19.0; // Yields: 0.0
ap_fixed<4, 4, AP_RND, AP_SAT_ZERO> UAPFixed4 = -19.0; // Yields: 0.0
ap_ufixed<4, 4, AP_RND, AP_SAT_ZERO> UAPFixed4 = 19.0; // Yields: 0.0
ap_ufixed<4, 4, AP_RND, AP_SAT_ZERO> UAPFixed4 = -19.0; // Yields: 0.0
```

AP_SAT_SYM

使值饱和为：

- 最大值，以防上溢。
- 最小值，以防负上溢。
 - 针对有符号的 ap_fixed 类型，设为负最大值
 - 针对无符号的 ap_ufixed 类型，设为 0

```
ap_fixed<4, 4, AP_RND, AP_SAT_SYM> UAPFixed4 = 19.0; // Yields: 7.0
ap_fixed<4, 4, AP_RND, AP_SAT_SYM> UAPFixed4 = -19.0; // Yields: -7.0
ap_ufixed<4, 4, AP_RND, AP_SAT_SYM> UAPFixed4 = 19.0; // Yields: 15.0
ap_ufixed<4, 4, AP_RND, AP_SAT_SYM> UAPFixed4 = -19.0; // Yields: 0.0
```

AP_WRAP

将值卷绕，以防上溢。

```

ap_fixed<4, 4, AP_RND, AP_WRAP> UAPFixed4 = 31.0; // Yields: -1.0
ap_fixed<4, 4, AP_RND, AP_WRAP> UAPFixed4 = -19.0; // Yields: -3.0
ap_ufixed<4, 4, AP_RND, AP_WRAP> UAPFixed4 = 19.0; // Yields: 3.0
ap_ufixed<4, 4, AP_RND, AP_WRAP> UAPFixed4 = -19.0; // Yields: 13.0

```

如果 N 的值设为 0（默认上溢模式）：

- 删除超出范围的所有 MSB 位。
- 针对无符号的数值：达到最大值后，卷绕为 0。
- 针对有符号的数值：达到最大值后，卷绕至最小值。

如果 N>0：

- 当 N > 0 时，N 个 MSB 位饱和为 1 或设为 1。
- 保留符号位，因此正数仍保留为正数，负数保留为负数。
- 复制不饱和的位，从 LSB 侧开始。

AP_WRAP_SM

应以符号量值卷绕该值。

```

ap_fixed<4, 4, AP_RND, AP_WRAP_SM> UAPFixed4 = 19.0; // Yields: -4.0
ap_fixed<4, 4, AP_RND, AP_WRAP_SM> UAPFixed4 = -19.0; // Yields: 2.0

```

如果 N 的值设为 0（默认上溢模式）：

- 此模式使用符号量值卷绕。
- 符号位设置为已删除的最低有效位的值。
- 如果剩余最高有效位不同于原 MSB，那么所有剩余位全部取反。
- 如果 MSB 相同，那么将复制其他位。
 1. 删除冗余 MSB。
 2. 新符号位是已删除位的最低有效位。在此情况下即为 0。
 3. 将新的符号位与新的值的符号进行比较。
- 如果不同，则对所有数值取反。在此情况下两者不同。

如果 N>0：

- 使用符号量级饱和
- N 个 MSB 饱和为 1。
- 与 N = 0 情况下的行为类似，但正数仍保留为正数，负数保留为负数。

编译 ap_[u]fixed<> 类型

要使用 ap_[u]fixed<> 类，必须将 ap_fixed.h 头文件包含在引用 ap_[u]fixed<> variables 的所有源文件中。

编译使用这些类的软件模型时，可能需要指定 Vitis HLS 头文件的位置，例如，为 g++ 编译添加 “-I/<HLS_HOME>/include” 选项。

声明和定义 ap_[u]fixed<> 变量

有符号的类与无符号的类不同：

- ap_fixed<W,I> (有符号)
- ap_ufixed<W,I> (无符号)

您可使用 C/C++ typedef 语句创建用户定义的类型：

```
#include "ap_fixed.h" // use ap_[u]fixed<> types

typedef ap_ufixed<128,32> uint128_t; // 128-bit user defined type,
// 32 integer bits
```

从常量初始化和赋值（字面值）

您可使用常用 C/C++ 宽度的正常浮点常量来初始化 ap_[u]fixed 变量：

- 32 位 (针对 float 类型)
- 64 位 (针对 double 类型)

即通常为单精度类型或双精度形式的浮点值。请注意分配到定点变量的值将受到常量精度的限制。

```
#include <ap_fixed.h>

ap_ufixed<30, 15> my15BitInt = 3.1415;
ap_fixed<42, 23> my42BitInt = -1158.987;
ap_ufixed<99, 40> = 287432.0382911;
ap_fixed<36, 30> = -0x123.456p-1;
```

您也可以使用字符串初始化以确保根据字符串所描述的精度来填充定点值的所有位。

```
ap_ufixed<2, 0> x      = "0b0.01";           // 0.25 in “dot” format
ap_ufixed<2, 0> y      = "0b01p-2";          // 0.25 in binary “scientific”
format
ap_ufixed<2, 0> z      = "0x4p-4";           // 0.25 in hex “scientific” format
ap_ufixed<62, 2> my_pi =
“0b11.001001000011111101101010100010001000010110100011000010001101”; // pi
with 60 fractional bits
```

在 std::complex 类型的阵列中使用 ap_[u]fixed 类型时，后者不支持初始化。

```
typedef ap_fixed<DIN_W, 1, AP_TRN, AP_SAT> coeff_t; // MUST have IW >= 1
std::complex<coeff_t> twid_rom[REAL_SZ/2] = {{ 1, -0 }, { 0.9, -0.006 }, etc.}
```

初始化值必须首先强制转换为 std::complex 类型：

```
typedef ap_fixed<DIN_W, 1, AP_TRN, AP_SAT> coeff_t; // MUST have IW >= 1
std::complex<coeff_t> twid_rom[REAL_SZ/2] = {std::complex<coeff_t>( 1,
-0 ),
std::complex<coeff_t>(0.9, -0.006 ), etc.}
```

控制台 I/O 支持（打印）

正如 `ap_[u]fixed<>` 变量初始化和赋值一样，Vitis HLS 支持打印显示超 64 位的值。

输出存储在 `ap_[u]fixed` 变量中的任意值的最简单方法是使用 C++ 标准输出串流：`std::cout` (`#include <iostream>` or `<iostream.h>`)。串流插入运算符 `<<` 已重载，可正确输出任意给定 `ap_[u]fixed` 变量的全部可能值。以下串流操控程序同样受支持，以便按所示方式对该值进行格式化。

- `dec`（十进制）
- `hex`（十六进制）
- `oct`（八进制）

```
#include <iostream.h>
// Alternative: #include <iostream>

ap_fixed<6,3, AP_RND, AP_WRAP> Val = 3.25;

cout << Val << endl;      // Yields: 3.25
```

使用标准 C 语言库

您还可使用标准 C 语言库 (`#include <stdio.h>`) 来打印大于 64 位的值：

1. 使用 `ap_[u]fixed` 类方法 `to_string()` 将值转换为 C++ `std::string`。
2. 使用 `std::string` 类方法 `c_str()` 将结果转换为以 `null` 结尾的 C 语言字符串。

可选实参 1（指定基数）

您可为 `ap[u]int::to_string()` 方法传递 1 个可选实参以指定期望的数字格式的基数。有效的基数实参值包括：

- 2（二进制）
- 8（八进制）
- 10（十进制）
- 16（十六进制，默认值）

可选实参 2（打印为有符号的值）

`ap_[u]int::to_string()` 的第 2 个可选实参可指定是否将非十进制格式打印为有符号的值。该实参为布尔值。默认值为 `false`，导致非十进制格式打印为无符号的值。

```
ap_fixed<6,3, AP_RND, AP_WRAP> Val = 3.25;

printf("%s \n", in2.to_string().c_str()); // Yields: 0b011.010
printf("%s \n", in2.to_string(10).c_str()); //Yields: 3.25
```

以下 C++ 操控程序函数支持 `ap_[u]fixed` 类型：

- `setprecision`
- `setw`
- `setfill`

setprecision 操控程序用于设置要使用的小数位精度。需 1 个参数 *f* 作为小数位精度的值，其中 *n* 用于指定要显示的有意义的最大位数（计算小数点前后的总位数）。

f 默认值为 6，与原生 C 浮点类型一致。

```
ap_fixed<64, 32> f = 3.14159;
cout << setprecision (5) << f << endl;
cout << setprecision (9) << f << endl;
f = 123456;
cout << setprecision (5) << f << endl;
```

以上示例显示了以下结果，其中当实际精度超过指定精度时，将舍入打印的结果：

```
3.1416
3.14159
1.2346e+05
```

setw 操控程序：

- 用于设置要用作为字段宽度的字符数。
- 需 1 个参数 *w* 作为宽度值

其中

- *w* 用于判定在某些输出表示法中将写入的最小字符数。

如果该表示法的标准宽度小于字段宽度，那么将以填充字符来填充该表示法。填充字符由 setfill 操控程序来控制，需 1 个参数 *f* 作为填充字符。

例如，给定：

```
ap_fixed<65,32> aa = 123456;
int precision = 5;
cout<<setprecision(precision)<<setw(13)<<setfill('T')<<a<<endl;
```

输出为：

```
TTT1.2346e+05
```

涉及 ap_[u]fixed<> 类型的表达式

任意精度定点值可用于使用受 C/C++ 支持的任意运算符的表达式。定义任意精度定点类型或变量后，其使用方式与 C/C++ 语言中的任意浮点类型或变量相同。

请注意以下附加说明：

- 0 值和符号扩展

根据源值符号，较小位宽的所有值均为 0 值或符号扩展值。您可能需要插入类型强制转换以在将较小位宽的值赋值到较大位宽的值时获取替代符号。

- 截位

如果赋值的任意精度定点的位宽大于目标变量，则会发生截位。

类方法、运算符和数据成员

一般情况下，支持针对 `ap_[u]fixed` 类型使用运算符重载来执行在原生 C/C++ 整数数据类型上执行的任意有效操作。除这些重载运算符外，还包含部分特定于类的运算符和方法以简化位级运算。

二进制算术运算符

加法

```
ap_[u]fixed::RType ap_[u]fixed::operator + (ap_[u]fixed op)
```

以任意精度定点加上给定操作数 `op`。

该操作数可采用以下任一整数类型：

- `ap_[u]fixed`
- `ap_[u]int`
- `C/C++`

返回类型 `ap_[u]fixed::RType` 取决于 2 个操作数的类型信息。

```
ap_fixed<76, 63> Result;

ap_fixed<5, 2> Val1 = 1.125;
ap_fixed<75, 62> Val2 = 6721.35595703125;

Result = Val1 + Val2; //Yields 6722.480957
```

由于 `Val2` 的整数部分和小数部分位宽均较大，因此返回类型采用相同位宽加 1 以便存储所有可能的结果值。

指定数据宽度可通过使用幂函数来控制资源，如下所示。在此类情况下，AMD 建议指定存储的结果的宽度，而不是指定定点运算的宽度。

```
ap_ufixed<16,6> x=5;
ap_ufixed<16,7>y=hl::rsqrt<16,6>(x+x);
```

减法

```
ap_[u]fixed::RType ap_[u]fixed::operator - (ap_[u]fixed op)
```

以任意精度定点减去给定操作数 `op`。

返回类型 `ap_[u]fixed::RType` 取决于 2 个操作数的类型信息。

```
ap_fixed<76, 63> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val2 - Val1; // Yields 6720.23057
```

由于 `Val2` 的整数部分和小数部分位宽均较大，因此返回类型采用相同位宽加 1 以便存储所有可能的结果值。

乘法

```
ap_[u]fixed::RType ap_[u]fixed::operator * (ap_[u]fixed op)
```

以任意精度定点乘以给定操作数 `op`。

```
ap_fixed<80, 64> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val1 * Val2; // Yields 7561.525452
```

这将显示 `Val1` 与 `Val2` 的乘法。返回类型为其整数部分位宽与其小数部分位宽之和。

除法

```
ap_[u]fixed::RType ap_[u]fixed::operator / (ap_[u]fixed op)
```

以任意精度定点除以给定操作数 `op`。

```
ap_fixed<84, 66> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Val2 / Val1; // Yields 5974.538628
```

这将显示 `Val2` 与 `Val1` 的除法。为保留足够精度：

- 返回类型的整数位宽为 `Val2` 的整数位宽与 `Val1` 的小数位宽之和。
- 返回类型的小数位宽等于 `Val2` 的小数位宽。

按位逻辑运算符

按位 OR

```
ap_[u]fixed::RType ap_[u]fixed::operator | (ap_[u]fixed op)
```

对任意精度定点和给定操作数 `op` 应用按位运算。

```
ap_fixed<75, 62> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val1 | Val2; // Yields 6271.480957
```

按位 AND

```
ap_[u]fixed::RType ap_[u]fixed::operator & (ap_[u]fixed op)
```

对任意精度定点和给定操作数 `op` 应用按位运算。

```
ap_fixed<75, 62> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val1 & Val2; // Yields 1.00000
```

按位 XOR

```
ap_[u]fixed::RType ap_[u]fixed::operator ^ (ap_[u]fixed op)
```

对任意精度定点和给定操作数 `op` 应用 `xor` 按位运算。

```
ap_fixed<75, 62> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val1 ^ Val2; // Yields 6720.480957
```

增量运算符和减量运算符

递增前

```
ap_[u]fixed ap_[u]fixed::operator ++ ()
```

此运算符函数前缀会将任意精度定点变量加 1。

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = ++Val1; // Yields 6.125000
```

递增后

```
ap_[u]fixed ap_[u]fixed::operator ++ (int)
```

此运算符函数后缀：

- 会将任意精度定点变量加 1。
- 返回此任意精度定点的原始 `val`。

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = Val1++; // Yields 5.125000
```

递减前

```
ap_[u]fixed ap_[u]fixed::operator -- ()
```


此运算符函数前缀会将任意精度定点变量减 1。

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = --Val1; // Yields 4.125000
```

递减后

```
ap_[u]fixed ap_[u]fixed::operator -- (int)
```

此运算符函数后缀：

- 会将任意精度定点变量减 1。
- 返回此任意精度定点的原始 val。

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = Val1--; // Yields 5.125000
```

一元运算符

加法

```
ap_[u]fixed ap_[u]fixed::operator + ()
```

返回任意精度定点变量的自拷贝。

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = +Val1; // Yields 5.125000
```

减法

```
ap_[u]fixed::RType ap_[u]fixed::operator - ()
```

返回任意精度定点变量的负值。

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = -Val1; // Yields -5.125000
```

等于 0

```
bool ap_[u]fixed::operator ! ()
```

此运算符函数：

- 将任意精度定点变量与 0 比较，

- 返回结果。

```
bool Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = !Val1; // Yields false
```

按位反转

```
ap_[u]fixed::RType ap_[u]fixed::operator ~ ()
```

返回任意精度定点变量的按位补码。

```
ap_fixed<25, 15> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = ~Val1; // Yields -5.25
```

移位运算符

无符号的左移

```
ap_[u]fixed ap_[u]fixed::operator << (ap_uint<_W2> op)
```

此运算符函数：

- 按给定整数操作数左移。
- 返回结果。

此操作数可采用以下 C/C++ 整数类型：

- char
- short
- int
- long

左移运算的返回类型与移位的类型的宽度相同。

注释：移位不支持上溢或量化模式。

```
ap_fixed<25, 15> Result;
ap_fixed<8, 5> Val = 5.375;

ap_uint<4> sh = 2;

Result = Val << sh; // Yields -10.5
```

结果的位宽为 (W = 25, I = 15)。由于左移运算返回类型与 Val 类型相同：

- Val 的高阶 2 位将移出。
- 结果为 -10.5。

如果需结果为 21.5，Val 必须首先强制转换为 ap_fixed<10, 7> - 例如，ap_ufixed<10, 7>(Val)。

有符号的左移

```
ap_[u]fixed ap_[u]fixed::operator << (ap_int<_W2> op)
```

此运算符：

- 按给定整数操作数左移。
- 返回结果。

移位方向取决于操作数为正还是为负。

- 如果操作数为正，则执行右移。
- 如果操作数为负，则执行左移（反方向）。

此操作数可采用以下 C/C++ 整数类型：

- char
- short
- int
- long

右移运算的返回类型与移位的类型的宽度相同。

```
ap_fixed<25, 15, false> Result;  
ap_uint<8, 5> Val = 5.375;  
  
ap_int<4> Sh = 2;  
Result = Val << sh; // Shift left, yields -10.25  
  
Sh = -2;  
Result = Val << sh; // Shift right, yields 1.25
```

无符号的右移

```
ap_[u]fixed ap_[u]fixed::operator >> (ap_uint<_W2> op)
```

此运算符函数：

- 按给定整数操作数右移。
- 返回结果。

此操作数可采用以下 C/C++ 整数类型：

- char
- short
- int
- long

右移运算的返回类型与移位的类型的宽度相同。

```
ap_fixed<25, 15> Result;
ap_fixed<8, 5> Val = 5.375;

ap_uint<4> sh = 2;

Result = Val >> sh; // Yields 1.25
```

如需保留所有有效位，请首先扩展 Val 的小数部分的位宽，例如，`ap_fixed<10, 5>(Val)`。

有符号的右移

```
ap_[u]fixed ap_[u]fixed::operator >> (ap_int<_W2> op)
```

此运算符：

- 按给定整数操作数右移。
- 返回结果。

移位方向取决于操作数为正还是为负。

- 如果操作数为正，则执行右移。
- 如果操作数为负，则执行左移（反方向）。

此操作数可采用任一 C/C++ 整数类型（char、short、int 或 long）。

右移运算的返回类型与移位的类型的宽度相同。例如：

```
ap_fixed<25, 15, false> Result;
ap_uint<8, 5> Val = 5.375;

ap_int<4> Sh = 2;
Result = Val >> sh; // Shift right, yields 1.25

Sh = -2;
Result = Val >> sh; // Shift left, yields -10.5

1.25
```

关系运算符

等于

```
bool ap_[u]fixed::operator == (ap_[u]fixed op)
```

此运算符会将任意精度定点变量与给定操作数进行比较。

如果两者相等，则返回 `true`；如果不相等，则返回 `false`。

此 `op` 操作数的类型可以是 `ap_[u]fixed`、`ap_int` 或 C/C++ 整数类型。例如：

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 == Val2; // Yields true
Result = Val1 == Val3; // Yields false
```

不等于

```
bool ap_[u]fixed::operator != (ap_[u]fixed op)
```

此运算符会将此任意精度定点变量与给定操作数进行比较。

如果两者不相等，则返回 `true`；如果相等，则返回 `false`。

`op` 操作数的类型可以是：

- `ap_[u]fixed`
- `ap_int`
- C 或 C++ 整数类型

例如：

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 != Val2; // Yields false
Result = Val1 != Val3; // Yields true
```

大于或等于

```
bool ap_[u]fixed::operator >= (ap_[u]fixed op)
```

此操作数会将变量与给定操作数比较。

如果两者相等或者如果变量大于运算符，则返回 `true`，否则返回 `false`。

此 `op` 操作数的类型可以是 `ap_[u]fixed`、`ap_int` 或 C/C++ 整数类型。

例如：

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 >= Val2; // Yields true
Result = Val1 >= Val3; // Yields false
```

小于或等于

```
bool ap_[u]fixed::operator <= (ap_[u]fixed op)
```

此运算符会将变量与给定操作数比较，如果变量等于或小于操作数，则返回 `true`，否则返回 `false`。

此 `op` 操作数的类型可以是 `ap_[u]fixed`、`ap_int` 或 C/C++ 整数类型。

例如：

```
bool Result;  
  
ap_ufixed<8, 5> Val1 = 1.25;  
ap_fixed<9, 4> Val2 = 17.25;  
ap_fixed<10, 5> Val3 = 3.25;  
  
Result = Val1 <= Val2; // Yields true  
Result = Val1 <= Val3; // Yields true
```

大于

```
bool ap_[u]fixed::operator > (ap_[u]fixed op)
```

此运算符会将变量与给定操作数比较，如果变量大于运算符，则返回 `true`，否则返回 `false`。

此 `op` 操作数的类型可以是 `ap_[u]fixed`、`ap_int` 或 C/C++ 整数类型。

例如：

```
bool Result;  
  
ap_ufixed<8, 5> Val1 = 1.25;  
ap_fixed<9, 4> Val2 = 17.25;  
ap_fixed<10, 5> Val3 = 3.25;  
  
Result = Val1 > Val2; // Yields false  
Result = Val1 > Val3; // Yields false
```

小于

```
bool ap_[u]fixed::operator < (ap_[u]fixed op)
```

此运算符会将变量与给定操作数比较，如果变量小于运算符，则返回 `true`，否则返回 `false`。

此 `op` 操作数的类型可以是 `ap_[u]fixed`、`ap_int` 或 C/C++ 整数类型。例如：

```
bool Result;  
  
ap_ufixed<8, 5> Val1 = 1.25;  
ap_fixed<9, 4> Val2 = 17.25;  
ap_fixed<10, 5> Val3 = 3.25;  
  
Result = Val1 < Val2; // Yields false  
Result = Val1 < Val3; // Yields true
```

位运算符

位选择和位设置

```
af_bit_ref ap_[u]fixed::operator [] (int bit)
```

此运算符从任意精度定点值中选择 1 个位，并将其返回。

返回的值为可用于设置或清除 `ap_[u]fixed` 变量中的对应位的参考值。位实参必须为整数值，它用于指定要选择的位的索引。最低有效位索引为 0。允许的最高索引是 `ap_[u]fixed` 变量的位宽减 1。

结果类型为 `af_bit_ref`，值为 0 或 1。例如：

```
ap_int<8, 5> Value = 1.375;

Value[3]; // Yields 1
Value[4]; // Yields 0

Value[2] = 1; // Yields 1.875
Value[3] = 0; // Yields 0.875
```

位范围

```
af_range_ref af_(u)fixed::range (unsigned Hi, unsigned Lo)
af_range_ref af_(u)fixed::operator [] (unsigned Hi, unsigned Lo)
```

此运算类似于位选择运算符 `[]`，但它对某一范围内的位而不是对单个位进行运算。

它可从任意精度定点变量中选择一组位。`Hi` 实参可提供要选择的位范围的上半部分。`Lo` 实参可提供要选择的最低有效位。如果 `Lo` 大于 `Hi`，那么所选的位将按逆序返回。

返回类型 `af_range_ref` 表示引用 `Hi` 和 `Lo` 所指定的 `ap_[u]fixed` 变量范围。例如：

```
ap_uint<4> Result = 0;
ap_ufixed<4, 2> Value = 1.25;
ap_uint<8> Repl = 0xAA;

Result = Value.range(3, 0); // Yields: 0x5
Value(3, 0) = Repl(3, 0); // Yields: -1.5

// when Lo > Hi, return the reverse bits string
Result = Value.range(0, 3); // Yields: 0xA
```

范围选择

```
af_range_ref af_(u)fixed::range ()
af_range_ref af_(u)fixed::operator []
```

此运算符是范围选择运算符 `[]` 的特例。它会按正常顺序选择该任意精度定点值中的所有位。

返回类型 `af_range_ref` 表示对 $Hi = W - 1$ 和 $Lo = 0$ 所指定的范围的引用。例如：

```
ap_uint<4> Result = 0;

ap_ufixed<4, 2> Value = 1.25;
ap_uint<8> Repl = 0xAA;

Result = Value.range(); // Yields: 0x5
Value() = Repl(3, 0); // Yields: -1.5
```

长度

```
int ap_[u]fixed::length ()
```

此函数返回的整数值可提供任意精度定点值内的位数。可配合类型或值一起使用。例如：

```
ap_ufixed<128, 64> My128APFixed;

int bitwidth = My128APFixed.length(); // Yields 128
```

显式转换方法

定点到双精度

```
double ap_[u]fixed::to_double ()
```

此成员函数以 IEEE 双精度格式返回此定点值。例如：

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
double Result;

Result = MyAPFixed.to_double(); // Yields 333.789
```

定点到浮点

```
float ap_[u]fixed::to_float()
```

此成员函数以 IEEE 浮点格式返回此定点值。例如：

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
float Result;

Result = MyAPFixed.to_float(); // Yields 333.789
```

定点到半精度浮点

```
half ap_[u]fixed::to_half()
```

此成员函数以 HLS 半精度（16 位）浮点精度格式返回此定点值。例如：

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
half Result;

Result = MyAPFixed.to_half(); // Yields 333.789
```


定点到 ap_int

```
ap_int ap_[u]fixed::to_ap_int ()
```

此成员函数将此定点值显式转换为 `ap_int` 以捕获所有整数位（小数位将被截位）。例如：

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
ap_uint<77> Result;

Result = MyAPFixed.to_ap_int(); //Yields 333
```

定点到整数

```
int ap_[u]fixed::to_int ()
unsigned ap_[u]fixed::to_uint ()
ap_slong ap_[u]fixed::to_int64 ()
ap_ulong ap_[u]fixed::to_uint64 ()
```

此成员函数将此定点值显式转换为 C 内置整数类型。例如：

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
unsigned int Result;

Result = MyAPFixed.to_uint(); //Yields 333

unsigned long long Result;
Result = MyAPFixed.to_uint64(); //Yields 333
```



建议：AMD 建议显式调用成员函数，而不是使用 C 语言样式强制转换来将 `ap_[u]fixed` 转换为其他数据类型。

编译时间访问数据类型属性

`ap_[u]fixed<>` 类型将随多个静态成员一起提供，以便在编译时判定数据类型的大小和配置。数据类型将随 `static const` 成员一起提供：`width`、`iwidth`、`qmode` 和 `omode`：

```
static const int width = _AP_W;
static const int iwidth = _AP_I;
static const ap_q_mode qmode = _AP_Q;
static const ap_o_mode omode = _AP_O;
```

您可使用这些数据成员从任何现有 `ap_[u]fixed<>` 数据类型提取以下信息：

- `width`：数据类型宽度。
- `iwidth`：数据类型的整数部分的宽度。
- `qmode`：数据类型的量化模式。
- `omode`：数据类型的上溢模式。

例如，您可使用这些数据成员来抽取现有 `ap_[u]fixed<>` 数据类型的数据宽度，以在编译时创建另一个 `ap_[u]fixed<>` 数据类型。

以下示例显示 `Res` 变量的大小自动定义为比使用相同量化模式的 `Val1` 变量和 `Val2` 变量位宽大 1 位：

```
// Definition of basic data type
#define INPUT_DATA_WIDTH 12
#define IN_INTG_WIDTH 6
#define IN_QMODE AP_RND_ZERO
#define IN_OMODE AP_WRAP
typedef ap_fixed<INPUT_DATA_WIDTH, IN_INTG_WIDTH, IN_QMODE, IN_OMODE>
data_t;
// Definition of variables
data_t Val1, Val2;
// Res is automatically sized at run-time to be 1-bit greater than
INPUT_DATA_WIDTH
// The bit growth in Res will be in the integer bits
ap_int<data_t::width+1, data_t::iwidth+1, data_t::qmode, data_t::omode> Res
= Val1 +
Val2;
```

这样可确保 Vitis HLS 对加法导致的位增长进行正确建模，即使为 `data_t` 更新 `INPUT_DATA_WIDTH`、`IN_INTG_WIDTH` 或量化模式的值也是如此。

HLS Print 函数

`hls::print` 函数类似于 C 语言中的 `printf`，因为它会将一条格式字符串和单个可选 `int` 或 `double` 实参打印至标准输出，并打印至 C 语言仿真、RTL 协同仿真和硬件仿真中的仿真 log 日志。

但它仅限于打印最多一个实参和一组数量有限的数据类型，如下所述。它还能更改流水线的启动时间间隔和时延，因此必须谨慎使用。

`hls::print` 函数可用于：

- 追踪选定变量的值
- 在整个复杂的控制和并发执行（例如，在数据流中）过程中，用于追踪代码块的执行顺序



提示： 它无法用于追踪基本代码块中各语句的调度顺序，因为调度器可能会显著更改此顺序。

在此简单示例中使用：

```
#include "hls_print.h"
...
for (int i=0; i<N; i++) {
#pragma HLS pipeline ii=1
  hls::print("loop %d\n", i);
...
}
```

在 C 语言仿真、软件仿真、RTL 协同仿真和硬件仿真中每次循环迭代时，它都会打印 “i” 值（当前如果目标为硬件实现，则将其忽略）。

请注意：

- `hls::print` 仅在 Verilog RTL 中受支持。
- 受支持的格式说明符仅限：
 - `%d` 对应整型或无符号
 - `%f` 对应浮点或双精度
- 类型为 `long` 和 `long long` 的值以及无符号变体的值都必须强制转换为 `int` 或 `uint`（原因是受 C++ 的实参提升规则所限）。
- 通过添加“观测”点，插入 `hls::print` 可能改变 HLS 所执行的优化。因此可能该 RTL 行为，就像软件中的 `printf` 可能改变二进制的行为一样，但由于 HLS 的性质，改变更为明显。
- 为了尽可能减小上述影响，仅限传递单个 `int` 值或 `double` 值。
- 任一代码块中不同 `hls::print` 函数的执行顺序都可能因优化和调度而发生更改。
- 在 RTL 中，还会打印输出当前仿真时间以简化调试。
- 由此生成的数据量可能极大，因此不应将其用于转储大型阵列。

HLS 数学库

AMD Vitis™ HLS 数学库 (`hls_math.h`) 可覆盖来自 C++ (`cmath`) 语言库的数学函数，并且可在 C 语言仿真与综合内使用。它可为所有函数提供浮点（单精度、双精度和半精度）支持，为大部分函数提供定点支持。`hls_math.h` 中的函数都组合到 `hls` 名称空间内，并且可代替来自标准 C++ 语言数学库 (`cmath`) 的 `std` 名称空间的函数就地使用。



重要提示！ 不支持在 C 语言代码中使用 `hls_math.h` 头文件。

HLS 数学库精度

HLS 数学函数由 `hls_math.h` 库实现为可综合的位近似函数。位近似 HLS 数学库函数无法提供与标准 C 语言函数相同的精度。为了获得期望的结果，位近似的实现可能使用与标准 C 语言数学库版本不同的底层算法。该函数的精度以 ULP（最小精度单位）的形式指定。精度上的差异对 C 语言仿真和 C/RTL 协同仿真都有影响。

ULP 差异通常在 1-4 ULP 范围内。

- 如果在 C 语言源代码中使用标准 C 语言数学库，则由于某些函数与标准 C 语言数学库存在 ULP 差异，因此 C 语言仿真与 C/RTL 协同仿真之间可能会有差异。
- 如果在 C 语言源代码中使用 HLS 数学库，则 C 语言仿真与 C/RTL 协同仿真之间不会有任何差异。使用 HLS 数学库的 C 语言仿真可能与使用标准 C 语言数学库的 C 语言仿真存在差异。

此外，以下 7 个函数可能会显示出一些差异，具体取决于用于编译和运行 C 语言仿真的 C 语言标准：

- `copysign`
- `fpclassify`
- `isinf`
- `isfinite`
- `isnan`
- `isnormal`
- `signbit`

C90 模式

系统头文件通常仅提供 `isinf`、`isnan` 和 `copysign`，并且这些函数仅适用于双精度类型。特别是，`copysign` 始终返回双精度结果。如果必须返回浮点类型，则在综合后可能会导致意外结果，这是因为在硬件中引入了双精度至浮点数转换块。

C99 模式 (-std=c99)

在此模式下通常系统头文件会提供全部 7 个函数，并且期望系统头文件会将其重定向到 `__isnan(double)` 和 `__isnan(float)`。这些头文件不会重定向 `isnormal`，而是根据 `fpclassify` 来实现。

使用 math.h 的 C++

系统头文件会提供全部 7 个函数，并且这些函数都适用于双精度类型。

`copysign` 始终返回双精度结果。如果必须返回浮点类型，则在综合后可能会导致意外结果，这是因为在硬件中引入了双精度至浮点数转换块。

使用 cmath 的 C++

与 `C99 mode(-std=c99)` 类似，以下情况除外：

- 系统头文件通常不同。
- 针对以下类型正确重载这些函数：

- `float().snan(double)`
- `isinf(double)`

`copysign` 和 `copysignf` 作为内置函数来处理，即使使用 `namespace std;` 时也是如此。

使用 cmath 和名称空间 std 的 C++

没有任何问题。AMD 建议使用以下方法以获得最佳效果：

- 对于 C，使用 `-std=c99`
- 对于 C 和 C++，使用 `-fno-builtin`

注释：要指定 `-std=c99` 等 C 语言编译选项，请使用 Tcl 命令 `add_files` 搭配 `-cflags` 选项。还可以在“Project Settings”（工程设置）对话框中使用“Edit CFLAGS”按钮。

HLS 数学库

在 HLS 数学库中提供了以下函数。其中每个函数都支持半精度（`half` 类型）、单精度（`float` 类型）和双精度（`double` 类型）。



重要提示！对于下列每个 `func` 函数，库中还提供了 1 个关联的仅限半精度函数（名为 `half_func`）和 1 个仅限单精度函数（名为 `funcf`）。

混用半精度、单精度和双精度数据类型时，请检查常见综合错误，防止在最终 FPGA 实现中引入类型转换硬件。

三角函数

<code>acos</code>	<code>acospi</code>	<code>asin</code>	<code>asinpi</code>
<code>atan</code>	<code>atan2</code>	<code>atan2pi</code>	<code>cos</code>
<code>cospi</code>	<code>sin</code>	<code>sincos</code>	<code>sinpi</code>

tan tanpi

双曲函数

acosh	asinh	atanh	cosh
sinh	tanh		

指数函数

exp	exp10	exp2	expm1
frexp	ldexp	modf	

对数函数

ilogb log log10 log1p

幂函数

cbt	hypot	pow	rsqrt
sqrt			

误差函数

erf erfc

舍入函数

ceil	floor	llrint	llround
lrint	lround	nearbyint	rint
round	trunc		

余数函数

fmodremainderremquo

浮点

copysign nan nextafter nexttoward

差分函数

fdim	fmax	fmin	maxmag
minmag			



重要提示! Vitis HLS 尚未支持技术规范 18661-1 sNaN 的 `fmin/fmax` 处理方式, 这可能导致与 glibc 2.25 和更高版本的结果进行比对时, 结果出现偏差。

其他函数

abs	divide	fabs	fma
fract	mad	recip	

分类函数

fpclassify	isfinite	isinf	isnan
isnormal	signbit		

比较函数

isgreater	isgreaterequal	isless	islessequal
islessgreater	isunordered		

关系函数

all	any	bitselect	isequal
isnotequal	isordered	select	

定点数学函数

针对以下数学函数还提供了定点实现。

所有定点数学函数都支持 `ap_[u]fixed` 数据类型和 `ap_[u]int` 数据类型，并采用以下位宽规格：

- 1. `ap_fixed<W,I>`（其中 $I \leq 33$ ， $W-I \leq 32$ ）
- 2. `ap_ufixed<W,I>`（其中 $I \leq 32$ ， $W-I \leq 32$ ）
- 3. `ap_int<I>`（其中 $I \leq 33$ ）
- 4. `ap_uint<I>`（其中 $I \leq 32$ ）



重要提示！ 来自 `hls_math` 库的定点数学函数不支持 `ap_[u]fixed` 模板参数 `Q`、`O` 和 `N`，这三个参数依次分别对应于量化模式、上溢模式和饱和位数。仅当 `ap_[u]fixed` 变量位于赋值左侧时或者在初始化期间，量化模式和上溢模式才有效，但在变量计算期间无效。

三角函数

cos	sin	tan	acos	asin	atan	atan2	sincos
cospi	sinpi						

双曲函数

cosh	sinh	tanh	acosh	asinh	atanh
------	------	------	-------	-------	-------

指数函数

exp frexp modf exp2 expm1

对数函数

log log10 ilogb log1p

幂函数

pow sqrt rsqrt cbrt hypot

误差函数

erf erfc

舍入函数

ceil floor trunc round rint nearbyint

浮点

nextafter nexttoward

差分函数

erf erfc fdim fmax fmin maxmag minmag

其他函数

fabs recip abs fract divide

分类函数

signbit

比较函数

isgreater isgreaterequal isless islessequal islessgreater

关系函数

isequal isnotequal any all bitselect

定点类型提供的函数值版本准确性略低，但 RTL 实现更小且更快。

使用定点数据类型实现数学函数的方法是：

- 1. 判定是否支持定点实现。

2. 将数学函数更新为使用 `ap_fixed` 类型。
3. 执行 C 语言仿真来确认设计按所需精度是否仍可正常操作。执行 C 语言仿真时所使用的位精度类型与 RTL 实现相同。
4. 对设计执行综合。

例如，使用含如下数学函数的定点类型来指定函数 `sin` 的定点实现：

```
#include "hls_math.h"
#include "ap_fixed.h"

ap_fixed<26,6> my_input, my_output;

my_input = 24.675;
my_output = sin(my_input);
```

验证和数学函数

如果在 C 语言源代码中使用标准 C 语言数学库，C 语言仿真结果与 C/RTL 协同仿真结果可能产生差异：如果源代码中的任一数学函数与标准 C 语言数学库存在 ULP 差异，则可能导致 RTL 仿真结果不同。

如果在 C 语言源代码中使用 `hls_math.h` 库，那么 C 语言仿真与 C/RTL 协同仿真结果可能不同。但使用 `hls_math.h` 时，C 语言仿真结果与使用标准 C 语言库时的结果不同。`hls_math.h` 库只确保 C 语言仿真与 C/RTL 协同仿真结果不同。在上述 2 种情况下，创建的 RTL 实现是相同的。以下解释了使用数学函数时用于执行验证的每一种可能的选项。

验证选项 1：标准数学库和验证差异

在此选项中，源代码中使用的是标准 C 语言数学库。如果已综合的任意函数的准确度与此完全相同，那么 C/RTL 协同仿真与 C 语言仿真不同。以下示例着重演示了此方法。

```
#include <cmath>
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

typedef float data_t;

data_t cpp_math(data_t angle) {
    data_t s = sinf(angle);
    data_t c = cosf(angle);
    return sqrtf(s*s+c*c);
}
```

在此情况下，C 语言仿真与 C/RTL 协同仿真之间的结果不同。比较仿真输出时请谨记，从测试平台写出的任意结果都将写入执行仿真的工作目录：

- C 语言仿真：<project>/<solution>/csim/build 文件夹
- C/RTL 协同仿真：<project>/<solution>/sim/<RTL>文件夹

其中，<project> 是工程文件夹，<solution> 是解决方案文件夹的名称，<RTL> 是验证的 RTL 的类型（Verilog 或 VHDL）。下图显示了综合前的结果文件（左侧）与分析后的 RTL 结果文件（右侧）的典型比较。输出显示在第 3 列中。

图 144：分析前和分析后仿真差异

	result.dat				proj_cpp_math.prj/solution1/sim/systemc/result.dat		
1	0.0000000000000000	0.009999999776483	1.0000000000000000	1	0.0000000000000000	0.009999999776483	1.0000000000000000
2	1.0000000000000000	0.109999999403954	1.0000000000000000	2	1.0000000000000000	0.109999999403954	1.0000000000000000
3	2.0000000000000000	0.2099999993443489	1.0000000000000000	3	2.0000000000000000	0.2099999993443489	1.0000000000000000
4	3.0000000000000000	0.310000002384186	1.0000000000000000	4	3.0000000000000000	0.310000002384186	1.0000000000000000
5	4.0000000000000000	0.409999996423721	1.0000000000000000	5	4.0000000000000000	0.409999996423721	1.0000000000000000
6	5.0000000000000000	0.509999990463257	1.0000000000000000	6	5.0000000000000000	0.509999990463257	1.0000000000000000
7	6.0000000000000000	0.610000014305115	0.999999940395355	7	6.0000000000000000	0.610000014305115	1.0000000000000000
8	7.0000000000000000	0.710000038146973	1.0000000000000000	8	7.0000000000000000	0.710000038146973	1.0000000000000000
9	8.0000000000000000	0.810000061988831	1.0000000000000000	9	8.0000000000000000	0.810000061988831	1.0000000000000000
10	9.0000000000000000	0.910000085830688	1.0000000000000000	10	9.0000000000000000	0.910000085830688	1.0000000000000000
11	10.0000000000000000	1.010000109672546	1.0000000000000000	11	10.0000000000000000	1.010000109672546	1.0000000000000000
12	11.0000000000000000	1.110000133514404	1.0000000000000000	12	11.0000000000000000	1.110000133514404	0.999999940395355
13	12.0000000000000000	1.210000157356262	0.999999940395355	13	12.0000000000000000	1.210000157356262	1.0000000000000000
14	13.0000000000000000	1.310000181198120	0.999999940395355	14	13.0000000000000000	1.310000181198120	0.999999940395355
15	14.0000000000000000	1.410000205039978	1.0000000000000000	15	14.0000000000000000	1.410000205039978	1.0000000000000000
16	15.0000000000000000	1.510000228881836	1.0000000000000000	16	15.0000000000000000	1.510000228881836	1.0000000000000000
17	16.0000000000000000	1.610000252723694	1.0000000000000000	17	16.0000000000000000	1.610000252723694	1.0000000000000000
18	17.0000000000000000	1.710000276565552	1.0000000000000000	18	17.0000000000000000	1.710000276565552	1.0000000000000000
19	18.0000000000000000	1.810000300407410	1.0000000000000000	19	18.0000000000000000	1.810000300407410	1.0000000000000000
20	19.0000000000000000	1.910000324249268	0.999999940395355	20	19.0000000000000000	1.910000324249268	1.0000000000000000
21	20.0000000000000000	2.010000228881836	0.999999940395355	21	20.0000000000000000	2.010000228881836	0.999999940395355
22	21.0000000000000000	2.110000133514404	1.0000000000000000	22	21.0000000000000000	2.110000133514404	1.0000000000000000
23	22.0000000000000000	2.210000038146973	1.0000000000000000	23	22.0000000000000000	2.210000038146973	1.0000000000000000
24	23.0000000000000000	2.309999942779541	1.0000000000000000	24	23.0000000000000000	2.309999942779541	1.0000000000000000
25	24.0000000000000000	2.409999847412109	1.0000000000000000	25	24.0000000000000000	2.409999847412109	1.0000000000000000
26	25.0000000000000000	2.509999752044678	1.0000000000000000	26	25.0000000000000000	2.509999752044678	1.0000000000000000
27	26.0000000000000000	2.609999656677246	1.0000000000000000	27	26.0000000000000000	2.609999656677246	1.0000000000000000
28	27.0000000000000000	2.709999561309814	0.999999940395355	28	27.0000000000000000	2.709999561309814	1.0000000000000000
29	28.0000000000000000	2.809999465942383	1.0000000000000000	29	28.0000000000000000	2.809999465942383	1.0000000000000000
30	29.0000000000000000	2.909999370574951	1.0000000000000000	30	29.0000000000000000	2.909999370574951	1.0000000000000000

分析前仿真与分析后仿真结果存在少量差异。您必须判定在最终 RTL 实现中此少量差异是否可接受。

处理此类差异的建议流程是使用测试平台检查结果，以确认结果处于可接受的误差范围内。这可通过为同一函数创建 2 个版本来完成，其中 1 个版本用于综合，另 1 个版本用作为参考版本。在此示例中，仅对 `cpp_math` 函数进行综合。

```
#include <cmath>
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

typedef float data_t;

data_t cpp_math(data_t angle) {
    data_t s = sinf(angle);
    data_t c = cosf(angle);
    return sqrtf(s*s+c*c);
}

data_t cpp_math_sw(data_t angle) {
    data_t s = sinf(angle);
    data_t c = cosf(angle);
    return sqrtf(s*s+c*c);
}
```

用于验证设计的测试平台会使用 `diff` 函数对 2 个函数的输出进行比较以判定差异，如以下示例所示。在 C 语言仿真期间，这 2 个函数会生成完全相同的输出。在 C/RTL 协同仿真期间，`cpp_math` 函数会生成不同结果，并检查结果的差异。

```
int main() {
    data_t angle = 0.01;
    data_t output, exp_output, diff;
    int retval=0;
```

```

for (data_t i = 0; i <= 250; i++) {
    output = cpp_math(angle);
    exp_output = cpp_math_sw(angle);

    // Check for differences
    diff = ( (exp_output > output) ? exp_output - output : output -
exp_output);
    if (diff > 0.00000005) {
        printf("Difference %.10f exceeds tolerance at angle %.10f \n", diff,
angle);
        retval=1;
    }

    angle = angle + .1;
}

if (retval != 0) {
    printf("Test failed !!!\n");
    retval=1;
} else {
    printf("Test passed !\n");
}
// Return 0 if the test passes
return retval;
}

```

如果差异裕度降低到 0.00000005，那么此测试平台会高亮 C/RTL 协同仿真期间的误差裕度：

```

Difference 0.0000000596 at angle 1.1100001335
Difference 0.0000000596 at angle 1.2100001574
Difference 0.0000000596 at angle 1.5100002289
Difference 0.0000000596 at angle 1.6100002527
etc..

```

使用标准 C 语言数学库（`math.h` 和 `cmath.h`）时，请创建一个“智能”测试平台以验证准确性差异是否可接受。

验证选项 2：HLS 函数库和确认差异

另一个验证选项是将源代码转换为使用 HLS 函数库。对于该选项，C 语言仿真与 C/RTL 协同仿真结果之间并无差异。以下示例显示了如何将以上代码修改为使用 `hls_math.h` 库。

注释：该选项仅限在 C++ 中才可用。

- 包含 `hls_math.h` 头文件。
- 将数学函数替换为等效的 `hls::` 函数。

```

#include <cmath>
#include "hls_math.h"
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

typedef float data_t;

```

```
data_t cpp_math(data_t angle) {
    data_t s = hls::sinf(angle);
    data_t c = hls::cosf(angle);
    return hls::sqrtf(s*s+c*c);
}
```

验证选项 3：HLS 数学库文件和确认差异

包含 HLS 数学库文件 `lib_hlsm.cpp` 作为设计文件可确保 Vitis HLS 使用 HLS 数学库进行 C 语言仿真。该选项与选项 2 相同，但无需修改 C 语言代码。

HLS 数学库文件位于 Vitis HLS 安装区域内的 `src` 目录中。只需将此文件复制到本地文件夹并将其添加为标准设计文件即可。

注释：该选项仅限在 C++ 中才可用。

正如选项 2 一样，该选项使用 HLS 数学库文件进行 C 语言仿真的结果与先前不添加此文件时获取的结果不同。应使用类似选项 1 中的“智能”测试平台通过 C 语言仿真来确认这些差异。

常见综合错误

以下是综合数学函数时常见的用法错误。这些错误通常是由于将 C 语言函数转换为 C++ 以便利用数学函数综合所导致的，但这并非唯一原因。

C++ `cmath.h`

如果使用 C++ `cmath.h` 头文件，则可使用浮点函数，如 `sinf` 和 `cosf`。这样可在硬件中生成 32 位运算。`cmath.h` 头文件还会导致标准函数（如 `sin` 和 `cos`）重载，以使其可用于浮点类型和双精度类型。

C `math.h`

如果使用 C `math.h` 库，则需要单精度函数（如 `sinf` 和 `cosf`）才能综合 32 位浮点运算。所有标准函数调用（如 `sin` 和 `cos`）都会导致对双精度和 64 位双精度运算进行综合。

警告

将 C 语言函数转换为 C++ 以利用 `math.h` 支持时，请确保在使用 Vitis HLS 综合前正确编译新 C++ 代码。例如，如果在代码中配合 `math.h` 使用 `sqrtf()`，则需将以下外部代码添加到 C++ 代码中以便为其提供支持：

```
#include <math.h>
extern "C" float sqrtf(float);
```

为避免因类型转换而导致产生不必要的硬件，请遵循“浮点与双精度”中所述的混合双精度与浮点类型进行操作。

HLS 串流库

串流是一种数据传输形式，其中数据样本从第一个样本开始按顺序发送。串流不需要地址管理。

在 C 语言中很难对使用串流数据的设计进行建模。使用指针执行多次读取和/或写入访问的方法可能会引发问题，因为其中隐式指定了类型限定符以及测试平台的构造方式。



重要提示! `hls::stream` 类仅限在 C++ 设计中使用。

Vitis HLS 提供了 C++ 模板类 `hls::stream<>` 用于对串流数据结构进行建模。使用 `hls::stream<>` 类实现的串流具有如下属性。

- 在 C 语言代码中，`hls::stream<>` 的行为与无限深度的 FIFO 相似。无需定义 `hls::stream<>` 的大小。
- 按顺序对其执行读取和写入。即，从 `hls::stream<>` 读取数据之后，无法再次对其进行读取。
- 顶层接口上的 `hls::stream<>` 默认情况下使用 `ap_fifo` 接口来实现（对应 AMD Vivado™ IP 流程）或者作为 AXIS 接口来实现（对应 AMD Vitis™ 内核流程）。
- 串流可定义为局部或全局，并且始终作为内部 FIFO 来实现。全局作用域内定义的串流遵循的规则与任何其他全局变量相同。
- 有两种可能的串流声明：
 - `hls::stream<Type>`：指定串流的数据类型。
设计内部的 `hls::stream<>` 作为 FIFO 来实现，默认深度为 2。[STREAM](#) 编译指示或指令可用于更改深度。
 - `hls::stream<Type, Depth>`：指定串流的数据类型和 FIFO 深度。
设置深度以防止停滞。如果设计中的任意任务生产或使用样本的速度大于指定深度，那么 FIFO 可能因无法读取（或写入）而变为空（或满），从而导致停滞。

本节显示了如何使用 `hls::stream<>` 类来对含串流数据的设计进行轻松建模。本节中的主题包括：

- 串流建模以及串流的 RTL 实现概述。
- 串流使用方式。
- 阻塞读写。
- 非阻塞读写。
- 控制 FIFO 深度。

注释：`hls::stream` 类应始终在函数之间作为 C++ 参考实参来进行传递。例如，`&my_stream`。

C 语言建模与 RTL 实现

串流在软件中（以及在测试平台中进行 RTL 协同仿真期间）作为无限队列来建模。在 C++ 中对串流进行仿真无需满足任意深度。串流可在函数内部使用，也可在函数接口上使用。内部串流可作为函数参数来进行传递。

串流仅限在基于 C++ 的设计中才能使用。每个 `hls::stream<>` 对象都仅限单一进程写入，且仅限单一进程读取。

如果在 Vivado IP 流程中，顶层接口使用 `hls::stream`，那么默认情况下在 RTL 中，将其作为 FIFO 接口 (`ap_fifo`) 来实现，但也可选择将其作为 AXI4-Stream 接口 (`axis`) 来实现。在 Vitis 内核流程中，默认将其作为 AXI4-Stream 接口 (`axis`) 来实现。

如果在设计函数中使用 `hls::stream` 并将其综合到硬件中，那么会将其作为 FIFO 来实现，且默认深度为 2。在某些情况下（如使用内插时），可能必须增大 FIFO 深度以确保 FIFO 可保存硬件生成的所有元素。如果无法确保 FIFO 大小足以保存硬件生成的所有数据样本，则可能导致设计停滞（可见于 C/RTL 协同仿真和硬件实现中）。FIFO 深度可使用 `STREAM` 指令搭配 `depth` 选项来进行调整。



重要提示！ 确保 `hls::stream` 变量在默认非 DATAFLOW 区域内使用时的大小设置正确。

如果使用 `hls::stream` 在不同任务（子函数或循环）间传输数据，应立即考量在不同任务间存在数据串流的 DATAFLOW 区域内实现这些任务。默认（非 DATAFLOW）行为是完成每一项任务后再开始下一项任务，在此情况下必须确保用于实现 `hls::stream` 变量的 FIFO 大小足以保存生产者任务所生成的所有数据采样。不增大 `hls::stream` 变量的大小会导致如下错误：

```
ERROR: [XFORM 203-733] An internal stream xxxx.xxxx.V.user.V' with default
size is
used in a non-dataflow region, which may result in deadlock. Please
consider to
resize the stream using the directive 'set_directive_stream' or the 'HLS
stream'
pragma.
```

此错误表示在非 DATAFLOW 区域内，默认 FIFO 深度为 2 可能不足以保存生产者任务写入 FIFO 的所有数据采样，并且可能发生死锁。

使用 HLS 串流

要在代码中使用 `hls::stream<>` 对象，请按如下所示方式包含头文件 `hls_stream.h`。串流数据对象是通过指定类型和变量名称来定义的。在此示例中，已定义 1 个 128 位无符号整数类型，用于创建名为 `my_wide_stream` 的串流变量。

```
#include "ap_int.h"
#include "hls_stream.h"

typedef ap_uint<128> uint128_t; // 128-bit user defined type
hls::stream<uint128_t> my_wide_stream; // A stream declaration
```

串流必须使用限定作用域的命名。AMD 建议使用限定作用域的 `hls::` 命名，如上示例所示。但如要使用 `hls` 名称空间，可将前述示例重写为：

```
#include <ap_int.h>
#include <hls_stream.h>
using namespace hls;

typedef ap_uint<128> uint128_t; // 128-bit user defined type
stream<uint128_t> my_wide_stream; // hls:: no longer required
```

给定指定为 `hls::stream<T>` 的串流的情况下，类型 `T` 可以是：

- 任何 C++ 原生数据类型
- Vitis HLS 任意精度类型（例如，`ap_int<>` 和 `ap_ufixed<>`）
- 用户定义的结构体（包含以上任意类型）

注释：用户定义的通用类（或结构），所含方法（成员函数）不应用作为串流变量的类型 (`T`)。

串流还可指定为 `hls::stream<Type, Depth>` 用于指示 HLS 工具为 RTL 协同仿真所创建的验证适配器中所需 FIFO 的深度。

串流命名为可选操作。为串流提供名称即可在报告中使用此名称。例如，Vitis HLS 会自动检查以确保仿真期间读取来自输入串流的所有元素。给定以下 2 个串流的情况下：

```
hls::stream<uint8_t> bytestr_in1;
hls::stream<uint8_t> bytestr_in2("input_stream2");
```

有关串流元素的任何警告均按如下方式报告，其中显而易见，`input_stream2` 表示 `bytestr_in2`：

```
WARNING: Hls::stream 'hls::stream<unsigned char>.1' contains leftover data,
which
may result in RTL simulation hanging.
WARNING: Hls::stream 'input_stream2' contains leftover data, which may
result in RTL
simulation hanging.
```

传入或传出函数的串流必须按引用传递，如下示例所示：

```
void stream_function (
    hls::stream<uint8_t> &strm_out,
    hls::stream<uint8_t> &strm_in,
    uint16_t strm_len
)
```

在 GitHub 上的 [Vitis-HLS-Introductory-Examples](#) 仓库内的 [Interface/Streaming](#) 下提供了串流示例。同样在 GitHub 上的 [Vitis_Accel_Examples](#) 中提供了使用串流的其他设计示例。

对于 `hls::stream` 对象，Vitis HLS 还支持阻塞访问方式和非阻塞访问方式，如下章节中所述。

阻塞 API

阻塞一词表示操作停滞，直至串流通道上刷新数据可用为止。阻塞 API 会自动检查 FIFO 是满还是空，并在 FIFO 上执行读/写操作。无需手动检查 FIFO 的满和空状态。阻塞 API 完全属于确定性行为，也就是说 C 语言仿真和 RTL/协同仿真行为方式将完全相同。仅限在如下章节所述场景中才应使用此阻塞 API。在任何其他情况下使用都会导致不确定性行为。



提示：在阻塞模式下使用 `hls::stream` 时，可能由于数据串流大小不足而导致死锁。

阻塞写入方法

在此示例中，变量 `src_var` 的值推送到串流中。

```
// Usage of void write(const T & wdata)

hls::stream<int> my_stream;
int src_var = 42;

my_stream.write(src_var);
```

“<<”运算符已重载，可按相似方式应用于 C++ 串流（例如，`istream`s 和 `ofstream`s）的串流插入运算符。要写入的 `hls::stream<>` 对象将作为左侧实参提供，而变量则作为右侧实参写入。

```
// Usage of void operator << (T & wdata)

hls::stream<int> my_stream;
int src_var = 42;

my_stream << src_var;
```

阻塞读取方法

此方法从串流头部读取并向 `dst_var` 变量赋值。

```
// Usage of void read(T &rdata)

hls::stream<int> my_stream;
int dst_var;

my_stream.read(dst_var);
```

或者，通过将串流赋值（例如，使用 `=` 或 `+=`）给左侧对象，即可读取串流中的下一个对象：

```
// Usage of T read(void)

hls::stream<int> my_stream;

int dst_var = my_stream.read();
```

`>>` 运算符已重载，支持以相似方式用于 C++ 串流（例如，`istream`s 和 `ofstream`s）的串流提取运算符。`hls::stream` 作为 LHS 实参和目标变量 RHS 提供。

```
// Usage of void operator >> (T & rdata)

hls::stream<int> my_stream;
int dst_var;

my_stream >> dst_var;
```


确定性行为

如 [阻塞 API](#) 中所述，阻塞 API 可包含确定性行为和不确定性行为，如以下代码示例所示：

```
func1()
{
    while(!s.empty()) {
        s.read();
    }
}
```

在 C 语言仿真期间，到串流的数据始终可用，因此 `while` 循环会运行直至完成。但如果数据串流在硬件中运行时有单个气泡，`while` 循环就将退出，`func1` 将提前返回。这将导致 C 语言仿真与 RTL 执行之间出现不确定性行为。

实现此循环的正确方法是使用旁路信号 `tlast`，如以下示例所示。如需了解更多信息，请参阅 [含旁路的 AXI4-Stream 接口](#)。

```
func1()
{
    while(!tlast) {
        s1.read()
    }
}
```

通过以下任一方式使用阻塞 API 即可得到确定性行为。

- 案例 1：简单读/写 FIFO：

```
int data = in.read();
if (data >= 10)
    out1.write(data);
else
    out2.write(data);
```

- 案例 2：使用阻塞 API 执行 FULL（满）和 EMPTY（空）检查：

此例不执行任何具有副作用的计算，即，读取或写入串流、存储器或静态变量仍被视为仅使用阻塞串流访问。

```
void df(hls::stream<...> &s1, hls::stream<...> &s2, ...) {
    #pragma HLS dataflow

    p1(s1, ...);
    p2(s2, ...);
    ...
}
void p1(hls::stream<...> &s1, ...) {
    if (s1.empty())
        return;
    ... = s1.read();
    ...
}
void p2(hls::stream<...> &s2, ...) {
    if (s2.full())
        return;
    ...
    s2.write(...);
}
```

非阻塞 API



重要提示！ 在使用 `ap_fifo` 协议的接口上完整支持非阻塞 API。但 AXI4-Stream (`axis`) 接口仅支持非阻塞读取访问，不支持写入。

非阻塞一词表示串流上缺少数据（或者数据过多）不会阻塞函数运算或循环迭代。非阻塞方法会返回布尔值以表示读取或写入的状态：`true` 表示成功，`false` 表示失败。但使用非阻塞 API 可能导致不确定行为，在 C 语言仿真或 RTL/协同仿真期间无法充分确认此类行为，需要通过 RTL 测试平台来对其进行详尽测试。从空 FIFO 读取或者写入已满的 FIFO 时，可能发生不确定行为。

在 C 语言仿真期间，串流大小无限。因此，无法通过 C 语言仿真确认串流是否已满。只能在 RTL 仿真期间已定义 FIFO 大小（默认大小 1，或者通过 `STREAM` 指令定义的任意大小）的情况下验证这些方法。



重要提示！ 如果设计指定为使用块级 I/O 协议 `ap_ctrl_none` 且设计包含采用非阻塞行为的任意 `hls::stream` 变量，则无法保证能否完成 C/RTL 协同仿真。

非阻塞写入

`hls::stream.write_nb()` 方法会尝试将数据推送到串流中，如果成功，则返回布尔值 `true`。否则返回 `false`，队列不受影响。

```
// Usage of bool write_nb(const T & wdata)
hls::stream<int> my_stream;
int src_var = 42;

if (my_stream.write_nb(src_var)) {
    // Perform standard operations
    ...
} else {
    // Write did not occur
    return;
}
```

非阻塞行为也可以使用含串流已满检查条件的阻塞写入操作来进行建模，如 `hls::stream.full()` 方法中所述。这可能导致不确定行为，应在含复杂测试平台的 RTL 仿真中进行验证。

```
hls::stream<int> my_stream;
int src_var = 42;
bool stream_full;

stream_full = my_stream.full();
if(!stream_full)
    my_stream.write_nb(src_var);
```

非阻塞读取

`hls::stream.read_nb()` 方法会尝试从串流中读取值，成功则返回 `true`。否则返回 `false`，队列不受影响。

```
// Usage of bool read_nb(const T & wdata)

hls::stream<int> my_stream;
int dst_var;

if (my_stream.read_nb(dst_var)) {
    // Perform standard operations
```

```

...
} else {
    // Read did not occur
    return;
}

```

非阻塞行为也可以使用含空串流检查的阻塞读取操作来进行建模，如 `hls::stream.empty()` 方法中所述。这可能导致不确定行为，应在含复杂测试平台的 RTL 仿真中进行验证。

```

READ_ONLY_LOOP:
while (check != 0) {
    if ( !addr_strm.empty() )
    {
        addr_strm.read_nb(addr_for_HBM);
        hbm[addr_for_HBM] = some_data;
        check[0] = 1;
        ...
    }
    ...
    ...
    check = (check << 1);
}

```

hls::stream.capacity() 方法

`hls::stream.capacity()` 方法会返回无符号整数，用于定义串流可保存的元素的最大数量，或者串流的总深度。这样您即可在写入前检查通道内可用的空间量。

```

hls::stream<int> my_stream;
int var = 16;

if ((my_stream.capacity() - my_stream.size()) >= N) {
    for (int=0; i<N;i++)
        my_stream.write(...); // this will not block
}

```



提示：在 C 语言仿真中，`hls::stream.capacity()` 会返回 MAXINT 作为理论限值。在综合与 C/RTL 协同仿真中，它会返回计算所得的串流容量。

仅当存在特定量的空间或数据可供传输事务使用时，`hls::stream.capacity()` 方法和 `hls::stream.size()` 方法才能启用高级流程控制用于读取或写入串流。例如，启动大小为 N 的读取突发前，您可检查输入串流中是否存在 N 个数据项可供突发正常完成操作而不会发生停滞。对于写入突发，您还可检查输出串流中是否存在足够空间可用于存储突发数据而不会发生停滞。

hls::stream.empty() 方法

如果串流为空，`hls::stream.empty()` 方法会返回 `true`，如下所示。

```

hls::stream<int> my_stream;
int dst_var;
bool stream_empty;

stream_empty = my_stream.empty();

```

以下示例显示了 RTL FIFO 已满或已空时，如何通过非阻塞访问与满/空测试的组合来提供错误处理功能：

```
#include "hls_stream.h"
using namespace hls;

typedef struct {
    short    data;
    bool     valid;
    bool     invert;
} input_interface;

bool invert(stream<input_interface>& in_data_1,
            stream<input_interface>& in_data_2,
            stream<short>& output
            ) {
    input_interface in;
    bool full_n;

    // Read an input value or return
    if (!in_data_1.read_nb(in))
        if (!in_data_2.read_nb(in))
            return false;

    // If the valid data is written, return not-full (full_n) as true
    if (in.valid) {
        if (in.invert)
            full_n = output.write_nb(~in.data);
        else
            full_n = output.write_nb(in.data);
    }
    return full_n;
}
```

hls::stream.full() 方法

仅当对象已满时，`hls::stream.full()` 方法才会返回 `true`，如下所示：

```
hls::stream<int> my_stream;
int src_var = 42;
bool stream_full;

stream_full = my_stream.full();
```

hls::stream.size() 方法

`hls::stream.size()` 方法会返回无符号整数，用于指定串流当前保存的元素数量。这样您即可先检查通道内是否存在数据或者存在的数据量，然后再读取数据，以免发生停滞或死锁。

注释： `hls::stream.size()` 可用于检查输入 FIFO 是否已满以及输出 FIFO 是否为空。但是，`hls::stream.empty()` 只能应用于输入 FIFO，`hls::stream.full()` 只能应用于输出 FIFO。

```
hls::stream<int> my_stream;
var = 16;

if (my_stream.size()>0) {
    my_stream.read(var);
}
```

仅当存在特定量的空间或数据可供传输事务使用时，`hls::stream.size()` 方法和 `hls::stream.capacity()` 方法才能启用高级流程控制用于读取或写入串流。例如，启动大小为 N 的读取突发前，您可检查输入串流中是否存在 N 个数据项可供突发正常完成操作而不会发生停滞。对于写入突发，您还可检查输出串流中是否存在足够空间可用于存储突发数据而不会发生停滞。

```
if (instream.size() > N)
    for(i=0, i<N; i++)
        ... = instream.read(); // will not block
```

如需仅在串流已满时才执行读取，请执行以下操作：

```
if (instream.size() == instream.capacity())
    ... = instream.read(); // read only if full
```



提示：您不能使用 `hls::stream.full()` 来检查此条件。以下操作将无效：

```
if (instream.full())
    ... = instream.read();
```

控制 RTL FIFO 深度

对于使用串流数据的大部分设计，默认 RTL FIFO 深度 2 足矣。每次仅处理一个采样的串流数据。

对于执行实现所需的 FIFO 深度大于 2 的多速率设计，必须判定（并使用 STREAM 指令设置）完成 RTL 仿真所需的深度。如果 FIFO 深度不足，RTL 协同仿真将停滞。

由于在 GUI 指令窗格内无法查看串流对象，因此在该窗格内无法直接应用 STREAM 指令。

对于包含已声明的 `hls::stream<>` 对象（或者实参列表中已使用或包含该对象）的函数，右键单击并执行以下操作：

- 选中 STREAM 指令。
- 使用串流变量的名称手动填充 `variable` 字段。

或者可以：

- 在 `directives.tcl` 文件中手动指定 STREAM 指令，或
- 在 `source` 中添加该指令作为编译指示。

HLS 直接 I/O

连续运行的内核是一种硬件内核类型，旨在无限期运行直至发生复位事件。它有别于受调用、执行并返回的传统软件加速内核。

在软件加速内核中，参数只能在调用内核时或内核空闲时进行更改。但对于连续运行的内核，由于这些内核始终运行，因此您无法在执行期间以通常方式修改其实参。为了动态修改连续运行的内核的内核实参（标量和存储器偏移），AMD Vitis™ HLS 提供了 Direct I/O 类，为动态更新这些实参提供了便捷的解决方案。

使用直接 I/O 动态调整参数

AMD Vitis™ HLS 提供了一个 C++ 模板类 `hls::direct_io`，专用于在执行期间修改内核的标量/存储器偏移实参。

`hls::direct_io` 的功能特性

- `hls::direct_io` 串流只能在顶层函数上使用
- 该类支持所有端口级协议。该类支持以下端口协议：`ap_none`、`ap_ack`、`ap_vld` 和 `ap_hs`。
- 直接 I/O 串流使用阻塞调用和非阻塞调用来进行读取和写入。
- 直接 I/O 串流的编码方式与 HLS 串流类似。但前者是根据端口协议作为连线来实现的。注释：直接 I/O 串流不作为 FIFO 来实现/综合。

使用直接 I/O 串流

要在您的高层次综合 (HLS) 代码中使用直接 I/O，您必须首先包含必要的头文件以提供直接 I/O 功能。首先，在源文件顶部添加以下行。

```
#include "hls_directio.h"
```

包含头文件后，您即可通过选择适当的端口级协议来定义直接 I/O 对象。这些协议会判定直接 I/O 串流与系统设计中其他组件的交互方式。随后，您可使用阻塞调用或非阻塞调用来读取或写入这些串流。在下一章节中，将详细解释阻塞和非阻塞 API。

定义和使用示例

在以下示例中，定义了两个直接 I/O 串流变量，`reset_value` 和 `reset_my_counter`，它们使用 `ap_vld` 握手协议和整数数据类型：

```
#include "hls_directio.h"
hls::ap_vld<int> &reset_value,
hls::ap_vld<int> &reset_myCounter
```

给定指定为 `hls::ap_vld<T>` 的直接 I/O 串流的情况下，类型 `T` 可以是：

- 任何 C++ 原生数据类型
- Vitis HLS 任意精度类型（例如，`ap_int<>` 或 `ap_ufixed<>`）
- 用户定义的结构体（包含以上任意类型）

在 GitHub 上的 [Vitis-HLS-Introductory-Examples](#) 仓库中提供了直接 I/O 串流示例。

将直接 I/O 串流映射到连线接口

直接 I/O 串流提供与硬件 (HW) 连线接口的灵活对接功能。通过选择适合的协议 [端口级 I/O：有线握手](#)，如，`ap_ack`、`ap_vld`、`ap_none` 或 `ap_hs`，您即可有效管理数据流信号、确认信号和应答信号。

直接 I/O 串流的协议选项

AP_HS

接口模式 `ap_hs` 包含一个双向握手信号，其中带有数据端口，并遵循业界标准的确认和应答握手。与 `ap_hs` 协议关联的信号包括：

- **数据端口：**通过 `<var>.read()` 访问数据。
- **确认信号：**通过 `<var>.valid()` 检查数据有效性，当数据准备好进行读取时返回 `true`。
- **应答信号：**通过 `<var>.ready()` 确认数据接收，读取数据后返回 `true`。

AP_VLD

`ap_vld` 协议是 `ap_hs` 的简化版本，可提供：

- **数据端口：**"`<var>.read()`"
- **确认信号：**"`<var>.valid()`" 用于指示数据信号有效并且可读。

使用 `ap_vld` 的内核函数示例如下：

```
void krnl_stream_vdatamover(hls::stream<pkt> &in,
                           hls::stream<pkt> &out,
                           int mem[DATA],
                           hls::ap_vld<int> &reset_value,
                           hls::ap_vld<int> &reset_myCounter
                           )

{
    for(int i=0;i<DATA;i++)
    {
        // ...
        if(reset_myCounter.valid()) {
            int reset = reset_myCounter.read();
            //...
        }
        // ..
    }
}
```

AP_ACK

ap_ack 端口级 I/O 协议是 ap_hs 接口类型的子集。ap_ack 端口级 I/O 协议提供以下信号：

- **数据端口：**通过 `<var>.read()` 访问数据。
- **应答信号：**通过 `<var>.ready()` 来表示数据耗用。

AP_NONE

ap_none 端口级 I/O 协议是最简单的接口类型，没有与之关联的其他信号。输入和输出数据信号都没有关联的控制端口以指示何时读取或写入数据。RTL 设计中仅有的端口是源代码中指定的端口。

注释：AP_NONE 协议不含非阻塞 API。

```
typedef hls::ap_none<int> mystream_in;
typedef hls::ap_none<int> mystream_out;
typedef hls::ap_none<int> stream_vld_in;
typedef hls::ap_none<int> stream_vld_out;

void addInputs(mystream_in &in, mystream_in
&dinB, stream_vld_in&vldIn, mystream_out &dout_Y, stream_vld_out & vldOut)
{
    #pragma HLS pipeline II=1
    int var = vldIn.read();
    if(var ==0)
    {
        vldOut.write(var);
        return;
    }
    int tmp1 = in.read(); // Blocking_read
    int tmp2 = dinB.read(); // Blocking read
    int out = tmp1+ tmp2;
    dout_Y.write(out);
    vldOut.write(1);
}
```

将直接 I/O 串流映射到 SAXI Lite 接口

Vitis HLS 工具能够将直接 I/O 串流（常用于较简单的数据传输或控制信号）直接映射到标准 SAXI Lite 接口，以便与处理器或其他 IP 核通信。以下过程概述了此映射的配置方式：

1. 选择要与 SAXI Lite 接口关联的端口协议。
2. 添加 SAXI Lite 接口编译指示。
3. 使用读/写方法。

```
void sub_task1(hls::stream<int>& in, hls::stream<int>& out) {
    int c = in.read();
    out.write(c + 2);
}

void sub_task2(hls::stream<int>& in, hls::stream<int>& out) {
    int c = in.read();
    out.write(c - 2);
}

void task2(hls::stream<int>& in, hls::stream<int>& out, hls::ap_none<int>
```



```

&n) {
    int c = in.read();
    int var = n.read();
    out.write(c + var);
}

void test(hls::stream<int>& in, hls::stream<int>& out, hls::ap_none<int>
&bias) {
#pragma HLS interface s_axilite port=bias
    HLS_TASK_STREAM<int> s1;
    HLS_TASK_STREAM<int> s2;
    HLS_TASK t(task2, s2, out, bias);
    HLS_TASK t1(sub_task1, in, s1);
    HLS_TASK t2(sub_task2, s1, s2);
}

```

阻塞/非阻塞 API

阻塞 API

阻塞一词表示操作停滞，直至直接 I/O 通道上刷新数据可用为止。阻塞 API 会自动检查直接 I/O 变量是否有任何输入数据。

阻塞写入

此 `.write()` 方法会将值 “counter” 写入直接 I/O 变量 “reset_value”。

```

// usage of void write( const T &din)

dut_top(hls::ap_vld<int> &reset_value,...)

...
    reset_value.write(counter);
..

```

“<<” 运算符已重载，可按相似方式应用于 C++ 串流（例如，`iostreams` 和 `filestreams`）的串流插入运算符。要写入的 `hls::ap_vld/ap_hs/ap_none<>` 对象将作为左侧实参提供，而变量则作为右侧实参写入。

```

// usage of void operator << (_const T & din) {

dut_top(hls::ap_vld<int> &reset_value,...)

...
    reset_value << counter;
..

```

阻塞读取

此 `<var>.read()` 方法会从 “reset_myCounter” 读取，并将这些值赋给 `reset` 变量。

```
// usage of void read(const T & dout)

void krnl_stream_vdatamover(...
                           hls::ap_hs<int> &reset_myCounter
                           ) {
...
    int reset = reset_myCounter.read();
...
}
```

“>>” 运算符已重载，可按相似方式应用于 C++ 串流（例如，`istream`s 和 `ofstream`s）的串流插入运算符。要读取的 `hls::ap_vld/ap_hs/ap_none<>` 对象是作为 LHS 实参来提供的，并且目标变量为 RHS。

```
// Usage of void operator >> (const T& rdata) void
krnl_stream_vdatamover(..., hls::ap_hs<int> &reset_myCounter ){
...
    reset_myCounter >> reset;
...
}
```

非阻塞 API

非阻塞一词表示串流上缺少数据（或者数据过多）不会阻塞函数运算或循环迭代。非阻塞方法会返回布尔值以表示读取或写入的状态：`true` 表示成功，`false` 表示失败。

非阻塞写入

`ap_hs/ap_vld` 支持此方法，它会尝试将数据推送到串流中，如果成功则返回布尔值 `true`。否则返回 `false`。

```
// usage of bool write_nb(const T& din)

dut_top(hls::ap_vld<int> &reset_value,...)

int counter =1;
reset_value.write_nb(counter);
```

“<<” 运算符已重载，可按相似方式应用于 C++ 串流（例如，`istream`s 和 `ofstream`s）的串流插入运算符。要写入的 `hls::ap_vld/ap_hs/ap_none<>` 对象将作为左侧实参提供，而变量则作为右侧实参写入。

```
// usage of void operator << (_const T & din) {

dut_top(hls::ap_vld<int> &reset_value,...)

...
    reset_value << counter;
..
```

非阻塞读取

ap_hs/ap_vld 支持此方法，它会尝试将数据推送到串流中，如果成功则返回布尔值 `true`。否则返回 `false`。

```
// Usage of bool read_nb(__STREAM_T__& dout)

hls::ap_vld<int> &reset_myCounterint reset =1;
reset_myCounter.read_nb(reset);
```

“>>” 运算符已重载，可按相似方式应用于 C++ 串流（例如，`istream`s 和 `ostream`s）的串流插入运算符。要读取的 `hls::ap_vld/ap_hs/ap_none<>` 对象是作为 LHS 实参来提供的，并且目标变量为 RHS。

```
// Usage of void operator >> (const T& rdata)

void krnl_stream_vdatamover(..., hls::ap_hs<int> &reset_myCounter ){
...
reset_myCounter >> reset;
...
}
```

HLS 围栏库

Vitis HLS 编译器可以对不相关的访问进行重新排序以提高性能。但为了确保逻辑正确（例如，保留程序语义），它保留了程序的顺序语义，用于保证“控制依赖关系”和“数据依赖关系”。

此外，如果用户希望在不相关的访问之间强制遵循某种顺序，则可使用 Vitis HLS 围栏以确保遵循源代码所暗示的顺序来对通过引用传递的串流、阵列和标量函数实参执行两次（或两次以上）的访问。



重要提示！ 要使用围栏，请在您的源代码中包含头文件 `hls_fence.h`。如需获取围栏用法的示例，请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Interface/Memory/aliasing_axi_master_ports](https://github.com/Xilinx/Vitis-HLS-Introductory-Examples/Interface/Memory/aliasing_axi_master_ports)。

全围栏和半围栏

该库提供了两种不同类型的围栏。

全围栏

```
hls::fence(obj_1, obj_2, ..., obj_n);
```

全围栏用于约束对 `obj_1`、`obj_2` ... `obj_n` 的访问的调度。从顺序的角度而言，原先位于围栏前的任何访问经调度后仍应保持位于围栏前，原先位于围栏后的任何访问经调度后都仍应位于围栏后。

半围栏

半围栏的限制程度较低，在流水线循环中很有用。

```
hls::fence({obj_1, obj_2, ..., obj_n}, {obj'_1, obj'_2, ..., obj'_p});
```

以上围栏用于按如下方式约束访问调度：第一组大括号中的对象可以调度为早于围栏。但如果这些对象按顺序位于围栏前，则不能将其调度到围栏后。同样，对于第二组（右大括号）中对象的访问也可以调度至晚于围栏。但如果这些对象按顺序位于围栏后，则不能将其调度到围栏前。

注释：全围栏 `hls::fence(a,b)` 等效于半围栏 `hls::fence({a, b}, {a, b});`。

实用示例

以下提供了两个 `hls::fence` 较为实用的典型场景。

避免死锁

假设有两个数据流进程彼此通信。

```

Producer(...) {
    int bound = ...;
    strm1.write(bound);
    for (int i=0; i<bound; i++) {
        strm2.write(...);
    }
}

Consumer(...) {
    int bound = strm1.read();
    for (int i=0; i<bound; i++) {
        ... = strm2.read();
    }
}

```

第二个进程（使用者）需要读取串流 strm1 以访问循环边界，并开始迭代读取串流 strm2。此循环边界是由第一个进程（生产者）发送的，然后再写入 strm2。

在这种安排下，这两个进程可以重叠，只需要小型 FIFO 即可存储 strm2 中的元素。原则上，您不希望编译器和/或调度器将 strm1 的写入移到该循环之后。在此情况下，在所有元素都写入 strm2 之前，使用者无法开始读取，因此需要一个深度为“bound”的串流来避免死锁。

使用围栏可保证 strm1 的写入发生在 strm2 的写入之前，如下所示。

```

Producer(...) {
    int bound = ...;
    strm1.write(bound);
    hls::fence({strm1}, {strm2});
    for (int i=0; i<bound; i++) {
        strm2.write(...);
    }
}

Consumer(...) {
    int bound = strm1.read();
    for (int i=0; i<bound; i++) {
        ... = strm2.read();
    }
}

```

配置 Vivado LogiCORE FFT

FFT 配置和数据发送循环的初始化可按如下方式来编写：

```

void inputdatamover(... input_parameters, config_t *fft_config, cmpxDataIn
in[FFT_LENGTH], cmpxDataIn xn[FFT_LENGTH]) {
    config_t fft_config_tmp;
    fft_config_tmp.setDir(...);
    fft_config_tmp.setSch(...);
    *fft_config = fft_config_tmp;

    for(int i=0; i<FFT_LENGTH; i++) {
        #pragma HLS pipeline rewind
        xn[i] = in[i];
    }
}

```

为保证正确，必须先捕获 FFT 的配置，然后才能实际处理数据。

按以上编写方式，无法保证正确，可能需要围栏：

```
void inputdatamover(... input_parameters, config_t *fft_config, cmpxDataIn
in[FFT_LENGTH], cmpxDataIn xn[FFT_LENGTH]) {
    config_t fft_config_tmp;
    fft_config_tmp.setDir(...);
    fft_config_tmp.setSch(...);
    *fft_config = fft_config_tmp;
    hls::fence({fft_config}, {xn});
    for(int i=0; i<FFT_LENGTH; i++) {
        #pragma HLS pipeline rewind style=flp
        xn[i] = in[i];
    }
}
```

已知限制

默认情况下，围栏无法用作涉及所有变量的屏障。需要约束的所有变量都应显式设为围栏的实参。

在调度区域（流水线或顺序）中可以应用围栏，但在数据流区域不能应用。

HLS 向量库



重要提示! 要在代码中使用 `hls::vector` 对象，请包含头文件 `hls_vector.h`。如需获取向量应用示例，请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples/Modeling](#)。

向量数据类型用法

通过提供向量数据类型即可轻松对 SIMD 类型向量运算进行建模与综合。AMD Vitis™ HLS 向量数据类型可按如下方式定义，其中 `T` 是原语或用户定义的类型，其中定义了大部分算术运算。`N` 是大于 0 的整数。声明向量类型变量后，它可像任何其他原语类型的变量一样用于执行算术运算和逻辑运算。

```
#include <hls_vector.h>
hls::vector<T,N> aVec;
```



提示: 当 `T` 的位宽和 `N` 均为 2 的幂整数时，即可实现最佳性能。

存储器布局

对于定义为 `hls::vector<T,N>` 的任何向量类型，存储器应保证连续、大小为 `sizeof(T)*N` 并对齐到最大 2 的幂值，因此，分配的大小至少为 `sizeof(T)*N`。具体来说，当 `N` 为 2 的幂并且 `sizeof(T)` 同样为 2 的幂时，`vector<T, N>` 即对齐到其总大小。这与大部分架构上的向量实现都匹配。



提示: `sizeof(T)*N` 为 2 的幂整数值时，分配的大小将精确设为 `sizeof(T)*N`，否则分配的大小将更大以保证能够对齐。

以下示例显示了按上述方式对齐自身的向量类的定义。

```
constexpr size_t gp2(size_t N)
{
    return (N > 0 && N % 2 == 0) ? 2 * gp2(N / 2) : 1;
}

template<typename T, size_t N> class alignas(gp2(sizeof(T) * N)) vector
{
    std::array<T, N> data;
};
```

以下是不同的对齐示例：

```
hls::vector<char,8> char8Vec; // aligns on 8 Bytes boundary
hls::vector<int,8> int8Vec; // aligns on 32 byte boundary
```

初始化

或者，`hls::vector<...>::iota` 有助于初始化一个从起始值开始递增的向量。这类似于 `std::iota` 函数。它会取一个实参作为初始值，并以 `start + i` 更新该向量的每个连续值，其中，`i` 是该值在向量中的索引。初始化会被完全展开（这与所有 `hls::vector` 运算的预期相符）。作为示例，以下代码可以更轻松地创建一个具有递增初始值的 `hls::vector`：

```
auto vec = hls::vector<int, 8>::iota(-4); The above code is equivalent to:
hls::vector<int, 8> vec = {-4, -3, -2, -1, 0, 1, 2, 3};
```

`lambda` 函数也可以初始化 `hls::vector` 向量。以下代码会添加初始值：

```
hls::vector<int, 8> vec([](size_t i) { return i*i; });
```

以上代码等效于：

```
hls::vector<int, 8> vec({0, 1, 4, 9, 16, 25, 36, 49});
```

要求和依赖关系

Vitis HLS 向量类型需支持 C++ 14 或更高版本。它与标准头文件之间存在如下依赖关系：

- `<array>`
 - `std::array<T, N>`
- `<cassert>`
 - `assert`
- `<initializer_list>`
 - `std::initializer_list<T>`

受支持的运算

· 初始化：

```
hls::vector<int, 4> x; // uninitialized
hls::vector<int, 4> y = 10; // scalar initialized: all elements set to 10
hls::vector<int, 4> z = {0, 1, 2, 3}; // initializer list (must have 4
elements)
hls::vector<ap_int, 4> a; // uninitialized arbitrary precision data type
```

· 访问：

`operator[]` 支持访问该向量的各元素，类似于标准阵列：

```
x[i] = ...; // set the element at index i
... = x[i]; // value of the element at index i
```

· 算术：

这些运算以递归方式定义，依赖于 `T` 上匹配的运算。

表 53：算术运算

运算	就地	表达式	归约（左折）
加法	<code>+=</code>	<code>+</code>	<code>reduce_add</code>
减法	<code>-=</code>	<code>-</code>	非结合
乘法	<code>*=</code>	<code>*</code>	<code>reduce_mult</code>
除法	<code>/=</code>	<code>/</code>	非结合
取余	<code>%=</code>	<code>%</code>	非结合
按位 AND	<code>&=</code>	<code>&</code>	<code>reduce_and</code>
按位 OR	<code> =</code>	<code> </code>	<code>reduce_or</code>
按位 XOR	<code>^=</code>	<code>^</code>	<code>reduce_xor</code>
左移	<code><<=</code>	<code><<</code>	非结合
右移	<code>>>=</code>	<code>>></code>	非结合
前置递增	<code>++x</code>	无	一元运算符
前置递减	<code>--x</code>	无	一元运算符
后置递增	<code>x++</code>	无	一元运算符
后置递减	<code>x--</code>	无	一元运算符

· 比较：
向量按字典顺序（返回布尔值）：

表 54：运算

运算	表达式
小于	<code><</code>
小于或等于	<code><=</code>
等于	<code>==</code>
不等于	<code>!=</code>
大于或等于	<code>>=</code>
大于	<code>></code>

HLS 任务库



重要提示! 要在代码中使用 `hls::task` 对象, 请包含头文件 `hls_task.h`。

`hls::task` 库提供了一种简单的方法用来对数据驱动的进程进行建模, 此方法支持对仅含串流 I/O (`hls::stream` 或 `hls::stream_of_blocks`) 的任务进行静态例化, 这样就不必检查对 C++ 中的并发进程进行建模所必需的空串流。



提示: `hls::task` 还支持为 `s_axilite` 接口上的标量值提供稳定的输入, 或者为 `m_axi` 接口上的阵列提供稳定的指针, 如 [DTLP 中对于标量/存储器接口的支持](#) 中所述。

GitHub 上的 [simple_data_driven](#) 提供了如下所示简单示例:

```
void odds_and_evens(hls::stream<int> &in, hls::stream<int> &out1,
hls::stream<int> &out2) {
    hls_thread_local hls::stream<int> s1; // channel connecting t1 and
t2
    hls_thread_local hls::stream<int> s2; // channel connecting t1 and t3

    // t1 infinitely runs splitter, with input in and outputs s1 and s2
    hls_thread_local hls::task t1(splitter, in, s1, s2);
    // t2 infinitely runs function odds, with input s1 and output out1
    hls_thread_local hls::task t2(odds, s1, out1);
    // t3 infinitely runs function evens, with input s2 and output
    hls_thread_local hls::task t3(evens, s2, out2);
}
```

请注意, 顶层函数 `odds_and_evens` 使用串流输入和输出接口。这是纯串流内核。顶层函数包含:

- `s1` 和 `s2` 均为线程本地的串流 (`hls_thread_local`), 并且均用于连接任务通道任务 `t1` 和 `t2`。这些串流需作为线程本地的串流, 这样在多次顶层调用之间才能保持处于活动状态。
- `t1`、`t2` 和 `t3` 均为线程本地的 `hls::task`, 分别用于执行 `splitter` 函数、`odds` 函数和 `evens` 函数。这些任务无限运行, 用于处理其输入串流上的数据。无需同步。

但这类仅限串流的模型存在一些限制, 如:

- 您无法访问非本地存储器
- 非串流数据 (如标量、阵列和指针) 可以作为参数来传入, 前提是这些端口已通过 STABLE 编译指示声明为稳定端口。当前含 `m_axi` 接口的顶层指针只能使用 `offset=off` 选项来传递



重要提示! 由于与 `hls::task` 之间不存在同步, 因此标量、阵列和指针的读取时间间隔未知。所以编写代码时, 需确保代码不依赖于这些端口的读取时机。

- 您必须按并行任务规范来显式描述设计中的并行度
- `hls::task` 必须始终在并行上下文中例化, 不得嵌套在顺序上下文中
 - 如果您的顶层函数中使用 `hls::task`, 那么该顶层函数就会成为并行上下文

- 由于数据流区域是并行上下文，因此您可在其中例化 `hls::task`
- 在顺序上下文中，如果您调用函数来例化任务，那么：
 - 该函数必须是并行上下文关联，如数据流区域
 - 必须由并行上下文中的常规数据流进程来生产和耗用任务输入串流和输出串流
- 控制驱动的 TLP 可位于顶层、位于另一个任务驱动的 TLP 内或者位于顺序区域内
- 数据驱动的 TLP 可位于顶层、位于另一个数据驱动的 TLP 内或者嵌套在两个控制驱动的任务之间
- 控制驱动的 TLP 不得位于流水线内，也不得直接位于数据驱动的 TLP 内。在后一种情况下，它必须位于由数据驱动的任务所执行的顺序区域内
- 数据驱动的 TLP 不得直接位于顺序、流水线或控制驱动的 TLP 内
- 顺序区域或流水线只能位于控制或数据驱动的任务主体（而非控制或数据驱动的区域）内

`hls::task` 对象可与标准数据流样式的函数调用自由混用，这些函数调用可将数据移入和移出存储器。任务还支持拆分通道 (`hls::split`) 与合并通道 (`hls::merge`)，拆分通道支持一对多数据分发，用于构建工作程序池来处理数据串流，合并通道则支持多对一数据聚集。

DTLP 中对于标量/存储器接口的支持

控制驱动的任务级并行 (CDTLP) 遵循标准 C 语言语义，例如：函数参数必须可用，而后方可受调用。这意味着在内核开始执行时会为其提供非串流 I/O 参数，这些参数保持不变直至内核完成。这些参数的同步点（如函数起始和结束）允许进行可预测的更新。

相反，数据驱动的任务级并行 (DTLP) 设计的工作方式与 RTL 设计类似。这些内核从 FPGA 编程时刻开始连续运行直至发生复位为止。由于内核始终处于活动状态，因此不存在传统意义上的函数起始点或完成点可供非串流 I/O 参数更新。

尽管 DTLP 设计中缺少同步点，但仍可支持非串流 I/O，其中关键考虑因素是：内核设计师不要求这些非串流输入更改时的时序保持精确。重要之处在于内核可以在其连续执行期间的某个时刻访问更新后的数据。

非串流 I/O 的行为有两种类型：

- 情况 (a)：可以随时更改，与内核是处于执行中还是空闲状态无关。如果内核实参是使用 `hls::direct` I/O 类来建模的，那么内核执行中任意时间点，这些内核都能更改该值。内核设计师对更改的时机并无特定要求，只要内核能在未来某个时间点看到更新后的值即可。
- 情况 (b)：在整个内核执行过程中，I/O 从不更改。默认情况下，编译器会将仅当内核空闲时才会更改的输入作为稳定输入来处理。这意味着编译器假定这些输入在内核活跃执行期间将保持不变。

在 KPN 上下文中，稳定存储器 I/O 应标记为 `stable`，如下示例所示。

```
void write_process(hls::stream<int>& in,
                  hls::stream<int>& out,
                  int* mem)
{
    #pragma HLS PIPELINE off
    int val;
    static int addr = 0;

    in.read(val);
    if (addr >= 32)
```

```

        addr = 0;
        //hls::print("writing %d\n", addr);
        mem[addr] = val;
        addr++;
        val = mem[addr-1];
        out.write(val);
    }
    ...
    ...
    ...
    void stable_pointer(int* mem,
        hls::stream<int>& in,
        hls::stream<int>& out)
    {
        #pragma HLS DATAFLOW
        #pragma HLS INTERFACE mode=s_axilite port=mem offset=30
        #pragma HLS INTERFACE mode=m_axi bundle=gmem depth=256
        max_read_burst_length=16 \
        max_widen_bitwidth=512 max_write_burst_length=16 num_read_outstanding=16 \
        num_write_outstanding=16 port=mem
        #pragma HLS stable variable=mem

        hls_thread_local hls::stream<int> int_fifo("int_fifo");
        #pragma HLS STREAM depth=512 type=fifo variable=int_fifo
        hls_thread_local hls::stream<int> int_fifo2("int_fifo2");
        #pragma HLS STREAM depth=512 type=fifo variable=int_fifo2

        hls_thread_local hls::task t1(process_23, in, int_fifo);
        hls_thread_local hls::task t2(process_11, int_fifo, int_fifo2);
        hls_thread_local hls::task t3(write_process, int_fifo2, out, mem);
    }

```

对于 C/RTL 协同仿真，需通过 `cosim.enable_tasks_with_m_axi` 命令为 `hls::task` 和 `M_AXI` 接口启用未同步的访问，如 [协同仿真配置](#) 中所述。

任务与通道

原 DATAFLOW 模型允许您写入顺序函数，然后要求 AMD Vitis™ HLS 工具识别数据流进程（任务）并使其并行执行、分析和依赖关系、执行标量传输以及优化（如阵列到串流）。或者，使用 `hls::task` 对象则要求您显式例化任务和通道，并在您的算法设计内自行管理并行度。`hls::task` 的用途是定义编程模型，该模型支持仅使用串流数据通道的并行任务。任务不受函数调用/返回的控制，但只要输入串流中存在数据就会运行任务。



提示： `hls::task` 库提供了并发语义，以使 C 语言仿真与 RTL 保持一致。这样即可消除顺序数据流模型中存在的部分问题。

任务与通道示例如下所示。您可以看到，其中仅使用串流接口（`hls::stream` 或 `hls::stream_of_blocks`）。您也可以看到，顶层函数使用 `hls_thread_local` 关键字来定义任务和串流通道。

```

void func1(hls::stream<int> &in, hls::stream<int> &out1, hls::stream<int>
&out2) {
    int data = in.read();
    if (data >= 10)
        out1.write(data);

```

```

    else
        out2.write(data);
    }
    void func2(hls::stream<int> &in, hls::stream<int> &out) {
        out.write(in.read() + 1);
    }
    void func3(hls::stream<int> &in, hls::stream<int> &out) {
        out.write(in.read() + 2);
    }
    void top_func(hls::stream<int> &in, hls::stream<int> &out1,
hls::stream<int> &out2) {
        hls_thread_local hls::stream<int> s1; // channel connecting t1 and t2
        hls_thread_local hls::stream<int> s2; // channel connecting t1 and t3

        hls_thread_local hls::task t1(func1, in, s1, s2); // t1 infinitely runs
        func1, with input in and outputs s1 and s2
        hls_thread_local hls::task t2(func2, s1, out1); // t2 infinitely runs
        func2, with input s1 and output out1
        hls_thread_local hls::task t3(func3, s2, out2); // t3 infinitely runs
        func3, with input s2 and output out2
    }

```

`hls::task` 对象为变量，这些变量应声明为 `hls_thread_local` 以便在上述示例中的例化函数 (`top_func`) 的多次调用之间，使变量和底层线程保持活动状态。任务对象会隐式管理连续运行函数的线程，例如以上示例中的 `func1`、`func2` 或 `func3`。该函数为任务主体，其周围具有隐含的无限循环。

对于每个 `hls::task`，必须为其传递一组实参，其中包含函数名称、输入和输出通道 `hls::streams` 或 `hls::stream_of_blocks`。`hls::task` 对象通常只能读写串流通道 `hls::stream` 和 `hls::stream_of_blocks`。

`hls::task` 和与之相连的通道都必须声明为 `hls_thread_local`。这些通道必须声明为 `hls_thread_local` 以便在顶层函数的多次调用之间使其保持活动状态。非串流数据（例如，标量和阵列变量）必须全部为任务函数的本地数据，不得作为实参来传递，下文《稳定的 M_AXI 和 S_AXILITE 访问》中另有声明除外。



重要提示！ 如在其中包含 `hls_task.h`，则会导致 `hls::stream` 和 `hls::stream_of_blocks` 读取调用在 C 语言仿真中发生阻塞。这意味着先前依赖于读取空串流的代码现在将在仿真期间导致死锁。

使用刷新流水线

总体上，`hls::task` 设计必须使用刷新流水线 (`flp`) 或自由运行的流水线 (`frp`)，如 [刷新流水线和流水线类型](#) 中所述。非刷新流水线会在进程执行之间引入依赖关系，这可能导致意外的死锁。

注释： 您可在 `hls::tasks` 中使用 `syn.compile.pipeline_flush_in_task` 来配置默认刷新行为，如 [编译选项](#) 中所述。

嵌套任务

在以下示例中，`task2` 中使用了 2 个 `task1` 的实例，这两个实例也例化为 `hls::task` 实例。这表示，`hls::task` 的主体不仅可作为顺序函数，也可作为仅包含 `hls::task` 对象的函数。

```

void task1(hls::stream<int> &in, hls::stream<int> &out) {
    hls_thread_local hls::stream<int> s1;

    hls_thread_local hls::task t1(func2, in, s1);
    hls_thread_local hls::task t2(func3, s1, out);
}

```

```
void task2(hls::stream<int> &in1, hls::stream<int> &in2, hls::stream<int>
&out1, hls::stream<int> &out2) {
    hls_thread_local hls::task tA(task1, in1, out1);
    hls_thread_local hls::task tB(task1, in2, out2);
}
```

`hls_thread_local` 的使用仍然是必需的，目的是为了确保中间网络 `tA` 和 `tB`（在此示例中均为 `task1` 的实例）的多次例化的安全性；此外还用于确保叶级进程 `t1`（位于用于执行 `func2` 的不同副本的 `tA` 和 `tB` 内）及 `t2`（`tA` 和 `tB` 内）的实例的安全性。

仿真与协同仿真

任务与通道模型的 C 语言仿真行为将与 C/RTL 协同仿真行为相同。原先允许读取空串流，仅显示警告以声明此状况可能在仿真期间导致挂起。在 Vitis HLS 2022.2 中，读取空串流可能导致死锁，即使在 C 语言仿真中也是如此，因此现在这属于错误，并显示如下消息：

- 在包含 `hls::task` 对象的设计中：

```
ERROR [HLS SIM]: deadlock detected when simulating hls::tasks.
Execute C-simulation in debug mode in the GUI and examine the source code
location of all the blocked hls::stream::read() calls
```

- 在不使用 `hls::task` 的设计中：

```
ERROR [HLS SIM]: an hls::stream is read while empty, which may result in
RTL simulation hanging. If this is not expected, execute C simulation in
debug mode
in the GUI and examine the source code location of the blocked
hls::stream::read()
call to debug. If this is expected, add
-DHLS_STREAM_READ_EMPTY_RETURNS_GARBAGE to
-cflags to turn this error into a warning and allow empty hls::stream
reads to return
the default value for the data type.
```



提示：给 `-cflags` 添加 `-DHLS_STREAM_READ_EMPTY_RETURNS_GARBAGE` 即可将此错误转为警告

任务与数据流

`hls::task` 还支持对数据流区域内部的任务进行定义。数据流区域允许将访问外部阵列的进程定义映射到来自设计层级上层的 `M_AXI`、标量或 `PIPO` 实参。这就需要由 `#pragma HLS dataflow` 语句来识别数据流进程，通过 `ap_ctrl_chain` 来同步这些进程，并由这些进程从任何非串流 C++ 数据结构读取数据，将数据作为 `hls::stream` 或 `hls::stream_of_blocks` 通道进行输出，以便连接到 `hls::tasks`。随后，任务即可输出由其他数据流进程所读取的串流，并将这些串流写入 `M_AXI`、标量或 `PIPO` 实参。必须先由进程或区域生成任务的输入串流，然后声明这些任务，最后由这些进程或区域耗用其输出串流。



重要提示！ 由于 `hls::task` 对象无法读取或写入 `M_AXI`、标量或 `PIPO` 实参，数据流进程必须在 `hls::tasks` 上读取或写入这些接口并写入或读取串流通道，如下所示。

以下示例演示了任务与数据流进程的结合。顶层函数 (top-func) 是数据流区域，可定义顺序函数 write_out() 和 read_in()，以及 hls::task 对象和 hls::stream 通道。

```
#include "hls_task.h"

// This is an I/O dataflow process
void write_out(int* out, int n, hls::stream<int> &s2) {
    for (int i=0; i<n; i++)
        out[i] = s2.read();
}
// This is an I/O dataflow process
void read_in(int* in, int n, hls::stream<int> &s1) {
    for (int i=0; i<n; i++)
        s1.write(in[i]);
}
// This is an hls::task body
void func1(hls::stream<int> &s1, hls::stream<int> &s3) {
    // No while(1) needed! This will be a task
    s3.write(... + s1.read());
}
// This is an hls::task body
void func2(hls::stream<int> &s3, hls::stream<int> &s2) {
    // No while(1) needed! This will be a task
    s2.write(... * s3.read());
}
// This could legally be at the top of the design hierarchy
void top-func(int *in, int *out, int n) {
#pragma HLS dataflow
    hls_thread_local hls::stream<int> sk3;
    hls_thread_local hls::stream<int> sk1;
    hls_thread_local hls::stream<int> sk2;

    read_in(in, n, sk1); // can access stream,
    scalar or array; calling order matters
    hls_thread_local hls::task t2(func2, sk3, sk2); // can access only
    stream; instance order does not matter
    hls_thread_local hls::task t1(func1, sk1, sk3); // can access only
    stream; instance order does not matter
    write_out(out, n, sk2); // can access stream,
    scalar or array; calling order matters
}
```

#pragma HLS DATAFLOW 对于这两个顺序函数而言是必需的，但 hls::task 对象并不需要它。Vitis HLS 将在内部自动拆分 top-func，包括将常规数据流进程和 hls::task 进程拆分为两个数据流区域：

1. 其中一个数据流区域使用的是 ap_ctrl_chain，其中包含常规数据流进程（如 read_in() 和 write_out()）且顺序与这些进程在 C++ 代码中显示的顺序相同，此外还包含对以下 ap_ctrl_none 区域的调用
2. 另一个数据流区域使用的是 ap_ctrl_none，其中包含任务与通道。

因此，您可在 Vitis HLS GUI 的“Dataflow Viewer”（数据流查看器）中看到两个层次构成的层级。

HLS 拆分/合并库



重要提示! 要在代码中使用 `hls::split<>` 或 `hls::merge<>` 对象，请按如下示例所示方式包含头文件 `hls_np_channel.h`

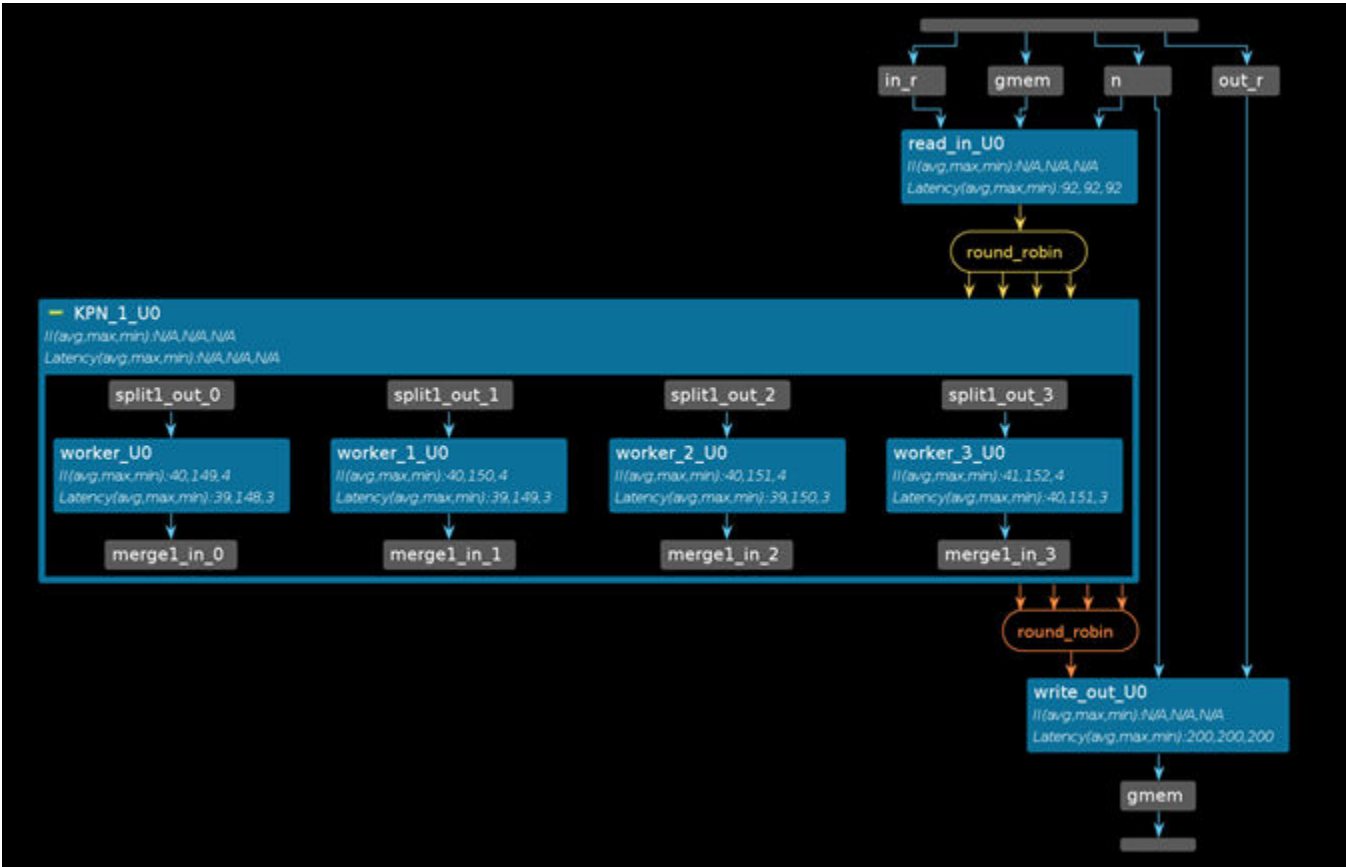
拆分/合并通道可供在数据流进程内使用，支持您创建一对多或多对一类型的通道，用于将数据分发给多项任务，或者聚集来自多项任务的数据。这些通道具有内置作业调度器，使用循环方法按顺序跨通道分发或收集数据，或者采用基于通道可用性来判定的负载均衡方法。



提示: 负载均衡可能导致 RTL/协同仿真内出现非确定性结果。在此情况下，您将需要重写测试平台，使其不受结果的顺序影响。

如下图所示，数据读取自输入串流，通过循环调度器机制进行拆分，并分发至关联的工作程序任务。当工作程序完成任务后，同样会采用循环调度器将合并的输出写入单一串流。

图 145：拆分/合并数据流



拆分通道具有单一生产者和众多使用者，通常可用于将任务分发给一系列工作程序，在 RTL 中将分发逻辑加以抽象化并实现，从而提升性能并减少耗用的资源。从 N 个输出中的任一输出分发输入的操作可采用：

- 循环法，其中使用者按固定轮换顺序读取输入数据，这样即可确保确定性行为，但不允许负载共享，且工作程序间存在动态可变的计算负载。
- 负载均衡，其中尝试执行读取的首个使用者将读取首个输入数据，这样可确保良好的负载均衡，但存在非确定性结果。

每条合并通道都具有多个生产者和单一使用者，基于相反逻辑来工作：

- 循环法，其中按固定轮换顺序合并生产者输出数据，这样即可确保确定性行为，但不允许负载共享，且工作程序间存在动态可变的计算负载。
- 负载均衡合并通道，其中完成工作的首个生产者将首先写入通道，这存在非确定性结果。

拆分与合并的一般构想是借助 round_robin（循环）调度器，以确定性方式将数据分发给周围工作程序以供拆分，从工作程序读取数据以供合并。因此，如果所有工作程序都计算相同函数，那么结果与使用单一工作程序的计算结果相同，但性能更好。

如果工作程序执行不同的函数，那么设计必须确保按工作程序循环方式（分别从 out[0] 或 in[0] 开始），将正确的数据项发送到正确的函数。

规格

拆分/合并通道规范如下：

```
hls::split::load_balancing<DATATYPE, NUM_PORTS[, DEPTH, N_PORT_DEPTH]>
name;
hls::split::round_robin<DATATYPE, NUM_PORTS[, DEPTH]> name
hls::merge::load_balancing<DATATYPE, NUM_PORTS[, DEPTH]> name
hls::merge::round_robin<DATATYPE, NUM_PORTS[, DEPTH]> name
```

其中：

- round_robin/load_balancing：指定用于该通道的调度器机制类型。
- DATATYPE：指定通道上的数据类型。这其中存在与标准 `hls::stream` 相同的限制。DATATYPE 可设为：
 - 任何 C++ 原生数据类型
 - Vitis HLS 任意精度类型（例如，`ap_int<>` 和 `ap_ufixed<>`）
 - 用户定义的结构体（包含以上任意类型）
- NUM_PORTS：表示拆分操作所需的写入端口数量 (`1:num`) 或者合并操作所需的读取端口数量 (`num:1`)。
- DEPTH：可选实参，表示主缓冲器的深度，位于拆分之前或合并之后。此项可选，如不指定，则默认深度为 2。
- N_PORT_DEPTH：可选字段，供循环法用于指定在拆分之后或合并之前应用的输出缓冲器的深度。此项可选，如不指定，则默认深度为 0。



提示：要指定可选 `N_PORT_DEPTH` 值，还必须指定 `DEPTH`。

- name：指定创建的通道对象的名称

GitHub 上的 [mixed_control_and_data_driven](#) 提供了如下所示示例：

```
#include "hls_np_channel.h"

const int N = 16;
const int NP = 4;

void dut(int in[N], int out[N], int n) {
#pragma HLS dataflow
    hls::split::round_robin<int, NP> split1;
    hls::merge::round_robin<int, NP> merge1;

    read_in(in, n, split1.in);

    // Task-Channels
    hls_thread_local hls::task t[NP];
    for (int i=0; i<NP; i++) {
#pragma HLS unroll
        t[i](worker, split1.out[i], merge1.in[i]);
    }

    write_out(merge1.out, out, n);
}
```



提示：以上示例显示了作为 `hls::task` 对象实现的工作程序。但这只是示例的一项特征，而非 `split/merge` 通道的要求。

拆分/合并的应用

拆分与合并的主要用途是支持多次计算引擎例化，以便充分利用 DDR 或 HBM 端口的带宽。在此情况下，生产者负载进程，从 MAXI 读取数据突发，然后通过拆分通道将各个待处理的数据包发送到多个工作程序。如果工作程序需耗费大量时间，请使用循环协议，或者如果输入可变，则根据输入对执行时间进行负载均衡。使用者则执行相反操作，将数据写回 DRAM。



提示：如果使用负载均衡，那么写回地址可随数据一起通过拆分与合并进行传递。

在拆分通道或合并通道两端，通过实现 `hls::stream` 对象的方式对这些通道进行建模。这意味着可将拆分或合并通道端连接到取 `hls::stream` 作为输入或输出的任何进程。此进程无需知晓通道连接的类型。因此，这两种类型的连接均可用于标准数据流或 `hls::task` 对象。

以下示例显示了单一生产者和多个使用者使用拆分的方式：

```
#include "hls_np_channel.h"

void producer(hls::stream<int> &s) {
    s.write(xxx);
}

void consumer1(hls::stream<int> &s) {
    ... = s.read();
}

void consumer2(hls::stream<int> &s) {
    ... = s.read();
}

void top-func() {
#pragma HLS dataflow
```

```
hls::split::load_balancing<int, 4, 6> s; // NUM_PORTS=4, DEPTH=6

producer(s.in, ...);
consumer1(s.out[0], ...);
consumer2(s.out[1], ...);
consumer3(s.out[2], ...);
consumer4(s.out[3], ...);
}
```

HLS 块串流库



重要提示! 要在代码中使用 `hls::stream_of_blocks` 对象，请包含头文件 `hls_streamofblocks.h`。

`hls::stream_of_blocks` 类型可提供用户同步的串流，支持对数据块进行串流以供数据流上下文中的进程级别接口使用，在此类上下文中每个块都是单个阵列或多维阵列。块串流的目标用途是替换数据流区域内的成对进程之间的基于阵列的通信。请参阅 Github 上的 [using_stream_of_blocks](#) 示例。

当前，AMD Vitis™ HLS 可在数据流区域内实现由生产者进程写入并由使用者进程读取的阵列，方法是将这些阵列映射到乒乓缓冲器 (PIPO)。在 C++ 中返回生产者函数和调用使用者函数时，PIPO 缓冲器就会发生缓冲器交换。

虽然这样可以确保并发通信语义与顺序 C++ 执行语义完全兼容，但也暗示只有在生产者完成后，使用者才能启动，如下代码示例所示。

```
void producer (int b[M][N], ...) {
    for (int i = 0; i < M; i++)
        for (int j = 0; j < N; j++)
            b[i][f(j)] = ...;
}

void consumer(int b[M][N], ...) {
    for (int i = 0; i < M; i++)
        for (int j = 0; j < N; j++)
            ... = b[i][g(j)] ...;
}

void top(...) {
    #pragma HLS dataflow
    int b[M][N];
    #pragma HLS stream off variable=b

    producer(b, ...);
    consumer(b, ...);
}
```

这样可能给吞吐量造成不必要的限制和/或增加资源，例如，如果生产者在一个嵌套循环内写入一行缓冲器输出，生成较小的数据块以供使用者使用，而使用者则以在嵌套循环内读取一行缓冲器输入的方式来使用较小的数据块，如以上示例所示。在此示例中，由于内层循环内发生非顺序缓冲器列访问，因此无法直接串流阵列 `b`。但外层循环内的行访问为顺序访问，因而支持 `hls::stream_of_blocks` 通信，并且其中每个块均为一维阵列（大小为 `N`）。

`hls::stream_of_blocks` 功能特性的主要用途是提供类 PIPO 功能，但具有用户管理的显式同步、访问和更好的编码样式。块串流使您得以避免在包含生产者和使用者的循环内使用数据流，使用数据流本可能对以上示例加以优化。但在此例中，使用包含生产者和使用者的数据流循环需要使用 PIPO 缓冲器 ($2 \times N$)，如下示例所示：

```
void producer (int b[N], ...) {
    for (int j = 0; j < N; j++)
        b[f(j)] = ...;
}

void consumer(int b[N], ...) {
```

```

    for (int j = 0; j < N; j++)
        ... = b[g(j)];
}

void top(...) {
// The loop below is very constrained in terms of how it must be written
    for (int i = 0; i < M; i++) {
#pragma HLS dataflow
        int b[N];
#pragma HLS stream off variable=b

        producer(b, ...); // writes b
        consumer(b, ...); // reads b
    }
}

```

以上循环内数据流代码并不推荐，因为此结构在 Vitis HLS 中存在诸多限制，例如，循环结构必须高度约束（单个归纳变量，从 0 开始并与常量或函数实参进行比较，按 1 递增）。

块串流建模样式

从另一方面来看，对于块串流，生产者与使用者之间的通信以类阵列对象串流方式来建模，这样与通过 PIPO 进行阵列传输相比存在多方面优势。

在代码中使用块串流需要以下 include 文件：

```
#include "hls_streamofblocks.h"
```

块串流对象模板为：`hls::stream_of_blocks<block_type, depth> v`

其中：

- `<block_type>` 用于指定块串流所包含的阵列或多位阵列的数据类型
- `<depth>` 是可选实参，可提供与 `hls::stream` 或 PIPO 相似的深度控制，并指定块的总数，包括在任意时间生产者获取的块和使用者获取的块。默认值为 2
- `v` 为块串流对象指定变量名称

使用以下步骤访问块串流中的某一个块：

1. 如果生产者或使用者进程要首先访问串流，则需要使用 `hls::write_lock` 或 `hls::read_lock` 对象获取对该串流的访问权。
2. 当生产者获取锁定后，它即可开始写入（或读取）获取的块。此块完全初始化后，当 `write_lock` 对象超出范围时，生产者即可将此块释放。

注释：具有 `write_lock` 的生产者进程也可以读取此块，前提是它从已写入的位置读取，因为新获取的块必须假定包含未初始化的数据。仅限生产者进程才能对块进行写入和读取，不支持使用者执行此操作。

3. 随后，此块将在块串流中以 FIFO 方式排队，当使用者获取 `read_lock` 对象后，此块即可供使用者进程读取。

前述示例中所示的 `hls::stream_of_blocks` 与 PIPO 机制之间的主要差异在于一旦 `write_lock` 超出范围，块就会变为对使用者可用，而不是仅在生产者进程返回时才可用。因此，通过块串流方式管理原示例（不含数据流循环）所需的存储器大小比仅使用 PIPO 要小得多：在该示例中，为 $2N$ 而不是 $2 \times M \times N$ 。

以下示例显示了如何重写前述示例以使用 `hls::stream_of_blocks`。生产者通过构造名为 `b` 的 `hls::write_lock` 对象，并将名为 `s` 的块串流对象引用传递给该对象，即可获取此块。`write_lock` 对象可提供重载阵列访问运算符，这样即可像阵列一样访问此对象，以便按随机顺序访问底层存储器，如以下示例所示。

通过构造 `write_lock/read_lock` 对象即可执行锁定的获取，当对象超出范围时，就会被销毁，从而自动释放锁定。此方法使用常用的“资源获取即初始化” (RAII) 样式的锁定和解锁。

```
#include "hls_streamofblocks.h"
typedef int buf[N];
void producer (hls::stream_of_blocks<buf> &s, ...) {
    for (int i = 0; i < M; i++) {
        // Allocation of hls::write_lock acquires the block for the producer
        hls::write_lock<buf> b(s);
        for (int j = 0; j < N; j++)
            b[f(j)] = ...;
        // Deallocation of hls::write_lock releases the block for the consumer
    }
}

void consumer(hls::stream_of_blocks<buf> &s, ...) {
    for (int i = 0; i < M; i++) {
        // Allocation of hls::read_lock acquires the block for the consumer
        hls::read_lock<buf> b(s);
        for (int j = 0; j < N; j++)
            ... = b[g(j)] ...;
        // Deallocation of hls::write_lock releases the block to be reused by
        the producer
    }
}

void top(...) {
    #pragma HLS dataflow
    hls::stream_of_blocks<buf> s;

    producer(b, ...);
    consumer(b, ...);
}
```

此方法的关键特征包括：

- 以上生产者的外层循环的目标性能是达成总体启动时间间隔 (II) 为 1 的目标
- 锁定的块虽可使用，但它仅供生产者或使用用户进程专用，直至被释放为止。
- 生产者的阵列对象的初始状态并未定义，但它包含由生产者为用户写入的值。
- 块串流的主要优势在于允许重叠执行使用者和生产者的多次迭代，以提升吞吐量。

资源使用

当深度增加超过默认值 2 时，资源成本与 PIPO 的资源成本类似。换言之，每递增 1，都将需要足够一个块使用的存储器，例如， $N * 32$ 位字。

块串流对象可绑定到特定 RAM 类型，方法是在声明块串流的位置放置 `BIND_STORAGE` 编译指示，例如，在顶层函数内。默认情况下，块串流使用 2 端口块 RAM (`type=RAM_2P`)。

检查满的块和空的块

`read_lock` 和 `write_lock` 与 `while(empty)` 或 `while (full)` 循环类似，会持续不断尝试获取资源，直至得到资源为止，因此代码执行将停滞直至获取锁定为止。您可以使用 `empty()` 和 `full()` 方法来判定缺少可供获取的可用块是否会导致对 `read_lock` 或 `write_lock` 执行的调用因此停滞，如下示例所示。

```
#include "hls_streamofblocks.h"

void reader(hls::stream_of_blocks<buf> &in1, hls::stream_of_blocks<buf>
&in2, int out[M][N], int c) {
    for(unsigned j = 0; j < M; j++) {
        if (!in1.empty()) {
            hls::read_lock<ppbuf> arr1(in1);
            for(unsigned i = 0; i < N; ++i) {
                out[j][i] = arr1[N-1-i];
            }
            j++;
        } else if (!in2.empty()) {
            hls::read_lock<ppbuf> arr2(in2);
            for(unsigned i = 0; i < N; ++i) {
                out[j][i] = arr2[N-1-i];
            }
            j++;
        }
    }
}

void writer(hls::stream_of_blocks<buf> &out1, hls::stream_of_blocks<buf>
&out2, int in[M][N], int d) {
    for(unsigned j = 0; j < M; ++j) {
        if (d < 2) {
            if (!out1.full()) {
                hls::write_lock<ppbuf> arr(out1);
                for(unsigned i = 0; i < N; ++i) {
                    arr[N-1-i] = in[j][i];
                }
            } else {
                if (!out2.full()) {
                    hls::write_lock<ppbuf> arr(out2);
                    for(unsigned i = 0; i < N; ++i) {
                        arr[N-1-i] = in[j][i];
                    }
                }
            }
        }
    }
}

void top(int in[M][N], int out[M][N], int c, int d) {
#pragma HLS dataflow
    hls::stream_of_blocks<buf, 3> strm1, strm2; // Depth=3
    writer(strm1, strm2, in, d);
    reader(strm1, strm2, out, c);
}
```

生产者进程和使用者进程可在其主体的任一范围内执行以下操作。如各示例所示，范围通常为循环，但这并非强制要求。诸如条件等其他范围也同样受支持。受支持操作包括：

- 获取某个块，例如，一个含任意受支持的数据类型的阵列。
 - 对于生产者，此阵列将为空，即，根据底层数据类型的构造函数（如有）进行初始化。
 - 对于使用者，此阵列将为满（当然不超过生产者对其填充的量，与对于 PIPO 缓冲器的要求相同，称为按需适用完整写入）。
- 将此块视作等同于本地存储器用于读取和写入，端口数量上限不超过根据 `BIND_STORAGE` 编译指示或指令所分配的最大端口数，此编译指示或指令是专为块的串流指定的，用于指定每一侧各自可看到哪些端口：
 - 1 个端口表示每一侧均只能访问 1 个端口，最终块串流可将单个双端口存储器用于实现。
 - 2 个端口表示每一侧根据调度可使用 1 个或 2 个端口：
 - 如果调度器在至少一侧使用 2 个端口，则将不会发生合并
 - 如果调度器使用 1 个端口，则可能发生合并
 - 如果不指定该编译指示，则调度器将根据当前用于局部阵列的条件来决定。此外：
 - 生产者可对其获取的块执行写入和读取
 - 使用者则只能对其获取的块执行读取
- 当块从获取此块的范围内退出时，将自动释放。释放后的块：
 - 如果由生产者释放块，则此块可供使用者获取。
 - 如果由使用者释放块，则此块可供生产者获取并复用，前提是由构造函数（如有）对此块进行重新初始化。此初始化可能导致设计变慢，因此通常不建议这样做。您可使用 `__no_ctor__` 属性（如前文 `std::complex` 所述）防止为阵列元素调用构造函数。

块串流与 PIPO 缓冲器精神上极为相似。对于 PIPO，获取与调用生产者或使用者进程函数相同，而释放则与从该函数返回相同。这意味着：

- PIPO 的握手符合以下描述
 - 在使用者一侧为 `ap_start/ap_ready`，且
 - 在生产者一侧为 `ap_done/ap_continue`。
- 块串流的握手则符合以下描述
 - 在生产者有其自己的 `read/empty_n`，且
 - 在使用者一侧有 `write/full_n`。

数据流区域内的建模反馈

PIPO 缓冲器的主要限制之一是只能按 C++ 中函数调用顺序正向流动。换言之，PIPO 不支持以下连接，但 `hls::stream_of_blocks` 可支持此连接：

```
void top(...) {
    int b[N];
    for (int i = 0; i < M; i++) {
#pragma HLS dataflow
#pragma HLS stream off variable=b
        consumer(b, ...); // reads b
        producer(b, ...); // writes b
    }
}
```

以下代码示例专用于演示概念：

```
#include "hls_streamofblocks.h"
typedef int buf[N];
void producer (hls::stream_of_blocks<buf> &out, ...) {
    for (int i = 0; i < M; i++) {
        hls::write_lock<buf> arr(out);
        for (int j = 0; j < N; j++)
            arr[f(j)] = ...;
    }
}

void consumer(hls::stream_of_blocks<buf> &in, ...) {
    if (in.empty()) // execute only when producer has already generated some
        meaningful data
        return;

    for (int i = 0; i < M; i++) {
        hls::read_lock<buf> arr(in);
        for (int j = 0; j < N; j++)
            ... = arr[g(j)];
        ...
    }
}

void top(...) {
    // Note the large non-default depth.
    // The producer must complete execution before the consumer can start
    // again, due to ap_ctrl_chain.
    // A smaller depth would require ap_ctrl_none
    hls::stream_of_blocks<buf, M+2> backward;

    for (int i = 0; i < M; i++) {
#pragma HLS dataflow
        consumer(backward, ...); // reads backward
        producer(backward, ...); // writes backward
    }
}
```

限制

使用 `hls::stream_of_blocks` 时有些限制需要注意：

- 每个 `hls::stream_of_blocks` 对象都必须具有单一的生产者和使用者进程，每个进程都必须不同。换言之，不支持单一进程内的本地块串流。
- 在顺序区域内，不能使用 `hls::stream_of_blocks`。在数据流区域内，生产者与使用者必须为独立的并发进程。
- 例如，在同一作用域或嵌套作用域内，不能使用多个嵌套的获取/释放语句 (`write_lock/read_lock`)，如以下示例所示：

```
using ppbuf = int[N];
void readerImplicitNested(hls::stream_of_blocks<ppbuf>& in, ...) {
    for(unsigned j = 0; j < M; ++j) {
        hls::read_lock<ppbuf> arrA(in); // constructor would acquire A
first
        hls::read_lock<ppbuf> arrB(in); // constructor would acquire B
second
        for(unsigned i = 0; i < N; ++i)
            ... = arrA[f(i)] + arrB[g(i)];
        // destructor would release B first
        // destructor would release A second
    }
}
```

但您可以使用多个顺序或互斥的获取/释放语句 (`write_lock/read_lock`)，例如在 IF/ELSE 分支内或者在 2 个后续代码块中。如下示例所示：

```
void readerImplicitNested(hls::stream_of_blocks<ppbuf>& in, ...) {
    for(unsigned j = 0; j < M; ++j) {
        {
            hls::read_lock<ppbuf> arrA(in); // constructor acquires A
            for(unsigned i = 0; i < N; ++i)
                ... = arrA[f(i)];
            // destructor releases A
        }
        {
            hls::read_lock<ppbuf> arrB(in); // constructor acquires B
            for(unsigned i = 0; i < N; ++i)
                ... = arrB[g(i)];
            // destructor releases B
        }
    }
}
```

- 不建议显式释放生产者和使用者进程中的锁定，因为这些锁定超出作用域之后就会自动释放。但您可通过在源代码中的 `#include "hls_streamofblocks.h` 前添加 `#define EXPLICIT_ACQUIRE_RELEASE` 来使用这些锁定。

HLS IP 库

AMD Vitis™ HLS 可提供 C++ 语言库，以实现多个 AMD IP 块。C 语言库允许从 C++ 源代码直接推断以下 AMD IP 块，以确保 FPGA 内的高质量实现。

表 55: HLS IP 库

库头文件	描述
hls_dsp_builtins.h	提供一组 C++ 本征函数和类，用于对 AMD DSP slice 原语的整数函数进行仿真并直接实现。
hls_fft.h	允许在 C 语言中对 AMD LogiCORE IP FFT 进行仿真，并使用 AMD LogiCORE 块来实现。
hls_fir.h	允许在 C 语言中对 AMD LogiCORE IP FIR 进行仿真，并使用 AMD LogiCORE 块来实现。
hls_dds.h	允许在 C 语言中对 AMD LogiCORE IP DDS 进行仿真，并使用 AMD LogiCORE 块来实现。
ap_shift_reg.h	提供 C++ 类用于实现移位寄存器，该移位寄存器是使用 AMD SRL 原语直接实现的。

DSP 本征函数

VHLS 编译器会将整数算术自动映射到 DSP 资源，但有经验且熟悉 DSP48E1、DSP48E2 或 DSP58 单元内部架构的设计师现在可以使用 DSP 本征函数来创建 DSP 实例，其中包含：

- 用户通过本征函数名指定的操作代码，例如，`add_mul_add` 或 `mul_add`（请注意特定名称空间内包含的操作代码，详见后文）。
- 通过本征函数的模板实参给定的一组寄存器，用于将 DSP 寄存器名称编码为一组值并通过 OR 运算来组合在一起以获取多个寄存器。
- 给定 DSP 级联端口可供使用，前提是底层布局工具允许使用这些端口。

具体方法是包含 `hls_dsp_builtins.h` 并使用如下所述的函数和类。

当前不支持映射到 DSP 的简单加法和乘法运算，因为在同样情况下可以通过 `bind_op` 来定期自动完成 DSP 自动推断和时延规格。

使用 DSP 本征函数

DSP 本征函数可以使用本征函数实现各种运算符、寄存器配置和级联连接。当使用级联或累加器时，这些对应操作代码都是使用类来实现的。下文对此进行了详细探讨，并提供了受支持的操作代码及其函数或类的列表。

模板实参定义在 DSP 内使用的寄存器。

这些函数可在用户代码中调用，并遵循 C++ 和 `ap_int` 转换规则在函数与用户代码变量之间进行适当的双向数据类型转换。

例如，DSP：

- 计算 add-mul-add 函数 ($P = (A + D) * B + C$).
- 时延为 2，因为使用 A1 (A1 是 DSP 的 A 输入上的两个寄存器之一) 和 P 寄存器 (这些寄存器是使用 REG_A1 | REG_P 作为模板实参来指定的)。
- 使用 DSP58 位宽 (位宽是使用名称空间 hls::dsp58; 指定的)，例化方式如下：

```
#include "hls_dsp_builtins.h"
using namespace hls::dsp58;
...
long test(long d, long a, long b, long c)
{
    return add_mul_add<REG_A1 | REG_P>(d, a, b, c);
}
```

注释：由于 Versal 之前的平台不支持 58 位 DSP，因此尝试对此类平台使用以上设计会导致错误。

mul_add	$P = A * B + C$	template<int64_t flags> C_t mul_add(A_t a, B_t b, C_t c);
mul_sub	$P = A * B - C$	template<int64_t flags> C_t mul_sub(A_t a, B_t b, C_t c);
mul_rev_sub	$P = C - A * B$	template<int64_t flags> C_t mul_rev_sub(A_t a, B_t b, C_t c);
add_mul_add	$P = (A + D) * B + C$	template<int64_t flags> C_t add_mul_add(D_t d, A_t a, B_t b, C_t c);
add_mul_sub	$P = (A + D) * B - C$	template<int64_t flags> C_t add_mul_sub(D_t d, A_t a, B_t b, C_t c);
add_mul_rev_sub	$P = C - (A + D) * B$	template<int64_t flags> C_t add_mul_rev_sub(D_t d, A_t a, B_t b, C_t c);
sub_mul_add	$P = (A - D) * B + C$	template<int64_t flags> C_t sub_mul_add(D_t d, A_t a, B_t b, C_t c);
sub_mul_sub	$P = (A - D) * B - C$	template<int64_t flags> C_t sub_mul_sub(D_t d, A_t a, B_t b, C_t c);
sub_mul_rev_sub	$P = C - (A - D) * B$	template<int64_t flags> C_t sub_mul_rev_sub(D_t d, A_t a, B_t b, C_t c);

该列表所含寄存器可用于将每个本征函数模板配置为在 RTL 模型中使用对应的寄存器：

- 输入 A 上的 REG_A1 和 REG_A2
- 输入 B 上的 REG_B1 和 REG_B2
- REG_C
- 输入 D 上的 REG_D
- 前加法器输出端的 REG_AD
- 乘法器输出端的 REG_M
- 后加法器输出上的 REG_P (也用作累加器)

上述函数的实参所使用的数据类型因平台而异。以下列出了头文件中定义的不同 typedef (因平台而异) 以便参考。

要指定目标 DSP 变体的具体版本，可在 C++ 源代码中的 #include "hls_dsp_builtins.h" 之后分别写入：using hls::dsp48e1; using hls::dsp48e2; using hls::dsp58;。

表 57：定义的数据类型

数据类型	DSP48E1	DSP48E2	DSP58
A_t	ap_int<25>	ap_int<27>	ap_int<27>
B_t	ap_int<18>	ap_int<18>	ap_int<24>

表 57: 定义的数据类型 (续)

数据类型	DSP48E1	DSP48E2	DSP58
C_t	ap_int<48>	ap_int<48>	ap_int<58>
D_t	ap_int<25>	ap_int<27>	ap_int<27>

无符号输入

DSP48 和 DSP58 都包含“有符号的乘法器”。这暗示使用 DSP 本征函数乘以无符号值时，“必须将 MSB 设为 0 来显式创建这些无符号值”，否则，MSB 设为 1 的无符号输入会被解读为负值。最简单的机制是显式使用少一位的无符号类型。

例如：

```
#include "hls_dsp_builtins.h"
using namespace hls::dsp58;
...
ap_uint<57> test(ap_uint<26> d, ap_uint<26> a, ap_uint<23> b, ap_uint<57>
c) {
    return add_mul_add<...>((ap_uint<27>(d), ap_uint<27>(a),
ap_uint<24>(b), ap_uint<58>(c));
}
```

累加

通过 `hls::dsp(48e1|48e2|58)::acc<>` 类来支持 REG_P 上的累加功能。

- 该功能可以例化以推断不同的 DSP 累加器实例（以下示例中仅提供一个此类实例）。
- 该功能具有两种方法：`mul_acc` ($P = A * B + P$) 和 `add_mul_acc` ($P = (A + D) * B + P$)。
 - 执行累加并返回其结果（即，寄存器 P）。
 - 如果初始化条件为 `true`（在下例中，迭代变量为 0 时），将寄存器 P 复位为 0。

<code>mul_acc</code>	<code>P += A * B</code>	<code>template <int64_t flags> C_t mul_acc(A_t a, B_t b, bool init);</code>
<code>add_mul_acc</code>	<code>P += (A + D) * B</code>	<code>template <int64_t flags> C_t mul_acc(D_t D, A_t a, B_t b, bool init);</code>

这是一个简单的 FIR 滤波示例，其中使用名称空间 `hls::dsp48e2` 来公开 `acc<>` 类以及 `REG_P` 和 `REG_M` 符号：

```
#include "hls_dsp_builtins.h"
using namespace hls::dsp48e2;
...
long test_mul_acc(long delay[N], long coeff[N])
{
    acc<REG_P|REG_M> my_acc;
    C_t res;
    for (int i=0; i<N; i++) {
        res = my_acc.mul_acc(delay[i], coeff[i], /* init condition */ i==0);
    }
    return res;
}
```

DSP 级联

只要布局允许，也可以生成 RTL 以供 Vivado 用于通过直接 ACIN、BCIN 和 PCIN 输入以及 ACOUT、BCOUT 和 PCOUT 输出来级联 DSP

这可通过例化 `hls::dsp(48e1|48e2|58)::cascade` 类来达成。

该类有个名为 `mul_add` 的公开方法（未来会添加更多方法）可用于对来自上一阶段的级联输入执行所需操作，并返回级联输出。

返回结构体

```
struct R_t {
    A_t acout;
    B_t bcout;
    C_t pcout;
};
R_t mul_add(A_t a, B_t b, C_t c);
```

`mul_add` 会生成一个结构体，举例来说，这个结构体允许设计师将 DSP 从 BCOUT 输出端口链接到 BCIN 输入端口，如此处示例所示，此示例对含有 `size` 抽头并使用脉动级联架构的模板化 FIR 进行建模。

```
#include "hls_dsp_builtins.h"
using namespace hls::dsp48e2;
...
template <int size>
class FIR {
private:
    const long* coeff_;
    long bias_;
    cascade<REG_A1 | REG_B1 | REG_M | REG_P > dsp0;
    cascade<REG_A1 | REG_B1 | REG_B2 | REG_M | REG_P > dspN[size - 1];

public:
    FIR(const long* coeff, long bias)
        : coeff_(coeff), bias_(bias) {
#pragma HLS ARRAY_PARTITION variable=dspN complete dim=1
    };

    long fir(B_t input){

        auto out = dsp0.mul_add(coeff_[0], input, bias_);
        for(int j=1; j < size ; j++){
#pragma HLS unroll
            out = dspN[j - 1].mul_add(coeff_[j], out.bcout, out.pcout);
        }
        return out.pcout;
    };
};

void test_cascaded_systolic_fir(long b[N], long c, long hw[N])
{
    long a[4] = {1, 12, 3, 4}; // coefficients
#pragma HLS ARRAY_PARTITION variable=a complete dim=1
#pragma HLS ARRAY_PARTITION variable=hw complete dim=1
    FIR<4> my_fir(a, c);
```

```

LOOP_FIR:
for(int i = 0 ; i< N ; i++){
    hw[i] = my_fir.fir(b[i]);
}
}

```

请注意以下细节：

- 第一阶段级联和其余阶段都需要两种不同配置的级联 DSP。
- 将 out.bcout 和 out.pcout 作为每个 mul_add 调用的 b 输入和 c 输入进行传递即可允许 Vivado 使用下一个 FIR 阶段的 bcin 输入和 pcin 输入。
- dsp(48e1|48e2|58)::cascade 对象的阵列必须完全分区，以便使用 array_partition 编译指示创建与 size 无关的 DSP 实例。
- Vitis HLS 会从以上代码生成可综合的 RTL，这表示：
 - Vivado 仅在布局时选择级联，前提是对应 DSP 单元彼此间布局距离足够近。
 - 用户应始终检查 Vivado 报告，查看是否发生级联，并使用 Vivado 来判定无法发生级联的原因（例如，没有足够的临近 DSP 资源）。
- 为了保证 Vivado 中完成正确级联，必须始终使用以下寄存器。
 - REG_A1 和/或 REG_A2
 - REG_B1 和/或 REG_B2
 - REG_P
- 对于要使用的 mul_add 和 mul_sub。
 - 不得使用 REG_AD 和 REG_D（在未来版本中可能撤销此限制）。
 - 在级联版本中不能使用 REG_C（在未来版本中可能撤销此限制）。

DSP 复数本征函数

仅限在 Versal 上，hls::dspcplx 名称空间内启用了一组 DSP 本征函数来执行复数类型的运算，如，乘、乘累加和累加，例如，通过使用 DSP CPLX 原语 2 DSP58 来执行 18x18 复数类型乘法。DSP 复数本征函数使用下列数据类型：

- std::complex<ap_int<18>> 用于乘法器输入，定义为 typedef A_t & B_t
- std::complex<ap_int<58>> 定义为 typedef C_t，用于：
 - 加法器输入（用于 mul_add）
 - 乘法器输出和累加器输出（用于 mul_acc）

仅使用 2 个 DSP58，而不是 3 个。由此导致在正常运作的 C++ 代码中使用 std::complex<ap_int<18>> 时可以显著节省资源。

可用函数包括 mul 和 mul_add，acc 类和 cascade 类可分别用于累加和级联，与整数数据类型的本征函数相似（包括用于捕获 Acout、Bcout 和 Pcout 的级联输出的 struct R_t）。

表 59: DSP 复数本征函数

函数	运算	签名
mul	$P=A*B$	template <int64_t flags> C_t mul_add(A_t a, B_t b)
mul_add	$P=A*B+C$	template <int64_t flags> C_t mul_add(A_t a, B_t b, C_t c)
mul_acc	$C=init?A*B:A*B+C$	template <int64_t flags> class acc { C_t mul_acc(A_t a, B_t b, bool init);}
cascade 和 mul_add	$P=A*B+C$	template <int64_t flags> class cascade { R_t mul_add(A_t a, B_t b, C_t c);}

以下代码示例显示了其用法：

- 对于寄存器标记的值，设计师可以自由选择仅使用 M 寄存器和 P 寄存器，而 A、B、C 和 AD 寄存器仅支持 3 种配置，称为
 - NO_PIPELINE（不含 REG_A1、REG_A2、REG_B1、REG_B2、REG_AD、REG_D 或 REG_C）
 - BALANCED_PIPELINE（REG_A1 和 REG_B1）
 - FULL_PIPELINE（REG_A1、REG_A2、REG_B1、REG_B2、REG_AD）
 - 以上 3 个符号在名称空间内的定义方式与其他 REG_xx 符号相似
- 这些示例显示了实数成对输入和输出的使用（例如，ap_int<18>）以及复数成对输入和输出的使用（例如，hls::dspcplx::A_t 作为 std::complex<ap_int<18>> 的 typedef）。
- test_mul 示例具有两个版本：
 - test_mul1 仅使用本征函数方法。因此其 RTL 实现：
 - 能保证使用 4 个 DSP（例如，2 个 DSPCPLX 实例）
 - 在这些 DSP 之间可能使用（也可能不使用）级联连接，因为 Vitis HLS 调度器可以在 DSPCPLX 之间插入寄存器，这可能阻止级联集合的推断。
 - test_mul2 使用级联类来确保 DSPCPLX 推断和级联连接的使用，在此例中，添加了调度约束以确保在 DSPCPLX 实例之间使用的寄存器数量正确。
 - 仅针对 std::complex<ap_int<8>> 和更高的位宽提供 DSPCPLX 的推断保证。

```
using namespace hls::dspcplx;

// example of intrinsic method, inferencing one DSPCPLX
void test_mul_add(
    ap_int<18> a_r, ap_int<18> a_i,
    ap_int<18> b_r, ap_int<18> b_i,
    ap_int<58> c_r, ap_int<58> c_i,
    ap_int<58>& r_r, ap_int<58>& r_i) {
    std::complex<ap_int<18>> a = {a_r, a_i};
    std::complex<ap_int<18>> b = {b_r, b_i};
    std::complex<ap_int<58>> c = {c_r, c_i};
    std::complex<ap_int<58>> r = mul_add<NO_PIPELINE | REG_M>(a, b, c);
    r_r = r.real();
    r_i = r.imag();
}

// example of two intrinsic methods, inferencing two DSPCPLX
void test_mul1(A_t a0, A_t a1, B_t b0, B_t b1, C_t &r) {
    // computes z = a0 * b0 + a1 * b1
```



```

    C_t t = mul<FULL_PIPELINE>(a0, b0);
    r = mul_add<FULL_PIPELINE | REG_M>(a1, b1, t);
}

// same example as above, but using the cascade class to ensure cascaded
connection
void test_mul2(A_t a0, A_t a1, B_t b0, B_t b1, C_t &z) {
    #pragma HLS pipeline II=1
    // computes z = a0 * b0 + a1 * b1
    cascade<REG_A1|REG_B1|REG_M|REG_P > dsp0;
    cascade<REG_A1|REG_B1|REG_M|REG_P > dsp1;
    C_t zero = {0, 0};
    // mul is not available for cascade, so use mul_add(...,0);
    auto tmp = dsp0.mul_add(a0, b0, zero);
    auto out = dsp0.mul_add(a1, b1, tmp.pcout);
    z=out.pcout;
}

// example showing the acc class, inferencing one DSPCPLX (with accumulator)
const int N=10;

void test_mul_acc(A_t sr[N], B_t coeff[N], C_t &res) {
    C_t t = {0, 0};
    // computes t += sr[i] * coeff[i] for all i in {0 .. N-1}
    acc<BALANCED_PIPELINE | REG_M | REG_P> my_acc;
acc_loop:
    for (int i = 0; i < N; i++) {
        #pragma HLS pipeline II=1
        t = my_acc.mul_acc( sr[i], coeff[i], /* init condition */ i == 0);
    }
    res = t;
}

// example showing cascades to infer "size" DSPCPLX and using cascaded
connections
const int N=10;
const int TAPS=4;

template <int size>
class FIR{
private:
    const A_t* coeff_;
    C_t bias_;
    cascade<BALANCED_PIPELINE|REG_M|REG_P > dsp0;
    cascade<      FULL_PIPELINE|REG_M|REG_P > dsp_full[size - 1];
public:
    FIR(const A_t* coeff, C_t bias) : coeff_(coeff), bias_(bias) {
        #pragma HLS ARRAY_PARTITION variable=dsp_full complete dim=1
    };

    C_t fir(B_t input){
        auto out = dsp0.mul_add(coeff_[0], input, bias_);
        for(int j=1; j < size ; j++){
            #pragma HLS unroll
            out = dsp_full[j - 1].mul_add(coeff_[j], out.bcout,
out.pcout);
        }
        return out.pcout;
    }; // end of fir function
}; // end of FIR class

void systolic_fir( A_t coeff[TAPS], B_t b[N], C_t bias, C_t hw[N]) {

```

```
#pragma HLS ARRAY_PARTITION variable=coeff complete dim=1

FIR<TAPS> my_fir(coeff, bias);

LOOP_FIR:
for(auto i = 0 ; i< N ; i++){
    #pragma HLS pipeline II=1
    hw[i] = my_fir.fir(b[i]);
}
}
```

快速傅里叶变换 (IP) 库

在 C++ 设计中可使用 `hls_fft.h` 库调用 AMD FFT LogiCORE IP。本节解释了在 C++ 代码中配置 FFT 的方式。如需获取 FFT 应用示例，请参阅 GitHub 上的 [Vitis-HLS-Introductory-Examples](#)。

以 Versal 器件为目标时，FFT 库支持浮点数据类型。通过 C++ 函数重载和模板特化，`hls::fft` 可以用不同的参数列表调用。`fft` 函数的这些重载版本使用不同的参数函数签名。

定点和浮点均支持超采样率 (SSR) 参数。



重要提示！ AMD 强烈建议您查阅《Fast Fourier Transform LogiCORE IP 产品指南》(PG109)，以获取有关如何实现和使用此 IP 的功能特性的信息。

要在 C++ 代码中使用 FFT，请执行以下操作：

1. 包含 `hls_fft.h` 库
2. 使用库预定义结构体 `hls::ip_fft::params_t` 设置默认参数，或继承其参数作为您自己的配置结构体（请参阅以下代码片段）。



提示： Vivado LogiCORE IP 通过其对话向导来检测并防止出现不兼容设置。请将您自己的配置结构体参数检入这些 Vivado IP 对话中，以避免出现不受支持的配置或错误配置。

3. 定义运行时配置
4. 调用 FFT 函数



提示： 由于它使用 DATAFLOW 编译指示或指令，因此必须在数据流循环（而非流水打拍循环中）执行 FFT 的流水打拍。

5. （可选）检查运行时状态

下文对上述每个步骤进行了详细讨论并配以代码片段：

首先，在源代码中包含 FFT 库。此头文件位于 Vitis HLS 安装区域内的 `include` 目录中，执行 Vitis HLS 时会自动搜索该目录。

```
#include "hls_fft.h"
```

定义 FFT 的静态参数，如输入宽度、通道数、架构类型等不会发生动态更改的参数。FFT 库包含 `hls::ip_fft::params_t` 参数化结构体，以默认值来初始化所有静态参数。

在以下 C++ 代码中，用户定义的新结构体 `config1` 会继承库结构体 `params_t` 的所有默认值。然后覆盖特定的结构体参数值，如输出排序和配置宽度。

下一步，使用 `hls::fft` API 的下列实参设置运行时配置：

- `fwd_inv`：FFT 的方向（不适用于原生浮点；必须改用静态配置参数 `systemicfft_inv`）。
- `scale_sch`：缩放调度（不适用于原生浮点）。
- `cp_len`：循环前缀长度（当 `cyclic_prefix_insertion==1` 时，此项为必需）。

使用 `hls` 命名空间调用 FFT 函数。

对于各种串流配置、单通道、带或不带运行时可配置的变换长度，函数参数如下：

```
// streaming 1 channel no run_time_configurable_transform_length
template<typename _CONFIG_T = hls::ip_fft::params_t,
        typename _DATA_IN_T,
        typename _DATA_OUT_T>
void fft(hls::stream<std::complex<_DATA_IN_T>>& in,
         hls::stream<std::complex<_DATA_OUT_T>>& out,
         bool fwd_inv = false, // direction
         int scale_sch = -1,
         int cp_len = -1,
         bool *ovflo = 0,
         unsigned *blk_exp = 0);
```

```
// streaming 1 channel run_time_configurable_transform_length
template<typename _CONFIG_T = hls::ip_fft::params_t,
        typename _DATA_IN_T,
        typename _DATA_OUT_T>
void fft(unsigned log2_transform_length,
         hls::stream<std::complex<_DATA_IN_T>>& in,
         hls::stream<std::complex<_DATA_OUT_T>>& out,
         bool fwd_inv = false, // direction
         int scale_sch = -1,
         int cp_len = -1,
         bool *ovflo = 0,
         unsigned *blk_exp = 0);
```

```
// streaming SSR>1 1 channel no run_time_configurable_transform_length
template<typename _CONFIG_T = hls::ip_fft::params_t,
        typename _DATA_IN_T,
        typename _DATA_OUT_T>
void fft(hls::stream<hls::vector<std::complex<_DATA_IN_T>,
CONFIG_T::super_sample_rate>>& in,
         hls::stream<hls::vector<std::complex<_DATA_OUT_T>,
CONFIG_T::super_sample_rate>>& out,
         bool fwd_inv = false, // direction
         int scale_sch = -1,
         int cp_len = -1,
         bool *ovflo = 0,
         unsigned *blk_exp = 0);
```

```
// streaming SSR>1 1 channel run_time_configurable_transform_length
template<typename _CONFIG_T = hls::ip_fft::params_t,
        typename _DATA_IN_T,
        typename _DATA_OUT_T>
void fft(unsigned log2_transform_length,
         hls::stream<hls::vector<std::complex<_DATA_IN_T>,
CONFIG_T::super_sample_rate>>& in,
         hls::stream<hls::vector<std::complex<_DATA_OUT_T>,
CONFIG_T::super_sample_rate>>& out,
         bool fwd_inv = false, // direction
```

```
int scale_sch = -1,
int cp_len = -1,
bool *ovflo = 0,
unsigned *blk_exp = 0);
```

对于各种阵列配置、单通道或多通道、带或不带运行时可配置的变换长度，函数参数如下：

```
// array 1 channel no run_time_configurable_transform_length
template<typename _CONFIG_T = hls::ip_fft::params_t,
        typename _DATA_IN_T,
        typename _DATA_OUT_T>
void fft(std::complex<_DATA_IN_T> in[1 <<
_CONFIG_T::log2_transform_length],
        std::complex<_DATA_OUT_T> out[1 <<
_CONFIG_T::log2_transform_length],
        bool fwd_inv = false, // direction
        int scale_sch = -1,
        int cp_len = -1,
        bool *ovflo = 0,
        unsigned *blk_exp = 0);
```

```
// array 1 channel run_time_configurable_transform_length
template<typename _CONFIG_T = hls::ip_fft::params_t,
        typename _DATA_IN_T,
        typename _DATA_OUT_T>
void fft(unsigned log2_transform_length,
        std::complex<_DATA_IN_T> in[1 <<
_CONFIG_T::log2_transform_length],
        std::complex<_DATA_OUT_T> out[1 <<
_CONFIG_T::log2_transform_length],
        bool fwd_inv = false, // direction
        int scale_sch = -1,
        int cp_len = -1,
        bool *ovflo = 0,
        unsigned *blk_exp = 0);
```

```
// array N channel no run_time_configurable_transform_length
template<typename _CONFIG_T = hls::ip_fft::params_t,
        typename _DATA_IN_T,
        typename _DATA_OUT_T>
void fft(std::complex<_DATA_IN_T> in[1 << _CONFIG_T::log2_transform_length]
[_CONFIG_T::channels],
        std::complex<_DATA_OUT_T> out[1 <<
_CONFIG_T::log2_transform_length][_CONFIG_T::channels],
        hls::vector<bool, _CONFIG_T::channels> &fwd_inv,
        hls::vector<int, _CONFIG_T::channels> &scale_sch, // used only if
enabled in config
        int cp_len = -1,
        hls::vector<bool, _CONFIG_T::channels> *ovflo = 0,
        hls::vector<unsigned, _CONFIG_T::channels> *blk_exp = 0);
```

```
// array N channel run_time_configurable_transform_length
template<typename _CONFIG_T = hls::ip_fft::params_t,
        typename _DATA_IN_T,
        typename _DATA_OUT_T>
void fft(unsigned log2_transform_length,
        std::complex<_DATA_IN_T> in[1 << _CONFIG_T::log2_transform_length]
[_CONFIG_T::channels],
        std::complex<_DATA_OUT_T> out[1 <<
_CONFIG_T::log2_transform_length][_CONFIG_T::channels],
```

```
hls::vector<bool, _CONFIG_T::channels> &fwd_inv,
hls::vector<int, _CONFIG_T::channels> &scale_sch, // used only if
enabled in config
int cp_len = -1,
hls::vector<bool, _CONFIG_T::channels> *ovflo = 0,
hls::vector<unsigned, _CONFIG_T::channels> *blk_exp = 0);
```

注释：在旧 FFT IP 使用示例中，API 的输入为 `hls::ip_fft::config_t`，输出为 `hls::ip_fft::status_t`，并通过数据搬运器函数（通常命名为 `dummy_fe()`、`dummy_be()`）进行配合，但在本版本中，不再需要这些数据搬运器函数。

以下是以固定变换长度 (`run_time_configurable_transform_length==0`) 调用 FFT 的示例，其中不含循环前缀长度或输出块指数。

```
hls::fft<config1>(xn, xk, dir, scaling, -1, ovflo);
```

最后（可选），检查输出状态，包括：上溢（仅适用于非原生浮点）和输出块指数（适用于含块浮点的非原生浮点）。

注释：原生浮点 FFT 不含输出状态。如果将状态实参（如上述的上溢）传递给 API，则其值始终为 0。

以下列出了 FFT 结构体 `hls::ip_fft::params_t` 的参数及其默认值。

表 60: FFT 结构体参数

C 语言变量名	C 语言类型	默认值	可能的值
input_width	unsigned	16	8-34
output_width	unsigned	16	input_width 到 (input_width + log2_transform_length + 1)
status_width	unsigned	8	已弃用。将在后续版本中移除。
config_width	unsigned	16	已弃用。将在后续版本中移除。
log2_transform_length	unsigned	10	3-16
run_time_configurable_transform_length	bool	false	True 和 False
channels	unsigned	1	1-12
implementation_options	unsigned	pipelined_streaming_io	automatically_select pipelined_streaming_io radix_4_burst_io radix_2_burst_io radix_2_lite_burst_io
phase_factor_width	unsigned	SSR==1 ? 16 : 32	8-34
output_ordering	unsigned	SSR==1 ? bit_reversed_order : natural_order	bit_reversed_order natural_order
ovflo	bool	SSR==1 ? true : false	true、false
scaling_options	unsigned	scaled	scaled、unscaled、block_floating_point
rounding_modes	unsigned	truncation	truncation convergent_rounding
memory_options_data	unsigned	block_ram	block_ram distributed_ram

表 60: FFT 结构体参数 (续)

C 语言变量名	C 语言类型	默认值	可能的值
memory_options_phase_factors	unsigned	block_ram	block_ram distributed_ram
memory_options_reorder	unsigned	block_ram	block_ram distributed_ram
number_of_stages_using_block_ram_for_data_and_phase_factors	unsigned	$((\log_2 \text{transform_length} < 10) ? 1 : (\log_2 \text{transform_length} - 9))$	0-11
memory_options_hybrid	bool	false	true、false
complex_mult_type	unsigned	use_mults_resources	use_luts use_mults_resources use_mults_performance
butterfly_type	unsigned	SSR==1 ? use_luts : use_xtremedsp_slices	use_luts use_xtremedsp_slices
super_sample_rates	unsigned	ssr_1	ssr_1、ssr_2、ssr_4、 ssr_8、ssr_16、ssr_32、 ssr_64
use_native_float	bool	false	true、false



重要提示！ 如果指定的参数值并非整数或布尔值，则应使用 Vitis HLS FFT 名称空间。例如，上表中参数 butterfly_type 的可能值为 use_luts 和 use_xtremedsp_slices。C 语言程序中使用的值应为 butterfly_type = hls::ip_fft::use_luts 和 butterfly_type = hls::ip_fft::use_xtremedsp_slices。



提示： 以上示例显示了标量值和阵列的使用，但 FFT 函数还支持对实参使用 hls::stream。

限制与已知问题

如果以大于 1 的超采样率 (SSR>1) (针对定点和原生浮点) 对 FFT 加以参数化，那么它与底层 LogiCORE IP 共享相同的配置限制，包括：

- FFT IP 核的 C 语言模型的数值精度受到 Linux 上 HLS 数学库的 C 语言模型的影响。为了保持 FFT 的 C 语言模型的精度，请为 csim_design 和 cosim_design 添加 -stdmath 选项，以便使用 std::math 作为 HLS 数学库的 C 语言模型。
 - 通过使用该选项，您即可在 FFT 模型外部调用浮点数学库函数（即，exp、pow、log 等）的 C 语言仿真，从 std::math 函数定义方面来看，这是精确的，但相对于 RTL 仿真则并非位精确。原因在于：
 - HLS 默认会在 FFT 外部使用 C 语言模型对这些在用户代码内调用的标准数学函数执行 C 语言仿真，该 C 语言模型相对于所生成的 RTL 是位精确的。
 - 随 LogiCORE IP 一并交付的 FFT C 语言仿真模型应使用标准数学函数，而非 HLS C 语言数学仿真模型。
- 不支持动态配置（例如方向、变换长度和缩放调度）。为保证兼容性，API 提供动态配置输入，但值会被忽略，如果指定这些值，那么 API 会检查它们与底层 IP 默认值是否相同。
- 变换方向是使用新的 config 静态参数（称为 systolicfft_inv）来指定的。
- 无输出状态（例如，无上溢输出）。为保证兼容性，API 提供了状态输出实参，其值始终设为 0。
- Phase_factor_width 对于浮点固定为 32 位，对于定点则为 19 位。
- 不支持 XK_INDEX。
- Output_ordering 固定为自然排序，即，其结果可按其自然出现的序列来呈现，不应用任何额外排序或重新排列。

- 不支持 `Cyclic_prefix_insertion`。
- `Complex_mult_type` 固定为 `use_mults_performance`。
- `Butterfly_type` 固定为 `use_extremesp_slice`。
- 每当 FFT 开始一个新的数据帧时，SSR FFT 的输入 `TREADY` 与输出 `TVALID` 都有 6 个周期的间隙。

FIR 滤波器 IP 库

在 C++ 设计中可使用 `hls_fir.h` 库调用 AMD FIR IP 块。本节解释了在 C++ 代码中配置 FIR 的方式。



重要提示！ AMD 强烈建议您查阅《FIR Compiler LogiCORE IP 产品指南》(PG149)，以获取有关如何实现和使用此 IP 的功能特性的信息。

要在 C++ 代码中使用 FIR，请执行以下操作：

1. 在代码中包含 `hls_fir.h` 库。
2. 使用库中预定义的结构体 `hls::ip_fir::params_t` 设置默认参数，或继承其参数作为您自己的配置结构体。



重要提示！ Vivado LogiCORE IP 通过其对话框向导来检测并防止出现不兼容设置。请将您自己的配置结构体参数检入这些 Vivado IP 对话框中，以避免出现不受支持的配置或错误配置。

3. (可选) 定义运行时输入配置以动态修改部分参数。
4. 调用 FIR 函数。

以下代码示例提供了每个步骤的执行方式的摘要信息。后文对每个步骤进行了更详细的探讨。

首先，在源代码中包含 FIR 库。此头文件位于 Vitis HLS 安装区域内的 `include` 目录中。执行 Vitis HLS 时，会自动搜索此目录。在 Vitis HLS 内编译时，无需指定到该目录的路径。

```
#include "hls_fir.h"
```

定义 FIR 的静态参数。包括诸如输入宽度、系数和滤波率 (`single`、`decimation` 和 `hilbert`) 等静态属性。FIR 库包含 `hls::ip_fir::params_t` 参数化结构体，以默认值来初始化所有静态参数。

在此示例中，系数定义为驻留在阵列 `coeff_vec` 中，并使用用户定义的 `myconfig` 结构体（继承自预定义结构体）来覆盖系数的数量、输入宽度和量化模式的默认值。

使用含已定义的静态参数（此示例中为 `myconfig`）的 HLS 名称空间来创建 FIR 块实例，然后使用 `run` 方法调用并执行该函数。函数实参按顺序为输入数据和输出数据。

```
static hls::FIR<param1> fir1;
fir1.run(fir_in, fir_out);
```

(可选) 可使用运行时输入配置。在 FIR 的某些模式下，此输入上的数据用于判定交织通道期间或者需要重新加载系数时的系数使用方式。此配置可采用动态配置，因而将其定义为变量。要获取有关哪些模式需使用此输入配置的完整说明，请参阅《FIR Compiler LogiCORE IP 产品指南》(PG149)。

使用运行时输入配置时，将使用以下 3 个实参来调用 FIR 函数：输入数据、输出数据和输入配置。

```
// Define the configuration type
typedef ap_uint<8> config_t;
// Define the configuration variable
config_t fir_config = 8;
// Use the configuration in the FIR
static hls::FIR<param1> fir1;
// Define input and output data types
const int axis_data_width = ((myconfig::data_width+7)>>3<<3); // AXIS data
width rounded up to byte boundary
const int axis_output_width = ((myconfig::output_width+7)>>3<<3); // AXIS
data width rounded up to byte boundary
typedef ap_fixed<axis_data_width, axis_data_width -
myconfig::data_frac_width> in_data_t;
typedef ap_fixed<axis_output_width, axis_output_width -
myconfig::output_frac_width> out_data_t;
// input and output data should be declared locally or as arguments as
follows:
// in_data_t fir_in[_CONFIG_T::data_length * _CONFIG_T::num_channels];
// out_data_t fir_out[_CONFIG_T::output_length * _CONFIG_T::num_channels];
fir1.run(fir_in, fir_out, &fir_config);
```



提示：以上示例显示了标量值和数组的使用，但 FIR 函数还支持对实参使用 `hls::stream`。如需了解更多信息，请参阅 [Vitis HLS 简介示例](#)。

FIR 静态参数

FIR 的静态参数可定义 FIR IP 的参数化方式，并指定非动态项，如输入和输出宽度、小数位数、系数值和内插与抽取速率。以上大部分配置都具有默认值：系数无默认值。

`hls_fir.h` 头文件定义了 `hls::ip_fir::params_t` 结构体，用于为大部分静态参数设置默认值。



重要提示！ 针对系数，未定义任何默认值。因此，AMD 不建议使用预定义的结构体来直接初始化 FIR。应始终使用用户定义的用于指定系数的新结构体来执行静态参数化。

在此示例中定义了 1 个新用户结构体 `my_config`，其中包含新的值用作系数。这些系数被指定为驻留在 `coeff_vec` 阵列内。FIR 的所有其他参数均使用默认值。

```
struct myconfig : hls::ip_fir::params_t {
    coeff_vec[sg_fir_srrc_coeffs_len] = {...};
};
static hls::FIR<myconfig> fir1;
fir1.run(fir_in, fir_out);
```

FIR 结构体参数

下表描述了用于参数化结构体 `hls::ip_fir::params_t` 的参数，并列出了参数的默认值和可能的值。

表 61：FIR 结构体参数值

C 语言变量名	C 语言类型	默认值	注释
data_width	unsigned	16	无限制
data_frac_width	unsigned	0	受 input_width 大小所限

表 61：FIR 结构体参数值 (续)

C 语言变量名	C 语言类型	默认值	注释
output_width	unsigned	24	无限制 (针对 output_rounding_mode=full_precision 不可设置)
coeff_width	unsigned	16	无限制
coeff_fract_width	unsigned	0	受 coeff_width 大小所限
config_width	unsigned	8	已弃用。将在后续版本中移除。
num_coefs	bool	21	Full
coeff_sets	unsigned	1	1-1024
data_length	unsigned	num_coefs	无限制 (请参阅下文, 了解依赖于 interp_rate、decim_rate 和 sample_period 的约束)
output_length	unsigned	num_coefs	无限制
num_channels	unsigned	1	1-1024
total_num_coeff	unsigned	num_coefs * coeff_sets	不适用
coeff_vec	double array	无	不适用
reloadable	bool	false	不适用
filter_type	unsigned	single_rate	single_rate、interpolation、decimation、hilbert_filter 或 interpolated
rate_change	unsigned	integer	integer 或 fixed_fractional
interp_rate	unsigned	1	1-1024
decim_rate	unsigned	1	1-1024
zero_pack_factor	unsigned	1	1-8
chan_seq	unsigned	basic	basic、advanced
rate_specification	unsigned	input_period	不适用
sample_period	double	1	无限制
quantization	unsigned	integer_coefficients	不适用
best_precision	bool	false	false true
coeff_structure	unsigned	non_symmetric	inferred、non_symmetric、symmetric、negative_symmetric、half_band 或 hilbert
output_rounding_mode	unsigned	full_precision	full_precision、truncate_lsbs、non_symmetric_rounding_down、non_symmetric_rounding_up、symmetric_rounding_to_zero、symmetric_rounding_to_infinity、convergent_rounding_to_even 或 convergent_rounding_to_odd
filter_arch	unsigned	systolic_multiply_accumulate	systolic_multiply_accumulate 或 transpose_multiply_accumulate
optimization_goal	unsigned	area	area 或 speed
inter_column_pipe_length	unsigned	4	1-16
column_config	unsigned	1	不适用

表 61: FIR 结构体参数值 (续)

C 语言变量名	C 语言类型	默认值	注释
config_method	unsigned	single	single 或 by_channel
coeff_padding	unsigned	0	添加到滤波器前的 0 值填充的数量
num_paths	unsigned	1	支持的数据路径数量
data_sign	unsigned	value_signed	不适用
coeff_sign	unsigned	value_signed	不适用



重要提示! 如果指定的参数值并非整数或布尔值, 则应使用 HLS FIR 名称空间。例如, 该表中显示 `rate_change` 的可能值为 `integer` 和 `fixed_fractional`。C 语言程序中使用的值应为 `rate_change = hls::ip_fir::integer` 和 `rate_change = hls::ip_fir::fixed_fractional`。

限制与已知问题

基于 Vivado IP 的限制如下:

- 虽然 FIR IP 内部使用的输入和输出数据宽度可取 FIR IP 支持的任意整数, 但由于 FIR IP 接口的限制, 输入和输出数据宽度必须向上舍入到 8 的下一个倍数。
- 当 `output_rounding_mode=full_precision` 时, 不可任意选择 `output_width`。请参阅《FIR Compiler LogiCORE IP 产品指南》(PG149), 或者使用 Vivado 集成设计环境 (IDE) 来检查合规值。
- 完全不能任意选择 `output_fract_width`。如需了解更多信息, 请参阅《FIR Compiler LogiCORE IP 产品指南》(PG149), 或者使用 Vivado 集成设计环境 (IDE) 来检查合规值。
- 当 FIR 的 `interpolation_rate>1` 时, 不支持切换滤波器参数 (`coeff_sets` 必须设为 1)。
- 一次执行 FIR 的输入和输出数据长度必须符合 SSR 速率以及内插和抽取所规定的长度要求:
 - `input_length` 必须能被 $\text{ceil}(1./\text{sample_period})$ (即, SSR 率) 整除。
 - `output_length` 必须能被 $\text{ceil}(1./\text{sample_period}) * \text{ceil}(\text{interp_rate} / \text{decim_rate})$ 整除
 - `output_length` 必须等于 $\text{input_length} * \text{interp_rate} / \text{decim_rate}$ 。
 - 例如, 如果 FIR 满足以下条件:
 - `sample_period = 0.5` (即, SSR 率 == $\text{ceil}(1./0.5) == 2$)
 - `interp_rate=3`
 - `decim_rate=2`
 - 那么:
 - 对于 2 项输入数据, 就必须有 $2 * 3 / 2 = 3$ 项输出数据
 - $\text{ceil}(1./\text{sample_period}) * \text{ceil}(\text{interp_rate}/\text{decim_rate}) = 2 * \text{ceil}(3 / 2) = 4$
 - 为确保 `output_length` 能被 4 整除, 必须使用:
 - `input_length=8`
 - `output_length=8 * 3 / 2 = 12`

- 对于其中一些违例，可以在 C 语言仿真和协同仿真期间生成断言，例如：

```
void hls::FIR<myconfig>::run_sim(in_data_t *, out_data_t *, config_t *,
    coeff_t *, sim_mode_t) [_CONFIG_T = myconfig]:
    Assertion `fir_out->data_size == _CONFIG_T::output_length *
        _CONFIG_T::num_channels' failed.
```

- 对于其他某些违例，Vivado 的 IP 封装器会生成错误。
- 对于其余违例，协同仿真会挂起，因为 FIR 可能会阻塞并等待输入，但输入永远不会到达，或者，FIR 不会产生测试平台所期望的所有输出。在此情况下：
- 必须遵从上述规则之一
- 或者，测试平台应生成一个额外的伪输入数据阵列，以生成输出数据。
- 如果 $SSR==1$ 且 $interpolation_rate==1$ ，那么 `fir_in` 和 `fir_out` 必须：
 - 包含一种阵列数据类型，在此情况下可连接到：
 - 含 `#pragma HLS stream` 的任一本地阵列
 - 含 `ap_fifo` 接口协议的顶层实参
 - 含 `hls::stream` 数据类型，在此情况下可连接到：
 - 任一本地 `hls::streams`
 - 含 `ap_fifo` 接口协议的顶层 `hls::stream` 实参。
 - 如果违反此规则，那么 Vivado 中执行协同仿真或逻辑综合期间会生成编译错误。
- 如果 $SSR>1$ 或 $interpolation_rate>1$ ，那么 `fir_in` 和 `fir_out` 实参只需满足常见 C++ 类型约束，并且可接受任意受支持的阵列和 `hls::streams` 接口类型。

例如，以下代码对于 $SSR==1$ 和 $interpolation_rate==1$ 无效（在 Vivado 中协同仿真或逻辑综合期间会生成错误）。

```
void top(s_data_t in[INPUT_LENGTH],
    m_data_t out[OUTPUT_LENGTH]) {
    #pragma HLS interface m_axi port=in,out
    static hls::FIR<myconfig> fir1;
    fir1.run(in, out);
}
```

而下面这段代码能正常运行：

```
void top(s_data_t in[INPUT_LENGTH],
    m_data_t out[OUTPUT_LENGTH]) {
    #pragma HLS interface ap_fifo port=in,out
    static hls::FIR<myconfig> fir1;
    fir1.run(in, out);
}
```

下面这段代码也能正常运行：

```
void top(s_data_t in[INPUT_LENGTH],
    m_data_t out[OUTPUT_LENGTH]) {
    #pragma HLS interface m_axi port=in,out
    hls::stream<s_data_t> in_t;
    hls::stream<m_data_t> out_t;
    static hls::FIR<myconfig> fir1;
    #pragma HLS dataflow
```

```

    for (int i = 0; i < INPUT_LENGTH; fir++)
        in_t.write(in[i]);
    fir1.run(in_t, out_t);
    for (int i = 0; i < OUTPUT_LENGTH; fir++)
        out[i] = out_t.read();
}

```

· 因此：

- 针对下列情况，无需添加复制输入和复制输出循环（如以上第二个示例所示，其中 `#pragma HLS interface ap_fifo port=in,out`）：
 - FIR 的 `SSR>1` 或 `interpolation_rate>1`
 - 或者，FIR 含本地输入和输出阵列或串流
- 针对 `SSR==1` 和 `interpolation_rate==1` 的情况，要将滤波器连接到非串流顶层实参，仍需要复制输入和复制输出循环（如以上第三个示例所示，其中可以使用对阵列合规的任意接口协议）。

DDS IP 库

您可在 C++ 设计内通过使用 `hls_dds.h` 库来使用 AMD 的 Direct Digital Synthesizer (DDS) IP 块。本节解释了如何在 C++ 代码中配置 DDS IP。



建议：AMD 强烈建议您查阅《DDS Compiler LogiCORE IP 产品指南》(PG141)，以获取有关如何实现和使用此 IP 的功能特性的信息。



重要提示！ DDS IP 核的 C 语言 IP 实现支持针对 `Phase_Increment` 与 `Phase_Offset` 参数采用混合模式，并且支持针对 `Phase_Offset` 采用 `none` 模式，但不支持针对这些参数采用 `programmable` 和 `streaming` 模式。

要在 C++ 代码中使用 DDS，请执行以下操作：

1. 在代码中包含 `hls_dds.h` 库。
2. 使用预定义的 `hls::ip_dds::params_t` 结构体来设置默认参数。
3. 调用 DDS 函数。

首先，在源代码中包含 DDS 库。此头文件位于 Vitis HLS 安装区域内的 `include` 目录中，执行 Vitis HLS 时会自动搜索该目录。

```
#include "hls_dds.h"
```

定义 DDS 的静态参数。例如，定义相位宽度、时钟速率和相位与增量偏移。DDS C 语言库包含 `hls::ip_dds::params_t` 参数化结构体，它可用于初始化含默认值的所有静态参数。通过重新定义此结构体中的任意值，即可自定义实现。

以下示例显示了如何使用用户定义的 `param1` 结构体覆盖相位宽度、时钟速率、相位偏移以及通道数量的默认值，此结构基于现有预定义的 `hls::ip_dds::params_t` 结构体：

```

struct param1 : hls::ip_dds::params_t {
    static const unsigned Phase_Width = PHASEWIDTH;
    static const double   DDS_Clock_Rate = 25.0;
    static const double PINC[16];
    static const double POFF[16];
};

```

使用含已定义的静态参数（例如，param1）的 HLS 名称空间创建 DDS 函数实例。然后，使用运行方法调用并执行该函数。以下是按顺序显示的数据和相位函数实参：

```
static hls::DDS<config1> dds1;  
dds1.run(data_channel, phase_channel);
```

DDS 静态参数

DDS 的静态参数定义了如何配置 DDS，例如时钟速率、相位间隔和模式。hls_dds.h 头文件可定义 hls::ip_dds::params_t 结构体，用于设置静态参数的默认值。要使用默认值，可以将参数化结构体直接与 DDS 函数一起使用。

```
static hls::DDS< hls::ip_dds::params_t > dds1;  
dds1.run(data_channel, phase_channel);
```

下表描述了 hls::ip_dds::params_t 参数化结构体的参数。

建议：AMD 强烈建议您查阅《DDS Compiler LogiCORE IP 产品指南》(PG141)，以获取有关参数和值的详细信息。

表 62：DDS 结构体参数

参数	描述
DDS_Clock_Rate	指定 DDS 输出的时钟速率。
Channels	指定通道数。DDS 和相位发生器最多可支持 16 个通道。这些通道属时分复用通道，可降低每个通道的有效时钟频率。
Mode_of_Operation	指定以下操作模式之一： 标准模式，当累加的相位用于访问 SIN/COS LUT 之前可被截位的情况下使用。 栅格化模式，当所需频率和系统时钟通过有理分式关联时使用。
Modulus	描述系统时钟频率和所需频率之间的关系。 仅在栅格化模式下使用此参数。
Spurious_Free_Dynamic_Range	指定 DDS 所产生音调的目标纯度。
Frequency_Resolution	以 Hz 为单位指定最小频率分辨率，并确定相位累加器使用的相位宽度，包括相关的相位增量 (PINC) 和相位偏移 (POFF) 值。
Noise_Shaping	用于控制是使用相位截位、抖动还是泰勒级数校正。
Phase_Width	设置以下各项的宽度： - m_axis_phase_tdata 中的“PHASE_OUT” - s_axis_phase_tdata 中的“Phase”（相位）字段（当 DDS 配置为仅 SIN/COS LUT 时） - 相位累加器 - 相关的相位增量和偏移寄存器 - s_axis_config_tdata 中的“Phase”字段 对于栅格化模式，相位宽度固定为描述有效输入范围 [0, Modulus-1] 所需的位数，即 log2 (Modulus-1) 取整。
Output_Width	设置 m_axis_data_tdata 中的“SINE”（正弦）和“COSINE”（余弦）字段的宽度。此参数提供的 SFDR 取决于所选的“Noise Shaping”（噪声成形）选项。
Phase_Increment	选择相位增量值。
Phase_Offset	选择相位偏移值。

表 62：DDS 结构体参数 (续)

参数	描述
Output_Selection	在 <code>m_axis_data_tdata</code> 中将输出选项设置为“SINE”和/或“COSINE”。
Negative_Sine	在运行时对“SINE”字段取反。
Negative_Cosine	在运行时对“COSINE”字段取反。
Amplitude_Mode	将振幅设置为全幅或单位圆。
Memory_Type	用于控制 SIN/COS LUT 的实现。
Optimization_Goal	用于控制实现决策的目标是最高速度还是最低资源。
DSP48_Use	用于控制相位偏移和/或抖动噪声加法的相位累加器和解决方案阶段的实现。
Latency_Configuration	根据优化目标设置核达到最佳值的时延。
Latency	指定手动时延值。
Output_Form	将输出形式设置为 2 的补码或符号和量级。通常，SINE 和 COSINE 的输出为 2 的补码形式。但是，使用象限对称时，可以将输出形式更改为符号和量级。
PINC[XIP_DDS_CHANNELS_MAX]	设置每个输出通道的相位增量值。
POFF[XIP_DDS_CHANNELS_MAX]	设置每个输出通道的相位偏移值。

DDS 结构体参数值

下表显示了 `hls::ip_dds::params_t` 参数化结构体参数的可能值。

表 63：DDS 结构体参数值

参数	C 语言类型	默认值	有效值
DDS_Clock_Rate	双精度	20.0	任意双精度值
Channels	无符号	1	1 - 16
Mode_of_Operation	无符号	XIP_DDS_MOO_CONVENTIONAL	XIP_DDS_MOO_CONVENTIONAL 用于对累加的相位进行截位。 XIP_DDS_MOO_RASTERIZED 用于选择栅格化模式。
Modulus	无符号	200	129 - 256
Spurious_Free_Dynamic_Range	双精度	20.0	18.0 - 150.0
Frequency_Resolution	双精度	10.0	0.000000001 - 125000000
Noise_Shaping	无符号	XIP_DDS_NS_NONE	XIP_DDS_NS_NONE 生成相位截位 DDS。 XIP_DDS_NS_DITHER 使用相位抖动来改进 SFDR，但会造成噪声本底增加。 XIP_DDS_NS_TAYLOR 使用其他情况下通过相位截位丢弃的位来插入正弦/余弦值 XIP_DDS_NS_AUTO 自动判定噪声成形。
Phase_Width	无符号	16	必须为 8 的倍数（整数）
Output_Width	无符号	16	必须为 8 的倍数（整数）

表 63: DDS 结构体参数值 (续)

参数	C 语言类型	默认值	有效值
Phase_Increment	无符号	XIP_DDS_PINCPOFF_FIXED	XIP_DDS_PINCPOFF_FIXED 在生成时修复 PINC，而 PINC 在运行时无法更改。 这是受支持的唯一值。
Phase_Offset	无符号	XIP_DDS_PINCPOFF_NONE	XIP_DDS_PINCPOFF_NONE 不生成相位偏移。 XIP_DDS_PINCPOFF_FIXED 在生成时修复 POFF，而 POFF 在运行时无法更改。
Output_Selection	无符号	XIP_DDS_OUT_SIN_AND_COS	XIP_DDS_OUT_SIN_ONLY 仅生成正弦输出。 XIP_DDS_OUT_COS_ONLY 仅生成余弦输出。 XIP_DDS_OUT_SIN_AND_COS 可生成正弦输出和余弦输出。
Negative_Sine	无符号	XIP_DDS_ABSENT	XIP_DDS_ABSENT 用于生成标准正弦波。 XIP_DDS_PRESENT 用于抵消正弦波。
Negative_Cosine	布尔值	XIP_DDS_ABSENT	XIP_DDS_ABSENT 用于生成标准正弦波。 XIP_DDS_PRESENT 用于抵消正弦波。
Amplitude_Mode	无符号	XIP_DDS_FULL_RANGE	XIP_DDS_FULL_RANGE 首先使用二进制小数点将振幅归一化为输出宽度。例如，8 位输出的二进制振幅为 100000000 - 10（对应 01111110 与 11111110 之间值），分别对应于小于 1 和大于 -1。 XIP_DDS_UNIT_CIRCLE 将振幅归一化到半满范围，即值范围为从 01000 ..(+0.5). 到 110000 ..(-0.5).
Memory_Type	无符号	XIP_DDS_MEM_AUTO	对于小规模案例（例如，如果该表可包含在单层存储器中），XIP_DDS_MEM_AUTO 会选择分布式 ROM，对于大型案例则选择块 ROM。 XIP_DDS_MEM_BLOCK 始终使用块 RAM。 XIP_DDS_MEM_DIST 始终使用分布式 RAM。
Optimization_Goal	无符号	XIP_DDS_OPTGOAL_AUTO	XIP_DDS_OPTGOAL_AUTO 会自动选择优化目标。 XIP_DDS_OPTGOAL_AREA 针对面积进行优化。 XIP_DDS_OPTGOAL_SPEED 针对性能进行优化。
DSP48_Use	无符号	XIP_DDS_DSP_MIN	XIP_DDS_DSP_MIN 用于在 FPGA 逻辑内实现相位累加器以及相位偏移和/或抖动噪声添加阶段。 XIP_DDS_DSP_MAX 用于使用 DSP 切片来实现相位累加器以及相位偏移和/或抖动噪声添加阶段。对于单通道，DSP 切片还可提供寄存器用于存储可编程相位增量和/或相位偏移，从而节省更多互连结构资源。

表 63: DDS 结构体参数值 (续)

参数	C 语言类型	默认值	有效值
Latency_Configuration	无符号	XIP_DDS_LATENCY_AUTO	XIP_DDS_LATENCY_AUTO 自动判定时延。 XIP_DDS_LATENCY_MANUAL 使用“Latency”（时延）选项手动指定时延。
Latency	无符号	5	任意值
Output_Form	无符号	XIP_DDS_OUTPUT_TWOS	XIP_DDS_OUTPUT_TWOS 输出 2 的补码。 XIP_DDS_OUTPUT_SIGN_MAG 输出有符号的量级。
PINC[XIP_DDS_CHANNELS_MAX]	无符号的阵列	{0}	对应每个通道的相位增量的任意值
POFF[XIP_DDS_CHANNELS_MAX]	无符号的阵列	{0}	对应每个通道的相位偏移的任意值

SRL IP 库

使用 C 语言编写代码可满足几种不同的要求：复用性、可读性和性能。但在目前情况下，使用 C 语言编写代码难以在进行高层次综合之后获得最理想的硬件。

与对复用性、可读性和性能的要求一样，某些编码技巧或预定义的结构可以确保综合输出产生更优化的硬件，或者以 C 语言更好地对硬件建模，从而使算法更易于确认。

直接映射到 SRL 资源中

许多 C 语言算法都会在阵列间按顺序移动数据。此类算法会在阵列开始处添加新的值、在阵列中移动现有数据，并删除最早的数据值。此操作在硬件中是通过移位寄存器来实现的。

这种将移位寄存器从 C 语言实现到硬件中的最常用的方式会将阵列完全分区为各元素，并支持 RTL 中元素间的数据依赖关系暗示需使用移位寄存器。

逻辑综合通常会将 RTL 移位寄存器实现到 AMD SRL 资源中，这样即可有效实现移位寄存器。问题在于有时逻辑综合不使用 SRL 组件来实现 RTL 移位寄存器：

- 在移位寄存器中间访问数据时，逻辑综合无法直接推断 SRL。
- 有时，即使 SRL 处于理想状态，逻辑综合也可能因其他因素而在触发器中实现移位寄存器。（逻辑综合同样是一个复杂的过程）。

Vitis HLS 可提供了 1 个 C++ 类 (ap_shift_reg) 用于确保 C 语言代码中定义的移位寄存器始终使用 SRL 资源来实现。ap_shift_reg 类采用 2 种方法来执行 SRL 组件支持的各种读写访问。

从移位器中读取

读取方法允许从移位器寄存器读取指定的位置。

Vitis HLS 还包含 `ap_shift_reg.h` 头文件作为独立封装，用于定义 `ap_shift_reg` 类。您有权在自己的源代码中使用此头文件。`xilinx_hls_lib_<release_number>.tgz` 封装位于 Vitis HLS 安装区域的 `include` 目录内。

```
// Include the Class
#include "ap_shift_reg.h"

// Define a variable of type ap_shift_reg<type, depth>
// - Sreg must use the static qualifier
// - Sreg will hold integer data types
// - Sreg will hold 4 data values
static ap_shift_reg<int, 4> Sreg;
int var1;

// Read location 2 of Sreg into var1
var1 = Sreg.read(2);
```

数据的读取、写入和移位

`shift` 方法允许执行读取、写入和移位操作。

```
// Include the Class
#include "ap_shift_reg.h"

// Define a variable of type ap_shift_reg<type, depth>
// - Sreg must use the static qualifier
// - Sreg will hold integer data types
// - Sreg will hold 4 data values
static ap_shift_reg<int, 4> Sreg;
int var1;

// Read location 3 of Sreg into var1
// THEN shift all values up one and load In1 into location 0
var1 = Sreg.shift(In1,3);
```

读、写和使能移位

`shift` 方法还支持已启用的输入，以支持通过变量来对移位进程进行控制和启用。

```
// Include the Class
#include "ap_shift_reg.h"

// Define a variable of type ap_shift_reg<type, depth>
// - Sreg must use the static qualifier
// - Sreg will hold integer data types
// - Sreg will hold 4 data values
static ap_shift_reg<int, 4> Sreg;
int var1, In1;
bool En;

// Read location 3 of Sreg into var1
// THEN if En=1
// Shift all values up one and load In1 into location 0
var1 = Sreg.shift(In1,3,En);
```

使用 `ap_shift_reg` 类时，Vitis HLS 会为每个移位器创建唯一的 RTL 组件。执行逻辑综合时，该组件会综合到 SRL 资源内。

处理 OpenCV

如 https://github.com/Xilinx/Vitis_Libraries/tree/main/vision 中所述，您还必须已安装 OpenCV 计算机视觉库，才能使用 Vitis 视觉库 (Vitis Vision Library)。Vitis HLS 并不含预安装的 OpenCV，如需了解如何查找和安装 OpenCV，请访问以下链接：[Vitis 库 - 编译和安装 OpenCV 库以搭配视觉库一起使用](#)。

第七部分

Vitis HLS 移植指南

本部分包含以下章节：

- [从 Vitis HLS 移植到 Vitis Unified HLS](#)
- [从 Vivado HLS 移植到 Vitis HLS](#)
- [已弃用功能特性和不受支持的功能特性](#)
- [不受支持的功能特性](#)

从 Vitis HLS 移植到 Vitis Unified HLS

Vitis Unified IDE 提供了通用工具用于创建异构系统设计或数据中心加速应用的不同组件。这些组件包括从 C/C++ 源代码生成的 HLS 组件。该组件包含 `v++` 和 `vitis-run` 通用命令行，用于系统设计的编译和链接。Vitis Unified IDE 提供了多种途径来将现有 Vitis HLS 传统工程移植到新工具。这些方法包括：

1. 从现有 Vitis HLS 传统工程创建 HLS 组件
2. 继续使用现有 Tcl 脚本
3. 转换为使用 Python 脚本

1. 从现有 Vitis HLS 工程创建 HLS 组件

如 [创建 HLS 组件](#) 中所述，您可从现有 Vitis HLS 传统工程指定 `hls.app` 文件，以便生成新的 HLS 配置文件。该工具允许您识别工程内的 `hls.app` 文件并为基于 `hls.app` 文件生成的配置文件输入“New configuration file name”（新配置文件名称）。



提示：如果工程包含多个解决方案，那么 `hls.app` 文件会识别这些解决方案，并且“HLS Component Creation” Wizard（HLS 组件创建向导）将提示您指定应导入哪个可用的解决方案。这样即可为工程中的每个解决方案创建单独的 HLS 组件。

2. 继续使用现有 Tcl 脚本

要继续使用现有 Tcl 脚本，请将 `open_project` 替换为 `open_component`，并指定 `-flow_target vivado` `vitis`。鉴于在 Vitis Unified IDE 中不需要 `open_solution` Tcl 命令，请将其移除，如下所示。

<pre> 17 # Create a project 18 open_project -reset proj_pointer_basic 19 20 # Add design files 21 add_files pointer_basic.c 22 # Add test bench & files 23 add_files -tb pointer_basic_test.c 24 add_files -tb result.golden.dat 25 26 # Set the top-level function 27 set_top pointer_basic 28 29 # ===== 30 # Create a solution 31 open_solution -reset solution1 -flow_target vitis 32 # Define technology and clock rate 33 set_part {xcvu9p-flga2104-2-i} 34 create_clock -period 4 </pre> Vitis HLS Classic	<pre> 15 # limitations under the license. 16 17 # Create a component 18 open_component -reset component_pointer_basic -flow_target vivado 19 20 # Add design files 21 add_files pointer_basic.c 22 # Add test bench & files 23 add_files -tb pointer_basic_test.c 24 add_files -tb result.golden.dat 25 26 # Set the top-level function 27 set_top pointer_basic 28 29 # ===== 30 # Define technology and clock rate 31 set_part {xcvu9p-flga2104-2-i} 32 create_clock -period 4 </pre> Vitis Unified IDE
---	---

使用 `open_component` 创建的输出结构与 Vitis Unified IDE 兼容，在 Vitis Unified IDE 内可打开要运行的 Tcl 脚本所在的目录作为工作空间。HLS 报告和生成的 RTL 文件当前位于组件目录下。Tcl 脚本可使用 `vitis-run` 来运行，如下所示。

```
vitis-run --mode hls --tcl <tcl_script>.tcl
```

3. 转换为使用 Python 脚本

Vitis Unified IDE 支持通过 Python API 来创建和管理各种组件，包括 HLS 组件。包含这些 API 的 Python 脚本可用于开发和执行 HLS 组件。HLS 组件配置既可在使用 `write_ini` 生成的配置文件（如上所述）内指定，也可以在 Python 脚本内以键值对的形式来指定。Python 脚本可使用 `vitis` 来运行，如下所示。

```
vitis -s <python_script>.py
```

使用 `write_ini` 创建 HLS 配置文件

`vitis-run` 可执行文件能使用 `write_ini` 命令从现有 Tcl 脚本创建配置文件。具体方法是在 Tcl 脚本内包含 `write_ini` 命令或者使用 `vitis-run` 启动 HLS 交互式 Tcl shell（如下所示），并指定 `write_ini` 命令将创建工程或特定解决方案所用 Tcl 命令导出。从现有工程或解决方案创建 HLS 配置文件后，即可在 Vitis Unified IDE 中使用这些配置文件搭配指定源文件来创建 HLS 组件。

```
$ vitis-run --mode hls --itcl
HLS> open_project <project_name>
HLS> write_ini <config_file_name>
```

此外，如果您倾向于使用 Vitis Unified IDE 通过脚本化流程进行 HLS 开发，请参阅 [Vitis-HLS-Introductory-Examples](#) 仓库的 [Migration](#) 中的示例，这些示例展示了多种方法可供您使用。

从 Vivado HLS 移植到 Vitis HLS

AMD Vivado™ HLS 是上一代 HLS 解决方案，最后一个官方版本为 2020.1。AMD Vitis™ HLS 提供了来自 AMD 的下一代 HLS 解决方案，相比先前 Vivado HLS 技术，新技术带来了多项改进。首先，AMD Vitis™ HLS 具有新的编译器，使用 LLVM 编译器标准的更新版本，并支持以 C/C++ 11/14 进行编译/仿真。对某一版本的 Vivado HLS 中已实现的内核模块或 IP 进行移植时，必须明确掌握 HLS 版本间的差异以及这些差异对于设计的影响。

这两种版本的 HLS 之间的主要差异分类如下：

- 关键行为差异
- 已弃用的命令
- 不受支持的功能特性

此外，Vitis HLS 现已支持对源代码的编译指示的使用执行如下更为严格的新语法检查：

- “Syntax Checker”（语法检查器）：Vitis HLS 将对数据流区域内或 RTL 生成期间未连接的端口报错并输出。
- “Pragma Conflict Checker”（编译指示冲突检查器）：用于针对编译指示选项中的编译指示冲突和/或错字提供全新的警告和错误报告功能。
- 当该工具忽略用户指定的编译指示时，新增并改善了显示的消息。

关键行为差异

本节着重讲解了 AMD Vivado™ HLS 与 AMD Vitis™ HLS 之间的部分关键行为差异。这些关键差异可能造成重大的结果质量 (QoR) 差异，因此在不同版本的 HLS 工具间比较 QoR 时，有必要了解这些关键差异。AMD 建议在使用 Vitis HLS 前复查本章节内容。



提示：鉴于 Vitis HLS 与 Vivado HLS 之间存在的差异，您可能需要更改自己的代码，才能在 Vitis 工具中使用。为了在两种工具内使用相同的源代码，Vitis HLS 支持使用 `__VITIS_HLS__` 预定义宏来对专为在该工具内使用而编写的源代码进行封装。`#if defined( __VITIS_HLS__ )` 类型的预处理器声明可用于封装特定于工具的代码。

默认用户控制设置

默认全局选项用于为 Vitis 内核加速开发流程或 Vivado IP 开发流程配置解决方案。

```
open_solution -flow_target [vitis | vivado]
```

此全局选项将替代先前配置选项 (`config_sdx`)。

Vivado IP 开发流程

将解决方案配置为在支持 Vivado IP 生成流程的前提下运行，其中包括要求严格按标准使用编译指示和指令，并将结果导出为 Vivado IP。用于为 Vivado IP 流程设置工程解决方案的命令如下：

```
open_solution -flow_target vivado
```

下表显示了 Vivado HLS 工具中命令选项的原始默认设置以及 Vitis HLS 中应用的新默认设置。

表 64：默认控制设置

默认控制设置	Vivado HLS	Vitis HLS
config_compile -pipeline_loops	0	64
config_export -vivado_optimization_level	2	0
set_clock_uncertainty	12.5%	27%
config_interface -m_axi_alignment_byte_size	不适用	0
config_interface -m_axi_max_widen_bitwidth	不适用	0
config_export -vivado_phys_opt	place	无
config_interface -m_axi_addr64	false	true
config_schedule -enable_dsp_full_reg	false	true
config_rtl -module_auto_prefix	false	true
interface 编译指示默认设置	ip 模式	ip 模式

Vitis 应用加速开发流程（内核模式）

将解决方案配置为在支持 Vivado IP 生成流程的前提下运行，其中包括要求严格按标准使用编译指示和指令，并将结果导出为 Vivado IP。用于为 Vivado IP 流程设置工程解决方案的命令如下：

```
open_solution -flow_target vivado
```

下表显示了 Vivado HLS 工具中命令选项的原始默认设置以及 Vitis HLS 工具中应用的新默认设置。

表 65：默认控制设置

默认控制设置	vitis_hls flow_target=vitis	vitis_hls flow_target=vivado（默认流程）
config_compile -pipeline_loops	0	64
config_export -vivado_optimization_level	2	0
set_clock_uncertainty	12.5%	27%
config_interface -m_axi_alignment_byte_size	0	64
config_interface -m_axi_latency	0	64
config_interface -m_axi_max_widen_bitwidth	0	512
config_export -vivado_phys_opt	place	无

表 65：默认控制设置 (续)

默认控制设置	vitis_hls flow_target=vitis	vitis_hls flow_target=vivado (默认流程)
config_interface -m_axi_addr64	false	true
config_schedule -enable_dsp_full_reg	false	true
config_rtl -module_auto_prefix	false	true
config_rtl -register_reset_num	0	3
interface 编译指示默认设置	ip 模式	内核模式

默认接口

AMD Vivado™ HLS 和 AMD Vitis™ HLS 都允许用户使用编译指示/指令来自定义与顶层函数对接的接口。Vitis HLS 还支持自动推断合适的接口的功能。Vitis HLS 创建的接口类型取决于：

- 顶层函数中 C 语言实参的数据类型。
- 目标流程。
- 默认接口模式。
- 任何用户指定的 **INTERFACE** 编译指示或指令。

在本用户指南 (UG1399) 的 [定义接口](#) 章节中对此进行了详细描述，应先审阅其中内容而后再继续操作。

接口捆绑规则

默认情况下，AMD Vitis™ HLS 会将具有兼容选项的函数实参组合到单个 `m_axi/s_axilite` 接口适配器中。将端口捆绑到单个接口中可消除 AXI4 逻辑，因而有助于节省器件资源，在拥塞的设计内工作时可能需要这些资源。但是，单一接口捆绑可能会限制内核性能，因为所有存储器传输都必须穿越单个接口。例如，`m_axi interface` 具有独立的 READ 通道和 WRITE 通道，因此单个接口即可同时执行读取和写入，但仅限单一位置。使用多个捆绑即可创建多个接口以连接到存储体，从而能够增大内核带宽和吞吐量。

在本用户指南 (UG1399) 的“[M_AXI 捆绑/S_AXILITE 捆绑](#)”部分中对 AXI 接口的捆绑规则进行了完整描述，应先审阅其中内容而后再继续操作。

接口上的存储器属性

INTERFACE 编译指示或指令上的 `storage_type` 选项允许用户显式定义所使用的 RAM 类型以及所创建的 RAM 端口（单端口或双端口）。如果不指定 `storage_type`，那么 Vitis HLS 会使用：

- 单端口 RAM（默认情况下）。
- 双端口 RAM，前提是这样可缩短启动时间间隔或减少时延。

对于 Vivado 流程，用户可以在指定接口上指定 RAM 存储类型，并将已弃用的 RESOURCE 编译指示替换为 `storage_type`。

```
#pragma HLS INTERFACE ap_memory port = in1 storage_type=RAM_2P
#pragma HLS INTERFACE ap_memory port = out storage_type=RAM_1P latency=3
```


含旁路的 AXI4-Stream 接口

旁路为可选信号，属于 AXI4-Stream 标准的一部分。C/C++ 语言代码中可使用结构体直接引用和控制旁路信号，前提是结构体的成员元素与 AXI4-Stream 旁路信号的名称相匹配。AXI4-Stream 旁路信号被视为数据信号，随 TDATA 一起寄存。在 Vivado HLS 与 Vitis HLS 中对此信号的处理方式截然不同。本用户指南 (UG1399) 中的 [含旁路的 AXI4-Stream 接口](#) 章节详细描述了 Vitis HLS 的限制和局限性。

存储器模型

AMD Vivado™ HLS 与 AMD Vitis™ HLS 之间的关键差异之一是内部存储器模型的实现方式。存储器模型可以定义计算机存储器中排列和访问数据的方式。其中涉及两个独立但相关的问题：数据对齐和数据结构填充。Vitis HLS 所使用的存储器模型不同于 Vivado HLS，关键差异如下：

- Vitis HLS 遵循 C++ 编译器标准来执行数据对齐
- 对接口中的结构体/类进行聚合和解聚时，Vitis HLS 采用的方法与 Vivado HLS 有着显著差异
 - 默认情况下，Vivado HLS 原先会对接口中的结构体进行解聚。Vitis HLS 则改为将结构体保留处于聚合状态。您可使用 AGGREGATE/[pragma HLS disaggregate](#) 编译指示/指令与 Vivado HLS 的行为相匹配。
 - 例外情况是如果结构体内有 hls::stream 对象，或者如果结构体内的阵列需分区，那么 Vitis HLS 在这两种情况下都会对该结构体进行解聚。
 - 代码中的结构体（内部变量和全局变量）默认情况下处于解聚 (disaggregated) 状态，并根据其成员元素进行分解，如 [结构体](#) 中所述。创建的元素数量和类型取决于结构体本身的内容。结构体阵列作为多个阵列来实现，每个结构体成员都具有独立的阵列。
- Vitis HLS 的接口协议（例如，AXI）的数据对齐处理方式也有所不同。
- 更改也包含已弃用的编译指示，例如，DATA_PACK。

在本用户指南 (UG1399) 的 [Vitis HLS 存储器布局模型](#) 章节中对这些更改进行了详细描述，应先审阅其中内容而后再继续操作。

未连接的端口

对于一维阵列和多维阵列，Vivado HLS 接受未连接的输出信号。

Vitis HLS 允许以下类型的未连接的输出：

- 未连接的输出标量
- 未连接的输出一维阵列和多维阵列，这些阵列在数据流区域内部使用时是作为 PIPO 来实现的

Vitis HLS 不允许：

- 未连接的输出串流
- 未连接的输出串流一维阵列或多维阵列

接口上的全局变量

Vivado HLS 和 AMD Vitis™ HLS 都允许在代码中自由使用全局变量，并且这些全局变量完全可综合。虽然 Vivado HLS 支持在顶层函数接口中导出全局变量，但是 AMD Vitis™ HLS 不支持此操作。

全局变量不能用作顶层函数的实参，也不能作为 AMD Vitis™ HLS 中的 RTL 接口上的端口予以公开。顶层函数的接口上所需的变量必须在接口中显式声明。

模块名称和模块前缀的行为更改

在 Vivado HLS 中，启用 `config_rtl -module_auto_prefix` 时，顶层 RTL 模块名称将采用其自己的名称作为前缀。从 2020.1 版 Vitis HLS 起，此自动前缀仅应用于子模块。

`-module_prefix` 行为无更改。如果使用该选项，那么指定的前缀值将置于所有模块（包括顶层模块）之前。`-module_prefix` 选项仍优先于 `-module_auto_prefix`。

```
# vivado HLS 2020.1 generated module names (top module is "top")
top_top.v
top_submodule1.v
top_submodule2.v

# Vitis HLS 2020.1 generated module names
top.v          <-- top module no longer has prefix
top_submodule1.v
top_submodule2.v
```

更新后的存储器 (RAM/ROM) 模块名称和 RTL 文件名

在模块的新名称后将追加存储器类型，如下所示。

表 66：模块和 RTL 文件名

旧模块和 RTL 文件名	新模块和 RTL 文件名
ncp_encoder_func_parbits_memcore_ram	ncp_encoder_func_parbits_RAM_1P_LUTRAM_1R1W
test_A_V_ROM	test_A_V_ROM_1P_BRAM_1R 和 test_A_V_ROM_1P_BRAM_1.v

内联函数之间的行为差异

在 Vivado HLS 中函数的自动内联由函数大小来管控。但在 Vitis HLS 中，自动内联优化由用户编译指示偏好或成本模型来驱动。以流水打拍循环为例，假设在此循环主体内部执行子函数的调用。在 Vivado HLS 中，这些子函数可能由于大小较小，已自动完成内联。在 Vitis HLS 中，仅当用户请求 `II=1` 的流水线时，这些子函数才会执行内联。因此，仅当 Vitis HLS 调度器判定在此流水打拍循环内，对这些子函数执行内联有利于达成 `II=1` 时，才会发生自动内联。

由于此更改，您可能会发现，在 Vitis HLS 与 Vivado HLS 所实现的 QoR 之间存在差异。您可通过在 Vitis HLS 中手动内联这些子函数来匹配先前 Vivado HLS 的行为，从而克服这些差异。

数据流

针对 `std::complex` 的支持：

在 Vivado HLS 中，由于存在多个写程序问题，无法在数据流 (DATAFLOW) 中直接使用 `std::complex` 数据类型。此多重读写程序问题原因在于调用了 `std` 类构造函数来初始化该值。如果同时在数据流中使用该变量作为通道，就会导致以上问题。但 Vitis 支持通过使用 `no_ctor` 属性来使用 `std::complex`，如下所示。

```
// Nothing to do here.
void proc_1(std::complex<float> (&buffer)[50], const std::complex<float>
*in);
void proc_2(hls::stream<std::complex<float>> &fifo, const
std::complex<float> (&buffer)[50], std::complex<float> &acc);
void proc_3(std::complex<float> *out, hls::stream<std::complex<float>>
```

```
&fifo, const std::complex<float> acc);

void top(std::complex<float> *out, const std::complex<float> *in) {
#pragma HLS DATAFLOW
    std::complex<float> acc __attribute__((no_ctor)); // here
    std::complex<float> buffer[50] __attribute__((no_ctor)); // here
    hls::stream<std::complex<float>, 5> fifo; // no need here (hls::stream
    has it internally)

    proc_1(buffer, in);
    proc_2(fifo, buffer, acc);
    proc_3(out, fifo, acc);
}
```

已弃用功能特性和不受支持的功能特性

下列已弃用的命令将在未来版本中停用，不建议使用：

表 67: Vitis HLS 中弃用的 Vivado HLS 命令

类型	命令	选项	Vitis HLS	详细信息
config	config_interface	-m_axi_max_data_size	已弃用	
config	config_interface	-m_axi_min_data_size	已弃用	
config	config_interface	-m_axi_alignment_byte_size	已弃用	
config	config_interface	-m_axi_offset=slave	不支持	现在通过 -m_axi_offset=direct 与 -default_slave_interface=s_axilite 组合来处理此操作
config	config_interface	-expose_global	不支持	在 Vitis HLS 中不支持将全局变量作为 IP 或内核中的顶层端口予以公开。
config	config_interface	-trim_dangling_port	不支持	
config	config_array_partition	-auto_promotion_threshold	已弃用	
config	config_array_partition	-auto_partition_threshold	已弃用	已重命名为 -complete_threshold
config	config_array_partition	-scalarize_all	不支持	
config	config_array_partition	-throughput_driven	不支持	
config	config_array_partition	-maximum_size	不支持	
config	config_array_partition	-include_external_globals	不支持	
config	config_array_partition	-include_ports	不支持	
config	config_schedule	除 -enable_dsp_fill_reg 外的所有选项	已弃用	
config	config_bind	* (所有选项)	已弃用	
config	config_rtl	-encoding	已弃用	FSM 编码为独热码。
config	config_sdx	* (所有选项)	已弃用	

表 67: Vitis HLS 中弃用的 Vivado HLS 命令 (续)

类型	命令	选项	Vitis HLS	详细信息
config	config_flow	* (所有选项)	已弃用	
config	config_dataflow	-disable_start_propagation	已弃用	
config	config_rtl	-auto_prefix	已弃用	已替换为 config_rtl-module_prefix。
config	config_rtl	-prefix	已弃用	已替换为 config_rtl-module_prefix。
config	config_rtl	-m_axi_conservative_mode	已弃用	使用 config_interface-m_axi_conservative_mode
指令/编译指示	set_directive_pipeline	-enable_flush	已弃用	
指令/编译指示	CLOCK	*	不支持	
指令/编译指示	DATA_PACK	*	不支持	如果需要, 请使用 AGGREGATE 编译指示或指令和 __attribute__((packed(X)))。
指令/编译指示	INLINE	-region	已弃用	
指令/编译指示	INTERFACE	-mode ap_bus	不支持	使用 m_axi 代替。
指令/编译指示	INTERFACE	-mode ap_stable	已弃用	改用 STABLE 编译指示或指令。
指令/编译指示	ARRAY_MAP	*	不支持	
指令/编译指示	RESOURCE	*	已弃用	已替换为 BIND_OP 和 BIND_STORAGE 编译指示和指令。对于顶层函数实参, 请将 INTERFACE 编译指示或指令与 storage_type 选项搭配使用。
指令/编译指示	SHARED	*	已弃用	SHARED 编译指示或指令已移至 STREAM 编译指示或指令的 type=shared 选项。
指令/编译指示	STREAM	-dim	不支持	
指令/编译指示	STREAM	-off	已弃用	STREAM off 已改为 STREAM type=pipo
project	csim_design	-clang_sanitizer	添加/重命名	
project	export_design	-use_netlist	已弃用	已替换为: export_design-format ip_catalog
project	export_design	-xo	已弃用	已替换为: export_design-format xo
project	add_files		不支持	System-C 文件不受 Vitis HLS 支持。



表 67: Vitis HLS 中弃用的 Vivado HLS 命令 (续)

类型	命令	选项	Vitis HLS	详细信息
config	config_export	-disable_deadlock_detection	已弃用	已替换为： config_export -deadlock_detecti on_sim

- 注释：**
- 1. 已弃用：在未来版本中将发出终止使用相关编译指示的警告消息。
 - 2. 不支持：Vitis HLS 将报错，并输出有效消息。
 - 3. *：命令中的所有选项。

下列库均已弃用。

表 68：已弃用的库

库	API	已弃用	建议的替代/操作
线性代数学库 hls_linear_algebra.h hls/hls_axi_io.h hls/ hls_linear_algebra_io.h hls/linear_algebra -- deprecated -- x_complex_back_substitute.h -- x_complex_cholesky.h -- x_complex_cholesky_inverse.h -- x_complex_matrix_multiply.h -- x_complex_matrix_tbtutils.h -- x_complex_matrix_utils.h -- x_complex_qr_inverse.h -- x_complex_qrf.h `-- x_complex_svd.h  -- hls_back_substitute.h  -- hls_cholesky.h  -- hls_cholesky_inverse.h  -- hls_matrix_multiply.h  -- hls_qr_inverse.h  -- hls_qrf.h  -- hls_svd.h `-- utils -- std_complex_utils.h -- x_hls_complex.h -- x_hls_matrix_tbtutils.h `-- x_hls_matrix_utils.h	cholesky float、ap_fixed、 x_complex<float> 或 x_complex<ap_fixed>	已弃用	Vitis 解算器 potrf float 或 double
	cholesky_inverse float、ap_fixed、 x_complex<float> 或 x_complex<ap_fixed>		Vitis 解算器 pomatrixinverse float 或 double
	matrix_multiply float、ap_fixed、 x_complex<float> 或 x_complex<ap_fixed>		
	qrf float 或 x_complex<float>		Vitis 解算器 geqrf float 或 double
	qr_inverse float 或 x_complex<float>		
	svd float 或 x_complex<float>		Vitis 解算器 gesvdj float 或 double

表 68：已弃用的库 (续)

库	API	已弃用	建议的替代/操作
DSP 库 hls_dsp.h hls/dsp -- hls_atan2_cordic.h -- hls_awgn.h -- hls_cmpy.h -- hls_convolution_encoder.h -- hls_nco.h -- hls_qam_demod.h -- hls_qam_mod.h -- hls_sqrt_cordic.h -- hls_viterbi_decoder.h `-- utils  -- hls_cordic.h  -- hls_cordic_functions.h `-- hls_dsp_common_utils.h	atan2 输入： std::complex<ap_fixed> 输出：ap_ufixed	已弃用	HLS 数学库 atan2 ap_fixed/ap_ufixed/float/ double
	sqrt 含 1 位整数的无符号二进制小 数，无符号整数		HLS 数学库 sqrt ap_fixed/ap_ufixed/float/ double
	awgn (加性高斯白噪声) 输入：ap_ufixed 输出：ap_int		
	cmpy (复数乘法) 输入：std::complex< ap_fixed > 输出：std::complex< ap_fixed >		
	convolution_encoder (用于 在含纠错的通道内进行数据传 输，搭配 Viterbi Decoder 一 起使用) 输入：ap_uint 输出：ap_uint		
	viterbi_decoder (用于在含纠 错的通道内进行数据传输，搭 配 Convolutional Encoder 一 起使用) 输入：ap_uint 输出：ap_uint		
	nco (数控振荡器) 输入：ap_uint 输出：std::complex< ap_int >		

不受支持的功能特性

在此版本中，不支持以下功能。



重要提示！ HLS 会对本章节中所提到的所有不受支持的功能发出警告或错误消息。

编译指示

- 如果实参包含 `m_axi` INTERFACE 编译指示用于指定含 2 个或 2 个以上端口的 `bundle`，则不支持此实参上同时包含 `DEPENDENCE` 编译指示。

```
void top(int *a, int *b) { // both a and b are bundled to m_axi port gmem
    #pragma HLS interface m_axi port=a offset=slave bundle=gmem
    #pragma HLS interface m_axi port=b offset=slave bundle=gmem
    #pragma HLS dependence variable=a false
}
```

- 编译指示 `INTERFACE` 不再支持原先 AMD Vivado™ HLS 中支持的 `ap_bus` 模式。您应改为使用 `m_axi` 接口。

顶层函数实参



重要提示！ AMD Vitis™ HLS 不支持在子函数内使用 `INTERFACE` 编译指示。

含如下 C 语言数据类型的顶层实参：

- `enum` 或 `enum` 的任意用法（结构体、`enum` 的阵列指针）
- `ap_int<N>`（其中 `N` 为 1-32768 之间的值）
- `_Complex`
- `_Half` 和 `__fp16`

HLS 视频库

适用于视频实用工具和功能的 `hls_video.h` 已弃用，并替换为 Vitis 视觉库。如需了解详细信息，请参阅 GitHub 上的 [将 HLS 视频库移植到 Vitis 视觉库](#)。

C 语言任意精度类型

Vitis HLS 不支持 C 语言任意精度类型。AMD 建议使用具有任意精度的 C++ 语言类型。

此外，Vitis HLS 中的 C++ 任意精度类型的最大宽度限制为 4096 位，而不是 Vivado HLS 所支持的 32K 位。

C 语言构造

- 不支持指针强制转换。
- 不支持虚拟函数。

Tcl 到配置文件命令映射

下表提供了 Vitis HLS Tcl 命令到 `v++ -c --mode hls` 和 `vitis-run` 配置文件命令的映射。

表 69: Tcl 工程命令

Tcl 命令	选项	默认	值类型	配置文件命令
add_files	appendflags	false	bool	syn.file_cflags syn.file_csimflags tb.file_cflags
add_files	blackbox	false	bool	syn.blackbox.file
add_files	cflags		string	syn.cflags syn.file_cflags tb.cflags tb.file_cflags
add_files	csimflags		string	syn.csimflags syn.file_csimflags
add_files	src_files		string	syn.file
add_files	tb	false	bool	tb.file
cosim_design	enable_tasks_with_m_axi		bool	
cosim_design	hwemu_trace_dir		string	
cosim_design	disable_binary_tv		bool	
cosim_design	stable_axilite_update		bool	
cosim_design	disable_dependency_check		bool	
cosim_design	enable_dataflow_profiling		bool	
cosim_design	enable_fifo_sizing		bool	
cosim_design	random_stall		bool	
cosim_design	user_stall		string	
cosim_design	disable_deadlock_detection		bool	
cosim_design	wave_debug		bool	
cosim_design	argv		string	
cosim_design	compiled_library_dir		string	
cosim_design	coverage		bool	
cosim_design	ldflags		string	
cosim_design	mflags		string	
cosim_design	O		bool	
cosim_design	rtl	verilog	enum	
cosim_design	setup		bool	

表 69: Tcl 工程命令 (续)

Tcl 命令	选项	默认	值类型	配置文件命令
cosim_design	tool	xsim	enum	
cosim_design	trace_level	无	enum	
cosim_stall	generate_burst_default		string	
cosim_stall	check		string	
cosim_stall	generate		string	
cosim_stall	list		bool	
create_clock	name		string	
create_clock	period	10ns	string	clock
csim_design	code_analyzer	0	bool	
csim_design	argv		string	
csim_design	clean		bool	
csim_design	ldflags		string	
csim_design	mflags		string	
csim_design	O		bool	
csim_design	setup		bool	
csynth_design	dump_post_cfg	0	bool	
csynth_design	synthesis_check	0	bool	
csynth_design	dump_cfg	0	bool	
delete_project	project		string	
delete_solution	solution		string	
enable_beta_device	pattern	*	string	
export_design	output		string	
export_design	description		string	
export_design	display_name		string	
export_design	flow	无	enum	
export_design	format	ip_catalog	enum	
export_design	ipname		string	
export_design	library		string	
export_design	rtl	verilog	enum	
export_design	taxonomy		string	
export_design	vendor		string	
export_design	version		string	
get_clock_period	units		enum	
get_clock_period	default	0	bool	
get_clock_period	name		string	
get_clock_uncertainty	default	1	bool	
get_clock_uncertainty	clock_name	default	string	
get_files	appfile		string	
get_files	blackbox	false	bool	

表 69: Tcl 工程命令 (续)

Tcl 命令	选项	默认	值类型	配置文件命令
get_files	filepaths	无	enum	
get_files	fileroot		string	
get_files	quiet	false	bool	
get_files	csimflags	false	bool	
get_files	cflags	false	bool	
get_files	tb	false	bool	
get_part	family	false	bool	
get_project	solutions	0	bool	
get_project	directory	0	bool	
get_project	name	0	bool	
get_solution	json	0	bool	
get_solution	directory	0	bool	
get_solution	flow_target	0	bool	
get_solution	name	0	bool	
help	ini	0	bool	
help	width	120	int	
help	cmd		string	
list_part	family	false	bool	
list_part	clock_regions	false	bool	
list_part	slr_pblocks	false	bool	
list_part	board	false	bool	
list_part	name		string	
open_project	diskless	0	bool	
open_project	upgrade	0	bool	
open_project	name		string	
open_project	reset	0	bool	
open_solution	flow_target		enum	flow_target
open_solution	name		string	
open_solution	reset	0	bool	
open_tcl_project	tclfile		string	
set_clock_uncertainty	clock_list		string	
set_clock_uncertainty	uncertainty		string	clock_uncertainty
set_part	board		string	board
set_part	value		string	part
set_top	name		string	syn.top

表 70: Tcl 配置命令

Tcl 命令	选项	默认	值类型	配置文件命令
config_array_partition	complete_threshold	4	uint	syn.array_partition.complete_threshold
config_array_partition	throughput_driven	auto	enum	syn.array_partition.throughput_driven
config_array_stencil	throughput_driven	off	enum	syn.array_stencil.throughput_driven
config_compile	clang_version	16	enum: 7 16	syn.compile.clang_version
config_compile	design_size_maximum_warning	100000	int	syn.compile.design_size_maximum_warning
config_compile	performance_budgeter	auto	enum: auto enable disable	syn.compile.performance_budgeter
config_compile	pipeline_flush_in_task	ii1	enum	syn.compile.pipeline_flush_in_task
config_compile	ignore_long_run_time	0	bool	syn.compile.ignore_long_run_time
config_compile	pre_tcl		string	
config_compile	enable_auto_rewind	1	bool	syn.compile.enable_auto_rewind
config_compile	pipeline_style	frp	enum	syn.compile.pipeline_style
config_compile	pragma_strict_mode	0	bool	syn.compile.pragma_strict_mode
config_compile	name_max_length	80	uint	syn.compile.name_max_length
config_compile	no_signed_zeros	0	bool	syn.compile.no_signed_zeros
config_compile	pipeline_loops	64	uint	syn.compile.pipeline_loops
config_compile	unsafe_math_optimizations	0	bool	syn.compile.unsafe_math_optimizations
config_cosim	enable_tasks_with_maxi		bool	cosim.enable_tasks_with_maxi
config_cosim	argv		string	cosim.argv
config_cosim	compiled_library_dir		string	cosim.compiled_library_dir
config_cosim	coverage		bool	cosim.coverage
config_cosim	disable_binary_tv		bool	cosim.disable_binary_tv
config_cosim	disable_deadlock_detection		bool	cosim.disable_deadlock_detection
config_cosim	disable_dependency_check		bool	cosim.disable_dependency_check
config_cosim	enable_dataflow_profiling		bool	cosim.enable_dataflow_profiling
config_cosim	enable_fifo_sizing		bool	cosim.enable_fifo_sizing

表 70: Tcl 配置命令 (续)

Tcl 命令	选项	默认	值类型	配置文件命令
config_cosim	hwemu_trace_dir		string	cosim.hwemu_trace_dir
config_cosim	ldflags		string	cosim.ldflags
config_cosim	mflags		string	cosim.mflags
config_cosim	O		bool	cosim.O
config_cosim	random_stall		bool	cosim.random_stall
config_cosim	rtl	verilog	enum	cosim.rtl
config_cosim	sanitize_address			csim.sanitize_address
config_cosim	sanitize_undefined			csim.sanitize_undefined
config_cosim	setup		bool	cosim.setup
config_cosim	stable_axilite_update		bool	cosim.stable_axilite_update
config_cosim	tool	xsim	enum	cosim.tool
config_cosim	trace_level	无	enum	cosim.trace_level
config_cosim	user_stall		string	cosim.user_stall
config_cosim	wave_debug		bool	cosim.wave_debug
config_csim	code_analyzer	0	bool	csim.code_analyzer
config_csim	argv		string	csim.argv
config_csim	clean		bool	csim.clean
config_csim	ldflags		string	csim.ldflags
config_csim	mflags		string	csim.mflags
config_csim	O		bool	csim.O
config_csim	setup		bool	csim.setup
config_dataflow	enable_canonicalization	true	bool (true false)	syn.dataflow.enable_canonicalization
config_dataflow	override_user_fifo_depth	0	uint	syn.dataflow.override_user_fifo_depth
config_dataflow	disable_fifo_sizing_opt	0	bool	syn.dataflow.disable_fifo_sizing_opt
config_dataflow	task_level_fifo_depth	2	uint	syn.dataflow.task_level_fifo_depth
config_dataflow	strict_stable_sync	0	bool	syn.dataflow.strict_stable_sync
config_dataflow	default_channel	pingpong	enum	syn.dataflow.default_channel
config_dataflow	fifo_depth	2	uint	syn.dataflow.fifo_depth
config_dataflow	scalar_fifo_depth	2	uint	syn.dataflow.scalar_fifo_depth
config_dataflow	start_fifo_depth	2	uint	syn.dataflow.start_fifo_depth
config_dataflow	strict_mode	warning	enum	syn.dataflow.strict_mode

表 70: Tcl 配置命令 (续)

Tcl 命令	选项	默认	值类型	配置文件命令
config_debug	directory	.debug	string	syn.debug.directory
config_debug	enable	0	bool	syn.debug.enable
config_export	xo_type	rtl	enum	package.output.xo_type
config_export	cosim_trace_generation	0	bool	syn.rtl.cosim_trace_generation
config_export	flow	无	enum	vivado.flow
config_export	ip_xdc_file		string	package.ip.xdc_file
config_export	ip_xdc_ooc_file		string	package.ip.xdc_ooc_file
config_export	vivado_clock		string	vivado.clock
config_export	vivado_max_timing_paths	10	uint	vivado.max_timing_paths
config_export	vivado_pblock		string	vivado.pblock
config_export	output		string	package.output.file
config_export	description		string	package.ip.description
config_export	display_name		string	package.ip.display_name
config_export	format	ip_catalog	enum	package.output.format
config_export	ipname		string	package.ip.name
config_export	library		string	package.ip.library
config_export	rtl	verilog	enum	vivado.rtl
config_export	taxonomy		string	package.ip.taxonomy
config_export	vendor		string	package.ip.vendor
config_export	version		string	package.ip.version
config_export	vivado_impl_strategy	default	string	vivado.impl_strategy
config_export	vivado_optimization_level	0	enum	vivado.optimization_level
config_export	vivado_phys_opt	无	enum	vivado.phys_opt
config_export	vivado_report_level	2	enum	vivado.report_level
config_export	vivado_synth_design_args	-directive sdx_optimization_effort_high	string	vivado.synth_design_args
config_export	vivado_synth_strategy	default	string	vivado.synth_strategy
config_interface	m_axi_cache_impl	auto	enum	syn.interface.m_axi_cache_impl
config_interface	m_axi_auto_id_channel	0	bool	syn.interface.m_axi_auto_id_channel
config_interface	s_axilite_interrupt_mode	tow	enum	syn.interface.s_axilite_interrupt_mode
config_interface	s_axilite_mailbox	无	enum	syn.interface.s_axilite_mailbox
config_interface	s_axilite_memory_auto_widen	true	bool (true false)	syn.interface.s_axilite_memory_auto_widen

表 70: Tcl 配置命令 (续)

Tcl 命令	选项	默认	值类型	配置文件命令
config_interface	s_axilite_status_regs	off	enum	syn.interface.s_axilite_status_regs
config_interface	s_axilite_sw_reset	0	bool	syn.interface.s_axilite_sw_reset
config_interface	m_axi_buffer_impl	bram	enum	syn.interface.m_axi_buffer_impl
config_interface	m_axi_conservative_mode	1	bool	syn.interface.m_axi_conservative_mode
config_interface	m_axi_flush_mode	0	bool	syn.interface.m_axi_flush_mode
config_interface	m_axi_alignment_byte_size	1	uint	syn.interface.m_axi_alignment_byte_size
config_interface	m_axi_alignment_size	0	uint	syn.interface.m_axi_alignment_size
config_interface	m_axi_auto_max_ports	0	bool	syn.interface.m_axi_auto_max_ports
config_interface	m_axi_latency	0	uint	syn.interface.m_axi_latency
config_interface	m_axi_max_bitwidth	1024	uint	syn.interface.m_axi_max_bitwidth
config_interface	m_axi_max_read_burst_length	16	uint	syn.interface.m_axi_max_read_burst_length
config_interface	m_axi_max_widen_bitwidth	0	uint	syn.interface.m_axi_max_widen_bitwidth
config_interface	m_axi_max_write_burst_length	16	uint	syn.interface.m_axi_max_write_burst_length
config_interface	m_axi_min_bitwidth	8	uint	syn.interface.m_axi_min_bitwidth
config_interface	m_axi_num_read_outstanding	16	uint	syn.interface.m_axi_num_read_outstanding
config_interface	m_axi_num_write_outstanding	16	uint	syn.interface.m_axi_num_write_outstanding
config_interface	default_slave_interface	s_axilite	enum	syn.interface.default_slave_interface
config_interface	s_axilite_data64	0	bool	syn.interface.s_axilite_data64
config_interface	clock_enable	0	bool	syn.interface.clock_enable
config_interface	m_axi_addr64	1	bool	syn.interface.m_axi_addr64
config_interface	m_axi_offset	slave	enum	syn.interface.m_axi_offset
config_interface	register_io	off	enum	syn.interface.register_io
config_op	precision	standard	enum	syn.op
config_op	impl	all	string	syn.op
config_op	latency	-1	int	syn.op
config_op	op		string	syn.op

表 70: Tcl 配置命令 (续)

Tcl 命令	选项	默认	值类型	配置文件命令
config_rtl	deadlock_detection	sim	enum	syn.rtl.deadlock_detection
config_rtl	register_all_io	0	bool	syn.rtl.register_all_io
config_rtl	register_reset_num	0	int	syn.rtl.register_reset_num
config_rtl	header		string	syn.rtl.header
config_rtl	kernel_profile	0	bool	syn.rtl.kernel_profile
config_rtl	module_auto_prefix	1	bool	syn.rtl.module_auto_prefix
config_rtl	module_prefix		string	syn.rtl.module_prefix
config_rtl	mult_keep_attribute	0	bool	syn.rtl.mult_keep_attribute
config_rtl	reset	control	enum	syn.rtl.reset
config_rtl	reset_async	0	bool	syn.rtl.reset_async
config_rtl	reset_level	high	enum	syn.rtl.reset_level
config_schedule	enable_dsp_full_reg	1	bool	syn.schedule.enable_dsp_full_reg
config_storage	auto_srl_max_bits	1024	uint	syn.storage
config_storage	auto_srl_max_depth	2	uint	syn.storage
config_storage	impl	autosrl	string	syn.storage
config_storage	type		string	syn.storage
config_unroll	tripcount_threshold	0	uint	syn.unroll.tripcount_threshold

表 71: Tcl 优化指令

Tcl 命令	配置文件命令
set_directive_aggregate	syn.directive.aggregate
set_directive_alias	syn.directive.alias
set_directive_allocation	syn.directive.allocation
set_directive_array_partition	syn.directive.array_partition
set_directive_array_reshape	syn.directive.array_reshape
set_directive_bind_op	syn.directive.bind_op
set_directive_bind_storage	syn.directive.bind_storage
set_directive_cache	syn.directive.cache
set_directive_dataflow	syn.directive.dataflow
set_directive_dependence	syn.directive.dependence
set_directive_disaggregate	syn.directive.disaggregate
set_directive_expression_balance	syn.directive.expression_balance
set_directive_function_instantiate	syn.directive.function_instantiate
set_directive_inline	syn.directive.inline
set_directive_interface	syn.directive.interface

表 71: Tcl 优化指令 (续)

Tcl 命令	配置文件命令
set_directive_latency	syn.directive.latency
set_directive_loop_flatten	syn.directive.loop_flatten
set_directive_loop_merge	syn.directive.loop_merge
set_directive_loop_tripcount	syn.directive.loop_tripcount
set_directive_occurrence	syn.directive.occurrence
set_directive_performance	syn.directive.performance
set_directive_pipeline	syn.directive.pipeline
set_directive_protocol	syn.directive.protocol
set_directive_reset	syn.directive.reset
set_directive_resource	syn.directive.resource
set_directive_shared	syn.directive.shared
set_directive_stable	syn.directive.stable
set_directive_stream	syn.directive.stream
set_directive_top	syn.top
set_directive_unroll	syn.directive.unroll

指令/运算符说明

下表提供了 HLS 编译器中使用的部分运算符的定义。如需获取标准 llvm 指令，请访问：<https://llvm.org/docs/LangRef.html#instruction-reference>。

名称	描述
FSqrt	计算浮点数的平方根
FRSqrt	计算浮点数的平方根倒数
FRecip	计算浮点数的倒数
FLog	计算浮点数的对数
FExp	计算浮点数的指数
DSqrt	计算双精度的平方根
DRSqrt	计算双精度的平方根倒数
DRecip	计算双精度的倒数
DLog	计算双精度的对数
DExp	计算双精度的指数
BlackBox	例化黑盒函数
BitSelect	选中某个变量的单个位
BitSet	将某个变量的单个位置位
PartSelect	选中某个变量的特定范围内的位
PartSet	将某个变量的特定范围内的位置位
BitConcatenate	串联 2 个变量
XorReduce	计算变量的 XOR reduce，该变量由 xor_reduce() 定义，类型为 ap_(u)int
XnorReduce	计算变量的 XNOR reduce，该变量由 xnor_reduce() 定义，类型为 ap_(u)int
AndReduce	计算变量的 AND reduce，该变量由 and_reduce() 定义，类型为 ap_(u)int
NandReduce	计算变量的 NAND reduce，该变量由 nand_reduce() 定义，类型为 ap_(u)int
OrReduce	计算变量的 OR reduce，该变量由 or_reduce() 定义，类型为 ap_(u)int
NorReduce	计算变量的 NOR reduce，该变量由 nor_reduce() 定义，类型为 ap_(u)int
Read	从接口读取值
Write	向接口写入值
NbRead	以非阻塞方式从接口读取值
NbWrite	以非阻塞方式向接口写入值
ReadReq	向总线发送读取请求。仅可用于 m_axi。
WriteReq	向总线发送写入请求。仅可用于 m_axi。
WriteResp	接收来自总线的写入响应。仅用于 m_axi。
MemShiftRead	对 SRL 原语的寄存器进行移位

名称	描述
Mux	从多个值中选中一个值

附加资源与法律声明

查找其他文档

技术信息门户网站

AMD 技术信息门户网站是旨在使用您的网页浏览器提供健全的文档搜索和导航的在线工具。要访问该技术信息门户网站，请转至 <https://docs.amd.com>。

注释：单击链接将打开英语版本，但您可从下拉列表中选择简体中文版本（如可用）。请注意，简体中文版本可能比英语版本旧。

Documentation Navigator

Documentation Navigator (DocNav) 是预安装的工具，支持访问 AMD 自适应计算文档、视频和支持资源，您可在其中通过筛选和搜索来查找信息。要打开 DocNav，请执行以下操作：

- 在 AMD Vivado™ IDE 中，依次选择“Help” → “Documentation and Tutorials”（帮助 > 文档和教程）。
- 在 Windows 上，单击“Start”（开始）按钮，然后选择“AMD Adaptive SoC and FPGA Tools” → “DocNav”（AMD 自适应 SoC 与 FPGA 工具 > DocNav）。
- 在 Linux 命令提示中输入 `docnav`。

注释：如需了解有关 DocNav 的更多信息，请参阅《Documentation Navigator 用户指南》(UG968)。

注释：您无法从 DocNav 访问简体中文版本。请使用设计中心网页。

设计中心

AMD 设计中心提供了根据设计任务和其他主题整理的文档链接，您可使用这些链接来了解关键概念以及获取常见问题的解答。要访问设计中心，请执行以下操作：

- 在 DocNav 中，单击“Design Hubs View”选项卡。
- 转至[设计中心](#)网页。

支持资源

如需获取答复记录、技术文档、下载以及论坛等支持资源，请访问[技术支持](#)。

参考资料

以下技术文档是非常实用的补充资料，可配合本指南一起使用：

- 1. 《UltraScale 架构 DSP Slice》(UG579)
- 2. 《Vivado Design Suite 用户指南：逻辑仿真》(UG900)
- 3. 《Vivado Design Suite 用户指南：综合》(UG901)
- 4. 《Vivado Design Suite 用户指南：实现》(UG904)
- 5. 《Vivado Design Suite Tcl 命令参考指南》(UG835)
- 6. 《采用 Vivado 高层次综合开展 FPGA 设计介绍》(UG998)
- 7. 《Vivado Design Suite 教程：高层次综合》(UG871)
- 8. 《Vivado Design Suite 用户指南：版本说明、安装和许可》(UG973)
- 9. 《Floating-Point Operator LogiCORE IP 产品指南》(PG060)
- 10. 《Fast Fourier Transform LogiCORE IP 产品指南》(PG109)
- 11. 《FIR Compiler LogiCORE IP 产品指南》(PG149)
- 12. 《DDS Compiler LogiCORE IP 产品指南》(PG141)
- 13. 《Vivado Design Suite：AXI 参考指南》(UG1037)
- 14. 《Vitis 统一软件平台文档：嵌入式软件开发》(UG1400)
- 15. 《使用 Vitis 进行数据中心加速》(UG1700)
- 16. 《使用 Vitis 进行嵌入式设计开发》(UG1701)
- 17. 《Vitis 参考指南》(UG1702)
- 18. 《Versal 自适应 SoC DSP 引擎架构手册》(AM004)
- 19. 《使用 Vivado HLS 进行浮点设计》(XAPP599)
- 20. GCC 网站上的“Option Summary”页面 (gcc.gnu.org/onlinedocs/gcc/Option-Summary.html)
- 21. Accellera 网站 (<http://www.accellera.org/>)
- 22. MathWorks 网站上的 AWGN 页面 (<http://www.mathworks.com/help/comm/ug/awgn-channel.html>)
- 23. [AMD Vivado™ Design Suite 文档](#)

修订历史

下表显示了 [第二部分：HLS 编程指南](#) 的修订历史。

章节	修订综述
2026 年 1 月 22 日 2025.2 版	
常规更新	编辑更新。

章节	修订综述
2025 年 11 月 20 日 2025.2 版	
常规更新	编辑更新。
2025 年 9 月 10 日 2025.1 版	
更新图示	更新章节。
2025 年 5 月 29 日 2025.1 版	
混用数据驱动的模式与控制驱动的模式 HLS 设计的执行模式	更新章节。

下表显示了 [第三部分：Vitis HLS 流程步骤](#) 的修订历史。

章节	修订综述
2025 年 5 月 29 日 2025.1 版	
组件比较功能	更新章节。

下表显示了 [第四部分：Vitis HLS 命令参考资料](#) 的修订历史。

章节	修订综述
2025 年 5 月 29 日 2025.1 版	
整个章节	本节无更改。

下表显示了 [第六部分：Vitis HLS 库参考资料](#) 的修订历史。

章节	修订综述
2025 年 5 月 29 日 2025.1 版	
C++ 任意精度定点类型 HLS 串流库 快速傅里叶变换 (IP) 库	小幅更新。

下表显示了 [第七部分：Vitis HLS 移植指南](#) 的修订历史。

章节	修订综述
2025 年 5 月 29 日 2025.1 版	
整个章节	本节无更改。

请阅读：重要法律声明

本文档所示信息仅做参考，其中可能包含不准确的技术信息、疏漏和印刷错误。受诸多原因影响，此处所含信息可能发生更改，也可能无法准确呈现，这些原因包括但不限于产品和路线图变更、组件和主板版本更改、新增模型和/或产品发布、不同制造商之间存在的差异、软件更改、BIOS 刷新、固件升级等。任何计算机系统均存在安全性漏洞风险，无法彻底阻止也无法缓解这类风险。AMD 没有任何义务来更新或者以任何其他方式纠正或修改这些信息。但 AMD 保留随时修改这些信息和更改文档内容的权利，AMD 没有任何义务将此类修改或更改通知任何人。此处信息“按原样”提供。AMD 对于本文档内容不作任何陈述或保证，并且对于这些可能出现的任何不准确、错误或疏漏问题不承担任何责任。对于有关任何暗含的非侵权、适销性及适合特定用途的保证，AMD 特此声明不承担任何责任。无论在任何情况下，对于任何人因使用此处包含的任何信息而形成的依赖或者引发的任何直接、间接、特殊或其他后果性损害，AMD 概不负责，即使 AMD 已明确获悉存在发生此类损害的可能性也是如此。

关于与汽车相关用途的免责声明

如将汽车产品（部件号中含“XA”字样）用于部署安全气囊或用于影响车辆控制的应用（“安全应用”），除非有符合 ISO 26262 汽车安全标准的安全概念或冗余特性（“安全设计”），否则不在质保范围内。客户应在使用或分销任何包含产品的系统之前为了安全的目的全面地测试此类系统。在未采用安全设计的条件下将产品用于安全应用的所有风险，由客户自行承担，并且仅在适用的法律法规对产品责任另有规定的情况下，适用该等法律法规的规定。

版权声明

© Copyright 2020 - 2026 AMD 公司，版权所有。AMD、AMD 箭头徽标 Alveo、UltraScale、UltraScale+、Versal、Virtex、Vitis、Vivado、Zynq 及其组合均为 Advanced Micro Devices, Inc. 的商标。“OpenCL”和“OpenCL”徽标均为 Apple Inc. 的商标，经 Khronos 许可后方能使用。“PCI”、“PCIe”和“PCI Express”均为 PCI-SIG 拥有的商标，且经授权使用。“AMBA”、“AMBA Designer”、“Arm”、“ARM1176JZ-S”、“CoreSight”、“Cortex”、“PrimeCell”、“Mali”和“MPCore”为 Arm Limited 在美国和/或其他国家或地区的商标。“DisplayPort”图标为 Video Electronics Standards Association 拥有的商标，在美国和其他国家或地区已获注册。此出版物中所使用的其他产品名称仅用于标识目的，可能是其各自所属公司的商标。